lombok/AccessLevel.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
/**
* Represents an AccessLevel. Used e.g. to specify the access level for generated methods and fields.
*/
public enum AccessLevel {
PUBLIC, MODULE, PROTECTED, PACKAGE, PRIVATE,
/** Represents not generating anything or the complete lack of a method. */
NONE;
}
```

lombok/AllArgsConstructor.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Generates an all-args constructor.
* An all-args constructor requires one argument for every field in the class.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/constructor">the
project lombok features page for &#64;Constructor</a>.
* <p>
* Even though it is not listed, this annotation also has the {@code onConstructor} parameter. See the
full documentation for more details.
*
* @see NoArgsConstructor
* @see RequiredArgsConstructor
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface AllArgsConstructor {
/**
* If set, the generated constructor will be private, and an additional static 'constructor'
* is generated with the same argument list that wraps the real constructor.
*
* Such a static 'constructor' is primarily useful as it infers type arguments.
*
* @return Name of static 'constructor' method to generate (blank = generate a normal constructor).
*/
String staticName() default "";
/**
* Any annotations listed here are put on the generated constructor.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @AllArgsConstructor(onConstructor=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @AllArgsConstructor(onConstructor_={@AnnotationsGohere})} // note the underscore after
{@code onConstructor}.
*
* @return List of annotations to apply to the generated constructor.
*/
AnyAnnotation[] onConstructor() default {};
/**
* Sets the access level of the constructor. By default, generated constructors are {@code public}.
*
* @return The constructor will be generated with this access modifier.
```

```
*/
AccessLevel access() default lombok.AccessLevel.PUBLIC;
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
*
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/Builder.java

```
/*
* Copyright (C) 2013-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import static java.lang.annotation.ElementType.*;
import static java.lang.annotation.RetentionPolicy.*;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;
/**
* The builder annotation creates a so-called 'builder' aspect to the class that is annotated or the class
* that contains a member which is annotated with {@code @Builder}.
* <p>
* If a member is annotated, it must be either a constructor or a method. If a class is annotated,
* then a package-private constructor is generated with all fields as arguments
* (as if {@code @AllArgsConstructor(access = AccessLevel.PACKAGE)} is present
* on the class), and it is as if this constructor has been annotated with {@code @Builder} instead.
* Note that this constructor is only generated if you haven't written any constructors and also haven't
```

* added any explicit {@code @XArgsConstructor} annotations. In those cases, lombok will assume an all-args
* constructor is present and generate code that uses it; this means you'd get a compiler error if this
* constructor is not present.
* <p>
* The effect of {@code @Builder} is that an inner class is generated named <code>TBuilder</code>,
* with a private constructor. Instances of <code>TBuilder</code> are made with the
* method named {@code builder()} which is also generated for you in the class itself (not in the builder class).
* <p>
* The <code>TBuilder</code> class contains 1 method for each parameter of the annotated
* constructor / method (each field, when annotating a class), which returns the builder itself.
* The builder also has a <code>build()</code> method which returns a completed instance of the original type,
* created by passing all parameters as set via the various other methods in the builder to the constructor
* or method that was annotated with {@code @Builder}. The return type of this method will be the same
* as the relevant class, unless a method has been annotated, in which case it'll be equal to the
* return type of that method.
* <p>
* Complete documentation is found at the project lombok features page for @Builder.
*
* <p>
* Before:
*
* <pre>
* @Builder
* class Example<T> {
* private T foo;
* private final String bar;
* }
* </pre>
*
* After:
*
* <pre>
* class Example<T> {
* private T foo;
* private final String bar;
*
* private Example(T foo, String bar) {
* this.foo = foo;
* this.bar = bar;
* }
*
* public static <T> ExampleBuilder<T> builder() {
* return new ExampleBuilder<T>();
* }
*
* public static class ExampleBuilder<T> {

```
* private T foo;
* private String bar;
*
* private ExampleBuilder() {}
*
* public ExampleBuilder foo(T foo) {
* this.foo = foo;
* return this;
* }
*
* public ExampleBuilder bar(String bar) {
* this.bar = bar;
* return this;
* }
*
* &#064;java.lang.Override public String toString() {
* return "ExampleBuilder(foo = " + foo + ", bar = " + bar + ")";
* }
*
* public Example build() {
* return new Example(foo, bar);
* }
* }
* }
* </pre>
*
* @see Singular
*/
@Target({TYPE, METHOD, CONSTRUCTOR})
@Retention(SOURCE)
public @interface Builder {
/**
* The field annotated with {@code @Default} must have an initializing expression; that expression is taken as the default to be used if not explicitly set during building.
*/
@Target(FIELD)
@Retention(SOURCE)
public @interface Default {}
/** @return Name of the method that creates a new builder instance. Default: {@code builder}. If the empty string, suppress generating the {@code builder} method. */
String builderMethodName() default "builder";
/** @return Name of the method in the builder class that creates an instance of your {@code @Builder}-annotated class. */
String buildMethodName() default "build";
/**
* Name of the builder class.
*
* Default for {@code @Builder} on types and constructors: see the configkey {@code lombok.builder.className}, which if not set defaults to {@code (TypeName)Builder}.
* <p>
* Default for {@code @Builder} on methods: see the configkey {@code lombok.builder.className}, which if not set defaults to {@code (ReturnTypeName)Builder}.
*
```

```
* @return Name of the builder class that will be generated (or if it already exists, will be filled with builder elements).
*/
String builderClassName() default "";
/**
* If true, generate an instance method to obtain a builder that is initialized with the values of this instance.
* Legal only if {@code @Builder} is used on a constructor, on the type itself, or on a static method that returns
* an instance of the declaring type.
*
* @return Whether to generate a {@code toBuilder()} method.
*/
boolean toBuilder() default false;
/**
* Sets the access level of the generated builder class. By default, generated builder classes are {@code public}.
* Note: This does nothing if you write your own builder class (we won't change its access level).
*
* @return The builder class will be generated with this access modifier.
*/
AccessLevel access() default lombok.AccessLevel.PUBLIC;
/**
* Prefix to prepend to 'set' methods in the generated builder class. By default, generated methods do not include a prefix.
*
* For example, a method normally generated as {@code someField(String someField)} would instead be
* generated as {@code withSomeField(String someField)} if using {@code @Builder(setterPrefix = "with")}.
*
* Note that using "with" to prefix builder setter methods is strongly discouraged as "with" normally
* suggests immutable data structures, and builders by definition are mutable objects.
*
* For {@code @Singular} fields, the generated methods are called {@code withName}, {@code withNames}, and {@code clearNames}, instead of
* the default {@code name}, {@code names}, and {@code clearNames}.
*
* @return The prefix to prepend to generated method names.
*/
String setterPrefix() default "";
/**
* Put on a field (in case of {@code @Builder} on a type) or a parameter (for {@code @Builder} on a constructor or static method) to
* indicate how lombok should obtain a value for this field or parameter given an instance; this is only relevant if {@code toBuilder} is {@code true}.
*
* You do not need to supply an {@code @ObtainVia} annotation unless you wish to change the default behaviour: Use a field with the same name.
* <p>
* Note that one of {@code field} or {@code method} should be set, or an error is generated.
* <p>
* The default behaviour is to obtain a value by referencing the name of the parameter as a field on 'this'.
```

```
*/
@Target({FIELD, PARAMETER})
@Retention(SOURCE)
public @interface ObtainVia {
/**
* @return Tells lombok to obtain a value with the expression {@code this.value}.
*/
String field() default "";
/**
* @return Tells lombok to obtain a value with the expression {@code this.method()}.
*/
String method() default "";
/**
* @return Tells lombok to obtain a value with the expression {@code SelfType.method(this)}; requires
{@code method} to be set.
*/
boolean isStatic() default false;
}
}
```

lombok/Cleanup.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Ensures the variable declaration that you annotate will be cleaned up by calling its close method,
regardless
```

```
* of what happens. Implemented by wrapping all statements following the local variable declaration to
the
* end of your scope into a try block that, as a finally action, closes the resource.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/Cleanup">the project
lombok features page for &#64;Cleanup</a>.
* <p>
* Example:
* <pre>
* public void copyFile(String in, String out) throws IOException {
* &#64;Cleanup FileInputStream inStream = new FileInputStream(in);
* &#64;Cleanup FileOutputStream outStream = new FileOutputStream(out);
* byte[] b = new byte[65536];
* while (true) {
* int r = inStream.read(b);
* if (r == -1) break;
* outStream.write(b, 0, r);
* }
* }
* </pre>
*
* Will generate:
* <pre>
* public void copyFile(String in, String out) throws IOException {
* &#64;Cleanup FileInputStream inStream = new FileInputStream(in);
* try {
* &#64;Cleanup FileOutputStream outStream = new FileOutputStream(out);
* try {
* byte[] b = new byte[65536];
* while (true) {
* int r = inStream.read(b);
* if (r == -1) break;
* outStream.write(b, 0, r);
* }
* } finally {
* if (outStream != null) outStream.close();
* }
* } finally {
* if (inStream != null) inStream.close();
* }
* }
* </pre>
*/
@Target(ElementType.LOCAL_VARIABLE)
@Retention(RetentionPolicy.SOURCE)
public @interface Cleanup {
/** @return The name of the method that cleans up the resource. By default, 'close'. The method must
not have any parameters. */
String value() default "close";
}
```

lombok/ConfigurationKeys.java

```
/*
* Copyright (C) 2013-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.util.List;
import lombok.core.configuration.CallSuperType;
import lombok.core.configuration.CapitalizationStrategy;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.configuration.ConfigurationKey;
import lombok.core.configuration.FlagUsageType;
import lombok.core.configuration.IdentifierName;
import lombok.core.configuration.LogDeclaration;
import lombok.core.configuration.NullAnnotationLibrary;
import lombok.core.configuration.NullCheckExceptionType;
import lombok.core.configuration.TypeName;
/**
* A container class containing all lombok configuration keys that do not belong to a specific annotation.
*/
public class ConfigurationKeys {
private ConfigurationKeys() {}
// ##### main package features #####
// ----- global -----
/**
* lombok configuration: {@code dangerousconfig.lombok.disable} = {@code true} | {@code false}.
*
* If {@code true}, lombok is disabled entirely.
*/
public static final ConfigurationKey<Boolean> LOMBOK_DISABLE = new
ConfigurationKey<Boolean>("dangerousconfig.lombok.disable", "Disables lombok transformers. It does
not flag any lombok mentions (so, @Cleanup silently does nothing), and does not disable patched
operations in eclipse either. Don't use this unless you know what you're doing. (default: false).", true) {};
/**
* lombok configuration: {@code lombok.addGeneratedAnnotation} = {@code true} | {@code false}.
*
```

```
* If {@code true}, lombok generates {@code @javax.annotation.Generated("lombok")} on all fields,
methods, and types that are generated, unless {@code lombok.addJavaxGeneratedAnnotation} is set.
* <br>
* <em>BREAKING CHANGE</em>: Starting with lombok v1.16.20, defaults to {@code false} instead of
{@code true}, as this annotation is broken in JDK9.
*
* @see ConfigurationKeys#ADD_JAVAX_GENERATED_ANNOTATIONS
* @see ConfigurationKeys#ADD_JAKARTA_GENERATED_ANNOTATIONS
* @see ConfigurationKeys#ADD_LOMBOK_GENERATED_ANNOTATIONS
* @deprecated Since version 1.16.14, use {@link #ADD_JAVAX_GENERATED_ANNOTATIONS}
instead.
*/
@Deprecated
public static final ConfigurationKey<Boolean> ADD_GENERATED_ANNOTATIONS = new
ConfigurationKey<Boolean>("lombok.addGeneratedAnnotation", "Generate
@javax.annotation.Generated on all generated code (default: false). Deprecated, use
'lombok.addJavaxGeneratedAnnotation' instead.") {};
/**
* lombok configuration: {@code lombok.addJavaxGeneratedAnnotation} = {@code true} | {@code
false}.
*
* If {@code true}, lombok generates {@code @javax.annotation.Generated("lombok")} on all fields,
methods, and types that are generated.
* <br>
* <em>BREAKING CHANGE</em>: Starting with lombok v1.16.20, defaults to {@code false} instead of
{@code true}, as this annotation is broken in JDK9.
*/
public static final ConfigurationKey<Boolean> ADD_JAVAX_GENERATED_ANNOTATIONS = new
ConfigurationKey<Boolean>("lombok.addJavaxGeneratedAnnotation", "Generate
@javax.annotation.Generated on all generated code (default: follow
lombok.addGeneratedAnnotation).") {};
/**
* lombok configuration: {@code lombok.addJakartaGeneratedAnnotation} = {@code true} | {@code
false}.
*
* If {@code true}, lombok generates {@code @jakarta.annotation.Generated("lombok")} on all fields,
methods, and types that are generated.
*/
public static final ConfigurationKey<Boolean> ADD_JAKARTA_GENERATED_ANNOTATIONS = new
ConfigurationKey<Boolean>("lombok.addJakartaGeneratedAnnotation", "Generate
@jakarta.annotation.Generated on all generated code (default: false).") {};
/**
* lombok configuration: {@code lombok.addLombokGeneratedAnnotation} = {@code true} | {@code
false}.
*
* If {@code true}, lombok generates {@code @lombok.Generated} on all fields, methods, and types
that are generated.
*/
public static final ConfigurationKey<Boolean> ADD_LOMBOK_GENERATED_ANNOTATIONS = new
ConfigurationKey<Boolean>("lombok.addLombokGeneratedAnnotation", "Generate
@lombok.Generated on all generated code (default: true).") {};
/**
```

```
* lombok configuration: {@code lombok.extern.findbugs.addSuppressFBWarnings} = {@code true} |
{@code false}.
*
* If {@code true}, lombok generates {@code edu.umd.cs.findbugs.annotations.SuppressFBWarnings}
on all fields, methods, and types that are generated.
*
* NB: If you enable this option, findbugs must be on the source or classpath, or you'll get errors that the
type {@code SuppressFBWarnings} cannot be found.
*/
public static final ConfigurationKey<Boolean>
ADD_FINDBUGS_SUPPRESSWARNINGS_ANNOTATIONS = new
ConfigurationKey<Boolean>("lombok.extern.findbugs.addSuppressFBWarnings", "Generate
@edu.umd.cs.findbugs.annotations.SuppressFBWarnings on all generated code (default: false).") {};
/**
* lombok configuration: {@code lombok.addSuppressWarnings} = {@code true} | {@code false}.
*
* If {@code true}, lombok generates {@code @java.lang.SuppressWarnings("all")} on all fields,
methods, and types that are generated.
*/
public static final ConfigurationKey<Boolean> ADD_SUPPRESSWARNINGS_ANNOTATIONS = new
ConfigurationKey<Boolean>("lombok.addSuppressWarnings", "Generate
@java.lang.SuppressWarnings(\"all\") on all generated code (default: true).") {};
/**
* lombok configuration: {@code lombok.addNullAnnotations = }one of: [{@code none}, {@code javax},
{@code eclipse}, {@code jetbrains}, {@code netbeans}, {@code androidx}, {@code android.support},
{@code checkerframework}, {@code findbugs}, {@code spring}, {@code JML}, or a custom set of fully
qualified annotation types].
*
* Lombok generally copies relevant nullity annotations from your source code to the right places.
However, sometimes lombok generates code where the nullability of some node is not dependent on
something in your source code. You can configure lombok to add an appropriate nullity annotation in
this case.<ul>
* <li>{@code none} (the default) - no annotations are added.</li>
* <li>{@code javax} - The annotations {@code javax.annotation.NonNull} and {@code
javax.annotation.Nullable} are used.</li>
* <li>{@code jakarta} - The annotations {@code jakarta.annotation.NonNull} and {@code
jakarta.annotation.Nullable} are used.</li>
* <li>{@code eclipse} - The annotations {@code org.eclipse.jdt.annotation.NonNull} and {@code
org.eclipse.jdt.annotation.Nullable} are used.</li>
* <li>{@code jetbrains} - The annotations {@code org.jetbrains.annotations.NotNull} and {@code
org.jetbrains.annotations.Nullable} are used.</li>
* <li>{@code netbeans} - The annotations {@code org.netbeans.api.annotations.common.NonNull}
and {@code org.netbeans.api.annotations.common.NullAllowed} are used.</li>
* <li>{@code androidx} - The annotations {@code androidx.annotation.NonNull} and {@code
androidx.annotation.Nullable} are used.</li>
* <li>{@code android.support} - The annotations {@code android.support.annotation.NonNull} and
{@code android.support.annotation.Nullable} are used.</li>
* <li>{@code checkerframework} - The annotations {@code
org.checkerframework.checker.nullness.qual.NonNull} and {@code
org.checkerframework.checker.nullness.qual.Nullable} are used.</li>
* <li>{@code findbugs} - The annotations {@code edu.umd.cs.findbugs.annotations.NonNull} and
{@code edu.umd.cs.findbugs.annotations.Nullable} are used.</li>
```

```
* <li>{@code spring} - The annotations {@code org.springframework.lang.NonNull} and {@code
org.springframework.lang.Nullable} are used.</li>
* <li>{@code jml} - The annotations {@code org.jmlspecs.annotation.NonNull} and {@code
org.jmlspecs.annotation.Nullable} are used.</li>
* <li><code>CUSTOM:<em>fully.qualified.nonnull.annotation</em>:<em>fully.qualified.nullable.annota
tion</em></code> to configure your own types; the nullable annotation (and the colon) are
optional.</li>
* </ul>
* <p>
* Lombok will not put these annotations on the classpath for you; your project must be set up such that
these annotations are available.
* <p>
* Current features which use this configuration:<ul>
* <li>{@code @Builder.Singular} makes methods that accept a collection, all of which must be added.
The parameter to this 'plural form' method is annotated.</li>
* </ul>
*/
public static final ConfigurationKey<NullAnnotationLibrary> ADD_NULL_ANNOTATIONS = new
ConfigurationKey<NullAnnotationLibrary>("lombok.addNullAnnotations", "Generate some style of null
annotation for generated code where this is relevant. (default: none).") {};
// ----- *ArgsConstructor -----
/**
* lombok configuration: {@code lombok.anyConstructor.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @AllArgsConstructor}, {@code @RequiredArgsConstructor},
or {@code @NoArgsConstructor} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> ANY_CONSTRUCTOR_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.anyConstructor.flagUsage", "Emit a warning or error if
any of the XxxArgsConstructor annotations are used.") {};
/**
* lombok configuration: {@code lombok.anyConstructor.suppressConstructorProperties} = {@code
true} | {@code false}.
*
* If {@code false} or this configuration is omitted, all generated constructors with at least 1 argument
get a {@code @ConstructorProperties}.
* To suppress the generation of it, set this configuration to {@code true}.
*
* NB: GWT projects, and probably android projects, should explicitly set this key to {@code true} for the
entire project.
*
* <br>
* <em>BREAKING CHANGE</em>: Starting with lombok v1.16.20, defaults to {@code true} instead of
{@code false}, as {@code @ConstructorProperties} requires extra modules in JDK9.
*
* @see ConfigurationKeys#ANY_CONSTRUCTOR_ADD_CONSTRUCTOR_PROPERTIES
* @deprecated Since version 2.0, use {@link
#ANY_CONSTRUCTOR_ADD_CONSTRUCTOR_PROPERTIES} instead.
*/
@Deprecated
public static final ConfigurationKey<Boolean>
ANY_CONSTRUCTOR_SUPPRESS_CONSTRUCTOR_PROPERTIES = new
```

```
ConfigurationKey<Boolean>("lombok.anyConstructor.suppressConstructorProperties", "Suppress the generation of @ConstructorProperties for generated constructors (default: false).") {};
/**
* lombok configuration: {@code lombok.anyConstructor.addConstructorProperties} = {@code true} | {@code false}.
*
* If {@code true}, all generated constructors with at least 1 argument get a {@code @ConstructorProperties}.
*
*/
public static final ConfigurationKey<Boolean> ANY_CONSTRUCTOR_ADD_CONSTRUCTOR_PROPERTIES = new ConfigurationKey<Boolean>("lombok.anyConstructor.addConstructorProperties", "Generate @ConstructorProperties for generated constructors (default: false).") {};
/**
* lombok configuration: {@code lombok.allArgsConstructor.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @AllArgsConstructor} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> ALL_ARGS_CONSTRUCTOR_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.allArgsConstructor.flagUsage", "Emit a warning or error if @AllArgsConstructor is used.") {};
/**
* lombok configuration: {@code lombok.noArgsConstructor.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @NoArgsConstructor} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> NO_ARGS_CONSTRUCTOR_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.noArgsConstructor.flagUsage", "Emit a warning or error if @NoArgsConstructor is used.") {};
/**
* lombok configuration: {@code lombok.noArgsConstructor.extraPrivate} = {@code true} | {@code false}.
*
* If {@code true}, @Data and @Value will also generate a private no-args constructor, if there isn't already one, setting all fields to their default values.
*/
public static final ConfigurationKey<Boolean> NO_ARGS_CONSTRUCTOR_EXTRA_PRIVATE = new ConfigurationKey<Boolean>("lombok.noArgsConstructor.extraPrivate", "Generate a private no-args constructor for @Data and @Value (default: false).") {};
/**
* lombok configuration: {@code lombok.requiredArgsConstructor.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @RequiredArgsConstructor} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> REQUIRED_ARGS_CONSTRUCTOR_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.requiredArgsConstructor.flagUsage", "Emit a warning or error if @RequiredArgsConstructor is used.") {};
// ##### Beanies #####
```

```
// ----- Data -----
/**
* lombok configuration: {@code lombok.data.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Data} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> DATA_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.data.flagUsage", "Emit a warning or error if @Data is
used.") {};
// ----- Value -----
/**
* lombok configuration: {@code lombok.value.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Value} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> VALUE_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.value.flagUsage", "Emit a warning or error if @Value is
used.") {};
// ----- Getter -----
/**
* lombok configuration: {@code lombok.getter.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Getter} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> GETTER_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.getter.flagUsage", "Emit a warning or error if @Getter is
used.") {};
/**
* lombok configuration: {@code lombok.getter.lazy.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @Getter(lazy=true)} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> GETTER_LAZY_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.getter.lazy.flagUsage", "Emit a warning or error if
@Getter(lazy=true) is used.") {};
/**
* lombok configuration: {@code lombok.getter.noIsPrefix} = {@code true} | {@code false}.
*
* If {@code true}, booleans getters are both referred to, and generated as {@code getFieldName()}. If
{@code false} (the default), the javabean-standard {@code isFieldName()} is generated / used instead.
*
*/
public static final ConfigurationKey<Boolean> GETTER_CONSEQUENT_BOOLEAN = new
ConfigurationKey<Boolean>("lombok.getter.noIsPrefix", "If true, generate and use getFieldName() for
boolean getters instead of isFieldName().") {};
// ----- Setter -----
/**
* lombok configuration: {@code lombok.setter.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Setter} results in a warning / error.
*/
```

```
public static final ConfigurationKey<FlagUsageType> SETTER_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.setter.flagUsage", "Emit a warning or error if @Setter is
used.") {};
// ----- EqualsAndHashCode -----
/**
* lombok configuration: {@code lombok.equalsAndHashCode.doNotUseGetters} = {@code true} |
{@code false}.
*
* For any class without an {@code @EqualsAndHashCode} that explicitly defines the {@code
doNotUseGetters} option, this value is used (default = false).
*/
public static final ConfigurationKey<Boolean>
EQUALS_AND_HASH_CODE_DO_NOT_USE_GETTERS = new
ConfigurationKey<Boolean>("lombok.equalsAndHashCode.doNotUseGetters", "Don't call the getters
but use the fields directly in the generated equals and hashCode method (default = false).") {};
/**
* lombok configuration: {@code lombok.equalsAndHashCode.callSuper} = {@code call} | {@code
ignore} | {@code warn}.
*
* For any class with an {@code @EqualsAndHashCode} annotation which extends a class other than
{@code java.lang.Object}, should a call to superclass's implementation of {@code equals} and {@code
hashCode} be included in the generated methods? (Default = warn)
*/
public static final ConfigurationKey<CallSuperType> EQUALS_AND_HASH_CODE_CALL_SUPER =
new ConfigurationKey<CallSuperType>("lombok.equalsAndHashCode.callSuper", "When generating
equals and hashCode for classes that extend something (other than Object), either automatically take
into account superclass implementation (call), or don't (skip), or warn and don't (warn). (default =
warn).") {};
/**
* lombok configuration: {@code lombok.equalsAndHashCode.flagUsage} = {@code WARNING} |
{@code ERROR}.
*
* If set, <em>any</em> usage of {@code @EqualsAndHashCode} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> EQUALS_AND_HASH_CODE_FLAG_USAGE =
new ConfigurationKey<FlagUsageType>("lombok.equalsAndHashCode.flagUsage", "Emit a warning or
error if @EqualsAndHashCode is used.") {};
// ----- ToString -----
/**
* lombok configuration: {@code lombok.toString.doNotUseGetters} = {@code true} | {@code false}.
*
* For any class without an {@code @ToString} that explicitly defines the {@code doNotUseGetters}
option, this value is used (default = false).
*/
public static final ConfigurationKey<Boolean> TO_STRING_DO_NOT_USE_GETTERS = new
ConfigurationKey<Boolean>("lombok.toString.doNotUseGetters", "Don't call the getters but use the
fields directly in the generated toString method (default = false).") {};
/**
* lombok configuration: {@code lombok.toString.callSuper} = {@code call} | {@code ignore} | {@code
warn}.
*
* For any class with an {@code @ToString} annotation which extends a class other than {@code
java.lang.Object}, should a call to superclass's implementation of {@code toString} be included in the
```

generated method? (Default = skip)
*/
public static final ConfigurationKey<CallSuperType> TO_STRING_CALL_SUPER = new ConfigurationKey<CallSuperType>("lombok.toString.callSuper", "When generating toString for classes that extend something (other than Object), either automatically take into account superclass implementation (call), or don't (skip), or warn and don't (warn). (default = skip).") {};
/**
* lombok configuration: {@code lombok.toString.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, any usage of {@code @ToString} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> TO_STRING_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.toString.flagUsage", "Emit a warning or error if @ToString is used.") {};
/**
* lombok configuration: {@code lombok.toString.includeFieldNames} = {@code true} | {@code false}.
*
* For any class without an {@code @ToString} that explicitly defines the {@code includeFieldNames} option, this value is used (default = true).
*/
public static final ConfigurationKey<Boolean> TO_STRING_INCLUDE_FIELD_NAMES = new ConfigurationKey<Boolean>("lombok.toString.includeFieldNames", "Include the field names in the generated toString method (default = true).") {};
/**
* lombok configuration: {@code lombok.toString.onlyExplicitlyIncluded} = {@code true} | {@code false}.
*
* If {@code true}, require a {@code @ToString.Include} annotation on any fields/no-args methods you want to include in lombok's generated `@ToString` method. Otherwise, every (non-static, non-dollar-named) field is included by default (default = false).
*/
public static final ConfigurationKey<Boolean> TO_STRING_ONLY_EXPLICITLY_INCLUDED = new ConfigurationKey<Boolean>("lombok.toString.onlyExplicitlyIncluded", "Include only fields/methods explicitly marked with @ToString.Include. Otherwise, include all non-static, non-dollar-named fields (default = false).") {};
// ----- Builder -----
/**
* lombok configuration: {@code lombok.builder.classNames} = <String: aJavaIdentifier (optionally with a star as placeholder for the type name)> (Default: {@code *Builder}).
*
* For any usage of the {@code @Builder} annotation without an explicit {@code builderClassName} parameter, this value is used to determine the name of the builder class to generate (or to adapt if such an inner class already exists).
*/
public static final ConfigurationKey<String> BUILDER_CLASS_NAME = new ConfigurationKey<String>("lombok.builder.className", "Default name of the generated builder class. A * is replaced with the name of the relevant type (default = *Builder).") {};
/**
* lombok configuration: {@code lombok.builder.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, any usage of {@code @Builder} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> BUILDER_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.builder.flagUsage", "Emit a warning or error if @Builder is

```
used.") {};
// ----- Singular -----
/**
* lombok configuration: {@code lombok.singular.useGuava} = {@code true} | {@code false}.
*
* If explicitly set to {@code true}, guava's {@code ImmutableList} etc are used to implement the immutable collection datatypes generated by @Singular @Builder for fields/parameters of type {@code java.util.List} and such.
* By default, unmodifiable-wrapped versions of {@code java.util} types are used.
*/
public static final ConfigurationKey<Boolean> SINGULAR_USE_GUAVA = new ConfigurationKey<Boolean>("lombok.singular.useGuava", "Generate backing immutable implementations for @Singular on java.util.* types by using guava's ImmutableList, etc. Normally java.util's mutable types are used and wrapped to make them immutable.") {};
/**
* lombok configuration: {@code lombok.singular.auto} = {@code true} | {@code false}.
*
* By default or if explicitly set to {@code true}, lombok will attempt to automatically singularize the name of your variable/parameter when using {@code @Singular}; the name is assumed to be written in english, and a plural. If explicitly to {@code false}, you must always specify the singular form; this is especially useful if your identifiers are in a foreign language.
*/
public static final ConfigurationKey<Boolean> SINGULAR_AUTO = new ConfigurationKey<Boolean>("lombok.singular.auto", "If true (default): Automatically singularize the assumed-to-be-plural name of your variable/parameter when using @Singular.") {};
// ###### Standalones ######
// ----- Cleanup -----
/**
* lombok configuration: {@code lombok.cleanup.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Cleanup} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> CLEANUP_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.cleanup.flagUsage", "Emit a warning or error if @Cleanup is used.") {};
// ----- Delegate -----
/**
* lombok configuration: {@code lombok.delegate.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Delegate} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> DELEGATE_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.delegate.flagUsage", "Emit a warning or error if @Delegate is used.") {};
// ----- NonNull -----
/**
* lombok configuration: {@code lombok.nonNull.exceptionType} = one of: [{@code IllegalArgumentException}, {@code NullPointerException}, {@code JDK}, {@code Guava}, or {@code Assertion}].
*
* Sets the behavior of the generated nullcheck if {@code @NonNull} is applied to a method parameter, and a caller passes in {@code null}.<ul>
```

```
* <li>If the chosen configuration is {@code NullPointerException} (the default), or {@code
IllegalArgumentException}, that exception type is a thrown, with as message
<code><em>field-name</em> is marked non-null but is null</code>.</li>
* <li>If the chosen configuration is {@code Assert}, then an {@code assert} statement is generated.
This means an {@code AssertionError} will be thrown if assertions are on (VM started with {@code -ea}
parameter), and nothing happens if not.</li>
* <li>If the chosen configuration is {@code JDK}, a call to {@code java.util.Objects.requireNonNull} is
generated with the fieldname passed along (which throws {@code NullPointerException}).</li>
* <li>If the chosen configuration is {@code Guava}, a call to {@code
com.google.common.base.Preconditions.checkNotNull} is generated with the fieldname passed along
(which throws {@code NullPointerException}).</li>
* </ul>
* NB: The chosen nullcheck style is also used by {@code @Builder}'s {@code @Singular} annotation
to check any collections passed to a plural-form method.
*/
public static final ConfigurationKey<NullCheckExceptionType> NON_NULL_EXCEPTION_TYPE =
new ConfigurationKey<NullCheckExceptionType>("lombok.nonNull.exceptionType", "The type of the
exception to throw if a passed-in argument is null (Default: NullPointerException).") {};
/**
* lombok configuration: {@code lombok.nonNull.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @NonNull} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> NON_NULL_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.nonNull.flagUsage", "Emit a warning or error if @NonNull
is used.") {};
// ----- SneakyThrows -----
/**
* lombok configuration: {@code lombok.sneakyThrows.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @SneakyThrows} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> SNEAKY_THROWS_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.sneakyThrows.flagUsage", "Emit a warning or error if
@SneakyThrows is used.") {};
// ----- Synchronized -----
/**
* lombok configuration: {@code lombok.synchronized.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @Synchronized} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> SYNCHRONIZED_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.synchronized.flagUsage", "Emit a warning or error if
@Synchronized is used.") {};
// ----- val -----
/**
* lombok configuration: {@code lombok.val.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code val} results in a warning / error.
*/
```

```
public static final ConfigurationKey<FlagUsageType> VAL_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.val.flagUsage", "Emit a warning or error if 'val' is used.")
{};
public static final ConfigurationKey<FlagUsageType> VAR_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.var.flagUsage", "Emit a warning or error if 'var' is used.")
{};
// ----- With -----
/**
* lombok configuration: {@code lombok.with.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @With} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> WITH_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.with.flagUsage", "Emit a warning or error if @With is
used.") {};
// ##### Extern #####
// ----- Logging -----
/**
* lombok configuration: {@code lombok.log.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of any of the log annotations in {@code lombok.extern}{@code @Slf4j}
results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_ANY_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.flagUsage", "Emit a warning or error if any of the log
annotations is used.") {};
/**
* lombok configuration: {@code lombok.log.apacheCommons.flagUsage} = {@code WARNING} |
{@code ERROR}.
*
* If set, <em>any</em> usage of {@code @CommonsLog} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_COMMONS_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.apacheCommons.flagUsage", "Emit a warning or
error if @CommonsLog is used.") {};
/**
* lombok configuration: {@code lombok.log.javaUtilLogging.flagUsage} = {@code WARNING} |
{@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Log} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_JUL_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.javaUtilLogging.flagUsage", "Emit a warning or error if
@Log is used.") {};
/**
* lombok configuration: {@code lombok.log.log4j.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Log4j} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_LOG4J_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.log4j.flagUsage", "Emit a warning or error if @Log4j is
used.") {};
/**
```

```
* lombok configuration: {@code lombok.log.log4j2.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @Log4j2} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_LOG4J2_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.log4j2.flagUsage", "Emit a warning or error if @Log4j2
is used.") {};
/**
* lombok configuration: {@code lombok.log.slf4j.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Slf4j} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_SLF4J_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.slf4j.flagUsage", "Emit a warning or error if @Slf4j is
used.") {};
/**
* lombok configuration: {@code lombok.log.xslf4j.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @XSlf4j} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_XSLF4J_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.xslf4j.flagUsage", "Emit a warning or error if @XSlf4j
is used.") {};
/**
* lombok configuration: {@code lombok.log.jbosslog.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @JBossLog} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_JBOSSLOG_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.jbosslog.flagUsage", "Emit a warning or error if
@JBossLog is used.") {};
/**
* lombok configuration: {@code lombok.log.flogger.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @Flogger} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_FLOGGER_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.flogger.flagUsage", "Emit a warning or error if
@Flogger is used.") {};
/**
* lombok configuration: {@code lombok.log.fieldName} = &lt;String: aJavaIdentifier&gt; (Default:
{@code log}).
*
* If set the various log annotations (which make a log field) will use the stated identifier instead of
{@code log} as a name.
*/
public static final ConfigurationKey<IdentifierName> LOG_ANY_FIELD_NAME = new
ConfigurationKey<IdentifierName>("lombok.log.fieldName", "Use this name for the generated logger
fields (default: 'log').") {};
/**
```

```
* lombok configuration: {@code lombok.log.fieldIsStatic} = {@code true} | {@code false}.
*
* If not set, or set to {@code true}, the log field generated by the various log annotations will be {@code
static}.
*
* If set to {@code false}, these will be generated as instance fields instead.
*/
public static final ConfigurationKey<Boolean> LOG_ANY_FIELD_IS_STATIC = new
ConfigurationKey<Boolean>("lombok.log.fieldIsStatic", "Make the generated logger fields static
(default: true).") {};
// ----- Custom Logging -----
/**
* lombok configuration: {@code lombok.log.custom.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @CustomLog} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOG_CUSTOM_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.log.custom.flagUsage", "Emit a warning or error if
@CustomLog is used.") {};
/**
* lombok configuration: {@code lombok.log.custom.declaration} = &lt;logDeclaration string&gt;.
*
* The log declaration must follow the pattern:
* <br>
* {@code [LoggerType ]LoggerFactoryType.loggerFactoryMethod(loggerFactoryMethodParams)[(logg
erFactoryMethodParams)]}
* <br>
* It consists of:
* <ul>
* <li>Optional fully qualified logger type, e.g. {@code my.cool.Logger}, followed by space. If not
specified, it defaults to the <em>LoggerFactoryType</em>.
* <li>Fully qualified logger factory type, e.g. {@code my.cool.LoggerFactory}, followed by dot.
* <li>Factory method, e.g. {@code createLogger}. This must be a {@code public static} method in the
<em>LoggerFactoryType</em>.
* <li>At least one definition of factory method parameters, e.g. {@code ()} or {@code (TOPIC,TYPE)}.
The format inside the parentheses is a comma-separated list of parameter kinds.<br>
* The allowed parameters are: {@code TYPE} | {@code NAME} | {@code TOPIC} | {@code
NULL}.<br>
* There can be at most one parameter definition with {@code TOPIC} and at most one without {@code
TOPIC}. You can specify both.
* </ul>
*
* An example: {@code my.cool.Logger
my.cool.LoggerFactory.createLogger(TYPE)(TYPE,TOPIC)}<br>
* If no topic is provided in the usage of {@code @CustomLog}, the above will invoke {@code
LoggerFactory}'s {@code createLogger} method, passing in the type as a {@code java.lang.Class}
variable.<br>
* If a topic is provided, the overload of that method is invoked with 2 parameters: First the type (as
{@code Class}), then the topic (as {@code String}).
* <p>
* If this configuration key is not set, any usage of {@code @CustomLog} will result in an error.
*/
```

```
public static final ConfigurationKey<LogDeclaration> LOG_CUSTOM_DECLARATION = new
ConfigurationKey<LogDeclaration>("lombok.log.custom.declaration", "Define the generated custom
logger field.") {};
// ##### Experimental #####
/**
* lombok configuration: {@code lombok.experimental.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of any experimental features (from package {@code
lombok.experimental}) that haven't been
* promoted to a main feature results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> EXPERIMENTAL_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.experimental.flagUsage", "Emit a warning or error if an
experimental feature is used.") {};
// ----- Accessors -----
/**
* lombok configuration: {@code lombok.accessors.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @Accessors} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> ACCESSORS_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.accessors.flagUsage", "Emit a warning or error if
@Accessors is used.") {};
/**
* lombok configuration: {@code lombok.accessors.prefix} += &lt;String: prefix&gt;.
*
* For any class without an {@code @Accessors} that explicitly defines the {@code prefix} option, this
list of prefixes is used.
*/
public static final ConfigurationKey<List<String>> ACCESSORS_PREFIX = new
ConfigurationKey<List<String>>("lombok.accessors.prefix", "Strip this field prefix, like 'f' or 'm_', from
the names of generated getters, setters, and with-ers.") {};
/**
* lombok configuration: {@code lombok.accessors.chain} = {@code true} | {@code false}.
*
* For any class without an {@code @Accessors} that explicitly defines the {@code chain} option, this
value is used (default = false).
*/
public static final ConfigurationKey<Boolean> ACCESSORS_CHAIN = new
ConfigurationKey<Boolean>("lombok.accessors.chain", "Generate setters that return 'this' instead of
'void' (default: false).") {};
/**
* lombok configuration: {@code lombok.accessors.fluent} = {@code true} | {@code false}.
*
* For any class without an {@code @Accessors} that explicitly defines the {@code fluent} option, this
value is used (default = false).
*/
public static final ConfigurationKey<Boolean> ACCESSORS_FLUENT = new
ConfigurationKey<Boolean>("lombok.accessors.fluent", "Generate getters and setters using only the
field name (no get/set prefix) (default: false).") {};
/**
```

```
* lombok configuration: {@code lombok.accessors.makeFinal} = {@code true} | {@code false}.
*
* Unless an explicit {@code @Accessors} that explicitly defines the {@code makeFinal} option, this value is used (default = false).
*/
public static final ConfigurationKey<Boolean> ACCESSORS_MAKE_FINAL = new ConfigurationKey<Boolean>("lombok.accessors.makeFinal", "Generate getters, setters and with-ers with the 'final' modifier (default: false).") {};
/**
* lombok configuration: {@code lombok.accessors.capitalization} = {@code basic} | {@code beanspec}.
*
* Which capitalization rule is used to turn field names into getter/setter/with names and vice versa for field names that start with 1 lowercase letter, then 1 uppercase letter.
* basic = {@code uShape} becomes {@code getUShape}, beanspec = {@code uShape} becomes {@code getuShape} (default = basic).
*/
public static final ConfigurationKey<CapitalizationStrategy> ACCESSORS_JAVA_BEANS_SPEC_CAPITALIZATION = new ConfigurationKey<CapitalizationStrategy>("lombok.accessors.capitalization", "Which capitalization strategy to use when converting field names to accessor names and vice versa (default: basic).") {};
// ----- ExtensionMethod -----
/**
* lombok configuration: {@code lombok.extensionMethod.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @ExtensionMethod} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> EXTENSION_METHOD_FLAG_USAGE = new ConfigurationKey<FlagUsageType>("lombok.extensionMethod.flagUsage", "Emit a warning or error if @ExtensionMethod is used.") {};
// ----- FieldDefaults -----
/**
* lombok configuration: {@code lombok.fieldDefaults.defaultPrivate} = {@code true} | {@code false}.
*
* If set to <code>true</code> <em>any</em> field without an access modifier or {@code @PackagePrivate} is marked as {@code private} by lombok, in all source files compiled.
*/
public static final ConfigurationKey<Boolean> FIELD_DEFAULTS_PRIVATE_EVERYWHERE = new ConfigurationKey<Boolean>("lombok.fieldDefaults.defaultPrivate", "If true, fields without any access modifier, in any file (lombok annotated or not) are marked as private. Use @PackagePrivate or an explicit modifier to override this.") {};
/**
* lombok configuration: {@code lombok.fieldDefaults.defaultFinal} = {@code true} | {@code false}.
*
* If set to <code>true</code> <em>any</em> field without {@code @NonFinal} is marked as {@code final} by lombok, in all source files compiled.
*/
public static final ConfigurationKey<Boolean> FIELD_DEFAULTS_FINAL_EVERYWHERE = new ConfigurationKey<Boolean>("lombok.fieldDefaults.defaultFinal", "If true, fields, in any file (lombok annotated or not) are marked as final. Use @NonFinal to override this.") {};
/**
```

```
* lombok configuration: {@code lombok.fieldDefaults.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @FieldDefaults} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> FIELD_DEFAULTS_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.fieldDefaults.flagUsage", "Emit a warning or error if
@FieldDefaults is used.") {};
// ----- Helper -----
/**
* lombok configuration: {@code lombok.helper.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Helper} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> HELPER_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.helper.flagUsage", "Emit a warning or error if @Helper is
used.") {};
// ----- LOCKED -----
/**
* lombok configuration: {@code lombok.locked.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @Locked} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> LOCKED_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.locked.flagUsage", "Emit a warning or error if @Locked is
used.") {};
// ----- onX -----
/**
* lombok configuration: {@code lombok.onX.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code onX} results in a warning / error.
* <br>
* Specifically, this flags usage of {@code @Getter(onMethod=...)}, {@code @Setter(onParam=...)},
{@code @Setter(onMethod=...)}, {@code @XArgsConstructor(onConstructor=...)}.
*/
public static final ConfigurationKey<FlagUsageType> ON_X_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.onX.flagUsage", "Emit a warning or error if onX is used.")
{};
// ----- UtilityClass -----
/**
* lombok configuration: {@code lombok.utilityClass.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @UtilityClass} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> UTILITY_CLASS_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.utilityClass.flagUsage", "Emit a warning or error if
@UtilityClass is used.") {};
// ----- FieldNameConstants -----
/**
* lombok configuration: {@code lombok.fieldNameConstants.flagUsage} = {@code WARNING} |
{@code ERROR}.
*
```

```
* If set, <em>any</em> usage of {@code @FieldNameConstants} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> FIELD_NAME_CONSTANTS_FLAG_USAGE =
new ConfigurationKey<FlagUsageType>("lombok.fieldNameConstants.flagUsage", "Emit a warning or
error if @FieldNameConstants is used.") {};
/**
* lombok configuration: {@code lombok.fieldNameConstants.innerTypeName} = &lt;String:
AValidJavaTypeName&gt; (Default: {@code Fields}).
*
* The names of the constants generated by {@code @FieldNameConstants} will be prefixed with this
value.
*/
public static final ConfigurationKey<IdentifierName>
FIELD_NAME_CONSTANTS_INNER_TYPE_NAME = new
ConfigurationKey<IdentifierName>("lombok.fieldNameConstants.innerTypeName", "The default name
of the inner type generated by @FieldNameConstants. (default: 'Fields').") {};
/**
* lombok configuration: {@code lombok.fieldNameConstants.uppercase} = {@code true} | {@code
false}.
*
* If true, names of constants generated by {@code @FieldNameConstants} will be
UPPER_CASED_LIKE_A_CONSTANT. (Default: {@code false}).
*/
public static final ConfigurationKey<Boolean> FIELD_NAME_CONSTANTS_UPPERCASE = new
ConfigurationKey<Boolean>("lombok.fieldNameConstants.uppercase", "The default name of the
constants inside the inner type generated by @FieldNameConstants follow the variable name
precisely. If this config key is true, lombok will uppercase them as best it can. (default: false).") {};
// ----- SuperBuilder -----
/**
* lombok configuration: {@code lombok.superBuilder.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @SuperBuilder} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> SUPERBUILDER_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.superBuilder.flagUsage", "Emit a warning or error if
@SuperBuilder is used.") {};
// ----- WithBy -----
/**
* lombok configuration: {@code lombok.withBy.flagUsage} = {@code WARNING} | {@code ERROR}.
*
* If set, <em>any</em> usage of {@code @WithBy} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> WITHBY_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.withBy.flagUsage", "Emit a warning or error if @WithBy is
used.") {};
// ----- Jacksonized -----
/**
* lombok configuration: {@code lombok.jacksonized.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @Jacksonized} results in a warning / error.
*/
```

```
public static final ConfigurationKey<FlagUsageType> JACKSONIZED_FLAG_USAGE = new
ConfigurationKey<FlagUsageType>("lombok.jacksonized.flagUsage", "Emit a warning or error if
@Jacksonized is used.") {};
// ----- Configuration System -----
/**
* lombok configuration: {@code config.stopBubbling} = {@code true} | {@code false}.
*
* If not set, or set to {@code false}, the configuration system will look for {@code lombok.config} files in
the parent directory.
*
* If set to {@code true}, no further {@code lombok.config} files will be checked.
*/
public static final ConfigurationKey<Boolean> STOP_BUBBLING = new
ConfigurationKey<Boolean>("config.stopBubbling", "Tell the configuration system it should stop looking
for other configuration files (default: false).") {};
/**
* lombok configuration: {@code lombok.copyableAnnotations} += &lt;TypeName: fully-qualified
annotation class name&gt;.
*
* Copy these annotations to getters, setters, with methods, builder-setters, etc.
*/
public static final ConfigurationKey<List<TypeName>> COPYABLE_ANNOTATIONS = new
ConfigurationKey<List<TypeName>>("lombok.copyableAnnotations", "Copy these annotations to
getters, setters, with methods, builder-setters, etc.") {};
/**
* lombok configuration: {@code checkerframework} = {@code true} | {@code false} | &lt;String:
MajorVer.MinorVer&gt; (Default: false).
*
* If set, lombok will generate appropriate annotations from checkerframework.org on generated code. If
set to {@code true}, all relevant annotations from the most recent version of
* checkerframework.org that lombok supports will be generated. If set to a specific major/minor version
number, only checkerframework annotations introduced on or before the stated
* checkerframework.org version will be generated.
*/
public static final ConfigurationKey<CheckerFrameworkVersion> CHECKER_FRAMEWORK = new
ConfigurationKey<CheckerFrameworkVersion>("checkerframework", "If set with the version of
checkerframework.org (in major.minor, or just 'true' for the latest supported version), create relevant
checkerframework.org annotations for code lombok generates (default: false).") {};
/**
* lombok configuration: {@code lombok.standardException.flagUsage} = {@code WARNING} | {@code
ERROR}.
*
* If set, <em>any</em> usage of {@code @StandardException} results in a warning / error.
*/
public static final ConfigurationKey<FlagUsageType> STANDARD_EXCEPTION_FLAG_USAGE =
new ConfigurationKey<FlagUsageType>("lombok.standardException.flagUsage", "Emit a warning or
error if @StandardException is used.") {};
}
```

lombok/CustomLog.java

```
/*
* Copyright (C) 2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field based on a custom logger implementation.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project
lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;CustomLog
* public class LogExample {
* }
* </pre>
* With configuration:
* <pre>
* lombok.log.custom.declaration=my.cool.Logger my.cool.LoggerFactory.getLogger(NAME)
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final my.cool.Logger log =
my.cool.LoggerFactory.getLogger(LogExample.class.getName());
* }
* </pre>
* <p>
```

```
* Configuration must be provided in lombok.config, otherwise any usage of this annotation will result in a compile-time error.
*
* This annotation is valid for classes and enumerations.<br>
* @see lombok.extern.java.Log &#64;Log
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.slf4j.Slf4j &#64;Slf4j
* @see lombok.extern.slf4j.XSlf4j &#64;XSlf4j
* @see lombok.extern.jbosslog.JBossLog &#64;JBossLog
* @see lombok.extern.flogger.Flogger &#64;Flogger
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface CustomLog {
/**
*
* Sets a custom topic/category. Note that this requires you to specify a parameter configuration for your custom logger that includes {@code TOPIC}.
*
* @return The topic/category of the constructed Logger. By default (or for the empty string as topic), the parameter configuration without {@code TOPIC} is invoked.
*/
String topic() default "";
}
```

lombok/Data.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
```

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Generates getters for all fields, a useful toString method, and hashCode and equals implementations that check
* all non-transient fields. Will also generate setters for all non-final fields, as well as a constructor
* (except that no constructor will be generated if any explicitly written constructors already exist).
* <p>
* Equivalent to {@code @Getter @Setter @RequiredArgsConstructor @ToString @EqualsAndHashCode}.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/Data">the project lombok features page for &#64;Data</a>.
*
* @see Getter
* @see Setter
* @see RequiredArgsConstructor
* @see ToString
* @see EqualsAndHashCode
* @see lombok.Value
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface Data {
/**
* If you specify a static constructor name, then the generated constructor will be private, and
* instead a static factory method is created that other classes can use to create instances.
* We suggest the name: "of", like so:
*
* <pre>
* public @Data(staticConstructor = "of") class Point { final int x, y; }
* </pre>
*
* Default: No static constructor, instead the normal constructor is public.
*
* @return Name of static 'constructor' method to generate (blank = generate a normal constructor).
*/
String staticConstructor() default "";
}
```

lombok/Delegate.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
```

```
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* @deprecated Use {@link lombok.experimental.Delegate} instead.
*/
@Target({ElementType.FIELD, ElementType.METHOD})
@Retention(RetentionPolicy.SOURCE)
@Deprecated
public @interface Delegate {
/**
* Normally the type of the field is used as delegate type. However, to choose a different type to
delegate, you can list one (or more) types here. Note that types with
* type arguments can only be done as a field type. A solution for this is to create a private inner
interface/class with the appropriate types extended, and possibly
* with all methods you'd like to delegate listed, and then supply that class here. The field does not
actually have to implement the type you're delegating; the
* type listed here is used only to determine which delegate methods to generate.
*
* NB: All methods in {@code Object}, as well as {@code canEqual(Object other)} will never be
delegated.
*
* @return For each method (not already in {@code java.lang.Object}) in these types, generate a
delegate method.
*/
Class<?>[] types() default {};
/**
* Each method in any of the types listed here (include supertypes) will <em>not</em> be delegated.
*
* NB: All methods in {@code Object}, as well as {@code canEqual(Object other)} will never be
delegated.
*
* @return For each method (not already in {@code java.lang.Object}) in these types, skip generating a
delegate method (overrides {@code types()}).
*/
Class<?>[] excludes() default {};
}
```

lombok/EqualsAndHashCode.java

```
/*
* Copyright (C) 2009-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Generates implementations for the {@code equals} and {@code hashCode} methods inherited by all
objects, based on relevant fields.
* <p>
* Complete documentation is found at <a
href="https://projectlombok.org/features/EqualsAndHashCode">the project lombok features page for
&#64;EqualsAndHashCode</a>.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface EqualsAndHashCode {
/**
* Any fields listed here will not be taken into account in the generated {@code equals} and {@code
hashCode} implementations.
* Mutually exclusive with {@link #of()}.
* <p>
* Will soon be marked {@code @Deprecated}; use the {@code @EqualsAndHashCode.Exclude}
annotation instead.
*
* @return A list of fields to exclude.
*/
String[] exclude() default {};
```

```
/**
* If present, explicitly lists the fields that are to be used for identity.
* Normally, all non-static, non-transient fields are used for identity.
* <p>
* Mutually exclusive with {@link #exclude()}.
* <p>
* Will soon be marked {@code @Deprecated}; use the {@code @EqualsAndHashCode.Include}
annotation together with {@code @EqualsAndHashCode(onlyExplicitlyIncluded = true)}.
*
* @return A list of fields to use (<em>default</em>: all of them).
*/
String[] of() default {};
/**
* Call on the superclass's implementations of {@code equals} and {@code hashCode} before
calculating for the fields in this class.
* <strong>default: false</strong>
*
* @return Whether to call the superclass's {@code equals} implementation as part of the generated
equals algorithm.
*/
boolean callSuper() default false;
/**
* Normally, if getters are available, those are called. To suppress this and let the generated code use
the fields directly, set this to {@code true}.
* <strong>default: false</strong>
*
* @return If {@code true}, always use direct field access instead of calling the getter method.
*/
boolean doNotUseGetters() default false;
/**
* Determines how the result of the {@code hashCode} method will be cached.
* <strong>default: {@link CacheStrategy#NEVER}</strong>
*
* @return The {@code hashCode} cache strategy to be used.
*/
CacheStrategy cacheStrategy() default CacheStrategy.NEVER;
/**
* Any annotations listed here are put on the generated parameter of {@code equals} and {@code
canEqual}.
* This is useful to add for example a {@code Nullable} annotation.<br>
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @EqualsAndHashCode(onParam=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @EqualsAndHashCode(onParam_={@AnnotationsGohere})} // note the underscore after
{@code onParam}.
*
* @return List of annotations to apply to the generated parameter in the {@code equals()} method.
*/
AnyAnnotation[] onParam() default {};
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
```

```
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
/**
* Only include fields and methods explicitly marked with {@code @EqualsAndHashCode.Include}.
* Normally, all (non-static, non-transient) fields are included by default.
*
* @return If {@code true}, don't include non-static non-transient fields automatically (default: {@code
false}).
*/
boolean onlyExplicitlyIncluded() default false;
/**
* If present, do not include this field in the generated {@code equals} and {@code hashCode} methods.
*/
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.SOURCE)
public @interface Exclude {}
/**
* Configure the behaviour of how this member is treated in the {@code equals} and {@code hashCode}
implementation; if on a method, include the method's return value as part of calculating
hashCode/equality.
*/
@Target({ElementType.FIELD, ElementType.METHOD})
@Retention(RetentionPolicy.SOURCE)
public @interface Include {
/**
* Defaults to the method name of the annotated member.
* If on a method and the name equals the name of a default-included field, this member takes its place.
*
* @return If present, this method serves as replacement for the named field.
*/
String replaces() default "";
/**
* Higher ranks are considered first. Members of the same rank are considered in the order they appear
in the source file.
*
* If not explicitly set, the {@code default} rank for primitives is 1000, and for primitive wrappers 800.
*
* @return ordering within the generating {@code equals} and {@code hashCode} methods; higher
numbers are considered first.
*/
int rank() default 0;
}
public enum CacheStrategy {
/**
* Never cache. Perform the calculation every time the method is called.
*/
NEVER,
/**
* Cache the result of the first invocation of {@code hashCode} and use it for subsequent invocations.
```

* This can improve performance if all fields used for calculating the {@code hashCode} are immutable
* and thus every invocation of {@code hashCode} will always return the same value.
* Do not use this if there's any chance that different invocations of {@code
hashCode}
* might return different values.
*/
LAZY
}
}

lombok/Generated.java

```
/*
* Copyright (C) 2015-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Lombok automatically adds this annotation to all generated constructors, methods, fields, and types.
*
* You can mark the presence of this annotation as 'ignore it' for all code style and bug finding tools.
* <p>
* NB: As of v1.18.34, lombok adds this annotation by default; before then you had to add a lombok
config key.
* <p>
* If you want to opt out (not recommended), you can add {@code
lombok.addLombokGeneratedAnnotation = false} to
* {@code lombok.config}.
*/
```

```
@Target({ElementType.CONSTRUCTOR, ElementType.METHOD, ElementType.FIELD,
ElementType.TYPE})
@Retention(RetentionPolicy.CLASS)
public @interface Generated {
}
```

lombok/Getter.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Put on any field to make lombok build a standard getter.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/GetterSetter">the
project lombok features page for &#64;Getter and &#64;Setter</a>.
* <p>
* Even though it is not listed, this annotation also has the {@code onMethod} parameter. See the full
documentation for more details.
* <p>
* Example:
* <pre>
* private &#64;Getter int foo;
* </pre>
*
* will generate:
*
* <pre>
```

```
* public int getFoo() {
* return this.foo;
* }
* </pre>
* <p>
* This annotation can also be applied to a class, in which case it'll be as if all non-static fields that don't already have
* a {@code @Getter} annotation have the annotation.
*/
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface Getter {
/**
* If you want your getter to be non-public, you can specify an alternate access level here.
*
* @return The getter method will be generated with this access modifier.
*/
lombok.AccessLevel value() default lombok.AccessLevel.PUBLIC;
/**
* Any annotations listed here are put on the generated method.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around javac bugs).<br>
* up to JDK7:<br>
* {@code @Getter(onMethod=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @Getter(onMethod_={@AnnotationsGohere})} // note the underscore after {@code onMethod}.
*
* @return List of annotations to apply to the generated getter method.
*/
AnyAnnotation[] onMethod() default {};
boolean lazy() default false;
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/Locked.java

```
/*
* Copyright (C) 2021-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
```

```
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Guards all statements in an annotation method with a {@link java.util.concurrent.locks.Lock}.
* <p>
* For non-static methods, a field named {@code $lock} is used, and for static methods,
* {@code $LOCK} is used. These will be generated if needed and if they aren't already present.
* <p>
* Because {@link Locked} uses a different type of lock from {@link Locked.Read} and {@link
Locked.Write}, using both in
* the same class using the default names will result in a compile time error.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/Locked">the project
lombok features page for &#64;Locked</a>.
*/
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.SOURCE)
public @interface Locked {
/**
* Locks using a {@link java.util.concurrent.locks.ReadWriteLock#readLock()}.
*/
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.SOURCE)
public @interface Read {
/**
* Optional: specify the name of a different field to lock on. It is a compile time error if this field
* doesn't already exist (the fields are automatically generated only if you don't specify a specific name).
*
* @return Name of the field to lock on (blank = generate one).
*/
String value() default "";
}
/**
* Locks using a {@link java.util.concurrent.locks.ReadWriteLock#writeLock()}.
*/
@Target(ElementType.METHOD)
```

```
@Retention(RetentionPolicy.SOURCE)
public @interface Write {
/**
* Optional: specify the name of a different field to lock on. It is a compile time error if this field
* doesn't already exist (the fields are automatically generated only if you don't specify a specific name).
*
* @return Name of the field to lock on (blank = generate one).
*/
String value() default "";
}
/**
* Optional: specify the name of a different field to lock on. It is a compile time error if this field
* doesn't already exist (the fields are automatically generated only if you don't specify a specific name).
*
* @return Name of the field to lock on (blank = generate one).
*/
String value() default "";
}
```

lombok/Lombok.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
/**
* Useful utility methods to manipulate lombok-generated code.
*/
public class Lombok {
/**
* Throws any throwable 'sneakily' - you don't need to catch it, nor declare that you throw it onwards.
* The exception is still thrown - javac will just stop whining about it.
* <p>
```

```
* Example usage:
* <pre>public void run() {
* throw sneakyThrow(new IOException("You don't need to catch me!"));
* }</pre>
* <p>
* NB: The exception is not wrapped, ignored, swallowed, or redefined. The JVM actually does not know or care
* about the concept of a 'checked exception'. All this method does is hide the act of throwing a checked exception
* from the java compiler.
* <p>
* Note that this method has a return type of {@code RuntimeException}; it is advised you always call this
* method as argument to the {@code throw} statement to avoid compiler errors regarding no return
* statement and similar problems. This method won't of course return an actual {@code RuntimeException} -
* it never returns, it always throws the provided exception.
*
* @param t The throwable to throw without requiring you to catch its type.
* @return A dummy RuntimeException; this method never returns normally, it <em>always</em> throws an exception!
*/
public static RuntimeException sneakyThrow(Throwable t) {
if (t == null) throw new NullPointerException("t");
return Lombok.<RuntimeException>sneakyThrow0(t);
}
@SuppressWarnings("unchecked")
private static <T extends Throwable> T sneakyThrow0(Throwable t) throws T {
throw (T)t;
}
/**
* Returns the parameter directly.
*
* This method can be used to prevent a static analyzer to determine the nullness of the passed parameter.
*
* @param <T> the type of the parameter.
* @param value the value to return.
* @return value (this method just returns the parameter).
*/
public static <T> T preventNullAnalysis(T value) {
return value;
}
/**
* Ensures that the {@code value} is not {@code null}.
*
* @param <T> Type of the parameter.
* @param value the value to test for null.
* @param message the message of the {@link NullPointerException}.
* @return the value if it is not null.
* @throws NullPointerException with the {@code message} if the value is null.
*/
public static <T> T checkNotNull(T value, String message) {
```

```
if (value == null) throw new NullPointerException(message);
return value;
}
}
```

lombok/NoArgsConstructor.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Generates a no-args constructor.
* Will generate an error message if such a constructor cannot be written due to the existence of final
fields.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/constructor">the
project lombok features page for &#64;Constructor</a>.
* <p>
* Even though it is not listed, this annotation also has the {@code onConstructor} parameter. See the
full documentation for more details.
* <p>
* NB: Fields with constraints such as {@code @NonNull} will <em>NOT</em> be checked in a {@code
@NoArgsConstructor} constructor, of course!
*
* @see RequiredArgsConstructor
* @see AllArgsConstructor
*/
@Target(ElementType.TYPE)
```

```
@Retention(RetentionPolicy.SOURCE)
public @interface NoArgsConstructor {
/**
* If set, the generated constructor will be private, and an additional static 'constructor'
* is generated with the same argument list that wraps the real constructor.
*
* Such a static 'constructor' is primarily useful as it infers type arguments.
*
* @return Name of static 'constructor' method to generate (blank = generate a normal constructor).
*/
String staticName() default "";
/**
* Any annotations listed here are put on the generated constructor.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around javac bugs).<br>
* up to JDK7:<br>
* {@code @NoArgsConstructor(onConstructor=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @NoArgsConstructor(onConstructor_={@AnnotationsGohere})} // note the underscore after {@code onConstructor}.
*
* @return List of annotations to apply to the generated constructor.
*/
AnyAnnotation[] onConstructor() default {};
/**
* Sets the access level of the constructor. By default, generated constructors are {@code public}.
*
* @return The constructor will be generated with this access modifier.
*/
AccessLevel access() default lombok.AccessLevel.PUBLIC;
/**
* If {@code true}, initializes all final fields to 0 / null / false.
* Otherwise, a compile time error occurs.
*
* @return {@code} true to force generation of a no-args constructor, picking defaults if necessary to assign required fields.
*/
boolean force() default false;
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/NonNull.java

```
/*
```

* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok;
import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* If put on a parameter, lombok will insert a null-check at the start of the method / constructor's body, throwing a
* {@code NullPointerException} with the parameter's name as message. If put on a field, any generated method assigning
* a value to this field will also produce these null-checks.
*/
@Target({ElementType.FIELD, ElementType.METHOD, ElementType.PARAMETER, ElementType.LOCAL_VARIABLE, ElementType.TYPE_USE})
@Retention(RetentionPolicy.CLASS)
@Documented
public @interface NonNull {
}
```

lombok/RequiredArgsConstructor.java

/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:

```
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Generates a constructor with required arguments.
* Required arguments are final fields and fields with constraints such as {@code @NonNull}.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/constructor">the
project lombok features page for &#64;Constructor</a>.
* <p>
* Even though it is not listed, this annotation also has the {@code onConstructor} parameter. See the
full documentation for more details.
*
* @see NoArgsConstructor
* @see AllArgsConstructor
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface RequiredArgsConstructor {
/**
* If set, the generated constructor will be private, and an additional static 'constructor'
* is generated with the same argument list that wraps the real constructor.
*
* Such a static 'constructor' is primarily useful as it infers type arguments.
*
* @return Name of static 'constructor' method to generate (blank = generate a normal constructor).
*/
String staticName() default "";
/**
* Any annotations listed here are put on the generated constructor.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @RequiredArgsConstructor(onConstructor=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @RequiredArgsConstructor(onConstructor_={@AnnotationsGohere})} // note the underscore
after {@code onConstructor}.
*
* @return List of annotations to apply to the generated constructor.
```

```
*/
AnyAnnotation[] onConstructor() default {};
/**
* Sets the access level of the constructor. By default, generated constructors are {@code public}.
*
* @return The constructor will be generated with this access modifier.
*/
AccessLevel access() default lombok.AccessLevel.PUBLIC;
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/Setter.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Put on any field to make lombok build a standard setter.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/GetterSetter">the
project lombok features page for &#64;Getter and &#64;Setter</a>.
```

```
* <p>
* Even though it is not listed, this annotation also has the {@code onParam} and {@code onMethod}
parameter. See the full documentation for more details.
* <p>
* Example:
* <pre>
* private &#64;Setter int foo;
* </pre>
*
* will generate:
*
* <pre>
* public void setFoo(int foo) {
* this.foo = foo;
* }
* </pre>
*
* <p>
* This annotation can also be applied to a class, in which case it'll be as if all non-static fields that don't
already have
* a {@code Setter} annotation have the annotation.
*/
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface Setter {
/**
* If you want your setter to be non-public, you can specify an alternate access level here.
*
* @return The setter method will be generated with this access modifier.
*/
lombok.AccessLevel value() default lombok.AccessLevel.PUBLIC;
/**
* Any annotations listed here are put on the generated method.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @Setter(onMethod=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @Setter(onMethod_={@AnnotationsGohere})} // note the underscore after {@code
onMethod}.
*
* @return List of annotations to apply to the generated setter method.
*/
AnyAnnotation[] onMethod() default {};
/**
* Any annotations listed here are put on the generated method's parameter.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @Setter(onParam=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @Setter(onParam_={@AnnotationsGohere})} // note the underscore after {@code onParam}.
*
```

```
* @return List of annotations to apply to the generated parameter in the setter method.
*/
AnyAnnotation[] onParam() default {};
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/Singular.java

```
/*
* Copyright (C) 2015-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import static java.lang.annotation.ElementType.*;
import static java.lang.annotation.RetentionPolicy.*;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;
/**
* The singular annotation is used together with {@code @Builder} to create single element 'add'
methods in the builder for collections.
*/
@Target({FIELD, PARAMETER})
@Retention(SOURCE)
public @interface Singular {
/** @return The singular name of this field. If it's a normal english plural, lombok will figure it out
automatically. Otherwise, this parameter is mandatory. */
String value() default "";
```

/** @return If true, the plural variant (which takes a collection and adds each element inside) will treat {@code null} as an empty collection, i.e. do nothing. If {@code false} (the default), it is null checked as if annotated with {@code @lombok.NonNull}. */
boolean ignoreNullCollections() default false;
}

lombok/SneakyThrows.java

/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* @SneakyThrows will avoid javac's insistence that you either catch or throw onward any checked exceptions that
* statements in your method body declare they generate.
* <p>
* @SneakyThrows does not silently swallow, wrap into RuntimeException, or otherwise modify any exceptions of the listed
* checked exception types. The JVM does not check for the consistency of the checked exception system; javac does,
* and this annotation lets you opt out of its mechanism.
* <p>
* Complete documentation is found at the project lombok features page for @SneakyThrows.
* <p>
* Example:
* <pre>
* @SneakyThrows(UnsupportedEncodingException.class)

```
* public String utf8ToString(byte[] bytes) {
* return new String(bytes, "UTF-8");
* }
* </pre>
*
* Becomes:
* <pre>
* public String utf8ToString(byte[] bytes) {
* try {
* return new String(bytes, "UTF-8");
* } catch (UnsupportedEncodingException $uniqueName) {
* throw useMagicTrickeryToHideThisFromTheCompiler($uniqueName);
* // This trickery involves a bytecode transformer run automatically during the final stages of compilation;
* // there is no runtime dependency on lombok.
* }
* </pre>
*/
@Target({ElementType.METHOD, ElementType.CONSTRUCTOR})
@Retention(RetentionPolicy.SOURCE)
public @interface SneakyThrows {
/** @return The exception type(s) you want to sneakily throw onward. */
Class<? extends Throwable>[] value() default java.lang.Throwable.class;
//The fully qualified name is used for java.lang.Throwable in the parameter only. This works around a bug in javac:
// presence of an annotation processor throws off the type resolver for some reason.
}
```

lombok/Synchronized.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```

```
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Almost exactly like putting the 'synchronized' keyword on a method, except will synchronize on a private internal
* Object, so that other code not under your control doesn't meddle with your thread management by locking on
* your own instance.
* <p>
* For non-static methods, a field named {@code $lock} is used, and for static methods,
* {@code $LOCK} is used. These will be generated if needed and if they aren't already present. The contents
* of the fields will be serializable.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/Synchronized">the project lombok features page for &#64;Synchronized</a>.
*/
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.SOURCE)
public @interface Synchronized {
/**
* Optional: specify the name of a different field to lock on. It is a compile time error if this field
* doesn't already exist (the fields are automatically generated only if you don't specify a specific name).
*
* @return Name of the field to lock on (blank = generate one).
*/
String value() default "";
}
```

lombok/ToString.java

```
/*
* Copyright (C) 2009-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Generates an implementation for the {@code toString} method inherited by all objects, consisting of
printing the values of relevant fields.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/ToString">the project
lombok features page for &#64;ToString</a>.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface ToString {
/**
* Include the name of each field when printing it.
* <strong>default: true</strong>
*
* @return Whether or not to include the names of fields in the string produced by the generated
{@code toString()}.
*/
boolean includeFieldNames() default true;
/**
* Any fields listed here will not be printed in the generated {@code toString} implementation.
* Mutually exclusive with {@link #of()}.
* <p>
* Will soon be marked {@code @Deprecated}; use the {@code @ToString.Exclude} annotation
instead.
*
* @return A list of fields to exclude.
*/
String[] exclude() default {};
/**
* If present, explicitly lists the fields that are to be printed.
* Normally, all non-static fields are printed.
* <p>
* Mutually exclusive with {@link #exclude()}.
* <p>
* Will soon be marked {@code @Deprecated}; use the {@code @ToString.Include} annotation together
with {@code @ToString(onlyExplicitlyIncluded = true)}.
*
* @return A list of fields to use (<em>default</em>: all of them).
*/
String[] of() default {};
/**
* Include the result of the superclass's implementation of {@code toString} in the output.
* <strong>default: false</strong>
*
```

```
* @return Whether to call the superclass's {@code toString} implementation as part of the generated toString algorithm.
*/
boolean callSuper() default false;
/**
* Normally, if getters are available, those are called. To suppress this and let the generated code use the fields directly, set this to {@code true}.
* <strong>default: false</strong>
*
* @return If {@code true}, always use direct field access instead of calling the getter method.
*/
boolean doNotUseGetters() default false;
/**
* Only include fields and methods explicitly marked with {@code @ToString.Include}.
* Normally, all (non-static) fields are included by default.
*
* @return If {@code true}, don't include non-static fields automatically (default: {@code false}).
*/
boolean onlyExplicitlyIncluded() default false;
/**
* If present, do not include this field in the generated {@code toString}.
*/
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.SOURCE)
public @interface Exclude {}
/**
* Configure the behaviour of how this member is rendered in the {@code toString}; if on a method, include the method's return value in the output.
*/
@Target({ElementType.FIELD, ElementType.METHOD})
@Retention(RetentionPolicy.SOURCE)
public @interface Include {
// /** If true and the return value is {@code null}, omit this member entirely from the {@code toString} output. */
// boolean skipNull() default false; // -- We'll add it later, it requires a complete rework on the toString code we generate.
/**
* Higher ranks are printed first. Members of the same rank are printed in the order they appear in the source file.
*
* @return ordering within the generating {@code toString()}; higher numbers are printed first.
*/
int rank() default 0;
/**
* Defaults to the field / method name of the annotated member.
* If the name equals the name of a default-included field, this member takes its place.
*
* @return The name to show in the generated {@code toString()}. Also, if this annotation is on a method and the name matches an existing field, it replaces that field.
*/
String name() default "";
}
}
```

lombok/Value.java

```
/*
* Copyright (C) 2012-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Generates a lot of code which fits with a class that is a representation of an immutable entity.
* <p>
* Equivalent to {@code @Getter @FieldDefaults(makeFinal=true, level=AccessLevel.PRIVATE)
@AllArgsConstructor @ToString @EqualsAndHashCode}.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/Value">the project
lombok features page for &#64;Value</a>.
*
* @see lombok.Getter
* @see lombok.experimental.FieldDefaults
* @see lombok.AllArgsConstructor
* @see lombok.ToString
* @see lombok.EqualsAndHashCode
* @see lombok.Data
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface Value {
/**
* If you specify a static constructor name, then the generated constructor will be private, and
* instead a static factory method is created that other classes can use to create instances.
```

* We suggest the name: "of", like so:
*
* <pre>
* public @Value(staticConstructor = "of") class Point { final int x, y; }
* </pre>
*
* Default: No static constructor, instead the normal constructor is public.
*
* @return Name of static 'constructor' method to generate (blank = generate a normal constructor).
*/
String staticConstructor() default "";
}

lombok/With.java

/*
* Copyright (C) 2012-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Put on any field to make lombok build a 'with' - a withX method which produces a clone of this object (except for 1 field which gets a new value).
* <p>
* Complete documentation is found at the project lombok features page for @With.
* <p>
* Example:
* <pre>
* private @With final int foo;

```
* </pre>
*
* will generate:
*
* <pre>
* public SELF_TYPE withFoo(int foo) {
* return this.foo == foo ? this : new SELF_TYPE(otherField1, otherField2, foo);
* }
* </pre>
* <p>
* This annotation can also be applied to a class, in which case it'll be as if all non-static fields that don't already have
* a {@code With} annotation have the annotation.
*/
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface With {
/**
* If you want your with method to be non-public, you can specify an alternate access level here.
*
* @return The method will be generated with this access modifier.
*/
AccessLevel value() default AccessLevel.PUBLIC;
/**
* Any annotations listed here are put on the generated method.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around javac bugs).<br>
* up to JDK7:<br>
* {@code @With(onMethod=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @With(onMethod_={@AnnotationsGohere})} // note the underscore after {@code onMethod}.
*
* @return List of annotations to apply to the generated method.
*/
AnyAnnotation[] onMethod() default {};
/**
* Any annotations listed here are put on the generated method's parameter.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around javac bugs).<br>
* up to JDK7:<br>
* {@code @With(onParam=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @With(onParam_={@AnnotationsGohere})} // note the underscore after {@code onParam}.
*
* @return List of annotations to apply to the generated parameter in the method.
*/
AnyAnnotation[] onParam() default {};
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
```

```
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/bytecode/AsmUtil.java

```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.bytecode;
import org.objectweb.asm.ClassReader;
import org.objectweb.asm.ClassVisitor;
import org.objectweb.asm.ClassWriter;
import org.objectweb.asm.MethodVisitor;
import org.objectweb.asm.Opcodes;
import org.objectweb.asm.commons.JSRInlinerAdapter;
class AsmUtil {
private AsmUtil() {
throw new UnsupportedOperationException();
}
static byte[] fixJSRInlining(byte[] byteCode) {
ClassReader reader = new ClassReader(byteCode);
ClassWriter writer = new FixedClassWriter(reader, 0);
ClassVisitor visitor = new ClassVisitor(Opcodes.ASM9, writer) {
@Override public MethodVisitor visitMethod(int access, String name, String desc, String signature,
String[] exceptions) {
return new JSRInlinerAdapter(super.visitMethod(access, name, desc, signature, exceptions), access,
name, desc, signature, exceptions);
}
};
reader.accept(visitor, 0);
return writer.toByteArray();
```

}
}

lombok/bytecode/ClassFileMetaData.java

```
/*
* Copyright (C) 2010-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.bytecode;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
/**
* Utility to read the constant pool, header, and inheritance information of any class file.
*
* THIS IS A COPY OF lombok.launch.ClassFileMetaData - it is used by the shadowclassloader which can't rely on anything from outside the lombok.launch package, but everything in the launch package has to be package private.
* If there is a bug or an update is needed here - make sure to also update it in the lombok.bytecode package.
*/
public class ClassFileMetaData {
private static final byte UTF8 = 1;
private static final byte INTEGER = 3;
private static final byte FLOAT = 4;
private static final byte LONG = 5;
private static final byte DOUBLE = 6;
private static final byte CLASS = 7;
private static final byte STRING = 8;
private static final byte FIELD = 9;
private static final byte METHOD = 10;
private static final byte INTERFACE_METHOD = 11;
```

```
private static final byte NAME_TYPE = 12;
// New in java7: support for methodhandles and invokedynamic
private static final byte METHOD_HANDLE = 15;
private static final byte METHOD_TYPE = 16;
private static final byte DYNAMIC = 17;
private static final byte INVOKE_DYNAMIC = 18;
// New in java9: support for modules
private static final byte MODULE = 19;
private static final byte PACKAGE = 20;
private static final int NOT_FOUND = -1;
private static final int START_OF_CONSTANT_POOL = 8;
private final byte[] byteCode;
private final int maxPoolSize;
private final int[] offsets;
private final byte[] types;
private final String[] utf8s;
private final int endOfPool;
public ClassFileMetaData(byte[] byteCode) {
this.byteCode = byteCode;
maxPoolSize = readValue(START_OF_CONSTANT_POOL);
offsets = new int[maxPoolSize];
types = new byte[maxPoolSize];
utf8s = new String[maxPoolSize];
int position = 10;
for (int i = 1; i < maxPoolSize; i++) {
byte type = byteCode[position];
types[i] = type;
position++;
offsets[i] = position;
switch (type) {
case UTF8:
int length = readValue(position);
position += 2;
utf8s[i] = decodeString(position, length);
position += length;
break;
case CLASS:
case STRING:
case METHOD_TYPE:
case MODULE:
case PACKAGE:
position += 2;
break;
case METHOD_HANDLE:
position += 3;
break;
case INTEGER:
case FLOAT:
case FIELD:
case METHOD:
case INTERFACE_METHOD:
case NAME_TYPE:
case INVOKE_DYNAMIC:
```

```
case DYNAMIC:
position += 4;
break;
case LONG:
case DOUBLE:
position += 8;
i++;
break;
case 0:
break;
default:
throw new AssertionError("Unknown constant pool type " + type);
}
}
endOfPool = position;
}
private String decodeString(int pos, int size) {
int end = pos + size;
// the resulting string might be smaller
char[] result = new char[size];
int length = 0;
while (pos < end) {
int first = (byteCode[pos++] & 0xFF);
if (first < 0x80) {
result[length++] = (char)first;
} else if ((first & 0xE0) == 0xC0) {
int x = (first & 0x1F) << 6;
int y = (byteCode[pos++] & 0x3F);
result[length++] = (char)(x | y);
} else {
int x = (first & 0x0F) << 12;
int y = (byteCode[pos++] & 0x3F) << 6;
int z = (byteCode[pos++] & 0x3F);
result[length++] = (char)(x | y | z);
}
}
return new String(result, 0, length);
}
public int[] getOffsets(byte type) {
int[] out = new int[types.length];
int ptr = 0;
for (int i = 0; i < types.length; i++) {
if (types[i] != type) continue;
out[ptr++] = offsets[i];
}
return Arrays.copyOf(out, ptr);
}
/**
* Checks if the constant pool contains the provided 'raw' string. These are used as source material for
further JVM types, such as string constants, type references, etcetera.
*/
public boolean containsUtf8(String value) {
return findUtf8(value) != NOT_FOUND;
```

```
}
/**
* Checks if the constant pool contains a reference to the provided class.
*
* NB: Most uses of a type do <em>NOT</em> show up as a class in the constant pool.
* For example, the parameter types and return type of any method you invoke or declare, are stored as signatures and not as type references,
* but the type to which any method you invoke belongs, is. Read the JVM Specification for more information.
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
*/
public boolean usesClass(String className) {
return findClass(className) != NOT_FOUND;
}
/**
* Checks if the constant pool contains a reference to a given field, either for writing or reading.
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
*/
public boolean usesField(String className, String fieldName) {
int classIndex = findClass(className);
if (classIndex == NOT_FOUND) return false;
int fieldNameIndex = findUtf8(fieldName);
if (fieldNameIndex == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == FIELD && readValue(offsets[i]) == classIndex) {
int nameAndTypeIndex = readValue(offsets[i] + 2);
if (readValue(offsets[nameAndTypeIndex]) == fieldNameIndex) return true;
}
}
return false;
}
/**
* Checks if the constant pool contains a reference to a given method, with any signature (return type and parameter types).
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
*/
public boolean usesMethod(String className, String methodName) {
int classIndex = findClass(className);
if (classIndex == NOT_FOUND) return false;
int methodNameIndex = findUtf8(methodName);
if (methodNameIndex == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (isMethod(i) && readValue(offsets[i]) == classIndex) {
int nameAndTypeIndex = readValue(offsets[i] + 2);
if (readValue(offsets[nameAndTypeIndex]) == methodNameIndex) return true;
}
}
return false;
}
/**
```

```
* Checks if the constant pool contains a reference to a given method.
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
* @param descriptor must be provided JVM-style, such as {@code (IZ)Ljava/lang/String;}
*/
public boolean usesMethod(String className, String methodName, String descriptor) {
int classIndex = findClass(className);
if (classIndex == NOT_FOUND) return false;
int nameAndTypeIndex = findNameAndType(methodName, descriptor);
if (nameAndTypeIndex == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (isMethod(i) &&
readValue(offsets[i]) == classIndex &&
readValue(offsets[i] + 2) == nameAndTypeIndex) return true;
}
return false;
}
/**
* Checks if the constant pool contains the provided string constant, which implies the constant is used
somewhere in the code.
*
* NB: String literals get concatenated by the compiler.
* NB2: This method does NOT do any kind of normalization.
*/
public boolean containsStringConstant(String value) {
int index = findUtf8(value);
if (index == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == STRING && readValue(offsets[i]) == index) return true;
}
return false;
}
/**
* Checks if the constant pool contains the provided long constant, which implies the constant is used
somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsLong(long value) {
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == LONG && readLong(i) == value) return true;
}
return false;
}
/**
* Checks if the constant pool contains the provided double constant, which implies the constant is used
somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsDouble(double value) {
boolean isNan = Double.isNaN(value);
for (int i = 1; i < maxPoolSize; i++) {
```

```
if (types[i] == DOUBLE) {
double d = readDouble(i);
if (d == value || (isNan && Double.isNaN(d))) return true;
}
}
return false;
}
/**
* Checks if the constant pool contains the provided int constant, which implies the constant is used
somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsInteger(int value) {
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == INTEGER && readInteger(i) == value) return true;
}
return false;
}
/**
* Checks if the constant pool contains the provided float constant, which implies the constant is used
somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsFloat(float value) {
boolean isNan = Float.isNaN(value);
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == FLOAT) {
float f = readFloat(i);
if (f == value || (isNan && Float.isNaN(f))) return true;
}
}
return false;
}
private long readLong(int index) {
int pos = offsets[index];
return ((long)read32(pos)) << 32 | (read32(pos + 4) & 0x00000000FFFFFFFFL);
}
private double readDouble(int index) {
return Double.longBitsToDouble(readLong(index));
}
private int readInteger(int index) {
return read32(offsets[index]);
}
private float readFloat(int index) {
return Float.intBitsToFloat(readInteger(index));
}
private int read32(int pos) {
return (byteCode[pos] & 0xFF) << 24 | (byteCode[pos + 1] & 0xFF) << 16 | (byteCode[pos + 2] & 0xFF)
<< 8 | (byteCode[pos + 3] & 0xFF);
}
/**
```

```
* Returns the name of the class in JVM format, such as {@code java/lang/String}
*/
public String getClassName() {
return getClassName(readValue(endOfPool + 2));
}
/**
* Returns the name of the superclass in JVM format, such as {@code java/lang/Object}
*
* NB: If you try this on Object itself, you'll get {@code null}.<br />
* NB2: For interfaces and annotation interfaces, you'll always get {@code java/lang/Object}
*/
public String getSuperClassName() {
return getClassName(readValue(endOfPool + 4));
}
/**
* Returns the name of all implemented interfaces.
*/
public List<String> getInterfaces() {
int size = readValue(endOfPool + 6);
if (size == 0) return Collections.emptyList();
List<String> result = new ArrayList<String>();
for (int i = 0; i < size; i++) {
result.add(getClassName(readValue(endOfPool + 8 + (i * 2))));
}
return result;
}
/**
* A {@code toString()} like utility to dump all contents of the constant pool into a string.
*
* NB: No guarantees are made about the exact layout of this string. It is for informational purposes only,
don't try to parse it.<br />
* NB2: After a double or long, there's a JVM spec-mandated gap, which is listed as {@code (cont.)} in
the returned string.
*/
public String poolContent() {
StringBuilder result = new StringBuilder();
for (int i = 1; i < maxPoolSize; i++) {
result.append(String.format("#%02x: ", i));
int pos = offsets[i];
switch(types[i]) {
case UTF8:
result.append("Utf8 ").append(utf8s[i]);
break;
case CLASS:
result.append("Class ").append(getClassName(i));
break;
case STRING:
result.append("String \"").append(utf8s[readValue(pos)]).append("\"");
break;
case INTEGER:
result.append("int ").append(readInteger(i));
break;
case FLOAT:
```

```
result.append("float ").append(readFloat(i));
break;
case FIELD:
appendAccess(result.append("Field "), i);
break;
case METHOD:
case INTERFACE_METHOD:
appendAccess(result.append("Method "), i);
break;
case NAME_TYPE:
appendNameAndType(result.append("Name&Type "), i);
break;
case LONG:
result.append("long ").append(readLong(i));
break;
case DOUBLE:
result.append("double ").append(readDouble(i));
break;
case METHOD_HANDLE:
result.append("MethodHandle...");
break;
case METHOD_TYPE:
result.append("MethodType...");
break;
case DYNAMIC:
result.append("Dynamic...");
break;
case INVOKE_DYNAMIC:
result.append("InvokeDynamic...");
break;
case 0:
result.append("(cont.)");
break;
}
result.append("\n");
}
return result.toString();
}
private void appendAccess(StringBuilder result, int index) {
int pos = offsets[index];
result.append(getClassName(readValue(pos))).append(".");
appendNameAndType(result, readValue(pos + 2));
}
private void appendNameAndType(StringBuilder result, int index) {
int pos = offsets[index];
result.append(utf8s[readValue(pos)]).append(":").append(utf8s[readValue(pos + 2)]);
}
private String getClassName(int classIndex) {
if (classIndex < 1) return null;
return utf8s[readValue(offsets[classIndex])];
}
private boolean isMethod(int i) {
byte type = types[i];
```

```
return type == METHOD || type == INTERFACE_METHOD;
}
private int findNameAndType(String name, String descriptor) {
int nameIndex = findUtf8(name);
if (nameIndex == NOT_FOUND) return NOT_FOUND;
int descriptorIndex = findUtf8(descriptor);
if (descriptorIndex == NOT_FOUND) return NOT_FOUND;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == NAME_TYPE &&
readValue(offsets[i]) == nameIndex &&
readValue(offsets[i] + 2) == descriptorIndex) return i;
}
return NOT_FOUND;
}
private int findUtf8(String value) {
for (int i = 1; i < maxPoolSize; i++) {
if (value.equals(utf8s[i])) {
return i;
}
}
return NOT_FOUND;
}
private int findClass(String className) {
int index = findUtf8(className);
if (index == -1) return NOT_FOUND;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == CLASS && readValue(offsets[i]) == index) return i;
}
return NOT_FOUND;
}
/**
* Reads a 16-bit value at {@code position}
*/
private int readValue(int position) {
return ((byteCode[position] & 0xFF) << 8) | (byteCode[position + 1] & 0xFF);
}
}
```

lombok/bytecode/FixedClassWriter.java

```
/*
* Copyright (C) 2010-2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.bytecode;
import org.objectweb.asm.ClassReader;
import org.objectweb.asm.ClassWriter;
class FixedClassWriter extends ClassWriter {
FixedClassWriter(ClassReader classReader, int flags) {
super(classReader, flags);
}
FixedClassWriter(int flags) {
super(flags);
}
@Override protected String getCommonSuperClass(String type1, String type2) {
//By default, ASM will attempt to live-load the class types, which will fail if meddling with classes in an
//environment with custom classloaders, such as Equinox. It's just an optimization; returning Object is
always legal.
try {
return super.getCommonSuperClass(type1, type2);
} catch (OutOfMemoryError e) {
throw e;
} catch (Throwable t) {
return "java/lang/Object";
}
}
}
```

lombok/bytecode/PoolConstantsApp.java

```
/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.bytecode;
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import lombok.core.LombokApp;
import lombok.spi.Provides;
import com.zwitserloot.cmdreader.CmdReader;
import com.zwitserloot.cmdreader.Description;
import com.zwitserloot.cmdreader.InvalidCommandLineException;
import com.zwitserloot.cmdreader.Mandatory;
import com.zwitserloot.cmdreader.Sequential;
import com.zwitserloot.cmdreader.Shorthand;
@Provides
public class PoolConstantsApp extends LombokApp {
@Override public String getAppName() {
return "Xprintpool";
}
@Override public String getAppDescription() {
return "Prints the content of the constant pool to standard out.";
}
@Override public boolean isDebugTool() {
return true;
}
public static class CmdArgs {
@Sequential
@Mandatory
@Description("paths to class files to be printed. If a directory is named, all files (recursively) in that
directory will be printed.")
private List<String> classFiles = new ArrayList<String>();
@Shorthand({"h", "?"})
@Description("Shows this help text")
boolean help = false;
}
@Override public int runApp(List<String> raw) throws Exception {
CmdReader<CmdArgs> reader = CmdReader.of(CmdArgs.class);
CmdArgs args;
try {
args = reader.make(raw.toArray(new String[0]));
if (args.help) {
System.out.println(reader.generateCommandLineHelp("java -jar lombok.jar -printpool"));
return 0;
}
} catch (InvalidCommandLineException e) {
System.err.println(e.getMessage());
System.err.println(reader.generateCommandLineHelp("java -jar lombok.jar -printpool"));
return 1;
}
```

```
List<File> filesToProcess = PostCompilerApp.cmdArgsToFiles(args.classFiles);
int filesVisited = 0;
boolean moreThanOne = filesToProcess.size() > 1;
for (File file : filesToProcess) {
if (!file.exists() || !file.isFile()) {
System.out.printf("Cannot find file '%s'\n", file.getAbsolutePath());
continue;
}
filesVisited++;
if (moreThanOne) System.out.printf("Processing '%s'\n", file.getAbsolutePath());
System.out.println(new ClassFileMetaData(PostCompilerApp.readFile(file)).poolContent());
}
if (moreThanOne) System.out.printf("Total files visited: %d\n", filesVisited);
return filesVisited == 0 ? 1 : 0;
}
}
```

lombok/bytecode/PostCompilerApp.java

```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.bytecode;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import lombok.core.DiagnosticsReceiver;
import lombok.core.LombokApp;
```

```
import lombok.core.PostCompiler;
import lombok.spi.Provides;
import com.zwitserloot.cmdreader.CmdReader;
import com.zwitserloot.cmdreader.Description;
import com.zwitserloot.cmdreader.InvalidCommandLineException;
import com.zwitserloot.cmdreader.Mandatory;
import com.zwitserloot.cmdreader.Sequential;
import com.zwitserloot.cmdreader.Shorthand;
@Provides
public class PostCompilerApp extends LombokApp {
@Override public List<String> getAppAliases() {
return Arrays.asList("post", "postcompile");
}
@Override public String getAppDescription() {
return "Runs registered post compiler handlers to against existing class files, modifying them in the
process.";
}
@Override public String getAppName() {
return "post-compile";
}
public static class CmdArgs {
@Sequential
@Mandatory
@Description("paths to class files to be converted. If a directory is named, all files (recursively) in that
directory will be converted.")
private List<String> classFiles = new ArrayList<String>();
@Shorthand("v")
@Description("Prints lots of status information as the post compiler runs")
boolean verbose = false;
@Shorthand({"h", "?"})
@Description("Shows this help text")
boolean help = false;
}
@Override public int runApp(List<String> raw) throws Exception {
CmdReader<CmdArgs> reader = CmdReader.of(CmdArgs.class);
CmdArgs args;
try {
args = reader.make(raw.toArray(new String[0]));
if (args.help) {
System.out.println(reader.generateCommandLineHelp("java -jar lombok.jar post-compile"));
return 0;
}
} catch (InvalidCommandLineException e) {
System.err.println(e.getMessage());
System.err.println(reader.generateCommandLineHelp("java -jar lombok.jar post-compile"));
return 1;
}
int filesVisited = 0, filesTouched = 0;
for (File file : cmdArgsToFiles(args.classFiles)) {
if (!file.exists() || !file.isFile()) {
System.out.printf("Cannot find file '%s'\n", file);
continue;
}
```

```
filesVisited++;
if (args.verbose) System.out.println("Processing " + file.getAbsolutePath());
byte[] original = readFile(file);
byte[] clone = original.clone();
byte[] transformed = PostCompiler.applyTransformations(clone, file.toString(),
DiagnosticsReceiver.CONSOLE);
if (clone != transformed && !Arrays.equals(original, transformed)) {
filesTouched++;
if (args.verbose) System.out.println("Rewriting " + file.getAbsolutePath());
writeFile(file, transformed);
}
}
if (args.verbose) {
System.out.printf("Total files visited: %d total files changed: %d\n", filesVisited, filesTouched);
}
return filesVisited == 0 ? 1 : 0;
}
static List<File> cmdArgsToFiles(List<String> fileNames) {
List<File> filesToProcess = new ArrayList<File>();
for (String f : fileNames) addFiles(filesToProcess, f);
return filesToProcess;
}
static void addFiles(List<File> filesToProcess, String f) {
File file = new File(f);
if (file.isDirectory()) {
addRecursively(filesToProcess, file);
} else {
filesToProcess.add(file);
}
}
static void addRecursively(List<File> filesToProcess, File file) {
for (File f : file.listFiles()) {
if (f.isDirectory()) addRecursively(filesToProcess, f);
else if (f.getName().endsWith(".class")) filesToProcess.add(f);
}
}
static byte[] readFile(File file) throws IOException {
byte[] buffer = new byte[1024];
ByteArrayOutputStream bytes = new ByteArrayOutputStream();
FileInputStream fileInputStream = new FileInputStream(file);
try {
while (true) {
int read = fileInputStream.read(buffer);
if (read == -1) break;
bytes.write(buffer, 0, read);
}
} finally {
fileInputStream.close();
}
return bytes.toByteArray();
}
static void writeFile(File file, byte[] transformed) throws IOException {
FileOutputStream out = new FileOutputStream(file);
```

```
try {
out.write(transformed);
} finally {
out.close();
}
}
}
```

lombok/bytecode/PreventNullAnalysisRemover.java

```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.bytecode;
import static lombok.bytecode.AsmUtil.*;
import java.util.concurrent.atomic.AtomicBoolean;
import lombok.core.DiagnosticsReceiver;
import lombok.core.PostCompilerTransformation;
import lombok.spi.Provides;
import org.objectweb.asm.ClassReader;
import org.objectweb.asm.ClassVisitor;
import org.objectweb.asm.ClassWriter;
import org.objectweb.asm.MethodVisitor;
import org.objectweb.asm.Opcodes;
@Provides
public class PreventNullAnalysisRemover implements PostCompilerTransformation {
@Override public byte[] applyTransformations(byte[] original, String fileName, DiagnosticsReceiver
diagnostics) {
if (!new ClassFileMetaData(original).usesMethod("lombok/Lombok", "preventNullAnalysis")) return null;
byte[] fixedByteCode = fixJSRInlining(original);
ClassReader reader = new ClassReader(fixedByteCode);
ClassWriter writer = new FixedClassWriter(0);
final AtomicBoolean changesMade = new AtomicBoolean();
```

```
class PreventNullAnalysisVisitor extends MethodVisitor {
PreventNullAnalysisVisitor(MethodVisitor mv) {
super(Opcodes.ASM9, mv);
}
@Override public void visitMethodInsn(int opcode, String owner, String name, String desc, boolean itf)
{
boolean hit = true;
if (hit && opcode != Opcodes.INVOKESTATIC) hit = false;
if (hit && !"preventNullAnalysis".equals(name)) hit = false;
if (hit && !"lombok/Lombok".equals(owner)) hit = false;
if (hit && !"(Ljava/lang/Object;)Ljava/lang/Object;".equals(desc)) hit = false;
if (hit) {
changesMade.set(true);
if (System.getProperty("lombok.debugAsmOnly", null) != null) super.visitMethodInsn(opcode, owner,
name, desc, itf); // DEBUG for issue 470!
} else {
super.visitMethodInsn(opcode, owner, name, desc, itf);
}
}
}
reader.accept(new ClassVisitor(Opcodes.ASM9, writer) {
@Override public MethodVisitor visitMethod(int access, String name, String desc, String signature,
String[] exceptions) {
return new PreventNullAnalysisVisitor(super.visitMethod(access, name, desc, signature, exceptions));
}
}, 0);
return changesMade.get() ? writer.toByteArray() : null;
}
}
```

lombok/bytecode/SneakyThrowsRemover.java


```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
```

```
* THE SOFTWARE.
*/
package lombok.bytecode;
import static lombok.bytecode.AsmUtil.*;
import java.util.concurrent.atomic.AtomicBoolean;
import lombok.core.DiagnosticsReceiver;
import lombok.core.PostCompilerTransformation;
import lombok.spi.Provides;
import org.objectweb.asm.ClassReader;
import org.objectweb.asm.ClassVisitor;
import org.objectweb.asm.ClassWriter;
import org.objectweb.asm.Handle;
import org.objectweb.asm.Label;
import org.objectweb.asm.MethodVisitor;
import org.objectweb.asm.Opcodes;
@Provides
public class SneakyThrowsRemover implements PostCompilerTransformation {
@Override public byte[] applyTransformations(byte[] original, String fileName, final
DiagnosticsReceiver diagnostics) {
if (!new ClassFileMetaData(original).usesMethod("lombok/Lombok", "sneakyThrow")) return null;
byte[] fixedByteCode = fixJSRInlining(original);
ClassReader reader = new ClassReader(fixedByteCode);
ClassWriter writer = new ClassWriter(0);
final AtomicBoolean changesMade = new AtomicBoolean();
class SneakyThrowsRemoverVisitor extends MethodVisitor {
SneakyThrowsRemoverVisitor(MethodVisitor mv) {
super(Opcodes.ASM9, mv);
}
private boolean methodInsnQueued = false;
@Override public void visitMethodInsn(int opcode, String owner, String name, String desc, boolean itf)
{
if (
opcode == Opcodes.INVOKESTATIC &&
"sneakyThrow".equals(name) &&
"lombok/Lombok".equals(owner) &&
"(Ljava/lang/Throwable;)Ljava/lang/RuntimeException;".equals(desc)) {
if (System.getProperty("lombok.debugAsmOnly", null) != null) {
super.visitMethodInsn(opcode, owner, name, desc, itf); // DEBUG for issue 470!
} else {
methodInsnQueued = true;
}
} else {
super.visitMethodInsn(opcode, owner, name, desc, itf);
}
}
private void handleQueue() {
if (!methodInsnQueued) return;
super.visitMethodInsn(Opcodes.INVOKESTATIC, "lombok/Lombok", "sneakyThrow",
"(Ljava/lang/Throwable;)Ljava/lang/RuntimeException;", false);
methodInsnQueued = false;
diagnostics.addWarning("Proper usage is: throw lombok.Lombok.sneakyThrow(someException);. You
did not 'throw' it. Because of this, the call to sneakyThrow " +
"remains in your classfile and you will need lombok.jar on the classpath at runtime.");
```

```
}
@Override public void visitInsn(int arg0) {
if (methodInsnQueued && arg0 == Opcodes.ATHROW) {
changesMade.set(true);
// As expected, the required ATHROW. We can now safely 'eat' the previous call.
methodInsnQueued = false;
}
handleQueue();
super.visitInsn(arg0);
}
@Override public void visitFrame(int arg0, int arg1, Object[] arg2, int arg3, Object[] arg4) {
handleQueue();
super.visitFrame(arg0, arg1, arg2, arg3, arg4);
}
@Override public void visitIincInsn(int arg0, int arg1) {
handleQueue();
super.visitIincInsn(arg0, arg1);
}
@Override public void visitFieldInsn(int arg0, String arg1, String arg2, String arg3) {
handleQueue();
super.visitFieldInsn(arg0, arg1, arg2, arg3);
}
@Override public void visitIntInsn(int arg0, int arg1) {
handleQueue();
super.visitIntInsn(arg0, arg1);
}
@Override public void visitEnd() {
handleQueue();
super.visitEnd();
}
@Override public void visitInvokeDynamicInsn(String arg0, String arg1, Handle arg2, Object... arg3) {
handleQueue();
super.visitInvokeDynamicInsn(arg0, arg1, arg2, arg3);
}
@Override public void visitLabel(Label arg0) {
handleQueue();
super.visitLabel(arg0);
}
@Override public void visitJumpInsn(int arg0, Label arg1) {
handleQueue();
super.visitJumpInsn(arg0, arg1);
}
@Override public void visitLdcInsn(Object arg0) {
handleQueue();
super.visitLdcInsn(arg0);
}
@Override public void visitLocalVariable(String arg0, String arg1, String arg2, Label arg3, Label arg4,
int arg5) {
handleQueue();
super.visitLocalVariable(arg0, arg1, arg2, arg3, arg4, arg5);
}
@Override public void visitMaxs(int arg0, int arg1) {
handleQueue();
```

```
super.visitMaxs(arg0, arg1);
}
@Override public void visitLookupSwitchInsn(Label arg0, int[] arg1, Label[] arg2) {
handleQueue();
super.visitLookupSwitchInsn(arg0, arg1, arg2);
}
@Override public void visitMultiANewArrayInsn(String arg0, int arg1) {
handleQueue();
super.visitMultiANewArrayInsn(arg0, arg1);
}
@Override public void visitVarInsn(int arg0, int arg1) {
handleQueue();
super.visitVarInsn(arg0, arg1);
}
@Override public void visitTryCatchBlock(Label arg0, Label arg1, Label arg2, String arg3) {
handleQueue();
super.visitTryCatchBlock(arg0, arg1, arg2, arg3);
}
@Override public void visitTableSwitchInsn(int arg0, int arg1, Label arg2, Label... arg3) {
handleQueue();
super.visitTableSwitchInsn(arg0, arg1, arg2, arg3);
}
@Override public void visitTypeInsn(int arg0, String arg1) {
handleQueue();
super.visitTypeInsn(arg0, arg1);
}
}
reader.accept(new ClassVisitor(Opcodes.ASM9, writer) {
@Override public MethodVisitor visitMethod(int access, String name, String desc, String signature,
String[] exceptions) {
return new SneakyThrowsRemoverVisitor(super.visitMethod(access, name, desc, signature,
exceptions));
}
}, 0);
return changesMade.get() ? writer.toByteArray() : null;
}
}
```

lombok/bytecode/package-info.java

```
/*
* Copyright (C) 2009-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* This package contains utilities and handlers for the 'post-process class files' aspect of
* lombok. Lombok's class file post processing capabilities are based on Objectweb's ASM library.
*
* NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.
*/
package lombok.bytecode;

lombok/core/AST.java

/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import static lombok.Lombok.sneakyThrow;
import java.lang.reflect.Array;
import java.lang.reflect.Field;
import java.lang.reflect.Modifier;
import java.lang.reflect.ParameterizedType;
import java.lang.reflect.Type;
import java.net.URI;
import java.util.ArrayList;

```
import java.util.Collection;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import lombok.core.configuration.ConfigurationKey;
import lombok.core.debug.HistogramTracker;
import lombok.permit.Permit;
/**
* Lombok wraps the AST produced by a target platform into its own AST system, mostly because both
Eclipse and javac
* do not allow upward traversal (from a method to its owning type, for example).
*
* @param A Self-type.
* @param L type of all LombokNodes.
* @param N The common type of all AST nodes in the internal representation of the target platform.
* For example, JCTree for javac, and ASTNode for Eclipse.
*/
public abstract class AST<A extends AST<A, L, N>, L extends LombokNode<A, L, N>, N> {
/** The kind of node represented by a given AST.Node object. */
public enum Kind {
COMPILATION_UNIT, TYPE, FIELD, INITIALIZER, METHOD, ANNOTATION, ARGUMENT, LOCAL,
STATEMENT, TYPE_USE;
}
private L top;
private final String fileName;
private final String packageDeclaration;
private final ImportList imports;
private TypeResolver importsAsResolver;
Map<N, N> identityDetector = new IdentityHashMap<N, N>();
private Map<N, L> nodeMap = new IdentityHashMap<N, L>();
private boolean changed = false;
// The supertypes which are considered AST Node children. Usually, the Statement, and the
Expression,
// though some platforms (such as Eclipse) group these under one common supertype.
private final Collection<Class<? extends N>> statementTypes;
private static final HistogramTracker configTracker = System.getProperty("lombok.timeConfig") == null
? null : new HistogramTracker("lombok.config");
protected AST(String fileName, String packageDeclaration, ImportList imports, Collection<Class<?
extends N>> statementTypes) {
this.fileName = fileName == null ? "(unknown).java" : fileName;
this.packageDeclaration = packageDeclaration;
this.imports = imports;
this.statementTypes = statementTypes;
}
/**
* Attempts to find the absolute path (in URI form) to the source file represented by this AST.
*
* May return {@code null} if this cannot be done. We don't yet know under which conditions this will
happen.
*/
public abstract URI getAbsoluteFileLocation();
```

```
public void setChanged() {
this.changed = true;
}
protected void clearChanged() {
this.changed = false;
}
public boolean isChanged() {
return changed;
}
/** Set the node object that wraps the internal Compilation Unit node. */
protected void setTop(L top) {
this.top = top;
}
/**
* Return the content of the package declaration on this AST's top (Compilation Unit) node.
*
* Example: "java.util".
*/
public final String getPackageDeclaration() {
return packageDeclaration;
}
/**
* Return the contents of each non-static import statement on this AST's top (Compilation Unit) node.
*/
public final ImportList getImportList() {
return imports;
}
/**
* Return the contents of each non-static import statement on this AST's top (Compilation Unit) node,
packed into a (cached) TypeResolver.
*/
public final TypeResolver getImportListAsTypeResolver() {
if (importsAsResolver != null) return importsAsResolver;
return importsAsResolver = new TypeResolver(getImportList());
}
/**
* Puts the given node in the map so that javac/Eclipse's own internal AST object can be translated to
* an AST.Node object. Also registers the object as visited to avoid endless loops.
*/
protected L putInMap(L node) {
nodeMap.put(node.get(), node);
identityDetector.put(node.get(), node.get());
return node;
}
/** Returns the node map, that can map javac/Eclipse internal AST objects to AST.Node objects. */
protected Map<N, L> getNodeMap() {
return nodeMap;
}
/** Clears the registry that avoids endless loops, and empties the node map. The existing node map
* object is left untouched, and instead a new map is created. */
protected void clearState() {
identityDetector = new IdentityHashMap<N, N>();
nodeMap = new IdentityHashMap<N, L>();
```

```
}
/**
* Marks the stated node as handled (to avoid endless loops if 2 nodes refer to each other, or a node
* refers to itself). Will then return true if it was already set as handled before this call - in which
* case you should do nothing lest the AST build process loops endlessly.
*/
protected boolean setAndGetAsHandled(N node) {
return identityDetector.put(node, node) != null;
}
public String getFileName() {
return fileName;
}
/** The AST.Node object representing the Compilation Unit. */
public L top() {
return top;
}
/** Maps a javac/Eclipse internal AST Node to the appropriate AST.Node object. */
public L get(N node) {
return nodeMap.get(node);
}
/**
* Returns the JLS spec version that the compiler uses to parse and compile this AST.
* For example, if -source 1.6 is on the command line, this will return {@code 6}.
*/
public int getSourceVersion() {
return 6;
}
/**
* Returns the latest version of the java language specification supported by the host compiler.
* For example, if compiling with javac v1.7, this returns {@code 7}.
*
* NB: Even if -source (lower than maximum) is specified, this method still returns the maximum
supported number.
*/
public int getLatestJavaSpecSupported() {
return 6;
}
@SuppressWarnings({"unchecked", "rawtypes"})
L replaceNewWithExistingOld(Map<N, L> oldNodes, L newNode) {
L oldNode = oldNodes.get(newNode.get());
L targetNode = oldNode == null ? newNode : oldNode;
List children = new ArrayList();
for (L child : newNode.children) {
L oldChild = replaceNewWithExistingOld(oldNodes, child);
children.add(oldChild);
oldChild.parent = targetNode;
}
targetNode.children = LombokImmutableList.copyOf(children);
return targetNode;
}
/** Build an AST.Node object for the stated internal (javac/Eclipse) AST Node object. */
protected abstract L buildTree(N item, Kind kind);
/**
```

```
* Represents a field that contains AST children.
*/
protected static class FieldAccess {
/** The actual field. */
public final Field field;
/** Dimensions of the field. Works for arrays, or for java.util.collections. */
public final int dim;
FieldAccess(Field field, int dim) {
this.field = field;
this.dim = dim;
}
}
private static final ConcurrentMap<Class<?>, FieldAccess[]> fieldsOfASTClasses = new
ConcurrentHashMap<Class<?>, FieldAccess[]>();
/** Returns FieldAccess objects for the stated class. Each field that contains objects of the kind
returned by
* {@link #getStatementTypes()}, either directly or inside of an array or java.util.collection (or
array-of-arrays,
* or collection-of-collections, et cetera), is returned.
*/
protected FieldAccess[] fieldsOf(Class<?> c) {
FieldAccess[] fields = fieldsOfASTClasses.get(c);
if (fields != null) return fields;
List<FieldAccess> fieldList = new ArrayList<FieldAccess>();
getFields(c, fieldList);
fieldsOfASTClasses.putIfAbsent(c, fieldList.toArray(new FieldAccess[0]));
return fieldsOfASTClasses.get(c);
}
private void getFields(Class<?> c, Collection<FieldAccess> fields) {
if (c == Object.class || c == null) return;
for (Field f : c.getDeclaredFields()) {
if (Modifier.isStatic(f.getModifiers())) continue;
Class<?> t = f.getType();
int dim = 0;
if (t.isArray()) {
while (t.isArray()) {
dim++;
t = t.getComponentType();
}
} else {
while (Collection.class.isAssignableFrom(t)) {
dim++;
t = getComponentType(f.getGenericType());
}
}
if (shouldDrill(c, t, f.getName())) {
Permit.setAccessible(f);
fields.add(new FieldAccess(f, dim));
}
}
getFields(c.getSuperclass(), fields);
}
private Class<?> getComponentType(Type type) {
```

```
if (type instanceof ParameterizedType) {
Type component = ((ParameterizedType) type).getActualTypeArguments()[0];
return component instanceof Class<?> ? (Class<?>) component : Object.class;
}
return Object.class;
}
private boolean shouldDrill(Class<?> parentType, Class<?> childType, String fieldName) {
for (Class<?> statementType : statementTypes) {
if (statementType.isAssignableFrom(childType)) return true;
}
return false;
}
/**
* buildTree implementation that uses reflection to find all child nodes by way of inspecting
* the fields. */
protected Collection<L> buildWithField(Class<L> nodeType, N statement, FieldAccess fa) {
List<L> list = new ArrayList<L>();
buildWithField0(nodeType, statement, fa, list);
return list;
}
/**
* Uses reflection to find the given direct child on the given statement, and replace it with a new child.
*/
protected boolean replaceStatementInNode(N statement, N oldN, N newN) {
for (FieldAccess fa : fieldsOf(statement.getClass())) {
if (replaceStatementInField(fa, statement, oldN, newN)) return true;
}
return false;
}
private boolean replaceStatementInField(FieldAccess fa, N statement, N oldN, N newN) {
try {
Object o = fa.field.get(statement);
if (o == null) return false;
if (o == oldN) {
fa.field.set(statement, newN);
return true;
}
if (fa.dim > 0) {
if (o.getClass().isArray()) {
return replaceStatementInArray(o, oldN, newN);
} else if (Collection.class.isInstance(o)) {
return replaceStatementInCollection(fa.field, statement, new ArrayList<Collection<?>>(),
(Collection<?>)o, oldN, newN);
}
}
return false;
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
}
}
private boolean replaceStatementInCollection(Field field, Object fieldRef, List<Collection<?>> chain,
Collection<?> collection, N oldN, N newN) throws IllegalAccessException {
if (collection == null) return false;
```

```
int idx = -1;
for (Object o : collection) {
idx++;
if (o == null) continue;
if (Collection.class.isInstance(o)) {
Collection<?> newC = (Collection<?>) o;
List<Collection<?>> newChain = new ArrayList<Collection<?>>(chain);
newChain.add(newC);
if (replaceStatementInCollection(field, fieldRef, newChain, newC, oldN, newN)) return true;
}
if (o == oldN) {
setElementInASTCollection(field, fieldRef, chain, collection, idx, newN);
return true;
}
}
return false;
}
/**
* Override if your AST collection does not support the set method. Javac's for example, does not.
*
* @param field The field that contains the array or list of AST nodes.
* @param fieldRef The object that you can supply to the field's {@code get} method.
* @param chain If the collection is immutable, you need to update the pointer to the collection in each
element in the chain.
*
* @throws IllegalAccessException This exception won't happen, but we allow you to throw it so you can
avoid having to catch it.
*/
@SuppressWarnings({"rawtypes", "unchecked"})
protected void setElementInASTCollection(Field field, Object fieldRef, List<Collection<?>> chain,
Collection<?> collection, int idx, N newN) throws IllegalAccessException {
if (collection instanceof List<?>) {
((List) collection).set(idx, newN);
}
}
private boolean replaceStatementInArray(Object array, N oldN, N newN) {
if (array == null) return false;
int len = Array.getLength(array);
for (int i = 0; i < len; i++) {
Object o = Array.get(array, i);
if (o == null) continue;
if (o.getClass().isArray()) {
if (replaceStatementInArray(o, oldN, newN)) return true;
} else if (o == oldN) {
Array.set(array, i, newN);
return true;
}
}
return false;
}
@SuppressWarnings("unchecked")
private void buildWithField0(Class<L> nodeType, N child, FieldAccess fa, Collection<L> list) {
try {
```

```
Object o = fa.field.get(child);
if (o == null) return;
if (fa.dim == 0) {
L node = buildTree((N) o, Kind.STATEMENT);
if (node != null) list.add(nodeType.cast(node));
} else if (o.getClass().isArray()) {
buildWithArray(nodeType, o, list, fa.dim);
} else if (Collection.class.isInstance(o)) {
buildWithCollection(nodeType, o, list, fa.dim);
}
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
}
}
@SuppressWarnings("unchecked")
private void buildWithArray(Class<L> nodeType, Object array, Collection<L> list, int dim) {
if (dim == 1) {
for (Object v : (Object[]) array) {
if (v == null) continue;
L node = buildTree((N)v, Kind.STATEMENT);
if (node != null) list.add(nodeType.cast(node));
}
} else for (Object v : (Object[]) array) {
if (v == null) return;
buildWithArray(nodeType, v, list, dim -1);
}
}
@SuppressWarnings("unchecked")
private void buildWithCollection(Class<L> nodeType, Object collection, Collection<L> list, int dim) {
if (dim == 1) {
for (Object v : (Collection<?>) collection) {
if (v == null) continue;
L node = buildTree((N) v, Kind.STATEMENT);
if (node != null) list.add(nodeType.cast(node));
}
} else for (Object v : (Collection<?>) collection) {
buildWithCollection(nodeType, v, list, dim - 1);
}
}
/**
* @return The {@code lombok.config} configuration value for the provided {@code key}, or {@code
null} if that key is not in the config / there is no config.
*/
public final <T> T readConfiguration(ConfigurationKey<T> key) {
long start = configTracker == null ? 0L : configTracker.start();
try {
return LombokConfiguration.read(key, this);
} finally {
if (configTracker != null) configTracker.end(start);
}
}
/**
```

```
* @return The {@code lombok.config} configuration value for the provided {@code key}, or {@code
defaultValue} if that key is not in the config / there is no config.
*/
public final <T> T readConfigurationOr(ConfigurationKey<T> key, T defaultValue) {
long start = configTracker == null ? 0L : configTracker.start();
try {
T value = LombokConfiguration.read(key, this);
return value != null ? value : defaultValue;
} finally {
if (configTracker != null) configTracker.end(start);
}
}
public boolean getBooleanAnnotationValue(AnnotationValues<?> annotation, String annoMethod,
ConfigurationKey<Boolean> confKey) {
Boolean conf = readConfiguration(confKey);
return annotation.isExplicit(annoMethod) || conf == null ? annotation.getAsBoolean(annoMethod) : conf;
}
}
```

lombok/core/AgentLauncher.java

```
/*
* Copyright (C) 2009-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.lang.instrument.Instrumentation;
import java.lang.reflect.InvocationTargetException;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
public class AgentLauncher {
public interface AgentLaunchable {
```

```
void runAgent(String agentArgs, Instrumentation instrumentation, boolean injected, Class<?>
launchingContext) throws Exception;
}
public static void runAgents(String agentArgs, Instrumentation instrumentation, boolean injected,
Class<?> launchingContext) throws Throwable {
for (AgentInfo info : AGENTS) {
try {
Class<?> agentClass = Class.forName(info.className());
AgentLaunchable agent = (AgentLaunchable) agentClass.getConstructor().newInstance();
agent.runAgent(agentArgs, instrumentation, injected, launchingContext);
} catch (Throwable t) {
if (t instanceof InvocationTargetException) t = t.getCause();
info.problem(t, instrumentation);
}
}
}
private static final List<AgentInfo> AGENTS = Collections.unmodifiableList(Arrays.<AgentInfo>asList(
new EclipsePatcherInfo()
));
private static abstract class AgentInfo {
abstract String className();
/**
* Called if an exception occurs while loading the agent represented by this AgentInfo object.
*
* @param t The throwable.
* @param instrumentation In case you want to take an alternative action.
*/
void problem(Throwable t, Instrumentation instrumentation) throws Throwable {
if (t instanceof ClassNotFoundException) {
//That's okay - this lombok evidently is a version with support for something stripped out.
return;
}
if (t instanceof ClassCastException) {
throw new InternalError("Lombok bug. Class: " + className() + " is not an implementation of
lombok.core.Agent");
}
if (t instanceof IllegalAccessError) {
throw new InternalError("Lombok bug. Class: " + className() + " is not public");
}
if (t instanceof InstantiationException) {
throw new InternalError("Lombok bug. Class: " + className() + " is not concrete or has no public
no-args constructor");
}
throw t;
}
}
private static class EclipsePatcherInfo extends AgentInfo {
@Override String className() {
return "lombok.eclipse.agent.EclipsePatcher";
}
}
}
```

lombok/core/AlreadyHandledAnnotations.java

```
/*
* Copyright (C) 2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/package lombok.core;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Used to indicate a handler is to be invoked for its marked annotation even if that annotation is already
handled. Useful for cleanup handlers
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface AlreadyHandledAnnotations {}
```

lombok/core/AnnotationProcessor.java

```
/*
* Copyright (C) 2009-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
```

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import static lombok.core.Augments.ClassLoader_lombokAlreadyAddedTo;
import java.io.File;
import java.io.PrintWriter;
import java.io.StringWriter;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.net.URL;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Set;
import javax.annotation.processing.AbstractProcessor;
import javax.annotation.processing.ProcessingEnvironment;
import javax.annotation.processing.Processor;
import javax.annotation.processing.RoundEnvironment;
import javax.annotation.processing.SupportedAnnotationTypes;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.Element;
import javax.lang.model.element.Name;
import javax.lang.model.element.TypeElement;
import javax.tools.Diagnostic.Kind;
import lombok.patcher.ClassRootFinder;
import lombok.permit.Permit;
@SupportedAnnotationTypes("*")
public class AnnotationProcessor extends AbstractProcessor {
private static String trace(Throwable t) {
StringWriter w = new StringWriter();
t.printStackTrace(new PrintWriter(w, true));
return w.toString();
}
static abstract class ProcessorDescriptor {
abstract boolean want(ProcessingEnvironment procEnv, List<String> delayedWarnings);
abstract String getName();
abstract boolean process(Set<? extends TypeElement> annotations, RoundEnvironment roundEnv);
}
private final List<ProcessorDescriptor> registered = Arrays.asList(new JavacDescriptor(), new
EcjDescriptor());
private final List<ProcessorDescriptor> active = new ArrayList<ProcessorDescriptor>();
private final List<String> delayedWarnings = new ArrayList<String>();
/**
* This method is a simplified version of {@link
lombok.javac.apt.LombokProcessor#getJavacProcessingEnvironment}
* It simply returns the processing environment, but in case of gradle incremental compilation,
```

```
* the delegate ProcessingEnvironment of the gradle wrapper is returned.
*/
public static ProcessingEnvironment getJavacProcessingEnvironment(ProcessingEnvironment
procEnv, List<String> delayedWarnings) {
return tryRecursivelyObtainJavacProcessingEnvironment(procEnv);
}
private static ProcessingEnvironment
tryRecursivelyObtainJavacProcessingEnvironment(ProcessingEnvironment procEnv) {
if (procEnv.getClass().getName().equals("com.sun.tools.javac.processing.JavacProcessingEnvironmen
t")) {
return procEnv;
}
for (Class<?> procEnvClass = procEnv.getClass(); procEnvClass != null; procEnvClass =
procEnvClass.getSuperclass()) {
try {
Object delegate = tryGetDelegateField(procEnvClass, procEnv);
if (delegate == null) delegate = tryGetProcessingEnvField(procEnvClass, procEnv);
if (delegate == null) delegate = tryGetProxyDelegateToField(procEnvClass, procEnv);
if (delegate != null) return tryRecursivelyObtainJavacProcessingEnvironment((ProcessingEnvironment)
delegate);
} catch (final Exception e) {
// no valid delegate, try superclass
}
}
return null;
}
/**
* Gradle incremental processing
*/
private static Object tryGetDelegateField(Class<?> delegateClass, Object instance) {
try {
return Permit.getField(delegateClass, "delegate").get(instance);
} catch (Exception e) {
return null;
}
}
/**
* Kotlin incremental processing
*/
private static Object tryGetProcessingEnvField(Class<?> delegateClass, Object instance) {
try {
return Permit.getField(delegateClass, "processingEnv").get(instance);
} catch (Exception e) {
return null;
}
}
/**
* IntelliJ IDEA >= 2020.3
*/
private static Object tryGetProxyDelegateToField(Class<?> delegateClass, Object instance) {
try {
InvocationHandler handler = Proxy.getInvocationHandler(instance);
return Permit.getField(handler.getClass(), "val$delegateTo").get(handler);
```

```
} catch (Exception e) {
return null;
}
}
static class JavacDescriptor extends ProcessorDescriptor {
private Processor processor;
@Override String getName() {
return "OpenJDK javac";
}
@Override boolean want(ProcessingEnvironment procEnv, List<String> delayedWarnings) {
// do not run on ECJ as it may print warnings
if (procEnv.getClass().getName().startsWith("org.eclipse.jdt.")) return false;
ProcessingEnvironment javacProcEnv = getJavacProcessingEnvironment(procEnv, delayedWarnings);
if (javacProcEnv == null) return false;
try {
ClassLoader classLoader = findAndPatchClassLoader(javacProcEnv);
processor = (Processor) Class.forName("lombok.javac.apt.LombokProcessor", false,
classLoader).getConstructor().newInstance();
} catch (Exception e) {
delayedWarnings.add("You found a bug in lombok; lombok.javac.apt.LombokProcessor is not
available. Lombok will not run during this compilation: " + trace(e));
return false;
} catch (NoClassDefFoundError e) {
delayedWarnings.add("Can't load javac processor due to (most likely) a class loader problem: " +
trace(e));
return false;
}
try {
processor.init(procEnv);
} catch (Exception e) {
delayedWarnings.add("lombok.javac.apt.LombokProcessor could not be initialized. Lombok will not run
during this compilation: " + trace(e));
return false;
} catch (NoClassDefFoundError e) {
delayedWarnings.add("Can't initialize javac processor due to (most likely) a class loader problem: " +
trace(e));
return false;
}
return true;
}
private ClassLoader findAndPatchClassLoader(ProcessingEnvironment procEnv) throws Exception {
ClassLoader environmentClassLoader = procEnv.getClass().getClassLoader();
if (environmentClassLoader != null && environmentClassLoader.getClass().getCanonicalName().equal
s("org.codehaus.plexus.compiler.javac.IsolatedClassLoader")) {
if (!ClassLoader_lombokAlreadyAddedTo.getAndSet(environmentClassLoader, true)) {
Method m = Permit.getMethod(environmentClassLoader.getClass(), "addURL", URL.class);
URL selfUrl = new
File(ClassRootFinder.findClassRootOfClass(AnnotationProcessor.class)).toURI().toURL();
Permit.invoke(m, environmentClassLoader, selfUrl);
}
}
ClassLoader ourClassLoader = JavacDescriptor.class.getClassLoader();
if (ourClassLoader == null) return ClassLoader.getSystemClassLoader();
```

```
return ourClassLoader;
}
@Override boolean process(Set<? extends TypeElement> annotations, RoundEnvironment roundEnv)
{
return processor.process(annotations, roundEnv);
}
}
static class EcjDescriptor extends ProcessorDescriptor {
@Override String getName() {
return "ECJ";
}
@Override boolean want(ProcessingEnvironment procEnv, List<String> delayedWarnings) {
if (!procEnv.getClass().getName().startsWith("org.eclipse.jdt.")) return false;
// Lombok used to work as annotation processor to ecj but that never actually worked properly, so we
disabled the feature in 0.10.0.
// Because loading lombok as an agent in any ECJ-based non-interactive tool works just fine, we're not
going to generate any warnings, as we'll
// likely generate more false positives than be helpful.
return true;
}
@Override boolean process(Set<? extends TypeElement> annotations, RoundEnvironment roundEnv)
{
return false;
}
}
@Override public void init(ProcessingEnvironment procEnv) {
super.init(procEnv);
for (ProcessorDescriptor proc : registered) {
if (proc.want(procEnv, delayedWarnings)) active.add(proc);
}
if (active.isEmpty() && delayedWarnings.isEmpty()) {
StringBuilder supported = new StringBuilder();
for (ProcessorDescriptor proc : registered) {
if (supported.length() > 0) supported.append(", ");
supported.append(proc.getName());
}
if (procEnv.getClass().getName().equals("com.google.turbine.processing.TurbineProcessingEnvironme
nt")) {
procEnv.getMessager().printMessage(Kind.ERROR, String.format("Turbine is not currently supported
by lombok."));
} else {
procEnv.getMessager().printMessage(Kind.WARNING, String.format("You aren't using a compiler
supported by lombok, so lombok will not work and has been disabled.\n" +
"Your processor is: %s\nLombok supports: %s", procEnv.getClass().getName(), supported));
}
}
}
@Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment
roundEnv) {
if (!delayedWarnings.isEmpty()) {
Set<? extends Element> rootElements = roundEnv.getRootElements();
if (!rootElements.isEmpty()) {
Element firstRoot = rootElements.iterator().next();
```

```
for (String warning : delayedWarnings) processingEnv.getMessager().printMessage(Kind.WARNING, warning, firstRoot);
delayedWarnings.clear();
}
}
for (ProcessorDescriptor proc : active) proc.process(annotations, roundEnv);
boolean onlyLombok = true;
boolean zeroElems = true;
for (TypeElement elem : annotations) {
zeroElems = false;
Name n = elem.getQualifiedName();
if (n.toString().startsWith("lombok.")) continue;
onlyLombok = false;
}
// Normally we rely on the claiming processor to claim away all lombok annotations.
// One of the many Java9 oversights is that this 'process' API has not been fixed to address the point that 'files I want to look at' and 'annotations I want to claim' must be one and the same,
// and yet in java9 you can no longer have 2 providers for the same service, thus, if you go by module path, lombok no longer loads the ClaimingProcessor.
// This doesn't do as good a job, but it'll have to do. The only way to go from here, I think, is either 2 modules, or use reflection hackery to add ClaimingProcessor during our init.
return onlyLombok && !zeroElems;
}
/**
* We just return the latest version of whatever JDK we run on. Stupid? Yeah, but it's either that or warnings on all versions but 1. Blame Joe.
*/
@Override public SourceVersion getSupportedSourceVersion() {
return SourceVersion.latest();
}
}
```

lombok/core/AnnotationValues.java

```
/*
* Copyright (C) 2009-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.lang.annotation.Annotation;
import java.lang.reflect.Array;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.Map;
import lombok.core.AST.Kind;
import lombok.permit.Permit;
/**
* Represents a single annotation in a source file and can be used to query the parameters present on it.
*
* @param A The annotation that this class represents, such as {@code lombok.Getter}
*/
public class AnnotationValues<A extends Annotation> {
private final Class<A> type;
private final Map<String, AnnotationValue> values;
private final LombokNode<?, ?, ?> ast;
/**
* Represents a single method on the annotation class. For example, the value() method on the Getter
annotation.
*/
public static class AnnotationValue {
/** A list of the raw expressions. List is size 1 unless an array is provided. */
public final List<String> raws;
/** Guesses for each raw expression. It's 'primitive' (String or primitive), an AV.ClassLiteral, an
AV.FieldSelect, or an array of one of those. */
public final List<Object> valueGuesses;
/** A list of the actual expressions. List is size 1 unless an array is provided. */
public final List<Object> expressions;
private final LombokNode<?, ?, ?> node;
private final boolean isExplicit;
/**
* Like the other constructor, but used for when the annotation method is initialized with an array value.
*/
public AnnotationValue(LombokNode<?, ?, ?> node, List<String> raws, List<Object> expressions,
List<Object> valueGuesses, boolean isExplicit) {
this.node = node;
this.raws = raws;
this.expressions = expressions;
this.valueGuesses = valueGuesses;
this.isExplicit = isExplicit;
}
/**
* Override this if you want more specific behaviour (to get the source position just right).
```

```
*
* @param message English message with the problem.
* @param valueIdx The index into the values for this annotation key that caused the problem.
* -1 for a problem that applies to all values, otherwise the 0-based index into an array of values.
* If there is no array for this value (e.g. value=1 instead of value={1,2}), then always -1 or 0.
*/
public void setError(String message, int valueIdx) {
node.addError(message);
}
/**
* Override this if you want more specific behaviour (to get the source position just right).
*
* @param message English message with the problem.
* @param valueIdx The index into the values for this annotation key that caused the problem.
* -1 for a problem that applies to all values, otherwise the 0-based index into an array of values.
* If there is no array for this value (e.g. value=1 instead of value={1,2}), then always -1 or 0.
*/
public void setWarning(String message, int valueIdx) {
node.addError(message);
}
/** {@inheritDoc} */
@Override public String toString() {
return "raws: " + raws + " valueGuesses: " + valueGuesses;
}
public boolean isExplicit() {
return isExplicit;
}
}
/**
* Creates a new AnnotationValues.
*
* @param type The annotation type. For example, "Getter.class"
* @param values a Map of method names to AnnotationValue instances, for example 'value ->
annotationValue instance'.
* @param ast The Annotation node.
*/
public AnnotationValues(Class<A> type, Map<String, AnnotationValue> values, LombokNode<?, ?, ?>
ast) {
this.type = type;
this.values = values;
this.ast = ast;
}
public static <A extends Annotation> AnnotationValues<A> of(Class<A> type) {
return new AnnotationValues<A>(type, Collections.<String, AnnotationValue>emptyMap(), null);
}
/**
* Creates a new annotation wrapper with all default values, and using the provided ast as lookup
anchor for
* class literals.
*/
public static <A extends Annotation> AnnotationValues<A> of(Class<A> type, LombokNode<?, ?, ?>
ast) {
return new AnnotationValues<A>(type, Collections.<String, AnnotationValue>emptyMap(), ast);
```

```
}
/**
* Thrown on the fly if an actual annotation instance procured via the {@link #getInstance()} method is
queried
* for a method for which this AnnotationValues instance either doesn't have a guess or can't manage to
fit
* the guess into the required data type.
*/
public static class AnnotationValueDecodeFail extends RuntimeException {
private static final long serialVersionUID = 1L;
/** The index into an array initializer (e.g. if the second value in an array initializer is
* an integer constant expression like '5+SomeOtherClass.CONSTANT', this exception will be thrown,
* and you'll get a '1' for idx. */
public final int idx;
/** The AnnotationValue object that goes with the annotation method for which the failure occurred. */
public final AnnotationValue owner;
public AnnotationValueDecodeFail(AnnotationValue owner, String msg, int idx) {
super(msg);
this.idx = idx;
this.owner = owner;
}
}
private static AnnotationValueDecodeFail makeNoDefaultFail(AnnotationValue owner, Method method)
{
return new AnnotationValueDecodeFail(owner,
"No value supplied but " + method.getName() + " has no default either.", -1);
}
private A cachedInstance = null;
public List<String> getAsStringList(String methodName) {
AnnotationValue v = values.get(methodName);
if (v == null) {
String[] s = getDefaultIf(methodName, new String[0]);
return Collections.unmodifiableList(Arrays.asList(s));
}
List<String> out = new ArrayList<String>(v.valueGuesses.size());
int idx = 0;
for (Object guess : v.valueGuesses) {
Object result = guess == null ? null : guessToType(guess, String.class, v, idx);
if (result == null) {
if (v.valueGuesses.size() == 1) {
String[] s = getDefaultIf(methodName, new String[0]);
return Collections.unmodifiableList(Arrays.asList(s));
}
throw new AnnotationValueDecodeFail(v,
"I can't make sense of this annotation value. Try using a fully qualified literal.", idx);
}
out.add((String) result);
}
return Collections.unmodifiableList(out);
}
public String getAsString(String methodName) {
AnnotationValue v = values.get(methodName);
if (v == null || v.valueGuesses.size() != 1) {
```

```
return getDefaultIf(methodName, "");
}
Object guess = guessToType(v.valueGuesses.get(0), String.class, v, 0);
if (guess instanceof String) return (String) guess;
return getDefaultIf(methodName, "");
}
public boolean getAsBoolean(String methodName) {
AnnotationValue v = values.get(methodName);
if (v == null || v.valueGuesses.size() != 1) {
return getDefaultIf(methodName, false);
}
Object guess = guessToType(v.valueGuesses.get(0), boolean.class, v, 0);
if (guess instanceof Boolean) return ((Boolean) guess).booleanValue();
return getDefaultIf(methodName, false);
}
@SuppressWarnings("unchecked")
public <T> T getDefaultIf(String methodName, T defaultValue) {
try {
return (T) Permit.getMethod(type, methodName).getDefaultValue();
} catch (Exception e) {
return defaultValue;
}
}
/**
* Creates an actual annotation instance. You can use this to query any annotation methods, except for
* those annotation methods with class literals, as those can most likely not be turned into Class objects.
*
* If some of the methods cannot be implemented, this method still works; it's only when you call a
method
* that has a problematic value that an AnnotationValueDecodeFail exception occurs.
*/
@SuppressWarnings("unchecked")
public A getInstance() {
if (cachedInstance != null) return cachedInstance;
InvocationHandler invocations = new InvocationHandler() {
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
AnnotationValue v = values.get(method.getName());
if (v == null) {
Object defaultValue = method.getDefaultValue();
if (defaultValue != null) return defaultValue;
throw makeNoDefaultFail(v, method);
}
boolean isArray = false;
Class<?> expected = method.getReturnType();
Object array = null;
if (expected.isArray()) {
isArray = true;
expected = expected.getComponentType();
array = Array.newInstance(expected, v.valueGuesses.size());
}
if (!isArray && v.valueGuesses.size() > 1) {
throw new AnnotationValueDecodeFail(v,
"Expected a single value, but " + method.getName() + " has an array of values", -1);
```

```
}
if (v.valueGuesses.size() == 0 && !isArray) {
Object defaultValue = method.getDefaultValue();
if (defaultValue == null) throw makeNoDefaultFail(v, method);
return defaultValue;
}
int idx = 0;
for (Object guess : v.valueGuesses) {
Object result = guess == null ? null : guessToType(guess, expected, v, idx);
if (!isArray) {
if (result == null) {
Object defaultValue = method.getDefaultValue();
if (defaultValue == null) throw makeNoDefaultFail(v, method);
return defaultValue;
}
return result;
}
if (result == null) {
if (v.valueGuesses.size() == 1) {
Object defaultValue = method.getDefaultValue();
if (defaultValue == null) throw makeNoDefaultFail(v, method);
return defaultValue;
}
throw new AnnotationValueDecodeFail(v,
"I can't make sense of this annotation value. Try using a fully qualified literal.", idx);
}
Array.set(array, idx++, result);
}
return array;
}
};
return cachedInstance = (A) Proxy.newProxyInstance(type.getClassLoader(), new Class[] { type },
invocations);
}
private Object guessToType(Object guess, Class<?> expected, AnnotationValue v, int pos) {
if (expected == int.class || expected == Integer.class) {
if (guess instanceof Integer || guess instanceof Short || guess instanceof Byte) {
return ((Number) guess).intValue();
}
}
if (expected == long.class || expected == Long.class) {
if (guess instanceof Long || guess instanceof Integer || guess instanceof Short || guess instanceof Byte)
{
return ((Number) guess).longValue();
}
}
if (expected == short.class || expected == Short.class) {
if (guess instanceof Integer || guess instanceof Short || guess instanceof Byte) {
int intVal = ((Number) guess).intValue();
int shortVal = ((Number) guess).shortValue();
if (shortVal == intVal) return shortVal;
}
}
```

```
if (expected == byte.class || expected == Byte.class) {
if (guess instanceof Integer || guess instanceof Short || guess instanceof Byte) {
int intVal = ((Number) guess).intValue();
int byteVal = ((Number) guess).byteValue();
if (byteVal == intVal) return byteVal;
}
}
if (expected == double.class || expected == Double.class) {
if (guess instanceof Number) return ((Number) guess).doubleValue();
}
if (expected == float.class || expected == Float.class) {
if (guess instanceof Number) return ((Number) guess).floatValue();
}
if (expected == boolean.class || expected == Boolean.class) {
if (guess instanceof Boolean) return ((Boolean) guess).booleanValue();
}
if (expected == char.class || expected == Character.class) {
if (guess instanceof Character) return ((Character) guess).charValue();
}
if (expected == String.class) {
if (guess instanceof String) return guess;
}
if (Enum.class.isAssignableFrom(expected) ) {
if (guess instanceof FieldSelect) {
String fieldSel = ((FieldSelect) guess).getFinalPart();
for (Object enumConstant : expected.getEnumConstants()) {
String target = ((Enum<?>) enumConstant).name();
if (target.equals(fieldSel)) return enumConstant;
}
throw new AnnotationValueDecodeFail(v,
"Can't translate " + fieldSel + " to an enum of type " + expected, pos);
}
}
if (expected == Class.class) {
if (guess instanceof ClassLiteral) try {
String classLit = ((ClassLiteral) guess).getClassName();
return Class.forName(toFQ(classLit));
} catch (ClassNotFoundException e) {
throw new AnnotationValueDecodeFail(v,
"Can't translate " + guess + " to a class object.", pos);
}
}
if (guess instanceof AnnotationValues) {
return ((AnnotationValues<?>) guess).getInstance();
}
if (guess instanceof FieldSelect) throw new AnnotationValueDecodeFail(v,
"You must use constant literals in lombok annotations; they cannot be references to (static) fields.",
pos);
throw new AnnotationValueDecodeFail(v,
"Can't translate a " + guess.getClass() + " to the expected " + expected, pos);
}
/**
* Returns the raw expressions used for the provided {@code annotationMethodName}.
```

```
*
* You should use this method for annotation methods that return {@code Class} objects. Remember
that
* class literals end in ".class" which you probably want to strip off.
*/
public List<String> getRawExpressions(String annotationMethodName) {
AnnotationValue v = values.get(annotationMethodName);
return v == null ? Collections.<String>emptyList() : v.raws;
}
/**
* Returns the actual expressions used for the provided {@code annotationMethodName}.
*/
public List<Object> getActualExpressions(String annotationMethodName) {
AnnotationValue v = values.get(annotationMethodName);
return v == null ? Collections.<Object>emptyList() : v.expressions;
}
public boolean isExplicit(String annotationMethodName) {
AnnotationValue annotationValue = values.get(annotationMethodName);
return annotationValue != null && annotationValue.isExplicit();
}
/**
* Convenience method to return the first result in a {@link #getRawExpressions(String)} call.
*
* You should use this method if the annotation method is not an array type.
*/
public String getRawExpression(String annotationMethodName) {
List<String> l = getRawExpressions(annotationMethodName);
return l.isEmpty() ? null : l.get(0);
}
/**
* Convenience method to return the first result in a {@link #getActualExpressions(String)} call.
*
* You should use this method if the annotation method is not an array type.
*/
public Object getActualExpression(String annotationMethodName) {
List<Object> l = getActualExpressions(annotationMethodName);
return l.isEmpty() ? null : l.get(0);
}
/**
* Returns the guessed value for the provided {@code annotationMethodName}.
*/
public Object getValueGuess(String annotationMethodName) {
AnnotationValue v = values.get(annotationMethodName);
return v == null || v.valueGuesses.isEmpty() ? null : v.valueGuesses.get(0);
}
/** Generates an error message on the stated annotation value (you should only call this method if you
know it's there!) */
public void setError(String annotationMethodName, String message) {
setError(annotationMethodName, message, -1);
}
/** Generates a warning message on the stated annotation value (you should only call this method if
you know it's there!) */
public void setWarning(String annotationMethodName, String message) {
```

```
setWarning(annotationMethodName, message, -1);
}
/** Generates an error message on the stated annotation value, which must have an array initializer.
* The index-th item in the initializer will carry the error (you should only call this method if you know it's there!) */
public void setError(String annotationMethodName, String message, int index) {
AnnotationValue v = values.get(annotationMethodName);
if (v == null) return;
v.setError(message, index);
}
/** Generates a warning message on the stated annotation value, which must have an array initializer.
* The index-th item in the initializer will carry the error (you should only call this method if you know it's there!) */
public void setWarning(String annotationMethodName, String message, int index) {
AnnotationValue v = values.get(annotationMethodName);
if (v == null) return;
v.setWarning(message, index);
}
/**
* Attempts to translate class literals to their fully qualified names, such as 'Throwable.class' to 'java.lang.Throwable'.
*
* This process is at best a guess, but it will take into account import statements.
*/
public List<String> getProbableFQTypes(String annotationMethodName) {
List<String> result = new ArrayList<String>();
AnnotationValue v = values.get(annotationMethodName);
if (v == null) return Collections.emptyList();
for (Object o : v.valueGuesses) result.add(o == null ? null : toFQ(o.toString()));
return result;
}
/**
* Convenience method to return the first result in a {@link #getProbableFQType(String)} call.
*
* You should use this method if the annotation method is not an array type.
*/
public String getProbableFQType(String annotationMethodName) {
List<String> l = getProbableFQTypes(annotationMethodName);
return l.isEmpty() ? null : l.get(0);
}
/*
* Credit goes to Petr Jiricka of Sun for highlighting the problems with the earlier version of this method.
*/
private String toFQ(String typeName) {
String prefix = typeName.indexOf('.') > -1 ? typeName.substring(0, typeName.indexOf('.')) : typeName;
/* 1. Walk through type names in this source file at this level. */ {
LombokNode<?, ?, ?> n = ast;
walkThroughCU:
while (n != null) {
if (n.getKind() == Kind.TYPE) {
String simpleName = n.getName();
if (prefix.equals(simpleName)) {
//We found a matching type name in the local hierarchy!
```

```
List<String> outerNames = new ArrayList<String>();
while (true) {
n = n.up();
if (n == null || n.getKind() == Kind.COMPILATION_UNIT) break;
if (n.getKind() == Kind.TYPE) outerNames.add(n.getName());
//If our type has a parent that isn't either the CompilationUnit or another type, then we are
//a method-local class or an anonymous inner class literal. These technically do have FQNs
//and we may, with a lot of effort, figure out their name, but, that's some fairly horrible code
//style and these methods have 'probable' in their name for a reason.
break walkThroughCU;
}
StringBuilder result = new StringBuilder();
if (ast.getPackageDeclaration() != null) result.append(ast.getPackageDeclaration());
if (result.length() > 0) result.append('.');
Collections.reverse(outerNames);
for (String outerName : outerNames) result.append(outerName).append('.');
result.append(typeName);
return result.toString();
}
}
n = n.up();
}
}
/* 2. Walk through non-star imports and search for a match. */ {
if (prefix.equals(typeName)) {
String fqn = ast.getImportList().getFullyQualifiedNameForSimpleName(typeName);
if (fqn != null) return fqn;
}
}
/* 3. Walk through star imports and, if they start with "java.", use Class.forName based resolution. */ {
for (String potential : ast.getImportList().applyNameToStarImports("java", typeName)) {
try {
Class<?> c = Class.forName(potential);
if (c != null) return c.getName();
} catch (Throwable t) {
//Class.forName failed for whatever reason - it most likely does not exist, continue.
}
}
}
/* 4. If the type name is a simple name, then our last guess is that it's another class in this package. */ {
if (typeName.indexOf('.') == -1) return inLocalPackage(ast, typeName);
}
/* 5. It's either an FQN or a nested class in another class in our package. Use code conventions to
guess. */ {
char firstChar = typeName.charAt(0);
if (Character.isTitleCase(firstChar) || Character.isUpperCase(firstChar)) {
//Class names start with uppercase letters, so presume it's a nested class in another class in our
package.
return inLocalPackage(ast, typeName);
}
//Presume it's fully qualified.
return typeName;
}
```

```
}
private static String inLocalPackage(LombokNode<?, ?, ?> node, String typeName) {
StringBuilder result = new StringBuilder();
if (node != null && node.getPackageDeclaration() != null)
result.append(node.getPackageDeclaration());
if (result.length() > 0) result.append('.');
result.append(typeName);
return result.toString();
}
/**
* Creates an amalgamation where any values in this AnnotationValues that aren't explicit are 'enriched' by explicitly set stuff from {@code defaults}.
* Note that this code may modify self and then returns self, or it returns defaults - do not rely on immutability nor on getting self.
*/
public AnnotationValues<A> integrate(AnnotationValues<A> defaults) {
if (values.isEmpty()) return defaults;
for (Map.Entry<String, AnnotationValue> entry : defaults.values.entrySet()) {
if (!entry.getValue().isExplicit) continue;
AnnotationValue existingValue = values.get(entry.getKey());
if (existingValue != null && existingValue.isExplicit) continue;
values.put(entry.getKey(), entry.getValue());
}
return this;
}
/** Returns {@code true} if the annotation has zero parameters. */
public boolean isMarking() {
for (AnnotationValue v : values.values()) if (v.isExplicit) return false;
return true;
}
}
```

lombok/core/Augments.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
public final class Augments {
private Augments() {
// prevent instantiation
}
public static final FieldAugment<ClassLoader, Boolean> ClassLoader_lombokAlreadyAddedTo =
FieldAugment.augment(ClassLoader.class, boolean.class, "lombok$alreadyAddedTo");
}
```

lombok/core/CleanupRegistry.java

```
/*
* Copyright (C) 2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
public class CleanupRegistry {
private static final class CleanupKey {
private final String key;
private final Object target;
CleanupKey(String key, Object target) {
this.key = key;
this.target = target;
}
@Override public boolean equals(Object other) {
if (other == null) return false;
if (other == this) return true;
if (!(other instanceof CleanupKey)) return false;
```

```
CleanupKey o = (CleanupKey) other;
if (!key.equals(o.key)) return false;
return target == o.target;
}
@Override public int hashCode() {
return 109 * System.identityHashCode(target) + key.hashCode();
}
}
private final ConcurrentMap<CleanupKey, CleanupTask> tasks = new
ConcurrentHashMap<CleanupKey, CleanupTask>();
public void registerTask(String key, Object target, CleanupTask task) {
CleanupKey ck = new CleanupKey(key, target);
tasks.putIfAbsent(ck, task);
}
public void run() {
for (CleanupTask task : tasks.values()) {
task.cleanup();
}
tasks.clear();
}
}
```

lombok/core/CleanupTask.java


```
/*
* Copyright (C) 2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.core;
public interface CleanupTask {
void cleanup();
}
```

lombok/core/DiagnosticsReceiver.java

```
/*
* Copyright (C) 2009-2010 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
public interface DiagnosticsReceiver {
DiagnosticsReceiver CONSOLE = new DiagnosticsReceiver() {
@Override public void addError(String message) {
System.err.println("Error: " + message);
}
@Override public void addWarning(String message) {
System.out.println("Warning: " + message);
}
};
/** Generate a compiler error on this node. */
void addError(String message);
/** Generate a compiler warning on this node. */
void addWarning(String message);
}
```

lombok/core/GuavaTypeMap.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
```

```
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
public final class GuavaTypeMap {
private static final Map<String, String> TYPE_TO_GUAVA_TYPE; static {
Map<String, String> m = new HashMap<String, String>();
m.put("java.util.NavigableSet", "ImmutableSortedSet");
m.put("java.util.NavigableMap", "ImmutableSortedMap");
m.put("java.util.SortedSet", "ImmutableSortedSet");
m.put("java.util.SortedMap", "ImmutableSortedMap");
m.put("java.util.Set", "ImmutableSet");
m.put("java.util.Map", "ImmutableMap");
m.put("java.util.Collection", "ImmutableList");
m.put("java.util.List", "ImmutableList");
m.put("com.google.common.collect.ImmutableSet", "ImmutableSet");
m.put("com.google.common.collect.ImmutableSortedSet", "ImmutableSortedSet");
m.put("com.google.common.collect.ImmutableMap", "ImmutableMap");
m.put("com.google.common.collect.ImmutableBiMap", "ImmutableBiMap");
m.put("com.google.common.collect.ImmutableSortedMap", "ImmutableSortedMap");
m.put("com.google.common.collect.ImmutableList", "ImmutableList");
m.put("com.google.common.collect.ImmutableCollection", "ImmutableList");
m.put("com.google.common.collect.ImmutableTable", "ImmutableTable");
TYPE_TO_GUAVA_TYPE = Collections.unmodifiableMap(m);
}
public static String getGuavaTypeName(String fqn) {
String target = TYPE_TO_GUAVA_TYPE.get(fqn);
return target != null ? target : "ImmutableList";
}
private GuavaTypeMap() {}
}
```

lombok/core/HandlerPriority.java

```
/*
* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
```

* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.core;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Used to order the way handlers are run. Handlers with a lower priority value are run before handlers with higher priority values.
* For example, {@code @Value} can cause the class to be marked final, and this affects the behaviour of {@code @EqualsAndHashCode}. By ensuring that
* the handler for {@code @Value} always runs before the handler for {@code @EqualsAndHashCode}, the code is simpler: The {@code @EqualsAndHashCode} handler
* does not have to check for the presence of a {@code @Value} annotation to determine whether to generate the {@code canEqual} method or not.
* <p>
* A new priority level can also be used to force a reset of the resolved model, i.e. to add generated methods to the symbol tables. Each platform implementation (javac, ecj, etc)
* may have additional marker annotations required to indicate the need for the reset.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface HandlerPriority {
int value();
/**
* This can be used to differentiate 2 handlers with the same value to be at a different handler priority anyway.
* <strong>DO NOT USE THIS</strong> unless someone has been crowding out the numbers and there's no room left.
*/
int subValue() default 0;
}
```

lombok/core/ImportList.java

```
/*
```

* Copyright (C) 2013-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
```
*/
package lombok.core;
import java.util.Collection;
public interface ImportList {
/**
* If there is an explicit import of the stated unqualified type name, return that. Otherwise, return null.
*/
String getFullyQualifiedNameForSimpleName(String unqualified);
/**
* If there is an explicit import of the stated unqualified type name, return that. Otherwise, return null.
* Do not translate the produced fully qualified name to the alias.
*/
String getFullyQualifiedNameForSimpleNameNoAliasing(String unqualified);
/**
* Returns true if the package name is explicitly star-imported, OR the packageName refers to this
source file's own package name, OR packageName is 'java.lang'.
*/
boolean hasStarImport(String packageName);
/**
* Takes all explicit non-static star imports whose first element is equal to {@code startsWith}, replaces
the star with {@code unqualified}, and returns these.
*/
Collection<String> applyNameToStarImports(String startsWith, String unqualified);
String applyUnqualifiedNameToPackage(String unqualified);
}
```

lombok/core/LombokApp.java

/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy

```
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.Collections;
import java.util.List;
/**
* Implement this class, and add yourself as a provider for it, to become an app runnable by running
lombok.jar as a jar.
*
* @see lombok.core.Main.VersionApp
*/
public abstract class LombokApp {
/**
* @param args The arguments; analogous to what's passed to {@code public static void main(String[]
args)} methods.
* @return The return value. Don't call {@code System.exit} yourself.
*/
public abstract int runApp(List<String> args) throws Exception;
/**
* @return Your app name. For example {@code delombok}.
*/
public abstract String getAppName();
/**
* @return Description of this app, for the command line.
*/
public abstract String getAppDescription();
/**
* @return When lombok.jar is executed with any of these strings as first argument, your app will be
started.
*/
public List<String> getAppAliases() {
return Collections.emptyList();
}
/**
* @return {@code true} if this app is an internal debugging tool and won't be listed by the default help
message.
*/
public boolean isDebugTool() {
```

```
return false;
}
}
```

lombok/core/LombokConfiguration.java

```
/*
* Copyright (C) 2013-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.net.URI;
import java.util.Collections;
import lombok.core.configuration.BubblingConfigurationResolver;
import lombok.core.configuration.ConfigurationFileToSource;
import lombok.core.configuration.ConfigurationKey;
import lombok.core.configuration.ConfigurationParser;
import lombok.core.configuration.ConfigurationProblemReporter;
import lombok.core.configuration.ConfigurationResolver;
import lombok.core.configuration.ConfigurationResolverFactory;
import lombok.core.configuration.FileSystemSourceCache;
public class LombokConfiguration {
private static final ConfigurationResolver NULL_RESOLVER = new ConfigurationResolver() {
@SuppressWarnings("unchecked") @Override public <T> T resolve(ConfigurationKey<T> key) {
if (key.getType().isList()) return (T) Collections.emptyList();
return null;
}
};
private static FileSystemSourceCache cache = new FileSystemSourceCache();
private static ConfigurationResolverFactory configurationResolverFactory;
static {
if (System.getProperty("lombok.disableConfig") != null) {
configurationResolverFactory = new ConfigurationResolverFactory() {
@Override public ConfigurationResolver createResolver(URI sourceLocation) {
```

```
return NULL_RESOLVER;
}
};
}
else {
configurationResolverFactory = createFileSystemBubblingResolverFactory();
}
}
private LombokConfiguration() {
// prevent instantiation
}
public static void overrideConfigurationResolverFactory(ConfigurationResolverFactory crf) {
configurationResolverFactory = crf == null ? createFileSystemBubblingResolverFactory() : crf;
}
static <T> T read(ConfigurationKey<T> key, AST<?, ?, ?> ast) {
return read(key, ast.getAbsoluteFileLocation());
}
public static <T> T read(ConfigurationKey<T> key, URI sourceLocation) {
return configurationResolverFactory.createResolver(sourceLocation).resolve(key);
}
private static ConfigurationResolverFactory createFileSystemBubblingResolverFactory() {
final ConfigurationFileToSource fileToSource = cache.fileToSource(new
ConfigurationParser(ConfigurationProblemReporter.CONSOLE));
return new ConfigurationResolverFactory() {
@Override public ConfigurationResolver createResolver(URI sourceLocation) {
return new BubblingConfigurationResolver(cache.forUri(sourceLocation), fileToSource);
}
};
}
}
```

lombok/core/LombokInternalAliasing.java

```
/*
* Copyright (C) 2013-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
```

```
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
public class LombokInternalAliasing {
public static final Map<String, String> ALIASES;
public static final Map<String, Collection<String>> REVERSE_ALIASES;
/**
* Provide a fully qualified name (FQN), and the canonical version of this is returned.
*/
public static String processAliases(String in) {
if (in == null) return null;
String ret = ALIASES.get(in);
return ret == null ? in : ret;
}
static {
Map<String, String> m1 = new HashMap<String, String>();
m1.put("lombok.experimental.Value", "lombok.Value");
m1.put("lombok.experimental.Builder", "lombok.Builder");
m1.put("lombok.experimental.var", "lombok.var");
m1.put("lombok.Delegate", "lombok.experimental.Delegate");
m1.put("lombok.experimental.Wither", "lombok.With");
ALIASES = Collections.unmodifiableMap(m1);
Map<String, Collection<String>> m2 = new HashMap<String, Collection<String>>();
for (Map.Entry<String, String> e : m1.entrySet()) {
Collection<String> c = m2.get(e.getValue());
if (c == null) {
m2.put(e.getValue(), Collections.singleton(e.getKey()));
} else if (c.size() == 1) {
Collection<String> newC = new ArrayList<String>(2);
newC.addAll(c);
m2.put(e.getValue(), c);
} else {
c.add(e.getKey());
}
}
for (Map.Entry<String, Collection<String>> e : m2.entrySet()) {
Collection<String> c = e.getValue();
if (c.size() > 1) e.setValue(Collections.unmodifiableList((ArrayList<String>) c));
}
REVERSE_ALIASES = Collections.unmodifiableMap(m2);
}
}
```

lombok/core/LombokNode.java

/*
* Copyright (C) 2009-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.lang.annotation.Annotation;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;
import lombok.core.AST.Kind;
/**
* An instance of this class wraps an Eclipse/javac internal node object.
*
* @param A Type of our owning AST.
* @param L self-type.
* @param N The common type of all AST nodes in the internal representation of the target platform.
* For example, JCTree for javac, and ASTNode for Eclipse.
*/
public abstract class LombokNode<A extends AST<A, L, N>, L extends LombokNode<A, L, N>, N>
implements DiagnosticsReceiver {
protected final Kind kind;
protected final N node;
protected LombokImmutableList<L> children;
protected L parent;
/** structurally significant are those nodes that can be annotated in java 1.6 or are method-like
toplevels,
* so fields, local declarations, method arguments, methods, types, the Compilation Unit itself, and
initializers. */
protected boolean isStructurallySignificant;
/**
* Creates a new Node object that represents the provided node.
*
* @param ast The owning AST - this node is part of this AST's tree of nodes.

```
* @param node The AST object in the target parser's own internal AST tree that this node object will
represent.
* @param children A list of child nodes. Passing in null results in the children list being empty, not null.
* @param kind The kind of node represented by this object.
*/
@SuppressWarnings("unchecked")
protected LombokNode(N node, List<L> children, Kind kind) {
this.kind = kind;
this.node = node;
this.children = children != null ? LombokImmutableList.copyOf(children) :
LombokImmutableList.<L>of();
for (L child : this.children) {
child.parent = (L) this;
if (!child.isStructurallySignificant)
child.isStructurallySignificant = calculateIsStructurallySignificant(node);
}
this.isStructurallySignificant = calculateIsStructurallySignificant(null);
}
public abstract A getAst();
/** {@inheritDoc} */
@Override public String toString() {
return String.format("NODE %s (%s) %s",
kind, node == null ? "(NULL)" : node.getClass(), node == null ? "" : node);
}
/**
* Convenient shortcut to the owning ast object's {@code getPackageDeclaration} method.
*
* @see AST#getPackageDeclaration()
*/
public String getPackageDeclaration() {
return getAst().getPackageDeclaration();
}
/**
* Convenient shortcut to the owning ast object's {@code getImportList} method.
*
* @see AST#getImportList()
*/
public ImportList getImportList() {
return getAst().getImportList();
}
/**
* Convenient shortcut to the owning ast object's {@code getImportListAsTypeResolver} method.
*
* @see AST#getImportListAsTypeResolver()
*/
public TypeResolver getImportListAsTypeResolver() {
return getAst().getImportListAsTypeResolver();
}
/**
* See {@link #isStructurallySignificant}.
*/
protected abstract boolean calculateIsStructurallySignificant(N parent);
/**
```

```
* Convenient shortcut to the owning ast object's get method.
*
* @see AST#get(Object)
*/
public L getNodeFor(N obj) {
return getAst().get(obj);
}
/**
* @return The javac/Eclipse internal AST object wrapped by this LombokNode object.
*/
public N get() {
return node;
}
public Kind getKind() {
return kind;
}
/**
* Return the name of your type (simple name), method, field, or local variable. Return null if this
* node doesn't really have a name, such as initializers, while statements, etc.
*/
public abstract String getName();
/** Returns the structurally significant node that encloses this one.
*
* @see #isStructurallySignificant()
*/
public L up() {
L result = parent;
while (result != null && !result.isStructurallySignificant) result = result.parent;
return result;
}
/**
* {@code @Foo int x, y;} is stored in both javac and ecj as 2 FieldDeclarations, both with the same
annotation as child.
* The normal {@code up()} method can't handle having multiple parents, but this one can.
*/
public Collection<L> upFromAnnotationToFields() {
if (getKind() != Kind.ANNOTATION) return Collections.emptyList();
L field = up();
if (field == null || field.getKind() != Kind.FIELD) return Collections.emptyList();
L type = field.up();
if (type == null || type.getKind() != Kind.TYPE) return Collections.emptyList();
List<L> fields = new ArrayList<L>();
for (L potentialField : type.down()) {
if (potentialField.getKind() != Kind.FIELD) continue;
for (L child : potentialField.down()) {
if (child.getKind() != Kind.ANNOTATION) continue;
if (child.get() == get()) fields.add(potentialField);
}
}
return fields;
}
/**
* Returns the direct parent node in the AST tree of this node. For example, a local variable declaration's
```

```
* direct parent can be e.g. an If block, but its {@code up()} {@code LombokNode} is the {@code
Method} that contains it.
*/
public L directUp() {
return parent;
}
/**
* Returns all children nodes.
*/
public LombokImmutableList<L> down() {
return children;
}
/**
* Convenient shortcut to the owning ast object's getLatestJavaSpecSupported method.
*
* @see AST#getLatestJavaSpecSupported()
*/
public int getLatestJavaSpecSupported() {
return getAst().getLatestJavaSpecSupported();
}
/**
* Convenient shortcut to the owning ast object's getSourceVersion method.
*
* @see AST#getSourceVersion()
*/
public int getSourceVersion() {
return getAst().getSourceVersion();
}
/**
* Convenient shortcut to the owning ast object's top method.
*
* @see AST#top()
*/
public L top() {
return getAst().top();
}
/**
* Convenient shortcut to the owning ast object's getFileName method.
*
* @see AST#getFileName()
*/
public String getFileName() {
return getAst().getFileName();
}
/**
* Adds the stated node as a direct child of this node.
*
* Does not change the underlying (javac/Eclipse) AST, only the wrapped view.
*/
@SuppressWarnings({"unchecked"})
public L add(N newChild, Kind newChildKind) {
getAst().setChanged();
L n = getAst().buildTree(newChild, newChildKind);
```

```
if (n == null) return null;
n.parent = (L) this;
children = children.append(n);
return n;
}
/**
* Reparses the AST node represented by this node. Any existing nodes that occupy a different space in the AST are rehomed, any
* nodes that no longer exist are removed, and new nodes are created.
*
* Careful - the node you call this on must not itself have been removed or rehomed - it rebuilds <i>all children</i>.
*/
public void rebuild() {
Map<N, L> oldNodes = new IdentityHashMap<N, L>();
gatherAndRemoveChildren(oldNodes);
L newNode = getAst().buildTree(get(), kind);
getAst().setChanged();
getAst().replaceNewWithExistingOld(oldNodes, newNode);
}
@SuppressWarnings({"unchecked", "rawtypes"})
private void gatherAndRemoveChildren(Map<N, L> map) {
for (LombokNode child : children) child.gatherAndRemoveChildren(map);
getAst().identityDetector.remove(get());
map.put(get(), (L) this);
children = LombokImmutableList.of();
getAst().getNodeMap().remove(get());
}
/**
* Removes the stated node, which must be a direct child of this node, from the AST.
*
* Does not change the underlying (javac/Eclipse) AST, only the wrapped view.
*/
public void removeChild(L child) {
getAst().setChanged();
children = children.removeElement(child);
}
/**
* Structurally significant means: LocalDeclaration, TypeDeclaration, MethodDeclaration, ConstructorDeclaration,
* FieldDeclaration, Initializer, and CompilationUnitDeclaration.
* The rest is e.g. if statements, while loops, etc.
*/
public boolean isStructurallySignificant() {
return isStructurallySignificant;
}
public abstract boolean hasAnnotation(Class<? extends Annotation> type);
public abstract <Z extends Annotation> AnnotationValues<Z> findAnnotation(Class<Z> type);
public abstract boolean isStatic();
public abstract boolean isFinal();
public abstract boolean isTransient();
public abstract boolean isPrimitive();
public abstract boolean isEnumMember();
```

```
public abstract boolean isEnumType();
/**
* The 'type' of the field or method, or {@code null} if this node is neither.
*
* The type is as it is written in the code (no resolution), includes array dimensions,
* but not necessarily generics.
*
* The main purpose of this method is to verify this type against a list of known types,
* like primitives or primitive wrappers.
*
* @return The 'type' of the field or method, or {@code null} if this node is neither.
*/
public abstract String fieldOrMethodBaseType();
public abstract int countMethodParameters();
public abstract int getStartPos();
}
```

lombok/core/Main.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.PrintStream;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import lombok.spi.Provides;
public class Main {
```

```
private static final Collection<?> HELP_SWITCHES = Collections.unmodifiableList(Arrays.asList(
"/?", "/h", "/help", "-h", "-help", "--help", "help", "h"
));
public static void main(String[] args) throws IOException {
Thread.currentThread().setContextClassLoader(Main.class.getClassLoader());
int err = new Main(SpiLoadUtil.readAllFromIterator(
SpiLoadUtil.findServices(LombokApp.class)), Arrays.asList(args)).go();
if (err != 0) {
System.exit(err);
}
}
@Provides
public static class VersionApp extends LombokApp {
@Override public String getAppName() {
return "version";
}
@Override public String getAppDescription() {
return "prints lombok's version.";
}
@Override public List<String> getAppAliases() {
return Arrays.asList("-version", "--version");
}
@Override public int runApp(List<String> args) {
System.out.println(Version.getFullVersion());
return 0;
}
}
@Provides
public static class LicenseApp extends LombokApp {
@Override public String getAppName() {
return "license";
}
@Override public String getAppDescription() {
return "prints license information.";
}
@Override public List<String> getAppAliases() {
return Arrays.asList("licence", "copyright", "copyleft", "gpl");
}
@Override public int runApp(List<String> args) {
try {
InputStream in = Main.class.getResourceAsStream("/LICENSE");
try {
ByteArrayOutputStream out = new ByteArrayOutputStream();
byte[] b = new byte[65536];
while (true) {
int r = in.read(b);
if (r == -1) break;
out.write(b, 0, r);
}
System.out.println(new String(out.toByteArray()));
return 0;
} finally {
in.close();
```

```
}
} catch (Exception e) {
System.err.println("License file not found. Check https://projectlombok.org/LICENSE");
return 1;
}
}
}
private final List<LombokApp> apps;
private final List<String> args;
public Main(List<LombokApp> apps, List<String> args) {
this.apps = apps;
this.args = args;
}
public int go() {
if (!args.isEmpty() && HELP_SWITCHES.contains(args.get(0))) {
printHelp(null, System.out);
return 0;
}
String command = args.isEmpty() ? "" : args.get(0).trim();
if (command.startsWith("--")) command = command.substring(2);
else if (command.startsWith("-")) command = command.substring(1);
List<String> subArgs = args.isEmpty() ? Collections.<String>emptyList() : Collections.unmodifiableList(
args.subList(1, args.size()));
for (LombokApp app : apps) {
if (app.getAppName().equals(command) || app.getAppAliases().contains(command)) {
try {
return app.runApp(subArgs);
} catch (Exception e) {
e.printStackTrace();
return 5;
}
}
}
printHelp("Unknown command: " + command, System.err);
return 1;
}
public void printHelp(String message, PrintStream out) {
if (message != null) {
out.println(message);
out.println("------------------------------");
}
out.println("projectlombok.org " + Version.getFullVersion());
out.println("Copyright (C) 2009-2021 The Project Lombok Authors.");
out.println("Run 'lombok license' to see the lombok license agreement.");
out.println();
out.println("Run lombok without any parameters to start the graphical installer.");
out.println("Other available commands:");
for (LombokApp app : apps) {
if (app.isDebugTool()) continue;
String[] desc = app.getAppDescription().split("\n");
for (int i = 0; i < desc.length; i++) {
out.printf(" %15s %s\n", i == 0 ? app.getAppName() : "", desc[i]);
}
```

```
}
out.println();
out.println("Run lombok commandName --help for more info on each command.");
}
}
```

lombok/core/PostCompiler.java

```
/*
* Copyright (C) 2010-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.io.PrintWriter;
import java.io.StringWriter;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.atomic.AtomicBoolean;
public final class PostCompiler {
private PostCompiler() {/* prevent instantiation*/};
private static List<PostCompilerTransformation> transformations;
public static byte[] applyTransformations(byte[] original, String fileName, DiagnosticsReceiver
diagnostics) {
if (System.getProperty("lombok.disablePostCompiler", null) != null) return original;
init(diagnostics);
byte[] previous = original;
for (PostCompilerTransformation transformation : transformations) {
try {
byte[] next = transformation.applyTransformations(previous, fileName, diagnostics);
if (next != null) {
previous = next;
```

```
}
} catch (Exception e) {
StringWriter sw = new StringWriter();
e.printStackTrace(new PrintWriter(sw, true));
diagnostics.addError(String.format("Error during the transformation of '%s'; post-compiler '%s' caused
an exception: %s", fileName, transformation.getClass().getName(), sw));
}
}
return previous;
}
private static synchronized void init(DiagnosticsReceiver diagnostics) {
if (transformations != null) return;
try {
transformations =
SpiLoadUtil.readAllFromIterator(SpiLoadUtil.findServices(PostCompilerTransformation.class,
PostCompilerTransformation.class.getClassLoader()));
} catch (IOException e) {
transformations = Collections.emptyList();
StringWriter sw = new StringWriter();
e.printStackTrace(new PrintWriter(sw, true));
diagnostics.addError("Could not load post-compile transformers: " + e.getMessage() + "\n" +
sw.toString());
}
}
public static OutputStream wrapOutputStream(final OutputStream originalStream, final String fileName,
final DiagnosticsReceiver diagnostics) throws IOException {
if (System.getProperty("lombok.disablePostCompiler", null) != null) return originalStream;
// close() can be called more than once and should be idempotent, therefore, ensure we never
transform more than once.
final AtomicBoolean closed = new AtomicBoolean();
return new ByteArrayOutputStream() {
@Override public void close() throws IOException {
if (closed.getAndSet(true)) {
originalStream.close();
return;
}
// no need to call super
byte[] original = toByteArray();
byte[] copy = null;
if (original.length > 0) {
try {
copy = applyTransformations(original, fileName, diagnostics);
} catch (Exception e) {
diagnostics.addWarning(String.format("Error during the transformation of '%s'; no post-compilation has
been applied", fileName));
}
}
if (copy == null) {
copy = original;
}
// Exceptions below should bubble
originalStream.write(copy);
originalStream.close();
```

```
}
};
}
}
```

lombok/core/PostCompilerTransformation.java

```
/*
* Copyright (C) 2010 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
public interface PostCompilerTransformation {
byte[] applyTransformations(byte[] original, String fileName, DiagnosticsReceiver diagnostics);
}
```

lombok/core/PrintAST.java

```
/*
* Copyright (C) 2009-2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Will print the tree structure of annotated node and all its children.
*
* This annotation is useful only for those working on Lombok, for example to test if a Lombok handlers is doing its
* job correctly, or to see what the imagined endresult of a transformation is supposed to look like.
*/
@Target({ElementType.CONSTRUCTOR, ElementType.FIELD, ElementType.LOCAL_VARIABLE, ElementType.METHOD, ElementType.TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface PrintAST {
/**
* Normally, the AST is printed to standard out, but you can pick a filename instead. Useful for many IDEs
* which don't have a console unless you start them from the command line.
*/
String outfile() default "";
/**
* Sets whether to print node structure (false) or generated java code (true).
*
* By setting printContent to true, the annotated element's java code representation is printed. If false,
* its node structure (e.g. node classname) is printed, and this process is repeated for all children.
*/
boolean printContent() default false;
/**
* if {@code true} prints the start and end position of each node.
*/
boolean printPositions() default false;
}
```

lombok/core/PublicApiCreatorApp.java

```
/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
```

```
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.util.ArrayList;
import java.util.Enumeration;
import java.util.List;
import java.util.jar.JarEntry;
import java.util.jar.JarFile;
import java.util.jar.JarOutputStream;
import java.util.zip.ZipEntry;
import lombok.Lombok;
import lombok.patcher.ClassRootFinder;
import lombok.spi.Provides;
@Provides
public class PublicApiCreatorApp extends LombokApp {
@Override public String getAppName() {
return "publicApi";
}
@Override public String getAppDescription() {
return "Creates a small lombok-api.jar with the annotations and other public API\n" +
"classes of all lombok features. This is primarily useful to include in your\n" +
"android projects.";
}
@Override public int runApp(List<String> rawArgs) throws Exception {
String loc = ".";
switch (rawArgs.size()) {
case 0: break;
case 1: loc = rawArgs.get(0); break;
default:
System.err.println("Supply 1 path to specify the directory where lombok-api.jar will be created. No path
means the current directory is used.");
return 1;
}
File out = new File(loc, "lombok-api.jar");
int errCode = 0;
try {
errCode = writeApiJar(out);
```

```
} catch (Exception e) {
System.err.println("ERROR: Creating " + canonical(out) + " failed: ");
e.printStackTrace();
return 1;
}
return errCode;
}
/**
* Returns a File object pointing to our own jar file. Will obviously fail if the installer was started via
* a jar that wasn't accessed via the file-system, or if its started via e.g. unpacking the jar.
*/
private static File findOurJar() {
return new File(ClassRootFinder.findClassRootOfClass(PublicApiCreatorApp.class));
}
private int writeApiJar(File outFile) throws Exception {
File selfRaw = findOurJar();
if (selfRaw == null) {
System.err.println("The publicApi option only works if lombok is a jar.");
return 2;
}
List<String> toCopy = new ArrayList<String>();
JarFile self = new JarFile(selfRaw);
try {
Enumeration<JarEntry> entries = self.entries();
while (entries.hasMoreElements()) {
JarEntry entry = entries.nextElement();
String name = entry.getName();
if (!name.startsWith("lombok/")) continue;
if (name.endsWith("/package-info.class")) continue;
if (!name.endsWith(".class")) continue;
String subName = name.substring(7, name.length() - 6);
int firstSlash = subName.indexOf('/');
if (firstSlash == -1) {
// direct member of the lombok package.
if (!subName.startsWith("ConfigurationKeys")) toCopy.add(name);
continue;
}
String topPkg = subName.substring(0, firstSlash);
if ("extern".equals(topPkg) || "experimental".equals(topPkg)) {
toCopy.add(name);
}
}
} finally {
self.close();
}
if (toCopy.isEmpty()) {
System.out.println("Not generating lombok-api.jar: No lombok api classes required!");
return 1;
}
OutputStream out = new FileOutputStream(outFile);
boolean success = false;
try {
JarOutputStream jar = new JarOutputStream(out);
```

```
for (String resourceName : toCopy) {
InputStream in = Lombok.class.getResourceAsStream("/" + resourceName);
try {
if (in == null) {
throw new Fail(String.format("api class %s cannot be found", resourceName));
}
writeIntoJar(jar, resourceName, in);
} finally {
if (in != null) in.close();
}
}
jar.close();
out.close();
System.out.println("Successfully created: " + canonical(outFile));
return 0;
} catch (Throwable t) {
try { out.close();} catch (Throwable ignore) {}
if (!success) outFile.delete();
if (t instanceof Fail) {
System.err.println(t.getMessage());
return 1;
} else if (t instanceof Exception) {
throw (Exception)t;
} else if (t instanceof Error) {
throw (Error)t;
} else {
throw new Exception(t);
}
}
}
private void writeIntoJar(JarOutputStream jar, String resourceName, InputStream in) throws
IOException {
jar.putNextEntry(new ZipEntry(resourceName));
byte[] b = new byte[65536];
while (true) {
int r = in.read(b);
if (r == -1) break;
jar.write(b, 0, r);
}
jar.closeEntry();
in.close();
}
private static class Fail extends Exception {
Fail(String message) {
super(message);
}
}
private static String canonical(File out) {
try {
return out.getCanonicalPath();
} catch (Exception e) {
return out.getAbsolutePath();
}
```

}
}

lombok/core/TypeLibrary.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
/**
* Library of types, which can be used to look up potential matching types.
*
* For example, if you put {@code foo.Spork} and {@code bar.Spork} into the library, and then ask for
* all compatible types given the type {@code Spork}, you'll get both of them, but you'll only
* get the one if you ask for compatible types given {@code foo.Spork}.
* <p>
* When adding {@code foo.Spork}, that FQN (Fully Qualified Name) will be returned as an option for
any of these queries:
* <ul><li>foo.Spork</li><li>Spork</li><li>foo.*</li></ul>
*/
public class TypeLibrary {
private final Map<String, Object> unqualifiedToQualifiedMap; // maps to usually a string, but could be a
string array in aliasing cases.
private final String unqualified, qualified;
private boolean locked;
public TypeLibrary() {
unqualifiedToQualifiedMap = new HashMap<String, Object>();
```

```
unqualified = null;
qualified = null;
}
public TypeLibrary(TypeLibrary parent) {
unqualifiedToQualifiedMap = new HashMap<String, Object>();
unqualified = null;
qualified = null;
}
public void lock() {
this.locked = true;
}
private TypeLibrary(String fqnSingleton) {
if (fqnSingleton.indexOf("$") != -1) {
unqualifiedToQualifiedMap = new HashMap<String, Object>();
unqualified = null;
qualified = null;
addType(fqnSingleton);
} else {
unqualifiedToQualifiedMap = null;
qualified = fqnSingleton;
int idx = fqnSingleton.lastIndexOf('.');
if (idx == -1) {
unqualified = fqnSingleton;
} else {
unqualified = fqnSingleton.substring(idx + 1);
}
}
locked = true;
}
public static TypeLibrary createLibraryForSingleType(String fqnSingleton) {
if (LombokInternalAliasing.REVERSE_ALIASES.containsKey(fqnSingleton)) {
// Internal aliasing is a little too complex to handle with the map-less 'efficient' implementation.
TypeLibrary tl = new TypeLibrary();
tl.addType(fqnSingleton);
tl.lock();
return tl;
}
return new TypeLibrary(fqnSingleton);
}
/**
* Add a type to the library.
*
* @param fullyQualifiedTypeName the FQN type name, such as 'java.lang.String'.
*/
public void addType(String fullyQualifiedTypeName) {
Collection<String> oldNames =
LombokInternalAliasing.REVERSE_ALIASES.get(fullyQualifiedTypeName);
if (oldNames != null) for (String oldName : oldNames) addType(oldName);
String dotBased = fullyQualifiedTypeName.replace("$", ".");
if (locked) throw new IllegalStateException("locked");
int idx = fullyQualifiedTypeName.lastIndexOf('.');
if (idx == -1) throw new IllegalArgumentException(
"Only fully qualified types are allowed (types in the default package cannot be added here either)");
```

```
String unqualified = fullyQualifiedTypeName.substring(idx + 1);
if (unqualifiedToQualifiedMap == null) throw new IllegalStateException("SingleType library");
put(unqualified.replace("$", "."), dotBased);
put(unqualified, dotBased);
put(fullyQualifiedTypeName, dotBased);
put(dotBased, dotBased);
int idx2 = fullyQualifiedTypeName.indexOf('$', idx + 1);
while (idx2 != -1) {
String unq = fullyQualifiedTypeName.substring(idx2 + 1);
put(unq.replace("$", "."), dotBased);
put(unq, dotBased);
idx2 = fullyQualifiedTypeName.indexOf('$', idx2 + 1);
}
}
/**
* Translates an unqualified name such as 'String' to 'java.lang.String', _if_ you added 'java.lang.String'
to the library via the {@code addType} method.
* Also returns the input if it is equal to a fully qualified name added to this type library.
*
* Returns an empty collection if it does not match any type in this type library.
*/
public List<String> toQualifieds(String typeReference) {
if (unqualifiedToQualifiedMap == null) {
if (typeReference.equals(unqualified) || typeReference.equals(qualified)) return
Collections.singletonList(qualified);
return null;
}
Object v = unqualifiedToQualifiedMap.get(typeReference);
if (v == null) return Collections.emptyList();
if (v instanceof String) return Collections.singletonList((String) v);
return Arrays.asList((String[]) v);
}
private void put(String k, String v) {
Object old = unqualifiedToQualifiedMap.put(k, v);
if (old == null) return;
String[] nv;
if (old instanceof String) {
if (old.equals(v)) return;
nv = new String[] {(String) old, v};
} else {
String[] s = (String[]) old;
nv = new String[s.length + 1];
System.arraycopy(s, 0, nv, 0, s.length);
nv[s.length] = v;
}
unqualifiedToQualifiedMap.put(k, nv);
}
}
```

lombok/core/TypeResolver.java

```
/*
* Copyright (C) 2009-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.List;
import lombok.core.AST.Kind;
/**
* Capable of resolving a simple type name such as 'String' into 'java.lang.String'.
* <p>
* NB: This resolver gives wrong answers when there's a class in the local package with the same name
as a class in a star-import,
* and this importer also can't find inner types from superclasses/interfaces.
*/
public class TypeResolver {
private ImportList imports;
/**
* Creates a new TypeResolver that can be used to resolve types in a source file with the given package
and import statements.
*/
public TypeResolver(ImportList importList) {
this.imports = importList;
}
public boolean typeMatches(LombokNode<?, ?, ?> context, String fqn, String typeRef) {
return typeRefToFullyQualifiedName(context, TypeLibrary.createLibraryForSingleType(fqn), typeRef)
!= null;
}
public String typeRefToFullyQualifiedName(LombokNode<?, ?, ?> context, TypeLibrary library, String
typeRef) {
// When asking if 'Foo' could possibly be referring to 'bar.Baz', the answer is obviously no.
List<String> qualifieds = library.toQualifieds(typeRef);
if (qualifieds == null || qualifieds.isEmpty()) return null;
// When asking if 'lombok.Getter' could possibly be referring to 'lombok.Getter', the answer is obviously
yes.
if (qualifieds.contains(typeRef)) return LombokInternalAliasing.processAliases(typeRef);
```

```
// When asking if 'Getter' could possibly be referring to 'lombok.Getter' if 'import lombok.Getter;' is in the source file, the answer is yes.
int firstDot = typeRef.indexOf('.');
if (firstDot == -1) firstDot = typeRef.length();
String firstTypeRef = typeRef.substring(0, firstDot);
String fromExplicitImport = imports.getFullyQualifiedNameForSimpleNameNoAliasing(firstTypeRef);
if (fromExplicitImport != null) {
String fqn = fromExplicitImport + typeRef.substring(firstDot);
if (qualifieds.contains(fqn)) return LombokInternalAliasing.processAliases(fqn);
// ... and if 'import foobar.Getter;' is in the source file, the answer is no.
return null;
}
// When asking if 'Getter' could possibly be referring to 'lombok.Getter' and 'import lombok.*; / package lombok;' isn't in the source file. the answer is no.
for (String qualified : qualifieds) {
String pkgName = qualified.substring(0, qualified.length() - typeRef.length() - 1);
if (!imports.hasStarImport(pkgName)) continue;
// Now the hard part: Given that there is a star import, 'Getter' most likely refers to 'lombok.Getter', but type shadowing may occur in which case it doesn't.
LombokNode<?, ?, ?> n = context;
mainLoop:
while (n != null) {
if (n.getKind() == Kind.TYPE && firstTypeRef.equals(n.getName())) {
// Our own class or one of our outer classes is named 'typeRef' so that's what 'typeRef' is referring to, not one of our type library classes.
return null;
}
if (n.getKind() == Kind.STATEMENT || n.getKind() == Kind.LOCAL) {
LombokNode<?, ?, ?> newN = n.directUp();
if (newN == null) break mainLoop;
if (newN.getKind() == Kind.STATEMENT || newN.getKind() == Kind.INITIALIZER || newN.getKind() == Kind.METHOD) {
for (LombokNode<?, ?, ?> child : newN.down()) {
// We found a method local with the same name above our code. That's the one 'typeRef' is referring to, not
// anything in the type library we're trying to find, so, no matches.
if (child.getKind() == Kind.TYPE && firstTypeRef.equals(child.getName())) return null;
if (child == n) break;
}
}
n = newN;
continue mainLoop;
}
if (n.getKind() == Kind.TYPE || n.getKind() == Kind.COMPILATION_UNIT) {
for (LombokNode<?, ?, ?> child : n.down()) {
// Inner class that's visible to us has 'typeRef' as name, so that's the one being referred to, not one of our type library classes.
if (child.getKind() == Kind.TYPE && firstTypeRef.equals(child.getName())) return null;
}
}
n = n.directUp();
}
// If no shadowing thing has been found, the star import 'wins', so, return that.
```

```
return LombokInternalAliasing.processAliases(qualified);
}
// No star import matches either.
return null;
}
}
```

lombok/core/Version.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.io.InputStream;
/**
* This class just holds lombok's current version.
*/
public class Version {
// ** CAREFUL ** - this class must always compile with 0 dependencies (it must not refer to any other
sources or libraries).
// Note: In 'X.Y.Z', if Z is odd, its a snapshot build built from the repository, so many different 0.10.3
versions can exist, for example.
// Official builds always end in an even number. (Since 0.10.2).
private static final String VERSION = "1.18.34";
// private static final String RELEASE_NAME = "Edgy Guinea Pig";
private static final String RELEASE_NAME = "Envious Ferret";
// Named version history:
// Angry Butterfly
// Branching Cobra
// Candid Duck
// Dancing Elephant
// Envious Ferret
private Version() {
```

```
//Prevent instantiation
}
/**
* Prints the version followed by a newline, and exits.
*/
public static void main(String[] args) {
if (args.length > 0) {
System.out.printf("%s\n", getFullVersion());
} else {
System.out.println(VERSION);
}
}
/**
* Get the current Lombok version.
*/
public static String getVersion() {
return VERSION;
}
/**
* Get the current release name.
*
* The release name is a string (not numbers). Every time a new release has a significantly improved feature set, a new release name is given.
* Thus, many versions can carry the same release name. Version bumps and release names are not related; if a new version of lombok is entirely
* backwards compatible with a previous one, but also adds many new features, it will get only a minor version bump, but also a new release name.
*/
public static String getReleaseName() {
return RELEASE_NAME;
}
public static String getFullVersion() {
String version = String.format("v%s \"%s\"", VERSION, RELEASE_NAME);
if (!isEdgeRelease()) return version;
InputStream in = Version.class.getResourceAsStream("/release-timestamp.txt");
if (in == null) return version;
try {
byte[] data = new byte[65536];
int p = 0;
while (p < data.length) {
int r = in.read(data, p, data.length - p);
if (r == -1) break;
p += r;
}
String timestamp = new String(data, "UTF-8").trim();
return version + " - " + timestamp;
} catch (Exception e) {
try {
in.close();
} catch (Exception ignore) {}
}
return version;
}
```

```
public static boolean isEdgeRelease() {
int lastIdx = VERSION.lastIndexOf('.');
if (lastIdx == -1) return false;
try {
return Integer.parseInt(VERSION.substring(lastIdx + 1)) % 2 == 1;
} catch (Exception e) {
return false;
}
}
}
```

lombok/core/configuration/AllowHelper.java

```
/*
* Copyright (C) 2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.util.Collection;
import java.util.Collections;
public final class AllowHelper {
private final static Collection<? extends ConfigurationKey<?>> ALLOWABLE = Collections.emptySet();
private AllowHelper() {
// Prevent instantiation
}
public static boolean isAllowable(ConfigurationKey<?> key) {
return ALLOWABLE.contains(key);
}
}
```

lombok/core/configuration/BubblingConfigurationResolver.java

/*
* Copyright (C) 2014-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.core.configuration;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Deque;
import java.util.HashSet;
import java.util.List;
import lombok.ConfigurationKeys;
import lombok.core.configuration.ConfigurationSource.ListModification;
import lombok.core.configuration.ConfigurationSource.Result;
public class BubblingConfigurationResolver implements ConfigurationResolver {
private final ConfigurationFile start;
private final ConfigurationFileToSource fileMapper;
public BubblingConfigurationResolver(ConfigurationFile start, ConfigurationFileToSource fileMapper) {
this.start = start;
this.fileMapper = fileMapper;
}
@SuppressWarnings("unchecked")
@Override
public <T> T resolve(ConfigurationKey<T> key) {
boolean isList = key.getType().isList();
List<List<ListModification>> listModificationsList = null;
boolean stopBubbling = false;
ConfigurationFile currentLevel = start;
Collection<ConfigurationFile> visited = new HashSet<ConfigurationFile>();
outer:
while (currentLevel != null) {
Deque<ConfigurationFile> round = new ArrayDeque<ConfigurationFile>();
round.push(currentLevel);
while (!round.isEmpty()) {
```

```
ConfigurationFile currentFile = round.pop();
if (currentFile == null || !visited.add(currentFile)) continue;
ConfigurationSource source = fileMapper.parsed(currentFile);
if (source == null) continue;
for (ConfigurationFile importFile : source.imports()) round.push(importFile);
Result stop = source.resolve(ConfigurationKeys.STOP_BUBBLING);
stopBubbling = stopBubbling || (stop != null && Boolean.TRUE.equals(stop.getValue()));
Result result = source.resolve(key);
if (result == null) continue;
if (isList) {
if (listModificationsList == null) listModificationsList = new ArrayList<List<ListModification>>();
listModificationsList.add((List<ListModification>) result.getValue());
}
if (result.isAuthoritative()) {
if (isList) break outer;
return (T) result.getValue();
}
}
if (stopBubbling) break;
currentLevel = currentLevel.parent();
}
if (!isList) return null;
if (listModificationsList == null) return (T) Collections.emptyList();
List<Object> listValues = new ArrayList<Object>();
Collections.reverse(listModificationsList);
for (List<ListModification> listModifications : listModificationsList) {
if (listModifications != null) for (ListModification modification : listModifications) {
listValues.remove(modification.getValue());
if (modification.isAdded()) listValues.add(modification.getValue());
}
}
return (T) listValues;
}
}
```

lombok/core/configuration/CallSuperType.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
/** Used for lombok configuration for configuration whether or not to call the super implementation for certain lombok feature. */
public enum CallSuperType {
CALL, SKIP, WARN;
}

lombok/core/configuration/CapitalizationStrategy.java

/*
* Copyright (C) 2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
/** Used for lombok configuration to determine how to transform field names when turning them into accessor method names and vice versa. */
public enum CapitalizationStrategy {
BASIC {
@Override public String capitalize(String in) {
if (in.length() == 0) return in;
char first = in.charAt(0);
if (!Character.isLowerCase(first)) return in;
boolean useUpperCase = in.length() > 2 &&
(Character.isTitleCase(in.charAt(1)) || Character.isUpperCase(in.charAt(1)));
return (useUpperCase ? Character.toUpperCase(first) : Character.toTitleCase(first)) + in.substring(1);

```
}
},
BEANSPEC {
@Override public String capitalize(String in) {
if (in.length() == 0) return in;
char first = in.charAt(0);
if (!Character.isLowerCase(first) || (in.length() > 1 && Character.isUpperCase(in.charAt(1)))) return in;
boolean useUpperCase = in.length() > 2 && Character.isTitleCase(in.charAt(1));
return (useUpperCase ? Character.toUpperCase(first) : Character.toTitleCase(first)) + in.substring(1);
}
},
;
public static CapitalizationStrategy defaultValue() {
return BASIC;
}
public abstract String capitalize(String in);
}
```

lombok/core/configuration/CheckerFrameworkVersion.java

```
/*
* Copyright (C) 2019-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public final class CheckerFrameworkVersion implements ConfigurationValueType {
private final int version;
private static final int DEFAULT = 3200;
private static final int MAX_SUPPORTED = 4000;
```

```
public static final String NAME__SIDE_EFFECT_FREE =
"org.checkerframework.dataflow.qual.SideEffectFree";
public static final String NAME__PURE = "org.checkerframework.dataflow.qual.Pure";
public static final String NAME__UNIQUE = "org.checkerframework.common.aliasing.qual.Unique";
public static final String NAME__RETURNS_RECEIVER =
"org.checkerframework.common.returnsreceiver.qual.This";
public static final String NAME__CALLED =
"org.checkerframework.checker.calledmethods.qual.CalledMethods";
public static final CheckerFrameworkVersion NONE = new CheckerFrameworkVersion(0);
private CheckerFrameworkVersion(int v) {
this.version = v;
}
private static final Pattern VERSION = Pattern.compile("^(\\d+)(?:\\.(\\d+))?(?:\\.\\d+)*$");
public boolean generateSideEffectFree() {
return version > 0;
}
public boolean generateUnique() {
return version > 2899;
}
public boolean generatePure() {
return version > 0;
}
public boolean generateReturnsReceiver() {
return version >= 3100;
}
public boolean generateCalledMethods() {
return version >= 3100;
}
public static CheckerFrameworkVersion valueOf(String versionString) {
if (versionString != null) versionString = versionString.trim();
if (versionString == null || versionString.equalsIgnoreCase("false") || versionString.equals("0")) return
new CheckerFrameworkVersion(0);
if (versionString.equalsIgnoreCase("true")) return new CheckerFrameworkVersion(DEFAULT);
Matcher m = VERSION.matcher(versionString);
if (!m.matches()) throw new IllegalArgumentException("Expected 'true' or 'false' or a major/minor
version, such as '2.9'");
int major = Integer.parseInt(m.group(1));
int minor = (m.group(2) != null && !m.group(2).isEmpty()) ? Integer.parseInt(m.group(2)) : 0;
if (minor > 999) throw new IllegalArgumentException("Minor version must be between 0 and 999");
int v = major * 1000 + minor;
if (v > MAX_SUPPORTED) {
String s = (v / 1000) + "." + (v % 1000);
throw new IllegalArgumentException("Lombok supports at most v" + s + "; reduce the value of key
'checkerframework' to " + s);
}
return new CheckerFrameworkVersion(v);
}
public static String description() {
return "checkerframework-version";
}
public static String exampleValue() {
String s = (MAX_SUPPORTED / 1000) + "." + (MAX_SUPPORTED % 1000);
return "major.minor (example: 3.2 - and no higher than " + s + ") or true or false";
```

```
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof CheckerFrameworkVersion)) return false;
return version == ((CheckerFrameworkVersion) obj).version;
}
@Override public int hashCode() {
return version;
}
}
```

lombok/core/configuration/ConfigurationApp.java

```
/*
* Copyright (C) 2014-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.io.File;
import java.io.IOException;
import java.io.PrintStream;
import java.net.URI;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Set;
```

```
import java.util.TreeMap;
import java.util.TreeSet;
import com.zwitserloot.cmdreader.CmdReader;
import com.zwitserloot.cmdreader.Description;
import com.zwitserloot.cmdreader.Excludes;
import com.zwitserloot.cmdreader.FullName;
import com.zwitserloot.cmdreader.InvalidCommandLineException;
import com.zwitserloot.cmdreader.Mandatory;
import com.zwitserloot.cmdreader.Requires;
import com.zwitserloot.cmdreader.Sequential;
import com.zwitserloot.cmdreader.Shorthand;
import lombok.ConfigurationKeys;
import lombok.core.LombokApp;
import lombok.core.configuration.ConfigurationParser.Collector;
import lombok.spi.Provides;
@Provides
public class ConfigurationApp extends LombokApp {
private static final URI NO_CONFIG = URI.create("");
private PrintStream out = System.out;
private PrintStream err = System.err;
@Override public String getAppName() {
return "config";
}
@Override public String getAppDescription() {
return "Prints the configurations for the provided paths to standard out.";
}
@Override public List<String> getAppAliases() {
return Arrays.asList("configuration", "config", "conf", "settings");
}
public static class CmdArgs {
@Sequential
@Mandatory(onlyIfNot={"help", "generate"})
@Description("Paths to java files or directories the configuration is to be printed for.")
private List<String> paths = new ArrayList<String>();
@Shorthand("g")
@Excludes("paths")
@Description("Generates a list containing all the available configuration parameters. Add --verbose to
print more information.")
boolean generate = false;
@Shorthand("v")
@Description("Displays more information.")
boolean verbose = false;
@Shorthand("n")
@FullName("not-mentioned")
@Requires("verbose")
@Description("Also display files that don't mention the key.")
boolean notMentioned = false;
@Shorthand("k")
@Description("Limit the result to these keys.")
private List<String> key = new ArrayList<String>();
@Shorthand({"h", "?"})
@Description("Shows this help text.")
boolean help = false;
```

```
}
@Override public int runApp(List<String> raw) throws Exception {
CmdReader<CmdArgs> reader = CmdReader.of(CmdArgs.class);
CmdArgs args;
try {
args = reader.make(raw.toArray(new String[0]));
if (args.help) {
out.println(reader.generateCommandLineHelp("java -jar lombok.jar configuration"));
return 0;
}
} catch (InvalidCommandLineException e) {
err.println(e.getMessage());
err.println(reader.generateCommandLineHelp("java -jar lombok.jar configuration"));
return 1;
}
ConfigurationKeysLoader.LoaderLoader.loadAllConfigurationKeys();
Collection<ConfigurationKey<?>> keys = checkKeys(args.key);
if (keys == null) return 1;
boolean verbose = args.verbose;
if (args.generate) {
return generate(keys, verbose, !args.key.isEmpty());
}
return display(keys, verbose, args.paths, !args.key.isEmpty(), args.notMentioned);
}
public ConfigurationApp redirectOutput(PrintStream out, PrintStream err) {
if (out != null) this.out = out;
if (err != null) this.err = err;
return this;
}
public int generate(Collection<ConfigurationKey<?>> keys, boolean verbose, boolean explicit) {
for (ConfigurationKey<?> key : keys) {
if (!explicit && key.isHidden()) continue;
String keyName = key.getKeyName();
ConfigurationDataType type = key.getType();
String description = key.getDescription();
boolean hasDescription = description != null && !description.isEmpty();
if (!verbose) {
out.println(keyName);
if (hasDescription) {
out.print(" ");
out.println(description);
}
out.println();
continue;
}
out.printf("##%n## Key : %s%n## Type: %s%n", keyName, type);
if (hasDescription) {
out.printf("##%n## %s%n", description);
}
out.printf("##%n## Examples:%n#%n");
out.printf("# clear %s%n", keyName);
String exampleValue = type.getParser().exampleValue();
if (type.isList()) {
```

```
out.printf("# %s += %s%n", keyName, exampleValue);
out.printf("# %s -= %s%n", keyName, exampleValue);
} else {
out.printf("# %s = %s%n", keyName, exampleValue);
}
out.printf("#%n%n");
}
if (!verbose) {
out.println("Use --verbose for more information.");
}
return 0;
}
public int display(Collection<ConfigurationKey<?>> keys, boolean verbose, Collection<String>
argsPaths, boolean explicitKeys, boolean notMentioned) throws Exception {
TreeMap<URI, Set<String>> sharedDirectories = findSharedDirectories(argsPaths);
if (sharedDirectories == null) return 1;
Set<String> none = sharedDirectories.remove(NO_CONFIG);
if (none != null) {
if (none.size() == 1) {
out.printf("No 'lombok.config' found for '%s'.%n", none.iterator().next());
} else {
out.println("No 'lombok.config' found for: ");
for (String path : none) out.printf("- %s%n", path);
}
}
final List<String> problems = new ArrayList<String>();
ConfigurationProblemReporter reporter = new ConfigurationProblemReporter() {
@Override public void report(String sourceDescription, String problem, int lineNumber, CharSequence
line) {
problems.add(String.format("%s: %s (%s:%d)", problem, line, sourceDescription, lineNumber));
}
};
FileSystemSourceCache cache = new FileSystemSourceCache();
ConfigurationParser parser = new ConfigurationParser(reporter);
boolean first = true;
for (Entry<URI, Set<String>> entry : sharedDirectories.entrySet()) {
if (!first) {
out.printf("%n%n");
}
Set<String> paths = entry.getValue();
if (paths.size() == 1) {
if (!(argsPaths.size() == 1)) out.printf("Configuration for '%s'.%n%n", paths.iterator().next());
} else {
out.printf("Configuration for:%n");
for (String path : paths) out.printf("- %s%n", path);
out.println();
}
URI directory = entry.getKey();
ConfigurationResolver resolver = new BubblingConfigurationResolver(cache.forUri(directory),
cache.fileToSource(parser));
Map<ConfigurationKey<?>, ? extends Collection<String>> traces = trace(keys, directory,
notMentioned);
boolean printed = false;
```

```
for (ConfigurationKey<?> key : keys) {
Object value = resolver.resolve(key);
Collection<String> modifications = traces.get(key);
if (!modifications.isEmpty() || explicitKeys) {
if (printed && verbose) out.println();
printValue(key, value, verbose, modifications);
printed = true;
}
}
if (!printed) out.println("<default>");
first = false;
}
if (!problems.isEmpty()) {
err.printf("Problems in the configuration files:%n");
for (String problem : problems) err.printf("- %s%n", problem);
return 2;
}
return 0;
}
private void printValue(ConfigurationKey<?> key, Object value, boolean verbose, Collection<String>
history) {
if (verbose) out.printf("# %s%n", key.getDescription());
if (value == null) {
out.printf("clear %s%n", key.getKeyName());
} else if (value instanceof List<?>) {
List<?> list = (List<?>)value;
if (list.isEmpty()) out.printf("clear %s%n", key.getKeyName());
for (Object element : list) out.printf("%s += %s%n", key.getKeyName(), element);
} else {
out.printf("%s = %s%n", key.getKeyName(), value);
}
if (!verbose) return;
for (String modification : history) out.printf("# %s%n", modification);
}
private static final ConfigurationProblemReporter VOID = new ConfigurationProblemReporter() {
@Override public void report(String sourceDescription, String problem, int lineNumber, CharSequence
line) {}
};
private Map<ConfigurationKey<?>, ? extends Collection<String>>
trace(Collection<ConfigurationKey<?>> keys, URI directory, boolean notMentioned) throws Exception {
Map<ConfigurationKey<?>, List<String>> result = new HashMap<ConfigurationKey<?>,
List<String>>();
for (ConfigurationKey<?> key : keys) result.put(key, new ArrayList<String>());
Set<ConfigurationKey<?>> used = new HashSet<ConfigurationKey<?>>();
boolean stopBubbling = false;
Collection<ConfigurationFile> visited = new HashSet<ConfigurationFile>();
for (ConfigurationFile context = ConfigurationFile.forDirectory(new File(directory)); context != null &&
!stopBubbling; context = context.parent()) {
if (!context.exists()) continue;
Deque<Source> round = new ArrayDeque<Source>();
round.push(new Source(context, context.description()));
while (!round.isEmpty()) {
Source current = round.pop();
```

```
if (current == null || !visited.add(current.file) || !current.file.exists()) continue;
Map<ConfigurationKey<?>, List<String>> traces = trace(current.file, keys, round);
stopBubbling = stopBubbling(traces.get(ConfigurationKeys.STOP_BUBBLING));
for (ConfigurationKey<?> key : keys) {
List<String> modifications = traces.get(key);
if (modifications == null) {
modifications = new ArrayList<String>();
if (notMentioned) {
modifications.add("");
modifications.add(current.description + ":");
modifications.add(" <'" + key.getKeyName() + "' not mentioned>");
}
} else {
used.add(key);
modifications.add(0, current.description + ":");
modifications.add(0, "");
}
result.get(key).addAll(0, modifications);
}
}
}
for (ConfigurationKey<?> key : keys) {
if (used.contains(key)) {
List<String> modifications = result.get(key);
modifications.remove(0);
if (stopBubbling) {
String mostRecent = modifications.get(0);
modifications.set(0, mostRecent.substring(0, mostRecent.length() - 1) + " (stopped bubbling):");
}
} else {
result.put(key, Collections.<String>emptyList());
}
}
return result;
}
private static final class Source {
final ConfigurationFile file;
final String description;
Source(ConfigurationFile file, String description) {
this.file = file;
this.description = description;
}
}
private Map<ConfigurationKey<?>, List<String>> trace(ConfigurationFile context, final
Collection<ConfigurationKey<?>> keys, final Deque<Source> round) throws IOException {
final Map<ConfigurationKey<?>, List<String>> result = new HashMap<ConfigurationKey<?>,
List<String>>();
Collector collector = new Collector() {
@Override public void addImport(ConfigurationFile importFile, ConfigurationFile context, int
lineNumber) {
round.push(new Source(importFile, importFile.description() + " (imported from " + context.description()
+ ":" + lineNumber + ")"));
}
```

```
@Override public void clear(ConfigurationKey<?> key, ConfigurationFile context, int lineNumber) {
trace(key, "clear " + key.getKeyName(), lineNumber);
}
@Override public void set(ConfigurationKey<?> key, Object value, ConfigurationFile context, int
lineNumber) {
trace(key, key.getKeyName() + " = " + value, lineNumber);
}
@Override public void add(ConfigurationKey<?> key, Object value, ConfigurationFile context, int
lineNumber) {
trace(key, key.getKeyName() + " += " + value, lineNumber);
}
@Override public void remove(ConfigurationKey<?> key, Object value, ConfigurationFile context, int
lineNumber) {
trace(key, key.getKeyName() + " -= " + value, lineNumber);
}
private void trace(ConfigurationKey<?> key, String message, int lineNumber) {
if (!keys.contains(key) && key != ConfigurationKeys.STOP_BUBBLING) return;
List<String> traces = result.get(key);
if (traces == null) {
traces = new ArrayList<String>();
result.put(key, traces);
}
traces.add(String.format("%4d: %s", lineNumber, message));
}
};
new ConfigurationParser(VOID).parse(context, collector);
return result;
}
private boolean stopBubbling(List<String> stops) {
return stops != null && !stops.isEmpty() && stops.get(stops.size() -1).endsWith("true");
}
private Collection<ConfigurationKey<?>> checkKeys(List<String> keyList) {
Map<String, ConfigurationKey<?>> registeredKeys = ConfigurationKey.registeredKeys();
if (keyList.isEmpty()) return registeredKeys.values();
Collection<ConfigurationKey<?>> keys = new ArrayList<ConfigurationKey<?>>();
for (String keyName : keyList) {
ConfigurationKey<?> key = registeredKeys.get(keyName);
if (key == null) {
err.printf("Unknown key '%s'%n", keyName);
return null;
}
keys.remove(key);
keys.add(key);
}
return keys;
}
private TreeMap<URI, Set<String>> findSharedDirectories(Collection<String> paths) {
TreeMap<URI,Set<String>> sharedDirectories = new TreeMap<URI, Set<String>>(new
Comparator<URI>() {
@Override public int compare(URI o1, URI o2) {
return o1.toString().compareTo(o2.toString());
}
});
```

```
for (String path : paths) {
File file = new File(path);
if (!file.exists()) {
err.printf("File not found: '%s'%n", path);
return null;
}
URI first = findFirstLombokDirectory(file);
Set<String> sharedBy = sharedDirectories.get(first);
if (sharedBy == null) {
sharedBy = new TreeSet<String>();
sharedDirectories.put(first, sharedBy);
}
sharedBy.add(path);
}
return sharedDirectories;
}
private URI findFirstLombokDirectory(File file) {
File current = new File(file.toURI().normalize());
if (file.isFile()) current = current.getParentFile();
while (current != null) {
if (new File(current, "lombok.config").exists()) return current.toURI();
current = current.getParentFile();
}
return NO_CONFIG;
}
}
```

lombok/core/configuration/ConfigurationDataType.java


```
/*
* Copyright (C) 2013-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
```

```
*/
package lombok.core.configuration;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.ParameterizedType;
import java.lang.reflect.Type;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
public final class ConfigurationDataType {
private static final Map<Class<?>, ConfigurationValueParser> SIMPLE_TYPES;
static {
Map<Class<?>, ConfigurationValueParser> map = new HashMap<Class<?>,
ConfigurationValueParser>();
map.put(String.class, new ConfigurationValueParser() {
@Override public Object parse(String value) {
return value;
}
@Override public String description() {
return "string";
}
@Override public String exampleValue() {
return "<text>";
}
});
map.put(Integer.class, new ConfigurationValueParser() {
@Override public Object parse(String value) {
return Integer.parseInt(value);
}
@Override public String description() {
return "int";
}
@Override public String exampleValue() {
return "<int>";
}
});
map.put(Long.class, new ConfigurationValueParser() {
@Override public Object parse(String value) {
return Long.parseLong(value);
}
@Override public String description() {
return "long";
}
@Override public String exampleValue() {
return "<long>";
}
});
map.put(Double.class, new ConfigurationValueParser() {
@Override public Object parse(String value) {
return Double.parseDouble(value);
}
@Override public String description() {
```

```
return "double";
}
@Override public String exampleValue() {
return "<double>";
}
});
map.put(Boolean.class, new ConfigurationValueParser() {
@Override public Object parse(String value) {
return Boolean.parseBoolean(value);
}
@Override public String description() {
return "boolean";
}
@Override public String exampleValue() {
return "[false | true]";
}
});
SIMPLE_TYPES = map;
}
private static ConfigurationValueParser enumParser(Type enumType) {
final Class<?> type = (Class<?>) enumType;
@SuppressWarnings("rawtypes") final Class rawType = type;
return new ConfigurationValueParser() {
@SuppressWarnings("unchecked")
@Override public Object parse(String value) {
try {
return Enum.valueOf(rawType, value);
} catch (Exception e) {
StringBuilder sb = new StringBuilder();
for (int i = 0; i < value.length(); i++) {
char c = value.charAt(i);
if (Character.isUpperCase(c) && i > 0) sb.append("_");
sb.append(Character.toUpperCase(c));
}
return Enum.valueOf(rawType, sb.toString());
}
}
@Override public String description() {
return "enum (" + type.getName() + ")";
}
@Override public String exampleValue() {
ExampleValueString evs = type.getAnnotation(ExampleValueString.class);
if (evs != null) return evs.value();
return Arrays.toString(type.getEnumConstants()).replace(",", " |");
}
};
}
private static ConfigurationValueParser valueTypeParser(Type argumentType) {
final Class<?> type = (Class<?>) argumentType;
final Method valueOfMethod = getMethod(type, "valueOf", String.class);
final Method descriptionMethod = getMethod(type, "description");
final Method exampleValueMethod = getMethod(type, "exampleValue");
return new ConfigurationValueParser() {
```

```
@Override public Object parse(String value) {
return invokeStaticMethod(valueOfMethod, value);
}
@Override public String description() {
return invokeStaticMethod(descriptionMethod);
}
@Override public String exampleValue() {
return invokeStaticMethod(exampleValueMethod);
}
@SuppressWarnings("unchecked")
private <R> R invokeStaticMethod(Method method, Object... arguments) {
try {
return (R) method.invoke(null, arguments);
} catch (IllegalAccessException e) {
throw new IllegalStateException("The method " + method.getName() + " ", e);
} catch (InvocationTargetException e) {
// There shouldn't be any checked Exception, only IllegalArgumentException is expected
throw (RuntimeException) e.getTargetException();
}
}
};
}
private final boolean isList;
private final ConfigurationValueParser parser;
public static ConfigurationDataType toDataType(Class<? extends ConfigurationKey<?>> keyClass) {
if (keyClass.getSuperclass() != ConfigurationKey.class) {
throw new IllegalArgumentException("No direct subclass of ConfigurationKey: " +
keyClass.getName());
}
Type type = keyClass.getGenericSuperclass();
if (!(type instanceof ParameterizedType)) {
throw new IllegalArgumentException("Missing type parameter in " + type);
}
ParameterizedType parameterized = (ParameterizedType) type;
Type argumentType = parameterized.getActualTypeArguments()[0];
boolean isList = false;
if (argumentType instanceof ParameterizedType) {
ParameterizedType parameterizedArgument = (ParameterizedType) argumentType;
if (parameterizedArgument.getRawType() == List.class) {
isList = true;
argumentType = parameterizedArgument.getActualTypeArguments()[0];
}
}
if (SIMPLE_TYPES.containsKey(argumentType)) {
return new ConfigurationDataType(isList, SIMPLE_TYPES.get(argumentType));
}
if (isEnum(argumentType)) {
return new ConfigurationDataType(isList, enumParser(argumentType));
}
if (isConfigurationValueType(argumentType)) {
return new ConfigurationDataType(isList, valueTypeParser(argumentType));
}
throw new IllegalArgumentException("Unsupported type parameter in " + type);
```

```
}
private ConfigurationDataType(boolean isList, ConfigurationValueParser parser) {
this.isList = isList;
this.parser = parser;
}
public boolean isList() {
return isList;
}
ConfigurationValueParser getParser() {
return parser;
}
@Override
public String toString() {
if (isList) return "list of " + parser.description();
return parser.description();
}
private static boolean isEnum(Type argumentType) {
return argumentType instanceof Class && ((Class<?>) argumentType).isEnum();
}
private static boolean isConfigurationValueType(Type argumentType) {
return argumentType instanceof Class && ConfigurationValueType.class.isAssignableFrom((Class<?>)
argumentType);
}
private static Method getMethod(Class<?> argumentType, String name, Class<?>... parameterTypes) {
try {
return argumentType.getMethod(name, parameterTypes);
} catch (NoSuchMethodException e) {
throw new IllegalStateException("Method " + name + " with parameters " +
Arrays.toString(parameterTypes) + " was not found.", e);
} catch (SecurityException e) {
throw new IllegalStateException("Cannot inspect methods of type " + argumentType, e);
}
}
}
```

lombok/core/configuration/ConfigurationFile.java

```
/*
* Copyright (C) 2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.net.URI;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import java.util.zip.ZipEntry;
import java.util.zip.ZipInputStream;
public abstract class ConfigurationFile {
private static final Pattern VARIABLE = Pattern.compile("\\<(.+?)\\>");
private static final String LOMBOK_CONFIG_FILENAME = "lombok.config";
private static final Map<String, String> ENV = new HashMap<String, String>(System.getenv());
private static final ThreadLocal<byte[]> buffers = new ThreadLocal<byte[]>() {
protected byte[] initialValue() {
return new byte[65536];
}
};
static void setEnvironment(String key, String value) {
ENV.put(key, value);
}
private final String identifier;
public static ConfigurationFile forFile(File file) {
return new RegularConfigurationFile(file);
}
public static ConfigurationFile forDirectory(File directory) {
return forFile(new File(directory, LOMBOK_CONFIG_FILENAME));
}
public static ConfigurationFile fromCharSequence(String identifier, CharSequence contents, long
lastModified) {
return new CharSequenceConfigurationFile(identifier, contents, lastModified);
}
private ConfigurationFile(String identifier) {
this.identifier = identifier;
}
abstract long getLastModifiedOrMissing();
abstract boolean exists();
abstract CharSequence contents() throws IOException;
public abstract ConfigurationFile resolve(String path);
abstract ConfigurationFile parent();
```

```
final String description() {
return identifier;
}
@Override public final boolean equals(Object obj) {
if (!(obj instanceof ConfigurationFile)) return false;
return identifier.equals(((ConfigurationFile)obj).identifier);
}
@Override public final int hashCode() {
return identifier.hashCode();
}
public static long getLastModifiedOrMissing(File file) {
if (!fileExists(file)) return FileSystemSourceCache.MISSING;
return file.lastModified();
}
private static boolean fileExists(File file) {
return file.exists() && file.isFile();
}
private static String read(InputStream is) throws IOException {
byte[] b = buffers.get();
ByteArrayOutputStream out = new ByteArrayOutputStream();
while (true) {
int r = is.read(b);
if (r == -1) break;
out.write(b, 0, r);
}
return new String(out.toByteArray(), "UTF-8");
}
private static class RegularConfigurationFile extends ConfigurationFile {
private final File file;
private ConfigurationFile parent;
private RegularConfigurationFile(File file) {
super(file.getPath());
this.file = file;
}
@Override boolean exists() {
return fileExists(file);
}
public ConfigurationFile resolve(String path) {
if (path.endsWith("!")) return null;
String[] parts = path.split("!");
if (parts.length > 2) return null;
String realFileName = parts[0];
File file = resolveFile(replaceEnvironmentVariables(realFileName));
if (realFileName.endsWith(".zip") || realFileName.endsWith(".jar")) {
try {
return ArchivedConfigurationFile.create(file, URI.create(parts.length == 1 ?
LOMBOK_CONFIG_FILENAME : parts[1]));
} catch (Exception e) {
return null;
}
}
if (parts.length > 1) return null;
return file == null ? null : forFile(file);
```

```
}
private File resolveFile(String path) {
boolean absolute = false;
int colon = path.indexOf(':');
if (colon != -1) {
if (colon != 1 || path.indexOf(':', colon + 1) != -1) return null;
char firstCharacter = Character.toLowerCase(path.charAt(0));
if (firstCharacter < 'a' || firstCharacter > 'z') return null;
absolute = true;
}
if (path.charAt(0) == '/') absolute = true;
try {
return absolute ? new File(path) : new File(file.toURI().resolve(path));
} catch (Exception e) {
return null;
}
}
@Override
long getLastModifiedOrMissing() {
return getLastModifiedOrMissing(file);
}
@Override
CharSequence contents() throws IOException {
FileInputStream is = new FileInputStream(file);
try {
return read(is);
} finally {
is.close();
}
}
@Override ConfigurationFile parent() {
if (parent == null) {
File parentFile = file.getParentFile().getParentFile();
parent = parentFile == null ? null : forDirectory(parentFile);
}
return parent;
}
private static String replaceEnvironmentVariables(String fileName) {
int start = 0;
StringBuffer result = new StringBuffer();
if (fileName.startsWith("~")) {
start = 1;
result.append(System.getProperty("user.home", "~"));
}
Matcher matcher = VARIABLE.matcher(fileName.substring(start));
while (matcher.find()) {
String key = matcher.group(1);
String value = ENV.get(key);
if (value == null) value = "<" + key + ">";
matcher.appendReplacement(result, value);
}
matcher.appendTail(result);
return result.toString();
```

```
}
}
private static class ArchivedConfigurationFile extends ConfigurationFile {
private static final URI ROOT1 = URI.create("http://x.y/a/");
private static final URI ROOT2 = URI.create("ftp://y.x/b/");
private static final ConcurrentMap<String, Object> locks = new ConcurrentHashMap<String, Object>();
private final File archive;
private final URI file;
private final Object lock;
private long lastModified = -2;
private String contents;
public static ConfigurationFile create(File archive, URI file) {
if (!isRelative(file)) return null;
return new ArchivedConfigurationFile(archive, file, archive.getPath() + "!" + file.getPath());
}
static boolean isRelative(URI path) {
try {
return ROOT1.resolve(path).toString().startsWith(ROOT1.toString()) &&
ROOT2.resolve(path).toString().startsWith(ROOT2.toString());
} catch (Exception e) {
return false;
}
}
ArchivedConfigurationFile(File archive, URI file, String description) {
super(description);
this.archive = archive;
this.file = file;
locks.putIfAbsent(archive.getPath(), new Object());
this.lock = locks.get(archive.getPath());
}
@Override
long getLastModifiedOrMissing() {
return getLastModifiedOrMissing(archive);
}
@Override
boolean exists() {
if (!fileExists(archive)) return false;
synchronized (lock) {
try {
readIfNeccesary();
return contents != null;
} catch (Exception e) {
return false;
}
}
}
@Override
CharSequence contents() throws IOException {
synchronized (lock) {
readIfNeccesary();
return contents;
}
}
```

```
void readIfNeccesary() throws IOException {
long archiveModified = getLastModifiedOrMissing();
if (archiveModified == lastModified) return;
contents = null;
lastModified = archiveModified;
if (archiveModified == FileSystemSourceCache.MISSING) return;
contents = read();
}
private String read() throws IOException {
FileInputStream is = new FileInputStream(archive);
try {
ZipInputStream zip = new ZipInputStream(is);
try {
while (true) {
ZipEntry entry = zip.getNextEntry();
if (entry == null) return null;
if (entry.getName().equals(file.getPath())) {
return read(zip);
}
}
} finally {
zip.close();
}
} finally {
is.close();
}
}
@Override
public ConfigurationFile resolve(String path) {
try {
URI resolved = file.resolve(path);
if (!isRelative(resolved)) return null;
return create(archive, resolved);
} catch (Exception e) {
return null;
}
}
@Override
ConfigurationFile parent() {
return null;
}
}
private static class CharSequenceConfigurationFile extends ConfigurationFile {
private final CharSequence contents;
private final long lastModified;
private CharSequenceConfigurationFile(String identifier, CharSequence contents, long lastModified) {
super(identifier);
this.contents = contents;
this.lastModified = lastModified;
}
@Override long getLastModifiedOrMissing() {
return lastModified;
}
```

```
@Override CharSequence contents() throws IOException {
return contents;
}
@Override boolean exists() {
return true;
}
@Override public ConfigurationFile resolve(String path) {
return null;
}
@Override ConfigurationFile parent() {
return null;
}
}
}
```

lombok/core/configuration/ConfigurationFileToSource.java

```
/*
* Copyright (C) 2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
public interface ConfigurationFileToSource {
ConfigurationSource parsed(ConfigurationFile fileLocation);
}
```

lombok/core/configuration/ConfigurationKey.java

```
/*
* Copyright (C) 2013-2018 The Project Lombok Authors.
*
```

```
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.util.Collections;
import java.util.Map;
import java.util.TreeMap;
import java.util.regex.Pattern;
/**
* Describes a configuration key and its type.
* <p>
* The recommended usage is to create a type token:
* <pre>
* private static ConfigurationKey&lt;String> KEY = new ConfigurationKey&lt;String>("keyName",
"description") {};
* </pre>
*/
public abstract class ConfigurationKey<T> {
private static final Pattern VALID_NAMES = Pattern.compile("[-_a-zA-Z][-.\\w]*(?<![-.])");
private static final TreeMap<String, ConfigurationKey<?>> registeredKeys = new TreeMap<String,
ConfigurationKey<?>>(String.CASE_INSENSITIVE_ORDER);
private static Map<String, ConfigurationKey<?>> copy;
private final String keyName;
private final String description;
private final ConfigurationDataType type;
private final boolean hidden;
public ConfigurationKey(String keyName, String description) {
this(keyName, description, false);
}
public ConfigurationKey(String keyName, String description, boolean hidden) {
this.keyName = checkName(keyName);
@SuppressWarnings("unchecked")
ConfigurationDataType type = ConfigurationDataType.toDataType((Class<? extends
ConfigurationKey<?>>)getClass());
this.type = type;
this.description = description;
this.hidden = hidden;
registerKey(keyName, this);
```

```
}
public final String getKeyName() {
return keyName;
}
public final String getDescription() {
return description;
}
public final ConfigurationDataType getType() {
return type;
}
public final boolean isHidden() {
return hidden;
}
@Override public String toString() {
return keyName + " (" + type + "): " + description;
}
private static String checkName(String keyName) {
if (keyName == null) throw new NullPointerException("keyName");
if (!VALID_NAMES.matcher(keyName).matches()) throw new IllegalArgumentException("Invalid
keyName: " + keyName);
return keyName;
}
/**
* Returns a copy of the currently registered keys.
*/
@SuppressWarnings("unchecked")
public static Map<String, ConfigurationKey<?>> registeredKeys() {
synchronized (registeredKeys) {
if (copy == null) copy = Collections.unmodifiableMap((Map<String, ConfigurationKey<?>>)
registeredKeys.clone());
return copy;
}
}
private static void registerKey(String keyName, ConfigurationKey<?> key) {
synchronized (registeredKeys) {
if (registeredKeys.containsKey(keyName)) throw new IllegalArgumentException("Key '" + keyName + "'
already registered");
registeredKeys.put(keyName, key);
copy = null;
}
}
}
```

lombok/core/configuration/ConfigurationKeysLoader.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
```

```
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.io.IOException;
import java.util.Iterator;
import java.util.concurrent.atomic.AtomicBoolean;
import lombok.ConfigurationKeys;
import lombok.core.SpiLoadUtil;
public interface ConfigurationKeysLoader {
public class LoaderLoader {
private static final AtomicBoolean alreadyLoaded = new AtomicBoolean(false);
private LoaderLoader() {}
public static void loadAllConfigurationKeys() {
if (alreadyLoaded.get()) return;
try {
Class.forName(ConfigurationKeys.class.getName());
} catch (Throwable ignore) {}
try {
Iterator<ConfigurationKeysLoader> iterator = SpiLoadUtil.findServices(ConfigurationKeysLoader.class,
ConfigurationKeysLoader.class.getClassLoader()).iterator();
while (iterator.hasNext()) {
try {
iterator.next();
} catch (Exception ignore) {}
}
} catch (IOException e) {
throw new RuntimeException("Can't load config keys; services file issue.", e);
} finally {
alreadyLoaded.set(true);
}
}
}
}
```

lombok/core/configuration/ConfigurationParser.java

```
/*
* Copyright (C) 2014-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.io.IOException;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public class ConfigurationParser {
private static final Pattern LINE = Pattern.compile("(?:clear\\s+([^=]+))|(?:(\\S*?)\\s*([-+]?=)\\s*(.*?))");
private static final Pattern NEWLINE_FINDER = Pattern.compile("^[\t ]*(.*?)[\t\r ]*$",
Pattern.MULTILINE);
private static final Pattern IMPORT = Pattern.compile("import\\s+(.+?)");
private ConfigurationProblemReporter reporter;
public ConfigurationParser(ConfigurationProblemReporter reporter) {
if (reporter == null) throw new NullPointerException("reporter");
this.reporter = reporter;
}
public void parse(ConfigurationFile context, Collector collector) {
CharSequence contents = contents(context);
if (contents == null) {
return;
}
Map<String, ConfigurationKey<?>> registeredKeys = ConfigurationKey.registeredKeys();
int lineNumber = 0;
Matcher lineMatcher = NEWLINE_FINDER.matcher(contents);
boolean importsAllowed = true;
while (lineMatcher.find()) {
CharSequence line = contents.subSequence(lineMatcher.start(1), lineMatcher.end(1));
lineNumber++;
if (line.length() == 0 || line.charAt(0) == '#') continue;
Matcher importMatcher = IMPORT.matcher(line);
if (importMatcher.matches()) {
if (!importsAllowed) {
reporter.report(context.description(), "Imports are only allowed in the top of the file", lineNumber, line);
```

```
continue;
}
String imported = importMatcher.group(1);
ConfigurationFile importFile = context.resolve(imported);
if (importFile == null) {
reporter.report(context.description(), "Import is not valid", lineNumber, line);
continue;
}
if (!importFile.exists()) {
reporter.report(context.description(), "Imported file does not exist", lineNumber, line);
continue;
}
collector.addImport(importFile, context, lineNumber);
continue;
}
Matcher matcher = LINE.matcher(line);
if (!matcher.matches()) {
reporter.report(context.description(), "Invalid line", lineNumber, line);
continue;
}
importsAllowed = false;
String operator = null;
String keyName = null;
String stringValue;
if (matcher.group(1) == null) {
keyName = matcher.group(2);
operator = matcher.group(3);
stringValue = matcher.group(4);
} else {
keyName = matcher.group(1);
operator = "clear";
stringValue = null;
}
ConfigurationKey<?> key = registeredKeys.get(keyName);
if (key == null) {
reporter.report(context.description(), "Unknown key '" + keyName + "'", lineNumber, line);
continue;
}
ConfigurationDataType type = key.getType();
boolean listOperator = operator.equals("+=") || operator.equals("-=");
if (listOperator && !type.isList()) {
reporter.report(context.description(), "'" + keyName + "' is not a list and doesn't support " + operator + "
(only = and clear)", lineNumber, line);
continue;
}
if (operator.equals("=") && type.isList()) {
reporter.report(context.description(), "'" + keyName + "' is a list and cannot be assigned to (use +=, -=
and clear instead)", lineNumber, line);
continue;
}
Object value = null;
if (stringValue != null) try {
value = type.getParser().parse(stringValue);
```

```
} catch (Exception e) {
reporter.report(context.description(), "Error while parsing the value for '" + keyName + "' value '" +
stringValue + "' (should be " + type.getParser().exampleValue() + ")", lineNumber, line);
continue;
}
if (operator.equals("clear")) {
collector.clear(key, context, lineNumber);
} else if (operator.equals("=")) {
collector.set(key, value, context, lineNumber);
} else if (operator.equals("+=")) {
collector.add(key, value, context, lineNumber);
} else {
collector.remove(key, value, context, lineNumber);
}
}
}
private CharSequence contents(ConfigurationFile context) {
try {
return context.contents();
} catch (IOException e) {
reporter.report(context.description(), "Exception while reading file: " + e.getMessage(), 0, null);
}
return null;
}
public interface Collector {
void addImport(ConfigurationFile importFile, ConfigurationFile context, int lineNumber);
void clear(ConfigurationKey<?> key, ConfigurationFile context, int lineNumber);
void set(ConfigurationKey<?> key, Object value, ConfigurationFile context, int lineNumber);
void add(ConfigurationKey<?> key, Object value, ConfigurationFile context, int lineNumber);
void remove(ConfigurationKey<?> key, Object value, ConfigurationFile context, int lineNumber);
}
}
```

lombok/core/configuration/ConfigurationProblemReporter.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import lombok.eclipse.handlers.EclipseHandlerUtil;
public interface ConfigurationProblemReporter {
void report(String sourceDescription, String problem, int lineNumber, CharSequence line);
ConfigurationProblemReporter CONSOLE = new ConfigurationProblemReporter() {
@Override public void report(String sourceDescription, String problem, int lineNumber, CharSequence
line) {
try {
// The console (System.err) is non-existent in eclipse environments, so we should try to
// log into at least the error log. This isn't really the appropriate place (should go in the
// relevant file instead, most people never see anything in the error log either!), but at least
// there is a way to see it, vs. System.err, which is completely invisible.
EclipseHandlerUtil.warning(String.format("%s (%s:%d)", problem, sourceDescription, lineNumber),
null);
} catch (Throwable ignore) {}
System.err.printf("%s (%s:%d)\n", problem, sourceDescription, lineNumber);
}
};
}
```

lombok/core/configuration/ConfigurationResolver.java

```
/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
```

```
*/
package lombok.core.configuration;
public interface ConfigurationResolver {
<T> T resolve(ConfigurationKey<T> key);
}
```

lombok/core/configuration/ConfigurationResolverFactory.java

```
/*
* Copyright (C) 2014-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.net.URI;
public interface ConfigurationResolverFactory {
ConfigurationResolver createResolver(URI sourceLocation);
}
```

lombok/core/configuration/ConfigurationSource.java

```
/*
* Copyright (C) 2014-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.util.List;
public interface ConfigurationSource {
Result resolve(ConfigurationKey<?> key);
List<ConfigurationFile> imports();
public static final class Result {
private final Object value;
private final boolean authoritative;
public Result(Object value, boolean authoritative) {
this.value = value;
this.authoritative = authoritative;
}
public Object getValue() {
return value;
}
public boolean isAuthoritative() {
return authoritative;
}
@Override public String toString() {
return String.valueOf(value) + (authoritative ? " (set)" : " (delta)");
}
}
public static final class ListModification {
private final Object value;
private final boolean added;
public ListModification(Object value, boolean added) {
this.value = value;
this.added = added;
}
public Object getValue() {
return value;
}
public boolean isAdded() {
return added;
}
}
}
```

lombok/core/configuration/ConfigurationValueParser.java

/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
interface ConfigurationValueParser {
Object parse(String value);
String description();
String exampleValue();
}

lombok/core/configuration/ConfigurationValueType.java

/*
* Copyright (C) 2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,

```
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
/**
* If a type used in {@link ConfigurationKey} type argument implements this interface,
* it is expected to provide the following three static methods:
* <ul>
* <li><code>public static SELF valueOf(String value)</code>
* <li><code>public static String description()</code>
* <li><code>public static String exampleValue()</code>
* </ul>
* None of them should throw checked exceptions.
* Based on these methods, an instance of {@link ConfigurationValueParser} is created
* and used by the configuration system.
*/
public interface ConfigurationValueType {
}
```

lombok/core/configuration/ExampleValueString.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* If a configuration key has an enum type, then the 'example values' string is built up by just joining all
enum keys together with a bar separator, but you
* can add this annotation to the enum type to override this string.
```

```
*/
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface ExampleValueString {
String value();
}
```

lombok/core/configuration/FileSystemSourceCache.java

```
/*
* Copyright (C) 2014-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.io.File;
import java.net.URI;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import java.util.concurrent.TimeUnit;
import lombok.core.debug.ProblemReporter;
public class FileSystemSourceCache {
private static final long FULL_CACHE_CLEAR_INTERVAL = TimeUnit.MINUTES.toMillis(30);
private static final long RECHECK_FILESYSTEM = TimeUnit.SECONDS.toMillis(2);
private static final long NEVER_CHECKED = -1;
static final long MISSING = -88; // Magic value; any lombok.config with this exact epochmillis last modified will never be read, so, let's ensure nobody accidentally has one with that exact last modified stamp.
private final ConcurrentMap<ConfigurationFile, Content> fileCache = new ConcurrentHashMap<ConfigurationFile, Content>(); // caches files to the content object that tracks content.
private final ConcurrentMap<URI, ConfigurationFile> uriCache = new ConcurrentHashMap<URI, ConfigurationFile>(); // caches URIs of java source files to the dir that contains it.
```

```
private volatile long lastCacheClear = System.currentTimeMillis();
private void cacheClear() {
// We never clear the caches, generally because it'd be weird if a compile run would continually create an endless stream of new java files.
// Still, eventually that's going to cause a bit of a memory leak, so lets just completely clear them out every many minutes.
long now = System.currentTimeMillis();
long delta = now - lastCacheClear;
if (delta > FULL_CACHE_CLEAR_INTERVAL) {
lastCacheClear = now;
fileCache.clear();
uriCache.clear();
}
}
public ConfigurationFileToSource fileToSource(final ConfigurationParser parser) {
return new ConfigurationFileToSource() {
@Override public ConfigurationSource parsed(ConfigurationFile fileLocation) {
return parseIfNeccesary(fileLocation, parser);
}
};
}
public ConfigurationFile forUri(URI javaFile) {
if (javaFile == null) return null;
cacheClear();
ConfigurationFile result = uriCache.get(javaFile);
if (result == null) {
URI uri = javaFile.normalize();
if (!uri.isAbsolute()) uri = URI.create("file:" + uri.toString());
try {
File file = new File(uri);
if (!file.exists()) throw new IllegalArgumentException("File does not exist: " + uri);
File directory = file.isDirectory() ? file : file.getParentFile();
if (directory != null) result = ConfigurationFile.forDirectory(directory);
uriCache.put(javaFile, result);
} catch (IllegalArgumentException e) {
// This means that the file as passed is not actually a file at all, and some exotic path system is involved.
// examples: sourcecontrol://jazz stuff, or an actual relative path (uri.isAbsolute() is completely different, that checks presence of schema!),
// or it's eclipse trying to parse a snippet, which has "/Foo.java" as uri.
// At some point it might be worth investigating abstracting away the notion of "I can read lombok.config if present in
// current context, and I can give you may parent context", using ResourcesPlugin.getWorkspace().getRoot().findFilesForLocationURI(javaFile) as basis.
// For now, we just carry on as if there is no lombok.config. (intentional fallthrough)
} catch (Exception e) {
// Especially for eclipse's sake, exceptions here make eclipse borderline unusable, so let's play nice.
ProblemReporter.error("Can't find absolute path of file being compiled: " + javaFile, e);
}
}
return result;
}
private ConfigurationSource parseIfNeccesary(ConfigurationFile file, ConfigurationParser parser) {
long now = System.currentTimeMillis();
```

```
Content content = ensureContent(file);
synchronized (content) {
if (content.lastChecked != NEVER_CHECKED && now - content.lastChecked <
RECHECK_FILESYSTEM) {
return content.source;
}
content.lastChecked = now;
long previouslyModified = content.lastModified;
content.lastModified = file.getLastModifiedOrMissing();
if (content.lastModified != previouslyModified) content.source = content.lastModified == MISSING ? null
: SingleConfigurationSource.parse(file, parser);
return content.source;
}
}
private Content ensureContent(ConfigurationFile context) {
Content content = fileCache.get(context);
if (content != null) {
return content;
}
fileCache.putIfAbsent(context, Content.empty());
return fileCache.get(context);
}
private static class Content {
ConfigurationSource source;
long lastModified;
long lastChecked;
private Content(ConfigurationSource source, long lastModified, long lastChecked) {
this.source = source;
this.lastModified = lastModified;
this.lastChecked = lastChecked;
}
static Content empty() {
return new Content(null, MISSING, NEVER_CHECKED);
}
}
}
```

lombok/core/configuration/FlagUsageType.java

```
/*
* Copyright (C) 2014-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
```

```
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
/** Used for lombok configuration to flag usages of certain lombok features. */
public enum FlagUsageType {
WARNING, ERROR, ALLOW;
}
```

lombok/core/configuration/IdentifierName.java

```
/*
* Copyright (C) 2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import lombok.core.JavaIdentifiers;
public final class IdentifierName implements ConfigurationValueType {
private final String name;
private IdentifierName(String name) {
this.name = name;
}
public static IdentifierName valueOf(String name) {
if (name == null || name.trim().isEmpty()) return null;
String trimmedName = name.trim();
if (!JavaIdentifiers.isValidJavaIdentifier(trimmedName)) throw new IllegalArgumentException("Invalid
identifier " + trimmedName);
return new IdentifierName(trimmedName);
```

```
}
public static String description() {
return "identifier-name";
}
public static String exampleValue() {
return "<javaIdentifier>";
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof IdentifierName)) return false;
return name.equals(((IdentifierName) obj).name);
}
@Override public int hashCode() {
return name.hashCode();
}
@Override public String toString() {
return name;
}
public String getName() {
return name;
}
public char[] getCharArray() {
return name.toCharArray();
}
}
```

lombok/core/configuration/LogDeclaration.java

```
/*
* Copyright (C) 2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.util.ArrayList;
import java.util.List;
```

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public final class LogDeclaration implements ConfigurationValueType {
private static final Pattern PARAMETERS_PATTERN = Pattern.compile("(?:\\(([A-Z,]*)\\))");
private static final Pattern DECLARATION_PATTERN = Pattern.compile("^(?:([^ ]+) )?([^(]+)\\.([^(]+)(" +
PARAMETERS_PATTERN.pattern() + "+)$");
public enum LogFactoryParameter {
TYPE, NAME, TOPIC, NULL;
}
private final TypeName loggerType;
private final TypeName loggerFactoryType;
private final IdentifierName loggerFactoryMethod;
private final List<LogFactoryParameter> parametersWithoutTopic;
private final List<LogFactoryParameter> parametersWithTopic;
private LogDeclaration(TypeName loggerType, TypeName loggerFactoryType, IdentifierName
loggerFactoryMethod, List<LogFactoryParameter> parametersWithoutTopic,
List<LogFactoryParameter> parametersWithTopic) {
this.loggerType = loggerType;
this.loggerFactoryType = loggerFactoryType;
this.loggerFactoryMethod = loggerFactoryMethod;
this.parametersWithoutTopic = parametersWithoutTopic;
this.parametersWithTopic = parametersWithTopic;
}
public static LogDeclaration valueOf(String declaration) {
if (declaration == null) return null;
Matcher matcher = DECLARATION_PATTERN.matcher(declaration);
if (!matcher.matches()) throw new IllegalArgumentException("The declaration must follow the pattern:
[LoggerType ]LoggerFactoryType.loggerFactoryMethod(loggerFactoryMethodParams)[(loggerFactory
MethodParams)]");
TypeName loggerFactoryType = TypeName.valueOf(matcher.group(2));
TypeName loggerType = TypeName.valueOf(matcher.group(1));
if (loggerType == null) loggerType = loggerFactoryType;
IdentifierName loggerFactoryMethod = IdentifierName.valueOf(matcher.group(3));
List<List<LogFactoryParameter>> allParameters = parseParameters(matcher.group(4));
List<LogFactoryParameter> parametersWithoutTopic = null;
List<LogFactoryParameter> parametersWithTopic = null;
for (List<LogFactoryParameter> parameters: allParameters) {
if (parameters.contains(LogFactoryParameter.TOPIC)) {
if (parametersWithTopic != null) throw new IllegalArgumentException("There is more than one
parameter definition that includes TOPIC: " + parametersWithTopic + " and " + parameters);
parametersWithTopic = parameters;
} else {
if (parametersWithoutTopic != null) throw new IllegalArgumentException("There is more than one
parmaeter definition that does not include TOPIC: " + parametersWithoutTopic + " and " + parameters);
parametersWithoutTopic = parameters;
}
}
// sanity check (the pattern should disallow this situation
if (parametersWithoutTopic == null && parametersWithTopic == null) throw new
IllegalArgumentException("No logger factory method parameters specified.");
return new LogDeclaration(loggerType, loggerFactoryType, loggerFactoryMethod,
parametersWithoutTopic, parametersWithTopic);
}
```

```
private static List<List<LogFactoryParameter>> parseParameters(String parametersDefinitions) {
List<List<LogFactoryParameter>> allParameters = new ArrayList<List<LogFactoryParameter>>();
Matcher matcher = PARAMETERS_PATTERN.matcher(parametersDefinitions);
while (matcher.find()) {
String parametersDefinition = matcher.group(1);
List<LogFactoryParameter> parameters = new ArrayList<LogFactoryParameter>();
if (!parametersDefinition.isEmpty()) {
for (String parameter : parametersDefinition.split(",")) {
parameters.add(LogFactoryParameter.valueOf(parameter));
}
}
allParameters.add(parameters);
}
return allParameters;
}
public static String description() {
return "custom-log-declaration";
}
public static String exampleValue() {
return "my.cool.Logger my.cool.LoggerFactory.createLogger()(TOPIC,TYPE)";
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof LogDeclaration)) return false;
return loggerType.equals(((LogDeclaration) obj).loggerType)
&& loggerFactoryType.equals(((LogDeclaration) obj).loggerFactoryType)
&& loggerFactoryMethod.equals(((LogDeclaration) obj).loggerFactoryMethod)
&& parametersWithoutTopic == ((LogDeclaration) obj).parametersWithoutTopic ||
parametersWithoutTopic.equals(((LogDeclaration) obj).parametersWithoutTopic)
&& parametersWithTopic == ((LogDeclaration) obj).parametersWithTopic ||
parametersWithTopic.equals(((LogDeclaration) obj).parametersWithTopic);
}
@Override public int hashCode() {
final int prime = 31;
int result = 1;
result = prime * result + loggerType.hashCode();
result = prime * result + loggerFactoryType.hashCode();
result = prime * result + loggerFactoryMethod.hashCode();
result = prime * result + ((parametersWithTopic == null) ? 0 : parametersWithTopic.hashCode());
result = prime * result + ((parametersWithoutTopic == null) ? 0 : parametersWithoutTopic.hashCode());
return result;
}
@Override public String toString() {
StringBuilder sb = new StringBuilder();
sb.append(loggerType);
sb.append(" ");
sb.append(loggerFactoryType);
sb.append(".");
sb.append(loggerFactoryMethod);
appendParams(sb, parametersWithoutTopic);
appendParams(sb, parametersWithTopic);
return sb.toString();
}
private static void appendParams(StringBuilder sb, List<LogFactoryParameter> params) {
```

```
if (params != null) {
sb.append("(");
boolean first = true;
for (LogFactoryParameter param : params) {
if (!first) {
sb.append(",");
}
first = false;
sb.append(param);
}
sb.append(")");
}
}
public TypeName getLoggerType() {
return loggerType;
}
public TypeName getLoggerFactoryType() {
return loggerFactoryType;
}
public IdentifierName getLoggerFactoryMethod() {
return loggerFactoryMethod;
}
public List<LogFactoryParameter> getParametersWithoutTopic() {
return parametersWithoutTopic;
}
public List<LogFactoryParameter> getParametersWithTopic() {
return parametersWithTopic;
}
}
```

lombok/core/configuration/NullAnnotationLibrary.java


```
/*
* Copyright (C) 2020-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.lang.reflect.Field;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
public final class NullAnnotationLibrary implements ConfigurationValueType {
private final String key;
private final String nonNullAnnotation;
private final String nullableAnnotation;
private final boolean typeUse;
private NullAnnotationLibrary(String key, String nonNullAnnotation, String nullableAnnotation, boolean
typeUse) {
this.key = key;
this.nonNullAnnotation = nonNullAnnotation;
this.nullableAnnotation = nullableAnnotation;
this.typeUse = typeUse;
}
/**
* Returns the fully qualified annotation name to apply to non-null elements. If {@code null} is returned,
apply no annotation.
*/
public String getNonNullAnnotation() {
return nonNullAnnotation;
}
/**
* Returns the fully qualified annotation name to apply to nullable elements. If {@code null} is returned,
apply no annotation.
*/
public String getNullableAnnotation() {
return nullableAnnotation;
}
/**
* If {@code true}, the annotation can only be used in TYPE_USE form, otherwise, prefer to annotate
the parameter, not the type of the parameter (or the method, not the return type, etc).
*/
public boolean isTypeUse() {
return typeUse;
}
public static final NullAnnotationLibrary NONE = new NullAnnotationLibrary("none", null, null, false);
public static final NullAnnotationLibrary JAVAX = new NullAnnotationLibrary("javax",
"javax.annotation.Nonnull", "javax.annotation.Nullable", false);
public static final NullAnnotationLibrary JAKARTA = new NullAnnotationLibrary("jakarta",
"jakarta.annotation.Nonnull", "jakarta.annotation.Nullable", false);
public static final NullAnnotationLibrary ECLIPSE = new NullAnnotationLibrary("eclipse",
"org.eclipse.jdt.annotation.NonNull", "org.eclipse.jdt.annotation.Nullable", true);
public static final NullAnnotationLibrary JETBRAINS = new NullAnnotationLibrary("jetbrains",
"org.jetbrains.annotations.NotNull", "org.jetbrains.annotations.Nullable", false);
```

```
public static final NullAnnotationLibrary NETBEANS = new NullAnnotationLibrary("netbeans",
"org.netbeans.api.annotations.common.NonNull", "org.netbeans.api.annotations.common.NullAllowed",
false);
public static final NullAnnotationLibrary ANDROIDX = new NullAnnotationLibrary("androidx",
"androidx.annotation.NonNull", "androidx.annotation.Nullable", false);
public static final NullAnnotationLibrary ANDROID_SUPPORT = new
NullAnnotationLibrary("android.support", "android.support.annotation.NonNull",
"android.support.annotation.Nullable", false);
public static final NullAnnotationLibrary CHECKERFRAMEWORK = new
NullAnnotationLibrary("checkerframework", "org.checkerframework.checker.nullness.qual.NonNull",
"org.checkerframework.checker.nullness.qual.Nullable", true);
public static final NullAnnotationLibrary FINDBUGS = new NullAnnotationLibrary("findbugs",
"edu.umd.cs.findbugs.annotations.NonNull", "edu.umd.cs.findbugs.annotations.Nullable", false);
public static final NullAnnotationLibrary SPRING = new NullAnnotationLibrary("spring",
"org.springframework.lang.NonNull", "org.springframework.lang.Nullable", false);
public static final NullAnnotationLibrary JML = new NullAnnotationLibrary("jml",
"org.jmlspecs.annotation.NonNull", "org.jmlspecs.annotation.Nullable", false);
private static final List<NullAnnotationLibrary> ALL_AVAILABLE;
private static final String EXAMPLE_VALUE;
static {
ArrayList<NullAnnotationLibrary> out = new ArrayList<NullAnnotationLibrary>();
StringBuilder example = new StringBuilder();
for (Field f : NullAnnotationLibrary.class.getDeclaredFields()) {
if (f.getType() != NullAnnotationLibrary.class || !Modifier.isStatic(f.getModifiers()) ||
!Modifier.isPublic(f.getModifiers())) continue;
try {
NullAnnotationLibrary nal = (NullAnnotationLibrary) f.get(null);
out.add(nal);
example.append(nal.key).append(" | ");
} catch (IllegalAccessException e) {
continue;
}
}
out.trimToSize();
example.append("CUSTOM:com.foo.my.nonnull.annotation:com.foo.my.nullable.annotation");
ALL_AVAILABLE = Collections.unmodifiableList(out);
EXAMPLE_VALUE = example.toString();
}
public static NullAnnotationLibrary custom(String nonNullAnnotation, String nullableAnnotation,
boolean typeUse) {
if (nonNullAnnotation == null && nullableAnnotation == null) return NONE;
String typeUseStr = typeUse ? "TYPE_USE:" : "";
if (nullableAnnotation == null) return new NullAnnotationLibrary("custom:" + typeUseStr +
nonNullAnnotation, nonNullAnnotation, null, typeUse);
if (nonNullAnnotation == null) return new NullAnnotationLibrary("custom::" + typeUseStr +
nullableAnnotation, null, nullableAnnotation, typeUse);
return new NullAnnotationLibrary("custom:" + typeUseStr + nonNullAnnotation + ":" +
nullableAnnotation, nonNullAnnotation, nullableAnnotation, typeUse);
}
public static String description() {
return "nullity-annotation-library";
}
public static String exampleValue() {
```

```
return EXAMPLE_VALUE;
}
public static NullAnnotationLibrary valueOf(String in) {
String ci = in == null ? "" : in.toLowerCase();
if (ci.length() == 0) return NONE;
for (NullAnnotationLibrary nal : ALL_AVAILABLE) if (nal.key.equals(ci)) return nal;
if (!ci.startsWith("custom:")) {
StringBuilder out = new StringBuilder("Invalid null annotation library. Valid options: ");
for (NullAnnotationLibrary nal : ALL_AVAILABLE) out.append(nal.key).append(", ");
out.setLength(out.length() - 2);
out.append(" or CUSTOM:[TYPE_USE:]nonnull.annotation.type:nullable.annotation.type");
throw new IllegalArgumentException(out.toString());
}
boolean typeUse = ci.startsWith("custom:type_use:");
int start = typeUse ? 16 : 7;
int split = ci.indexOf(':', start);
if (split == -1) {
String nonNullAnnotation = in.substring(start);
return custom(verifyTypeName(nonNullAnnotation), null, typeUse);
}
String nonNullAnnotation = in.substring(start, split);
String nullableAnnotation = in.substring(split + 1);
return custom(verifyTypeName(nonNullAnnotation), verifyTypeName(nullableAnnotation), typeUse);
}
private static final String MSG = "Not an identifier (provide a fully qualified type for custom: nullity
annotations): ";
private static String verifyTypeName(String fqn) {
boolean atStart = true;
for (int i = 0; i < fqn.length(); i++) {
char c = fqn.charAt(i);
if (Character.isJavaIdentifierStart(c)) {
atStart = false;
continue;
}
if (atStart) throw new IllegalArgumentException(MSG + fqn);
if (c == '.') {
atStart = true;
continue;
}
if (Character.isJavaIdentifierPart(c)) continue;
throw new IllegalArgumentException(MSG + fqn);
}
if (atStart) throw new IllegalArgumentException(MSG + fqn);
return fqn;
}
}
```

lombok/core/configuration/NullCheckExceptionType.java

```
/*
* Copyright (C) 2014-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import lombok.core.LombokImmutableList;
@ExampleValueString("[NullPointerException | IllegalArgumentException | Assertion | JDK | Guava]")
public enum NullCheckExceptionType {
ILLEGAL_ARGUMENT_EXCEPTION {
@Override public String getExceptionType() {
return "java.lang.IllegalArgumentException";
}
@Override public LombokImmutableList<String> getMethod() {
return null;
}
},
NULL_POINTER_EXCEPTION {
@Override public String getExceptionType() {
return "java.lang.NullPointerException";
}
@Override public LombokImmutableList<String> getMethod() {
return null;
}
},
ASSERTION {
@Override public String getExceptionType() {
return null;
}
@Override public LombokImmutableList<String> getMethod() {
return null;
}
},
JDK {
@Override public String getExceptionType() {
return null;
}
```

```
@Override public LombokImmutableList<String> getMethod() {
return METHOD_JDK;
}
},
GUAVA {
@Override public String getExceptionType() {
return null;
}
@Override public LombokImmutableList<String> getMethod() {
return METHOD_GUAVA;
}
};
private static final LombokImmutableList<String> METHOD_JDK = LombokImmutableList.of("java",
"util", "Objects", "requireNonNull");
private static final LombokImmutableList<String> METHOD_GUAVA = LombokImmutableList.of("com",
"google", "common", "base", "Preconditions", "checkNotNull");
public String toExceptionMessage(String fieldName, String customMessage) {
if (customMessage == null) return fieldName + " is marked non-null but is null";
return customMessage.replace("%s", fieldName);
}
public abstract String getExceptionType();
public abstract LombokImmutableList<String> getMethod();
}
```

lombok/core/configuration/SingleConfigurationSource.java

```
/*
* Copyright (C) 2014-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import java.util.ArrayList;
```

```
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import lombok.core.configuration.ConfigurationParser.Collector;
public final class SingleConfigurationSource implements ConfigurationSource {
private final Map<ConfigurationKey<?>, Result> values;
private final List<ConfigurationFile> imports;
public static ConfigurationSource parse(ConfigurationFile context, ConfigurationParser parser) {
final Map<ConfigurationKey<?>, Result> values = new HashMap<ConfigurationKey<?>, Result>();
final List<ConfigurationFile> imports = new ArrayList<ConfigurationFile>();
Collector collector = new Collector() {
@Override public void addImport(ConfigurationFile importFile, ConfigurationFile context, int
lineNumber) {
imports.add(importFile);
}
@Override public void clear(ConfigurationKey<?> key, ConfigurationFile context, int lineNumber) {
values.put(key, new Result(null, true));
}
@Override public void set(ConfigurationKey<?> key, Object value, ConfigurationFile context, int
lineNumber) {
values.put(key, new Result(value, true));
}
@Override public void add(ConfigurationKey<?> key, Object value, ConfigurationFile context, int
lineNumber) {
modifyList(key, value, true);
}
@Override public void remove(ConfigurationKey<?> key, Object value, ConfigurationFile context, int
lineNumber) {
modifyList(key, value, false);
}
@SuppressWarnings("unchecked")
private void modifyList(ConfigurationKey<?> key, Object value, boolean add) {
Result result = values.get(key);
List<ListModification> list;
if (result == null || result.getValue() == null) {
list = new ArrayList<ConfigurationSource.ListModification>();
values.put(key, new Result(list, result != null));
} else {
list = (List<ListModification>) result.getValue();
}
list.add(new ListModification(value, add));
}
};
parser.parse(context, collector);
return new SingleConfigurationSource(values, imports);
}
private SingleConfigurationSource(Map<ConfigurationKey<?>, Result> values, List<ConfigurationFile>
imports) {
this.values = new HashMap<ConfigurationKey<?>, Result>();
for (Entry<ConfigurationKey<?>, Result> entry : values.entrySet()) {
Result result = entry.getValue();
```

```
if (result.getValue() instanceof List<?>) {
this.values.put(entry.getKey(), new Result(Collections.unmodifiableList((List<?>) result.getValue()),
result.isAuthoritative()));
} else {
this.values.put(entry.getKey(), result);
}
}
this.imports = Collections.unmodifiableList(imports);
}
@Override
public Result resolve(ConfigurationKey<?> key) {
return values.get(key);
}
@Override
public List<ConfigurationFile> imports() {
return imports;
}
}
```

lombok/core/configuration/TypeName.java

```
/*
* Copyright (C) 2013-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.configuration;
import lombok.core.JavaIdentifiers;
public final class TypeName implements ConfigurationValueType {
private final String name;
private TypeName(String name) {
this.name = name;
}
public static TypeName valueOf(String name) {
if (name == null || name.trim().isEmpty()) return null;
```

```
String trimmedName = name.trim();
for (String identifier : trimmedName.split("\\.")) {
if (!JavaIdentifiers.isValidJavaIdentifier(identifier)) throw new IllegalArgumentException("Invalid type
name " + trimmedName + " (part " + identifier + ")");
}
return new TypeName(trimmedName);
}
public static String description() {
return "type-name";
}
public static String exampleValue() {
return "<fully.qualified.Type>";
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof TypeName)) return false;
return name.equals(((TypeName) obj).name);
}
@Override public int hashCode() {
return name.hashCode();
}
@Override public String toString() {
return name;
}
public String getName() {
return name;
}
public char[] getCharArray() {
return name.toCharArray();
}
}
```

lombok/core/debug/AssertionLogger.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
```

```
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.debug;
import java.io.FileOutputStream;
import java.io.OutputStream;
import java.io.PrintWriter;
import java.io.StringWriter;
import java.util.Date;
import java.util.Random;
import java.util.concurrent.atomic.AtomicBoolean;
import lombok.core.Version;
/**
* We make a number of assumptions in lombok code, and if these assumptions fail, we try to fall back to a 'least bad' scenario. However, we would prefer to
* just know about these cases, without confronting our users with error messages. The 'fix' is to log such assertion failures to this logger, which promptly
* ignores them, _unless_ you specifically enable logging them to a file. If you'd like to help out or want to assist in debugging, turn this on.
*/
public class AssertionLogger {
private static final String LOG_PATH;
static {
String log = System.getProperty("lombok.assertion.log", null);
if (log != null) {
LOG_PATH = log.isEmpty() ? null : log;
} else {
try {
log = System.getenv("LOMBOK_ASSERTION_LOG");
} catch (Exception e) {
log = null;
}
LOG_PATH = (log == null || log.isEmpty()) ? null : log;
}
}
private static final AtomicBoolean loggedIntro = new AtomicBoolean(false);
private static final String PROCESS_ID = generateProcessId();
private static final String ID_CHARS =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
private static String generateProcessId() {
char[] ID = new char[4];
Random r = new Random();
for (int i = 0; i < ID.length; i++) ID[i] = ID_CHARS.charAt(r.nextInt(ID_CHARS.length()));
return new String(ID);
}
private static synchronized void logToFile(String msg) {
if (msg == null) return;
try {
OutputStream out = new FileOutputStream(LOG_PATH, true);
out.write(msg.getBytes("UTF-8"));
out.close();
} catch (Exception e) {
throw new RuntimeException("assertion logging can't write to log file", e);
```

```
}
}
private static void logIntro() {
if (loggedIntro.getAndSet(true)) return;
String version;
try {
version = Version.getFullVersion();
} catch (Exception e) {
version = Version.getVersion();
}
logToFile(String.format("{%s} [%s -- START %s]\n", PROCESS_ID, new Date(), version));
}
public static <T extends Throwable> T assertLog(String message, T throwable) {
if (LOG_PATH == null) return throwable;
logIntro();
if (message == null) message = "(No message)";
String stackMsg = "";
if (throwable != null) {
StringWriter sw = new StringWriter();
PrintWriter pw = new PrintWriter(sw);
throwable.printStackTrace(pw);
pw.close();
stackMsg = "\n " + sw.toString().replace("\r", "").replace("\n", "\n ").trim();
}
logToFile(String.format("{%s} [%ty%<tm%<tdT%<tH%<tM%<tS.%<tL] %s%s\n", PROCESS_ID, new
Date(), message, stackMsg));
return throwable;
}
public static void assertLog(String message) {
if (LOG_PATH == null) return;
assertLog(message, null);
}
}
```

lombok/core/debug/DebugSnapshot.java


```
/*
* Copyright (C) 2012-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.debug;
import java.lang.ref.WeakReference;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
public class DebugSnapshot implements Comparable<DebugSnapshot> {
private static AtomicLong counter = new AtomicLong();
private final long when, id = counter.getAndIncrement();
private final long bits;
private final List<StackTraceElement> trace;
private final String threadName;
private final String message;
private final Object[] params;
private final WeakReference<CompilationUnitDeclaration> owner;
public DebugSnapshot(CompilationUnitDeclaration owner, int stackHiding, String message, Object...
params) {
this.when = System.currentTimeMillis();
this.bits = owner.bits;
if (stackHiding < 0) {
this.trace = null;
} else {
StackTraceElement[] stackTrace = new Throwable().getStackTrace();
this.trace = new ArrayList<StackTraceElement>(Math.max(0, stackTrace.length - stackHiding - 1));
for (int i = 1 + stackHiding; i < stackTrace.length; i++) trace.add(stackTrace[i]);
}
this.threadName = Thread.currentThread().getName();
this.message = message;
this.params = params == null ? new Object[0] : params;
this.owner = new WeakReference<CompilationUnitDeclaration>(owner);
}
private String ownerName() {
CompilationUnitDeclaration node = owner.get();
if (node == null) return "--GCed--";
char[] tn = node.getMainTypeName();
char[] fs = node.getFileName();
if (tn == null || tn.length == 0) {
return (fs == null || fs.length == 0) ? "--UNKNOWN--" : new String(fs);
}
return new String(tn);
}
public String shortToString() {
StringBuilder out = new StringBuilder();
out.append(String.format("WHEN: %14d THREAD: %s AST: %s HAMB: %b -- ", when, threadName,
ownerName(), 0 != (bits & ASTNode.HasAllMethodBodies)));
if (message != null) out.append(" ").append(String.format(message, params));
```

```
return out.toString();
}
@Override public String toString() {
StringBuilder out = new StringBuilder();
out.append(shortToString()).append("\n");
if (trace == null) {
out.append(" Stack Omitted");
} else {
for (StackTraceElement elem : trace) {
out.append(" ").append(elem.toString()).append("\n");
}
}
return out.toString();
}
@Override public int compareTo(DebugSnapshot o) {
return Long.valueOf(id).compareTo(o.id);
}
}
```

lombok/core/debug/DebugSnapshotStore.java


```
/*
* Copyright (C) 2012-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.core.debug;
import java.io.File;
import java.io.FileOutputStream;
import java.io.OutputStream;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.WeakHashMap;
```

```
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
public class DebugSnapshotStore {
public static final DebugSnapshotStore INSTANCE = new DebugSnapshotStore();
public static final boolean GLOBAL_DSS_DISABLE_SWITCH = true;
// All access should synchronize on the map
private final Map<CompilationUnitDeclaration, List<DebugSnapshot>> map =
new WeakHashMap<CompilationUnitDeclaration, List<DebugSnapshot>>();
public void snapshot(CompilationUnitDeclaration owner, String message, Object... params) {
if (GLOBAL_DSS_DISABLE_SWITCH) return;
DebugSnapshot snapshot = new DebugSnapshot(owner, 1, message, params);
List<DebugSnapshot> list;
synchronized (map) {
list = map.get(owner);
if (list == null) {
list = new ArrayList<DebugSnapshot>();
map.put(owner, list);
list.add(snapshot);
} else if (!list.isEmpty()) {
list.add(snapshot);
} else {
// An empty list is an indicator that we no longer care about that particular CUD.
}
}
}
public void log(CompilationUnitDeclaration owner, String message, Object... params) {
if (GLOBAL_DSS_DISABLE_SWITCH) return;
DebugSnapshot snapshot = new DebugSnapshot(owner, -1, message, params);
List<DebugSnapshot> list;
synchronized (map) {
list = map.get(owner);
if (list == null) {
list = new ArrayList<DebugSnapshot>();
map.put(owner, list);
list.add(snapshot);
} else if (!list.isEmpty()) {
list.add(snapshot);
} else {
// An empty list is an indicator that we no longer care about that particular CUD.
}
}
}
public String print(CompilationUnitDeclaration owner, String message, Object... params) {
if (GLOBAL_DSS_DISABLE_SWITCH) return null;
List<DebugSnapshot> list;
synchronized (map) {
snapshot(owner, message == null ? "Printing" : message, params);
list = new ArrayList<DebugSnapshot>();
list.addAll(map.get(owner));
if (list.isEmpty()) return null; // An empty list is an indicator that we no longer care about that particular
CUD.
map.get(owner).clear();
}
Collections.sort(list);
```

```
int idx = 1;
StringBuilder out = new StringBuilder();
out.append("--------------------------\n");
for (DebugSnapshot snapshot : list) {
out.append(String.format("%3d: %s\n", idx++, snapshot.shortToString()));
}
out.append("******\n");
idx = 1;
for (DebugSnapshot snapshot : list) {
out.append(String.format("%3d: %s", idx++, snapshot.toString()));
}
try {
File logFile = new File(System.getProperty("user.home", "."), String.format("lombokdss-%d.err",
System.currentTimeMillis()));
OutputStream stream = new FileOutputStream(logFile);
try {
stream.write(out.toString().getBytes("UTF-8"));
} finally {
stream.close();
}
return logFile.getAbsolutePath();
} catch (Exception e) {
System.err.println(out);
return "(can't write log file - emitted to system err)";
}
}
}
```

lombok/core/debug/HistogramTracker.java

```
package lombok.core.debug;
import java.io.PrintStream;
import java.util.Calendar;
import java.util.GregorianCalendar;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicStampedReference;
/**
* Create one of these and call .report() on it a lot to emit histogram data about the times taken by some
process.
* Currently the results are broken down into 10 buckets, from 0 millis to a quarter of a second, and a
report is emitted
* to the ProblemReporter every minute. If there are 0 entries for a given minute, it is not reported.
Reports aren't
* emitted until you call report().
*/
public class HistogramTracker {
private static final long[] RANGES = {
250001L, 500001L, 1000001L, 2000001L, 4000001L, 8000001L, 16000001L, 32000001L,
64000001L, 128000001L, 256000001L, 512000001L, 1024000001L, 2048000001L, 10000000001L};
private static final long REPORT_WINDOW = 1000 * 60;
private final String category;
```

```
private final AtomicStampedReference<long[]> bars = new AtomicStampedReference<long[]>(new
long[RANGES.length + 2], 0);
private final AtomicBoolean addedSysHook = new AtomicBoolean(false);
private final PrintStream out;
public HistogramTracker(String category) {
this.category = category;
this.out = null;
printInit();
}
public HistogramTracker(String category, PrintStream out) {
this.category = category;
this.out = out;
printInit();
}
private void printInit() {
if (category == null) {
if (out == null) ProblemReporter.info("Initialized histogram", null);
else out.println("Initialized histogram");
} else {
if (out == null) ProblemReporter.info(String.format("Initialized histogram tracker for '%s'", category),
null);
else out.printf("Initialized histogram tracker for '%s'%n", category);
}
}
public long start() {
return System.nanoTime();
}
public void end(long startToken) {
if (!addedSysHook.getAndSet(true)) Runtime.getRuntime().addShutdownHook(new Thread("Histogram
Printer") {
@Override public void run() {
int[] currentInterval = {0};
long[] b = bars.get(currentInterval);
printReport(currentInterval[0], b);
}
});
long end = System.nanoTime();
long now = System.currentTimeMillis();
long delta = end - startToken;
if (delta < 0L) delta = 0L;
int interval = (int) (now / REPORT_WINDOW);
int[] currentInterval = {0};
long[] bars = this.bars.get(currentInterval);
long[] newBars;
if (currentInterval[0] != interval) {
printReport(currentInterval[0], bars);
newBars = new long[RANGES.length + 2];
if (!this.bars.compareAndSet(bars, newBars, currentInterval[0], interval)) {
newBars = this.bars.get(currentInterval);
}
} else {
newBars = bars;
}
```

```
newBars[RANGES.length + 1] += delta;
for (int i = 0; i < RANGES.length; i++) {
if (delta < RANGES[i]) {
newBars[i]++;
return;
}
}
newBars[RANGES.length]++;
}
private void printReport(int interval, long[] bars) {
StringBuilder sb = new StringBuilder();
if (category != null) sb.append(category).append(" ");
sb.append("[");
GregorianCalendar gc = new GregorianCalendar();
gc.setTimeInMillis(interval * REPORT_WINDOW);
int hour = gc.get(Calendar.HOUR_OF_DAY);
int minute = gc.get(Calendar.MINUTE);
if (hour < 10) sb.append('0');
sb.append(hour).append(":");
if (minute < 10) sb.append('0');
sb.append(minute).append("] {");
long sum = bars[RANGES.length];
int count = 0;
int lastZeroPos = sb.length();
for (int i = 0; i < RANGES.length; i++) {
sum += bars[i];
sb.append(bars[i]);
if (bars[i] != 0) lastZeroPos = sb.length();
sb.append(" ");
count++;
if (count == 3) sb.append("-- ");
if (count == 9) sb.append("-- ");
}
if (sum == 0) return;
sb.setLength(lastZeroPos);
double millis = bars[RANGES.length + 1] / 1000000.0;
long over = bars[RANGES.length];
if (over > 0L) {
sb.append(" -- ").append(bars[RANGES.length]);
}
sb.append("} total calls: ").append(sum).append(" total time (millis): ").append((int)(millis + 0.5));
if (out == null) ProblemReporter.info(sb.toString(), null);
else out.println(sb.toString());
}
}
```

lombok/core/debug/ProblemReporter.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
```

```
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.debug;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import org.eclipse.core.runtime.ILog;
import org.eclipse.core.runtime.IStatus;
import org.eclipse.core.runtime.Platform;
import org.eclipse.core.runtime.Status;
import org.osgi.framework.Bundle;
public class ProblemReporter {
public static void info(String msg, Throwable ex) {
init();
try {
logger.info(msg, ex);
} catch (Throwable t) {
logger = new TerminalLogger();
logger.info(msg, ex);
}
}
public static void warning(String msg, Throwable ex) {
init();
try {
logger.warning(msg, ex);
} catch (Throwable t) {
logger = new TerminalLogger();
logger.warning(msg, ex);
}
}
public static void error(String msg, Throwable ex) {
init();
try {
logger.error(msg, ex);
} catch (Throwable t) {
logger = new TerminalLogger();
logger.error(msg, ex);
}
```

```
}
private static void init() {
if (logger != null) return;
try {
logger = new EclipseWorkspaceLogger();
} catch (Throwable t) {
logger = new TerminalLogger();
}
}
private static ErrorLogger logger;
private interface ErrorLogger {
void info(String message, Throwable ex);
void warning(String message, Throwable ex);
void error(String message, Throwable ex);
}
private static class TerminalLogger implements ErrorLogger {
@Override
public void info(String message, Throwable ex) {
System.err.println(message);
if (ex != null) ex.printStackTrace();
}
@Override
public void warning(String message, Throwable ex) {
System.err.println(message);
if (ex != null) ex.printStackTrace();
}
@Override
public void error(String message, Throwable ex) {
System.err.println(message);
if (ex != null) ex.printStackTrace();
}
}
private static class EclipseWorkspaceLogger implements ErrorLogger {
private static final String DEFAULT_BUNDLE_NAME = "org.eclipse.jdt.core";
private static final Bundle bundle;
private static final int MAX_LOG = 200;
private static final long SQUELCH_TIMEOUT = TimeUnit.HOURS.toMillis(1);
private static final AtomicInteger counter = new AtomicInteger();
private static volatile long squelchTimeout = 0L;
static {
bundle = Platform.getBundle(DEFAULT_BUNDLE_NAME);
if (bundle == null) throw new NoClassDefFoundError(); // this means some weird RCP build or possible
ecj. At any rate, we can't report this way so act as if this isn't an eclipse.
}
@Override
public void info(String message, Throwable error) {
msg(IStatus.INFO, message, error);
}
@Override
public void warning(String message, Throwable error) {
msg(IStatus.WARNING, message, error);
}
@Override
```

```
public void error(String message, Throwable error) {
msg(IStatus.ERROR, message, error);
}
private void msg(int msgType, String message, Throwable error) {
int ct = squelchTimeout != 0L ? 0 : counter.incrementAndGet();
boolean printSquelchWarning = false;
if (squelchTimeout != 0L) {
long now = System.currentTimeMillis();
if (squelchTimeout > now) return;
squelchTimeout = now + SQUELCH_TIMEOUT;
printSquelchWarning = true;
} else if (ct >= MAX_LOG) {
squelchTimeout = System.currentTimeMillis() + SQUELCH_TIMEOUT;
printSquelchWarning = true;
}
ILog log = Platform.getLog(bundle);
log.log(new Status(msgType, DEFAULT_BUNDLE_NAME, message, error));
if (printSquelchWarning) {
log.log(new Status(IStatus.WARNING, DEFAULT_BUNDLE_NAME, "Lombok has logged too many
messages; to avoid memory issues, further lombok logs will be squelched for a while. Restart eclipse to
start over."));
}
}
}
}
```

lombok/core/debug/package-info.java

```
/*
* Copyright (C) 2012-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* This package contains tooling used only to debug issues that cannot be found with research but
```

```
* which require releasing a production or edge release with extra introspective facilities in
* an attempt to add clarity to the exceptions or other messages that result when the bug occurs.
*
* <strong>NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.</strong>
*/
package lombok.core.debug;
```

lombok/core/handlers/HandlerUtil.java

```
/*
* Copyright (C) 2013-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.handlers;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import lombok.AllArgsConstructor;
import lombok.ConfigurationKeys;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;
import lombok.Setter;
import lombok.ToString;
import lombok.Value;
import lombok.With;
```

```
import lombok.core.AST;
import lombok.core.AnnotationValues;
import lombok.core.JavaIdentifiers;
import lombok.core.LombokNode;
import lombok.core.configuration.AllowHelper;
import lombok.core.configuration.CapitalizationStrategy;
import lombok.core.configuration.ConfigurationKey;
import lombok.core.configuration.FlagUsageType;
import lombok.experimental.Accessors;
import lombok.experimental.FieldDefaults;
/**
* Container for static utility methods useful for some of the standard lombok handlers, regardless of
* target platform (e.g. useful for both javac and Eclipse lombok implementations).
*/
public class HandlerUtil {
private HandlerUtil() {}
public enum FieldAccess {
GETTER, PREFER_FIELD, ALWAYS_FIELD;
}
public static int primeForHashcode() {
return 59;
}
public static int primeForTrue() {
return 79;
}
public static int primeForFalse() {
return 97;
}
public static int primeForNull() {
return 43;
}
public static final List<String> NONNULL_ANNOTATIONS, BASE_COPYABLE_ANNOTATIONS,
COPY_TO_SETTER_ANNOTATIONS, COPY_TO_BUILDER_SINGULAR_SETTER_ANNOTATIONS,
JACKSON_COPY_TO_BUILDER_ANNOTATIONS;
static {
// This is a list of annotations with a __highly specific meaning__: All annotations in this list indicate that
passing null for the relevant item is __never__ acceptable, regardless of settings or circumstance.
// In other words, things like 'this models a database table, and the db table column has a nonnull
constraint', or 'this represents a web form, and if this is null, the form is invalid' __do not count__ and
should not be in this list;
// after all, you should be able to model invalid rows, or invalid forms.
// In addition, the intent for these annotations is that they can be used 'in public' - it's not for internal-only
usage annotations.
// Presence of these annotations mean that lombok will generate null checks in any created setters and
constructors.
NONNULL_ANNOTATIONS = Collections.unmodifiableList(Arrays.asList(new String[] {
"android.annotation.NonNull",
"android.support.annotation.NonNull",
"android.support.annotation.RecentlyNonNull",
"androidx.annotation.NonNull",
"androidx.annotation.RecentlyNonNull",
"com.android.annotations.NonNull",
```

"com.google.firebase.database.annotations.NotNull", // Even though it's in a database package, it does mean semantically: "Check if never null at the language level", and not 'db column cannot be null'.
"com.mongodb.lang.NonNull", // Even though mongo is a DB engine, this semantically refers to language, not DB table designs (mongo is a document DB engine, so this isn't surprising perhaps).
"com.sun.istack.NotNull",
"com.unboundid.util.NotNull",
"edu.umd.cs.findbugs.annotations.NonNull",
"io.micrometer.core.lang.NonNull",
"io.reactivex.annotations.NonNull",
"io.reactivex.rxjava3.annotations.NonNull",
"jakarta.annotation.Nonnull",
"javax.annotation.Nonnull",
// "javax.validation.constraints.NotNull", // The field might contain a null value until it is persisted.
"libcore.util.NonNull",
"lombok.NonNull",
"org.checkerframework.checker.nullness.qual.NonNull",
"org.checkerframework.checker.nullness.compatqual.NonNullDecl",
"org.checkerframework.checker.nullness.compatqual.NonNullType",
"org.codehaus.commons.nullanalysis.NotNull",
"org.eclipse.jdt.annotation.NonNull",
"org.jetbrains.annotations.NotNull",
"org.jmlspecs.annotation.NonNull",
"org.jspecify.annotations.NonNull",
"org.netbeans.api.annotations.common.NonNull",
"org.springframework.lang.NonNull",
"reactor.util.annotation.NonNull",
}));
// This is a list of annotations that lombok will automatically 'copy' - be it to the method (when generating a getter for a field annotated with one of these), or to a parameter (generating a setter, with-er, or builder 'setter').
// You can't disable this behaviour, so the list should only contain annotations where 'copy it!' is the desired behaviour in at least 95%, preferably 98%, of all non-buggy usages.
// As a general rule, lombok takes on maintenance of adding all nullity-related annotations here, _if_ they fit the definition of language-level nullity as per {@see #NONNULL_ANNOTATIONS}. As a consequence, everything from the NONNULL list should probably
// also be in this list, and any nullity-related annotation in this list implies the non-null variant should be in the NONNULL_ANNOTATIONS list, unless there is no such annotation.
// NB: Intent is that we get rid of a lot of this list and instead move to a system whereby lombok users explicitly opt in to the desired behaviour per 'library' (e.g per "Jackson annotations", "Checker framework annotations", etc.
// - the problem is, how do we know that the owners of a certain annotation intend for it to be copied in this fashion? What to do if a bug report is filed that we should not always copy it? Hence, care should be taken when editing this list.
// When in doubt, leave it out - this list can be added to dynamically by {See lombok.ConfigurationKeys#COPYABLE_ANNOTATIONS}.
BASE_COPYABLE_ANNOTATIONS = Collections.unmodifiableList(Arrays.asList(new String[] {
"android.annotation.NonNull",
"android.annotation.Nullable",
"android.support.annotation.NonNull",
"android.support.annotation.Nullable",
"android.support.annotation.RecentlyNonNull",
"android.support.annotation.RecentlyNullable",
"androidx.annotation.NonNull",

"androidx.annotation.Nullable",
"androidx.annotation.RecentlyNonNull",
"androidx.annotation.RecentlyNullable",
"com.android.annotations.NonNull",
"com.android.annotations.Nullable",
// "com.google.api.server.spi.config.Nullable", - let's think about this one a little, as it is targeted solely at parameters, so you can't even put it on fields. If we choose to support it, we should REMOVE it from the field, then - that's not something we currently support.
"com.google.firebase.database.annotations.NotNull",
"com.google.firebase.database.annotations.Nullable",
"com.mongodb.lang.NonNull",
"com.mongodb.lang.Nullable",
"com.sun.istack.NotNull",
"com.sun.istack.Nullable",
"com.unboundid.util.NotNull",
"com.unboundid.util.Nullable",
"edu.umd.cs.findbugs.annotations.CheckForNull",
"edu.umd.cs.findbugs.annotations.NonNull",
"edu.umd.cs.findbugs.annotations.Nullable",
"edu.umd.cs.findbugs.annotations.PossiblyNull",
"edu.umd.cs.findbugs.annotations.UnknownNullness",
"io.micrometer.core.lang.NonNull",
"io.micrometer.core.lang.Nullable",
"io.reactivex.annotations.NonNull",
"io.reactivex.annotations.Nullable",
"io.reactivex.rxjava3.annotations.NonNull",
"io.reactivex.rxjava3.annotations.Nullable",
"jakarta.annotation.Nonnull",
"jakarta.annotation.Nullable",
"javax.annotation.CheckForNull",
"javax.annotation.Nonnull",
"javax.annotation.Nullable",
// "javax.validation.constraints.NotNull", // - this should definitely not be included; validation is not about language-level nullity, therefore should not be in this core list.
"libcore.util.NonNull",
"libcore.util.Nullable",
"lombok.NonNull",
"org.checkerframework.checker.nullness.compatqual.NonNullDecl",
"org.checkerframework.checker.nullness.compatqual.NonNullType",
"org.checkerframework.checker.nullness.compatqual.NullableDecl",
"org.checkerframework.checker.nullness.compatqual.NullableType",
"org.checkerframework.checker.nullness.qual.NonNull",
"org.checkerframework.checker.nullness.qual.Nullable",
"org.codehaus.commons.nullanalysis.NotNull",
"org.codehaus.commons.nullanalysis.Nullable",
"org.eclipse.jdt.annotation.NonNull",
"org.eclipse.jdt.annotation.Nullable",
"org.jetbrains.annotations.NotNull",
"org.jetbrains.annotations.Nullable",
"org.jetbrains.annotations.UnknownNullability",
"org.jmlspecs.annotation.NonNull",
"org.jmlspecs.annotation.Nullable",
"org.jspecify.annotations.Nullable",

"org.jspecify.annotations.NonNull",
"org.netbeans.api.annotations.common.CheckForNull",
"org.netbeans.api.annotations.common.NonNull",
"org.netbeans.api.annotations.common.NullAllowed",
"org.netbeans.api.annotations.common.NullUnknown",
"org.springframework.lang.NonNull",
"org.springframework.lang.Nullable",
"reactor.util.annotation.NonNull",
"reactor.util.annotation.Nullable",
// Checker Framework annotations.
// To update Checker Framework annotations, run:
// grep --recursive --files-with-matches -e '^@Target\b.*TYPE_USE'
$CHECKERFRAMEWORK/checker/src/main/java
$CHECKERFRAMEWORK/checker-qual/src/main/java
$CHECKERFRAMEWORK/checker-util/src/main/java
$CHECKERFRAMEWORK/framework/src/main/java | grep '\.java$' | sed 's/.*\/java\//\t\t\t"/' | sed
's/\.java$/",/' | sed 's/\//./g' | sort
// Only add new annotations, do not remove annotations that have been removed from the latest
version of the Checker Framework.
"org.checkerframework.checker.builder.qual.CalledMethods",
"org.checkerframework.checker.builder.qual.NotCalledMethods",
"org.checkerframework.checker.calledmethods.qual.CalledMethods",
"org.checkerframework.checker.calledmethods.qual.CalledMethodsBottom",
"org.checkerframework.checker.calledmethods.qual.CalledMethodsPredicate",
"org.checkerframework.checker.compilermsgs.qual.CompilerMessageKey",
"org.checkerframework.checker.compilermsgs.qual.CompilerMessageKeyBottom",
"org.checkerframework.checker.compilermsgs.qual.UnknownCompilerMessageKey",
"org.checkerframework.checker.fenum.qual.AwtAlphaCompositingRule",
"org.checkerframework.checker.fenum.qual.AwtColorSpace",
"org.checkerframework.checker.fenum.qual.AwtCursorType",
"org.checkerframework.checker.fenum.qual.AwtFlowLayout",
"org.checkerframework.checker.fenum.qual.Fenum",
"org.checkerframework.checker.fenum.qual.FenumBottom",
"org.checkerframework.checker.fenum.qual.FenumTop",
"org.checkerframework.checker.fenum.qual.PolyFenum",
"org.checkerframework.checker.fenum.qual.SwingBoxOrientation",
"org.checkerframework.checker.fenum.qual.SwingCompassDirection",
"org.checkerframework.checker.fenum.qual.SwingElementOrientation",
"org.checkerframework.checker.fenum.qual.SwingHorizontalOrientation",
"org.checkerframework.checker.fenum.qual.SwingSplitPaneOrientation",
"org.checkerframework.checker.fenum.qual.SwingTextOrientation",
"org.checkerframework.checker.fenum.qual.SwingTitleJustification",
"org.checkerframework.checker.fenum.qual.SwingTitlePosition",
"org.checkerframework.checker.fenum.qual.SwingVerticalOrientation",
"org.checkerframework.checker.formatter.qual.Format",
"org.checkerframework.checker.formatter.qual.FormatBottom",
"org.checkerframework.checker.formatter.qual.InvalidFormat",
"org.checkerframework.checker.formatter.qual.UnknownFormat",
"org.checkerframework.checker.guieffect.qual.AlwaysSafe",
"org.checkerframework.checker.guieffect.qual.PolyUI",
"org.checkerframework.checker.guieffect.qual.UI",
"org.checkerframework.checker.i18nformatter.qual.I18nFormat",
"org.checkerframework.checker.i18nformatter.qual.I18nFormatBottom",

"org.checkerframework.checker.i18nformatter.qual.I18nFormatFor",
"org.checkerframework.checker.i18nformatter.qual.I18nInvalidFormat",
"org.checkerframework.checker.i18nformatter.qual.I18nUnknownFormat",
"org.checkerframework.checker.i18n.qual.LocalizableKey",
"org.checkerframework.checker.i18n.qual.LocalizableKeyBottom",
"org.checkerframework.checker.i18n.qual.Localized",
"org.checkerframework.checker.i18n.qual.UnknownLocalizableKey",
"org.checkerframework.checker.i18n.qual.UnknownLocalized",
"org.checkerframework.checker.index.qual.GTENegativeOne",
"org.checkerframework.checker.index.qual.IndexFor",
"org.checkerframework.checker.index.qual.IndexOrHigh",
"org.checkerframework.checker.index.qual.IndexOrLow",
"org.checkerframework.checker.index.qual.LengthOf",
"org.checkerframework.checker.index.qual.LessThan",
"org.checkerframework.checker.index.qual.LessThanBottom",
"org.checkerframework.checker.index.qual.LessThanUnknown",
"org.checkerframework.checker.index.qual.LowerBoundBottom",
"org.checkerframework.checker.index.qual.LowerBoundUnknown",
"org.checkerframework.checker.index.qual.LTEqLengthOf",
"org.checkerframework.checker.index.qual.LTLengthOf",
"org.checkerframework.checker.index.qual.LTOMLengthOf",
"org.checkerframework.checker.index.qual.NegativeIndexFor",
"org.checkerframework.checker.index.qual.NonNegative",
"org.checkerframework.checker.index.qual.PolyIndex",
"org.checkerframework.checker.index.qual.PolyLength",
"org.checkerframework.checker.index.qual.PolyLowerBound",
"org.checkerframework.checker.index.qual.PolySameLen",
"org.checkerframework.checker.index.qual.PolyUpperBound",
"org.checkerframework.checker.index.qual.Positive",
"org.checkerframework.checker.index.qual.SameLen",
"org.checkerframework.checker.index.qual.SameLenBottom",
"org.checkerframework.checker.index.qual.SameLenUnknown",
"org.checkerframework.checker.index.qual.SearchIndexBottom",
"org.checkerframework.checker.index.qual.SearchIndexFor",
"org.checkerframework.checker.index.qual.SearchIndexUnknown",
"org.checkerframework.checker.index.qual.SubstringIndexBottom",
"org.checkerframework.checker.index.qual.SubstringIndexFor",
"org.checkerframework.checker.index.qual.SubstringIndexUnknown",
"org.checkerframework.checker.index.qual.UpperBoundBottom",
"org.checkerframework.checker.index.qual.UpperBoundLiteral",
"org.checkerframework.checker.index.qual.UpperBoundUnknown",
"org.checkerframework.checker.initialization.qual.FBCBottom",
"org.checkerframework.checker.initialization.qual.Initialized",
"org.checkerframework.checker.initialization.qual.UnderInitialization",
"org.checkerframework.checker.initialization.qual.UnknownInitialization",
"org.checkerframework.checker.interning.qual.Interned",
"org.checkerframework.checker.interning.qual.InternedDistinct",
"org.checkerframework.checker.interning.qual.PolyInterned",
"org.checkerframework.checker.interning.qual.UnknownInterned",
"org.checkerframework.checker.lock.qual.GuardedBy",
"org.checkerframework.checker.lock.qual.GuardedByBottom",
"org.checkerframework.checker.lock.qual.GuardedByUnknown",
"org.checkerframework.checker.lock.qual.GuardSatisfied",

"org.checkerframework.checker.lock.qual.NewObject",
"org.checkerframework.checker.mustcall.qual.MustCall",
"org.checkerframework.checker.mustcall.qual.MustCallAlias",
"org.checkerframework.checker.mustcall.qual.MustCallUnknown",
"org.checkerframework.checker.mustcall.qual.PolyMustCall",
"org.checkerframework.checker.nullness.qual.KeyFor",
"org.checkerframework.checker.nullness.qual.KeyForBottom",
"org.checkerframework.checker.nullness.qual.MonotonicNonNull",
"org.checkerframework.checker.nullness.qual.NonNull",
"org.checkerframework.checker.nullness.qual.Nullable",
"org.checkerframework.checker.nullness.qual.PolyKeyFor",
"org.checkerframework.checker.nullness.qual.PolyNull",
"org.checkerframework.checker.nullness.qual.UnknownKeyFor",
"org.checkerframework.checker.optional.qual.MaybePresent",
"org.checkerframework.checker.optional.qual.OptionalBottom",
"org.checkerframework.checker.optional.qual.PolyPresent",
"org.checkerframework.checker.optional.qual.Present",
"org.checkerframework.checker.propkey.qual.PropertyKey",
"org.checkerframework.checker.propkey.qual.PropertyKeyBottom",
"org.checkerframework.checker.propkey.qual.UnknownPropertyKey",
"org.checkerframework.checker.regex.qual.PolyRegex",
"org.checkerframework.checker.regex.qual.Regex",
"org.checkerframework.checker.regex.qual.RegexBottom",
"org.checkerframework.checker.regex.qual.UnknownRegex",
"org.checkerframework.checker.signature.qual.ArrayWithoutPackage",
"org.checkerframework.checker.signature.qual.BinaryName",
"org.checkerframework.checker.signature.qual.BinaryNameOrPrimitiveType",
"org.checkerframework.checker.signature.qual.BinaryNameWithoutPackage",
"org.checkerframework.checker.signature.qual.CanonicalName",
"org.checkerframework.checker.signature.qual.CanonicalNameAndBinaryName",
"org.checkerframework.checker.signature.qual.CanonicalNameOrEmpty",
"org.checkerframework.checker.signature.qual.CanonicalNameOrPrimitiveType",
"org.checkerframework.checker.signature.qual.ClassGetName",
"org.checkerframework.checker.signature.qual.ClassGetSimpleName",
"org.checkerframework.checker.signature.qual.DotSeparatedIdentifiers",
"org.checkerframework.checker.signature.qual.DotSeparatedIdentifiersOrPrimitiveType",
"org.checkerframework.checker.signature.qual.FieldDescriptor",
"org.checkerframework.checker.signature.qual.FieldDescriptorForPrimitive",
"org.checkerframework.checker.signature.qual.FieldDescriptorWithoutPackage",
"org.checkerframework.checker.signature.qual.FqBinaryName",
"org.checkerframework.checker.signature.qual.FullyQualifiedName",
"org.checkerframework.checker.signature.qual.Identifier",
"org.checkerframework.checker.signature.qual.IdentifierOrPrimitiveType",
"org.checkerframework.checker.signature.qual.InternalForm",
"org.checkerframework.checker.signature.qual.MethodDescriptor",
"org.checkerframework.checker.signature.qual.PolySignature",
"org.checkerframework.checker.signature.qual.PrimitiveType",
"org.checkerframework.checker.signature.qual.SignatureBottom",
"org.checkerframework.checker.signedness.qual.PolySigned",
"org.checkerframework.checker.signedness.qual.Signed",
"org.checkerframework.checker.signedness.qual.SignednessBottom",
"org.checkerframework.checker.signedness.qual.SignednessGlb",
"org.checkerframework.checker.signedness.qual.SignedPositive",

"org.checkerframework.checker.signedness.qual.SignedPositiveFromUnsigned",
"org.checkerframework.checker.signedness.qual.UnknownSignedness",
"org.checkerframework.checker.signedness.qual.Unsigned",
"org.checkerframework.checker.tainting.qual.PolyTainted",
"org.checkerframework.checker.tainting.qual.Tainted",
"org.checkerframework.checker.tainting.qual.Untainted",
"org.checkerframework.checker.units.qual.A",
"org.checkerframework.checker.units.qual.Acceleration",
"org.checkerframework.checker.units.qual.Angle",
"org.checkerframework.checker.units.qual.Area",
"org.checkerframework.checker.units.qual.C",
"org.checkerframework.checker.units.qual.cd",
"org.checkerframework.checker.units.qual.Current",
"org.checkerframework.checker.units.qual.degrees",
"org.checkerframework.checker.units.qual.Force",
"org.checkerframework.checker.units.qual.g",
"org.checkerframework.checker.units.qual.h",
"org.checkerframework.checker.units.qual.K",
"org.checkerframework.checker.units.qual.kg",
"org.checkerframework.checker.units.qual.km",
"org.checkerframework.checker.units.qual.km2",
"org.checkerframework.checker.units.qual.km3",
"org.checkerframework.checker.units.qual.kmPERh",
"org.checkerframework.checker.units.qual.kN",
"org.checkerframework.checker.units.qual.Length",
"org.checkerframework.checker.units.qual.Luminance",
"org.checkerframework.checker.units.qual.m",
"org.checkerframework.checker.units.qual.m2",
"org.checkerframework.checker.units.qual.m3",
"org.checkerframework.checker.units.qual.Mass",
"org.checkerframework.checker.units.qual.min",
"org.checkerframework.checker.units.qual.mm",
"org.checkerframework.checker.units.qual.mm2",
"org.checkerframework.checker.units.qual.mm3",
"org.checkerframework.checker.units.qual.mol",
"org.checkerframework.checker.units.qual.mPERs",
"org.checkerframework.checker.units.qual.mPERs2",
"org.checkerframework.checker.units.qual.N",
"org.checkerframework.checker.units.qual.PolyUnit",
"org.checkerframework.checker.units.qual.radians",
"org.checkerframework.checker.units.qual.s",
"org.checkerframework.checker.units.qual.Speed",
"org.checkerframework.checker.units.qual.Substance",
"org.checkerframework.checker.units.qual.t",
"org.checkerframework.checker.units.qual.Temperature",
"org.checkerframework.checker.units.qual.Time",
"org.checkerframework.checker.units.qual.UnitsBottom",
"org.checkerframework.checker.units.qual.UnknownUnits",
"org.checkerframework.checker.units.qual.Volume",
"org.checkerframework.common.aliasing.qual.LeakedToResult",
"org.checkerframework.common.aliasing.qual.MaybeAliased",
"org.checkerframework.common.aliasing.qual.NonLeaked",
"org.checkerframework.common.aliasing.qual.Unique",

```
"org.checkerframework.common.initializedfields.qual.InitializedFields",
"org.checkerframework.common.initializedfields.qual.InitializedFieldsBottom",
"org.checkerframework.common.initializedfields.qual.PolyInitializedFields",
"org.checkerframework.common.reflection.qual.ClassBound",
"org.checkerframework.common.reflection.qual.ClassVal",
"org.checkerframework.common.reflection.qual.ClassValBottom",
"org.checkerframework.common.reflection.qual.MethodVal",
"org.checkerframework.common.reflection.qual.MethodValBottom",
"org.checkerframework.common.reflection.qual.UnknownClass",
"org.checkerframework.common.reflection.qual.UnknownMethod",
"org.checkerframework.common.returnsreceiver.qual.BottomThis",
"org.checkerframework.common.returnsreceiver.qual.This",
"org.checkerframework.common.returnsreceiver.qual.UnknownThis",
"org.checkerframework.common.subtyping.qual.Bottom",
"org.checkerframework.common.util.report.qual.ReportUnqualified",
"org.checkerframework.common.value.qual.ArrayLen",
"org.checkerframework.common.value.qual.ArrayLenRange",
"org.checkerframework.common.value.qual.BoolVal",
"org.checkerframework.common.value.qual.BottomVal",
"org.checkerframework.common.value.qual.DoubleVal",
"org.checkerframework.common.value.qual.EnumVal",
"org.checkerframework.common.value.qual.IntRange",
"org.checkerframework.common.value.qual.IntVal",
"org.checkerframework.common.value.qual.MatchesRegex",
"org.checkerframework.common.value.qual.MinLen",
"org.checkerframework.common.value.qual.PolyValue",
"org.checkerframework.common.value.qual.StringVal",
"org.checkerframework.common.value.qual.UnknownVal",
"org.checkerframework.framework.qual.PurityUnqualified",
}));
COPY_TO_SETTER_ANNOTATIONS = Collections.unmodifiableList(Arrays.asList(new String[] {
"com.fasterxml.jackson.annotation.JacksonInject",
"com.fasterxml.jackson.annotation.JsonAlias",
"com.fasterxml.jackson.annotation.JsonFormat",
"com.fasterxml.jackson.annotation.JsonIgnore",
"com.fasterxml.jackson.annotation.JsonIgnoreProperties",
"com.fasterxml.jackson.annotation.JsonProperty",
"com.fasterxml.jackson.annotation.JsonSetter",
"com.fasterxml.jackson.annotation.JsonSubTypes",
"com.fasterxml.jackson.annotation.JsonTypeInfo",
"com.fasterxml.jackson.annotation.JsonUnwrapped",
"com.fasterxml.jackson.annotation.JsonView",
"com.fasterxml.jackson.databind.annotation.JsonDeserialize",
"com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlElementWrapper",
"com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlProperty",
"com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlText",
}));
COPY_TO_BUILDER_SINGULAR_SETTER_ANNOTATIONS =
Collections.unmodifiableList(Arrays.asList(new String[] {
"com.fasterxml.jackson.annotation.JsonAnySetter",
}));
JACKSON_COPY_TO_BUILDER_ANNOTATIONS = Collections.unmodifiableList(Arrays.asList(new
String[] {
```

```
"com.fasterxml.jackson.annotation.JsonAutoDetect",
"com.fasterxml.jackson.annotation.JsonFormat",
"com.fasterxml.jackson.annotation.JsonIgnoreProperties",
"com.fasterxml.jackson.annotation.JsonIgnoreType",
"com.fasterxml.jackson.annotation.JsonPropertyOrder",
"com.fasterxml.jackson.annotation.JsonRootName",
"com.fasterxml.jackson.annotation.JsonSubTypes",
"com.fasterxml.jackson.annotation.JsonTypeInfo",
"com.fasterxml.jackson.annotation.JsonTypeName",
"com.fasterxml.jackson.annotation.JsonView",
"com.fasterxml.jackson.databind.annotation.JsonNaming",
}));
}
/** Checks if the given name is a valid identifier.
*
* If it is, this returns {@code true} and does nothing else.
* If it isn't, this returns {@code false} and adds an error message to the supplied node.
*/
public static boolean checkName(String nameSpec, String identifier, LombokNode<?, ?, ?> errorNode)
{
if (identifier.isEmpty()) {
errorNode.addError(nameSpec + " cannot be the empty string.");
return false;
}
if (!JavaIdentifiers.isValidJavaIdentifier(identifier)) {
errorNode.addError(nameSpec + " must be a valid java identifier.");
return false;
}
return true;
}
public static String autoSingularize(String plural) {
return Singulars.autoSingularize(plural);
}
public static void handleFlagUsage(LombokNode<?, ?, ?> node, ConfigurationKey<FlagUsageType>
key, String featureName) {
FlagUsageType fut = node.getAst().readConfiguration(key);
if (fut == null && AllowHelper.isAllowable(key)) {
node.addError("Use of " + featureName + " is disabled by default. Please add '" + key.getKeyName() +
" = " + FlagUsageType.ALLOW + "' to 'lombok.config' if you want to enable is.");
}
if (fut != null) {
String msg = "Use of " + featureName + " is flagged according to lombok configuration.";
if (fut == FlagUsageType.WARNING) node.addWarning(msg);
else if (fut == FlagUsageType.ERROR) node.addError(msg);
}
}
@SuppressWarnings("deprecation")
public static boolean shouldAddGenerated(LombokNode<?, ?, ?> node) {
Boolean add =
node.getAst().readConfiguration(ConfigurationKeys.ADD_JAVAX_GENERATED_ANNOTATIONS);
if (add != null) return add;
return Boolean.TRUE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_GENERATED_
ANNOTATIONS));
```

```
}
public static void handleExperimentalFlagUsage(LombokNode<?, ?, ?> node,
ConfigurationKey<FlagUsageType> key, String featureName) {
handleFlagUsage(node, key, featureName, ConfigurationKeys.EXPERIMENTAL_FLAG_USAGE, "any
lombok.experimental feature");
}
public static void handleFlagUsage(LombokNode<?, ?, ?> node, ConfigurationKey<FlagUsageType>
key1, String featureName1, ConfigurationKey<FlagUsageType> key2, String featureName2) {
FlagUsageType fut1 = node.getAst().readConfiguration(key1);
FlagUsageType fut2 = node.getAst().readConfiguration(key2);
FlagUsageType fut = null;
String featureName = null;
if (fut1 == FlagUsageType.ERROR) {
fut = fut1;
featureName = featureName1;
} else if (fut2 == FlagUsageType.ERROR) {
fut = fut2;
featureName = featureName2;
} else if (fut1 == FlagUsageType.WARNING) {
fut = fut1;
featureName = featureName1;
} else {
fut = fut2;
featureName = featureName2;
}
if (fut != null) {
String msg = "Use of " + featureName + " is flagged according to lombok configuration.";
if (fut == FlagUsageType.WARNING) node.addWarning(msg);
else if (fut == FlagUsageType.ERROR) node.addError(msg);
}
}
public static boolean shouldReturnThis0(AnnotationValues<Accessors> accessors, AST<?, ?, ?> ast) {
boolean chainForced = accessors.isExplicit("chain");
boolean fluentForced = accessors.isExplicit("fluent");
Accessors instance = accessors.getInstance();
boolean chain = instance.chain();
boolean fluent = instance.fluent();
if (chainForced) return chain;
if (!chainForced) {
Boolean chainConfig = ast.readConfiguration(ConfigurationKeys.ACCESSORS_CHAIN);
if (chainConfig != null) return chainConfig;
}
if (!fluentForced) {
Boolean fluentConfig = ast.readConfiguration(ConfigurationKeys.ACCESSORS_FLUENT);
if (fluentConfig != null) fluent = fluentConfig;
}
return chain || fluent;
}
public static boolean shouldMakeFinal0(AnnotationValues<Accessors> accessors, AST<?, ?, ?> ast) {
boolean isExplicit = accessors.isExplicit("makeFinal");
if (isExplicit) return accessors.getAsBoolean("makeFinal");
Boolean config = ast.readConfiguration(ConfigurationKeys.ACCESSORS_MAKE_FINAL);
if (config != null) return config.booleanValue();
```

```
return false;
}
@SuppressWarnings({"all", "unchecked", "deprecation"})
public static final List<String> INVALID_ON_BUILDERS = Collections.unmodifiableList(
Arrays.<String>asList(
Getter.class.getName(), Setter.class.getName(), With.class.getName(), "lombok.experimental.Wither",
ToString.class.getName(), EqualsAndHashCode.class.getName(),
RequiredArgsConstructor.class.getName(), AllArgsConstructor.class.getName(),
NoArgsConstructor.class.getName(),
Data.class.getName(), Value.class.getName(), "lombok.experimental.Value",
FieldDefaults.class.getName()));
/**
* Given the name of a field, return the 'base name' of that field. For example, {@code fFoobar}
becomes {@code foobar} if {@code f} is in the prefix list.
* For prefixes that end in a letter character, the next character must be a non-lowercase character (i.e.
{@code hashCode} is not {@code ashCode} even if
* {@code h} is in the prefix list, but {@code hAshcode} would become {@code ashCode}). The first
prefix that matches is used. If the prefix list is empty,
* or the empty string is in the prefix list and no prefix before it matches, the fieldName will be returned
verbatim.
*
* If no prefix matches and the empty string is not in the prefix list and the prefix list is not empty,
{@code null} is returned.
*
* @param fieldName The full name of a field.
* @param prefixes A list of prefixes, usually provided by the {@code Accessors} settings annotation,
listing field prefixes.
* @return The base name of the field.
*/
public static CharSequence removePrefix(CharSequence fieldName, List<String> prefixes) {
if (prefixes == null || prefixes.isEmpty()) return fieldName;
fieldName = fieldName.toString();
outer:
for (String prefix : prefixes) {
if (prefix.length() == 0) return fieldName;
if (fieldName.length() <= prefix.length()) continue outer;
for (int i = 0; i < prefix.length(); i++) {
if (fieldName.charAt(i) != prefix.charAt(i)) continue outer;
}
char followupChar = fieldName.charAt(prefix.length());
// if prefix is a letter then follow up letter needs to not be lowercase, i.e. 'foo' is not a match
// as field named 'oo' with prefix 'f', but 'fOo' would be.
if (Character.isLetter(prefix.charAt(prefix.length() - 1)) &&
Character.isLowerCase(followupChar)) continue outer;
return "" + Character.toLowerCase(followupChar) + fieldName.subSequence(prefix.length() + 1,
fieldName.length());
}
return null;
}
public static final String DEFAULT_EXCEPTION_FOR_NON_NULL = "java.lang.NullPointerException";
/**
* Generates a getter name from a given field name.
*
```

```
* Strategy:
* <ul>
* <li>Reduce the field's name to its base name by stripping off any prefix (from {@code Accessors}). If the field name does not fit
* the prefix list, this method immediately returns {@code null}.</li>
* <li>If {@code Accessors} has {@code fluent=true}, then return the basename.</li>
* <li>Pick a prefix. 'get' normally, but 'is' if {@code isBoolean} is true.</li>
* <li>Only if {@code isBoolean} is true: Check if the field starts with {@code is} followed by a non-lowercase character. If so, return the field name verbatim.</li>
* <li>Check if the first character of the field is lowercase. If so, check if the second character
* exists and is title or upper case. If so, uppercase the first character. If not, titlecase the first character.</li>
* <li>Return the prefix plus the possibly title/uppercased first character, and the rest of the field name.</li>
* </ul>
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
* @param isBoolean if the field is of type 'boolean'. For fields of type {@code java.lang.Boolean}, you should provide {@code false}.
* @return The getter name for this field, or {@code null} if this field does not fit expected patterns and therefore cannot be turned into a getter name.
*/
public static String toGetterName(AST<?, ?, ?> ast, AnnotationValues<Accessors> accessors, CharSequence fieldName, boolean isBoolean) {
return toAccessorName(ast, accessors, fieldName, isBoolean, "is", "get", true);
}
/**
* Generates a setter name from a given field name.
*
* Strategy:
* <ul>
* <li>Reduce the field's name to its base name by stripping off any prefix (from {@code Accessors}). If the field name does not fit
* the prefix list, this method immediately returns {@code null}.</li>
* <li>If {@code Accessors} has {@code fluent=true}, then return the basename.</li>
* <li>Only if {@code isBoolean} is true: Check if the field starts with {@code is} followed by a non-lowercase character.
* If so, replace {@code is} with {@code set} and return that.</li>
* <li>Check if the first character of the field is lowercase. If so, check if the second character
* exists and is title or upper case. If so, uppercase the first character. If not, titlecase the first character.</li>
* <li>Return {@code "set"} plus the possibly title/uppercased first character, and the rest of the field name.</li>
* </ul>
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
* @param isBoolean if the field is of type 'boolean'. For fields of type {@code java.lang.Boolean}, you should provide {@code false}.
* @return The setter name for this field, or {@code null} if this field does not fit expected patterns and therefore cannot be turned into a getter name.
*/
```

```
public static String toSetterName(AST<?, ?, ?> ast, AnnotationValues<Accessors> accessors,
CharSequence fieldName, boolean isBoolean) {
return toAccessorName(ast, accessors, fieldName, isBoolean, "set", "set", true);
}
/**
* Generates a with name from a given field name.
*
* Strategy:
* <ul>
* <li>Reduce the field's name to its base name by stripping off any prefix (from {@code Accessors}). If
the field name does not fit
* the prefix list, this method immediately returns {@code null}.</li>
* <li>Only if {@code isBoolean} is true: Check if the field starts with {@code is} followed by a
non-lowercase character.
* If so, replace {@code is} with {@code with} and return that.</li>
* <li>Check if the first character of the field is lowercase. If so, check if the second character
* exists and is title or upper case. If so, uppercase the first character. If not, titlecase the first
character.</li>
* <li>Return {@code "with"} plus the possibly title/uppercased first character, and the rest of the field
name.</li>
* </ul>
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
* @param isBoolean if the field is of type 'boolean'. For fields of type {@code java.lang.Boolean}, you
should provide {@code false}.
* @return The with name for this field, or {@code null} if this field does not fit expected patterns and
therefore cannot be turned into a getter name.
*/
public static String toWithName(AST<?, ?, ?> ast, AnnotationValues<Accessors> accessors,
CharSequence fieldName, boolean isBoolean) {
return toAccessorName(ast, accessors, fieldName, isBoolean, "with", "with", false);
}
/**
* Generates a withBy name from a given field name.
*
* Strategy: The same as the {@code toWithName} strategy, but then append {@code "By"} at the end.
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
* @param isBoolean if the field is of type 'boolean'. For fields of type {@code java.lang.Boolean}, you
should provide {@code false}.
* @return The with name for this field, or {@code null} if this field does not fit expected patterns and
therefore cannot be turned into a getter name.
*/
public static String toWithByName(AST<?, ?, ?> ast, AnnotationValues<Accessors> accessors,
CharSequence fieldName, boolean isBoolean) {
return toAccessorName(ast, accessors, fieldName, isBoolean, "with", "with", false) + "By";
}
private static String toAccessorName(AST<?, ?, ?> ast, AnnotationValues<Accessors> accessors,
CharSequence fieldName, boolean isBoolean,
String booleanPrefix, String normalPrefix, boolean adhereToFluent) {
fieldName = fieldName.toString();
```

```
if (fieldName.length() == 0) return null;
if (Boolean.TRUE.equals(ast.readConfiguration(ConfigurationKeys.GETTER_CONSEQUENT_BOOLE
AN))) isBoolean = false;
boolean explicitPrefix = accessors != null && accessors.isExplicit("prefix");
boolean explicitFluent = accessors != null && accessors.isExplicit("fluent");
boolean explicitJavaBeansSpecCapitalization = accessors != null &&
accessors.isExplicit("javaBeansSpecCapitalization");
Accessors ac = (explicitPrefix || explicitFluent || explicitJavaBeansSpecCapitalization) ?
accessors.getInstance() : null;
List<String> prefix = explicitPrefix ? Arrays.asList(ac.prefix()) :
ast.readConfiguration(ConfigurationKeys.ACCESSORS_PREFIX);
boolean fluent = explicitFluent ? ac.fluent() :
Boolean.TRUE.equals(ast.readConfiguration(ConfigurationKeys.ACCESSORS_FLUENT));
CapitalizationStrategy capitalizationStrategy =
ast.readConfigurationOr(ConfigurationKeys.ACCESSORS_JAVA_BEANS_SPEC_CAPITALIZATION,
CapitalizationStrategy.defaultValue());
fieldName = removePrefix(fieldName, prefix);
if (fieldName == null) return null;
String fName = fieldName.toString();
if (adhereToFluent && fluent) return fName;
if (isBoolean && fName.startsWith("is") && fieldName.length() > 2 &&
!Character.isLowerCase(fieldName.charAt(2))) {
// The field is for example named 'isRunning'.
return booleanPrefix + fName.substring(2);
}
return buildAccessorName(isBoolean ? booleanPrefix : normalPrefix, fName, capitalizationStrategy);
}
/**
* Returns all names of methods that would represent the getter for a field with the provided name.
*
* For example if {@code isBoolean} is true, then a field named {@code isRunning} would produce:<br
/>
* {@code [isRunning, getRunning, isIsRunning, getIsRunning]}
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
* @param isBoolean if the field is of type 'boolean'. For fields of type 'java.lang.Boolean', you should
provide {@code false}.
*/
public static List<String> toAllGetterNames(AST<?, ?, ?> ast, AnnotationValues<Accessors>
accessors, CharSequence fieldName, boolean isBoolean) {
return toAllAccessorNames(ast, accessors, fieldName, isBoolean, "is", "get", true);
}
/**
* Returns all names of methods that would represent the setter for a field with the provided name.
*
* For example if {@code isBoolean} is true, then a field named {@code isRunning} would produce:<br
/>
* {@code [setRunning, setIsRunning]}
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
```

```
* @param isBoolean if the field is of type 'boolean'. For fields of type 'java.lang.Boolean', you should
provide {@code false}.
*/
public static List<String> toAllSetterNames(AST<?, ?, ?> ast, AnnotationValues<Accessors>
accessors, CharSequence fieldName, boolean isBoolean) {
return toAllAccessorNames(ast, accessors, fieldName, isBoolean, "set", "set", true);
}
/**
* Returns all names of methods that would represent the with for a field with the provided name.
*
* For example if {@code isBoolean} is true, then a field named {@code isRunning} would produce:<br
/>
* {@code [withRunning, withIsRunning]}
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
* @param isBoolean if the field is of type 'boolean'. For fields of type 'java.lang.Boolean', you should
provide {@code false}.
*/
public static List<String> toAllWithNames(AST<?, ?, ?> ast, AnnotationValues<Accessors> accessors,
CharSequence fieldName, boolean isBoolean) {
return toAllAccessorNames(ast, accessors, fieldName, isBoolean, "with", "with", false);
}
/**
* Returns all names of methods that would represent the withBy for a field with the provided name.
*
* For example if {@code isBoolean} is true, then a field named {@code isRunning} would produce:<br
/>
* {@code [withRunningBy, withIsRunningBy]}
*
* @param accessors Accessors configuration.
* @param fieldName the name of the field.
* @param isBoolean if the field is of type 'boolean'. For fields of type 'java.lang.Boolean', you should
provide {@code false}.
*/
public static List<String> toAllWithByNames(AST<?, ?, ?> ast, AnnotationValues<Accessors>
accessors, CharSequence fieldName, boolean isBoolean) {
List<String> in = toAllAccessorNames(ast, accessors, fieldName, isBoolean, "with", "with", false);
if (!(in instanceof ArrayList)) in = new ArrayList<String>(in);
for (int i = 0; i < in.size(); i++) in.set(i, in.get(i) + "By");
return in;
}
private static List<String> toAllAccessorNames(AST<?, ?, ?> ast, AnnotationValues<Accessors>
accessors, CharSequence fieldName, boolean isBoolean,
String booleanPrefix, String normalPrefix, boolean adhereToFluent) {
if (Boolean.TRUE.equals(ast.readConfiguration(ConfigurationKeys.GETTER_CONSEQUENT_BOOLE
AN))) isBoolean = false;
if (!isBoolean) {
String accessorName = toAccessorName(ast, accessors, fieldName, false, booleanPrefix,
normalPrefix, adhereToFluent);
return (accessorName == null) ? Collections.<String>emptyList() :
Collections.singletonList(accessorName);
}
```

```
boolean explicitPrefix = accessors != null && accessors.isExplicit("prefix");
boolean explicitFluent = accessors != null && accessors.isExplicit("fluent");
Accessors ac = (explicitPrefix || explicitFluent) ? accessors.getInstance() : null;
List<String> prefix = explicitPrefix ? Arrays.asList(ac.prefix()) :
ast.readConfiguration(ConfigurationKeys.ACCESSORS_PREFIX);
boolean fluent = explicitFluent ? ac.fluent() :
Boolean.TRUE.equals(ast.readConfiguration(ConfigurationKeys.ACCESSORS_FLUENT));
CapitalizationStrategy capitalizationStrategy =
ast.readConfigurationOr(ConfigurationKeys.ACCESSORS_JAVA_BEANS_SPEC_CAPITALIZATION,
CapitalizationStrategy.defaultValue());
fieldName = removePrefix(fieldName, prefix);
if (fieldName == null) return Collections.emptyList();
List<String> baseNames = toBaseNames(fieldName, isBoolean, fluent);
Set<String> names = new HashSet<String>();
for (String baseName : baseNames) {
if (adhereToFluent && fluent) {
names.add(baseName);
} else {
names.add(buildAccessorName(normalPrefix, baseName, capitalizationStrategy));
if (!normalPrefix.equals(booleanPrefix)) names.add(buildAccessorName(booleanPrefix, baseName,
capitalizationStrategy));
}
}
return new ArrayList<String>(names);
}
private static List<String> toBaseNames(CharSequence fieldName, boolean isBoolean, boolean fluent)
{
List<String> baseNames = new ArrayList<String>();
baseNames.add(fieldName.toString());
// isPrefix = field is called something like 'isRunning', so 'running' could also be the fieldname.
String fName = fieldName.toString();
if (fName.startsWith("is") && fName.length() > 2 && !Character.isLowerCase(fName.charAt(2))) {
String baseName = fName.substring(2);
if (fluent) {
baseNames.add("" + Character.toLowerCase(baseName.charAt(0)) + baseName.substring(1));
} else {
baseNames.add(baseName);
}
}
return baseNames;
}
/**
* @param node Any node (used to fetch config of capitalization strategy).
* @param prefix Something like {@code get} or {@code set} or {@code is}.
* @param suffix Something like {@code running}.
* @return prefix + smartly title-cased suffix. For example, {@code setRunning}.
*/
public static String buildAccessorName(AST<?, ?, ?> ast, String prefix, String suffix) {
CapitalizationStrategy capitalizationStrategy =
ast.readConfigurationOr(ConfigurationKeys.ACCESSORS_JAVA_BEANS_SPEC_CAPITALIZATION,
CapitalizationStrategy.defaultValue());
return buildAccessorName(prefix, suffix, capitalizationStrategy);
}
```

```
/**
* @param node Any node (used to fetch config of capitalization strategy).
* @param prefix Something like {@code get} or {@code set} or {@code is}.
* @param suffix Something like {@code running}.
* @return prefix + smartly title-cased suffix. For example, {@code setRunning}.
*/
public static String buildAccessorName(LombokNode<?, ?, ?> node, String prefix, String suffix) {
CapitalizationStrategy capitalizationStrategy = node.getAst().readConfigurationOr(ConfigurationKeys.A
CCESSORS_JAVA_BEANS_SPEC_CAPITALIZATION, CapitalizationStrategy.defaultValue());
return buildAccessorName(prefix, suffix, capitalizationStrategy);
}
/**
* @param prefix Something like {@code get} or {@code set} or {@code is}.
* @param suffix Something like {@code running}.
* @param capitalizationStrategy Which strategy to use to capitalize the name part.
*/
private static String buildAccessorName(String prefix, String suffix, CapitalizationStrategy
capitalizationStrategy) {
if (suffix.length() == 0) return prefix;
if (prefix.length() == 0) return suffix;
return prefix + capitalizationStrategy.capitalize(suffix);
}
public static String camelCaseToConstant(String fieldName) {
if (fieldName == null || fieldName.isEmpty()) return "";
StringBuilder b = new StringBuilder();
b.append(Character.toUpperCase(fieldName.charAt(0)));
for (int i = 1; i < fieldName.length(); i++) {
char c = fieldName.charAt(i);
if (Character.isUpperCase(c)) b.append('_');
b.append(Character.toUpperCase(c));
}
return b.toString();
}
/** Matches any of the 8 primitive wrapper names, such as {@code Boolean}. */
private static final Pattern PRIMITIVE_WRAPPER_TYPE_NAME_PATTERN =
Pattern.compile("^(?:java\\.lang\\.)?(?:Boolean|Byte|Short|Integer|Long|Float|Double|Character)$");
public static int defaultEqualsAndHashcodeIncludeRank(String typeName) {
// Modification in this code should be documented
// 1. In the changelog this should be marked as an INPROBABLE BREAKING CHANGE, since the
hashcode will change
// 2. In the javadoc of EqualsAndHashcode.Include#rank
if (JavaIdentifiers.isPrimitive(typeName)) return 1000;
if (PRIMITIVE_WRAPPER_TYPE_NAME_PATTERN.matcher(typeName).matches()) return 800;
return 0;
}
private static final Pattern SECTION_FINDER =
Pattern.compile("^\\s*\\**\\s*[-*][-*]+\\s*([GS]ETTER|WITH(?:ER)?)\\s*[-*][-*]+\\s*\\**\\s*$",
Pattern.MULTILINE | Pattern.CASE_INSENSITIVE);
private static final Pattern LINE_BREAK_FINDER = Pattern.compile("(\\r?\\n)?");
public enum JavadocTag {
PARAM("@param(?:eter)?"),
RETURN("@returns?");
private Pattern pattern;
```

```
JavadocTag(String regexpFragment) {
pattern = Pattern.compile("\\s?^[ \\t]*\\**[ \\t]*" + regexpFragment + "(\\S|\\s)*?(?=(\\s^\\s*\\**\\s*@|\\Z))",
Pattern.MULTILINE | Pattern.CASE_INSENSITIVE);
}
}
public static String stripLinesWithTagFromJavadoc(String javadoc, JavadocTag... tags) {
if (javadoc == null || javadoc.isEmpty()) return javadoc;
String result = javadoc;
for (JavadocTag tag : tags) {
result = tag.pattern.matcher(result).replaceAll("").trim();
}
return result;
}
public static String stripSectionsFromJavadoc(String javadoc) {
if (javadoc == null || javadoc.isEmpty()) return javadoc;
Matcher sectionMatcher = SECTION_FINDER.matcher(javadoc);
if (!sectionMatcher.find()) return javadoc;
return javadoc.substring(0, sectionMatcher.start());
}
public static String getJavadocSection(String javadoc, String sectionNameSpec) {
if (javadoc == null || javadoc.isEmpty()) return null;
String[] sectionNames = sectionNameSpec.split("\\|");
Matcher sectionMatcher = SECTION_FINDER.matcher(javadoc);
Matcher lineBreakMatcher = LINE_BREAK_FINDER.matcher(javadoc);
int sectionStart = -1;
int sectionEnd = -1;
while (sectionMatcher.find()) {
boolean found = false;
for (String sectionName : sectionNames) if (sectionMatcher.group(1).equalsIgnoreCase(sectionName))
{
found = true;
break;
}
if (found) {
lineBreakMatcher.find(sectionMatcher.end());
sectionStart = lineBreakMatcher.end();
} else if (sectionStart != -1) {
sectionEnd = sectionMatcher.start();
}
}
if (sectionStart != -1) {
if (sectionEnd != -1) return javadoc.substring(sectionStart, sectionEnd);
return javadoc.substring(sectionStart);
}
return null;
}
private static final Pattern FIND_RETURN = Pattern.compile("^\\s*\\**\\s*@returns?\\s+.*$",
Pattern.MULTILINE | Pattern.CASE_INSENSITIVE);
public static String addReturnsThisIfNeeded(String in) {
if (in != null && FIND_RETURN.matcher(in).find()) return in;
return addJavadocLine(in, "@return {@code this}.");
}
public static String addReturnsUpdatedSelfIfNeeded(String in) {
```

```
if (in != null && FIND_RETURN.matcher(in).find()) return in;
return addJavadocLine(in, "@return a clone of this object, except with this updated property (returns
{@code this} if an identical value is passed).");
}
public static String addJavadocLine(String in, String line) {
if (in == null) return line;
if (in.endsWith("\n")) return in + line;
return in + "\n" + line;
}
public static String getParamJavadoc(String methodComment, String param) {
if (methodComment == null || methodComment.isEmpty()) return methodComment;
Pattern pattern = Pattern.compile("@param " + param + " (\\S|\\s)+?(?=^ ?@|\\z)", Pattern.MULTILINE |
Pattern.CASE_INSENSITIVE);
Matcher matcher = pattern.matcher(methodComment);
if (matcher.find()) {
return matcher.group();
}
return null;
}
public static String getConstructorJavadocHeader(String typeName) {
return "Creates a new {@code " + typeName + "} instance.\n\n";
}
public static String getConstructorParameterJavadoc(String paramName, String fieldJavadoc) {
String fieldBaseJavadoc = stripSectionsFromJavadoc(fieldJavadoc);
String paramJavadoc = getParamJavadoc(fieldBaseJavadoc, paramName);
if (paramJavadoc != null) {
return paramJavadoc;
}
String javadocWithoutTags = stripLinesWithTagFromJavadoc(fieldBaseJavadoc, JavadocTag.PARAM,
JavadocTag.RETURN);
if (javadocWithoutTags != null) {
return "@param " + paramName + " " + javadocWithoutTags;
}
return null;
}
}
```

lombok/core/handlers/InclusionExclusionUtils.java


```
/*
* Copyright (C) 2009-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
```

```
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.handlers;
import java.lang.annotation.Annotation;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.Iterator;
import java.util.List;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.core.AST;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.LombokNode;
public class InclusionExclusionUtils {
private static List<Integer> createListOfNonExistentFields(List<String> list, LombokNode<?, ?, ?> type,
boolean excludeStandard, boolean excludeTransient) {
boolean[] matched = new boolean[list.size()];
for (LombokNode<?, ?, ?> child : type.down()) {
if (list.isEmpty()) break;
if (child.getKind() != Kind.FIELD) continue;
if (excludeStandard) {
if (child.isStatic()) continue;
if (child.getName().startsWith("$")) continue;
}
if (excludeTransient && child.isTransient()) continue;
int idx = list.indexOf(child.getName());
if (idx > -1) matched[idx] = true;
}
List<Integer> problematic = new ArrayList<Integer>();
for (int i = 0 ; i < list.size() ; i++) if (!matched[i]) problematic.add(i);
return problematic;
}
public static void checkForBogusFieldNames(LombokNode<?, ?, ?> type, AnnotationValues<?>
annotation, List<String> excludes, List<String> includes) {
if (excludes != null && !excludes.isEmpty()) {
for (int i : createListOfNonExistentFields(excludes, type, true, false)) {
if (annotation != null) annotation.setWarning("exclude", "This field does not exist, or would have been
excluded anyway.", i);
}
}
if (includes != null && !includes.isEmpty()) {
for (int i : createListOfNonExistentFields(includes, type, false, false)) {
if (annotation != null) annotation.setWarning("of", "This field does not exist.", i);
}
```

```
}
}
public static class Included<L, I extends Annotation> {
private final L node;
private final I inc;
private final boolean defaultInclude;
private final boolean explicitRank;
public Included(L node, I inc, boolean defaultInclude, boolean explicitRank) {
this.node = node;
this.inc = inc;
this.defaultInclude = defaultInclude;
this.explicitRank = explicitRank;
}
public L getNode() {
return node;
}
public I getInc() {
return inc;
}
public boolean isDefaultInclude() {
return defaultInclude;
}
public boolean hasExplicitRank() {
return explicitRank;
}
}
private static String innerAnnName(Class<? extends Annotation> type) {
String name = type.getSimpleName();
Class<?> c = type.getEnclosingClass();
while (c != null) {
name = c.getSimpleName() + "." + name;
c = c.getEnclosingClass();
}
return name;
}
private static <A extends AST<A, L, N>, L extends LombokNode<A, L, N>, N, I extends Annotation>
List<Included<L, I>> handleIncludeExcludeMarking(Class<I> inclType, String replaceName, Class<?
extends Annotation> exclType, LombokNode<A, L, N> typeNode, AnnotationValues<?> annotation,
LombokNode<A, L, N> annotationNode, boolean includeTransient) {
boolean onlyExplicitlyIncluded = annotation != null ? annotation.getAsBoolean("onlyExplicitlyIncluded")
: false;
return handleIncludeExcludeMarking(inclType, onlyExplicitlyIncluded, replaceName, exclType,
typeNode, annotation, annotationNode, includeTransient);
}
private static <A extends AST<A, L, N>, L extends LombokNode<A, L, N>, N, I extends Annotation>
List<Included<L, I>> handleIncludeExcludeMarking(Class<I> inclType, boolean onlyExplicitlyIncluded,
String replaceName, Class<? extends Annotation> exclType, LombokNode<A, L, N> typeNode,
AnnotationValues<?> annotation, LombokNode<A, L, N> annotationNode, boolean includeTransient) {
List<String> oldExcludes = (annotation != null && annotation.isExplicit("exclude")) ?
annotation.getAsStringList("exclude") : null;
List<String> oldIncludes = (annotation != null && annotation.isExplicit("of")) ?
annotation.getAsStringList("of") : null;
boolean memberAnnotationMode = onlyExplicitlyIncluded;
```

```
List<Included<L, I>> members = new ArrayList<Included<L, I>>();
List<String> namesToAutoExclude = new ArrayList<String>();
if (typeNode == null || typeNode.getKind() != Kind.TYPE) return null;
checkForBogusFieldNames(typeNode, annotation, oldExcludes, oldIncludes);
if (oldExcludes != null && oldIncludes != null) {
oldExcludes = null;
if (annotation != null) annotation.setWarning("exclude", "exclude and of are mutually exclusive; the
'exclude' parameter will be ignored.");
}
for (L child : typeNode.down()) {
boolean markExclude = child.getKind() == Kind.FIELD && child.hasAnnotation(exclType);
AnnotationValues<I> markInclude = null;
if (child.getKind() == Kind.FIELD || child.getKind() == Kind.METHOD) markInclude =
child.findAnnotation(inclType);
if (markExclude || markInclude != null) memberAnnotationMode = true;
if (markInclude != null && markExclude) {
child.addError("@" + innerAnnName(exclType) + " and @" + innerAnnName(inclType) + " are mutually
exclusive; the @Include annotation will be ignored");
markInclude = null;
}
String name = child.getName();
if (markExclude) {
if (onlyExplicitlyIncluded) {
child.addWarning("The @Exclude annotation is not needed; 'onlyExplicitlyIncluded' is set, so this
member would be excluded anyway");
} else if (child.isStatic()) {
child.addWarning("The @Exclude annotation is not needed; static fields aren't included anyway");
} else if (name.startsWith("$")) {
child.addWarning("The @Exclude annotation is not needed; fields that start with $ aren't included
anyway");
}
continue;
}
if (oldExcludes != null && oldExcludes.contains(name)) continue;
if (markInclude != null) {
I inc = markInclude.getInstance();
if (child.getKind() == Kind.METHOD) {
if (child.countMethodParameters() > 0) {
child.addError("Methods included with @" + innerAnnName(inclType) + " must have no arguments; it
will not be included");
continue;
}
String n = replaceName != null ? markInclude.getAsString(replaceName) : "";
if (n.isEmpty()) n = name;
namesToAutoExclude.add(n);
}
members.add(new Included<L, I>(child, inc, false, markInclude.isExplicit("rank")));
continue;
}
if (onlyExplicitlyIncluded) continue;
if (oldIncludes != null) {
if (child.getKind() == Kind.FIELD && oldIncludes.contains(name)) members.add(new Included<L,
I>(child, null, false, false));
```

```
continue;
}
if (child.getKind() != Kind.FIELD) continue;
if (child.isStatic()) continue;
if (child.isTransient() && !includeTransient) continue;
if (name.startsWith("$")) continue;
if (child.isEnumMember()) continue;
members.add(new Included<L, I>(child, null, true, false));
}
/* delete default-included fields with the same name as an explicit inclusion */ {
Iterator<Included<L, I>> it = members.iterator();
while (it.hasNext()) {
Included<L, I> m = it.next();
if (m.isDefaultInclude() && namesToAutoExclude.contains(m.getNode().getName())) it.remove();
}
}
if (annotation == null || !annotation.isExplicit("exclude")) oldExcludes = null;
if (annotation == null || !annotation.isExplicit("of")) oldIncludes = null;
if (memberAnnotationMode && (oldExcludes != null || oldIncludes != null)) {
annotationNode.addError("The old-style 'exclude/of' parameter cannot be used together with the
new-style @Include / @Exclude annotations.");
return null;
}
return members;
}
public static <A extends AST<A, L, N>, L extends LombokNode<A, L, N>, N> List<Included<L,
ToString.Include>> handleToStringMarking(LombokNode<A, L, N> typeNode, boolean
onlyExplicitlyIncluded, AnnotationValues<ToString> annotation, LombokNode<A, L, N>
annotationNode) {
List<Included<L, ToString.Include>> members = handleIncludeExcludeMarking(ToString.Include.class,
onlyExplicitlyIncluded, "name", ToString.Exclude.class, typeNode, annotation, annotationNode, true);
Collections.sort(members, new Comparator<Included<L, ToString.Include>>() {
@Override public int compare(Included<L, ToString.Include> a, Included<L, ToString.Include> b) {
int ra = a.getInc() == null ? 0 : a.getInc().rank();
int rb = b.getInc() == null ? 0 : b.getInc().rank();
return compareRankOrPosition(ra, rb, a.getNode(), b.getNode());
}
});
return members;
}
public static <A extends AST<A, L, N>, L extends LombokNode<A, L, N>, N> List<Included<L,
EqualsAndHashCode.Include>> handleEqualsAndHashCodeMarking(LombokNode<A, L, N>
typeNode, AnnotationValues<EqualsAndHashCode> annotation, LombokNode<A, L, N>
annotationNode) {
List<Included<L, EqualsAndHashCode.Include>> members =
handleIncludeExcludeMarking(EqualsAndHashCode.Include.class, "replaces",
EqualsAndHashCode.Exclude.class, typeNode, annotation, annotationNode, false);
Collections.sort(members, new Comparator<Included<L, EqualsAndHashCode.Include>>() {
@Override public int compare(Included<L, EqualsAndHashCode.Include> a, Included<L,
EqualsAndHashCode.Include> b) {
int ra = a.hasExplicitRank() ? a.getInc().rank() :
HandlerUtil.defaultEqualsAndHashcodeIncludeRank(a.node.fieldOrMethodBaseType());
```

```
int rb = b.hasExplicitRank() ? b.getInc().rank() :
HandlerUtil.defaultEqualsAndHashcodeIncludeRank(b.node.fieldOrMethodBaseType());
return compareRankOrPosition(ra, rb, a.getNode(), b.getNode());
}
});
return members;
}
private static <A extends AST<A, L, N>, L extends LombokNode<A, L, N>, N> int
compareRankOrPosition(int ra, int rb, LombokNode<A, L, N> nodeA, LombokNode<A, L, N> nodeB) {
if (ra < rb) return +1;
if (ra > rb) return -1;
int pa = nodeA.getStartPos();
int pb = nodeB.getStartPos();
if (pa < pb) return -1;
if (pa > pb) return +1;
return 0;
}
}
```

lombok/core/handlers/LoggingFramework.java

```
/*
* Copyright (C) 2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.handlers;
import java.lang.annotation.Annotation;
import lombok.core.configuration.LogDeclaration;
public class LoggingFramework {
// private static final org.apache.commons.logging.Log log =
org.apache.commons.logging.LogFactory.getLog(TargetType.class);
public static final LoggingFramework COMMONS = new LoggingFramework(
lombok.extern.apachecommons.CommonsLog.class,
```

```
LogDeclaration.valueOf("org.apache.commons.logging.Log
org.apache.commons.logging.LogFactory.getLog(TYPE)(TOPIC)")
);
// private static final java.util.logging.Logger log =
java.util.logging.Logger.getLogger(TargetType.class.getName());
public static final LoggingFramework JUL = new LoggingFramework(
lombok.extern.java.Log.class,
LogDeclaration.valueOf("java.util.logging.Logger java.util.logging.Logger.getLogger(NAME)(TOPIC)")
);
// private static final org.apache.log4j.Logger log =
org.apache.log4j.Logger.getLogger(TargetType.class);
public static final LoggingFramework LOG4J = new LoggingFramework(
lombok.extern.log4j.Log4j.class,
LogDeclaration.valueOf("org.apache.log4j.Logger org.apache.log4j.Logger.getLogger(TYPE)(TOPIC)")
);
// private static final org.apache.logging.log4j.Logger log =
org.apache.logging.log4j.LogManager.getLogger(TargetType.class);
public static final LoggingFramework LOG4J2 = new LoggingFramework(
lombok.extern.log4j.Log4j2.class,
LogDeclaration.valueOf("org.apache.logging.log4j.Logger
org.apache.logging.log4j.LogManager.getLogger(TYPE)(TOPIC)")
);
// private static final org.slf4j.Logger log = org.slf4j.LoggerFactory.getLogger(TargetType.class);
public static final LoggingFramework SLF4J = new LoggingFramework(
lombok.extern.slf4j.Slf4j.class,
LogDeclaration.valueOf("org.slf4j.Logger org.slf4j.LoggerFactory.getLogger(TYPE)(TOPIC)")
);
// private static final org.slf4j.ext.XLogger log =
org.slf4j.ext.XLoggerFactory.getXLogger(TargetType.class);
public static final LoggingFramework XSLF4J = new LoggingFramework(
lombok.extern.slf4j.XSlf4j.class,
LogDeclaration.valueOf("org.slf4j.ext.XLogger
org.slf4j.ext.XLoggerFactory.getXLogger(TYPE)(TOPIC)")
);
// private static final org.jboss.logging.Logger log =
org.jboss.logging.Logger.getLogger(TargetType.class);
public static final LoggingFramework JBOSSLOG = new LoggingFramework(
lombok.extern.jbosslog.JBossLog.class,
LogDeclaration.valueOf("org.jboss.logging.Logger
org.jboss.logging.Logger.getLogger(TYPE)(TOPIC)")
);
// private static final com.google.common.flogger.FluentLogger log =
com.google.common.flogger.FluentLogger.forEnclosingClass();
public static final LoggingFramework FLOGGER = new LoggingFramework(
lombok.extern.flogger.Flogger.class,
LogDeclaration.valueOf("com.google.common.flogger.FluentLogger
com.google.common.flogger.FluentLogger.forEnclosingClass()")
);
private final Class<? extends Annotation> annotationClass;
private final String annotationAsString;
private final LogDeclaration declaration;
public LoggingFramework(Class<? extends Annotation> annotationClass, LogDeclaration declaration) {
this.annotationClass = annotationClass;
```

```
this.annotationAsString = "@" + annotationClass.getSimpleName();
this.declaration = declaration;
}
public Class<? extends Annotation> getAnnotationClass() {
return annotationClass;
}
public String getAnnotationAsString() {
return annotationAsString;
}
public LogDeclaration getDeclaration() {
return declaration;
}
}
```

lombok/core/handlers/Singulars.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.handlers;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.List;
public class Singulars {
private static final List<String> SINGULAR_STORE; // intended to be immutable.
static {
SINGULAR_STORE = new ArrayList<String>();
try {
InputStream in = Singulars.class.getResourceAsStream("singulars.txt");
try {
```

```
BufferedReader br = new BufferedReader(new InputStreamReader(in, "UTF-8"));
for (String line = br.readLine(); line != null; line = br.readLine()) {
line = line.trim();
if (line.startsWith("#") || line.isEmpty()) continue;
if (line.endsWith(" =")) {
SINGULAR_STORE.add(line.substring(0, line.length() - 2));
SINGULAR_STORE.add("");
continue;
}
int idx = line.indexOf(" = ");
SINGULAR_STORE.add(line.substring(0, idx));
SINGULAR_STORE.add(line.substring(idx + 3));
}
} finally {
try {
in.close();
} catch (Throwable ignore) {}
}
} catch (IOException e) {
SINGULAR_STORE.clear();
}
}
public static String autoSingularize(String in) {
final int inLen = in.length();
for (int i = 0; i < SINGULAR_STORE.size(); i+= 2) {
final String lastPart = SINGULAR_STORE.get(i);
final boolean wholeWord = Character.isUpperCase(lastPart.charAt(0));
final int endingOnly = lastPart.charAt(0) == '-' ? 1 : 0;
final int len = lastPart.length();
if (inLen < len) continue;
if (!in.regionMatches(true, inLen - len + endingOnly, lastPart, endingOnly, len - endingOnly)) continue;
if (wholeWord && inLen != len && !Character.isUpperCase(in.charAt(inLen - len))) continue;
String replacement = SINGULAR_STORE.get(i + 1);
if (replacement.equals("!")) return null;
boolean capitalizeFirst = !replacement.isEmpty() && Character.isUpperCase(in.charAt(inLen - len +
endingOnly));
String pre = in.substring(0, inLen - len + endingOnly);
String post = capitalizeFirst ? Character.toUpperCase(replacement.charAt(0)) +
replacement.substring(1) : replacement;
return pre + post;
}
return null;
}
}
```

lombok/core/handlers/SneakyThrowsAndCleanupDependencyInfo.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
```

```
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.handlers;
import java.util.Arrays;
import java.util.List;
import lombok.core.runtimeDependencies.RuntimeDependencyInfo;
import lombok.spi.Provides;
@Provides
public class SneakyThrowsAndCleanupDependencyInfo implements RuntimeDependencyInfo {
@Override public List<String> getRuntimeDependencies() {
return Arrays.asList(
"lombok/Lombok.class"
);
}
@Override public List<String> getRuntimeDependentsDescriptions() {
return Arrays.asList(
"@SneakyThrows (only when delomboking - using @SneakyThrows in code that is compiled with lombok on the classpath does not create the dependency)",
"@Cleanup (only when delomboking - using @Cleanup in code that is compiled with lombok on the classpath does not create the dependency)"
);
}
}
```

lombok/core/handlers/package-info.java

```
/*
* Copyright (C) 2009-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
```

* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* This package contains utility methods and classes shared between javac-specific feature implementations
* and eclipse-specific feature implementations.
*
* NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.
*/
package lombok.core.handlers;

lombok/core/package-info.java

/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* Contains the platform-agnostic core of lombok.
* Includes the lombok AST superclasses, annotation introspection support,
* an implementation of SPI service loader (to avoid being dependent on a v1.6 JVM),
* lombok's version, and annotations and support classes for your normal java code

* that's primarily useful for developing and debugging lombok.
*
* NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.
*/
package lombok.core;

lombok/core/runtimeDependencies/CreateLombokRuntimeApp.java

/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.runtimeDependencies;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.io.PrintStream;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.jar.JarOutputStream;
import java.util.zip.ZipEntry;
import lombok.core.LombokApp;
import lombok.core.SpiLoadUtil;
import lombok.spi.Provides;
import com.zwitserloot.cmdreader.CmdReader;

```
import com.zwitserloot.cmdreader.Description;
import com.zwitserloot.cmdreader.InvalidCommandLineException;
import com.zwitserloot.cmdreader.Mandatory;
import com.zwitserloot.cmdreader.Requires;
import com.zwitserloot.cmdreader.Shorthand;
@Provides
public class CreateLombokRuntimeApp extends LombokApp {
private List<RuntimeDependencyInfo> infoObjects;
@Override public String getAppName() {
return "createRuntime";
}
@Override public String getAppDescription() {
return "Creates a small lombok-runtime.jar with the runtime\n" +
"dependencies of all lombok transformations that have them,\n" +
"and prints the names of each lombok transformation that\n" +
"requires the lombok-runtime.jar at runtime.";
}
@Override public List<String> getAppAliases() {
return Arrays.asList("runtime");
}
private static class CmdArgs {
@Shorthand("p")
@Description("Prints those lombok transformations that require lombok-runtime.jar.")
@Mandatory(onlyIfNot="create")
boolean print;
@Shorthand("c")
@Description("Creates the lombok-runtime.jar.")
@Mandatory(onlyIfNot="print")
boolean create;
@Shorthand("o")
@Description("Where to write the lombok-runtime.jar. Defaults to the current working directory.")
@Requires("create")
String output;
@Description("Shows this help text")
boolean help;
}
@Override public int runApp(List<String> rawArgs) throws Exception {
CmdReader<CmdArgs> reader = CmdReader.of(CmdArgs.class);
CmdArgs args;
try {
args = reader.make(rawArgs.toArray(new String[0]));
} catch (InvalidCommandLineException e) {
printHelp(reader, e.getMessage(), System.err);
return 1;
}
if (args.help) {
printHelp(reader, null, System.out);
return 0;
}
initializeInfoObjects();
if (args.print) {
printRuntimeDependents();
}
```

```
int errCode = 0;
if (args.create) {
File out = new File("./lombok-runtime.jar");
if (args.output != null) {
out = new File(args.output);
if (out.isDirectory()) out = new File(out, "lombok-runtime.jar");
}
try {
errCode = writeRuntimeJar(out);
} catch (Exception e) {
System.err.println("ERROR: Creating " + canonical(out) + " failed: ");
e.printStackTrace();
return 1;
}
}
return errCode;
}
private void printRuntimeDependents() {
List<String> descriptions = new ArrayList<String>();
for (RuntimeDependencyInfo info : infoObjects)
descriptions.addAll(info.getRuntimeDependentsDescriptions());
if (descriptions.isEmpty()) {
System.out.println("Not printing dependents: No lombok transformations currently have any runtime
dependencies!");
} else {
System.out.println("Using any of these lombok features means your app will need
lombok-runtime.jar:");
for (String desc : descriptions) {
System.out.println(desc);
}
}
}
private int writeRuntimeJar(File outFile) throws Exception {
Map<String, Class<?>> deps = new LinkedHashMap<String, Class<?>>();
for (RuntimeDependencyInfo info : infoObjects) {
List<String> depNames = info.getRuntimeDependencies();
if (depNames != null) for (String depName : depNames) {
if (!deps.containsKey(depName)) deps.put(depName, info.getClass());
}
}
if (deps.isEmpty()) {
System.out.println("Not generating lombok-runtime.jar: No lombok transformations currently have any
runtime dependencies!");
return 1;
}
OutputStream out = new FileOutputStream(outFile);
boolean success = false;
try {
JarOutputStream jar = new JarOutputStream(out);
deps.put("LICENSE", CreateLombokRuntimeApp.class);
deps.put("AUTHORS", CreateLombokRuntimeApp.class);
for (Entry<String, Class<?>> dep : deps.entrySet()) {
InputStream in = dep.getValue().getResourceAsStream("/" + dep.getKey());
```

```
try {
if (in == null) {
throw new Fail(String.format("Dependency %s contributed by %s cannot be found", dep.getKey(),
dep.getValue()));
}
writeIntoJar(jar, dep.getKey(), in);
} finally {
if (in != null) in.close();
}
}
jar.close();
out.close();
System.out.println("Successfully created: " + canonical(outFile));
return 0;
} catch (Throwable t) {
try { out.close();} catch (Throwable ignore) {}
if (!success) outFile.delete();
if (t instanceof Fail) {
System.err.println(t.getMessage());
return 1;
} else if (t instanceof Exception) {
throw (Exception)t;
} else if (t instanceof Error) {
throw (Error)t;
} else {
throw new Exception(t);
}
}
}
private void writeIntoJar(JarOutputStream jar, String depName, InputStream in) throws IOException {
jar.putNextEntry(new ZipEntry(depName));
byte[] b = new byte[65536];
while (true) {
int r = in.read(b);
if (r == -1) break;
jar.write(b, 0, r);
}
jar.closeEntry();
in.close();
}
private static class Fail extends Exception {
Fail(String message) {
super(message);
}
}
private void initializeInfoObjects() throws IOException {
infoObjects = SpiLoadUtil.readAllFromIterator(
SpiLoadUtil.findServices(RuntimeDependencyInfo.class));
}
private static String canonical(File out) {
try {
return out.getCanonicalPath();
} catch (Exception e) {
```

```
return out.getAbsolutePath();
}
}
private void printHelp(CmdReader<CmdArgs> reader, String message, PrintStream out) {
if (message != null) {
out.println(message);
out.println("----------------------------");
}
out.println(reader.generateCommandLineHelp("java -jar lombok.jar createRuntime"));
}
}
```

lombok/core/runtimeDependencies/RuntimeDependencyInfo.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.runtimeDependencies;
import java.util.List;
/**
* Implement and provide this interface to specify which transformations have a runtime dependency on
* {@code lombok-runtime.jar}, as well as which files of your transformation must be in {@code
lombok-runtime.jar}.
*/
public interface RuntimeDependencyInfo {
/**
* @return A list of strings describing each lombok transformation that has a runtime dependency.
*/
public List<String> getRuntimeDependentsDescriptions();
/**
```

* @return A list of files (findable via {@code yourClass.getResourceAsStream("/" + NAME)}) to include in
* {@code lombok-runtime.jar}.
*/
public List<String> getRuntimeDependencies();
}

lombok/core/runtimeDependencies/package-info.java

/*
* Copyright (C) 2009-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* This package is the basis for lombok's (the jar, run as an application) ability to spin off
* a clone of itself that only contains files required as a runtime dependency.
*
* This feature was used for a short while to support {@code @SneakyThrows}, but this is no longer
* necessary. Currently no lombok features have any such dependencies though at some point we may
* reintroduce the concept, for example to support properties.
*
* It is possible we'll use a different mechanism at that point; use the infrastructure in this package
* with knowledge that it may be eliminated at any time.
*/
package lombok.core.runtimeDependencies;

lombok/eclipse/DeferUntilPostDiet.java

/*
* Copyright (C) 2012 The Project Lombok Authors.
*

* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.eclipse;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Mark a handler class with this annotation to indicate that this handler should not be run in the diet parse phase.
* 'diet parse' is where method bodies aren't filled in yet. If you have a method-level annotation that modifies the contents of that method,
* you need to put this annotation on your handler. Otherwise, do not put this annotation on your handler.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface DeferUntilPostDiet {
}
```

lombok/eclipse/EcjAugments.java

/*
* Copyright (C) 2014-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentMap;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.env.ICompilationUnit;
import org.eclipse.jdt.internal.core.CompilationUnit;
import org.eclipse.jdt.internal.core.SourceMethod;
import lombok.core.FieldAugment;
public final class EcjAugments {
private EcjAugments() {
// Prevent instantiation
}
public static final FieldAugment<FieldDeclaration, Boolean> FieldDeclaration_booleanLazyGetter =
FieldAugment.augment(FieldDeclaration.class, boolean.class, "lombok$booleanLazyGetter");
public static final FieldAugment<ASTNode, Boolean> ASTNode_handled =
FieldAugment.augment(ASTNode.class, boolean.class, "lombok$handled");
public static final FieldAugment<ASTNode, ASTNode> ASTNode_generatedBy =
FieldAugment.augment(ASTNode.class, ASTNode.class, "$generatedBy");
public static final FieldAugment<Annotation, Boolean> Annotation_applied =
FieldAugment.augment(Annotation.class, boolean.class, "lombok$applied");
public static final FieldAugment<ICompilationUnit, Map<String, String>> CompilationUnit_javadoc =
FieldAugment.augment(ICompilationUnit.class, Map.class, "$javadoc");
public static final FieldAugment<CompilationUnitDeclaration, TransformationState>
CompilationUnitDeclaration_transformationState =
FieldAugment.augment(CompilationUnitDeclaration.class, Object.class, "$transformationState");
public static final class EclipseAugments {
private EclipseAugments() {
// Prevent instantiation
}
public static final FieldAugment<CompilationUnit, ConcurrentMap<String, List<SourceMethod>>>
CompilationUnit_delegateMethods = FieldAugment.augment(CompilationUnit.class,
ConcurrentMap.class, "$delegateMethods");
}
}
```

lombok/eclipse/EclipseAST.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
```

*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import java.io.File;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.net.URI;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import lombok.core.AST;
import lombok.core.LombokImmutableList;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import lombok.permit.Permit;
import org.eclipse.core.resources.ResourcesPlugin;
import org.eclipse.core.runtime.Path;
import org.eclipse.jdt.internal.compiler.CompilationResult;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ImportReference;
import org.eclipse.jdt.internal.compiler.ast.Initializer;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.Wildcard;
/**

```
* Wraps around Eclipse's internal AST view to add useful features as well as the ability to visit parents from children,
* something Eclipse own AST system does not offer.
*/
public class EclipseAST extends AST<EclipseAST, EclipseNode, ASTNode> {
/**
* Creates a new EclipseAST of the provided Compilation Unit.
*
* @param ast The compilation unit, which serves as the top level node in the tree to be built.
*/
public EclipseAST(CompilationUnitDeclaration ast) {
super(toFileName(ast), packageDeclaration(ast), new EclipseImportList(ast), statementTypes());
this.compilationUnitDeclaration = ast;
setTop(buildCompilationUnit(ast));
this.completeParse = isComplete(ast);
clearChanged();
}
private static volatile boolean skipEclipseWorkspaceBasedFileResolver = false;
private static final URI NOT_CALCULATED_MARKER =
URI.create("https://projectlombok.org/not/calculated");
private URI memoizedAbsoluteFileLocation = NOT_CALCULATED_MARKER;
public static URI getAbsoluteFileLocation(CompilationUnitDeclaration ast) {
return getAbsoluteFileLocation0(ast);
}
public URI getAbsoluteFileLocation() {
if (memoizedAbsoluteFileLocation != NOT_CALCULATED_MARKER) return
memoizedAbsoluteFileLocation;
memoizedAbsoluteFileLocation = getAbsoluteFileLocation0(this.compilationUnitDeclaration);
return memoizedAbsoluteFileLocation;
}
/** This is the call, but we wrapped it to memoize this. */
private static URI getAbsoluteFileLocation0(CompilationUnitDeclaration ast) {
String fileName = toFileName(ast);
if (fileName != null && (fileName.startsWith("file:") || fileName.startsWith("sourcecontrol:"))) {
// Some exotic build systems get real fancy with filenames. Known culprits:
// The 'jazz' source control system _probably_ (not confirmed yet) uses sourcecontrol://jazz: urls.
// GWT puts file:/D:/etc/etc/etc/Foo.java in here.
return URI.create(fileName);
}
// state of the research in this:
// * We need an abstraction of a 'directory level'. This abstraction needs 'read()' which returns a string (content of lombok.config) and 'getParent()'.
// * sometimes, cud.compilationResult.compilationUnit is an 'openable', you can chase this down to end up with a path, you can jigger this into being the sibling 'lombok.config', and then use:
// InputStream in = ResourcesPlugin.getWorkspace().getRoot().getFile(ResourcesPlugin.getWorkspac
e().getRoot().getFile(new Path(x)).getFullPath()).getContents(true);
// to read out this data. Our theory is that this will work even with very crazy virtual filesystems such as sourcecontrol://jazz/blabla.
// * With jazz and other creative file backed systems, is there even a 'project root' concept? Surely there won't be a 'workspace root' concept so how do we abstract the idea that, from jazz://whatever/projectroot, the parent is c:\myWorkspace?
// * Check the .getAlternateAbsolutePath() impl which has the research done so far.
```

```
// * VIRTUAL FILES: Sometimes virtual files are created; their location tends to be /FileName.java which cannot be resolved. Optimally speaking we should find the 'source' of the virtual code and use IT for determining lombok.config, but that may not be feasible. If not, can we get at project or at least workspace?
// * Either way there are sufficiently many WTF situations, that in case of error, as painful as this is, we should just carry on and not apply lombok.config, though at least if we don't recognize the scenario we should write a log file imploring the user to send us a bunch of feedback on the situation.
// Relevant issues: Comment 2 on #683, all of #682
if (!skipEclipseWorkspaceBasedFileResolver) {
// if (Boolean.FALSE) throw new IllegalArgumentException("Here's the alt strat result: " + getAlternativeAbsolutePathDEBUG());
try {
/*if (fileName.startsWith("/") && fileName.indexOf('/', 1) > -1) */
try {
return EclipseWorkspaceBasedFileResolver.resolve(fileName);
} catch (IllegalArgumentException e) {
warning("Finding 'lombok.config' file failed for '" + fileName + "'", e);
// String msg = e.getMessage();
// if (msg != null && msg.startsWith("Path must include project and resource name")) {
// // We shouldn't throw an exception at all, but we can't reproduce this so we need help from our users to figure this out.
// // Let's bother them with an error that slows their eclipse down to a crawl and makes it unusable.
// throw new IllegalArgumentException("Path resolution for lombok.config failed. Path: " + fileName + " -- package of this class: " + this.getPackageDeclaration());
// } else throw e;
}
} catch (NoClassDefFoundError e) {
skipEclipseWorkspaceBasedFileResolver = true;
}
}
// Our fancy workspace based source file to absolute disk location algorithm only works in a fully fledged eclipse.
// This fallback works when using 'ecj', which has a much simpler project/path system. For example, no 'linked' resources.
try {
return new File(fileName).getAbsoluteFile().toURI();
} catch (Exception e) {
// This is a temporary workaround while we try and gather all the various exotic shenanigans where lombok.config resolution is not going to work!
return null;
}
}
// /** This is ongoing research for issues with lombok.config resolution. */
// @SuppressWarnings("unused") private String getAlternativeAbsolutePathDEBUG() {
// try {
// ICompilationUnit cu = this.compilationUnitDeclaration.compilationResult.compilationUnit;
//
// if (cu instanceof Openable) {
// String x = ((Openable) cu).getResource().getFullPath().makeAbsolute().toString();
// int lastLoc = x.lastIndexOf('/');
// x = x.substring(0, lastLoc + 1) + "lombok.config";
// URI lombokConfigLoc = ResourcesPlugin.getWorkspace().getRoot().getFile(new Path(x)).getLocationURI();
```

```
// InputStream in = ResourcesPlugin.getWorkspace().getRoot().getFile(ResourcesPlugin.getWorkspac
e().getRoot().getFile(new Path(x)).getFullPath()).getContents(true);
// byte[] b = new byte[100000];
// int p = 0;
// while (true) {
// int r = in.read(b, p, b.length - p);
// if (r == -1) break;
// p += r;
// }
// in.close();
// return "(Contents of lombok.config: " + new String(b, 0, p, "UTF-8");
//
//// return "(alt strategy result C: '" + ((Openable)
cu).getResource().getFullPath().makeAbsolute().toString() + "'): resolved: " +
EclipseWorkspaceBasedFileResolver.resolve(((Openable)
cu).getResource().getFullPath().makeAbsolute().toString());
// }
// if (cu instanceof SourceFile) {
// String cuFileName = new String(((SourceFile) cu).getFileName());
// String cuIFilePath = ((SourceFile) cu).resource.getFullPath().toString();
// return "(alt strategy result A: \"" + cuFileName + "\" B: \"" + cuIFilePath + "\")";
// }
// return "(alt strategy failed: cu isn't a SourceFile or Openable but a " + cu.getClass() + ")";
// } catch (Exception e) {
// return "(alt strategy failed: " + e + ")";
// }
// }
private static class EclipseWorkspaceBasedFileResolver {
public static URI resolve(String path) {
/* eclipse issue: When creating snippets, for example to calculate 'find callers', refactor scripts, save
actions, etc,
* eclipse creates a psuedo-file whose path is simply "/SimpleName.java", which cannot be turned back
into a real location.
* What we really need to do is find out which file is the source of this script job and use its directory
instead. For now,
* we just go with all defaults; these operations are often not sensitive to proper lomboking or aren't even
lomboked at all.
*
* Reliable way to reproduce this (Kepler, possibly with JDK8 beta support):
* * Have a method, called once by some code in another class.
* * Refactor it with the 'change method signature' refactor script, and add a parameter and hit 'ok'.
*/
if (path == null || path.indexOf('/', 1) == -1) {
return null;
}
try {
return ResourcesPlugin.getWorkspace().getRoot().getFile(new Path(path)).getLocationURI();
} catch (Exception e) {
// One of the exceptions that can occur is IllegalStateException (during getWorkspace())
// if you try to run this while eclipse is shutting down.
return null;
}
}
```

```
}
private static String packageDeclaration(CompilationUnitDeclaration cud) {
ImportReference pkg = cud.currentPackage;
return pkg == null ? null : Eclipse.toQualifiedName(pkg.getImportName());
}
@Override public int getSourceVersion() {
long sl = compilationUnitDeclaration.problemReporter.options.sourceLevel;
long cl = compilationUnitDeclaration.problemReporter.options.complianceLevel;
sl >>= 16;
cl >>= 16;
if (sl == 0) sl = cl;
if (cl == 0) cl = sl;
return Math.min((int)(sl - 44), (int)(cl - 44));
}
@Override public int getLatestJavaSpecSupported() {
return Eclipse.getEcjCompilerVersion();
}
/**
* Runs through the entire AST, starting at the compilation unit, calling the provided visitor's visit
methods
* for each node, depth first.
*/
public void traverse(EclipseASTVisitor visitor) {
top().traverse(visitor);
}
void traverseChildren(EclipseASTVisitor visitor, EclipseNode node) {
LombokImmutableList<EclipseNode> children = node.down();
int len = children.size();
for (int i = 0; i < len; i++) {
children.get(i).traverse(visitor);
}
}
public void setSource(char[] source) {
this.source = source;
}
public char[] getSource() {
return source;
}
/**
* Eclipse starts off with a 'diet' parse which leaves method bodies blank, amongst other shortcuts.
*
* For such diet parses, this method returns false, otherwise it returns true. Any lombok processor
* that needs the contents of methods should just do nothing (and return false so it gets another shot
later!)
* when this is false.
*/
public boolean isCompleteParse() {
return completeParse;
}
class ParseProblem {
final boolean isWarning;
final String message;
final int sourceStart;
```

```
final int sourceEnd;
ParseProblem(boolean isWarning, String message, int sourceStart, int sourceEnd) {
this.isWarning = isWarning;
this.message = message;
this.sourceStart = sourceStart;
this.sourceEnd = sourceEnd;
}
void addToCompilationResult() {
CompilationUnitDeclaration cud = (CompilationUnitDeclaration) top().get();
addProblemToCompilationResult(cud.getFileName(), cud.compilationResult,
isWarning, message, sourceStart, sourceEnd);
}
}
private void propagateProblems() {
if (queuedProblems.isEmpty()) return;
CompilationUnitDeclaration cud = (CompilationUnitDeclaration) top().get();
if (cud.compilationResult == null) return;
for (ParseProblem problem : queuedProblems) problem.addToCompilationResult();
queuedProblems.clear();
}
private final List<ParseProblem> queuedProblems = new ArrayList<ParseProblem>();
void addProblem(ParseProblem problem) {
queuedProblems.add(problem);
propagateProblems();
}
/**
* Adds a problem to the provided CompilationResult object so that it will show up
* in the Problems/Warnings view.
*/
public static void addProblemToCompilationResult(char[] fileNameArray, CompilationResult result,
boolean isWarning, String message, int sourceStart, int sourceEnd) {
Permit.invokeSneaky(EcjReflectionCheck.problemAddProblemToCompilationResult,
EcjReflectionCheck.addProblemToCompilationResult, null, fileNameArray, result, isWarning, message,
sourceStart, sourceEnd);
}
public static Annotation[] getTopLevelTypeReferenceAnnotations(TypeReference tr) {
Method m = EcjReflectionCheck.typeReferenceGetAnnotationsOnDimensions;
if (m == null) return null;
Annotation[][] annss = null;
try {
annss = (Annotation[][]) Permit.invoke(m, tr);
if (annss != null) return annss[0];
} catch (Throwable ignore) {}
try {
Field f = EcjReflectionCheck.typeReferenceAnnotations;
if (f == null) return null;
annss = (Annotation[][]) Permit.get(f, tr);
if (annss == null) return null;
return annss[annss.length - 1];
} catch (Throwable t) {
return null;
}
}
```

```
private final CompilationUnitDeclaration compilationUnitDeclaration;
private char[] source;
private boolean completeParse;
private static String toFileName(CompilationUnitDeclaration ast) {
return ast.compilationResult.fileName == null ? null : new String(ast.compilationResult.fileName);
}
/**
* Call this method to move an EclipseAST generated for a diet parse to rebuild itself for the full parse -
* with filled in method bodies and such. Also propagates problems and errors, which in diet parse
* mode can't be reliably added to the problems/warnings view.
*/
public void rebuild(boolean force) {
propagateProblems();
if (completeParse && !force) return;
boolean changed = isChanged();
boolean newCompleteParse = isComplete(compilationUnitDeclaration);
if (!newCompleteParse && !force) return;
top().rebuild();
this.completeParse = newCompleteParse;
if (!changed) clearChanged();
}
public static boolean isComplete(CompilationUnitDeclaration unit) {
return (unit.bits & ASTNode.HasAllMethodBodies) != 0;
}
/** {@inheritDoc} */
@Override protected EclipseNode buildTree(ASTNode node, Kind kind) {
switch (kind) {
case COMPILATION_UNIT:
return buildCompilationUnit((CompilationUnitDeclaration) node);
case TYPE:
return buildType((TypeDeclaration) node);
case FIELD:
return buildField((FieldDeclaration) node, null);
case INITIALIZER:
return buildInitializer((Initializer) node);
case METHOD:
return buildMethod((AbstractMethodDeclaration) node);
case ARGUMENT:
return buildLocal((Argument) node, kind);
case LOCAL:
return buildLocal((LocalDeclaration) node, kind);
case STATEMENT:
return buildStatement((Statement) node);
case ANNOTATION:
return buildAnnotation((Annotation) node, false);
case TYPE_USE:
return buildTypeUse((TypeReference) node);
default:
throw new AssertionError("Did not expect to arrive here: " + kind);
}
}
private EclipseNode buildCompilationUnit(CompilationUnitDeclaration top) {
if (setAndGetAsHandled(top)) return null;
```

```
List<EclipseNode> children = buildTypes(top.types);
return putInMap(new EclipseNode(this, top, children, Kind.COMPILATION_UNIT));
}
private void addIfNotNull(Collection<EclipseNode> collection, EclipseNode n) {
if (n != null) collection.add(n);
}
private List<EclipseNode> buildTypes(TypeDeclaration[] children) {
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
if (children != null) for (TypeDeclaration type : children) addIfNotNull(childNodes, buildType(type));
return childNodes;
}
private EclipseNode buildType(TypeDeclaration type) {
if (setAndGetAsHandled(type)) return null;
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
childNodes.addAll(buildFields(type.fields, getRecordFieldAnnotations(type)));
childNodes.addAll(buildTypes(type.memberTypes));
childNodes.addAll(buildMethods(type.methods));
childNodes.addAll(buildAnnotations(type.annotations, false));
return putInMap(new EclipseNode(this, type, childNodes, Kind.TYPE));
}
private Collection<EclipseNode> buildFields(FieldDeclaration[] children, Annotation[][] annotations) {
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
if (children != null) for (int i = 0; i < children.length; i++) {
addIfNotNull(childNodes, buildField(children[i], annotations[i]));
}
return childNodes;
}
private static <T> List<T> singleton(T item) {
List<T> list = new ArrayList<T>();
if (item != null) list.add(item);
return list;
}
private EclipseNode buildField(FieldDeclaration field, Annotation[] annotations) {
if (field instanceof Initializer) return buildInitializer((Initializer) field);
if (setAndGetAsHandled(field)) return null;
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
addIfNotNull(childNodes, buildTypeUse(field.type));
addIfNotNull(childNodes, buildStatement(field.initialization));
childNodes.addAll(buildAnnotations(annotations != null ? annotations : field.annotations, true));
return putInMap(new EclipseNode(this, field, childNodes, Kind.FIELD));
}
private EclipseNode buildInitializer(Initializer initializer) {
if (setAndGetAsHandled(initializer)) return null;
return putInMap(new EclipseNode(this, initializer, singleton(buildStatement(initializer.block)),
Kind.INITIALIZER));
}
private Collection<EclipseNode> buildMethods(AbstractMethodDeclaration[] children) {
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
if (children != null) for (AbstractMethodDeclaration method : children) addIfNotNull(childNodes,
buildMethod(method));
return childNodes;
}
private EclipseNode buildMethod(AbstractMethodDeclaration method) {
```

```
if (setAndGetAsHandled(method)) return null;
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
childNodes.addAll(buildArguments(method.arguments));
if (method instanceof ConstructorDeclaration) {
ConstructorDeclaration constructor = (ConstructorDeclaration) method;
addIfNotNull(childNodes, buildStatement(constructor.constructorCall));
}
childNodes.addAll(buildStatements(method.statements));
childNodes.addAll(buildAnnotations(method.annotations, false));
return putInMap(new EclipseNode(this, method, childNodes, Kind.METHOD));
}
//Arguments are a kind of LocalDeclaration. They can definitely contain lombok annotations, so we care about them.
private Collection<EclipseNode> buildArguments(Argument[] children) {
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
if (children != null) for (LocalDeclaration local : children) {
addIfNotNull(childNodes, buildLocal(local, Kind.ARGUMENT));
}
return childNodes;
}
private EclipseNode buildLocal(LocalDeclaration local, Kind kind) {
if (setAndGetAsHandled(local)) return null;
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
addIfNotNull(childNodes, buildTypeUse(local.type));
addIfNotNull(childNodes, buildStatement(local.initialization));
childNodes.addAll(buildAnnotations(local.annotations, true));
return putInMap(new EclipseNode(this, local, childNodes, kind));
}
private EclipseNode buildTypeUse(TypeReference tr) {
if (setAndGetAsHandled(tr)) return null;
if (tr == null) return null;
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
Annotation[] anns = getTopLevelTypeReferenceAnnotations(tr);
if (anns != null) for (Annotation ann : anns) addIfNotNull(childNodes, buildAnnotation(ann, false));
if (tr instanceof ParameterizedQualifiedTypeReference) {
ParameterizedQualifiedTypeReference pqtr = (ParameterizedQualifiedTypeReference) tr;
int len = pqtr.tokens.length;
for (int i = 0; i < len; i++) {
TypeReference[] typeArgs = pqtr.typeArguments[i];
if (typeArgs != null) for (TypeReference tArg : typeArgs) addIfNotNull(childNodes, buildTypeUse(tArg));
}
} else if (tr instanceof ParameterizedSingleTypeReference) {
ParameterizedSingleTypeReference pstr = (ParameterizedSingleTypeReference) tr;
if (pstr.typeArguments != null) for (TypeReference tArg : pstr.typeArguments) {
addIfNotNull(childNodes, buildTypeUse(tArg));
}
} else if (tr instanceof Wildcard) {
TypeReference bound = ((Wildcard) tr).bound;
if (bound != null) addIfNotNull(childNodes, buildTypeUse(bound));
}
return putInMap(new EclipseNode(this, tr, childNodes, Kind.TYPE_USE));
}
private Collection<EclipseNode> buildAnnotations(Annotation[] annotations, boolean varDecl) {
```

```
List<EclipseNode> elements = new ArrayList<EclipseNode>();
if (annotations != null) for (Annotation an : annotations) addIfNotNull(elements, buildAnnotation(an,
varDecl));
return elements;
}
private EclipseNode buildAnnotation(Annotation annotation, boolean field) {
if (annotation == null) return null;
boolean handled = setAndGetAsHandled(annotation);
if (!field && handled) {
// @Foo int x, y; is handled in eclipse by putting the same annotation node on 2 FieldDeclarations.
return null;
}
return putInMap(new EclipseNode(this, annotation, null, Kind.ANNOTATION));
}
private Collection<EclipseNode> buildStatements(Statement[] children) {
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
if (children != null) for (Statement child : children) addIfNotNull(childNodes, buildStatement(child));
return childNodes;
}
private EclipseNode buildStatement(Statement child) {
if (child == null) return null;
if (child instanceof TypeDeclaration) return buildType((TypeDeclaration) child);
if (child instanceof LocalDeclaration) return buildLocal((LocalDeclaration) child, Kind.LOCAL);
if (setAndGetAsHandled(child)) return null;
return drill(child);
}
private EclipseNode drill(Statement statement) {
List<EclipseNode> childNodes = new ArrayList<EclipseNode>();
for (FieldAccess fa : fieldsOf(statement.getClass()))
childNodes.addAll(buildWithField(EclipseNode.class, statement, fa));
return putInMap(new EclipseNode(this, statement, childNodes, Kind.STATEMENT));
}
/* For Eclipse, only Statement counts, as Expression is a subclass of it, even though this isn't
* entirely correct according to the JLS spec (only some expressions can be used as statements, not all
of them). */
private static Collection<Class<? extends ASTNode>> statementTypes() {
return Collections.<Class<? extends ASTNode>>singleton(Statement.class);
}
private static class EcjReflectionCheck {
private static final String COMPILATIONRESULT_TYPE =
"org.eclipse.jdt.internal.compiler.CompilationResult";
public static final Method addProblemToCompilationResult;
public static final Throwable problemAddProblemToCompilationResult;
public static final Method typeReferenceGetAnnotationsOnDimensions;
public static final Field typeReferenceAnnotations;
static {
Throwable problem_ = null;
Method m1 = null, m2;
Field f;
try {
m1 = Permit.getMethod(EclipseAstProblemView.class, "addProblemToCompilationResult",
char[].class, Class.forName(COMPILATIONRESULT_TYPE), boolean.class, String.class, int.class,
int.class);
```

```
} catch (Throwable t) {
// That's problematic, but as long as no local classes are used we don't actually need it.
// Better fail on local classes than crash altogether.
problem_ = t;
}
try {
m2 = Permit.getMethod(TypeReference.class, "getAnnotationsOnDimensions");
} catch (Throwable t) {
m2 = null;
}
try {
f = Permit.getField(TypeReference.class, "annotations");
} catch (Throwable t) {
f = null;
}
addProblemToCompilationResult = m1;
problemAddProblemToCompilationResult = problem_;
typeReferenceGetAnnotationsOnDimensions = m2;
typeReferenceAnnotations = f;
}
}
}
```

lombok/eclipse/EclipseASTAdapter.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
```

```
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Initializer;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
/**
* Standard adapter for the {@link EclipseASTVisitor} interface. Every method on that interface
* has been implemented with an empty body. Override whichever methods you need.
*/
public abstract class EclipseASTAdapter implements EclipseASTVisitor {
private final boolean deferUntilPostDiet = getClass().isAnnotationPresent(DeferUntilPostDiet.class);
/** {@inheritDoc} */
public void visitCompilationUnit(EclipseNode top, CompilationUnitDeclaration unit) {}
/** {@inheritDoc} */
public void endVisitCompilationUnit(EclipseNode top, CompilationUnitDeclaration unit) {}
/** {@inheritDoc} */
public void visitType(EclipseNode typeNode, TypeDeclaration type) {}
/** {@inheritDoc} */
public void visitAnnotationOnType(TypeDeclaration type, EclipseNode annotationNode, Annotation
annotation) {}
/** {@inheritDoc} */
public void endVisitType(EclipseNode typeNode, TypeDeclaration type) {}
/** {@inheritDoc} */
public void visitInitializer(EclipseNode initializerNode, Initializer initializer) {}
/** {@inheritDoc} */
public void endVisitInitializer(EclipseNode initializerNode, Initializer initializer) {}
/** {@inheritDoc} */
public void visitField(EclipseNode fieldNode, FieldDeclaration field) {}
/** {@inheritDoc} */
public void visitAnnotationOnField(FieldDeclaration field, EclipseNode annotationNode, Annotation
annotation) {}
/** {@inheritDoc} */
public void endVisitField(EclipseNode fieldNode, FieldDeclaration field) {}
/** {@inheritDoc} */
public void visitMethod(EclipseNode methodNode, AbstractMethodDeclaration method) {}
/** {@inheritDoc} */
public void visitAnnotationOnMethod(AbstractMethodDeclaration method, EclipseNode
annotationNode, Annotation annotation) {}
/** {@inheritDoc} */
public void endVisitMethod(EclipseNode methodNode, AbstractMethodDeclaration method) {}
/** {@inheritDoc} */
public void visitMethodArgument(EclipseNode argNode, Argument arg, AbstractMethodDeclaration
method) {}
/** {@inheritDoc} */
public void visitAnnotationOnMethodArgument(Argument arg, AbstractMethodDeclaration method,
EclipseNode annotationNode, Annotation annotation) {}
/** {@inheritDoc} */
public void endVisitMethodArgument(EclipseNode argNode, Argument arg, AbstractMethodDeclaration
method) {}
/** {@inheritDoc} */
public void visitLocal(EclipseNode localNode, LocalDeclaration local) {}
/** {@inheritDoc} */
```

```
public void visitAnnotationOnLocal(LocalDeclaration local, EclipseNode annotationNode, Annotation
annotation) {}
/** {@inheritDoc} */
public void endVisitLocal(EclipseNode localNode, LocalDeclaration local) {}
/** {@inheritDoc} */
@Override public void visitTypeUse(EclipseNode typeUseNode, TypeReference typeUse) {}
/** {@inheritDoc} */
public void visitAnnotationOnTypeUse(TypeReference typeUse, EclipseNode annotationNode,
Annotation annotation) {}
/** {@inheritDoc} */
@Override public void endVisitTypeUse(EclipseNode typeUseNode, TypeReference typeUse) {}
/** {@inheritDoc} */
public void visitStatement(EclipseNode statementNode, Statement statement) {}
/** {@inheritDoc} */
public void endVisitStatement(EclipseNode statementNode, Statement statement) {}
public boolean isDeferUntilPostDiet() {
return deferUntilPostDiet ;
}
}
```

lombok/eclipse/EclipseASTVisitor.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.io.PrintStream;
import java.lang.reflect.Modifier;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
```

```
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Initializer;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MarkerAnnotation;
import org.eclipse.jdt.internal.compiler.ast.MemberValuePair;
import org.eclipse.jdt.internal.compiler.ast.NormalAnnotation;
import org.eclipse.jdt.internal.compiler.ast.SingleMemberAnnotation;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
/**
* Implement so you can ask any EclipseAST.Node to traverse depth-first through all children,
* calling the appropriate visit and endVisit methods.
*/
public interface EclipseASTVisitor {
/**
* Called at the very beginning and end.
*/
void visitCompilationUnit(EclipseNode top, CompilationUnitDeclaration unit);
void endVisitCompilationUnit(EclipseNode top, CompilationUnitDeclaration unit);
/**
* Called when visiting a type (a class, interface, annotation, enum, etcetera).
*/
void visitType(EclipseNode typeNode, TypeDeclaration type);
void visitAnnotationOnType(TypeDeclaration type, EclipseNode annotationNode, Annotation
annotation);
void endVisitType(EclipseNode typeNode, TypeDeclaration type);
/**
* Called when visiting a field of a class.
* Even though in Eclipse initializers (both instance and static) are represented as Initializer objects,
* which are a subclass of FieldDeclaration, those do NOT result in a call to this method. They result
* in a call to the visitInitializer method.
*/
void visitField(EclipseNode fieldNode, FieldDeclaration field);
void visitAnnotationOnField(FieldDeclaration field, EclipseNode annotationNode, Annotation
annotation);
void endVisitField(EclipseNode fieldNode, FieldDeclaration field);
/**
* Called for static and instance initializers. You can tell the difference via the modifier flag on the
* ASTNode (8 for static, 0 for not static). The content is in the 'block', not in the 'initialization',
* which would always be null for an initializer instance.
*/
void visitInitializer(EclipseNode initializerNode, Initializer initializer);
void endVisitInitializer(EclipseNode initializerNode, Initializer initializer);
/**
* Called for both methods (MethodDeclaration) and constructors (ConstructorDeclaration), but not for
* Clinit objects, which are a vestigial Eclipse thing that never contain anything. Static initializers
```

```
* show up as 'Initializer', in the visitInitializer method, with modifier bit STATIC set.
*/
void visitMethod(EclipseNode methodNode, AbstractMethodDeclaration method);
void visitAnnotationOnMethod(AbstractMethodDeclaration method, EclipseNode annotationNode,
Annotation annotation);
void endVisitMethod(EclipseNode methodNode, AbstractMethodDeclaration method);
/**
* Visits a method argument
*/
void visitMethodArgument(EclipseNode argNode, Argument arg, AbstractMethodDeclaration method);
void visitAnnotationOnMethodArgument(Argument arg, AbstractMethodDeclaration method,
EclipseNode annotationNode, Annotation annotation);
void endVisitMethodArgument(EclipseNode argNode, Argument arg, AbstractMethodDeclaration
method);
/**
* Visits a local declaration - that is, something like 'int x = 10;' on the method level.
*/
void visitLocal(EclipseNode localNode, LocalDeclaration local);
void visitAnnotationOnLocal(LocalDeclaration local, EclipseNode annotationNode, Annotation
annotation);
void endVisitLocal(EclipseNode localNode, LocalDeclaration local);
/**
* Visits a node that represents a type reference. Anything from {@code int} to {@code T} to {@code
foo,.pkg.Bar<T>.Baz<?> @Ann []}.
*/
void visitTypeUse(EclipseNode typeUseNode, TypeReference typeUse);
void visitAnnotationOnTypeUse(TypeReference typeUse, EclipseNode annotationNode, Annotation
annotation);
void endVisitTypeUse(EclipseNode typeUseNode, TypeReference typeUse);
/**
* Visits a statement that isn't any of the other visit methods (e.g. TypeDeclaration).
*/
void visitStatement(EclipseNode statementNode, Statement statement);
void endVisitStatement(EclipseNode statementNode, Statement statement);
/**
* Prints the structure of an AST.
*/
public static class Printer implements EclipseASTVisitor {
private final PrintStream out;
private final boolean printContent;
private int disablePrinting = 0;
private int indent = 0;
private boolean printClassNames = false;
private final boolean printPositions;
public boolean deferUntilPostDiet() {
return false;
}
/**
* @param printContent if true, bodies are printed directly, as java code,
* instead of a tree listing of every AST node inside it.
*/
public Printer(boolean printContent) {
this(printContent, System.out, false);
```

```
}
/**
* @param printContent if true, bodies are printed directly, as java code,
* instead of a tree listing of every AST node inside it.
* @param out write output to this stream. You must close it yourself. flush() is called after every line.
*
* @see java.io.PrintStream#flush()
*/
public Printer(boolean printContent, PrintStream out, boolean printPositions) {
this.printContent = printContent;
this.out = out;
this.printPositions = printPositions;
}
private void forcePrint(String text, Object... params) {
StringBuilder sb = new StringBuilder();
for (int i = 0; i < indent; i++) sb.append(" ");
sb.append(text);
Object[] t;
if (printClassNames && params.length > 0) {
sb.append(" [");
for (int i = 0; i < params.length; i++) {
if (i > 0) sb.append(", ");
sb.append("%s");
}
sb.append("]");
t = new Object[params.length + params.length];
for (int i = 0; i < params.length; i++) {
t[i] = params[i];
t[i + params.length] = (params[i] == null) ? "NULL " : params[i].getClass();
}
} else {
t = params;
}
sb.append("\n");
out.printf(sb.toString(), t);
out.flush();
}
private void print(String text, Object... params) {
if (disablePrinting == 0) forcePrint(text, params);
}
private String str(char[] c) {
if (c == null) return "(NULL)";
return new String(c);
}
private String str(TypeReference type) {
if (type == null) return "(NULL)";
char[][] c = type.getTypeName();
StringBuilder sb = new StringBuilder();
boolean first = true;
for (char[] d : c) {
sb.append(first ? "" : ".").append(new String(d));
first = false;
}
```

```
return sb.toString();
}
public void visitCompilationUnit(EclipseNode node, CompilationUnitDeclaration unit) {
out.println("------------------------------------------------------------");
out.println(node.isCompleteParse() ? "COMPLETE" : "incomplete");
print("<CUD %s%s%s>", node.getFileName(), isGenerated(unit) ? " (GENERATED)" : "",
position(node));
indent++;
}
public void endVisitCompilationUnit(EclipseNode node, CompilationUnitDeclaration unit) {
indent--;
print("</CUD>");
}
private String printFlags(int flags, ASTNode node) {
StringBuilder out = new StringBuilder();
if ((flags & ClassFileConstants.AccPublic) != 0) {
flags &= ~ClassFileConstants.AccPublic;
out.append("public ");
}
if ((flags & ClassFileConstants.AccPrivate) != 0) {
flags &= ~ClassFileConstants.AccPrivate;
out.append("private ");
}
if ((flags & ClassFileConstants.AccProtected) != 0) {
flags &= ~ClassFileConstants.AccProtected;
out.append("protected ");
}
if ((flags & ClassFileConstants.AccStatic) != 0) {
flags &= ~ClassFileConstants.AccStatic;
out.append("static ");
}
if ((flags & ClassFileConstants.AccFinal) != 0) {
flags &= ~ClassFileConstants.AccFinal;
out.append("final ");
}
if ((flags & ClassFileConstants.AccSynchronized) != 0) {
flags &= ~ClassFileConstants.AccSynchronized;
out.append("synchronized ");
}
if ((flags & ClassFileConstants.AccNative) != 0) {
flags &= ~ClassFileConstants.AccNative;
out.append("native ");
}
if ((flags & ClassFileConstants.AccInterface) != 0) {
flags &= ~ClassFileConstants.AccInterface;
out.append("interface ");
}
if ((flags & ClassFileConstants.AccAbstract) != 0) {
flags &= ~ClassFileConstants.AccAbstract;
out.append("abstract ");
}
if ((flags & ClassFileConstants.AccStrictfp) != 0) {
flags &= ~ClassFileConstants.AccStrictfp;
```

```
out.append("strictfp ");
}
if ((flags & ClassFileConstants.AccSynthetic) != 0) {
flags &= ~ClassFileConstants.AccSynthetic;
out.append("synthetic ");
}
if ((flags & ClassFileConstants.AccAnnotation) != 0) {
flags &= ~ClassFileConstants.AccAnnotation;
out.append("annotation ");
}
if ((flags & ClassFileConstants.AccEnum) != 0) {
flags &= ~ClassFileConstants.AccEnum;
out.append("enum ");
}
if ((flags & ClassFileConstants.AccVolatile) != 0) {
flags &= ~ClassFileConstants.AccVolatile;
if (node instanceof FieldDeclaration) out.append("volatile ");
else out.append("volatile/bridge ");
}
if ((flags & ClassFileConstants.AccTransient) != 0) {
flags &= ~ClassFileConstants.AccTransient;
if (node instanceof Argument) out.append("varargs ");
else if (node instanceof FieldDeclaration) out.append("transient ");
else out.append("transient/varargs ");
}
if (flags != 0) {
out.append(String.format(" 0x%08X ", flags));
}
return out.toString().trim();
}
public void visitType(EclipseNode node, TypeDeclaration type) {
print("<TYPE %s%s%s> %s", str(type.name), isGenerated(type) ? " (GENERATED)" : "",
position(node), printFlags(type.modifiers, type));
indent++;
if (printContent) {
print("%s", type);
disablePrinting++;
}
}
public void visitAnnotationOnType(TypeDeclaration type, EclipseNode node, Annotation annotation) {
forcePrint("<ANNOTATION%s: %s%s />", isGenerated(annotation) ? " (GENERATED)" : "", annotation,
position(node));
}
public void endVisitType(EclipseNode node, TypeDeclaration type) {
if (printContent) disablePrinting--;
indent--;
print("</TYPE %s>", str(type.name));
}
public void visitInitializer(EclipseNode node, Initializer initializer) {
Block block = initializer.block;
boolean s = (block != null && block.statements != null);
print("<%s INITIALIZER: %s%s%s>",
(initializer.modifiers & Modifier.STATIC) != 0 ? "static" : "instance",
```

```
s ? "filled" : "blank",
isGenerated(initializer) ? " (GENERATED)" : "", position(node));
indent++;
if (printContent) {
if (initializer.block != null) print("%s", initializer.block);
disablePrinting++;
}
}
public void endVisitInitializer(EclipseNode node, Initializer initializer) {
if (printContent) disablePrinting--;
indent--;
print("</%s INITIALIZER>", (initializer.modifiers & Modifier.STATIC) != 0 ? "static" : "instance");
}
public void visitField(EclipseNode node, FieldDeclaration field) {
print("<FIELD%s %s %s = %s%s> %s", isGenerated(field) ? " (GENERATED)" : "",
str(field.type), str(field.name), field.initialization, position(node), printFlags(field.modifiers, field));
indent++;
if (printContent) {
if (field.initialization != null) print("%s", field.initialization);
disablePrinting++;
}
}
public void visitAnnotationOnField(FieldDeclaration field, EclipseNode node, Annotation annotation) {
forcePrint("<ANNOTATION%s: %s%s />", isGenerated(annotation) ? " (GENERATED)" : "", annotation,
position(node));
}
public void endVisitField(EclipseNode node, FieldDeclaration field) {
if (printContent) disablePrinting--;
indent--;
print("</FIELD %s %s>", str(field.type), str(field.name));
}
public void visitMethod(EclipseNode node, AbstractMethodDeclaration method) {
String type = method instanceof ConstructorDeclaration ? "CONSTRUCTOR" : "METHOD";
print("<%s %s: %s%s%s> %s", type, str(method.selector), method.statements != null ? ("filled(" +
method.statements.length + ")") : "blank",
isGenerated(method) ? " (GENERATED)" : "", position(node), printFlags(method.modifiers, method));
indent++;
if (method instanceof ConstructorDeclaration) {
ConstructorDeclaration cd = (ConstructorDeclaration) method;
print("--> constructorCall: %s", cd.constructorCall == null ? "-NONE-" : cd.constructorCall);
}
if (printContent) {
if (method.statements != null) print("%s", method);
disablePrinting++;
}
}
public void visitAnnotationOnMethod(AbstractMethodDeclaration method, EclipseNode node,
Annotation annotation) {
forcePrint("<ANNOTATION%s: %s%s>", isGenerated(method) ? " (GENERATED)" : "", annotation,
position(node));
if (annotation instanceof MarkerAnnotation || disablePrinting != 0) {
forcePrint("<ANNOTATION%s: %s%s />", isGenerated(method) ? " (GENERATED)" : "", annotation,
position(node));
```

```
} else {
forcePrint("<ANNOTATION%s: %s%s>", isGenerated(method) ? " (GENERATED)" : "", annotation,
position(node));
indent++;
if (annotation instanceof SingleMemberAnnotation) {
Expression expr = ((SingleMemberAnnotation) annotation).memberValue;
print("<SINGLE-MEMBER-VALUE %s /> %s", expr.getClass(), expr);
}
if (annotation instanceof NormalAnnotation) {
for (MemberValuePair mvp : ((NormalAnnotation) annotation).memberValuePairs) {
print("<Member %s: %s /> %s", new String(mvp.name), mvp.value.getClass(), mvp.value);
}
}
indent--;
}
}
public void endVisitMethod(EclipseNode node, AbstractMethodDeclaration method) {
if (printContent) disablePrinting--;
String type = method instanceof ConstructorDeclaration ? "CONSTRUCTOR" : "METHOD";
indent--;
print("</%s %s>", type, str(method.selector));
}
public void visitMethodArgument(EclipseNode node, Argument arg, AbstractMethodDeclaration
method) {
print("<METHODARG%s %s %s = %s%s> %s", isGenerated(arg) ? " (GENERATED)" : "",
str(arg.type), str(arg.name), arg.initialization, position(node), printFlags(arg.modifiers, arg));
indent++;
}
public void visitAnnotationOnMethodArgument(Argument arg, AbstractMethodDeclaration method,
EclipseNode node, Annotation annotation) {
print("<ANNOTATION%s: %s%s />", isGenerated(annotation) ? " (GENERATED)" : "", annotation,
position(node));
}
public void endVisitMethodArgument(EclipseNode node, Argument arg, AbstractMethodDeclaration
method) {
indent--;
print("</METHODARG %s %s>", str(arg.type), str(arg.name));
}
public void visitLocal(EclipseNode node, LocalDeclaration local) {
print("<LOCAL%s %s %s = %s%s> %s", isGenerated(local) ? " (GENERATED)" : "",
str(local.type), str(local.name), local.initialization, position(node), printFlags(local.modifiers, local));
indent++;
}
public void visitAnnotationOnLocal(LocalDeclaration local, EclipseNode node, Annotation annotation) {
print("<ANNOTATION%s: %s />", isGenerated(annotation) ? " (GENERATED)" : "", annotation);
}
public void endVisitLocal(EclipseNode node, LocalDeclaration local) {
indent--;
print("</LOCAL %s %s>", str(local.type), str(local.name));
}
@Override public void visitTypeUse(EclipseNode typeUseNode, TypeReference typeUse) {
print("<TYPE %s>", typeUse.getClass());
indent++;
```

```
print("%s", typeUse);
}
@Override public void visitAnnotationOnTypeUse(TypeReference typeUse, EclipseNode
annotationNode, Annotation annotation) {
print("<ANNOTATION%s: %s />", isGenerated(annotation) ? " (GENERATED)" : "", annotation);
}
@Override public void endVisitTypeUse(EclipseNode typeUseNode, TypeReference typeUse) {
indent--;
print("</TYPE %s>", typeUse.getClass());
}
public void visitStatement(EclipseNode node, Statement statement) {
print("<%s%s%s>", statement.getClass(), isGenerated(statement) ? " (GENERATED)" : "",
position(node));
if (statement instanceof AllocationExpression) {
AllocationExpression alloc = (AllocationExpression) statement;
print(" --> arguments: %s", alloc.arguments == null ? "NULL" : alloc.arguments.length);
print(" --> genericTypeArguments: %s", alloc.genericTypeArguments == null ? "NULL" :
alloc.genericTypeArguments.length);
print(" --> typeArguments: %s", alloc.typeArguments == null ? "NULL" : alloc.typeArguments.length);
print(" --> enumConstant: %s", alloc.enumConstant);
print(" --> inferredReturnType: %s", alloc.inferredReturnType);
}
indent++;
print("%s", statement);
}
public void endVisitStatement(EclipseNode node, Statement statement) {
indent--;
print("</%s>", statement.getClass());
}
String position(EclipseNode node) {
if (!printPositions) return "";
int start = node.get().sourceStart();
int end = node.get().sourceEnd();
return String.format(" [%d, %d]", start, end);
}
public boolean isDeferUntilPostDiet() {
return false;
}
}
boolean isDeferUntilPostDiet();
}
```

lombok/eclipse/EclipseAnnotationHandler.java

```
/*
* Copyright (C) 2009-2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
```

```
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import lombok.core.AnnotationValues;
import lombok.core.SpiLoadUtil;
/**
* Implement this interface if you want to be triggered for a specific annotation.
*
* You MUST replace 'T' with a specific annotation type, such as:
*
* {@code public class HandleGetter extends EclipseAnnotationHandler<Getter>}
*
* Because this generics parameter is inspected to figure out which class you're interested in.
*
* You also need to register yourself via SPI discovery as being an implementation of {@code
EclipseAnnotationHandler}.
*/
public abstract class EclipseAnnotationHandler<T extends java.lang.annotation.Annotation> {
/**
* Called when an annotation is found that is likely to match the annotation you're interested in.
*
* Be aware that you'll be called for ANY annotation node in the source that looks like a match. There is,
* for example, no guarantee that the annotation node belongs to a method, even if you set your
* TargetType in the annotation to methods only.
*
* @param annotation The actual annotation - use this object to retrieve the annotation parameters.
* @param ast The Eclipse AST node representing the annotation.
* @param annotationNode The Lombok AST wrapper around the 'ast' parameter. You can use this
object
* to travel back up the chain (something javac AST can't do) to the parent of the annotation, as well
* as access useful methods such as generating warnings or errors focused on the annotation.
*/
public abstract void handle(AnnotationValues<T> annotation,
org.eclipse.jdt.internal.compiler.ast.Annotation ast, EclipseNode annotationNode);
/**
* Called when you want to defer until post diet, and we're still in pre-diet. May be called not at all or
multiple times, so make sure
* this method is idempotent if run more than once, and whatever you do here should also be done in the
main 'handle' method.
*
```

```
* NB: This method exists because in certain cases, within eclipse, you have to create i.e. a field before referencing it in generated code. You still
* have to create the field, if its not already there, in 'handle', because for example preHandle would never even be called in ecj mode.
*/
public void preHandle(AnnotationValues<T> annotation, org.eclipse.jdt.internal.compiler.ast.Annotation ast, EclipseNode annotationNode) {
}
/**
* This handler is a handler for the given annotation; you don't normally need to override this class
* as the annotation type is extracted from your {@code extends EclipseAnnotationHandler<AnnotationTypeHere>}
* signature.
*/
@SuppressWarnings("unchecked") public Class<T> getAnnotationHandledByThisHandler() {
return (Class<T>) SpiLoadUtil.findAnnotationClass(getClass(), EclipseAnnotationHandler.class);
}
}
```

lombok/eclipse/EclipseAstProblemView.java

```
package lombok.eclipse;
import org.eclipse.jdt.core.compiler.CategorizedProblem;
import org.eclipse.jdt.internal.compiler.CompilationResult;
import org.eclipse.jdt.internal.compiler.problem.DefaultProblem;
import org.eclipse.jdt.internal.compiler.problem.ProblemSeverities;
import org.eclipse.jdt.internal.compiler.util.Util;
public class EclipseAstProblemView {
/**
* Adds a problem to the provided CompilationResult object so that it will show up
* in the Problems/Warnings view.
*/
public static void addProblemToCompilationResult(char[] fileNameArray, CompilationResult result, boolean isWarning, String message, int sourceStart, int sourceEnd) {
if (result == null) return;
if (fileNameArray == null) fileNameArray = "(unknown).java".toCharArray();
int lineNumber = 0;
int columnNumber = 1;
int[] lineEnds = null;
lineNumber = sourceStart >= 0
? Util.getLineNumber(sourceStart, lineEnds = result.getLineSeparatorPositions(), 0, lineEnds.length-1)
: 0;
columnNumber = sourceStart >= 0
? Util.searchColumnNumber(result.getLineSeparatorPositions(), lineNumber,sourceStart)
: 0;
CategorizedProblem ecProblem = new LombokProblem(
fileNameArray, message, 0, new String[0],
isWarning ? ProblemSeverities.Warning : ProblemSeverities.Error,
sourceStart, sourceEnd, lineNumber, columnNumber);
result.record(ecProblem, null);
}
```

```
private static class LombokProblem extends DefaultProblem {
private static final String MARKER_ID = "org.eclipse.jdt.apt.pluggable.core.compileProblem";
//$NON-NLS-1$
public LombokProblem(char[] originatingFileName, String message, int id,
String[] stringArguments, int severity,
int startPosition, int endPosition, int line, int column) {
super(originatingFileName, message, id, stringArguments, severity, startPosition, endPosition, line,
column);
}
@Override public int getCategoryID() {
return CAT_UNSPECIFIED;
}
@Override public String getMarkerType() {
return MARKER_ID;
}
}
}
```

lombok/eclipse/EclipseImportList.java

```
/*
* Copyright (C) 2013-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import static lombok.eclipse.Eclipse.toQualifiedName;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import lombok.core.ImportList;
import lombok.core.LombokInternalAliasing;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
```

```
import org.eclipse.jdt.internal.compiler.ast.ImportReference;
public class EclipseImportList implements ImportList {
private ImportReference[] imports;
private ImportReference pkg;
public EclipseImportList(CompilationUnitDeclaration cud) {
this.pkg = cud.currentPackage;
this.imports = cud.imports;
}
@Override public String getFullyQualifiedNameForSimpleName(String unqualified) {
String q = getFullyQualifiedNameForSimpleNameNoAliasing(unqualified);
return q == null ? null : LombokInternalAliasing.processAliases(q);
}
@Override public String getFullyQualifiedNameForSimpleNameNoAliasing(String unqualified) {
if (imports != null) {
outer:
for (ImportReference imp : imports) {
if ((imp.bits & ASTNode.OnDemand) != 0) continue;
char[][] tokens = imp.tokens;
char[] token = tokens.length == 0 ? new char[0] : tokens[tokens.length - 1];
int len = token.length;
if (len != unqualified.length()) continue;
for (int i = 0; i < len; i++) if (token[i] != unqualified.charAt(i)) continue outer;
return toQualifiedName(tokens);
}
}
return null;
}
@Override public boolean hasStarImport(String packageName) {
if (isEqual(packageName, pkg)) return true;
if ("java.lang".equals(packageName)) return true;
if (imports != null) for (ImportReference imp : imports) {
if ((imp.bits & ASTNode.OnDemand) == 0) continue;
if (imp.isStatic()) continue;
if (isEqual(packageName, imp)) return true;
}
return false;
}
private static boolean isEqual(String packageName, ImportReference pkgOrStarImport) {
if (pkgOrStarImport == null || pkgOrStarImport.tokens == null || pkgOrStarImport.tokens.length == 0)
return packageName.isEmpty();
int pos = 0;
int len = packageName.length();
for (int i = 0; i < pkgOrStarImport.tokens.length; i++) {
if (i != 0) {
if (pos >= len) return false;
if (packageName.charAt(pos++) != '.') return false;
}
for (int j = 0; j < pkgOrStarImport.tokens[i].length; j++) {
if (pos >= len) return false;
if (packageName.charAt(pos++) != pkgOrStarImport.tokens[i][j]) return false;
}
}
return true;
```

```
}
@Override public Collection<String> applyNameToStarImports(String startsWith, String name) {
List<String> out = Collections.emptyList();
if (pkg != null && pkg.tokens != null && pkg.tokens.length != 0) {
char[] first = pkg.tokens[0];
int len = first.length;
boolean match = true;
if (startsWith.length() == len) {
for (int i = 0; match && i < len; i++) {
if (startsWith.charAt(i) != first[i]) match = false;
}
if (match) out.add(toQualifiedName(pkg.tokens) + "." + name);
}
}
if (imports != null) {
outer:
for (ImportReference imp : imports) {
if ((imp.bits & ASTNode.OnDemand) == 0) continue;
if (imp.isStatic()) continue;
if (imp.tokens == null || imp.tokens.length == 0) continue;
char[] firstToken = imp.tokens[0];
if (firstToken.length != startsWith.length()) continue;
for (int i = 0; i < firstToken.length; i++) if (startsWith.charAt(i) != firstToken[i]) continue outer;
String fqn = toQualifiedName(imp.tokens) + "." + name;
if (out.isEmpty()) out = Collections.singletonList(fqn);
else if (out.size() == 1) {
out = new ArrayList<String>(out);
out.add(fqn);
} else {
out.add(fqn);
}
}
}
return out;
}
@Override public String applyUnqualifiedNameToPackage(String unqualified) {
if (pkg == null || pkg.tokens == null || pkg.tokens.length == 0) return unqualified;
return toQualifiedName(pkg.tokens) + "." + unqualified;
}
}
```

lombok/eclipse/EclipseNode.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
```

```
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Clinit;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Initializer;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.eclipse.handlers.EclipseHandlerUtil;
/**
* Eclipse specific version of the LombokNode class.
*/
public class EclipseNode extends lombok.core.LombokNode<EclipseAST, EclipseNode, ASTNode> {
private EclipseAST ast;
/** {@inheritDoc} */
EclipseNode(EclipseAST ast, ASTNode node, List<EclipseNode> children, Kind kind) {
super(node, children, kind);
this.ast = ast;
}
@Override
public EclipseAST getAst() {
return ast;
}
/**
* Visits this node and all child nodes depth-first, calling the provided visitor's visit methods.
*/
public void traverse(EclipseASTVisitor visitor) {
if (visitor.isDeferUntilPostDiet() && !isCompleteParse()) return;
switch (getKind()) {
case COMPILATION_UNIT:
```

```
visitor.visitCompilationUnit(this, (CompilationUnitDeclaration) get());
ast.traverseChildren(visitor, this);
visitor.endVisitCompilationUnit(this, (CompilationUnitDeclaration) get());
break;
case TYPE:
visitor.visitType(this, (TypeDeclaration) get());
ast.traverseChildren(visitor, this);
visitor.endVisitType(this, (TypeDeclaration) get());
break;
case FIELD:
visitor.visitField(this, (FieldDeclaration) get());
ast.traverseChildren(visitor, this);
visitor.endVisitField(this, (FieldDeclaration) get());
break;
case INITIALIZER:
visitor.visitInitializer(this, (Initializer) get());
ast.traverseChildren(visitor, this);
visitor.endVisitInitializer(this, (Initializer) get());
break;
case METHOD:
if (get() instanceof Clinit) return;
visitor.visitMethod(this, (AbstractMethodDeclaration) get());
ast.traverseChildren(visitor, this);
visitor.endVisitMethod(this, (AbstractMethodDeclaration) get());
break;
case ARGUMENT:
AbstractMethodDeclaration method = (AbstractMethodDeclaration) up().get();
visitor.visitMethodArgument(this, (Argument) get(), method);
ast.traverseChildren(visitor, this);
visitor.endVisitMethodArgument(this, (Argument) get(), method);
break;
case LOCAL:
visitor.visitLocal(this, (LocalDeclaration) get());
ast.traverseChildren(visitor, this);
visitor.endVisitLocal(this, (LocalDeclaration) get());
break;
case ANNOTATION:
switch (up().getKind()) {
case TYPE:
visitor.visitAnnotationOnType((TypeDeclaration) up().get(), this, (Annotation) get());
break;
case FIELD:
visitor.visitAnnotationOnField((FieldDeclaration) up().get(), this, (Annotation) get());
break;
case METHOD:
visitor.visitAnnotationOnMethod((AbstractMethodDeclaration) up().get(), this, (Annotation) get());
break;
case ARGUMENT:
visitor.visitAnnotationOnMethodArgument(
(Argument) parent.get(),
(AbstractMethodDeclaration) parent.directUp().get(),
this, (Annotation) get());
break;
```

```
case LOCAL:
visitor.visitAnnotationOnLocal((LocalDeclaration) parent.get(), this, (Annotation) get());
break;
case TYPE_USE:
visitor.visitAnnotationOnTypeUse((TypeReference) parent.get(), this, (Annotation) get());
break;
default:
throw new AssertionError("Annotation not expected as child of a " + up().getKind());
}
break;
case TYPE_USE:
visitor.visitTypeUse(this, (TypeReference) get());
ast.traverseChildren(visitor, this);
visitor.endVisitTypeUse(this, (TypeReference) get());
break;
case STATEMENT:
visitor.visitStatement(this, (Statement) get());
ast.traverseChildren(visitor, this);
visitor.endVisitStatement(this, (Statement) get());
break;
default:
throw new AssertionError("Unexpected kind during node traversal: " + getKind());
}
}
/** {@inheritDoc} */
@Override public String getName() {
final char[] n;
if (node instanceof TypeDeclaration) n = ((TypeDeclaration)node).name;
else if (node instanceof FieldDeclaration) n = ((FieldDeclaration)node).name;
else if (node instanceof AbstractMethodDeclaration) n = ((AbstractMethodDeclaration)node).selector;
else if (node instanceof LocalDeclaration) n = ((LocalDeclaration)node).name;
else n = null;
return n == null ? null : new String(n);
}
/** {@inheritDoc} */
@Override public void addError(String message) {
this.addError(message, this.get().sourceStart, this.get().sourceEnd);
}
/** Generate a compiler error that shows the wavy underline from-to the stated character positions. */
public void addError(String message, int sourceStart, int sourceEnd) {
ast.addProblem(ast.new ParseProblem(false, message, sourceStart, sourceEnd));
}
/** {@inheritDoc} */
@Override public void addWarning(String message) {
this.addWarning(message, this.get().sourceStart, this.get().sourceEnd);
}
/** Generate a compiler warning that shows the wavy underline from-to the stated character positions.
*/
public void addWarning(String message, int sourceStart, int sourceEnd) {
ast.addProblem(ast.new ParseProblem(true, message, sourceStart, sourceEnd));
}
/** {@inheritDoc} */
@Override protected boolean calculateIsStructurallySignificant(ASTNode parent) {
```

```
if (node instanceof TypeDeclaration) return true;
if (node instanceof AbstractMethodDeclaration) return true;
if (node instanceof FieldDeclaration) return true;
if (node instanceof LocalDeclaration) return true;
if (node instanceof CompilationUnitDeclaration) return true;
return false;
}
/**
* Convenient shortcut to the owning EclipseAST object's isCompleteParse method.
*
* @see EclipseAST#isCompleteParse()
*/
public boolean isCompleteParse() {
return ast.isCompleteParse();
}
@Override public boolean hasAnnotation(Class<? extends java.lang.annotation.Annotation> type) {
return EclipseHandlerUtil.hasAnnotation(type, this);
}
@Override public <Z extends java.lang.annotation.Annotation> AnnotationValues<Z>
findAnnotation(Class<Z> type) {
EclipseNode annotation = EclipseHandlerUtil.findAnnotation(type, this);
if (annotation == null) return null;
return EclipseHandlerUtil.createAnnotation(type, annotation);
}
private Integer getModifiers() {
if (node instanceof TypeDeclaration) return ((TypeDeclaration) node).modifiers;
if (node instanceof FieldDeclaration) return ((FieldDeclaration) node).modifiers;
if (node instanceof LocalDeclaration) return ((LocalDeclaration) node).modifiers;
if (node instanceof AbstractMethodDeclaration) return ((AbstractMethodDeclaration) node).modifiers;
return null;
}
@Override public boolean isStatic() {
if (node instanceof TypeDeclaration) {
TypeDeclaration t = (TypeDeclaration) node;
int f = t.modifiers;
if (((ClassFileConstants.AccInterface | ClassFileConstants.AccEnum | Eclipse.AccRecord) & f) != 0)
return true;
EclipseNode directUp = directUp();
if (directUp == null || directUp.getKind() == Kind.COMPILATION_UNIT) return true;
if (!(directUp.get() instanceof TypeDeclaration)) return false;
TypeDeclaration p = (TypeDeclaration) directUp.get();
f = p.modifiers;
if (((ClassFileConstants.AccInterface | ClassFileConstants.AccEnum) & f) != 0) return true;
}
if (node instanceof FieldDeclaration) {
EclipseNode directUp = directUp();
if (directUp != null && directUp.get() instanceof TypeDeclaration) {
TypeDeclaration p = (TypeDeclaration) directUp.get();
int f = p.modifiers;
if ((ClassFileConstants.AccInterface & f) != 0) return true;
}
}
Integer i = getModifiers();
```

```
if (i == null) return false;
int f = i.intValue();
return (ClassFileConstants.AccStatic & f) != 0;
}
@Override public boolean isFinal() {
if (node instanceof FieldDeclaration) {
EclipseNode directUp = directUp();
if (directUp != null && directUp.get() instanceof TypeDeclaration) {
TypeDeclaration p = (TypeDeclaration) directUp.get();
int f = p.modifiers;
if (((ClassFileConstants.AccInterface | ClassFileConstants.AccEnum) & f) != 0) return true;
}
}
Integer i = getModifiers();
if (i == null) return false;
int f = i.intValue();
return (ClassFileConstants.AccFinal & f) != 0;
}
@Override public boolean isPrimitive() {
if (node instanceof FieldDeclaration && !isEnumMember()) {
return Eclipse.isPrimitive(((FieldDeclaration) node).type);
}
if (node instanceof MethodDeclaration) {
return Eclipse.isPrimitive(((MethodDeclaration) node).returnType);
}
return false;
}
/**
* {@inheritDoc}
*/
@Override public String fieldOrMethodBaseType() {
TypeReference typeReference = null;
if (node instanceof FieldDeclaration && !isEnumMember()) {
typeReference = ((FieldDeclaration) node).type;
}
if (node instanceof MethodDeclaration) {
typeReference = ((MethodDeclaration) node).returnType;
}
if (typeReference == null) return null;
String fqn = Eclipse.toQualifiedName(typeReference.getTypeName());
if (typeReference.dimensions() == 0) return fqn;
StringBuilder result = new StringBuilder(fqn.length() + 2 * typeReference.dimensions());
result.append(fqn);
for (int i = 0; i < typeReference.dimensions(); i++) {
result.append("[]");
}
return result.toString();
}
@Override public boolean isTransient() {
if (getKind() != Kind.FIELD) return false;
Integer i = getModifiers();
return i != null && (i.intValue() & ClassFileConstants.AccTransient) != 0;
}
```

```
@Override public boolean isEnumMember() {
if (getKind() != Kind.FIELD) return false;
return ((FieldDeclaration) node).getKind() == 3;
}
@Override public boolean isEnumType() {
if (getKind() != Kind.TYPE) return false;
return (((TypeDeclaration) node).modifiers & ClassFileConstants.AccEnum) != 0;
}
@Override public int countMethodParameters() {
if (getKind() != Kind.METHOD) return 0;
Argument[] a = ((AbstractMethodDeclaration) node).arguments;
if (a == null) return 0;
return a.length;
}
@Override public int getStartPos() {
return node.sourceStart;
}
}
```

lombok/eclipse/HandlerLibrary.java


```
/*
* Copyright (C) 2009-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.eclipse;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import static lombok.eclipse.EcjAugments.ASTNode_handled;
import java.io.IOException;
import java.lang.annotation.Annotation;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
```

```
import java.util.HashMap;
import java.util.Map;
import java.util.SortedSet;
import java.util.TreeSet;
import lombok.Lombok;
import lombok.core.AnnotationValues;
import lombok.core.AnnotationValues.AnnotationValueDecodeFail;
import lombok.core.configuration.ConfigurationKeysLoader;
import lombok.core.HandlerPriority;
import lombok.core.SpiLoadUtil;
import lombok.core.TypeLibrary;
import lombok.core.TypeResolver;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
/**
* This class tracks 'handlers' and knows how to invoke them for any given AST node.
*
* This class can find the handlers (via SPI discovery) and will set up the given AST node, such as
* building an AnnotationValues instance.
*/
public class HandlerLibrary {
/**
* Creates a new HandlerLibrary. Errors will be reported to the Eclipse Error log.
* You probably want to use {@link #load()} instead.
*/
public HandlerLibrary() {
ConfigurationKeysLoader.LoaderLoader.loadAllConfigurationKeys();
}
private TypeLibrary typeLibrary = new TypeLibrary();
private static class VisitorContainer {
private final EclipseASTVisitor visitor;
private final long priority;
private final boolean deferUntilPostDiet;
VisitorContainer(EclipseASTVisitor visitor) {
this.visitor = visitor;
this.deferUntilPostDiet = visitor.getClass().isAnnotationPresent(DeferUntilPostDiet.class);
HandlerPriority hp = visitor.getClass().getAnnotation(HandlerPriority.class);
this.priority = hp == null ? 0L : (((long)hp.value()) << 32) + hp.subValue();
}
public boolean deferUntilPostDiet() {
return deferUntilPostDiet;
}
public long getPriority() {
return priority;
}
}
private static class AnnotationHandlerContainer<T extends Annotation> {
private final EclipseAnnotationHandler<T> handler;
private final Class<T> annotationClass;
private final long priority;
private final boolean deferUntilPostDiet;
AnnotationHandlerContainer(EclipseAnnotationHandler<T> handler, Class<T> annotationClass) {
```

```
this.handler = handler;
this.annotationClass = annotationClass;
this.deferUntilPostDiet = handler.getClass().isAnnotationPresent(DeferUntilPostDiet.class);
HandlerPriority hp = handler.getClass().getAnnotation(HandlerPriority.class);
this.priority = hp == null ? 0L : (((long)hp.value()) << 32) + hp.subValue();
}
public void handle(org.eclipse.jdt.internal.compiler.ast.Annotation annotation,
final EclipseNode annotationNode) {
AnnotationValues<T> annValues = createAnnotation(annotationClass, annotationNode);
handler.handle(annValues, annotation, annotationNode);
}
public void preHandle(org.eclipse.jdt.internal.compiler.ast.Annotation annotation,
final EclipseNode annotationNode) {
AnnotationValues<T> annValues = createAnnotation(annotationClass, annotationNode);
handler.preHandle(annValues, annotation, annotationNode);
}
public boolean deferUntilPostDiet() {
return deferUntilPostDiet;
}
public long getPriority() {
return priority;
}
}
private Map<String, AnnotationHandlerContainer<?>> annotationHandlers =
new HashMap<String, AnnotationHandlerContainer<?>>();
private Collection<VisitorContainer> visitorHandlers = new ArrayList<VisitorContainer>();
/**
* Creates a new HandlerLibrary. Errors will be reported to the Eclipse Error log.
* then uses SPI discovery to load all annotation and visitor based handlers so that future calls
* to the handle methods will defer to these handlers.
*/
public static HandlerLibrary load() {
HandlerLibrary lib = new HandlerLibrary();
loadAnnotationHandlers(lib);
loadVisitorHandlers(lib);
lib.calculatePriorities();
return lib;
}
private SortedSet<Long> priorities;
public SortedSet<Long> getPriorities() {
return priorities;
}
private void calculatePriorities() {
SortedSet<Long> set = new TreeSet<Long>();
for (AnnotationHandlerContainer<?> container : annotationHandlers.values())
set.add(container.getPriority());
for (VisitorContainer container : visitorHandlers) set.add(container.getPriority());
this.priorities = Collections.unmodifiableSortedSet(set);
}
/** Uses SPI Discovery to find implementations of {@link EclipseAnnotationHandler}. */
@SuppressWarnings({"rawtypes", "unchecked"})
private static void loadAnnotationHandlers(HandlerLibrary lib) {
try {
```

```
for (EclipseAnnotationHandler<?> handler : SpiLoadUtil.findServices(EclipseAnnotationHandler.class,
EclipseAnnotationHandler.class.getClassLoader())) {
try {
Class<? extends Annotation> annotationClass = handler.getAnnotationHandledByThisHandler();
AnnotationHandlerContainer<?> container = new AnnotationHandlerContainer(handler,
annotationClass);
String annotationClassName = container.annotationClass.getName().replace("$", ".");
if (lib.annotationHandlers.put(annotationClassName, container) != null) {
error(null, "Duplicate handlers for annotation type: " + annotationClassName, null);
}
lib.typeLibrary.addType(container.annotationClass.getName());
} catch (Throwable t) {
error(null, "Can't load Lombok annotation handler for Eclipse: ", t);
}
}
} catch (IOException e) {
throw Lombok.sneakyThrow(e);
}
}
/** Uses SPI Discovery to find implementations of {@link EclipseASTVisitor}. */
private static void loadVisitorHandlers(HandlerLibrary lib) {
try {
for (EclipseASTVisitor visitor : SpiLoadUtil.findServices(EclipseASTVisitor.class,
EclipseASTVisitor.class.getClassLoader())) {
lib.visitorHandlers.add(new VisitorContainer(visitor));
}
} catch (Throwable t) {
throw Lombok.sneakyThrow(t);
}
}
private boolean checkAndSetHandled(ASTNode node) {
return !ASTNode_handled.getAndSet(node, true);
}
private boolean needsHandling(ASTNode node) {
return !ASTNode_handled.get(node);
}
/**
* Handles the provided annotation node by first finding a qualifying instance of
* {@link EclipseAnnotationHandler} and if one exists, calling it with a freshly cooked up
* instance of {@link AnnotationValues}.
*
* Note that depending on the printASTOnly flag, the {@link lombok.core.PrintAST} annotation
* will either be silently skipped, or everything that isn't {@code PrintAST} will be skipped.
*
* The HandlerLibrary will attempt to guess if the given annotation node represents a lombok annotation.
* For example, if {@code lombok.*} is in the import list, then this method will guess that
* {@code Getter} refers to {@code lombok.Getter}, presuming that {@link
lombok.eclipse.handlers.HandleGetter}
* has been loaded.
*
* @param ast The Compilation Unit that contains the Annotation AST Node.
* @param annotationNode The Lombok AST Node representing the Annotation AST Node.
* @param annotation 'node.get()' - convenience parameter.
```

```
* @param priority current prioritiy
* @return the priority we want to run - MAX_VALUE means never
*/
public long handleAnnotation(CompilationUnitDeclaration ast, EclipseNode annotationNode,
org.eclipse.jdt.internal.compiler.ast.Annotation annotation, long priority) {
TypeResolver resolver = new TypeResolver(annotationNode.getImportList());
TypeReference rawType = annotation.type;
if (rawType == null) return Long.MAX_VALUE;
String fqn = resolver.typeRefToFullyQualifiedName(annotationNode, typeLibrary,
toQualifiedName(annotation.type.getTypeName()));
if (fqn == null) return Long.MAX_VALUE;
AnnotationHandlerContainer<?> container = annotationHandlers.get(fqn);
if (container == null) return Long.MAX_VALUE;
if (priority < container.getPriority()) return container.getPriority(); // we want to run at this priority
if (priority > container.getPriority()) return Long.MAX_VALUE; // it's over- we do not want to run again
if (!annotationNode.isCompleteParse() && container.deferUntilPostDiet()) {
if (needsHandling(annotation)) container.preHandle(annotation, annotationNode);
return Long.MAX_VALUE;
}
try {
if (checkAndSetHandled(annotation)) container.handle(annotation, annotationNode);
} catch (AnnotationValueDecodeFail fail) {
fail.owner.setError(fail.getMessage(), fail.idx);
} catch (Throwable t) {
error(ast, String.format("Lombok annotation handler %s failed", container.handler.getClass()), t);
}
return Long.MAX_VALUE;
}
/**
* Will call all registered {@link EclipseASTVisitor} instances.
*/
public long callASTVisitors(EclipseAST ast, long priority, boolean isCompleteParse) {
long nearestPriority = Long.MAX_VALUE;
for (VisitorContainer container : visitorHandlers) {
if (priority < container.getPriority()) nearestPriority = Math.min(container.getPriority(), nearestPriority);
if (!isCompleteParse && container.deferUntilPostDiet()) continue;
if (priority != container.getPriority()) continue;
try {
ast.traverse(container.visitor);
} catch (Throwable t) {
error((CompilationUnitDeclaration) ast.top().get(),
String.format("Lombok visitor handler %s failed", container.visitor.getClass()), t);
}
}
return nearestPriority;
}
}
```

lombok/eclipse/TransformEclipseAST.java

```
/*
```

* Copyright (C) 2009-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import static lombok.eclipse.EcjAugments.CompilationUnitDeclaration_transformationState;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Field;
import lombok.ConfigurationKeys;
import lombok.core.LombokConfiguration;
import lombok.core.debug.DebugSnapshotStore;
import lombok.core.debug.HistogramTracker;
import lombok.patcher.Symbols;
import lombok.permit.Permit;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.parser.Parser;
/**
* Entry point for the Eclipse Parser patch that lets lombok modify the Abstract Syntax Tree as generated by
* Eclipse's parser implementations. This class is injected into the appropriate OSGi ClassLoader and can thus
* use any classes that belong to org.eclipse.jdt.(apt.)core.
*
* Note that, for any Method body, if Bit24 is set, the Eclipse parser has been patched to never attempt to
* (re)parse it. You should set Bit24 on any MethodDeclaration object you inject into the AST:
*
* {@code methodDeclaration.bits |= ASTNode.Bit24; //0x800000}
*/
public class TransformEclipseAST {

```
private final EclipseAST ast;
//The patcher hacks this field onto CUD. It's public.
private static final Field astCacheField;
private static final HandlerLibrary handlers;
public static boolean disableLombok = false;
private static final HistogramTracker lombokTracker;
static {
String v = System.getProperty("lombok.histogram");
if (v == null) lombokTracker = null;
else if (v.toLowerCase().equals("sysout")) lombokTracker = new HistogramTracker("lombok.histogram",
System.out);
else lombokTracker = new HistogramTracker("lombok.histogram");
}
static {
Field f = null;
HandlerLibrary h = null;
if (System.getProperty("lombok.disable") != null) {
disableLombok = true;
astCacheField = null;
handlers = null;
} else {
try {
h = HandlerLibrary.load();
} catch (Throwable t) {
try {
error(null, "Problem initializing lombok", t);
} catch (Throwable t2) {
System.err.println("Problem initializing lombok");
t.printStackTrace();
}
disableLombok = true;
}
try {
f = Permit.getField(CompilationUnitDeclaration.class, "$lombokAST");
} catch (Throwable t) {
//I guess we're in an ecj environment; we'll just not cache stuff then.
}
astCacheField = f;
handlers = h;
}
}
public static void transform_swapped(CompilationUnitDeclaration ast, Parser parser) {
transform(parser, ast);
}
public static EclipseAST getAST(CompilationUnitDeclaration ast, boolean forceRebuild) {
EclipseAST existing = null;
if (astCacheField != null) {
try {
existing = (EclipseAST) astCacheField.get(ast);
} catch (Exception e) {
// existing remains null
}
}
```

```
if (existing == null) {
existing = new EclipseAST(ast);
if (astCacheField != null) try {
astCacheField.set(ast, existing);
} catch (Exception ignore) {
}
} else {
existing.rebuild(forceRebuild);
}
return existing;
}
/**
* Check if lombok already handled the given AST. This method will return
* <code>true</code> once for diet mode and once for full mode.
*
* The reason for this is that Eclipse invokes the transform method multiple
* times during compilation and it is enough to transform it once and not
* repeat the whole thing over and over again.
*
* @param ast The AST node belonging to the compilation unit (java speak for a single source file).
* @return <code>true</code> if this AST was already handled by lombok.
*/
public static boolean alreadyTransformed(CompilationUnitDeclaration ast) {
TransformationState state = CompilationUnitDeclaration_transformationState.get(ast);
if (state == TransformationState.FULL) return true;
if (state == TransformationState.DIET) {
if (!EclipseAST.isComplete(ast)) return true;
CompilationUnitDeclaration_transformationState.set(ast, TransformationState.FULL);
} else {
CompilationUnitDeclaration_transformationState.set(ast, TransformationState.DIET);
}
return false;
}
/**
* This method is called immediately after Eclipse finishes building a CompilationUnitDeclaration, which
is
* the top-level AST node when Eclipse parses a source file. The signature is 'magic' - you should not
* change it!
*
* Eclipse's parsers often operate in diet mode, which means many parts of the AST have been left
blank.
* Be ready to deal with just about anything being null, such as the Statement[] arrays of the Method
AST nodes.
*
* @param parser The Eclipse parser object that generated the AST. Not actually used; mostly there to
satisfy parameter rules for lombok.patcher scripts.
* @param ast The AST node belonging to the compilation unit (java speak for a single source file).
*/
public static void transform(Parser parser, CompilationUnitDeclaration ast) {
if (disableLombok) return;
// Skip module-info.java
char[] fileName = ast.getFileName();
if (fileName != null && String.valueOf(fileName).endsWith("module-info.java")) return;
```

```
if (Symbols.hasSymbol("lombok.disable")) return;
// The IndexingParser only supports a single import statement, restricting lombok annotations to either fully qualified ones or
// those specified in the last import statement. To avoid handling hard to reproduce edge cases, we opt to ignore the entire parser.
if ("org.eclipse.jdt.internal.core.search.indexing.IndexingParser".equals(parser.getClass().getName())) return;
if (alreadyTransformed(ast)) return;
// Do NOT abort if (ast.bits & ASTNode.HasAllMethodBodies) != 0 - that doesn't work.
if (Boolean.TRUE.equals(LombokConfiguration.read(ConfigurationKeys.LOMBOK_DISABLE, EclipseAST.getAbsoluteFileLocation(ast)))) return;
try {
DebugSnapshotStore.INSTANCE.snapshot(ast, "transform entry");
long histoToken = lombokTracker == null ? 0L : lombokTracker.start();
EclipseAST existing = getAST(ast, false);
existing.setSource(parser.scanner.getSource());
new TransformEclipseAST(existing).go();
if (lombokTracker != null) lombokTracker.end(histoToken);
DebugSnapshotStore.INSTANCE.snapshot(ast, "transform exit");
} catch (Throwable t) {
DebugSnapshotStore.INSTANCE.snapshot(ast, "transform error: %s", t.getClass().getSimpleName());
try {
String message = "Lombok can't parse this source: " + t.toString();
EclipseAST.addProblemToCompilationResult(ast.getFileName(), ast.compilationResult, false, message, 0, 0);
t.printStackTrace();
} catch (Throwable t2) {
try {
error(ast, "Can't create an error in the problems dialog while adding: " + t.toString(), t2);
} catch (Throwable t3) {
//This seems risky to just silently turn off lombok, but if we get this far, something pretty
//drastic went wrong. For example, the eclipse help system's JSP compiler will trigger a lombok call,
//but due to class loader shenanigans we'll actually get here due to a cascade of
//ClassNotFoundErrors. This is the right action for the help system (no lombok needed for that JSP compiler,
//of course). 'disableLombok' is static, but each context classloader (e.g. each eclipse OSGi plugin) has
//it's own edition of this class, so this won't turn off lombok everywhere.
disableLombok = true;
}
}
}
}
public TransformEclipseAST(EclipseAST ast) {
this.ast = ast;
}
/**
* First handles all lombok annotations except PrintAST, then calls all non-annotation based handlers.
* then handles any PrintASTs.
*/
public void go() {
long nextPriority = Long.MIN_VALUE;
for (Long d : handlers.getPriorities()) {
if (nextPriority > d) continue;
```

```
AnnotationVisitor visitor = new AnnotationVisitor(d);
ast.traverse(visitor);
// if no visitor interested for this AST, nextPriority would be MAX_VALUE and we bail out immediatetly
nextPriority = visitor.getNextPriority();
nextPriority = Math.min(nextPriority, handlers.callASTVisitors(ast, d, ast.isCompleteParse()));
}
}
private static class AnnotationVisitor extends EclipseASTAdapter {
private final long priority;
// this is the next priority we continue to visit.
// Long.MAX_VALUE means never. Each visit method will potentially reduce the next priority
private long nextPriority = Long.MAX_VALUE;
public AnnotationVisitor(long priority) {
this.priority = priority;
}
public long getNextPriority() {
return nextPriority;
}
@Override public void visitAnnotationOnField(FieldDeclaration field, EclipseNode annotationNode,
Annotation annotation) {
CompilationUnitDeclaration top = (CompilationUnitDeclaration) annotationNode.top().get();
nextPriority = Math.min(nextPriority, handlers.handleAnnotation(top, annotationNode, annotation,
priority));
}
@Override public void visitAnnotationOnMethodArgument(Argument arg, AbstractMethodDeclaration
method, EclipseNode annotationNode, Annotation annotation) {
CompilationUnitDeclaration top = (CompilationUnitDeclaration) annotationNode.top().get();
nextPriority = Math.min(nextPriority, handlers.handleAnnotation(top, annotationNode, annotation,
priority));
}
@Override public void visitAnnotationOnLocal(LocalDeclaration local, EclipseNode annotationNode,
Annotation annotation) {
CompilationUnitDeclaration top = (CompilationUnitDeclaration) annotationNode.top().get();
nextPriority = Math.min(nextPriority, handlers.handleAnnotation(top, annotationNode, annotation,
priority));
}
@Override public void visitAnnotationOnMethod(AbstractMethodDeclaration method, EclipseNode
annotationNode, Annotation annotation) {
CompilationUnitDeclaration top = (CompilationUnitDeclaration) annotationNode.top().get();
nextPriority = Math.min(nextPriority, handlers.handleAnnotation(top, annotationNode, annotation,
priority));
}
@Override public void visitAnnotationOnType(TypeDeclaration type, EclipseNode annotationNode,
Annotation annotation) {
CompilationUnitDeclaration top = (CompilationUnitDeclaration) annotationNode.top().get();
nextPriority = Math.min(nextPriority, handlers.handleAnnotation(top, annotationNode, annotation,
priority));
}
@Override public void visitAnnotationOnTypeUse(TypeReference typeUse, EclipseNode
annotationNode, Annotation annotation) {
CompilationUnitDeclaration top = (CompilationUnitDeclaration) annotationNode.top().get();
nextPriority = Math.min(nextPriority, handlers.handleAnnotation(top, annotationNode, annotation,
priority));
```

```
}
}
}
```

lombok/eclipse/TransformationState.java

```
/*
* Copyright (C) 2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
enum TransformationState {
DIET,
FULL
}
```

lombok/eclipse/handlers/EclipseHandlerUtil.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.EcjAugments.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.EclipseReflectiveMembers.*;
import java.lang.reflect.Array;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import org.eclipse.jdt.core.compiler.CharOperation;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.ArrayQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ArrayTypeReference;
import org.eclipse.jdt.internal.compiler.ast.AssertStatement;
import org.eclipse.jdt.internal.compiler.ast.BinaryExpression;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.CastExpression;
import org.eclipse.jdt.internal.compiler.ast.CharLiteral;
import org.eclipse.jdt.internal.compiler.ast.ClassLiteralAccess;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.DoubleLiteral;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FalseLiteral;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.FloatLiteral;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.IntLiteral;
import org.eclipse.jdt.internal.compiler.ast.Literal;
import org.eclipse.jdt.internal.compiler.ast.LongLiteral;
import org.eclipse.jdt.internal.compiler.ast.MarkerAnnotation;

```
import org.eclipse.jdt.internal.compiler.ast.MemberValuePair;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NameReference;
import org.eclipse.jdt.internal.compiler.ast.NormalAnnotation;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleMemberAnnotation;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.ThrowStatement;
import org.eclipse.jdt.internal.compiler.ast.TrueLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeParameter;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.Wildcard;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.env.ICompilationUnit;
import org.eclipse.jdt.internal.compiler.lookup.Binding;
import org.eclipse.jdt.internal.compiler.lookup.CaptureBinding;
import org.eclipse.jdt.internal.compiler.lookup.ParameterizedTypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.RawTypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.ReferenceBinding;
import org.eclipse.jdt.internal.compiler.lookup.TypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
import org.eclipse.jdt.internal.compiler.lookup.WildcardBinding;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.Data;
import lombok.Getter;
import lombok.Lombok;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.AnnotationValues.AnnotationValue;
import lombok.core.LombokImmutableList;
import lombok.core.TypeResolver;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.configuration.NullAnnotationLibrary;
import lombok.core.configuration.NullCheckExceptionType;
import lombok.core.configuration.TypeName;
import lombok.core.debug.ProblemReporter;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.core.handlers.HandlerUtil.JavadocTag;
import lombok.eclipse.EcjAugments;
```

```
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAST;
import lombok.eclipse.EclipseNode;
import lombok.experimental.Accessors;
import lombok.experimental.Tolerate;
import lombok.permit.Permit;
/**
* Container for static utility methods useful to handlers written for eclipse.
*/
public class EclipseHandlerUtil {
private EclipseHandlerUtil() {
//Prevent instantiation
}
/**
* Generates an error in the Eclipse error log. Note that most people never look at it!
*
* @param cud The {@code CompilationUnitDeclaration} where the error occurred.
* An error will be generated on line 0 linking to the error log entry. Can be {@code null}.
* @param message Human readable description of the problem.
* @param ex The associated exception. Can be {@code null}.
*/
public static void error(CompilationUnitDeclaration cud, String message, Throwable ex) {
ProblemReporter.error(message, ex);
if (cud != null) EclipseAST.addProblemToCompilationResult(cud.getFileName(), cud.compilationResult,
false, message + " - See error log.", 0, 0);
}
/**
* Generates a warning in the Eclipse error log. Note that most people never look at it!
*
* @param message Human readable description of the problem.
* @param ex The associated exception. Can be {@code null}.
*/
public static void warning(String message, Throwable ex) {
ProblemReporter.warning(message, ex);
}
public static ASTNode getGeneratedBy(ASTNode node) {
return ASTNode_generatedBy.get(node);
}
public static boolean isGenerated(ASTNode node) {
return getGeneratedBy(node) != null;
}
public static <T extends ASTNode> T setGeneratedBy(T node, ASTNode source) {
ASTNode_generatedBy.set(node, source);
return node;
}
public static MarkerAnnotation generateDeprecatedAnnotation(ASTNode source) {
QualifiedTypeReference qtr = new QualifiedTypeReference(new char[][] {
{'j', 'a', 'v', 'a'}, {'l', 'a', 'n', 'g'}, {'D', 'e', 'p', 'r', 'e', 'c', 'a', 't', 'e', 'd'}}, poss(source, 3));
setGeneratedBy(qtr, source);
MarkerAnnotation ma = new MarkerAnnotation(qtr, source.sourceStart);
// No matter what value you input for sourceEnd, the AST->DOM converter of eclipse will reparse to find
the end, and will fail as
```

```
// it can't find code that isn't really there. This results in the end position being set to 2 or 0 or some weird magic value, and thus,
// length, as calculated by end-start, is all screwed up, resulting in IllegalArgumentException during a setSourceRange call MUCH later in the process.
// We solve it by going with a voodoo magic source start value such that the calculated length so happens to exactly be 0. 0 lengths are accepted
// by eclipse. For some reason.
// TL;DR: Don't change 1. 1 is sacred. Trust the 1.
// issue: #408.
ma.sourceStart = 1;
setGeneratedBy(ma, source);
return ma;
}
public static MarkerAnnotation generateNamedAnnotation(ASTNode source, String typeName) {
char[][] cc = fromQualifiedName(typeName);
QualifiedTypeReference qtr = new QualifiedTypeReference(cc, poss(source, cc.length));
setGeneratedBy(qtr, source);
MarkerAnnotation ma = new MarkerAnnotation(qtr, source.sourceStart);
// No matter what value you input for sourceEnd, the AST->DOM converter of eclipse will reparse to find the end, and will fail as
// it can't find code that isn't really there. This results in the end position being set to 2 or 0 or some weird magic value, and thus,
// length, as calculated by end-start, is all screwed up, resulting in IllegalArgumentException during a setSourceRange call MUCH later in the process.
// We solve it by going with a voodoo magic source start value such that the calculated length so happens to exactly be 0. 0 lengths are accepted
// by eclipse. For some reason.
// TL;DR: Don't change 1. 1 is sacred. Trust the 1.
// issue: #408.
ma.sourceStart = 1;
setGeneratedBy(ma, source);
return ma;
}
public static boolean isFieldDeprecated(EclipseNode fieldNode) {
if (!(fieldNode.get() instanceof FieldDeclaration)) return false;
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
if ((field.modifiers & ClassFileConstants.AccDeprecated) != 0) {
return true;
}
if (field.annotations == null) return false;
for (Annotation annotation : field.annotations) {
if (typeMatches(Deprecated.class, fieldNode, annotation.type)) {
return true;
}
}
return false;
}
public static CheckerFrameworkVersion getCheckerFrameworkVersion(EclipseNode node) {
CheckerFrameworkVersion cfv =
node.getAst().readConfiguration(ConfigurationKeys.CHECKER_FRAMEWORK);
return cfv != null ? cfv : CheckerFrameworkVersion.NONE;
}
/**
```

```
* Checks if the given TypeReference node is likely to be a reference to the provided class.
*
* @param type An actual type. This method checks if {@code typeNode} is likely to be a reference to this type.
* @param node A Lombok AST node. Any node in the appropriate compilation unit will do (used to get access to import statements).
* @param typeRef A type reference to check.
*/
public static boolean typeMatches(Class<?> type, EclipseNode node, TypeReference typeRef) {
return typeMatches(type.getName(), node, typeRef);
}
/**
* Checks if the given TypeReference node is likely to be a reference to the provided class.
*
* @param type An actual type. This method checks if {@code typeNode} is likely to be a reference to this type.
* @param node A Lombok AST node. Any node in the appropriate compilation unit will do (used to get access to import statements).
* @param typeRef A type reference to check.
*/
public static boolean typeMatches(String type, EclipseNode node, TypeReference typeRef) {
char[][] tn = typeRef == null ? null : typeRef.getTypeName();
if (tn == null || tn.length == 0) return false;
char[] lastPartA = tn[tn.length - 1];
int lastIndex = Math.max(type.lastIndexOf('.'), type.lastIndexOf('$')) + 1;
if (lastPartA.length != type.length() - lastIndex) return false;
for (int i = 0; i < lastPartA.length; i++) if (lastPartA[i] != type.charAt(i + lastIndex)) return false;
String typeName = toQualifiedName(tn);
TypeResolver resolver = node.getImportListAsTypeResolver();
return resolver.typeMatches(node, type, typeName);
}
public static void sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(EclipseNode typeNode, EclipseNode errorNode) {
List<String> disallowed = null;
for (EclipseNode child : typeNode.down()) {
if (child.getKind() != Kind.ANNOTATION) continue;
for (String annType : INVALID_ON_BUILDERS) {
if (annotationTypeMatches(annType, child)) {
if (disallowed == null) disallowed = new ArrayList<String>();
int lastIndex = annType.lastIndexOf('.');
disallowed.add(lastIndex == -1 ? annType : annType.substring(lastIndex + 1));
}
}
}
int size = disallowed == null ? 0 : disallowed.size();
if (size == 0) return;
if (size == 1) {
errorNode.addError("@" + disallowed.get(0) + " is not allowed on builder classes.");
return;
}
StringBuilder out = new StringBuilder();
for (String a : disallowed) out.append("@").append(a).append(", ");
out.setLength(out.length() - 2);
```

```
errorNode.addError(out.append(" are not allowed on builder classes.").toString());
}
public static Annotation copyAnnotation(Annotation annotation, ASTNode source) {
int pS = source.sourceStart, pE = source.sourceEnd;
if (annotation instanceof MarkerAnnotation) {
MarkerAnnotation ann = new MarkerAnnotation(copyType(annotation.type, source), pS);
setGeneratedBy(ann, source);
ann.declarationSourceEnd = ann.sourceEnd = ann.statementEnd = pE;
copyMemberValuePairName(ann, annotation);
return ann;
}
if (annotation instanceof SingleMemberAnnotation) {
SingleMemberAnnotation ann = new SingleMemberAnnotation(copyType(annotation.type, source),
pS);
setGeneratedBy(ann, source);
ann.declarationSourceEnd = ann.sourceEnd = ann.statementEnd = pE;
ann.memberValue = copyAnnotationMemberValue(((SingleMemberAnnotation)
annotation).memberValue);
copyMemberValuePairName(ann, annotation);
return ann;
}
if (annotation instanceof NormalAnnotation) {
NormalAnnotation ann = new NormalAnnotation(copyType(annotation.type, source), pS);
setGeneratedBy(ann, source);
ann.declarationSourceEnd = ann.statementEnd = ann.sourceEnd = pE;
MemberValuePair[] inPairs = ((NormalAnnotation) annotation).memberValuePairs;
if (inPairs == null) {
ann.memberValuePairs = null;
} else {
ann.memberValuePairs = new MemberValuePair[inPairs.length];
for (int i = 0; i < inPairs.length; i++) ann.memberValuePairs[i] =
new MemberValuePair(inPairs[i].name, inPairs[i].sourceStart, inPairs[i].sourceEnd,
copyAnnotationMemberValue(inPairs[i].value));
}
copyMemberValuePairName(ann, annotation);
return ann;
}
return annotation;
}
private static void copyMemberValuePairName(Annotation source, Annotation target) {
if (ANNOTATION__MEMBER_VALUE_PAIR_NAME == null) return;
try {
reflectSet(ANNOTATION__MEMBER_VALUE_PAIR_NAME, source,
reflect(ANNOTATION__MEMBER_VALUE_PAIR_NAME, target));
} catch (Exception ignore) { /* Various eclipse versions don't have it */ }
}
static class EclipseReflectiveMembers {
public static final Field STRING_LITERAL__LINE_NUMBER;
public static final Field ANNOTATION__MEMBER_VALUE_PAIR_NAME;
public static final Field TYPE_REFERENCE__ANNOTATIONS;
public static final Class<?> INTERSECTION_BINDING1, INTERSECTION_BINDING2;
public static final Field INTERSECTION_BINDING_TYPES1, INTERSECTION_BINDING_TYPES2;
public static final Field TYPE_DECLARATION_RECORD_COMPONENTS;
```

```
public static final Class<?> COMPILATION_UNIT;
public static final Method COMPILATION_UNIT_ORIGINAL_FROM_CLONE;
static {
STRING_LITERAL__LINE_NUMBER = getField(StringLiteral.class, "lineNumber");
ANNOTATION__MEMBER_VALUE_PAIR_NAME = getField(Annotation.class,
"memberValuePairName");
TYPE_REFERENCE__ANNOTATIONS = getField(TypeReference.class, "annotations");
INTERSECTION_BINDING1 =
getClass("org.eclipse.jdt.internal.compiler.lookup.IntersectionTypeBinding18");
INTERSECTION_BINDING2 =
getClass("org.eclipse.jdt.internal.compiler.lookup.IntersectionCastTypeBinding");
INTERSECTION_BINDING_TYPES1 = INTERSECTION_BINDING1 == null ? null :
getField(INTERSECTION_BINDING1, "intersectingTypes");
INTERSECTION_BINDING_TYPES2 = INTERSECTION_BINDING2 == null ? null :
getField(INTERSECTION_BINDING2, "intersectingTypes");
TYPE_DECLARATION_RECORD_COMPONENTS = getField(TypeDeclaration.class,
"recordComponents");
COMPILATION_UNIT = getClass("org.eclipse.jdt.internal.core.CompilationUnit");
COMPILATION_UNIT_ORIGINAL_FROM_CLONE = COMPILATION_UNIT == null ? null :
Permit.permissiveGetMethod(COMPILATION_UNIT, "originalFromClone");
}
public static int reflectInt(Field f, Object o) {
try {
return ((Number) f.get(o)).intValue();
} catch (IllegalAccessException e) {
throw new RuntimeException(e);
}
}
public static void reflectSet(Field f, Object o, Object v) {
try {
f.set(o, v);
} catch (IllegalAccessException e) {
throw new RuntimeException(e);
}
}
public static Object reflect(Field f, Object o) {
try {
return f.get(o);
} catch (IllegalAccessException e) {
throw new RuntimeException(e);
}
}
private static Class<?> getClass(String fqn) {
try {
return Class.forName(fqn);
} catch (Throwable t) {
return null;
}
}
private static Field getField(Class<?> c, String fName) {
try {
return Permit.getField(c, fName);
} catch (Throwable t) {
```

```
return null;
}
}
}
public static Expression copyAnnotationMemberValue(Expression in) {
Expression out = copyAnnotationMemberValue0(in);
out.constant = in.constant;
return out;
}
private static Expression copyAnnotationMemberValue0(Expression in) {
int s = in.sourceStart, e = in.sourceEnd;
// literals
if (in instanceof FalseLiteral) return new FalseLiteral(s, e);
if (in instanceof TrueLiteral) return new TrueLiteral(s, e);
if (in instanceof NullLiteral) return new NullLiteral(s, e);
if (in instanceof CharLiteral) return new CharLiteral(((Literal) in).source(), s, e);
if (in instanceof DoubleLiteral) return new DoubleLiteral(((Literal) in).source(), s, e);
if (in instanceof FloatLiteral) return new FloatLiteral(((Literal) in).source(), s, e);
if (in instanceof IntLiteral) return IntLiteral.buildIntLiteral(((Literal) in).source(), s, e);
if (in instanceof LongLiteral) return LongLiteral.buildLongLiteral(((Literal) in).source(), s, e);
if (in instanceof StringLiteral) return new StringLiteral(((Literal) in).source(), s, e,
reflectInt(STRING_LITERAL__LINE_NUMBER, in) + 1);
// enums and field accesses (as long as those are references to compile time constant literals that's
also acceptable)
if (in instanceof SingleNameReference) {
SingleNameReference snr = (SingleNameReference) in;
return new SingleNameReference(snr.token, pos(in));
}
if (in instanceof QualifiedNameReference) {
QualifiedNameReference qnr = (QualifiedNameReference) in;
return new QualifiedNameReference(qnr.tokens, poss(in, qnr.tokens.length), s, e);
}
// class refs
if (in instanceof ClassLiteralAccess) return new ClassLiteralAccess(e, copyType(((ClassLiteralAccess)
in).type));
// arrays
if (in instanceof ArrayInitializer) {
ArrayInitializer out = new ArrayInitializer();
out.sourceStart = s;
out.sourceEnd = e;
out.bits = in.bits;
out.implicitConversion = in.implicitConversion;
out.statementEnd = e;
Expression[] exprs = ((ArrayInitializer) in).expressions;
if (exprs != null) {
Expression[] copy = new Expression[exprs.length];
for (int i = 0; i < exprs.length; i++) copy[i] = copyAnnotationMemberValue(exprs[i]);
out.expressions = copy;
}
return out;
}
if (in instanceof BinaryExpression) {
BinaryExpression be = (BinaryExpression) in;
```

```
BinaryExpression out = new BinaryExpression(be);
out.left = copyAnnotationMemberValue(be.left);
out.right = copyAnnotationMemberValue(be.right);
out.sourceStart = s;
out.sourceEnd = e;
out.statementEnd = e;
return out;
}
return in;
}
/**
* You can't share TypeParameter objects or bad things happen; for example, one 'T' resolves differently
* from another 'T', even for the same T in a single class file. Unfortunately the TypeParameter type
hierarchy
* is complicated and there's no clone method on TypeParameter itself. This method can clone them.
*/
public static TypeParameter[] copyTypeParams(TypeParameter[] params, ASTNode source) {
if (params == null) return null;
TypeParameter[] out = new TypeParameter[params.length];
int idx = 0;
for (TypeParameter param : params) {
TypeParameter o = new TypeParameter();
setGeneratedBy(o, source);
o.annotations = param.annotations;
o.bits = param.bits;
o.modifiers = param.modifiers;
o.name = param.name;
o.type = copyType(param.type, source);
o.sourceStart = param.sourceStart;
o.sourceEnd = param.sourceEnd;
o.declarationEnd = param.declarationEnd;
o.declarationSourceStart = param.declarationSourceStart;
o.declarationSourceEnd = param.declarationSourceEnd;
if (param.bounds != null) {
TypeReference[] b = new TypeReference[param.bounds.length];
int idx2 = 0;
for (TypeReference ref : param.bounds) b[idx2++] = copyType(ref, source);
o.bounds = b;
}
out[idx++] = o;
}
return out;
}
public static Annotation[] getTypeUseAnnotations(TypeReference from) {
Annotation[][] a;
try {
a = (Annotation[][]) reflect(TYPE_REFERENCE__ANNOTATIONS, from);
} catch (Exception e) {
return null;
}
if (a == null) return null;
Annotation[] b = a[a.length - 1];
return b.length == 0 ? null : b;
```

```
}
public static void removeTypeUseAnnotations(TypeReference from) {
try {
reflectSet(TYPE_REFERENCE__ANNOTATIONS, from, null);
} catch (Exception ignore) {}
}
public static TypeReference namePlusTypeParamsToTypeReference(EclipseNode type,
TypeParameter[] params, long p) {
TypeDeclaration td = (TypeDeclaration) type.get();
boolean instance = (td.modifiers & MODIFIERS_INDICATING_STATIC) == 0;
return namePlusTypeParamsToTypeReference(type.up(), td.name, instance, params, p);
}
public static TypeReference namePlusTypeParamsToTypeReference(EclipseNode parentType, char[]
typeName, boolean instance, TypeParameter[] params, long p) {
if (params != null && params.length > 0) {
TypeReference[] refs = new TypeReference[params.length];
int idx = 0;
for (TypeParameter param : params) {
TypeReference typeRef = new SingleTypeReference(param.name, p);
refs[idx++] = typeRef;
}
return generateParameterizedTypeReference(parentType, typeName, instance, refs, p);
}
return generateTypeReference(parentType, typeName, instance, p);
}
public static TypeReference[] copyTypes(TypeReference[] refs) {
return copyTypes(refs, null);
}
/**
* Convenience method that creates a new array and copies each TypeReference in the source array
via
* {@link #copyType(TypeReference, ASTNode)}.
*/
public static TypeReference[] copyTypes(TypeReference[] refs, ASTNode source) {
if (refs == null) return null;
TypeReference[] outs = new TypeReference[refs.length];
int idx = 0;
for (TypeReference ref : refs) {
outs[idx++] = copyType(ref, source);
}
return outs;
}
public static TypeReference copyType(TypeReference ref) {
return copyType(ref, null);
}
/**
* You can't share TypeReference objects or subtle errors start happening.
* Unfortunately the TypeReference type hierarchy is complicated and there's no clone
* method on TypeReference itself. This method can clone them.
*/
public static TypeReference copyType(TypeReference ref, ASTNode source) {
if (ref instanceof ParameterizedQualifiedTypeReference) {
ParameterizedQualifiedTypeReference iRef = (ParameterizedQualifiedTypeReference) ref;
```

```
TypeReference[][] args = null;
if (iRef.typeArguments != null) {
args = new TypeReference[iRef.typeArguments.length][];
int idx = 0;
for (TypeReference[] inRefArray : iRef.typeArguments) {
if (inRefArray == null) args[idx++] = null;
else {
TypeReference[] outRefArray = new TypeReference[inRefArray.length];
int idx2 = 0;
for (TypeReference inRef : inRefArray) {
outRefArray[idx2++] = copyType(inRef, source);
}
args[idx++] = outRefArray;
}
}
}
TypeReference typeRef = new ParameterizedQualifiedTypeReference(iRef.tokens, args,
iRef.dimensions(), copy(iRef.sourcePositions));
copyTypeAnns(ref, typeRef);
if (source != null) setGeneratedBy(typeRef, source);
return typeRef;
}
if (ref instanceof ArrayQualifiedTypeReference) {
ArrayQualifiedTypeReference iRef = (ArrayQualifiedTypeReference) ref;
TypeReference typeRef = new ArrayQualifiedTypeReference(iRef.tokens, iRef.dimensions(),
copy(iRef.sourcePositions));
copyTypeAnns(ref, typeRef);
if (source != null) setGeneratedBy(typeRef, source);
return typeRef;
}
if (ref instanceof QualifiedTypeReference) {
QualifiedTypeReference iRef = (QualifiedTypeReference) ref;
TypeReference typeRef = new QualifiedTypeReference(iRef.tokens, copy(iRef.sourcePositions));
copyTypeAnns(ref, typeRef);
if (source != null) setGeneratedBy(typeRef, source);
return typeRef;
}
if (ref instanceof ParameterizedSingleTypeReference) {
ParameterizedSingleTypeReference iRef = (ParameterizedSingleTypeReference) ref;
TypeReference[] args = null;
if (iRef.typeArguments != null) {
args = new TypeReference[iRef.typeArguments.length];
int idx = 0;
for (TypeReference inRef : iRef.typeArguments) {
if (inRef == null) args[idx++] = null;
else args[idx++] = copyType(inRef, source);
}
}
TypeReference typeRef = new ParameterizedSingleTypeReference(iRef.token, args,
iRef.dimensions(), (long) iRef.sourceStart << 32 | iRef.sourceEnd);
copyTypeAnns(ref, typeRef);
if (source != null) setGeneratedBy(typeRef, source);
return typeRef;
```

```
}
if (ref instanceof ArrayTypeReference) {
ArrayTypeReference iRef = (ArrayTypeReference) ref;
TypeReference typeRef = new ArrayTypeReference(iRef.token, iRef.dimensions(), (long)
iRef.sourceStart << 32 | iRef.sourceEnd);
copyTypeAnns(ref, typeRef);
if (source != null) setGeneratedBy(typeRef, source);
return typeRef;
}
if (ref instanceof Wildcard) {
Wildcard original = (Wildcard) ref;
Wildcard wildcard = new Wildcard(original.kind);
wildcard.sourceStart = original.sourceStart;
wildcard.sourceEnd = original.sourceEnd;
if (original.bound != null) wildcard.bound = copyType(original.bound, source);
copyTypeAnns(ref, wildcard);
if (source != null) setGeneratedBy(wildcard, source);
return wildcard;
}
if (ref instanceof SingleTypeReference) {
SingleTypeReference iRef = (SingleTypeReference) ref;
TypeReference typeRef = new SingleTypeReference(iRef.token, (long) iRef.sourceStart << 32 |
iRef.sourceEnd);
copyTypeAnns(ref, typeRef);
if (source != null) setGeneratedBy(typeRef, source);
return typeRef;
}
return ref;
}
private static void copyTypeAnns(TypeReference in, TypeReference out) {
Annotation[][] a;
try {
a = (Annotation[][]) reflect(TYPE_REFERENCE__ANNOTATIONS, in);
} catch (Exception e) {
return;
}
if (a == null) {
reflectSet(TYPE_REFERENCE__ANNOTATIONS, out, null);
return;
}
Annotation[][] b = new Annotation[a.length][];
for (int i = 0; i < a.length; i++) {
if (a[i] != null) {
b[i] = new Annotation[a[i].length];
for (int j = 0 ; j < a[i].length; j++) {
b[i][j] = copyAnnotation(a[i][j], a[i][j]);
}
}
}
reflectSet(TYPE_REFERENCE__ANNOTATIONS, out, b);
}
public static Annotation[] copyAnnotations(ASTNode source, Annotation[]... allAnnotations) {
List<Annotation> result = null;
```

```
for (Annotation[] annotations : allAnnotations) {
if (annotations != null) {
for (Annotation annotation : annotations) {
if (result == null) result = new ArrayList<Annotation>();
result.add(copyAnnotation(annotation, source));
}
}
}
return result == null ? null : result.toArray(new Annotation[0]);
}
public static boolean hasAnnotation(Class<? extends java.lang.annotation.Annotation> type,
EclipseNode node) {
if (node == null) return false;
if (type == null) return false;
switch (node.getKind()) {
case ARGUMENT:
case FIELD:
case LOCAL:
case TYPE:
case METHOD:
for (EclipseNode child : node.down()) {
if (annotationTypeMatches(type, child)) return true;
}
// intentional fallthrough
default:
return false;
}
}
public static boolean hasAnnotation(String type, EclipseNode node) {
if (node == null) return false;
if (type == null) return false;
switch (node.getKind()) {
case ARGUMENT:
case FIELD:
case LOCAL:
case TYPE:
case METHOD:
for (EclipseNode child : node.down()) {
if (annotationTypeMatches(type, child)) return true;
}
// intentional fallthrough
default:
return false;
}
}
public static EclipseNode findInnerClass(EclipseNode parent, String name) {
char[] c = name.toCharArray();
for (EclipseNode child : parent.down()) {
if (child.getKind() != Kind.TYPE) continue;
TypeDeclaration td = (TypeDeclaration) child.get();
if (Arrays.equals(td.name, c)) return child;
}
return null;
```

```
}
public static EclipseNode findAnnotation(Class<? extends java.lang.annotation.Annotation> type,
EclipseNode node) {
if (node == null) return null;
if (type == null) return null;
switch (node.getKind()) {
case ARGUMENT:
case FIELD:
case LOCAL:
case TYPE:
case METHOD:
for (EclipseNode child : node.down()) {
if (annotationTypeMatches(type, child)) return child;
}
// intentional fallthrough
default:
return null;
}
}
public static String scanForNearestAnnotation(EclipseNode node, String... anns) {
while (node != null) {
for (EclipseNode ann : node.down()) {
if (ann.getKind() != Kind.ANNOTATION) continue;
Annotation a = (Annotation) ann.get();
TypeReference aType = a.type;
for (String annToFind : anns) if (typeMatches(annToFind, node, aType)) return annToFind;
}
node = node.up();
}
return null;
}
public static boolean hasNonNullAnnotations(EclipseNode node) {
for (EclipseNode child : node.down()) {
if (child.getKind() != Kind.ANNOTATION) continue;
Annotation annotation = (Annotation) child.get();
for (String bn : NONNULL_ANNOTATIONS) if (typeMatches(bn, node, annotation.type)) return true;
}
return false;
}
public static boolean hasNonNullAnnotations(EclipseNode node, List<Annotation> anns) {
if (anns == null) return false;
for (Annotation annotation : anns) {
TypeReference typeRef = annotation.type;
if (typeRef != null && typeRef.getTypeName() != null) {
for (String bn : NONNULL_ANNOTATIONS) if (typeMatches(bn, node, typeRef)) return true;
}
}
return false;
}
private static final Annotation[] EMPTY_ANNOTATIONS_ARRAY = new Annotation[0];
/**
* Searches the given field node for annotations and returns each one that is 'copyable' (either via
configuration or from the base list).
```

```
*/
public static Annotation[] findCopyableAnnotations(EclipseNode node) {
List<Annotation> result = new ArrayList<Annotation>();
List<TypeName> configuredCopyable =
node.getAst().readConfiguration(ConfigurationKeys.COPYABLE_ANNOTATIONS);
for (EclipseNode child : node.down()) {
if (child.getKind() != Kind.ANNOTATION) continue;
Annotation annotation = (Annotation) child.get();
TypeReference typeRef = annotation.type;
boolean match = false;
if (typeRef != null && typeRef.getTypeName() != null) {
for (TypeName cn : configuredCopyable) if (cn != null && typeMatches(cn.toString(), node, typeRef)) {
result.add(annotation);
match = true;
break;
}
if (!match) for (String bn : BASE_COPYABLE_ANNOTATIONS) if (typeMatches(bn, node, typeRef)) {
result.add(annotation);
break;
}
}
}
return result.toArray(EMPTY_ANNOTATIONS_ARRAY);
}
/**
* Searches the given field node for annotations that are specifically intentioned to be copied to the
setter.
*/
public static Annotation[] findCopyableToSetterAnnotations(EclipseNode node) {
return findAnnotationsInList(node, COPY_TO_SETTER_ANNOTATIONS);
}
/**
* Searches the given field node for annotations that are specifically intentioned to be copied to the
builder's singular method.
*/
public static Annotation[] findCopyableToBuilderSingularSetterAnnotations(EclipseNode node) {
return findAnnotationsInList(node, COPY_TO_BUILDER_SINGULAR_SETTER_ANNOTATIONS);
}
/**
* Searches the given field node for annotations that are in the given list, and returns those.
*/
private static Annotation[] findAnnotationsInList(EclipseNode node, java.util.List<String>
annotationsToFind) {
List<Annotation> result = new ArrayList<Annotation>();
for (EclipseNode child : node.down()) {
if (child.getKind() != Kind.ANNOTATION) continue;
Annotation annotation = (Annotation) child.get();
TypeReference typeRef = annotation.type;
if (typeRef != null && typeRef.getTypeName() != null) {
for (String bn : annotationsToFind) if (typeMatches(bn, node, typeRef)) {
result.add(annotation);
break;
}
```

```
}
}
return result.toArray(EMPTY_ANNOTATIONS_ARRAY);
}
/**
* Checks if the provided annotation type is likely to be the intended type for the given annotation node.
*
* This is a guess, but a decent one.
*/
public static boolean annotationTypeMatches(Class<? extends java.lang.annotation.Annotation> type,
EclipseNode node) {
if (node.getKind() != Kind.ANNOTATION) return false;
return typeMatches(type, node, ((Annotation) node.get()).type);
}
/**
* Checks if the provided annotation type is likely to be the intended type for the given annotation node.
*
* This is a guess, but a decent one.
*/
public static boolean annotationTypeMatches(String type, EclipseNode node) {
if (node.getKind() != Kind.ANNOTATION) return false;
return typeMatches(type, node, ((Annotation) node.get()).type);
}
public static TypeReference cloneSelfType(EclipseNode context) {
return cloneSelfType(context, null);
}
public static TypeReference cloneSelfType(EclipseNode context, ASTNode source) {
int pS = source == null ? 0 : source.sourceStart, pE = source == null ? 0 : source.sourceEnd;
long p = (long) pS << 32 | pE;
EclipseNode type = context;
TypeReference result = null;
while (type != null && type.getKind() != Kind.TYPE) type = type.up();
if (type != null && type.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration) type.get();
if (typeDecl.typeParameters != null && typeDecl.typeParameters.length > 0) {
TypeReference[] refs = new TypeReference[typeDecl.typeParameters.length];
int idx = 0;
for (TypeParameter param : typeDecl.typeParameters) {
TypeReference typeRef = new SingleTypeReference(param.name, (long)param.sourceStart << 32 |
param.sourceEnd);
if (source != null) setGeneratedBy(typeRef, source);
refs[idx++] = typeRef;
}
result = generateParameterizedTypeReference(type, refs, p);
} else {
result = generateTypeReference(type, p);
}
}
if (result != null && source != null) setGeneratedBy(result, source);
return result;
}
public static TypeReference generateParameterizedTypeReference(EclipseNode type,
TypeReference[] typeParams, long p) {
```

```
TypeDeclaration td = (TypeDeclaration) type.get();
char[][] tn = getQualifiedInnerName(type.up(), td.name);
if (tn.length == 1) return new ParameterizedSingleTypeReference(tn[0], typeParams, 0, p);
int tnLen = tn.length;
long[] ps = new long[tnLen];
for (int i = 0; i < tnLen; i++) ps[i] = p;
TypeReference[][] rr = new TypeReference[tnLen][];
rr[tnLen - 1] = typeParams;
boolean instance = (td.modifiers & MODIFIERS_INDICATING_STATIC) == 0;
if (instance) fillOuterTypeParams(rr, tnLen - 2, type.up(), p);
return new ParameterizedQualifiedTypeReference(tn, rr, 0, ps);
}
public static TypeReference generateParameterizedTypeReference(EclipseNode parent, char[] name,
boolean instance, TypeReference[] typeParams, long p) {
char[][] tn = getQualifiedInnerName(parent, name);
if (tn.length == 1) return new ParameterizedSingleTypeReference(tn[0], typeParams, 0, p);
int tnLen = tn.length;
long[] ps = new long[tnLen];
for (int i = 0; i < tnLen; i++) ps[i] = p;
TypeReference[][] rr = new TypeReference[tnLen][];
rr[tnLen - 1] = typeParams;
if (instance) fillOuterTypeParams(rr, tnLen - 2, parent, p);
return new ParameterizedQualifiedTypeReference(tn, rr, 0, ps);
}
private static final int MODIFIERS_INDICATING_STATIC = ClassFileConstants.AccInterface |
ClassFileConstants.AccStatic | ClassFileConstants.AccEnum | Eclipse.AccRecord;
/**
* This class will add type params to fully qualified chain of type references for inner types, such as
{@code GrandParent.Parent.Child}; this is needed only as long as the chain does not involve static.
*
* @return {@code true} if at least one parameterization is actually added, {@code false} otherwise.
*/
private static boolean fillOuterTypeParams(TypeReference[][] rr, int idx, EclipseNode node, long p) {
if (idx < 0 || node == null || !(node.get() instanceof TypeDeclaration)) return false;
boolean filled = false;
TypeDeclaration td = (TypeDeclaration) node.get();
if (0 != (td.modifiers & (ClassFileConstants.AccInterface | ClassFileConstants.AccEnum))) {
// any class defs inside an enum or interface are static, even if not marked as such.
return false;
}
TypeParameter[] tps = td.typeParameters;
if (tps != null && tps.length > 0) {
TypeReference[] trs = new TypeReference[tps.length];
for (int i = 0; i < tps.length; i++) {
trs[i] = new SingleTypeReference(tps[i].name, p);
}
rr[idx] = trs;
filled = true;
}
if ((td.modifiers & MODIFIERS_INDICATING_STATIC) != 0) return filled; // Once we hit a static class,
no further typeparams needed.
boolean f2 = fillOuterTypeParams(rr, idx - 1, node.up(), p);
return f2 || filled;
```

```
}
public static NameReference generateNameReference(EclipseNode type, long p) {
char[][] tn = getQualifiedInnerName(type.up(), ((TypeDeclaration) type.get()).name);
if (tn.length == 1) return new SingleNameReference(tn[0], p);
int tnLen = tn.length;
long[] ps = new long[tnLen];
for (int i = 0; i < tnLen; i++) ps[i] = p;
int ss = (int) (p >> 32);
int se = (int) p;
return new QualifiedNameReference(tn, ps, ss, se);
}
public static NameReference generateNameReference(EclipseNode parent, char[] name, long p) {
char[][] tn = getQualifiedInnerName(parent, name);
if (tn.length == 1) return new SingleNameReference(tn[0], p);
int tnLen = tn.length;
long[] ps = new long[tnLen];
for (int i = 0; i < tnLen; i++) ps[i] = p;
int ss = (int) (p >> 32);
int se = (int) p;
return new QualifiedNameReference(tn, ps, ss, se);
}
public static TypeReference generateTypeReference(EclipseNode type, long p) {
TypeDeclaration td = (TypeDeclaration) type.get();
char[][] tn = getQualifiedInnerName(type.up(), td.name);
if (tn.length == 1) return new SingleTypeReference(tn[0], p);
int tnLen = tn.length;
long[] ps = new long[tnLen];
for (int i = 0; i < tnLen; i++) ps[i] = p;
boolean instance = (td.modifiers & MODIFIERS_INDICATING_STATIC) == 0 && type.up() != null &&
type.up().get() instanceof TypeDeclaration;
if (instance) {
TypeReference[][] trs = new TypeReference[tn.length][];
boolean filled = fillOuterTypeParams(trs, trs.length - 2, type.up(), p);
if (filled) return new ParameterizedQualifiedTypeReference(tn, trs, 0, ps);
}
return new QualifiedTypeReference(tn, ps);
}
public static TypeReference generateTypeReference(EclipseNode parent, char[] name, boolean
instance, long p) {
char[][] tn = getQualifiedInnerName(parent, name);
if (tn.length == 1) return new SingleTypeReference(tn[0], p);
int tnLen = tn.length;
long[] ps = new long[tnLen];
for (int i = 0; i < tnLen; i++) ps[i] = p;
if (instance && parent != null && parent.get() instanceof TypeDeclaration) {
TypeReference[][] trs = new TypeReference[tn.length][];
if (fillOuterTypeParams(trs, tn.length - 2, parent, p)) return new
ParameterizedQualifiedTypeReference(tn, trs, 0, ps);
}
return new QualifiedTypeReference(tn, ps);
}
/**
* Generate a chain of names for the enclosing classes.
```

```
*
* Given for example {@code class Outer { class Inner {} }} this would generate {@code char[][] {
"Outer", "Inner" }}.
* For method local and top level types, this generates a size-1 char[][] where the only char[] element is
{@code name} itself.
*/
public static char[][] getQualifiedInnerName(EclipseNode parent, char[] name) {
int count = 0;
EclipseNode n = parent;
while (n != null && n.getKind() == Kind.TYPE && n.get() instanceof TypeDeclaration) {
TypeDeclaration td = (TypeDeclaration) n.get();
if (td.name == null || td.name.length == 0) break;
count++;
n = n.up();
}
if (count == 0) return new char[][] { name };
char[][] res = new char[count + 1][];
res[count] = name;
n = parent;
while (count > 0) {
TypeDeclaration td = (TypeDeclaration) n.get();
res[--count] = td.name;
n = n.up();
}
return res;
}
private static final char[] OBJECT_SIG = "Ljava/lang/Object;".toCharArray();
private static int compare(char[] a, char[] b) {
if (a == null) return b == null ? 0 : -1;
if (b == null) return +1;
int len = Math.min(a.length, b.length);
for (int i = 0; i < len; i++) {
if (a[i] < b[i]) return -1;
if (a[i] > b[i]) return +1;
}
return a.length < b.length ? -1 : a.length > b.length ? +1 : 0;
}
public static TypeReference makeType(TypeBinding binding, ASTNode pos, boolean allowCompound)
{
Object[] arr = null;
if (binding.getClass() == EclipseReflectiveMembers.INTERSECTION_BINDING1) {
arr = (Object[])
EclipseReflectiveMembers.reflect(EclipseReflectiveMembers.INTERSECTION_BINDING_TYPES1,
binding);
} else if (binding.getClass() == EclipseReflectiveMembers.INTERSECTION_BINDING2) {
arr = (Object[])
EclipseReflectiveMembers.reflect(EclipseReflectiveMembers.INTERSECTION_BINDING_TYPES2,
binding);
}
if (arr != null) {
// Is there a class? Alphabetically lowest wins.
TypeBinding winner = null;
int winLevel = 0; // 100 = array, 50 = class, 20 = typevar, 15 = wildcard, 10 = interface, 1 = Object.
```

```
for (Object b : arr) {
if (b instanceof TypeBinding) {
TypeBinding tb = (TypeBinding) b;
int level = 0;
if (tb.isArrayType()) level = 100;
else if (tb.isClass()) level = 50;
else if (tb.isTypeVariable()) level = 20;
else if (tb.isWildcard()) level = 15;
else level = 10;
if (level == 50 && compare(tb.signature(), OBJECT_SIG) == 0) level = 1;
if (winLevel > level) continue;
if (winLevel < level) {
winner = tb;
winLevel = level;
continue;
}
if (compare(winner.signature(), tb.signature()) > 0) winner = tb;
}
}
binding = winner;
}
int dims = binding.dimensions();
binding = binding.leafComponentType();
// Primitives
char[] base = null;
switch (binding.id) {
case TypeIds.T_int:
base = TypeConstants.INT;
break;
case TypeIds.T_long:
base = TypeConstants.LONG;
break;
case TypeIds.T_short:
base = TypeConstants.SHORT;
break;
case TypeIds.T_byte:
base = TypeConstants.BYTE;
break;
case TypeIds.T_double:
base = TypeConstants.DOUBLE;
break;
case TypeIds.T_float:
base = TypeConstants.FLOAT;
break;
case TypeIds.T_boolean:
base = TypeConstants.BOOLEAN;
break;
case TypeIds.T_char:
base = TypeConstants.CHAR;
break;
case TypeIds.T_void:
base = TypeConstants.VOID;
break;
```

```
case TypeIds.T_null:
return null;
}
if (base != null) {
if (dims > 0) {
TypeReference result = new ArrayTypeReference(base, dims, pos(pos));
setGeneratedBy(result, pos);
return result;
}
TypeReference result = new SingleTypeReference(base, pos(pos));
setGeneratedBy(result, pos);
return result;
}
if (binding.isAnonymousType()) {
ReferenceBinding ref = (ReferenceBinding)binding;
ReferenceBinding[] supers = ref.superInterfaces();
if (supers == null || supers.length == 0) supers = new ReferenceBinding[] {ref.superclass()};
if (supers[0] == null) {
TypeReference result = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT,
poss(pos, 3));
setGeneratedBy(result, pos);
return result;
}
return makeType(supers[0], pos, false);
}
if (binding instanceof CaptureBinding) {
return makeType(((CaptureBinding)binding).wildcard, pos, allowCompound);
}
if (binding.isUnboundWildcard()) {
if (!allowCompound) {
TypeReference result = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT,
poss(pos, 3));
setGeneratedBy(result, pos);
return result;
} else {
Wildcard out = new Wildcard(Wildcard.UNBOUND);
setGeneratedBy(out, pos);
out.sourceStart = pos.sourceStart;
out.sourceEnd = pos.sourceEnd;
return out;
}
}
if (binding.isWildcard()) {
WildcardBinding wildcard = (WildcardBinding) binding;
if (wildcard.boundKind == Wildcard.EXTENDS) {
if (!allowCompound) {
TypeBinding bound = wildcard.bound;
boolean isObject = bound.id == TypeIds.T_JavaLangObject;
TypeBinding[] otherBounds = wildcard.otherBounds;
if (isObject && otherBounds != null && otherBounds.length > 0) {
return makeType(otherBounds[0], pos, false);
} else return makeType(bound, pos, false);
} else {
```

```
Wildcard out = new Wildcard(Wildcard.EXTENDS);
setGeneratedBy(out, pos);
out.bound = makeType(wildcard.bound, pos, false);
out.sourceStart = pos.sourceStart;
out.sourceEnd = pos.sourceEnd;
return out;
}
} else if (allowCompound && wildcard.boundKind == Wildcard.SUPER) {
Wildcard out = new Wildcard(Wildcard.SUPER);
setGeneratedBy(out, pos);
out.bound = makeType(wildcard.bound, pos, false);
out.sourceStart = pos.sourceStart;
out.sourceEnd = pos.sourceEnd;
return out;
} else {
TypeReference result = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT,
poss(pos, 3));
setGeneratedBy(result, pos);
return result;
}
}
// Keep moving up via 'binding.enclosingType()' and gather generics from each binding. We stop after a
local type, or a static type, or a top-level type.
// Finally, add however many nullTypeArgument[] arrays as that are missing, inverse the list, toArray it,
and use that as PTR's typeArgument argument.
List<TypeReference[]> params = new ArrayList<TypeReference[]>();
/* Calculate generics */
if (!(binding instanceof RawTypeBinding)) {
TypeBinding b = binding;
while (true) {
boolean isFinalStop = b.isLocalType() || !b.isMemberType() || b.enclosingType() == null;
TypeReference[] tyParams = null;
if (b instanceof ParameterizedTypeBinding) {
ParameterizedTypeBinding paramized = (ParameterizedTypeBinding) b;
if (paramized.arguments != null) {
tyParams = new TypeReference[paramized.arguments.length];
for (int i = 0; i < tyParams.length; i++) {
tyParams[i] = makeType(paramized.arguments[i], pos, true);
}
}
}
params.add(tyParams);
if (isFinalStop) break;
b = b.enclosingType();
}
}
char[][] parts;
if (binding.isTypeVariable()) {
parts = new char[][] { binding.shortReadableName() };
} else if (binding.isLocalType()) {
parts = new char[][] { binding.sourceName() };
} else {
String[] pkg = new String(binding.qualifiedPackageName()).split("\\.");
```

```
String[] name = new String(binding.qualifiedSourceName()).split("\\.");
if (pkg.length == 1 && pkg[0].isEmpty()) pkg = new String[0];
parts = new char[pkg.length + name.length][];
int ptr;
for (ptr = 0; ptr < pkg.length; ptr++) parts[ptr] = pkg[ptr].toCharArray();
for (; ptr < pkg.length + name.length; ptr++) parts[ptr] = name[ptr - pkg.length].toCharArray();
}
while (params.size() < parts.length) params.add(null);
Collections.reverse(params);
boolean isParamized = false;
for (TypeReference[] tyParams : params) {
if (tyParams != null) {
isParamized = true;
break;
}
}
if (isParamized) {
if (parts.length > 1) {
TypeReference[][] typeArguments = params.toArray(new TypeReference[0][]);
TypeReference result = new ParameterizedQualifiedTypeReference(parts, typeArguments, dims,
poss(pos, parts.length));
setGeneratedBy(result, pos);
return result;
}
TypeReference result = new ParameterizedSingleTypeReference(parts[0], params.get(0), dims,
pos(pos));
setGeneratedBy(result, pos);
return result;
}
if (dims > 0) {
if (parts.length > 1) {
TypeReference result = new ArrayQualifiedTypeReference(parts, dims, poss(pos, parts.length));
setGeneratedBy(result, pos);
return result;
}
TypeReference result = new ArrayTypeReference(parts[0], dims, pos(pos));
setGeneratedBy(result, pos);
return result;
}
if (parts.length > 1) {
TypeReference result = new QualifiedTypeReference(parts, poss(pos, parts.length));
setGeneratedBy(result, pos);
return result;
}
TypeReference result = new SingleTypeReference(parts[0], pos(pos));
setGeneratedBy(result, pos);
return result;
}
/**
* Provides AnnotationValues with the data it needs to do its thing.
*/
public static <A extends java.lang.annotation.Annotation> AnnotationValues<A>
createAnnotation(Class<A> type, final EclipseNode annotationNode) {
```

```
final Annotation annotation = (Annotation) annotationNode.get();
Map<String, AnnotationValue> values = new HashMap<String, AnnotationValue>();
MemberValuePair[] memberValuePairs = annotation.memberValuePairs();
if (memberValuePairs != null) for (final MemberValuePair pair : memberValuePairs) {
List<String> raws = new ArrayList<String>();
List<Object> expressionValues = new ArrayList<Object>();
List<Object> guesses = new ArrayList<Object>();
Expression[] expressions = null;
char[] n = pair.name;
String mName = (n == null || n.length == 0) ? "value" : new String(pair.name);
final Expression rhs = pair.value;
if (rhs instanceof ArrayInitializer) {
expressions = ((ArrayInitializer) rhs).expressions;
} else if (rhs != null) {
expressions = new Expression[] { rhs };
}
if (expressions != null) for (Expression ex : expressions) {
raws.add(ex.toString());
expressionValues.add(ex);
guesses.add(calculateValue(ex));
}
final Expression[] exprs = expressions;
values.put(mName, new AnnotationValue(annotationNode, raws, expressionValues, guesses, true) {
@Override public void setError(String message, int valueIdx) {
Expression ex;
if (valueIdx == -1) ex = rhs;
else ex = exprs != null ? exprs[valueIdx] : null;
if (ex == null) ex = annotation;
int sourceStart = ex.sourceStart;
int sourceEnd = ex.sourceEnd;
annotationNode.addError(message, sourceStart, sourceEnd);
}
@Override public void setWarning(String message, int valueIdx) {
Expression ex;
if (valueIdx == -1) ex = rhs;
else ex = exprs != null ? exprs[valueIdx] : null;
if (ex == null) ex = annotation;
int sourceStart = ex.sourceStart;
int sourceEnd = ex.sourceEnd;
annotationNode.addWarning(message, sourceStart, sourceEnd);
}
});
}
for (Method m : type.getDeclaredMethods()) {
if (!Modifier.isPublic(m.getModifiers())) continue;
String name = m.getName();
if (!values.containsKey(name)) {
values.put(name, new AnnotationValue(annotationNode, new ArrayList<String>(), new
ArrayList<Object>(), new ArrayList<Object>(), false) {
@Override public void setError(String message, int valueIdx) {
annotationNode.addError(message);
}
@Override public void setWarning(String message, int valueIdx) {
```

```
annotationNode.addWarning(message);
}
});
}
}
return new AnnotationValues<A>(type, values, annotationNode);
}
/**
* Turns an {@code AccessLevel} instance into the flag bit used by eclipse.
*/
public static int toEclipseModifier(AccessLevel value) {
switch (value) {
case MODULE:
case PACKAGE:
return 0;
default:
case PUBLIC:
return ClassFileConstants.AccPublic;
case PROTECTED:
return ClassFileConstants.AccProtected;
case NONE:
case PRIVATE:
return ClassFileConstants.AccPrivate;
}
}
private static class GetterMethod {
private final char[] name;
private final TypeReference type;
GetterMethod(char[] name, TypeReference type) {
this.name = name;
this.type = type;
}
}
static void registerCreatedLazyGetter(FieldDeclaration field, char[] methodName, TypeReference
returnType) {
if (isBoolean(returnType)) {
FieldDeclaration_booleanLazyGetter.set(field, true);
}
}
public static boolean isBoolean(TypeReference typeReference) {
return nameEquals(typeReference.getTypeName(), "boolean") && typeReference.dimensions() == 0;
}
private static GetterMethod findGetter(EclipseNode field) {
FieldDeclaration fieldDeclaration = (FieldDeclaration) field.get();
boolean forceBool = FieldDeclaration_booleanLazyGetter.get(fieldDeclaration);
TypeReference fieldType = fieldDeclaration.type;
boolean isBoolean = forceBool || isBoolean(fieldType);
EclipseNode typeNode = field.up();
for (String potentialGetterName : toAllGetterNames(field, isBoolean)) {
for (EclipseNode potentialGetter : typeNode.down()) {
if (potentialGetter.getKind() != Kind.METHOD) continue;
if (!(potentialGetter.get() instanceof MethodDeclaration)) continue;
MethodDeclaration method = (MethodDeclaration) potentialGetter.get();
```

```
if (!potentialGetterName.equalsIgnoreCase(new String(method.selector))) continue;
/** static getX() methods don't count. */
if ((method.modifiers & ClassFileConstants.AccStatic) != 0) continue;
/** Nor do getters with a non-empty parameter list. */
if (method.arguments != null && method.arguments.length > 0) continue;
return new GetterMethod(method.selector, method.returnType);
}
}
// Check if the field has a @Getter annotation.
boolean hasGetterAnnotation = false;
for (EclipseNode child : field.down()) {
if (child.getKind() == Kind.ANNOTATION && annotationTypeMatches(Getter.class, child)) {
AnnotationValues<Getter> ann = createAnnotation(Getter.class, child);
if (ann.getInstance().value() == AccessLevel.NONE) return null; //Definitely WONT have a getter.
hasGetterAnnotation = true;
}
}
// Check if the class has a @Getter annotation.
if (!hasGetterAnnotation && HandleGetter.fieldQualifiesForGetterGeneration(field)) {
//Check if the class has @Getter or @Data annotation.
EclipseNode containingType = field.up();
if (containingType != null) for (EclipseNode child : containingType.down()) {
if (child.getKind() == Kind.ANNOTATION && annotationTypeMatches(Data.class, child))
hasGetterAnnotation = true;
if (child.getKind() == Kind.ANNOTATION && annotationTypeMatches(Getter.class, child)) {
AnnotationValues<Getter> ann = createAnnotation(Getter.class, child);
if (ann.getInstance().value() == AccessLevel.NONE) return null; //Definitely WONT have a getter.
hasGetterAnnotation = true;
}
}
}
if (hasGetterAnnotation) {
String getterName = toGetterName(field, isBoolean);
if (getterName == null) return null;
return new GetterMethod(getterName.toCharArray(), fieldType);
}
return null;
}
static boolean lookForGetter(EclipseNode field, FieldAccess fieldAccess) {
if (fieldAccess == FieldAccess.GETTER) return true;
if (fieldAccess == FieldAccess.ALWAYS_FIELD) return false;
// If @Getter(lazy = true) is used, then using it is mandatory.
for (EclipseNode child : field.down()) {
if (child.getKind() != Kind.ANNOTATION) continue;
if (annotationTypeMatches(Getter.class, child)) {
AnnotationValues<Getter> ann = createAnnotation(Getter.class, child);
if (ann.getInstance().lazy()) return true;
}
}
return false;
}
static TypeReference getFieldType(EclipseNode field, FieldAccess fieldAccess) {
if (field.get() instanceof MethodDeclaration) return ((MethodDeclaration) field.get()).returnType;
```

```
boolean lookForGetter = lookForGetter(field, fieldAccess);
GetterMethod getter = lookForGetter ? findGetter(field) : null;
if (getter == null) {
return ((FieldDeclaration) field.get()).type;
}
return getter.type;
}
static Expression createFieldAccessor(EclipseNode field, FieldAccess fieldAccess, ASTNode source) {
int pS = source == null ? 0 : source.sourceStart, pE = source == null ? 0 : source.sourceEnd;
long p = (long) pS << 32 | pE;
boolean lookForGetter = lookForGetter(field, fieldAccess);
GetterMethod getter = lookForGetter ? findGetter(field) : null;
if (getter == null) {
FieldDeclaration fieldDecl = (FieldDeclaration)field.get();
FieldReference ref = new FieldReference(fieldDecl.name, p);
if ((fieldDecl.modifiers & ClassFileConstants.AccStatic) != 0) {
EclipseNode containerNode = field.up();
if (containerNode != null && containerNode.get() instanceof TypeDeclaration) {
ref.receiver = new SingleNameReference(((TypeDeclaration)containerNode.get()).name, p);
} else {
Expression smallRef = new FieldReference(field.getName().toCharArray(), p);
if (source != null) setGeneratedBy(smallRef, source);
return smallRef;
}
} else {
ref.receiver = new ThisReference(pS, pE);
}
if (source != null) {
setGeneratedBy(ref, source);
setGeneratedBy(ref.receiver, source);
}
return ref;
}
MessageSend call = new MessageSend();
setGeneratedBy(call, source);
call.sourceStart = pS; call.statementEnd = call.sourceEnd = pE;
call.receiver = new ThisReference(pS, pE);
setGeneratedBy(call.receiver, source);
call.selector = getter.name;
return call;
}
static Expression createFieldAccessor(EclipseNode field, FieldAccess fieldAccess, ASTNode source,
char[] receiver) {
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long)pS << 32 | pE;
boolean lookForGetter = lookForGetter(field, fieldAccess);
GetterMethod getter = lookForGetter ? findGetter(field) : null;
if (getter == null) {
NameReference ref;
char[][] tokens = new char[2][];
tokens[0] = receiver;
tokens[1] = field.getName().toCharArray();
long[] poss = {p, p};
```

```
ref = new QualifiedNameReference(tokens, poss, pS, pE);
setGeneratedBy(ref, source);
return ref;
}
MessageSend call = new MessageSend();
setGeneratedBy(call, source);
call.sourceStart = pS; call.statementEnd = call.sourceEnd = pE;
call.receiver = new SingleNameReference(receiver, p);
setGeneratedBy(call.receiver, source);
call.selector = getter.name;
return call;
}
static Expression createMethodAccessor(EclipseNode method, ASTNode source) {
int pS = source == null ? 0 : source.sourceStart, pE = source == null ? 0 : source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration methodDecl = (MethodDeclaration) method.get();
MessageSend call = new MessageSend();
setGeneratedBy(call, source);
call.sourceStart = pS; call.statementEnd = call.sourceEnd = pE;
if ((methodDecl.modifiers & ClassFileConstants.AccStatic) == 0) {
call.receiver = new ThisReference(pS, pE);
setGeneratedBy(call.receiver, source);
} else {
EclipseNode containerNode = method.up();
if (containerNode != null && containerNode.get() instanceof TypeDeclaration) {
call.receiver = new SingleNameReference(((TypeDeclaration) containerNode.get()).name, p);
setGeneratedBy(call.receiver, source);
}
}
call.selector = methodDecl.selector;
return call;
}
static Expression createMethodAccessor(EclipseNode method, ASTNode source, char[] receiver) {
int pS = source == null ? 0 : source.sourceStart, pE = source == null ? 0 : source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration methodDecl = (MethodDeclaration) method.get();
MessageSend call = new MessageSend();
setGeneratedBy(call, source);
call.sourceStart = pS; call.statementEnd = call.sourceEnd = pE;
call.receiver = new SingleNameReference(receiver, p);
setGeneratedBy(call.receiver, source);
call.selector = methodDecl.selector;
return call;
}
/** Serves as return value for the methods that check for the existence of fields and methods. */
public enum MemberExistsResult {
NOT_EXISTS, EXISTS_BY_LOMBOK, EXISTS_BY_USER;
}
/**
* Translates the given field into all possible getter names.
* Convenient wrapper around {@link HandlerUtil#toAllGetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
```

```
public static List<String> toAllGetterNames(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toAllGetterNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean);
}
/**
* Translates the given field into all possible getter names.
* Convenient wrapper around {@link HandlerUtil#toAllGetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static List<String> toAllGetterNames(EclipseNode field, boolean isBoolean,
AnnotationValues<Accessors> accessors) {
return HandlerUtil.toAllGetterNames(field.getAst(), accessors, field.getName(), isBoolean);
}
/**
* @return the likely getter name for the stated field. (e.g. private boolean foo; to isFoo).
*
* Convenient wrapper around {@link HandlerUtil#toGetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toGetterName(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toGetterName(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean);
}
/**
* @return the likely getter name for the stated field. (e.g. private boolean foo; to isFoo).
*
* Convenient wrapper around {@link HandlerUtil#toGetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toGetterName(EclipseNode field, boolean isBoolean, AnnotationValues<Accessors>
accessors) {
return HandlerUtil.toGetterName(field.getAst(), accessors, field.getName(), isBoolean);
}
/**
* Translates the given field into all possible setter names.
* Convenient wrapper around {@link HandlerUtil#toAllSetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllSetterNames(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toAllSetterNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean);
}
/**
* Translates the given field into all possible setter names.
* Convenient wrapper around {@link HandlerUtil#toAllSetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllSetterNames(EclipseNode field, boolean isBoolean,
AnnotationValues<Accessors> accessors) {
return HandlerUtil.toAllSetterNames(field.getAst(), accessors, field.getName(), isBoolean);
}
/**
* @return the likely setter name for the stated field. (e.g. private boolean foo; to setFoo).
```

```
*
* Convenient wrapper around {@link HandlerUtil#toSetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toSetterName(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toSetterName(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean);
}
/**
* @return the likely setter name for the stated field. (e.g. private boolean foo; to setFoo).
*
* Convenient wrapper around {@link HandlerUtil#toSetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toSetterName(EclipseNode field, boolean isBoolean, AnnotationValues<Accessors>
accessors) {
return HandlerUtil.toSetterName(field.getAst(), accessors, field.getName(), isBoolean);
}
/**
* Translates the given field into all possible with names.
* Convenient wrapper around {@link HandlerUtil#toAllWithNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithNames(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toAllWithNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean);
}
/**
* Translates the given field into all possible with names.
* Convenient wrapper around {@link HandlerUtil#toAllWithNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithNames(EclipseNode field, boolean isBoolean,
AnnotationValues<Accessors> accessors) {
return HandlerUtil.toAllWithNames(field.getAst(), accessors, field.getName(), isBoolean);
}
/**
* Translates the given field into all possible withBy names.
* Convenient wrapper around {@link HandlerUtil#toAllWithByNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithByNames(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toAllWithByNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean);
}
/**
* Translates the given field into all possible withBy names.
* Convenient wrapper around {@link HandlerUtil#toAllWithByNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithByNames(EclipseNode field, boolean isBoolean,
AnnotationValues<Accessors> accessors) {
return HandlerUtil.toAllWithByNames(field.getAst(), accessors, field.getName(), isBoolean);
```

```
}
/**
* @return the likely with name for the stated field. (e.g. private boolean foo; to withFoo).
*
* Convenient wrapper around {@link HandlerUtil#toWithName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithName(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toWithName(field.getAst(), getAccessorsForField(field), field.getName(), isBoolean);
}
/**
* @return the likely with name for the stated field. (e.g. private boolean foo; to withFoo).
*
* Convenient wrapper around {@link HandlerUtil#toWithName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithName(EclipseNode field, boolean isBoolean, AnnotationValues<Accessors>
accessors) {
return HandlerUtil.toWithName(field.getAst(), accessors, field.getName(), isBoolean);
}
/**
* @return the likely withBy name for the stated field. (e.g. private boolean foo; to withFooBy).
*
* Convenient wrapper around {@link HandlerUtil#toWithByName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithByName(EclipseNode field, boolean isBoolean) {
return HandlerUtil.toWithByName(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean);
}
/**
* @return the likely withBy name for the stated field. (e.g. private boolean foo; to withFooBy).
*
* Convenient wrapper around {@link HandlerUtil#toWithByName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithByName(EclipseNode field, boolean isBoolean,
AnnotationValues<Accessors> accessors) {
return HandlerUtil.toWithByName(field.getAst(), accessors, field.getName(), isBoolean);
}
/**
* When generating a setter/getter/wither, should it be made final?
*/
public static boolean shouldMakeFinal(EclipseNode field, AnnotationValues<Accessors> accessors) {
if ((((FieldDeclaration) field.get()).modifiers & ClassFileConstants.AccStatic) != 0) return false;
return shouldMakeFinal0(accessors, field.getAst());
}
/**
* When generating a setter, the setter either returns void (beanspec) or Self (fluent).
* This method scans for the {@code Accessors} annotation and associated config properties to figure
that out.
*/
public static boolean shouldReturnThis(EclipseNode field, AnnotationValues<Accessors> accessors) {
```

```
if ((((FieldDeclaration) field.get()).modifiers & ClassFileConstants.AccStatic) != 0) return false;
return shouldReturnThis0(accessors, field.getAst());
}
/**
* Checks if the field should be included in operations that work on 'all' fields:
* If the field is static, or starts with a '$', or is actually an enum constant, 'false' is returned, indicating
you should skip it.
*/
public static boolean filterField(FieldDeclaration declaration) {
return filterField(declaration, true);
}
public static boolean filterField(FieldDeclaration declaration, boolean skipStatic) {
// Skip the fake fields that represent enum constants.
if (declaration.initialization instanceof AllocationExpression &&
((AllocationExpression) declaration.initialization).enumConstant != null) return false;
if (declaration.type == null) return false;
// Skip fields that start with $
if (declaration.name.length > 0 && declaration.name[0] == '$') return false;
// Skip static fields.
if (skipStatic && (declaration.modifiers & ClassFileConstants.AccStatic) != 0) return false;
return true;
}
public static char[] removePrefixFromField(EclipseNode field) {
List<String> prefixes = null;
for (EclipseNode node : field.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
AnnotationValues<Accessors> ann = createAnnotation(Accessors.class, node);
if (ann.isExplicit("prefix")) prefixes = Arrays.asList(ann.getInstance().prefix());
break;
}
}
if (prefixes == null) {
EclipseNode current = field.up();
outer:
while (current != null) {
for (EclipseNode node : current.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
AnnotationValues<Accessors> ann = createAnnotation(Accessors.class, node);
if (ann.isExplicit("prefix")) prefixes = Arrays.asList(ann.getInstance().prefix());
break outer;
}
}
current = current.up();
}
}
if (prefixes == null) prefixes =
field.getAst().readConfiguration(ConfigurationKeys.ACCESSORS_PREFIX);
if (!prefixes.isEmpty()) {
CharSequence newName = removePrefix(field.getName(), prefixes);
if (newName != null) return newName.toString().toCharArray();
}
return ((FieldDeclaration) field.get()).name;
}
```

```
public static AnnotationValues<Accessors> getAccessorsForField(EclipseNode field) {
AnnotationValues<Accessors> values = null;
for (EclipseNode node : field.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
values = createAnnotation(Accessors.class, node);
break;
}
}
EclipseNode current = field.up();
while (current != null) {
for (EclipseNode node : current.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
AnnotationValues<Accessors> onType = createAnnotation(Accessors.class, node);
values = values == null ? onType : values.integrate(onType);
}
}
current = current.up();
}
return values == null ? AnnotationValues.of(Accessors.class, field) : values;
}
public static EclipseNode upToTypeNode(EclipseNode node) {
if (node == null) throw new NullPointerException("node");
while (node != null && !(node.get() instanceof TypeDeclaration)) node = node.up();
return node;
}
/**
* Checks if there is a field with the provided name.
*
* @param fieldName the field name to check for.
* @param node Any node that represents the Type (TypeDeclaration) to look in, or any child node
thereof.
*/
public static MemberExistsResult fieldExists(String fieldName, EclipseNode node) {
node = upToTypeNode(node);
char[] fieldNameChars = null;
if (node != null && node.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration) node.get();
if (typeDecl.fields != null) for (FieldDeclaration def : typeDecl.fields) {
char[] fName = def.name;
if (fName == null) continue;
if (fieldNameChars == null) fieldNameChars = fieldName.toCharArray();
if (Arrays.equals(fName, fieldNameChars)) {
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
}
return MemberExistsResult.NOT_EXISTS;
}
/**
* Wrapper for {@link #methodExists(String, EclipseNode, boolean, int)} with {@code caseSensitive} =
{@code true}.
*/
```

```
public static MemberExistsResult methodExists(String methodName, EclipseNode node, int params) {
return methodExists(methodName, node, true, params);
}
/**
* Checks if there is a method with the provided name. In case of multiple methods (overloading), only
* the first method decides if EXISTS_BY_USER or EXISTS_BY_LOMBOK is returned.
*
* @param methodName the method name to check for.
* @param node Any node that represents the Type (TypeDeclaration) to look in, or any child node
thereof.
* @param caseSensitive If the search should be case sensitive.
* @param params The number of parameters the method should have; varargs count as 0-*. Set to -1
to find any method with the appropriate name regardless of parameter count.
*/
public static MemberExistsResult methodExists(String methodName, EclipseNode node, boolean
caseSensitive, int params) {
while (node != null && !(node.get() instanceof TypeDeclaration)) {
node = node.up();
}
if (node != null && node.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration)node.get();
if (typeDecl.methods != null) top: for (AbstractMethodDeclaration def : typeDecl.methods) {
if (def instanceof MethodDeclaration) {
char[] mName = def.selector;
if (mName == null) continue;
boolean nameEquals = caseSensitive ? methodName.equals(new String(mName)) :
methodName.equalsIgnoreCase(new String(mName));
if (nameEquals) {
if (params > -1) {
int minArgs = 0;
int maxArgs = 0;
if (def.arguments != null && def.arguments.length > 0) {
minArgs = def.arguments.length;
if ((def.arguments[def.arguments.length - 1].type.bits & ASTNode.IsVarArgs) != 0) {
minArgs--;
maxArgs = Integer.MAX_VALUE;
} else {
maxArgs = minArgs;
}
}
if (params < minArgs || params > maxArgs) continue;
}
if (isTolerate(node, def)) continue top;
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
}
}
return MemberExistsResult.NOT_EXISTS;
}
public static boolean isTolerate(EclipseNode node, AbstractMethodDeclaration def) {
if (def.annotations != null) for (Annotation anno : def.annotations) {
```

```
if (typeMatches(Tolerate.class, node, anno.type)) return true;
}
return false;
}
/**
* Checks if there is a (non-default) constructor. In case of multiple constructors (overloading), only
* the first constructor decides if EXISTS_BY_USER or EXISTS_BY_LOMBOK is returned.
*
* @param node Any node that represents the Type (TypeDeclaration) to look in, or any child node
thereof.
*/
public static MemberExistsResult constructorExists(EclipseNode node) {
node = upToTypeNode(node);
if (node != null && node.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration) node.get();
if (typeDecl.methods != null) for (AbstractMethodDeclaration def : typeDecl.methods) {
if (!(def instanceof ConstructorDeclaration)) continue;
if ((def.bits & ASTNode.IsDefaultConstructor) != 0) continue;
if (isTolerate(node, def)) continue;
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
return MemberExistsResult.NOT_EXISTS;
}
/**
* Inserts a field into an existing type. The type must represent a {@code TypeDeclaration}.
* The field carries the &#64;{@link SuppressWarnings}("all") annotation.
*/
public static EclipseNode injectFieldAndMarkGenerated(EclipseNode type, FieldDeclaration field) {
field.annotations = addSuppressWarningsAll(type, field, field.annotations);
field.annotations = addGenerated(type, field, field.annotations);
return injectField(type, field);
}
/**
* Inserts a field into an existing type. The type must represent a {@code TypeDeclaration}.
*/
public static EclipseNode injectField(EclipseNode type, FieldDeclaration field) {
TypeDeclaration parent = (TypeDeclaration) type.get();
if (parent.fields == null) {
parent.fields = new FieldDeclaration[1];
parent.fields[0] = field;
} else {
int size = parent.fields.length;
FieldDeclaration[] newArray = new FieldDeclaration[size + 1];
System.arraycopy(parent.fields, 0, newArray, 0, size);
int index = 0;
for (; index < size; index++) {
FieldDeclaration f = newArray[index];
if (isEnumConstant(f) || isGenerated(f) || isRecordField(f)) continue;
break;
}
System.arraycopy(newArray, index, newArray, index + 1, size - index);
```

```
newArray[index] = field;
parent.fields = newArray;
}
if (isEnumConstant(field) || (field.modifiers & Modifier.STATIC) != 0) {
if (!hasClinit(parent)) {
parent.addClinit();
}
}
return type.add(field, Kind.FIELD);
}
public static boolean isEnumConstant(final FieldDeclaration field) {
return ((field.initialization instanceof AllocationExpression) && (((AllocationExpression)
field.initialization).enumConstant == field));
}
/**
* Inserts a method into an existing type. The type must represent a {@code TypeDeclaration}.
*/
public static EclipseNode injectMethod(EclipseNode type, AbstractMethodDeclaration method) {
method.annotations = addSuppressWarningsAll(type, method, method.annotations);
method.annotations = addGenerated(type, method, method.annotations);
TypeDeclaration parent = (TypeDeclaration) type.get();
if (parent.methods == null) {
parent.methods = new AbstractMethodDeclaration[1];
parent.methods[0] = method;
} else {
if (method instanceof ConstructorDeclaration) {
for (int i = 0 ; i < parent.methods.length ; i++) {
if (parent.methods[i] instanceof ConstructorDeclaration &&
(parent.methods[i].bits & ASTNode.IsDefaultConstructor) != 0) {
EclipseNode tossMe = type.getNodeFor(parent.methods[i]);
AbstractMethodDeclaration[] withoutGeneratedConstructor = new
AbstractMethodDeclaration[parent.methods.length - 1];
System.arraycopy(parent.methods, 0, withoutGeneratedConstructor, 0, i);
System.arraycopy(parent.methods, i + 1, withoutGeneratedConstructor, i, parent.methods.length - i -
1);
parent.methods = withoutGeneratedConstructor;
if (tossMe != null) tossMe.up().removeChild(tossMe);
break;
}
}
}
//We insert the method in the last position of the methods registered to the type
//When changing this behavior, this may trigger issue #155 and #377
AbstractMethodDeclaration[] newArray = new AbstractMethodDeclaration[parent.methods.length + 1];
System.arraycopy(parent.methods, 0, newArray, 0, parent.methods.length);
newArray[parent.methods.length] = method;
parent.methods = newArray;
}
return type.add(method, Kind.METHOD);
}
/**
* Adds an inner type (class, interface, enum) to the given type. Cannot inject top-level types.
*
```

```
* @param typeNode parent type to inject new type into
* @param type New type (class, interface, etc) to inject.
*/
public static EclipseNode injectType(final EclipseNode typeNode, final TypeDeclaration type) {
type.annotations = addSuppressWarningsAll(typeNode, type, type.annotations);
type.annotations = addGenerated(typeNode, type, type.annotations);
TypeDeclaration parent = (TypeDeclaration) typeNode.get();
if (parent.memberTypes == null) {
parent.memberTypes = new TypeDeclaration[] { type };
} else {
TypeDeclaration[] newArray = new TypeDeclaration[parent.memberTypes.length + 1];
System.arraycopy(parent.memberTypes, 0, newArray, 0, parent.memberTypes.length);
newArray[parent.memberTypes.length] = type;
parent.memberTypes = newArray;
}
return typeNode.add(type, Kind.TYPE);
}
static final char[] ALL = "all".toCharArray();
static final char[] UNCHECKED = "unchecked".toCharArray();
private static final char[] JUSTIFICATION = "justification".toCharArray();
private static final char[] GENERATED_CODE = "generated code".toCharArray();
private static final char[] LOMBOK = "lombok".toCharArray();
private static final char[][] JAVAX_ANNOTATION_GENERATED =
Eclipse.fromQualifiedName("javax.annotation.Generated");
private static final char[][] JAKARTA_ANNOTATION_GENERATED =
Eclipse.fromQualifiedName("jakarta.annotation.Generated");
private static final char[][] LOMBOK_GENERATED = Eclipse.fromQualifiedName("lombok.Generated");
private static final char[][] EDU_UMD_CS_FINDBUGS_ANNOTATIONS_SUPPRESSFBWARNINGS =
Eclipse.fromQualifiedName("edu.umd.cs.findbugs.annotations.SuppressFBWarnings");
public static Annotation[] addSuppressWarningsAll(EclipseNode node, ASTNode source, Annotation[]
originalAnnotationArray) {
Annotation[] anns = originalAnnotationArray;
if (!Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_SUPPRESSWAR
NINGS_ANNOTATIONS))) {
anns = addAnnotation(source, anns, TypeConstants.JAVA_LANG_SUPPRESSWARNINGS, new
StringLiteral(ALL, 0, 0, 0));
}
if (Boolean.TRUE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_FINDBUGS_SUPP
RESSWARNINGS_ANNOTATIONS))) {
MemberValuePair mvp = new MemberValuePair(JUSTIFICATION, 0, 0, new
StringLiteral(GENERATED_CODE, 0, 0, 0));
anns = addAnnotation(source, anns,
EDU_UMD_CS_FINDBUGS_ANNOTATIONS_SUPPRESSFBWARNINGS, mvp);
}
return anns;
}
public static Annotation[] addGenerated(EclipseNode node, ASTNode source, Annotation[]
originalAnnotationArray) {
Annotation[] result = originalAnnotationArray;
if (HandlerUtil.shouldAddGenerated(node)) {
result = addAnnotation(source, result, JAVAX_ANNOTATION_GENERATED, new
StringLiteral(LOMBOK, 0, 0, 0));
}
```

```
if (Boolean.TRUE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_JAKARTA_GENE
RATED_ANNOTATIONS))) {
result = addAnnotation(source, result, JAKARTA_ANNOTATION_GENERATED, new
StringLiteral(LOMBOK, 0, 0, 0));
}
if (!Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_LOMBOK_GENE
RATED_ANNOTATIONS))) {
result = addAnnotation(source, result, LOMBOK_GENERATED);
}
return result;
}
static Annotation[] addAnnotation(ASTNode source, Annotation[] originalAnnotationArray, char[][]
annotationTypeFqn) {
return addAnnotation(source, originalAnnotationArray, annotationTypeFqn, (ASTNode[]) null);
}
static Annotation[] addAnnotation(ASTNode source, Annotation[] originalAnnotationArray, char[][]
annotationTypeFqn, ASTNode... args) {
char[] simpleName = annotationTypeFqn[annotationTypeFqn.length - 1];
if (originalAnnotationArray != null) for (Annotation ann : originalAnnotationArray) {
if (ann.type instanceof QualifiedTypeReference) {
char[][] t = ((QualifiedTypeReference) ann.type).tokens;
if (Arrays.deepEquals(t, annotationTypeFqn)) return originalAnnotationArray;
}
if (ann.type instanceof SingleTypeReference) {
char[] lastToken = ((SingleTypeReference) ann.type).token;
if (Arrays.equals(lastToken, simpleName)) return originalAnnotationArray;
}
}
int pS = source.sourceStart, pE = source.sourceEnd;
TypeReference qualifiedType = generateQualifiedTypeRef(source, annotationTypeFqn);
Annotation ann;
if (args != null && args.length == 1 && args[0] instanceof Expression) {
SingleMemberAnnotation sma = new SingleMemberAnnotation(qualifiedType, pS);
sma.declarationSourceEnd = pE;
args[0].sourceStart = pS;
args[0].sourceEnd = pE;
sma.memberValue = (Expression) args[0];
setGeneratedBy(sma.memberValue, source);
ann = sma;
} else if (args != null && args.length >= 1 && arrayHasOnlyElementsOfType(args,
MemberValuePair.class)) {
NormalAnnotation na = new NormalAnnotation(qualifiedType, pS);
na.declarationSourceEnd = pE;
na.memberValuePairs = new MemberValuePair[args.length];
for (int i = 0; i < args.length; i++) {
args[i].sourceStart = pS;
args[i].sourceEnd = pE;
na.memberValuePairs[i] = (MemberValuePair) args[i];
}
setGeneratedBy(na.memberValuePairs[0], source);
setGeneratedBy(na.memberValuePairs[0].value, source);
na.memberValuePairs[0].value.sourceStart = pS;
na.memberValuePairs[0].value.sourceEnd = pE;
```

```
ann = na;
} else {
MarkerAnnotation ma = new MarkerAnnotation(qualifiedType, pS);
ma.declarationSourceEnd = pE;
ann = ma;
}
setGeneratedBy(ann, source);
if (originalAnnotationArray == null) return new Annotation[] { ann };
Annotation[] newAnnotationArray = new Annotation[originalAnnotationArray.length + 1];
System.arraycopy(originalAnnotationArray, 0, newAnnotationArray, 0, originalAnnotationArray.length);
newAnnotationArray[originalAnnotationArray.length] = ann;
return newAnnotationArray;
}
public static void addCheckerFrameworkReturnsReceiver(TypeReference returnType, ASTNode
source, CheckerFrameworkVersion cfv) {
if (cfv.generateReturnsReceiver()) {
Annotation rrAnn = generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__RETURNS_RECEIVER);
int levels = returnType.getAnnotatableLevels();
returnType.annotations = new Annotation[levels][];
returnType.annotations[levels-1] = new Annotation[] {rrAnn};
}
}
private static boolean arrayHasOnlyElementsOfType(Object[] array, Class<?> clazz) {
for (Object element : array) {
if (!clazz.isInstance(element))
return false;
}
return true;
}
/**
* Generates a new statement that checks if the given local variable is null, and if so, throws a specified
exception with the
* variable name as message.
*/
public static Statement generateNullCheck(TypeReference type, char[] variable, EclipseNode
sourceNode, String customMessage) {
NullCheckExceptionType exceptionType =
sourceNode.getAst().readConfiguration(ConfigurationKeys.NON_NULL_EXCEPTION_TYPE);
if (exceptionType == null) exceptionType = NullCheckExceptionType.NULL_POINTER_EXCEPTION;
ASTNode source = sourceNode.get();
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
if (type != null && isPrimitive(type)) return null;
SingleNameReference varName = new SingleNameReference(variable, p);
setGeneratedBy(varName, source);
StringLiteral message = new StringLiteral(exceptionType.toExceptionMessage(new String(variable),
customMessage).toCharArray(), pS, pE, 0);
setGeneratedBy(message, source);
LombokImmutableList<String> method = exceptionType.getMethod();
if (method != null) {
MessageSend invocation = new MessageSend();
invocation.sourceStart = pS; invocation.sourceEnd = pE;
```

```
setGeneratedBy(invocation, source);
char[][] utilityTypeName = new char[method.size() - 1][];
for (int i = 0; i < method.size() - 1; i++) {
utilityTypeName[i] = method.get(i).toCharArray();
}
invocation.receiver = new QualifiedNameReference(utilityTypeName, new long[method.size() - 1], pS,
pE);
setGeneratedBy(invocation.receiver, source);
invocation.selector = method.get(method.size() - 1).toCharArray();
invocation.arguments = new Expression[] {varName, message};
return invocation;
}
AllocationExpression exception = new AllocationExpression();
setGeneratedBy(exception, source);
NullLiteral nullLiteral = new NullLiteral(pS, pE);
setGeneratedBy(nullLiteral, source);
int equalOperator = exceptionType == NullCheckExceptionType.ASSERTION ?
OperatorIds.NOT_EQUAL : OperatorIds.EQUAL_EQUAL;
EqualExpression equalExpression = new EqualExpression(varName, nullLiteral, equalOperator);
equalExpression.sourceStart = pS; equalExpression.statementEnd = equalExpression.sourceEnd =
pE;
setGeneratedBy(equalExpression, source);
if (exceptionType == NullCheckExceptionType.ASSERTION) {
Statement assertStatement = new AssertStatement(message, equalExpression, pS);
setGeneratedBy(assertStatement, source);
return assertStatement;
}
String exceptionTypeStr = exceptionType.getExceptionType();
int partCount = 1;
for (int i = 0; i < exceptionTypeStr.length(); i++) if (exceptionTypeStr.charAt(i) == '.') partCount++;
long[] ps = new long[partCount];
Arrays.fill(ps, 0L);
exception.type = new QualifiedTypeReference(fromQualifiedName(exceptionTypeStr), ps);
setGeneratedBy(exception.type, source);
exception.arguments = new Expression[] {message};
ThrowStatement throwStatement = new ThrowStatement(exception, pS, pE);
setGeneratedBy(throwStatement, source);
Block throwBlock = new Block(0);
throwBlock.statements = new Statement[] {throwStatement};
throwBlock.sourceStart = pS; throwBlock.sourceEnd = pE;
setGeneratedBy(throwBlock, source);
IfStatement ifStatement = new IfStatement(equalExpression, throwBlock, 0, 0);
setGeneratedBy(ifStatement, source);
return ifStatement;
}
/**
* Generates a new statement that checks if the given variable is null, and if so, throws a specified
exception with the
* variable name as message.
*
* @param exName The name of the exception to throw; normally {@code
java.lang.NullPointerException}.
*/
```

```
public static Statement generateNullCheck(AbstractVariableDeclaration variable, EclipseNode
sourceNode, String customMessage) {
return generateNullCheck(variable.type, variable.name, sourceNode, customMessage);
}
/**
* Create an annotation of the given name, and is marked as being generated by the given source.
*/
public static MarkerAnnotation makeMarkerAnnotation(char[][] name, ASTNode source) {
long pos = (long) source.sourceStart << 32 | source.sourceEnd;
long[] poss = new long[name.length];
Arrays.fill(poss, pos);
TypeReference typeRef = new QualifiedTypeReference(name, poss);
setGeneratedBy(typeRef, source);
MarkerAnnotation ann = new MarkerAnnotation(typeRef, (int) (pos >> 32));
ann.declarationSourceEnd = ann.sourceEnd = ann.statementEnd = (int) pos;
setGeneratedBy(ann, source);
return ann;
}
/**
* Given a list of field names and a node referring to a type, finds each name in the list that does not
match a field within the type.
*/
public static List<Integer> createListOfNonExistentFields(List<String> list, EclipseNode type, boolean
excludeStandard, boolean excludeTransient) {
boolean[] matched = new boolean[list.size()];
for (EclipseNode child : type.down()) {
if (list.isEmpty()) break;
if (child.getKind() != Kind.FIELD) continue;
if (excludeStandard) {
if ((((FieldDeclaration) child.get()).modifiers & ClassFileConstants.AccStatic) != 0) continue;
if (child.getName().startsWith("$")) continue;
}
if (excludeTransient && (((FieldDeclaration)child.get()).modifiers & ClassFileConstants.AccTransient) !=
0) continue;
int idx = list.indexOf(child.getName());
if (idx > -1) matched[idx] = true;
}
List<Integer> problematic = new ArrayList<Integer>();
for (int i = 0 ; i < list.size() ; i++) {
if (!matched[i]) problematic.add(i);
}
return problematic;
}
/**
* In eclipse 3.7+, the CastExpression constructor was changed from a really weird version to
* a less weird one. Unfortunately that means we need to use reflection as we want to be compatible
* with eclipse versions before 3.7 and 3.7+.
*
* @param ref The {@code foo} in {@code (String)foo}.
* @param castTo The {@code String} in {@code (String)foo}.
*/
public static CastExpression makeCastExpression(Expression ref, TypeReference castTo, ASTNode
source) {
```

```
CastExpression result;
try {
if (castExpressionConstructorIsTypeRefBased) {
result = castExpressionConstructor.newInstance(ref, castTo);
} else {
Expression castToConverted = castTo;
if (castTo.getClass() == SingleTypeReference.class && !isPrimitive(castTo)) {
SingleTypeReference str = (SingleTypeReference) castTo;
//Why a SingleNameReference instead of a SingleTypeReference you ask? I don't know. It seems
dumb. Ask the ecj guys.
castToConverted = new SingleNameReference(str.token, 0);
castToConverted.bits = (castToConverted.bits & ~Binding.VARIABLE) | Binding.TYPE;
castToConverted.sourceStart = str.sourceStart;
castToConverted.sourceEnd = str.sourceEnd;
setGeneratedBy(castToConverted, source);
} else if (castTo.getClass() == QualifiedTypeReference.class) {
QualifiedTypeReference qtr = (QualifiedTypeReference) castTo;
//Same here, but for the more complex types, they stay types.
castToConverted = new QualifiedNameReference(qtr.tokens, copy(qtr.sourcePositions),
qtr.sourceStart, qtr.sourceEnd);
castToConverted.bits = (castToConverted.bits & ~Binding.VARIABLE) | Binding.TYPE;
setGeneratedBy(castToConverted, source);
}
result = castExpressionConstructor.newInstance(ref, castToConverted);
}
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InstantiationException e) {
throw Lombok.sneakyThrow(e);
}
result.sourceStart = source.sourceStart;
result.sourceEnd = source.sourceEnd;
result.statementEnd = source.sourceEnd;
setGeneratedBy(result, source);
return result;
}
private static final Constructor<CastExpression> castExpressionConstructor;
private static final boolean castExpressionConstructorIsTypeRefBased;
static {
Constructor<?> constructor = null;
for (Constructor<?> ctor : CastExpression.class.getConstructors()) {
if (ctor.getParameterTypes().length != 2) continue;
constructor = ctor;
}
@SuppressWarnings("unchecked")
Constructor<CastExpression> castExpressionConstructor_ = (Constructor<CastExpression>)
constructor;
castExpressionConstructor = castExpressionConstructor_;
castExpressionConstructorIsTypeRefBased =
(castExpressionConstructor.getParameterTypes()[1] == TypeReference.class);
}
```

```
/**
* In eclipse 3.7+, IntLiterals are created using a factory-method
* Unfortunately that means we need to use reflection as we want to be compatible
* with eclipse versions before 3.7.
*/
public static IntLiteral makeIntLiteral(char[] token, ASTNode source) {
int pS = source == null ? 0 : source.sourceStart, pE = source == null ? 0 : source.sourceEnd;
IntLiteral result;
if (intLiteralConstructor != null) {
result = Permit.newInstanceSneaky(intLiteralConstructor, token, pS, pE);
} else {
result = (IntLiteral) Permit.invokeSneaky(intLiteralFactoryMethod, null, token, pS, pE);
}
if (source != null) setGeneratedBy(result, source);
return result;
}
private static final Constructor<IntLiteral> intLiteralConstructor;
private static final Method intLiteralFactoryMethod;
static {
Class<?>[] parameterTypes = {char[].class, int.class, int.class};
Constructor<IntLiteral> intLiteralConstructor_ = null;
Method intLiteralFactoryMethod_ = null;
try {
intLiteralConstructor_ = Permit.getConstructor(IntLiteral.class, parameterTypes);
} catch (Throwable ignore) {
// probably eclipse 3.7++
}
try {
intLiteralFactoryMethod_ = Permit.getMethod(IntLiteral.class, "buildIntLiteral", parameterTypes);
} catch (Throwable ignore) {
// probably eclipse versions before 3.7
}
intLiteralConstructor = intLiteralConstructor_;
intLiteralFactoryMethod = intLiteralFactoryMethod_;
}
private static boolean isAllValidOnXCharacters(char[] in) {
if (in == null || in.length == 0) return false;
for (char c : in) if (c != '_' && c != 'X' && c != 'x' && c != '$') return false;
return true;
}
public static void addError(String errorName, EclipseNode node) {
if (node.getLatestJavaSpecSupported() < 8) {
node.addError("The correct format is " + errorName + "_={@SomeAnnotation,
@SomeOtherAnnotation})");
} else {
node.addError("The correct format is " + errorName + "=@__({@SomeAnnotation,
@SomeOtherAnnotation}))");
}
}
public static List<Annotation> unboxAndRemoveAnnotationParameter(Annotation annotation, String
annotationName, String errorName, EclipseNode errorNode) {
if ("value".equals(annotationName)) {
// We can't unbox this, because SingleMemberAnnotation REQUIRES a value, and this method
```

```
// is supposed to remove the value. That means we need to replace the SMA with either
// MarkerAnnotation or NormalAnnotation and that is beyond the scope of this method as we
// don't need that at the time of writing this method; we only unbox onMethod, onParameter
// and onConstructor. Let's exit early and very obviously:
throw new UnsupportedOperationException("Lombok cannot unbox 'value' from
SingleMemberAnnotation at this time.");
}
if (!NormalAnnotation.class.equals(annotation.getClass())) {
// Prevent MarkerAnnotation, SingleMemberAnnotation, and
// CompletionOnAnnotationMemberValuePair from triggering this handler.
return Collections.emptyList();
}
NormalAnnotation normalAnnotation = (NormalAnnotation) annotation;
MemberValuePair[] pairs = normalAnnotation.memberValuePairs;
if (pairs == null) return Collections.emptyList();
char[] nameAsCharArray = annotationName.toCharArray();
top:
for (int i = 0; i < pairs.length; i++) {
boolean allowRaw;
char[] name = pairs[i].name;
if (name == null) continue;
if (name.length < nameAsCharArray.length) continue;
for (int j = 0; j < nameAsCharArray.length; j++) {
if (name[j] != nameAsCharArray[j]) continue top;
}
allowRaw = name.length > nameAsCharArray.length;
for (int j = nameAsCharArray.length; j < name.length; j++) {
if (name[j] != '_') continue top;
}
// If we're still here it's the targeted annotation param.
Expression value = pairs[i].value;
MemberValuePair[] newPairs = new MemberValuePair[pairs.length - 1];
if (i > 0) System.arraycopy(pairs, 0, newPairs, 0, i);
if (i < pairs.length - 1) System.arraycopy(pairs, i + 1, newPairs, i, pairs.length - i - 1);
normalAnnotation.memberValuePairs = newPairs;
// We have now removed the annotation parameter and stored the value,
// which we must now unbox. It's either annotations, or @__(annotations).
Expression content = null;
if (value instanceof ArrayInitializer) {
if (!allowRaw) {
addError(errorName, errorNode);
return Collections.emptyList();
}
content = value;
} else if (!(value instanceof Annotation)) {
addError(errorName, errorNode);
return Collections.emptyList();
} else {
Annotation atDummyIdentifier = (Annotation) value;
if (atDummyIdentifier.type instanceof SingleTypeReference &&
isAllValidOnXCharacters(((SingleTypeReference) atDummyIdentifier.type).token)) {
if (atDummyIdentifier instanceof MarkerAnnotation) {
return Collections.emptyList();
```

```
} else if (atDummyIdentifier instanceof NormalAnnotation) {
MemberValuePair[] mvps = ((NormalAnnotation) atDummyIdentifier).memberValuePairs;
if (mvps == null || mvps.length == 0) {
return Collections.emptyList();
}
if (mvps.length == 1 && Arrays.equals("value".toCharArray(), mvps[0].name)) {
content = mvps[0].value;
}
} else if (atDummyIdentifier instanceof SingleMemberAnnotation) {
content = ((SingleMemberAnnotation) atDummyIdentifier).memberValue;
} else {
addError(errorName, errorNode);
return Collections.emptyList();
}
} else {
if (allowRaw) {
content = atDummyIdentifier;
} else {
addError(errorName, errorNode);
return Collections.emptyList();
}
}
}
if (content == null) {
addError(errorName, errorNode);
return Collections.emptyList();
}
if (content instanceof Annotation) {
return Collections.singletonList((Annotation) content);
} else if (content instanceof ArrayInitializer) {
Expression[] expressions = ((ArrayInitializer) content).expressions;
List<Annotation> result = new ArrayList<Annotation>();
if (expressions != null) for (Expression ex : expressions) {
if (ex instanceof Annotation) result.add((Annotation) ex);
else {
addError(errorName, errorNode);
return Collections.emptyList();
}
}
return result;
} else {
addError(errorName, errorNode);
return Collections.emptyList();
}
}
return Collections.emptyList();
}
public static NameReference createNameReference(String name, Annotation source) {
return generateQualifiedNameRef(source, fromQualifiedName(name));
}
private static long[] copy(long[] array) {
return array == null ? null : array.clone();
}
```

```
public static <T> T[] concat(T[] first, T[] second, Class<T> type) {
if (first == null)
return second;
if (second == null)
return first;
if (first.length == 0)
return second;
if (second.length == 0)
return first;
T[] result = newArray(type, first.length + second.length);
System.arraycopy(first, 0, result, 0, first.length);
System.arraycopy(second, 0, result, first.length, second.length);
return result;
}
@SuppressWarnings("unchecked")
private static <T> T[] newArray(Class<T> type, int length) {
return (T[]) Array.newInstance(type, length);
}
public static boolean isDirectDescendantOfObject(EclipseNode typeNode) {
if (!(typeNode.get() instanceof TypeDeclaration)) throw new IllegalArgumentException("not a type
node");
TypeDeclaration typeDecl = (TypeDeclaration) typeNode.get();
if (typeDecl.superclass == null) return true;
String p = typeDecl.superclass.toString();
return p.equals("Object") || p.equals("java.lang.Object");
}
public static void createRelevantNullableAnnotation(EclipseNode typeNode, MethodDeclaration mth) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToMethodDecl(typeNode, mth, lib.getNullableAnnotation(), lib.isTypeUse());
}
public static void createRelevantNonNullAnnotation(EclipseNode typeNode, MethodDeclaration mth) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToMethodDecl(typeNode, mth, lib.getNonNullAnnotation(), lib.isTypeUse());
}
public static void createRelevantNullableAnnotation(EclipseNode typeNode, Argument arg,
MethodDeclaration mth) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToVarDecl(typeNode, arg, mth, lib.getNullableAnnotation(), lib.isTypeUse());
}
public static void createRelevantNullableAnnotation(EclipseNode typeNode, Argument arg,
MethodDeclaration mth, List<NullAnnotationLibrary> applied) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null || applied.contains(lib)) return;
applyAnnotationToVarDecl(typeNode, arg, mth, lib.getNullableAnnotation(), lib.isTypeUse());
}
```

```
public static void createRelevantNonNullAnnotation(EclipseNode typeNode, Argument arg,
MethodDeclaration mth) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToVarDecl(typeNode, arg, mth, lib.getNonNullAnnotation(), lib.isTypeUse());
}
private static void applyAnnotationToMethodDecl(EclipseNode typeNode, MethodDeclaration mth,
String annType, boolean typeUse) {
if (annType == null) return;
int partCount = 1;
for (int i = 0; i < annType.length(); i++) if (annType.charAt(i) == '.') partCount++;
long[] ps = new long[partCount];
Arrays.fill(ps, 0L);
Annotation ann = new MarkerAnnotation(new
QualifiedTypeReference(Eclipse.fromQualifiedName(annType), ps), 0);
if (!typeUse || mth.returnType == null || mth.returnType.getTypeName().length < 2) {
Annotation[] a = mth.annotations;
if (a == null) a = new Annotation[1];
else {
Annotation[] b = new Annotation[a.length + 1];
System.arraycopy(a, 0, b, 0, a.length);
a = b;
}
a[a.length - 1] = ann;
mth.annotations = a;
} else {
int len = mth.returnType.getTypeName().length;
if (mth.returnType.annotations == null) mth.returnType.annotations = new Annotation[len][];
Annotation[] a = mth.returnType.annotations[len - 1];
if (a == null) a = new Annotation[1];
else {
Annotation[] b = new Annotation[a.length + 1];
System.arraycopy(a, 0, b, 1, a.length);
a = b;
}
a[0] = ann;
mth.returnType.annotations[len - 1] = a;
mth.bits |= Eclipse.HasTypeAnnotations;
}
}
private static void applyAnnotationToVarDecl(EclipseNode typeNode, Argument arg,
MethodDeclaration mth, String annType, boolean typeUse) {
if (annType == null) return;
int partCount = 1;
for (int i = 0; i < annType.length(); i++) if (annType.charAt(i) == '.') partCount++;
long[] ps = new long[partCount];
Arrays.fill(ps, 0L);
Annotation ann = new MarkerAnnotation(new
QualifiedTypeReference(Eclipse.fromQualifiedName(annType), ps), 0);
if (!typeUse || arg.type.getTypeName().length < 2) {
Annotation[] a = arg.annotations;
if (a == null) a = new Annotation[1];
```

```
else {
Annotation[] b = new Annotation[a.length + 1];
System.arraycopy(a, 0, b, 0, a.length);
a = b;
}
a[a.length - 1] = ann;
arg.annotations = a;
} else {
int len = arg.type.getTypeName().length;
if (arg.type.annotations == null) arg.type.annotations = new Annotation[len][];
Annotation[] a = arg.type.annotations[len - 1];
if (a == null) a = new Annotation[1];
else {
Annotation[] b = new Annotation[a.length + 1];
System.arraycopy(a, 0, b, 1, a.length);
a = b;
}
a[0] = ann;
arg.type.annotations[len - 1] = a;
arg.type.bits |= Eclipse.HasTypeAnnotations;
arg.bits |= Eclipse.HasTypeAnnotations;
mth.bits |= Eclipse.HasTypeAnnotations;
}
}
public static NameReference generateQualifiedNameRef(ASTNode source, char[]... varNames) {
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long)pS << 32 | pE;
NameReference ref;
if (varNames.length > 1) ref = new QualifiedNameReference(varNames, new long[varNames.length],
pS, pE);
else ref = new SingleNameReference(varNames[0], p);
setGeneratedBy(ref, source);
return ref;
}
public static TypeReference generateQualifiedTypeRef(ASTNode source, char[]... varNames) {
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long)pS << 32 | pE;
TypeReference ref;
long[] poss = Eclipse.poss(source, varNames.length);
if (varNames.length > 1) ref = new QualifiedTypeReference(varNames, poss);
else ref = new SingleTypeReference(varNames[0], p);
setGeneratedBy(ref, source);
return ref;
}
public static TypeReference createTypeReference(String typeName, ASTNode source) {
return generateQualifiedTypeRef(source, fromQualifiedName(typeName));
}
/**
* Returns {@code true} if the provided node is an actual class and not some other type declaration (so,
not an annotation definition, interface, enum, or record).
*/
public static boolean isClass(EclipseNode typeNode) {
```

```
return isTypeAndDoesNotHaveFlags(typeNode, ClassFileConstants.AccInterface |
ClassFileConstants.AccEnum | ClassFileConstants.AccAnnotation | AccRecord);
}
/**
* Returns {@code true} if the provided node is an actual class or enum and not some other type
declaration (so, not an annotation definition, interface, or record).
*/
public static boolean isClassOrEnum(EclipseNode typeNode) {
return isTypeAndDoesNotHaveFlags(typeNode, ClassFileConstants.AccInterface |
ClassFileConstants.AccAnnotation | AccRecord);
}
/**
* Returns {@code true} if the provided node is an actual class, an enum or a record and not some other
type declaration (so, not an annotation definition or interface).
*/
public static boolean isClassEnumOrRecord(EclipseNode typeNode) {
return isTypeAndDoesNotHaveFlags(typeNode, ClassFileConstants.AccInterface |
ClassFileConstants.AccAnnotation);
}
/**
* Returns {@code true} if the provided node is a record declaration (so, not an annotation definition,
interface, enum, or plain class).
*/
public static boolean isRecord(EclipseNode typeNode) {
TypeDeclaration typeDecl = null;
if (typeNode.get() instanceof TypeDeclaration) typeDecl = (TypeDeclaration) typeNode.get();
int modifiers = typeDecl == null ? 0 : typeDecl.modifiers;
return (modifiers & AccRecord) != 0;
}
/**
* Returns {@code true} If the provided node is a field declaration, and represents a field in a {@code
record} declaration.
*/
public static boolean isRecordField(EclipseNode fieldNode) {
return fieldNode.getKind() == Kind.FIELD && (((FieldDeclaration) fieldNode.get()).modifiers &
AccRecord) != 0;
}
/**
* Returns {@code true} If the provided node is a field declaration, and represents a field in a {@code
record} declaration.
*/
public static boolean isRecordField(FieldDeclaration fieldDeclaration) {
return (fieldDeclaration.modifiers & AccRecord) != 0;
}
/**
* Returns {@code true) if the provided node is a type declaration <em>and</em> is
<strong>not</strong> of any kind indicated by the flags (the intent is to pass flags usch as
`ClassFileConstants.AccEnum`).
*/
static boolean isTypeAndDoesNotHaveFlags(EclipseNode typeNode, long flags) {
TypeDeclaration typeDecl = null;
if (typeNode.get() instanceof TypeDeclaration) typeDecl = (TypeDeclaration) typeNode.get();
int modifiers = typeDecl == null ? 0 : typeDecl.modifiers;
```

```
return (modifiers & flags) == 0;
}
/**
* Returns {@code true} if the provided node supports static methods and types (top level or static class)
*/
public static boolean isStaticAllowed(EclipseNode typeNode) {
return typeNode.isStatic() || typeNode.up() == null || typeNode.up().getKind() ==
Kind.COMPILATION_UNIT || isRecord(typeNode);
}
public static AbstractVariableDeclaration[] getRecordComponents(TypeDeclaration typeDeclaration) {
if (typeDeclaration == null || (typeDeclaration.modifiers & AccRecord) == 0) return null;
try {
return (AbstractVariableDeclaration[])
TYPE_DECLARATION_RECORD_COMPONENTS.get(typeDeclaration);
} catch (Exception e) {
// This presumably means this isn't a JDK16 - fall through.
}
return null;
}
public static Annotation[][] getRecordFieldAnnotations(TypeDeclaration typeDeclaration) {
if (typeDeclaration.fields == null) return null;
Annotation[][] annotations = new Annotation[typeDeclaration.fields.length][];
AbstractVariableDeclaration[] recordComponents = getRecordComponents(typeDeclaration);
if (recordComponents != null) {
int j = 0;
for (int i = 0; i < typeDeclaration.fields.length; i++) {
if ((typeDeclaration.fields[i].modifiers & AccRecord) != 0) {
annotations[i] = recordComponents[j++].annotations;
}
}
}
return annotations;
}
public static String getDocComment(EclipseNode eclipseNode) {
if (eclipseNode.getAst().getSource() == null) return null;
int start = -1;
int end = -1;
final ASTNode node = eclipseNode.get();
if (node instanceof FieldDeclaration) {
FieldDeclaration fieldDeclaration = (FieldDeclaration) node;
start = fieldDeclaration.declarationSourceStart;
end = fieldDeclaration.declarationSourceEnd;
} else if (node instanceof AbstractMethodDeclaration) {
AbstractMethodDeclaration abstractMethodDeclaration = (AbstractMethodDeclaration) node;
start = abstractMethodDeclaration.declarationSourceStart;
end = abstractMethodDeclaration.declarationSourceEnd;
}
if (start != -1 && end != -1) {
char[] rawContent = CharOperation.subarray(eclipseNode.getAst().getSource(), start, end);
String rawContentString = new String(rawContent);
int startIndex = rawContentString.indexOf("/**");
int endIndex = rawContentString.indexOf("*/");
if (startIndex != -1 && endIndex != -1) {
```

```
/* Remove all leading asterisks */
return rawContentString.substring(startIndex + 3, endIndex).replaceAll("(?m)^\\s*\\* ?", "").trim();
}
}
return null;
}
public static void setDocComment(EclipseNode typeNode, EclipseNode eclipseNode, String doc) {
setDocComment((CompilationUnitDeclaration) eclipseNode.top().get(), (TypeDeclaration)
typeNode.get(), eclipseNode.get(), doc);
}
public static void setDocComment(CompilationUnitDeclaration cud, EclipseNode eclipseNode, String
doc) {
setDocComment(cud, (TypeDeclaration) upToTypeNode(eclipseNode).get(), eclipseNode.get(), doc);
}
public static void setDocComment(CompilationUnitDeclaration cud, TypeDeclaration type, ASTNode
node, String doc) {
if (doc == null) return;
ICompilationUnit compilationUnit = cud.compilationResult.compilationUnit;
if (compilationUnit.getClass().equals(COMPILATION_UNIT)) {
try {
compilationUnit = (ICompilationUnit) Permit.invoke(COMPILATION_UNIT_ORIGINAL_FROM_CLONE,
compilationUnit);
} catch (Throwable t) { }
}
Map<String, String> docs = EcjAugments.CompilationUnit_javadoc.setIfAbsent(compilationUnit, new
HashMap<String, String>());
if (node instanceof AbstractMethodDeclaration) {
AbstractMethodDeclaration methodDeclaration = (AbstractMethodDeclaration) node;
String signature = getSignature(type, methodDeclaration);
/* Add javadoc start marker, remove trailing line break, add leading asterisks to each line, add javadoc
end marker */
docs.put(signature, String.format("/**%n%s%n */", doc.replaceAll("$\\r?\\n", "").replaceAll("(?m)^", " *
")));
}
}
public static String getSignature(TypeDeclaration type, AbstractMethodDeclaration methodDeclaration)
{
StringBuilder sb = new StringBuilder();
sb.append(type.name);
sb.append(".");
sb.append(methodDeclaration.selector);
sb.append("(");
Argument[] arguments = methodDeclaration.arguments;
if (arguments != null) {
for (Argument argument : arguments) {
sb.append(String.valueOf(argument.type));
}
}
sb.append(")");
return sb.toString();
}
public static enum CopyJavadoc {
VERBATIM {
```

```
@Override public String apply(final EclipseNode node) {
return getDocComment(node);
}
},
GETTER {
@Override public String apply(final EclipseNode node) {
String javadoc = getDocComment(node);
// step 1: Check if there is a 'GETTER' section. If yes, that becomes the new method's javadoc.
String out = getJavadocSection(javadoc, "GETTER");
final boolean sectionBased = out != null;
if (!sectionBased) {
out = stripLinesWithTagFromJavadoc(stripSectionsFromJavadoc(javadoc), JavadocTag.PARAM);
}
return out;
}
},
SETTER {
@Override public String apply(final EclipseNode node) {
return applySetter(node, "SETTER");
}
},
WITH {
@Override public String apply(final EclipseNode node) {
return addReturnsUpdatedSelfIfNeeded(applySetter(node, "WITH|WITHER"));
}
},
WITH_BY {
@Override public String apply( final EclipseNode node) {
return applySetter(node, "WITHBY|WITH_BY");
}
};
public abstract String apply(final EclipseNode node);
private static String applySetter(EclipseNode node, String sectionName) {
String javadoc = getDocComment(node);
// step 1: Check if there is a 'SETTER' section. If yes, that becomes the new method's javadoc.
String out = getJavadocSection(javadoc, sectionName);
final boolean sectionBased = out != null;
if (!sectionBased) {
out = stripLinesWithTagFromJavadoc(stripSectionsFromJavadoc(javadoc), JavadocTag.RETURN);
}
return shouldReturnThis(node, EclipseHandlerUtil.getAccessorsForField(node)) ?
addReturnsThisIfNeeded(out) : out;
}
}
/**
* Copies javadoc on one node to the other.
*
* This one is a shortcut for {@link EclipseHandlerUtil#copyJavadoc(EclipseNode, ASTNode,
TypeDeclaration, CopyJavadoc, boolean)}
* if source and target node are in the same type.
*/
public static void copyJavadoc(EclipseNode from, ASTNode to, CopyJavadoc copyMode) {
copyJavadoc(from, to, (TypeDeclaration) upToTypeNode(from).get(), copyMode, false);
```

```
}
/**
* Copies javadoc on one node to the other.
*
* This one is a shortcut for {@link EclipseHandlerUtil#copyJavadoc(EclipseNode, ASTNode,
TypeDeclaration, CopyJavadoc, boolean)}
* if source and target node are in the same type.
*/
public static void copyJavadoc(EclipseNode from, ASTNode to, CopyJavadoc copyMode, boolean
forceAddReturn) {
copyJavadoc(from, to, (TypeDeclaration) upToTypeNode(from).get(), copyMode, forceAddReturn);
}
public static void copyJavadoc(EclipseNode from, ASTNode to, TypeDeclaration type, CopyJavadoc
copyMode) {
copyJavadoc(from, to, type, copyMode, false);
}
/**
* Copies javadoc on one node to the other.
*
* in 'GETTER' copyMode, first a 'GETTER' segment is searched for. If it exists, that will become the
javadoc for the 'to' node, and this section is
* stripped out of the 'from' node. If no 'GETTER' segment is found, then the entire javadoc is taken
minus any {@code @param} lines and other sections.
* any {@code @return} lines are stripped from 'from'.
*
* in 'SETTER' mode, stripping works similarly to 'GETTER' mode, except {@code param} are copied
and stripped from the original and {@code @return} are skipped.
*/
public static void copyJavadoc(EclipseNode from, ASTNode to, TypeDeclaration type, CopyJavadoc
copyMode, boolean forceAddReturn) {
if (copyMode == null) copyMode = CopyJavadoc.VERBATIM;
try {
CompilationUnitDeclaration cud = ((CompilationUnitDeclaration) from.top().get());
String newJavadoc = copyMode.apply(from);
if (forceAddReturn) {
newJavadoc = addReturnsThisIfNeeded(newJavadoc);
}
setDocComment(cud, type, to, newJavadoc);
} catch (Exception ignore) {}
}
public static void copyJavadocFromParam(EclipseNode from, MethodDeclaration to, TypeDeclaration
type, String param) {
try {
CompilationUnitDeclaration cud = (CompilationUnitDeclaration) from.top().get();
String methodComment = getDocComment(from);
String newJavadoc = addReturnsThisIfNeeded(getParamJavadoc(methodComment, param));
setDocComment(cud, type, to, newJavadoc);
} catch (Exception ignore) {}
}
/**
* Returns the method node containing the given annotation, or {@code null} if the given annotation
node is not on a method or argument.
*/
```

```
public static EclipseNode getAnnotatedMethod(EclipseNode node) {
if (node == null || node.getKind() != Kind.ANNOTATION) return null;
EclipseNode result = node.up();
if (result.getKind() == Kind.ARGUMENT) {
result = node.up();
}
if (result.getKind() != Kind.METHOD) {
result = null;
}
return result;
}
/**
* Returns {@code true} if the given method node body was parsed.
*/
public static boolean hasParsedBody(EclipseNode method) {
if (method == null || method.getKind() != Kind.METHOD) return false;
boolean isCompleteParse = method.getAst().isCompleteParse();
if (isCompleteParse) return true;
AbstractMethodDeclaration methodDecl = (AbstractMethodDeclaration) method.get();
if (methodDecl.statements != null) return true;
// If the method is part of a field initializer it was parsed
EclipseNode parent = method.up();
while (parent != null) {
if (parent.getKind() == Kind.FIELD) return true;
parent = parent.up();
}
return false;
}
}
```

lombok/eclipse/handlers/EclipseSingularsRecipes.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.IntLiteral;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Reference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.Wildcard;
import org.eclipse.jdt.internal.compiler.lookup.ClassScope;
import org.eclipse.jdt.internal.compiler.lookup.MethodScope;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
import lombok.AccessLevel;
import lombok.core.LombokImmutableList;
import lombok.core.SpiLoadUtil;
import lombok.core.TypeLibrary;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.HandleBuilder.BuilderJob;
public class EclipseSingularsRecipes {
public interface TypeReferenceMaker {
```

```
TypeReference make();
}
public interface StatementMaker {
Statement make();
}
private static final EclipseSingularsRecipes INSTANCE = new EclipseSingularsRecipes();
private final Map<String, EclipseSingularizer> singularizers = new HashMap<String,
EclipseSingularizer>();
private final TypeLibrary singularizableTypes = new TypeLibrary();
private EclipseSingularsRecipes() {
try {
loadAll(singularizableTypes, singularizers);
singularizableTypes.lock();
} catch (IOException e) {
System.err.println("Lombok's @Singularizable feature is broken due to misconfigured SPI files: " + e);
}
}
private static void loadAll(TypeLibrary library, Map<String, EclipseSingularizer> map) throws
IOException {
for (EclipseSingularizer handler : SpiLoadUtil.findServices(EclipseSingularizer.class,
EclipseSingularizer.class.getClassLoader())) {
for (String type : handler.getSupportedTypes()) {
EclipseSingularizer existingSingularizer = map.get(type);
if (existingSingularizer != null) {
EclipseSingularizer toKeep =
existingSingularizer.getClass().getName().compareTo(handler.getClass().getName()) > 0 ? handler :
existingSingularizer;
System.err.println("Multiple singularizers found for type " + type + "; the alphabetically first class is
used: " + toKeep.getClass().getName());
map.put(type, toKeep);
} else {
map.put(type, handler);
library.addType(type);
}
}
}
}
public static EclipseSingularsRecipes get() {
return INSTANCE;
}
public String toQualified(String typeReference) {
List<String> q = singularizableTypes.toQualifieds(typeReference);
if (q.isEmpty()) return null;
return q.get(0);
}
public EclipseSingularizer getSingularizer(String fqn) {
return singularizers.get(fqn);
}
public static final class SingularData {
private final EclipseNode annotation;
private final char[] singularName;
private final char[] pluralName;
private final char[] setterPrefix;
```

```
private final List<TypeReference> typeArgs;
private final String targetFqn;
private final EclipseSingularizer singularizer;
private final boolean ignoreNullCollections;
private final ASTNode source;
public SingularData(EclipseNode annotation, char[] singularName, char[] pluralName,
List<TypeReference> typeArgs, String targetFqn, EclipseSingularizer singularizer, ASTNode source,
boolean ignoreNullCollections) {
this(annotation, singularName, pluralName, typeArgs, targetFqn, singularizer, source,
ignoreNullCollections, new char[0]);
}
public SingularData(EclipseNode annotation, char[] singularName, char[] pluralName,
List<TypeReference> typeArgs, String targetFqn, EclipseSingularizer singularizer, ASTNode source,
boolean ignoreNullCollections, char[] setterPrefix) {
this.annotation = annotation;
this.singularName = singularName;
this.pluralName = pluralName;
this.typeArgs = typeArgs;
this.targetFqn = targetFqn;
this.singularizer = singularizer;
this.source = source;
this.ignoreNullCollections = ignoreNullCollections;
this.setterPrefix = setterPrefix;
}
public void setGeneratedByRecursive(ASTNode target) {
SetGeneratedByVisitor visitor = new SetGeneratedByVisitor(source);
if (target instanceof AbstractMethodDeclaration) {
((AbstractMethodDeclaration) target).traverse(visitor, (ClassScope) null);
} else if (target instanceof FieldDeclaration) {
((FieldDeclaration) target).traverse(visitor, (MethodScope) null);
} else {
target.traverse(visitor, null);
}
}
public ASTNode getSource() {
return source;
}
public EclipseNode getAnnotation() {
return annotation;
}
public char[] getSingularName() {
return singularName;
}
public char[] getPluralName() {
return pluralName;
}
public char[] getSetterPrefix() {
return setterPrefix;
}
public List<TypeReference> getTypeArgs() {
return typeArgs;
}
public String getTargetFqn() {
```

```
return targetFqn;
}
public EclipseSingularizer getSingularizer() {
return singularizer;
}
public boolean isIgnoreNullCollections() {
return ignoreNullCollections;
}
public String getTargetSimpleType() {
int idx = targetFqn.lastIndexOf(".");
return idx == -1 ? targetFqn : targetFqn.substring(idx + 1);
}
}
public static abstract class EclipseSingularizer {
protected static final long[] NULL_POSS = {0L};
public abstract LombokImmutableList<String> getSupportedTypes();
/** Checks if any of the to-be-generated nodes (fields, methods) already exist. If so, errors on these
(singulars don't support manually writing some of it, and returns true). */
public boolean checkForAlreadyExistingNodesAndGenerateError(EclipseNode builderType,
SingularData data) {
for (EclipseNode child : builderType.down()) {
switch (child.getKind()) {
case FIELD: {
FieldDeclaration fd = (FieldDeclaration) child.get();
char[] name = fd.name;
if (name == null) continue;
if (getGeneratedBy(fd) != null) continue;
for (char[] fieldToBeGenerated : listFieldsToBeGenerated(data, builderType)) {
if (!Arrays.equals(name, fieldToBeGenerated)) continue;
child.addError("Manually adding a field that @Singular @Builder would generate is not supported. If
you want to manually manage the builder aspect for this field/parameter, don't use @Singular.");
return true;
}
break;
}
case METHOD: {
AbstractMethodDeclaration method = (AbstractMethodDeclaration) child.get();
char[] name = method.selector;
if (name == null) continue;
if (getGeneratedBy(method) != null) continue;
for (char[] methodToBeGenerated : listMethodsToBeGenerated(data, builderType)) {
if (!Arrays.equals(name, methodToBeGenerated)) continue;
child.addError("Manually adding a method that @Singular @Builder would generate is not supported. If
you want to manually manage the builder aspect for this field/parameter, don't use @Singular.");
return true;
}
break;
}}
}
return false;
}
public List<char[]> listFieldsToBeGenerated(SingularData data, EclipseNode builderType) {
return Collections.singletonList(data.pluralName);
```

```
}
public List<char[]> listMethodsToBeGenerated(SingularData data, EclipseNode builderType) {
char[] p = data.pluralName;
char[] s = data.singularName;
if (Arrays.equals(p, s)) return Collections.singletonList(p);
return Arrays.asList(p, s);
}
public abstract List<EclipseNode> generateFields(SingularData data, EclipseNode builderType);
/**
* Generates the singular, plural, and clear methods for the given {@link SingularData}.
* Uses the given {@code builderType} as return type if {@code chain == true}, {@code void} otherwise.
* If you need more control over the return type and value, use
* {@link #generateMethods(SingularData, boolean, EclipseNode, boolean, TypeReferenceMaker,
StatementMaker)}.
*/
public void generateMethods(final BuilderJob job, SingularData data, boolean deprecate) {
TypeReferenceMaker returnTypeMaker = new TypeReferenceMaker() {
@Override public TypeReference make() {
return job.oldChain ? cloneSelfType(job.builderType) :
TypeReference.baseTypeReference(TypeIds.T_void, 0);
}
};
StatementMaker returnStatementMaker = new StatementMaker() {
@Override public ReturnStatement make() {
return job.oldChain ? new ReturnStatement(new ThisReference(0, 0), 0, 0) : null;
}
};
generateMethods(job.checkerFramework, data, deprecate, job.builderType, job.oldFluent,
returnTypeMaker, returnStatementMaker, job.accessInners);
}
/**
* Generates the singular, plural, and clear methods for the given {@link SingularData}.
* Uses the given {@code returnTypeMaker} and {@code returnStatementMaker} for the generated
methods.
*/
public abstract void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, EclipseNode builderType, boolean fluent, TypeReferenceMaker returnTypeMaker,
StatementMaker returnStatementMaker, AccessLevel access);
public abstract void appendBuildCode(SingularData data, EclipseNode builderType, List<Statement>
statements, char[] targetVariableName, String builderVariable);
public boolean shadowedDuringBuild() {
return true;
}
public boolean requiresCleaning() {
try {
return !getClass().getMethod("appendCleaningCode", SingularData.class, EclipseNode.class,
List.class).getDeclaringClass().equals(EclipseSingularizer.class);
} catch (NoSuchMethodException e) {
return false;
}
}
public void appendCleaningCode(SingularData data, EclipseNode builderType, List<Statement>
statements) {
```

```
}
// -- Utility methods --
protected Annotation[] generateSelfReturnAnnotations(boolean deprecate, ASTNode source) {
Annotation deprecated = deprecate ? generateDeprecatedAnnotation(source) : null;
if (deprecated == null) return null;
return new Annotation[] {deprecated};
}
/**
* Adds the requested number of type arguments to the provided type, copying each argument in
{@code typeArgs}. If typeArgs is too long, the extra elements are ignored.
* If {@code typeArgs} is null or too short, {@code java.lang.Object} will be substituted for each missing
type argument.
*
* @param count The number of type arguments requested.
* @param addExtends If {@code true}, all bounds are either '? extends X' or just '?'. If false, the
reverse is applied, and '? extends Foo' is converted to Foo, '?' to Object, etc.
* @param node Some node in the same AST. Just used to obtain makers and contexts and such.
* @param type The type to add generics to.
* @param typeArgs the list of type args to clone.
* @param source The source annotation that is the root cause of this code generation.
*/
protected TypeReference addTypeArgs(int count, boolean addExtends, EclipseNode node,
TypeReference type, List<TypeReference> typeArgs) {
TypeReference[] clonedAndFixedArgs = createTypeArgs(count, addExtends, node, typeArgs);
if (type instanceof SingleTypeReference) {
type = new ParameterizedSingleTypeReference(((SingleTypeReference) type).token,
clonedAndFixedArgs, 0, 0L);
} else if (type instanceof QualifiedTypeReference) {
QualifiedTypeReference qtr = (QualifiedTypeReference) type;
TypeReference[][] trs = new TypeReference[qtr.tokens.length][];
trs[qtr.tokens.length - 1] = clonedAndFixedArgs;
type = new ParameterizedQualifiedTypeReference(((QualifiedTypeReference) type).tokens, trs, 0,
NULL_POSS);
} else {
node.addError("Don't know how to clone-and-parameterize type: " + type);
}
return type;
}
protected TypeReference[] createTypeArgs(int count, boolean addExtends, EclipseNode node,
List<TypeReference> typeArgs) {
if (count < 0) throw new IllegalArgumentException("count is negative");
if (count == 0) return null;
List<TypeReference> arguments = new ArrayList<TypeReference>();
if (typeArgs != null) for (TypeReference orig : typeArgs) {
Wildcard wildcard = orig instanceof Wildcard ? (Wildcard) orig : null;
if (!addExtends) {
if (wildcard != null && (wildcard.kind == Wildcard.UNBOUND || wildcard.kind == Wildcard.SUPER)) {
arguments.add(new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, NULL_POSS));
} else if (wildcard != null && wildcard.kind == Wildcard.EXTENDS) {
try {
arguments.add(copyType(wildcard.bound));
} catch (Exception e) {
arguments.add(new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, NULL_POSS));
```

```
}
} else {
arguments.add(copyType(orig));
}
} else {
if (wildcard != null && (wildcard.kind == Wildcard.UNBOUND || wildcard.kind == Wildcard.SUPER)) {
Wildcard w = new Wildcard(Wildcard.UNBOUND);
arguments.add(w);
} else if (wildcard != null && wildcard.kind == Wildcard.EXTENDS) {
arguments.add(copyType(orig));
} else {
Wildcard w = new Wildcard(Wildcard.EXTENDS);
w.bound = copyType(orig);
arguments.add(w);
}
}
if (--count == 0) break;
}
while (count-- > 0) {
if (addExtends) {
arguments.add(new Wildcard(Wildcard.UNBOUND));
} else {
arguments.add(new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, NULL_POSS));
}
}
if (arguments.isEmpty()) return null;
return arguments.toArray(new TypeReference[0]);
}
private static final char[] SIZE_TEXT = new char[] {'s', 'i', 'z', 'e'};
/** Generates 'this.<em>name</em>.size()' as an expression; if nullGuard is true, it's this.name == null
? 0 : this.name.size(). */
protected Expression getSize(EclipseNode builderType, char[] name, boolean nullGuard, String
builderVariable) {
MessageSend invoke = new MessageSend();
Reference thisRef = getBuilderReference(builderVariable);
FieldReference thisDotName = new FieldReference(name, 0L);
thisDotName.receiver = thisRef;
invoke.receiver = thisDotName;
invoke.selector = SIZE_TEXT;
if (!nullGuard) return invoke;
Reference cdnThisRef = getBuilderReference(builderVariable);
FieldReference cdnThisDotName = new FieldReference(name, 0L);
cdnThisDotName.receiver = cdnThisRef;
NullLiteral nullLiteral = new NullLiteral(0, 0);
EqualExpression isNull = new EqualExpression(cdnThisDotName, nullLiteral,
OperatorIds.EQUAL_EQUAL);
IntLiteral zeroLiteral = makeIntLiteral(new char[] {'0'}, null);
ConditionalExpression conditional = new ConditionalExpression(isNull, zeroLiteral, invoke);
return conditional;
}
protected TypeReference cloneParamType(int index, List<TypeReference> typeArgs, EclipseNode
builderType) {
if (typeArgs != null && typeArgs.size() > index) {
```

```
TypeReference originalType = typeArgs.get(index);
if (originalType instanceof Wildcard) {
Wildcard wOriginalType = (Wildcard) originalType;
if (wOriginalType.kind == Wildcard.EXTENDS) {
try {
return copyType(wOriginalType.bound);
} catch (Exception e) {
// fallthrough
}
}
} else {
return copyType(originalType);
}
}
return new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, NULL_POSS);
}
/** @return a {@code SingleNameReference} to the builder in the variable
<code>builderVariable</code>. If {@ code builderVariable == "this"}, a {@code ThisReference} is
returned. */
protected static Reference getBuilderReference(String builderVariable) {
if ("this".equals(builderVariable)) {
return new ThisReference(0, 0);
} else {
return new SingleNameReference(builderVariable.toCharArray(), 0);
}
}
protected void nullBehaviorize(EclipseNode typeNode, SingularData data, List<Statement> statements,
Argument arg, MethodDeclaration md) {
boolean ignoreNullCollections = data.isIgnoreNullCollections();
if (ignoreNullCollections) {
Expression isNotNull = new EqualExpression(new SingleNameReference(data.getPluralName(), 0L),
new NullLiteral(0, 0), OperatorIds.NOT_EQUAL);
Block b = new Block(0);
b.statements = statements.toArray(new Statement[statements.size()]);
statements.clear();
statements.add(new IfStatement(isNotNull, b, 0, 0));
EclipseHandlerUtil.createRelevantNullableAnnotation(typeNode, arg, md);
return;
}
EclipseHandlerUtil.createRelevantNonNullAnnotation(typeNode, arg, md);
Statement nullCheck = EclipseHandlerUtil.generateNullCheck(null, data.getPluralName(), typeNode,
"%s cannot be null");
statements.add(0, nullCheck);
}
protected abstract int getTypeArgumentsCount();
protected abstract char[][] getEmptyMakerReceiver(String targetFqn);
protected abstract char[] getEmptyMakerSelector(String targetFqn);
public MessageSend getEmptyExpression(String targetFqn, SingularData data, EclipseNode
typeNode, ASTNode source) {
MessageSend send = new MessageSend();
send.receiver = generateQualifiedNameRef(source, getEmptyMakerReceiver(targetFqn));
send.selector = getEmptyMakerSelector(targetFqn);
```

```
send.typeArguments = createTypeArgs(getTypeArgumentsCount(), false, typeNode,
data.getTypeArgs());
return send;
}
}
}
```

lombok/eclipse/handlers/HandleAccessors.java

```
/*
* Copyright (C) 2014-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.Accessors;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
@Provides
@HandlerPriority(65536)
public class HandleAccessors extends EclipseAnnotationHandler<Accessors> {
@Override public void handle(AnnotationValues<Accessors> annotation, Annotation ast, EclipseNode
annotationNode) {
// Accessors itself is handled by HandleGetter/Setter; this is just to ensure that usages are flagged if
requested.
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.ACCESSORS_FLAG_USAGE,
"@Accessors");
if (annotation.isMarking()) annotationNode.addWarning("Accessors on its own does nothing. Set at
least one parameter");
```

}
}

lombok/eclipse/handlers/HandleBuilder.java

/*
* Copyright (C) 2013-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.ArrayAllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FalseLiteral;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;

```
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedAllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.QualifiedThisReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Receiver;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleMemberAnnotation;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TrueLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeParameter;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.UnaryExpression;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.ClassScope;
import org.eclipse.jdt.internal.compiler.lookup.MethodScope;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
import lombok.AccessLevel;
import lombok.Builder;
import lombok.Builder.ObtainVia;
import lombok.ConfigurationKeys;
import lombok.Singular;
import lombok.ToString;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.CopyJavadoc;
import lombok.eclipse.handlers.EclipseHandlerUtil.MemberExistsResult;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
import lombok.eclipse.handlers.HandleConstructor.SkipIfConstructorExists;
import lombok.experimental.NonFinal;
import lombok.spi.Provides;
@Provides
```

```
@HandlerPriority(-1024) //-2^10; to ensure we've picked up @FieldDefault's changes (-2048) but
@Value hasn't removed itself yet (-512), so that we can error on presence of it on the builder classes.
public class HandleBuilder extends EclipseAnnotationHandler<Builder> {
private HandleConstructor handleConstructor = new HandleConstructor();
static final char[] CLEAN_FIELD_NAME = "$lombokUnclean".toCharArray();
static final char[] CLEAN_METHOD_NAME = "$lombokClean".toCharArray();
static final String TO_BUILDER_METHOD_NAME_STRING = "toBuilder";
static final char[] TO_BUILDER_METHOD_NAME =
TO_BUILDER_METHOD_NAME_STRING.toCharArray();
static final char[] DEFAULT_PREFIX = {'$', 'd', 'e', 'f', 'a', 'u', 'l', 't', '$'};
static final char[] SET_PREFIX = {'$', 's', 'e', 't'};
static final char[] VALUE_PREFIX = {'$', 'v', 'a', 'l', 'u', 'e'};
static final char[] BUILDER_TEMP_VAR = {'b', 'u', 'i', 'l', 'd', 'e', 'r'};
static final AbstractMethodDeclaration[] EMPTY_METHODS = {};
static final String TO_BUILDER_NOT_SUPPORTED = "@Builder(toBuilder=true) is only supported if
you return your own type.";
private static final boolean toBoolean(Object expr, boolean defaultValue) {
if (expr == null) return defaultValue;
if (expr instanceof FalseLiteral) return false;
if (expr instanceof TrueLiteral) return true;
return ((Boolean) expr).booleanValue();
}
static class BuilderJob {
CheckerFrameworkVersion checkerFramework;
EclipseNode parentType;
String builderMethodName, buildMethodName;
boolean isStatic;
TypeParameter[] typeParams;
TypeParameter[] builderTypeParams;
ASTNode source;
EclipseNode sourceNode;
List<BuilderFieldData> builderFields;
AccessLevel accessInners, accessOuters;
boolean oldFluent, oldChain, toBuilder;
EclipseNode builderType;
String builderClassName;
char[] builderClassNameArr;
void setBuilderClassName(String builderClassName) {
this.builderClassName = builderClassName;
this.builderClassNameArr = builderClassName.toCharArray();
}
TypeParameter[] copyTypeParams() {
return EclipseHandlerUtil.copyTypeParams(typeParams, source);
}
long getPos() {
return ((long) source.sourceStart) << 32 | source.sourceEnd;
}
public TypeReference createBuilderTypeReference() {
return namePlusTypeParamsToTypeReference(parentType, builderClassNameArr, !isStatic,
builderTypeParams, getPos());
}
public TypeReference createBuilderTypeReferenceForceStatic() {
```

```
return namePlusTypeParamsToTypeReference(parentType, builderClassNameArr, false,
builderTypeParams, getPos());
}
public TypeReference createBuilderParentTypeReference() {
return namePlusTypeParamsToTypeReference(parentType, typeParams, getPos());
}
public EclipseNode getTopNode() {
return parentType.top();
}
void init(AnnotationValues<Builder> annValues, Builder ann, EclipseNode node) {
accessOuters = ann.access();
if (accessOuters == null) accessOuters = AccessLevel.PUBLIC;
if (accessOuters == AccessLevel.NONE) {
sourceNode.addError("AccessLevel.NONE is not valid here");
accessOuters = AccessLevel.PUBLIC;
}
accessInners = accessOuters == AccessLevel.PROTECTED ? AccessLevel.PUBLIC : accessOuters;
// These exist just to support the 'old' lombok.experimental.Builder, which had these properties.
lombok.Builder no longer has them.
oldFluent = toBoolean(annValues.getActualExpression("fluent"), true);
oldChain = toBoolean(annValues.getActualExpression("chain"), true);
builderMethodName = ann.builderMethodName();
buildMethodName = ann.buildMethodName();
setBuilderClassName(getBuilderClassNameTemplate(node, ann.builderClassName()));
toBuilder = ann.toBuilder();
if (builderMethodName == null) builderMethodName = "builder";
if (buildMethodName == null) buildMethodName = "build";
}
static String getBuilderClassNameTemplate(EclipseNode node, String override) {
if (override != null && !override.isEmpty()) return override;
override = node.getAst().readConfiguration(ConfigurationKeys.BUILDER_CLASS_NAME);
if (override != null && !override.isEmpty()) return override;
return "*Builder";
}
MethodDeclaration createNewMethodDeclaration() {
return new MethodDeclaration(((CompilationUnitDeclaration) getTopNode().get()).compilationResult);
}
String replaceBuilderClassName(char[] name) {
return replaceBuilderClassName(name, builderClassName);
}
String replaceBuilderClassName(char[] name, String template) {
if (template.indexOf('*') == -1) return template;
return template.replace("*", new String(name));
}
String replaceBuilderClassName(String name) {
return builderClassName.replace("*", name);
}
}
static class BuilderFieldData {
Annotation[] annotations;
TypeReference type;
char[] rawName;
char[] name;
```

```
char[] builderFieldName;
char[] nameOfDefaultProvider;
char[] nameOfSetFlag;
SingularData singularData;
ObtainVia obtainVia;
EclipseNode obtainViaNode;
EclipseNode originalFieldNode;
List<EclipseNode> createdFields = new ArrayList<EclipseNode>();
}
private static boolean equals(String a, char[] b) {
if (a.length() != b.length) return false;
for (int i = 0; i < b.length; i++) {
if (a.charAt(i) != b[i]) return false;
}
return true;
}
private static boolean equals(String a, char[][] b) {
if (a == null || a.isEmpty()) return b.length == 0;
String[] aParts = a.split("\\.");
if (aParts.length != b.length) return false;
for (int i = 0; i < b.length; i++) {
if (!equals(aParts[i], b[i])) return false;
}
return true;
}
private static final char[] prefixWith(char[] prefix, char[] name) {
char[] out = new char[prefix.length + name.length];
System.arraycopy(prefix, 0, out, 0, prefix.length);
System.arraycopy(name, 0, out, prefix.length, name.length);
return out;
}
@Override public void handle(AnnotationValues<Builder> annotation, Annotation ast, EclipseNode
annotationNode) {
final String BUILDER_NODE_NOT_SUPPORTED_ERR = "@Builder is only supported on classes,
records, constructors, and methods.";
handleFlagUsage(annotationNode, ConfigurationKeys.BUILDER_FLAG_USAGE, "@Builder");
BuilderJob job = new BuilderJob();
job.sourceNode = annotationNode;
job.source = ast;
job.checkerFramework = getCheckerFrameworkVersion(annotationNode);
job.isStatic = true;
Builder annInstance = annotation.getInstance();
job.init(annotation, annInstance, annotationNode);
List<char[]> typeArgsForToBuilder = null;
boolean generateBuilderMethod;
if (job.builderMethodName.isEmpty()) {
generateBuilderMethod = false;
} else if (!checkName("builderMethodName", job.builderMethodName, annotationNode)) {
return;
} else {
generateBuilderMethod = true;
}
if (!checkName("buildMethodName", job.buildMethodName, annotationNode)) return;
```

```
EclipseNode parent = annotationNode.up();
job.builderFields = new ArrayList<BuilderFieldData>();
TypeReference buildMethodReturnType;
TypeReference[] buildMethodThrownExceptions;
char[] nameOfBuilderMethod;
EclipseNode fillParametersFrom = parent.get() instanceof AbstractMethodDeclaration ? parent : null;
boolean addCleaning = false;
List<EclipseNode> nonFinalNonDefaultedFields = null;
if (!isStaticAllowed(upToTypeNode(parent))) {
annotationNode.addError("@Builder is not supported on non-static nested classes.");
return;
}
if (parent.get() instanceof TypeDeclaration) {
if (!isClass(parent) && !isRecord(parent)) {
annotationNode.addError(BUILDER_NODE_NOT_SUPPORTED_ERR);
return;
}
job.parentType = parent;
TypeDeclaration td = (TypeDeclaration) parent.get();
List<EclipseNode> allFields = new ArrayList<EclipseNode>();
boolean valuePresent = (hasAnnotation(lombok.Value.class, parent) ||
hasAnnotation("lombok.experimental.Value", parent));
for (EclipseNode fieldNode : HandleConstructor.findAllFields(parent, true)) {
FieldDeclaration fd = (FieldDeclaration) fieldNode.get();
EclipseNode isDefault = findAnnotation(Builder.Default.class, fieldNode);
boolean isFinal = ((fd.modifiers & ClassFileConstants.AccFinal) != 0) || (valuePresent &&
!hasAnnotation(NonFinal.class, fieldNode));
BuilderFieldData bfd = new BuilderFieldData();
bfd.rawName = fieldNode.getName().toCharArray();
bfd.name = removePrefixFromField(fieldNode);
bfd.builderFieldName = bfd.name;
bfd.annotations = copyAnnotations(fd, findCopyableAnnotations(fieldNode));
bfd.type = fd.type;
bfd.singularData = getSingularData(fieldNode, ast, annInstance.setterPrefix());
bfd.originalFieldNode = fieldNode;
if (bfd.singularData != null && isDefault != null) {
isDefault.addError("@Builder.Default and @Singular cannot be mixed.");
isDefault = null;
}
if (fd.initialization == null && isDefault != null) {
isDefault.addWarning("@Builder.Default requires an initializing expression (' = something;').");
isDefault = null;
}
if (fd.initialization != null && isDefault == null) {
if (isFinal) continue;
if (nonFinalNonDefaultedFields == null) nonFinalNonDefaultedFields = new ArrayList<EclipseNode>();
nonFinalNonDefaultedFields.add(fieldNode);
}
if (isDefault != null) {
bfd.nameOfDefaultProvider = prefixWith(DEFAULT_PREFIX, bfd.name);
bfd.nameOfSetFlag = prefixWith(bfd.name, SET_PREFIX);
bfd.builderFieldName = prefixWith(bfd.name, VALUE_PREFIX);
```

```
MethodDeclaration md = generateDefaultProvider(bfd.nameOfDefaultProvider, td.typeParameters,
fieldNode, ast);
if (md != null) injectMethod(parent, md);
}
addObtainVia(bfd, fieldNode);
job.builderFields.add(bfd);
allFields.add(fieldNode);
}
if (!isRecord(parent)) {
// Records ship with a canonical constructor that acts as @AllArgsConstructor - just use that one.
handleConstructor.generateConstructor(parent, AccessLevel.PACKAGE, allFields, false, null,
SkipIfConstructorExists.I_AM_BUILDER,
Collections.<Annotation>emptyList(), annotationNode);
}
job.typeParams = job.builderTypeParams = td.typeParameters;
buildMethodReturnType = job.createBuilderParentTypeReference();
buildMethodThrownExceptions = null;
nameOfBuilderMethod = null;
job.setBuilderClassName(job.replaceBuilderClassName(td.name));
if (!checkName("builderClassName", job.builderClassName, annotationNode)) return;
} else if (parent.get() instanceof ConstructorDeclaration) {
ConstructorDeclaration cd = (ConstructorDeclaration) parent.get();
if (cd.typeParameters != null && cd.typeParameters.length > 0) {
annotationNode.addError("@Builder is not supported on constructors with constructor type
parameters.");
return;
}
job.parentType = parent.up();
TypeDeclaration td = (TypeDeclaration) job.parentType.get();
job.typeParams = job.builderTypeParams = td.typeParameters;
buildMethodReturnType = job.createBuilderParentTypeReference();
buildMethodThrownExceptions = cd.thrownExceptions;
nameOfBuilderMethod = null;
job.setBuilderClassName(job.replaceBuilderClassName(cd.selector));
if (!checkName("builderClassName", job.builderClassName, annotationNode)) return;
} else if (parent.get() instanceof MethodDeclaration) {
MethodDeclaration md = (MethodDeclaration) parent.get();
job.parentType = parent.up();
job.isStatic = md.isStatic();
if (job.toBuilder) {
char[] token;
char[][] pkg = null;
if (md.returnType.dimensions() > 0) {
annotationNode.addError(TO_BUILDER_NOT_SUPPORTED);
return;
}
if (md.returnType instanceof SingleTypeReference) {
token = ((SingleTypeReference) md.returnType).token;
} else if (md.returnType instanceof QualifiedTypeReference) {
pkg = ((QualifiedTypeReference) md.returnType).tokens;
token = pkg[pkg.length];
char[][] pkg_ = new char[pkg.length - 1][];
System.arraycopy(pkg, 0, pkg_, 0, pkg_.length);
```

```
pkg = pkg_;
} else {
annotationNode.addError(TO_BUILDER_NOT_SUPPORTED);
return;
}
if (pkg != null && !equals(parent.getPackageDeclaration(), pkg)) {
annotationNode.addError(TO_BUILDER_NOT_SUPPORTED);
return;
}
if (job.parentType == null || !equals(job.parentType.getName(), token)) {
annotationNode.addError(TO_BUILDER_NOT_SUPPORTED);
return;
}
TypeParameter[] tpOnType = ((TypeDeclaration) job.parentType.get()).typeParameters;
TypeParameter[] tpOnMethod = md.typeParameters;
TypeReference[][] tpOnRet_ = null;
if (md.returnType instanceof ParameterizedSingleTypeReference) {
tpOnRet_ = new TypeReference[1][];
tpOnRet_[0] = ((ParameterizedSingleTypeReference) md.returnType).typeArguments;
} else if (md.returnType instanceof ParameterizedQualifiedTypeReference) {
tpOnRet_ = ((ParameterizedQualifiedTypeReference) md.returnType).typeArguments;
}
if (tpOnRet_ != null) for (int i = 0; i < tpOnRet_.length - 1; i++) {
if (tpOnRet_[i] != null && tpOnRet_[i].length > 0) {
annotationNode.addError("@Builder(toBuilder=true) is not supported if returning a type with generics
applied to an intermediate.");
return;
}
}
TypeReference[] tpOnRet = tpOnRet_ == null ? null : tpOnRet_[tpOnRet_.length - 1];
typeArgsForToBuilder = new ArrayList<char[]>();
// Every typearg on this method needs to be found in the return type, but the reverse is not true.
// We also need to 'map' them.
if (tpOnMethod != null) for (TypeParameter onMethod : tpOnMethod) {
int pos = -1;
if (tpOnRet != null) for (int i = 0; i < tpOnRet.length; i++) {
if (tpOnRet[i].getClass() != SingleTypeReference.class) continue;
if (!Arrays.equals(((SingleTypeReference) tpOnRet[i]).token, onMethod.name)) continue;
pos = i;
}
if (pos == -1 || tpOnType == null || tpOnType.length <= pos) {
annotationNode.addError("@Builder(toBuilder=true) requires that each type parameter on the static
method is part of the typeargs of the return value. Type parameter " + new String(onMethod.name) + "
is not part of the return type.");
return;
}
typeArgsForToBuilder.add(tpOnType[pos].name);
}
}
job.typeParams = job.builderTypeParams = md.typeParameters;
buildMethodReturnType = copyType(md.returnType, ast);
buildMethodThrownExceptions = md.thrownExceptions;
nameOfBuilderMethod = md.selector;
```

```
if (job.builderClassName.indexOf('*') > -1) {
char[] token = returnTypeToBuilderClassName(annotationNode, md, job.typeParams);
if (token == null) return; // should not happen.
job.setBuilderClassName(job.replaceBuilderClassName(token));
if (!checkName("builderClassName", job.builderClassName, annotationNode)) return;
}
} else {
annotationNode.addError(BUILDER_NODE_NOT_SUPPORTED_ERR);
return;
}
if (fillParametersFrom != null) {
for (EclipseNode param : fillParametersFrom.down()) {
if (param.getKind() != Kind.ARGUMENT) continue;
BuilderFieldData bfd = new BuilderFieldData();
Argument arg = (Argument) param.get();
Annotation[] copyableAnnotations = findCopyableAnnotations(param);
bfd.rawName = arg.name;
bfd.name = arg.name;
bfd.builderFieldName = bfd.name;
bfd.annotations = copyAnnotations(arg, copyableAnnotations);
bfd.type = arg.type;
bfd.singularData = getSingularData(param, ast, annInstance.setterPrefix());
bfd.originalFieldNode = param;
addObtainVia(bfd, param);
job.builderFields.add(bfd);
}
}
job.builderType = findInnerClass(job.parentType, job.builderClassName);
if (job.builderType == null) makeBuilderClass(job);
else {
TypeDeclaration builderTypeDeclaration = (TypeDeclaration) job.builderType.get();
if (job.isStatic && (builderTypeDeclaration.modifiers & ClassFileConstants.AccStatic) == 0) {
annotationNode.addError("Existing Builder must be a static inner class.");
return;
} else if (!job.isStatic && (builderTypeDeclaration.modifiers & ClassFileConstants.AccStatic) != 0) {
annotationNode.addError("Existing Builder must be a non-static inner class.");
return;
}
sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(job.builderType, annotationNode);
/* generate errors for @Singular BFDs that have one already defined node. */ {
for (BuilderFieldData bfd : job.builderFields) {
SingularData sd = bfd.singularData;
if (sd == null) continue;
EclipseSingularizer singularizer = sd.getSingularizer();
if (singularizer == null) continue;
if (singularizer.checkForAlreadyExistingNodesAndGenerateError(job.builderType, sd)) {
bfd.singularData = null;
}
}
}
}
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
```

```
if (bfd.singularData.getSingularizer().requiresCleaning()) {
addCleaning = true;
break;
}
}
if (bfd.obtainVia != null) {
if (bfd.obtainVia.field().isEmpty() == bfd.obtainVia.method().isEmpty()) {
bfd.obtainViaNode.addError("The syntax is either @ObtainVia(field = \"fieldName\") or
@ObtainVia(method = \"methodName\").");
return;
}
if (bfd.obtainVia.method().isEmpty() && bfd.obtainVia.isStatic()) {
bfd.obtainViaNode.addError("@ObtainVia(isStatic = true) is not valid unless 'method' has been set.");
return;
}
}
}
generateBuilderFields(job);
if (addCleaning) {
FieldDeclaration cleanDecl = new FieldDeclaration(CLEAN_FIELD_NAME, 0, -1);
cleanDecl.declarationSourceEnd = -1;
cleanDecl.modifiers = ClassFileConstants.AccPrivate;
cleanDecl.type = TypeReference.baseTypeReference(TypeIds.T_boolean, 0);
cleanDecl.traverse(new SetGeneratedByVisitor(ast), (MethodScope) null);
injectFieldAndMarkGenerated(job.builderType, cleanDecl);
}
if (constructorExists(job.builderType) == MemberExistsResult.NOT_EXISTS) {
ConstructorDeclaration cd = HandleConstructor.createConstructor(
AccessLevel.PACKAGE, job.builderType, Collections.<EclipseNode>emptyList(), false,
annotationNode, Collections.<Annotation>emptyList());
if (cd != null) injectMethod(job.builderType, cd);
}
for (BuilderFieldData bfd : job.builderFields) {
makePrefixedSetterMethodsForBuilder(job, bfd, annInstance.setterPrefix());
}
{
MemberExistsResult methodExists = methodExists(job.buildMethodName, job.builderType, -1);
if (methodExists == MemberExistsResult.EXISTS_BY_LOMBOK) methodExists =
methodExists(job.buildMethodName, job.builderType, 0);
if (methodExists == MemberExistsResult.NOT_EXISTS) {
MethodDeclaration md = generateBuildMethod(job, nameOfBuilderMethod, buildMethodReturnType,
buildMethodThrownExceptions, addCleaning);
if (md != null) injectMethod(job.builderType, md);
}
}
if (methodExists("toString", job.builderType, 0) == MemberExistsResult.NOT_EXISTS) {
List<Included<EclipseNode, ToString.Include>> fieldNodes = new ArrayList<Included<EclipseNode,
ToString.Include>>();
for (BuilderFieldData bfd : job.builderFields) {
for (EclipseNode f : bfd.createdFields) {
fieldNodes.add(new Included<EclipseNode, ToString.Include>(f, null, true, false));
}
}
```

```
MethodDeclaration md = HandleToString.createToString(job.builderType, fieldNodes, true, false, ast,
FieldAccess.ALWAYS_FIELD);
if (md != null) injectMethod(job.builderType, md);
}
if (addCleaning) {
MethodDeclaration cleanMethod = generateCleanMethod(job);
if (cleanMethod != null) injectMethod(job.builderType, cleanMethod);
}
if (generateBuilderMethod && methodExists(job.builderMethodName, job.parentType, -1) !=
MemberExistsResult.NOT_EXISTS) generateBuilderMethod = false;
if (generateBuilderMethod) {
MethodDeclaration md = generateBuilderMethod(job);
if (md != null) injectMethod(job.parentType, md);
}
if (job.toBuilder) switch (methodExists(TO_BUILDER_METHOD_NAME_STRING, job.parentType, 0)) {
case EXISTS_BY_USER:
annotationNode.addWarning("Not generating toBuilder() as it already exists.");
break;
case NOT_EXISTS:
TypeParameter[] tps = job.typeParams;
if (typeArgsForToBuilder != null) {
tps = new TypeParameter[typeArgsForToBuilder.size()];
for (int i = 0; i < tps.length; i++) {
tps[i] = new TypeParameter();
tps[i].name = typeArgsForToBuilder.get(i);
}
}
MethodDeclaration md = generateToBuilderMethod(job, tps, annInstance.setterPrefix());
if (md != null) injectMethod(job.parentType, md);
}
if (nonFinalNonDefaultedFields != null && generateBuilderMethod) {
for (EclipseNode fieldNode : nonFinalNonDefaultedFields) {
fieldNode.addWarning("@Builder will ignore the initializing expression entirely. If you want the
initializing expression to serve as default, add @Builder.Default. If it is not supposed to be settable
during building, make the field final.");
}
}
}
static char[] returnTypeToBuilderClassName(EclipseNode annotationNode, MethodDeclaration md,
TypeParameter[] typeParams) {
char[] token;
if (md.returnType instanceof QualifiedTypeReference) {
char[][] tokens = ((QualifiedTypeReference) md.returnType).tokens;
token = tokens[tokens.length - 1];
} else if (md.returnType instanceof SingleTypeReference) {
token = ((SingleTypeReference) md.returnType).token;
if (!(md.returnType instanceof ParameterizedSingleTypeReference) && typeParams != null) {
for (TypeParameter tp : typeParams) {
if (Arrays.equals(tp.name, token)) {
annotationNode.addError("@Builder requires specifying 'builderClassName' if used on methods with a
type parameter as return type.");
return null;
}
```

```
}
}
} else {
annotationNode.addError("Unexpected kind of return type on annotated method. Specify
'builderClassName' to solve this problem.");
return null;
}
if (Character.isLowerCase(token[0])) {
char[] newToken = new char[token.length];
System.arraycopy(token, 1, newToken, 1, token.length - 1);
newToken[0] = Character.toTitleCase(token[0]);
token = newToken;
}
return token;
}
private MethodDeclaration generateToBuilderMethod(BuilderJob job, TypeParameter[]
typeParameters, String prefix) {
int pS = job.source.sourceStart, pE = job.source.sourceEnd;
long p = job.getPos();
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = TO_BUILDER_METHOD_NAME;
out.modifiers = toEclipseModifier(job.accessOuters);
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.returnType = job.createBuilderTypeReference();
if (job.checkerFramework.generateUnique()) {
int len = out.returnType.getTypeName().length;
out.returnType.annotations = new Annotation[len][];
out.returnType.annotations[len - 1] = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__UNIQUE)};
}
AllocationExpression invoke = new AllocationExpression();
invoke.type = job.createBuilderTypeReference();
Expression receiver = invoke;
List<Statement> preStatements = null;
List<Statement> postStatements = null;
for (BuilderFieldData bfd : job.builderFields) {
String setterName = new String(bfd.name);
String setterPrefix = !prefix.isEmpty() ? prefix : job.oldFluent ? "" : "set";
if (!setterPrefix.isEmpty()) setterName = HandlerUtil.buildAccessorName(job.sourceNode, setterPrefix,
setterName);
MessageSend ms = new MessageSend();
Expression[] tgt = new Expression[bfd.singularData == null ? 1 : 2];
if (bfd.obtainVia == null || !bfd.obtainVia.field().isEmpty()) {
char[] fieldName = bfd.obtainVia == null ? bfd.rawName : bfd.obtainVia.field().toCharArray();
for (int i = 0; i < tgt.length; i++) {
FieldReference fr = new FieldReference(fieldName, 0);
fr.receiver = new ThisReference(0, 0);
tgt[i] = fr;
}
} else {
String obtainName = bfd.obtainVia.method();
boolean obtainIsStatic = bfd.obtainVia.isStatic();
MessageSend obtainExpr = new MessageSend();
```

```
if (obtainIsStatic) {
if (typeParameters != null && typeParameters.length > 0) {
obtainExpr.typeArguments = new TypeReference[typeParameters.length];
for (int j = 0; j < typeParameters.length; j++) {
obtainExpr.typeArguments[j] = new SingleTypeReference(typeParameters[j].name, 0);
}
}
obtainExpr.receiver = generateNameReference(job.parentType, 0);
} else {
obtainExpr.receiver = new ThisReference(0, 0);
}
obtainExpr.selector = obtainName.toCharArray();
if (obtainIsStatic) obtainExpr.arguments = new Expression[] {new ThisReference(0, 0)};
for (int i = 0; i < tgt.length; i++) tgt[i] = new SingleNameReference(bfd.name, 0L);
// javac appears to cache the type of JCMethodInvocation expressions based on position, meaning, if
you have 2 ObtainVia-based method invokes on different types, you get bizarre type mismatch errors.
// going via a local variable declaration solves the problem. We copy this behaviour
// for ecj so we match what javac's handler does.
LocalDeclaration ld = new LocalDeclaration(bfd.name, 0, 0);
ld.modifiers = ClassFileConstants.AccFinal;
ld.type = EclipseHandlerUtil.copyType(bfd.type, job.source);
ld.initialization = obtainExpr;
if (preStatements == null) preStatements = new ArrayList<Statement>();
preStatements.add(ld);
}
ms.selector = setterName.toCharArray();
if (bfd.singularData == null) {
ms.arguments = tgt;
ms.receiver = receiver;
receiver = ms;
} else {
ms.arguments = new Expression[] {tgt[1]};
ms.receiver = new SingleNameReference(BUILDER_TEMP_VAR, p);
EqualExpression isNotNull = new EqualExpression(tgt[0], new NullLiteral(pS, pE),
OperatorIds.NOT_EQUAL);
if (postStatements == null) postStatements = new ArrayList<Statement>();
postStatements.add(new IfStatement(isNotNull, ms, pS, pE));
}
}
int preSs = preStatements == null ? 0 : preStatements.size();
int postSs = postStatements == null ? 0 : postStatements.size();
if (postSs > 0) {
out.statements = new Statement[preSs + postSs + 2];
for (int i = 0; i < preSs; i++) out.statements[i] = preStatements.get(i);
for (int i = 0; i < postSs; i++) out.statements[preSs + 1 + i] = postStatements.get(i);
LocalDeclaration b = new LocalDeclaration(BUILDER_TEMP_VAR, pS, pE);
out.statements[preSs] = b;
b.modifiers |= ClassFileConstants.AccFinal;
b.type = job.createBuilderTypeReference();
b.type.sourceStart = pS; b.type.sourceEnd = pE;
b.initialization = receiver;
out.statements[preSs + postSs + 1] = new ReturnStatement(new
SingleNameReference(BUILDER_TEMP_VAR, p), pS, pE);
```

```
} else {
out.statements = new Statement[preSs + 1];
for (int i = 0; i < preSs; i++) out.statements[i] = preStatements.get(i);
out.statements[preSs] = new ReturnStatement(receiver, pS, pE);
}
createRelevantNonNullAnnotation(job.parentType, out);
out.traverse(new SetGeneratedByVisitor(job.source), ((TypeDeclaration) job.parentType.get()).scope);
return out;
}
private MethodDeclaration generateCleanMethod(BuilderJob job) {
List<Statement> statements = new ArrayList<Statement>();
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.singularData.getSingularizer().appendCleaningCode(bfd.singularData, job.builderType,
statements);
}
}
FieldReference thisUnclean = new FieldReference(CLEAN_FIELD_NAME, 0);
thisUnclean.receiver = new ThisReference(0, 0);
statements.add(new Assignment(thisUnclean, new FalseLiteral(0, 0), 0));
MethodDeclaration decl = job.createNewMethodDeclaration();
decl.selector = CLEAN_METHOD_NAME;
decl.modifiers = ClassFileConstants.AccPrivate;
decl.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
decl.returnType = TypeReference.baseTypeReference(TypeIds.T_void, 0);
decl.statements = statements.toArray(new Statement[0]);
decl.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return decl;
}
static Receiver generateBuildReceiver(BuilderJob job) {
if (!job.checkerFramework.generateCalledMethods()) return null;
List<char[]> mandatories = new ArrayList<char[]>();
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData == null && bfd.nameOfSetFlag == null) mandatories.add(bfd.name);
}
if (mandatories.size() == 0) return null;
int pS = job.source.sourceStart, pE = job.source.sourceEnd;
char[][] nameCalled = fromQualifiedName(CheckerFrameworkVersion.NAME__CALLED);
SingleMemberAnnotation ann = new SingleMemberAnnotation(new
QualifiedTypeReference(nameCalled, poss(job.source, nameCalled.length)), pS);
if (mandatories.size() == 1) {
ann.memberValue = new StringLiteral(mandatories.get(0), 0, 0, 0);
} else {
ArrayInitializer arr = new ArrayInitializer();
arr.sourceStart = pS;
arr.sourceEnd = pE;
arr.expressions = new Expression[mandatories.size()];
for (int i = 0; i < arr.expressions.length; i++) {
arr.expressions[i] = new StringLiteral(mandatories.get(i), pS, pE, 0);
}
ann.memberValue = arr;
}
TypeReference typeReference = job.createBuilderTypeReference();
```

```
int len = typeReference.getTypeName().length;
typeReference.annotations = new Annotation[len][];
typeReference.annotations[len - 1] = new Annotation[] {ann};
return new Receiver(new char[] { 't', 'h', 'i', 's' }, 0, typeReference, null, 0);
}
public MethodDeclaration generateBuildMethod(BuilderJob job, char[] staticName, TypeReference
returnType, TypeReference[] thrownExceptions, boolean addCleaning) {
MethodDeclaration out = job.createNewMethodDeclaration();
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
List<Statement> statements = new ArrayList<Statement>();
if (addCleaning) {
FieldReference thisUnclean = new FieldReference(CLEAN_FIELD_NAME, 0);
thisUnclean.receiver = new ThisReference(0, 0);
Expression notClean = new UnaryExpression(thisUnclean, OperatorIds.NOT);
MessageSend invokeClean = new MessageSend();
invokeClean.selector = CLEAN_METHOD_NAME;
statements.add(new IfStatement(notClean, invokeClean, 0, 0));
}
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.singularData.getSingularizer().appendBuildCode(bfd.singularData, job.builderType, statements,
bfd.builderFieldName, "this");
}
}
List<Expression> args = new ArrayList<Expression>();
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.nameOfSetFlag != null) {
LocalDeclaration ld = new LocalDeclaration(bfd.builderFieldName, 0, 0);
ld.type = copyType(bfd.type);
FieldReference builderAssign = new FieldReference(bfd.builderFieldName, 0);
builderAssign.receiver = new ThisReference(0, 0);
ld.initialization = builderAssign;
statements.add(ld);
MessageSend inv = new MessageSend();
inv.sourceStart = job.source.sourceStart;
inv.sourceEnd = job.source.sourceEnd;
inv.receiver = new SingleNameReference(((TypeDeclaration) job.parentType.get()).name, 0L);
inv.selector = bfd.nameOfDefaultProvider;
inv.typeArguments = typeParameterNames(((TypeDeclaration) job.builderType.get()).typeParameters);
Assignment defaultAssign = new Assignment(new SingleNameReference(bfd.builderFieldName, 0L),
inv, 0);
FieldReference thisSet = new FieldReference(bfd.nameOfSetFlag, 0L);
thisSet.receiver = new ThisReference(0, 0);
Expression thisNotSet = new UnaryExpression(thisSet, OperatorIds.NOT);
statements.add(new IfStatement(thisNotSet, defaultAssign, 0, 0));
}
if (bfd.nameOfSetFlag != null || (bfd.singularData != null &&
bfd.singularData.getSingularizer().shadowedDuringBuild())) {
args.add(new SingleNameReference(bfd.builderFieldName, 0L));
} else {
FieldReference fr = new FieldReference(bfd.builderFieldName, 0L);
fr.receiver = new ThisReference(0, 0);
args.add(fr);
```

```
}
}
if (addCleaning) {
FieldReference thisUnclean = new FieldReference(CLEAN_FIELD_NAME, 0);
thisUnclean.receiver = new ThisReference(0, 0);
statements.add(new Assignment(thisUnclean, new TrueLiteral(0, 0), 0));
}
out.modifiers = toEclipseModifier(job.accessInners);
out.selector = job.buildMethodName.toCharArray();
out.thrownExceptions = copyTypes(thrownExceptions);
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.returnType = returnType;
if (staticName == null) {
AllocationExpression allocationStatement = new AllocationExpression();
allocationStatement.type = copyType(out.returnType);
allocationStatement.arguments = args.isEmpty() ? null : args.toArray(new Expression[0]);
statements.add(new ReturnStatement(allocationStatement, 0, 0));
} else {
MessageSend invoke = new MessageSend();
invoke.selector = staticName;
if (job.isStatic) {
invoke.receiver = new SingleNameReference(job.builderType.up().getName().toCharArray(), 0);
} else {
invoke.receiver = new QualifiedThisReference(generateTypeReference(job.builderType.up(), 0) , 0, 0);
}
invoke.typeArguments = typeParameterNames(((TypeDeclaration)
job.builderType.get()).typeParameters);
invoke.arguments = args.isEmpty() ? null : args.toArray(new Expression[0]);
if (returnType instanceof SingleTypeReference && Arrays.equals(TypeConstants.VOID,
((SingleTypeReference) returnType).token)) {
statements.add(invoke);
} else {
statements.add(new ReturnStatement(invoke, 0, 0));
}
}
out.statements = statements.isEmpty() ? null : statements.toArray(new Statement[0]);
if (job.checkerFramework.generateSideEffectFree()) {
out.annotations = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE)};
}
out.receiver = generateBuildReceiver(job);
if (staticName == null) createRelevantNonNullAnnotation(job.builderType, out);
out.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return out;
}
private TypeReference[] typeParameterNames(TypeParameter[] typeParameters) {
if (typeParameters == null) return null;
TypeReference[] trs = new TypeReference[typeParameters.length];
for (int i = 0; i < trs.length; i++) {
trs[i] = new SingleTypeReference(typeParameters[i].name, 0);
}
return trs;
}
```

```
public static MethodDeclaration generateDefaultProvider(char[] methodName, TypeParameter[]
typeParameters, EclipseNode fieldNode, ASTNode source) {
int pS = source.sourceStart, pE = source.sourceEnd;
MethodDeclaration out = new MethodDeclaration(((CompilationUnitDeclaration)
fieldNode.top().get()).compilationResult);
out.typeParameters = copyTypeParams(typeParameters, source);
out.selector = methodName;
out.modifiers = ClassFileConstants.AccPrivate | ClassFileConstants.AccStatic;
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
FieldDeclaration fd = (FieldDeclaration) fieldNode.get();
out.returnType = copyType(fd.type, source);
// Convert short array initializers from `{1,2}` to `new int[]{1,2}`
Expression initialization;
if (fd.initialization instanceof ArrayInitializer) {
ArrayAllocationExpression arrayAllocationExpression = new ArrayAllocationExpression();
arrayAllocationExpression.initializer = (ArrayInitializer) fd.initialization;
arrayAllocationExpression.type = generateQualifiedTypeRef(fd, fd.type.getTypeName());
arrayAllocationExpression.dimensions = new Expression[fd.type.dimensions()];
initialization = arrayAllocationExpression;
} else {
initialization = fd.initialization;
}
out.statements = new Statement[] {new ReturnStatement(initialization, pS, pE)};
fd.initialization = null;
out.traverse(new SetGeneratedByVisitor(source), ((TypeDeclaration) fieldNode.up().get()).scope);
return out;
}
public MethodDeclaration generateBuilderMethod(BuilderJob job) {
int pS = job.source.sourceStart, pE = job.source.sourceEnd;
long p = job.getPos();
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = job.builderMethodName.toCharArray();
out.modifiers = toEclipseModifier(job.accessOuters);
if (job.isStatic) out.modifiers |= ClassFileConstants.AccStatic;
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.returnType = job.createBuilderTypeReference();
if (job.checkerFramework.generateUnique()) {
int len = out.returnType.getTypeName().length;
out.returnType.annotations = new Annotation[len][];
out.returnType.annotations[len - 1] = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__UNIQUE)};
}
out.typeParameters = job.copyTypeParams();
AllocationExpression invoke = new AllocationExpression();
if (job.isStatic) {
invoke.type = job.createBuilderTypeReferenceForceStatic();
out.statements = new Statement[] {new ReturnStatement(invoke, pS, pE)};
} else {
// return this.new Builder();
QualifiedAllocationExpression qualifiedInvoke = new QualifiedAllocationExpression();
qualifiedInvoke.enclosingInstance = new ThisReference(pS, pE);
if (job.typeParams == null || job.typeParams.length == 0) {
qualifiedInvoke.type = new SingleTypeReference(job.builderClassNameArr, p);
```

```
} else {
qualifiedInvoke.type = namePlusTypeParamsToTypeReference(null, job.builderClassNameArr, false,
job.typeParams, p);
}
out.statements = new Statement[] {new ReturnStatement(qualifiedInvoke, pS, pE)};
}
if (job.checkerFramework.generateSideEffectFree()) {
out.annotations = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE)};
}
createRelevantNonNullAnnotation(job.builderType, out);
out.traverse(new SetGeneratedByVisitor(job.source), ((TypeDeclaration) job.builderType.get()).scope);
return out;
}
public void generateBuilderFields(BuilderJob job) {
List<EclipseNode> existing = new ArrayList<EclipseNode>();
for (EclipseNode child : job.builderType.down()) {
if (child.getKind() == Kind.FIELD) existing.add(child);
}
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.createdFields.addAll(bfd.singularData.getSingularizer().generateFields(bfd.singularData,
job.builderType));
} else {
EclipseNode field = null, setFlag = null;
for (EclipseNode exists : existing) {
char[] n = ((FieldDeclaration) exists.get()).name;
if (Arrays.equals(n, bfd.builderFieldName)) field = exists;
if (bfd.nameOfSetFlag != null && Arrays.equals(n, bfd.nameOfSetFlag)) setFlag = exists;
}
if (field == null) {
FieldDeclaration fd = new FieldDeclaration(bfd.builderFieldName.clone(), 0, 0);
fd.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
fd.modifiers = ClassFileConstants.AccPrivate;
fd.type = copyType(bfd.type);
fd.traverse(new SetGeneratedByVisitor(job.source), (MethodScope) null);
field = injectFieldAndMarkGenerated(job.builderType, fd);
}
if (setFlag == null && bfd.nameOfSetFlag != null) {
FieldDeclaration fd = new FieldDeclaration(bfd.nameOfSetFlag, 0, 0);
fd.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
fd.modifiers = ClassFileConstants.AccPrivate;
fd.type = TypeReference.baseTypeReference(TypeIds.T_boolean, 0);
fd.traverse(new SetGeneratedByVisitor(job.source), (MethodScope) null);
injectFieldAndMarkGenerated(job.builderType, fd);
}
bfd.createdFields.add(field);
}
}
}
public void makePrefixedSetterMethodsForBuilder(BuilderJob job, BuilderFieldData bfd, String prefix) {
boolean deprecate = isFieldDeprecated(bfd.originalFieldNode);
if (bfd.singularData == null || bfd.singularData.getSingularizer() == null) {
```

```
makePrefixedSetterMethodForBuilder(job, bfd, deprecate, prefix);
} else {
bfd.singularData.getSingularizer().generateMethods(job, bfd.singularData, deprecate);
}
}
private void makePrefixedSetterMethodForBuilder(BuilderJob job, BuilderFieldData bfd, boolean
deprecate, String prefix) {
TypeDeclaration td = (TypeDeclaration) job.builderType.get();
EclipseNode fieldNode = bfd.createdFields.get(0);
AbstractMethodDeclaration[] existing = td.methods;
if (existing == null) existing = EMPTY_METHODS;
int len = existing.length;
String setterPrefix = prefix.isEmpty() ? "set" : prefix;
String setterName;
if (job.oldFluent) {
setterName = prefix.isEmpty() ? new String(bfd.name) :
HandlerUtil.buildAccessorName(job.sourceNode, setterPrefix, new String(bfd.name));
} else {
setterName = HandlerUtil.buildAccessorName(job.sourceNode, setterPrefix, new String(bfd.name));
}
for (int i = 0; i < len; i++) {
if (!(existing[i] instanceof MethodDeclaration)) continue;
char[] existingName = existing[i].selector;
if (Arrays.equals(setterName.toCharArray(), existingName) && !isTolerate(fieldNode, existing[i])) return;
}
List<Annotation> methodAnnsList = Collections.<Annotation>emptyList();
Annotation[] methodAnns =
EclipseHandlerUtil.findCopyableToSetterAnnotations(bfd.originalFieldNode);
if (methodAnns != null && methodAnns.length > 0) methodAnnsList = Arrays.asList(methodAnns);
ASTNode source = job.sourceNode.get();
MethodDeclaration setter = HandleSetter.createSetter(td, deprecate, fieldNode, setterName, bfd.name,
bfd.nameOfSetFlag, job.oldChain, toEclipseModifier(job.accessInners),
job.sourceNode, methodAnnsList, bfd.annotations != null ? Arrays.asList(copyAnnotations(source,
bfd.annotations)) : Collections.<Annotation>emptyList());
if (job.sourceNode.up().getKind() == Kind.METHOD) {
copyJavadocFromParam(bfd.originalFieldNode.up(), setter, td, new String(bfd.name));
} else {
copyJavadoc(bfd.originalFieldNode, setter, td, CopyJavadoc.SETTER, true);
}
injectMethod(job.builderType, setter);
}
public void makeBuilderClass(BuilderJob job) {
TypeDeclaration parent = (TypeDeclaration) job.parentType.get();
TypeDeclaration builder = new TypeDeclaration(parent.compilationResult);
builder.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
builder.modifiers |= toEclipseModifier(job.accessOuters);
if (job.isStatic) builder.modifiers |= ClassFileConstants.AccStatic;
builder.typeParameters = job.copyTypeParams();
builder.name = job.builderClassNameArr;
builder.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
job.builderType = injectType(job.parentType, builder);
}
private void addObtainVia(BuilderFieldData bfd, EclipseNode node) {
```

```
for (EclipseNode child : node.down()) {
if (!annotationTypeMatches(ObtainVia.class, child)) continue;
AnnotationValues<ObtainVia> ann = createAnnotation(ObtainVia.class, child);
bfd.obtainVia = ann.getInstance();
bfd.obtainViaNode = child;
return;
}
}
/**
* Returns the explicitly requested singular annotation on this node (field
* or parameter), or null if there's no {@code @Singular} annotation on it.
*
* @param node The node (field or method param) to inspect for its name and potential {@code
@Singular} annotation.
* @param setterPrefix Explicitly requested setter prefix.
*/
private SingularData getSingularData(EclipseNode node, ASTNode source, final String setterPrefix) {
for (EclipseNode child : node.down()) {
if (!annotationTypeMatches(Singular.class, child)) continue;
char[] pluralName = node.getKind() == Kind.FIELD ? removePrefixFromField(node) :
((AbstractVariableDeclaration) node.get()).name;
AnnotationValues<Singular> ann = createAnnotation(Singular.class, child);
Singular singularInstance = ann.getInstance();
String explicitSingular = singularInstance.value();
if (explicitSingular.isEmpty()) {
if (Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKeys.SINGULAR_AUTO))) {
node.addError("The singular must be specified explicitly (e.g. @Singular(\"task\")) because auto
singularization is disabled.");
explicitSingular = new String(pluralName);
} else {
explicitSingular = autoSingularize(new String(pluralName));
if (explicitSingular == null) {
node.addError("Can't singularize this name; please specify the singular explicitly (i.e.
@Singular(\"sheep\"))");
explicitSingular = new String(pluralName);
}
}
}
char[] singularName = explicitSingular.toCharArray();
TypeReference type = ((AbstractVariableDeclaration) node.get()).type;
TypeReference[] typeArgs = null;
String typeName;
if (type instanceof ParameterizedSingleTypeReference) {
typeArgs = ((ParameterizedSingleTypeReference) type).typeArguments;
typeName = new String(((ParameterizedSingleTypeReference) type).token);
} else if (type instanceof ParameterizedQualifiedTypeReference) {
TypeReference[][] tr = ((ParameterizedQualifiedTypeReference) type).typeArguments;
if (tr != null) typeArgs = tr[tr.length - 1];
char[][] tokens = ((ParameterizedQualifiedTypeReference) type).tokens;
StringBuilder sb = new StringBuilder();
for (int i = 0; i < tokens.length; i++) {
if (i > 0) sb.append(".");
sb.append(tokens[i]);
```

```
}
typeName = sb.toString();
} else {
typeName = type.toString();
}
String targetFqn = EclipseSingularsRecipes.get().toQualified(typeName);
EclipseSingularizer singularizer = EclipseSingularsRecipes.get().getSingularizer(targetFqn);
if (singularizer == null) {
node.addError("Lombok does not know how to create the singular-form builder methods for type '" +
typeName + "'; they won't be generated.");
return null;
}
return new SingularData(child, singularName, pluralName, typeArgs == null ?
Collections.<TypeReference>emptyList() : Arrays.asList(typeArgs), targetFqn, singularizer, source,
singularInstance.ignoreNullCollections(), setterPrefix.toCharArray());
}
return null;
}
}
```

lombok/eclipse/handlers/HandleBuilderDefault.java

```
/*
* Copyright (C) 2017-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import lombok.Builder;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.EclipseAnnotationHandler;
```

```
import lombok.eclipse.EclipseNode;
import lombok.experimental.SuperBuilder;
import lombok.spi.Provides;
@Provides
@HandlerPriority(-1025) //HandleBuilder's level, minus one.
public class HandleBuilderDefault extends EclipseAnnotationHandler<Builder.Default> {
@Override public void handle(AnnotationValues<Builder.Default> annotation, Annotation ast,
EclipseNode annotationNode) {
EclipseNode annotatedField = annotationNode.up();
if (annotatedField.getKind() != Kind.FIELD) return;
EclipseNode classWithAnnotatedField = annotatedField.up();
if (!hasAnnotation(Builder.class, classWithAnnotatedField) &&
!hasAnnotation("lombok.experimental.Builder", classWithAnnotatedField)
&& !hasAnnotation(SuperBuilder.class, classWithAnnotatedField)) {
annotationNode.addWarning("@Builder.Default requires @Builder or @SuperBuilder on the class for it
to mean anything.");
}
}
}
```

lombok/eclipse/handlers/HandleCleanup.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.Arrays;
import lombok.Cleanup;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.AST.Kind;
```

```
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.CaseStatement;
import org.eclipse.jdt.internal.compiler.ast.CastExpression;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MemberValuePair;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SwitchStatement;
import org.eclipse.jdt.internal.compiler.ast.TryStatement;
/**
* Handles the {@code lombok.Cleanup} annotation for eclipse.
*/
@Provides
public class HandleCleanup extends EclipseAnnotationHandler<Cleanup> {
public void handle(AnnotationValues<Cleanup> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.CLEANUP_FLAG_USAGE, "@Cleanup");
String cleanupName = annotation.getInstance().value();
if (cleanupName.length() == 0) {
annotationNode.addError("cleanupName cannot be the empty string.");
return;
}
if (annotationNode.up().getKind() != Kind.LOCAL) {
annotationNode.addError("@Cleanup is legal only on local variable declarations.");
return;
}
LocalDeclaration decl = (LocalDeclaration)annotationNode.up().get();
if (decl.initialization == null) {
annotationNode.addError("@Cleanup variable declarations need to be initialized.");
return;
}
EclipseNode ancestor = annotationNode.up().directUp();
ASTNode blockNode = ancestor.get();
final boolean isSwitch;
final Statement[] statements;
if (blockNode instanceof AbstractMethodDeclaration) {
isSwitch = false;
statements = ((AbstractMethodDeclaration)blockNode).statements;
} else if (blockNode instanceof Block) {
isSwitch = false;
```

```
statements = ((Block)blockNode).statements;
} else if (blockNode instanceof SwitchStatement) {
isSwitch = true;
statements = ((SwitchStatement)blockNode).statements;
} else {
annotationNode.addError("@Cleanup is legal only on a local variable declaration inside a block.");
return;
}
if (statements == null) {
annotationNode.addError("LOMBOK BUG: Parent block does not contain any statements.");
return;
}
int start = 0;
for (; start < statements.length ; start++) {
if (statements[start] == decl) break;
}
if (start == statements.length) {
annotationNode.addError("LOMBOK BUG: Can't find this local variable declaration inside its parent.");
return;
}
start++; //We start with try{} *AFTER* the var declaration.
int end;
if (isSwitch) {
end = start + 1;
for (; end < statements.length ; end++) {
if (statements[end] instanceof CaseStatement) {
break;
}
}
} else end = statements.length;
//At this point:
// start-1 = Local Declaration marked with @Cleanup
// start = first instruction that needs to be wrapped into a try block
// end = last instruction of the scope -OR- last instruction before the next case label in switch
statements.
// hence:
// [start, end) = statements for the try block.
Statement[] tryBlock = new Statement[end - start];
System.arraycopy(statements, start, tryBlock, 0, end-start);
//Remove the stuff we just dumped into the tryBlock, and then leave room for the try node.
int newStatementsLength = statements.length - (end-start); //Remove room for every statement moved
into try block...
newStatementsLength += 1; //But add room for the TryStatement node itself.
Statement[] newStatements = new Statement[newStatementsLength];
System.arraycopy(statements, 0, newStatements, 0, start); //copy all statements before the try block
verbatim.
System.arraycopy(statements, end, newStatements, start+1, statements.length - end); //For switch
statements.
doAssignmentCheck(annotationNode, tryBlock, decl.name);
TryStatement tryStatement = new TryStatement();
setGeneratedBy(tryStatement, ast);
tryStatement.tryBlock = new Block(0);
tryStatement.tryBlock.statements = tryBlock;
```

```
setGeneratedBy(tryStatement.tryBlock, ast);
// Positions for in-method generated nodes are special
int ss = decl.declarationSourceEnd + 1;
int se = ss;
if (tryBlock.length > 0) {
se = tryBlock[tryBlock.length - 1].sourceEnd + 1; //+1 for the closing semicolon. Yes, there could be
spaces. Bummer.
tryStatement.sourceStart = ss;
tryStatement.sourceEnd = se;
tryStatement.tryBlock.sourceStart = ss;
tryStatement.tryBlock.sourceEnd = se;
}
newStatements[start] = tryStatement;
Statement[] finallyBlock = new Statement[1];
MessageSend unsafeClose = new MessageSend();
setGeneratedBy(unsafeClose, ast);
unsafeClose.sourceStart = ast.sourceStart;
unsafeClose.sourceEnd = ast.sourceEnd;
SingleNameReference receiver = new SingleNameReference(decl.name, 0);
setGeneratedBy(receiver, ast);
unsafeClose.receiver = receiver;
long nameSourcePosition = (long)ast.sourceStart << 32 | ast.sourceEnd;
if (ast.memberValuePairs() != null) for (MemberValuePair pair : ast.memberValuePairs()) {
if (pair.name != null && new String(pair.name).equals("value")) {
nameSourcePosition = (long)pair.value.sourceStart << 32 | pair.value.sourceEnd;
break;
}
}
unsafeClose.nameSourcePosition = nameSourcePosition;
unsafeClose.selector = cleanupName.toCharArray();
int pS = ast.sourceStart, pE = ast.sourceEnd;
long p = (long)pS << 32 | pE;
SingleNameReference varName = new SingleNameReference(decl.name, p);
setGeneratedBy(varName, ast);
NullLiteral nullLiteral = new NullLiteral(pS, pE);
setGeneratedBy(nullLiteral, ast);
MessageSend preventNullAnalysis = preventNullAnalysis(ast, varName);
EqualExpression equalExpression = new EqualExpression(preventNullAnalysis, nullLiteral,
OperatorIds.NOT_EQUAL);
equalExpression.sourceStart = pS; equalExpression.sourceEnd = pE;
setGeneratedBy(equalExpression, ast);
Block closeBlock = new Block(0);
closeBlock.statements = new Statement[1];
closeBlock.statements[0] = unsafeClose;
setGeneratedBy(closeBlock, ast);
IfStatement ifStatement = new IfStatement(equalExpression, closeBlock, 0, 0);
setGeneratedBy(ifStatement, ast);
finallyBlock[0] = ifStatement;
tryStatement.finallyBlock = new Block(0);
// Positions for in-method generated nodes are special
if (!isSwitch) {
tryStatement.finallyBlock.sourceStart = blockNode.sourceEnd;
tryStatement.finallyBlock.sourceEnd = blockNode.sourceEnd;
```

```
}
setGeneratedBy(tryStatement.finallyBlock, ast);
tryStatement.finallyBlock.statements = finallyBlock;
tryStatement.catchArguments = null;
tryStatement.catchBlocks = null;
if (blockNode instanceof AbstractMethodDeclaration) {
((AbstractMethodDeclaration)blockNode).statements = newStatements;
} else if (blockNode instanceof Block) {
((Block)blockNode).statements = newStatements;
} else if (blockNode instanceof SwitchStatement) {
((SwitchStatement)blockNode).statements = newStatements;
}
ancestor.rebuild();
}
public MessageSend preventNullAnalysis(Annotation ast, Expression expr) {
MessageSend singletonList = new MessageSend();
setGeneratedBy(singletonList, ast);
int pS = ast.sourceStart, pE = ast.sourceEnd;
long p = (long)pS << 32 | pE;
singletonList.receiver = createNameReference("java.util.Collections", ast);
singletonList.selector = "singletonList".toCharArray();
singletonList.arguments = new Expression[] { expr };
singletonList.nameSourcePosition = p;
singletonList.sourceStart = pS;
singletonList.sourceEnd = singletonList.statementEnd = pE;
MessageSend preventNullAnalysis = new MessageSend();
setGeneratedBy(preventNullAnalysis, ast);
preventNullAnalysis.receiver = singletonList;
preventNullAnalysis.selector = "get".toCharArray();
preventNullAnalysis.arguments = new Expression[] { makeIntLiteral("0".toCharArray(), ast) };
preventNullAnalysis.nameSourcePosition = p;
preventNullAnalysis.sourceStart = pS;
preventNullAnalysis.sourceEnd = singletonList.statementEnd = pE;
return preventNullAnalysis;
}
public void doAssignmentCheck(EclipseNode node, Statement[] tryBlock, char[] varName) {
for (Statement statement : tryBlock) doAssignmentCheck0(node, statement, varName);
}
private void doAssignmentCheck0(EclipseNode node, Statement statement, char[] varName) {
if (statement instanceof Assignment)
doAssignmentCheck0(node, ((Assignment)statement).expression, varName);
else if (statement instanceof LocalDeclaration)
doAssignmentCheck0(node, ((LocalDeclaration)statement).initialization, varName);
else if (statement instanceof CastExpression)
doAssignmentCheck0(node, ((CastExpression)statement).expression, varName);
else if (statement instanceof SingleNameReference) {
if (Arrays.equals(((SingleNameReference)statement).token, varName)) {
EclipseNode problemNode = node.getNodeFor(statement);
if (problemNode != null) problemNode.addWarning(
"You're assigning an auto-cleanup variable to something else. This is a bad idea.");
}
}
}
```

}

lombok/eclipse/handlers/HandleConstructor.java

```
/*
* Copyright (C) 2010-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.ConfigurationKeys;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;
import lombok.core.AST.Kind;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.AnnotationValues;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
```

```
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.ArrayTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.CharLiteral;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.DoubleLiteral;
import org.eclipse.jdt.internal.compiler.ast.ExplicitConstructorCall;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FalseLiteral;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.FloatLiteral;
import org.eclipse.jdt.internal.compiler.ast.IntLiteral;
import org.eclipse.jdt.internal.compiler.ast.LongLiteral;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleMemberAnnotation;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
public class HandleConstructor {
@Provides
public static class HandleNoArgsConstructor extends EclipseAnnotationHandler<NoArgsConstructor> {
private static final String NAME = NoArgsConstructor.class.getSimpleName();
private HandleConstructor handleConstructor = new HandleConstructor();
@Override public void handle(AnnotationValues<NoArgsConstructor> annotation, Annotation ast,
EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.NO_ARGS_CONSTRUCTOR_FLAG_USAGE,
"@NoArgsConstructor", ConfigurationKeys.ANY_CONSTRUCTOR_FLAG_USAGE, "any
@xArgsConstructor");
EclipseNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode, NAME)) return;
NoArgsConstructor ann = annotation.getInstance();
AccessLevel level = ann.access();
String staticName = ann.staticName();
if (level == AccessLevel.NONE) return;
boolean force = ann.force();
List<Annotation> onConstructor = unboxAndRemoveAnnotationParameter(ast, "onConstructor",
"@NoArgsConstructor(onConstructor", annotationNode);
if (!onConstructor.isEmpty()) {
```

```
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@NoArgsConstructor(onConstructor=...)");
}
handleConstructor.generateConstructor(typeNode, level, Collections.<EclipseNode>emptyList(), force,
staticName, SkipIfConstructorExists.NO, onConstructor, annotationNode);
}
}
@Provides
public static class HandleRequiredArgsConstructor extends
EclipseAnnotationHandler<RequiredArgsConstructor> {
private static final String NAME = RequiredArgsConstructor.class.getSimpleName();
private HandleConstructor handleConstructor = new HandleConstructor();
@Override public void handle(AnnotationValues<RequiredArgsConstructor> annotation, Annotation
ast, EclipseNode annotationNode) {
handleFlagUsage(annotationNode,
ConfigurationKeys.REQUIRED_ARGS_CONSTRUCTOR_FLAG_USAGE,
"@RequiredArgsConstructor", ConfigurationKeys.ANY_CONSTRUCTOR_FLAG_USAGE, "any
@xArgsConstructor");
EclipseNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode, NAME)) return;
RequiredArgsConstructor ann = annotation.getInstance();
AccessLevel level = ann.access();
if (level == AccessLevel.NONE) return;
String staticName = ann.staticName();
if (annotation.isExplicit("suppressConstructorProperties")) {
annotationNode.addError("This deprecated feature is no longer supported. Remove it; you can create a
lombok.config file with 'lombok.anyConstructor.suppressConstructorProperties = true'.");
}
List<Annotation> onConstructor = unboxAndRemoveAnnotationParameter(ast, "onConstructor",
"@RequiredArgsConstructor(onConstructor", annotationNode);
if (!onConstructor.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@RequiredArgsConstructor(onConstructor=...)");
}
handleConstructor.generateConstructor(
typeNode, level, findRequiredFields(typeNode), false, staticName, SkipIfConstructorExists.NO,
onConstructor, annotationNode);
}
}
@Provides
public static class HandleAllArgsConstructor extends EclipseAnnotationHandler<AllArgsConstructor> {
private static final String NAME = AllArgsConstructor.class.getSimpleName();
private HandleConstructor handleConstructor = new HandleConstructor();
@Override public void handle(AnnotationValues<AllArgsConstructor> annotation, Annotation ast,
EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.ALL_ARGS_CONSTRUCTOR_FLAG_USAGE,
"@AllArgsConstructor", ConfigurationKeys.ANY_CONSTRUCTOR_FLAG_USAGE, "any
@xArgsConstructor");
EclipseNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode, NAME)) return;
AllArgsConstructor ann = annotation.getInstance();
AccessLevel level = ann.access();
if (level == AccessLevel.NONE) return;
```

```
String staticName = ann.staticName();
if (annotation.isExplicit("suppressConstructorProperties")) {
annotationNode.addError("This deprecated feature is no longer supported. Remove it; you can create a
lombok.config file with 'lombok.anyConstructor.suppressConstructorProperties = true'.");
}
List<Annotation> onConstructor = unboxAndRemoveAnnotationParameter(ast, "onConstructor",
"@AllArgsConstructor(onConstructor", annotationNode);
if (!onConstructor.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@AllArgsConstructor(onConstructor=...)");
}
handleConstructor.generateConstructor(
typeNode, level, findAllFields(typeNode), false, staticName, SkipIfConstructorExists.NO,
onConstructor, annotationNode);
}
}
private static List<EclipseNode> findRequiredFields(EclipseNode typeNode) {
return findFields(typeNode, true);
}
private static List<EclipseNode> findFields(EclipseNode typeNode, boolean nullMarked) {
List<EclipseNode> fields = new ArrayList<EclipseNode>();
for (EclipseNode child : typeNode.down()) {
if (child.getKind() != Kind.FIELD) continue;
FieldDeclaration fieldDecl = (FieldDeclaration) child.get();
if (!filterField(fieldDecl)) continue;
boolean isFinal = (fieldDecl.modifiers & ClassFileConstants.AccFinal) != 0;
boolean isNonNull = nullMarked && hasNonNullAnnotations(child);
if ((isFinal || isNonNull) && fieldDecl.initialization == null) fields.add(child);
}
return fields;
}
static List<EclipseNode> findAllFields(EclipseNode typeNode) {
return findAllFields(typeNode, false);
}
static List<EclipseNode> findAllFields(EclipseNode typeNode, boolean evenFinalInitialized) {
List<EclipseNode> fields = new ArrayList<EclipseNode>();
for (EclipseNode child : typeNode.down()) {
if (child.getKind() != Kind.FIELD) continue;
FieldDeclaration fieldDecl = (FieldDeclaration) child.get();
if (!filterField(fieldDecl)) continue;
if (!evenFinalInitialized && ((fieldDecl.modifiers & ClassFileConstants.AccFinal) != 0) &&
fieldDecl.initialization != null) continue;
fields.add(child);
}
return fields;
}
static boolean checkLegality(EclipseNode typeNode, EclipseNode errorNode, String name) {
if (!isClassOrEnum(typeNode)) {
errorNode.addError(name + " is only supported on a class or an enum.");
return false;
}
return true;
}
```

```
public enum SkipIfConstructorExists {
YES, NO, I_AM_BUILDER;
}
public void generateExtraNoArgsConstructor(EclipseNode typeNode, EclipseNode sourceNode) {
if (!isDirectDescendantOfObject(typeNode)) return;
Boolean v = typeNode.getAst().readConfiguration(ConfigurationKeys.NO_ARGS_CONSTRUCTOR_E
XTRA_PRIVATE);
if (v == null || !v) return;
generate(typeNode, AccessLevel.PRIVATE, Collections.<EclipseNode>emptyList(), true, null,
SkipIfConstructorExists.NO, Collections.<Annotation>emptyList(), sourceNode, true);
}
public void generateRequiredArgsConstructor(
EclipseNode typeNode, AccessLevel level, String staticName, SkipIfConstructorExists
skipIfConstructorExists,
List<Annotation> onConstructor, EclipseNode sourceNode) {
generateConstructor(typeNode, level, findRequiredFields(typeNode), false, staticName,
skipIfConstructorExists, onConstructor, sourceNode);
}
public void generateAllArgsConstructor(
EclipseNode typeNode, AccessLevel level, String staticName, SkipIfConstructorExists
skipIfConstructorExists,
List<Annotation> onConstructor, EclipseNode sourceNode) {
generateConstructor(typeNode, level, findAllFields(typeNode), false, staticName,
skipIfConstructorExists, onConstructor, sourceNode);
}
public void generateConstructor(
EclipseNode typeNode, AccessLevel level, List<EclipseNode> fieldsToParam, boolean forceDefaults,
String staticName, SkipIfConstructorExists skipIfConstructorExists,
List<Annotation> onConstructor, EclipseNode sourceNode) {
generate(typeNode, level, fieldsToParam, forceDefaults, staticName, skipIfConstructorExists,
onConstructor, sourceNode, false);
}
public void generate(
EclipseNode typeNode, AccessLevel level, List<EclipseNode> fieldsToParam, boolean forceDefaults,
String staticName, SkipIfConstructorExists skipIfConstructorExists,
List<Annotation> onConstructor, EclipseNode sourceNode, boolean noArgs) {
ASTNode source = sourceNode.get();
boolean staticConstrRequired = staticName != null && !staticName.equals("");
if (skipIfConstructorExists != SkipIfConstructorExists.NO) {
for (EclipseNode child : typeNode.down()) {
if (child.getKind() == Kind.ANNOTATION) {
boolean skipGeneration = (annotationTypeMatches(NoArgsConstructor.class, child) ||
annotationTypeMatches(AllArgsConstructor.class, child) ||
annotationTypeMatches(RequiredArgsConstructor.class, child));
if (!skipGeneration && skipIfConstructorExists == SkipIfConstructorExists.YES) {
skipGeneration = annotationTypeMatches(Builder.class, child);
}
if (skipGeneration) {
if (staticConstrRequired) {
// @Data has asked us to generate a constructor, but we're going to skip this instruction, as an explicit
'make a constructor' annotation
// will take care of it. However, @Data also wants a specific static name; this will be ignored; the
appropriate way to do this is to use
```

```
// the 'staticName' parameter of the @XArgsConstructor you've stuck on your type.
// We should warn that we're ignoring @Data's 'staticConstructor' param.
typeNode.addWarning(
"Ignoring static constructor name: explicit @XxxArgsConstructor annotation present; its `staticName`
parameter will be used.",
source.sourceStart, source.sourceEnd);
}
return;
}
}
}
}
if (noArgs && noArgsConstructorExists(typeNode)) return;
if (!(skipIfConstructorExists != SkipIfConstructorExists.NO && constructorExists(typeNode) !=
MemberExistsResult.NOT_EXISTS)) {
ConstructorDeclaration constr = createConstructor(
staticConstrRequired ? AccessLevel.PRIVATE : level, typeNode, fieldsToParam, forceDefaults,
sourceNode, onConstructor);
EclipseNode constructorNode = injectMethod(typeNode, constr);
generateConstructorJavadoc(typeNode, constructorNode, fieldsToParam);
}
generateStaticConstructor(staticConstrRequired, typeNode, staticName, level, fieldsToParam, source);
}
private void generateStaticConstructor(boolean staticConstrRequired, EclipseNode typeNode, String
staticName, AccessLevel level, Collection<EclipseNode> fields, ASTNode source) {
if (staticConstrRequired) {
MethodDeclaration staticConstr = createStaticConstructor(level, staticName, typeNode, fields, source);
EclipseNode constructorNode = injectMethod(typeNode, staticConstr);
generateConstructorJavadoc(typeNode, constructorNode, fields);
}
}
private static boolean noArgsConstructorExists(EclipseNode node) {
node = EclipseHandlerUtil.upToTypeNode(node);
if (node != null && node.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration)node.get();
if (typeDecl.methods != null) for (AbstractMethodDeclaration def : typeDecl.methods) {
if (def instanceof ConstructorDeclaration) {
Argument[] arguments = ((ConstructorDeclaration) def).arguments;
if (arguments == null || arguments.length == 0) return true;
}
}
}
for (EclipseNode child : node.down()) {
if (annotationTypeMatches(NoArgsConstructor.class, child)) return true;
if (annotationTypeMatches(RequiredArgsConstructor.class, child) &&
findRequiredFields(node).isEmpty()) return true;
if (annotationTypeMatches(AllArgsConstructor.class, child) && findAllFields(node).isEmpty()) return
true;
}
return false;
}
private static final char[][] JAVA_BEANS_CONSTRUCTORPROPERTIES = new char[][] {
"java".toCharArray(), "beans".toCharArray(), "ConstructorProperties".toCharArray() };
```

```
public static Annotation[] createConstructorProperties(ASTNode source, Collection<EclipseNode>
fields) {
if (fields.isEmpty()) return null;
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
long[] poss = new long[3];
Arrays.fill(poss, p);
QualifiedTypeReference constructorPropertiesType = new
QualifiedTypeReference(JAVA_BEANS_CONSTRUCTORPROPERTIES, poss);
setGeneratedBy(constructorPropertiesType, source);
SingleMemberAnnotation ann = new SingleMemberAnnotation(constructorPropertiesType, pS);
ann.declarationSourceEnd = pE;
ArrayInitializer fieldNames = new ArrayInitializer();
fieldNames.sourceStart = pS;
fieldNames.sourceEnd = pE;
fieldNames.expressions = new Expression[fields.size()];
int ctr = 0;
for (EclipseNode field : fields) {
char[] fieldName = removePrefixFromField(field);
fieldNames.expressions[ctr] = new StringLiteral(fieldName, pS, pE, 0);
setGeneratedBy(fieldNames.expressions[ctr], source);
ctr++;
}
ann.memberValue = fieldNames;
setGeneratedBy(ann, source);
setGeneratedBy(ann.memberValue, source);
return new Annotation[] { ann };
}
private static final char[] DEFAULT_PREFIX = {'$', 'd', 'e', 'f', 'a', 'u', 'l', 't', '$'};
private static final char[] prefixWith(char[] prefix, char[] name) {
char[] out = new char[prefix.length + name.length];
System.arraycopy(prefix, 0, out, 0, prefix.length);
System.arraycopy(name, 0, out, prefix.length, name.length);
return out;
}
@SuppressWarnings("deprecation") public static ConstructorDeclaration createConstructor(
AccessLevel level, EclipseNode type, Collection<EclipseNode> fieldsToParam, boolean forceDefaults,
EclipseNode sourceNode, List<Annotation> onConstructor) {
ASTNode source = sourceNode.get();
TypeDeclaration typeDeclaration = ((TypeDeclaration) type.get());
long p = (long) source.sourceStart << 32 | source.sourceEnd;
boolean isEnum = (((TypeDeclaration) type.get()).modifiers & ClassFileConstants.AccEnum) != 0;
if (isEnum) level = AccessLevel.PRIVATE;
List<EclipseNode> fieldsToDefault = fieldsNeedingBuilderDefaults(type, fieldsToParam);
List<EclipseNode> fieldsToExplicit = forceDefaults ? fieldsNeedingExplicitDefaults(type,
fieldsToParam) : Collections.<EclipseNode>emptyList();
boolean addConstructorProperties;
if (fieldsToParam.isEmpty()) {
addConstructorProperties = false;
} else {
Boolean v = type.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTOR_ADD_CONST
RUCTOR_PROPERTIES);
addConstructorProperties = v != null ? v.booleanValue() :
```

```
Boolean.FALSE.equals(type.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTOR_SUPPRESS_CONSTRUCTOR_PROPERTIES));
}
ConstructorDeclaration constructor = new ConstructorDeclaration(((CompilationUnitDeclaration) type.top().get()).compilationResult);
constructor.modifiers = toEclipseModifier(level);
constructor.selector = typeDeclaration.name;
constructor.constructorCall = new ExplicitConstructorCall(ExplicitConstructorCall.ImplicitSuper);
constructor.constructorCall.sourceStart = source.sourceStart;
constructor.constructorCall.sourceEnd = source.sourceEnd;
constructor.thrownExceptions = null;
constructor.typeParameters = null;
constructor.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
constructor.bodyStart = constructor.declarationSourceStart = constructor.sourceStart = source.sourceStart;
constructor.bodyEnd = constructor.declarationSourceEnd = constructor.sourceEnd = source.sourceEnd;
constructor.arguments = null;
List<Argument> params = new ArrayList<Argument>();
List<Statement> assigns = new ArrayList<Statement>();
List<Statement> nullChecks = new ArrayList<Statement>();
for (EclipseNode fieldNode : fieldsToParam) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
char[] rawName = field.name;
char[] fieldName = removePrefixFromField(fieldNode);
FieldReference thisX = new FieldReference(rawName, p);
int s = (int) (p >> 32);
int e = (int) p;
thisX.receiver = new ThisReference(s, e);
Expression assignmentExpr = new SingleNameReference(fieldName, p);
Assignment assignment = new Assignment(thisX, assignmentExpr, (int) p);
assignment.sourceStart = (int) (p >> 32); assignment.sourceEnd = assignment.statementEnd = (int) (p >> 32);
assigns.add(assignment);
long fieldPos = (((long) field.sourceStart) << 32) | field.sourceEnd;
Argument parameter = new Argument(fieldName, fieldPos, copyType(field.type, source), Modifier.FINAL);
Annotation[] copyableAnnotations = findCopyableAnnotations(fieldNode);
if (hasNonNullAnnotations(fieldNode)) {
Statement nullCheck = generateNullCheck(parameter, sourceNode, null);
if (nullCheck != null) nullChecks.add(nullCheck);
}
parameter.annotations = copyAnnotations(source, copyableAnnotations);
if (parameter.annotations != null) {
parameter.bits |= Eclipse.HasTypeAnnotations;
constructor.bits |= Eclipse.HasTypeAnnotations;
}
params.add(parameter);
}
for (EclipseNode fieldNode : fieldsToExplicit) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
char[] rawName = field.name;
FieldReference thisX = new FieldReference(rawName, p);
```

```
int s = (int) (p >> 32);
int e = (int) p;
thisX.receiver = new ThisReference(s, e);
Expression assignmentExpr = getDefaultExpr(field.type, s, e);
Assignment assignment = new Assignment(thisX, assignmentExpr, (int) p);
assignment.sourceStart = (int) (p >> 32); assignment.sourceEnd = assignment.statementEnd = (int) (p
>> 32);
assigns.add(assignment);
}
for (EclipseNode fieldNode : fieldsToDefault) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
char[] rawName = field.name;
FieldReference thisX = new FieldReference(rawName, p);
int s = (int) (p >> 32);
int e = (int) p;
thisX.receiver = new ThisReference(s, e);
MessageSend inv = new MessageSend();
inv.sourceStart = source.sourceStart;
inv.sourceEnd = source.sourceEnd;
inv.receiver = new SingleNameReference(((TypeDeclaration) type.get()).name, 0L);
inv.selector = prefixWith(DEFAULT_PREFIX, removePrefixFromField(fieldNode));
Assignment assignment = new Assignment(thisX, inv, (int) p);
assignment.sourceStart = (int) (p >> 32); assignment.sourceEnd = assignment.statementEnd = (int) (p
>> 32);
assigns.add(assignment);
}
nullChecks.addAll(assigns);
constructor.statements = nullChecks.isEmpty() ? null : nullChecks.toArray(new Statement[0]);
constructor.arguments = params.isEmpty() ? null : params.toArray(new Argument[0]);
/* Generate annotations that must be put on the generated method, and attach them. */ {
Annotation[] constructorProperties = null;
if (addConstructorProperties && !isLocalType(type)) constructorProperties =
createConstructorProperties(source, fieldsToParam);
constructor.annotations = copyAnnotations(source,
onConstructor.toArray(new Annotation[0]),
constructorProperties);
}
constructor.traverse(new SetGeneratedByVisitor(source), typeDeclaration.scope);
return constructor;
}
private static List<EclipseNode> fieldsNeedingBuilderDefaults(EclipseNode type,
Collection<EclipseNode> fieldsToParam) {
List<EclipseNode> out = new ArrayList<EclipseNode>();
top:
for (EclipseNode node : type.down()) {
if (node.getKind() != Kind.FIELD) continue top;
FieldDeclaration fd = (FieldDeclaration) node.get();
if ((fd.modifiers & ClassFileConstants.AccStatic) != 0) continue top;
for (EclipseNode ftp : fieldsToParam) if (node == ftp) continue top;
if (EclipseHandlerUtil.hasAnnotation(Builder.Default.class, node)) out.add(node);
}
return out;
}
```

```
private static List<EclipseNode> fieldsNeedingExplicitDefaults(EclipseNode type,
Collection<EclipseNode> fieldsToParam) {
List<EclipseNode> out = new ArrayList<EclipseNode>();
top:
for (EclipseNode node : type.down()) {
if (node.getKind() != Kind.FIELD) continue top;
FieldDeclaration fd = (FieldDeclaration) node.get();
if (fd.initialization != null) continue top;
if ((fd.modifiers & ClassFileConstants.AccFinal) == 0) continue top;
if ((fd.modifiers & ClassFileConstants.AccStatic) != 0) continue top;
for (EclipseNode ftp : fieldsToParam) if (node == ftp) continue top;
if (EclipseHandlerUtil.hasAnnotation(Builder.Default.class, node)) continue top;
out.add(node);
}
return out;
}
private static Expression getDefaultExpr(TypeReference type, int s, int e) {
boolean array = type instanceof ArrayTypeReference;
if (array) return new NullLiteral(s, e);
char[] lastToken = type.getLastToken();
if (Arrays.equals(TypeConstants.BOOLEAN, lastToken)) return new FalseLiteral(s, e);
if (Arrays.equals(TypeConstants.CHAR, lastToken)) return new CharLiteral(new char[] {'\'', '\\', '0', '\'}, s,
e);
if (Arrays.equals(TypeConstants.BYTE, lastToken) || Arrays.equals(TypeConstants.SHORT, lastToken)
||
Arrays.equals(TypeConstants.INT, lastToken)) return IntLiteral.buildIntLiteral(new char[] {'0'}, s, e);
if (Arrays.equals(TypeConstants.LONG, lastToken)) return LongLiteral.buildLongLiteral(new char[] {'0',
'L'}, s, e);
if (Arrays.equals(TypeConstants.FLOAT, lastToken)) return new FloatLiteral(new char[] {'0', 'F'}, s, e);
if (Arrays.equals(TypeConstants.DOUBLE, lastToken)) return new DoubleLiteral(new char[] {'0', 'D'}, s,
e);
return new NullLiteral(s, e);
}
public static boolean isLocalType(EclipseNode type) {
Kind kind = type.up().getKind();
if (kind == Kind.COMPILATION_UNIT) return false;
if (kind == Kind.TYPE) return isLocalType(type.up());
return true;
}
public MethodDeclaration createStaticConstructor(AccessLevel level, String name, EclipseNode type,
Collection<EclipseNode> fields, ASTNode source) {
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration constructor = new MethodDeclaration(((CompilationUnitDeclaration)
type.top().get()).compilationResult);
constructor.modifiers = toEclipseModifier(level) | ClassFileConstants.AccStatic;
TypeDeclaration typeDecl = (TypeDeclaration) type.get();
constructor.returnType = EclipseHandlerUtil.namePlusTypeParamsToTypeReference(type,
typeDecl.typeParameters, p);
constructor.annotations = null;
if (getCheckerFrameworkVersion(type).generateUnique()) {
int len = constructor.returnType.getTypeName().length;
constructor.returnType.annotations = new Annotation[len][];
```

```
constructor.returnType.annotations[len - 1] = new Annotation[] {generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__UNIQUE)};
}
constructor.selector = name.toCharArray();
constructor.thrownExceptions = null;
constructor.typeParameters = copyTypeParams(((TypeDeclaration) type.get()).typeParameters,
source);
constructor.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
constructor.bodyStart = constructor.declarationSourceStart = constructor.sourceStart =
source.sourceStart;
constructor.bodyEnd = constructor.declarationSourceEnd = constructor.sourceEnd =
source.sourceEnd;
List<Argument> params = new ArrayList<Argument>();
List<Expression> assigns = new ArrayList<Expression>();
AllocationExpression statement = new AllocationExpression();
statement.sourceStart = pS; statement.sourceEnd = pE;
statement.type = copyType(constructor.returnType, source);
for (EclipseNode fieldNode : fields) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
long fieldPos = (((long) field.sourceStart) << 32) | field.sourceEnd;
SingleNameReference nameRef = new SingleNameReference(field.name, fieldPos);
assigns.add(nameRef);
Argument parameter = new Argument(field.name, fieldPos, copyType(field.type, source),
Modifier.FINAL);
parameter.annotations = copyAnnotations(source, findCopyableAnnotations(fieldNode));
if (parameter.annotations != null) {
parameter.bits |= Eclipse.HasTypeAnnotations;
constructor.bits |= Eclipse.HasTypeAnnotations;
}
params.add(parameter);
}
statement.arguments = assigns.isEmpty() ? null : assigns.toArray(new Expression[0]);
constructor.arguments = params.isEmpty() ? null : params.toArray(new Argument[0]);
constructor.statements = new Statement[] { new ReturnStatement(statement, (int) (p >> 32), (int)p) };
createRelevantNonNullAnnotation(type, constructor);
constructor.traverse(new SetGeneratedByVisitor(source), typeDecl.scope);
return constructor;
}
private void generateConstructorJavadoc(EclipseNode typeNode, EclipseNode constructorNode,
Collection<EclipseNode> fields) {
if (fields.isEmpty()) return;
String constructorJavadoc = getConstructorJavadocHeader(typeNode.getName());
boolean fieldDescriptionAdded = false;
for (EclipseNode fieldNode : fields) {
String paramName = String.valueOf(removePrefixFromField(fieldNode));
String fieldJavadoc = getDocComment(fieldNode);
String paramJavadoc = getConstructorParameterJavadoc(paramName, fieldJavadoc);
if (paramJavadoc == null) {
paramJavadoc = "@param " + paramName;
} else {
fieldDescriptionAdded = true;
}
constructorJavadoc = addJavadocLine(constructorJavadoc, paramJavadoc);
```

```
}
if (fieldDescriptionAdded) {
setDocComment(typeNode, constructorNode, constructorJavadoc);
}
}
}
```

lombok/eclipse/handlers/HandleData.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.Collections;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.Data;
import lombok.core.AnnotationValues;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.HandleConstructor.SkipIfConstructorExists;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
/**
* Handles the {@code lombok.Data} annotation for eclipse.
*/
@Provides
public class HandleData extends EclipseAnnotationHandler<Data> {
private HandleGetter handleGetter = new HandleGetter();
private HandleSetter handleSetter = new HandleSetter();
```

```
private HandleEqualsAndHashCode handleEqualsAndHashCode = new
HandleEqualsAndHashCode();
private HandleToString handleToString = new HandleToString();
private HandleConstructor handleConstructor = new HandleConstructor();
@Override public void handle(AnnotationValues<Data> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.DATA_FLAG_USAGE, "@Data");
Data ann = annotation.getInstance();
EclipseNode typeNode = annotationNode.up();
if (!isClass(typeNode)) {
annotationNode.addError("@Data is only supported on a class.");
return;
}
//Careful: Generate the public static constructor (if there is one) LAST, so that any attempt to
//'find callers' on the annotation node will find callers of the constructor, which is by far the
//most useful of the many methods built by @Data. This trick won't work for the non-static constructor,
//for whatever reason, though you can find callers of that one by focusing on the class name itself
//and hitting 'find callers'.
handleGetter.generateGetterForType(typeNode, annotationNode, AccessLevel.PUBLIC, true,
Collections.<Annotation>emptyList());
handleSetter.generateSetterForType(typeNode, annotationNode, AccessLevel.PUBLIC, true,
Collections.<Annotation>emptyList(), Collections.<Annotation>emptyList());
handleEqualsAndHashCode.generateEqualsAndHashCodeForType(typeNode, annotationNode);
handleToString.generateToStringForType(typeNode, annotationNode);
handleConstructor.generateRequiredArgsConstructor(
typeNode, AccessLevel.PUBLIC, ann.staticConstructor(), SkipIfConstructorExists.YES,
Collections.<Annotation>emptyList(), annotationNode);
handleConstructor.generateExtraNoArgsConstructor(typeNode, annotationNode);
}
}
```

lombok/eclipse/handlers/HandleDelegate.java


```
/*
* Copyright (C) 2014-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
```

```
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.Delegate;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
/**
* This class just handles basic error cases. The real meat of eclipse '@Delegate' support is in {@code
PatchDelegate}.
*/
@Provides
public class HandleDelegate extends EclipseAnnotationHandler<Delegate> {
public void handle(AnnotationValues<Delegate> annotation, Annotation ast, EclipseNode
annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.DELEGATE_FLAG_USAGE,
"@Delegate");
}
}
```

lombok/eclipse/handlers/HandleEqualsAndHashCode.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
```

```
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.EqualsAndHashCode;
import lombok.EqualsAndHashCode.CacheStrategy;
import lombok.core.AST.Kind;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.InclusionExclusionUtils;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CallSuperType;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.configuration.NullAnnotationLibrary;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.BinaryExpression;
import org.eclipse.jdt.internal.compiler.ast.CastExpression;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FalseLiteral;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.InstanceOfExpression;
import org.eclipse.jdt.internal.compiler.ast.IntLiteral;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MarkerAnnotation;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NameReference;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
```

```
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SuperReference;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TrueLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.UnaryExpression;
import org.eclipse.jdt.internal.compiler.ast.Wildcard;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
/**
* Handles the {@code EqualsAndHashCode} annotation for eclipse.
*/
@Provides
public class HandleEqualsAndHashCode extends EclipseAnnotationHandler<EqualsAndHashCode> {
private static final String HASH_CODE_CACHE_NAME = "$hashCodeCache";
private final char[] HASH_CODE_CACHE_NAME_ARR =
HASH_CODE_CACHE_NAME.toCharArray();
private final char[] PRIME = "PRIME".toCharArray();
private final char[] RESULT = "result".toCharArray();
public static final Set<String> BUILT_IN_TYPES = Collections.unmodifiableSet(new
HashSet<String>(Arrays.asList(
"byte", "short", "int", "long", "char", "boolean", "double", "float")));
@Override public void handle(AnnotationValues<EqualsAndHashCode> annotation, Annotation ast,
EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.EQUALS_AND_HASH_CODE_FLAG_USAGE,
"@EqualsAndHashCode");
EqualsAndHashCode ann = annotation.getInstance();
List<Included<EclipseNode, EqualsAndHashCode.Include>> members =
InclusionExclusionUtils.handleEqualsAndHashCodeMarking(annotationNode.up(), annotation,
annotationNode);
if (members == null) return;
List<Annotation> onParam = unboxAndRemoveAnnotationParameter(ast, "onParam",
"@EqualsAndHashCode(onParam", annotationNode);
Boolean callSuper = ann.callSuper();
if (!annotation.isExplicit("callSuper")) callSuper = null;
Boolean doNotUseGettersConfiguration = annotationNode.getAst().readConfiguration(ConfigurationKe
ys.EQUALS_AND_HASH_CODE_DO_NOT_USE_GETTERS);
boolean doNotUseGetters = annotation.isExplicit("doNotUseGetters") || doNotUseGettersConfiguration
== null ? ann.doNotUseGetters() : doNotUseGettersConfiguration;
FieldAccess fieldAccess = doNotUseGetters ? FieldAccess.PREFER_FIELD : FieldAccess.GETTER;
boolean cacheHashCode = ann.cacheStrategy() == CacheStrategy.LAZY;
generateMethods(annotationNode.up(), annotationNode, members, callSuper, true, cacheHashCode,
fieldAccess, onParam);
}
public void generateEqualsAndHashCodeForType(EclipseNode typeNode, EclipseNode errorNode) {
if (hasAnnotation(EqualsAndHashCode.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
```

```
}
List<Included<EclipseNode, EqualsAndHashCode.Include>> members =
InclusionExclusionUtils.handleEqualsAndHashCodeMarking(typeNode, null, null);
Boolean doNotUseGettersConfiguration = typeNode.getAst().readConfiguration(ConfigurationKeys.EQ
UALS_AND_HASH_CODE_DO_NOT_USE_GETTERS);
FieldAccess access = doNotUseGettersConfiguration == null || !doNotUseGettersConfiguration ?
FieldAccess.GETTER : FieldAccess.PREFER_FIELD;
generateMethods(typeNode, errorNode, members, null, false, false, access, new
ArrayList<Annotation>());
}
public void generateMethods(EclipseNode typeNode, EclipseNode errorNode,
List<Included<EclipseNode, EqualsAndHashCode.Include>> members,
Boolean callSuper, boolean whineIfExists, boolean cacheHashCode, FieldAccess fieldAccess,
List<Annotation> onParam) {
if (!isClass(typeNode)) {
errorNode.addError("@EqualsAndHashCode is only supported on a class.");
return;
}
TypeDeclaration typeDecl = (TypeDeclaration) typeNode.get();
boolean implicitCallSuper = callSuper == null;
if (callSuper == null) {
try {
callSuper = ((Boolean)
EqualsAndHashCode.class.getMethod("callSuper").getDefaultValue()).booleanValue();
} catch (Exception ignore) {
throw new InternalError("Lombok bug - this cannot happen - can't find callSuper field in
EqualsAndHashCode annotation.");
}
}
boolean isDirectDescendantOfObject = isDirectDescendantOfObject(typeNode);
boolean isFinal = (typeDecl.modifiers & ClassFileConstants.AccFinal) != 0;
boolean needsCanEqual = !isFinal || !isDirectDescendantOfObject;
MemberExistsResult equalsExists = methodExists("equals", typeNode, 1);
MemberExistsResult hashCodeExists = methodExists("hashCode", typeNode, 0);
MemberExistsResult canEqualExists = methodExists("canEqual", typeNode, 1);
switch (Collections.max(Arrays.asList(equalsExists, hashCodeExists))) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String msg = "Not generating equals and hashCode: A method with one of those names already exists.
(Either both or none of these methods will be generated).";
errorNode.addWarning(msg);
} else if (equalsExists == MemberExistsResult.NOT_EXISTS || hashCodeExists ==
MemberExistsResult.NOT_EXISTS) {
// This means equals OR hashCode exists and not both.
// Even though we should suppress the message about not generating these, this is such a weird and
surprising situation we should ALWAYS generate a warning.
// The user code couldn't possibly (barring really weird subclassing shenanigans) be in a shippable
state anyway; the implementations of these 2 methods are
// all inter-related and should be written by the same entity.
String msg = String.format("Not generating %s: One of equals or hashCode exists. " +
"You should either write both of these or none of these (in the latter case, lombok generates them).",
```

```
equalsExists == MemberExistsResult.NOT_EXISTS ? "equals" : "hashCode");
errorNode.addWarning(msg);
}
return;
case NOT_EXISTS:
default:
//fallthrough
}
if (isDirectDescendantOfObject && callSuper) {
errorNode.addError("Generating equals/hashCode with a supercall to java.lang.Object is pointless.");
return;
}
if (implicitCallSuper && !isDirectDescendantOfObject) {
CallSuperType cst = typeNode.getAst().readConfiguration(ConfigurationKeys.EQUALS_AND_HASH_
CODE_CALL_SUPER);
if (cst == null) cst = CallSuperType.WARN;
switch (cst) {
default:
case WARN:
errorNode.addWarning("Generating equals/hashCode implementation but without a call to superclass,
even though this class does not extend java.lang.Object. If this is intentional, add
'@EqualsAndHashCode(callSuper=false)' to your type.");
callSuper = false;
break;
case SKIP:
callSuper = false;
break;
case CALL:
callSuper = true;
break;
}
}
MethodDeclaration equalsMethod = createEquals(typeNode, members, callSuper, errorNode.get(),
fieldAccess, needsCanEqual, onParam);
equalsMethod.traverse(new SetGeneratedByVisitor(errorNode.get()),
((TypeDeclaration)typeNode.get()).scope);
injectMethod(typeNode, equalsMethod);
if (needsCanEqual && canEqualExists == MemberExistsResult.NOT_EXISTS) {
MethodDeclaration canEqualMethod = createCanEqual(typeNode, errorNode.get(), onParam);
canEqualMethod.traverse(new SetGeneratedByVisitor(errorNode.get()),
((TypeDeclaration)typeNode.get()).scope);
injectMethod(typeNode, canEqualMethod);
}
if (cacheHashCode){
if (fieldExists(HASH_CODE_CACHE_NAME, typeNode) != MemberExistsResult.NOT_EXISTS) {
String msg = String.format("Not caching the result of hashCode: A field named %s already exists.",
HASH_CODE_CACHE_NAME);
errorNode.addWarning(msg);
cacheHashCode = false;
} else {
createHashCodeCacheField(typeNode, errorNode.get());
}
}
```

```
MethodDeclaration hashCodeMethod = createHashCode(typeNode, members, callSuper,
cacheHashCode, errorNode.get(), fieldAccess);
hashCodeMethod.traverse(new SetGeneratedByVisitor(errorNode.get()),
((TypeDeclaration)typeNode.get()).scope);
injectMethod(typeNode, hashCodeMethod);
}
private void createHashCodeCacheField(EclipseNode typeNode, ASTNode source) {
FieldDeclaration hashCodeCacheDecl = new FieldDeclaration(HASH_CODE_CACHE_NAME_ARR, 0,
0);
hashCodeCacheDecl.modifiers = ClassFileConstants.AccPrivate | ClassFileConstants.AccTransient;
hashCodeCacheDecl.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
hashCodeCacheDecl.type = TypeReference.baseTypeReference(TypeIds.T_int, 0);
hashCodeCacheDecl.declarationSourceEnd = -1;
injectFieldAndMarkGenerated(typeNode, hashCodeCacheDecl);
setGeneratedBy(hashCodeCacheDecl, source);
setGeneratedBy(hashCodeCacheDecl.type, source);
}
private static final char[] HASH_CODE = "hashCode".toCharArray(), FLOAT_TO_INT_BITS =
"floatToIntBits".toCharArray(), DOUBLE_TO_LONG_BITS = "doubleToLongBits".toCharArray();
public MethodDeclaration createHashCode(EclipseNode type, Collection<Included<EclipseNode,
EqualsAndHashCode.Include>> members, boolean callSuper, boolean cacheHashCode, ASTNode
source, FieldAccess fieldAccess) {
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration method = new MethodDeclaration(((CompilationUnitDeclaration)
type.top().get()).compilationResult);
setGeneratedBy(method, source);
method.modifiers = toEclipseModifier(AccessLevel.PUBLIC);
method.returnType = TypeReference.baseTypeReference(TypeIds.T_int, 0);
setGeneratedBy(method.returnType, source);
Annotation overrideAnnotation = makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE,
source);
CheckerFrameworkVersion checkerFramework = getCheckerFrameworkVersion(type);
if (cacheHashCode && checkerFramework.generatePure()) {
method.annotations = new Annotation[] { overrideAnnotation, generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__PURE) };
} else if (checkerFramework.generateSideEffectFree()) {
method.annotations = new Annotation[] { overrideAnnotation, generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE) };
} else {
method.annotations = new Annotation[] { overrideAnnotation };
}
method.selector = "hashCode".toCharArray();
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
method.arguments = null;
List<Statement> statements = new ArrayList<Statement>();
boolean isEmpty = true;
for (Included<EclipseNode, EqualsAndHashCode.Include> member : members) {
TypeReference fType = getFieldType(member.getNode(), fieldAccess);
```

```
if (fType.getLastToken() != null) {
isEmpty = false;
break;
}
}
/* if (this.$hashCodeCache != 0) return this.$hashCodeCache; */ {
if (cacheHashCode) {
FieldReference hashCodeCacheRef = new FieldReference(HASH_CODE_CACHE_NAME_ARR, p);
hashCodeCacheRef.receiver = new ThisReference(pS, pE);
setGeneratedBy(hashCodeCacheRef, source);
setGeneratedBy(hashCodeCacheRef.receiver, source);
EqualExpression cacheNotZero = new EqualExpression(hashCodeCacheRef,
makeIntLiteral("0".toCharArray(), source), OperatorIds.NOT_EQUAL);
setGeneratedBy(cacheNotZero, source);
ReturnStatement returnCache = new ReturnStatement(hashCodeCacheRef, pS, pE);
setGeneratedBy(returnCache, source);
IfStatement ifStatement = new IfStatement(cacheNotZero, returnCache, pS, pE);
setGeneratedBy(ifStatement, source);
statements.add(ifStatement);
}
}
/* final int PRIME = X; */ {
/* Without members, PRIME isn't used, as that would trigger a 'local variable not used' warning. */
if (!isEmpty) {
LocalDeclaration primeDecl = new LocalDeclaration(PRIME, pS, pE);
setGeneratedBy(primeDecl, source);
primeDecl.modifiers |= Modifier.FINAL;
primeDecl.type = TypeReference.baseTypeReference(TypeIds.T_int, 0);
primeDecl.type.sourceStart = pS; primeDecl.type.sourceEnd = pE;
setGeneratedBy(primeDecl.type, source);
primeDecl.initialization =
makeIntLiteral(String.valueOf(HandlerUtil.primeForHashcode()).toCharArray(), source);
statements.add(primeDecl);
}
}
/* int result = ... */ {
LocalDeclaration resultDecl = new LocalDeclaration(RESULT, pS, pE);
setGeneratedBy(resultDecl, source);
final Expression init;
if (callSuper) {
/* ... super.hashCode(); */
MessageSend callToSuper = new MessageSend();
setGeneratedBy(callToSuper, source);
callToSuper.sourceStart = pS; callToSuper.sourceEnd = pE;
callToSuper.receiver = new SuperReference(pS, pE);
setGeneratedBy(callToSuper.receiver, source);
callToSuper.selector = "hashCode".toCharArray();
init = callToSuper;
} else {
/* ... 1; */
init = makeIntLiteral("1".toCharArray(), source);
}
resultDecl.initialization = init;
```

```
resultDecl.type = TypeReference.baseTypeReference(TypeIds.T_int, 0);
resultDecl.type.sourceStart = pS; resultDecl.type.sourceEnd = pE;
if (isEmpty && !cacheHashCode) resultDecl.modifiers |= Modifier.FINAL;
setGeneratedBy(resultDecl.type, source);
statements.add(resultDecl);
}
for (Included<EclipseNode, EqualsAndHashCode.Include> member : members) {
EclipseNode memberNode = member.getNode();
boolean isMethod = memberNode.getKind() == Kind.METHOD;
TypeReference fType = getFieldType(memberNode, fieldAccess);
char[] dollarFieldName = ((isMethod ? "$$" : "$") + memberNode.getName()).toCharArray();
char[] token = fType.getLastToken();
Expression fieldAccessor = isMethod ? createMethodAccessor(memberNode, source) :
createFieldAccessor(memberNode, fieldAccess, source);
if (fType.dimensions() == 0 && token != null) {
if (Arrays.equals(TypeConstants.BOOLEAN, token)) {
/* booleanField ? X : Y */
IntLiteral intTrue = makeIntLiteral(String.valueOf(HandlerUtil.primeForTrue()).toCharArray(), source);
IntLiteral intFalse = makeIntLiteral(String.valueOf(HandlerUtil.primeForFalse()).toCharArray(), source);
ConditionalExpression intForBool = new ConditionalExpression(fieldAccessor, intTrue, intFalse);
setGeneratedBy(intForBool, source);
statements.add(createResultCalculation(source, intForBool));
} else if (Arrays.equals(TypeConstants.LONG, token)) {
/* (int)(ref >>> 32 ^ ref) */
statements.add(createLocalDeclaration(source, dollarFieldName,
TypeReference.baseTypeReference(TypeIds.T_long, 0), fieldAccessor));
SingleNameReference copy1 = new SingleNameReference(dollarFieldName, p);
setGeneratedBy(copy1, source);
SingleNameReference copy2 = new SingleNameReference(dollarFieldName, p);
setGeneratedBy(copy2, source);
statements.add(createResultCalculation(source, longToIntForHashCode(copy1, copy2, source)));
} else if (Arrays.equals(TypeConstants.FLOAT, token)) {
/* Float.floatToIntBits(fieldName) */
MessageSend floatToIntBits = new MessageSend();
floatToIntBits.sourceStart = pS; floatToIntBits.sourceEnd = pE;
setGeneratedBy(floatToIntBits, source);
floatToIntBits.receiver = generateQualifiedNameRef(source, TypeConstants.JAVA_LANG_FLOAT);
floatToIntBits.selector = FLOAT_TO_INT_BITS;
floatToIntBits.arguments = new Expression[] { fieldAccessor };
statements.add(createResultCalculation(source, floatToIntBits));
} else if (Arrays.equals(TypeConstants.DOUBLE, token)) {
/* longToIntForHashCode(Double.doubleToLongBits(fieldName)) */
MessageSend doubleToLongBits = new MessageSend();
doubleToLongBits.sourceStart = pS; doubleToLongBits.sourceEnd = pE;
setGeneratedBy(doubleToLongBits, source);
doubleToLongBits.receiver = generateQualifiedNameRef(source,
TypeConstants.JAVA_LANG_DOUBLE);
doubleToLongBits.selector = DOUBLE_TO_LONG_BITS;
doubleToLongBits.arguments = new Expression[] { fieldAccessor };
statements.add(createLocalDeclaration(source, dollarFieldName,
TypeReference.baseTypeReference(TypeIds.T_long, 0), doubleToLongBits));
SingleNameReference copy1 = new SingleNameReference(dollarFieldName, p);
setGeneratedBy(copy1, source);
```

```
SingleNameReference copy2 = new SingleNameReference(dollarFieldName, p);
setGeneratedBy(copy2, source);
statements.add(createResultCalculation(source, longToIntForHashCode(copy1, copy2, source)));
} else if (BUILT_IN_TYPES.contains(new String(token))) {
statements.add(createResultCalculation(source, fieldAccessor));
} else /* objects */ {
/* final java.lang.Object $fieldName = this.fieldName; */
/* $fieldName == null ? NULL_PRIME : $fieldName.hashCode() */
statements.add(createLocalDeclaration(source, dollarFieldName, generateQualifiedTypeRef(source,
TypeConstants.JAVA_LANG_OBJECT), fieldAccessor));
SingleNameReference copy1 = new SingleNameReference(dollarFieldName, p);
setGeneratedBy(copy1, source);
SingleNameReference copy2 = new SingleNameReference(dollarFieldName, p);
setGeneratedBy(copy2, source);
MessageSend hashCodeCall = new MessageSend();
hashCodeCall.sourceStart = pS; hashCodeCall.sourceEnd = pE;
setGeneratedBy(hashCodeCall, source);
hashCodeCall.receiver = copy1;
hashCodeCall.selector = HASH_CODE;
NullLiteral nullLiteral = new NullLiteral(pS, pE);
setGeneratedBy(nullLiteral, source);
EqualExpression objIsNull = new EqualExpression(copy2, nullLiteral, OperatorIds.EQUAL_EQUAL);
setGeneratedBy(objIsNull, source);
IntLiteral intMagic = makeIntLiteral(String.valueOf(HandlerUtil.primeForNull()).toCharArray(), source);
ConditionalExpression nullOrHashCode = new ConditionalExpression(objIsNull, intMagic,
hashCodeCall);
nullOrHashCode.sourceStart = pS; nullOrHashCode.sourceEnd = pE;
setGeneratedBy(nullOrHashCode, source);
statements.add(createResultCalculation(source, nullOrHashCode));
}
} else if (fType.dimensions() > 0 && token != null) {
/* Arrays.deepHashCode(array) //just hashCode for simple arrays */
MessageSend arraysHashCodeCall = new MessageSend();
arraysHashCodeCall.sourceStart = pS; arraysHashCodeCall.sourceEnd = pE;
setGeneratedBy(arraysHashCodeCall, source);
arraysHashCodeCall.receiver = generateQualifiedNameRef(source, TypeConstants.JAVA,
TypeConstants.UTIL, "Arrays".toCharArray());
if (fType.dimensions() > 1 || !BUILT_IN_TYPES.contains(new String(token))) {
arraysHashCodeCall.selector = "deepHashCode".toCharArray();
} else {
arraysHashCodeCall.selector = "hashCode".toCharArray();
}
arraysHashCodeCall.arguments = new Expression[] { fieldAccessor };
statements.add(createResultCalculation(source, arraysHashCodeCall));
}
}
/*
* if (result == 0) result = Integer.MIN_VALUE;
* this.$hashCodeCache = result;
*
*/ {
if (cacheHashCode) {
SingleNameReference resultRef = new SingleNameReference(RESULT, p);
```

```
setGeneratedBy(resultRef, source);
EqualExpression resultIsZero = new EqualExpression(resultRef, makeIntLiteral("0".toCharArray(),
source), OperatorIds.EQUAL_EQUAL);
setGeneratedBy(resultIsZero, source);
resultRef = new SingleNameReference(RESULT, p);
setGeneratedBy(resultRef, source);
FieldReference integerMinValue = new FieldReference("MIN_VALUE".toCharArray(), p);
integerMinValue.receiver = generateQualifiedNameRef(source,
TypeConstants.JAVA_LANG_INTEGER);
setGeneratedBy(integerMinValue, source);
Assignment newResult = new Assignment(resultRef, integerMinValue, pE);
newResult.sourceStart = pS; newResult.statementEnd = newResult.sourceEnd = pE;
setGeneratedBy(newResult, source);
IfStatement ifStatement = new IfStatement(resultIsZero, newResult, pS, pE);
setGeneratedBy(ifStatement, source);
statements.add(ifStatement);
FieldReference hashCodeCacheRef = new FieldReference(HASH_CODE_CACHE_NAME_ARR, p);
hashCodeCacheRef.receiver = new ThisReference(pS, pE);
setGeneratedBy(hashCodeCacheRef, source);
setGeneratedBy(hashCodeCacheRef.receiver, source);
resultRef = new SingleNameReference(RESULT, p);
setGeneratedBy(resultRef, source);
Assignment cacheResult = new Assignment(hashCodeCacheRef, resultRef, pE);
cacheResult.sourceStart = pS; cacheResult.statementEnd = cacheResult.sourceEnd = pE;
setGeneratedBy(cacheResult, source);
statements.add(cacheResult);
}
}
/* return result; */ {
SingleNameReference resultRef = new SingleNameReference(RESULT, p);
setGeneratedBy(resultRef, source);
ReturnStatement returnStatement = new ReturnStatement(resultRef, pS, pE);
setGeneratedBy(returnStatement, source);
statements.add(returnStatement);
}
method.statements = statements.toArray(new Statement[0]);
return method;
}
public LocalDeclaration createLocalDeclaration(ASTNode source, char[] dollarFieldName,
TypeReference type, Expression initializer) {
int pS = source.sourceStart, pE = source.sourceEnd;
LocalDeclaration tempVar = new LocalDeclaration(dollarFieldName, pS, pE);
setGeneratedBy(tempVar, source);
tempVar.initialization = initializer;
tempVar.type = type;
tempVar.type.sourceStart = pS; tempVar.type.sourceEnd = pE;
setGeneratedBy(tempVar.type, source);
tempVar.modifiers = Modifier.FINAL;
return tempVar;
}
public Expression createResultCalculation(ASTNode source, Expression ex) {
/* result = result * PRIME + (ex); */
int pS = source.sourceStart, pE = source.sourceEnd;
```

```
long p = (long)pS << 32 | pE;
SingleNameReference resultRef = new SingleNameReference(RESULT, p);
setGeneratedBy(resultRef, source);
SingleNameReference primeRef = new SingleNameReference(PRIME, p);
setGeneratedBy(primeRef, source);
BinaryExpression multiplyByPrime = new BinaryExpression(resultRef, primeRef,
OperatorIds.MULTIPLY);
multiplyByPrime.sourceStart = pS; multiplyByPrime.sourceEnd = pE;
setGeneratedBy(multiplyByPrime, source);
BinaryExpression addItem = new BinaryExpression(multiplyByPrime, ex, OperatorIds.PLUS);
addItem.sourceStart = pS; addItem.sourceEnd = pE;
setGeneratedBy(addItem, source);
resultRef = new SingleNameReference(RESULT, p);
setGeneratedBy(resultRef, source);
Assignment assignment = new Assignment(resultRef, addItem, pE);
assignment.sourceStart = pS; assignment.sourceEnd = assignment.statementEnd = pE;
setGeneratedBy(assignment, source);
return assignment;
}
/**
* @param type Type to 'copy' into a typeref
* @param p position
* @param addWildcards If false, all generics are cut off. If true, replaces all genericparams with a ?.
* @return
*/
public TypeReference createTypeReference(EclipseNode type, long p, ASTNode source, boolean
addWildcards) {
int pS = source.sourceStart; int pE = source.sourceEnd;
List<String> list = new ArrayList<String>();
List<Integer> genericsCount = addWildcards ? new ArrayList<Integer>() : null;
list.add(type.getName());
if (addWildcards) genericsCount.add(arraySizeOf(((TypeDeclaration) type.get()).typeParameters));
boolean staticContext = (((TypeDeclaration) type.get()).modifiers & ClassFileConstants.AccStatic) != 0;
EclipseNode tNode = type.up();
while (tNode != null && tNode.getKind() == Kind.TYPE) {
TypeDeclaration td = (TypeDeclaration) tNode.get();
if (td.name == null || td.name.length == 0) break;
list.add(tNode.getName());
if (!staticContext && tNode.getKind() == Kind.TYPE && (td.modifiers &
ClassFileConstants.AccInterface) != 0) staticContext = true;
if (addWildcards) genericsCount.add(staticContext ? 0 : arraySizeOf(td.typeParameters));
if (!staticContext) staticContext = (td.modifiers & Modifier.STATIC) != 0;
tNode = tNode.up();
}
Collections.reverse(list);
if (addWildcards) Collections.reverse(genericsCount);
if (list.size() == 1) {
if (!addWildcards || genericsCount.get(0) == 0) {
return new SingleTypeReference(list.get(0).toCharArray(), p);
} else {
return new ParameterizedSingleTypeReference(list.get(0).toCharArray(), wildcardify(pS, pE, source,
genericsCount.get(0)), 0, p);
}
```

```
}
if (addWildcards) {
addWildcards = false;
for (int i : genericsCount) if (i > 0) addWildcards = true;
}
long[] ps = new long[list.size()];
char[][] tokens = new char[list.size()][];
for (int i = 0; i < list.size(); i++) {
ps[i] = p;
tokens[i] = list.get(i).toCharArray();
}
if (!addWildcards) return new QualifiedTypeReference(tokens, ps);
TypeReference[][] typeArgs2 = new TypeReference[tokens.length][];
for (int i = 0; i < tokens.length; i++) typeArgs2[i] = wildcardify(pS, pE, source, genericsCount.get(i));
return new ParameterizedQualifiedTypeReference(tokens, typeArgs2, 0, ps);
}
private TypeReference[] wildcardify(int pS, int pE, ASTNode source, int count) {
if (count == 0) return null;
TypeReference[] typeArgs = new TypeReference[count];
for (int i = 0; i < count; i++) {
typeArgs[i] = new Wildcard(Wildcard.UNBOUND);
typeArgs[i].sourceStart = pS; typeArgs[i].sourceEnd = pE;
setGeneratedBy(typeArgs[i], source);
}
return typeArgs;
}
private int arraySizeOf(Object[] arr) {
return arr == null ? 0 : arr.length;
}
/*
* scan method, then class, then enclosing classes, then package for the first of:
* javax.annotation.ParametersAreNonnullByDefault or
javax.annotation.ParametersAreNullableByDefault. If it's the NN variant, generate
javax.annotation.Nullable _on the type_.
* org.eclipse.jdt.annotation.NonNullByDefault -> org.eclipse.jdt.annotation.Nullable
*/
private static final char[][] JAVAX_ANNOTATION_NULLABLE =
Eclipse.fromQualifiedName("javax.annotation.Nullable");
private static final char[][] ORG_ECLIPSE_JDT_ANNOTATION_NULLABLE =
Eclipse.fromQualifiedName("org.eclipse.jdt.annotation.Nullable");
public MethodDeclaration createEquals(EclipseNode type, Collection<Included<EclipseNode,
EqualsAndHashCode.Include>> members, boolean callSuper, ASTNode source, FieldAccess
fieldAccess, boolean needsCanEqual, List<Annotation> onParam) {
int pS = source.sourceStart; int pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
List<NullAnnotationLibrary> applied = new ArrayList<NullAnnotationLibrary>();
Annotation[] onParamNullable = null;
String nearest = scanForNearestAnnotation(type, "javax.annotation.ParametersAreNullableByDefault",
"javax.annotation.ParametersAreNonnullByDefault");
if ("javax.annotation.ParametersAreNonnullByDefault".equals(nearest)) {
onParamNullable = new Annotation[] { new MarkerAnnotation(generateQualifiedTypeRef(source,
JAVAX_ANNOTATION_NULLABLE), 0) };
applied.add(NullAnnotationLibrary.JAVAX);
```

```
}
Annotation[] onParamTypeNullable = null;
nearest = scanForNearestAnnotation(type, "org.eclipse.jdt.annotation.NonNullByDefault");
if (nearest != null) {
onParamTypeNullable = new Annotation[] { new MarkerAnnotation(generateQualifiedTypeRef(source,
ORG_ECLIPSE_JDT_ANNOTATION_NULLABLE), 0) };
applied.add(NullAnnotationLibrary.ECLIPSE);
}
MethodDeclaration method = new MethodDeclaration(((CompilationUnitDeclaration)
type.top().get()).compilationResult);
setGeneratedBy(method, source);
method.modifiers = toEclipseModifier(AccessLevel.PUBLIC);
method.returnType = TypeReference.baseTypeReference(TypeIds.T_boolean, 0);
method.returnType.sourceStart = pS; method.returnType.sourceEnd = pE;
setGeneratedBy(method.returnType, source);
Annotation overrideAnnotation = makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE,
source);
if (getCheckerFrameworkVersion(type).generateSideEffectFree()) {
method.annotations = new Annotation[] { overrideAnnotation, generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE) };
} else {
method.annotations = new Annotation[] { overrideAnnotation };
}
method.selector = "equals".toCharArray();
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
QualifiedTypeReference objectRef = new
QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, new long[] { p, p, p });
if (onParamTypeNullable != null) {
objectRef.annotations = new Annotation[][] {null, null, onParamTypeNullable};
objectRef.bits |= Eclipse.HasTypeAnnotations;
method.bits |= Eclipse.HasTypeAnnotations;
}
setGeneratedBy(objectRef, source);
method.arguments = new Argument[] {new Argument(new char[] { 'o' }, 0, objectRef, Modifier.FINAL)};
method.arguments[0].sourceStart = pS; method.arguments[0].sourceEnd = pE;
if (!onParam.isEmpty() || onParamNullable != null)
method.arguments[0].annotations = copyAnnotations(source, onParam.toArray(new Annotation[0]),
onParamNullable);
EclipseHandlerUtil.createRelevantNullableAnnotation(type, method.arguments[0], method, applied);
setGeneratedBy(method.arguments[0], source);
List<Statement> statements = new ArrayList<Statement>();
/* if (o == this) return true; */ {
SingleNameReference oRef = new SingleNameReference(new char[] { 'o' }, p);
setGeneratedBy(oRef, source);
ThisReference thisRef = new ThisReference(pS, pE);
setGeneratedBy(thisRef, source);
EqualExpression otherEqualsThis = new EqualExpression(oRef, thisRef,
OperatorIds.EQUAL_EQUAL);
setGeneratedBy(otherEqualsThis, source);
```

```
TrueLiteral trueLiteral = new TrueLiteral(pS, pE);
setGeneratedBy(trueLiteral, source);
ReturnStatement returnTrue = new ReturnStatement(trueLiteral, pS, pE);
setGeneratedBy(returnTrue, source);
IfStatement ifOtherEqualsThis = new IfStatement(otherEqualsThis, returnTrue, pS, pE);
setGeneratedBy(ifOtherEqualsThis, source);
statements.add(ifOtherEqualsThis);
}
/* if (!(o instanceof Outer.Inner.MyType) return false; */ {
SingleNameReference oRef = new SingleNameReference(new char[] { 'o' }, p);
setGeneratedBy(oRef, source);
TypeReference typeReference = createTypeReference(type, p, source, false);
setGeneratedBy(typeReference, source);
InstanceOfExpression instanceOf = new InstanceOfExpression(oRef, typeReference);
instanceOf.sourceStart = pS; instanceOf.sourceEnd = pE;
setGeneratedBy(instanceOf, source);
Expression notInstanceOf = new UnaryExpression(instanceOf, OperatorIds.NOT);
setGeneratedBy(notInstanceOf, source);
FalseLiteral falseLiteral = new FalseLiteral(pS, pE);
setGeneratedBy(falseLiteral, source);
ReturnStatement returnFalse = new ReturnStatement(falseLiteral, pS, pE);
setGeneratedBy(returnFalse, source);
IfStatement ifNotInstanceOf = new IfStatement(notInstanceOf, returnFalse, pS, pE);
setGeneratedBy(ifNotInstanceOf, source);
statements.add(ifNotInstanceOf);
}
char[] otherName = "other".toCharArray();
/* Outer.Inner.MyType<?> other = (Outer.Inner.MyType<?>) o; */ {
if (!members.isEmpty() || needsCanEqual) {
LocalDeclaration other = new LocalDeclaration(otherName, pS, pE);
other.modifiers |= ClassFileConstants.AccFinal;
setGeneratedBy(other, source);
TypeReference targetType = createTypeReference(type, p, source, true);
setGeneratedBy(targetType, source);
other.type = createTypeReference(type, p, source, true);
setGeneratedBy(other.type, source);
NameReference oRef = new SingleNameReference(new char[] { 'o' }, p);
setGeneratedBy(oRef, source);
other.initialization = makeCastExpression(oRef, targetType, source);
statements.add(other);
}
}
/* if (!other.canEqual((java.lang.Object) this)) return false; */ {
if (needsCanEqual) {
MessageSend otherCanEqual = new MessageSend();
otherCanEqual.sourceStart = pS; otherCanEqual.sourceEnd = pE;
setGeneratedBy(otherCanEqual, source);
otherCanEqual.receiver = new SingleNameReference(otherName, p);
setGeneratedBy(otherCanEqual.receiver, source);
otherCanEqual.selector = "canEqual".toCharArray();
ThisReference thisReference = new ThisReference(pS, pE);
setGeneratedBy(thisReference, source);
```

```
CastExpression castThisRef = makeCastExpression(thisReference, generateQualifiedTypeRef(source,
TypeConstants.JAVA_LANG_OBJECT), source);
castThisRef.sourceStart = pS; castThisRef.sourceEnd = pE;
otherCanEqual.arguments = new Expression[] {castThisRef};
Expression notOtherCanEqual = new UnaryExpression(otherCanEqual, OperatorIds.NOT);
setGeneratedBy(notOtherCanEqual, source);
FalseLiteral falseLiteral = new FalseLiteral(pS, pE);
setGeneratedBy(falseLiteral, source);
ReturnStatement returnFalse = new ReturnStatement(falseLiteral, pS, pE);
setGeneratedBy(returnFalse, source);
IfStatement ifNotCanEqual = new IfStatement(notOtherCanEqual, returnFalse, pS, pE);
setGeneratedBy(ifNotCanEqual, source);
statements.add(ifNotCanEqual);
}
}
/* if (!super.equals(o)) return false; */
if (callSuper) {
MessageSend callToSuper = new MessageSend();
callToSuper.sourceStart = pS; callToSuper.sourceEnd = pE;
setGeneratedBy(callToSuper, source);
callToSuper.receiver = new SuperReference(pS, pE);
setGeneratedBy(callToSuper.receiver, source);
callToSuper.selector = "equals".toCharArray();
SingleNameReference oRef = new SingleNameReference(new char[] { 'o' }, p);
setGeneratedBy(oRef, source);
callToSuper.arguments = new Expression[] {oRef};
Expression superNotEqual = new UnaryExpression(callToSuper, OperatorIds.NOT);
setGeneratedBy(superNotEqual, source);
FalseLiteral falseLiteral = new FalseLiteral(pS, pE);
setGeneratedBy(falseLiteral, source);
ReturnStatement returnFalse = new ReturnStatement(falseLiteral, pS, pE);
setGeneratedBy(returnFalse, source);
IfStatement ifSuperEquals = new IfStatement(superNotEqual, returnFalse, pS, pE);
setGeneratedBy(ifSuperEquals, source);
statements.add(ifSuperEquals);
}
for (Included<EclipseNode, EqualsAndHashCode.Include> member : members) {
EclipseNode memberNode = member.getNode();
boolean isMethod = memberNode.getKind() == Kind.METHOD;
TypeReference fType = getFieldType(memberNode, fieldAccess);
char[] token = fType.getLastToken();
Expression thisFieldAccessor = isMethod ? createMethodAccessor(memberNode, source) :
createFieldAccessor(memberNode, fieldAccess, source);
Expression otherFieldAccessor = isMethod ? createMethodAccessor(memberNode, source,
otherName) : createFieldAccessor(memberNode, fieldAccess, source, otherName);
if (fType.dimensions() == 0 && token != null) {
if (Arrays.equals(TypeConstants.FLOAT, token)) {
statements.add(generateCompareFloatOrDouble(thisFieldAccessor, otherFieldAccessor,
"Float".toCharArray(), source));
} else if (Arrays.equals(TypeConstants.DOUBLE, token)) {
statements.add(generateCompareFloatOrDouble(thisFieldAccessor, otherFieldAccessor,
"Double".toCharArray(), source));
} else if (BUILT_IN_TYPES.contains(new String(token))) {
```

```
EqualExpression fieldsNotEqual = new EqualExpression(thisFieldAccessor, otherFieldAccessor,
OperatorIds.NOT_EQUAL);
setGeneratedBy(fieldsNotEqual, source);
FalseLiteral falseLiteral = new FalseLiteral(pS, pE);
setGeneratedBy(falseLiteral, source);
ReturnStatement returnStatement = new ReturnStatement(falseLiteral, pS, pE);
setGeneratedBy(returnStatement, source);
IfStatement ifStatement = new IfStatement(fieldsNotEqual, returnStatement, pS, pE);
setGeneratedBy(ifStatement, source);
statements.add(ifStatement);
} else /* objects */ {
/* final java.lang.Object this$fieldName = this.fieldName; */
/* final java.lang.Object other$fieldName = other.fieldName; */
/* if (this$fieldName == null ? other$fieldName != null : !this$fieldName.equals(other$fieldName)) return
false; */
char[] thisDollarFieldName = ("this" + (isMethod ? "$$" : "$") + memberNode.getName()).toCharArray();
char[] otherDollarFieldName = ("other" + (isMethod ? "$$" : "$") +
memberNode.getName()).toCharArray();
statements.add(createLocalDeclaration(source, thisDollarFieldName,
generateQualifiedTypeRef(source, TypeConstants.JAVA_LANG_OBJECT), thisFieldAccessor));
statements.add(createLocalDeclaration(source, otherDollarFieldName,
generateQualifiedTypeRef(source, TypeConstants.JAVA_LANG_OBJECT), otherFieldAccessor));
SingleNameReference this1 = new SingleNameReference(thisDollarFieldName, p);
setGeneratedBy(this1, source);
SingleNameReference this2 = new SingleNameReference(thisDollarFieldName, p);
setGeneratedBy(this2, source);
SingleNameReference other1 = new SingleNameReference(otherDollarFieldName, p);
setGeneratedBy(other1, source);
SingleNameReference other2 = new SingleNameReference(otherDollarFieldName, p);
setGeneratedBy(other2, source);
NullLiteral nullLiteral = new NullLiteral(pS, pE);
setGeneratedBy(nullLiteral, source);
EqualExpression fieldIsNull = new EqualExpression(this1, nullLiteral, OperatorIds.EQUAL_EQUAL);
nullLiteral = new NullLiteral(pS, pE);
setGeneratedBy(nullLiteral, source);
EqualExpression otherFieldIsntNull = new EqualExpression(other1, nullLiteral,
OperatorIds.NOT_EQUAL);
MessageSend equalsCall = new MessageSend();
equalsCall.sourceStart = pS; equalsCall.sourceEnd = pE;
setGeneratedBy(equalsCall, source);
equalsCall.receiver = this2;
equalsCall.selector = "equals".toCharArray();
equalsCall.arguments = new Expression[] { other2 };
UnaryExpression fieldsNotEqual = new UnaryExpression(equalsCall, OperatorIds.NOT);
fieldsNotEqual.sourceStart = pS; fieldsNotEqual.sourceEnd = pE;
setGeneratedBy(fieldsNotEqual, source);
ConditionalExpression fullEquals = new ConditionalExpression(fieldIsNull, otherFieldIsntNull,
fieldsNotEqual);
fullEquals.sourceStart = pS; fullEquals.sourceEnd = pE;
setGeneratedBy(fullEquals, source);
FalseLiteral falseLiteral = new FalseLiteral(pS, pE);
setGeneratedBy(falseLiteral, source);
ReturnStatement returnStatement = new ReturnStatement(falseLiteral, pS, pE);
```

```
setGeneratedBy(returnStatement, source);
IfStatement ifStatement = new IfStatement(fullEquals, returnStatement, pS, pE);
setGeneratedBy(ifStatement, source);
statements.add(ifStatement);
}
} else if (fType.dimensions() > 0 && token != null) {
MessageSend arraysEqualCall = new MessageSend();
arraysEqualCall.sourceStart = pS; arraysEqualCall.sourceEnd = pE;
setGeneratedBy(arraysEqualCall, source);
arraysEqualCall.receiver = generateQualifiedNameRef(source, TypeConstants.JAVA,
TypeConstants.UTIL, "Arrays".toCharArray());
if (fType.dimensions() > 1 || !BUILT_IN_TYPES.contains(new String(token))) {
arraysEqualCall.selector = "deepEquals".toCharArray();
} else {
arraysEqualCall.selector = "equals".toCharArray();
}
arraysEqualCall.arguments = new Expression[] { thisFieldAccessor, otherFieldAccessor };
UnaryExpression arraysNotEqual = new UnaryExpression(arraysEqualCall, OperatorIds.NOT);
arraysNotEqual.sourceStart = pS; arraysNotEqual.sourceEnd = pE;
setGeneratedBy(arraysNotEqual, source);
FalseLiteral falseLiteral = new FalseLiteral(pS, pE);
setGeneratedBy(falseLiteral, source);
ReturnStatement returnStatement = new ReturnStatement(falseLiteral, pS, pE);
setGeneratedBy(returnStatement, source);
IfStatement ifStatement = new IfStatement(arraysNotEqual, returnStatement, pS, pE);
setGeneratedBy(ifStatement, source);
statements.add(ifStatement);
}
}
/* return true; */ {
TrueLiteral trueLiteral = new TrueLiteral(pS, pE);
setGeneratedBy(trueLiteral, source);
ReturnStatement returnStatement = new ReturnStatement(trueLiteral, pS, pE);
setGeneratedBy(returnStatement, source);
statements.add(returnStatement);
}
method.statements = statements.toArray(new Statement[0]);
return method;
}
public MethodDeclaration createCanEqual(EclipseNode type, ASTNode source, List<Annotation>
onParam) {
/* protected boolean canEqual(final java.lang.Object other) {
* return other instanceof Outer.Inner.MyType;
* }
*/
int pS = source.sourceStart; int pE = source.sourceEnd;
long p = (long)pS << 32 | pE;
char[] otherName = "other".toCharArray();
MethodDeclaration method = new MethodDeclaration(((CompilationUnitDeclaration)
type.top().get()).compilationResult);
setGeneratedBy(method, source);
method.modifiers = toEclipseModifier(AccessLevel.PROTECTED);
method.returnType = TypeReference.baseTypeReference(TypeIds.T_boolean, 0);
```

```
method.returnType.sourceStart = pS; method.returnType.sourceEnd = pE;
setGeneratedBy(method.returnType, source);
method.selector = "canEqual".toCharArray();
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
TypeReference objectRef = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, new
long[] { p, p, p });
setGeneratedBy(objectRef, source);
method.arguments = new Argument[] {new Argument(otherName, 0, objectRef, Modifier.FINAL)};
method.arguments[0].sourceStart = pS; method.arguments[0].sourceEnd = pE;
if (!onParam.isEmpty()) method.arguments[0].annotations = onParam.toArray(new Annotation[0]);
EclipseHandlerUtil.createRelevantNullableAnnotation(type, method.arguments[0], method);
setGeneratedBy(method.arguments[0], source);
SingleNameReference otherRef = new SingleNameReference(otherName, p);
setGeneratedBy(otherRef, source);
TypeReference typeReference = createTypeReference(type, p, source, false);
setGeneratedBy(typeReference, source);
InstanceOfExpression instanceOf = new InstanceOfExpression(otherRef, typeReference);
instanceOf.sourceStart = pS; instanceOf.sourceEnd = pE;
setGeneratedBy(instanceOf, source);
ReturnStatement returnStatement = new ReturnStatement(instanceOf, pS, pE);
setGeneratedBy(returnStatement, source);
method.statements = new Statement[] {returnStatement};
if (getCheckerFrameworkVersion(type).generatePure()) method.annotations = new Annotation[] {
generateNamedAnnotation(source, CheckerFrameworkVersion.NAME__PURE) };
return method;
}
public IfStatement generateCompareFloatOrDouble(Expression thisRef, Expression otherRef, char[]
floatOrDouble, ASTNode source) {
int pS = source.sourceStart, pE = source.sourceEnd;
/* if (Float.compare(fieldName, other.fieldName) != 0) return false */
MessageSend floatCompare = new MessageSend();
floatCompare.sourceStart = pS; floatCompare.sourceEnd = pE;
setGeneratedBy(floatCompare, source);
floatCompare.receiver = generateQualifiedNameRef(source, TypeConstants.JAVA,
TypeConstants.LANG, floatOrDouble);
floatCompare.selector = "compare".toCharArray();
floatCompare.arguments = new Expression[] {thisRef, otherRef};
IntLiteral int0 = makeIntLiteral("0".toCharArray(), source);
EqualExpression ifFloatCompareIsNot0 = new EqualExpression(floatCompare, int0,
OperatorIds.NOT_EQUAL);
ifFloatCompareIsNot0.sourceStart = pS; ifFloatCompareIsNot0.sourceEnd = pE;
setGeneratedBy(ifFloatCompareIsNot0, source);
FalseLiteral falseLiteral = new FalseLiteral(pS, pE);
setGeneratedBy(falseLiteral, source);
ReturnStatement returnFalse = new ReturnStatement(falseLiteral, pS, pE);
setGeneratedBy(returnFalse, source);
IfStatement ifStatement = new IfStatement(ifFloatCompareIsNot0, returnFalse, pS, pE);
setGeneratedBy(ifStatement, source);
return ifStatement;
```

```
}
/** Give 2 clones! */
public Expression longToIntForHashCode(Expression ref1, Expression ref2, ASTNode source) {
int pS = source.sourceStart, pE = source.sourceEnd;
/* (int)(ref >>> 32 ^ ref) */
IntLiteral int32 = makeIntLiteral("32".toCharArray(), source);
BinaryExpression higherBits = new BinaryExpression(ref1, int32,
OperatorIds.UNSIGNED_RIGHT_SHIFT);
setGeneratedBy(higherBits, source);
BinaryExpression xorParts = new BinaryExpression(ref2, higherBits, OperatorIds.XOR);
setGeneratedBy(xorParts, source);
TypeReference intRef = TypeReference.baseTypeReference(TypeIds.T_int, 0);
intRef.sourceStart = pS; intRef.sourceEnd = pE;
setGeneratedBy(intRef, source);
CastExpression expr = makeCastExpression(xorParts, intRef, source);
expr.sourceStart = pS; expr.sourceEnd = pE;
return expr;
}
}
```

lombok/eclipse/handlers/HandleExtensionMethod.java

```
/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
```

```
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.ExtensionMethod;
import lombok.spi.Provides;
// This handler just does some additional error checking; the real work is done in the agent.
@Provides
@HandlerPriority(66560) // 2^16 + 2^10; we must run AFTER HandleVal which is at 2^16
public class HandleExtensionMethod extends EclipseAnnotationHandler<ExtensionMethod> {
@Override public void handle(AnnotationValues<ExtensionMethod> annotation, Annotation ast,
EclipseNode annotationNode) {
handleExperimentalFlagUsage(annotationNode,
ConfigurationKeys.EXTENSION_METHOD_FLAG_USAGE, "@ExtensionMethod");
TypeDeclaration typeDecl = null;
EclipseNode owner = annotationNode.up();
if (owner.get() instanceof TypeDeclaration) typeDecl = (TypeDeclaration) owner.get();
int modifiers = typeDecl == null ? 0 : typeDecl.modifiers;
boolean notAClass = (modifiers &
(ClassFileConstants.AccAnnotation)) != 0;
if (typeDecl == null || notAClass) {
annotationNode.addError("@ExtensionMethod is legal only on classes and enums and interfaces.");
return;
}
List<Object> listenerInterfaces = annotation.getActualExpressions("value");
if (listenerInterfaces.isEmpty()) {
annotationNode.addWarning(String.format("@ExtensionMethod has no effect since no extension types
were specified."));
return;
}
}
}
```

lombok/eclipse/handlers/HandleFieldDefaults.java

```
/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.Arrays;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseASTAdapter;
import lombok.eclipse.EclipseASTVisitor;
import lombok.eclipse.EclipseNode;
import lombok.experimental.FieldDefaults;
import lombok.experimental.NonFinal;
import lombok.experimental.PackagePrivate;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
/**
* Handles the {@code lombok.FieldDefaults} annotation for eclipse.
*/
@Provides(EclipseASTVisitor.class)
@HandlerPriority(-2048) //-2^11; to ensure @Value picks up on messing with the fields' 'final' state, run
earlier.
public class HandleFieldDefaults extends EclipseASTAdapter {
public boolean generateFieldDefaultsForType(EclipseNode typeNode, EclipseNode pos, AccessLevel
level, boolean makeFinal, boolean checkForTypeLevelFieldDefaults) {
if (checkForTypeLevelFieldDefaults) {
if (hasAnnotation(FieldDefaults.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return true;
}
}
if (!isClassOrEnum(typeNode)) {
pos.addError("@FieldDefaults is only supported on a class or an enum.");
return false;
}
for (EclipseNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
FieldDeclaration fieldDecl = (FieldDeclaration) field.get();
```

```
if (!filterField(fieldDecl, false)) continue;
Class<?> t = field.get().getClass();
if (t == FieldDeclaration.class) {
// There are various other things that extend FieldDeclaration that really
// aren't field declarations. Typing 'ma' in an otherwise blank class is a
// CompletionOnFieldType object (extends FieldDeclaration). If we mess with the
// modifiers of such a thing, you take away template suggestions such as
// 'main method'. See issue 411.
setFieldDefaultsForField(field, pos.get(), level, makeFinal);
}
}
return true;
}
public void setFieldDefaultsForField(EclipseNode fieldNode, ASTNode pos, AccessLevel level, boolean
makeFinal) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
if (level != null && level != AccessLevel.NONE) {
if ((field.modifiers & (ClassFileConstants.AccPublic | ClassFileConstants.AccPrivate |
ClassFileConstants.AccProtected)) == 0) {
if (!hasAnnotation(PackagePrivate.class, fieldNode)) {
if ((field.modifiers & ClassFileConstants.AccStatic) == 0) {
field.modifiers |= EclipseHandlerUtil.toEclipseModifier(level);
}
}
}
}
if (makeFinal && (field.modifiers & ClassFileConstants.AccFinal) == 0) {
if (!hasAnnotation(NonFinal.class, fieldNode)) {
if ((field.modifiers & ClassFileConstants.AccStatic) == 0) {
field.modifiers |= ClassFileConstants.AccFinal;
}
}
}
fieldNode.rebuild();
}
private static final char[] FIELD_DEFAULTS = "FieldDefaults".toCharArray();
@Override public void visitType(EclipseNode typeNode, TypeDeclaration type) {
AnnotationValues<FieldDefaults> fieldDefaults = null;
EclipseNode source = typeNode;
boolean levelIsExplicit = false;
boolean makeFinalIsExplicit = false;
FieldDefaults fd = null;
for (EclipseNode jn : typeNode.down()) {
if (jn.getKind() != Kind.ANNOTATION) continue;
Annotation ann = (Annotation) jn.get();
TypeReference typeTree = ann.type;
if (typeTree == null) continue;
if (typeTree instanceof SingleTypeReference) {
char[] t = ((SingleTypeReference) typeTree).token;
if (!Arrays.equals(t, FIELD_DEFAULTS)) continue;
} else if (typeTree instanceof QualifiedTypeReference) {
char[][] t = ((QualifiedTypeReference) typeTree).tokens;
if (!Eclipse.nameEquals(t, "lombok.experimental.FieldDefaults")) continue;
```

```
} else {
continue;
}
if (!typeMatches(FieldDefaults.class, jn, typeTree)) continue;
source = jn;
fieldDefaults = createAnnotation(FieldDefaults.class, jn);
levelIsExplicit = fieldDefaults.isExplicit("level");
makeFinalIsExplicit = fieldDefaults.isExplicit("makeFinal");
handleExperimentalFlagUsage(jn, ConfigurationKeys.FIELD_DEFAULTS_FLAG_USAGE,
"@FieldDefaults");
fd = fieldDefaults.getInstance();
if (!levelIsExplicit && !makeFinalIsExplicit) {
jn.addError("This does nothing; provide either level or makeFinal or both.");
}
if (levelIsExplicit && fd.level() == AccessLevel.NONE) {
jn.addError("AccessLevel.NONE doesn't mean anything here. Pick another value.");
levelIsExplicit = false;
}
break;
}
if (fd == null && (type.modifiers & (ClassFileConstants.AccInterface |
ClassFileConstants.AccAnnotation)) != 0) return;
boolean defaultToPrivate = levelIsExplicit ? false : Boolean.TRUE.equals(typeNode.getAst().readConfi
guration(ConfigurationKeys.FIELD_DEFAULTS_PRIVATE_EVERYWHERE));
boolean defaultToFinal = makeFinalIsExplicit ? false : Boolean.TRUE.equals(typeNode.getAst().readC
onfiguration(ConfigurationKeys.FIELD_DEFAULTS_FINAL_EVERYWHERE));
if (!defaultToPrivate && !defaultToFinal && fieldDefaults == null) return;
// Do not apply field defaults to records if set using the config system
if (fieldDefaults == null && !isClassOrEnum(typeNode)) return;
AccessLevel fdAccessLevel = (fieldDefaults != null && levelIsExplicit) ? fd.level() : defaultToPrivate ?
AccessLevel.PRIVATE : null;
boolean fdToFinal = (fieldDefaults != null && makeFinalIsExplicit) ? fd.makeFinal() : defaultToFinal;
generateFieldDefaultsForType(typeNode, source, fdAccessLevel, fdToFinal, false);
}
}
```

lombok/eclipse/handlers/HandleFieldNameConstants.java

```
/*
* Copyright (C) 2014-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ASTVisitor;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Clinit;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ExplicitConstructorCall;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.IdentifierName;
import lombok.core.handlers.HandlerUtil;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.MemberExistsResult;
import lombok.experimental.FieldNameConstants;
import lombok.spi.Provides;
@Provides
public class HandleFieldNameConstants extends EclipseAnnotationHandler<FieldNameConstants> {
private static final IdentifierName FIELDS = IdentifierName.valueOf("Fields");
public void generateFieldNameConstantsForType(EclipseNode typeNode, EclipseNode errorNode,
AccessLevel level, boolean asEnum, IdentifierName innerTypeName, boolean onlyExplicit, boolean
uppercase) {
if (!isClassEnumOrRecord(typeNode)) {
errorNode.addError("@FieldNameConstants is only supported on a class, an enum or a record.");
return;
}
if (!isStaticAllowed(typeNode)) {
errorNode.addError("@FieldNameConstants is not supported on non-static nested classes.");
```

```
return;
}
List<EclipseNode> qualified = new ArrayList<EclipseNode>();
for (EclipseNode field : typeNode.down()) {
if (fieldQualifiesForFieldNameConstantsGeneration(field, onlyExplicit)) qualified.add(field);
}
if (qualified.isEmpty()) {
errorNode.addWarning("No fields qualify for @FieldNameConstants, therefore this annotation does
nothing");
} else {
createInnerTypeFieldNameConstants(typeNode, errorNode, errorNode.get(), level, qualified, asEnum,
innerTypeName, uppercase);
}
}
private boolean fieldQualifiesForFieldNameConstantsGeneration(EclipseNode field, boolean
onlyExplicit) {
if (field.getKind() != Kind.FIELD) return false;
if (hasAnnotation(FieldNameConstants.Exclude.class, field)) return false;
if (hasAnnotation(FieldNameConstants.Include.class, field)) return true;
if (onlyExplicit) return false;
FieldDeclaration fieldDecl = (FieldDeclaration) field.get();
return filterField(fieldDecl);
}
@Override public void handle(AnnotationValues<FieldNameConstants> annotation, Annotation ast,
EclipseNode annotationNode) {
handleExperimentalFlagUsage(annotationNode,
ConfigurationKeys.FIELD_NAME_CONSTANTS_FLAG_USAGE, "@FieldNameConstants");
EclipseNode node = annotationNode.up();
FieldNameConstants annotationInstance = annotation.getInstance();
AccessLevel level = annotationInstance.level();
boolean asEnum = annotationInstance.asEnum();
boolean usingLombokv1_18_2 = annotation.isExplicit("prefix") || annotation.isExplicit("suffix") ||
node.getKind() == Kind.FIELD;
if (usingLombokv1_18_2) {
annotationNode.addError("@FieldNameConstants has been redesigned in lombok v1.18.4; please
upgrade your project dependency on lombok. See
https://projectlombok.org/features/experimental/FieldNameConstants for more information.");
return;
}
if (level == AccessLevel.NONE) {
annotationNode.addWarning("AccessLevel.NONE is not compatible with @FieldNameConstants. If you
don't want the inner type, simply remove FieldNameConstants.");
return;
}
IdentifierName innerTypeName;
try {
innerTypeName = IdentifierName.valueOf(annotationInstance.innerTypeName());
} catch(IllegalArgumentException e) {
annotationNode.addError("InnerTypeName " + annotationInstance.innerTypeName() + " is not a valid
Java identifier.");
return;
}
```

```
if (innerTypeName == null) innerTypeName = annotationNode.getAst().readConfiguration(Configuratio
nKeys.FIELD_NAME_CONSTANTS_INNER_TYPE_NAME);
if (innerTypeName == null) innerTypeName = FIELDS;
Boolean uppercase = annotationNode.getAst().readConfiguration(ConfigurationKeys.FIELD_NAME_C
ONSTANTS_UPPERCASE);
if (uppercase == null) uppercase = false;
generateFieldNameConstantsForType(node, annotationNode, level, asEnum, innerTypeName,
annotationInstance.onlyExplicitlyIncluded(), uppercase);
}
private void createInnerTypeFieldNameConstants(EclipseNode typeNode, EclipseNode errorNode,
ASTNode source, AccessLevel level, List<EclipseNode> fields, boolean asEnum, IdentifierName
innerTypeName, boolean uppercase) {
if (fields.isEmpty()) return;
ASTVisitor generatedByVisitor = new SetGeneratedByVisitor(source);
TypeDeclaration parent = (TypeDeclaration) typeNode.get();
EclipseNode fieldsType = findInnerClass(typeNode, innerTypeName.getName());
boolean genConstr = false, genClinit = false;
char[] name = innerTypeName.getCharArray();
if (fieldsType == null) {
TypeDeclaration generatedInnerType = new TypeDeclaration(parent.compilationResult);
generatedInnerType.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
generatedInnerType.modifiers = toEclipseModifier(level) | (asEnum ? ClassFileConstants.AccEnum :
(ClassFileConstants.AccStatic | ClassFileConstants.AccFinal));
generatedInnerType.name = name;
fieldsType = injectType(typeNode, generatedInnerType);
genConstr = true;
genClinit = asEnum;
generatedInnerType.traverse(generatedByVisitor, ((TypeDeclaration) typeNode.get()).scope);
} else {
TypeDeclaration builderTypeDeclaration = (TypeDeclaration) fieldsType.get();
if (asEnum && (builderTypeDeclaration.modifiers & ClassFileConstants.AccEnum) == 0) {
errorNode.addError("Existing " + innerTypeName + " must be declared as an 'enum'.");
return;
}
if (!asEnum && (builderTypeDeclaration.modifiers & ClassFileConstants.AccStatic) == 0) {
errorNode.addError("Existing " + innerTypeName + " must be declared as a 'static class'.");
return;
}
genConstr = constructorExists(fieldsType) == MemberExistsResult.NOT_EXISTS;
}
if (genConstr) {
ConstructorDeclaration constructor = new ConstructorDeclaration(parent.compilationResult);
constructor.selector = name;
constructor.modifiers = ClassFileConstants.AccPrivate;
ExplicitConstructorCall superCall = new ExplicitConstructorCall(0);
superCall.sourceStart = source.sourceStart;
superCall.sourceEnd = source.sourceEnd;
superCall.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
constructor.constructorCall = superCall;
if (!asEnum) constructor.statements = new Statement[0];
injectMethod(fieldsType, constructor);
}
Clinit cli = null;
```

```
if (genClinit) {
cli = new Clinit(parent.compilationResult);
injectMethod(fieldsType, cli);
}
for (EclipseNode fieldNode : fields) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
char[] fName = field.name;
if (uppercase) fName = HandlerUtil.camelCaseToConstant(new String(fName)).toCharArray();
if (fieldExists(new String(fName), fieldsType) != MemberExistsResult.NOT_EXISTS) continue;
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
FieldDeclaration constantField = new FieldDeclaration(fName, pS, pE);
constantField.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
if (asEnum) {
AllocationExpression ac = new AllocationExpression();
ac.enumConstant = constantField;
ac.sourceStart = source.sourceStart;
ac.sourceEnd = source.sourceEnd;
constantField.initialization = ac;
constantField.modifiers = 0;
((TypeDeclaration) fieldsType.get()).enumConstantsCounter++;
} else {
constantField.type = new QualifiedTypeReference(TypeConstants.JAVA_LANG_STRING, new long[]
{p, p, p});
constantField.initialization = new StringLiteral(field.name, pS, pE, 0);
constantField.modifiers = ClassFileConstants.AccPublic | ClassFileConstants.AccStatic |
ClassFileConstants.AccFinal;
}
injectField(fieldsType, constantField);
constantField.traverse(generatedByVisitor, ((TypeDeclaration) fieldsType.get()).initializerScope);
}
if (genClinit) {
cli.traverse(generatedByVisitor, ((TypeDeclaration) fieldsType.get()).scope);
}
}
}
```

lombok/eclipse/handlers/HandleGetter.java


```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
```

*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.experimental.Accessors;
import lombok.experimental.Delegate;
import lombok.spi.Provides;
import lombok.Getter;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.agent.PatchDelegate;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.ArrayTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.BinaryExpression;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.CastExpression;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
```

```
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.SynchronizedStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
/**
* Handles the {@code lombok.Getter} annotation for eclipse.
*/
@Provides
public class HandleGetter extends EclipseAnnotationHandler<Getter> {
private static final Annotation[] EMPTY_ANNOTATIONS_ARRAY = new Annotation[0];
private static final String GETTER_NODE_NOT_SUPPORTED_ERR = "@Getter is only supported on
a class, an enum, or a field.";
public boolean generateGetterForType(EclipseNode typeNode, EclipseNode pos, AccessLevel level,
boolean checkForTypeLevelGetter, List<Annotation> onMethod) {
if (checkForTypeLevelGetter) {
if (hasAnnotation(Getter.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return true;
}
}
if (!isClassOrEnum(typeNode)) {
pos.addError(GETTER_NODE_NOT_SUPPORTED_ERR);
return false;
}
for (EclipseNode field : typeNode.down()) {
if (fieldQualifiesForGetterGeneration(field)) generateGetterForField(field, pos.get(), level, false,
onMethod);
}
return true;
}
public static boolean fieldQualifiesForGetterGeneration(EclipseNode field) {
if (field.getKind() != Kind.FIELD) return false;
FieldDeclaration fieldDecl = (FieldDeclaration) field.get();
return filterField(fieldDecl);
}
/**
* Generates a getter on the stated field.
*
* Used by {@link HandleData}.
*
* The difference between this call and the handle method is as follows:
*
* If there is a {@code lombok.Getter} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
* If not, the getter is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*/
public void generateGetterForField(EclipseNode fieldNode, ASTNode pos, AccessLevel level, boolean
lazy, List<Annotation> onMethod) {
```

```
if (hasAnnotation(Getter.class, fieldNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
createGetterForField(level, fieldNode, fieldNode, pos, false, lazy, onMethod);
}
public void handle(AnnotationValues<Getter> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.GETTER_FLAG_USAGE, "@Getter");
EclipseNode node = annotationNode.up();
Getter annotationInstance = annotation.getInstance();
AccessLevel level = annotationInstance.value();
boolean lazy = annotationInstance.lazy();
if (lazy) handleFlagUsage(annotationNode, ConfigurationKeys.GETTER_LAZY_FLAG_USAGE,
"@Getter(lazy=true)");
if (level == AccessLevel.NONE) {
if (lazy) annotationNode.addWarning("'lazy' does not work with AccessLevel.NONE.");
return;
}
if (node == null) return;
List<Annotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@Getter(onMethod", annotationNode);
if (!onMethod.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@Getter(onMethod=...)");
}
switch (node.getKind()) {
case FIELD:
createGetterForFields(level, annotationNode.upFromAnnotationToFields(), annotationNode,
annotationNode.get(), true, lazy, onMethod);
break;
case TYPE:
if (lazy) annotationNode.addError("'lazy' is not supported for @Getter on a type.");
generateGetterForType(node, annotationNode, level, false, onMethod);
break;
}
}
public void createGetterForFields(AccessLevel level, Collection<EclipseNode> fieldNodes,
EclipseNode errorNode, ASTNode source, boolean whineIfExists, boolean lazy, List<Annotation>
onMethod) {
for (EclipseNode fieldNode : fieldNodes) {
createGetterForField(level, fieldNode, errorNode, source, whineIfExists, lazy, onMethod);
}
}
public void createGetterForField(AccessLevel level,
EclipseNode fieldNode, EclipseNode errorNode, ASTNode source, boolean whineIfExists, boolean
lazy, List<Annotation> onMethod) {
if (fieldNode.getKind() != Kind.FIELD || fieldNode.isEnumMember()) {
errorNode.addError(GETTER_NODE_NOT_SUPPORTED_ERR);
return;
}
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
if (lazy) {
```

```
if ((field.modifiers & ClassFileConstants.AccPrivate) == 0 || (field.modifiers &
ClassFileConstants.AccFinal) == 0) {
errorNode.addError("'lazy' requires the field to be private and final.");
return;
}
if ((field.modifiers & ClassFileConstants.AccTransient) != 0) {
errorNode.addError("'lazy' is not supported on transient fields.");
return;
}
if (field.initialization == null) {
errorNode.addError("'lazy' requires field initialization.");
return;
}
}
TypeReference fieldType = copyType(field.type, source);
boolean isBoolean = isBoolean(fieldType);
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
String getterName = toGetterName(fieldNode, isBoolean, accessors);
if (getterName == null) {
errorNode.addWarning("Not generating getter for this field: It does not fit your @Accessors prefix list.");
return;
}
int modifier = toEclipseModifier(level) | (field.modifiers & ClassFileConstants.AccStatic);
for (String altName : toAllGetterNames(fieldNode, isBoolean, accessors)) {
switch (methodExists(altName, fieldNode, false, 0)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String altNameExpl = "";
if (!altName.equals(getterName)) altNameExpl = String.format(" (%s)", altName);
errorNode.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", getterName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
//continue scanning the other alt names.
}
}
MethodDeclaration method = createGetter((TypeDeclaration) fieldNode.up().get(), fieldNode,
getterName, modifier, source, lazy, onMethod);
injectMethod(fieldNode.up(), method);
}
public static Annotation[] findDelegatesAndMarkAsHandled(EclipseNode fieldNode) {
List<Annotation> delegates = new ArrayList<Annotation>();
for (EclipseNode child : fieldNode.down()) {
if (annotationTypeMatches(Delegate.class, child)) {
Annotation delegate = (Annotation)child.get();
PatchDelegate.markHandled(delegate);
delegates.add(delegate);
}
```

```
}
return delegates.toArray(EMPTY_ANNOTATIONS_ARRAY);
}
public MethodDeclaration createGetter(TypeDeclaration parent, EclipseNode fieldNode, String name,
int modifier, ASTNode source, boolean lazy, List<Annotation> onMethod) {
// Remember the type; lazy will change it;
TypeReference returnType = copyType(((FieldDeclaration) fieldNode.get()).type, source);
Statement[] statements;
boolean addSuppressWarningsUnchecked = false;
if (lazy) {
statements = createLazyGetterBody(source, fieldNode);
addSuppressWarningsUnchecked = true;
} else {
statements = createSimpleGetterBody(source, fieldNode);
}
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
MethodDeclaration method = new MethodDeclaration(parent.compilationResult);
if (shouldMakeFinal(fieldNode, accessors)) modifier |= ClassFileConstants.AccFinal;
method.modifiers = modifier;
method.returnType = returnType;
method.annotations = null;
method.arguments = null;
method.selector = name.toCharArray();
method.binding = null;
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
method.statements = statements;
EclipseHandlerUtil.registerCreatedLazyGetter((FieldDeclaration) fieldNode.get(), method.selector,
returnType);
/* Generate annotations that must be put on the generated method, and attach them. */ {
Annotation[] deprecated = null, checkerFramework = null;
if (isFieldDeprecated(fieldNode)) deprecated = new Annotation[] {
generateDeprecatedAnnotation(source) };
if (fieldNode.isFinal()) {
if (getCheckerFrameworkVersion(fieldNode).generatePure()) checkerFramework = new Annotation[] {
generateNamedAnnotation(source, CheckerFrameworkVersion.NAME__PURE) };
} else {
if (getCheckerFrameworkVersion(fieldNode).generateSideEffectFree()) checkerFramework = new
Annotation[] { generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE) };
}
method.annotations = copyAnnotations(source,
onMethod.toArray(new Annotation[0]),
findCopyableAnnotations(fieldNode),
findDelegatesAndMarkAsHandled(fieldNode),
checkerFramework,
deprecated);
}
if (addSuppressWarningsUnchecked) {
List<Expression> suppressions = new ArrayList<Expression>(2);
```

```
if (!Boolean.FALSE.equals(fieldNode.getAst().readConfiguration(ConfigurationKeys.ADD_SUPPRESS
WARNINGS_ANNOTATIONS))) {
suppressions.add(new StringLiteral(ALL, 0, 0, 0));
}
suppressions.add(new StringLiteral(UNCHECKED, 0, 0, 0));
ArrayInitializer arr = new ArrayInitializer();
arr.expressions = suppressions.toArray(new Expression[0]);
method.annotations = addAnnotation(source, method.annotations,
TypeConstants.JAVA_LANG_SUPPRESSWARNINGS, arr);
}
method.traverse(new SetGeneratedByVisitor(source), parent.scope);
copyJavadoc(fieldNode, method, CopyJavadoc.GETTER);
return method;
}
public Statement[] createSimpleGetterBody(ASTNode source, EclipseNode fieldNode) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
Expression fieldRef = createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source);
Statement returnStatement = new ReturnStatement(fieldRef, field.sourceStart, field.sourceEnd);
return new Statement[] {returnStatement};
}
private static final char[][] AR = fromQualifiedName("java.util.concurrent.atomic.AtomicReference");
public static final java.util.Map<String, char[][]> TYPE_MAP;
static {
Map<String, char[][]> m = new HashMap<String, char[][]>();
m.put("int", fromQualifiedName("java.lang.Integer"));
m.put("double", fromQualifiedName("java.lang.Double"));
m.put("float", fromQualifiedName("java.lang.Float"));
m.put("short", fromQualifiedName("java.lang.Short"));
m.put("byte", fromQualifiedName("java.lang.Byte"));
m.put("long", fromQualifiedName("java.lang.Long"));
m.put("boolean", fromQualifiedName("java.lang.Boolean"));
m.put("char", fromQualifiedName("java.lang.Character"));
TYPE_MAP = Collections.unmodifiableMap(m);
}
private static char[] valueName = "$value".toCharArray();
private static char[] actualValueName = "actualValue".toCharArray();
private static final int PARENTHESIZED = (1 << ASTNode.ParenthesizedSHIFT) &
ASTNode.ParenthesizedMASK;
public Statement[] createLazyGetterBody(ASTNode source, EclipseNode fieldNode) {
/*
java.lang.Object value = this.fieldName.get();
if (value == null) {
synchronized (this.fieldName) {
value = this.fieldName.get();
if (value == null) {
final RawValueType actualValue = INITIALIZER_EXPRESSION;
[IF PRIMITIVE]
value = actualValue;
[ELSE]
value = actualValue == null ? this.fieldName : actualValue;
[END IF]
this.fieldName.set(value);
}
```

```
}
}
[IF PRIMITIVE]
return (BoxedValueType) value;
[ELSE]
return (BoxedValueType) (value == this.fieldName ? null : value);
[END IF]
*/
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long)pS << 32 | pE;
TypeReference rawComponentType = copyType(field.type, source);
TypeReference boxedComponentType = null;
boolean isPrimitive = false;
if (field.type instanceof SingleTypeReference && !(field.type instanceof ArrayTypeReference)) {
char[][] newType = TYPE_MAP.get(new String(((SingleTypeReference)field.type).token));
if (newType != null) {
boxedComponentType = new QualifiedTypeReference(newType, poss(source, 3));
isPrimitive = true;
}
}
if (boxedComponentType == null) boxedComponentType = copyType(field.type, source);
boxedComponentType.sourceStart = pS; boxedComponentType.sourceEnd =
boxedComponentType.statementEnd = pE;
Statement[] statements = new Statement[3];
/* java.lang.Object value = this.fieldName.get(); */ {
LocalDeclaration valueDecl = new LocalDeclaration(valueName, pS, pE);
valueDecl.type = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, poss(source,
3));
valueDecl.type.sourceStart = pS; valueDecl.type.sourceEnd = valueDecl.type.statementEnd = pE;
MessageSend getter = new MessageSend();
getter.sourceStart = pS; getter.statementEnd = getter.sourceEnd = pE;
getter.selector = new char[] {'g', 'e', 't'};
getter.receiver = createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source);
valueDecl.initialization = getter;
statements[0] = valueDecl;
}
/*
if (value == null) {
synchronized (this.fieldName) {
value = this.fieldName.get();
if (value == null) {
final ValueType actualValue = INITIALIZER_EXPRESSION;
[IF PRIMITIVE]
value = actualValue;
[ELSE]
value = actualValue == null ? this.fieldName : actualValue;
[END IF]
this.fieldName.set(value);
}
}
}
*/ {
```

```
EqualExpression cond = new EqualExpression(
new SingleNameReference(valueName, p), new NullLiteral(pS, pE),
BinaryExpression.EQUAL_EQUAL);
Block then = new Block(0);
Expression lock = createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source);
Block inner = new Block(0);
inner.statements = new Statement[2];
/* value = this.fieldName.get(); */ {
MessageSend getter = new MessageSend();
getter.sourceStart = pS; getter.sourceEnd = getter.statementEnd = pE;
getter.selector = new char[] {'g', 'e', 't'};
getter.receiver = createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source);
Assignment assign = new Assignment(new SingleNameReference(valueName, p), getter, pE);
assign.sourceStart = pS; assign.statementEnd = assign.sourceEnd = pE;
inner.statements[0] = assign;
}
/* if (value == null) */ {
EqualExpression innerCond = new EqualExpression(
new SingleNameReference(valueName, p), new NullLiteral(pS, pE),
BinaryExpression.EQUAL_EQUAL);
innerCond.sourceStart = pS; innerCond.sourceEnd = innerCond.statementEnd = pE;
Block innerThen = new Block(0);
innerThen.statements = new Statement[3];
/* final ValueType actualValue = INITIALIZER_EXPRESSION */ {
LocalDeclaration actualValueDecl = new LocalDeclaration(actualValueName, pS, pE);
actualValueDecl.type = rawComponentType;
actualValueDecl.type.sourceStart = pS; actualValueDecl.type.sourceEnd =
actualValueDecl.type.statementEnd = pE;
actualValueDecl.initialization = field.initialization;
actualValueDecl.modifiers = ClassFileConstants.AccFinal;
innerThen.statements[0] = actualValueDecl;
}
/* [IF PRIMITIVE] value = actualValue; */ {
if (isPrimitive) {
Assignment innerAssign = new Assignment(new SingleNameReference(valueName, p), new
SingleNameReference(actualValueName, p), pE);
innerAssign.sourceStart = pS; innerAssign.statementEnd = innerAssign.sourceEnd = pE;
innerThen.statements[1] = innerAssign;
}
}
/* [ELSE] value = actualValue == null ? this.fieldName : actualValue; */ {
if (!isPrimitive) {
EqualExpression avIsNull = new EqualExpression(
new SingleNameReference(actualValueName, p), new NullLiteral(pS, pE),
BinaryExpression.EQUAL_EQUAL);
avIsNull.sourceStart = pS; avIsNull.sourceEnd = avIsNull.statementEnd = pE;
Expression fieldRef = createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source);
ConditionalExpression ternary = new ConditionalExpression(avIsNull, fieldRef, new
SingleNameReference(actualValueName, p));
ternary.sourceStart = pS; ternary.sourceEnd = ternary.statementEnd = pE;
Assignment innerAssign = new Assignment(new SingleNameReference(valueName, p), ternary, pE);
innerAssign.sourceStart = pS; innerAssign.statementEnd = innerAssign.sourceEnd = pE;
innerThen.statements[1] = innerAssign;
```

```
}
}
/* this.fieldName.set(value); */ {
MessageSend setter = new MessageSend();
setter.sourceStart = pS; setter.sourceEnd = setter.statementEnd = pE;
setter.receiver = createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source);
setter.selector = new char[] { 's', 'e', 't' };
setter.arguments = new Expression[] {
new SingleNameReference(valueName, p)};
innerThen.statements[2] = setter;
}
IfStatement innerIf = new IfStatement(innerCond, innerThen, pS, pE);
inner.statements[1] = innerIf;
}
SynchronizedStatement sync = new SynchronizedStatement(lock, inner, pS, pE);
then.statements = new Statement[] {sync};
IfStatement ifStatement = new IfStatement(cond, then, pS, pE);
statements[1] = ifStatement;
}
/* [IF PRIMITIVE] return (BoxedValueType)value; */ {
if (isPrimitive) {
CastExpression cast = makeCastExpression(new SingleNameReference(valueName, p),
boxedComponentType, source);
statements[2] = new ReturnStatement(cast, pS, pE);
}
}
/* [ELSE] return (BoxedValueType)(value == this.fieldName ? null : value); */ {
if (!isPrimitive) {
EqualExpression vIsThisFieldName = new EqualExpression(
new SingleNameReference(valueName, p), createFieldAccessor(fieldNode,
FieldAccess.ALWAYS_FIELD, source),
BinaryExpression.EQUAL_EQUAL);
vIsThisFieldName.sourceStart = pS; vIsThisFieldName.sourceEnd = vIsThisFieldName.statementEnd
= pE;
ConditionalExpression ternary = new ConditionalExpression(vIsThisFieldName, new NullLiteral(pS,
pE), new SingleNameReference(valueName, p));
ternary.sourceStart = pS; ternary.sourceEnd = ternary.statementEnd = pE;
ternary.bits |= PARENTHESIZED;
CastExpression cast = makeCastExpression(ternary, boxedComponentType, source);
statements[2] = new ReturnStatement(cast, pS, pE);
}
}
// update the field type and init last
/* private final java.util.concurrent.atomic.AtomicReference<java.lang.Object> fieldName = new
java.util.concurrent.atomic.AtomicReference<java.lang.Object>(); */ {
TypeReference innerType = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT,
poss(source, 3));
TypeReference[][] typeParams = new TypeReference[5][];
typeParams[4] = new TypeReference[] {innerType};
TypeReference type = new ParameterizedQualifiedTypeReference(AR, typeParams, 0, poss(source,
5));
// Some magic here
type.sourceStart = -1; type.sourceEnd = -2;
```

```
field.type = type;
AllocationExpression init = new AllocationExpression();
// Some magic here
init.sourceStart = field.initialization.sourceStart; init.sourceEnd = init.statementEnd =
field.initialization.sourceEnd;
init.type = copyType(type, source);
field.initialization = init;
}
return statements;
}
}
```

lombok/eclipse/handlers/HandleHelper.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ASTVisitor;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
```

```
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SwitchStatement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.BlockScope;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.Helper;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.Cleanup} annotation for eclipse.
*/
@Provides
public class HandleHelper extends EclipseAnnotationHandler<Helper> {
private Statement[] getStatementsFromAstNode(ASTNode node) {
if (node instanceof Block) return ((Block) node).statements;
if (node instanceof AbstractMethodDeclaration) return ((AbstractMethodDeclaration) node).statements;
if (node instanceof SwitchStatement) return ((SwitchStatement) node).statements;
return null;
}
private void setStatementsOfAstNode(ASTNode node, Statement[] statements) {
if (node instanceof Block) ((Block) node).statements = statements;
else if (node instanceof AbstractMethodDeclaration) ((AbstractMethodDeclaration) node).statements =
statements;
else if (node instanceof SwitchStatement) ((SwitchStatement) node).statements = statements;
else throw new IllegalArgumentException("Can't set statements on node type: " + node.getClass());
}
@Override public void handle(AnnotationValues<Helper> annotation, Annotation ast, EclipseNode
annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.HELPER_FLAG_USAGE,
"@Helper");
EclipseNode annotatedType = annotationNode.up();
EclipseNode containingBlock = annotatedType == null ? null : annotatedType.directUp();
Statement[] origStatements = getStatementsFromAstNode(containingBlock == null ? null :
containingBlock.get());
if (annotatedType == null || annotatedType.getKind() != Kind.TYPE || origStatements == null) {
annotationNode.addError("@Helper is legal only on method-local classes.");
return;
}
TypeDeclaration annotatedType_ = (TypeDeclaration) annotatedType.get();
int indexOfType = -1;
for (int i = 0; i < origStatements.length; i++) {
if (origStatements[i] == annotatedType_) {
indexOfType = i;
break;
}
}
final List<String> knownMethodNames = new ArrayList<String>();
```

```
for (AbstractMethodDeclaration methodOfHelper : annotatedType_.methods) {
if (!(methodOfHelper instanceof MethodDeclaration)) continue;
char[] name = methodOfHelper.selector;
if (name != null && name.length > 0 && name[0] != '<') knownMethodNames.add(new String(name));
}
Collections.sort(knownMethodNames);
final String[] knownMethodNames_ = knownMethodNames.toArray(new String[0]);
final char[] helperName = new char[annotatedType_.name.length + 1];
final boolean[] helperUsed = new boolean[1];
helperName[0] = '$';
System.arraycopy(annotatedType_.name, 0, helperName, 1, helperName.length - 1);
ASTVisitor visitor = new ASTVisitor() {
@Override public boolean visit(MessageSend messageSend, BlockScope scope) {
if (messageSend.receiver instanceof ThisReference) {
if ((((ThisReference) messageSend.receiver).bits & ASTNode.IsImplicitThis) == 0) return true;
} else if (messageSend.receiver != null) return true;
char[] name = messageSend.selector;
if (name == null || name.length == 0 || name[0] == '<') return true;
String n = new String(name);
if (Arrays.binarySearch(knownMethodNames_, n) < 0) return true;
messageSend.receiver = new SingleNameReference(helperName,
messageSend.nameSourcePosition);
helperUsed[0] = true;
return true;
}
};
for (int i = indexOfType + 1; i < origStatements.length; i++) {
origStatements[i].traverse(visitor, null);
}
if (!helperUsed[0]) {
annotationNode.addWarning("No methods of this helper class are ever used.");
return;
}
Statement[] newStatements = new Statement[origStatements.length + 1];
System.arraycopy(origStatements, 0, newStatements, 0, indexOfType + 1);
System.arraycopy(origStatements, indexOfType + 1, newStatements, indexOfType + 2,
origStatements.length - indexOfType - 1);
LocalDeclaration decl = new LocalDeclaration(helperName, 0, 0);
decl.modifiers |= ClassFileConstants.AccFinal;
AllocationExpression alloc = new AllocationExpression();
alloc.type = new SingleTypeReference(annotatedType_.name, 0L);
decl.initialization = alloc;
decl.type = new SingleTypeReference(annotatedType_.name, 0L);
SetGeneratedByVisitor sgbvVisitor = new SetGeneratedByVisitor(annotationNode.get());
decl.traverse(sgbvVisitor, null);
newStatements[indexOfType + 1] = decl;
setStatementsOfAstNode(containingBlock.get(), newStatements);
}
}
```

lombok/eclipse/handlers/HandleJacksonized.java

/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.ClassLiteralAccess;
import org.eclipse.jdt.internal.compiler.ast.MemberValuePair;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import lombok.Builder;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.AST.Kind;
import lombok.core.handlers.HandlerUtil;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.SuperBuilder;
import lombok.extern.jackson.Jacksonized;
import lombok.spi.Provides;
/**
* This (ecj) handler deals with {@code @Jacksonized} modifying the (already
* generated) {@code @Builder} or {@code @SuperBuilder} to conform to Jackson's
* needs for builders.

```
*/
@Provides
@HandlerPriority(-512) // Above Handle(Super)Builder's level (builders must be already generated).
public class HandleJacksonized extends EclipseAnnotationHandler<Jacksonized> {
private static final char[][] JSON_POJO_BUILDER_ANNOTATION =
Eclipse.fromQualifiedName("com.fasterxml.jackson.databind.annotation.JsonPOJOBuilder");
private static final char[][] JSON_DESERIALIZE_ANNOTATION =
Eclipse.fromQualifiedName("com.fasterxml.jackson.databind.annotation.JsonDeserialize");
@Override public void handle(AnnotationValues<Jacksonized> annotation, Annotation ast,
EclipseNode annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.JACKSONIZED_FLAG_USAGE,
"@Jacksonized");
EclipseNode annotatedNode = annotationNode.up();
EclipseNode tdNode;
if (annotatedNode.getKind() != Kind.TYPE)
tdNode = annotatedNode.up(); // @Jacksonized on a constructor or a static factory method.
else
tdNode = annotatedNode; // @Jacksonized on the class.
TypeDeclaration td = (TypeDeclaration) tdNode.get();
EclipseNode builderAnnotationNode = findAnnotation(Builder.class, annotatedNode);
EclipseNode superBuilderAnnotationNode = findAnnotation(SuperBuilder.class, annotatedNode);
if (builderAnnotationNode == null && superBuilderAnnotationNode == null) {
annotationNode.addWarning("@Jacksonized requires @Builder or @SuperBuilder for it to mean
anything.");
return;
}
if (builderAnnotationNode != null && superBuilderAnnotationNode != null) {
annotationNode.addError("@Jacksonized cannot process both @Builder and @SuperBuilder on the
same class.");
return;
}
boolean isAbstract = (td.modifiers & ClassFileConstants.AccAbstract) != 0;
if (isAbstract) {
annotationNode.addError("Builders on abstract classes cannot be @Jacksonized (the builder would
never be used).");
return;
}
AnnotationValues<Builder> builderAnnotation = builderAnnotationNode != null ?
createAnnotation(Builder.class, builderAnnotationNode) : null;
AnnotationValues<SuperBuilder> superBuilderAnnotation = superBuilderAnnotationNode != null ?
createAnnotation(SuperBuilder.class, superBuilderAnnotationNode) : null;
String setPrefix = builderAnnotation != null ? builderAnnotation.getInstance().setterPrefix() :
superBuilderAnnotation.getInstance().setterPrefix();
String buildMethodName = builderAnnotation != null ?
builderAnnotation.getInstance().buildMethodName() :
superBuilderAnnotation.getInstance().buildMethodName();
// Now lets find the generated builder class.
EclipseNode builderClassNode = null;
TypeDeclaration builderClass = null;
String builderClassName = getBuilderClassName(ast, annotationNode, annotatedNode, td,
builderAnnotation);
for (EclipseNode member : tdNode.down()) {
ASTNode astNode = member.get();
```

```
if (astNode instanceof TypeDeclaration && Arrays.equals(((TypeDeclaration)astNode).name,
builderClassName.toCharArray())) {
builderClassNode = member;
builderClass = (TypeDeclaration) astNode;
break;
}
}
if (builderClass == null) {
annotationNode.addError("Could not find @(Super)Builder's generated builder class for @Jacksonized
processing. If there are other compiler errors, fix them first.");
return;
}
// Insert @JsonDeserialize on annotated class.
if (hasAnnotation("com.fasterxml.jackson.databind.annotation.JsonDeserialize", tdNode)) {
annotationNode.addError("@JsonDeserialize already exists on class. Either delete @JsonDeserialize,
or remove @Jacksonized and manually configure Jackson.");
return;
}
long p = (long) ast.sourceStart << 32 | ast.sourceEnd;
TypeReference builderClassExpression = namePlusTypeParamsToTypeReference(builderClassNode,
null, p);
ClassLiteralAccess builderClassLiteralAccess = new ClassLiteralAccess(td.sourceEnd,
builderClassExpression);
MemberValuePair builderMvp = new MemberValuePair("builder".toCharArray(), td.sourceStart,
td.sourceEnd, builderClassLiteralAccess);
td.annotations = addAnnotation(td, td.annotations, JSON_DESERIALIZE_ANNOTATION, builderMvp);
// Copy annotations from the class to the builder class.
Annotation[] copyableAnnotations = findJacksonAnnotationsOnClass(td, tdNode);
builderClass.annotations = copyAnnotations(builderClass, builderClass.annotations,
copyableAnnotations);
// Insert @JsonPOJOBuilder on the builder class.
StringLiteral withPrefixLiteral = new StringLiteral(setPrefix.toCharArray(), builderClass.sourceStart,
builderClass.sourceEnd, 0);
MemberValuePair withPrefixMvp = new MemberValuePair("withPrefix".toCharArray(),
builderClass.sourceStart, builderClass.sourceEnd, withPrefixLiteral);
StringLiteral buildMethodNameLiteral = new StringLiteral(buildMethodName.toCharArray(),
builderClass.sourceStart, builderClass.sourceEnd, 0);
MemberValuePair buildMethodNameMvp = new MemberValuePair("buildMethodName".toCharArray(),
builderClass.sourceStart, builderClass.sourceEnd, buildMethodNameLiteral);
builderClass.annotations = addAnnotation(builderClass, builderClass.annotations,
JSON_POJO_BUILDER_ANNOTATION, withPrefixMvp, buildMethodNameMvp);
// @SuperBuilder? Make it package-private!
if (superBuilderAnnotationNode != null)
builderClass.modifiers = builderClass.modifiers & ~ClassFileConstants.AccPrivate;
}
private String getBuilderClassName(Annotation ast, EclipseNode annotationNode, EclipseNode
annotatedNode, TypeDeclaration td, AnnotationValues<Builder> builderAnnotation) {
String builderClassName = builderAnnotation != null ?
builderAnnotation.getInstance().builderClassName() : null;
if (builderClassName == null || builderClassName.isEmpty()) {
builderClassName =
annotationNode.getAst().readConfiguration(ConfigurationKeys.BUILDER_CLASS_NAME);
if (builderClassName == null || builderClassName.isEmpty())
```

```
builderClassName = "*Builder";
MethodDeclaration fillParametersFrom = annotatedNode.get() instanceof MethodDeclaration ?
(MethodDeclaration) annotatedNode.get() : null;
char[] replacement;
if (fillParametersFrom != null) {
// @Builder on a method: Use name of return type for builder class name.
replacement = HandleBuilder.returnTypeToBuilderClassName(annotationNode, fillParametersFrom,
fillParametersFrom.typeParameters);
} else {
// @Builder on class or constructor: Use the class name.
replacement = td.name;
}
builderClassName = builderClassName.replace("*", new String(replacement));
}
if (builderAnnotation == null)
builderClassName += "Impl"; // For @SuperBuilder, all Jackson annotations must be put on the
BuilderImpl class.
return builderClassName;
}
private static final Annotation[] EMPTY_ANNOTATIONS_ARRAY = new Annotation[0];
private static Annotation[] findJacksonAnnotationsOnClass(TypeDeclaration td, EclipseNode node) {
if (td.annotations == null) return EMPTY_ANNOTATIONS_ARRAY;
List<Annotation> result = new ArrayList<Annotation>();
for (Annotation annotation : td.annotations) {
TypeReference typeRef = annotation.type;
if (typeRef != null && typeRef.getTypeName() != null) {
for (String bn : HandlerUtil.JACKSON_COPY_TO_BUILDER_ANNOTATIONS) {
if (typeMatches(bn, node, typeRef)) {
result.add(annotation);
break;
}
}
}
}
return result.toArray(EMPTY_ANNOTATIONS_ARRAY);
}
}
```

lombok/eclipse/handlers/HandleLocked.java

```
/*
* Copyright (C) 2021-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.eclipse.EcjAugments.ASTNode_handled;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import lombok.Locked;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.DeferUntilPostDiet;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.Locked} annotation for eclipse.
*/
@Provides
@DeferUntilPostDiet
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the
statements generated by it.
public class HandleLocked extends EclipseAnnotationHandler<Locked> {
private static final String ANNOTATION_NAME = "@Locked";
private static final char[][] LOCK_TYPE_CLASS = new char[][] { TypeConstants.JAVA,
TypeConstants.UTIL,
"concurrent".toCharArray(), "locks".toCharArray(), "Lock".toCharArray() };
private static final char[][] LOCK_IMPL_CLASS = new char[][] { TypeConstants.JAVA,
TypeConstants.UTIL,
"concurrent".toCharArray(), "locks".toCharArray(), "ReentrantLock".toCharArray() };
@Override public void handle(AnnotationValues<Locked> annotation, Annotation source, EclipseNode
annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.handle(annotationValue, source, annotationNode, ANNOTATION_NAME,
LOCK_TYPE_CLASS, LOCK_IMPL_CLASS);
}
@Override public void preHandle(AnnotationValues<Locked> annotation, Annotation source,
EclipseNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.preHandle(annotationValue, LOCK_TYPE_CLASS, LOCK_IMPL_CLASS,
annotationNode);
if (hasParsedBody(getAnnotatedMethod(annotationNode))) {
// This method has a body in diet mode, so we have to handle it now.
handle(annotation, source, annotationNode);
ASTNode_handled.set(source, true);
}
```

}
}

lombok/eclipse/handlers/HandleLockedRead.java

```
/*
* Copyright (C) 2021-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.eclipse.EcjAugments.ASTNode_handled;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import lombok.Locked;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.DeferUntilPostDiet;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.Locked.Read} annotation for eclipse.
*/
@Provides
@DeferUntilPostDiet
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the
statements generated by it.
public class HandleLockedRead extends EclipseAnnotationHandler<Locked.Read> {
private static final char[] LOCK_METHOD = "readLock".toCharArray();
private static final String ANNOTATION_NAME = "@Locked.Read";
private static final char[][] LOCK_TYPE_CLASS = new char[][] { TypeConstants.JAVA,
TypeConstants.UTIL,
"concurrent".toCharArray(), "locks".toCharArray(), "ReadWriteLock".toCharArray() };
```

```
private static final char[][] LOCK_IMPL_CLASS = new char[][] { TypeConstants.JAVA,
TypeConstants.UTIL,
"concurrent".toCharArray(), "locks".toCharArray(), "ReentrantReadWriteLock".toCharArray() };
@Override public void handle(AnnotationValues<Locked.Read> annotation, Annotation source,
EclipseNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.handle(annotationValue, source, annotationNode, ANNOTATION_NAME,
LOCK_TYPE_CLASS, LOCK_IMPL_CLASS, LOCK_METHOD);
}
@Override public void preHandle(AnnotationValues<Locked.Read> annotation, Annotation source,
EclipseNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.preHandle(annotationValue, LOCK_TYPE_CLASS, LOCK_IMPL_CLASS,
annotationNode);
if (hasParsedBody(getAnnotatedMethod(annotationNode))) {
// This method has a body in diet mode, so we have to handle it now.
handle(annotation, source, annotationNode);
ASTNode_handled.set(source, true);
}
}
}
```

lombok/eclipse/handlers/HandleLockedUtil.java


```
/*
* Copyright (C) 2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/* Copyright (C) 2021-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
```

```
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import lombok.ConfigurationKeys;
import lombok.core.AST;
import lombok.eclipse.EclipseNode;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Reference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TryStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import java.lang.reflect.Modifier;
import java.util.Arrays;
import java.util.concurrent.atomic.AtomicBoolean;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
/**
* Container for static utility methods used by the Locked[.Read/Write] annotations for eclipse.
*/
public final class HandleLockedUtil {
private static final char[] INSTANCE_LOCK_NAME = "$lock".toCharArray();
private static final char[] STATIC_LOCK_NAME = "$LOCK".toCharArray();
private static final char[] LOCK_METHOD = "lock".toCharArray();
private static final char[] UNLOCK_METHOD = "unlock".toCharArray();
private HandleLockedUtil() {
//Prevent instantiation
}
public static void preHandle(String annotationValue, char[][] lockTypeClass, char[][] lockImplClass,
EclipseNode annotationNode) {
EclipseNode methodNode = annotationNode.up();
```

```
if (methodNode == null || methodNode.getKind() != AST.Kind.METHOD || !(methodNode.get()
instanceof MethodDeclaration)) return;
MethodDeclaration method = (MethodDeclaration) methodNode.get();
if (method.isAbstract()) return;
createLockField(annotationValue, annotationNode, lockTypeClass, lockImplClass, new
AtomicBoolean(method.isStatic()), false);
}
private static char[] createLockField(String name, EclipseNode annotationNode, char[][] lockTypeClass,
char[][] lockImplClass, AtomicBoolean isStatic, boolean reportErrors) {
char[] lockName = name.toCharArray();
Annotation source = (Annotation) annotationNode.get();
if (lockName.length == 0) {
lockName = isStatic.get() ? STATIC_LOCK_NAME : INSTANCE_LOCK_NAME;
}
EclipseNode typeNode = upToTypeNode(annotationNode);
MemberExistsResult exists = MemberExistsResult.NOT_EXISTS;
QualifiedTypeReference lockType = new QualifiedTypeReference(lockTypeClass, new long[] { 0, 0, 0,
0, 0 });
if (typeNode != null && typeNode.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration) typeNode.get();
if (typeDecl.fields != null) for (FieldDeclaration def : typeDecl.fields) {
char[] fName = def.name;
if (fName == null) continue;
if (Arrays.equals(fName, lockName)) {
exists = getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
boolean st = def.isStatic();
if (st != isStatic.get() && exists == MemberExistsResult.EXISTS_BY_LOMBOK) {
if (reportErrors) annotationNode.addError(String.format("The generated field %s does not match the
static status of this method", new String(lockName)));
return null;
}
isStatic.set(st);
if (exists == MemberExistsResult.EXISTS_BY_LOMBOK &&
!Arrays.deepEquals(lockType.getTypeName(), def.type.getTypeName())) {
annotationNode.addError("Expected field " + new String(lockName) + " to be of type " + lockType +
" but got type " + def.type + ". Did you mix @Locked with @Locked.Read/Write on the same generated
field?");
return null;
}
break;
}
}
}
if (exists == MemberExistsResult.NOT_EXISTS) {
FieldDeclaration fieldDecl = setGeneratedBy(new FieldDeclaration(lockName, 0, -1), source);
fieldDecl.declarationSourceEnd = -1;
fieldDecl.modifiers = (isStatic.get() ? Modifier.STATIC : 0) | Modifier.FINAL | Modifier.PRIVATE;
AllocationExpression lockAlloc = setGeneratedBy(new AllocationExpression(), source);
lockAlloc.type = setGeneratedBy(new QualifiedTypeReference(lockImplClass, new long[] { 0, 0, 0, 0, 0
}), source);
fieldDecl.type = setGeneratedBy(new QualifiedTypeReference(lockTypeClass, new long[] { 0, 0, 0, 0, 0
}), source);
```

```
fieldDecl.initialization = lockAlloc;
injectFieldAndMarkGenerated(annotationNode.up().up(), fieldDecl);
}
return lockName;
}
/**
* See {@link #handle(String, Annotation, EclipseNode, String, char[][], char[])} for
* {@code lockableMethodName = null}.
*/
public static void handle(String annotationValue, Annotation ast, EclipseNode annotationNode, String
annotationName, char[][] lockTypeClass, char[][] lockImplClass) {
handle(annotationValue, ast, annotationNode, annotationName, lockTypeClass, lockImplClass, null);
}
/**
* Called when an annotation is found that is likely to match the annotation you're interested in.
*
* Be aware that you'll be called for ANY annotation node in the source that looks like a match. There is,
* for example, no guarantee that the annotation node belongs to a method, even if you set your
* TargetType in the annotation to methods only.
*
* @param annotationValue The value of the annotation. This will be the name of the object used for
locking.
* @param source The Eclipse AST node representing the annotation.
* @param annotationNode The Lombok AST wrapper around the 'ast' parameter. You can use this
object
* to travel back up the chain (something javac AST can't do) to the parent of the annotation, as well
* as access useful methods such as generating warnings or errors focused on the annotation.
* @param annotationName The name of the annotation to use when referencing it in errors.
* @param lockTypeClass The fully qualified type of the variable when generating a lock to use.
* @param lockImplClass Call the constructor of this fully qualified classname to generate a lock to use.
* @param lockableMethodName The name of the method in the {@code lockClass} that returns a
* {@link java.util.concurrent.locks.Lock} object. When this is {@code null}, it is assumed that {@code
lockClass}
* itself can be locked/unlocked.
*/
public static void handle(String annotationValue, Annotation source, EclipseNode annotationNode,
String annotationName, char[][] lockTypeClass, char[][] lockImplClass, char[] lockableMethodName) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOCKED_FLAG_USAGE, annotationName);
int p1 = source.sourceStart -1;
int p2 = source.sourceStart -2;
long pos = (((long) p1) << 32) | p2;
EclipseNode methodNode = annotationNode.up();
if (methodNode == null || methodNode.getKind() != AST.Kind.METHOD || !(methodNode.get()
instanceof MethodDeclaration)) {
annotationNode.addError(annotationName + " is legal only on methods.");
return;
}
MethodDeclaration method = (MethodDeclaration) methodNode.get();
if (method.isAbstract()) {
annotationNode.addError(annotationName + " is legal only on concrete methods.");
return;
}
EclipseNode typeNode = upToTypeNode(annotationNode);
```

```
if (!isClassOrEnum(typeNode)) {
annotationNode.addError(annotationName + " is legal only on methods in classes and enums.");
return;
}
AtomicBoolean isStatic = new AtomicBoolean(method.isStatic());
char[] lockName = createLockField(annotationValue, annotationNode, lockTypeClass, lockImplClass,
isStatic, true);
if (lockName == null) return;
if (method.statements == null) return;
Block block = new Block(0);
block.statements = method.statements;
setGeneratedBy(block, source);
// Positions for in-method generated nodes are special
block.sourceEnd = method.bodyEnd;
block.sourceStart = method.bodyStart;
Statement acquireLock = getLockingStatement(source, typeNode, LOCK_METHOD, lockName,
lockableMethodName, isStatic.get(), p1, p2, pos);
Statement unLock = getLockingStatement(source, typeNode, UNLOCK_METHOD, lockName,
lockableMethodName, isStatic.get(), p1, p2, pos);
TryStatement tryStatement = new TryStatement();
tryStatement.tryBlock = block;
tryStatement.finallyBlock = new Block(0);
tryStatement.finallyBlock.statements = new Statement[] { unLock };
method.statements = new Statement[] { acquireLock, tryStatement };
// Positions for in-method generated nodes are special
method.statements[0].sourceEnd = method.bodyEnd;
method.statements[0].sourceStart = method.bodyStart;
methodNode.rebuild();
}
private static Statement getLockingStatement(ASTNode source, EclipseNode typeNode, char[]
lockMethod,
char[] lockableObjectName, char[] lockableMethodName, boolean isStatic, int p1, int p2, long pos) {
MessageSend lockStat = setGeneratedBy(new MessageSend(), source);
lockStat.receiver = getLockable(source, typeNode, lockableObjectName, lockableMethodName,
isStatic, p1, p2, pos);
lockStat.selector = lockMethod;
lockStat.nameSourcePosition = pos;
lockStat.sourceStart = p1;
lockStat.sourceEnd = lockStat.statementEnd = p2;
return lockStat;
}
private static Expression getLockable(ASTNode source, EclipseNode typeNode, char[] lockName,
char[] lockableMethodName, boolean isStatic, int p1, int p2, long pos) {
Reference lockVariable;
if (isStatic) {
char[][] n = getQualifiedInnerName(typeNode, lockName);
long[] ps = new long[n.length];
Arrays.fill(ps, pos);
lockVariable = new QualifiedNameReference(n, ps, p1, p2);
} else {
lockVariable = new FieldReference(lockName, pos);
ThisReference thisReference = new ThisReference(p1, p2);
setGeneratedBy(thisReference, source);
```

```
((FieldReference) lockVariable).receiver = thisReference;
}
setGeneratedBy(lockVariable, source);
Expression lockable;
if (lockableMethodName == null) lockable = lockVariable;
else {
lockable = new MessageSend();
((MessageSend) lockable).receiver = lockVariable;
((MessageSend) lockable).selector = lockableMethodName;
((MessageSend) lockable).nameSourcePosition = pos;
lockable.sourceStart = p1;
lockable.sourceEnd = lockable.statementEnd = p2;
}
return setGeneratedBy(lockable, source);
}
}
```

lombok/eclipse/handlers/HandleLockedWrite.java

```
/*
* Copyright (C) 2021-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.eclipse.EcjAugments.ASTNode_handled;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import lombok.Locked;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.DeferUntilPostDiet;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
```

```
import lombok.spi.Provides;
/**
* Handles the {@code lombok.Locked.Write} annotation for eclipse.
*/
@Provides
@DeferUntilPostDiet
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the statements generated by it.
public class HandleLockedWrite extends EclipseAnnotationHandler<Locked.Write> {
private static final char[] LOCK_METHOD = "writeLock".toCharArray();
private static final String ANNOTATION_NAME = "@Locked.Write";
private static final char[][] LOCK_TYPE_CLASS = new char[][] { TypeConstants.JAVA, TypeConstants.UTIL,
"concurrent".toCharArray(), "locks".toCharArray(), "ReadWriteLock".toCharArray() };
private static final char[][] LOCK_IMPL_CLASS = new char[][] { TypeConstants.JAVA, TypeConstants.UTIL,
"concurrent".toCharArray(), "locks".toCharArray(), "ReentrantReadWriteLock".toCharArray() };
@Override public void handle(AnnotationValues<Locked.Write> annotation, Annotation source, EclipseNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.handle(annotationValue, source, annotationNode, ANNOTATION_NAME, LOCK_TYPE_CLASS, LOCK_IMPL_CLASS, LOCK_METHOD);
}
@Override public void preHandle(AnnotationValues<Locked.Write> annotation, Annotation source, EclipseNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.preHandle(annotationValue, LOCK_TYPE_CLASS, LOCK_IMPL_CLASS, annotationNode);
if (hasParsedBody(getAnnotatedMethod(annotationNode))) {
// This method has a body in diet mode, so we have to handle it now.
handle(annotation, source, annotationNode);
ASTNode_handled.set(source, true);
}
}
}
```

lombok/eclipse/handlers/HandleLog.java


```
/*
* Copyright (C) 2010-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
```

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Modifier;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.ClassLiteralAccess;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.configuration.IdentifierName;
import lombok.core.configuration.LogDeclaration;
import lombok.core.configuration.LogDeclaration.LogFactoryParameter;
import lombok.core.handlers.LoggingFramework;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
public class HandleLog {
private static final IdentifierName LOG = IdentifierName.valueOf("log");
private HandleLog() {
throw new UnsupportedOperationException();
}
public static void processAnnotation(LoggingFramework framework, AnnotationValues<? extends
java.lang.annotation.Annotation> annotation, Annotation source, EclipseNode annotationNode) {
EclipseNode owner = annotationNode.up();
switch (owner.getKind()) {
case TYPE:
IdentifierName logFieldName =
annotationNode.getAst().readConfiguration(ConfigurationKeys.LOG_ANY_FIELD_NAME);
if (logFieldName == null) logFieldName = LOG;
boolean useStatic = !Boolean.FALSE.equals(annotationNode.getAst().readConfiguration(Configuration
Keys.LOG_ANY_FIELD_IS_STATIC));
TypeDeclaration typeDecl = null;
if (owner.get() instanceof TypeDeclaration) typeDecl = (TypeDeclaration) owner.get();
int modifiers = typeDecl == null ? 0 : typeDecl.modifiers;
```

```
boolean notAClass = (modifiers & (ClassFileConstants.AccInterface |
ClassFileConstants.AccAnnotation)) != 0;
if (typeDecl == null || notAClass) {
annotationNode.addError(framework.getAnnotationAsString() + " is legal only on classes and enums.");
return;
}
if (fieldExists(logFieldName.getName(), owner) != MemberExistsResult.NOT_EXISTS) {
annotationNode.addWarning("Field '" + logFieldName + "' already exists.");
return;
}
if (isRecord(owner) && !useStatic) {
annotationNode.addError("Logger fields must be static in records.");
return;
}
if (useStatic && !isStaticAllowed(owner)) {
annotationNode.addError(framework.getAnnotationAsString() + " is not supported on non-static nested
classes.");
return;
}
Object valueGuess = annotation.getValueGuess("topic");
Expression loggerTopic = (Expression) annotation.getActualExpression("topic");
if (valueGuess instanceof String && ((String) valueGuess).trim().isEmpty()) loggerTopic = null;
if (framework.getDeclaration().getParametersWithTopic() == null && loggerTopic != null) {
annotationNode.addError(framework.getAnnotationAsString() + " does not allow a topic.");
loggerTopic = null;
}
if (framework.getDeclaration().getParametersWithoutTopic() == null && loggerTopic == null) {
annotationNode.addError(framework.getAnnotationAsString() + " requires a topic.");
loggerTopic = new StringLiteral(new char[]{}, 0, 0, 0);
}
ClassLiteralAccess loggingType = selfType(owner, source);
FieldDeclaration fieldDeclaration = createField(framework, source, loggingType,
logFieldName.getName(), useStatic, loggerTopic);
fieldDeclaration.traverse(new SetGeneratedByVisitor(source), typeDecl.staticInitializerScope);
injectFieldAndMarkGenerated(owner, fieldDeclaration);
owner.rebuild();
break;
default:
break;
}
}
public static ClassLiteralAccess selfType(EclipseNode type, Annotation source) {
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
TypeDeclaration typeDeclaration = (TypeDeclaration)type.get();
TypeReference typeReference = new SingleTypeReference(typeDeclaration.name, p);
setGeneratedBy(typeReference, source);
ClassLiteralAccess result = new ClassLiteralAccess(source.sourceEnd, typeReference);
setGeneratedBy(result, source);
return result;
}
private static FieldDeclaration createField(LoggingFramework framework, Annotation source,
ClassLiteralAccess loggingType, String logFieldName, boolean useStatic, Expression loggerTopic) {
```

```
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
// private static final <loggerType> log = <factoryMethod>(<parameter>);
FieldDeclaration fieldDecl = new FieldDeclaration(logFieldName.toCharArray(), 0, -1);
setGeneratedBy(fieldDecl, source);
fieldDecl.declarationSourceEnd = -1;
fieldDecl.modifiers = Modifier.PRIVATE | (useStatic ? Modifier.STATIC : 0) | Modifier.FINAL;
LogDeclaration logDeclaration = framework.getDeclaration();
fieldDecl.type = createTypeReference(logDeclaration.getLoggerType().getName(), source);
MessageSend factoryMethodCall = new MessageSend();
setGeneratedBy(factoryMethodCall, source);
factoryMethodCall.receiver =
createNameReference(logDeclaration.getLoggerFactoryType().getName(), source);
factoryMethodCall.selector = logDeclaration.getLoggerFactoryMethod().getCharArray();
List<LogFactoryParameter> parameters = loggerTopic != null ?
logDeclaration.getParametersWithTopic() : logDeclaration.getParametersWithoutTopic();
factoryMethodCall.arguments = createFactoryParameters(loggingType, source, parameters,
loggerTopic);
factoryMethodCall.nameSourcePosition = p;
factoryMethodCall.sourceStart = pS;
factoryMethodCall.sourceEnd = factoryMethodCall.statementEnd = pE;
fieldDecl.initialization = factoryMethodCall;
return fieldDecl;
}
private static final Expression[] createFactoryParameters(ClassLiteralAccess loggingType, Annotation
source, List<LogFactoryParameter> parameters, Expression loggerTopic) {
Expression[] expressions = new Expression[parameters.size()];
int pS = source.sourceStart, pE = source.sourceEnd;
for (int i = 0; i < parameters.size(); i++) {
LogFactoryParameter parameter = parameters.get(i);
switch(parameter) {
case TYPE:
expressions[i] = createFactoryTypeParameter(loggingType, source);
break;
case NAME:
long p = (long) pS << 32 | pE;
MessageSend factoryParameterCall = new MessageSend();
setGeneratedBy(factoryParameterCall, source);
factoryParameterCall.receiver = createFactoryTypeParameter(loggingType, source);
factoryParameterCall.selector = "getName".toCharArray();
factoryParameterCall.nameSourcePosition = p;
factoryParameterCall.sourceStart = pS;
factoryParameterCall.sourceEnd = factoryParameterCall.statementEnd = pE;
expressions[i] = factoryParameterCall;
break;
case TOPIC:
expressions[i] = EclipseHandlerUtil.copyAnnotationMemberValue(loggerTopic);
break;
case NULL:
expressions[i] = new NullLiteral(pS, pE);
break;
default:
throw new IllegalStateException("Unknown logger factory parameter type: " + parameter);
```

```
}
}
return expressions;
}
private static final Expression createFactoryTypeParameter(ClassLiteralAccess loggingType,
Annotation source) {
TypeReference copy = copyType(loggingType.type, source);
ClassLiteralAccess result = new ClassLiteralAccess(source.sourceEnd, copy);
setGeneratedBy(result, source);
return result;
}
/**
* Handles the {@link lombok.extern.apachecommons.CommonsLog} annotation for Eclipse.
*/
@Provides
public static class HandleCommonsLog extends
EclipseAnnotationHandler<lombok.extern.apachecommons.CommonsLog> {
@Override public void handle(AnnotationValues<lombok.extern.apachecommons.CommonsLog>
annotation, Annotation source, EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_COMMONS_FLAG_USAGE,
"@apachecommons.CommonsLog", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.COMMONS, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.java.Log} annotation for Eclipse.
*/
@Provides
public static class HandleJulLog extends EclipseAnnotationHandler<lombok.extern.java.Log> {
@Override public void handle(AnnotationValues<lombok.extern.java.Log> annotation, Annotation
source, EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_JUL_FLAG_USAGE, "@java.Log",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.JUL, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.log4j.Log4j} annotation for Eclipse.
*/
@Provides
public static class HandleLog4jLog extends EclipseAnnotationHandler<lombok.extern.log4j.Log4j> {
@Override public void handle(AnnotationValues<lombok.extern.log4j.Log4j> annotation, Annotation
source, EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_LOG4J_FLAG_USAGE, "@Log4j",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.LOG4J, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.log4j.Log4j2} annotation for Eclipse.
*/
@Provides
public static class HandleLog4j2Log extends EclipseAnnotationHandler<lombok.extern.log4j.Log4j2> {
```

```
@Override public void handle(AnnotationValues<lombok.extern.log4j.Log4j2> annotation, Annotation source, EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_LOG4J2_FLAG_USAGE, "@Log4j2", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.LOG4J2, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.slf4j.Slf4j} annotation for Eclipse.
*/
@Provides
public static class HandleSlf4jLog extends EclipseAnnotationHandler<lombok.extern.slf4j.Slf4j> {
@Override public void handle(AnnotationValues<lombok.extern.slf4j.Slf4j> annotation, Annotation source, EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_SLF4J_FLAG_USAGE, "@Slf4j", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.SLF4J, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.slf4j.XSlf4j} annotation for Eclipse.
*/
@Provides
public static class HandleXSlf4jLog extends EclipseAnnotationHandler<lombok.extern.slf4j.XSlf4j> {
@Override public void handle(AnnotationValues<lombok.extern.slf4j.XSlf4j> annotation, Annotation source, EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_XSLF4J_FLAG_USAGE, "@XSlf4j", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.XSLF4J, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.jbosslog.JBossLog} annotation for Eclipse.
*/
@Provides
public static class HandleJBossLog extends
EclipseAnnotationHandler<lombok.extern.jbosslog.JBossLog> {
@Override public void handle(AnnotationValues<lombok.extern.jbosslog.JBossLog> annotation, Annotation source, EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_JBOSSLOG_FLAG_USAGE, "@JBossLog", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.JBOSSLOG, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.flogger.Flogger} annotation for Eclipse.
*/
@Provides
public static class HandleFloggerLog extends
EclipseAnnotationHandler<lombok.extern.flogger.Flogger> {
@Override public void handle(AnnotationValues<lombok.extern.flogger.Flogger> annotation, Annotation source, EclipseNode annotationNode) {
```

```
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_FLOGGER_FLAG_USAGE, "@Flogger",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.FLOGGER, annotation, source, annotationNode);
}
}
/**
* Handles the {@link lombok.CustomLog} annotation for Eclipse.
*/
@Provides
public static class HandleCustomLog extends EclipseAnnotationHandler<lombok.CustomLog> {
@Override public void handle(AnnotationValues<lombok.CustomLog> annotation, Annotation source,
EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_CUSTOM_FLAG_USAGE,
"@CustomLog", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
LogDeclaration logDeclaration =
annotationNode.getAst().readConfiguration(ConfigurationKeys.LOG_CUSTOM_DECLARATION);
if (logDeclaration == null) {
annotationNode.addError("The @CustomLog annotation is not configured; please set
lombok.log.custom.declaration in lombok.config.");
return;
}
LoggingFramework framework = new LoggingFramework(lombok.CustomLog.class, logDeclaration);
processAnnotation(framework, annotation, source, annotationNode);
}
}
}
```

lombok/eclipse/handlers/HandleNonNull.java

```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
```

```
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.AssertStatement;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.ExplicitConstructorCall;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SynchronizedStatement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.ThrowStatement;
import org.eclipse.jdt.internal.compiler.ast.TryStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import lombok.ConfigurationKeys;
import lombok.NonNull;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.EcjAugments;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
@Provides
@HandlerPriority(value = 512) // 2^9; onParameter=@__(@NonNull) has to run first.
public class HandleNonNull extends EclipseAnnotationHandler<NonNull> {
private static final char[] REQUIRE_NON_NULL = "requireNonNull".toCharArray();
private static final char[] CHECK_NOT_NULL = "checkNotNull".toCharArray();
public static final HandleNonNull INSTANCE = new HandleNonNull();
public void fix(EclipseNode method) {
for (EclipseNode m : method.down()) {
if (m.getKind() != Kind.ARGUMENT) continue;
```

```
for (EclipseNode c : m.down()) {
if (c.getKind() == Kind.ANNOTATION) {
if (annotationTypeMatches(NonNull.class, c)) {
handle0((Annotation) c.get(), c, true);
}
}
}
}
}
private List<FieldDeclaration> getRecordComponents(EclipseNode typeNode) {
List<FieldDeclaration> list = new ArrayList<FieldDeclaration>();
for (EclipseNode child : typeNode.down()) {
if (child.getKind() == Kind.FIELD) {
FieldDeclaration fd = (FieldDeclaration) child.get();
if ((fd.modifiers & AccRecord) != 0) list.add(fd);
}
}
return list;
}
private EclipseNode addCompactConstructorIfNeeded(EclipseNode typeNode, EclipseNode
annotationNode) {
// explicit Compact Constructor has bits set: Bit32, IsCanonicalConstructor (10).
// implicit Compact Constructor has bits set: Bit32, IsCanonicalConstructor (10), and IsImplicit (11).
// explicit constructor with long-form shows up as a normal constructor (Bit32 set, that's all), but the
// implicit CC is then also present and will presumably be stripped out in some later phase.
EclipseNode toRemove = null;
EclipseNode existingCompactConstructor = null;
List<FieldDeclaration> recordComponents = null;
for (EclipseNode child : typeNode.down()) {
if (!(child.get() instanceof ConstructorDeclaration)) continue;
ConstructorDeclaration cd = (ConstructorDeclaration) child.get();
if ((cd.bits & IsCanonicalConstructor) != 0) {
if ((cd.bits & IsImplicit) != 0) {
toRemove = child;
} else {
existingCompactConstructor = child;
}
} else {
// If this constructor has exact matching types vs. the record components,
// this is the canonical constructor in long form and we should not generate one.
if (recordComponents == null) recordComponents = getRecordComponents(typeNode);
int argLength = cd.arguments == null ? 0 : cd.arguments.length;
int compLength = recordComponents.size();
boolean isCanonical = argLength == compLength;
if (isCanonical) top: for (int i = 0; i < argLength; i++) {
TypeReference a = recordComponents.get(i).type;
TypeReference b = cd.arguments[i] == null ? null : cd.arguments[i].type;
// technically this won't match e.g. `java.lang.String` to just `String`;
// to use this feature you'll need to use the same way to write it, which seems
// like a fair requirement.
char[][] ta = getRawTypeName(a);
char[][] tb = getRawTypeName(b);
if (ta == null || tb == null || ta.length != tb.length) {
```

```
isCanonical = false;
break top;
}
for (int j = 0; j < ta.length; j++) {
if (!Arrays.equals(ta[j], tb[j])) {
isCanonical = false;
break top;
}
}
}
if (isCanonical) {
return null;
}
}
}
if (existingCompactConstructor != null) return existingCompactConstructor;
int posToInsert = -1;
TypeDeclaration td = (TypeDeclaration) typeNode.get();
if (toRemove != null) {
int idxToRemove = -1;
for (int i = 0; i < td.methods.length; i++) {
if (td.methods[i] == toRemove.get()) idxToRemove = i;
}
if (idxToRemove != -1) {
System.arraycopy(td.methods, idxToRemove + 1, td.methods, idxToRemove, td.methods.length -
idxToRemove - 1);
posToInsert = td.methods.length - 1;
typeNode.removeChild(toRemove);
}
}
if (posToInsert == -1) {
AbstractMethodDeclaration[] na = new AbstractMethodDeclaration[td.methods.length + 1];
posToInsert = td.methods.length;
System.arraycopy(td.methods, 0, na, 0, posToInsert);
td.methods = na;
}
ConstructorDeclaration cd = new ConstructorDeclaration(((CompilationUnitDeclaration)
typeNode.top().get()).compilationResult);
cd.modifiers = ClassFileConstants.AccPublic;
cd.bits = ASTNode.Bit32 | ECLIPSE_DO_NOT_TOUCH_FLAG | IsCanonicalConstructor;
cd.selector = td.name;
cd.constructorCall = new ExplicitConstructorCall(ExplicitConstructorCall.ImplicitSuper);
if (recordComponents == null) recordComponents = getRecordComponents(typeNode);
cd.arguments = new Argument[recordComponents.size()];
cd.statements = new Statement[recordComponents.size()];
cd.bits = IsCanonicalConstructor;
for (int i = 0; i < cd.arguments.length; i++) {
FieldDeclaration cmp = recordComponents.get(i);
cd.arguments[i] = new Argument(cmp.name, cmp.sourceStart, cmp.type, 0);
cd.arguments[i].bits = ASTNode.IsArgument | ASTNode.IgnoreRawTypeCheck |
ASTNode.IsReachable;
FieldReference lhs = new FieldReference(cmp.name, 0);
lhs.receiver = new ThisReference(0, 0);
```

```
SingleNameReference rhs = new SingleNameReference(cmp.name, 0);
cd.statements[i] = new Assignment(lhs, rhs, cmp.sourceEnd);
}
setGeneratedBy(cd, annotationNode.get());
for (int i = 0; i < cd.arguments.length; i++) {
FieldDeclaration cmp = recordComponents.get(i);
cd.arguments[i].sourceStart = cmp.sourceStart;
cd.arguments[i].sourceEnd = cmp.sourceStart;
cd.arguments[i].declarationSourceEnd = cmp.sourceStart;
cd.arguments[i].declarationEnd = cmp.sourceStart;
}
td.methods[posToInsert] = cd;
cd.annotations = addSuppressWarningsAll(typeNode, cd, cd.annotations);
cd.annotations = addGenerated(typeNode, cd, cd.annotations);
return typeNode.add(cd, Kind.METHOD);
}
private static char[][] getRawTypeName(TypeReference a) {
if (a instanceof QualifiedTypeReference) return ((QualifiedTypeReference) a).tokens;
if (a instanceof SingleTypeReference) return new char[][] {((SingleTypeReference) a).token};
return null;
}
@Override public void handle(AnnotationValues<NonNull> annotation, Annotation ast, EclipseNode
annotationNode) {
// Generating new methods is only possible during diet parse but modifying existing methods requires a
full parse.
// As we need both for @NonNull we reset the handled flag during diet parse.
if (!annotationNode.isCompleteParse()) {
final EclipseNode node;
if (annotationNode.up().getKind() == Kind.TYPE_USE) {
node = annotationNode.directUp().directUp();
} else {
node = annotationNode.up();
}
if (node.getKind() == Kind.FIELD) {
//Check if this is a record and we need to generate the compact form constructor.
EclipseNode typeNode = node.up();
if (typeNode.getKind() == Kind.TYPE) {
if (isRecord(typeNode)) {
addCompactConstructorIfNeeded(typeNode, annotationNode);
}
}
}
EcjAugments.ASTNode_handled.clear(ast);
return;
}
handle0(ast, annotationNode, false);
}
private EclipseNode findCompactConstructor(EclipseNode typeNode) {
for (EclipseNode child : typeNode.down()) {
if (!(child.get() instanceof ConstructorDeclaration)) continue;
ConstructorDeclaration cd = (ConstructorDeclaration) child.get();
if ((cd.bits & IsCanonicalConstructor) != 0 && (cd.bits & IsImplicit) == 0) return child;
}
```

```
return null;
}
private void handle0(Annotation ast, EclipseNode annotationNode, boolean force) {
handleFlagUsage(annotationNode, ConfigurationKeys.NON_NULL_FLAG_USAGE, "@NonNull");
final EclipseNode node;
if (annotationNode.up().getKind() == Kind.TYPE_USE) {
node = annotationNode.directUp().directUp();
} else {
node = annotationNode.up();
}
if (node.getKind() == Kind.FIELD) {
// This is meaningless unless the field is used to generate a method (@Setter,
@RequiredArgsConstructor, etc),
// but in that case those handlers will take care of it. However, we DO check if the annotation is applied
to
// a primitive, because those handlers trigger on any annotation named @NonNull and we only want the
warning
// behaviour on _OUR_ 'lombok.NonNull'.
EclipseNode fieldNode = node;
EclipseNode typeNode = fieldNode.up();
try {
if (isPrimitive(((AbstractVariableDeclaration) node.get()).type)) {
annotationNode.addWarning("@NonNull is meaningless on a primitive.");
return;
}
} catch (Exception ignore) {}
if (isRecord(typeNode)) {
// well, these kinda double as parameters (of the compact constructor), so we do some work here.
// NB:Tthe diet parse run already added an explicit compact constructor if we need to take any actions.
EclipseNode compactConstructor = findCompactConstructor(typeNode);
if (compactConstructor != null) {
addNullCheckIfNeeded((AbstractMethodDeclaration) compactConstructor.get(),
(AbstractVariableDeclaration) fieldNode.get(), annotationNode);
}
}
return;
}
if (node.getKind() != Kind.ARGUMENT) return;
Argument param;
AbstractMethodDeclaration declaration;
try {
param = (Argument) node.get();
declaration = (AbstractMethodDeclaration) node.up().get();
} catch (Exception e) {
return;
}
if (!force && isGenerated(declaration)) return;
if (declaration.isAbstract()) {
// This used to be a warning, but as @NonNull also has a documentary purpose, better to not warn
about this. Since 1.16.7
return;
}
addNullCheckIfNeeded(declaration, param, annotationNode);
```

```
node.up().rebuild();
}
private void addNullCheckIfNeeded(AbstractMethodDeclaration declaration,
AbstractVariableDeclaration param, EclipseNode annotationNode) {
// Possibly, if 'declaration instanceof ConstructorDeclaration', fetch declaration.constructorCall, search it
for any references to our parameter,
// and if they exist, create a new method in the class: 'private static <T> T lombok$nullCheck(T expr,
String msg) {if (expr == null) throw NPE; return expr;}' and
// wrap all references to it in the super/this to a call to this method.
Statement nullCheck = generateNullCheck(param, annotationNode, null);
if (nullCheck == null) {
// @NonNull applied to a primitive. Kinda pointless. Let's generate a warning.
annotationNode.addWarning("@NonNull is meaningless on a primitive.");
return;
}
if (declaration.statements == null) {
declaration.statements = new Statement[] {nullCheck};
} else {
char[] expectedName = param.name;
/* Abort if the null check is already there, delving into try and synchronized statements */ {
Statement[] stats = declaration.statements;
int idx = 0;
while (stats != null && stats.length > idx) {
Statement stat = stats[idx++];
if (stat instanceof TryStatement) {
stats = ((TryStatement) stat).tryBlock.statements;
idx = 0;
continue;
}
if (stat instanceof SynchronizedStatement) {
stats = ((SynchronizedStatement) stat).block.statements;
idx = 0;
continue;
}
char[] varNameOfNullCheck = returnVarNameIfNullCheck(stat);
if (varNameOfNullCheck == null) break;
if (Arrays.equals(varNameOfNullCheck, expectedName)) return;
}
}
Statement[] newStatements = new Statement[declaration.statements.length + 1];
int skipOver = 0;
for (Statement stat : declaration.statements) {
if (isGenerated(stat) && isNullCheck(stat)) skipOver++;
else break;
}
System.arraycopy(declaration.statements, 0, newStatements, 0, skipOver);
System.arraycopy(declaration.statements, skipOver, newStatements, skipOver + 1,
declaration.statements.length - skipOver);
newStatements[skipOver] = nullCheck;
declaration.statements = newStatements;
}
}
public boolean isNullCheck(Statement stat) {
```

```
return returnVarNameIfNullCheck(stat) != null;
}
public char[] returnVarNameIfNullCheck(Statement stat) {
boolean isIf = stat instanceof IfStatement;
boolean isExpression = stat instanceof Expression;
if (!isIf && !(stat instanceof AssertStatement) && !isExpression) return null;
if (isExpression) {
/* Check if the statements contains a call to checkNotNull or requireNonNull */
Expression expression = (Expression) stat;
if (expression instanceof Assignment) expression = ((Assignment) expression).expression;
if (!(expression instanceof MessageSend)) return null;
MessageSend invocation = (MessageSend) expression;
if (!Arrays.equals(invocation.selector, CHECK_NOT_NULL) && !Arrays.equals(invocation.selector,
REQUIRE_NON_NULL)) return null;
if (invocation.arguments == null || invocation.arguments.length == 0) return null;
Expression firstArgument = invocation.arguments[0];
if (!(firstArgument instanceof SingleNameReference)) return null;
return ((SingleNameReference) firstArgument).token;
}
if (isIf) {
/* Check that the if's statement is a throw statement, possibly in a block. */
Statement then = ((IfStatement) stat).thenStatement;
if (then instanceof Block) {
Statement[] blockStatements = ((Block) then).statements;
if (blockStatements == null || blockStatements.length == 0) return null;
then = blockStatements[0];
}
if (!(then instanceof ThrowStatement)) return null;
}
/* Check that the if's conditional is like 'x == null'. Return from this method (don't generate
a nullcheck) if 'x' is equal to our own variable's name: There's already a nullcheck here. */ {
Expression cond = isIf ? ((IfStatement) stat).condition : ((AssertStatement) stat).assertExpression;
if (!(cond instanceof EqualExpression)) return null;
EqualExpression bin = (EqualExpression) cond;
String op = bin.operatorToString();
if (isIf) {
if (!"==".equals(op)) return null;
} else {
if (!"!=".equals(op)) return null;
}
if (!(bin.left instanceof SingleNameReference)) return null;
if (!(bin.right instanceof NullLiteral)) return null;
return ((SingleNameReference) bin.left).token;
}
}
}
```

lombok/eclipse/handlers/HandlePrintAST.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
```

*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.eclipse.handlers;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintStream;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import lombok.Lombok;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.PrintAST;
import lombok.eclipse.DeferUntilPostDiet;
import lombok.eclipse.EclipseASTVisitor;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.core.PrintAST} annotation for eclipse.
*/
@Provides
@DeferUntilPostDiet
@HandlerPriority(536870912) // 2^29; this handler is customarily run at the very end.
public class HandlePrintAST extends EclipseAnnotationHandler<PrintAST> {
public void handle(AnnotationValues<PrintAST> annotation, Annotation ast, EclipseNode
annotationNode) {
PrintStream stream = System.out;
String fileName = annotation.getInstance().outfile();
if (fileName.length() > 0) try {
stream = new PrintStream(new File(fileName));
} catch (FileNotFoundException e) {
throw Lombok.sneakyThrow(e);
}
try {
annotationNode.up().traverse(new EclipseASTVisitor.Printer(annotation.getInstance().printContent(),
stream, annotation.getInstance().printPositions()));
} finally {
```

```
if (stream != System.out) {
try {
stream.close();
} catch (Exception e) {
throw Lombok.sneakyThrow(e);
}
}
}
}
}
```

lombok/eclipse/handlers/HandleSetter.java


```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.Setter;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.Accessors;
```

```
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NameReference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TrueLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
/**
* Handles the {@code lombok.Setter} annotation for eclipse.
*/
@Provides
public class HandleSetter extends EclipseAnnotationHandler<Setter> {
private static final String SETTER_NODE_NOT_SUPPORTED_ERR = "@Setter is only supported on a
class or a field.";
public boolean generateSetterForType(EclipseNode typeNode, EclipseNode pos, AccessLevel level,
boolean checkForTypeLevelSetter, List<Annotation> onMethod, List<Annotation> onParam) {
if (checkForTypeLevelSetter) {
if (hasAnnotation(Setter.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return true;
}
}
if (!isClass(typeNode)) {
pos.addError(SETTER_NODE_NOT_SUPPORTED_ERR);
return false;
}
for (EclipseNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
FieldDeclaration fieldDecl = (FieldDeclaration) field.get();
if (!filterField(fieldDecl)) continue;
//Skip final fields.
if ((fieldDecl.modifiers & ClassFileConstants.AccFinal) != 0) continue;
generateSetterForField(field, pos, level, onMethod, onParam);
}
return true;
}
/**
* Generates a setter on the stated field.
*
* Used by {@link HandleData}.
*
* The difference between this call and the handle method is as follows:
```

```
*
* If there is a {@code lombok.Setter} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
* If not, the setter is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*/
public void generateSetterForField(EclipseNode fieldNode, EclipseNode sourceNode, AccessLevel
level, List<Annotation> onMethod, List<Annotation> onParam) {
if (hasAnnotation(Setter.class, fieldNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
createSetterForField(level, fieldNode, sourceNode, false, onMethod, onParam);
}
@Override public void handle(AnnotationValues<Setter> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.SETTER_FLAG_USAGE, "@Setter");
EclipseNode node = annotationNode.up();
AccessLevel level = annotation.getInstance().value();
if (level == AccessLevel.NONE || node == null) return;
List<Annotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@Setter(onMethod", annotationNode);
if (!onMethod.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@Setter(onMethod=...)");
}
List<Annotation> onParam = unboxAndRemoveAnnotationParameter(ast, "onParam",
"@Setter(onParam", annotationNode);
if (!onParam.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@Setter(onParam=...)");
}
switch (node.getKind()) {
case FIELD:
createSetterForFields(level, annotationNode.upFromAnnotationToFields(), annotationNode, true,
onMethod, onParam);
break;
case TYPE:
generateSetterForType(node, annotationNode, level, false, onMethod, onParam);
break;
}
}
public void createSetterForFields(AccessLevel level, Collection<EclipseNode> fieldNodes,
EclipseNode sourceNode, boolean whineIfExists, List<Annotation> onMethod, List<Annotation>
onParam) {
for (EclipseNode fieldNode : fieldNodes) {
createSetterForField(level, fieldNode, sourceNode, whineIfExists, onMethod, onParam);
}
}
public void createSetterForField(
AccessLevel level, EclipseNode fieldNode, EclipseNode sourceNode,
boolean whineIfExists, List<Annotation> onMethod,
List<Annotation> onParam) {
```

```
ASTNode source = sourceNode.get();
if (fieldNode.getKind() != Kind.FIELD) {
sourceNode.addError(SETTER_NODE_NOT_SUPPORTED_ERR);
return;
}
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
TypeReference fieldType = copyType(field.type, source);
boolean isBoolean = isBoolean(fieldType);
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
String setterName = toSetterName(fieldNode, isBoolean, accessors);
boolean shouldReturnThis = shouldReturnThis(fieldNode, accessors);
if (setterName == null) {
fieldNode.addWarning("Not generating setter for this field: It does not fit your @Accessors prefix list.");
return;
}
int modifier = toEclipseModifier(level) | (field.modifiers & ClassFileConstants.AccStatic);
for (String altName : toAllSetterNames(fieldNode, isBoolean, accessors)) {
switch (methodExists(altName, fieldNode, false, 1)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String altNameExpl = "";
if (!altName.equals(setterName)) altNameExpl = String.format(" (%s)", altName);
fieldNode.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", setterName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
//continue scanning the other alt names.
}
}
MethodDeclaration method = createSetter((TypeDeclaration) fieldNode.up().get(), false, fieldNode,
setterName, null, null, shouldReturnThis, modifier, sourceNode, onMethod, onParam);
injectMethod(fieldNode.up(), method);
}
static MethodDeclaration createSetter(TypeDeclaration parent, boolean deprecate, EclipseNode
fieldNode, String name, char[] paramName, char[] booleanFieldToSet, boolean shouldReturnThis, int
modifier, EclipseNode sourceNode, List<Annotation> onMethod, List<Annotation> onParam) {
ASTNode source = sourceNode.get();
int pS = source.sourceStart, pE = source.sourceEnd;
TypeReference returnType = null;
ReturnStatement returnThis = null;
if (shouldReturnThis) {
returnType = cloneSelfType(fieldNode, source);
addCheckerFrameworkReturnsReceiver(returnType, source,
getCheckerFrameworkVersion(sourceNode));
ThisReference thisRef = new ThisReference(pS, pE);
returnThis = new ReturnStatement(thisRef, pS, pE);
}
```

```
MethodDeclaration d = createSetter(parent, deprecate, fieldNode, name, paramName,
booleanFieldToSet, returnType, returnThis, modifier, sourceNode, onMethod, onParam);
return d;
}
static MethodDeclaration createSetter(TypeDeclaration parent, boolean deprecate, EclipseNode
fieldNode, String name, char[] paramName, char[] booleanFieldToSet, TypeReference returnType,
Statement returnStatement, int modifier, EclipseNode sourceNode, List<Annotation> onMethod,
List<Annotation> onParam) {
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
if (paramName == null) paramName = field.name;
ASTNode source = sourceNode.get();
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration method = new MethodDeclaration(parent.compilationResult);
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
if (shouldMakeFinal(fieldNode, accessors)) modifier |= ClassFileConstants.AccFinal;
method.modifiers = modifier;
if (returnType != null) {
method.returnType = returnType;
} else {
method.returnType = TypeReference.baseTypeReference(TypeIds.T_void, 0);
method.returnType.sourceStart = pS; method.returnType.sourceEnd = pE;
}
Annotation[] deprecated = null;
if (isFieldDeprecated(fieldNode) || deprecate) {
deprecated = new Annotation[] { generateDeprecatedAnnotation(source) };
}
method.annotations = copyAnnotations(source, onMethod.toArray(new Annotation[0]), deprecated,
findCopyableToSetterAnnotations(fieldNode));
Argument param = new Argument(paramName, p, copyType(field.type, source), Modifier.FINAL);
param.sourceStart = pS; param.sourceEnd = pE;
method.arguments = new Argument[] { param };
method.selector = name.toCharArray();
method.binding = null;
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
Expression fieldRef = createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source);
NameReference fieldNameRef = new SingleNameReference(paramName, p);
Assignment assignment = new Assignment(fieldRef, fieldNameRef, (int) p);
assignment.sourceStart = pS; assignment.sourceEnd = assignment.statementEnd = pE;
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
Annotation[] copyableAnnotations = findCopyableAnnotations(fieldNode);
List<Statement> statements = new ArrayList<Statement>(5);
if (!hasNonNullAnnotations(fieldNode) && !hasNonNullAnnotations(fieldNode, onParam)) {
statements.add(assignment);
} else {
Statement nullCheck = generateNullCheck(field.type, paramName, sourceNode, null);
if (nullCheck != null) statements.add(nullCheck);
statements.add(assignment);
}
if (booleanFieldToSet != null) {
```

```
statements.add(new Assignment(new SingleNameReference(booleanFieldToSet, p), new
TrueLiteral(pS, pE), pE));
}
if (returnType != null && returnStatement != null) {
statements.add(returnStatement);
}
method.statements = statements.toArray(new Statement[0]);
param.annotations = copyAnnotations(source, copyableAnnotations, onParam.toArray(new
Annotation[0]));
if (param.annotations != null) {
param.bits |= Eclipse.HasTypeAnnotations;
method.bits |= Eclipse.HasTypeAnnotations;
}
if (returnType != null && returnStatement != null) createRelevantNonNullAnnotation(sourceNode,
method);
method.traverse(new SetGeneratedByVisitor(source), parent.scope);
copyJavadoc(fieldNode, method, CopyJavadoc.SETTER, returnStatement != null);
return method;
}
}
```

lombok/eclipse/handlers/HandleSneakyThrows.java


```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.EcjAugments.ASTNode_handled;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Arrays;
```

```
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ExplicitConstructorCall;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.MemberValuePair;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThrowStatement;
import org.eclipse.jdt.internal.compiler.ast.TryStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import lombok.ConfigurationKeys;
import lombok.SneakyThrows;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.DeferUntilPostDiet;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.HandleSneakyThrows} annotation for eclipse.
*/
@Provides
@DeferUntilPostDiet
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the
statements generated by it.
public class HandleSneakyThrows extends EclipseAnnotationHandler<SneakyThrows> {
private static class DeclaredException {
final String exceptionName;
final ASTNode node;
DeclaredException(String exceptionName, ASTNode node) {
this.exceptionName = exceptionName;
this.node = node;
}
}
@Override
public void preHandle(AnnotationValues<SneakyThrows> annotation, Annotation ast, EclipseNode
annotationNode) {
if (hasParsedBody(getAnnotatedMethod(annotationNode))) {
// This method has a body in diet mode, so we have to handle it now.
handle(annotation, ast, annotationNode);
ASTNode_handled.set(ast, true);
}
}
```

```
@Override public void handle(AnnotationValues<SneakyThrows> annotation, Annotation source,
EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.SNEAKY_THROWS_FLAG_USAGE,
"@SneakyThrows");
List<String> exceptionNames = annotation.getRawExpressions("value");
List<DeclaredException> exceptions = new ArrayList<DeclaredException>();
MemberValuePair[] memberValuePairs = source.memberValuePairs();
if (memberValuePairs == null || memberValuePairs.length == 0) {
exceptions.add(new DeclaredException("java.lang.Throwable", source));
} else {
Expression arrayOrSingle = memberValuePairs[0].value;
final Expression[] exceptionNameNodes;
if (arrayOrSingle instanceof ArrayInitializer) {
exceptionNameNodes = ((ArrayInitializer)arrayOrSingle).expressions;
} else exceptionNameNodes = new Expression[] { arrayOrSingle };
if (exceptionNames.size() != exceptionNameNodes.length) {
annotationNode.addError(
"LOMBOK BUG: The number of exception classes in the annotation isn't the same pre- and post-
guessing.");
}
int idx = 0;
for (String exceptionName : exceptionNames) {
if (exceptionName.endsWith(".class")) exceptionName = exceptionName.substring(0,
exceptionName.length() - 6);
exceptions.add(new DeclaredException(exceptionName, exceptionNameNodes[idx++]));
}
}
EclipseNode owner = annotationNode.up();
switch (owner.getKind()) {
// case FIELD:
// return handleField(annotationNode, (FieldDeclaration)owner.get(), exceptions);
case METHOD:
handleMethod(annotationNode, (AbstractMethodDeclaration)owner.get(), exceptions);
break;
default:
annotationNode.addError("@SneakyThrows is legal only on methods and constructors.");
}
}
// private boolean handleField(Node annotation, FieldDeclaration field, List<DeclaredException>
exceptions) {
// if (field.initialization == null) {
// annotation.addError("@SneakyThrows can only be used on fields with an initialization statement.");
// return true;
// }
//
// Expression expression = field.initialization;
// Statement[] content = new Statement[] {new Assignment(
// new SingleNameReference(field.name, 0), expression, 0)};
// field.initialization = null;
//
// for (DeclaredException exception : exceptions) {
// content = new Statement[] { buildTryCatchBlock(content, exception) };
// }
```

```
//
// Block block = new Block(0);
// block.statements = content;
//
// Node typeNode = annotation.up().up();
//
// Initializer initializer = new Initializer(block, field.modifiers & Modifier.STATIC);
// initializer.sourceStart = expression.sourceStart;
// initializer.sourceEnd = expression.sourceEnd;
// initializer.declarationSourceStart = expression.sourceStart;
// initializer.declarationSourceEnd = expression.sourceEnd;
// injectField(typeNode, initializer);
//
// typeNode.rebuild();
//
// return true;
// }
public void handleMethod(EclipseNode annotation, AbstractMethodDeclaration method,
List<DeclaredException> exceptions) {
if (method.isAbstract()) {
annotation.addError("@SneakyThrows can only be used on concrete methods.");
return;
}
if (method.statements == null || method.statements.length == 0) {
boolean hasConstructorCall = false;
if (method instanceof ConstructorDeclaration) {
ExplicitConstructorCall constructorCall = ((ConstructorDeclaration) method).constructorCall;
hasConstructorCall = constructorCall != null && !constructorCall.isImplicitSuper() &&
!constructorCall.isImplicitThis();
}
if (hasConstructorCall) {
annotation.addWarning("Calls to sibling / super constructors are always excluded from
@SneakyThrows; @SneakyThrows has been ignored because there is no other code in this
constructor.");
} else {
annotation.addWarning("This method or constructor is empty; @SneakyThrows has been ignored.");
}
return;
}
Statement[] contents = method.statements;
for (DeclaredException exception : exceptions) {
contents = new Statement[] { buildTryCatchBlock(contents, exception, exception.node, method) };
}
method.statements = contents;
annotation.up().rebuild();
}
public Statement buildTryCatchBlock(Statement[] contents, DeclaredException exception, ASTNode
source, AbstractMethodDeclaration method) {
int methodStart = method.bodyStart;
int methodEnd = method.bodyEnd;
long methodPosEnd = ((long) methodEnd) << 32 | (methodEnd & 0xFFFFFFFFL);
TryStatement tryStatement = new TryStatement();
setGeneratedBy(tryStatement, source);
```

```
tryStatement.tryBlock = new Block(0);
// Positions for in-method generated nodes are special
tryStatement.tryBlock.sourceStart = methodStart; tryStatement.tryBlock.sourceEnd = methodEnd;
setGeneratedBy(tryStatement.tryBlock, source);
tryStatement.tryBlock.statements = contents;
TypeReference typeReference;
if (exception.exceptionName.indexOf('.') == -1) {
typeReference = new SingleTypeReference(exception.exceptionName.toCharArray(), methodPosEnd);
typeReference.statementEnd = methodEnd;
} else {
String[] x = exception.exceptionName.split("\\.");
char[][] elems = new char[x.length][];
long[] poss = new long[x.length];
Arrays.fill(poss, methodPosEnd);
for (int i = 0; i < x.length; i++) {
elems[i] = x[i].trim().toCharArray();
}
typeReference = new QualifiedTypeReference(elems, poss);
}
setGeneratedBy(typeReference, source);
Argument catchArg = new Argument("$ex".toCharArray(), methodPosEnd, typeReference,
Modifier.FINAL);
setGeneratedBy(catchArg, source);
catchArg.declarationSourceEnd = catchArg.declarationEnd = catchArg.sourceEnd = methodEnd;
catchArg.declarationSourceStart = catchArg.modifiersSourceStart = catchArg.sourceStart =
methodEnd;
tryStatement.catchArguments = new Argument[] { catchArg };
MessageSend sneakyThrowStatement = new MessageSend();
setGeneratedBy(sneakyThrowStatement, source);
sneakyThrowStatement.receiver = new QualifiedNameReference(new char[][] {
"lombok".toCharArray(), "Lombok".toCharArray() }, new long[2], methodEnd, methodEnd);
setGeneratedBy(sneakyThrowStatement.receiver, source);
sneakyThrowStatement.receiver.statementEnd = methodEnd;
sneakyThrowStatement.selector = "sneakyThrow".toCharArray();
SingleNameReference exRef = new SingleNameReference("$ex".toCharArray(), methodPosEnd);
setGeneratedBy(exRef, source);
exRef.statementEnd = methodEnd;
sneakyThrowStatement.arguments = new Expression[] { exRef };
// This is the magic fix for rendering issues
// In
org.eclipse.jdt.core.dom.ASTConverter#convert(org.eclipse.jdt.internal.compiler.ast.MessageSend)
// a new SimpleName is created and the setSourceRange should receive -1, 0. That's why we provide
-2L :-)
sneakyThrowStatement.nameSourcePosition = -2L;
sneakyThrowStatement.sourceStart = methodEnd;
sneakyThrowStatement.sourceEnd = sneakyThrowStatement.statementEnd = methodEnd;
Statement rethrowStatement = new ThrowStatement(sneakyThrowStatement, methodEnd,
methodEnd);
setGeneratedBy(rethrowStatement, source);
Block block = new Block(0);
block.sourceStart = methodEnd;
block.sourceEnd = methodEnd;
setGeneratedBy(block, source);
```

```
block.statements = new Statement[] { rethrowStatement };
tryStatement.catchBlocks = new Block[] { block };
// Positions for in-method generated nodes are special
tryStatement.sourceStart = method.bodyStart;
tryStatement.sourceEnd = method.bodyEnd;
return tryStatement;
}
}
```

lombok/eclipse/handlers/HandleStandardException.java

```
/*
* Copyright (C) 2021-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.experimental.StandardException;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.*;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.*;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import java.lang.reflect.Modifier;
import java.util.*;
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.eclipse.Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
```

```
@Provides
public class HandleStandardException extends EclipseAnnotationHandler<StandardException> {
@Override
public void handle(AnnotationValues<StandardException> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.STANDARD_EXCEPTION_FLAG_USAGE,
"@StandardException");
EclipseNode typeNode = annotationNode.up();
if (!isClass(typeNode)) {
annotationNode.addError("@StandardException is only supported on a class");
return;
}
TypeDeclaration classDef = (TypeDeclaration) typeNode.get();
if (classDef.superclass == null) {
annotationNode.addError("@StandardException requires that you extend a Throwable type");
return;
}
AccessLevel access = annotation.getInstance().access();
generateNoArgsConstructor(typeNode, access, annotationNode);
generateMsgOnlyConstructor(typeNode, access, annotationNode);
generateCauseOnlyConstructor(typeNode, access, annotationNode);
generateFullConstructor(typeNode, access, annotationNode);
}
private void generateNoArgsConstructor(EclipseNode typeNode, AccessLevel level, EclipseNode
source) {
if (hasConstructor(typeNode) != MemberExistsResult.NOT_EXISTS) return;
int pS = source.get().sourceStart, pE = source.get().sourceEnd;
Expression messageArgument = new CastExpression(new NullLiteral(pS, pE),
generateQualifiedTypeRef(source.get(), TypeConstants.JAVA_LANG_STRING));
Expression causeArgument = new CastExpression(new NullLiteral(pS, pE),
generateQualifiedTypeRef(source.get(), TypeConstants.JAVA_LANG_THROWABLE));
ExplicitConstructorCall explicitCall = new ExplicitConstructorCall(ExplicitConstructorCall.This);
explicitCall.arguments = new Expression[] {messageArgument, causeArgument};
ConstructorDeclaration constructor = createConstructor(level, typeNode, false, false, source,
explicitCall, null);
injectMethod(typeNode, constructor);
}
private void generateMsgOnlyConstructor(EclipseNode typeNode, AccessLevel level, EclipseNode
source) {
if (hasConstructor(typeNode, String.class) != MemberExistsResult.NOT_EXISTS) return;
int pS = source.get().sourceStart, pE = source.get().sourceEnd;
long p = (long) pS << 32 | pE;
Expression messageArgument = new SingleNameReference(MESSAGE, p);
Expression causeArgument = new CastExpression(new NullLiteral(pS, pE),
generateQualifiedTypeRef(source.get(), TypeConstants.JAVA_LANG_THROWABLE));
ExplicitConstructorCall explicitCall = new ExplicitConstructorCall(ExplicitConstructorCall.This);
explicitCall.arguments = new Expression[] {messageArgument, causeArgument};
ConstructorDeclaration constructor = createConstructor(level, typeNode, true, false, source,
explicitCall, null);
injectMethod(typeNode, constructor);
}
private void generateCauseOnlyConstructor(EclipseNode typeNode, AccessLevel level, EclipseNode
source) {
```

```
if (hasConstructor(typeNode, Throwable.class) != MemberExistsResult.NOT_EXISTS) return;
int pS = source.get().sourceStart, pE = source.get().sourceEnd;
long p = (long) pS << 32 | pE;
ExplicitConstructorCall explicitCall = new ExplicitConstructorCall(ExplicitConstructorCall.This);
Expression causeNotNull = new EqualExpression(new SingleNameReference(CAUSE, p), new
NullLiteral(pS, pE), OperatorIds.NOT_EQUAL);
MessageSend causeDotGetMessage = new MessageSend();
causeDotGetMessage.sourceStart = pS; causeDotGetMessage.sourceEnd = pE;
causeDotGetMessage.receiver = new SingleNameReference(CAUSE, p);
causeDotGetMessage.selector = GET_MESSAGE;
Expression messageExpr = new ConditionalExpression(causeNotNull, causeDotGetMessage, new
NullLiteral(pS, pE));
explicitCall.arguments = new Expression[] {messageExpr, new SingleNameReference(CAUSE, p)};
ConstructorDeclaration constructor = createConstructor(level, typeNode, false, true, source,
explicitCall, null);
injectMethod(typeNode, constructor);
}
private void generateFullConstructor(EclipseNode typeNode, AccessLevel level, EclipseNode source) {
if (hasConstructor(typeNode, String.class, Throwable.class) != MemberExistsResult.NOT_EXISTS)
return;
int pS = source.get().sourceStart, pE = source.get().sourceEnd;
long p = (long) pS << 32 | pE;
ExplicitConstructorCall explicitCall = new ExplicitConstructorCall(ExplicitConstructorCall.Super);
explicitCall.arguments = new Expression[] {new SingleNameReference(MESSAGE, p)};
Expression causeNotNull = new EqualExpression(new SingleNameReference(CAUSE, p), new
NullLiteral(pS, pE), OperatorIds.NOT_EQUAL);
MessageSend causeDotInitCause = new MessageSend();
causeDotInitCause.sourceStart = pS; causeDotInitCause.sourceEnd = pE;
causeDotInitCause.receiver = new SuperReference(pS, pE);
causeDotInitCause.selector = INIT_CAUSE;
causeDotInitCause.arguments = new Expression[] {new SingleNameReference(CAUSE, p)};
IfStatement ifs = new IfStatement(causeNotNull, causeDotInitCause, pS, pE);
ConstructorDeclaration constructor = createConstructor(level, typeNode, true, true, source, explicitCall,
ifs);
injectMethod(typeNode, constructor);
}
/**
* Checks if a constructor with the provided parameters exists under the type node.
*/
public static MemberExistsResult hasConstructor(EclipseNode node, Class<?>... paramTypes) {
node = upToTypeNode(node);
if (node != null && node.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration) node.get();
if (typeDecl.methods != null) for (AbstractMethodDeclaration def : typeDecl.methods) {
if (def instanceof ConstructorDeclaration) {
if ((def.bits & ASTNode.IsDefaultConstructor) != 0) continue;
if (!paramsMatch(node, def.arguments, paramTypes)) continue;
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
}
return MemberExistsResult.NOT_EXISTS;
```

```
}
private static boolean paramsMatch(EclipseNode node, Argument[] a, Class<?>[] b) {
if (a == null) return b == null || b.length == 0;
if (b == null) return a.length == 0;
if (a.length != b.length) return false;
for (int i = 0; i < a.length; i++) {
if (!typeMatches(b[i], node, a[i].type)) return false;
}
return true;
}
private static final char[][] JAVA_BEANS_CONSTRUCTORPROPERTIES = new char[][] {
"java".toCharArray(), "beans".toCharArray(), "ConstructorProperties".toCharArray() };
private static final char[] MESSAGE = "message".toCharArray(), CAUSE = "cause".toCharArray(),
GET_MESSAGE = "getMessage".toCharArray(), INIT_CAUSE = "initCause".toCharArray();
public static Annotation[] createConstructorProperties(ASTNode source, boolean msgParam, boolean
causeParam) {
if (!msgParam && !causeParam) return null;
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
long[] poss = new long[3];
Arrays.fill(poss, p);
QualifiedTypeReference constructorPropertiesType = new
QualifiedTypeReference(JAVA_BEANS_CONSTRUCTORPROPERTIES, poss);
setGeneratedBy(constructorPropertiesType, source);
SingleMemberAnnotation ann = new SingleMemberAnnotation(constructorPropertiesType, pS);
ann.declarationSourceEnd = pE;
ArrayInitializer fieldNames = new ArrayInitializer();
fieldNames.sourceStart = pS;
fieldNames.sourceEnd = pE;
fieldNames.expressions = new Expression[(msgParam && causeParam) ? 2 : 1];
int ctr = 0;
if (msgParam) {
fieldNames.expressions[ctr] = new StringLiteral(MESSAGE, pS, pE, 0);
setGeneratedBy(fieldNames.expressions[ctr], source);
ctr++;
}
if (causeParam) {
fieldNames.expressions[ctr] = new StringLiteral(CAUSE, pS, pE, 0);
setGeneratedBy(fieldNames.expressions[ctr], source);
ctr++;
}
ann.memberValue = fieldNames;
setGeneratedBy(ann, source);
setGeneratedBy(ann.memberValue, source);
return new Annotation[] { ann };
}
@SuppressWarnings("deprecation") public static ConstructorDeclaration
createConstructor(AccessLevel level, EclipseNode typeNode, boolean msgParam, boolean
causeParam, EclipseNode sourceNode, ExplicitConstructorCall explicitCall, Statement extra) {
ASTNode source = sourceNode.get();
TypeDeclaration typeDeclaration = ((TypeDeclaration) typeNode.get());
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
```

```
boolean addConstructorProperties;
if ((!msgParam && !causeParam) || isLocalType(typeNode)) {
addConstructorProperties = false;
} else {
Boolean v = typeNode.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTOR_ADD_C
ONSTRUCTOR_PROPERTIES);
addConstructorProperties = v != null ? v.booleanValue() :
Boolean.FALSE.equals(typeNode.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTO
R_SUPPRESS_CONSTRUCTOR_PROPERTIES));
}
ConstructorDeclaration constructor = new ConstructorDeclaration(((CompilationUnitDeclaration)
typeNode.top().get()).compilationResult);
constructor.modifiers = toEclipseModifier(level);
constructor.selector = typeDeclaration.name;
constructor.thrownExceptions = null;
constructor.typeParameters = null;
constructor.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
constructor.bodyStart = constructor.declarationSourceStart = constructor.sourceStart = pS;
constructor.bodyEnd = constructor.declarationSourceEnd = constructor.sourceEnd = pE;
constructor.arguments = null;
List<Argument> params = new ArrayList<Argument>();
if (msgParam) {
TypeReference typeRef = new QualifiedTypeReference(TypeConstants.JAVA_LANG_STRING, new
long[] {p, p, p});
Argument parameter = new Argument(MESSAGE, p, typeRef, Modifier.FINAL);
params.add(parameter);
}
if (causeParam) {
TypeReference typeRef = new QualifiedTypeReference(TypeConstants.JAVA_LANG_THROWABLE,
new long[] {p, p, p});
Argument parameter = new Argument(CAUSE, p, typeRef, Modifier.FINAL);
params.add(parameter);
}
explicitCall.sourceStart = pS;
explicitCall.sourceEnd = pE;
constructor.constructorCall = explicitCall;
constructor.statements = extra != null ? new Statement[] {extra} : null;
constructor.arguments = params.isEmpty() ? null : params.toArray(new Argument[0]);
Annotation[] constructorProperties = null;
if (addConstructorProperties) constructorProperties = createConstructorProperties(source, msgParam,
causeParam);
constructor.annotations = copyAnnotations(source, constructorProperties);
constructor.traverse(new SetGeneratedByVisitor(source), typeDeclaration.scope);
return constructor;
}
public static boolean isLocalType(EclipseNode type) {
Kind kind = type.up().getKind();
if (kind == Kind.COMPILATION_UNIT) return false;
if (kind == Kind.TYPE) return isLocalType(type.up());
return true;
}
}
```

lombok/eclipse/handlers/HandleSuperBuilder.java

```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.eclipse.handlers.HandleBuilder.*;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.ExplicitConstructorCall;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FalseLiteral;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
```

```
import org.eclipse.jdt.internal.compiler.ast.Initializer;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SuperReference;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeParameter;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.Wildcard;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.ClassScope;
import org.eclipse.jdt.internal.compiler.lookup.ExtraCompilerModifiers;
import org.eclipse.jdt.internal.compiler.lookup.MethodScope;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
import lombok.AccessLevel;
import lombok.Builder;
import lombok.Builder.ObtainVia;
import lombok.ConfigurationKeys;
import lombok.Singular;
import lombok.ToString;
import lombok.core.AST.Kind;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.CopyJavadoc;
import lombok.eclipse.handlers.EclipseHandlerUtil.MemberExistsResult;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
import lombok.eclipse.handlers.EclipseSingularsRecipes.StatementMaker;
import lombok.eclipse.handlers.EclipseSingularsRecipes.TypeReferenceMaker;
import lombok.eclipse.handlers.HandleBuilder.BuilderFieldData;
import lombok.eclipse.handlers.HandleBuilder.BuilderJob;
import lombok.experimental.NonFinal;
import lombok.experimental.SuperBuilder;
import lombok.spi.Provides;
@Provides
```

```
@HandlerPriority(-1024) //-2^10; to ensure we've picked up @FieldDefault's changes (-2048) but
@Value hasn't removed itself yet (-512), so that we can error on presence of it on the builder classes.
public class HandleSuperBuilder extends EclipseAnnotationHandler<SuperBuilder> {
private static final char[] SELF_METHOD_NAME = "self".toCharArray();
private static final char[] FILL_VALUES_METHOD_NAME = "$fillValuesFrom".toCharArray();
private static final char[] FILL_VALUES_STATIC_METHOD_NAME =
"$fillValuesFromInstanceIntoBuilder".toCharArray();
private static final char[] INSTANCE_VARIABLE_NAME = "instance".toCharArray();
private static final String BUILDER_VARIABLE_NAME_STRING = "b";
private static final char[] BUILDER_VARIABLE_NAME =
BUILDER_VARIABLE_NAME_STRING.toCharArray();
class SuperBuilderJob extends BuilderJob {
void init(AnnotationValues<SuperBuilder> annValues, SuperBuilder ann, EclipseNode node) {
accessOuters = accessInners = AccessLevel.PUBLIC;
oldFluent = true;
oldChain = true;
builderMethodName = ann.builderMethodName();
buildMethodName = ann.buildMethodName();
toBuilder = ann.toBuilder();
if (builderMethodName == null) builderMethodName = "builder";
if (buildMethodName == null) buildMethodName = "build";
builderClassName = getBuilderClassNameTemplate(node, null);
}
EclipseNode builderAbstractType;
String builderAbstractClassName;
char[] builderAbstractClassNameArr;
EclipseNode builderImplType;
String builderImplClassName;
char[] builderImplClassNameArr;
private TypeParameter[] builderTypeParams_;
void setBuilderToImpl() {
builderType = builderImplType;
builderClassName = builderImplClassName;
builderClassNameArr = builderImplClassNameArr;
builderTypeParams = typeParams;
}
void setBuilderToAbstract() {
builderType = builderAbstractType;
builderClassName = builderAbstractClassName;
builderClassNameArr = builderAbstractClassNameArr;
builderTypeParams = builderTypeParams_;
}
}
@Override public void handle(AnnotationValues<SuperBuilder> annotation, Annotation ast,
EclipseNode annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.SUPERBUILDER_FLAG_USAGE,
"@SuperBuilder");
SuperBuilderJob job = new SuperBuilderJob();
job.sourceNode = annotationNode;
job.source = ast;
job.checkerFramework = getCheckerFrameworkVersion(annotationNode);
job.isStatic = true;
SuperBuilder annInstance = annotation.getInstance();
```

```
job.init(annotation, annInstance, annotationNode);
boolean generateBuilderMethod;
if (job.builderMethodName.isEmpty()) {
generateBuilderMethod = false;
} else if (!checkName("builderMethodName", job.builderMethodName, annotationNode)) {
return;
} else {
generateBuilderMethod = true;
}
if (!checkName("buildMethodName", job.buildMethodName, annotationNode)) return;
EclipseNode parent = annotationNode.up();
job.builderFields = new ArrayList<BuilderFieldData>();
TypeReference buildMethodReturnType;
boolean addCleaning = false;
List<EclipseNode> nonFinalNonDefaultedFields = null;
if (!isClass(parent)) {
annotationNode.addError("@SuperBuilder is only supported on classes.");
return;
}
if (!isStaticAllowed(parent)) {
annotationNode.addError("@SuperBuilder is not supported on non-static nested classes.");
return;
}
job.parentType = parent;
TypeDeclaration td = (TypeDeclaration) parent.get();
// Gather all fields of the class that should be set by the builder.
List<EclipseNode> allFields = new ArrayList<EclipseNode>();
boolean valuePresent = (hasAnnotation(lombok.Value.class, parent) ||
hasAnnotation("lombok.experimental.Value", parent));
for (EclipseNode fieldNode : HandleConstructor.findAllFields(parent, true)) {
FieldDeclaration fd = (FieldDeclaration) fieldNode.get();
EclipseNode isDefault = findAnnotation(Builder.Default.class, fieldNode);
boolean isFinal = ((fd.modifiers & ClassFileConstants.AccFinal) != 0) || (valuePresent &&
!hasAnnotation(NonFinal.class, fieldNode));
Annotation[] copyableAnnotations = findCopyableAnnotations(fieldNode);
BuilderFieldData bfd = new BuilderFieldData();
bfd.rawName = fieldNode.getName().toCharArray();
bfd.name = removePrefixFromField(fieldNode);
bfd.builderFieldName = bfd.name;
bfd.annotations = copyAnnotations(fd, copyableAnnotations);
bfd.type = fd.type;
bfd.singularData = getSingularData(fieldNode, ast, annInstance.setterPrefix());
bfd.originalFieldNode = fieldNode;
if (bfd.singularData != null && isDefault != null) {
isDefault.addError("@Builder.Default and @Singular cannot be mixed.");
isDefault = null;
}
if (fd.initialization == null && isDefault != null) {
isDefault.addWarning("@Builder.Default requires an initializing expression (' = something;').");
isDefault = null;
}
if (fd.initialization != null && isDefault == null) {
if (isFinal) continue;
```

```
if (nonFinalNonDefaultedFields == null) nonFinalNonDefaultedFields = new ArrayList<EclipseNode>();
nonFinalNonDefaultedFields.add(fieldNode);
}
if (isDefault != null) {
bfd.nameOfDefaultProvider = prefixWith(DEFAULT_PREFIX, bfd.name);
bfd.nameOfSetFlag = prefixWith(bfd.name, SET_PREFIX);
bfd.builderFieldName = prefixWith(bfd.name, VALUE_PREFIX);
MethodDeclaration md = HandleBuilder.generateDefaultProvider(bfd.nameOfDefaultProvider,
td.typeParameters, fieldNode, ast);
if (md != null) injectMethod(parent, md);
}
addObtainVia(bfd, fieldNode);
job.builderFields.add(bfd);
allFields.add(fieldNode);
}
job.typeParams = td.typeParameters != null ? td.typeParameters : new TypeParameter[0];
buildMethodReturnType = job.createBuilderParentTypeReference();
// <C, B> are the generics for our builder.
String classGenericName = "C";
String builderGenericName = "B";
// We have to make sure that the generics' names do not collide with any generics on the annotated
class,
// the classname itself, or any member type name of the annotated class.
// For instance, if there are generics <B, B2, C> on the annotated class, use "C2" and "B3" for our
builder.
java.util.Set<String> usedNames = gatherUsedTypeNames(job.typeParams, td);
classGenericName = generateNonclashingNameFor(classGenericName, usedNames);
builderGenericName = generateNonclashingNameFor(builderGenericName, usedNames);
TypeParameter[] paddedTypeParameters; {
paddedTypeParameters = new TypeParameter[job.typeParams.length + 2];
System.arraycopy(job.typeParams, 0, paddedTypeParameters, 0, job.typeParams.length);
TypeParameter c = new TypeParameter();
c.name = classGenericName.toCharArray();
c.type = cloneSelfType(job.parentType, job.source);
paddedTypeParameters[paddedTypeParameters.length - 2] = c;
TypeParameter b = new TypeParameter();
b.name = builderGenericName.toCharArray();
b.type = cloneSelfType(job.parentType, job.source);
paddedTypeParameters[paddedTypeParameters.length - 1] = b;
}
job.builderTypeParams = job.builderTypeParams_ = paddedTypeParameters;
TypeReference extendsClause = td.superclass;
TypeReference superclassBuilderClass = null;
TypeReference[] typeArguments = new TypeReference[] {
new SingleTypeReference(classGenericName.toCharArray(), 0),
new SingleTypeReference(builderGenericName.toCharArray(), 0),
};
if (extendsClause instanceof QualifiedTypeReference) {
QualifiedTypeReference qualifiedTypeReference = (QualifiedTypeReference)extendsClause;
char[] superclassClassName = qualifiedTypeReference.getLastToken();
String builderClassNameTemplate = BuilderJob.getBuilderClassNameTemplate(annotationNode, null);
String superclassBuilderClassName = job.replaceBuilderClassName(superclassClassName,
builderClassNameTemplate);
```

```
char[][] tokens = Arrays.copyOf(qualifiedTypeReference.tokens, qualifiedTypeReference.tokens.length
+ 1);
tokens[tokens.length-1] = superclassBuilderClassName.toCharArray();
long[] poss = new long[tokens.length];
Arrays.fill(poss, job.getPos());
TypeReference[] superclassTypeArgs = getTypeParametersFrom(extendsClause);
// Every token may potentially have type args. Here, we only have
// type args for the last token, the superclass' builder.
TypeReference[][] typeArgsForTokens = new TypeReference[tokens.length][];
typeArgsForTokens[typeArgsForTokens.length-1] = mergeTypeReferences(superclassTypeArgs,
typeArguments);
superclassBuilderClass = new ParameterizedQualifiedTypeReference(tokens, typeArgsForTokens, 0,
poss);
} else if (extendsClause != null) {
char[] superclassClassName = extendsClause.getTypeName()[0];
String builderClassNameTemplate = BuilderJob.getBuilderClassNameTemplate(annotationNode, null);
String superclassBuilderClassName = job.replaceBuilderClassName(superclassClassName,
builderClassNameTemplate);
char[][] tokens = new char[][] {superclassClassName, superclassBuilderClassName.toCharArray()};
long[] poss = new long[tokens.length];
Arrays.fill(poss, job.getPos());
TypeReference[] superclassTypeArgs = getTypeParametersFrom(extendsClause);
// Every token may potentially have type args. Here, we only have
// type args for the last token, the superclass' builder.
TypeReference[][] typeArgsForTokens = new TypeReference[tokens.length][];
typeArgsForTokens[typeArgsForTokens.length-1] = mergeTypeReferences(superclassTypeArgs,
typeArguments);
superclassBuilderClass = new ParameterizedQualifiedTypeReference(tokens, typeArgsForTokens, 0,
poss);
}
job.builderAbstractClassName = job.builderClassName = job.replaceBuilderClassName(td.name);
job.builderAbstractClassNameArr = job.builderClassNameArr =
job.builderAbstractClassName.toCharArray();
job.builderImplClassName = job.builderAbstractClassName + "Impl";
job.builderImplClassNameArr = job.builderImplClassName.toCharArray();
// If there is no superclass, superclassBuilderClassExpression is still == null at this point.
// You can use it to check whether to inherit or not.
if (!constructorExists(parent, job.builderClassName)) {
generateBuilderBasedConstructor(job, superclassBuilderClass != null);
}
// Create the abstract builder class, or reuse an existing one.
job.builderAbstractType = findInnerClass(parent, job.builderClassName);
if (job.builderAbstractType == null) {
job.builderAbstractType = generateBuilderAbstractClass(job, superclassBuilderClass,
classGenericName, builderGenericName);
} else {
TypeDeclaration builderTypeDeclaration = (TypeDeclaration) job.builderAbstractType.get();
if ((builderTypeDeclaration.modifiers & (ClassFileConstants.AccStatic |
ClassFileConstants.AccAbstract)) == 0) {
annotationNode.addError("Existing Builder must be an abstract static inner class.");
return;
}
```

```
sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(job.builderAbstractType,
annotationNode);
// Generate errors for @Singular BFDs that have one already defined node.
for (BuilderFieldData bfd : job.builderFields) {
SingularData sd = bfd.singularData;
if (sd == null) continue;
EclipseSingularizer singularizer = sd.getSingularizer();
if (singularizer == null) continue;
if (singularizer.checkForAlreadyExistingNodesAndGenerateError(job.builderAbstractType, sd)) {
bfd.singularData = null;
}
}
}
// Check validity of @ObtainVia fields, and add check if adding cleaning for @Singular is necessary.
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
if (bfd.singularData.getSingularizer().requiresCleaning()) {
addCleaning = true;
break;
}
}
if (bfd.obtainVia != null) {
if (bfd.obtainVia.field().isEmpty() == bfd.obtainVia.method().isEmpty()) {
bfd.obtainViaNode.addError("The syntax is either @ObtainVia(field = \"fieldName\") or
@ObtainVia(method = \"methodName\").");
return;
}
if (bfd.obtainVia.method().isEmpty() && bfd.obtainVia.isStatic()) {
bfd.obtainViaNode.addError("@ObtainVia(isStatic = true) is not valid unless 'method' has been set.");
return;
}
}
}
// Generate the fields in the abstract builder class that hold the values for the instance.
job.setBuilderToAbstract();
generateBuilderFields(job);
if (addCleaning) {
FieldDeclaration cleanDecl = new FieldDeclaration(CLEAN_FIELD_NAME, 0, -1);
cleanDecl.declarationSourceEnd = -1;
cleanDecl.modifiers = ClassFileConstants.AccPrivate;
cleanDecl.type = TypeReference.baseTypeReference(TypeIds.T_boolean, 0);
injectFieldAndMarkGenerated(job.builderType, cleanDecl);
}
if (job.toBuilder) {
// Generate $fillValuesFrom() method in the abstract builder.
injectMethod(job.builderType, generateFillValuesMethod(job, superclassBuilderClass != null,
builderGenericName, classGenericName));
// Generate $fillValuesFromInstanceIntoBuilder() method in the builder implementation class.
injectMethod(job.builderType, generateStaticFillValuesMethod(job, annInstance.setterPrefix()));
}
// Create the setter methods in the abstract builder.
for (BuilderFieldData bfd : job.builderFields) {
generateSetterMethodsForBuilder(job, bfd, builderGenericName, annInstance.setterPrefix());
```

```
}
// Generate abstract self() and build() methods in the abstract builder.
injectMethod(job.builderType, generateAbstractSelfMethod(job, superclassBuilderClass != null,
builderGenericName));
job.setBuilderToAbstract();
injectMethod(job.builderType, generateAbstractBuildMethod(job, superclassBuilderClass != null,
classGenericName));
// Create the toString() method for the abstract builder.
if (methodExists("toString", job.builderType, 0) == MemberExistsResult.NOT_EXISTS) {
List<Included<EclipseNode, ToString.Include>> fieldNodes = new ArrayList<Included<EclipseNode,
ToString.Include>>();
for (BuilderFieldData bfd : job.builderFields) {
for (EclipseNode f : bfd.createdFields) {
fieldNodes.add(new Included<EclipseNode, ToString.Include>(f, null, true, false));
}
}
// Let toString() call super.toString() if there is a superclass, so that it also shows fields from the
superclass' builder.
MethodDeclaration md = HandleToString.createToString(job.builderType, fieldNodes, true,
superclassBuilderClass != null, ast, FieldAccess.ALWAYS_FIELD);
if (md != null) {
injectMethod(job.builderType, md);
}
}
if (addCleaning) {
job.setBuilderToAbstract();
injectMethod(job.builderType, generateCleanMethod(job));
}
boolean isAbstract = (td.modifiers & ClassFileConstants.AccAbstract) != 0;
if (isAbstract) {
// Only non-abstract classes get the Builder implementation.
return;
}
// Create the builder implementation class, or reuse an existing one.
job.builderImplType = findInnerClass(parent, job.builderImplClassName);
if (job.builderImplType == null) {
job.builderImplType = generateBuilderImplClass(job, job.builderImplClassName);
} else {
TypeDeclaration builderImplTypeDeclaration = (TypeDeclaration) job.builderImplType.get();
if ((builderImplTypeDeclaration.modifiers & ClassFileConstants.AccAbstract) != 0 ||
(builderImplTypeDeclaration.modifiers & ClassFileConstants.AccStatic) == 0) {
annotationNode.addError("Existing BuilderImpl must be a non-abstract static inner class.");
return;
}
sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(job.builderImplType, annotationNode);
}
job.setBuilderToImpl();
if (job.toBuilder) {
// Add the toBuilder() method to the annotated class.
switch (methodExists(TO_BUILDER_METHOD_NAME_STRING, job.parentType, 0)) {
case EXISTS_BY_USER:
break;
case NOT_EXISTS:
```

```
injectMethod(parent, generateToBuilderMethod(job));
break;
default:
// Should not happen.
}
}
// Create the self() and build() methods in the BuilderImpl.
job.setBuilderToImpl();
injectMethod(job.builderImplType, generateSelfMethod(job));
if (methodExists(job.buildMethodName, job.builderImplType, -1) ==
MemberExistsResult.NOT_EXISTS) {
job.setBuilderToImpl();
injectMethod(job.builderImplType, generateBuildMethod(job, buildMethodReturnType));
}
// Add the builder() method to the annotated class.
if (generateBuilderMethod && methodExists(job.builderMethodName, parent, -1) !=
MemberExistsResult.NOT_EXISTS) generateBuilderMethod = false;
if (generateBuilderMethod) {
MethodDeclaration md = generateBuilderMethod(job);
if (md != null) injectMethod(parent, md);
}
if (nonFinalNonDefaultedFields != null && generateBuilderMethod) {
for (EclipseNode fieldNode : nonFinalNonDefaultedFields) {
fieldNode.addWarning("@SuperBuilder will ignore the initializing expression entirely. If you want the
initializing expression to serve as default, add @Builder.Default. If it is not supposed to be settable
during building, make the field final.");
}
}
}
private EclipseNode generateBuilderAbstractClass(BuilderJob job, TypeReference
superclassBuilderClass, String classGenericName, String builderGenericName) {
TypeDeclaration parent = (TypeDeclaration) job.parentType.get();
TypeDeclaration builder = new TypeDeclaration(parent.compilationResult);
builder.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
builder.modifiers |= ClassFileConstants.AccPublic | ClassFileConstants.AccStatic |
ClassFileConstants.AccAbstract;
builder.name = job.builderClassNameArr;
// Keep any type params of the annotated class.
builder.typeParameters = Arrays.copyOf(copyTypeParams(job.typeParams, job.source),
job.typeParams.length + 2);
// Add builder-specific type params required for inheritable builders.
// 1. The return type for the build() method, named "C", which extends the annotated class.
TypeParameter o = new TypeParameter();
o.name = classGenericName.toCharArray();
o.type = cloneSelfType(job.parentType, job.source);
builder.typeParameters[builder.typeParameters.length - 2] = o;
// 2. The return type for all setter methods, named "B", which extends this builder class.
o = new TypeParameter();
o.name = builderGenericName.toCharArray();
TypeReference[] typerefs = appendBuilderTypeReferences(job.typeParams, classGenericName,
builderGenericName);
o.type = generateParameterizedTypeReference(job.parentType, job.builderClassNameArr, false,
typerefs, 0);
```

```
builder.typeParameters[builder.typeParameters.length - 1] = o;
if (superclassBuilderClass != null) builder.superclass = copyType(superclassBuilderClass, job.source);
builder.createDefaultConstructor(false, true);
builder.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return injectType(job.parentType, builder);
}
private EclipseNode generateBuilderImplClass(BuilderJob job, String builderImplClass) {
TypeDeclaration parent = (TypeDeclaration) job.parentType.get();
TypeDeclaration builder = new TypeDeclaration(parent.compilationResult);
builder.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
builder.modifiers |= ClassFileConstants.AccPrivate | ClassFileConstants.AccStatic |
ClassFileConstants.AccFinal;
builder.name = builderImplClass.toCharArray();
// Add type params if there are any.
if (job.typeParams != null && job.typeParams.length > 0) builder.typeParameters =
copyTypeParams(job.typeParams, job.source);
if (job.builderClassName != null) {
// Extend the abstract builder.
// 1. Add any type params of the annotated class.
TypeReference[] typeArgs = new TypeReference[job.typeParams.length + 2];
for (int i = 0; i < job.typeParams.length; i++) {
typeArgs[i] = new SingleTypeReference(job.typeParams[i].name, 0);
}
// 2. The return type for the build() method (named "C" in the abstract builder), which is the annotated
class.
// 3. The return type for all setter methods (named "B" in the abstract builder), which is this builder
class.
typeArgs[typeArgs.length - 2] = cloneSelfType(job.parentType, job.source);
typeArgs[typeArgs.length - 1] = createTypeReferenceWithTypeParameters(job.parentType,
builderImplClass, job.typeParams);
builder.superclass = generateParameterizedTypeReference(job.parentType, job.builderClassNameArr,
false, typeArgs, 0);
}
builder.createDefaultConstructor(false, true);
builder.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return injectType(job.parentType, builder);
}
/**
* Generates a constructor that has a builder as the only parameter.
* The values from the builder are used to initialize the fields of new instances.
*
* @param callBuilderBasedSuperConstructor
* If {@code true}, the constructor will explicitly call a super
* constructor with the builder as argument. Requires
* {@code builderClassAsParameter != null}.
*/
private void generateBuilderBasedConstructor(BuilderJob job, boolean
callBuilderBasedSuperConstructor) {
TypeDeclaration typeDeclaration = ((TypeDeclaration) job.parentType.get());
long p = job.getPos();
ConstructorDeclaration constructor = new ConstructorDeclaration(((CompilationUnitDeclaration)
job.parentType.top().get()).compilationResult);
constructor.modifiers = toEclipseModifier(AccessLevel.PROTECTED);
```

```
constructor.selector = typeDeclaration.name;
if (callBuilderBasedSuperConstructor) {
constructor.constructorCall = new ExplicitConstructorCall(ExplicitConstructorCall.Super);
constructor.constructorCall.arguments = new Expression[] {new
SingleNameReference(BUILDER_VARIABLE_NAME, p)};
} else {
constructor.constructorCall = new ExplicitConstructorCall(ExplicitConstructorCall.ImplicitSuper);
}
constructor.constructorCall.sourceStart = job.source.sourceStart;
constructor.constructorCall.sourceEnd = job.source.sourceEnd;
constructor.thrownExceptions = null;
constructor.typeParameters = null;
constructor.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
constructor.bodyStart = constructor.declarationSourceStart = constructor.sourceStart =
job.source.sourceStart;
constructor.bodyEnd = constructor.declarationSourceEnd = constructor.sourceEnd =
job.source.sourceEnd;
TypeReference[] wildcards = new TypeReference[] {new Wildcard(Wildcard.UNBOUND), new
Wildcard(Wildcard.UNBOUND)};
TypeReference builderType = generateParameterizedTypeReference(job.parentType,
job.builderClassNameArr, false, mergeToTypeReferences(job.typeParams, wildcards), p);
constructor.arguments = new Argument[] {new Argument(BUILDER_VARIABLE_NAME, p,
builderType, Modifier.FINAL)};
List<Statement> statements = new ArrayList<Statement>();
for (BuilderFieldData fieldNode : job.builderFields) {
FieldReference fieldInThis = new FieldReference(fieldNode.rawName, p);
int s = (int) (p >> 32);
int e = (int) p;
fieldInThis.receiver = new ThisReference(s, e);
Expression assignmentExpr;
if (fieldNode.singularData != null && fieldNode.singularData.getSingularizer() != null) {
fieldNode.singularData.getSingularizer().appendBuildCode(fieldNode.singularData, job.parentType,
statements, fieldNode.builderFieldName, BUILDER_VARIABLE_NAME_STRING);
assignmentExpr = new SingleNameReference(fieldNode.builderFieldName, p);
} else {
char[][] variableInBuilder = new char[][] {BUILDER_VARIABLE_NAME, fieldNode.builderFieldName};
long[] positions = new long[] {p, p};
assignmentExpr = new QualifiedNameReference(variableInBuilder, positions, s, e);
}
Statement assignment = new Assignment(fieldInThis, assignmentExpr, (int) p);
// In case of @Builder.Default, set the value to the default if it was NOT set in the builder.
if (fieldNode.nameOfSetFlag != null) {
char[][] setVariableInBuilder = new char[][] {BUILDER_VARIABLE_NAME, fieldNode.nameOfSetFlag};
long[] positions = new long[] {p, p};
QualifiedNameReference setVariableInBuilderRef = new
QualifiedNameReference(setVariableInBuilder, positions, s, e);
MessageSend defaultMethodCall = new MessageSend();
defaultMethodCall.sourceStart = job.source.sourceStart;
defaultMethodCall.sourceEnd = job.source.sourceEnd;
defaultMethodCall.receiver = generateNameReference(job.parentType, 0L);
defaultMethodCall.selector = fieldNode.nameOfDefaultProvider;
defaultMethodCall.typeArguments = typeParameterNames(((TypeDeclaration)
job.parentType.get()).typeParameters);
```

```
Statement defaultAssignment = new Assignment(fieldInThis, defaultMethodCall, (int) p);
IfStatement ifBlockForDefault = new IfStatement(setVariableInBuilderRef, assignment,
defaultAssignment, s, e);
statements.add(ifBlockForDefault);
} else {
statements.add(assignment);
}
if (hasNonNullAnnotations(fieldNode.originalFieldNode)) {
Statement nullCheck = generateNullCheck((FieldDeclaration) fieldNode.originalFieldNode.get(),
job.sourceNode, null);
if (nullCheck != null) statements.add(nullCheck);
}
}
constructor.statements = statements.isEmpty() ? null : statements.toArray(new Statement[0]);
if (job.checkerFramework.generateSideEffectFree()) {
constructor.annotations = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE)};
}
constructor.traverse(new SetGeneratedByVisitor(job.source), typeDeclaration.scope);
injectMethod(job.parentType, constructor);
}
private MethodDeclaration generateBuilderMethod(SuperBuilderJob job) {
int pS = job.source.sourceStart, pE = job.source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = job.builderMethodName.toCharArray();
out.modifiers = ClassFileConstants.AccPublic | ClassFileConstants.AccStatic;
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
// Add type params if there are any.
if (job.typeParams != null && job.typeParams.length > 0) out.typeParameters =
copyTypeParams(job.typeParams, job.source);
TypeReference[] wildcards = new TypeReference[] {new Wildcard(Wildcard.UNBOUND), new
Wildcard(Wildcard.UNBOUND) };
out.returnType = generateParameterizedTypeReference(job.parentType,
job.builderAbstractClassNameArr, false, mergeToTypeReferences(job.typeParams, wildcards), p);
if (job.checkerFramework.generateUnique()) {
int len = out.returnType.getTypeName().length;
out.returnType.annotations = new Annotation[len][];
out.returnType.annotations[len - 1] = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__UNIQUE)};
}
AllocationExpression invoke = new AllocationExpression();
invoke.type = namePlusTypeParamsToTypeReference(job.parentType, job.builderImplClassNameArr,
false, job.typeParams, p);
out.statements = new Statement[] {new ReturnStatement(invoke, pS, pE)};
if (job.checkerFramework.generateSideEffectFree()) {
out.annotations = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE)};
}
createRelevantNonNullAnnotation(job.parentType, out);
out.traverse(new SetGeneratedByVisitor(job.source), ((TypeDeclaration) job.parentType.get()).scope);
return out;
}
```

```
/**
* Generates a <code>toBuilder()</code> method in the annotated class that looks like this:
* <pre>
* public <i>Foobar</i>.<i>Foobar</i>Builder&lt;?, ?&gt; toBuilder() {
* return new <i.Foobar</i>.<i>Foobar</i>BuilderImpl().$fillValuesFrom(this);
* }
* </pre>
*/
private MethodDeclaration generateToBuilderMethod(SuperBuilderJob job) {
int pS = job.source.sourceStart, pE = job.source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = TO_BUILDER_METHOD_NAME;
out.modifiers = ClassFileConstants.AccPublic;
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
TypeReference[] wildcards = new TypeReference[] {new Wildcard(Wildcard.UNBOUND), new
Wildcard(Wildcard.UNBOUND) };
out.returnType = generateParameterizedTypeReference(job.parentType,
job.builderAbstractClassNameArr, false, mergeToTypeReferences(job.typeParams, wildcards), p);
if (job.checkerFramework.generateUnique()) {
int len = out.returnType.getTypeName().length;
out.returnType.annotations = new Annotation[len][];
out.returnType.annotations[len - 1] = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__UNIQUE)};
}
AllocationExpression newClass = new AllocationExpression();
newClass.type = namePlusTypeParamsToTypeReference(job.parentType,
job.builderImplClassNameArr, false, job.typeParams, p);
MessageSend invokeFillMethod = new MessageSend();
invokeFillMethod.receiver = newClass;
invokeFillMethod.selector = FILL_VALUES_METHOD_NAME;
invokeFillMethod.arguments = new Expression[] {new ThisReference(0, 0)};
out.statements = new Statement[] {new ReturnStatement(invokeFillMethod, pS, pE)};
if (job.checkerFramework.generateSideEffectFree()) {
out.annotations = new Annotation[] {generateNamedAnnotation(job.source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE)};
}
createRelevantNonNullAnnotation(job.parentType, out);
out.traverse(new SetGeneratedByVisitor(job.source), ((TypeDeclaration) job.parentType.get()).scope);
return out;
}
/**
* Generates a <code>$fillValuesFrom()</code> method in the abstract builder class that looks
* like this:
* <pre>
* protected B $fillValuesFrom(final C instance) {
* super.$fillValuesFrom(instance);
* Foobar.FoobarBuilder.$fillValuesFromInstanceIntoBuilder(instance, this);
* return self();
* }
* </pre>
*/
```

```
private MethodDeclaration generateFillValuesMethod(SuperBuilderJob job, boolean inherited, String
builderGenericName, String classGenericName) {
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = FILL_VALUES_METHOD_NAME;
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.modifiers = ClassFileConstants.AccProtected;
if (inherited) out.annotations = new Annotation[]
{makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE, job.parentType.get())};
out.returnType = new SingleTypeReference(builderGenericName.toCharArray(), 0);
TypeReference builderType = new SingleTypeReference(classGenericName.toCharArray(), 0);
out.arguments = new Argument[] {new Argument(INSTANCE_VARIABLE_NAME, 0, builderType,
Modifier.FINAL)};
List<Statement> body = new ArrayList<Statement>();
if (inherited) {
// Call super.
MessageSend callToSuper = new MessageSend();
callToSuper.receiver = new SuperReference(0, 0);
callToSuper.selector = FILL_VALUES_METHOD_NAME;
callToSuper.arguments = new Expression[] {new
SingleNameReference(INSTANCE_VARIABLE_NAME, 0)};
body.add(callToSuper);
}
// Call the builder implemention's helper method that actually fills the values from the instance.
MessageSend callStaticFillValuesMethod = new MessageSend();
callStaticFillValuesMethod.receiver = generateNameReference(job.parentType,
job.builderAbstractClassNameArr, 0);
callStaticFillValuesMethod.selector = FILL_VALUES_STATIC_METHOD_NAME;
callStaticFillValuesMethod.arguments = new Expression[] {new
SingleNameReference(INSTANCE_VARIABLE_NAME, 0), new ThisReference(0, 0)};
body.add(callStaticFillValuesMethod);
// Return self().
MessageSend returnCall = new MessageSend();
returnCall.receiver = ThisReference.implicitThis();
returnCall.selector = SELF_METHOD_NAME;
body.add(new ReturnStatement(returnCall, 0, 0));
out.statements = body.isEmpty() ? null : body.toArray(new Statement[0]);
return out;
}
/**
* Generates a <code>$fillValuesFromInstanceIntoBuilder()</code> method in
* the builder implementation class that copies all fields from the instance
* to the builder. It looks like this:
*
* <pre>
* protected B $fillValuesFromInstanceIntoBuilder(Foobar instance, FoobarBuilder&lt;?, ?&gt; b) {
* b.field(instance.field);
* }
* </pre>
* @param setterPrefix the prefix for setter methods
*/
private MethodDeclaration generateStaticFillValuesMethod(BuilderJob job, String setterPrefix) {
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = FILL_VALUES_STATIC_METHOD_NAME;
```

```
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.modifiers = ClassFileConstants.AccPrivate | ClassFileConstants.AccStatic;
out.returnType = TypeReference.baseTypeReference(TypeIds.T_void, 0);
TypeReference[] wildcards = new TypeReference[] {new Wildcard(Wildcard.UNBOUND), new
Wildcard(Wildcard.UNBOUND)};
TypeReference builderType = generateParameterizedTypeReference(job.parentType,
job.builderClassNameArr, false, mergeToTypeReferences(job.typeParams, wildcards), 0);
Argument builderArgument = new Argument(BUILDER_VARIABLE_NAME, 0, builderType,
Modifier.FINAL);
TypeReference[] typerefs = null;
if (job.typeParams.length > 0) {
typerefs = new TypeReference[job.typeParams.length];
for (int i = 0; i < job.typeParams.length; i++) typerefs[i] = new
SingleTypeReference(job.typeParams[i].name, 0);
}
long p = job.getPos();
TypeReference parentArgument = typerefs == null ? generateTypeReference(job.parentType, p) :
generateParameterizedTypeReference(job.parentType, typerefs, p);
out.arguments = new Argument[] {new Argument(INSTANCE_VARIABLE_NAME, 0, parentArgument,
Modifier.FINAL), builderArgument};
// Add type params if there are any.
if (job.typeParams.length > 0) out.typeParameters = copyTypeParams(job.typeParams, job.source);
List<Statement> body = new ArrayList<Statement>();
// Call the builder's setter methods to fill the values from the instance.
for (BuilderFieldData bfd : job.builderFields) {
MessageSend exec = createSetterCallWithInstanceValue(bfd, job.parentType, job.source,
setterPrefix);
body.add(exec);
}
out.statements = body.isEmpty() ? null : body.toArray(new Statement[0]);
out.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return out;
}
private MessageSend createSetterCallWithInstanceValue(BuilderFieldData bfd, EclipseNode type,
ASTNode source, String setterPrefix) {
char[] setterName = HandlerUtil.buildAccessorName(type, setterPrefix,
String.valueOf(bfd.name)).toCharArray();
MessageSend ms = new MessageSend();
Expression[] tgt = new Expression[bfd.singularData == null ? 1 : 2];
if (bfd.obtainVia == null || !bfd.obtainVia.field().isEmpty()) {
char[] fieldName = bfd.obtainVia == null ? bfd.rawName : bfd.obtainVia.field().toCharArray();
for (int i = 0; i < tgt.length; i++) {
FieldReference fr = new FieldReference(fieldName, 0);
fr.receiver = new SingleNameReference(INSTANCE_VARIABLE_NAME, 0);
tgt[i] = fr;
}
} else {
String obtainName = bfd.obtainVia.method();
boolean obtainIsStatic = bfd.obtainVia.isStatic();
for (int i = 0; i < tgt.length; i++) {
MessageSend obtainExpr = new MessageSend();
obtainExpr.receiver = obtainIsStatic ? generateNameReference(type, 0) : new
SingleNameReference(INSTANCE_VARIABLE_NAME, 0);
```

```
obtainExpr.selector = obtainName.toCharArray();
if (obtainIsStatic) obtainExpr.arguments = new Expression[] {new
SingleNameReference(INSTANCE_VARIABLE_NAME, 0)};
tgt[i] = obtainExpr;
}
}
if (bfd.singularData == null) {
ms.arguments = tgt;
} else {
Expression ifNull = new EqualExpression(tgt[0], new NullLiteral(0, 0), OperatorIds.EQUAL_EQUAL);
MessageSend emptyCollection =
bfd.singularData.getSingularizer().getEmptyExpression(bfd.singularData.getTargetFqn(),
bfd.singularData, type, source);
ms.arguments = new Expression[] {new ConditionalExpression(ifNull, emptyCollection, tgt[1])};
}
ms.receiver = new SingleNameReference(BUILDER_VARIABLE_NAME, 0);
ms.selector = setterName;
return ms;
}
private MethodDeclaration generateAbstractSelfMethod(BuilderJob job, boolean override, String
builderGenericName) {
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = SELF_METHOD_NAME;
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.modifiers = ClassFileConstants.AccAbstract | ClassFileConstants.AccProtected |
ExtraCompilerModifiers.AccSemicolonBody;
Annotation overrideAnn = override ?
makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE, job.parentType.get()) : null;
Annotation sefAnn = job.checkerFramework.generatePure() ?
generateNamedAnnotation(job.parentType.get(), CheckerFrameworkVersion.NAME__PURE): null;
if (overrideAnn != null && sefAnn != null) out.annotations = new Annotation[] {overrideAnn, sefAnn};
else if (overrideAnn != null) out.annotations = new Annotation[] {overrideAnn};
else if (sefAnn != null) out.annotations = new Annotation[] {sefAnn};
out.returnType = new SingleTypeReference(builderGenericName.toCharArray(), 0);
addCheckerFrameworkReturnsReceiver(out.returnType, job.parentType.get(), job.checkerFramework);
return out;
}
private MethodDeclaration generateSelfMethod(BuilderJob job) {
MethodDeclaration out = job.createNewMethodDeclaration();
out.selector = SELF_METHOD_NAME;
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.modifiers = ClassFileConstants.AccProtected;
Annotation overrideAnn = makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE,
job.builderType.get());
Annotation sefAnn = job.checkerFramework.generatePure() ?
generateNamedAnnotation(job.builderType.get(), CheckerFrameworkVersion.NAME__PURE) : null;
if (sefAnn != null) out.annotations = new Annotation[] {overrideAnn, sefAnn};
else out.annotations = new Annotation[] {overrideAnn};
out.returnType = namePlusTypeParamsToTypeReference(job.builderType, job.typeParams,
job.getPos());
addCheckerFrameworkReturnsReceiver(out.returnType, job.parentType.get(), job.checkerFramework);
out.statements = new Statement[] {new ReturnStatement(new ThisReference(0, 0), 0, 0)};
return out;
```

```
}
private MethodDeclaration generateAbstractBuildMethod(BuilderJob job, boolean override, String
classGenericName) {
MethodDeclaration out = job.createNewMethodDeclaration();
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.modifiers = ClassFileConstants.AccPublic | ClassFileConstants.AccAbstract |
ExtraCompilerModifiers.AccSemicolonBody;
out.selector = job.buildMethodName.toCharArray();
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.returnType = new SingleTypeReference(classGenericName.toCharArray(), 0);
Annotation overrideAnn = override ?
makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE, job.source) : null;
Annotation sefAnn = job.checkerFramework.generateSideEffectFree() ?
generateNamedAnnotation(job.source, CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE):
null;
if (overrideAnn != null && sefAnn != null) out.annotations = new Annotation[] {overrideAnn, sefAnn};
else if (overrideAnn != null) out.annotations = new Annotation[] {overrideAnn};
else if (sefAnn != null) out.annotations = new Annotation[] {sefAnn};
out.receiver = HandleBuilder.generateBuildReceiver(job);
out.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return out;
}
private MethodDeclaration generateBuildMethod(BuilderJob job, TypeReference returnType) {
MethodDeclaration out = job.createNewMethodDeclaration();
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
List<Statement> statements = new ArrayList<Statement>();
out.modifiers = ClassFileConstants.AccPublic;
out.selector = job.buildMethodName.toCharArray();
out.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
out.returnType = returnType;
Annotation overrideAnn = makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE,
job.source);
Annotation sefAnn = job.checkerFramework.generateSideEffectFree() ?
generateNamedAnnotation(job.source, CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE):
null;
if (sefAnn != null) out.annotations = new Annotation[] {overrideAnn, sefAnn};
else out.annotations = new Annotation[] {overrideAnn};
AllocationExpression allocationStatement = new AllocationExpression();
allocationStatement.type = copyType(out.returnType);
// Use a constructor that only has this builder as parameter.
allocationStatement.arguments = new Expression[] {new ThisReference(0, 0)};
statements.add(new ReturnStatement(allocationStatement, 0, 0));
out.statements = statements.isEmpty() ? null : statements.toArray(new Statement[0]);
out.receiver = HandleBuilder.generateBuildReceiver(job);
createRelevantNonNullAnnotation(job.builderType, out);
out.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return out;
}
private MethodDeclaration generateCleanMethod(BuilderJob job) {
List<Statement> statements = new ArrayList<Statement>();
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
```

```
bfd.singularData.getSingularizer().appendCleaningCode(bfd.singularData, job.builderType,
statements);
}
}
FieldReference thisUnclean = new FieldReference(CLEAN_FIELD_NAME, 0);
thisUnclean.receiver = new ThisReference(0, 0);
statements.add(new Assignment(thisUnclean, new FalseLiteral(0, 0), 0));
MethodDeclaration decl = job.createNewMethodDeclaration();
//new MethodDeclaration(((CompilationUnitDeclaration) builderType.top().get()).compilationResult);
decl.selector = CLEAN_METHOD_NAME;
decl.modifiers = ClassFileConstants.AccPrivate;
decl.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
decl.returnType = TypeReference.baseTypeReference(TypeIds.T_void, 0);
decl.statements = statements.toArray(new Statement[0]);
decl.traverse(new SetGeneratedByVisitor(job.source), (ClassScope) null);
return decl;
}
private void generateBuilderFields(BuilderJob job) {
List<EclipseNode> existing = new ArrayList<EclipseNode>();
for (EclipseNode child : job.builderType.down()) {
if (child.getKind() == Kind.FIELD) existing.add(child);
}
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.createdFields.addAll(bfd.singularData.getSingularizer().generateFields(bfd.singularData,
job.builderType));
} else {
EclipseNode field = null, setFlag = null;
for (EclipseNode exists : existing) {
char[] n = ((FieldDeclaration) exists.get()).name;
if (Arrays.equals(n, bfd.builderFieldName)) field = exists;
if (bfd.nameOfSetFlag != null && Arrays.equals(n, bfd.nameOfSetFlag)) setFlag = exists;
}
if (field == null) {
FieldDeclaration fd = new FieldDeclaration(bfd.builderFieldName.clone(), 0, 0);
fd.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
fd.modifiers = ClassFileConstants.AccPrivate;
fd.type = copyType(bfd.type);
fd.traverse(new SetGeneratedByVisitor(job.source), (MethodScope) null);
field = injectFieldAndMarkGenerated(job.builderType, fd);
}
if (setFlag == null && bfd.nameOfSetFlag != null) {
FieldDeclaration fd = new FieldDeclaration(bfd.nameOfSetFlag, 0, 0);
fd.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
fd.modifiers = ClassFileConstants.AccPrivate;
fd.type = TypeReference.baseTypeReference(TypeIds.T_boolean, 0);
fd.traverse(new SetGeneratedByVisitor(job.source), (MethodScope) null);
injectFieldAndMarkGenerated(job.builderType, fd);
}
bfd.createdFields.add(field);
}
}
}
```

```
private void generateSetterMethodsForBuilder(BuilderJob job, BuilderFieldData bfd, final String
builderGenericName, String setterPrefix) {
boolean deprecate = isFieldDeprecated(bfd.originalFieldNode);
TypeReferenceMaker returnTypeMaker = new TypeReferenceMaker() {
@Override public TypeReference make() {
return new SingleTypeReference(builderGenericName.toCharArray(), 0);
}
};
StatementMaker returnStatementMaker = new StatementMaker() {
@Override public ReturnStatement make() {
MessageSend returnCall = new MessageSend();
returnCall.receiver = ThisReference.implicitThis();
returnCall.selector = SELF_METHOD_NAME;
return new ReturnStatement(returnCall, 0, 0);
}
};
if (bfd.singularData == null || bfd.singularData.getSingularizer() == null) {
generateSimpleSetterMethodForBuilder(job, deprecate, bfd.createdFields.get(0), bfd.name,
bfd.nameOfSetFlag, returnTypeMaker.make(), returnStatementMaker.make(), bfd.annotations,
bfd.originalFieldNode, setterPrefix);
} else {
bfd.singularData.getSingularizer().generateMethods(job.checkerFramework, bfd.singularData,
deprecate, job.builderType, true, returnTypeMaker, returnStatementMaker, AccessLevel.PUBLIC);
}
}
private void generateSimpleSetterMethodForBuilder(BuilderJob job, boolean deprecate, EclipseNode
fieldNode, char[] paramName, char[] nameOfSetFlag, TypeReference returnType, Statement
returnStatement, Annotation[] annosOnParam, EclipseNode originalFieldNode, String setterPrefix) {
TypeDeclaration td = (TypeDeclaration) job.builderType.get();
AbstractMethodDeclaration[] existing = td.methods;
if (existing == null) existing = EMPTY_METHODS;
int len = existing.length;
String setterName = HandlerUtil.buildAccessorName(job.sourceNode, setterPrefix, new
String(paramName));
for (int i = 0; i < len; i++) {
if (!(existing[i] instanceof MethodDeclaration)) continue;
char[] existingName = existing[i].selector;
if (Arrays.equals(setterName.toCharArray(), existingName) && !isTolerate(fieldNode, existing[i])) return;
}
List<Annotation> methodAnnsList =
Arrays.asList(EclipseHandlerUtil.findCopyableToSetterAnnotations(originalFieldNode));
addCheckerFrameworkReturnsReceiver(returnType, job.source, job.checkerFramework);
MethodDeclaration setter = HandleSetter.createSetter(td, deprecate, fieldNode, setterName,
paramName, nameOfSetFlag, returnType, returnStatement, ClassFileConstants.AccPublic,
job.sourceNode, methodAnnsList, annosOnParam != null ? Arrays.asList(copyAnnotations(job.source,
annosOnParam)) : Collections.<Annotation>emptyList());
if (job.sourceNode.up().getKind() == Kind.METHOD) {
copyJavadocFromParam(originalFieldNode.up(), setter, td, paramName.toString());
} else {
copyJavadoc(originalFieldNode, setter, td, CopyJavadoc.SETTER, true);
}
injectMethod(job.builderType, setter);
}
```

```
private void addObtainVia(BuilderFieldData bfd, EclipseNode node) {
for (EclipseNode child : node.down()) {
if (!annotationTypeMatches(ObtainVia.class, child)) continue;
AnnotationValues<ObtainVia> ann = createAnnotation(ObtainVia.class, child);
bfd.obtainVia = ann.getInstance();
bfd.obtainViaNode = child;
return;
}
}
/**
* Returns the explicitly requested singular annotation on this node (field
* or parameter), or null if there's no {@code @Singular} annotation on it.
*
* @param node The node (field or method param) to inspect for its name and potential {@code
@Singular} annotation.
* @param setterPrefix the prefix for setter methods
*/
private SingularData getSingularData(EclipseNode node, ASTNode source, String setterPrefix) {
for (EclipseNode child : node.down()) {
if (!annotationTypeMatches(Singular.class, child)) continue;
char[] pluralName = node.getKind() == Kind.FIELD ? removePrefixFromField(node) :
((AbstractVariableDeclaration) node.get()).name;
AnnotationValues<Singular> ann = createAnnotation(Singular.class, child);
Singular singularInstance = ann.getInstance();
String explicitSingular = singularInstance.value();
if (explicitSingular.isEmpty()) {
if (Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKeys.SINGULAR_AUTO))) {
node.addError("The singular must be specified explicitly (e.g. @Singular(\"task\")) because auto
singularization is disabled.");
explicitSingular = new String(pluralName);
} else {
explicitSingular = autoSingularize(new String(pluralName));
if (explicitSingular == null) {
node.addError("Can't singularize this name; please specify the singular explicitly (i.e.
@Singular(\"sheep\"))");
explicitSingular = new String(pluralName);
}
}
}
char[] singularName = explicitSingular.toCharArray();
TypeReference type = ((AbstractVariableDeclaration) node.get()).type;
TypeReference[] typeArgs = null;
String typeName;
if (type instanceof ParameterizedSingleTypeReference) {
typeArgs = ((ParameterizedSingleTypeReference) type).typeArguments;
typeName = new String(((ParameterizedSingleTypeReference) type).token);
} else if (type instanceof ParameterizedQualifiedTypeReference) {
TypeReference[][] tr = ((ParameterizedQualifiedTypeReference) type).typeArguments;
if (tr != null) typeArgs = tr[tr.length - 1];
char[][] tokens = ((ParameterizedQualifiedTypeReference) type).tokens;
StringBuilder sb = new StringBuilder();
for (int i = 0; i < tokens.length; i++) {
if (i > 0) sb.append(".");
```

```
sb.append(tokens[i]);
}
typeName = sb.toString();
} else {
typeName = type.toString();
}
String targetFqn = EclipseSingularsRecipes.get().toQualified(typeName);
EclipseSingularizer singularizer = EclipseSingularsRecipes.get().getSingularizer(targetFqn);
if (singularizer == null) {
node.addError("Lombok does not know how to create the singular-form builder methods for type '" +
typeName + "'; they won't be generated.");
return null;
}
return new SingularData(child, singularName, pluralName, typeArgs == null ?
Collections.<TypeReference>emptyList() : Arrays.asList(typeArgs), targetFqn, singularizer, source,
singularInstance.ignoreNullCollections(), setterPrefix.toCharArray());
}
return null;
}
private java.util.Set<String> gatherUsedTypeNames(TypeParameter[] typeParams, TypeDeclaration
td) {
java.util.HashSet<String> usedNames = new HashSet<String>();
// 1. Add type parameter names.
for (TypeParameter typeParam : typeParams)
usedNames.add(typeParam.toString());
// 2. Add class name.
usedNames.add(String.valueOf(td.name));
// 3. Add used type names.
if (td.fields != null) {
for (FieldDeclaration field : td.fields) {
if (field instanceof Initializer) continue;
addFirstToken(usedNames, field.type);
}
}
// 4. Add extends and implements clauses.
addFirstToken(usedNames, td.superclass);
if (td.superInterfaces != null) {
for (TypeReference typeReference : td.superInterfaces) {
addFirstToken(usedNames, typeReference);
}
}
return usedNames;
}
private void addFirstToken(java.util.Set<String> usedNames, TypeReference type) {
if (type == null)
return;
// Add the first token, because only that can collide.
char[][] typeName = type.getTypeName();
if (typeName != null && typeName.length >= 1)
usedNames.add(String.valueOf(typeName[0]));
}
private String generateNonclashingNameFor(String classGenericName, java.util.Set<String>
typeParamStrings) {
```

```
if (!typeParamStrings.contains(classGenericName)) return classGenericName;
int counter = 2;
while (typeParamStrings.contains(classGenericName + counter)) counter++;
return classGenericName + counter;
}
private TypeReference[] appendBuilderTypeReferences(TypeParameter[] typeParams, String
classGenericName, String builderGenericName) {
TypeReference[] typeReferencesToAppend = new TypeReference[2];
typeReferencesToAppend[typeReferencesToAppend.length - 2] = new
SingleTypeReference(classGenericName.toCharArray(), 0);
typeReferencesToAppend[typeReferencesToAppend.length - 1] = new
SingleTypeReference(builderGenericName.toCharArray(), 0);
return mergeToTypeReferences(typeParams, typeReferencesToAppend);
}
private TypeReference[] getTypeParametersFrom(TypeReference typeRef) {
TypeReference[][] typeArgss = null;
if (typeRef instanceof ParameterizedQualifiedTypeReference) {
typeArgss = ((ParameterizedQualifiedTypeReference) typeRef).typeArguments;
} else if (typeRef instanceof ParameterizedSingleTypeReference) {
typeArgss = new TypeReference[][] {((ParameterizedSingleTypeReference) typeRef).typeArguments};
}
TypeReference[] typeArgs = new TypeReference[0];
if (typeArgss != null && typeArgss.length > 0) {
typeArgs = typeArgss[typeArgss.length - 1];
}
return typeArgs;
}
private static TypeReference createTypeReferenceWithTypeParameters(EclipseNode parent, String
referenceName, TypeParameter[] typeParams) {
if (typeParams.length > 0) {
TypeReference[] typerefs = new TypeReference[typeParams.length];
for (int i = 0; i < typeParams.length; i++) {
typerefs[i] = new SingleTypeReference(typeParams[i].name, 0);
}
return generateParameterizedTypeReference(parent, referenceName.toCharArray(), false, typerefs, 0);
} else {
return generateTypeReference(parent, referenceName.toCharArray(), false, 0);
}
}
private TypeReference[] mergeToTypeReferences(TypeParameter[] typeParams, TypeReference[]
typeReferencesToAppend) {
TypeReference[] typerefs = new TypeReference[typeParams.length +
typeReferencesToAppend.length];
for (int i = 0; i < typeParams.length; i++) {
typerefs[i] = new SingleTypeReference(typeParams[i].name, 0);
}
for (int i = 0; i < typeReferencesToAppend.length; i++) {
typerefs[typeParams.length + i] = typeReferencesToAppend[i];
}
return typerefs;
}
private TypeReference[] mergeTypeReferences(TypeReference[] refs1, TypeReference[] refs2) {
TypeReference[] result = new TypeReference[refs1.length + refs2.length];
```

```
for (int i = 0; i < refs1.length; i++) result[i] = refs1[i];
for (int i = 0; i < refs2.length; i++) result[refs1.length + i] = refs2[i];
return result;
}
private TypeReference[] typeParameterNames(TypeParameter[] typeParameters) {
if (typeParameters == null) return null;
TypeReference[] trs = new TypeReference[typeParameters.length];
for (int i = 0; i < trs.length; i++) {
trs[i] = new SingleTypeReference(typeParameters[i].name, 0);
}
return trs;
}
private EclipseNode findInnerClass(EclipseNode parent, String name) {
char[] c = name.toCharArray();
for (EclipseNode child : parent.down()) {
if (child.getKind() != Kind.TYPE) continue;
TypeDeclaration td = (TypeDeclaration) child.get();
if (Arrays.equals(td.name, c)) return child;
}
return null;
}
private static final char[] prefixWith(char[] prefix, char[] name) {
char[] out = new char[prefix.length + name.length];
System.arraycopy(prefix, 0, out, 0, prefix.length);
System.arraycopy(name, 0, out, prefix.length, name.length);
return out;
}
private boolean constructorExists(EclipseNode type, String builderClassName) {
if (type != null && type.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration)type.get();
if (typeDecl.methods != null) for (AbstractMethodDeclaration def : typeDecl.methods) {
if (def instanceof ConstructorDeclaration) {
if ((def.bits & ASTNode.IsDefaultConstructor) != 0) continue;
if (!def.isConstructor()) continue;
if (isTolerate(type, def)) continue;
if (def.arguments == null || def.arguments.length != 1) continue;
// Cannot use typeMatches() here, because the parameter could be fully-qualified, partially-qualified, or
not qualified.
// A string-compare of the last part should work. If it's a false-positive, users could still @Tolerate it.
char[] typeName = def.arguments[0].type.getLastToken();
if (builderClassName.equals(String.valueOf(typeName)))
return true;
}
}
}
return false;
}
}
```

lombok/eclipse/handlers/HandleSynchronized.java

/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.EcjAugments.ASTNode_handled;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Modifier;
import java.util.Arrays;
import lombok.ConfigurationKeys;
import lombok.Synchronized;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.AST.Kind;
import lombok.eclipse.DeferUntilPostDiet;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.ArrayAllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SynchronizedStatement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
/**
* Handles the {@code lombok.Synchronized} annotation for eclipse.

```
*/
@Provides
@DeferUntilPostDiet
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the
statements generated by it.
public class HandleSynchronized extends EclipseAnnotationHandler<Synchronized> {
private static final char[] INSTANCE_LOCK_NAME = "$lock".toCharArray();
private static final char[] STATIC_LOCK_NAME = "$LOCK".toCharArray();
@Override public void preHandle(AnnotationValues<Synchronized> annotation, Annotation source,
EclipseNode annotationNode) {
EclipseNode methodNode = annotationNode.up();
if (methodNode == null || methodNode.getKind() != Kind.METHOD || !(methodNode.get() instanceof
MethodDeclaration)) return;
MethodDeclaration method = (MethodDeclaration) methodNode.get();
if (method.isAbstract()) return;
createLockField(annotation, annotationNode, new boolean[] {method.isStatic()}, false);
if (hasParsedBody(getAnnotatedMethod(annotationNode))) {
// This method has a body in diet mode, so we have to handle it now.
handle(annotation, source, annotationNode);
ASTNode_handled.set(source, true);
}
}
public char[] createLockField(AnnotationValues<Synchronized> annotation, EclipseNode
annotationNode, boolean[] isStatic, boolean reportErrors) {
char[] lockName = annotation.getInstance().value().toCharArray();
Annotation source = (Annotation) annotationNode.get();
boolean autoMake = false;
if (lockName.length == 0) {
autoMake = true;
lockName = isStatic[0] ? STATIC_LOCK_NAME : INSTANCE_LOCK_NAME;
}
EclipseNode typeNode = upToTypeNode(annotationNode);
MemberExistsResult exists = MemberExistsResult.NOT_EXISTS;
if (typeNode != null && typeNode.get() instanceof TypeDeclaration) {
TypeDeclaration typeDecl = (TypeDeclaration) typeNode.get();
if (typeDecl.fields != null) for (FieldDeclaration def : typeDecl.fields) {
char[] fName = def.name;
if (fName == null) continue;
if (Arrays.equals(fName, lockName)) {
exists = getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
boolean st = def.isStatic();
if (!st && isStatic[0]) {
if (reportErrors) annotationNode.addError(String.format("The field %s is non-static and thus cannot be
used on this static method", new String(lockName)));
return null;
}
isStatic[0] = st;
break;
}
}
}
if (exists == MemberExistsResult.NOT_EXISTS) {
```

```
if (!autoMake) {
if (reportErrors) annotationNode.addError(String.format("The field %s does not exist", new
String(lockName)));
return null;
}
FieldDeclaration fieldDecl = new FieldDeclaration(lockName, 0, -1);
setGeneratedBy(fieldDecl, source);
fieldDecl.declarationSourceEnd = -1;
fieldDecl.modifiers = (isStatic[0] ? Modifier.STATIC : 0) | Modifier.FINAL | Modifier.PRIVATE;
//We use 'new Object[0];' because unlike 'new Object();', empty arrays *ARE* serializable!
ArrayAllocationExpression arrayAlloc = new ArrayAllocationExpression();
setGeneratedBy(arrayAlloc, source);
arrayAlloc.dimensions = new Expression[] { makeIntLiteral("0".toCharArray(), source) };
arrayAlloc.type = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, new long[] { 0,
0, 0 });
setGeneratedBy(arrayAlloc.type, source);
fieldDecl.type = new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, new long[] { 0, 0,
0 });
setGeneratedBy(fieldDecl.type, source);
fieldDecl.initialization = arrayAlloc;
injectFieldAndMarkGenerated(annotationNode.up().up(), fieldDecl);
}
return lockName;
}
@Override public void handle(AnnotationValues<Synchronized> annotation, Annotation source,
EclipseNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.SYNCHRONIZED_FLAG_USAGE,
"@Synchronized");
int p1 = source.sourceStart -1;
int p2 = source.sourceStart -2;
long pos = (((long) p1) << 32) | p2;
EclipseNode methodNode = annotationNode.up();
if (methodNode == null || methodNode.getKind() != Kind.METHOD || !(methodNode.get() instanceof
MethodDeclaration)) {
annotationNode.addError("@Synchronized is legal only on methods.");
return;
}
MethodDeclaration method = (MethodDeclaration) methodNode.get();
if (method.isAbstract()) {
annotationNode.addError("@Synchronized is legal only on concrete methods.");
return;
}
EclipseNode typeNode = upToTypeNode(annotationNode);
if (!isClassOrEnum(typeNode)) {
annotationNode.addError("@Synchronized is legal only on methods in classes and enums.");
return;
}
boolean[] isStatic = { method.isStatic() };
char[] lockName = createLockField(annotation, annotationNode, isStatic, true);
if (lockName == null) return;
if (method.statements == null) return;
Block block = new Block(0);
setGeneratedBy(block, source);
```

```
block.statements = method.statements;
// Positions for in-method generated nodes are special
block.sourceEnd = method.bodyEnd;
block.sourceStart = method.bodyStart;
Expression lockVariable;
if (isStatic[0]) {
char[][] n = getQualifiedInnerName(typeNode, lockName);
long[] ps = new long[n.length];
Arrays.fill(ps, pos);
lockVariable = new QualifiedNameReference(n, ps, p1, p2);
} else {
lockVariable = new FieldReference(lockName, pos);
ThisReference thisReference = new ThisReference(p1, p2);
setGeneratedBy(thisReference, source);
((FieldReference) lockVariable).receiver = thisReference;
}
setGeneratedBy(lockVariable, source);
method.statements = new Statement[] {
new SynchronizedStatement(lockVariable, block, 0, 0)
};
// Positions for in-method generated nodes are special
method.statements[0].sourceEnd = method.bodyEnd;
method.statements[0].sourceStart = method.bodyStart;
setGeneratedBy(method.statements[0], source);
methodNode.rebuild();
}
}
```

lombok/eclipse/handlers/HandleToString.java


```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```

```
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.BinaryExpression;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NameReference;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.SuperReference;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.ToString;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CallSuperType;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.core.handlers.InclusionExclusionUtils;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
/**
* Handles the {@code ToString} annotation for eclipse.
*/
@Provides
public class HandleToString extends EclipseAnnotationHandler<ToString> {
public void handle(AnnotationValues<ToString> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.TO_STRING_FLAG_USAGE, "@ToString");
ToString ann = annotation.getInstance();
```

```
boolean onlyExplicitlyIncluded = annotationNode.getAst().getBooleanAnnotationValue(annotation,
"onlyExplicitlyIncluded", ConfigurationKeys.TO_STRING_ONLY_EXPLICITLY_INCLUDED);
List<Included<EclipseNode, ToString.Include>> members =
InclusionExclusionUtils.handleToStringMarking(annotationNode.up(), onlyExplicitlyIncluded,
annotation, annotationNode);
if (members == null) return;
Boolean callSuper = ann.callSuper();
if (!annotation.isExplicit("callSuper")) callSuper = null;
Boolean doNotUseGettersConfiguration = annotationNode.getAst().readConfiguration(ConfigurationKe
ys.TO_STRING_DO_NOT_USE_GETTERS);
boolean doNotUseGetters = annotation.isExplicit("doNotUseGetters") || doNotUseGettersConfiguration
== null ? ann.doNotUseGetters() : doNotUseGettersConfiguration;
FieldAccess fieldAccess = doNotUseGetters ? FieldAccess.PREFER_FIELD : FieldAccess.GETTER;
Boolean fieldNamesConfiguration = annotationNode.getAst().readConfiguration(ConfigurationKeys.TO
_STRING_INCLUDE_FIELD_NAMES);
boolean includeFieldNames = annotation.isExplicit("includeFieldNames") || fieldNamesConfiguration ==
null ? ann.includeFieldNames() : fieldNamesConfiguration;
generateToString(annotationNode.up(), annotationNode, members, includeFieldNames, callSuper,
true, fieldAccess);
}
public void generateToStringForType(EclipseNode typeNode, EclipseNode errorNode) {
if (hasAnnotation(ToString.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
AnnotationValues<ToString> anno = AnnotationValues.of(ToString.class);
boolean includeFieldNames = typeNode.getAst().getBooleanAnnotationValue(anno,
"includeFieldNames", ConfigurationKeys.TO_STRING_INCLUDE_FIELD_NAMES);
boolean onlyExplicitlyIncluded = typeNode.getAst().getBooleanAnnotationValue(anno,
"onlyExplicitlyIncluded", ConfigurationKeys.TO_STRING_ONLY_EXPLICITLY_INCLUDED);
Boolean doNotUseGettersConfiguration =
typeNode.getAst().readConfiguration(ConfigurationKeys.TO_STRING_DO_NOT_USE_GETTERS);
FieldAccess access = doNotUseGettersConfiguration == null || !doNotUseGettersConfiguration ?
FieldAccess.GETTER : FieldAccess.PREFER_FIELD;
List<Included<EclipseNode, ToString.Include>> members =
InclusionExclusionUtils.handleToStringMarking(typeNode, onlyExplicitlyIncluded, null, null);
generateToString(typeNode, errorNode, members, includeFieldNames, null, false, access);
}
public void generateToString(EclipseNode typeNode, EclipseNode errorNode,
List<Included<EclipseNode, ToString.Include>> members,
boolean includeFieldNames, Boolean callSuper, boolean whineIfExists, FieldAccess fieldAccess) {
if (!isClassOrEnum(typeNode)) {
errorNode.addError("@ToString is only supported on a class or enum.");
return;
}
switch (methodExists("toString", typeNode, 0)) {
case NOT_EXISTS:
if (callSuper == null) {
if (isDirectDescendantOfObject(typeNode)) {
callSuper = false;
} else {
CallSuperType cst =
typeNode.getAst().readConfiguration(ConfigurationKeys.TO_STRING_CALL_SUPER);
```

```
if (cst == null) cst = CallSuperType.SKIP;
switch (cst) {
default:
case SKIP:
callSuper = false;
break;
case WARN:
errorNode.addWarning("Generating toString implementation but without a call to superclass, even
though this class does not extend java.lang.Object. If this intentional, add '@ToString(callSuper=false)'
to your type.");
callSuper = false;
break;
case CALL:
callSuper = true;
break;
}
}
}
MethodDeclaration toString = createToString(typeNode, members, includeFieldNames, callSuper,
errorNode.get(), fieldAccess);
injectMethod(typeNode, toString);
break;
case EXISTS_BY_LOMBOK:
break;
default:
case EXISTS_BY_USER:
if (whineIfExists) {
errorNode.addWarning("Not generating toString(): A method with that name already exists");
}
}
}
public static MethodDeclaration createToString(EclipseNode type, Collection<Included<EclipseNode,
ToString.Include>> members,
boolean includeNames, boolean callSuper, ASTNode source, FieldAccess fieldAccess) {
String typeName = getTypeName(type);
boolean isEnum = type.isEnumType();
char[] suffix = ")".toCharArray();
String infixS = ", ";
char[] infix = infixS.toCharArray();
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
final int PLUS = OperatorIds.PLUS;
String prefix;
if (callSuper) {
prefix = "(super=";
} else if (members.isEmpty()) {
prefix = isEnum ? "" : "()";
} else if (includeNames) {
Included<EclipseNode, ToString.Include> firstMember = members.iterator().next();
String name = firstMember.getInc() == null ? "" : firstMember.getInc().name();
if (name.isEmpty()) name = firstMember.getNode().getName();
prefix = "(" + name + "=";
} else {
```

```
prefix = "(";
}
boolean first = true;
Expression current;
if (!isEnum) {
current = new StringLiteral((typeName + prefix).toCharArray(), pS, pE, 0);
setGeneratedBy(current, source);
} else {
current = new StringLiteral((typeName + ".").toCharArray(), pS, pE, 0);
setGeneratedBy(current, source);
MessageSend thisName = new MessageSend();
thisName.sourceStart = pS; thisName.sourceEnd = pE;
setGeneratedBy(thisName, source);
thisName.receiver = new ThisReference(pS, pE);
setGeneratedBy(thisName.receiver, source);
thisName.selector = "name".toCharArray();
current = new BinaryExpression(current, thisName, PLUS);
setGeneratedBy(current, source);
if (!prefix.isEmpty()) {
StringLiteral px = new StringLiteral(prefix.toCharArray(), pS, pE, 0);
setGeneratedBy(px, source);
current = new BinaryExpression(current, px, PLUS);
current.sourceStart = pS; current.sourceEnd = pE;
setGeneratedBy(current, source);
}
}
if (callSuper) {
MessageSend callToSuper = new MessageSend();
callToSuper.sourceStart = pS; callToSuper.sourceEnd = pE;
setGeneratedBy(callToSuper, source);
callToSuper.receiver = new SuperReference(pS, pE);
setGeneratedBy(callToSuper.receiver, source);
callToSuper.selector = "toString".toCharArray();
current = new BinaryExpression(current, callToSuper, PLUS);
setGeneratedBy(current, source);
first = false;
}
for (Included<EclipseNode, ToString.Include> member : members) {
EclipseNode memberNode = member.getNode();
TypeReference fieldType = getFieldType(memberNode, fieldAccess);
Expression memberAccessor;
if (memberNode.getKind() == Kind.METHOD) {
memberAccessor = createMethodAccessor(memberNode, source);
} else {
memberAccessor = createFieldAccessor(memberNode, fieldAccess, source);
}
// The distinction between primitive and object will be useful if we ever add a 'hideNulls' option.
boolean fieldBaseTypeIsPrimitive = BUILT_IN_TYPES.contains(new String(fieldType.getLastToken()));
@SuppressWarnings("unused")
boolean fieldIsPrimitive = fieldType.dimensions() == 0 && fieldBaseTypeIsPrimitive;
boolean fieldIsPrimitiveArray = fieldType.dimensions() == 1 && fieldBaseTypeIsPrimitive;
boolean fieldIsObjectArray = fieldType.dimensions() > 0 && !fieldIsPrimitiveArray;
Expression ex;
```

```
if (fieldIsPrimitiveArray || fieldIsObjectArray) {
MessageSend arrayToString = new MessageSend();
arrayToString.sourceStart = pS; arrayToString.sourceEnd = pE;
arrayToString.receiver = generateQualifiedNameRef(source, TypeConstants.JAVA,
TypeConstants.UTIL, "Arrays".toCharArray());
arrayToString.arguments = new Expression[] { memberAccessor };
setGeneratedBy(arrayToString.arguments[0], source);
arrayToString.selector = (fieldIsObjectArray ? "deepToString" : "toString").toCharArray();
ex = arrayToString;
} else {
ex = memberAccessor;
}
setGeneratedBy(ex, source);
if (first) {
current = new BinaryExpression(current, ex, PLUS);
current.sourceStart = pS; current.sourceEnd = pE;
setGeneratedBy(current, source);
first = false;
continue;
}
StringLiteral fieldNameLiteral;
if (includeNames) {
String n = member.getInc() == null ? "" : member.getInc().name();
if (n.isEmpty()) n = memberNode.getName();
char[] namePlusEqualsSign = (infixS + n + "=").toCharArray();
fieldNameLiteral = new StringLiteral(namePlusEqualsSign, pS, pE, 0);
} else {
fieldNameLiteral = new StringLiteral(infix, pS, pE, 0);
}
setGeneratedBy(fieldNameLiteral, source);
current = new BinaryExpression(current, fieldNameLiteral, PLUS);
setGeneratedBy(current, source);
current = new BinaryExpression(current, ex, PLUS);
setGeneratedBy(current, source);
}
if (!first) {
StringLiteral suffixLiteral = new StringLiteral(suffix, pS, pE, 0);
setGeneratedBy(suffixLiteral, source);
current = new BinaryExpression(current, suffixLiteral, PLUS);
setGeneratedBy(current, source);
}
ReturnStatement returnStatement = new ReturnStatement(current, pS, pE);
setGeneratedBy(returnStatement, source);
MethodDeclaration method = new MethodDeclaration(((CompilationUnitDeclaration)
type.top().get()).compilationResult);
setGeneratedBy(method, source);
method.modifiers = toEclipseModifier(AccessLevel.PUBLIC);
method.returnType = new QualifiedTypeReference(TypeConstants.JAVA_LANG_STRING, new long[]
{p, p, p});
setGeneratedBy(method.returnType, source);
Annotation overrideAnnotation = makeMarkerAnnotation(TypeConstants.JAVA_LANG_OVERRIDE,
source);
if (getCheckerFrameworkVersion(type).generateSideEffectFree()) {
```

```
method.annotations = new Annotation[] { overrideAnnotation, generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE) };
} else {
method.annotations = new Annotation[] { overrideAnnotation };
}
method.arguments = null;
method.selector = "toString".toCharArray();
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
method.statements = new Statement[] { returnStatement };
EclipseHandlerUtil.createRelevantNonNullAnnotation(type, method);
return method;
}
public static String getTypeName(EclipseNode type) {
String typeName = getSingleTypeName(type);
EclipseNode upType = type.up();
while (upType.getKind() == Kind.TYPE) {
String upTypeName = getSingleTypeName(upType);
if (upTypeName.isEmpty()) break;
typeName = upTypeName + "." + typeName;
upType = upType.up();
}
return typeName;
}
public static String getSingleTypeName(EclipseNode type) {
TypeDeclaration typeDeclaration = (TypeDeclaration)type.get();
char[] rawTypeName = typeDeclaration.name;
return rawTypeName == null ? "" : new String(rawTypeName);
}
private static final Set<String> BUILT_IN_TYPES = Collections.unmodifiableSet(new
HashSet<String>(Arrays.asList(
"byte", "short", "int", "long", "char", "boolean", "double", "float")));
public static NameReference generateQualifiedNameRef(ASTNode source, char[]... varNames) {
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long)pS << 32 | pE;
NameReference ref;
if (varNames.length > 1) ref = new QualifiedNameReference(varNames, new long[varNames.length],
pS, pE);
else ref = new SingleNameReference(varNames[0], p);
setGeneratedBy(ref, source);
return ref;
}
}
```

lombok/eclipse/handlers/HandleUtilityClass.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
```

*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Clinit;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ExplicitConstructorCall;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.ThrowStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.AST.Kind;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.UtilityClass;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.experimental.UtilityClass} annotation for eclipse.
*/
@Provides
```

```
@HandlerPriority(-4096) //-2^12; to ensure @FieldDefaults picks up on the 'static' we set here.
public class HandleUtilityClass extends EclipseAnnotationHandler<UtilityClass> {
@Override public void handle(AnnotationValues<UtilityClass> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.UTILITY_CLASS_FLAG_USAGE,
"@UtilityClass");
EclipseNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode)) return;
changeModifiersAndGenerateConstructor(annotationNode.up(), annotationNode);
}
private static boolean checkLegality(EclipseNode typeNode, EclipseNode errorNode) {
if (!isClass(typeNode)) {
errorNode.addError("@UtilityClass is only supported on a class.");
return false;
}
// It might be an inner class. This is okay, but only if it is / can be a static inner class. Thus, all of its
parents have to be static inner classes until the top-level.
EclipseNode typeWalk = typeNode;
while (true) {
typeWalk = typeWalk.up();
switch (typeWalk.getKind()) {
case TYPE:
if ((((TypeDeclaration) typeWalk.get()).modifiers & (ClassFileConstants.AccStatic |
ClassFileConstants.AccInterface | ClassFileConstants.AccAnnotation | ClassFileConstants.AccEnum))
!= 0) continue;
if (typeWalk.up().getKind() == Kind.COMPILATION_UNIT) return true;
errorNode.addError("@UtilityClass automatically makes the class static, however, this class cannot be
made static.");
return false;
case COMPILATION_UNIT:
return true;
default:
errorNode.addError("@UtilityClass cannot be placed on a method local or anonymous inner class, or
any class nested in such a class.");
return false;
}
}
}
private void changeModifiersAndGenerateConstructor(EclipseNode typeNode, EclipseNode
annotationNode) {
TypeDeclaration classDecl = (TypeDeclaration) typeNode.get();
boolean makeConstructor = true;
classDecl.modifiers |= ClassFileConstants.AccFinal;
boolean markStatic = true;
boolean requiresClInit = false;
boolean alreadyHasClinit = false;
if (typeNode.up().getKind() == Kind.COMPILATION_UNIT) markStatic = false;
if (markStatic && typeNode.up().getKind() == Kind.TYPE) {
TypeDeclaration typeDecl = (TypeDeclaration) typeNode.up().get();
if ((typeDecl.modifiers & (ClassFileConstants.AccInterface | ClassFileConstants.AccAnnotation)) != 0)
markStatic = false;
}
if (markStatic) classDecl.modifiers |= ClassFileConstants.AccStatic;
```

```
for (EclipseNode element : typeNode.down()) {
if (element.getKind() == Kind.FIELD) {
FieldDeclaration fieldDecl = (FieldDeclaration) element.get();
if ((fieldDecl.modifiers & ClassFileConstants.AccStatic) == 0) {
requiresClInit = true;
fieldDecl.modifiers |= ClassFileConstants.AccStatic;
}
} else if (element.getKind() == Kind.METHOD) {
AbstractMethodDeclaration amd = (AbstractMethodDeclaration) element.get();
if (amd instanceof ConstructorDeclaration) {
ConstructorDeclaration constrDecl = (ConstructorDeclaration) element.get();
if (getGeneratedBy(constrDecl) == null && (constrDecl.bits & ASTNode.IsDefaultConstructor) == 0) {
element.addError("@UtilityClasses cannot have declared constructors.");
makeConstructor = false;
continue;
}
} else if (amd instanceof MethodDeclaration) {
amd.modifiers |= ClassFileConstants.AccStatic;
} else if (amd instanceof Clinit) {
alreadyHasClinit = true;
}
} else if (element.getKind() == Kind.TYPE) {
((TypeDeclaration) element.get()).modifiers |= ClassFileConstants.AccStatic;
}
}
if (makeConstructor) createPrivateDefaultConstructor(typeNode, annotationNode);
if (requiresClInit && !alreadyHasClinit) classDecl.addClinit();
}
private static final char[][] JAVA_LANG_UNSUPPORTED_OPERATION_EXCEPTION = new char[][] {
TypeConstants.JAVA, TypeConstants.LANG, "UnsupportedOperationException".toCharArray()
};
private static final char[] UNSUPPORTED_MESSAGE = "This is a utility class and cannot be
instantiated".toCharArray();
private void createPrivateDefaultConstructor(EclipseNode typeNode, EclipseNode sourceNode) {
ASTNode source = sourceNode.get();
TypeDeclaration typeDeclaration = ((TypeDeclaration) typeNode.get());
ConstructorDeclaration constructor = new ConstructorDeclaration(((CompilationUnitDeclaration)
typeNode.top().get()).compilationResult);
constructor.modifiers = ClassFileConstants.AccPrivate;
constructor.selector = typeDeclaration.name;
constructor.constructorCall = new ExplicitConstructorCall(ExplicitConstructorCall.ImplicitSuper);
constructor.thrownExceptions = null;
constructor.typeParameters = null;
constructor.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
constructor.arguments = null;
long[] ps = new long[JAVA_LANG_UNSUPPORTED_OPERATION_EXCEPTION.length];
AllocationExpression exception = new AllocationExpression();
exception.type = new
QualifiedTypeReference(JAVA_LANG_UNSUPPORTED_OPERATION_EXCEPTION, ps);
exception.arguments = new Expression[] {
new StringLiteral(UNSUPPORTED_MESSAGE, 0, 0, 0)
};
ThrowStatement throwStatement = new ThrowStatement(exception, 0, 0);
```

```
constructor.statements = new Statement[] {throwStatement};
constructor.traverse(new SetGeneratedByVisitor(source), typeDeclaration.scope);
injectMethod(typeNode, constructor);
}
}
```

lombok/eclipse/handlers/HandleVal.java

```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.eclipse.handlers.EclipseHandlerUtil.typeMatches;
import lombok.ConfigurationKeys;
import lombok.val;
import lombok.var;
import lombok.core.HandlerPriority;
import lombok.eclipse.DeferUntilPostDiet;
import lombok.eclipse.EclipseASTAdapter;
import lombok.eclipse.EclipseASTVisitor;
import lombok.eclipse.EclipseNode;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.ForStatement;
import org.eclipse.jdt.internal.compiler.ast.ForeachStatement;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
/*
* This class just handles 3 basic error cases. The real meat of eclipse 'val' support is in {@code
PatchVal} and {@code PatchValEclipse}
```

```
*/
@Provides(EclipseASTVisitor.class)
@DeferUntilPostDiet
@HandlerPriority(65536) // 2^16; resolution needs to work, so if the RHS expression is i.e. a call to a generated getter, we have to run after that getter has been generated.
public class HandleVal extends EclipseASTAdapter {
@Override public void visitLocal(EclipseNode localNode, LocalDeclaration local) {
TypeReference type = local.type;
boolean isVal = typeMatches(val.class, localNode, type);
boolean isVar = typeMatches(var.class, localNode, type);
if (!(isVal || isVar)) return;
if (isVal) handleFlagUsage(localNode, ConfigurationKeys.VAL_FLAG_USAGE, "val");
if (isVar) handleFlagUsage(localNode, ConfigurationKeys.VAR_FLAG_USAGE, "var");
boolean variableOfForEach = false;
if (localNode.directUp().get() instanceof ForeachStatement) {
ForeachStatement fs = (ForeachStatement) localNode.directUp().get();
variableOfForEach = fs.elementVariable == local;
}
String annotation = isVal ? "val" : "var";
if (local.initialization == null && !variableOfForEach) {
localNode.addError("'" + annotation + "' on a local variable requires an initializer expression");
return;
}
if (local.initialization instanceof ArrayInitializer) {
localNode.addError("'" + annotation + "' is not compatible with array initializer expressions. Use the full form (new int[] { ... } instead of just { ... })");
return;
}
ASTNode parentRaw = localNode.directUp().get();
if (isVal && parentRaw instanceof ForStatement) {
localNode.addError("'val' is not allowed in old-style for loops");
return;
}
if (parentRaw instanceof ForStatement && ((ForStatement) parentRaw).initializations != null && ((ForStatement) parentRaw).initializations.length > 1) {
localNode.addError("'var' is not allowed in old-style for loops if there is more than 1 initializer");
return;
}
if (local.initialization != null && local.initialization.getClass().getName().equals("org.eclipse.jdt.internal.compiler.ast.LambdaExpression")) {
localNode.addError("'" + annotation + "' is not allowed with lambda expressions.");
return;
}
if(isVar && local.initialization instanceof NullLiteral) {
localNode.addError("variable initializer is 'null'");
return;
}
}
}
```

lombok/eclipse/handlers/HandleValue.java

/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.Collections;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.HandleConstructor.SkipIfConstructorExists;
import lombok.experimental.NonFinal;
import lombok.spi.Provides;
import lombok.Value;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
/**
* Handles the {@code lombok.Value} annotation for eclipse.
*/
@Provides
@HandlerPriority(-512) //-2^9; to ensure @EqualsAndHashCode and such pick up on this handler making the class final and messing with the fields' access levels, run earlier.
public class HandleValue extends EclipseAnnotationHandler<Value> {
private HandleFieldDefaults handleFieldDefaults = new HandleFieldDefaults();
private HandleGetter handleGetter = new HandleGetter();
private HandleEqualsAndHashCode handleEqualsAndHashCode = new HandleEqualsAndHashCode();
private HandleToString handleToString = new HandleToString();
private HandleConstructor handleConstructor = new HandleConstructor();

```
public void handle(AnnotationValues<Value> annotation, Annotation ast, EclipseNode annotationNode)
{
handleFlagUsage(annotationNode, ConfigurationKeys.VALUE_FLAG_USAGE, "@Value");
Value ann = annotation.getInstance();
EclipseNode typeNode = annotationNode.up();
if (!isClass(typeNode)) {
annotationNode.addError("@Value is only supported on a class.");
return;
}
TypeDeclaration typeDecl = (TypeDeclaration) typeNode.get();
// Make class final.
if (!hasAnnotation(NonFinal.class, typeNode)) {
if ((typeDecl.modifiers & ClassFileConstants.AccFinal) == 0) {
typeDecl.modifiers |= ClassFileConstants.AccFinal;
typeNode.rebuild();
}
}
handleFieldDefaults.generateFieldDefaultsForType(typeNode, annotationNode,
AccessLevel.PRIVATE, true, true);
//Careful: Generate the public static constructor (if there is one) LAST, so that any attempt to
//'find callers' on the annotation node will find callers of the constructor, which is by far the
//most useful of the many methods built by @Value. This trick won't work for the non-static constructor,
//for whatever reason, though you can find callers of that one by focusing on the class name itself
//and hitting 'find callers'.
handleGetter.generateGetterForType(typeNode, annotationNode, AccessLevel.PUBLIC, true,
Collections.<Annotation>emptyList());
handleEqualsAndHashCode.generateEqualsAndHashCodeForType(typeNode, annotationNode);
handleToString.generateToStringForType(typeNode, annotationNode);
handleConstructor.generateAllArgsConstructor(typeNode, AccessLevel.PUBLIC,
ann.staticConstructor(), SkipIfConstructorExists.YES,
Collections.<Annotation>emptyList(), annotationNode);
handleConstructor.generateExtraNoArgsConstructor(typeNode, annotationNode);
}
}
```

lombok/eclipse/handlers/HandleWith.java

```
/*
* Copyright (C) 2012-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.With;
import lombok.core.AST.Kind;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.AnnotationValues;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.Accessors;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.ExtraCompilerModifiers;
@Provides
public class HandleWith extends EclipseAnnotationHandler<With> {
public boolean generateWithForType(EclipseNode typeNode, EclipseNode pos, AccessLevel level,
boolean checkForTypeLevelWith) {
if (checkForTypeLevelWith) {
if (hasAnnotation(With.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return true;
}
}
```

```
TypeDeclaration typeDecl = null;
if (typeNode.get() instanceof TypeDeclaration) typeDecl = (TypeDeclaration) typeNode.get();
int modifiers = typeDecl == null ? 0 : typeDecl.modifiers;
boolean notAClass = (modifiers &
(ClassFileConstants.AccInterface | ClassFileConstants.AccAnnotation | ClassFileConstants.AccEnum))
!= 0;
if (typeDecl == null || notAClass) {
pos.addError("@With is only supported on a class or a field.");
return false;
}
for (EclipseNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
FieldDeclaration fieldDecl = (FieldDeclaration) field.get();
if (!filterField(fieldDecl)) continue;
//Skip final fields.
if ((fieldDecl.modifiers & ClassFileConstants.AccFinal) != 0 && fieldDecl.initialization != null) continue;
generateWithForField(field, pos, level);
}
return true;
}
/**
* Generates a with on the stated field.
*
* Used by {@link HandleValue}.
*
* The difference between this call and the handle method is as follows:
*
* If there is a {@code lombok.With} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
* If not, the with method is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*/
public void generateWithForField(EclipseNode fieldNode, EclipseNode sourceNode, AccessLevel
level) {
for (EclipseNode child : fieldNode.down()) {
if (child.getKind() == Kind.ANNOTATION) {
if (annotationTypeMatches(With.class, child)) {
//The annotation will make it happen, so we can skip it.
return;
}
}
}
List<Annotation> empty = Collections.emptyList();
createWithForField(level, fieldNode, sourceNode, false, empty, empty);
}
@Override public void handle(AnnotationValues<With> annotation, Annotation ast, EclipseNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.WITH_FLAG_USAGE, "@With");
EclipseNode node = annotationNode.up();
AccessLevel level = annotation.getInstance().value();
if (level == AccessLevel.NONE || node == null) return;
List<Annotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@With(onMethod", annotationNode);
```

```
List<Annotation> onParam = unboxAndRemoveAnnotationParameter(ast, "onParam",
"@With(onParam", annotationNode);
switch (node.getKind()) {
case FIELD:
createWithForFields(level, annotationNode.upFromAnnotationToFields(), annotationNode, true,
onMethod, onParam);
break;
case TYPE:
if (!onMethod.isEmpty()) {
annotationNode.addError("'onMethod' is not supported for @With on a type.");
}
if (!onParam.isEmpty()) {
annotationNode.addError("'onParam' is not supported for @With on a type.");
}
generateWithForType(node, annotationNode, level, false);
break;
}
}
public void createWithForFields(AccessLevel level, Collection<EclipseNode> fieldNodes, EclipseNode
sourceNode, boolean whineIfExists, List<Annotation> onMethod, List<Annotation> onParam) {
for (EclipseNode fieldNode : fieldNodes) {
createWithForField(level, fieldNode, sourceNode, whineIfExists, onMethod, onParam);
}
}
public void createWithForField(
AccessLevel level, EclipseNode fieldNode, EclipseNode sourceNode,
boolean whineIfExists, List<Annotation> onMethod,
List<Annotation> onParam) {
ASTNode source = sourceNode.get();
if (fieldNode.getKind() != Kind.FIELD) {
sourceNode.addError("@With is only supported on a class or a field.");
return;
}
EclipseNode typeNode = fieldNode.up();
boolean makeAbstract = typeNode != null && typeNode.getKind() == Kind.TYPE &&
(((TypeDeclaration) typeNode.get()).modifiers & ClassFileConstants.AccAbstract) != 0;
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
TypeReference fieldType = copyType(field.type, source);
boolean isBoolean = isBoolean(fieldType);
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
String withName = toWithName(fieldNode, isBoolean, accessors);
if (withName == null) {
fieldNode.addWarning("Not generating a with method for this field: It does not fit your @Accessors
prefix list.");
return;
}
if ((field.modifiers & ClassFileConstants.AccStatic) != 0) {
fieldNode.addWarning("Not generating " + withName + " for this field: With methods cannot be
generated for static fields.");
return;
}
if ((field.modifiers & ClassFileConstants.AccFinal) != 0 && field.initialization != null) {
```

```
fieldNode.addWarning("Not generating " + withName + " for this field: With methods cannot be
generated for final, initialized fields.");
return;
}
if (field.name != null && field.name.length > 0 && field.name[0] == '$') {
fieldNode.addWarning("Not generating " + withName + " for this field: With methods cannot be
generated for fields starting with $.");
return;
}
for (String altName : toAllWithNames(fieldNode, isBoolean, accessors)) {
switch (methodExists(altName, fieldNode, false, 1)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String altNameExpl = "";
if (!altName.equals(withName)) altNameExpl = String.format(" (%s)", altName);
fieldNode.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", withName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
//continue scanning the other alt names.
}
}
int modifier = toEclipseModifier(level);
MethodDeclaration method = createWith((TypeDeclaration) fieldNode.up().get(), fieldNode, withName,
modifier, sourceNode, onMethod, onParam, makeAbstract);
injectMethod(fieldNode.up(), method);
}
public MethodDeclaration createWith(TypeDeclaration parent, EclipseNode fieldNode, String name, int
modifier, EclipseNode sourceNode, List<Annotation> onMethod, List<Annotation> onParam, boolean
makeAbstract) {
ASTNode source = sourceNode.get();
if (name == null) return null;
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration method = new MethodDeclaration(parent.compilationResult);
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
if (makeAbstract) modifier |= ClassFileConstants.AccAbstract |
ExtraCompilerModifiers.AccSemicolonBody;
if (shouldMakeFinal(fieldNode, accessors)) modifier |= ClassFileConstants.AccFinal;
method.modifiers = modifier;
method.returnType = cloneSelfType(fieldNode, source);
if (method.returnType == null) return null;
Annotation[] deprecated = null, checkerFramework = null;
if (isFieldDeprecated(fieldNode)) deprecated = new Annotation[] {
generateDeprecatedAnnotation(source) };
if (getCheckerFrameworkVersion(fieldNode).generateSideEffectFree()) checkerFramework = new
Annotation[] { generateNamedAnnotation(source,
```

```
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE) };
method.annotations = copyAnnotations(source, onMethod.toArray(new Annotation[0]),
checkerFramework, deprecated);
Argument param = new Argument(field.name, p, copyType(field.type, source),
ClassFileConstants.AccFinal);
param.sourceStart = pS; param.sourceEnd = pE;
method.arguments = new Argument[] { param };
method.selector = name.toCharArray();
method.binding = null;
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
Annotation[] copyableAnnotations = findCopyableAnnotations(fieldNode);
if (!makeAbstract) {
List<Expression> args = new ArrayList<Expression>();
for (EclipseNode child : fieldNode.up().down()) {
if (child.getKind() != Kind.FIELD) continue;
FieldDeclaration childDecl = (FieldDeclaration) child.get();
// Skip fields that start with $
if (childDecl.name != null && childDecl.name.length > 0 && childDecl.name[0] == '$') continue;
long fieldFlags = childDecl.modifiers;
// Skip static fields.
if ((fieldFlags & ClassFileConstants.AccStatic) != 0) continue;
// Skip initialized final fields.
if (((fieldFlags & ClassFileConstants.AccFinal) != 0) && childDecl.initialization != null) continue;
if (child.get() == fieldNode.get()) {
args.add(new SingleNameReference(field.name, p));
} else {
args.add(createFieldAccessor(child, FieldAccess.ALWAYS_FIELD, source));
}
}
AllocationExpression constructorCall = new AllocationExpression();
constructorCall.arguments = args.toArray(new Expression[0]);
constructorCall.type = cloneSelfType(fieldNode, source);
Expression identityCheck = new EqualExpression(
createFieldAccessor(fieldNode, FieldAccess.ALWAYS_FIELD, source),
new SingleNameReference(field.name, p),
OperatorIds.EQUAL_EQUAL);
ThisReference thisRef = new ThisReference(pS, pE);
Expression conditional = new ConditionalExpression(identityCheck, thisRef, constructorCall);
Statement returnStatement = new ReturnStatement(conditional, pS, pE);
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
List<Statement> statements = new ArrayList<Statement>(5);
if (hasNonNullAnnotations(fieldNode)) {
Statement nullCheck = generateNullCheck(field, sourceNode, null);
if (nullCheck != null) statements.add(nullCheck);
}
statements.add(returnStatement);
method.statements = statements.toArray(new Statement[0]);
}
param.annotations = copyAnnotations(source, copyableAnnotations, onParam.toArray(new
Annotation[0]));
```

```
EclipseHandlerUtil.createRelevantNonNullAnnotation(fieldNode, method);
method.traverse(new SetGeneratedByVisitor(source), parent.scope);
copyJavadoc(fieldNode, method, CopyJavadoc.WITH);
return method;
}
}
```

lombok/eclipse/handlers/HandleWithBy.java

```
/*
* Copyright (C) 2020-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import static lombok.eclipse.Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
```

```
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.Wildcard;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.ExtraCompilerModifiers;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.AST.Kind;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAnnotationHandler;
import lombok.eclipse.EclipseNode;
import lombok.experimental.Accessors;
import lombok.experimental.WithBy;
import lombok.spi.Provides;
@Provides
public class HandleWithBy extends EclipseAnnotationHandler<WithBy> {
public boolean generateWithByForType(EclipseNode typeNode, EclipseNode pos, AccessLevel level,
boolean checkForTypeLevelWithBy) {
if (checkForTypeLevelWithBy) {
if (hasAnnotation(WithBy.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return true;
}
}
TypeDeclaration typeDecl = null;
if (typeNode.get() instanceof TypeDeclaration) typeDecl = (TypeDeclaration) typeNode.get();
int modifiers = typeDecl == null ? 0 : typeDecl.modifiers;
boolean notAClass = (modifiers &
(ClassFileConstants.AccInterface | ClassFileConstants.AccAnnotation | ClassFileConstants.AccEnum))
!= 0;
if (typeDecl == null || notAClass) {
pos.addError("@WithBy is only supported on a class or a field.");
return false;
}
for (EclipseNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
FieldDeclaration fieldDecl = (FieldDeclaration) field.get();
if (!filterField(fieldDecl)) continue;
//Skip final fields.
if ((fieldDecl.modifiers & ClassFileConstants.AccFinal) != 0 && fieldDecl.initialization != null) continue;
generateWithByForField(field, pos, level);
}
return true;
}
/**
* Generates a withBy on the stated field.
*
```

```
* Used by {@link HandleValue}.
*
* The difference between this call and the handle method is as follows:
*
* If there is a {@code lombok.experimental.WithBy} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
* If not, the with method is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*/
public void generateWithByForField(EclipseNode fieldNode, EclipseNode sourceNode, AccessLevel
level) {
for (EclipseNode child : fieldNode.down()) {
if (child.getKind() == Kind.ANNOTATION) {
if (annotationTypeMatches(WithBy.class, child)) {
//The annotation will make it happen, so we can skip it.
return;
}
}
}
List<Annotation> empty = Collections.emptyList();
createWithByForField(level, fieldNode, sourceNode, false, empty);
}
@Override public void handle(AnnotationValues<WithBy> annotation, Annotation ast, EclipseNode
annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.WITHBY_FLAG_USAGE,
"@WithBy");
EclipseNode node = annotationNode.up();
AccessLevel level = annotation.getInstance().value();
if (level == AccessLevel.NONE || node == null) return;
List<Annotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@WithBy(onMethod", annotationNode);
switch (node.getKind()) {
case FIELD:
createWithByForFields(level, annotationNode.upFromAnnotationToFields(), annotationNode, true,
onMethod);
break;
case TYPE:
if (!onMethod.isEmpty()) {
annotationNode.addError("'onMethod' is not supported for @WithBy on a type.");
}
generateWithByForType(node, annotationNode, level, false);
break;
}
}
public void createWithByForFields(AccessLevel level, Collection<EclipseNode> fieldNodes,
EclipseNode sourceNode, boolean whineIfExists, List<Annotation> onMethod) {
for (EclipseNode fieldNode : fieldNodes) {
createWithByForField(level, fieldNode, sourceNode, whineIfExists, onMethod);
}
}
public void createWithByForField(
AccessLevel level, EclipseNode fieldNode, EclipseNode sourceNode,
boolean whineIfExists, List<Annotation> onMethod) {
```

```
ASTNode source = sourceNode.get();
if (fieldNode.getKind() != Kind.FIELD) {
sourceNode.addError("@WithBy is only supported on a class or a field.");
return;
}
EclipseNode typeNode = fieldNode.up();
boolean makeAbstract = typeNode != null && typeNode.getKind() == Kind.TYPE &&
(((TypeDeclaration) typeNode.get()).modifiers & ClassFileConstants.AccAbstract) != 0;
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
TypeReference fieldType = copyType(field.type, source);
boolean isBoolean = isBoolean(fieldType);
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
String withName = toWithByName(fieldNode, isBoolean, accessors);
if (withName == null) {
fieldNode.addWarning("Not generating a withXBy method for this field: It does not fit your @Accessors
prefix list.");
return;
}
if ((field.modifiers & ClassFileConstants.AccStatic) != 0) {
fieldNode.addWarning("Not generating " + withName + " for this field: With methods cannot be
generated for static fields.");
return;
}
if ((field.modifiers & ClassFileConstants.AccFinal) != 0 && field.initialization != null) {
fieldNode.addWarning("Not generating " + withName + " for this field: With methods cannot be
generated for final, initialized fields.");
return;
}
if (field.name != null && field.name.length > 0 && field.name[0] == '$') {
fieldNode.addWarning("Not generating " + withName + " for this field: With methods cannot be
generated for fields starting with $.");
return;
}
for (String altName : toAllWithByNames(fieldNode, isBoolean, accessors)) {
switch (methodExists(altName, fieldNode, false, 1)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String altNameExpl = "";
if (!altName.equals(withName)) altNameExpl = String.format(" (%s)", altName);
fieldNode.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", withName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
//continue scanning the other alt names.
}
}
int modifier = toEclipseModifier(level);
```

```
MethodDeclaration method = createWithBy((TypeDeclaration) fieldNode.up().get(), fieldNode,
withName, modifier, sourceNode, onMethod, makeAbstract);
injectMethod(fieldNode.up(), method);
}
private static final char[][] NAME_JUF_FUNCTION =
Eclipse.fromQualifiedName("java.util.function.Function");
private static final char[][] NAME_JUF_OP =
Eclipse.fromQualifiedName("java.util.function.UnaryOperator");
private static final char[][] NAME_JUF_DOUBLEOP =
Eclipse.fromQualifiedName("java.util.function.DoubleUnaryOperator");
private static final char[][] NAME_JUF_INTOP =
Eclipse.fromQualifiedName("java.util.function.IntUnaryOperator");
private static final char[][] NAME_JUF_LONGOP =
Eclipse.fromQualifiedName("java.util.function.LongUnaryOperator");
private static final char[] NAME_CHAR = {'c', 'h', 'a', 'r'};
private static final char[] NAME_SHORT = {'s', 'h', 'o', 'r', 't'};
private static final char[] NAME_BYTE = {'b', 'y', 't', 'e'};
private static final char[] NAME_INT = {'i', 'n', 't'};
private static final char[] NAME_LONG = {'l', 'o', 'n', 'g'};
private static final char[] NAME_DOUBLE = {'d', 'o', 'u', 'b', 'l', 'e'};
private static final char[] NAME_FLOAT = {'f', 'l', 'o', 'a', 't'};
private static final char[] NAME_BOOLEAN = {'b', 'o', 'o', 'l', 'e', 'a', 'n'};
private static final char[][] NAME_JAVA_LANG_BOOLEAN =
Eclipse.fromQualifiedName("java.lang.Boolean");
private static final char[] NAME_APPLY = {'a', 'p', 'p', 'l', 'y'};
private static final char[] NAME_APPLY_AS_INT = {'a', 'p', 'p', 'l', 'y', 'A', 's', 'I', 'n', 't'};
private static final char[] NAME_APPLY_AS_LONG = {'a', 'p', 'p', 'l', 'y', 'A', 's', 'L', 'o', 'n', 'g'};
private static final char[] NAME_APPLY_AS_DOUBLE = {'a', 'p', 'p', 'l', 'y', 'A', 's', 'D', 'o', 'u', 'b', 'l', 'e'};
private static final char[] NAME_TRANSFORMER = {'t', 'r', 'a', 'n', 's', 'f', 'o', 'r', 'm', 'e', 'r'};
public MethodDeclaration createWithBy(TypeDeclaration parent, EclipseNode fieldNode, String name,
int modifier, EclipseNode sourceNode, List<Annotation> onMethod, boolean makeAbstract) {
ASTNode source = sourceNode.get();
if (name == null) return null;
FieldDeclaration field = (FieldDeclaration) fieldNode.get();
int pS = source.sourceStart, pE = source.sourceEnd;
long p = (long) pS << 32 | pE;
MethodDeclaration method = new MethodDeclaration(parent.compilationResult);
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
if (makeAbstract) modifier |= ClassFileConstants.AccAbstract |
ExtraCompilerModifiers.AccSemicolonBody;
if (shouldMakeFinal(fieldNode, accessors)) modifier |= ClassFileConstants.AccFinal;
method.modifiers = modifier;
method.returnType = cloneSelfType(fieldNode, source);
if (method.returnType == null) return null;
Annotation[] deprecated = null, checkerFramework = null;
if (isFieldDeprecated(fieldNode)) deprecated = new Annotation[] {
generateDeprecatedAnnotation(source) };
if (getCheckerFrameworkVersion(fieldNode).generateSideEffectFree()) checkerFramework = new
Annotation[] { generateNamedAnnotation(source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE) };
char[][] functionalInterfaceName = null;
int requiredCast = -1;
TypeReference parameterizer = null;
```

```
boolean superExtendsStyle = true;
char[] applyMethodName = NAME_APPLY;
if (field.type instanceof SingleTypeReference) {
char[] token = ((SingleTypeReference) field.type).token;
if (Arrays.equals(token, NAME_CHAR)) {
requiredCast = TypeIds.T_char;
functionalInterfaceName = NAME_JUF_INTOP;
} else if (Arrays.equals(token, NAME_SHORT)) {
requiredCast = TypeIds.T_short;
functionalInterfaceName = NAME_JUF_INTOP;
} else if (Arrays.equals(token, NAME_BYTE)) {
requiredCast = TypeIds.T_byte;
functionalInterfaceName = NAME_JUF_INTOP;
} else if (Arrays.equals(token, NAME_INT)) {
functionalInterfaceName = NAME_JUF_INTOP;
} else if (Arrays.equals(token, NAME_LONG)) {
functionalInterfaceName = NAME_JUF_LONGOP;
} else if (Arrays.equals(token, NAME_FLOAT)) {
requiredCast = TypeIds.T_float;
functionalInterfaceName = NAME_JUF_DOUBLEOP;
} else if (Arrays.equals(token, NAME_DOUBLE)) {
functionalInterfaceName = NAME_JUF_DOUBLEOP;
} else if (Arrays.equals(token, NAME_BOOLEAN)) {
functionalInterfaceName = NAME_JUF_OP;
parameterizer = new QualifiedTypeReference(NAME_JAVA_LANG_BOOLEAN, new long[] {0, 0, 0});
superExtendsStyle = false;
}
}
if (functionalInterfaceName == NAME_JUF_INTOP) applyMethodName = NAME_APPLY_AS_INT;
if (functionalInterfaceName == NAME_JUF_LONGOP) applyMethodName =
NAME_APPLY_AS_LONG;
if (functionalInterfaceName == NAME_JUF_DOUBLEOP) applyMethodName =
NAME_APPLY_AS_DOUBLE;
if (functionalInterfaceName == null) {
functionalInterfaceName = NAME_JUF_FUNCTION;
parameterizer = copyType(field.type, source);
}
method.annotations = copyAnnotations(source, onMethod.toArray(new Annotation[0]),
checkerFramework, deprecated);
TypeReference fType = null;
if (parameterizer != null && superExtendsStyle) {
Wildcard w1 = new Wildcard(Wildcard.SUPER);
w1.bound = parameterizer;
Wildcard w2 = new Wildcard(Wildcard.EXTENDS);
w2.bound = copyType(field.type, source);
TypeReference[][] ta = new TypeReference[functionalInterfaceName.length][];
ta[functionalInterfaceName.length - 1] = new TypeReference[] {w1, w2};
long[] ps = new long[functionalInterfaceName.length];
fType = new ParameterizedQualifiedTypeReference(functionalInterfaceName, ta, 0, ps);
}
if (parameterizer != null && !superExtendsStyle) {
TypeReference[][] ta = new TypeReference[functionalInterfaceName.length][];
ta[functionalInterfaceName.length - 1] = new TypeReference[] {parameterizer};
```

```
long[] ps = new long[functionalInterfaceName.length];
fType = new ParameterizedQualifiedTypeReference(functionalInterfaceName, ta, 0, ps);
}
if (parameterizer == null) {
long[] ps = new long[functionalInterfaceName.length];
fType = new QualifiedTypeReference(functionalInterfaceName, ps);
}
Argument param = new Argument(NAME_TRANSFORMER, p, fType, ClassFileConstants.AccFinal);
param.sourceStart = pS; param.sourceEnd = pE;
method.arguments = new Argument[] { param };
method.selector = name.toCharArray();
method.binding = null;
method.thrownExceptions = null;
method.typeParameters = null;
method.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
if (!makeAbstract) {
List<Expression> args = new ArrayList<Expression>();
for (EclipseNode child : fieldNode.up().down()) {
if (child.getKind() != Kind.FIELD) continue;
FieldDeclaration childDecl = (FieldDeclaration) child.get();
// Skip fields that start with $
if (childDecl.name != null && childDecl.name.length > 0 && childDecl.name[0] == '$') continue;
long fieldFlags = childDecl.modifiers;
// Skip static fields.
if ((fieldFlags & ClassFileConstants.AccStatic) != 0) continue;
// Skip initialized final fields.
if (((fieldFlags & ClassFileConstants.AccFinal) != 0) && childDecl.initialization != null) continue;
if (child.get() == fieldNode.get()) {
MessageSend ms = new MessageSend();
ms.receiver = new SingleNameReference(NAME_TRANSFORMER, 0);
ms.selector = applyMethodName;
ms.arguments = new Expression[] {createFieldAccessor(child, FieldAccess.ALWAYS_FIELD, source)};
if (requiredCast != -1) {
args.add(makeCastExpression(ms, TypeReference.baseTypeReference(requiredCast, 0), source));
} else {
args.add(ms);
}
} else {
args.add(createFieldAccessor(child, FieldAccess.ALWAYS_FIELD, source));
}
}
AllocationExpression constructorCall = new AllocationExpression();
constructorCall.arguments = args.toArray(new Expression[0]);
constructorCall.type = cloneSelfType(fieldNode, source);
Statement returnStatement = new ReturnStatement(constructorCall, pS, pE);
method.bodyStart = method.declarationSourceStart = method.sourceStart = source.sourceStart;
method.bodyEnd = method.declarationSourceEnd = method.sourceEnd = source.sourceEnd;
List<Statement> statements = new ArrayList<Statement>(5);
if (hasNonNullAnnotations(fieldNode)) {
Statement nullCheck = generateNullCheck(field, sourceNode, null);
if (nullCheck != null) statements.add(nullCheck);
}
statements.add(returnStatement);
```

```
method.statements = statements.toArray(new Statement[0]);
}
createRelevantNonNullAnnotation(sourceNode, param, method);
createRelevantNonNullAnnotation(fieldNode, method);
method.traverse(new SetGeneratedByVisitor(source), parent.scope);
copyJavadoc(fieldNode, method, CopyJavadoc.WITH_BY);
return method;
}
}
```

lombok/eclipse/handlers/SetGeneratedByVisitor.java


```
/*
* Copyright (C) 2011-2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.eclipse.handlers;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.Arrays;
import org.eclipse.jdt.internal.compiler.ASTVisitor;
import org.eclipse.jdt.internal.compiler.ast.AND_AND_Expression;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.AnnotationMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.ArrayAllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.ArrayQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ArrayReference;
import org.eclipse.jdt.internal.compiler.ast.ArrayTypeReference;
import org.eclipse.jdt.internal.compiler.ast.AssertStatement;
```

```
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.BinaryExpression;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.BreakStatement;
import org.eclipse.jdt.internal.compiler.ast.CaseStatement;
import org.eclipse.jdt.internal.compiler.ast.CastExpression;
import org.eclipse.jdt.internal.compiler.ast.CharLiteral;
import org.eclipse.jdt.internal.compiler.ast.ClassLiteralAccess;
import org.eclipse.jdt.internal.compiler.ast.Clinit;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.CompoundAssignment;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ContinueStatement;
import org.eclipse.jdt.internal.compiler.ast.DoStatement;
import org.eclipse.jdt.internal.compiler.ast.DoubleLiteral;
import org.eclipse.jdt.internal.compiler.ast.EmptyStatement;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.ExplicitConstructorCall;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.ExtendedStringLiteral;
import org.eclipse.jdt.internal.compiler.ast.FalseLiteral;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.FloatLiteral;
import org.eclipse.jdt.internal.compiler.ast.ForStatement;
import org.eclipse.jdt.internal.compiler.ast.ForeachStatement;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.ImportReference;
import org.eclipse.jdt.internal.compiler.ast.Initializer;
import org.eclipse.jdt.internal.compiler.ast.InstanceOfExpression;
import org.eclipse.jdt.internal.compiler.ast.IntLiteral;
import org.eclipse.jdt.internal.compiler.ast.Javadoc;
import org.eclipse.jdt.internal.compiler.ast.JavadocAllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.JavadocArgumentExpression;
import org.eclipse.jdt.internal.compiler.ast.JavadocArrayQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.JavadocArraySingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.JavadocFieldReference;
import org.eclipse.jdt.internal.compiler.ast.JavadocImplicitTypeReference;
import org.eclipse.jdt.internal.compiler.ast.JavadocMessageSend;
import org.eclipse.jdt.internal.compiler.ast.JavadocQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.JavadocReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.JavadocSingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.JavadocSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.LabeledStatement;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.LongLiteral;
import org.eclipse.jdt.internal.compiler.ast.MarkerAnnotation;
import org.eclipse.jdt.internal.compiler.ast.MemberValuePair;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NormalAnnotation;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
```

```
import org.eclipse.jdt.internal.compiler.ast.OR_OR_Expression;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedQualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ParameterizedSingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.PostfixExpression;
import org.eclipse.jdt.internal.compiler.ast.PrefixExpression;
import org.eclipse.jdt.internal.compiler.ast.QualifiedAllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedSuperReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedThisReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleMemberAnnotation;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.StringLiteral;
import org.eclipse.jdt.internal.compiler.ast.StringLiteralConcatenation;
import org.eclipse.jdt.internal.compiler.ast.SuperReference;
import org.eclipse.jdt.internal.compiler.ast.SwitchStatement;
import org.eclipse.jdt.internal.compiler.ast.SynchronizedStatement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.ThrowStatement;
import org.eclipse.jdt.internal.compiler.ast.TrueLiteral;
import org.eclipse.jdt.internal.compiler.ast.TryStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeParameter;
import org.eclipse.jdt.internal.compiler.ast.UnaryExpression;
import org.eclipse.jdt.internal.compiler.ast.WhileStatement;
import org.eclipse.jdt.internal.compiler.ast.Wildcard;
import org.eclipse.jdt.internal.compiler.lookup.BlockScope;
import org.eclipse.jdt.internal.compiler.lookup.ClassScope;
import org.eclipse.jdt.internal.compiler.lookup.CompilationUnitScope;
import org.eclipse.jdt.internal.compiler.lookup.MethodScope;
public final class SetGeneratedByVisitor extends ASTVisitor {
private static final long INT_TO_LONG_MASK = 0x00000000FFFFFFFFL;
private final ASTNode source;
private final int sourceStart;
private final int sourceEnd;
private final long sourcePos;
public SetGeneratedByVisitor(ASTNode source) {
this.source = source;
this.sourceStart = this.source.sourceStart;
this.sourceEnd = this.source.sourceEnd;
this.sourcePos = (long)sourceStart << 32 | (sourceEnd & INT_TO_LONG_MASK);
}
private void fixPositions(JavadocAllocationExpression node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.memberStart = sourceStart;
node.tagSourceEnd = sourceEnd;
node.tagSourceStart = sourceStart;
}
private void fixPositions(JavadocMessageSend node) {
```

```
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.nameSourcePosition = sourcePos;
node.tagSourceEnd = sourceEnd;
node.tagSourceStart = sourceStart;
}
private void fixPositions(JavadocSingleNameReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.tagSourceEnd = sourceEnd;
node.tagSourceStart = sourceStart;
}
private void fixPositions(JavadocSingleTypeReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.tagSourceEnd = sourceEnd;
node.tagSourceStart = sourceStart;
}
private void fixPositions(JavadocFieldReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.nameSourcePosition = sourcePos;
node.tagSourceEnd = sourceEnd;
node.tagSourceStart = sourceStart;
}
private void fixPositions(JavadocArrayQualifiedTypeReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
if (node.sourcePositions == null || node.sourcePositions.length != node.tokens.length)
node.sourcePositions = new long[node.tokens.length];
Arrays.fill(node.sourcePositions, sourcePos);
node.tagSourceEnd = sourceEnd;
node.tagSourceStart = sourceStart;
}
private void fixPositions(JavadocQualifiedTypeReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
if (node.sourcePositions == null || node.sourcePositions.length != node.tokens.length)
node.sourcePositions = new long[node.tokens.length];
Arrays.fill(node.sourcePositions, sourcePos);
node.tagSourceEnd = sourceEnd;
node.tagSourceStart = sourceStart;
}
private void fixPositions(Annotation node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
```

```
node.declarationSourceEnd = sourceEnd;
}
private void fixPositions(ArrayTypeReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.originalSourceEnd = sourceEnd;
}
private void fixPositions(AbstractMethodDeclaration node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.bodyEnd = sourceEnd;
node.bodyStart = sourceStart;
node.declarationSourceEnd = sourceEnd;
node.declarationSourceStart = sourceStart;
node.modifiersSourceStart = sourceStart;
}
private void fixPositions(Javadoc node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.valuePositions = sourceStart;
}
private void fixPositions(Initializer node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.declarationEnd = sourceEnd;
node.declarationSourceEnd = sourceEnd;
node.declarationSourceStart = sourceStart;
node.modifiersSourceStart = sourceStart;
node.endPart1Position = sourceEnd;
node.endPart2Position = sourceEnd;
node.bodyStart = sourceStart;
node.bodyEnd = sourceEnd;
}
private void fixPositions(TypeDeclaration node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.bodyEnd = sourceEnd;
node.bodyStart = sourceStart;
node.declarationSourceEnd = sourceEnd;
node.declarationSourceStart = sourceStart;
node.modifiersSourceStart = sourceStart;
}
private void fixPositions(ImportReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.declarationEnd = sourceEnd;
node.declarationSourceEnd = sourceEnd;
node.declarationSourceStart = sourceStart;
if (node.sourcePositions == null || node.sourcePositions.length != node.tokens.length)
node.sourcePositions = new long[node.tokens.length];
Arrays.fill(node.sourcePositions, sourcePos);
}
```

```
private void fixPositions(ASTNode node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
}
private void fixPositions(SwitchStatement node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.blockStart = sourceStart;
}
private void fixPositions(Expression node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
}
private void fixPositions(AbstractVariableDeclaration node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.declarationEnd = sourceEnd;
node.declarationSourceEnd = sourceEnd;
node.declarationSourceStart = sourceStart;
node.modifiersSourceStart = sourceStart;
}
private void fixPositions(FieldDeclaration node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.declarationEnd = sourceEnd;
node.declarationSourceEnd = sourceEnd;
node.declarationSourceStart = sourceStart;
node.modifiersSourceStart = sourceStart;
node.endPart1Position = sourceEnd;
node.endPart2Position = sourceEnd;
}
private void fixPositions(FieldReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.nameSourcePosition = sourcePos;
}
private void fixPositions(MessageSend node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
node.nameSourcePosition = sourcePos;
}
private void fixPositions(QualifiedNameReference node) {
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
if (node.sourcePositions == null || node.sourcePositions.length != node.tokens.length)
node.sourcePositions = new long[node.tokens.length];
Arrays.fill(node.sourcePositions, sourcePos);
}
private void fixPositions(QualifiedTypeReference node) {
```

```
node.sourceEnd = sourceEnd;
node.sourceStart = sourceStart;
node.statementEnd = sourceEnd;
if (node.sourcePositions == null || node.sourcePositions.length != node.tokens.length)
node.sourcePositions = new long[node.tokens.length];
Arrays.fill(node.sourcePositions, sourcePos);
}
@Override public boolean visit(AllocationExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(AND_AND_Expression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(AnnotationMethodDeclaration node, ClassScope classScope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, classScope);
}
@Override public boolean visit(Argument node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Argument node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ArrayAllocationExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ArrayInitializer node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ArrayQualifiedTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ArrayQualifiedTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ArrayReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ArrayTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ArrayTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
```

```
return super.visit(node, scope);
}
@Override public boolean visit(AssertStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Assignment node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(BinaryExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Block node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(BreakStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(CaseStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(CastExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(CharLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ClassLiteralAccess node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Clinit node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(CompilationUnitDeclaration node, CompilationUnitScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(CompoundAssignment node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ConditionalExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
```

```
}
@Override public boolean visit(ConstructorDeclaration node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ContinueStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(DoStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(DoubleLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(EmptyStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(EqualExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ExplicitConstructorCall node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ExtendedStringLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(FalseLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(FieldDeclaration node, MethodScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(FieldReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(FieldReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(FloatLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
```

```
@Override public boolean visit(ForeachStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ForStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(IfStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ImportReference node, CompilationUnitScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Initializer node, MethodScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(InstanceOfExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(IntLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Javadoc node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Javadoc node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocAllocationExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocAllocationExpression node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocArgumentExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocArgumentExpression node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocArrayQualifiedTypeReference node, BlockScope scope) {
```

```
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocArrayQualifiedTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocArraySingleTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocArraySingleTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocFieldReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocFieldReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocImplicitTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocImplicitTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocMessageSend node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocMessageSend node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocQualifiedTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocQualifiedTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocReturnStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocReturnStatement node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
```

```
return super.visit(node, scope);
}
@Override public boolean visit(JavadocSingleNameReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocSingleNameReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocSingleTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(JavadocSingleTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(LabeledStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(LocalDeclaration node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(LongLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(MarkerAnnotation node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(MemberValuePair node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(MessageSend node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(MethodDeclaration node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(StringLiteralConcatenation node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(NormalAnnotation node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
```

```
}
@Override public boolean visit(NullLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(OR_OR_Expression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ParameterizedQualifiedTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ParameterizedQualifiedTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ParameterizedSingleTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ParameterizedSingleTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(PostfixExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(PrefixExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedAllocationExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedNameReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedNameReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedSuperReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedSuperReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
```

```
@Override public boolean visit(QualifiedThisReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedThisReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(QualifiedTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ReturnStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SingleMemberAnnotation node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SingleNameReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SingleNameReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SingleTypeReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SingleTypeReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(StringLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SuperReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SwitchStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(SynchronizedStatement node, BlockScope scope) {
```

```
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ThisReference node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ThisReference node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(ThrowStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(TrueLiteral node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(TryStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(TypeDeclaration node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(TypeDeclaration node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(TypeDeclaration node, CompilationUnitScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(TypeParameter node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(TypeParameter node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(UnaryExpression node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(WhileStatement node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
@Override public boolean visit(Wildcard node, BlockScope scope) {
fixPositions(setGeneratedBy(node, source));
```

```
return super.visit(node, scope);
}
@Override public boolean visit(Wildcard node, ClassScope scope) {
fixPositions(setGeneratedBy(node, source));
return super.visit(node, scope);
}
// missing methods
// public boolean visit(MarkerAnnotation node, ClassScope scope){ return true;}
// public boolean visit(MemberValuePair node, ClassScope scope){ return true;}
// public boolean visit(NormalAnnotation node, ClassScope scope){ return true;}
// public boolean visit(SingleMemberAnnotation node, ClassScope scope){ return true;}
// missing methods from later versions
// public boolean visit(UnionTypeReference node, BlockScope scope){ return true;}
// public boolean visit(UnionTypeReference node, ClassScope scope){ return true;}
// public boolean visit(LambdaExpression node, BlockScope scope){ return true;}
// public boolean visit(ReferenceExpression node, BlockScope scope){ return true;}
// public boolean visit(IntersectionCastTypeReference node, ClassScope scope){ return true;}
// public boolean visit(IntersectionCastTypeReference node, BlockScope scope){ return true;}
}
```

lombok/eclipse/handlers/package-info.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* Contains the classes that implement the transformations for all of lombok's various features on the
eclipse platform.
*
* <strong>NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.</strong>
*/
package lombok.eclipse.handlers;
```

lombok/eclipse/handlers/singulars/EclipseGuavaMapSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import lombok.core.LombokImmutableList;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.spi.Provides;
@Provides(EclipseSingularizer.class)
public class EclipseGuavaMapSingularizer extends EclipseGuavaSingularizer {
// TODO cgcc.ImmutableMultimap, cgcc.ImmutableListMultimap, cgcc.ImmutableSetMultimap
// TODO cgcc.ImmutableClassToInstanceMap
// TODO cgcc.ImmutableRangeMap
private static final LombokImmutableList<String> SUFFIXES =
LombokImmutableList.of("key", "value");
private static final LombokImmutableList<String> SUPPORTED_TYPES = LombokImmutableList.of(
"com.google.common.collect.ImmutableMap",
"com.google.common.collect.ImmutableBiMap",
"com.google.common.collect.ImmutableSortedMap"
);
@Override public LombokImmutableList<String> getSupportedTypes() {
return SUPPORTED_TYPES;
}
@Override protected LombokImmutableList<String> getArgumentSuffixes() {
return SUFFIXES;
}
@Override protected String getAddMethodName() {
return "put";
}
```

```
@Override protected String getAddAllTypeName() {
return "java.util.Map";
}
}
```

lombok/eclipse/handlers/singulars/EclipseGuavaSetListSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import lombok.core.LombokImmutableList;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.spi.Provides;
@Provides(EclipseSingularizer.class)
public class EclipseGuavaSetListSingularizer extends EclipseGuavaSingularizer {
// TODO com.google.common.collect.ImmutableRangeSet
// TODO com.google.common.collect.ImmutableMultiset and
com.google.common.collect.ImmutableSortedMultiset
private static final LombokImmutableList<String> SUFFIXES = LombokImmutableList.of("");
private static final LombokImmutableList<String> SUPPORTED_TYPES = LombokImmutableList.of(
"com.google.common.collect.ImmutableCollection",
"com.google.common.collect.ImmutableList",
"com.google.common.collect.ImmutableSet",
"com.google.common.collect.ImmutableSortedSet"
);
@Override public LombokImmutableList<String> getSupportedTypes() {
return SUPPORTED_TYPES;
}
@Override protected LombokImmutableList<String> getArgumentSuffixes() {
return SUFFIXES;
```

```
}
@Override protected String getAddMethodName() {
return "add";
}
@Override protected String getAddAllTypeName() {
return "java.lang.Iterable";
}
}
```

lombok/eclipse/handlers/singulars/EclipseGuavaSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import lombok.AccessLevel;
import lombok.core.GuavaTypeMap;
import lombok.core.LombokImmutableList;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.HandleNonNull;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
import lombok.eclipse.handlers.EclipseSingularsRecipes.StatementMaker;
import lombok.eclipse.handlers.EclipseSingularsRecipes.TypeReferenceMaker;
```

```
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
abstract class EclipseGuavaSingularizer extends EclipseSingularizer {
protected static final char[] OF = {'o', 'f'};
protected static final char[][] CGCC = {{'c', 'o', 'm'}, {'g', 'o', 'o', 'g', 'l', 'e'}, {'c', 'o', 'm', 'm', 'o', 'n'}, {'c', 'o', 'l',
'l', 'e', 'c', 't'}};
protected String getSimpleTargetTypeName(SingularData data) {
return GuavaTypeMap.getGuavaTypeName(data.getTargetFqn());
}
protected char[] getBuilderMethodName(SingularData data) {
String simpleTypeName = getSimpleTargetTypeName(data);
if ("ImmutableSortedSet".equals(simpleTypeName) ||
"ImmutableSortedMap".equals(simpleTypeName)) return "naturalOrder".toCharArray();
return "builder".toCharArray();
}
protected char[][] makeGuavaTypeName(String simpleName, boolean addBuilder) {
char[][] tokenizedName = new char[addBuilder ? 6 : 5][];
tokenizedName[0] = CGCC[0];
tokenizedName[1] = CGCC[1];
tokenizedName[2] = CGCC[2];
tokenizedName[3] = CGCC[3];
tokenizedName[4] = simpleName.toCharArray();
if (addBuilder) tokenizedName[5] = new char[] { 'B', 'u', 'i', 'l', 'd', 'e', 'r'};
return tokenizedName;
}
@Override protected char[] getEmptyMakerSelector(String targetFqn) {
return OF;
}
@Override protected char[][] getEmptyMakerReceiver(String targetFqn) {
return makeGuavaTypeName(GuavaTypeMap.getGuavaTypeName(targetFqn), false);
}
@Override public List<EclipseNode> generateFields(SingularData data, EclipseNode builderType) {
String simpleTypeName = getSimpleTargetTypeName(data);
```

```
char[][] tokenizedName = makeGuavaTypeName(simpleTypeName, true);
TypeReference type = new QualifiedTypeReference(tokenizedName, NULL_POSS);
type = addTypeArgs(getTypeArgumentsCount(), false, builderType, type, data.getTypeArgs());
FieldDeclaration buildField = new FieldDeclaration(data.getPluralName(), 0, -1);
buildField.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
buildField.modifiers = ClassFileConstants.AccPrivate;
buildField.declarationSourceEnd = -1;
buildField.type = type;
data.setGeneratedByRecursive(buildField);
return Collections.singletonList(injectFieldAndMarkGenerated(builderType, buildField));
}
@Override public void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, EclipseNode builderType, boolean fluent, TypeReferenceMaker returnTypeMaker,
StatementMaker returnStatementMaker, AccessLevel access) {
generateSingularMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(),
data, builderType, fluent, access);
generatePluralMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(), data,
builderType, fluent, access);
generateClearMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(), data,
builderType, access);
}
void generateClearMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, AccessLevel
access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = new ThisReference(0, 0);
Assignment a = new Assignment(thisDotField, new NullLiteral(0, 0), 0);
md.selector = HandlerUtil.buildAccessorName(builderType, "clear", new
String(data.getPluralName())).toCharArray();
md.statements = returnStatement != null ? new Statement[] {a, returnStatement} : new Statement[] {a};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
md.annotations = generateSelfReturnAnnotations(deprecate, data.getSource());
data.setGeneratedByRecursive(md);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
injectMethod(builderType, md);
}
void generateSingularMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, boolean fluent,
AccessLevel access) {
LombokImmutableList<String> suffixes = getArgumentSuffixes();
char[][] names = new char[suffixes.size()][];
for (int i = 0; i < suffixes.size(); i++) {
String s = suffixes.get(i);
char[] n = data.getSingularName();
names[i] = s.isEmpty() ? n : s.toCharArray();
}
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
```

```
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
List<Statement> statements = new ArrayList<Statement>();
statements.add(createConstructBuilderVarIfNeeded(data, builderType));
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = new ThisReference(0, 0);
MessageSend thisDotFieldDotAdd = new MessageSend();
thisDotFieldDotAdd.arguments = new Expression[suffixes.size()];
for (int i = 0; i < suffixes.size(); i++) {
thisDotFieldDotAdd.arguments[i] = new SingleNameReference(names[i], 0L);
}
thisDotFieldDotAdd.receiver = thisDotField;
thisDotFieldDotAdd.selector = getAddMethodName().toCharArray();
statements.add(thisDotFieldDotAdd);
if (returnStatement != null) statements.add(returnStatement);
md.statements = statements.toArray(new Statement[0]);
md.arguments = new Argument[suffixes.size()];
for (int i = 0; i < suffixes.size(); i++) {
TypeReference tr = cloneParamType(i, data.getTypeArgs(), builderType);
Annotation[] typeUseAnns = getTypeUseAnnotations(tr);
removeTypeUseAnnotations(tr);
md.arguments[i] = new Argument(names[i], 0, tr, ClassFileConstants.AccFinal);
md.arguments[i].annotations = typeUseAnns;
}
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
char[] prefixedSingularName = data.getSetterPrefix().length == 0 ? data.getSingularName() :
HandlerUtil.buildAccessorName(builderType, new String(data.getSetterPrefix()), new
String(data.getSingularName())).toCharArray();
md.selector = fluent ? prefixedSingularName : HandlerUtil.buildAccessorName(builderType, "add", new
String(data.getSingularName())).toCharArray();
Annotation[] selfReturnAnnotations = generateSelfReturnAnnotations(deprecate, data.getSource());
Annotation[] copyToSetterAnnotations = copyAnnotations(md,
findCopyableToBuilderSingularSetterAnnotations(data.getAnnotation().up()));
md.annotations = concat(selfReturnAnnotations, copyToSetterAnnotations, Annotation.class);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
data.setGeneratedByRecursive(md);
HandleNonNull.INSTANCE.fix(injectMethod(builderType, md));
}
void generatePluralMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, boolean fluent,
AccessLevel access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
List<Statement> statements = new ArrayList<Statement>();
statements.add(createConstructBuilderVarIfNeeded(data, builderType));
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = new ThisReference(0, 0);
MessageSend thisDotFieldDotAddAll = new MessageSend();
thisDotFieldDotAddAll.arguments = new Expression[] {new
SingleNameReference(data.getPluralName(), 0L)};
```

```
thisDotFieldDotAddAll.receiver = thisDotField;
thisDotFieldDotAddAll.selector = (getAddMethodName() + "All").toCharArray();
statements.add(thisDotFieldDotAddAll);
TypeReference paramType;
paramType = new QualifiedTypeReference(fromQualifiedName(getAddAllTypeName()), NULL_POSS);
paramType = addTypeArgs(getTypeArgumentsCount(), true, builderType, paramType,
data.getTypeArgs());
Argument param = new Argument(data.getPluralName(), 0, paramType, ClassFileConstants.AccFinal);
nullBehaviorize(builderType, data, statements, param, md);
if (returnStatement != null) statements.add(returnStatement);
md.statements = statements.toArray(new Statement[0]);
md.arguments = new Argument[] {param};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
char[] prefixedSelector = data.getSetterPrefix().length == 0 ? data.getPluralName() :
HandlerUtil.buildAccessorName(builderType, new String(data.getSetterPrefix()), new
String(data.getPluralName())).toCharArray();
md.selector = fluent ? prefixedSelector : HandlerUtil.buildAccessorName(builderType, "addAll", new
String(data.getPluralName())).toCharArray();
Annotation[] selfReturnAnnotations = generateSelfReturnAnnotations(deprecate, data.getSource());
Annotation[] copyToSetterAnnotations = copyAnnotations(md,
findCopyableToSetterAnnotations(data.getAnnotation().up()));
md.annotations = concat(selfReturnAnnotations, copyToSetterAnnotations, Annotation.class);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
data.setGeneratedByRecursive(md);
injectMethod(builderType, md);
}
@Override public void appendBuildCode(SingularData data, EclipseNode builderType,
List<Statement> statements, char[] targetVariableName, String builderVariable) {
TypeReference varType = new QualifiedTypeReference(fromQualifiedName(data.getTargetFqn()),
NULL_POSS);
String simpleTypeName = getSimpleTargetTypeName(data);
int agrumentsCount = getTypeArgumentsCount();
varType = addTypeArgs(agrumentsCount, false, builderType, varType, data.getTypeArgs());
MessageSend emptyInvoke; {
//ImmutableX.of()
emptyInvoke = new MessageSend();
emptyInvoke.selector = new char[] {'o', 'f'};
emptyInvoke.receiver = new QualifiedNameReference(makeGuavaTypeName(simpleTypeName,
false), NULL_POSS, 0, 0);
emptyInvoke.typeArguments = createTypeArgs(agrumentsCount, false, builderType,
data.getTypeArgs());
}
MessageSend invokeBuild; {
//this.pluralName.build();
invokeBuild = new MessageSend();
invokeBuild.selector = new char[] {'b', 'u', 'i', 'l', 'd'};
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = getBuilderReference(builderVariable);
invokeBuild.receiver = thisDotField;
}
Expression isNull; {
//this.pluralName == null
```

```
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = getBuilderReference(builderVariable);
isNull = new EqualExpression(thisDotField, new NullLiteral(0, 0), OperatorIds.EQUAL_EQUAL);
}
Expression init = new ConditionalExpression(isNull, emptyInvoke, invokeBuild);
LocalDeclaration varDefStat = new LocalDeclaration(data.getPluralName(), 0, 0);
varDefStat.type = varType;
varDefStat.initialization = init;
statements.add(varDefStat);
}
protected Statement createConstructBuilderVarIfNeeded(SingularData data, EclipseNode builderType)
{
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = new ThisReference(0, 0);
FieldReference thisDotField2 = new FieldReference(data.getPluralName(), 0L);
thisDotField2.receiver = new ThisReference(0, 0);
Expression cond = new EqualExpression(thisDotField, new NullLiteral(0, 0),
OperatorIds.EQUAL_EQUAL);
MessageSend createBuilderInvoke = new MessageSend();
char[][] tokenizedName = makeGuavaTypeName(getSimpleTargetTypeName(data), false);
createBuilderInvoke.receiver = new QualifiedNameReference(tokenizedName, NULL_POSS, 0, 0);
createBuilderInvoke.selector = getBuilderMethodName(data);
return new IfStatement(cond, new Assignment(thisDotField2, createBuilderInvoke, 0), 0, 0);
}
protected abstract LombokImmutableList<String> getArgumentSuffixes();
protected abstract String getAddMethodName();
protected abstract String getAddAllTypeName();
@Override protected int getTypeArgumentsCount() {
return getArgumentSuffixes().size();
}
}
```

lombok/eclipse/handlers/singulars/EclipseGuavaTableSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import lombok.core.LombokImmutableList;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.spi.Provides;
@Provides(EclipseSingularizer.class)
public class EclipseGuavaTableSingularizer extends EclipseGuavaSingularizer {
private static final LombokImmutableList<String> SUFFIXES =
LombokImmutableList.of("rowKey", "columnKey", "value");
private static final LombokImmutableList<String> SUPPORTED_TYPES =
LombokImmutableList.of("com.google.common.collect.ImmutableTable");
@Override public LombokImmutableList<String> getSupportedTypes() {
return SUPPORTED_TYPES;
}
@Override protected LombokImmutableList<String> getArgumentSuffixes() {
return SUFFIXES;
}
@Override protected String getAddMethodName() {
return "put";
}
@Override protected String getAddAllTypeName() {
return "com.google.common.collect.Table";
}
}

lombok/eclipse/handlers/singulars/EclipseJavaUtilListSetSingularizer.java

/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import static lombok.eclipse.Eclipse.ECLIPSE_DO_NOT_TOUCH_FLAG;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import lombok.AccessLevel;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.HandleNonNull;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
import lombok.eclipse.handlers.EclipseSingularsRecipes.StatementMaker;
import lombok.eclipse.handlers.EclipseSingularsRecipes.TypeReferenceMaker;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
abstract class EclipseJavaUtilListSetSingularizer extends EclipseJavaUtilSingularizer {
@Override public List<char[]> listFieldsToBeGenerated(SingularData data, EclipseNode builderType) {
if (useGuavaInstead(builderType)) {
return guavaListSetSingularizer.listFieldsToBeGenerated(data, builderType);
}
return super.listFieldsToBeGenerated(data, builderType);
}
@Override public List<char[]> listMethodsToBeGenerated(SingularData data, EclipseNode
builderType) {
if (useGuavaInstead(builderType)) {
return guavaListSetSingularizer.listMethodsToBeGenerated(data, builderType);
}
return super.listMethodsToBeGenerated(data, builderType);
}
@Override public List<EclipseNode> generateFields(SingularData data, EclipseNode builderType) {
if (useGuavaInstead(builderType)) {
```

```
return guavaListSetSingularizer.generateFields(data, builderType);
}
TypeReference type = new QualifiedTypeReference(JAVA_UTIL_ARRAYLIST, NULL_POSS);
type = addTypeArgs(1, false, builderType, type, data.getTypeArgs());
FieldDeclaration buildField = new FieldDeclaration(data.getPluralName(), 0, -1);
buildField.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
buildField.modifiers = ClassFileConstants.AccPrivate;
buildField.declarationSourceEnd = -1;
buildField.type = type;
data.setGeneratedByRecursive(buildField);
return Collections.singletonList(injectFieldAndMarkGenerated(builderType, buildField));
}
@Override public void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, EclipseNode builderType, boolean fluent, TypeReferenceMaker returnTypeMaker,
StatementMaker returnStatementMaker, AccessLevel access) {
if (useGuavaInstead(builderType)) {
guavaListSetSingularizer.generateMethods(cfv, data, deprecate, builderType, fluent, returnTypeMaker,
returnStatementMaker, access);
return;
}
generateSingularMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(),
data, builderType, fluent, access);
generatePluralMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(), data,
builderType, fluent, access);
generateClearMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(), data,
builderType, access);
}
private void generateClearMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, AccessLevel
access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = new ThisReference(0, 0);
FieldReference thisDotField2 = new FieldReference(data.getPluralName(), 0L);
thisDotField2.receiver = new ThisReference(0, 0);
md.selector = HandlerUtil.buildAccessorName(builderType, "clear", new
String(data.getPluralName())).toCharArray();
MessageSend clearMsg = new MessageSend();
clearMsg.receiver = thisDotField2;
clearMsg.selector = "clear".toCharArray();
Statement clearStatement = new IfStatement(new EqualExpression(thisDotField, new NullLiteral(0, 0),
OperatorIds.NOT_EQUAL), clearMsg, 0, 0);
md.statements = returnStatement != null ? new Statement[] {clearStatement, returnStatement} : new
Statement[] {clearStatement};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
md.annotations = generateSelfReturnAnnotations(deprecate, data.getSource());
data.setGeneratedByRecursive(md);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
injectMethod(builderType, md);
```

```
}
void generateSingularMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, boolean fluent,
AccessLevel access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
List<Statement> statements = new ArrayList<Statement>();
statements.add(createConstructBuilderVarIfNeeded(data, builderType, false));
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = new ThisReference(0, 0);
MessageSend thisDotFieldDotAdd = new MessageSend();
thisDotFieldDotAdd.arguments = new Expression[] {new
SingleNameReference(data.getSingularName(), 0L)};
thisDotFieldDotAdd.receiver = thisDotField;
thisDotFieldDotAdd.selector = "add".toCharArray();
statements.add(thisDotFieldDotAdd);
if (returnStatement != null) statements.add(returnStatement);
md.statements = statements.toArray(new Statement[0]);
TypeReference paramType = cloneParamType(0, data.getTypeArgs(), builderType);
Annotation[] typeUseAnns = getTypeUseAnnotations(paramType);
removeTypeUseAnnotations(paramType);
Argument param = new Argument(data.getSingularName(), 0, paramType,
ClassFileConstants.AccFinal);
param.annotations = typeUseAnns;
md.arguments = new Argument[] {param};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
char[] prefixedSingularName = data.getSetterPrefix().length == 0 ? data.getSingularName() :
HandlerUtil.buildAccessorName(builderType, new String(data.getSetterPrefix()), new
String(data.getSingularName())).toCharArray();
md.selector = fluent ? prefixedSingularName : HandlerUtil.buildAccessorName(builderType, "add", new
String(data.getSingularName())).toCharArray();
Annotation[] selfReturnAnnotations = generateSelfReturnAnnotations(deprecate, data.getSource());
Annotation[] copyToSetterAnnotations = copyAnnotations(md,
findCopyableToBuilderSingularSetterAnnotations(data.getAnnotation().up()));
md.annotations = concat(selfReturnAnnotations, copyToSetterAnnotations, Annotation.class);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
data.setGeneratedByRecursive(md);
HandleNonNull.INSTANCE.fix(injectMethod(builderType, md));
}
void generatePluralMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, boolean fluent,
AccessLevel access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
List<Statement> statements = new ArrayList<Statement>();
statements.add(createConstructBuilderVarIfNeeded(data, builderType, false));
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = new ThisReference(0, 0);
```

```
MessageSend thisDotFieldDotAddAll = new MessageSend();
thisDotFieldDotAddAll.arguments = new Expression[] {new
SingleNameReference(data.getPluralName(), 0L)};
thisDotFieldDotAddAll.receiver = thisDotField;
thisDotFieldDotAddAll.selector = "addAll".toCharArray();
statements.add(thisDotFieldDotAddAll);
TypeReference paramType = new
QualifiedTypeReference(TypeConstants.JAVA_UTIL_COLLECTION, NULL_POSS);
paramType = addTypeArgs(1, true, builderType, paramType, data.getTypeArgs());
Argument param = new Argument(data.getPluralName(), 0, paramType, ClassFileConstants.AccFinal);
nullBehaviorize(builderType, data, statements, param, md);
if (returnStatement != null) statements.add(returnStatement);
md.statements = statements.toArray(new Statement[0]);
md.arguments = new Argument[] {param};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
char[] prefixedSelector = data.getSetterPrefix().length == 0 ? data.getPluralName() :
HandlerUtil.buildAccessorName(builderType, new String(data.getSetterPrefix()), new
String(data.getPluralName())).toCharArray();
md.selector = fluent ? prefixedSelector : HandlerUtil.buildAccessorName(builderType, "addAll", new
String(data.getPluralName())).toCharArray();
Annotation[] selfReturnAnnotations = generateSelfReturnAnnotations(deprecate, data.getSource());
Annotation[] copyToSetterAnnotations = copyAnnotations(md,
findCopyableToSetterAnnotations(data.getAnnotation().up()));
md.annotations = concat(selfReturnAnnotations, copyToSetterAnnotations, Annotation.class);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
data.setGeneratedByRecursive(md);
injectMethod(builderType, md);
}
@Override protected int getTypeArgumentsCount() {
return 1;
}
}
```

lombok/eclipse/handlers/singulars/EclipseJavaUtilListSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
```

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import static lombok.eclipse.handlers.EclipseHandlerUtil.makeIntLiteral;
import java.util.ArrayList;
import java.util.List;
import lombok.core.LombokImmutableList;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.BreakStatement;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SwitchStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
@Provides(EclipseSingularizer.class)
public class EclipseJavaUtilListSingularizer extends EclipseJavaUtilListSetSingularizer {
@Override public LombokImmutableList<String> getSupportedTypes() {
return LombokImmutableList.of("java.util.List", "java.util.Collection", "java.lang.Iterable");
}
private static final char[] EMPTY_LIST = {'e', 'm', 'p', 't', 'y', 'L', 'i', 's', 't'};
@Override protected char[][] getEmptyMakerReceiver(String targetFqn) {
return JAVA_UTIL_COLLECTIONS;
}
@Override protected char[] getEmptyMakerSelector(String targetFqn) {
return EMPTY_LIST;
}
@Override public void appendBuildCode(SingularData data, EclipseNode builderType,
List<Statement> statements, char[] targetVariableName, String builderVariable) {
if (useGuavaInstead(builderType)) {
guavaListSetSingularizer.appendBuildCode(data, builderType, statements, targetVariableName,
builderVariable);
return;
}
List<Statement> switchContents = new ArrayList<Statement>();
/* case 0: (empty) break; */ {
switchContents.add(Eclipse.createCaseStatement(makeIntLiteral(new char[] {'0'}, null)));
```

```
MessageSend invoke = new MessageSend();
invoke.receiver = new QualifiedNameReference(JAVA_UTIL_COLLECTIONS, NULL_POSS, 0, 0);
invoke.selector = "emptyList".toCharArray();
switchContents.add(new Assignment(new SingleNameReference(data.getPluralName(), 0), invoke,
0));
switchContents.add(new BreakStatement(null, 0, 0));
}
/* case 1: (singleton) break; */ {
switchContents.add(Eclipse.createCaseStatement(makeIntLiteral(new char[] {'1'}, null)));
FieldReference thisDotField = new FieldReference(data.getPluralName(), 0L);
thisDotField.receiver = getBuilderReference(builderVariable);
MessageSend thisDotFieldGet0 = new MessageSend();
thisDotFieldGet0.receiver = thisDotField;
thisDotFieldGet0.selector = new char[] {'g', 'e', 't'};
thisDotFieldGet0.arguments = new Expression[] {makeIntLiteral(new char[] {'0'}, null)};
Expression[] args = new Expression[] {thisDotFieldGet0};
MessageSend invoke = new MessageSend();
invoke.receiver = new QualifiedNameReference(JAVA_UTIL_COLLECTIONS, NULL_POSS, 0, 0);
invoke.selector = "singletonList".toCharArray();
invoke.arguments = args;
switchContents.add(new Assignment(new SingleNameReference(data.getPluralName(), 0), invoke,
0));
switchContents.add(new BreakStatement(null, 0, 0));
}
/* default: Create by passing builder field to constructor. */ {
switchContents.add(Eclipse.createCaseStatement(null));
Expression argToUnmodifiable;
/* new j.u.ArrayList<Generics>(this.pluralName); */ {
FieldReference thisDotPluralName = new FieldReference(data.getPluralName(), 0L);
thisDotPluralName.receiver = getBuilderReference(builderVariable);
TypeReference targetTypeExpr = new QualifiedTypeReference(JAVA_UTIL_ARRAYLIST,
NULL_POSS);
targetTypeExpr = addTypeArgs(1, false, builderType, targetTypeExpr, data.getTypeArgs());
AllocationExpression constructorCall = new AllocationExpression();
constructorCall.type = targetTypeExpr;
constructorCall.arguments = new Expression[] {thisDotPluralName};
argToUnmodifiable = constructorCall;
}
/* pluralname = Collections.unmodifiableList(-newlist-); */ {
MessageSend unmodInvoke = new MessageSend();
unmodInvoke.receiver = new QualifiedNameReference(JAVA_UTIL_COLLECTIONS, NULL_POSS, 0,
0);
unmodInvoke.selector = "unmodifiableList".toCharArray();
unmodInvoke.arguments = new Expression[] {argToUnmodifiable};
switchContents.add(new Assignment(new SingleNameReference(data.getPluralName(), 0),
unmodInvoke, 0));
}
}
SwitchStatement switchStat = new SwitchStatement();
switchStat.statements = switchContents.toArray(new Statement[0]);
switchStat.expression = getSize(builderType, data.getPluralName(), true, builderVariable);
TypeReference localShadowerType = new
QualifiedTypeReference(Eclipse.fromQualifiedName(data.getTargetFqn()), NULL_POSS);
```

```
localShadowerType = addTypeArgs(1, false, builderType, localShadowerType, data.getTypeArgs());
LocalDeclaration varDefStat = new LocalDeclaration(data.getPluralName(), 0, 0);
varDefStat.type = localShadowerType;
statements.add(varDefStat);
statements.add(switchStat);
}
}
```

lombok/eclipse/handlers/singulars/EclipseJavaUtilMapSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.ForeachStatement;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
```

```
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import lombok.AccessLevel;
import lombok.core.LombokImmutableList;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
import lombok.eclipse.handlers.EclipseSingularsRecipes.StatementMaker;
import lombok.eclipse.handlers.EclipseSingularsRecipes.TypeReferenceMaker;
import lombok.eclipse.handlers.HandleNonNull;
import lombok.spi.Provides;
@Provides(EclipseSingularizer.class)
public class EclipseJavaUtilMapSingularizer extends EclipseJavaUtilSingularizer {
@Override public LombokImmutableList<String> getSupportedTypes() {
return LombokImmutableList.of("java.util.Map", "java.util.SortedMap", "java.util.NavigableMap");
}
private static final char[] EMPTY_SORTED_MAP = {'e', 'm', 'p', 't', 'y', 'S', 'o', 'r', 't', 'e', 'd', 'M', 'a', 'p'};
private static final char[] EMPTY_NAVIGABLE_MAP = {'e', 'm', 'p', 't', 'y', 'N', 'a', 'v', 'i', 'g', 'a', 'b', 'l', 'e',
'M', 'a', 'p'};
private static final char[] EMPTY_MAP = {'e', 'm', 'p', 't', 'y', 'M', 'a', 'p'};
@Override protected char[][] getEmptyMakerReceiver(String targetFqn) {
return JAVA_UTIL_COLLECTIONS;
}
@Override protected char[] getEmptyMakerSelector(String targetFqn) {
if (targetFqn.endsWith("SortedMap")) return EMPTY_SORTED_MAP;
if (targetFqn.endsWith("NavigableMap")) return EMPTY_NAVIGABLE_MAP;
return EMPTY_MAP;
}
@Override public List<char[]> listFieldsToBeGenerated(SingularData data, EclipseNode builderType) {
if (useGuavaInstead(builderType)) {
return guavaMapSingularizer.listFieldsToBeGenerated(data, builderType);
}
char[] p = data.getPluralName();
int len = p.length;
char[] k = new char[len + 4];
char[] v = new char[len + 6];
System.arraycopy(p, 0, k, 0, len);
System.arraycopy(p, 0, v, 0, len);
k[len] = '$';
k[len + 1] = 'k';
k[len + 2] = 'e';
k[len + 3] = 'y';
v[len] = '$';
```

```
v[len + 1] = 'v';
v[len + 2] = 'a';
v[len + 3] = 'l';
v[len + 4] = 'u';
v[len + 5] = 'e';
return Arrays.asList(k, v);
}
@Override public List<char[]> listMethodsToBeGenerated(SingularData data, EclipseNode
builderType) {
if (useGuavaInstead(builderType)) {
return guavaMapSingularizer.listFieldsToBeGenerated(data, builderType);
} else {
return super.listMethodsToBeGenerated(data, builderType);
}
}
@Override public List<EclipseNode> generateFields(SingularData data, EclipseNode builderType) {
if (useGuavaInstead(builderType)) {
return guavaMapSingularizer.generateFields(data, builderType);
}
char[] keyName = (new String(data.getPluralName()) + "$key").toCharArray();
char[] valueName = (new String(data.getPluralName()) + "$value").toCharArray();
FieldDeclaration buildKeyField; {
TypeReference type = new QualifiedTypeReference(JAVA_UTIL_ARRAYLIST, NULL_POSS);
type = addTypeArgs(1, false, builderType, type, data.getTypeArgs());
buildKeyField = new FieldDeclaration(keyName, 0, -1);
buildKeyField.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
buildKeyField.modifiers = ClassFileConstants.AccPrivate;
buildKeyField.declarationSourceEnd = -1;
buildKeyField.type = type;
}
FieldDeclaration buildValueField; {
TypeReference type = new QualifiedTypeReference(JAVA_UTIL_ARRAYLIST, NULL_POSS);
List<TypeReference> tArgs = data.getTypeArgs();
if (tArgs != null && tArgs.size() > 1) tArgs = Collections.singletonList(tArgs.get(1));
else tArgs = Collections.emptyList();
type = addTypeArgs(1, false, builderType, type, tArgs);
buildValueField = new FieldDeclaration(valueName, 0, -1);
buildValueField.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
buildValueField.modifiers = ClassFileConstants.AccPrivate;
buildValueField.declarationSourceEnd = -1;
buildValueField.type = type;
}
data.setGeneratedByRecursive(buildKeyField);
data.setGeneratedByRecursive(buildValueField);
EclipseNode keyFieldNode = injectFieldAndMarkGenerated(builderType, buildKeyField);
EclipseNode valueFieldNode = injectFieldAndMarkGenerated(builderType, buildValueField);
return Arrays.asList(keyFieldNode, valueFieldNode);
}
@Override public void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, EclipseNode builderType, boolean fluent, TypeReferenceMaker returnTypeMaker,
StatementMaker returnStatementMaker, AccessLevel access) {
if (useGuavaInstead(builderType)) {
```

```
guavaMapSingularizer.generateMethods(cfv, data, deprecate, builderType, fluent, returnTypeMaker,
returnStatementMaker, access);
return;
}
generateSingularMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(),
data, builderType, fluent, access);
generatePluralMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(), data,
builderType, fluent, access);
generateClearMethod(cfv, deprecate, returnTypeMaker.make(), returnStatementMaker.make(), data,
builderType, access);
}
private void generateClearMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, AccessLevel
access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
String pN = new String(data.getPluralName());
char[] keyFieldName = (pN + "$key").toCharArray();
char[] valueFieldName = (pN + "$value").toCharArray();
FieldReference thisDotField = new FieldReference(keyFieldName, 0L);
thisDotField.receiver = new ThisReference(0, 0);
FieldReference thisDotField2 = new FieldReference(keyFieldName, 0L);
thisDotField2.receiver = new ThisReference(0, 0);
FieldReference thisDotField3 = new FieldReference(valueFieldName, 0L);
thisDotField3.receiver = new ThisReference(0, 0);
md.selector = HandlerUtil.buildAccessorName(builderType, "clear", new
String(data.getPluralName())).toCharArray();
MessageSend clearMsg1 = new MessageSend();
clearMsg1.receiver = thisDotField2;
clearMsg1.selector = "clear".toCharArray();
MessageSend clearMsg2 = new MessageSend();
clearMsg2.receiver = thisDotField3;
clearMsg2.selector = "clear".toCharArray();
Block clearMsgs = new Block(2);
clearMsgs.statements = new Statement[] {clearMsg1, clearMsg2};
Statement clearStatement = new IfStatement(new EqualExpression(thisDotField, new NullLiteral(0, 0),
OperatorIds.NOT_EQUAL), clearMsgs, 0, 0);
md.statements = returnStatement != null ? new Statement[] {clearStatement, returnStatement} : new
Statement[] {clearStatement};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
md.annotations = generateSelfReturnAnnotations(deprecate, data.getSource());
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
data.setGeneratedByRecursive(md);
injectMethod(builderType, md);
}
private void generateSingularMethod(CheckerFrameworkVersion cfv, boolean deprecate,
TypeReference returnType, Statement returnStatement, SingularData data, EclipseNode builderType,
boolean fluent, AccessLevel access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
```

```
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
List<Statement> statements = new ArrayList<Statement>();
statements.add(createConstructBuilderVarIfNeeded(data, builderType, true));
String sN = new String(data.getSingularName());
String pN = new String(data.getPluralName());
char[] keyParamName = (sN + "Key").toCharArray();
char[] valueParamName = (sN + "Value").toCharArray();
char[] keyFieldName = (pN + "$key").toCharArray();
char[] valueFieldName = (pN + "$value").toCharArray();
/* this.pluralname$key.add(singularnameKey); */ {
FieldReference thisDotKeyField = new FieldReference(keyFieldName, 0L);
thisDotKeyField.receiver = new ThisReference(0, 0);
MessageSend thisDotKeyFieldDotAdd = new MessageSend();
thisDotKeyFieldDotAdd.arguments = new Expression[] {new SingleNameReference(keyParamName,
0L)};
thisDotKeyFieldDotAdd.receiver = thisDotKeyField;
thisDotKeyFieldDotAdd.selector = "add".toCharArray();
statements.add(thisDotKeyFieldDotAdd);
}
/* this.pluralname$value.add(singularnameValue); */ {
FieldReference thisDotValueField = new FieldReference(valueFieldName, 0L);
thisDotValueField.receiver = new ThisReference(0, 0);
MessageSend thisDotValueFieldDotAdd = new MessageSend();
thisDotValueFieldDotAdd.arguments = new Expression[] {new
SingleNameReference(valueParamName, 0L)};
thisDotValueFieldDotAdd.receiver = thisDotValueField;
thisDotValueFieldDotAdd.selector = "add".toCharArray();
statements.add(thisDotValueFieldDotAdd);
}
if (returnStatement != null) statements.add(returnStatement);
md.statements = statements.toArray(new Statement[0]);
TypeReference keyParamType = cloneParamType(0, data.getTypeArgs(), builderType);
TypeReference valueParamType = cloneParamType(1, data.getTypeArgs(), builderType);
Annotation[] typeUseAnnsKey = getTypeUseAnnotations(keyParamType);
Annotation[] typeUseAnnsValue = getTypeUseAnnotations(valueParamType);
removeTypeUseAnnotations(keyParamType);
removeTypeUseAnnotations(valueParamType);
Argument keyParam = new Argument(keyParamName, 0, keyParamType,
ClassFileConstants.AccFinal);
Argument valueParam = new Argument(valueParamName, 0, valueParamType,
ClassFileConstants.AccFinal);
keyParam.annotations = typeUseAnnsKey;
valueParam.annotations = typeUseAnnsValue;
md.arguments = new Argument[] {keyParam, valueParam};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
String name = new String(data.getSingularName());
String setterPrefix = data.getSetterPrefix().length > 0 ? new String(data.getSetterPrefix()) : fluent ? "" :
"put";
String setterName = HandlerUtil.buildAccessorName(builderType, setterPrefix, name);
md.selector = setterName.toCharArray();
Annotation[] selfReturnAnnotations = generateSelfReturnAnnotations(deprecate, data.getSource());
```

```
Annotation[] copyToSetterAnnotations = copyAnnotations(md,
findCopyableToBuilderSingularSetterAnnotations(data.getAnnotation().up()));
md.annotations = concat(selfReturnAnnotations, copyToSetterAnnotations, Annotation.class);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
data.setGeneratedByRecursive(md);
HandleNonNull.INSTANCE.fix(injectMethod(builderType, md));
}
private void generatePluralMethod(CheckerFrameworkVersion cfv, boolean deprecate, TypeReference
returnType, Statement returnStatement, SingularData data, EclipseNode builderType, boolean fluent,
AccessLevel access) {
MethodDeclaration md = new MethodDeclaration(((CompilationUnitDeclaration)
builderType.top().get()).compilationResult);
md.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
md.modifiers = toEclipseModifier(access);
String pN = new String(data.getPluralName());
char[] keyFieldName = (pN + "$key").toCharArray();
char[] valueFieldName = (pN + "$value").toCharArray();
List<Statement> statements = new ArrayList<Statement>();
statements.add(createConstructBuilderVarIfNeeded(data, builderType, true));
char[] entryName = "$lombokEntry".toCharArray();
TypeReference forEachType = new QualifiedTypeReference(JAVA_UTIL_MAP_ENTRY,
NULL_POSS);
forEachType = addTypeArgs(2, true, builderType, forEachType, data.getTypeArgs());
MessageSend keyArg = new MessageSend();
keyArg.receiver = new SingleNameReference(entryName, 0L);
keyArg.selector = "getKey".toCharArray();
MessageSend addKey = new MessageSend();
FieldReference thisDotKeyField = new FieldReference(keyFieldName, 0L);
thisDotKeyField.receiver = new ThisReference(0, 0);
addKey.receiver = thisDotKeyField;
addKey.selector = new char[] {'a', 'd', 'd'};
addKey.arguments = new Expression[] {keyArg};
MessageSend valueArg = new MessageSend();
valueArg.receiver = new SingleNameReference(entryName, 0L);
valueArg.selector = "getValue".toCharArray();
MessageSend addValue = new MessageSend();
FieldReference thisDotValueField = new FieldReference(valueFieldName, 0L);
thisDotValueField.receiver = new ThisReference(0, 0);
addValue.receiver = thisDotValueField;
addValue.selector = new char[] {'a', 'd', 'd'};
addValue.arguments = new Expression[] {valueArg};
LocalDeclaration elementVariable = new LocalDeclaration(entryName, 0, 0);
elementVariable.type = forEachType;
ForeachStatement forEach = new ForeachStatement(elementVariable, 0);
MessageSend invokeEntrySet = new MessageSend();
invokeEntrySet.selector = new char[] { 'e', 'n', 't', 'r', 'y', 'S', 'e', 't'};
invokeEntrySet.receiver = new SingleNameReference(data.getPluralName(), 0L);
forEach.collection = invokeEntrySet;
Block forEachContent = new Block(0);
forEachContent.statements = new Statement[] {addKey, addValue};
forEach.action = forEachContent;
statements.add(forEach);
TypeReference paramType = new QualifiedTypeReference(JAVA_UTIL_MAP, NULL_POSS);
```

```
paramType = addTypeArgs(2, true, builderType, paramType, data.getTypeArgs());
Argument param = new Argument(data.getPluralName(), 0, paramType, ClassFileConstants.AccFinal);
nullBehaviorize(builderType, data, statements, param, md);
if (returnStatement != null) statements.add(returnStatement);
md.statements = statements.toArray(new Statement[0]);
md.arguments = new Argument[] {param};
md.returnType = returnType;
addCheckerFrameworkReturnsReceiver(md.returnType, data.getSource(), cfv);
String name = new String(data.getPluralName());
String setterPrefix = data.getSetterPrefix().length > 0 ? new String(data.getSetterPrefix()) : fluent ? "" :
"put";
String setterName = HandlerUtil.buildAccessorName(builderType, setterPrefix, name);
md.selector = setterName.toCharArray();
Annotation[] selfReturnAnnotations = generateSelfReturnAnnotations(deprecate, data.getSource());
Annotation[] copyToSetterAnnotations = copyAnnotations(md,
findCopyableToSetterAnnotations(data.getAnnotation().up()));
md.annotations = concat(selfReturnAnnotations, copyToSetterAnnotations, Annotation.class);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, md);
data.setGeneratedByRecursive(md);
injectMethod(builderType, md);
}
@Override public void appendBuildCode(SingularData data, EclipseNode builderType,
List<Statement> statements, char[] targetVariableName, String builderVariable) {
if (useGuavaInstead(builderType)) {
guavaMapSingularizer.appendBuildCode(data, builderType, statements, targetVariableName,
builderVariable);
return;
}
if (data.getTargetFqn().equals("java.util.Map")) {
statements.addAll(createJavaUtilSetMapInitialCapacitySwitchStatements(data, builderType, true,
"emptyMap", "singletonMap", "LinkedHashMap", builderVariable));
} else {
statements.addAll(createJavaUtilSimpleCreationAndFillStatements(data, builderType, true, true, false,
true, "TreeMap", builderVariable));
}
}
@Override protected int getTypeArgumentsCount() {
return 2;
}
}
```

lombok/eclipse/handlers/singulars/EclipseJavaUtilSetSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
```

```
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import java.util.List;
import lombok.core.LombokImmutableList;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
import lombok.spi.Provides;
import org.eclipse.jdt.internal.compiler.ast.Statement;
@Provides(EclipseSingularizer.class)
public class EclipseJavaUtilSetSingularizer extends EclipseJavaUtilListSetSingularizer {
@Override public LombokImmutableList<String> getSupportedTypes() {
return LombokImmutableList.of("java.util.Set", "java.util.SortedSet", "java.util.NavigableSet");
}
private static final char[] EMPTY_SORTED_SET = {'e', 'm', 'p', 't', 'y', 'S', 'o', 'r', 't', 'e', 'd', 'S', 'e', 't'};
private static final char[] EMPTY_NAVIGABLE_SET = {'e', 'm', 'p', 't', 'y', 'N', 'a', 'v', 'i', 'g', 'a', 'b', 'l', 'e',
'S', 'e', 't'};
private static final char[] EMPTY_SET = {'e', 'm', 'p', 't', 'y', 'S', 'e', 't'};
@Override protected char[][] getEmptyMakerReceiver(String targetFqn) {
return JAVA_UTIL_COLLECTIONS;
}
@Override protected char[] getEmptyMakerSelector(String targetFqn) {
if (targetFqn.endsWith("SortedSet")) return EMPTY_SORTED_SET;
if (targetFqn.endsWith("NavigableSet")) return EMPTY_NAVIGABLE_SET;
return EMPTY_SET;
}
@Override public void appendBuildCode(SingularData data, EclipseNode builderType,
List<Statement> statements, char[] targetVariableName, String builderVariable) {
if (useGuavaInstead(builderType)) {
guavaListSetSingularizer.appendBuildCode(data, builderType, statements, targetVariableName,
builderVariable);
return;
}
if (data.getTargetFqn().equals("java.util.Set")) {
statements.addAll(createJavaUtilSetMapInitialCapacitySwitchStatements(data, builderType, false,
"emptySet", "singleton", "LinkedHashSet", builderVariable));
} else {
statements.addAll(createJavaUtilSimpleCreationAndFillStatements(data, builderType, false, true, false,
true, "TreeSet", builderVariable));
```

}
}
}

lombok/eclipse/handlers/singulars/EclipseJavaUtilSingularizer.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.handlers.singulars;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import org.eclipse.jdt.internal.compiler.ast.AllocationExpression;
import org.eclipse.jdt.internal.compiler.ast.Assignment;
import org.eclipse.jdt.internal.compiler.ast.BinaryExpression;
import org.eclipse.jdt.internal.compiler.ast.Block;
import org.eclipse.jdt.internal.compiler.ast.BreakStatement;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.EqualExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.ForStatement;
import org.eclipse.jdt.internal.compiler.ast.IfStatement;
import org.eclipse.jdt.internal.compiler.ast.IntLiteral;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.NullLiteral;
```

```
import org.eclipse.jdt.internal.compiler.ast.OperatorIds;
import org.eclipse.jdt.internal.compiler.ast.PostfixExpression;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.SwitchStatement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
import lombok.ConfigurationKeys;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.handlers.EclipseSingularsRecipes.EclipseSingularizer;
import lombok.eclipse.handlers.EclipseSingularsRecipes.SingularData;
abstract class EclipseJavaUtilSingularizer extends EclipseSingularizer {
protected static final char[][] JAVA_UTIL_ARRAYLIST = {
{'j', 'a', 'v', 'a'}, {'u', 't', 'i', 'l'}, {'A', 'r', 'r', 'a', 'y', 'L', 'i', 's', 't'}
};
protected static final char[][] JAVA_UTIL_LIST = {
{'j', 'a', 'v', 'a'}, {'u', 't', 'i', 'l'}, {'L', 'i', 's', 't'}
};
protected static final char[][] JAVA_UTIL_MAP = {
{'j', 'a', 'v', 'a'}, {'u', 't', 'i', 'l'}, {'M', 'a', 'p'}
};
protected static final char[][] JAVA_UTIL_MAP_ENTRY = {
{'j', 'a', 'v', 'a'}, {'u', 't', 'i', 'l'}, {'M', 'a', 'p'}, {'E', 'n', 't', 'r', 'y'}
};
protected static final char[][] JAVA_UTIL_COLLECTIONS = {
{'j', 'a', 'v', 'a'}, {'u', 't', 'i', 'l'}, {'C', 'o', 'l', 'l', 'e', 'c', 't', 'i', 'o', 'n', 's'}
};
protected final EclipseSingularizer guavaListSetSingularizer = new EclipseGuavaSetListSingularizer();
protected final EclipseSingularizer guavaMapSingularizer = new EclipseGuavaMapSingularizer();
protected boolean useGuavaInstead(EclipseNode node) {
return Boolean.TRUE.equals(node.getAst().readConfiguration(ConfigurationKeys.SINGULAR_USE_G
UAVA));
}
protected List<Statement> createJavaUtilSetMapInitialCapacitySwitchStatements(SingularData data,
EclipseNode builderType, boolean mapMode, String emptyCollectionMethod, String
singletonCollectionMethod, String targetType, String builderVariable) {
List<Statement> switchContents = new ArrayList<Statement>();
char[] keyName = mapMode ? (new String(data.getPluralName()) + "$key").toCharArray() :
data.getPluralName();
if (emptyCollectionMethod != null) { // case 0: (empty); break;
switchContents.add(Eclipse.createCaseStatement(makeIntLiteral(new char[] {'0'}, null)));
/* pluralName = java.util.Collections.emptyCollectionMethod(); */ {
MessageSend invoke = new MessageSend();
invoke.receiver = new QualifiedNameReference(JAVA_UTIL_COLLECTIONS, NULL_POSS, 0, 0);
invoke.selector = emptyCollectionMethod.toCharArray();
switchContents.add(new Assignment(new SingleNameReference(data.getPluralName(), 0), invoke,
0));
}
```

```
switchContents.add(new BreakStatement(null, 0, 0));
}
if (singletonCollectionMethod != null) { // case 1: (singleton); break;
switchContents.add(Eclipse.createCaseStatement(makeIntLiteral(new char[] {'1'}, null)));
/* !mapMode: pluralName = java.util.Collections.singletonCollectionMethod(this.pluralName.get(0));
mapMode: pluralName = java.util.Collections.singletonCollectionMethod(this.pluralName$key.get(0),
this.pluralName$value.get(0)); */ {
FieldReference thisDotKey = new FieldReference(keyName, 0L);
thisDotKey.receiver = getBuilderReference(builderVariable);
MessageSend thisDotKeyGet0 = new MessageSend();
thisDotKeyGet0.receiver = thisDotKey;
thisDotKeyGet0.selector = new char[] {'g', 'e', 't'};
thisDotKeyGet0.arguments = new Expression[] {makeIntLiteral(new char[] {'0'}, null)};
Expression[] args;
if (mapMode) {
char[] valueName = (new String(data.getPluralName()) + "$value").toCharArray();
FieldReference thisDotValue = new FieldReference(valueName, 0L);
thisDotValue.receiver = getBuilderReference(builderVariable);
MessageSend thisDotValueGet0 = new MessageSend();
thisDotValueGet0.receiver = thisDotValue;
thisDotValueGet0.selector = new char[] {'g', 'e', 't'};
thisDotValueGet0.arguments = new Expression[] {makeIntLiteral(new char[] {'0'}, null)};
args = new Expression[] {thisDotKeyGet0, thisDotValueGet0};
} else {
args = new Expression[] {thisDotKeyGet0};
}
MessageSend invoke = new MessageSend();
invoke.receiver = new QualifiedNameReference(JAVA_UTIL_COLLECTIONS, NULL_POSS, 0, 0);
invoke.selector = singletonCollectionMethod.toCharArray();
invoke.arguments = args;
switchContents.add(new Assignment(new SingleNameReference(data.getPluralName(), 0), invoke,
0));
}
switchContents.add(new BreakStatement(null, 0, 0));
}
{ // default:
switchContents.add(Eclipse.createCaseStatement(null));
switchContents.addAll(createJavaUtilSimpleCreationAndFillStatements(data, builderType, mapMode,
false, true, emptyCollectionMethod == null, targetType, builderVariable));
}
SwitchStatement switchStat = new SwitchStatement();
switchStat.statements = switchContents.toArray(new Statement[0]);
switchStat.expression = getSize(builderType, keyName, true, builderVariable);
TypeReference localShadowerType = new
QualifiedTypeReference(fromQualifiedName(data.getTargetFqn()), NULL_POSS);
localShadowerType = addTypeArgs(mapMode ? 2 : 1, false, builderType, localShadowerType,
data.getTypeArgs());
LocalDeclaration varDefStat = new LocalDeclaration(data.getPluralName(), 0, 0);
varDefStat.type = localShadowerType;
return Arrays.asList(varDefStat, switchStat);
}
protected List<Statement> createJavaUtilSimpleCreationAndFillStatements(SingularData data,
EclipseNode builderType, boolean mapMode, boolean defineVar, boolean addInitialCapacityArg,
```

```
boolean nullGuard, String targetType, String builderVariable) {
char[] varName = mapMode ? (new String(data.getPluralName()) + "$key").toCharArray() :
data.getPluralName();
Statement createStat; {
// pluralName = new java.util.TargetType(initialCap);
Expression[] constructorArgs = null;
if (addInitialCapacityArg) {
// this.varName.size() < MAX_POWER_OF_2 ? 1 + this.varName.size() + (this.varName.size() - 3) / 3 :
Integer.MAX_VALUE;
// lessThanCutOff = this.varName.size() < MAX_POWER_OF_2
Expression lessThanCutoff = new BinaryExpression(getSize(builderType, varName, nullGuard,
builderVariable), makeIntLiteral("0x40000000".toCharArray(), null), OperatorIds.LESS);
FieldReference integerMaxValue = new FieldReference("MAX_VALUE".toCharArray(), 0L);
integerMaxValue.receiver = new QualifiedNameReference(TypeConstants.JAVA_LANG_INTEGER,
NULL_POSS, 0, 0);
Expression sizeFormulaLeft = new BinaryExpression(makeIntLiteral(new char[] {'1'}, null),
getSize(builderType, varName, nullGuard, builderVariable), OperatorIds.PLUS);
Expression sizeFormulaRightLeft = new BinaryExpression(getSize(builderType, varName, nullGuard,
builderVariable), makeIntLiteral(new char[] {'3'}, null), OperatorIds.MINUS);
Expression sizeFormulaRight = new BinaryExpression(sizeFormulaRightLeft, makeIntLiteral(new char[]
{'3'}, null), OperatorIds.DIVIDE);
Expression sizeFormula = new BinaryExpression(sizeFormulaLeft, sizeFormulaRight,
OperatorIds.PLUS);
Expression cond = new ConditionalExpression(lessThanCutoff, sizeFormula, integerMaxValue);
constructorArgs = new Expression[] {cond};
}
TypeReference targetTypeRef = new QualifiedTypeReference(new char[][] {TypeConstants.JAVA,
TypeConstants.UTIL, targetType.toCharArray()}, NULL_POSS);
targetTypeRef = addTypeArgs(mapMode ? 2 : 1, false, builderType, targetTypeRef,
data.getTypeArgs());
AllocationExpression constructorCall = new AllocationExpression();
constructorCall.type = targetTypeRef;
constructorCall.arguments = constructorArgs;
if (defineVar) {
TypeReference localShadowerType = new
QualifiedTypeReference(fromQualifiedName(data.getTargetFqn()), NULL_POSS);
localShadowerType = addTypeArgs(mapMode ? 2 : 1, false, builderType, localShadowerType,
data.getTypeArgs());
LocalDeclaration localShadowerDecl = new LocalDeclaration(data.getPluralName(), 0, 0);
localShadowerDecl.type = localShadowerType;
localShadowerDecl.initialization = constructorCall;
createStat = localShadowerDecl;
} else {
createStat = new Assignment(new SingleNameReference(data.getPluralName(), 0L), constructorCall,
0);
}
}
Statement fillStat; {
if (mapMode) {
// for (int $i = 0; $i < this.pluralname$key.size(); i++) pluralname.put(this.pluralname$key.get($i),
this.pluralname$value.get($i));
char[] iVar = new char[] {'$', 'i'};
MessageSend pluralnameDotPut = new MessageSend();
```

```
pluralnameDotPut.selector = new char[] {'p', 'u', 't'};
pluralnameDotPut.receiver = new SingleNameReference(data.getPluralName(), 0L);
FieldReference thisDotKey = new FieldReference(varName, 0L);
thisDotKey.receiver = getBuilderReference(builderVariable);
FieldReference thisDotValue = new FieldReference((new String(data.getPluralName()) +
"$value").toCharArray(), 0L);
thisDotValue.receiver = getBuilderReference(builderVariable);
MessageSend keyArg = new MessageSend();
keyArg.receiver = thisDotKey;
keyArg.arguments = new Expression[] {new SingleNameReference(iVar, 0L)};
keyArg.selector = new char[] {'g', 'e', 't'};
MessageSend valueArg = new MessageSend();
valueArg.receiver = thisDotValue;
valueArg.arguments = new Expression[] {new SingleNameReference(iVar, 0L)};
valueArg.selector = new char[] {'g', 'e', 't'};
pluralnameDotPut.arguments = new Expression[] {keyArg, valueArg};
LocalDeclaration forInit = new LocalDeclaration(iVar, 0, 0);
forInit.type = TypeReference.baseTypeReference(TypeIds.T_int, 0);
forInit.initialization = makeIntLiteral(new char[] {'0'}, null);
Expression checkExpr = new BinaryExpression(new SingleNameReference(iVar, 0L),
getSize(builderType, varName, nullGuard, builderVariable), OperatorIds.LESS);
Expression incrementExpr = new PostfixExpression(new SingleNameReference(iVar, 0L),
IntLiteral.One, OperatorIds.PLUS, 0);
fillStat = new ForStatement(new Statement[] {forInit}, checkExpr, new Statement[] {incrementExpr},
pluralnameDotPut, true, 0, 0);
} else {
// pluralname.addAll(this.pluralname);
MessageSend pluralnameDotAddAll = new MessageSend();
pluralnameDotAddAll.selector = new char[] {'a', 'd', 'd', 'A', 'l', 'l'};
pluralnameDotAddAll.receiver = new SingleNameReference(data.getPluralName(), 0L);
FieldReference thisDotPluralname = new FieldReference(varName, 0L);
thisDotPluralname.receiver = getBuilderReference(builderVariable);
pluralnameDotAddAll.arguments = new Expression[] {thisDotPluralname};
fillStat = pluralnameDotAddAll;
}
if (nullGuard) {
FieldReference thisDotField = new FieldReference(varName, 0L);
thisDotField.receiver = getBuilderReference(builderVariable);
Expression cond = new EqualExpression(thisDotField, new NullLiteral(0, 0),
OperatorIds.NOT_EQUAL);
fillStat = new IfStatement(cond, fillStat, 0, 0);
}
}
Statement unmodifiableStat; {
// pluralname = Collections.unmodifiableInterfaceType(pluralname);
Expression arg = new SingleNameReference(data.getPluralName(), 0L);
MessageSend invoke = new MessageSend();
invoke.arguments = new Expression[] {arg};
invoke.selector = ("unmodifiable" + data.getTargetSimpleType()).toCharArray();
invoke.receiver = new QualifiedNameReference(JAVA_UTIL_COLLECTIONS, NULL_POSS, 0, 0);
unmodifiableStat = new Assignment(new SingleNameReference(data.getPluralName(), 0L), invoke, 0);
}
return Arrays.asList(createStat, fillStat, unmodifiableStat);
```

```
}
protected Statement createConstructBuilderVarIfNeeded(SingularData data, EclipseNode builderType,
boolean mapMode) {
char[] v1Name, v2Name;
if (mapMode) {
String n = new String(data.getPluralName());
v1Name = (n + "$key").toCharArray();
v2Name = (n + "$value").toCharArray();
} else {
v1Name = data.getPluralName();
v2Name = null;
}
FieldReference thisDotField = new FieldReference(v1Name, 0L);
thisDotField.receiver = new ThisReference(0, 0);
Expression cond = new EqualExpression(thisDotField, new NullLiteral(0, 0),
OperatorIds.EQUAL_EQUAL);
thisDotField = new FieldReference(v1Name, 0L);
thisDotField.receiver = new ThisReference(0, 0);
TypeReference v1Type = new QualifiedTypeReference(JAVA_UTIL_ARRAYLIST, NULL_POSS);
v1Type = addTypeArgs(1, false, builderType, v1Type, data.getTypeArgs());
AllocationExpression constructArrayList = new AllocationExpression();
constructArrayList.type = v1Type;
Assignment initV1 = new Assignment(thisDotField, constructArrayList, 0);
Statement thenPart;
if (mapMode) {
thisDotField = new FieldReference(v2Name, 0L);
thisDotField.receiver = new ThisReference(0, 0);
TypeReference v2Type = new QualifiedTypeReference(JAVA_UTIL_ARRAYLIST, NULL_POSS);
List<TypeReference> tArgs = data.getTypeArgs();
if (tArgs != null && tArgs.size() > 1) tArgs = Collections.singletonList(tArgs.get(1));
else tArgs = Collections.emptyList();
v2Type = addTypeArgs(1, false, builderType, v2Type, tArgs);
constructArrayList = new AllocationExpression();
constructArrayList.type = v2Type;
Assignment initV2 = new Assignment(thisDotField, constructArrayList, 0);
Block b = new Block(0);
b.statements = new Statement[] {initV1, initV2};
thenPart = b;
} else {
thenPart = initV1;
}
return new IfStatement(cond, thenPart, 0, 0);
}
}
```

lombok/eclipse/package-info.java

```
/*
* Copyright (C) 2009-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
```

* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* Includes the eclipse-specific implementations of the lombok AST and annotation introspection
support.
*
* NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.
*/
package lombok.eclipse;

lombok/experimental/Accessors.java

/*
* Copyright (C) 2012-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* A container for settings for the generation of getters, setters and "with"-ers.
* <p>
* Complete documentation is found at <a
href="https://projectlombok.org/features/experimental/Accessors">the project lombok features page for
&amp;#64;Accessors</a>.
* <p>
* Using this annotation does nothing by itself; an annotation that makes lombok generate getters,
setters, or "with"-ers
* such as {@link lombok.Setter} or {@link lombok.Data} is also required.
*/
@Target({ElementType.TYPE, ElementType.FIELD})
@Retention(RetentionPolicy.SOURCE)
public @interface Accessors {
/**
* If true, accessors will be named after the field and not include a {@code get} or {@code set}
* prefix. If true and {@code chain} is omitted, {@code chain} defaults to {@code true}.<br>
* NB: This setting has no effect on {@code @With}; they always get a "with" prefix.<br>
* <strong>default: false</strong>
*
* @return Whether or not to make fluent methods (named {@code fieldName()}, not for example
{@code setFieldName}).
*/
boolean fluent() default false;
/**
* If true, setters return {@code this} instead of {@code void}.
* <strong>default: false</strong>, unless {@code fluent=true}, then <strong>default: true</strong>
*
* @return Whether or not setters should return themselves (chaining) or {@code void} (no chaining).
*/
boolean chain() default false;
/**
* If true, generated accessors will be marked {@code final}.
* <strong>default: false</strong>
*
* @return Whether or not accessors should be marked {@code final}.
*/
boolean makeFinal() default false;
/**
* If present, only fields with any of the stated prefixes are given the getter/setter treatment.
* Note that a prefix only counts if the next character is NOT a lowercase character or the last
* letter of the prefix is not a letter (for instance an underscore). If multiple fields
* all turn into the same name when the prefix is stripped, an error will be generated.
*
* @return If you are in the habit of prefixing your fields (for example, you name them {@code
fFieldName}, specify such prefixes here).
*/
String[] prefix() default {};
}
```

lombok/experimental/Delegate.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Put on any field to make lombok generate delegate methods that forward the call to this field.
*
* Example:
* <pre>
* private &#64;Delegate List&lt;String&gt; foo;
* </pre>
*
* will generate for example an {@code boolean add(String)} method, which contains: {@code return
foo.add(arg);}, as well as all other methods in {@code List}.
*
* All public instance methods of the field's type, as well as all public instance methods of all the field's
type's superfields are delegated, except for all methods
* that exist in {@link Object}, the {@code canEqual(Object)} method, and any methods that appear in
types
* that are listed in the {@code excludes} property.
* <p>
* Complete documentation is found at <a
href="https://projectlombok.org/features/experimental/Delegate">the project lombok features page for
&#64;Delegate</a>.
*/
@Target({ElementType.FIELD, ElementType.METHOD})
@Retention(RetentionPolicy.SOURCE)
```

```
public @interface Delegate {
/**
* Normally the type of the field is used as delegate type. However, to choose a different type to delegate, you can list one (or more) types here. Note that types with
* type arguments can only be done as a field type. A solution for this is to create a private inner interface/class with the appropriate types extended, and possibly
* with all methods you'd like to delegate listed, and then supply that class here. The field does not actually have to implement the type you're delegating; the
* type listed here is used only to determine which delegate methods to generate.
*
* NB: All methods in {@code Object}, as well as {@code canEqual(Object other)} will never be delegated.
*
* @return For each method (not already in {@code java.lang.Object}) in these types, generate a delegate method.
*/
Class<?>[] types() default {};
/**
* Each method in any of the types listed here (include supertypes) will <em>not</em> be delegated.
*
* NB: All methods in {@code Object}, as well as {@code canEqual(Object other)} will never be delegated.
*
* @return For each method (not already in {@code java.lang.Object}) in these types, skip generating a delegate method (overrides {@code types()}).
*/
Class<?>[] excludes() default {};
}
```

lombok/experimental/ExtensionMethod.java


```
/*
* Copyright (C) 2012-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
```

```
*/
package lombok.experimental;
import static java.lang.annotation.ElementType.*;
import static java.lang.annotation.RetentionPolicy.*;
import java.lang.annotation.*;
/**
* Extension methods enable you to "add" methods to existing types without creating a new derived type, recompiling, or
* otherwise modifying the original type. Extension methods are a special kind of static method, but they are called as
* if they were instance methods on the extended type.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/experimental/ExtensionMethod">the project lombok features page for &#64;ExtensionMethod</a>.
* <br>
* <p>
* Before:
*
* <pre>
* &#064;ExtensionMethod(java.util.Arrays.class)
* class Example {
* private void example() {
* long[] values = new long[] { 2, 5, 7, 9 };
* values.copyOf(3).sort();
* }
* }
* </pre>
*
* After:
*
* <pre>
* class Example {
* private void example() {
* long[] values = new long[] { 2, 5, 7, 9 };
* java.util.Arrays.sort(java.util.Arrays.copyOf(values, 3));
* }
* }
* </pre>
*/
@Target(TYPE)
@Retention(SOURCE)
public @interface ExtensionMethod {
/** @return All types whose static methods will be exposed as extension methods. */
Class<?>[] value();
/**
* If {@code true}, an applicable extension method is used (if found) even if the method call already was compilable (this is the default).
* If {@code false}, an extension method is only used if the method call is not also defined by the type itself.
*
* @return Whether or not to override already existing methods with the extension.
*/
```

```
boolean suppressBaseMethods() default true;
}
```

lombok/experimental/FieldDefaults.java

```
/*
* Copyright (C) 2012-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import lombok.AccessLevel;
/**
* Adds modifiers to each field in the type with this annotation.
* <p>
* Complete documentation is found at <a
href="https://projectlombok.org/features/experimental/FieldDefaults">the project lombok features page
for &#64;FieldDefaults</a>.
* <p>
* If {@code makeFinal} is {@code true}, then each (instance) field that is not annotated with {@code
@NonFinal} will have the {@code final} modifier added.
* <p>
* If {@code level} is set, then each (instance) field that is package private (i.e. no access modifier) and
does not have the {@code @PackagePrivate} annotation will
* have the appropriate access level modifier added.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface FieldDefaults {
AccessLevel level() default AccessLevel.NONE;
boolean makeFinal() default false;
```

}

lombok/experimental/FieldNameConstants.java

```
/*
* Copyright (C) 2014-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import lombok.AccessLevel;
/**
* Generates an inner type, containing String constants containing the field name for each field.
Alternatively, generates an inner enum with enum values matching each field name.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface FieldNameConstants {
lombok.AccessLevel level() default AccessLevel.PUBLIC;
boolean asEnum() default false;
String innerTypeName() default "";
/**
* Only include fields and methods explicitly marked with {@code @FieldNameConstants.Include}.
* Normally, all (non-static) fields are included by default.
*
* @return If {@code true}, don't include non-static fields automatically (default: {@code false}).
*/
boolean onlyExplicitlyIncluded() default false;
/**
* If present, do not include this field in the generated fieldnames inner type.
*/
```

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.SOURCE)
public @interface Exclude {}
/**
* If present, include this field in the generated fieldnames inner type (default).
*/
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.SOURCE)
public @interface Include {}
}
```

lombok/experimental/Helper.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Use on a method local class to indicate that all methods inside should be exposed to the rest of
* the method as if they were helper methods.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface Helper {}
```

lombok/experimental/NonFinal.java

```
/*
```

* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Used to indicate the explicit intention for the annotated entity to <em>not</em> be {@code final}.
* Currently used by {@code FieldDefaults} and {@code Value} to avoid having it make a field final.
*/
@Target({ElementType.FIELD, ElementType.PARAMETER, ElementType.LOCAL_VARIABLE,
ElementType.ANNOTATION_TYPE, ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.SOURCE)
public @interface NonFinal {}
```

lombok/experimental/PackagePrivate.java

/*
* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Used to indicate the explicit intention for the annotated entity to have the <em>package private</em>
access level.
* Currently used by {@code FieldDefaults} and {@code Value} to avoid having it make a field one of
{@code public}, {@code protected}, or {@code private}.
*/
@Target({ElementType.TYPE, ElementType.FIELD, ElementType.CONSTRUCTOR,
ElementType.METHOD, ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface PackagePrivate {}
```

lombok/experimental/StandardException.java

```
/*
* Copyright (C) 2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import lombok.AccessLevel;
```

```
/**
* Put on any class that extends some {@code java.lang.Throwable} type to add the 4 common exception constructors.
*
* Specifically, all 4 constructors derived from the combinatorial explosion of {@code String message} and {@code Throwable cause}.
* You may write any or all of these 4 constructors by hand; lombok will only generate the missing ones.
* <p>
* All but the full {@code (String message, Throwable cause)} constructor are implemented as a {@code this(msg, cause)} call; it is therefore
* possibly to write code to run on construction by writing just the {@code (String message, Throwable cause)} constructor.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface StandardException {
/**
* Sets the access level of the generated constuctors. By default, generated constructors are {@code public}.
* Note: This does nothing if you write your own constructors (we won't change their access levels).
*
* @return The constructors will be generated with this access modifier.
*/
AccessLevel access() default lombok.AccessLevel.PUBLIC;
}
```

lombok/experimental/SuperBuilder.java

```
/*
* Copyright (C) 2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import static java.lang.annotation.ElementType.*;
```

```
import static java.lang.annotation.RetentionPolicy.*;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;
import lombok.Singular;
/**
* The SuperBuilder annotation creates a so-called 'builder' aspect to the class that is annotated with {@code @SuperBuilder}, but which works well when extending.
* It is similar to {@code @Builder}, except it is only legal on types, is less configurable, but allows you to {@code extends} other builder-able classes.
* <p>
* All classes in the hierarchy must be annotated with {@code @SuperBuilder}.
* <p>
* Lombok generates 2 inner 'builder' classes, which extend the parent class' builder class (unless your class doesn't have an extends clause).
* Lombok also generates a static method named {@code builder()}, and a protected constructor that takes 1 argument of the builderclass type.
* <p>
* The <code><strong>T</strong>Builder</code> class contains 1 method for each parameter of the annotated
* constructor / method (each field, when annotating a class), which returns the builder itself.
* The builder also has a <code>build()</code> method which returns a completed instance of the original type.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/experimental/SuperBuilder">the project lombok features page for &#64;SuperBuilder</a>.
*
* @see Singular
*/
@Target(TYPE)
@Retention(SOURCE)
public @interface SuperBuilder {
/** @return Name of the method that creates a new builder instance. Default: {@code builder}. If the empty string, suppress generating the {@code builder} method. */
String builderMethodName() default "builder";
/** @return Name of the method in the builder class that creates an instance of your {@code @Builder}-annotated class. */
String buildMethodName() default "build";
/**
* If <code>true</code>, generate an instance method to obtain a builder that is initialized with the values of this instance.
* In this case, all superclasses must also have <code>toBuilder=true</code>.
*
* @return Whether to generate a {@code toBuilder()} method.
*/
boolean toBuilder() default false;
/**
* Prefix to prepend to 'set' methods in the generated builder class. By default, generated methods do not include a prefix.
*
* For example, a method normally generated as {@code someField(String someField)} would instead be
```

```
* generated as {@code withSomeField(String someField)} if using {@code @SuperBuilder(setterPrefix
= "with")}.
*
* Note that using "with" to prefix builder setter methods is strongly discouraged as "with" normally
* suggests immutable data structures, and builders by definition are mutable objects.
*
* For {@code @Singular} fields, the generated methods are called {@code withName}, {@code
withNames}, and {@code clearNames}, instead of
* the default {@code name}, {@code names}, and {@code clearNames}.
*
* This prefix only applies to the 'set' methods for the fields of the annotated class.
* For consistency reasons, you should use the same prefix on all superclasses and subclasses that use
{@code @SuperBuilder}.
*
* @return The prefix to prepend to generated method names.
*/
String setterPrefix() default "";
}
```

lombok/experimental/Tolerate.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Put on any method or constructor to make lombok pretend it doesn't exist,
* i.e., to generate a method which would otherwise be skipped due to possible conflicts.
*/
```

```
@Target({ElementType.METHOD, ElementType.CONSTRUCTOR})
@Retention(RetentionPolicy.SOURCE)
public @interface Tolerate {
}
```

lombok/experimental/UtilityClass.java

```
/*
* Copyright (C) 2015-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* An annotation to create utility classes.
*
* If a class is annotated with {@code @UtilityClass}, the following things happen to it:<ul>
* <li>It is marked final.</li>
* <li>If any constructors are declared in it, an error is generated. Otherwise, a private no-args
constructor is generated; it throws a {@code UnsupportedOperationException}.</li>
* <li>All methods, inner classes, and fields in the class are marked static.</li>
* <li><em>WARNING:</em> Do not use non-star static imports to import these members; javac won't
be able to figure it out. Use either:
* <code>import static ThisType.*;</code> or don't static-import.</li>
* </ul>
*/
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface UtilityClass {
}
```

lombok/experimental/WithBy.java

```
/*
* Copyright (C) 2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import lombok.AccessLevel;
/**
* Put on any field to make lombok build a 'withBy' - a withFieldNameBy method which produces a clone
of this object (except for 1 field which gets a new value).
* <p>
* Complete documentation is found at <a
href="https://projectlombok.org/features/experimental/WithBy">the project lombok features page for
&#64;WithBy</a>.
* <p>
* Example:
* <pre>
* private &#64;WithBy final int foo;
* private &#64;WithBy final String bar;
* </pre>
*
* will generate:
*
* <pre>
* public SELF_TYPE withFooBy(@lombok.NonNull IntUnaryOperator operator) {
* int foo = operator.apply(this.foo);
* return this.foo == foo ? this : new SELF_TYPE(foo, bar);
* }
```

```
* public SELF_TYPE withBarBy(@lombok.NonNull Function&lt;? super String, ? extends String&gt;
operator) {
* String bar = operator.apply(this.bar);
* return this.bar == bar ? this : new SELF_TYPE(foo, bar);
* }
* </pre>
* <p>
* This annotation can also be applied to a class, in which case it'll be as if all non-static fields that don't
already have
* a {@code WithBy} annotation have the annotation.
* <p>
* This annotation is primarily useful for hierarchical immutable data structures. For example:
*
* <pre>
* class Movie {
* &#64;WithBy private final Director director;
* }
*
* class Director {
* &#64;WithBy private final LocalDate birthDate;
* }
* </pre>
*
* Using plain old {@code @With}, to increment a movie's director's birth date by one, you would write:
*
* <pre>
* movie =
movie.withDirector(movie.getDirector().withBirthDate(movie.getDirector().getBirthDate().plusDays(1)));
* </pre>
*
* but with {@code @WithBy}, you'd write:
*
* <pre>
* movie = movie.withDirectorBy(d -&gt; d.withBirthDateBy(bd -&gt; bd.plusDays(1)));
* </pre>
*/
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface WithBy {
/**
* If you want your with method to be non-public, you can specify an alternate access level here.
*
* @return The method will be generated with this access modifier.
*/
AccessLevel value() default AccessLevel.PUBLIC;
/**
* Any annotations listed here are put on the generated method.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @With(onMethod=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
```

```
* {@code @With(onMethod_={@AnnotationsGohere})} // note the underscore after {@code
onMethod}.
*
* @return List of annotations to apply to the generated method.
*/
AnyAnnotation[] onMethod() default {};
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/experimental/Wither.java

```
/*
* Copyright (C) 2012-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import lombok.AccessLevel;
/**
* Put on any field to make lombok build a 'wither' - a withX method which produces a clone of this object
(except for 1 field which gets a new value).
* <p>
* Complete documentation is found at <a
href="https://projectlombok.org/features/experimental/Wither">the project lombok features page for
```

```
&#64;Wither</a>.
* <p>
* Even though it is not listed, this annotation also has the {@code onParam} and {@code onMethod}
parameter. See the full documentation for more details.
* <p>
* Example:
* <pre>
* private &#64;Wither final int foo;
* </pre>
*
* will generate:
*
* <pre>
* public SELF_TYPE withFoo(int foo) {
* return this.foo == foo ? this : new SELF_TYPE(otherField1, otherField2, foo);
* }
* </pre>
* <p>
* This annotation can also be applied to a class, in which case it'll be as if all non-static fields that don't
already have
* a {@code Wither} annotation have the annotation.
*
* @deprecated {@link lombok.With} has been promoted to the main package, so use that one instead.
*/
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.SOURCE)
public @interface Wither {
/**
* If you want your wither to be non-public, you can specify an alternate access level here.
*
* @return The method will be generated with this access modifier.
*/
AccessLevel value() default AccessLevel.PUBLIC;
/**
* Any annotations listed here are put on the generated method.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @Wither(onMethod=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
* {@code @Wither(onMethod_={@AnnotationsGohere})} // note the underscore after {@code
onMethod}.
*
* @return List of annotations to apply to the generated method.
*/
AnyAnnotation[] onMethod() default {};
/**
* Any annotations listed here are put on the generated method's parameter.
* The syntax for this feature depends on JDK version (nothing we can do about that; it's to work around
javac bugs).<br>
* up to JDK7:<br>
* {@code @Wither(onParam=@__({@AnnotationsGoHere}))}<br>
* from JDK8:<br>
```

```
* {@code @Wither(onParam_={@AnnotationsGohere})} // note the underscore after {@code
onParam}.
*
* @return List of annotations to apply to the generated parameter in the method.
*/
AnyAnnotation[] onParam() default {};
/**
* Placeholder annotation to enable the placement of annotations on the generated code.
* @deprecated Don't use this annotation, ever - Read the documentation.
*/
@Deprecated
@Retention(RetentionPolicy.SOURCE)
@Target({})
@interface AnyAnnotation {}
}
```

lombok/experimental/package-info.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* This package contains the annotations and support classes you need as a user of lombok, for
* all features which aren't (yet) supported as a first class feature. Features that involve the
* annotations and support classes in this package may change or may be removed entirely in future
versions,
* and bugs may not be solved as expediently. For the status and likely future of any feature, refer
* to the official feature documentation.
*
* @see lombok
* @see <a href="https://projectlombok.org/features/experimental/all">Lombok features
(experimental)</a>
*/
```

package lombok.experimental;

lombok/experimental/var.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.experimental;
/**
* like val but not final
*
* @deprecated {@code var} has been promoted to the main package; use {@link lombok.var} instead.
*/
@Deprecated
public @interface var {
}
```

lombok/extern/apachecommons/CommonsLog.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
```

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.apachecommons;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project
lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;CommonsLog
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final org.apache.commons.logging.Log log =
org.apache.commons.logging.LogFactory.getLog(LogExample.class);
* }
* </pre>
*
* This annotation is valid for classes and enumerations.<br>
*
* @see <a href="https://commons.apache.org/proper/commons-logging/apidocs/org/apache/commons/
logging/Log.html">org.apache.commons.logging.Log</a>
* @see <a href="https://commons.apache.org/proper/commons-logging/apidocs/org/apache/commons/
logging/LogFactory.html#getLog(java.lang.Class)">org.apache.commons.logging.LogFactory#getLog(j
ava.lang.Class)</a>
* @see lombok.extern.java.Log &#64;Log
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.slf4j.Slf4j &#64;Slf4j
* @see lombok.extern.slf4j.XSlf4j &#64;XSlf4j
* @see lombok.extern.jbosslog.JBossLog &#64;JBossLog
* @see lombok.extern.flogger.Flogger &#64;Flogger
* @see lombok.CustomLog &#64;CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
```

```
public @interface CommonsLog {
/** @return The category of the constructed Logger. By default, it will use the type where the annotation is placed. */
String topic() default "";
}
```

lombok/extern/flogger/Flogger.java

```
/*
* Copyright (C) 2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.flogger;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;Flogger
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
```

```
* private static final com.google.common.flogger.FluentLogger log =
com.google.common.flogger.FluentLogger.forEnclosingClass();
* }
* </pre>
*
* This annotation is valid for classes and enumerations.<br>
* @see <a href="https://google.github.io/flogger/">com.google.common.flogger</a>
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.java.Log &#64;Log
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.slf4j.Slf4j &#64;Slf4j
* @see lombok.extern.slf4j.XSlf4j &#64;XSlf4j
* @see lombok.extern.jbosslog.JBossLog &#64;JBossLog
* @see lombok.CustomLog &#64;CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface Flogger {
}
```

lombok/extern/jackson/Jacksonized.java

```
/*
* Copyright (C) 2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.jackson;
import static java.lang.annotation.ElementType.*;
import static java.lang.annotation.RetentionPolicy.*;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;
import lombok.Builder;
import lombok.experimental.SuperBuilder;
```

```
/**
* The {@code @Jacksonized} annotation is an add-on annotation for
* {@code @}{@link Builder} and {@code @}{@link SuperBuilder}. It automatically
* configures the generated builder class to be used by Jackson's
* deserialization. It only has an effect if present at a context where there is
* also a {@code @Builder} or a {@code @SuperBuilder}; a warning is emitted
* otherwise.
* <p>
* In particular, the annotation does the following:
* <ul>
* <li>Configure Jackson to use the builder for deserialization using
* {@code @JsonDeserialize(builder=Foobar.FoobarBuilder[Impl].class)}
* on the class (where <em>Foobar</em> is the name of the annotated class).</li>
* <li>Copy Jackson-related configuration annotations (like
* {@code @JsonIgnoreProperties}) from the class to the builder class. This is
* necessary so that Jackson recognizes them when using the builder.</li>
* <li>Insert {@code @JsonPOJOBuilder(withPrefix="")} on the generated builder
* class to override Jackson's default prefix "with". If you configured a
* different prefix in lombok using {@code setterPrefix}, this value is used. If
* you changed the name of the {@code build()} method using
* {@code buildMethodName}, this is also made known to Jackson.</li>
* <li>For {@code @SuperBuilder}, make the builder implementation class
* package-private.</li>
* </ul>
* This annotation does <em>not</em> change the behavior of the generated builder.
* A {@code @Jacksonized} {@code @SuperBuilder} remains fully compatible to
* regular {@code @SuperBuilder}s.
*/
@Target({TYPE, METHOD, CONSTRUCTOR})
@Retention(SOURCE)
public @interface Jacksonized {
}
```

lombok/extern/java/Log.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.java;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project
lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;Log
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final java.util.logging.Logger log =
java.util.logging.Logger.getLogger(LogExample.class.getName());
* }
* </pre>
*
* This annotation is valid for classes and enumerations.<br>
* @see <a href="https://docs.oracle.com/javase/8/docs/api/java/util/logging/Logger.html">java.util.loggi
ng.Logger</a>
* @see <a href="https://docs.oracle.com/javase/8/docs/api/java/util/logging/Logger.html#getLogger-jav
a.lang.String-">java.util.logging.Logger#getLogger(java.lang.String)</a>
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.slf4j.Slf4j &#64;Slf4j
* @see lombok.extern.slf4j.XSlf4j &#64;XSlf4j
* @see lombok.extern.jbosslog.JBossLog &#64;JBossLog
* @see lombok.extern.flogger.Flogger &#64;Flogger
* @see lombok.CustomLog &#64;CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface Log {
/** @return The category of the constructed Logger. By default, it will use the type where the annotation
is placed. */
String topic() default "";
}
```

lombok/extern/jbosslog/JBossLog.java

```
/*
* Copyright (C) 2016-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.jbosslog;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project
lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;JBossLog
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final org.jboss.logging.Logger log =
org.jboss.logging.Logger.getLogger(LogExample.class);
* }
* </pre>
*
```

```
* This annotation is valid for classes and enumerations.<br>
* @see <a href="https://docs.jboss.org/jbosslogging/latest/org/jboss/logging/Logger.html">org.jboss.lo
gging.Logger</a>
* @see <a href="https://docs.jboss.org/jbosslogging/latest/org/jboss/logging/Logger.html#getLogger(ja
va.lang.Class)">org.jboss.logging.Logger#getLogger(java.lang.Class)</a>
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.java.Log &#64;Log
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.slf4j.Slf4j &#64;Slf4j
* @see lombok.extern.slf4j.XSlf4j &#64;XSlf4j
* @see lombok.extern.flogger.Flogger &#64;Flogger
* @see lombok.CustomLog &#64;CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface JBossLog {
/** @return The category of the constructed Logger. By default, it will use the type where the annotation
is placed. */
String topic() default "";
}
```

lombok/extern/log4j/Log4j.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.log4j;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
```

```
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;Log4j
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final org.apache.log4j.Logger log = org.apache.log4j.Logger.getLogger(LogExample.class);
* }
* </pre>
*
* This annotation is valid for classes and enumerations.<br>
*
* @see <a href="https://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/Logger.html">org.apache.log4j.Logger</a>
* @see <a href="https://logging.apache.org/log4j/1.2/apidocs/org/apache/log4j/Logger.html#getLogger(java.lang.Class)">org.apache.log4j.Logger#getLogger(java.lang.Class)</a>
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.java.Log &#64;Log
* @see lombok.extern.slf4j.Slf4j &#64;Slf4j
* @see lombok.extern.slf4j.XSlf4j &#64;XSlf4j
* @see lombok.extern.jbosslog.JBossLog &#64;JBossLog
* @see lombok.extern.flogger.Flogger &#64;Flogger
* @see lombok.CustomLog &#64;CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface Log4j {
/** @return The category of the constructed Logger. By default, it will use the type where the annotation is placed. */
String topic() default "";
}
```

lombok/extern/log4j/Log4j2.java

```
/*
* Copyright (C) 2013-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
```

```
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.log4j;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project
lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;Log4j2
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final org.apache.logging.log4j.Logger log =
org.apache.logging.log4j.LogManager.getLogger(LogExample.class);
* }
* </pre>
*
* This annotation is valid for classes and enumerations.<br>
*
* @see <a href="https://logging.apache.org/log4j/2.x/log4j-api/apidocs/org/apache/logging/log4j/Logger
.html">org.apache.logging.log4j.Logger</a>
* @see <a href="https://logging.apache.org/log4j/2.x/log4j-api/apidocs/org/apache/logging/log4j/LogMa
nager.html#getLogger-java.lang.Class-">org.apache.logging.log4j.LogManager#getLogger(java.lang.Cl
ass)</a>
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.java.Log &#64;Log
```

* @see lombok.extern.slf4j.Slf4j @Slf4j
* @see lombok.extern.slf4j.XSlf4j @XSlf4j
* @see lombok.extern.jbosslog.JBossLog @JBossLog
* @see lombok.extern.flogger.Flogger @Flogger
* @see lombok.CustomLog @CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface Log4j2 {
/** @return The category of the constructed Logger. By default, it will use the type where the annotation is placed. */
String topic() default "";
}

lombok/extern/slf4j/Slf4j.java

/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.slf4j;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at the project lombok features page for lombok log annotations.
* <p>
* Example:
* <pre>
* @Slf4j

```
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final org.slf4j.Logger log = org.slf4j.LoggerFactory.getLogger(LogExample.class);
* }
* </pre>
*
* This annotation is valid for classes and enumerations.<br>
* @see <a href="https://www.slf4j.org/api/org/slf4j/Logger.html">org.slf4j.Logger</a>
* @see <a href="https://www.slf4j.org/api/org/slf4j/LoggerFactory.html#getLogger(java.lang.Class)">or
g.slf4j.LoggerFactory#getLogger(java.lang.Class)</a>
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.java.Log &#64;Log
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.slf4j.XSlf4j &#64;XSlf4j
* @see lombok.extern.jbosslog.JBossLog &#64;JBossLog
* @see lombok.extern.flogger.Flogger &#64;Flogger
* @see lombok.CustomLog &#64;CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface Slf4j {
/** @return The category of the constructed Logger. By default, it will use the type where the annotation
is placed. */
String topic() default "";
}
```

lombok/extern/slf4j/XSlf4j.java

```
/*
* Copyright (C) 2012-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.extern.slf4j;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Causes lombok to generate a logger field.
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/log">the project
lombok features page for lombok log annotations</a>.
* <p>
* Example:
* <pre>
* &#64;XSlf4j
* public class LogExample {
* }
* </pre>
*
* will generate:
*
* <pre>
* public class LogExample {
* private static final org.slf4j.ext.XLogger log =
org.slf4j.ext.XLoggerFactory.getXLogger(LogExample.class);
* }
* </pre>
*
* This annotation is valid for classes and enumerations.<br>
* @see <a href="https://www.slf4j.org/api/org/slf4j/ext/XLogger.html">org.slf4j.ext.XLogger</a>
* @see <a href="https://www.slf4j.org/api/org/slf4j/ext/XLoggerFactory.html#getXLogger(java.lang.Clas
s)">org.slf4j.ext.XLoggerFactory#getLogger(java.lang.Class)</a>
* @see lombok.extern.apachecommons.CommonsLog &#64;CommonsLog
* @see lombok.extern.java.Log &#64;Log
* @see lombok.extern.log4j.Log4j &#64;Log4j
* @see lombok.extern.log4j.Log4j2 &#64;Log4j2
* @see lombok.extern.slf4j.Slf4j &#64;Slf4j
* @see lombok.extern.jbosslog.JBossLog &#64;JBossLog
* @see lombok.extern.flogger.Flogger &#64;Flogger
* @see lombok.CustomLog &#64;CustomLog
*/
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface XSlf4j {
/** @return The category of the constructed Logger. By default, it will use the type where the annotation
is placed. */
String topic() default "";
}
```

lombok/javac/CapturingDiagnosticListener.java

```
/*
* Copyright (C) 2012-2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.io.File;
import java.util.Collection;
import java.util.Iterator;
import java.util.Locale;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import javax.tools.Diagnostic;
import javax.tools.Diagnostic.Kind;
import javax.tools.DiagnosticListener;
import javax.tools.JavaFileObject;
/**
* This class stores any reported errors as {@code CompilerMessage} objects and supports removing
some of these.
* Currently this class is only used for testing purposes.
*/
public class CapturingDiagnosticListener implements DiagnosticListener<JavaFileObject> {
private final File file;
private final Collection<CompilerMessage> messages;
public CapturingDiagnosticListener(File file, Collection<CompilerMessage> messages) {
this.file = file;
this.messages = messages;
}
@Override public void report(Diagnostic<? extends JavaFileObject> d) {
String msg = d.getMessage(Locale.ENGLISH);
Matcher m = Pattern.compile(
"^" + Pattern.quote(file.getAbsolutePath()) +
"\\s*:\\s*\\d+\\s*:\\s*(?:warning:\\s*)?(.*)$", Pattern.DOTALL).matcher(msg);
```

```
if (m.matches()) msg = m.group(1);
if (msg.equals("deprecated item is not annotated with @Deprecated")) {
// This is new in JDK9; prior to that you don't see this. We shall ignore these.
return;
}
messages.add(new CompilerMessage(d.getLineNumber(), d.getStartPosition(), d.getKind() ==
Kind.ERROR, msg));
}
public void suppress(int start, int end) {
Iterator<CompilerMessage> it = messages.iterator();
while (it.hasNext()) {
long pos = it.next().getPosition();
if (pos >= start && pos < end) it.remove();
}
}
public static final class CompilerMessage {
/** Line Number (starting at 1) */
private final long line;
private final long position;
private final boolean isError;
private final String message;
public CompilerMessage(long line, long position, boolean isError, String message) {
this.line = line;
this.position = position;
this.isError = isError;
this.message = message;
}
public long getLine() {
return line;
}
public long getPosition() {
return position;
}
public boolean isError() {
return isError;
}
public String getMessage() {
return message;
}
@Override public int hashCode() {
final int prime = 31;
int result = 1;
result = prime * result + (isError ? 1231 : 1237);
result = prime * result + (int) (line ^ (line >>> 32));
result = prime * result + ((message == null) ? 0 : message.hashCode());
return result;
}
@Override public boolean equals(Object obj) {
if (this == obj) return true;
if (obj == null) return false;
if (getClass() != obj.getClass()) return false;
CompilerMessage other = (CompilerMessage) obj;
if (isError != other.isError) return false;
```

```
if (line != other.line) return false;
if (message == null) {
if (other.message != null) return false;
} else if (!message.equals(other.message)) return false;
return true;
}
@Override public String toString() {
return String.format("%d %s %s", line, isError ? "ERROR" : "WARNING", message);
}
}
}
```

lombok/javac/CompilerMessageSuppressor.java

```
/*
* Copyright (C) 2011-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.io.IOException;
import java.io.OutputStream;
import java.io.PrintWriter;
import java.lang.reflect.Field;
import java.util.LinkedList;
import java.util.Map;
import java.util.Queue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import javax.tools.DiagnosticListener;
import javax.tools.JavaFileObject;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.JCDiagnostic;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Log;
```

```
import lombok.permit.Permit;
/**
* During resolution, the resolver will emit resolution errors, but without appropriate file names and line numbers. If these resolution errors stick around
* then they will be generated AGAIN, this time with proper names and line numbers, at the end. Therefore, we want to suppress the logger.
*/
public final class CompilerMessageSuppressor {
private final Log log;
private static final WriterField errWriterField, warnWriterField, noticeWriterField;
private static final Field dumpOnErrorField, promptOnErrorField, diagnosticListenerField;
private static final Field deferDiagnosticsField, deferredDiagnosticsField, diagnosticHandlerField;
private static final ConcurrentMap<Class<?>, Field> handlerDeferredFields = new ConcurrentHashMap<Class<?>, Field>();
private static final Field NULL_FIELD;
private Boolean dumpOnError, promptOnError;
private DiagnosticListener<?> contextDiagnosticListener, logDiagnosticListener;
private final Context context;
private static final ThreadLocal<Queue<?>> queueCache = new ThreadLocal<Queue<?>>();
enum Writers {
ERROR("errWriter", "ERROR"),
WARNING("warnWriter", "WARNING"),
NOTICE("noticeWriter", "NOTICE");
final String fieldName;
final String keyName;
Writers(String fieldName, String keyName) {
this.fieldName = fieldName;
this.keyName = keyName;
}
}
static {
errWriterField = createWriterField(Writers.ERROR);
warnWriterField = createWriterField(Writers.WARNING);
noticeWriterField = createWriterField(Writers.NOTICE);
dumpOnErrorField = getDeclaredField(Log.class, "dumpOnError");
promptOnErrorField = getDeclaredField(Log.class, "promptOnError");
diagnosticListenerField = getDeclaredField(Log.class, "diagListener");
deferDiagnosticsField = getDeclaredField(Log.class, "deferDiagnostics");
deferredDiagnosticsField = getDeclaredField(Log.class, "deferredDiagnostics");
// javac8
diagnosticHandlerField = getDeclaredField(Log.class, "diagnosticHandler");
NULL_FIELD = getDeclaredField(JavacResolution.class, "NULL_FIELD");
}
static Field getDeclaredField(Class<?> c, String fieldName) {
try {
return Permit.getField(c, fieldName);
} catch (Throwable t) {
return null;
}
}
public CompilerMessageSuppressor(Context context) {
this.log = Log.instance(context);
this.context = context;
```

```
}
public void disableLoggers() {
contextDiagnosticListener = context.get(DiagnosticListener.class);
context.put(DiagnosticListener.class, (DiagnosticListener<?>) null);
errWriterField.pauze(log);
warnWriterField.pauze(log);
noticeWriterField.pauze(log);
if (deferDiagnosticsField != null) try {
if (Boolean.TRUE.equals(deferDiagnosticsField.get(log))) {
queueCache.set((Queue<?>) deferredDiagnosticsField.get(log));
Queue<?> empty = new LinkedList<Object>();
deferredDiagnosticsField.set(log, empty);
}
} catch (Exception e) {}
if (diagnosticHandlerField != null) try {
Object handler = diagnosticHandlerField.get(log);
Field field = getDeferredField(handler);
if (field != null) {
queueCache.set((Queue<?>) field.get(handler));
Queue<?> empty = new LinkedList<Object>();
field.set(handler, empty);
}
} catch (Exception e) {}
if (dumpOnErrorField != null) try {
dumpOnError = (Boolean) dumpOnErrorField.get(log);
dumpOnErrorField.set(log, false);
} catch (Exception e) {
}
if (promptOnErrorField != null) try {
promptOnError = (Boolean) promptOnErrorField.get(log);
promptOnErrorField.set(log, false);
} catch (Exception e) {
}
if (diagnosticListenerField != null) try {
logDiagnosticListener = (DiagnosticListener<?>) diagnosticListenerField.get(log);
diagnosticListenerField.set(log, null);
} catch (Exception e) {
}
}
private static Field getDeferredField(Object handler) {
Class<? extends Object> key = handler.getClass();
Field field = handlerDeferredFields.get(key);
if (field != null) {
return field == NULL_FIELD ? null : field;
}
Field value = getDeclaredField(key, "deferred");
handlerDeferredFields.put(key, value == null ? NULL_FIELD : value);
return getDeferredField(handler);
}
public void enableLoggers() {
if (contextDiagnosticListener != null) {
context.put(DiagnosticListener.class, contextDiagnosticListener);
contextDiagnosticListener = null;
```

```
}
errWriterField.resume(log);
warnWriterField.resume(log);
noticeWriterField.resume(log);
if (dumpOnError != null) try {
dumpOnErrorField.set(log, dumpOnError);
dumpOnError = null;
} catch (Exception e) {}
if (promptOnError != null) try {
promptOnErrorField.set(log, promptOnError);
promptOnError = null;
} catch (Exception e) {}
if (logDiagnosticListener != null) try {
diagnosticListenerField.set(log, logDiagnosticListener);
logDiagnosticListener = null;
} catch (Exception e) {}
if (diagnosticHandlerField != null && queueCache.get() != null) try {
Object handler = diagnosticHandlerField.get(log);
Field field = getDeferredField(handler);
if (field != null) {
field.set(handler, queueCache.get());
queueCache.set(null);
}
} catch (Exception e) {}
if (deferDiagnosticsField != null && queueCache.get() != null) try {
deferredDiagnosticsField.set(log, queueCache.get());
queueCache.set(null);
} catch (Exception e) {}
}
public void removeAllBetween(JavaFileObject sourcefile, int startPos, int endPos) {
DiagnosticListener<?> listener = context.get(DiagnosticListener.class);
if (listener instanceof CapturingDiagnosticListener) {
((CapturingDiagnosticListener) listener).suppress(startPos, endPos);
}
Field field = null;
Object receiver = null;
if (deferDiagnosticsField != null) try {
if (Boolean.TRUE.equals(deferDiagnosticsField.get(log))) {
field = deferredDiagnosticsField;
receiver = log;
}
} catch (Exception e) {}
if (diagnosticHandlerField != null) try {
Object handler = diagnosticHandlerField.get(log);
field = getDeferredField(handler);
receiver = handler;
} catch (Exception e) {}
if (field == null || receiver == null) return;
try {
ListBuffer<?> deferredDiagnostics = (ListBuffer<?>) field.get(receiver);
ListBuffer<Object> newDeferredDiagnostics = new ListBuffer<Object>();
for (Object diag_ : deferredDiagnostics) {
if (!(diag_ instanceof JCDiagnostic)) {
```

```
newDeferredDiagnostics.append(diag_);
continue;
}
JCDiagnostic diag = (JCDiagnostic) diag_;
long here = diag.getStartPosition();
if (here >= startPos && here < endPos && diag.getSource() == sourcefile) {
// We eliminate it
} else {
newDeferredDiagnostics.append(diag);
}
}
field.set(receiver, newDeferredDiagnostics);
} catch (Exception e) {
// We do not expect failure here; if failure does occur, the best course of action is to silently continue;
the result will be that the error output of
// javac will contain rather a lot of messages, but this is a lot better than just crashing during
compilation!
}
}
private static WriterField createWriterField(Writers w) {
// jdk9
try {
Field writers = getDeclaredField(Log.class, "writer");
if (writers != null) {
Class<?> kindsClass = Class.forName("com.sun.tools.javac.util.Log$WriterKind");
for (Object enumConstant : kindsClass.getEnumConstants()) {
if (enumConstant.toString().equals(w.keyName)) {
return new Java9WriterField(writers, enumConstant);
}
}
return WriterField.NONE;
}
} catch (Exception e) {
}
// jdk8
Field writerField = getDeclaredField(Log.class, w.fieldName);
if (writerField != null) return new Java8WriterField(writerField);
// other jdk
return WriterField.NONE;
}
interface WriterField {
final PrintWriter NO_WRITER = new PrintWriter(new OutputStream() {
@Override public void write(int b) throws IOException {
// Do nothing on purpose
}
});
final WriterField NONE = new WriterField() {
@Override public void pauze(Log log) {
// do nothing
}
@Override public void resume(Log log) {
// no nothing
}
```

```
};
void pauze(Log log);
void resume(Log log);
}
static class Java8WriterField implements WriterField {
private final Field field;
private PrintWriter writer;
public Java8WriterField(Field field) {
this.field = field;
}
@Override public void pauze(Log log) {
try {
writer = (PrintWriter) field.get(log);
field.set(log, NO_WRITER);
} catch (Exception e) {
}
}
@Override public void resume(Log log) {
if (writer != null) {
try {
field.set(log, writer);
} catch (Exception e) {
}
}
writer = null;
}
}
static class Java9WriterField implements WriterField {
private final Field field;
private final Object key;
private PrintWriter writer;
public Java9WriterField(Field field, Object key) {
this.field = field;
this.key = key;
}
@Override public void pauze(Log log) {
try {
@SuppressWarnings("unchecked") Map<Object,PrintWriter> map =
(Map<Object,PrintWriter>)field.get(log);
writer = map.get(key);
map.put(key, NO_WRITER);
} catch (Exception e) {
}
}
@Override public void resume(Log log) {
if (writer != null) {
try {
@SuppressWarnings("unchecked") Map<Object,PrintWriter> map =
(Map<Object,PrintWriter>)field.get(log);
map.put(key, writer);
} catch (Exception e) {
}
}
```

```
writer = null;
}
}
}
```

lombok/javac/FindTypeVarScanner.java

```
/*
* Copyright (C) 2010 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.util.HashSet;
import java.util.Set;
import javax.lang.model.element.Name;
import javax.lang.model.type.ArrayType;
import javax.lang.model.type.DeclaredType;
import javax.lang.model.type.ErrorType;
import javax.lang.model.type.ExecutableType;
import javax.lang.model.type.NoType;
import javax.lang.model.type.NullType;
import javax.lang.model.type.PrimitiveType;
import javax.lang.model.type.TypeMirror;
import javax.lang.model.type.TypeVariable;
import javax.lang.model.type.WildcardType;
import javax.lang.model.util.AbstractTypeVisitor6;
import com.sun.tools.javac.code.Type;
/**
* scanner (i.e. visits child nodes all the way to the leaves) that accumulates type variables. Call {@code visit} on any {@code TypeMirror} object with an instance
* to add all used type variable names such as {@code T} or {@code E} to the set that is returned by the {@link #getTypeVariables} method.
*/
public class FindTypeVarScanner extends AbstractTypeVisitor6<Void, Void> {
```

```
private Set<String> typeVariables = new HashSet<String>();
public Set<String> getTypeVariables() {
return typeVariables;
}
private Void subVisit(TypeMirror mirror) {
if (mirror == null) return null;
return mirror.accept(this, null);
}
@Override public Void visitPrimitive(PrimitiveType t, Void p) {
return null;
}
@Override public Void visitNull(NullType t, Void p) {
return null;
}
@Override public Void visitNoType(NoType t, Void p) {
return null;
}
@Override public Void visitUnknown(TypeMirror t, Void p) {
return null;
}
@Override public Void visitError(ErrorType t, Void p) {
return null;
}
@Override public Void visitArray(ArrayType t, Void p) {
return subVisit(t.getComponentType());
}
@Override public Void visitDeclared(DeclaredType t, Void p) {
for (TypeMirror subT : t.getTypeArguments()) subVisit(subT);
return null;
}
@Override public Void visitTypeVariable(TypeVariable t, Void p) {
Name name = null;
try {
name = ((Type) t).tsym.name;
} catch (NullPointerException e) {}
if (name != null) typeVariables.add(name.toString());
subVisit(t.getLowerBound());
subVisit(t.getUpperBound());
return null;
}
@Override public Void visitWildcard(WildcardType t, Void p) {
subVisit(t.getSuperBound());
subVisit(t.getExtendsBound());
return null;
}
@Override public Void visitExecutable(ExecutableType t, Void p) {
subVisit(t.getReturnType());
for (TypeMirror subT : t.getParameterTypes()) subVisit(subT);
for (TypeMirror subT : t.getThrownTypes()) subVisit(subT);
for (TypeVariable subT : t.getTypeVariables()) subVisit(subT);
return null;
}
}
```

lombok/javac/HandlerLibrary.java

```
/*
* Copyright (C) 2009-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import static lombok.javac.JavacAugments.JCTree_handled;
import java.io.IOException;
import java.lang.annotation.Annotation;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.SortedSet;
import java.util.TreeSet;
import javax.annotation.processing.Messager;
import javax.tools.Diagnostic;
import lombok.core.AlreadyHandledAnnotations;
import lombok.core.AnnotationValues.AnnotationValueDecodeFail;
import lombok.core.HandlerPriority;
import lombok.core.SpiLoadUtil;
import lombok.core.TypeLibrary;
import lombok.core.TypeResolver;
import lombok.core.configuration.ConfigurationKeysLoader;
import lombok.javac.handlers.JavacHandlerUtil;
import com.sun.source.util.Trees;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
/**
```

```
* This class tracks 'handlers' and knows how to invoke them for any given AST node.
*
* This class can find the handlers (via SPI discovery) and will set up the given AST node, such as
* building an AnnotationValues instance.
*/
public class HandlerLibrary {
private final TypeLibrary typeLibrary = new TypeLibrary();
private final Map<String, List<AnnotationHandlerContainer<?>>> annotationHandlers = new
HashMap<String, List<AnnotationHandlerContainer<?>>>();
private final Collection<VisitorContainer> visitorHandlers = new ArrayList<VisitorContainer>();
private final Messager messager;
/**
* Creates a new HandlerLibrary that will report any problems or errors to the provided messager.
* You probably want to use {@link #load(Messager, Trees)} instead.
*/
public HandlerLibrary(Messager messager) {
ConfigurationKeysLoader.LoaderLoader.loadAllConfigurationKeys();
this.messager = messager;
}
private static class VisitorContainer {
private final JavacASTVisitor visitor;
private final long priority;
private final boolean resolutionResetNeeded;
VisitorContainer(JavacASTVisitor visitor) {
this.visitor = visitor;
HandlerPriority hp = visitor.getClass().getAnnotation(HandlerPriority.class);
this.priority = hp == null ? 0L : (((long) hp.value()) << 32) + hp.subValue();
this.resolutionResetNeeded = visitor.getClass().isAnnotationPresent(ResolutionResetNeeded.class);
}
public long getPriority() {
return priority;
}
public boolean isResolutionResetNeeded() {
return resolutionResetNeeded;
}
}
private static class AnnotationHandlerContainer<T extends Annotation> {
private final JavacAnnotationHandler<T> handler;
private final Class<T> annotationClass;
private final long priority;
private final boolean resolutionResetNeeded;
private final boolean evenIfAlreadyHandled;
AnnotationHandlerContainer(JavacAnnotationHandler<T> handler, Class<T> annotationClass) {
this.handler = handler;
this.annotationClass = annotationClass;
HandlerPriority hp = handler.getClass().getAnnotation(HandlerPriority.class);
this.priority = hp == null ? 0L : (((long) hp.value()) << 32) + hp.subValue();
this.resolutionResetNeeded = handler.getClass().isAnnotationPresent(ResolutionResetNeeded.class);
this.evenIfAlreadyHandled =
handler.getClass().isAnnotationPresent(AlreadyHandledAnnotations.class);
}
public void handle(final JavacNode node) {
```

```
handler.handle(JavacHandlerUtil.createAnnotation(annotationClass, node), (JCAnnotation) node.get(),
node);
}
public long getPriority() {
return priority;
}
public boolean isResolutionResetNeeded() {
return resolutionResetNeeded;
}
public boolean isEvenIfAlreadyHandled() {
return evenIfAlreadyHandled;
}
}
private SortedSet<Long> priorities;
private SortedSet<Long> prioritiesRequiringResolutionReset;
public SortedSet<Long> getPriorities() {
return priorities;
}
public SortedSet<Long> getPrioritiesRequiringResolutionReset() {
return prioritiesRequiringResolutionReset;
}
private void calculatePriorities() {
SortedSet<Long> set = new TreeSet<Long>();
SortedSet<Long> resetNeeded = new TreeSet<Long>();
for (List<AnnotationHandlerContainer<?>> containers : annotationHandlers.values()) {
for (AnnotationHandlerContainer<?> container : containers) {
set.add(container.getPriority());
if (container.isResolutionResetNeeded()) resetNeeded.add(container.getPriority());
}
}
for (VisitorContainer container : visitorHandlers) {
set.add(container.getPriority());
if (container.isResolutionResetNeeded()) resetNeeded.add(container.getPriority());
}
this.priorities = Collections.unmodifiableSortedSet(set);
this.prioritiesRequiringResolutionReset = Collections.unmodifiableSortedSet(resetNeeded);
}
/**
* Creates a new HandlerLibrary that will report any problems or errors to the provided messager,
* then uses SPI discovery to load all annotation and visitor based handlers so that future calls
* to the handle methods will defer to these handlers.
*/
public static HandlerLibrary load(Messager messager, Trees trees) {
HandlerLibrary library = new HandlerLibrary(messager);
try {
loadAnnotationHandlers(library, trees);
loadVisitorHandlers(library, trees);
} catch (IOException e) {
System.err.println("Lombok isn't running due to misconfigured SPI files: " + e);
}
library.calculatePriorities();
return library;
}
```

```
/** Uses SPI Discovery to find implementations of {@link JavacAnnotationHandler}. */
@SuppressWarnings({"rawtypes", "unchecked"})
private static void loadAnnotationHandlers(HandlerLibrary lib, Trees trees) throws IOException {
//No, that seemingly superfluous reference to JavacAnnotationHandler's classloader is not in fact
superfluous!
for (JavacAnnotationHandler handler : SpiLoadUtil.findServices(JavacAnnotationHandler.class,
JavacAnnotationHandler.class.getClassLoader())) {
handler.setTrees(trees);
Class<? extends Annotation> annotationClass = handler.getAnnotationHandledByThisHandler();
AnnotationHandlerContainer<?> container = new AnnotationHandlerContainer(handler,
annotationClass);
String annotationClassName = container.annotationClass.getName().replace("$", ".");
List<AnnotationHandlerContainer<?>> list = lib.annotationHandlers.get(annotationClassName);
if (list == null) lib.annotationHandlers.put(annotationClassName, list = new
ArrayList<AnnotationHandlerContainer<?>>(1));
list.add(container);
lib.typeLibrary.addType(container.annotationClass.getName());
}
}
/** Uses SPI Discovery to find implementations of {@link JavacASTVisitor}. */
private static void loadVisitorHandlers(HandlerLibrary lib, Trees trees) throws IOException {
//No, that seemingly superfluous reference to JavacASTVisitor's classloader is not in fact superfluous!
for (JavacASTVisitor visitor : SpiLoadUtil.findServices(JavacASTVisitor.class,
JavacASTVisitor.class.getClassLoader())) {
visitor.setTrees(trees);
lib.visitorHandlers.add(new VisitorContainer(visitor));
}
}
/** Generates a warning in the Messager that was used to initialize this HandlerLibrary. */
public void javacWarning(String message) {
javacWarning(message, null);
}
/** Generates a warning in the Messager that was used to initialize this HandlerLibrary. */
public void javacWarning(String message, Throwable t) {
messager.printMessage(Diagnostic.Kind.WARNING, message + (t == null ? "" : (": " + t)));
}
/** Generates an error in the Messager that was used to initialize this HandlerLibrary. */
public void javacError(String message) {
javacError(message, null);
}
/** Generates an error in the Messager that was used to initialize this HandlerLibrary. */
public void javacError(String message, Throwable t) {
messager.printMessage(Diagnostic.Kind.ERROR, message + (t == null ? "" : (": " + t)));
if (t != null) t.printStackTrace();
}
private boolean checkAndSetHandled(JCTree node) {
return !JCTree_handled.getAndSet(node, true);
}
/**
* Handles the provided annotation node by first finding a qualifying instance of
* {@link JavacAnnotationHandler} and if one exists, calling it with a freshly cooked up
* instance of {@link lombok.core.AnnotationValues}.
*
```

```
* Note that depending on the printASTOnly flag, the {@link lombok.core.PrintAST} annotation
* will either be silently skipped, or everything that isn't {@code PrintAST} will be skipped.
*
* The HandlerLibrary will attempt to guess if the given annotation node represents a lombok annotation.
* For example, if {@code lombok.*} is in the import list, then this method will guess that
* {@code Getter} refers to {@code lombok.Getter}, presuming that {@link
lombok.javac.handlers.HandleGetter}
* has been loaded.
*
* @param unit The Compilation Unit that contains the Annotation AST Node.
* @param node The Lombok AST Node representing the Annotation AST Node.
* @param annotation 'node.get()' - convenience parameter.
*/
public void handleAnnotation(JCCompilationUnit unit, JavacNode node, JCAnnotation annotation, long
priority) {
TypeResolver resolver = new TypeResolver(node.getImportList());
String rawType = annotation.annotationType.toString();
String fqn = resolver.typeRefToFullyQualifiedName(node, typeLibrary, rawType);
if (fqn == null) return;
List<AnnotationHandlerContainer<?>> containers = annotationHandlers.get(fqn);
if (containers == null) return;
for (AnnotationHandlerContainer<?> container : containers) {
try {
if (container.getPriority() == priority) {
if (checkAndSetHandled(annotation)) {
container.handle(node);
} else {
if (container.isEvenIfAlreadyHandled()) container.handle(node);
}
}
} catch (AnnotationValueDecodeFail fail) {
fail.owner.setError(fail.getMessage(), fail.idx);
} catch (Throwable t) {
String sourceName = "(unknown).java";
if (unit != null && unit.sourcefile != null) sourceName = unit.sourcefile.getName();
javacError(String.format("Lombok annotation handler %s failed on " + sourceName,
container.handler.getClass()), t);
}
}
}
/**
* Will call all registered {@link JavacASTVisitor} instances.
*/
public void callASTVisitors(JavacAST ast, long priority) {
for (VisitorContainer container : visitorHandlers) try {
if (container.getPriority() == priority) ast.traverse(container.visitor);
} catch (Throwable t) {
javacError(String.format("Lombok visitor handler %s failed", container.visitor.getClass()), t);
}
}
}
```

lombok/javac/Javac6BasedLombokOptions.java

```
/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import lombok.Lombok;
import lombok.permit.Permit;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.Options;
public class Javac6BasedLombokOptions extends LombokOptions {
private static final Method optionName_valueOf;
private static final Method options_put;
static {
try {
Class<?> optionNameClass = Class.forName("com.sun.tools.javac.main.OptionName");
optionName_valueOf = Permit.getMethod(optionNameClass, "valueOf", String.class);
options_put = Permit.getMethod(Class.forName("com.sun.tools.javac.util.Options"), "put",
optionNameClass, String.class);
} catch (Exception e) {
throw new IllegalArgumentException("Can't initialize Javac6-based lombok options due to reflection
issue.", e);
}
}
public static Javac6BasedLombokOptions replaceWithDelombokOptions(Context context) {
Options options = Options.instance(context);
context.put(optionsKey, (Options)null);
Javac6BasedLombokOptions result = new Javac6BasedLombokOptions(context);
result.putAll(options);
return result;
}
```

```
private Javac6BasedLombokOptions(Context context) {
super(context);
}
@Override public void putJavacOption(String optionName, String value) {
try {
Permit.invoke(options_put, this, Permit.invoke(optionName_valueOf, null, optionName), value);
} catch (IllegalAccessException e) {
throw new IllegalArgumentException("Can't initialize Javac6-based lombok options due to reflection
issue.", e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
}
}
}
```

lombok/javac/Javac8BasedLombokOptions.java

```
/*
* Copyright (C) 2013-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import com.sun.tools.javac.main.Option;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.Options;
public class Javac8BasedLombokOptions extends LombokOptions {
public static Javac8BasedLombokOptions replaceWithDelombokOptions(Context context) {
Options options = Options.instance(context);
context.put(optionsKey, (Options) null);
Javac8BasedLombokOptions result = new Javac8BasedLombokOptions(context);
result.putAll(options);
return result;
}
private Javac8BasedLombokOptions(Context context) {
```

```
super(context);
}
@Override public void putJavacOption(String optionName, String value) {
String optionText = Option.valueOf(optionName).text;
put(optionText, value);
}
}
```

lombok/javac/Javac9BasedLombokOptions.java

```
/*
* Copyright (C) 2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import com.sun.tools.javac.main.Option;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.Options;
public class Javac9BasedLombokOptions extends LombokOptions {
public static Javac9BasedLombokOptions replaceWithDelombokOptions(Context context) {
Options options = Options.instance(context);
context.put(optionsKey, (Options) null);
Javac9BasedLombokOptions result = new Javac9BasedLombokOptions(context);
result.putAll(options);
return result;
}
private Javac9BasedLombokOptions(Context context) {
super(context);
}
@Override public void putJavacOption(String optionName, String value) {
if (optionName.equals("CLASSPATH")) optionName = "CLASS_PATH";
if (optionName.equals("SOURCEPATH")) optionName = "SOURCE_PATH";
if (optionName.equals("BOOTCLASSPATH")) optionName = "BOOT_CLASS_PATH";
String optionText = Option.valueOf(optionName).primaryName;
```

```
put(optionText, value);
}
}
```

lombok/javac/JavacAST.java

```
/*
* Copyright (C) 2009-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.io.File;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.net.URI;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import javax.annotation.processing.Messager;
import javax.tools.Diagnostic;
import javax.tools.JavaFileObject;
import lombok.core.AST;
import lombok.core.CleanupRegistry;
import lombok.core.CleanupTask;
import lombok.permit.Permit;
import com.sun.tools.javac.code.Source;
import com.sun.tools.javac.code.Symtab;
import com.sun.tools.javac.model.JavacElements;
import com.sun.tools.javac.model.JavacTypes;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
```

```
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCCatch;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCEnhancedForLoop;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTry;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.JCWildcard;
import com.sun.tools.javac.tree.TreeMaker;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.JCDiagnostic;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import com.sun.tools.javac.util.Log;
import com.sun.tools.javac.util.Name;
/**
* Wraps around javac's internal AST view to add useful features as well as the ability to visit parents
from children,
* something javac's own AST system does not offer.
*/
public class JavacAST extends AST<JavacAST, JavacNode, JCTree> {
private final CleanupRegistry cleanup;
private final JavacElements elements;
private final JavacTreeMaker treeMaker;
private final Symtab symtab;
private final JavacTypes javacTypes;
private final ErrorLog errorLogger;
private final Context context;
private static final URI NOT_CALCULATED_MARKER =
URI.create("https://projectlombok.org/not/calculated");
private URI memoizedAbsoluteFileLocation = NOT_CALCULATED_MARKER;
/**
* Creates a new JavacAST of the provided Compilation Unit.
*
* @param errorLog A logger for warning and error reporting.
* @param context A Context object for interfacing with the compiler.
* @param top The compilation unit, which serves as the top level node in the tree to be built.
* @param cleanup The registry for cleanup tasks.
*/
public JavacAST(ErrorLog errorLog, Context context, JCCompilationUnit top, CleanupRegistry
cleanup) {
super(sourceName(top), PackageName.getPackageName(top), new JavacImportList(top),
statementTypes());
setTop(buildCompilationUnit(top));
this.context = context;
this.errorLogger = errorLog;
this.elements = JavacElements.instance(context);
this.treeMaker = new JavacTreeMaker(TreeMaker.instance(context));
```

```
this.symtab = Symtab.instance(context);
this.javacTypes = JavacTypes.instance(context);
this.cleanup = cleanup;
clearChanged();
}
@Override public URI getAbsoluteFileLocation() {
if (memoizedAbsoluteFileLocation == NOT_CALCULATED_MARKER) {
memoizedAbsoluteFileLocation = getAbsoluteFileLocation((JCCompilationUnit) top().get());
}
return memoizedAbsoluteFileLocation;
}
private static Class<?> wrappedFileObjectClass, sbtJavaFileObjectClass, sbtMappedVirtualFileClass,
sbtOptionClass;
private static Field wrappedFileObjectField, sbtJavaFileObjectField, sbtMappedVirtualFilePathField,
sbtMappedVirtualFileRootsField, sbtOptionField;
private static Method sbtMapGetMethod;
public static URI getAbsoluteFileLocation(JCCompilationUnit cu) {
try {
URI uri = cu.sourcefile.toUri();
String fn = uri.toString();
if (fn.startsWith("file:")) return uri;
URI sbtUri = tryGetSbtFile(cu.sourcefile);
if (sbtUri != null) return sbtUri;
return uri;
} catch (Exception e) {
return null;
}
}
private static URI tryGetSbtFile(JavaFileObject sourcefile) {
try {
return tryGetSbtFile_(sourcefile);
} catch (Exception e) {
return null;
}
}
private static URI tryGetSbtFile_(JavaFileObject sourcefile) throws Exception {
Class<?> c = sourcefile.getClass();
String cn;
if (wrappedFileObjectClass == null) {
if (!c.getName().equals("com.sun.tools.javac.api.ClientCodeWrapper$WrappedJavaFileObject")) return
null;
wrappedFileObjectClass = c;
}
if (c != wrappedFileObjectClass) return null;
if (wrappedFileObjectField == null) wrappedFileObjectField =
Permit.permissiveGetField(wrappedFileObjectClass.getSuperclass(), "clientFileObject");
if (wrappedFileObjectField == null) return null;
Object fileObject = wrappedFileObjectField.get(sourcefile);
c = fileObject.getClass();
if (sbtJavaFileObjectClass == null) {
cn = c.getName();
if (!cn.startsWith("sbt.") || !cn.endsWith("JavaFileObject")) return null;
sbtJavaFileObjectClass = c;
```

```
}
if (sbtJavaFileObjectClass != c) return null;
if (sbtJavaFileObjectField == null) sbtJavaFileObjectField =
Permit.permissiveGetField(sbtJavaFileObjectClass, "underlying");
if (sbtJavaFileObjectField == null) return null;
Object mappedVirtualFile = sbtJavaFileObjectField.get(fileObject);
c = mappedVirtualFile.getClass();
if (sbtMappedVirtualFileClass == null) {
cn = c.getName();
if (!cn.startsWith("sbt.") || !cn.endsWith("MappedVirtualFile")) return null;
sbtMappedVirtualFileClass = c;
}
if (sbtMappedVirtualFilePathField == null) sbtMappedVirtualFilePathField =
Permit.permissiveGetField(sbtMappedVirtualFileClass, "encodedPath");
if (sbtMappedVirtualFilePathField == null) return null;
if (sbtMappedVirtualFileRootsField == null) sbtMappedVirtualFileRootsField =
Permit.permissiveGetField(sbtMappedVirtualFileClass, "rootPathsMap");
if (sbtMappedVirtualFileRootsField == null) return null;
String encodedPath = (String) sbtMappedVirtualFilePathField.get(mappedVirtualFile);
if (!encodedPath.startsWith("${")) {
File maybeAbsoluteFile = new File(encodedPath);
if (maybeAbsoluteFile.exists()) {
return maybeAbsoluteFile.toURI();
} else {
return null;
}
}
int idx = encodedPath.indexOf('}');
if (idx == -1) return null;
String base = encodedPath.substring(2, idx);
Object roots = sbtMappedVirtualFileRootsField.get(mappedVirtualFile);
if (sbtMapGetMethod == null) sbtMapGetMethod = Permit.getMethod(roots.getClass(), "get",
Object.class);
if (sbtMapGetMethod == null) return null;
Object option = sbtMapGetMethod.invoke(roots, base);
c = option.getClass();
if (sbtOptionClass == null) {
if (c.getName().equals("scala.Some")) sbtOptionClass = c;
}
if (c != sbtOptionClass) return null;
if (sbtOptionField == null) sbtOptionField = Permit.permissiveGetField(sbtOptionClass, "value");
if (sbtOptionField == null) return null;
Object path = sbtOptionField.get(option);
return new File(path.toString() + encodedPath.substring(idx + 1)).toURI();
}
private static String sourceName(JCCompilationUnit cu) {
return cu.sourcefile == null ? null : cu.sourcefile.toString();
}
public Context getContext() {
return context;
}
/**
```

```
* Runs through the entire AST, starting at the compilation unit, calling the provided visitor's visit
methods
* for each node, depth first.
*/
public void traverse(JavacASTVisitor visitor) {
top().traverse(visitor);
}
void traverseChildren(JavacASTVisitor visitor, JavacNode node) {
for (JavacNode child : node.down()) child.traverse(visitor);
}
@Override public int getSourceVersion() {
try {
String nm = Source.instance(context).name();
int underscoreIdx = nm.indexOf('_');
if (underscoreIdx > -1) return Integer.parseInt(nm.substring(underscoreIdx + 1));
// assume java9+
return Integer.parseInt(nm.substring(3));
} catch (Exception ignore) {}
return 6;
}
@Override public int getLatestJavaSpecSupported() {
return Javac.getJavaCompilerVersion();
}
public void cleanupTask(String key, JCTree target, CleanupTask task) {
cleanup.registerTask(key, target, task);
}
/** @return A Name object generated for the proper name table belonging to this AST. */
public Name toName(String name) {
return elements.getName(name);
}
/** @return A TreeMaker instance that you can use to create new AST nodes. */
public JavacTreeMaker getTreeMaker() {
treeMaker.at(-1);
return treeMaker;
}
/** @return The symbol table used by this AST for symbols. */
public Symtab getSymbolTable() {
return symtab;
}
/**
* @return The implementation of {@link javax.lang.model.util.Types} of javac. Contains a few extra
methods beyond
* the ones listed in the official annotation API interface. */
public JavacTypes getTypesUtil() {
return javacTypes;
}
/** {@inheritDoc} */
@Override protected JavacNode buildTree(JCTree node, Kind kind) {
switch (kind) {
case COMPILATION_UNIT:
return buildCompilationUnit((JCCompilationUnit) node);
case TYPE:
return buildType((JCClassDecl) node);
```

```
case FIELD:
return buildField((JCVariableDecl) node);
case INITIALIZER:
return buildInitializer((JCBlock) node);
case METHOD:
return buildMethod((JCMethodDecl) node);
case ARGUMENT:
return buildLocalVar((JCVariableDecl) node, kind);
case LOCAL:
return buildLocalVar((JCVariableDecl) node, kind);
case STATEMENT:
return buildStatementOrExpression(node);
case ANNOTATION:
return buildAnnotation((JCAnnotation) node, false);
case TYPE_USE:
return buildTypeUse(node);
default:
throw new AssertionError("Did not expect: " + kind);
}
}
private JavacNode buildCompilationUnit(JCCompilationUnit top) {
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (JCTree s : top.defs) {
if (s instanceof JCClassDecl) {
addIfNotNull(childNodes, buildType((JCClassDecl) s));
} // else they are import statements, which we don't care about. Or Skip objects, whatever those are.
}
return new JavacNode(this, top, childNodes, Kind.COMPILATION_UNIT);
}
private JavacNode buildType(JCClassDecl type) {
if (setAndGetAsHandled(type)) return null;
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (JCAnnotation annotation : type.mods.annotations) addIfNotNull(childNodes,
buildAnnotation(annotation, false));
for (JCTree def : type.defs) {
/* A def can be:
* JCClassDecl for inner types
* JCMethodDecl for constructors and methods
* JCVariableDecl for fields
* JCBlock for (static) initializers
*/
if (def instanceof JCMethodDecl) addIfNotNull(childNodes, buildMethod((JCMethodDecl) def));
else if (def instanceof JCClassDecl) addIfNotNull(childNodes, buildType((JCClassDecl) def));
else if (def instanceof JCVariableDecl) addIfNotNull(childNodes, buildField((JCVariableDecl) def));
else if (def instanceof JCBlock) addIfNotNull(childNodes, buildInitializer((JCBlock) def));
}
return putInMap(new JavacNode(this, type, childNodes, Kind.TYPE));
}
private JavacNode buildField(JCVariableDecl field) {
if (setAndGetAsHandled(field)) return null;
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (JCAnnotation annotation : field.mods.annotations) addIfNotNull(childNodes,
buildAnnotation(annotation, true));
```

```
addIfNotNull(childNodes, buildTypeUse(field.vartype));
addIfNotNull(childNodes, buildExpression(field.init));
return putInMap(new JavacNode(this, field, childNodes, Kind.FIELD));
}
private JavacNode buildLocalVar(JCVariableDecl local, Kind kind) {
if (setAndGetAsHandled(local)) return null;
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (JCAnnotation annotation : local.mods.annotations) addIfNotNull(childNodes,
buildAnnotation(annotation, true));
addIfNotNull(childNodes, buildTypeUse(local.vartype));
addIfNotNull(childNodes, buildExpression(local.init));
return putInMap(new JavacNode(this, local, childNodes, kind));
}
private JavacNode buildTypeUse(JCTree typeUse) {
if (setAndGetAsHandled(typeUse)) return null;
if (typeUse == null) return null;
if (typeUse.getClass().getName().equals("com.sun.tools.javac.tree.JCTree$JCAnnotatedType")) {
initJcAnnotatedType(typeUse.getClass());
Collection<?> anns = Permit.permissiveReadField(Collection.class,
JCANNOTATEDTYPE_ANNOTATIONS, typeUse);
JCExpression underlying = Permit.permissiveReadField(JCExpression.class,
JCANNOTATEDTYPE_UNDERLYINGTYPE, typeUse);
List<JavacNode> childNodes = new ArrayList<JavacNode>();
if (anns != null) for (Object annotation : anns) if (annotation instanceof JCAnnotation)
addIfNotNull(childNodes, buildAnnotation((JCAnnotation) annotation, true));
addIfNotNull(childNodes, buildTypeUse(underlying));
return putInMap(new JavacNode(this, typeUse, childNodes, Kind.TYPE_USE));
}
if (typeUse instanceof JCWildcard) {
JCTree inner = ((JCWildcard) typeUse).inner;
List<JavacNode> childNodes = inner == null ? Collections.<JavacNode>emptyList() : new
ArrayList<JavacNode>();
if (inner != null) addIfNotNull(childNodes, buildTypeUse(inner));
return putInMap(new JavacNode(this, typeUse, childNodes, Kind.TYPE_USE));
}
if (typeUse instanceof JCArrayTypeTree) {
JCTree inner = ((JCArrayTypeTree) typeUse).elemtype;
List<JavacNode> childNodes = inner == null ? Collections.<JavacNode>emptyList() : new
ArrayList<JavacNode>();
if (inner != null) addIfNotNull(childNodes, buildTypeUse(inner));
return putInMap(new JavacNode(this, typeUse, childNodes, Kind.TYPE_USE));
}
if (typeUse instanceof JCFieldAccess) {
JCTree inner = ((JCFieldAccess) typeUse).selected;
List<JavacNode> childNodes = inner == null ? Collections.<JavacNode>emptyList() : new
ArrayList<JavacNode>();
if (inner != null) addIfNotNull(childNodes, buildTypeUse(inner));
return putInMap(new JavacNode(this, typeUse, childNodes, Kind.TYPE_USE));
}
if (typeUse instanceof JCIdent) {
return putInMap(new JavacNode(this, typeUse, Collections.<JavacNode>emptyList(),
Kind.TYPE_USE));
}
```

```
return null;
}
private static boolean JCTRY_RESOURCES_FIELD_INITIALIZED = false;
private static Field JCTRY_RESOURCES_FIELD;
@SuppressWarnings("unchecked")
private static List<JCTree> getResourcesForTryNode(JCTry tryNode) {
if (!JCTRY_RESOURCES_FIELD_INITIALIZED) {
JCTRY_RESOURCES_FIELD = Permit.permissiveGetField(JCTry.class, "resources");
JCTRY_RESOURCES_FIELD_INITIALIZED = true;
}
if (JCTRY_RESOURCES_FIELD == null) return Collections.emptyList();
Object rv = null;
try {
rv = JCTRY_RESOURCES_FIELD.get(tryNode);
} catch (Exception ignore) {}
if (rv instanceof List) return (List<JCTree>) rv;
return Collections.emptyList();
}
private static boolean JCANNOTATEDTYPE_FIELDS_INITIALIZED = false;
private static Field JCANNOTATEDTYPE_ANNOTATIONS,
JCANNOTATEDTYPE_UNDERLYINGTYPE;
private static void initJcAnnotatedType(Class<?> context) {
if (JCANNOTATEDTYPE_FIELDS_INITIALIZED) return;
JCANNOTATEDTYPE_ANNOTATIONS = Permit.permissiveGetField(context, "annotations");
JCANNOTATEDTYPE_UNDERLYINGTYPE = Permit.permissiveGetField(context, "underlyingType");
JCANNOTATEDTYPE_FIELDS_INITIALIZED = true;
}
private static Field JCENHANCEDFORLOOP_VARORRECORDPATTERN_FIELD =
Permit.permissiveGetField(JCEnhancedForLoop.class, "varOrRecordPattern");
private static JCTree getVarOrRecordPattern(JCEnhancedForLoop loop) {
if (JCENHANCEDFORLOOP_VARORRECORDPATTERN_FIELD == null) {
return loop.var;
}
try {
return (JCTree) JCENHANCEDFORLOOP_VARORRECORDPATTERN_FIELD.get(loop);
} catch (Exception ignore) {}
return null;
}
private JavacNode buildTry(JCTry tryNode) {
if (setAndGetAsHandled(tryNode)) return null;
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (JCTree varDecl : getResourcesForTryNode(tryNode)) {
if (varDecl instanceof JCVariableDecl) {
addIfNotNull(childNodes, buildLocalVar((JCVariableDecl) varDecl, Kind.LOCAL));
}
}
addIfNotNull(childNodes, buildStatement(tryNode.body));
for (JCCatch jcc : tryNode.catchers) addIfNotNull(childNodes, buildTree(jcc, Kind.STATEMENT));
addIfNotNull(childNodes, buildStatement(tryNode.finalizer));
return putInMap(new JavacNode(this, tryNode, childNodes, Kind.STATEMENT));
}
private JavacNode buildInitializer(JCBlock initializer) {
if (setAndGetAsHandled(initializer)) return null;
```

```
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (JCStatement statement: initializer.stats) addIfNotNull(childNodes, buildStatement(statement));
return putInMap(new JavacNode(this, initializer, childNodes, Kind.INITIALIZER));
}
private JavacNode buildMethod(JCMethodDecl method) {
if (setAndGetAsHandled(method)) return null;
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (JCAnnotation annotation : method.mods.annotations) addIfNotNull(childNodes,
buildAnnotation(annotation, false));
for (JCVariableDecl param : method.params) addIfNotNull(childNodes, buildLocalVar(param,
Kind.ARGUMENT));
if (method.body != null && method.body.stats != null) {
for (JCStatement statement : method.body.stats) addIfNotNull(childNodes, buildStatement(statement));
}
return putInMap(new JavacNode(this, method, childNodes, Kind.METHOD));
}
private JavacNode buildAnnotation(JCAnnotation annotation, boolean varDecl) {
boolean handled = setAndGetAsHandled(annotation);
if (!varDecl && handled) {
// @Foo int x, y; is handled in javac by putting the same annotation node on 2 JCVariableDecls.
return null;
}
return putInMap(new JavacNode(this, annotation, null, Kind.ANNOTATION));
}
private JavacNode buildExpression(JCExpression expression) {
return buildStatementOrExpression(expression);
}
private JavacNode buildStatement(JCStatement statement) {
return buildStatementOrExpression(statement);
}
private JavacNode buildStatementOrExpression(JCTree statement) {
if (statement == null) return null;
if (statement instanceof JCAnnotation) return null;
if (statement instanceof JCClassDecl) return buildType((JCClassDecl) statement);
if (statement instanceof JCVariableDecl) return buildLocalVar((JCVariableDecl) statement,
Kind.LOCAL);
if (statement instanceof JCTry) return buildTry((JCTry) statement);
if (statement.getClass().getName().equals("com.sun.tools.javac.tree.JCTree$JCLambda")) return
buildLambda(statement);
if (statement instanceof JCEnhancedForLoop) return buildEnhancedForLoop((JCEnhancedForLoop)
statement);
if (setAndGetAsHandled(statement)) return null;
return drill(statement);
}
private JavacNode buildLambda(JCTree jcTree) {
return buildStatementOrExpression(getBody(jcTree));
}
private JCTree getBody(JCTree jcTree) {
return (JCTree) Permit.invokeSneaky(getBodyMethod(jcTree.getClass()), jcTree);
}
private final static ConcurrentMap<Class<?>, Method> getBodyMethods = new
ConcurrentHashMap<Class<?>, Method>();
private Method getBodyMethod(Class<?> c) {
```

```
Method m = getBodyMethods.get(c);
if (m != null) {
return m;
}
try {
m = Permit.getMethod(c, "getBody");
} catch (NoSuchMethodException e) {
throw Javac.sneakyThrow(e);
}
getBodyMethods.putIfAbsent(c, m);
return getBodyMethods.get(c);
}
private JavacNode buildEnhancedForLoop(JCEnhancedForLoop loop) {
if (setAndGetAsHandled(loop)) return null;
List<JavacNode> childNodes = new ArrayList<JavacNode>();
// The order of the child elements is important and must be kept
addIfNotNull(childNodes, buildTree(getVarOrRecordPattern(loop), Kind.STATEMENT));
addIfNotNull(childNodes, buildTree(loop.expr, Kind.STATEMENT));
addIfNotNull(childNodes, buildStatement(loop.body));
return putInMap(new JavacNode(this, loop, childNodes, Kind.STATEMENT));
}
private JavacNode drill(JCTree statement) {
try {
List<JavacNode> childNodes = new ArrayList<JavacNode>();
for (FieldAccess fa : fieldsOf(statement.getClass()))
childNodes.addAll(buildWithField(JavacNode.class, statement, fa));
return putInMap(new JavacNode(this, statement, childNodes, Kind.STATEMENT));
} catch (OutOfMemoryError oome) {
String msg = oome.getMessage();
if (msg == null) msg = "(no original message)";
OutOfMemoryError newError = new OutOfMemoryError(getFileName() + "@pos" +
statement.getPreferredPosition() + ": " + msg);
// We could try to set the stack trace of the new exception to the same one as the old exception, but this
costs memory,
// and we're already in an extremely fragile situation in regards to remaining heap space, so let's not do
that.
throw newError;
}
}
/* For javac, both JCExpression and JCStatement are considered as valid children types. */
private static Collection<Class<? extends JCTree>> statementTypes() {
Collection<Class<? extends JCTree>> collection = new ArrayList<Class<? extends JCTree>>(3);
collection.add(JCStatement.class);
collection.add(JCExpression.class);
collection.add(JCCatch.class);
return collection;
}
private static void addIfNotNull(Collection<JavacNode> nodes, JavacNode node) {
if (node != null) nodes.add(node);
}
/**
* Attempts to remove any compiler errors generated by java whose reporting position is located
anywhere between the start and end of the supplied node.
```

```
*/
void removeDeferredErrors(JavacNode node) {
DiagnosticPosition pos = node.get().pos();
JCCompilationUnit top = (JCCompilationUnit) top().get();
removeFromDeferredDiagnostics(pos.getStartPosition(), Javac.getEndPosition(pos, top));
}
/** Supply either a position or a node (in that case, position of the node is used) */
void printMessage(Diagnostic.Kind kind, String message, JavacNode node, DiagnosticPosition pos,
boolean attemptToRemoveErrorsInRange) {
JavaFileObject oldSource = null;
JavaFileObject newSource = null;
JCTree astObject = node == null ? null : node.get();
JCCompilationUnit top = (JCCompilationUnit) top().get();
newSource = top.sourcefile;
if (newSource != null) {
oldSource = errorLogger.useSource(newSource);
if (pos == null) pos = astObject.pos();
}
if (pos != null && node != null && attemptToRemoveErrorsInRange) {
removeFromDeferredDiagnostics(pos.getStartPosition(), node.getEndPosition(pos));
}
try {
switch (kind) {
case ERROR:
errorLogger.error(pos, message);
break;
case MANDATORY_WARNING:
errorLogger.mandatoryWarning(pos, message);
break;
case WARNING:
errorLogger.warning(pos, message);
break;
default:
case NOTE:
errorLogger.note(pos, message);
break;
}
} finally {
if (newSource != null) errorLogger.useSource(oldSource);
}
}
public void removeFromDeferredDiagnostics(int startPos, int endPos) {
JCCompilationUnit self = (JCCompilationUnit) top().get();
new CompilerMessageSuppressor(getContext()).removeAllBetween(self.sourcefile, startPos, endPos);
}
/** {@inheritDoc} */
@Override protected void setElementInASTCollection(Field field, Object refField, List<Collection<?>>
chain, Collection<?> collection, int idx, JCTree newN) throws IllegalAccessException {
com.sun.tools.javac.util.List<?> list = setElementInConsList(chain, collection,
((List<?>)collection).get(idx), newN);
field.set(refField, list);
}
```

```
private com.sun.tools.javac.util.List<?> setElementInConsList(List<Collection<?>> chain,
Collection<?> current, Object oldO, Object newO) {
com.sun.tools.javac.util.List<?> oldL = (com.sun.tools.javac.util.List<?>) current;
com.sun.tools.javac.util.List<?> newL = replaceInConsList(oldL, oldO, newO);
if (chain.isEmpty()) return newL;
List<Collection<?>> reducedChain = new ArrayList<Collection<?>>(chain);
Collection<?> newCurrent = reducedChain.remove(reducedChain.size() -1);
return setElementInConsList(reducedChain, newCurrent, oldL, newL);
}
private com.sun.tools.javac.util.List<?> replaceInConsList(com.sun.tools.javac.util.List<?> oldL, Object
oldO, Object newO) {
boolean repl = false;
Object[] a = oldL.toArray();
for (int i = 0; i < a.length; i++) {
if (a[i] == oldO) {
a[i] = newO;
repl = true;
}
}
if (repl) return com.sun.tools.javac.util.List.<Object>from(a);
return oldL;
}
abstract static class ErrorLog {
final Log log;
private final Messager messager;
private final Field errorCount;
private final Field warningCount;
private ErrorLog(Log log, Messager messager, Field errorCount, Field warningCount) {
this.log = log;
this.messager = messager;
this.errorCount = errorCount;
this.warningCount = warningCount;
}
final JavaFileObject useSource(JavaFileObject file) {
return log.useSource(file);
}
final void error(DiagnosticPosition pos, String message) {
increment(errorCount);
error1(pos, message);
}
final void warning(DiagnosticPosition pos, String message) {
increment(warningCount);
warning1(pos, message);
}
final void mandatoryWarning(DiagnosticPosition pos, String message) {
increment(warningCount);
mandatoryWarning1(pos, message);
}
abstract void error1(DiagnosticPosition pos, String message);
abstract void warning1(DiagnosticPosition pos, String message);
abstract void mandatoryWarning1(DiagnosticPosition pos, String message);
abstract void note(DiagnosticPosition pos, String message);
private void increment(Field field) {
```

```
if (field == null) return;
try {
int val = ((Number)field.get(messager)).intValue();
field.set(messager, val +1);
} catch (Throwable t) {
//Very unfortunate, but in most cases it still works fine, so we'll silently swallow it.
}
}
static ErrorLog create(Messager messager, Context context) {
Field errorCount; try {
errorCount = Permit.getField(messager.getClass(), "errorCount");
} catch (Throwable t) {
errorCount = null;
}
Log log = Log.instance(context);
boolean hasMultipleErrors = false;
for (Field field : log.getClass().getFields()) {
if (field.getName().equals("multipleErrors")) {
hasMultipleErrors = true;
break;
}
}
if (hasMultipleErrors) return new JdkBefore9(log, messager, errorCount);
Field warningCount; try {
warningCount = Permit.getField(messager.getClass(), "warningCount");
} catch (Throwable t) {
warningCount = null;
}
return new Jdk9Plus(log, messager, errorCount, warningCount);
}
}
static class JdkBefore9 extends ErrorLog {
private JdkBefore9(Log log, Messager messager, Field errorCount) {
super(log, messager, errorCount, null);
}
@Override void error1(DiagnosticPosition pos, String message) {
boolean prev = log.multipleErrors;
log.multipleErrors = true;
try {
log.error(pos, "proc.messager", message);
} finally {
log.multipleErrors = prev;
}
}
@Override void warning1(DiagnosticPosition pos, String message) {
log.warning(pos, "proc.messager", message);
}
@Override void mandatoryWarning1(DiagnosticPosition pos, String message) {
log.mandatoryWarning(pos, "proc.messager", message);
}
@Override void note(DiagnosticPosition pos, String message) {
log.note(pos, "proc.messager", message);
}
```

```
}
static class Jdk9Plus extends ErrorLog {
private static final String PROC_MESSAGER = "proc.messager";
private Object multiple;
private Method errorMethod, warningMethod, mandatoryWarningMethod, noteMethod;
private Method errorKey, warningKey, noteKey;
private JCDiagnostic.Factory diags;
private Jdk9Plus(Log log, Messager messager, Field errorCount, Field warningCount) {
super(log, messager, errorCount, warningCount);
try {
final String jcd = "com.sun.tools.javac.util.JCDiagnostic";
Class<?> df = Class.forName(jcd + "$DiagnosticFlag");
for (Object constant : df.getEnumConstants()) {
if (constant.toString().equals("MULTIPLE")) this.multiple = constant;
}
Class<?> errorCls = Class.forName(jcd + "$Error");
Class<?> warningCls = Class.forName(jcd + "$Warning");
Class<?> noteCls = Class.forName(jcd + "$Note");
Class<?> lc = log.getClass();
this.errorMethod = Permit.getMethod(lc, "error", df, DiagnosticPosition.class, errorCls);
this.warningMethod = Permit.getMethod(lc, "warning", DiagnosticPosition.class, warningCls);
this.mandatoryWarningMethod = Permit.getMethod(lc, "mandatoryWarning", DiagnosticPosition.class,
warningCls);
this.noteMethod = Permit.getMethod(lc, "note", DiagnosticPosition.class, noteCls);
Field diagsField = Permit.getField(lc.getSuperclass(), "diags");
this.diags = (JCDiagnostic.Factory) diagsField.get(log);
Class<?> dc = this.diags.getClass();
this.errorKey = Permit.getMethod(dc, "errorKey", String.class, Object[].class);
this.warningKey = Permit.getMethod(dc, "warningKey", String.class, Object[].class);
this.noteKey = Permit.getMethod(dc, "noteKey", String.class, Object[].class);
} catch (Throwable t) {
//t.printStackTrace();
}
}
@Override void error1(DiagnosticPosition pos, String message) {
Object error = Permit.invokeSneaky(this.errorKey, diags, PROC_MESSAGER, new Object[] { message
});
if (error != null) Permit.invokeSneaky(errorMethod, log, multiple, pos, error);
}
@Override
void warning1(DiagnosticPosition pos, String message) {
Object warning = Permit.invokeSneaky(this.warningKey, diags, PROC_MESSAGER, new Object[] {
message });
if (warning != null) Permit.invokeSneaky(warningMethod, log, pos, warning);
}
@Override
void mandatoryWarning1(DiagnosticPosition pos, String message) {
Object warning = Permit.invokeSneaky(this.warningKey, diags, PROC_MESSAGER, new Object[] {
message });
if (warning != null) Permit.invokeSneaky(mandatoryWarningMethod, log, pos, warning);
}
@Override
void note(DiagnosticPosition pos, String message) {
```

```
Object note = Permit.invokeSneaky(this.noteKey, diags, PROC_MESSAGER, new Object[] { message
});
if (note != null) Permit.invokeSneaky(noteMethod, log, pos, note);
}
}
}
```

lombok/javac/JavacASTAdapter.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import com.sun.source.util.Trees;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
/**
* Standard adapter for the {@link JavacASTVisitor} interface. Every method on that interface
* has been implemented with an empty body. Override whichever methods you need.
*/
public class JavacASTAdapter implements JavacASTVisitor {
/** {@inheritDoc} */
@Override public void setTrees(Trees trees) {}
/** {@inheritDoc} */
@Override public void visitCompilationUnit(JavacNode top, JCCompilationUnit unit) {}
/** {@inheritDoc} */
@Override public void endVisitCompilationUnit(JavacNode top, JCCompilationUnit unit) {}
/** {@inheritDoc} */
```

```
@Override public void visitType(JavacNode typeNode, JCClassDecl type) {}
/** {@inheritDoc} */
@Override public void visitAnnotationOnType(JCClassDecl type, JavacNode annotationNode,
JCAnnotation annotation) {}
/** {@inheritDoc} */
@Override public void endVisitType(JavacNode typeNode, JCClassDecl type) {}
/** {@inheritDoc} */
@Override public void visitField(JavacNode fieldNode, JCVariableDecl field) {}
/** {@inheritDoc} */
@Override public void visitAnnotationOnField(JCVariableDecl field, JavacNode annotationNode,
JCAnnotation annotation) {}
/** {@inheritDoc} */
@Override public void endVisitField(JavacNode fieldNode, JCVariableDecl field) {}
/** {@inheritDoc} */
@Override public void visitInitializer(JavacNode initializerNode, JCBlock initializer) {}
/** {@inheritDoc} */
@Override public void endVisitInitializer(JavacNode initializerNode, JCBlock initializer) {}
/** {@inheritDoc} */
@Override public void visitMethod(JavacNode methodNode, JCMethodDecl method) {}
/** {@inheritDoc} */
@Override public void visitAnnotationOnMethod(JCMethodDecl method, JavacNode annotationNode,
JCAnnotation annotation) {}
/** {@inheritDoc} */
@Override public void endVisitMethod(JavacNode methodNode, JCMethodDecl method) {}
/** {@inheritDoc} */
@Override public void visitMethodArgument(JavacNode argumentNode, JCVariableDecl argument,
JCMethodDecl method) {}
/** {@inheritDoc} */
@Override public void visitAnnotationOnMethodArgument(JCVariableDecl argument, JCMethodDecl
method, JavacNode annotationNode, JCAnnotation annotation) {}
/** {@inheritDoc} */
@Override public void endVisitMethodArgument(JavacNode argumentNode, JCVariableDecl
argument, JCMethodDecl method) {}
/** {@inheritDoc} */
@Override public void visitLocal(JavacNode localNode, JCVariableDecl local) {}
/** {@inheritDoc} */
@Override public void visitAnnotationOnLocal(JCVariableDecl local, JavacNode annotationNode,
JCAnnotation annotation) {}
/** {@inheritDoc} */
@Override public void endVisitLocal(JavacNode localNode, JCVariableDecl local) {}
/** {@inheritDoc} */
@Override public void visitTypeUse(JavacNode typeUseNode, JCTree typeUse) {}
/** {@inheritDoc} */
@Override public void visitAnnotationOnTypeUse(JCTree typeUse, JavacNode annotationNode,
JCAnnotation annotation) {}
/** {@inheritDoc} */
@Override public void endVisitTypeUse(JavacNode typeUseNode, JCTree typeUse) {}
/** {@inheritDoc} */
@Override public void visitStatement(JavacNode statementNode, JCTree statement) {}
/** {@inheritDoc} */
@Override public void endVisitStatement(JavacNode statementNode, JCTree statement) {}
}
```

lombok/javac/JavacASTVisitor.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.io.PrintStream;
import java.lang.reflect.Field;
import com.sun.source.util.Trees;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
/**
* Implement so you can ask any JavacAST.LombokNode to traverse depth-first through all children,
* calling the appropriate visit and endVisit methods.
*/
public interface JavacASTVisitor {
void setTrees(Trees trees);
/**
* Called at the very beginning and end.
*/
void visitCompilationUnit(JavacNode top, JCCompilationUnit unit);
void endVisitCompilationUnit(JavacNode top, JCCompilationUnit unit);
/**
* Called when visiting a type (a class, interface, annotation, enum, etcetera).
*/
void visitType(JavacNode typeNode, JCClassDecl type);
```

```
void visitAnnotationOnType(JCClassDecl type, JavacNode annotationNode, JCAnnotation annotation);
void endVisitType(JavacNode typeNode, JCClassDecl type);
/**
* Called when visiting a field of a class.
*/
void visitField(JavacNode fieldNode, JCVariableDecl field);
void visitAnnotationOnField(JCVariableDecl field, JavacNode annotationNode, JCAnnotation
annotation);
void endVisitField(JavacNode fieldNode, JCVariableDecl field);
/**
* Called for static and instance initializers. You can tell the difference via the isStatic() method.
*/
void visitInitializer(JavacNode initializerNode, JCBlock initializer);
void endVisitInitializer(JavacNode initializerNode, JCBlock initializer);
/**
* Called for both methods and constructors.
*/
void visitMethod(JavacNode methodNode, JCMethodDecl method);
void visitAnnotationOnMethod(JCMethodDecl method, JavacNode annotationNode, JCAnnotation
annotation);
void endVisitMethod(JavacNode methodNode, JCMethodDecl method);
/**
* Visits a method argument.
*/
void visitMethodArgument(JavacNode argumentNode, JCVariableDecl argument, JCMethodDecl
method);
void visitAnnotationOnMethodArgument(JCVariableDecl argument, JCMethodDecl method, JavacNode
annotationNode, JCAnnotation annotation);
void endVisitMethodArgument(JavacNode argumentNode, JCVariableDecl argument, JCMethodDecl
method);
/**
* Visits a local declaration - that is, something like 'int x = 10;' on the method level. Also called
* for method parameters.
*/
void visitLocal(JavacNode localNode, JCVariableDecl local);
void visitAnnotationOnLocal(JCVariableDecl local, JavacNode annotationNode, JCAnnotation
annotation);
void endVisitLocal(JavacNode localNode, JCVariableDecl local);
/**
* Visits a node that represents a type reference. Anything from {@code int} to {@code T} to {@code
foo.pkg.Bar<T>.Baz<?> @Ann []}.
*/
void visitTypeUse(JavacNode typeUseNode, JCTree typeUse);
void visitAnnotationOnTypeUse(JCTree typeUse, JavacNode annotationNode, JCAnnotation
annotation);
void endVisitTypeUse(JavacNode typeUseNode, JCTree typeUse);
/**
* Visits a statement that isn't any of the other visit methods (e.g. JCClassDecl).
* The statement object is guaranteed to be either a JCStatement or a JCExpression.
*/
void visitStatement(JavacNode statementNode, JCTree statement);
void endVisitStatement(JavacNode statementNode, JCTree statement);
/**
```

```
* Prints the structure of an AST.
*/
public static class Printer implements JavacASTVisitor {
private final PrintStream out;
private final boolean printContent;
private int disablePrinting = 0;
private int indent = 0;
/**
* @param printContent if true, bodies are printed directly, as java code,
* instead of a tree listing of every AST node inside it.
*/
public Printer(boolean printContent) {
this(printContent, System.out);
}
/**
* @param printContent if true, bodies are printed directly, as java code,
* instead of a tree listing of every AST node inside it.
* @param out write output to this stream. You must close it yourself. flush() is called after every line.
*
* @see java.io.PrintStream#flush()
*/
public Printer(boolean printContent, PrintStream out) {
this.printContent = printContent;
this.out = out;
}
@Override public void setTrees(Trees trees) {}
private void forcePrint(String text, Object... params) {
StringBuilder sb = new StringBuilder();
for (int i = 0; i < indent; i++) sb.append(" ");
out.printf(sb.append(text).append('\n').toString(), params);
out.flush();
}
private void print(String text, Object... params) {
if (disablePrinting == 0) forcePrint(text, params);
}
@Override public void visitCompilationUnit(JavacNode LombokNode, JCCompilationUnit unit) {
out.println("------------------------------------------------------------");
print("<CU %s>", LombokNode.getFileName());
indent++;
}
@Override public void endVisitCompilationUnit(JavacNode node, JCCompilationUnit unit) {
indent--;
print("</CUD>");
}
private String printFlags(long f) {
return Flags.toString(f);
}
@Override public void visitType(JavacNode node, JCClassDecl type) {
print("<TYPE %s> %s", type.name, printFlags(type.mods.flags));
indent++;
if (printContent) {
print("%s", type);
disablePrinting++;
```

```
}
}
@Override public void visitAnnotationOnType(JCClassDecl type, JavacNode node, JCAnnotation
annotation) {
forcePrint("<ANNOTATION: %s />", annotation);
}
@Override public void endVisitType(JavacNode node, JCClassDecl type) {
if (printContent) disablePrinting--;
indent--;
print("</TYPE %s>", type.name);
}
@Override public void visitInitializer(JavacNode node, JCBlock initializer) {
print("<%s INITIALIZER>",
initializer.isStatic() ? "static" : "instance");
indent++;
if (printContent) {
print("%s", initializer);
disablePrinting++;
}
}
@Override public void endVisitInitializer(JavacNode node, JCBlock initializer) {
if (printContent) disablePrinting--;
indent--;
print("</%s INITIALIZER>", initializer.isStatic() ? "static" : "instance");
}
@Override public void visitField(JavacNode node, JCVariableDecl field) {
print("<FIELD %s %s> %s", field.vartype, field.name, printFlags(field.mods.flags));
indent++;
if (printContent) {
if (field.init != null) print("%s", field.init);
disablePrinting++;
}
}
@Override public void visitAnnotationOnField(JCVariableDecl field, JavacNode node, JCAnnotation
annotation) {
forcePrint("<ANNOTATION: %s />", annotation);
}
@Override public void endVisitField(JavacNode node, JCVariableDecl field) {
if (printContent) disablePrinting--;
indent--;
print("</FIELD %s %s>", field.vartype, field.name);
}
@Override public void visitMethod(JavacNode node, JCMethodDecl method) {
final String type;
if (method.name.contentEquals("<init>")) {
if ((method.mods.flags & Flags.GENERATEDCONSTR) != 0) {
type = "DEFAULTCONSTRUCTOR";
} else type = "CONSTRUCTOR";
} else type = "METHOD";
print("<%s %s> %s returns: %s", type, method.name, printFlags(method.mods.flags), method.restype);
indent++;
JCVariableDecl recv;
try {
```

```
Field f = JCMethodDecl.class.getField("recvparam");
recv = (JCVariableDecl) f.get(method);
} catch (Exception ignore) {
recv = null;
}
if (recv != null) {
List<JCAnnotation> annotations = recv.mods.annotations;
if (recv.mods != null) annotations = recv.mods.annotations;
boolean innerContent = annotations != null && annotations.isEmpty();
print("<RECEIVER-PARAM (%s) %s %s%s> %s", recv.vartype == null ? "null" :
recv.vartype.getClass().toString(), recv.vartype, recv.name, innerContent ? "" : " /",
printFlags(recv.mods.flags));
if (innerContent) {
indent++;
for (JCAnnotation ann : annotations) print("<ANNOTATION: %s />", ann);
indent--;
print("</RECEIVER-PARAM>");
}
}
if (printContent) {
if (method.body == null) print("(ABSTRACT)");
else print("%s", method.body);
disablePrinting++;
}
}
@Override public void visitAnnotationOnMethod(JCMethodDecl method, JavacNode node,
JCAnnotation annotation) {
forcePrint("<ANNOTATION: %s />", annotation);
}
@Override public void endVisitMethod(JavacNode node, JCMethodDecl method) {
if (printContent) disablePrinting--;
indent--;
print("</%s %s>", "METHOD", method.name);
}
@Override public void visitMethodArgument(JavacNode node, JCVariableDecl arg, JCMethodDecl
method) {
print("<METHODARG (%s) %s %s> %s", arg.vartype.getClass().toString(), arg.vartype, arg.name,
printFlags(arg.mods.flags));
indent++;
}
@Override public void visitAnnotationOnMethodArgument(JCVariableDecl arg, JCMethodDecl method,
JavacNode nodeAnnotation, JCAnnotation annotation) {
forcePrint("<ANNOTATION: %s />", annotation);
}
@Override public void endVisitMethodArgument(JavacNode node, JCVariableDecl arg, JCMethodDecl
method) {
indent--;
print("</METHODARG %s %s>", arg.vartype, arg.name);
}
@Override public void visitLocal(JavacNode node, JCVariableDecl local) {
print("<LOCAL %s %s> %s", local.vartype, local.name, printFlags(local.mods.flags));
indent++;
}
```

```
@Override public void visitAnnotationOnLocal(JCVariableDecl local, JavacNode node, JCAnnotation
annotation) {
print("<ANNOTATION: %s />", annotation);
}
@Override public void endVisitLocal(JavacNode node, JCVariableDecl local) {
indent--;
print("</LOCAL %s %s>", local.vartype, local.name);
}
@Override public void visitTypeUse(JavacNode node, JCTree typeUse) {
print("<TYPE %s>", typeUse.getClass());
indent++;
print("%s", typeUse);
}
@Override public void visitAnnotationOnTypeUse(JCTree typeUse, JavacNode node, JCAnnotation
annotation) {
print("<ANNOTATION: %s />", annotation);
}
@Override public void endVisitTypeUse(JavacNode node, JCTree typeUse) {
indent--;
print("</TYPE %s>", typeUse.getClass());
}
@Override public void visitStatement(JavacNode node, JCTree statement) {
print("<%s>", statement.getClass());
indent++;
print("%s", statement);
}
@Override public void endVisitStatement(JavacNode node, JCTree statement) {
indent--;
print("</%s>", statement.getClass());
}
}
}
```

lombok/javac/JavacAnnotationHandler.java


```
/*
* Copyright (C) 2009-2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.lang.annotation.Annotation;
import lombok.core.AnnotationValues;
import lombok.core.SpiLoadUtil;
import com.sun.source.util.Trees;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
/**
* Implement this interface if you want to be triggered for a specific annotation.
*
* You MUST replace 'T' with a specific annotation type, such as:
*
* {@code public class HandleGetter extends JavacAnnotationHandler<Getter>}
*
* Because this generics parameter is inspected to figure out which class you're interested in.
*
* You also need to register yourself via SPI discovery as being an implementation of {@code
JavacAnnotationHandler}.
*/
public abstract class JavacAnnotationHandler<T extends Annotation> {
protected Trees trees;
/**
* Called when an annotation is found that is likely to match the annotation you're interested in.
*
* Be aware that you'll be called for ANY annotation node in the source that looks like a match. There is,
* for example, no guarantee that the annotation node belongs to a method, even if you set your
* TargetType in the annotation to methods only.
*
* @param annotation The actual annotation - use this object to retrieve the annotation parameters.
* @param ast The javac AST node representing the annotation.
* @param annotationNode The Lombok AST wrapper around the 'ast' parameter. You can use this
object
* to travel back up the chain (something javac AST can't do) to the parent of the annotation, as well
* as access useful methods such as generating warnings or errors focused on the annotation.
*/
public abstract void handle(AnnotationValues<T> annotation, JCAnnotation ast, JavacNode
annotationNode);
/**
* This handler is a handler for the given annotation; you don't normally need to override this class
* as the annotation type is extracted from your {@code extends
EclipseAnnotationHandler<AnnotationTypeHere>}
* signature.
*/
@SuppressWarnings("unchecked") public Class<T> getAnnotationHandledByThisHandler() {
return (Class<T>) SpiLoadUtil.findAnnotationClass(getClass(), JavacAnnotationHandler.class);
}
public void setTrees(Trees trees) {
this.trees = trees;
}
```

}

lombok/javac/JavacAugments.java

```
/*
* Copyright (C) 2014-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import lombok.core.FieldAugment;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCImport;
public final class JavacAugments {
private JavacAugments() {
// Prevent instantiation
}
public static final FieldAugment<JCTree, Boolean> JCTree_handled =
FieldAugment.augment(JCTree.class, boolean.class, "lombok$handled");
public static final FieldAugment<JCTree, JCTree> JCTree_generatedNode =
FieldAugment.circularSafeAugment(JCTree.class, JCTree.class, "lombok$generatedNode");
public static final FieldAugment<JCImport, Boolean> JCImport_deletable =
FieldAugment.circularSafeAugment(JCImport.class, Boolean.class, "lombok$deletable");
public static final FieldAugment<JCTree, Boolean> JCTree_keepPosition =
FieldAugment.augment(JCTree.class, boolean.class, "lombok$keepPosition");
}
```

lombok/javac/JavacImportList.java

```
/*
* Copyright (C) 2013-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
```

```
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.util.ArrayList;
import java.util.Collection;
import lombok.core.ImportList;
import lombok.core.LombokInternalAliasing;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCImport;
import com.sun.tools.javac.util.List;
public class JavacImportList implements ImportList {
private final String pkgStr;
private final List<JCTree> defs;
public JavacImportList(JCCompilationUnit cud) {
this.pkgStr = PackageName.getPackageName(cud);
this.defs = cud.defs;
}
@Override public String getFullyQualifiedNameForSimpleName(String unqualified) {
String q = getFullyQualifiedNameForSimpleNameNoAliasing(unqualified);
return q == null ? null : LombokInternalAliasing.processAliases(q);
}
@Override public String getFullyQualifiedNameForSimpleNameNoAliasing(String unqualified) {
for (JCTree def : defs) {
if (!(def instanceof JCImport)) continue;
JCTree qual = Javac.getQualid((JCImport) def);
if (!(qual instanceof JCFieldAccess)) continue;
String simpleName = ((JCFieldAccess) qual).name.toString();
if (simpleName.equals(unqualified)) {
return qual.toString();
}
}
return null;
}
@Override public boolean hasStarImport(String packageName) {
if (pkgStr != null && pkgStr.equals(packageName)) return true;
if ("java.lang".equals(packageName)) return true;
for (JCTree def : defs) {
```

```
if (!(def instanceof JCImport)) continue;
if (((JCImport) def).staticImport) continue;
JCTree qual = Javac.getQualid((JCImport) def);
if (!(qual instanceof JCFieldAccess)) continue;
String simpleName = ((JCFieldAccess) qual).name.toString();
if (!"*".equals(simpleName)) continue;
String starImport = ((JCFieldAccess) qual).selected.toString();
if (packageName.equals(starImport)) return true;
}
return false;
}
@Override public Collection<String> applyNameToStarImports(String startsWith, String name) {
ArrayList<String> out = new ArrayList<String>();
if (pkgStr != null && topLevelName(pkgStr).equals(startsWith)) out.add(pkgStr + "." + name);
for (JCTree def : defs) {
if (!(def instanceof JCImport)) continue;
if (((JCImport) def).staticImport) continue;
JCTree qual = Javac.getQualid((JCImport) def);
if (!(qual instanceof JCFieldAccess)) continue;
String simpleName = ((JCFieldAccess) qual).name.toString();
if (!"*".equals(simpleName)) continue;
String topLevelName = topLevelName(qual);
if (topLevelName.equals(startsWith)) {
out.add(((JCFieldAccess) qual).selected.toString() + "." + name);
}
}
return out;
}
private String topLevelName(JCTree tree) {
while (tree instanceof JCFieldAccess) tree = ((JCFieldAccess) tree).selected;
return tree.toString();
}
private String topLevelName(String packageName) {
int idx = packageName.indexOf(".");
if (idx == -1) return packageName;
return packageName.substring(0, idx);
}
@Override public String applyUnqualifiedNameToPackage(String unqualified) {
if (pkgStr == null) return unqualified;
return pkgStr + "." + unqualified;
}
}
```

lombok/javac/JavacNode.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
```

* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac;
import java.lang.annotation.Annotation;
import java.util.List;
import javax.lang.model.element.Element;
import javax.tools.Diagnostic;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.code.Symtab;
import com.sun.tools.javac.model.JavacTypes;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import com.sun.tools.javac.util.Name;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.javac.handlers.JavacHandlerUtil;
/**
* Javac specific version of the LombokNode class.
*/
public class JavacNode extends lombok.core.LombokNode<JavacAST, JavacNode, JCTree> {
private JavacAST ast;
/**
* Passes through to the parent constructor.
*/
public JavacNode(JavacAST ast, JCTree node, List<JavacNode> children, Kind kind) {
super(node, children, kind);
this.ast = ast;
}
@Override
public JavacAST getAst() {
return ast;
}
```

```
public Element getElement() {
if (node instanceof JCClassDecl) return ((JCClassDecl) node).sym;
if (node instanceof JCMethodDecl) return ((JCMethodDecl) node).sym;
if (node instanceof JCVariableDecl) return ((JCVariableDecl) node).sym;
return null;
}
public int getEndPosition(DiagnosticPosition pos) {
JCCompilationUnit cu = (JCCompilationUnit) top().get();
return Javac.getEndPosition(pos, cu);
}
public int getEndPosition() {
return getEndPosition(node);
}
/**
* Visits this node and all child nodes depth-first, calling the provided visitor's visit methods.
*/
public void traverse(JavacASTVisitor visitor) {
switch (this.getKind()) {
case COMPILATION_UNIT:
visitor.visitCompilationUnit(this, (JCCompilationUnit) get());
ast.traverseChildren(visitor, this);
visitor.endVisitCompilationUnit(this, (JCCompilationUnit) get());
break;
case TYPE:
visitor.visitType(this, (JCClassDecl) get());
ast.traverseChildren(visitor, this);
visitor.endVisitType(this, (JCClassDecl) get());
break;
case FIELD:
visitor.visitField(this, (JCVariableDecl) get());
ast.traverseChildren(visitor, this);
visitor.endVisitField(this, (JCVariableDecl) get());
break;
case METHOD:
visitor.visitMethod(this, (JCMethodDecl) get());
ast.traverseChildren(visitor, this);
visitor.endVisitMethod(this, (JCMethodDecl) get());
break;
case INITIALIZER:
visitor.visitInitializer(this, (JCBlock) get());
ast.traverseChildren(visitor, this);
visitor.endVisitInitializer(this, (JCBlock) get());
break;
case ARGUMENT:
JCMethodDecl parentMethod = (JCMethodDecl) up().get();
visitor.visitMethodArgument(this, (JCVariableDecl) get(), parentMethod);
ast.traverseChildren(visitor, this);
visitor.endVisitMethodArgument(this, (JCVariableDecl) get(), parentMethod);
break;
case LOCAL:
visitor.visitLocal(this, (JCVariableDecl) get());
ast.traverseChildren(visitor, this);
visitor.endVisitLocal(this, (JCVariableDecl) get());
```

```
break;
case STATEMENT:
visitor.visitStatement(this, get());
ast.traverseChildren(visitor, this);
visitor.endVisitStatement(this, get());
break;
case ANNOTATION:
switch (up().getKind()) {
case TYPE:
visitor.visitAnnotationOnType((JCClassDecl) up().get(), this, (JCAnnotation) get());
break;
case FIELD:
visitor.visitAnnotationOnField((JCVariableDecl) up().get(), this, (JCAnnotation) get());
break;
case METHOD:
visitor.visitAnnotationOnMethod((JCMethodDecl) up().get(), this, (JCAnnotation) get());
break;
case ARGUMENT:
JCVariableDecl argument = (JCVariableDecl) up().get();
JCMethodDecl method = (JCMethodDecl) up().up().get();
visitor.visitAnnotationOnMethodArgument(argument, method, this, (JCAnnotation) get());
break;
case LOCAL:
visitor.visitAnnotationOnLocal((JCVariableDecl) up().get(), this, (JCAnnotation) get());
break;
case TYPE_USE:
visitor.visitAnnotationOnTypeUse(up().get(), this, (JCAnnotation) get());
break;
default:
throw new AssertionError("Annotion not expected as child of a " + up().getKind());
}
break;
case TYPE_USE:
visitor.visitTypeUse(this, get());
ast.traverseChildren(visitor, this);
visitor.endVisitTypeUse(this, get());
break;
default:
throw new AssertionError("Unexpected kind during node traversal: " + getKind());
}
}
/** {@inheritDoc} */
@Override public String getName() {
final Name n;
if (node instanceof JCClassDecl) n = ((JCClassDecl) node).name;
else if (node instanceof JCMethodDecl) n = ((JCMethodDecl) node).name;
else if (node instanceof JCVariableDecl) n = ((JCVariableDecl) node).name;
else n = null;
return n == null ? null : n.toString();
}
/** {@inheritDoc} */
@Override protected boolean calculateIsStructurallySignificant(JCTree parent) {
if (node instanceof JCClassDecl) return true;
```

```
if (node instanceof JCMethodDecl) return true;
if (node instanceof JCVariableDecl) return true;
if (node instanceof JCCompilationUnit) return true;
//Static and instance initializers
if (node instanceof JCBlock) return parent instanceof JCClassDecl;
return false;
}
/**
* Convenient shortcut to the owning JavacAST object's getTreeMaker method.
*
* @see JavacAST#getTreeMaker()
*/
public JavacTreeMaker getTreeMaker() {
return ast.getTreeMaker();
}
/**
* Convenient shortcut to the owning JavacAST object's getSymbolTable method.
*
* @see JavacAST#getSymbolTable()
*/
public Symtab getSymbolTable() {
return ast.getSymbolTable();
}
/**
* Convenient shortcut to the owning JavacAST object's getTypesUtil method.
*
* @see JavacAST#getTypesUtil()
*/
public JavacTypes getTypesUtil() {
return ast.getTypesUtil();
}
/**
* Convenient shortcut to the owning JavacAST object's getContext method.
*
* @see JavacAST#getContext()
*/
public Context getContext() {
return ast.getContext();
}
public boolean shouldDeleteLombokAnnotations() {
return LombokOptions.shouldDeleteLombokAnnotations(ast.getContext());
}
/**
* Convenient shortcut to the owning JavacAST object's toName method.
*
* @see JavacAST#toName(String)
*/
public Name toName(String name) {
return ast.toName(name);
}
public void removeDeferredErrors() {
ast.removeDeferredErrors(this);
}
```

```
/**
* Generates an compiler error focused on the AST node represented by this node object.
*/
@Override public void addError(String message) {
ast.printMessage(Diagnostic.Kind.ERROR, message, this, null, true);
}
/**
* Generates an compiler error focused on the AST node represented by this node object.
*/
public void addError(String message, DiagnosticPosition pos) {
ast.printMessage(Diagnostic.Kind.ERROR, message, null, pos, true);
}
/**
* Generates a compiler warning focused on the AST node represented by this node object.
*/
@Override public void addWarning(String message) {
ast.printMessage(Diagnostic.Kind.WARNING, message, this, null, false);
}
/**
* Generates a compiler warning focused on the AST node represented by this node object.
*/
public void addWarning(String message, DiagnosticPosition pos) {
ast.printMessage(Diagnostic.Kind.WARNING, message, null, pos, false);
}
@Override public boolean hasAnnotation(Class<? extends Annotation> type) {
return JavacHandlerUtil.hasAnnotationAndDeleteIfNeccessary(type, this);
}
@Override public <Z extends Annotation> AnnotationValues<Z> findAnnotation(Class<Z> type) {
JavacNode annotation = JavacHandlerUtil.findAnnotation(type, this, true);
if (annotation == null) return null;
return JavacHandlerUtil.createAnnotation(type, annotation);
}
private JCModifiers getModifiers() {
if (node instanceof JCClassDecl) return ((JCClassDecl) node).getModifiers();
if (node instanceof JCMethodDecl) return ((JCMethodDecl) node).getModifiers();
if (node instanceof JCVariableDecl) return ((JCVariableDecl) node).getModifiers();
return null;
}
@Override public boolean isStatic() {
if (node instanceof JCClassDecl) {
JCClassDecl t = (JCClassDecl) node;
long f = t.mods.flags;
if (((Flags.INTERFACE | Flags.ENUM | Javac.RECORD) & f) != 0) return true;
JavacNode directUp = directUp();
if (directUp == null || directUp.getKind() == Kind.COMPILATION_UNIT) return true;
if (!(directUp.get() instanceof JCClassDecl)) return false;
JCClassDecl p = (JCClassDecl) directUp.get();
f = p.mods.flags;
if (((Flags.INTERFACE | Flags.ENUM) & f) != 0) return true;
}
if (node instanceof JCVariableDecl) {
JavacNode directUp = directUp();
if (directUp != null && directUp.get() instanceof JCClassDecl) {
```

```
JCClassDecl p = (JCClassDecl) directUp.get();
long f = p.mods.flags;
if ((Flags.INTERFACE & f) != 0) return true;
}
}
JCModifiers mods = getModifiers();
if (mods == null) return false;
return (mods.flags & Flags.STATIC) != 0;
}
@Override public boolean isFinal() {
if (node instanceof JCVariableDecl) {
JavacNode directUp = directUp();
if (directUp != null && directUp.get() instanceof JCClassDecl) {
JCClassDecl p = (JCClassDecl) directUp.get();
long f = p.mods.flags;
if (((Flags.INTERFACE | Flags.ENUM) & f) != 0) return true;
}
}
JCModifiers mods = getModifiers();
return mods != null && (Flags.FINAL & mods.flags) != 0;
}
@Override public boolean isEnumMember() {
if (getKind() != Kind.FIELD) return false;
JCModifiers mods = getModifiers();
return mods != null && (Flags.ENUM & mods.flags) != 0;
}
@Override public boolean isEnumType() {
if (getKind() != Kind.TYPE) return false;
JCModifiers mods = getModifiers();
return mods != null && (Flags.ENUM & mods.flags) != 0;
}
@Override public boolean isPrimitive() {
if (node instanceof JCVariableDecl && !isEnumMember()) {
return Javac.isPrimitive(((JCVariableDecl) node).vartype);
}
if (node instanceof JCMethodDecl) {
return Javac.isPrimitive(((JCMethodDecl) node).restype);
}
return false;
}
/**
* {@inheritDoc}
*/
@Override public String fieldOrMethodBaseType() {
if (node instanceof JCVariableDecl && !isEnumMember()) {
return (((JCVariableDecl) node).vartype).toString();
}
if (node instanceof JCMethodDecl) {
return (((JCMethodDecl) node).restype).toString();
}
return null;
}
@Override public boolean isTransient() {
```

```
if (getKind() != Kind.FIELD) return false;
JCModifiers mods = getModifiers();
return mods != null && (Flags.TRANSIENT & mods.flags) != 0;
}
@Override public int countMethodParameters() {
if (getKind() != Kind.METHOD) return 0;
com.sun.tools.javac.util.List<JCVariableDecl> params = ((JCMethodDecl) node).params;
if (params == null) return 0;
return params.size();
}
@Override public int getStartPos() {
return node.getPreferredPosition();
}
}
```

lombok/javac/JavacResolution.java

```
/*
* Copyright (C) 2011-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import static lombok.javac.Javac.*;
import static lombok.javac.JavacTreeMaker.TypeTag.typeTag;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.util.ArrayDeque;
import java.util.Collection;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.NoSuchElementException;
import javax.lang.model.type.TypeKind;
import javax.tools.JavaFileObject;
```

```
import com.sun.tools.javac.code.BoundKind;
import com.sun.tools.javac.code.Symbol.TypeSymbol;
import com.sun.tools.javac.code.Symtab;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.code.Type.ArrayType;
import com.sun.tools.javac.code.Type.CapturedType;
import com.sun.tools.javac.code.Type.ClassType;
import com.sun.tools.javac.code.Type.TypeVar;
import com.sun.tools.javac.code.Type.WildcardType;
import com.sun.tools.javac.code.Types;
import com.sun.tools.javac.comp.ArgumentAttr;
import com.sun.tools.javac.comp.Attr;
import com.sun.tools.javac.comp.AttrContext;
import com.sun.tools.javac.comp.Enter;
import com.sun.tools.javac.comp.Env;
import com.sun.tools.javac.comp.MemberEnter;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.JCWildcard;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Log;
import com.sun.tools.javac.util.Name;
import lombok.core.debug.AssertionLogger;
import lombok.permit.Permit;
public class JavacResolution {
private final Context context;
private final Attr attr;
private final CompilerMessageSuppressor messageSuppressor;
private static final Method isLocal;
static {
Method local = Permit.permissiveGetMethod(TypeSymbol.class, "isLocal");
if (local == null) {
local = Permit.permissiveGetMethod(TypeSymbol.class, "isDirectlyOrIndirectlyLocal");
}
isLocal = local;
}
public JavacResolution(Context context) {
this.context = context;
attr = Attr.instance(context);
messageSuppressor = new CompilerMessageSuppressor(context);
}
/*
* We need to dig down to the level of the method or field declaration or (static) initializer block, then
attribute that entire method/field/block using
* the appropriate environment. So, we start from the top and walk down the node tree until we hit that
method/field/block and stop there, recording both
```

```
* the environment object (`env`) and the exact tree node (`copyAt`) at which to begin the attr process.
*/
private static final class EnvFinder extends JCTree.Visitor {
private Env<AttrContext> env = null;
private Enter enter;
private MemberEnter memberEnter;
private JCTree copyAt = null;
EnvFinder(Context context) {
this.enter = Enter.instance(context);
this.memberEnter = MemberEnter.instance(context);
}
Env<AttrContext> get() {
return env;
}
JCTree copyAt() {
return copyAt;
}
@Override public void visitTopLevel(JCCompilationUnit tree) {
if (copyAt != null) return;
env = enter.getTopLevelEnv(tree);
}
@Override public void visitClassDef(JCClassDecl tree) {
if (copyAt != null) return;
if (tree.sym != null) env = enter.getClassEnv(tree.sym);
}
@Override public void visitMethodDef(JCMethodDecl tree) {
if (copyAt != null) return;
env = memberEnter.getMethodEnv(tree, env);
copyAt = tree;
}
public void visitVarDef(JCVariableDecl tree) {
if (copyAt != null) return;
env = memberEnter.getInitEnv(tree, env);
copyAt = tree;
}
@Override public void visitBlock(JCBlock tree) {
if (copyAt != null) return;
copyAt = tree;
}
@Override public void visitTree(JCTree that) {
}
}
public Map<JCTree, JCTree> resolveMethodMember(JavacNode node) {
ArrayDeque<JCTree> stack = new ArrayDeque<JCTree>();
{
JavacNode n = node;
while (n != null) {
stack.push(n.get());
n = n.up();
}
}
messageSuppressor.disableLoggers();
try {
```

```
EnvFinder finder = new EnvFinder(node.getContext());
while (!stack.isEmpty()) stack.pop().accept(finder);
TreeMirrorMaker mirrorMaker = new TreeMirrorMaker(node.getTreeMaker(), node.getContext());
JCTree copy = mirrorMaker.copy(finder.copyAt());
Log log = Log.instance(node.getContext());
JavaFileObject oldFileObject = log.useSource(((JCCompilationUnit) node.top().get()).getSourceFile());
try {
memberEnterAndAttribute(copy, finder.get(), node.getContext());
return mirrorMaker.getOriginalToCopyMap();
} finally {
log.useSource(oldFileObject);
}
} finally {
messageSuppressor.enableLoggers();
}
}
private static Field memberEnterDotEnv;
private static Field getMemberEnterDotEnv() {
if (memberEnterDotEnv != null) return memberEnterDotEnv;
try {
return memberEnterDotEnv = Permit.getField(MemberEnter.class, "env");
} catch (NoSuchFieldException e) {
return null;
}
}
@SuppressWarnings("unchecked")
private static Env<AttrContext> getEnvOfMemberEnter(MemberEnter memberEnter) {
Field f = getMemberEnterDotEnv();
try {
return (Env<AttrContext>) f.get(memberEnter);
} catch (Exception e) {
return null;
}
}
private static void setEnvOfMemberEnter(MemberEnter memberEnter, Env<AttrContext> env) {
Field f = getMemberEnterDotEnv();
try {
f.set(memberEnter, env);
} catch (Exception e) {
return;
}
}
private void memberEnterAndAttribute(JCTree copy, Env<AttrContext> env, Context context) {
MemberEnter memberEnter = MemberEnter.instance(context);
Env<AttrContext> oldEnv = getEnvOfMemberEnter(memberEnter);
setEnvOfMemberEnter(memberEnter, env);
try {
copy.accept(memberEnter);
} catch (Exception ignore) {
// intentionally ignored; usually even if this step fails, val will work (but not for val in method local inner
classes and anonymous inner classes).
AssertionLogger.assertLog("member enter failed.", ignore);
} finally {
```

```
setEnvOfMemberEnter(memberEnter, oldEnv);
}
attrib(copy, env);
}
public void resolveClassMember(JavacNode node) {
ArrayDeque<JCTree> stack = new ArrayDeque<JCTree>();
{
JavacNode n = node;
while (n != null) {
stack.push(n.get());
n = n.up();
}
}
messageSuppressor.disableLoggers();
try {
EnvFinder finder = new EnvFinder(node.getContext());
while (!stack.isEmpty()) stack.pop().accept(finder);
attrib(node.get(), finder.get());
} finally {
messageSuppressor.enableLoggers();
}
}
private void attrib(JCTree tree, Env<AttrContext> env) {
try {
if (env.enclClass.type == null) {
if (env.enclClass.sym != null) {
env.enclClass.type = env.enclClass.sym.type;
}
}
if (env.enclClass.type == null) {
env.enclClass.type = Type.noType;
}
} catch (Throwable ignore) {
// This addresses issue #1553 which involves JDK9; if it doesn't exist, we probably don't need to set it.
}
Map<?,?> cache = null;
try {
cache = ArgumentAttrReflect.enableTempCache(context);
if (tree instanceof JCBlock) attr.attribStat(tree, env);
else if (tree instanceof JCMethodDecl) attr.attribStat(((JCMethodDecl) tree).body, env);
else if (tree instanceof JCVariableDecl) attr.attribStat(tree, env);
else throw new IllegalStateException("Called with something that isn't a block, method decl, or variable
decl");
} finally {
ArgumentAttrReflect.restoreCache(cache, context);
}
}
public static class TypeNotConvertibleException extends Exception {
public TypeNotConvertibleException(String msg) {
super(msg);
}
}
private static class ReflectiveAccess {
```

```
private static Method UPPER_BOUND;
private static Throwable initError;
static {
Method upperBound = null;
try {
upperBound = Permit.getMethod(Types.class, "upperBound", Type.class);
} catch (Throwable e) {
initError = e;
}
if (upperBound == null) try {
upperBound = Permit.getMethod(Types.class, "wildUpperBound", Type.class);
} catch (Throwable e) {
initError = e;
}
UPPER_BOUND = upperBound;
}
public static Type Types_upperBound(Types types, Type type) {
return (Type) Permit.invokeSneaky(initError, UPPER_BOUND, types, type);
}
}
/**
* ArgumentAttr was added in Java 9 and caches some method arguments. Lombok should cleanup its
changes after resolution.
*/
private static class ArgumentAttrReflect {
private static Field ARGUMENT_TYPE_CACHE;
static {
if (Javac.getJavaCompilerVersion() >= 9) {
try {
ARGUMENT_TYPE_CACHE = Permit.getField(ArgumentAttr.class, "argumentTypeCache");
} catch (Exception ignore) {}
}
}
public static Map<?, ?> enableTempCache(Context context) {
if (ARGUMENT_TYPE_CACHE == null) return null;
ArgumentAttr argumentAttr = ArgumentAttr.instance(context);
try {
Map<?, ?> cache = (Map<?, ?>) Permit.get(ARGUMENT_TYPE_CACHE, argumentAttr);
Permit.set(ARGUMENT_TYPE_CACHE, argumentAttr, new LinkedHashMap<Object, Object>(cache));
return cache;
} catch (Exception ignore) { }
return null;
}
public static void restoreCache(Map<?, ?> cache, Context context) {
if (ARGUMENT_TYPE_CACHE == null) return;
ArgumentAttr argumentAttr = ArgumentAttr.instance(context);
try {
Permit.set(ARGUMENT_TYPE_CACHE, argumentAttr, cache);
} catch (Exception ignore) { }
}
}
public static Type ifTypeIsIterableToComponent(Type type, JavacAST ast) {
if (type == null) return null;
```

```
Types types = Types.instance(ast.getContext());
Symtab syms = Symtab.instance(ast.getContext());
Type boundType = ReflectiveAccess.Types_upperBound(types, type);
// Type boundType = types.upperBound(type);
Type elemTypeIfArray = types.elemtype(boundType);
if (elemTypeIfArray != null) return elemTypeIfArray;
Type base = types.asSuper(boundType, syms.iterableType.tsym);
if (base == null) return syms.objectType;
List<Type> iterableParams = base.allparams();
return iterableParams.isEmpty() ? syms.objectType : ReflectiveAccess.Types_upperBound(types,
iterableParams.head);
}
public static JCExpression typeToJCTree(Type type, JavacAST ast, boolean allowVoid) throws
TypeNotConvertibleException {
return typeToJCTree(type, ast, false, allowVoid, false);
}
public static JCExpression createJavaLangObject(JavacAST ast) {
JavacTreeMaker maker = ast.getTreeMaker();
JCExpression out = maker.Ident(ast.toName("java"));
out = maker.Select(out, ast.toName("lang"));
out = maker.Select(out, ast.toName("Object"));
return out;
}
private static JCExpression typeToJCTree(Type type, JavacAST ast, boolean allowCompound,
boolean allowVoid, boolean allowCapture) throws TypeNotConvertibleException {
int dims = 0;
Type type0 = type;
while (type0 instanceof ArrayType) {
dims++;
type0 = ((ArrayType) type0).elemtype;
}
JCExpression result = typeToJCTree0(type0, ast, allowCompound, allowVoid, allowCapture);
while (dims > 0) {
result = ast.getTreeMaker().TypeArray(result);
dims--;
}
return result;
}
private static Iterable<? extends Type> concat(final Type t, final Collection<? extends Type> ts) {
if (t == null) return ts;
return new Iterable<Type>() {
@Override public Iterator<Type> iterator() {
return new Iterator<Type>() {
private boolean first = true;
private Iterator<? extends Type> wrap = ts == null ? null : ts.iterator();
@Override public boolean hasNext() {
if (first) return true;
if (wrap == null) return false;
return wrap.hasNext();
}
@Override public Type next() {
if (first) {
first = false;
```

```
return t;
}
if (wrap == null) throw new NoSuchElementException();
return wrap.next();
}
@Override public void remove() {
throw new UnsupportedOperationException();
}
};
}
};
}
private static int compare(Name a, Name b) {
return a.compareTo(b);
}
private static boolean isLocalType(TypeSymbol symbol) {
try {
return (Boolean) Permit.invoke(isLocal, symbol);
}
catch (Exception e) {
return false;
}
}
private static JCExpression typeToJCTree0(Type type, JavacAST ast, boolean allowCompound,
boolean allowVoid, boolean allowCapture) throws TypeNotConvertibleException {
// NB: There's such a thing as maker.Type(type), but this doesn't work very well; it screws up
anonymous classes, captures, and adds an extra prefix dot for some reason too.
// -- so we write our own take on that here.
JavacTreeMaker maker = ast.getTreeMaker();
if (CTC_BOT.equals(typeTag(type))) return createJavaLangObject(ast);
if (CTC_VOID.equals(typeTag(type))) return allowVoid ? primitiveToJCTree(type.getKind(), maker) :
createJavaLangObject(ast);
if (type.isPrimitive()) return primitiveToJCTree(type.getKind(), maker);
if (type.isErroneous()) throw new TypeNotConvertibleException("Type cannot be resolved");
TypeSymbol symbol = type.asElement();
List<Type> generics = type.getTypeArguments();
JCExpression replacement = null;
if (symbol == null) throw new TypeNotConvertibleException("Null or compound type");
if (symbol.name.length() == 0) {
// Anonymous inner class
if (type instanceof ClassType) {
Type winner = null;
int winLevel = 0; // 100 = array, 50 = class, 20 = typevar, 15 = wildcard, 10 = interface, 1 = Object.
Type supertype = ((ClassType) type).supertype_field;
List<Type> ifaces = ((ClassType) type).interfaces_field;
for (Type t : concat(supertype, ifaces)) {
int level = 0;
if (t instanceof ArrayType) level = 100;
else if (t instanceof TypeVar) level = 20;
else if (t instanceof WildcardType) level = 15;
else if (t.isInterface()) level = 10;
else if (isObject(t)) level = 1;
else if (t instanceof ClassType) level = 50;
```

```
else level = 5;
if (winLevel > level) continue;
if (winLevel < level) {
winner = t;
winLevel = level;
continue;
}
if (compare(winner.tsym.getQualifiedName(), t.tsym.getQualifiedName()) < 0) winner = t;
}
if (winner == null) return createJavaLangObject(ast);
return typeToJCTree(winner, ast, allowCompound, allowVoid, allowCapture);
}
throw new TypeNotConvertibleException("Anonymous inner class");
}
if (type instanceof WildcardType || type instanceof CapturedType) {
Type lower, upper;
if (type instanceof WildcardType) {
upper = ((WildcardType) type).getExtendsBound();
lower = ((WildcardType) type).getSuperBound();
} else {
lower = type.getLowerBound();
upper = type.getUpperBound();
if (allowCapture) {
BoundKind bk = ((CapturedType) type).wildcard.kind;
if (bk == BoundKind.UNBOUND) {
return maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
} else if (bk == BoundKind.EXTENDS) {
lower = null;
upper = ((CapturedType) type).wildcard.type;
} else if (bk == BoundKind.SUPER) {
lower = ((CapturedType) type).wildcard.type;
upper = null;
}
}
}
if (allowCompound) {
if (lower == null || CTC_BOT.equals(typeTag(lower))) {
if (upper == null || upper.toString().equals("java.lang.Object")) {
return maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
}
if (upper.getTypeArguments().contains(type)) {
return maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
}
JCExpression bound = typeToJCTree(upper, ast, false, false, true);
if (bound instanceof JCWildcard) return
maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
return maker.Wildcard(maker.TypeBoundKind(BoundKind.EXTENDS), bound);
} else {
JCExpression bound = typeToJCTree(lower, ast, false, false, true);
if (bound instanceof JCWildcard) return
maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
return maker.Wildcard(maker.TypeBoundKind(BoundKind.SUPER), bound);
}
```

```
}
if (upper != null) {
if (upper.getTypeArguments().contains(type)) {
return maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
}
return typeToJCTree(upper, ast, allowCompound, allowVoid, true);
}
return createJavaLangObject(ast);
}
String qName;
if (isLocalType(symbol)) {
qName = symbol.getSimpleName().toString();
} else if (symbol.type != null && symbol.type.getEnclosingType() != null &&
typeTag(symbol.type.getEnclosingType()).equals(typeTag("CLASS"))) {
replacement = typeToJCTree0(type.getEnclosingType(), ast, false, false, false);
qName = symbol.getSimpleName().toString();
} else {
qName = symbol.getQualifiedName().toString();
}
if (qName.isEmpty()) throw new TypeNotConvertibleException("unknown type");
if (qName.startsWith("<")) throw new TypeNotConvertibleException(qName);
String[] baseNames = qName.split("\\.");
int i = 0;
if (replacement == null) {
replacement = maker.Ident(ast.toName(baseNames[0]));
i = 1;
}
for (; i < baseNames.length; i++) {
replacement = maker.Select(replacement, ast.toName(baseNames[i]));
}
return genericsToJCTreeNodes(generics, ast, replacement);
}
private static boolean isObject(Type supertype) {
return supertype.tsym.toString().equals("java.lang.Object");
}
private static JCExpression genericsToJCTreeNodes(List<Type> generics, JavacAST ast,
JCExpression rawTypeNode) throws TypeNotConvertibleException {
if (generics != null && !generics.isEmpty()) {
ListBuffer<JCExpression> args = new ListBuffer<JCExpression>();
for (Type t : generics) args.append(typeToJCTree(t, ast, true, false, true));
return ast.getTreeMaker().TypeApply(rawTypeNode, args.toList());
}
return rawTypeNode;
}
private static JCExpression primitiveToJCTree(TypeKind kind, JavacTreeMaker maker) throws
TypeNotConvertibleException {
switch (kind) {
case BYTE:
return maker.TypeIdent(CTC_BYTE);
case CHAR:
return maker.TypeIdent( CTC_CHAR);
case SHORT:
return maker.TypeIdent(CTC_SHORT);
```

```
case INT:
return maker.TypeIdent(CTC_INT);
case LONG:
return maker.TypeIdent(CTC_LONG);
case FLOAT:
return maker.TypeIdent(CTC_FLOAT);
case DOUBLE:
return maker.TypeIdent(CTC_DOUBLE);
case BOOLEAN:
return maker.TypeIdent(CTC_BOOLEAN);
case VOID:
return maker.TypeIdent(CTC_VOID);
case NULL:
case NONE:
case OTHER:
default:
throw new TypeNotConvertibleException("Nulltype");
}
}
public static boolean platformHasTargetTyping() {
return Javac.getJavaCompilerVersion() >= 8;
}
}
```

lombok/javac/JavacTransformer.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.util.List;
import java.util.SortedSet;
import javax.annotation.processing.Messager;
```

```
import com.sun.source.util.Trees;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.Context;
import lombok.ConfigurationKeys;
import lombok.core.CleanupRegistry;
import lombok.core.LombokConfiguration;
public class JavacTransformer {
private final HandlerLibrary handlers;
private final Messager messager;
public JavacTransformer(Messager messager, Trees trees) {
this.messager = messager;
this.handlers = HandlerLibrary.load(messager, trees);
}
public SortedSet<Long> getPriorities() {
return handlers.getPriorities();
}
public SortedSet<Long> getPrioritiesRequiringResolutionReset() {
return handlers.getPrioritiesRequiringResolutionReset();
}
public void transform(long priority, Context context, List<JCCompilationUnit> compilationUnits,
CleanupRegistry cleanup) {
if (compilationUnits.isEmpty()) {
return;
}
JavacAST.ErrorLog errorLog = JavacAST.ErrorLog.create(messager, context);
for (JCCompilationUnit unit : compilationUnits) {
if (!Boolean.TRUE.equals(LombokConfiguration.read(ConfigurationKeys.LOMBOK_DISABLE,
JavacAST.getAbsoluteFileLocation(unit)))) {
JavacAST ast = new JavacAST(errorLog, context, unit, cleanup);
ast.traverse(new AnnotationVisitor(priority));
handlers.callASTVisitors(ast, priority);
if (ast.isChanged()) LombokOptions.markChanged(context, (JCCompilationUnit) ast.top().get());
}
}
}
private class AnnotationVisitor extends JavacASTAdapter {
private final long priority;
AnnotationVisitor(long priority) {
this.priority = priority;
}
@Override public void visitAnnotationOnType(JCClassDecl type, JavacNode annotationNode,
JCAnnotation annotation) {
JCCompilationUnit top = (JCCompilationUnit) annotationNode.top().get();
handlers.handleAnnotation(top, annotationNode, annotation, priority);
}
@Override public void visitAnnotationOnField(JCVariableDecl field, JavacNode annotationNode,
JCAnnotation annotation) {
JCCompilationUnit top = (JCCompilationUnit) annotationNode.top().get();
```

```
handlers.handleAnnotation(top, annotationNode, annotation, priority);
}
@Override public void visitAnnotationOnMethod(JCMethodDecl method, JavacNode annotationNode,
JCAnnotation annotation) {
JCCompilationUnit top = (JCCompilationUnit) annotationNode.top().get();
handlers.handleAnnotation(top, annotationNode, annotation, priority);
}
@Override public void visitAnnotationOnMethodArgument(JCVariableDecl argument, JCMethodDecl
method, JavacNode annotationNode, JCAnnotation annotation) {
JCCompilationUnit top = (JCCompilationUnit) annotationNode.top().get();
handlers.handleAnnotation(top, annotationNode, annotation, priority);
}
@Override public void visitAnnotationOnLocal(JCVariableDecl local, JavacNode annotationNode,
JCAnnotation annotation) {
JCCompilationUnit top = (JCCompilationUnit) annotationNode.top().get();
handlers.handleAnnotation(top, annotationNode, annotation, priority);
}
@Override public void visitAnnotationOnTypeUse(JCTree typeUse, JavacNode annotationNode,
JCAnnotation annotation) {
JCCompilationUnit top = (JCCompilationUnit) annotationNode.top().get();
handlers.handleAnnotation(top, annotationNode, annotation, priority);
}
}
}
```

lombok/javac/LombokOptions.java


```
/*
* Copyright (C) 2010-2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.javac;
import java.util.HashSet;
import java.util.Set;
```

```
import lombok.delombok.FormatPreferences;
import lombok.delombok.LombokOptionsFactory;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.Options;
public abstract class LombokOptions extends Options {
private boolean deleteLombokAnnotations = false;
private final Set<JCCompilationUnit> changed = new HashSet<JCCompilationUnit>();
private FormatPreferences formatPreferences = new FormatPreferences(null);
public boolean isChanged(JCCompilationUnit ast) {
return changed.contains(ast);
}
public void setFormatPreferences(FormatPreferences formatPreferences) {
this.formatPreferences = formatPreferences;
}
public FormatPreferences getFormatPreferences() {
return this.formatPreferences;
}
public static void markChanged(Context context, JCCompilationUnit ast) {
LombokOptions options = LombokOptionsFactory.getDelombokOptions(context);
options.changed.add(ast);
}
public static boolean shouldDeleteLombokAnnotations(Context context) {
LombokOptions options = LombokOptionsFactory.getDelombokOptions(context);
return options.deleteLombokAnnotations;
}
protected LombokOptions(Context context) {
super(context);
}
public abstract void putJavacOption(String optionName, String value);
public void deleteLombokAnnotations() {
this.deleteLombokAnnotations = true;
}
}
```

lombok/javac/ResolutionResetNeeded.java

```
/*
* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
/**
* Marks the annotated handler as needing a reset on the resolved model (resulting in generated methods' signatures from becoming part of the symbol table and such) before
* the priority level of this handler is reached.
*/
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface ResolutionResetNeeded {
}

lombok/javac/apt/InterceptingJavaFileManager.java

/*
* Copyright (C) 2010-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import java.io.IOException;
import javax.tools.FileObject;
import javax.tools.ForwardingJavaFileManager;
import javax.tools.JavaFileManager;
import javax.tools.JavaFileObject;

```
import javax.tools.JavaFileObject.Kind;
import lombok.core.DiagnosticsReceiver;
final class InterceptingJavaFileManager extends ForwardingJavaFileManager<JavaFileManager> {
private final DiagnosticsReceiver diagnostics;
private final LombokFileObjects.Compiler compiler;
InterceptingJavaFileManager(JavaFileManager original, DiagnosticsReceiver diagnostics) {
super(original);
this.compiler = LombokFileObjects.getCompiler(original);
this.diagnostics = diagnostics;
}
@Override public JavaFileObject getJavaFileForOutput(Location location, String className, final Kind
kind, FileObject sibling) throws IOException {
JavaFileObject fileObject = fileManager.getJavaFileForOutput(location, className, kind, sibling);
if (kind != Kind.CLASS) return fileObject;
return LombokFileObjects.createIntercepting(compiler, fileObject, className, diagnostics);
}
}
```

lombok/javac/apt/InterceptingJavaFileObject.java

```
/*
* Copyright (C) 2010-2011 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.io.Reader;
import java.io.Writer;
import java.lang.reflect.Method;
import java.net.URI;
import java.nio.charset.CharsetDecoder;
import javax.lang.model.element.Modifier;
```

```
import javax.lang.model.element.NestingKind;
import javax.tools.JavaFileObject;
import lombok.core.DiagnosticsReceiver;
import lombok.core.PostCompiler;
import lombok.permit.Permit;
final class InterceptingJavaFileObject implements LombokFileObject {
private final JavaFileObject delegate;
private final String fileName;
private final DiagnosticsReceiver diagnostics;
private final Method decoderMethod;
public InterceptingJavaFileObject(JavaFileObject original, String fileName, DiagnosticsReceiver
diagnostics, Method decoderMethod) {
this.delegate = original;
this.fileName = fileName;
this.diagnostics = diagnostics;
this.decoderMethod = decoderMethod;
}
@Override
public OutputStream openOutputStream() throws IOException {
return PostCompiler.wrapOutputStream(delegate.openOutputStream(), fileName, diagnostics);
}
@Override
public Writer openWriter() throws IOException {
throw new UnsupportedOperationException("Can't use a write for class files");
}
@Override public CharsetDecoder getDecoder(boolean ignoreEncodingErrors) {
if (decoderMethod == null) throw new UnsupportedOperationException();
return (CharsetDecoder) Permit.invokeSneaky(decoderMethod, delegate, ignoreEncodingErrors);
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof InterceptingJavaFileObject)) {
return false;
}
if (obj == this) {
return true;
}
InterceptingJavaFileObject other = (InterceptingJavaFileObject) obj;
return fileName.equals(other.fileName) && delegate.equals(other.delegate);
}
@Override public int hashCode() {
return fileName.hashCode() ^ delegate.hashCode();
}
////////////////////// NOTHING CHANGED BELOW ////////////////////////////////////
@Override
public boolean delete() {
return delegate.delete();
}
@Override
public Modifier getAccessLevel() {
return delegate.getAccessLevel();
}
@Override
public CharSequence getCharContent(boolean ignoreEncodingErrors) throws IOException {
```

```
return delegate.getCharContent(ignoreEncodingErrors);
}
@Override
public Kind getKind() {
return delegate.getKind();
}
@Override
public long getLastModified() {
return delegate.getLastModified();
}
@Override
@SuppressWarnings("all")
public String getName() {
return delegate.getName();
}
@Override
public NestingKind getNestingKind() {
return delegate.getNestingKind();
}
@Override
public boolean isNameCompatible(String simpleName, Kind kind) {
return delegate.isNameCompatible(simpleName, kind);
}
@Override
public InputStream openInputStream() throws IOException {
return delegate.openInputStream();
}
@Override
public Reader openReader(boolean ignoreEncodingErrors) throws IOException {
return delegate.openReader(ignoreEncodingErrors);
}
@Override
public URI toUri() {
return delegate.toUri();
}
@Override
public String toString() {
return delegate.toString();
}
}
```

lombok/javac/apt/Javac6BaseFileObjectWrapper.java

```
/*
* Copyright (C) 2010-2011 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
```

```
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.io.Reader;
import java.io.Writer;
import java.net.URI;
import java.nio.charset.CharsetDecoder;
import javax.lang.model.element.Modifier;
import javax.lang.model.element.NestingKind;
// Weird SuppressWarnings, but javac doesn't understand 'all' and eclipse doesn't understand that this
file contains deprecation references.
@SuppressWarnings({"all", "deprecation"})
class Javac6BaseFileObjectWrapper extends com.sun.tools.javac.util.BaseFileObject {
private final LombokFileObject delegate;
public Javac6BaseFileObjectWrapper(LombokFileObject delegate) {
this.delegate = delegate;
}
@Override public boolean isNameCompatible(String simpleName, Kind kind) {
return delegate.isNameCompatible(simpleName, kind);
}
@Override public URI toUri() {
return delegate.toUri();
}
@Override public String getName() {
return delegate.getName();
}
@Override public CharSequence getCharContent(boolean ignoreEncodingErrors) throws IOException
{
return delegate.getCharContent(ignoreEncodingErrors);
}
@Override public InputStream openInputStream() throws IOException {
return delegate.openInputStream();
}
@Override public Reader openReader(boolean ignoreEncodingErrors) throws IOException {
return delegate.openReader(ignoreEncodingErrors);
}
@Override public Writer openWriter() throws IOException {
return delegate.openWriter();
}
```

```
@Override public OutputStream openOutputStream() throws IOException {
return delegate.openOutputStream();
}
@Override public long getLastModified() {
return delegate.getLastModified();
}
@Override public boolean delete() {
return delegate.delete();
}
@Override public Kind getKind() {
return delegate.getKind();
}
@Override public NestingKind getNestingKind() {
return delegate.getNestingKind();
}
@Override public Modifier getAccessLevel() {
return delegate.getAccessLevel();
}
protected CharsetDecoder getDecoder(boolean ignoreEncodingErrors) {
return delegate.getDecoder(ignoreEncodingErrors);
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof Javac6BaseFileObjectWrapper)) {
return false;
}
return delegate.equals(((Javac6BaseFileObjectWrapper)obj).delegate);
}
@Override public int hashCode() {
return delegate.hashCode();
}
@Override public String toString() {
return delegate.toString();
}
}
```

lombok/javac/apt/Javac7BaseFileObjectWrapper.java

```
/*
* Copyright (C) 2010-2011 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.io.Reader;
import java.io.Writer;
import java.net.URI;
import java.nio.charset.CharsetDecoder;
import javax.lang.model.element.Modifier;
import javax.lang.model.element.NestingKind;
class Javac7BaseFileObjectWrapper extends com.sun.tools.javac.file.BaseFileObject {
private final LombokFileObject delegate;
public Javac7BaseFileObjectWrapper(LombokFileObject delegate) {
super(null);
this.delegate = delegate;
}
@Override public boolean isNameCompatible(String simpleName, Kind kind) {
return delegate.isNameCompatible(simpleName, kind);
}
@Override public URI toUri() {
return delegate.toUri();
}
@SuppressWarnings("all")
@Override public String getName() {
return delegate.getName();
}
@Override public CharSequence getCharContent(boolean ignoreEncodingErrors) throws IOException
{
return delegate.getCharContent(ignoreEncodingErrors);
}
@Override public InputStream openInputStream() throws IOException {
return delegate.openInputStream();
}
@Override public Reader openReader(boolean ignoreEncodingErrors) throws IOException {
return delegate.openReader(ignoreEncodingErrors);
}
@Override public Writer openWriter() throws IOException {
return delegate.openWriter();
}
@Override public OutputStream openOutputStream() throws IOException {
return delegate.openOutputStream();
}
@Override public long getLastModified() {
return delegate.getLastModified();
}
@Override public boolean delete() {
```

```
return delegate.delete();
}
@Override public Kind getKind() {
return delegate.getKind();
}
@Override public NestingKind getNestingKind() {
return delegate.getNestingKind();
}
@Override public Modifier getAccessLevel() {
return delegate.getAccessLevel();
}
protected CharsetDecoder getDecoder(boolean ignoreEncodingErrors) {
return delegate.getDecoder(ignoreEncodingErrors);
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof Javac7BaseFileObjectWrapper)) {
return false;
}
return delegate.equals(((Javac7BaseFileObjectWrapper)obj).delegate);
}
@Override public int hashCode() {
return delegate.hashCode();
}
@Override public String toString() {
return delegate.toString();
}
}
```

lombok/javac/apt/Javac9JavaFileObjectWrapper.java


```
/*
* Copyright (C) 2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```

```
package lombok.javac.apt;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.io.Reader;
import java.io.Writer;
import java.net.URI;
import javax.lang.model.element.Modifier;
import javax.lang.model.element.NestingKind;
class Javac9JavaFileObjectWrapper implements javax.tools.JavaFileObject {
private final LombokFileObject delegate;
public Javac9JavaFileObjectWrapper(LombokFileObject delegate) {
this.delegate = delegate;
}
@Override public boolean isNameCompatible(String simpleName, Kind kind) {
return delegate.isNameCompatible(simpleName, kind);
}
@Override public URI toUri() {
return delegate.toUri();
}
@SuppressWarnings("all")
@Override public String getName() {
return delegate.getName();
}
@Override public CharSequence getCharContent(boolean ignoreEncodingErrors) throws IOException
{
return delegate.getCharContent(ignoreEncodingErrors);
}
@Override public InputStream openInputStream() throws IOException {
return delegate.openInputStream();
}
@Override public Reader openReader(boolean ignoreEncodingErrors) throws IOException {
return delegate.openReader(ignoreEncodingErrors);
}
@Override public Writer openWriter() throws IOException {
return delegate.openWriter();
}
@Override public OutputStream openOutputStream() throws IOException {
return delegate.openOutputStream();
}
@Override public long getLastModified() {
return delegate.getLastModified();
}
@Override public boolean delete() {
return delegate.delete();
}
@Override public Kind getKind() {
return delegate.getKind();
}
@Override public NestingKind getNestingKind() {
return delegate.getNestingKind();
}
@Override public Modifier getAccessLevel() {
```

```
return delegate.getAccessLevel();
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof Javac9JavaFileObjectWrapper)) return false;
return delegate.equals(((Javac9JavaFileObjectWrapper)obj).delegate);
}
@Override public int hashCode() {
return delegate.hashCode();
}
@Override public String toString() {
return delegate.toString();
}
}
```

lombok/javac/apt/LombokFileObject.java

```
/*
* Copyright (C) 2010-2011 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import java.nio.charset.CharsetDecoder;
import javax.tools.JavaFileObject;
interface LombokFileObject extends JavaFileObject {
CharsetDecoder getDecoder(boolean ignoreEncodingErrors);
}
```

lombok/javac/apt/LombokFileObjects.java

```
/*
* Copyright (C) 2010-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
```

```
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import java.lang.reflect.Constructor;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.List;
import java.util.concurrent.atomic.AtomicBoolean;
import javax.tools.JavaFileManager;
import javax.tools.JavaFileObject;
import lombok.core.DiagnosticsReceiver;
import lombok.permit.Permit;
//Can't use SimpleJavaFileObject so we copy/paste most of its content here, because javac doesn't
follow the interface,
//and casts to its own BaseFileObject type. D'oh!
final class LombokFileObjects {
interface Compiler {
Compiler JAVAC6 = new Compiler() {
private Method decoderMethod = null;
private final AtomicBoolean decoderIsSet = new AtomicBoolean();
@Override public JavaFileObject wrap(LombokFileObject fileObject) {
return new Javac6BaseFileObjectWrapper(fileObject);
}
@Override public Method getDecoderMethod() {
synchronized (decoderIsSet) {
if (decoderIsSet.get()) return decoderMethod;
decoderMethod = LombokFileObjects.getDecoderMethod("com.sun.tools.javac.util.BaseFileObject");
decoderIsSet.set(true);
return decoderMethod;
}
}
};
Compiler JAVAC7 = new Compiler() {
private Method decoderMethod = null;
private final AtomicBoolean decoderIsSet = new AtomicBoolean();
@Override public JavaFileObject wrap(LombokFileObject fileObject) {
return new Javac7BaseFileObjectWrapper(fileObject);
```

```
}
@Override public Method getDecoderMethod() {
synchronized (decoderIsSet) {
if (decoderIsSet.get()) return decoderMethod;
decoderMethod = LombokFileObjects.getDecoderMethod("com.sun.tools.javac.file.BaseFileObject");
decoderIsSet.set(true);
return decoderMethod;
}
}
};
JavaFileObject wrap(LombokFileObject fileObject);
Method getDecoderMethod();
}
static Method getDecoderMethod(String className) {
try {
return Permit.getMethod(Class.forName(className), "getDecoder", boolean.class);
} catch (NoSuchMethodException e) {
// Intentional fallthrough - getDecoder(boolean) is not always present.
} catch (ClassNotFoundException e) {
// Intentional fallthrough - getDecoder(boolean) is not always present.
}
return null;
}
private LombokFileObjects() {}
private static final List<String> KNOWN_JAVA9_FILE_MANAGERS = Arrays.asList(
"com.google.errorprone.MaskedClassLoader$MaskedFileManager",
"com.google.devtools.build.buildjar.javac.BlazeJavacMain$ClassloaderMaskingFileManager",
"com.google.devtools.build.java.turbine.javac.JavacTurbineCompiler$ClassloaderMaskingFileManager
",
"org.netbeans.modules.java.source.parsing.ProxyFileManager",
"com.sun.tools.javac.api.ClientCodeWrapper$WrappedStandardJavaFileManager",
"com.sun.tools.javac.main.DelegatingJavaFileManager$DelegatingSJFM" // IntelliJ + JDK10
);
static Compiler getCompiler(JavaFileManager jfm) {
String jfmClassName = jfm != null ? jfm.getClass().getName() : "null";
if (jfmClassName.equals("com.sun.tools.javac.util.DefaultFileManager")) return Compiler.JAVAC6;
if (jfmClassName.equals("com.sun.tools.javac.util.JavacFileManager")) return Compiler.JAVAC6;
if (jfmClassName.equals("com.sun.tools.javac.file.JavacFileManager")) {
try {
Class<?> superType = Class.forName("com.sun.tools.javac.file.BaseFileManager");
if (superType.isInstance(jfm)) return java9Compiler(jfm);
}
catch (Throwable e) {}
return Compiler.JAVAC7;
}
if (KNOWN_JAVA9_FILE_MANAGERS.contains(jfmClassName)) {
try {
return java9Compiler(jfm);
}
catch (Throwable e) {}
}
try {
if (Class.forName("com.sun.tools.javac.file.PathFileObject") == null) throw new NullPointerException();
```

```
return java9Compiler(jfm);
} catch (Throwable e) {}
try {
if (Class.forName("com.sun.tools.javac.file.BaseFileObject") == null) throw new NullPointerException();
return Compiler.JAVAC7;
} catch (Throwable e) {}
try {
if (Class.forName("com.sun.tools.javac.util.BaseFileObject") == null) throw new NullPointerException();
return Compiler.JAVAC6;
} catch (Throwable e) {}
StringBuilder sb = new StringBuilder(jfmClassName);
if (jfm != null) {
sb.append(" extends ").append(jfm.getClass().getSuperclass().getName());
for (Class<?> cls : jfm.getClass().getInterfaces()) {
sb.append(" implements ").append(cls.getName());
}
}
throw new IllegalArgumentException(sb.toString());
}
static JavaFileObject createIntercepting(Compiler compiler, JavaFileObject delegate, String fileName,
DiagnosticsReceiver diagnostics) {
return compiler.wrap(new InterceptingJavaFileObject(delegate, fileName, diagnostics,
compiler.getDecoderMethod()));
}
private static Constructor<?> j9CompilerConstructor = null;
private static Compiler java9Compiler(JavaFileManager jfm) {
try {
if (j9CompilerConstructor == null) j9CompilerConstructor =
Class.forName("lombok.javac.apt.Java9Compiler").getConstructor(JavaFileManager.class);
return (Compiler) j9CompilerConstructor.newInstance(jfm);
} catch (ClassNotFoundException e) {
return null;
} catch (NoSuchMethodException e) {
return null;
} catch (InvocationTargetException e) {
Throwable t = e.getCause();
if (t instanceof RuntimeException) throw (RuntimeException) t;
if (t instanceof Error) throw (Error) t;
throw new RuntimeException(t);
} catch (IllegalAccessException e) {
throw new RuntimeException(e);
} catch (InstantiationException e) {
throw new RuntimeException(e);
}
}
}
```

lombok/javac/apt/LombokProcessor.java

```
/*
* Copyright (C) 2009-2020 The Project Lombok Authors.
```

*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.apt;
import java.io.IOException;
import java.io.InputStream;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.net.URL;
import java.util.ArrayList;
import java.util.Enumeration;
import java.util.HashSet;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.SortedSet;
import javax.annotation.processing.AbstractProcessor;
import javax.annotation.processing.Messager;
import javax.annotation.processing.ProcessingEnvironment;
import javax.annotation.processing.RoundEnvironment;
import javax.annotation.processing.SupportedAnnotationTypes;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.Element;
import javax.lang.model.element.TypeElement;
import javax.tools.Diagnostic.Kind;
import javax.tools.JavaFileManager;
import com.sun.source.util.TreePath;
import com.sun.source.util.Trees;
import com.sun.tools.javac.jvm.ClassWriter;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.processing.JavacFiler;
import com.sun.tools.javac.processing.JavacProcessingEnvironment;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.util.Context;
```

```
import lombok.Lombok;
import lombok.core.CleanupRegistry;
import lombok.core.DiagnosticsReceiver;
import lombok.javac.JavacTransformer;
import lombok.permit.Permit;
import lombok.permit.dummy.Parent;
import sun.misc.Unsafe;
/**
* This Annotation Processor is the standard injection mechanism for lombok-enabling the javac
compiler.
*
* To actually enable lombok in a javac compilation run, this class should be in the classpath when
* running javac; that's the only requirement.
*/
@SupportedAnnotationTypes("*")
public class LombokProcessor extends AbstractProcessor {
private ProcessingEnvironment processingEnv;
private JavacProcessingEnvironment javacProcessingEnv;
private JavacFiler javacFiler;
private JavacTransformer transformer;
private Trees trees;
private boolean lombokDisabled = false;
/** {@inheritDoc} */
@Override public void init(ProcessingEnvironment procEnv) {
super.init(procEnv);
if (System.getProperty("lombok.disable") != null) {
lombokDisabled = true;
return;
}
this.processingEnv = procEnv;
this.javacProcessingEnv = getJavacProcessingEnvironment(procEnv);
this.javacFiler = getJavacFiler(procEnv.getFiler());
placePostCompileAndDontMakeForceRoundDummiesHook();
trees = Trees.instance(javacProcessingEnv);
transformer = new JavacTransformer(procEnv.getMessager(), trees);
SortedSet<Long> p = transformer.getPriorities();
if (p.isEmpty()) {
this.priorityLevels = new long[] {0L};
this.priorityLevelsRequiringResolutionReset = new HashSet<Long>();
} else {
this.priorityLevels = new long[p.size()];
int i = 0;
for (Long prio : p) this.priorityLevels[i++] = prio;
this.priorityLevelsRequiringResolutionReset = transformer.getPrioritiesRequiringResolutionReset();
}
}
private static final String JPE = "com.sun.tools.javac.processing.JavacProcessingEnvironment";
private static final Field javacProcessingEnvironment_discoveredProcs = getFieldAccessor(JPE,
"discoveredProcs");
private static final Field discoveredProcessors_procStateList = getFieldAccessor(JPE +
"$DiscoveredProcessors", "procStateList");
private static final Field processorState_processor = getFieldAccessor(JPE + "$processor",
"processor");
```

```
private static final Field getFieldAccessor(String typeName, String fieldName) {
try {
return Permit.getField(Class.forName(typeName), fieldName);
} catch (ClassNotFoundException e) {
return null;
} catch (NoSuchFieldException e) {
return null;
}
}
// The intent of this method is to have lombok emit a warning if it's not 'first in line'. However,
pragmatically speaking, you're always looking at one of two cases:
// (A) The other processor(s) running before lombok require lombok to have run or they crash. So, they
crash, and unfortunately we are never even init-ed; the warning is never emitted.
// (B) The other processor(s) don't care about it at all. So, it doesn't actually matter that lombok isn't
first.
// Hence, for now, no warnings.
@SuppressWarnings("unused")
private String listAnnotationProcessorsBeforeOurs() {
try {
Object discoveredProcessors =
javacProcessingEnvironment_discoveredProcs.get(this.javacProcessingEnv);
ArrayList<?> states = (ArrayList<?>) discoveredProcessors_procStateList.get(discoveredProcessors);
if (states == null || states.isEmpty()) return null;
if (states.size() == 1) return processorState_processor.get(states.get(0)).getClass().getName();
int idx = 0;
StringBuilder out = new StringBuilder();
for (Object processState : states) {
idx++;
String name = processorState_processor.get(processState).getClass().getName();
if (out.length() > 0) out.append(", ");
out.append("[").append(idx).append("] ").append(name);
}
return out.toString();
} catch (Exception e) {
return null;
}
}
private void placePostCompileAndDontMakeForceRoundDummiesHook() {
stopJavacProcessingEnvironmentFromClosingOurClassloader();
forceMultipleRoundsInNetBeansEditor();
Context context = javacProcessingEnv.getContext();
disablePartialReparseInNetBeansEditor(context);
try {
Method keyMethod = Permit.getMethod(Context.class, "key", Class.class);
Object key = Permit.invoke(keyMethod, context, JavaFileManager.class);
Field htField = Permit.getField(Context.class, "ht");
@SuppressWarnings("unchecked")
Map<Object,Object> ht = (Map<Object,Object>) Permit.get(htField, context);
final JavaFileManager originalFiler = (JavaFileManager) ht.get(key);
if (!(originalFiler instanceof InterceptingJavaFileManager)) {
final Messager messager = processingEnv.getMessager();
DiagnosticsReceiver receiver = new MessagerDiagnosticsReceiver(messager);
JavaFileManager newFilerManager = new InterceptingJavaFileManager(originalFiler, receiver);
```

```
ht.put(key, newFilerManager);
Field filerFileManagerField = Permit.getField(JavacFiler.class, "fileManager");
filerFileManagerField.set(javacFiler, newFilerManager);
if (lombok.javac.Javac.getJavaCompilerVersion() > 8
&& !lombok.javac.handlers.JavacHandlerUtil.inNetbeansCompileOnSave(context)) {
replaceFileManagerJdk9(context, newFilerManager);
}
}
} catch (Exception e) {
throw Lombok.sneakyThrow(e);
}
}
private void replaceFileManagerJdk9(Context context, JavaFileManager newFiler) {
try {
JavaCompiler compiler = (JavaCompiler) Permit.invoke(Permit.getMethod(JavaCompiler.class,
"instance", Context.class), null, context);
try {
Field fileManagerField = Permit.getField(JavaCompiler.class, "fileManager");
Permit.set(fileManagerField, compiler, newFiler);
}
catch (Exception e) {}
try {
Field writerField = Permit.getField(JavaCompiler.class, "writer");
ClassWriter writer = (ClassWriter) writerField.get(compiler);
Field fileManagerField = Permit.getField(ClassWriter.class, "fileManager");
Permit.set(fileManagerField, writer, newFiler);
}
catch (Exception e) {}
}
catch (Exception e) {
}
}
private void forceMultipleRoundsInNetBeansEditor() {
try {
Field f = Permit.getField(JavacProcessingEnvironment.class, "isBackgroundCompilation");
f.set(javacProcessingEnv, true);
} catch (NoSuchFieldException e) {
// only NetBeans has it
} catch (Throwable t) {
throw Lombok.sneakyThrow(t);
}
}
private void disablePartialReparseInNetBeansEditor(Context context) {
try {
Class<?> cancelServiceClass = Class.forName("com.sun.tools.javac.util.CancelService");
Method cancelServiceInstance = Permit.getMethod(cancelServiceClass, "instance", Context.class);
Object cancelService = Permit.invoke(cancelServiceInstance, null, context);
if (cancelService == null) return;
Field parserField = Permit.getField(cancelService.getClass(), "parser");
Object parser = parserField.get(cancelService);
Field supportsReparseField = Permit.getField(parser.getClass(), "supportsReparse");
supportsReparseField.set(parser, false);
} catch (ClassNotFoundException e) {
```

```
// only NetBeans has it
} catch (NoSuchFieldException e) {
// only NetBeans has it
} catch (Throwable t) {
throw Lombok.sneakyThrow(t);
}
}
private static ClassLoader wrapClassLoader(final ClassLoader parent) {
return new ClassLoader() {
public Class<?> loadClass(String name) throws ClassNotFoundException {
return parent.loadClass(name);
}
public String toString() {
return parent.toString();
}
public URL getResource(String name) {
return parent.getResource(name);
}
public Enumeration<URL> getResources(String name) throws IOException {
return parent.getResources(name);
}
public InputStream getResourceAsStream(String name) {
return parent.getResourceAsStream(name);
}
public void setDefaultAssertionStatus(boolean enabled) {
parent.setDefaultAssertionStatus(enabled);
}
public void setPackageAssertionStatus(String packageName, boolean enabled) {
parent.setPackageAssertionStatus(packageName, enabled);
}
public void setClassAssertionStatus(String className, boolean enabled) {
parent.setClassAssertionStatus(className, enabled);
}
public void clearAssertionStatus() {
parent.clearAssertionStatus();
}
};
}
private void stopJavacProcessingEnvironmentFromClosingOurClassloader() {
try {
Field f = Permit.getField(JavacProcessingEnvironment.class, "processorClassLoader");
ClassLoader unwrapped = (ClassLoader) f.get(javacProcessingEnv);
if (unwrapped == null) return;
ClassLoader wrapped = wrapClassLoader(unwrapped);
f.set(javacProcessingEnv, wrapped);
} catch (NoSuchFieldException e) {
// Some versions of javac have this (and call close on it), some don't. I guess this one doesn't have it.
} catch (Throwable t) {
throw Lombok.sneakyThrow(t);
}
}
private final IdentityHashMap<JCCompilationUnit, Long> roots = new
IdentityHashMap<JCCompilationUnit, Long>();
```

```
private long[] priorityLevels;
private Set<Long> priorityLevelsRequiringResolutionReset;
private CleanupRegistry cleanup = new CleanupRegistry();
/** {@inheritDoc} */
@Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment
roundEnv) {
if (lombokDisabled) return false;
if (roundEnv.processingOver()) {
cleanup.run();
return false;
}
// We have: A sorted set of all priority levels: 'priorityLevels'
// Step 1: Take all CUs which aren't already in the map. Give them the first priority level.
for (Element element : roundEnv.getRootElements()) {
JCCompilationUnit unit = toUnit(element);
if (unit == null) continue;
if (roots.containsKey(unit)) continue;
roots.put(unit, priorityLevels[0]);
}
while (true) {
// Step 2: For all CUs (in the map, not the roundEnv!), run them across all handlers at their current prio
level.
for (long prio : priorityLevels) {
List<JCCompilationUnit> cusForThisRound = new ArrayList<JCCompilationUnit>();
for (Map.Entry<JCCompilationUnit, Long> entry : roots.entrySet()) {
Long prioOfCu = entry.getValue();
if (prioOfCu == null || prioOfCu != prio) continue;
cusForThisRound.add(entry.getKey());
}
transformer.transform(prio, javacProcessingEnv.getContext(), cusForThisRound, cleanup);
}
// Step 3: Push up all CUs to the next level. Set level to null if there is no next level.
Set<Long> newLevels = new HashSet<Long>();
for (int i = priorityLevels.length - 1; i >= 0; i--) {
Long curLevel = priorityLevels[i];
Long nextLevel = (i == priorityLevels.length - 1) ? null : priorityLevels[i + 1];
List<JCCompilationUnit> cusToAdvance = new ArrayList<JCCompilationUnit>();
for (Map.Entry<JCCompilationUnit, Long> entry : roots.entrySet()) {
if (curLevel.equals(entry.getValue())) {
cusToAdvance.add(entry.getKey());
newLevels.add(nextLevel);
}
}
for (JCCompilationUnit unit : cusToAdvance) {
roots.put(unit, nextLevel);
}
}
newLevels.remove(null);
// Step 4: If ALL values are null, quit. Else, either do another loop right now or force a resolution reset by
forcing a new round in the annotation processor.
if (newLevels.isEmpty()) return false;
newLevels.retainAll(priorityLevelsRequiringResolutionReset);
if (!newLevels.isEmpty()) {
```

```
// Force a new round to reset resolution. The next round will cause this method (process) to be called again.
forceNewRound(javacFiler);
return false;
}
// None of the new levels need resolution, so just keep going.
}
}
private int dummyCount = 0;
private void forceNewRound(JavacFiler filer) {
if (!filer.newFiles()) {
try {
filer.getGeneratedSourceNames().add("lombok.dummy.ForceNewRound" + (dummyCount++));
} catch (Exception e) {
e.printStackTrace();
processingEnv.getMessager().printMessage(Kind.WARNING,
"Can't force a new processing round. Lombok won't work.");
}
}
}
private JCCompilationUnit toUnit(Element element) {
TreePath path = null;
if (trees != null) {
try {
path = trees.getPath(element);
} catch (NullPointerException ignore) {
// Happens if a package-info.java doesn't contain a package declaration.
// https://github.com/projectlombok/lombok/issues/2184
// We can safely ignore those, since they do not need any processing
}
}
if (path == null) return null;
return (JCCompilationUnit) path.getCompilationUnit();
}
/**
* We just return the latest version of whatever JDK we run on. Stupid? Yeah, but it's either that or warnings on all versions but 1.
*/
@Override public SourceVersion getSupportedSourceVersion() {
return SourceVersion.latest();
}
/**
* This class casts the given processing environment to a JavacProcessingEnvironment. In case of
* gradle incremental compilation, the delegate ProcessingEnvironment of the gradle wrapper is returned.
*/
public JavacProcessingEnvironment getJavacProcessingEnvironment(Object procEnv) {
addOpensForLombok();
if (procEnv instanceof JavacProcessingEnvironment) return (JavacProcessingEnvironment) procEnv;
// try to find a "delegate" field in the object, and use this to try to obtain a JavacProcessingEnvironment
for (Class<?> procEnvClass = procEnv.getClass(); procEnvClass != null; procEnvClass = procEnvClass.getSuperclass()) {
Object delegate = tryGetDelegateField(procEnvClass, procEnv);
```

```
if (delegate == null) delegate = tryGetProxyDelegateToField(procEnvClass, procEnv);
if (delegate == null) delegate = tryGetProcessingEnvField(procEnvClass, procEnv);
if (delegate != null) return getJavacProcessingEnvironment(delegate);
// delegate field was not found, try on superclass
}
processingEnv.getMessager().printMessage(Kind.WARNING,
"Can't get the delegate of the gradle IncrementalProcessingEnvironment. Lombok won't work.");
return null;
}
private static Object getOwnModule() {
try {
Method m = Permit.getMethod(Class.class, "getModule");
return m.invoke(LombokProcessor.class);
} catch (Exception e) {
return null;
}
}
private static Object getJdkCompilerModule() {
/* call public api: ModuleLayer.boot().findModule("jdk.compiler").get();
but use reflection because we don't want this code to crash on jdk1.7 and below.
In that case, none of this stuff was needed in the first place, so we just exit via
the catch block and do nothing.
*/
try {
Class<?> cModuleLayer = Class.forName("java.lang.ModuleLayer");
Method mBoot = cModuleLayer.getDeclaredMethod("boot");
Object bootLayer = mBoot.invoke(null);
Class<?> cOptional = Class.forName("java.util.Optional");
Method mFindModule = cModuleLayer.getDeclaredMethod("findModule", String.class);
Object oCompilerO = mFindModule.invoke(bootLayer, "jdk.compiler");
return cOptional.getDeclaredMethod("get").invoke(oCompilerO);
} catch (Exception e) {
return null;
}
}
/** Useful from jdk9 and up; required from jdk16 and up. This code is supposed to gracefully do nothing
on jdk8 and below, as this operation isn't needed there. */
public static void addOpensForLombok() {
Class<?> cModule;
try {
cModule = Class.forName("java.lang.Module");
} catch (ClassNotFoundException e) {
return; //jdk8-; this is not needed.
}
Unsafe unsafe = getUnsafe();
Object jdkCompilerModule = getJdkCompilerModule();
Object ownModule = getOwnModule();
String[] allPkgs = {
"com.sun.tools.javac.code",
"com.sun.tools.javac.comp",
"com.sun.tools.javac.file",
"com.sun.tools.javac.main",
"com.sun.tools.javac.model",
```

```
"com.sun.tools.javac.parser",
"com.sun.tools.javac.processing",
"com.sun.tools.javac.tree",
"com.sun.tools.javac.util",
"com.sun.tools.javac.jvm",
};
try {
Method m = cModule.getDeclaredMethod("implAddOpens", String.class, cModule);
long firstFieldOffset = getFirstFieldOffset(unsafe);
unsafe.putBooleanVolatile(m, firstFieldOffset, true);
for (String p : allPkgs) m.invoke(jdkCompilerModule, p, ownModule);
} catch (Exception ignore) {}
}
private static long getFirstFieldOffset(Unsafe unsafe) {
try {
return unsafe.objectFieldOffset(Parent.class.getDeclaredField("first"));
} catch (NoSuchFieldException e) {
// can't happen.
throw new RuntimeException(e);
} catch (SecurityException e) {
// can't happen
throw new RuntimeException(e);
}
}
private static Unsafe getUnsafe() {
try {
Field theUnsafe = Unsafe.class.getDeclaredField("theUnsafe");
theUnsafe.setAccessible(true);
return (Unsafe) theUnsafe.get(null);
} catch (Exception e) {
return null;
}
}
/**
* This class returns the given filer as a JavacFiler. In case the filer is no
* JavacFiler (e.g. the Gradle IncrementalFiler), its "delegate" field is used to get the JavacFiler
* (directly or through a delegate field again)
*/
public JavacFiler getJavacFiler(Object filer) {
if (filer instanceof JavacFiler) return (JavacFiler) filer;
// try to find a "delegate" field in the object, and use this to check for a JavacFiler
for (Class<?> filerClass = filer.getClass(); filerClass != null; filerClass = filerClass.getSuperclass()) {
Object delegate = tryGetDelegateField(filerClass, filer);
if (delegate == null) delegate = tryGetProxyDelegateToField(filerClass, filer);
if (delegate == null) delegate = tryGetFilerField(filerClass, filer);
if (delegate != null) return getJavacFiler(delegate);
// delegate field was not found, try on superclass
}
processingEnv.getMessager().printMessage(Kind.WARNING,
"Can't get a JavacFiler from " + filer.getClass().getName() + ". Lombok won't work.");
return null;
}
/**
```

```
* Gradle incremental processing
*/
private Object tryGetDelegateField(Class<?> delegateClass, Object instance) {
try {
return Permit.getField(delegateClass, "delegate").get(instance);
} catch (Exception e) {
return null;
}
}
/**
* Kotlin incremental processing
*/
private Object tryGetProcessingEnvField(Class<?> delegateClass, Object instance) {
try {
return Permit.getField(delegateClass, "processingEnv").get(instance);
} catch (Exception e) {
return null;
}
}
/**
* Kotlin incremental processing
*/
private Object tryGetFilerField(Class<?> delegateClass, Object instance) {
try {
return Permit.getField(delegateClass, "filer").get(instance);
} catch (Exception e) {
return null;
}
}
/**
* IntelliJ IDEA >= 2020.3
*/
private Object tryGetProxyDelegateToField(Class<?> delegateClass, Object instance) {
try {
InvocationHandler handler = Proxy.getInvocationHandler(instance);
return Permit.getField(handler.getClass(), "val$delegateTo").get(handler);
} catch (Exception e) {
return null;
}
}
}
```

lombok/javac/apt/MessagerDiagnosticsReceiver.java

```
/*
* Copyright (C) 2009-2010 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
```

* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import javax.annotation.processing.Messager;
import javax.tools.Diagnostic.Kind;
import lombok.core.DiagnosticsReceiver;
public class MessagerDiagnosticsReceiver implements DiagnosticsReceiver {
private final Messager messager;
public MessagerDiagnosticsReceiver(Messager messager) {
this.messager = messager;
}
@Override public void addWarning(String message) {
messager.printMessage(Kind.WARNING, message);
}
@Override public void addError(String message) {
messager.printMessage(Kind.ERROR, message);
}
}

lombok/javac/apt/Processor.java

/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,

```
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.apt;
import static javax.tools.StandardLocation.*;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.StringWriter;
import java.lang.reflect.Field;
import java.net.URL;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Enumeration;
import java.util.Map;
import java.util.Properties;
import java.util.Set;
import javax.annotation.processing.AbstractProcessor;
import javax.annotation.processing.Filer;
import javax.annotation.processing.ProcessingEnvironment;
import javax.annotation.processing.RoundEnvironment;
import javax.annotation.processing.SupportedAnnotationTypes;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.TypeElement;
import javax.tools.Diagnostic;
import javax.tools.JavaFileManager;
import com.sun.tools.javac.processing.JavacFiler;
import com.sun.tools.javac.processing.JavacProcessingEnvironment;
import com.sun.tools.javac.util.Options;
import lombok.permit.Permit;
/**
* This processor should not be used. It used to be THE processor. This class is only there to warn
people that something went wrong, and for the
* lombok developers to see if what the reason for those failures is.
*/
@Deprecated
@SupportedAnnotationTypes("*")
public class Processor extends AbstractProcessor {
/** {@inheritDoc} */
@Override public void init(ProcessingEnvironment procEnv) {
super.init(procEnv);
if (System.getProperty("lombok.disable") != null) {
return;
}
procEnv.getMessager().printMessage(Diagnostic.Kind.WARNING, "Wrong usage of
'lombok.javac.apt.Processor'. " + report(procEnv));
}
private String report(ProcessingEnvironment procEnv) {
String data = collectData(procEnv);
try {
```

```
return writeFile(data);
} catch (Exception e) {
return "Report:\n\n" + data;
}
}
private String writeFile(String data) throws IOException {
File file = File.createTempFile("lombok-processor-report-", ".txt");
OutputStreamWriter writer = new OutputStreamWriter(new FileOutputStream(file));
writer.write(data);
writer.close();
return "Report written to '" + file.getCanonicalPath() + "'\n";
}
private String collectData(ProcessingEnvironment procEnv) {
StringBuilder message = new StringBuilder();
message.append("Problem report for usages of 'lombok.javac.apt.Processor'\n\n");
listOptions(message, procEnv);
findServices(message, procEnv.getFiler());
addStacktrace(message);
listProperties(message);
return message.toString();
}
private void listOptions(StringBuilder message, ProcessingEnvironment procEnv) {
try {
JavacProcessingEnvironment environment = (JavacProcessingEnvironment) procEnv;
Options instance = Options.instance(environment.getContext());
Field field = Permit.getField(Options.class, "values");
@SuppressWarnings("unchecked") Map<String, String> values = (Map<String, String>)
field.get(instance);
if (values.isEmpty()) {
message.append("Options: empty\n\n");
return;
}
message.append("Compiler Options:\n");
for (Map.Entry<String, String> value : values.entrySet()) {
message.append("- ");
string(message, value.getKey());
message.append(" = ");
string(message, value.getValue());
message.append("\n");
}
message.append("\n");
} catch (Exception e) {
message.append("No options available\n\n");
}
}
private void findServices(StringBuilder message, Filer filer) {
try {
Field filerFileManagerField = Permit.getField(JavacFiler.class, "fileManager");
JavaFileManager jfm = (JavaFileManager) filerFileManagerField.get(filer);
ClassLoader processorClassLoader = jfm.hasLocation(ANNOTATION_PROCESSOR_PATH) ?
jfm.getClassLoader(ANNOTATION_PROCESSOR_PATH) : jfm.getClassLoader(CLASS_PATH);
Enumeration<URL> resources =
processorClassLoader.getResources("META-INF/services/javax.annotation.processing.Processor");
```

```
if (!resources.hasMoreElements()) {
message.append("No processors discovered\n\n");
return;
}
message.append("Discovered processors:\n");
while (resources.hasMoreElements()) {
URL processorUrl = resources.nextElement();
message.append("- '").append(processorUrl).append("'");
InputStream content = (InputStream) processorUrl.getContent();
if (content != null) try {
InputStreamReader reader = new InputStreamReader(content, "UTF-8");
StringWriter sw = new StringWriter();
char[] buffer = new char[8192];
int read = 0;
while ((read = reader.read(buffer))!= -1) {
sw.write(buffer, 0, read);
}
String wholeFile = sw.toString();
if (wholeFile.contains("lombok.javac.apt.Processor")) {
message.append(" <= problem\n");
} else {
message.append(" (ok)\n");
}
message.append(" ").append(wholeFile.replace("\n", "\n ")).append("\n");
} finally {
content.close();
}
}
} catch (Exception e) {
message.append("Filer information unavailable\n");
}
message.append("\n");
}
private void addStacktrace(StringBuilder message) {
StackTraceElement[] stackTraceElements = Thread.currentThread().getStackTrace();
if (stackTraceElements != null) {
message.append("Called from\n");
for (int i = 1; i < stackTraceElements.length; i++) {
StackTraceElement element = stackTraceElements[i];
if (!element.getClassName().equals("lombok.javac.apt.Processor")) message.append("-
").append(element).append("\n");
}
} else {
message.append("No stacktrace available\n");
}
message.append("\n");
}
private void listProperties(StringBuilder message) {
Properties properties = System.getProperties();
ArrayList<String> propertyNames = new ArrayList<String>(properties.stringPropertyNames());
Collections.sort(propertyNames);
message.append("Properties: \n");
for (String propertyName : propertyNames) {
```

```
if (propertyName.startsWith("user.")) continue;
message.append("- ").append(propertyName).append(" = ");
string(message, System.getProperty(propertyName));
message.append("\n");
}
message.append("\n");
}
private static void string(StringBuilder sb, String s) {
if (s == null) {
sb.append("null");
return;
}
sb.append("\"");
for (int i = 0; i < s.length(); i++) sb.append(escape(s.charAt(i)));
sb.append("\"");
}
private static String escape(char ch) {
switch (ch) {
case '\b': return "\\b";
case '\f': return "\\f";
case '\n': return "\\n";
case '\r': return "\\r";
case '\t': return "\\t";
case '\'': return "\\'";
case '\"': return "\\\"";
case '\\': return "\\\\";
default:
if (ch < 32) return String.format("\\%03o", (int) ch);
return String.valueOf(ch);
}
}
/**
* We just return the latest version of whatever JDK we run on. Stupid? Yeah, but it's either that or warnings on all versions but 1.
*/
@Override public SourceVersion getSupportedSourceVersion() {
return SourceVersion.latest();
}
@Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment roundEnv) {
return false;
}
}
```

lombok/javac/apt/package-info.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
```

```
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* Contains the mechanism that instruments javac as an annotation processor.
*
* <strong>NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.</strong>
*/
package lombok.javac.apt;
```

lombok/javac/handlers/HandleAccessors.java

```
/*
* Copyright (C) 2012-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
```

```
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.experimental.Accessors;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
@Provides
@HandlerPriority(65536)
public class HandleAccessors extends JavacAnnotationHandler<Accessors> {
@Override public void handle(AnnotationValues<Accessors> annotation, JCAnnotation ast,
JavacNode annotationNode) {
// Accessors itself is handled by HandleGetter/Setter; this is just to ensure that the annotation is
removed
// from the AST when delomboking.
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.ACCESSORS_FLAG_USAGE,
"@Accessors");
deleteAnnotationIfNeccessary(annotationNode, Accessors.class);
if (annotation.isMarking()) annotationNode.addWarning("Accessors on its own does nothing. Set at
least one parameter");
}
}
```

lombok/javac/handlers/HandleBuilder.java

```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.*;
import static lombok.javac.JavacTreeMaker.TypeTag.typeTag;
import static lombok.javac.handlers.JavacHandlerUtil.*;
```

```
import java.util.ArrayList;
import javax.lang.model.element.Modifier;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCIf;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCNewArray;
import com.sun.tools.javac.tree.JCTree.JCNewClass;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeApply;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.AccessLevel;
import lombok.Builder;
import lombok.Builder.ObtainVia;
import lombok.ConfigurationKeys;
import lombok.Singular;
import lombok.ToString;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.experimental.NonFinal;
import lombok.javac.Javac;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.HandleConstructor.SkipIfConstructorExists;
import lombok.javac.handlers.JavacHandlerUtil.CopyJavadoc;
import lombok.javac.handlers.JavacHandlerUtil.MemberExistsResult;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
import lombok.spi.Provides;
@Provides
```

```
@HandlerPriority(-1024) //-2^10; to ensure we've picked up @FieldDefault's changes (-2048) but
@Value hasn't removed itself yet (-512), so that we can error on presence of it on the builder classes.
public class HandleBuilder extends JavacAnnotationHandler<Builder> {
private HandleConstructor handleConstructor = new HandleConstructor();
static final String CLEAN_FIELD_NAME = "$lombokUnclean";
static final String CLEAN_METHOD_NAME = "$lombokClean";
static final String TO_BUILDER_METHOD_NAME = "toBuilder";
static final String DEFAULT_PREFIX = "$default$";
static final String SET_PREFIX = "$set";
static final String VALUE_PREFIX = "$value";
static final String BUILDER_TEMP_VAR = "builder";
static final String TO_BUILDER_NOT_SUPPORTED = "@Builder(toBuilder=true) is only supported if
you return your own type.";
private static final boolean toBoolean(Object expr, boolean defaultValue) {
if (expr == null) return defaultValue;
if (expr instanceof JCLiteral) return ((Integer) ((JCLiteral) expr).value) != 0;
return ((Boolean) expr).booleanValue();
}
static class BuilderJob {
CheckerFrameworkVersion checkerFramework;
JavacNode parentType;
String builderMethodName, buildMethodName;
boolean isStatic;
List<JCTypeParameter> typeParams;
List<JCTypeParameter> builderTypeParams;
JavacNode sourceNode;
java.util.List<BuilderFieldData> builderFields;
AccessLevel accessInners, accessOuters;
boolean oldFluent, oldChain, toBuilder;
JavacNode builderType;
String builderClassName;
void init(AnnotationValues<Builder> annValues, Builder ann, JavacNode node) {
accessOuters = ann.access();
if (accessOuters == null) accessOuters = AccessLevel.PUBLIC;
if (accessOuters == AccessLevel.NONE) {
sourceNode.addError("AccessLevel.NONE is not valid here");
accessOuters = AccessLevel.PUBLIC;
}
accessInners = accessOuters == AccessLevel.PROTECTED ? AccessLevel.PUBLIC : accessOuters;
oldFluent = toBoolean(annValues.getActualExpression("fluent"), true);
oldChain = toBoolean(annValues.getActualExpression("chain"), true);
builderMethodName = ann.builderMethodName();
buildMethodName = ann.buildMethodName();
builderClassName = getBuilderClassNameTemplate(node, ann.builderClassName());
toBuilder = ann.toBuilder();
if (builderMethodName == null) builderMethodName = "builder";
if (buildMethodName == null) buildMethodName = "build";
if (builderClassName == null) builderClassName = "";
}
static String getBuilderClassNameTemplate(JavacNode node, String override) {
if (override != null && !override.isEmpty()) return override;
override = node.getAst().readConfiguration(ConfigurationKeys.BUILDER_CLASS_NAME);
if (override != null && !override.isEmpty()) return override;
```

```
return "*Builder";
}
String replaceBuilderClassName(Name name) {
return replaceBuilderClassName(name.toString(), builderClassName);
}
String replaceBuilderClassName(String name, String template) {
if (template.indexOf('*') == -1) return template;
return template.replace("*", name);
}
JCExpression createBuilderParentTypeReference() {
return namePlusTypeParamsToTypeReference(parentType.getTreeMaker(), parentType,
typeParams);
}
Name getBuilderClassName() {
return parentType.toName(builderClassName);
}
List<JCTypeParameter> copyTypeParams() {
return JavacHandlerUtil.copyTypeParams(sourceNode, typeParams);
}
Name toName(String name) {
return parentType.toName(name);
}
Context getContext() {
return parentType.getContext();
}
JavacTreeMaker getTreeMaker() {
return parentType.getTreeMaker();
}
}
static class BuilderFieldData {
List<JCAnnotation> annotations;
JCExpression type;
Name rawName;
Name name;
Name builderFieldName;
Name nameOfDefaultProvider;
Name nameOfSetFlag;
SingularData singularData;
ObtainVia obtainVia;
JavacNode obtainViaNode;
JavacNode originalFieldNode;
java.util.List<JavacNode> createdFields = new ArrayList<JavacNode>();
}
@Override public void handle(AnnotationValues<Builder> annotation, JCAnnotation ast, JavacNode
annotationNode) {
final String BUILDER_NODE_NOT_SUPPORTED_ERR = "@Builder is only supported on classes,
records, constructors, and methods.";
handleFlagUsage(annotationNode, ConfigurationKeys.BUILDER_FLAG_USAGE, "@Builder");
BuilderJob job = new BuilderJob();
job.sourceNode = annotationNode;
job.checkerFramework = getCheckerFrameworkVersion(annotationNode);
job.isStatic = true;
Builder annInstance = annotation.getInstance();
```

```
job.init(annotation, annInstance, annotationNode);
java.util.List<Name> typeArgsForToBuilder = null;
boolean generateBuilderMethod;
if (job.builderMethodName.isEmpty()) {
generateBuilderMethod = false;
} else if (!checkName("builderMethodName", job.builderMethodName, annotationNode)) {
return;
} else {
generateBuilderMethod = true;
}
if (!checkName("buildMethodName", job.buildMethodName, annotationNode)) return;
// Do not delete the Builder annotation yet, we need it for @Jacksonized.
JavacNode parent = annotationNode.up();
job.builderFields = new ArrayList<BuilderFieldData>();
JCExpression buildMethodReturnType;
job.typeParams = List.nil();
List<JCExpression> buildMethodThrownExceptions;
Name nameOfBuilderMethod;
JavacNode fillParametersFrom = parent.get() instanceof JCMethodDecl ? parent : null;
boolean addCleaning = false;
ArrayList<JavacNode> nonFinalNonDefaultedFields = null;
if (!isStaticAllowed(upToTypeNode(parent))) {
annotationNode.addError("@Builder is not supported on non-static nested classes.");
return;
}
if (parent.get() instanceof JCClassDecl) {
if (!isClass(parent) && !isRecord(parent)) {
annotationNode.addError(BUILDER_NODE_NOT_SUPPORTED_ERR);
return;
}
job.parentType = parent;
JCClassDecl td = (JCClassDecl) parent.get();
ListBuffer<JavacNode> allFields = new ListBuffer<JavacNode>();
boolean valuePresent = (hasAnnotation(lombok.Value.class, parent) ||
hasAnnotation("lombok.experimental.Value", parent));
for (JavacNode fieldNode : HandleConstructor.findAllFields(parent, true)) {
JCVariableDecl fd = (JCVariableDecl) fieldNode.get();
JavacNode isDefault = findAnnotation(Builder.Default.class, fieldNode, false);
boolean isFinal = (fd.mods.flags & Flags.FINAL) != 0 || (valuePresent &&
!hasAnnotation(NonFinal.class, fieldNode));
BuilderFieldData bfd = new BuilderFieldData();
bfd.rawName = fd.name;
bfd.name = removePrefixFromField(fieldNode);
bfd.builderFieldName = bfd.name;
bfd.annotations = findCopyableAnnotations(fieldNode);
bfd.type = fd.vartype;
bfd.singularData = getSingularData(fieldNode, annInstance.setterPrefix());
bfd.originalFieldNode = fieldNode;
if (bfd.singularData != null && isDefault != null) {
isDefault.addError("@Builder.Default and @Singular cannot be mixed.");
findAnnotation(Builder.Default.class, fieldNode, true);
isDefault = null;
}
```

```
if (fd.init == null && isDefault != null) {
isDefault.addWarning("@Builder.Default requires an initializing expression (' = something;').");
findAnnotation(Builder.Default.class, fieldNode, true);
isDefault = null;
}
if (fd.init != null && isDefault == null) {
if (isFinal) continue;
if (nonFinalNonDefaultedFields == null) nonFinalNonDefaultedFields = new ArrayList<JavacNode>();
nonFinalNonDefaultedFields.add(fieldNode);
}
if (isDefault != null) {
bfd.nameOfDefaultProvider = parent.toName(DEFAULT_PREFIX + bfd.name);
bfd.nameOfSetFlag = parent.toName(bfd.name + SET_PREFIX);
bfd.builderFieldName = parent.toName(bfd.name + VALUE_PREFIX);
JCMethodDecl md = generateDefaultProvider(bfd.nameOfDefaultProvider, fieldNode, td.typarams,
job);
if (md != null) injectMethod(parent, md);
}
addObtainVia(bfd, fieldNode);
job.builderFields.add(bfd);
allFields.append(fieldNode);
}
if (!isRecord(parent)) {
// Records ship with a canonical constructor that acts as @AllArgsConstructor - just use that one.
handleConstructor.generateConstructor(parent, AccessLevel.PACKAGE, List.<JCAnnotation>nil(),
allFields.toList(), false, null, SkipIfConstructorExists.I_AM_BUILDER, annotationNode);
}
buildMethodReturnType = namePlusTypeParamsToTypeReference(parent.getTreeMaker(), parent,
td.typarams);
job.typeParams = job.builderTypeParams = td.typarams;
buildMethodThrownExceptions = List.nil();
nameOfBuilderMethod = null;
job.builderClassName = job.replaceBuilderClassName(td.name);
if (!checkName("builderClassName", job.builderClassName, annotationNode)) return;
} else if (fillParametersFrom != null && fillParametersFrom.getName().toString().equals("<init>")) {
JCMethodDecl jmd = (JCMethodDecl) fillParametersFrom.get();
if (!jmd.typarams.isEmpty()) {
annotationNode.addError("@Builder is not supported on constructors with constructor type
parameters.");
return;
}
job.parentType = parent.up();
JCClassDecl td = (JCClassDecl) job.parentType.get();
job.typeParams = job.builderTypeParams = td.typarams;
buildMethodReturnType = job.createBuilderParentTypeReference();
buildMethodThrownExceptions = jmd.thrown;
nameOfBuilderMethod = null;
job.builderClassName = job.replaceBuilderClassName(td.name);
if (!checkName("builderClassName", job.builderClassName, annotationNode)) return;
} else if (fillParametersFrom != null) {
job.parentType = parent.up();
JCClassDecl td = (JCClassDecl) job.parentType.get();
JCMethodDecl jmd = (JCMethodDecl) fillParametersFrom.get();
```

```
job.isStatic = (jmd.mods.flags & Flags.STATIC) != 0;
JCExpression fullReturnType = jmd.restype;
buildMethodReturnType = fullReturnType;
job.typeParams = job.builderTypeParams = jmd.typarams;
buildMethodThrownExceptions = jmd.thrown;
nameOfBuilderMethod = jmd.name;
if (buildMethodReturnType instanceof JCTypeApply) {
buildMethodReturnType = cloneType(job.getTreeMaker(), buildMethodReturnType, annotationNode);
}
if (job.builderClassName.indexOf('*') > -1) {
String replStr = returnTypeToBuilderClassName(annotationNode, td, buildMethodReturnType,
job.typeParams);
if (replStr == null) return; // shuold not happen
job.builderClassName = job.builderClassName.replace("*", replStr);
}
if (job.toBuilder) {
if (fullReturnType instanceof JCArrayTypeTree) {
annotationNode.addError(TO_BUILDER_NOT_SUPPORTED);
return;
}
Name simpleName;
String pkg;
List<JCExpression> tpOnRet = List.nil();
if (fullReturnType instanceof JCTypeApply) {
tpOnRet = ((JCTypeApply) fullReturnType).arguments;
}
JCExpression namingType = fullReturnType;
if (buildMethodReturnType instanceof JCTypeApply) namingType = ((JCTypeApply)
buildMethodReturnType).clazz;
if (namingType instanceof JCIdent) {
simpleName = ((JCIdent) namingType).name;
pkg = null;
} else if (namingType instanceof JCFieldAccess) {
JCFieldAccess jcfa = (JCFieldAccess) namingType;
simpleName = jcfa.name;
pkg = unpack(jcfa.selected);
if (pkg.startsWith("ERR:")) {
String err = pkg.substring(4, pkg.indexOf("__ERR__"));
annotationNode.addError(err);
return;
}
} else {
annotationNode.addError("Expected a (parameterized) type here instead of a " +
namingType.getClass().getName());
return;
}
if (pkg != null && !parent.getPackageDeclaration().equals(pkg)) {
annotationNode.addError(TO_BUILDER_NOT_SUPPORTED);
return;
}
if (!job.parentType.getName().contentEquals(simpleName)) {
annotationNode.addError(TO_BUILDER_NOT_SUPPORTED);
return;
```

```
}
List<JCTypeParameter> tpOnMethod = jmd.typarams;
List<JCTypeParameter> tpOnType = ((JCClassDecl) job.parentType.get()).typarams;
typeArgsForToBuilder = new ArrayList<Name>();
for (JCTypeParameter tp : tpOnMethod) {
int pos = -1;
int idx = -1;
for (JCExpression tOnRet : tpOnRet) {
idx++;
if (!(tOnRet instanceof JCIdent)) continue;
if (((JCIdent) tOnRet).name != tp.name) continue;
pos = idx;
}
if (pos == -1 || tpOnType.size() <= pos) {
annotationNode.addError("@Builder(toBuilder=true) requires that each type parameter on the static
method is part of the typeargs of the return value. Type parameter " + tp.name + " is not part of the
return type.");
return;
}
typeArgsForToBuilder.add(tpOnType.get(pos).name);
}
}
} else {
annotationNode.addError(BUILDER_NODE_NOT_SUPPORTED_ERR);
return;
}
if (fillParametersFrom != null) {
for (JavacNode param : fillParametersFrom.down()) {
if (param.getKind() != Kind.ARGUMENT) continue;
BuilderFieldData bfd = new BuilderFieldData();
JCVariableDecl raw = (JCVariableDecl) param.get();
bfd.name = raw.name;
bfd.builderFieldName = bfd.name;
bfd.rawName = raw.name;
bfd.annotations = findCopyableAnnotations(param);
bfd.type = raw.vartype;
bfd.singularData = getSingularData(param, annInstance.setterPrefix());
bfd.originalFieldNode = param;
addObtainVia(bfd, param);
job.builderFields.add(bfd);
}
}
job.builderType = findInnerClass(job.parentType, job.builderClassName);
if (job.builderType == null) {
job.builderType = makeBuilderClass(job);
recursiveSetGeneratedBy(job.builderType.get(), annotationNode);
} else {
JCClassDecl builderTypeDeclaration = (JCClassDecl) job.builderType.get();
if (job.isStatic && !builderTypeDeclaration.getModifiers().getFlags().contains(Modifier.STATIC)) {
annotationNode.addError("Existing Builder must be a static inner class.");
return;
} else if (!job.isStatic && builderTypeDeclaration.getModifiers().getFlags().contains(Modifier.STATIC)) {
annotationNode.addError("Existing Builder must be a non-static inner class.");
```

```
return;
}
sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(job.builderType, annotationNode);
/* generate errors for @Singular BFDs that have one already defined node. */ {
for (BuilderFieldData bfd : job.builderFields) {
SingularData sd = bfd.singularData;
if (sd == null) continue;
JavacSingularizer singularizer = sd.getSingularizer();
if (singularizer == null) continue;
if (singularizer.checkForAlreadyExistingNodesAndGenerateError(job.builderType, sd)) {
bfd.singularData = null;
}
}
}
}
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
if (bfd.singularData.getSingularizer().requiresCleaning()) {
addCleaning = true;
break;
}
}
if (bfd.obtainVia != null) {
if (bfd.obtainVia.field().isEmpty() == bfd.obtainVia.method().isEmpty()) {
bfd.obtainViaNode.addError("The syntax is either @ObtainVia(field = \"fieldName\") or
@ObtainVia(method = \"methodName\").");
return;
}
if (bfd.obtainVia.method().isEmpty() && bfd.obtainVia.isStatic()) {
bfd.obtainViaNode.addError("@ObtainVia(isStatic = true) is not valid unless 'method' has been set.");
return;
}
}
}
generateBuilderFields(job);
if (addCleaning) {
JavacTreeMaker maker = job.getTreeMaker();
JCVariableDecl uncleanField = maker.VarDef(maker.Modifiers(Flags.PRIVATE),
job.builderType.toName(CLEAN_FIELD_NAME), maker.TypeIdent(CTC_BOOLEAN), null);
injectFieldAndMarkGenerated(job.builderType, uncleanField);
recursiveSetGeneratedBy(uncleanField, annotationNode);
}
if (constructorExists(job.builderType) == MemberExistsResult.NOT_EXISTS) {
JCMethodDecl cd = HandleConstructor.createConstructor(AccessLevel.PACKAGE,
List.<JCAnnotation>nil(), job.builderType, List.<JavacNode>nil(), false, annotationNode);
if (cd != null) injectMethod(job.builderType, cd);
}
for (BuilderFieldData bfd : job.builderFields) {
makePrefixedSetterMethodsForBuilder(job, bfd, annInstance.setterPrefix());
}
{
MemberExistsResult methodExists = methodExists(job.buildMethodName, job.builderType, -1);
```

```
if (methodExists == MemberExistsResult.EXISTS_BY_LOMBOK) methodExists =
methodExists(job.buildMethodName, job.builderType, 0);
if (methodExists == MemberExistsResult.NOT_EXISTS) {
JCMethodDecl md = generateBuildMethod(job, nameOfBuilderMethod, buildMethodReturnType,
buildMethodThrownExceptions, addCleaning);
if (md != null) {
recursiveSetGeneratedBy(md, annotationNode);
injectMethod(job.builderType, md);
}
}
}
if (methodExists("toString", job.builderType, 0) == MemberExistsResult.NOT_EXISTS) {
java.util.List<Included<JavacNode, ToString.Include>> fieldNodes = new
ArrayList<Included<JavacNode, ToString.Include>>();
for (BuilderFieldData bfd : job.builderFields) {
for (JavacNode f : bfd.createdFields) {
fieldNodes.add(new Included<JavacNode, ToString.Include>(f, null, true, false));
}
}
JCMethodDecl md = HandleToString.createToString(job.builderType, fieldNodes, true, false,
FieldAccess.ALWAYS_FIELD, job.sourceNode);
if (md != null) injectMethod(job.builderType, md);
}
if (addCleaning) injectMethod(job.builderType, generateCleanMethod(job));
if (generateBuilderMethod && methodExists(job.builderMethodName, job.parentType, -1) !=
MemberExistsResult.NOT_EXISTS) generateBuilderMethod = false;
if (generateBuilderMethod) {
JCMethodDecl md = generateBuilderMethod(job);
recursiveSetGeneratedBy(md, annotationNode);
if (md != null) injectMethod(job.parentType, md);
}
if (job.toBuilder) {
switch (methodExists(TO_BUILDER_METHOD_NAME, job.parentType, 0)) {
case EXISTS_BY_USER:
annotationNode.addWarning("Not generating toBuilder() as it already exists.");
return;
case NOT_EXISTS:
List<JCTypeParameter> tps = job.typeParams;
if (typeArgsForToBuilder != null) {
ListBuffer<JCTypeParameter> lb = new ListBuffer<JCTypeParameter>();
JavacTreeMaker maker = job.getTreeMaker();
for (Name n : typeArgsForToBuilder) {
lb.append(maker.TypeParameter(n, List.<JCExpression>nil()));
}
tps = lb.toList();
}
JCMethodDecl md = generateToBuilderMethod(job, tps, annInstance.setterPrefix());
if (md != null) {
recursiveSetGeneratedBy(md, annotationNode);
injectMethod(job.parentType, md);
}
}
}
```

```
if (nonFinalNonDefaultedFields != null && generateBuilderMethod) {
for (JavacNode fieldNode : nonFinalNonDefaultedFields) {
fieldNode.addWarning("@Builder will ignore the initializing expression entirely. If you want the
initializing expression to serve as default, add @Builder.Default. If it is not supposed to be settable
during building, make the field final.");
}
}
}
static String returnTypeToBuilderClassName(JavacNode annotationNode, JCClassDecl td,
JCExpression returnType, List<JCTypeParameter> typeParams) {
String replStr = null;
if (returnType instanceof JCFieldAccess) {
replStr = ((JCFieldAccess) returnType).name.toString();
} else if (returnType instanceof JCIdent) {
Name n = ((JCIdent) returnType).name;
for (JCTypeParameter tp : typeParams) {
if (tp.name.equals(n)) {
annotationNode.addError("@Builder requires specifying 'builderClassName' if used on methods with a
type parameter as return type.");
return null;
}
}
replStr = n.toString();
} else if (returnType instanceof JCPrimitiveTypeTree) {
replStr = returnType.toString();
if (Character.isLowerCase(replStr.charAt(0))) {
replStr = Character.toTitleCase(replStr.charAt(0)) + replStr.substring(1);
}
} else if (returnType instanceof JCTypeApply) {
JCExpression clazz = ((JCTypeApply) returnType).clazz;
if (clazz instanceof JCFieldAccess) {
replStr = ((JCFieldAccess) clazz).name.toString();
} else if (clazz instanceof JCIdent) {
replStr = ((JCIdent) clazz).name.toString();
}
}
if (replStr == null || replStr.isEmpty()) {
// This shouldn't happen.
System.err.println("Lombok bug ID#20140614-1651: javac HandleBuilder: return type to name
conversion failed: " + returnType.getClass());
replStr = td.name.toString();
}
return replStr;
}
private static String unpack(JCExpression expr) {
StringBuilder sb = new StringBuilder();
unpack(sb, expr);
return sb.toString();
}
private static void unpack(StringBuilder sb, JCExpression expr) {
if (expr instanceof JCIdent) {
sb.append(((JCIdent) expr).name.toString());
return;
```

```
}
if (expr instanceof JCFieldAccess) {
JCFieldAccess jcfa = (JCFieldAccess) expr;
unpack(sb, jcfa.selected);
sb.append(".").append(jcfa.name.toString());
return;
}
if (expr instanceof JCTypeApply) {
sb.setLength(0);
sb.append("ERR:");
sb.append("@Builder(toBuilder=true) is not supported if returning a type with generics applied to an
intermediate.");
sb.append("__ERR__");
return;
}
sb.setLength(0);
sb.append("ERR:");
sb.append("Expected a type of some sort, not a " + expr.getClass().getName());
sb.append("__ERR__");
}
private JCMethodDecl generateToBuilderMethod(BuilderJob job, List<JCTypeParameter>
typeParameters, String prefix) {
// return new ThingieBuilder<A, B>().setA(this.a).setB(this.b);
JavacTreeMaker maker = job.getTreeMaker();
JCExpression call = maker.NewClass(null, List.<JCExpression>nil(),
namePlusTypeParamsToTypeReference(maker, job.parentType, job.toName(job.builderClassName),
!job.isStatic, job.builderTypeParams), List.<JCExpression>nil(), null);
JCExpression invoke = call;
ListBuffer<JCStatement> preStatements = null;
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
for (BuilderFieldData bfd : job.builderFields) {
String setterPrefix = !prefix.isEmpty() ? prefix : job.oldFluent ? "" : "set";
String prefixedSetterName = bfd.name.toString();
if (!setterPrefix.isEmpty()) prefixedSetterName = HandlerUtil.buildAccessorName(job.sourceNode,
setterPrefix, prefixedSetterName);
Name setterName = job.toName(prefixedSetterName);
JCExpression[] tgt = new JCExpression[bfd.singularData == null ? 1 : 2];
if (bfd.obtainVia == null || !bfd.obtainVia.field().isEmpty()) {
for (int i = 0; i < tgt.length; i++) {
tgt[i] = maker.Select(maker.Ident(job.toName("this")), bfd.obtainVia == null ? bfd.rawName :
job.toName(bfd.obtainVia.field()));
}
} else {
String name = bfd.obtainVia.method();
JCMethodInvocation inv;
if (bfd.obtainVia.isStatic()) {
JCExpression c = maker.Select(maker.Ident(job.toName(job.parentType.getName())),
job.toName(name));
inv = maker.Apply(typeParameterNames(maker, typeParameters), c,
List.<JCExpression>of(maker.Ident(job.toName("this"))));
} else {
JCExpression c = maker.Select(maker.Ident(job.toName("this")), job.toName(name));
inv = maker.Apply(List.<JCExpression>nil(), c, List.<JCExpression>nil());
```

```
}
for (int i = 0; i < tgt.length; i++) tgt[i] = maker.Ident(bfd.name);
// javac appears to cache the type of JCMethodInvocation expressions based on position, meaning, if
you have 2 ObtainVia-based method invokes on different types, you get bizarre type mismatch errors.
// going via a local variable declaration solves the problem.
JCExpression varType = JavacHandlerUtil.cloneType(maker, bfd.type, job.sourceNode);
if (preStatements == null) preStatements = new ListBuffer<JCStatement>();
preStatements.append(maker.VarDef(maker.Modifiers(Flags.FINAL), bfd.name, varType, inv));
}
JCExpression arg;
if (bfd.singularData == null) {
arg = tgt[0];
invoke = maker.Apply(List.<JCExpression>nil(), maker.Select(invoke, setterName), List.of(arg));
} else {
JCExpression isNotNull = maker.Binary(CTC_NOT_EQUAL, tgt[0], maker.Literal(CTC_BOT, null));
JCExpression invokeBuilder = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(job.toName(BUILDER_TEMP_VAR)), setterName),
List.<JCExpression>of(tgt[1]));
statements.append(maker.If(isNotNull, maker.Exec(invokeBuilder), null));
}
}
if (!statements.isEmpty()) {
JCExpression tempVarType = namePlusTypeParamsToTypeReference(maker, job.parentType,
job.getBuilderClassName(), !job.isStatic, typeParameters);
statements.prepend(maker.VarDef(maker.Modifiers(Flags.FINAL),
job.toName(BUILDER_TEMP_VAR), tempVarType, invoke));
statements.append(maker.Return(maker.Ident(job.toName(BUILDER_TEMP_VAR))));
} else {
statements.append(maker.Return(invoke));
}
if (preStatements != null) {
preStatements.appendList(statements);
statements = preStatements;
}
JCBlock body = maker.Block(0, statements.toList());
List<JCAnnotation> annsOnParamType = List.nil();
if (job.checkerFramework.generateUnique()) annsOnParamType =
List.of(maker.Annotation(genTypeRef(job.parentType, CheckerFrameworkVersion.NAME__UNIQUE),
List.<JCExpression>nil()));
JCMethodDecl methodDef = maker.MethodDef(maker.Modifiers(toJavacModifier(job.accessOuters)),
job.toName(TO_BUILDER_METHOD_NAME), namePlusTypeParamsToTypeReference(maker,
job.parentType, job.getBuilderClassName(), !job.isStatic, typeParameters, annsOnParamType),
List.<JCTypeParameter>nil(), List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
createRelevantNonNullAnnotation(job.parentType, methodDef);
return methodDef;
}
private JCMethodDecl generateCleanMethod(BuilderJob job) {
JavacTreeMaker maker = job.getTreeMaker();
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.singularData.getSingularizer().appendCleaningCode(bfd.singularData, job.builderType,
job.sourceNode, statements);
```

```
}
}
statements.append(maker.Exec(maker.Assign(maker.Select(maker.Ident(job.toName("this")),
job.toName(CLEAN_FIELD_NAME)), maker.Literal(CTC_BOOLEAN, 0))));
JCBlock body = maker.Block(0, statements.toList());
JCMethodDecl method = maker.MethodDef(maker.Modifiers(toJavacModifier(AccessLevel.PRIVATE)),
job.toName(CLEAN_METHOD_NAME),
maker.Type(Javac.createVoidType(job.builderType.getSymbolTable(), CTC_VOID)),
List.<JCTypeParameter>nil(), List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
recursiveSetGeneratedBy(method, job.sourceNode);
return method;
}
static JCVariableDecl generateReceiver(BuilderJob job) {
if (!job.checkerFramework.generateCalledMethods()) return null;
ArrayList<String> mandatories = new ArrayList<String>();
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData == null && bfd.nameOfSetFlag == null) mandatories.add(bfd.name.toString());
}
JCExpression arg;
JavacTreeMaker maker = job.getTreeMaker();
if (mandatories.size() == 0) return null;
if (mandatories.size() == 1) arg = maker.Literal(mandatories.get(0));
else {
List<JCExpression> elems = List.nil();
for (int i = mandatories.size() - 1; i >= 0; i--) elems = elems.prepend(maker.Literal(mandatories.get(i)));
arg = maker.NewArray(null, List.<JCExpression>nil(), elems);
}
JCAnnotation recvAnno = maker.Annotation(genTypeRef(job.builderType,
CheckerFrameworkVersion.NAME__CALLED), List.of(arg));
JCClassDecl builderTypeNode = (JCClassDecl) job.builderType.get();
JCVariableDecl recv = maker.VarDef(maker.Modifiers(Flags.PARAMETER, List.<JCAnnotation>nil()),
job.toName("this"), namePlusTypeParamsToTypeReference(maker, job.builderType,
builderTypeNode.typarams, List.<JCAnnotation>of(recvAnno)), null);
return recv;
}
private JCMethodDecl generateBuildMethod(BuilderJob job, Name staticName, JCExpression
returnType, List<JCExpression> thrownExceptions, boolean addCleaning) {
JavacTreeMaker maker = job.getTreeMaker();
JCExpression call;
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
if (addCleaning) {
JCExpression notClean = maker.Unary(CTC_NOT, maker.Select(maker.Ident(job.toName("this")),
job.toName(CLEAN_FIELD_NAME)));
JCStatement invokeClean = maker.Exec(maker.Apply(List.<JCExpression>nil(),
maker.Ident(job.toName(CLEAN_METHOD_NAME)), List.<JCExpression>nil()));
JCIf ifUnclean = maker.If(notClean, invokeClean, null);
statements.append(ifUnclean);
}
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.singularData.getSingularizer().appendBuildCode(bfd.singularData, job.builderType,
job.sourceNode, statements, bfd.builderFieldName, "this");
}
```

```
}
ListBuffer<JCExpression> args = new ListBuffer<JCExpression>();
Name thisName = job.toName("this");
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.nameOfSetFlag != null) {
statements.append(maker.VarDef(maker.Modifiers(0L), bfd.builderFieldName, cloneType(maker,
bfd.type, job.sourceNode), maker.Select(maker.Ident(thisName), bfd.builderFieldName)));
statements.append(maker.If(maker.Unary(CTC_NOT, maker.Select(maker.Ident(thisName),
bfd.nameOfSetFlag)), maker.Exec(maker.Assign(maker.Ident(bfd.builderFieldName),
maker.Apply(typeParameterNames(maker, ((JCClassDecl) job.parentType.get()).typarams),
maker.Select(maker.Ident(((JCClassDecl) job.parentType.get()).name), bfd.nameOfDefaultProvider),
List.<JCExpression>nil()))), null));
}
if (bfd.nameOfSetFlag != null || (bfd.singularData != null &&
bfd.singularData.getSingularizer().shadowedDuringBuild())) {
args.append(maker.Ident(bfd.builderFieldName));
} else {
args.append(maker.Select(maker.Ident(thisName), bfd.builderFieldName));
}
}
if (addCleaning) {
statements.append(maker.Exec(maker.Assign(maker.Select(maker.Ident(job.toName("this")),
job.toName(CLEAN_FIELD_NAME)), maker.Literal(CTC_BOOLEAN, 1))));
}
if (staticName == null) {
call = maker.NewClass(null, List.<JCExpression>nil(), returnType, args.toList(), null);
statements.append(maker.Return(call));
} else {
ListBuffer<JCExpression> typeParams = new ListBuffer<JCExpression>();
for (JCTypeParameter tp : ((JCClassDecl) job.builderType.get()).typarams) {
typeParams.append(maker.Ident(tp.name));
}
JCExpression callee = maker.Ident(((JCClassDecl) job.parentType.get()).name);
if (!job.isStatic) callee = maker.Select(callee, job.toName("this"));
JCExpression fn = maker.Select(callee, staticName);
call = maker.Apply(typeParams.toList(), fn, args.toList());
if (returnType instanceof JCPrimitiveTypeTree && CTC_VOID.equals(typeTag(returnType))) {
statements.append(maker.Exec(call));
} else {
statements.append(maker.Return(call));
}
}
JCBlock body = maker.Block(0, statements.toList());
List<JCAnnotation> annsOnMethod = job.checkerFramework.generateSideEffectFree() ?
List.of(maker.Annotation(genTypeRef(job.builderType,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil())) :
List.<JCAnnotation>nil();
JCVariableDecl recv = generateReceiver(job);
JCMethodDecl methodDef;
JCExpression returnTypeCopy = cloneType(maker, returnType, job.sourceNode);
if (recv != null && maker.hasMethodDefWithRecvParam()) {
methodDef = maker.MethodDefWithRecvParam(maker.Modifiers(toJavacModifier(job.accessInners),
annsOnMethod), job.toName(job.buildMethodName), returnTypeCopy, List.<JCTypeParameter>nil(),
```

```
recv, List.<JCVariableDecl>nil(), thrownExceptions, body, null);
} else {
methodDef = maker.MethodDef(maker.Modifiers(toJavacModifier(job.accessInners), annsOnMethod),
job.toName(job.buildMethodName), returnTypeCopy, List.<JCTypeParameter>nil(),
List.<JCVariableDecl>nil(), thrownExceptions, body, null);
}
if (staticName == null) createRelevantNonNullAnnotation(job.builderType, methodDef);
return methodDef;
}
public static JCMethodDecl generateDefaultProvider(Name methodName, JavacNode fieldNode,
List<JCTypeParameter> params, BuilderJob job) {
JavacTreeMaker maker = fieldNode.getTreeMaker();
JCVariableDecl field = (JCVariableDecl) fieldNode.get();
// Lombok tries to keep the position of the original initializer. First we save the expression ...
JCExpression init = field.init;
field.init = null;
// ... then we generate an empty return statement ...
JCReturn statement = maker.Return(null);
JCBlock body = maker.Block(0, List.<JCStatement>of(statement));
int modifiers = Flags.PRIVATE | Flags.STATIC;
JCMethodDecl defaultProvider = maker.MethodDef(maker.Modifiers(modifiers), methodName,
cloneType(maker, field.vartype, fieldNode), copyTypeParams(fieldNode, params),
List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
// ... then we convert short array initializers from `{1,2}` to `new int[]{1,2}` ...
if (init instanceof JCNewArray && field.vartype instanceof JCArrayTypeTree) {
JCNewArray arrayInitializer = (JCNewArray) init;
JCArrayTypeTree fieldType = (JCArrayTypeTree) field.vartype;
if (arrayInitializer.elemtype == null) {
arrayInitializer.elemtype = cloneType(maker, fieldType.elemtype, fieldNode);
}
}
// ... then we set positions for everything else ...
recursiveSetGeneratedBy(defaultProvider, job.sourceNode);
// ... and finally add back the original expression
statement.expr = init;
return defaultProvider;
}
public JCMethodDecl generateBuilderMethod(BuilderJob job) {
//String builderClassName, JavacNode source, JavacNode type, List<JCTypeParameter> typeParams,
AccessLevel access) {
//builderClassName, annotationNode, tdParent, typeParams, accessForOuters);
JavacTreeMaker maker = job.getTreeMaker();
JCExpression call;
if (job.isStatic) {
call = maker.NewClass(null, List.<JCExpression>nil(), namePlusTypeParamsToTypeReference(maker,
job.parentType, job.toName(job.builderClassName), false, job.typeParams), List.<JCExpression>nil(),
null);
} else {
call = maker.NewClass(null, List.<JCExpression>nil(), namePlusTypeParamsToTypeReference(maker,
null, job.toName(job.builderClassName), false, job.typeParams), List.<JCExpression>nil(), null);
((JCNewClass) call).encl = maker.Ident(job.toName("this"));
}
JCStatement statement = maker.Return(call);
```

```
JCBlock body = maker.Block(0, List.<JCStatement>of(statement));
int modifiers = toJavacModifier(job.accessOuters);
if (job.isStatic) modifiers |= Flags.STATIC;
List<JCAnnotation> annsOnMethod = List.nil();
if (job.checkerFramework.generateSideEffectFree()) annsOnMethod =
List.of(maker.Annotation(genTypeRef(job.parentType,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
List<JCAnnotation> annsOnParamType = List.nil();
if (job.checkerFramework.generateUnique()) annsOnParamType =
List.of(maker.Annotation(genTypeRef(job.parentType, CheckerFrameworkVersion.NAME__UNIQUE),
List.<JCExpression>nil()));
JCExpression returnType = namePlusTypeParamsToTypeReference(maker, job.parentType,
job.getBuilderClassName(), !job.isStatic, job.builderTypeParams, annsOnParamType);
JCMethodDecl methodDef = maker.MethodDef(maker.Modifiers(modifiers, annsOnMethod),
job.toName(job.builderMethodName), returnType, job.copyTypeParams(), List.<JCVariableDecl>nil(),
List.<JCExpression>nil(), body, null);
createRelevantNonNullAnnotation(job.parentType, methodDef);
return methodDef;
}
public void generateBuilderFields(BuilderJob job) {
int len = job.builderFields.size();
java.util.List<JavacNode> existing = new ArrayList<JavacNode>();
for (JavacNode child : job.builderType.down()) {
if (child.getKind() == Kind.FIELD) existing.add(child);
}
java.util.List<JCVariableDecl> generated = new ArrayList<JCVariableDecl>();
for (int i = len - 1; i >= 0; i--) {
BuilderFieldData bfd = job.builderFields.get(i);
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
java.util.List<JavacNode> generateSingularFields =
bfd.singularData.getSingularizer().generateFields(bfd.singularData, job.builderType, job.sourceNode);
for (JavacNode field : generateSingularFields) {
generated.add((JCVariableDecl) field.get());
}
bfd.createdFields.addAll(generateSingularFields);
} else {
JavacNode field = null, setFlag = null;
for (JavacNode exists : existing) {
Name n = ((JCVariableDecl) exists.get()).name;
if (n.equals(bfd.builderFieldName)) field = exists;
if (n.equals(bfd.nameOfSetFlag)) setFlag = exists;
}
JavacTreeMaker maker = job.getTreeMaker();
if (field == null) {
JCModifiers mods = maker.Modifiers(Flags.PRIVATE);
JCVariableDecl newField = maker.VarDef(mods, bfd.builderFieldName, cloneType(maker, bfd.type,
job.sourceNode), null);
field = injectFieldAndMarkGenerated(job.builderType, newField);
generated.add(newField);
}
if (setFlag == null && bfd.nameOfSetFlag != null) {
JCModifiers mods = maker.Modifiers(Flags.PRIVATE);
```

```
JCVariableDecl newField = maker.VarDef(mods, bfd.nameOfSetFlag,
maker.TypeIdent(CTC_BOOLEAN), null);
injectFieldAndMarkGenerated(job.builderType, newField);
generated.add(newField);
}
bfd.createdFields.add(field);
}
}
for (JCVariableDecl gen : generated) recursiveSetGeneratedBy(gen, job.sourceNode);
}
public void makePrefixedSetterMethodsForBuilder(BuilderJob job, BuilderFieldData bfd, String prefix) {
boolean deprecate = isFieldDeprecated(bfd.originalFieldNode);
if (bfd.singularData == null || bfd.singularData.getSingularizer() == null) {
makePrefixedSetterMethodForBuilder(job, bfd, deprecate, prefix);
} else {
bfd.singularData.getSingularizer().generateMethods(job, bfd.singularData, deprecate);
}
}
private void makePrefixedSetterMethodForBuilder(BuilderJob job, BuilderFieldData bfd, boolean
deprecate, String prefix) {
JavacNode fieldNode = bfd.createdFields.get(0);
String setterPrefix = !prefix.isEmpty() ? prefix : job.oldFluent ? "" : "set";
String setterName = HandlerUtil.buildAccessorName(job.sourceNode, setterPrefix,
bfd.name.toString());
Name setterName_ = job.builderType.toName(setterName);
for (JavacNode child : job.builderType.down()) {
if (child.getKind() != Kind.METHOD) continue;
JCMethodDecl methodDecl = (JCMethodDecl) child.get();
Name existingName = methodDecl.name;
if (existingName.equals(setterName_) && !isTolerate(fieldNode, methodDecl)) return;
}
JavacTreeMaker maker = fieldNode.getTreeMaker();
List<JCAnnotation> methodAnns =
JavacHandlerUtil.findCopyableToSetterAnnotations(bfd.originalFieldNode);
JCMethodDecl newMethod = HandleSetter.createSetter(toJavacModifier(job.accessInners), deprecate,
fieldNode, maker, setterName, bfd.name, bfd.nameOfSetFlag, job.oldChain, job.sourceNode,
methodAnns, bfd.annotations);
recursiveSetGeneratedBy(newMethod, job.sourceNode);
if (job.sourceNode.up().getKind() == Kind.METHOD) {
copyJavadocFromParam(bfd.originalFieldNode.up(), newMethod, bfd.name.toString());
} else {
copyJavadoc(bfd.originalFieldNode, newMethod, CopyJavadoc.SETTER, true);
}
injectMethod(job.builderType, newMethod);
}
public JavacNode makeBuilderClass(BuilderJob job) {
//boolean isStatic, JavacNode source, JavacNode tdParent, String builderClassName,
List<JCTypeParameter> typeParams, JCAnnotation ast, AccessLevel access) {
//isStatic, annotationNode, tdParent, builderClassName, typeParams, ast, accessForOuters
JavacTreeMaker maker = job.getTreeMaker();
int modifiers = toJavacModifier(job.accessOuters);
if (job.isStatic) modifiers |= Flags.STATIC;
JCModifiers mods = maker.Modifiers(modifiers);
```

```
JCClassDecl builder = maker.ClassDef(mods, job.getBuilderClassName(), job.copyTypeParams(), null,
List.<JCExpression>nil(), List.<JCTree>nil());
recursiveSetGeneratedBy(builder, job.sourceNode);
return injectType(job.parentType, builder);
}
private void addObtainVia(BuilderFieldData bfd, JavacNode node) {
for (JavacNode child : node.down()) {
if (!annotationTypeMatches(ObtainVia.class, child)) continue;
AnnotationValues<ObtainVia> ann = createAnnotation(ObtainVia.class, child);
bfd.obtainVia = ann.getInstance();
bfd.obtainViaNode = child;
deleteAnnotationIfNeccessary(child, ObtainVia.class);
return;
}
}
/**
* Returns the explicitly requested singular annotation on this node (field
* or parameter), or null if there's no {@code @Singular} annotation on it.
*
* @param node The node (field or method param) to inspect for its name and potential {@code
@Singular} annotation.
* @param setterPrefix Explicitly requested setter prefix.
*/
private SingularData getSingularData(JavacNode node, String setterPrefix) {
for (JavacNode child : node.down()) {
if (!annotationTypeMatches(Singular.class, child)) continue;
Name pluralName = node.getKind() == Kind.FIELD ? removePrefixFromField(node) : ((JCVariableDecl)
node.get()).name;
AnnotationValues<Singular> ann = createAnnotation(Singular.class, child);
Singular singularInstance = ann.getInstance();
deleteAnnotationIfNeccessary(child, Singular.class);
String explicitSingular = singularInstance.value();
if (explicitSingular.isEmpty()) {
if (Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKeys.SINGULAR_AUTO))) {
node.addError("The singular must be specified explicitly (e.g. @Singular(\"task\")) because auto
singularization is disabled.");
explicitSingular = pluralName.toString();
} else {
explicitSingular = autoSingularize(pluralName.toString());
if (explicitSingular == null) {
node.addError("Can't singularize this name; please specify the singular explicitly (i.e.
@Singular(\"sheep\"))");
explicitSingular = pluralName.toString();
}
}
}
Name singularName = node.toName(explicitSingular);
JCExpression type = null;
if (node.get() instanceof JCVariableDecl) {
type = ((JCVariableDecl) node.get()).vartype;
}
String name = null;
List<JCExpression> typeArgs = List.nil();
```

```
if (type instanceof JCTypeApply) {
typeArgs = ((JCTypeApply) type).arguments;
type = ((JCTypeApply) type).clazz;
}
name = type.toString();
String targetFqn = JavacSingularsRecipes.get().toQualified(name);
JavacSingularizer singularizer = JavacSingularsRecipes.get().getSingularizer(targetFqn, node);
if (singularizer == null) {
node.addError("Lombok does not know how to create the singular-form builder methods for type '" +
name + "'; they won't be generated.");
return null;
}
return new SingularData(child, singularName, pluralName, typeArgs, targetFqn, singularizer,
singularInstance.ignoreNullCollections(), setterPrefix);
}
return null;
}
}
```

lombok/javac/handlers/HandleBuilderDefault.java

```
/*
* Copyright (C) 2017-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import lombok.Builder;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
```

```
import lombok.experimental.SuperBuilder;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
@Provides
@HandlerPriority(-1025) //HandleBuilder's level, minus one.
public class HandleBuilderDefault extends JavacAnnotationHandler<Builder.Default> {
@Override public void handle(AnnotationValues<Builder.Default> annotation, JCAnnotation ast,
JavacNode annotationNode) {
JavacNode annotatedField = annotationNode.up();
if (annotatedField.getKind() != Kind.FIELD) return;
JavacNode classWithAnnotatedField = annotatedField.up();
if (!hasAnnotation(Builder.class, classWithAnnotatedField) &&
!hasAnnotation("lombok.experimental.Builder", classWithAnnotatedField)
&& !hasAnnotation(SuperBuilder.class, classWithAnnotatedField)) {
annotationNode.addWarning("@Builder.Default requires @Builder or @SuperBuilder on the class for it
to mean anything.");
deleteAnnotationIfNeccessary(annotationNode, Builder.Default.class);
}
/** HandleBuilder is going to wipe out the import, at which point '@Builder.Default' is no longer clear. */
if (ast.annotationType instanceof JCFieldAccess) {
JCFieldAccess jfa = (JCFieldAccess) ast.annotationType;
if (jfa.selected instanceof JCIdent && ((JCIdent) jfa.selected).name.contentEquals("Builder") &&
jfa.name.contentEquals("Default")) {
JCFieldAccess newJfaSel = annotationNode.getTreeMaker().Select(annotationNode.getTreeMaker().I
dent(annotationNode.toName("lombok")), ((JCIdent) jfa.selected).name);
recursiveSetGeneratedBy(newJfaSel, annotationNode);
jfa.selected = newJfaSel;
}
}
}
}
```

lombok/javac/handlers/HandleBuilderDefaultRemove.java

```
/*
* Copyright (C) 2018-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import lombok.Builder;
import lombok.Builder.Default;
import lombok.core.AlreadyHandledAnnotations;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
@Provides
@HandlerPriority(32768)
@AlreadyHandledAnnotations
public class HandleBuilderDefaultRemove extends JavacAnnotationHandler<Builder.Default> {
@Override public void handle(AnnotationValues<Default> annotation, JCAnnotation ast, JavacNode
annotationNode) {
deleteAnnotationIfNeccessary(annotationNode, Builder.Default.class);
deleteImportFromCompilationUnit(annotationNode, Builder.class.getName());
deleteImportFromCompilationUnit(annotationNode, Builder.Default.class.getName());
}
}

lombok/javac/handlers/HandleBuilderRemove.java

/*
* Copyright (C) 2020-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN

```
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import lombok.Builder;
import lombok.core.AlreadyHandledAnnotations;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
@Provides
@HandlerPriority(32768)
@AlreadyHandledAnnotations
public class HandleBuilderRemove extends JavacAnnotationHandler<Builder> {
@Override public void handle(AnnotationValues<Builder> annotation, JCAnnotation ast, JavacNode
annotationNode) {
deleteAnnotationIfNeccessary(annotationNode, Builder.class, "lombok.experimental.Builder");
}
}
```

lombok/javac/handlers/HandleCleanup.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import static lombok.javac.Javac.*;
import lombok.Cleanup;
import lombok.ConfigurationKeys;
```

```
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.delombok.LombokOptionsFactory;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCBinary;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCCase;
import com.sun.tools.javac.tree.JCTree.JCCatch;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCIf;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeCast;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
/**
* Handles the {@code lombok.Cleanup} annotation for javac.
*/
@Provides
public class HandleCleanup extends JavacAnnotationHandler<Cleanup> {
@Override public void handle(AnnotationValues<Cleanup> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.CLEANUP_FLAG_USAGE, "@Cleanup");
if (inNetbeansEditor(annotationNode)) return;
deleteAnnotationIfNeccessary(annotationNode, Cleanup.class);
String cleanupName = annotation.getInstance().value();
if (cleanupName.length() == 0) {
annotationNode.addError("cleanupName cannot be the empty string.");
return;
}
if (annotationNode.up().getKind() != Kind.LOCAL) {
annotationNode.addError("@Cleanup is legal only on local variable declarations.");
return;
}
JCVariableDecl decl = (JCVariableDecl)annotationNode.up().get();
if (decl.init == null) {
annotationNode.addError("@Cleanup variable declarations need to be initialized.");
return;
}
JavacNode ancestor = annotationNode.up().directUp();
JCTree blockNode = ancestor.get();
```

```
final List<JCStatement> statements;
if (blockNode instanceof JCBlock) {
statements = ((JCBlock)blockNode).stats;
} else if (blockNode instanceof JCCase) {
statements = ((JCCase)blockNode).stats;
} else if (blockNode instanceof JCMethodDecl) {
statements = ((JCMethodDecl)blockNode).body.stats;
} else {
annotationNode.addError("@Cleanup is legal only on a local variable declaration inside a block.");
return;
}
boolean seenDeclaration = false;
ListBuffer<JCStatement> newStatements = new ListBuffer<JCStatement>();
ListBuffer<JCStatement> tryBlock = new ListBuffer<JCStatement>();
for (JCStatement statement : statements) {
if (!seenDeclaration) {
if (statement == decl) seenDeclaration = true;
newStatements.append(statement);
} else {
tryBlock.append(statement);
}
}
if (!seenDeclaration) {
annotationNode.addError("LOMBOK BUG: Can't find this local variable declaration inside its parent.");
return;
}
doAssignmentCheck(annotationNode, tryBlock.toList(), decl.name);
JavacTreeMaker maker = annotationNode.getTreeMaker();
JCFieldAccess cleanupMethod = maker.Select(maker.Ident(decl.name),
annotationNode.toName(cleanupName));
List<JCStatement> cleanupCall = List.<JCStatement>of(maker.Exec(
maker.Apply(List.<JCExpression>nil(), cleanupMethod, List.<JCExpression>nil())));
JCExpression preventNullAnalysis = preventNullAnalysis(maker, annotationNode,
maker.Ident(decl.name));
JCBinary isNull = maker.Binary(CTC_NOT_EQUAL, preventNullAnalysis, maker.Literal(CTC_BOT,
null));
JCIf ifNotNullCleanup = maker.If(isNull, maker.Block(0, cleanupCall), null);
JCBlock finalizer = recursiveSetGeneratedBy(maker.Block(0, List.<JCStatement>of(ifNotNullCleanup)),
annotationNode);
newStatements.append(setGeneratedBy(maker.Try(setGeneratedBy(maker.Block(0, tryBlock.toList()),
annotationNode), List.<JCCatch>nil(), finalizer), annotationNode));
if (blockNode instanceof JCBlock) {
((JCBlock)blockNode).stats = newStatements.toList();
} else if (blockNode instanceof JCCase) {
((JCCase)blockNode).stats = newStatements.toList();
} else if (blockNode instanceof JCMethodDecl) {
((JCMethodDecl)blockNode).body.stats = newStatements.toList();
} else throw new AssertionError("Should not get here");
ancestor.rebuild();
}
public JCExpression preventNullAnalysis(JavacTreeMaker maker, JavacNode node, JCExpression
expression) {
```

```
if (LombokOptionsFactory.getDelombokOptions(node.getContext()).getFormatPreferences().danceAro
undIdeChecks()) {
JCMethodInvocation singletonList = maker.Apply(List.<JCExpression>nil(), chainDotsString(node,
"java.util.Collections.singletonList"), List.of(expression));
JCMethodInvocation cleanedExpr = maker.Apply(List.<JCExpression>nil(), maker.Select(singletonList,
node.toName("get")) , List.<JCExpression>of(maker.Literal(CTC_INT, 0)));
return cleanedExpr;
} else {
return expression;
}
}
public void doAssignmentCheck(JavacNode node, List<JCStatement> statements, Name name) {
for (JCStatement statement : statements) doAssignmentCheck0(node, statement, name);
}
public void doAssignmentCheck0(JavacNode node, JCTree statement, Name name) {
if (statement instanceof JCAssign) doAssignmentCheck0(node, ((JCAssign)statement).rhs, name);
if (statement instanceof JCExpressionStatement) doAssignmentCheck0(node,
((JCExpressionStatement)statement).expr, name);
if (statement instanceof JCVariableDecl) doAssignmentCheck0(node, ((JCVariableDecl)statement).init,
name);
if (statement instanceof JCTypeCast) doAssignmentCheck0(node, ((JCTypeCast)statement).expr,
name);
if (statement instanceof JCIdent) {
if (((JCIdent)statement).name.contentEquals(name)) {
JavacNode problemNode = node.getNodeFor(statement);
if (problemNode != null) problemNode.addWarning(
"You're assigning an auto-cleanup variable to something else. This is a bad idea.");
}
}
}
}
```

lombok/javac/handlers/HandleConstructor.java

```
/*
* Copyright (C) 2010-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.ConfigurationKeys;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.delombok.LombokOptionsFactory;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
public class HandleConstructor {
@Provides
public static class HandleNoArgsConstructor extends JavacAnnotationHandler<NoArgsConstructor> {
private static final String NAME = NoArgsConstructor.class.getSimpleName();
private HandleConstructor handleConstructor = new HandleConstructor();
@Override public void handle(AnnotationValues<NoArgsConstructor> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.NO_ARGS_CONSTRUCTOR_FLAG_USAGE,
"@NoArgsConstructor", ConfigurationKeys.ANY_CONSTRUCTOR_FLAG_USAGE, "any
@xArgsConstructor");
deleteAnnotationIfNeccessary(annotationNode, NoArgsConstructor.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
```

```
JavacNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode, NAME)) return;
List<JCAnnotation> onConstructor = unboxAndRemoveAnnotationParameter(ast, "onConstructor",
"@NoArgsConstructor(onConstructor", annotationNode);
if (!onConstructor.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@NoArgsConstructor(onConstructor=...)");
}
NoArgsConstructor ann = annotation.getInstance();
AccessLevel level = ann.access();
if (level == AccessLevel.NONE) return;
String staticName = ann.staticName();
boolean force = ann.force();
handleConstructor.generateConstructor(typeNode, level, onConstructor, List.<JavacNode>nil(), force,
staticName, SkipIfConstructorExists.NO, annotationNode);
}
}
@Provides
public static class HandleRequiredArgsConstructor extends
JavacAnnotationHandler<RequiredArgsConstructor> {
private static final String NAME = RequiredArgsConstructor.class.getSimpleName();
private HandleConstructor handleConstructor = new HandleConstructor();
@Override public void handle(AnnotationValues<RequiredArgsConstructor> annotation, JCAnnotation
ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode,
ConfigurationKeys.REQUIRED_ARGS_CONSTRUCTOR_FLAG_USAGE,
"@RequiredArgsConstructor", ConfigurationKeys.ANY_CONSTRUCTOR_FLAG_USAGE, "any
@xArgsConstructor");
deleteAnnotationIfNeccessary(annotationNode, RequiredArgsConstructor.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode, NAME)) return;
List<JCAnnotation> onConstructor = unboxAndRemoveAnnotationParameter(ast, "onConstructor",
"@RequiredArgsConstructor(onConstructor", annotationNode);
if (!onConstructor.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@RequiredArgsConstructor(onConstructor=...)");
}
RequiredArgsConstructor ann = annotation.getInstance();
AccessLevel level = ann.access();
if (level == AccessLevel.NONE) return;
String staticName = ann.staticName();
if (annotation.isExplicit("suppressConstructorProperties")) {
annotationNode.addError("This deprecated feature is no longer supported. Remove it; you can create a
lombok.config file with 'lombok.anyConstructor.suppressConstructorProperties = true'.");
}
handleConstructor.generateConstructor(typeNode, level, onConstructor,
findRequiredFields(typeNode), false, staticName, SkipIfConstructorExists.NO, annotationNode);
}
}
@Provides
public static class HandleAllArgsConstructor extends JavacAnnotationHandler<AllArgsConstructor> {
private static final String NAME = AllArgsConstructor.class.getSimpleName();
```

```
private HandleConstructor handleConstructor = new HandleConstructor();
@Override public void handle(AnnotationValues<AllArgsConstructor> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.ALL_ARGS_CONSTRUCTOR_FLAG_USAGE,
"@AllArgsConstructor", ConfigurationKeys.ANY_CONSTRUCTOR_FLAG_USAGE, "any
@xArgsConstructor");
deleteAnnotationIfNeccessary(annotationNode, AllArgsConstructor.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode, NAME)) return;
List<JCAnnotation> onConstructor = unboxAndRemoveAnnotationParameter(ast, "onConstructor",
"@AllArgsConstructor(onConstructor", annotationNode);
if (!onConstructor.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@AllArgsConstructor(onConstructor=...)");
}
AllArgsConstructor ann = annotation.getInstance();
AccessLevel level = ann.access();
if (level == AccessLevel.NONE) return;
String staticName = ann.staticName();
if (annotation.isExplicit("suppressConstructorProperties")) {
annotationNode.addError("This deprecated feature is no longer supported. Remove it; you can create a
lombok.config file with 'lombok.anyConstructor.suppressConstructorProperties = true'.");
}
handleConstructor.generateConstructor(typeNode, level, onConstructor, findAllFields(typeNode), false,
staticName, SkipIfConstructorExists.NO, annotationNode);
}
}
public static List<JavacNode> findRequiredFields(JavacNode typeNode) {
return findFields(typeNode, true);
}
public static List<JavacNode> findFinalFields(JavacNode typeNode) {
return findFields(typeNode, false);
}
public static List<JavacNode> findFields(JavacNode typeNode, boolean nullMarked) {
ListBuffer<JavacNode> fields = new ListBuffer<JavacNode>();
for (JavacNode child : typeNode.down()) {
if (child.getKind() != Kind.FIELD) continue;
JCVariableDecl fieldDecl = (JCVariableDecl) child.get();
//Skip fields that start with $
if (fieldDecl.name.toString().startsWith("$")) continue;
long fieldFlags = fieldDecl.mods.flags;
//Skip static fields.
if ((fieldFlags & Flags.STATIC) != 0) continue;
boolean isFinal = (fieldFlags & Flags.FINAL) != 0;
boolean isNonNull = nullMarked && hasNonNullAnnotations(child);
if ((isFinal || isNonNull) && fieldDecl.init == null) fields.append(child);
}
return fields.toList();
}
public static List<JavacNode> findAllFields(JavacNode typeNode) {
return findAllFields(typeNode, false);
}
```

```
public static List<JavacNode> findAllFields(JavacNode typeNode, boolean evenFinalInitialized) {
ListBuffer<JavacNode> fields = new ListBuffer<JavacNode>();
for (JavacNode child : typeNode.down()) {
if (child.getKind() != Kind.FIELD) continue;
JCVariableDecl fieldDecl = (JCVariableDecl) child.get();
//Skip fields that start with $
if (fieldDecl.name.toString().startsWith("$")) continue;
long fieldFlags = fieldDecl.mods.flags;
//Skip static fields.
if ((fieldFlags & Flags.STATIC) != 0) continue;
//Skip initialized final fields
boolean isFinal = (fieldFlags & Flags.FINAL) != 0;
if (evenFinalInitialized || !isFinal || fieldDecl.init == null) fields.append(child);
}
return fields.toList();
}
public static boolean checkLegality(JavacNode typeNode, JavacNode errorNode, String name) {
if (!isClassOrEnum(typeNode)) {
errorNode.addError(name + " is only supported on a class or an enum.");
return false;
}
return true;
}
public enum SkipIfConstructorExists {
YES, NO, I_AM_BUILDER;
}
public void generateExtraNoArgsConstructor(JavacNode typeNode, JavacNode source) {
if (!isDirectDescendantOfObject(typeNode)) return;
Boolean v = typeNode.getAst().readConfiguration(ConfigurationKeys.NO_ARGS_CONSTRUCTOR_E
XTRA_PRIVATE);
if (v == null || !v) return;
generate(typeNode, AccessLevel.PRIVATE, List.<JCAnnotation>nil(), List.<JavacNode>nil(), true, null,
SkipIfConstructorExists.NO, source, true);
}
public void generateRequiredArgsConstructor(JavacNode typeNode, AccessLevel level, String
staticName, SkipIfConstructorExists skipIfConstructorExists, JavacNode source) {
generateConstructor(typeNode, level, List.<JCAnnotation>nil(), findRequiredFields(typeNode), false,
staticName, skipIfConstructorExists, source);
}
public void generateAllArgsConstructor(JavacNode typeNode, AccessLevel level, String staticName,
SkipIfConstructorExists skipIfConstructorExists, JavacNode source) {
generateConstructor(typeNode, level, List.<JCAnnotation>nil(), findAllFields(typeNode), false,
staticName, skipIfConstructorExists, source);
}
public void generateConstructor(JavacNode typeNode, AccessLevel level, List<JCAnnotation>
onConstructor, List<JavacNode> fields, boolean allToDefault, String staticName,
SkipIfConstructorExists skipIfConstructorExists, JavacNode source) {
generate(typeNode, level, onConstructor, fields, allToDefault, staticName, skipIfConstructorExists,
source, false);
}
private void generate(JavacNode typeNode, AccessLevel level, List<JCAnnotation> onConstructor,
List<JavacNode> fields, boolean allToDefault, String staticName, SkipIfConstructorExists
skipIfConstructorExists, JavacNode source, boolean noArgs) {
```

```
boolean staticConstrRequired = staticName != null && !staticName.equals("");
if (skipIfConstructorExists != SkipIfConstructorExists.NO) {
for (JavacNode child : typeNode.down()) {
if (child.getKind() == Kind.ANNOTATION) {
boolean skipGeneration = annotationTypeMatches(NoArgsConstructor.class, child) ||
annotationTypeMatches(AllArgsConstructor.class, child) ||
annotationTypeMatches(RequiredArgsConstructor.class, child);
if (!skipGeneration && skipIfConstructorExists == SkipIfConstructorExists.YES) {
skipGeneration = annotationTypeMatches(Builder.class, child);
}
if (skipGeneration) {
if (staticConstrRequired) {
// @Data has asked us to generate a constructor, but we're going to skip this instruction, as an explicit
'make a constructor' annotation
// will take care of it. However, @Data also wants a specific static name; this will be ignored; the
appropriate way to do this is to use
// the 'staticName' parameter of the @XArgsConstructor you've stuck on your type.
// We should warn that we're ignoring @Data's 'staticConstructor' param.
source.addWarning("Ignoring static constructor name: explicit @XxxArgsConstructor annotation
present; its `staticName` parameter will be used.");
}
return;
}
}
}
}
if (noArgs && noArgsConstructorExists(typeNode)) return;
if (!(skipIfConstructorExists != SkipIfConstructorExists.NO && constructorExists(typeNode) !=
MemberExistsResult.NOT_EXISTS)) {
JCMethodDecl constr = createConstructor(staticConstrRequired ? AccessLevel.PRIVATE : level,
onConstructor, typeNode, fields, allToDefault, source);
generateConstructorJavadoc(constr, typeNode, fields);
injectMethod(typeNode, constr);
}
generateStaticConstructor(staticConstrRequired, typeNode, staticName, level, allToDefault, fields,
source);
}
private void generateStaticConstructor(boolean staticConstrRequired, JavacNode typeNode, String
staticName, AccessLevel level, boolean allToDefault, List<JavacNode> fields, JavacNode source) {
if (staticConstrRequired) {
JCMethodDecl staticConstr = createStaticConstructor(staticName, level, typeNode, allToDefault ?
List.<JavacNode>nil() : fields, source);
generateConstructorJavadoc(staticConstr, typeNode, fields);
injectMethod(typeNode, staticConstr);
}
}
private static boolean noArgsConstructorExists(JavacNode node) {
node = upToTypeNode(node);
if (node != null && node.get() instanceof JCClassDecl) {
for (JCTree def : ((JCClassDecl) node.get()).defs) {
if (def instanceof JCMethodDecl) {
JCMethodDecl md = (JCMethodDecl) def;
if (md.name.contentEquals("<init>") && md.params.size() == 0) return true;
```

```
}
}
}
for (JavacNode child : node.down()) {
if (annotationTypeMatches(NoArgsConstructor.class, child)) return true;
if (annotationTypeMatches(RequiredArgsConstructor.class, child) &&
findRequiredFields(node).isEmpty()) return true;
if (annotationTypeMatches(AllArgsConstructor.class, child) && findAllFields(node).isEmpty()) return
true;
}
return false;
}
public static void addConstructorProperties(JCModifiers mods, JavacNode node, List<JavacNode>
fields) {
if (fields.isEmpty()) return;
JavacTreeMaker maker = node.getTreeMaker();
JCExpression constructorPropertiesType = chainDots(node, "java", "beans", "ConstructorProperties");
ListBuffer<JCExpression> fieldNames = new ListBuffer<JCExpression>();
for (JavacNode field : fields) {
Name fieldName = removePrefixFromField(field);
fieldNames.append(maker.Literal(fieldName.toString()));
}
JCExpression fieldNamesArray = maker.NewArray(null, List.<JCExpression>nil(), fieldNames.toList());
JCAnnotation annotation = maker.Annotation(constructorPropertiesType, List.of(fieldNamesArray));
mods.annotations = mods.annotations.append(annotation);
}
@SuppressWarnings("deprecation") public static JCMethodDecl createConstructor(AccessLevel level,
List<JCAnnotation> onConstructor, JavacNode typeNode, List<JavacNode> fieldsToParam, boolean
forceDefaults, JavacNode source) {
JavacTreeMaker maker = typeNode.getTreeMaker();
boolean isEnum = (((JCClassDecl) typeNode.get()).mods.flags & Flags.ENUM) != 0;
if (isEnum) level = AccessLevel.PRIVATE;
boolean addConstructorProperties;
List<JavacNode> fieldsToDefault = fieldsNeedingBuilderDefaults(typeNode, fieldsToParam);
List<JavacNode> fieldsToExplicit = forceDefaults ? fieldsNeedingExplicitDefaults(typeNode,
fieldsToParam) : List.<JavacNode>nil();
if (fieldsToParam.isEmpty()) {
addConstructorProperties = false;
} else {
Boolean v = typeNode.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTOR_ADD_C
ONSTRUCTOR_PROPERTIES);
addConstructorProperties = v != null ? v.booleanValue() :
Boolean.FALSE.equals(typeNode.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTO
R_SUPPRESS_CONSTRUCTOR_PROPERTIES));
}
ListBuffer<JCStatement> nullChecks = new ListBuffer<JCStatement>();
ListBuffer<JCStatement> assigns = new ListBuffer<JCStatement>();
ListBuffer<JCVariableDecl> params = new ListBuffer<JCVariableDecl>();
for (JavacNode fieldNode : fieldsToParam) {
JCVariableDecl field = (JCVariableDecl) fieldNode.get();
Name fieldName = removePrefixFromField(fieldNode);
Name rawName = field.name;
List<JCAnnotation> copyableAnnotations = findCopyableAnnotations(fieldNode);
```

```
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, typeNode.getContext());
JCExpression pType = cloneType(fieldNode.getTreeMaker(), field.vartype, source);
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags, copyableAnnotations), fieldName,
pType, null);
params.append(param);
if (hasNonNullAnnotations(fieldNode)) {
JCStatement nullCheck = generateNullCheck(maker, param, source);
if (nullCheck != null) nullChecks.append(nullCheck);
}
JCFieldAccess thisX = maker.Select(maker.Ident(fieldNode.toName("this")), rawName);
JCExpression assign = maker.Assign(thisX, maker.Ident(fieldName));
assigns.append(maker.Exec(assign));
}
for (JavacNode fieldNode : fieldsToExplicit) {
JCVariableDecl field = (JCVariableDecl) fieldNode.get();
Name rawName = field.name;
JCFieldAccess thisX = maker.Select(maker.Ident(fieldNode.toName("this")), rawName);
JCExpression assign = maker.Assign(thisX, getDefaultExpr(maker, field.vartype));
assigns.append(maker.Exec(assign));
}
for (JavacNode fieldNode : fieldsToDefault) {
JCVariableDecl field = (JCVariableDecl) fieldNode.get();
Name rawName = field.name;
Name fieldName = removePrefixFromField(fieldNode);
Name nameOfDefaultProvider = typeNode.toName("$default$" + fieldName);
JCFieldAccess thisX = maker.Select(maker.Ident(fieldNode.toName("this")), rawName);
JCExpression assign = maker.Assign(thisX, maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(((JCClassDecl) typeNode.get()).name), nameOfDefaultProvider),
List.<JCExpression>nil()));
assigns.append(maker.Exec(assign));
}
JCModifiers mods = maker.Modifiers(toJavacModifier(level), List.<JCAnnotation>nil());
if (addConstructorProperties && !isLocalType(typeNode) && LombokOptionsFactory.getDelombokOpti
ons(typeNode.getContext()).getFormatPreferences().generateConstructorProperties()) {
addConstructorProperties(mods, typeNode, fieldsToParam);
}
if (onConstructor != null) mods.annotations =
mods.annotations.appendList(copyAnnotations(onConstructor));
return recursiveSetGeneratedBy(maker.MethodDef(mods, typeNode.toName("<init>"),
null, List.<JCTypeParameter>nil(), params.toList(), List.<JCExpression>nil(),
maker.Block(0L, nullChecks.appendList(assigns).toList()), null), source);
}
/**
* For each field which is not final and has no initializer that gets 'removed' by {@code @Builder.Default}
there is no need to
* write an explicit 'this.x = foo' in the constructor, so strip them away here.
*/
private static List<JavacNode> fieldsNeedingBuilderDefaults(JavacNode typeNode, List<JavacNode>
fieldsToParam) {
ListBuffer<JavacNode> out = new ListBuffer<JavacNode>();
top:
for (JavacNode node : typeNode.down()) {
if (node.getKind() != Kind.FIELD) continue top;
```

```
JCVariableDecl varDecl = (JCVariableDecl) node.get();
if ((varDecl.mods.flags & Flags.STATIC) != 0) continue top;
for (JavacNode ftp : fieldsToParam) if (node == ftp) continue top;
if (JavacHandlerUtil.hasAnnotation(Builder.Default.class, node)) out.append(node);
}
return out.toList();
}
/**
* Return each field which is final and has no initializer, and which is not already a parameter.
*/
private static List<JavacNode> fieldsNeedingExplicitDefaults(JavacNode typeNode, List<JavacNode>
fieldsToParam) {
ListBuffer<JavacNode> out = new ListBuffer<JavacNode>();
top:
for (JavacNode node : typeNode.down()) {
if (node.getKind() != Kind.FIELD) continue top;
JCVariableDecl varDecl = (JCVariableDecl) node.get();
if (varDecl.init != null) continue top;
if ((varDecl.mods.flags & Flags.FINAL) == 0) continue top;
if ((varDecl.mods.flags & Flags.STATIC) != 0) continue top;
for (JavacNode ftp : fieldsToParam) if (node == ftp) continue top;
if (JavacHandlerUtil.hasAnnotation(Builder.Default.class, node)) continue top;
out.append(node);
}
return out.toList();
}
private static JCExpression getDefaultExpr(JavacTreeMaker maker, JCExpression type) {
if (type instanceof JCPrimitiveTypeTree) {
switch (((JCPrimitiveTypeTree) type).getPrimitiveTypeKind()) {
case BOOLEAN:
return maker.Literal(CTC_BOOLEAN, 0);
case CHAR:
return maker.Literal(CTC_CHAR, 0);
default:
case BYTE:
case SHORT:
case INT:
return maker.Literal(CTC_INT, 0);
case LONG:
return maker.Literal(CTC_LONG, 0L);
case FLOAT:
return maker.Literal(CTC_FLOAT, 0F);
case DOUBLE:
return maker.Literal(CTC_DOUBLE, 0D);
}
}
return maker.Literal(CTC_BOT, null);
}
public static boolean isLocalType(JavacNode type) {
Kind kind = type.up().getKind();
if (kind == Kind.COMPILATION_UNIT) return false;
if (kind == Kind.TYPE) return isLocalType(type.up());
return true;
```

```
}
public JCMethodDecl createStaticConstructor(String name, AccessLevel level, JavacNode typeNode,
List<JavacNode> fields, JavacNode source) {
JavacTreeMaker maker = typeNode.getTreeMaker();
JCClassDecl type = (JCClassDecl) typeNode.get();
JCModifiers mods = maker.Modifiers(Flags.STATIC | toJavacModifier(level));
JCExpression returnType, constructorType;
ListBuffer<JCTypeParameter> typeParams = new ListBuffer<JCTypeParameter>();
ListBuffer<JCVariableDecl> params = new ListBuffer<JCVariableDecl>();
ListBuffer<JCExpression> args = new ListBuffer<JCExpression>();
if (!type.typarams.isEmpty()) {
for (JCTypeParameter param : type.typarams) {
typeParams.append(maker.TypeParameter(param.name, cloneTypes(maker, param.bounds,
source)));
}
}
List<JCAnnotation> annsOnReturnType = List.nil();
if (getCheckerFrameworkVersion(typeNode).generateUnique()) annsOnReturnType =
List.of(maker.Annotation(genTypeRef(typeNode, CheckerFrameworkVersion.NAME__UNIQUE),
List.<JCExpression>nil()));
returnType = namePlusTypeParamsToTypeReference(maker, typeNode, type.typarams,
annsOnReturnType);
constructorType = namePlusTypeParamsToTypeReference(maker, typeNode, type.typarams);
for (JavacNode fieldNode : fields) {
JCVariableDecl field = (JCVariableDecl) fieldNode.get();
Name fieldName = removePrefixFromField(fieldNode);
JCExpression pType = cloneType(maker, field.vartype, source);
List<JCAnnotation> copyableAnnotations = findCopyableAnnotations(fieldNode);
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, typeNode.getContext());
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags, copyableAnnotations), fieldName,
pType, null);
params.append(param);
args.append(maker.Ident(fieldName));
}
JCReturn returnStatement = maker.Return(maker.NewClass(null, List.<JCExpression>nil(),
constructorType, args.toList(), null));
JCBlock body = maker.Block(0, List.<JCStatement>of(returnStatement));
JCMethodDecl methodDef = maker.MethodDef(mods, typeNode.toName(name), returnType,
typeParams.toList(), params.toList(), List.<JCExpression>nil(), body, null);
createRelevantNonNullAnnotation(typeNode, methodDef);
return recursiveSetGeneratedBy(methodDef, source);
}
private void generateConstructorJavadoc(JCMethodDecl constructor, JavacNode typeNode,
List<JavacNode> fields) {
if (fields.isEmpty()) return;
JCCompilationUnit cu = ((JCCompilationUnit) typeNode.top().get());
String constructorJavadoc = getConstructorJavadocHeader(typeNode.getName());
boolean fieldDescriptionAdded = false;
for (JavacNode fieldNode : fields) {
String paramName = removePrefixFromField(fieldNode).toString();
String fieldJavadoc = getDocComment(cu, fieldNode.get());
String paramJavadoc = getConstructorParameterJavadoc(paramName, fieldJavadoc);
if (paramJavadoc == null) {
```

```
paramJavadoc = "@param " + paramName;
} else {
fieldDescriptionAdded = true;
}
constructorJavadoc = addJavadocLine(constructorJavadoc, paramJavadoc);
}
if (fieldDescriptionAdded) {
setDocComment(cu, constructor, constructorJavadoc);
}
}
}
```

lombok/javac/handlers/HandleData.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.Data;
import lombok.core.AnnotationValues;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.handlers.HandleConstructor.SkipIfConstructorExists;
import lombok.spi.Provides;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.util.List;
/**
* Handles the {@code lombok.Data} annotation for javac.
*/
```

```
@Provides
public class HandleData extends JavacAnnotationHandler<Data> {
private HandleConstructor handleConstructor = new HandleConstructor();
private HandleGetter handleGetter = new HandleGetter();
private HandleSetter handleSetter = new HandleSetter();
private HandleEqualsAndHashCode handleEqualsAndHashCode = new
HandleEqualsAndHashCode();
private HandleToString handleToString = new HandleToString();
@Override public void handle(AnnotationValues<Data> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.DATA_FLAG_USAGE, "@Data");
deleteAnnotationIfNeccessary(annotationNode, Data.class);
JavacNode typeNode = annotationNode.up();
if (!isClass(typeNode)) {
annotationNode.addError("@Data is only supported on a class.");
return;
}
String staticConstructorName = annotation.getInstance().staticConstructor();
// TODO move this to the end OR move it to the top in eclipse.
handleConstructor.generateRequiredArgsConstructor(typeNode, AccessLevel.PUBLIC,
staticConstructorName, SkipIfConstructorExists.YES, annotationNode);
handleConstructor.generateExtraNoArgsConstructor(typeNode, annotationNode);
handleGetter.generateGetterForType(typeNode, annotationNode, AccessLevel.PUBLIC, true,
List.<JCAnnotation>nil());
handleSetter.generateSetterForType(typeNode, annotationNode, AccessLevel.PUBLIC, true,
List.<JCAnnotation>nil(), List.<JCAnnotation>nil());
handleEqualsAndHashCode.generateEqualsAndHashCodeForType(typeNode, annotationNode);
handleToString.generateToStringForType(typeNode, annotationNode);
}
}
```

lombok/javac/handlers/HandleDelegate.java


```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
```

* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static com.sun.tools.javac.code.Flags.*;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.lang.annotation.Annotation;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import javax.lang.model.element.ElementKind;
import javax.lang.model.element.ExecutableElement;
import javax.lang.model.element.Modifier;
import javax.lang.model.element.TypeParameterElement;
import javax.lang.model.element.VariableElement;
import javax.lang.model.type.ExecutableType;
import javax.lang.model.type.TypeKind;
import javax.lang.model.type.TypeMirror;
import com.sun.tools.javac.code.Attribute.Compound;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.code.Symbol;
import com.sun.tools.javac.code.Symbol.MethodSymbol;
import com.sun.tools.javac.code.Symbol.TypeSymbol;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.code.Type.ClassType;
import com.sun.tools.javac.code.Type.TypeVar;
import com.sun.tools.javac.code.Types;
import com.sun.tools.javac.model.JavacTypes;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.experimental.Delegate;
import lombok.javac.FindTypeVarScanner;
import lombok.javac.JavacAnnotationHandler;
```

```
import lombok.javac.JavacNode;
import lombok.javac.JavacResolution;
import lombok.javac.JavacResolution.TypeNotConvertibleException;
import lombok.javac.JavacTreeMaker;
import lombok.javac.ResolutionResetNeeded;
import lombok.permit.Permit;
import lombok.spi.Provides;
@Provides
@HandlerPriority(HandleDelegate.HANDLE_DELEGATE_PRIORITY) //2^16; to make sure that we
also delegate generated methods.
@ResolutionResetNeeded
public class HandleDelegate extends JavacAnnotationHandler<Delegate> {
private static final List<String> METHODS_IN_OBJECT = Collections.unmodifiableList(Arrays.asList(
"hashCode()",
"canEqual(java.lang.Object)", //Not in j.l.Object, but it goes with hashCode and equals so if we ignore
those two, we should ignore this one.
"equals(java.lang.Object)",
"wait()",
"wait(long)",
"wait(long, int)",
"notify()",
"notifyAll()",
"toString()",
"getClass()",
"clone()",
"finalize()"));
private static final String LEGALITY_OF_DELEGATE = "@Delegate is legal only on instance fields or
no-argument instance methods.";
private static final String RECURSION_NOT_ALLOWED = "@Delegate does not support recursion
(delegating to a type that itself has @Delegate members). Member \"%s\" is @Delegate in type \"%s\"";
public static final int HANDLE_DELEGATE_PRIORITY = 65536;
@Override public void handle(AnnotationValues<Delegate> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.DELEGATE_FLAG_USAGE,
"@Delegate");
@SuppressWarnings("deprecation") Class<? extends Annotation> oldDelegate =
lombok.Delegate.class;
deleteAnnotationIfNeccessary(annotationNode, Delegate.class, oldDelegate);
Type delegateType;
Name delegateName = annotationNode.toName(annotationNode.up().getName());
DelegateReceiver delegateReceiver;
JavacResolution reso = new JavacResolution(annotationNode.getContext());
JCTree member = annotationNode.up().get();
if (annotationNode.up().getKind() == Kind.FIELD) {
if ((((JCVariableDecl) member).mods.flags & Flags.STATIC) != 0) {
annotationNode.addError(LEGALITY_OF_DELEGATE);
return;
}
delegateReceiver = DelegateReceiver.FIELD;
delegateType = member.type;
if (delegateType == null) reso.resolveClassMember(annotationNode.up());
delegateType = member.type;
} else if (annotationNode.up().getKind() == Kind.METHOD) {
```

```
if (!(member instanceof JCMethodDecl)) {
annotationNode.addError(LEGALITY_OF_DELEGATE);
return;
}
JCMethodDecl methodDecl = (JCMethodDecl) member;
if (!methodDecl.params.isEmpty() || (methodDecl.mods.flags & Flags.STATIC) != 0) {
annotationNode.addError(LEGALITY_OF_DELEGATE);
return;
}
delegateReceiver = DelegateReceiver.METHOD;
delegateType = methodDecl.restype.type;
if (delegateType == null) reso.resolveClassMember(annotationNode.up());
delegateType = methodDecl.restype.type;
} else {
// As the annotation is legal on fields and methods only, javac itself will take care of printing an error
message for this.
return;
}
List<Object> delegateTypes = annotation.getActualExpressions("types");
List<Object> excludeTypes = annotation.getActualExpressions("excludes");
List<Type> toDelegate = new ArrayList<Type>();
List<Type> toExclude = new ArrayList<Type>();
if (delegateTypes.isEmpty()) {
if (delegateType != null) toDelegate.add(delegateType);
} else {
for (Object dt : delegateTypes) {
if (dt instanceof JCFieldAccess && ((JCFieldAccess)dt).name.toString().equals("class")) {
Type type = ((JCFieldAccess)dt).selected.type;
if (type == null) reso.resolveClassMember(annotationNode);
type = ((JCFieldAccess)dt).selected.type;
if (type != null) toDelegate.add(type);
}
}
}
for (Object et : excludeTypes) {
if (et instanceof JCFieldAccess && ((JCFieldAccess)et).name.toString().equals("class")) {
Type type = ((JCFieldAccess)et).selected.type;
if (type == null) reso.resolveClassMember(annotationNode);
type = ((JCFieldAccess)et).selected.type;
if (type != null) toExclude.add(type);
}
}
List<MethodSig> signaturesToDelegate = new ArrayList<MethodSig>();
List<MethodSig> signaturesToExclude = new ArrayList<MethodSig>();
Set<String> banList = new HashSet<String>();
banList.addAll(METHODS_IN_OBJECT);
// Add already implemented methods to ban list
JavacNode typeNode = upToTypeNode(annotationNode);
for (Symbol m : ((JCClassDecl)typeNode.get()).sym.getEnclosedElements()) {
if (m instanceof MethodSymbol) {
banList.add(printSig((ExecutableType) m.asType(), m.name, annotationNode.getTypesUtil()));
}
}
```

```
try {
for (Type t : toExclude) {
if (t instanceof ClassType) {
ClassType ct = (ClassType) t;
addMethodBindings(signaturesToExclude, ct, annotationNode.getTypesUtil(), banList);
} else {
annotationNode.addError("@Delegate can only use concrete class types, not wildcards, arrays, type
variables, or primitives.");
return;
}
}
for (MethodSig sig : signaturesToExclude) {
banList.add(printSig(sig.type, sig.name, annotationNode.getTypesUtil()));
}
for (Type t : toDelegate) {
Type unannotatedType = Unannotated.unannotatedType(t);
if (unannotatedType instanceof ClassType) {
ClassType ct = (ClassType) unannotatedType;
addMethodBindings(signaturesToDelegate, ct, annotationNode.getTypesUtil(), banList);
} else {
annotationNode.addError("@Delegate can only use concrete class types, not wildcards, arrays, type
variables, or primitives.");
return;
}
}
for (MethodSig sig : signaturesToDelegate) generateAndAdd(sig, annotationNode, delegateName,
delegateReceiver);
} catch (DelegateRecursion e) {
annotationNode.addError(String.format(RECURSION_NOT_ALLOWED, e.member, e.type));
}
}
public void generateAndAdd(MethodSig sig, JavacNode annotation, Name delegateName,
DelegateReceiver delegateReceiver) {
List<JCMethodDecl> toAdd = new ArrayList<JCMethodDecl>();
try {
toAdd.add(createDelegateMethod(sig, annotation, delegateName, delegateReceiver));
} catch (TypeNotConvertibleException e) {
annotation.addError("Can't create delegate method for " + sig.name + ": " + e.getMessage());
return;
} catch (CantMakeDelegates e) {
annotation.addError("There's a conflict in the names of type parameters. Fix it by renaming the
following type parameters of your class: " + e.conflicted);
return;
}
for (JCMethodDecl method : toAdd) {
injectMethod(annotation.up().up(), method);
}
}
public static class CantMakeDelegates extends Exception {
Set<String> conflicted;
}
/**
```

```
* There's a rare but problematic case if a delegate method has its own type variables, and the delegated type does too, and the method uses both.
* If for example the delegated type has {@code <E>}, and the method has {@code <T>}, but in our class we have a {@code <T>} at the class level, then we have two different
* type variables both named {@code T}. We detect this situation and error out asking the programmer to rename their type variable.
*
* @throws CantMakeDelegates If there's a conflict. Conflict list is in ex.conflicted.
*/
public void checkConflictOfTypeVarNames(MethodSig sig, JavacNode annotation) throws CantMakeDelegates {
// As first step, we check if there's a conflict between the delegate method's type vars and our own class.
if (sig.elem.getTypeParameters().isEmpty()) return;
Set<String> usedInOurType = new HashSet<String>();
JavacNode enclosingType = annotation;
while (enclosingType != null) {
if (enclosingType.getKind() == Kind.TYPE) {
List<JCTypeParameter> typarams = ((JCClassDecl)enclosingType.get()).typarams;
if (typarams != null) for (JCTypeParameter param : typarams) {
if (param.name != null) usedInOurType.add(param.name.toString());
}
}
enclosingType = enclosingType.up();
}
Set<String> usedInMethodSig = new HashSet<String>();
for (TypeParameterElement param : sig.elem.getTypeParameters()) {
usedInMethodSig.add(param.getSimpleName().toString());
}
usedInMethodSig.retainAll(usedInOurType);
if (usedInMethodSig.isEmpty()) return;
// We might be delegating a List<T>, and we are making method <T> toArray(). A conflict is possible.
// But only if the toArray method also uses type vars from its class, otherwise we're only shadowing,
// which is okay as we'll add a @SuppressWarnings.
FindTypeVarScanner scanner = new FindTypeVarScanner();
sig.elem.asType().accept(scanner, null);
Set<String> names = new HashSet<String>(scanner.getTypeVariables());
names.removeAll(usedInMethodSig);
if (!names.isEmpty()) {
// We have a confirmed conflict. We could dig deeper as this may still be a false alarm, but its already an exceedingly rare case.
CantMakeDelegates cmd = new CantMakeDelegates();
cmd.conflicted = usedInMethodSig;
throw cmd;
}
}
public JCMethodDecl createDelegateMethod(MethodSig sig, JavacNode annotation, Name delegateName, DelegateReceiver delegateReceiver) throws TypeNotConvertibleException, CantMakeDelegates {
/* public <T, U, ...> ReturnType methodName(ParamType1 name1, ParamType2 name2, ...) throws T1, T2, ... {
* (return) delegate.<T, U>methodName(name1, name2);
* }
```

```
*/
checkConflictOfTypeVarNames(sig, annotation);
JavacTreeMaker maker = annotation.getTreeMaker();
com.sun.tools.javac.util.List<JCAnnotation> annotations;
if (sig.isDeprecated) {
annotations = com.sun.tools.javac.util.List.of(maker.Annotation(
genJavaLangTypeRef(annotation, "Deprecated"),
com.sun.tools.javac.util.List.<JCExpression>nil()));
} else {
annotations = com.sun.tools.javac.util.List.nil();
}
JCModifiers mods = maker.Modifiers(PUBLIC, annotations);
JCExpression returnType = JavacResolution.typeToJCTree((Type) sig.type.getReturnType(),
annotation.getAst(), true);
boolean useReturn = sig.type.getReturnType().getKind() != TypeKind.VOID;
ListBuffer<JCVariableDecl> params = sig.type.getParameterTypes().isEmpty() ? null : new
ListBuffer<JCVariableDecl>();
ListBuffer<JCExpression> args = sig.type.getParameterTypes().isEmpty() ? null : new
ListBuffer<JCExpression>();
ListBuffer<JCExpression> thrown = sig.type.getThrownTypes().isEmpty() ? null : new
ListBuffer<JCExpression>();
ListBuffer<JCTypeParameter> typeParams = sig.type.getTypeVariables().isEmpty() ? null : new
ListBuffer<JCTypeParameter>();
ListBuffer<JCExpression> typeArgs = sig.type.getTypeVariables().isEmpty() ? null : new
ListBuffer<JCExpression>();
Types types = Types.instance(annotation.getContext());
for (TypeMirror param : sig.type.getTypeVariables()) {
Name name = ((TypeVar) param).tsym.name;
ListBuffer<JCExpression> bounds = new ListBuffer<JCExpression>();
for (Type type : types.getBounds((TypeVar) param)) {
bounds.append(JavacResolution.typeToJCTree(type, annotation.getAst(), true));
}
typeParams.append(maker.TypeParameter(name, bounds.toList()));
typeArgs.append(maker.Ident(name));
}
for (TypeMirror ex : sig.type.getThrownTypes()) {
thrown.append(JavacResolution.typeToJCTree((Type) ex, annotation.getAst(), true));
}
int idx = 0;
String[] paramNames = sig.getParameterNames();
boolean varargs = sig.elem.isVarArgs();
for (TypeMirror param : sig.type.getParameterTypes()) {
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, annotation.getContext());
JCModifiers paramMods = maker.Modifiers(flags);
Name name = annotation.toName(paramNames[idx++]);
if (varargs && idx == paramNames.length) {
paramMods.flags |= VARARGS;
}
params.append(maker.VarDef(paramMods, name, JavacResolution.typeToJCTree((Type) param,
annotation.getAst(), true), null));
args.append(maker.Ident(name));
}
```

```
JCExpression delegateCall = maker.Apply(toList(typeArgs),
maker.Select(delegateReceiver.get(annotation, delegateName), sig.name), toList(args));
JCStatement body = useReturn ? maker.Return(delegateCall) : maker.Exec(delegateCall);
JCBlock bodyBlock = maker.Block(0, com.sun.tools.javac.util.List.of(body));
return recursiveSetGeneratedBy(maker.MethodDef(mods, sig.name, returnType, toList(typeParams),
toList(params), toList(thrown), bodyBlock, null), annotation);
}
public static <T> com.sun.tools.javac.util.List<T> toList(ListBuffer<T> collection) {
return collection == null ? com.sun.tools.javac.util.List.<T>nil() : collection.toList();
}
private static class DelegateRecursion extends Throwable {
final String type, member;
public DelegateRecursion(String type, String member) {
this.type = type;
this.member = member;
}
}
public void addMethodBindings(List<MethodSig> signatures, ClassType ct, JavacTypes types,
Set<String> banList) throws DelegateRecursion {
TypeSymbol tsym = ct.asElement();
if (tsym == null) return;
for (Symbol member : tsym.getEnclosedElements()) {
for (Compound am : member.getAnnotationMirrors()) {
String name = null;
try {
name = am.type.tsym.flatName().toString();
} catch (Exception ignore) {}
if ("lombok.Delegate".equals(name) || "lombok.experimental.Delegate".equals(name)) {
throw new DelegateRecursion(ct.tsym.name.toString(), member.name.toString());
}
}
if (member.getKind() != ElementKind.METHOD) continue;
if (member.isStatic()) continue;
if (member.isConstructor()) continue;
ExecutableElement exElem = (ExecutableElement)member;
if (!exElem.getModifiers().contains(Modifier.PUBLIC)) continue;
ExecutableType methodType = (ExecutableType) types.asMemberOf(ct, member);
String sig = printSig(methodType, member.name, types);
if (!banList.add(sig)) continue; //If add returns false, it was already in there
boolean isDeprecated = (member.flags() & DEPRECATED) != 0;
signatures.add(new MethodSig(member.name, methodType, isDeprecated, exElem));
}
for (Type type : types.directSupertypes(ct)) {
if (type instanceof ClassType) {
addMethodBindings(signatures, (ClassType) type, types, banList);
}
}
}
public static class MethodSig {
final Name name;
final ExecutableType type;
final boolean isDeprecated;
final ExecutableElement elem;
```

```
MethodSig(Name name, ExecutableType type, boolean isDeprecated, ExecutableElement elem) {
this.name = name;
this.type = type;
this.isDeprecated = isDeprecated;
this.elem = elem;
}
String[] getParameterNames() {
List<? extends VariableElement> paramList = elem.getParameters();
String[] paramNames = new String[paramList.size()];
for (int i = 0; i < paramNames.length; i++) {
paramNames[i] = paramList.get(i).getSimpleName().toString();
}
return paramNames;
}
@Override public String toString() {
return (isDeprecated ? "@Deprecated " : "") + name + " " + type;
}
}
public static String printSig(ExecutableType method, Name name, JavacTypes types) {
StringBuilder sb = new StringBuilder();
sb.append(name.toString()).append("(");
boolean first = true;
for (TypeMirror param : method.getParameterTypes()) {
if (!first) sb.append(", ");
first = false;
sb.append(typeBindingToSignature(param, types));
}
return sb.append(")").toString();
}
public static String typeBindingToSignature(TypeMirror binding, JavacTypes types) {
binding = types.erasure(binding);
return binding.toString();
}
public enum DelegateReceiver {
METHOD {
public JCExpression get(final JavacNode node, final Name name) {
com.sun.tools.javac.util.List<JCExpression> nilExprs = com.sun.tools.javac.util.List.nil();
final JavacTreeMaker maker = node.getTreeMaker();
return maker.Apply(nilExprs, maker.Select(maker.Ident(node.toName("this")), name), nilExprs);
}
},
FIELD {
public JCExpression get(final JavacNode node, final Name name) {
final JavacTreeMaker maker = node.getTreeMaker();
return maker.Select(maker.Ident(node.toName("this")), name);
}
};
public abstract JCExpression get(final JavacNode node, final Name name);
}
private static class Unannotated {
private static final Method unannotated;
static {
Method m = null;
```

```
try {
m = Permit.getMethod(Type.class, "unannotatedType");
} catch (Exception e) {/* ignore */}
unannotated = m;
}
static Type unannotatedType(Type t) {
if (unannotated == null) return t;
try {
return (Type) Permit.invoke(unannotated, t);
} catch (Exception e) {
return t;
}
}
}
}
```

lombok/javac/handlers/HandleEqualsAndHashCode.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import com.sun.tools.javac.code.BoundKind;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
```

```
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCBinary;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCUnary;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.ConfigurationKeys;
import lombok.EqualsAndHashCode;
import lombok.EqualsAndHashCode.CacheStrategy;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CallSuperType;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.core.handlers.InclusionExclusionUtils;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.EqualsAndHashCode} annotation for javac.
*/
@Provides
public class HandleEqualsAndHashCode extends JavacAnnotationHandler<EqualsAndHashCode> {
private static final String RESULT_NAME = "result";
private static final String PRIME_NAME = "PRIME";
private static final String HASH_CODE_CACHE_NAME = "$hashCodeCache";
@Override public void handle(AnnotationValues<EqualsAndHashCode> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.EQUALS_AND_HASH_CODE_FLAG_USAGE,
"@EqualsAndHashCode");
deleteAnnotationIfNeccessary(annotationNode, EqualsAndHashCode.class);
deleteImportFromCompilationUnit(annotationNode, CacheStrategy.class.getName());
EqualsAndHashCode ann = annotation.getInstance();
java.util.List<Included<JavacNode, EqualsAndHashCode.Include>> members =
InclusionExclusionUtils.handleEqualsAndHashCodeMarking(annotationNode.up(), annotation,
annotationNode);
```

```
JavacNode typeNode = annotationNode.up();
List<JCAnnotation> onParam = unboxAndRemoveAnnotationParameter(ast, "onParam",
"@EqualsAndHashCode(onParam", annotationNode);
Boolean callSuper = ann.callSuper();
if (!annotation.isExplicit("callSuper")) callSuper = null;
Boolean doNotUseGettersConfiguration = annotationNode.getAst().readConfiguration(ConfigurationKe
ys.EQUALS_AND_HASH_CODE_DO_NOT_USE_GETTERS);
boolean doNotUseGetters = annotation.isExplicit("doNotUseGetters") || doNotUseGettersConfiguration
== null ? ann.doNotUseGetters() : doNotUseGettersConfiguration;
FieldAccess fieldAccess = doNotUseGetters ? FieldAccess.PREFER_FIELD : FieldAccess.GETTER;
boolean cacheHashCode = ann.cacheStrategy() == CacheStrategy.LAZY;
generateMethods(typeNode, annotationNode, members, callSuper, true, cacheHashCode, fieldAccess,
onParam);
}
public void generateEqualsAndHashCodeForType(JavacNode typeNode, JavacNode source) {
if (hasAnnotation(EqualsAndHashCode.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
Boolean doNotUseGettersConfiguration = typeNode.getAst().readConfiguration(ConfigurationKeys.EQ
UALS_AND_HASH_CODE_DO_NOT_USE_GETTERS);
FieldAccess access = doNotUseGettersConfiguration == null || !doNotUseGettersConfiguration ?
FieldAccess.GETTER : FieldAccess.PREFER_FIELD;
java.util.List<Included<JavacNode, EqualsAndHashCode.Include>> members =
InclusionExclusionUtils.handleEqualsAndHashCodeMarking(typeNode, null, null);
generateMethods(typeNode, source, members, null, false, false, access, List.<JCAnnotation>nil());
}
public void generateMethods(JavacNode typeNode, JavacNode source,
java.util.List<Included<JavacNode, EqualsAndHashCode.Include>> members,
Boolean callSuper, boolean whineIfExists, boolean cacheHashCode, FieldAccess fieldAccess,
List<JCAnnotation> onParam) {
if (!isClass(typeNode)) {
source.addError("@EqualsAndHashCode is only supported on a class.");
return;
}
boolean implicitCallSuper = callSuper == null;
if (callSuper == null) {
try {
callSuper = ((Boolean)
EqualsAndHashCode.class.getMethod("callSuper").getDefaultValue()).booleanValue();
} catch (Exception ignore) {
throw new InternalError("Lombok bug - this cannot happen - can't find callSuper field in
EqualsAndHashCode annotation.");
}
}
boolean isDirectDescendantOfObject = isDirectDescendantOfObject(typeNode);
boolean isFinal = (((JCClassDecl) typeNode.get()).mods.flags & Flags.FINAL) != 0;
boolean needsCanEqual = !isFinal || !isDirectDescendantOfObject;
MemberExistsResult equalsExists = methodExists("equals", typeNode, 1);
MemberExistsResult hashCodeExists = methodExists("hashCode", typeNode, 0);
MemberExistsResult canEqualExists = methodExists("canEqual", typeNode, 1);
switch (Collections.max(Arrays.asList(equalsExists, hashCodeExists))) {
case EXISTS_BY_LOMBOK:
```

```
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String msg = "Not generating equals and hashCode: A method with one of those names already exists.
(Either both or none of these methods will be generated).";
source.addWarning(msg);
} else if (equalsExists == MemberExistsResult.NOT_EXISTS || hashCodeExists ==
MemberExistsResult.NOT_EXISTS) {
// This means equals OR hashCode exists and not both.
// Even though we should suppress the message about not generating these, this is such a weird and
surprising situation we should ALWAYS generate a warning.
// The user code couldn't possibly (barring really weird subclassing shenanigans) be in a shippable
state anyway; the implementations of these 2 methods are
// all inter-related and should be written by the same entity.
String msg = String.format("Not generating %s: One of equals or hashCode exists. " +
"You should either write both of these or none of these (in the latter case, lombok generates them).",
equalsExists == MemberExistsResult.NOT_EXISTS ? "equals" : "hashCode");
source.addWarning(msg);
}
return;
case NOT_EXISTS:
default:
//fallthrough
}
if (isDirectDescendantOfObject && callSuper) {
source.addError("Generating equals/hashCode with a supercall to java.lang.Object is pointless.");
return;
}
if (implicitCallSuper && !isDirectDescendantOfObject) {
CallSuperType cst = typeNode.getAst().readConfiguration(ConfigurationKeys.EQUALS_AND_HASH_
CODE_CALL_SUPER);
if (cst == null) cst = CallSuperType.WARN;
switch (cst) {
default:
case WARN:
source.addWarning("Generating equals/hashCode implementation but without a call to superclass,
even though this class does not extend java.lang.Object. If this is intentional, add
'@EqualsAndHashCode(callSuper=false)' to your type.");
callSuper = false;
break;
case SKIP:
callSuper = false;
break;
case CALL:
callSuper = true;
break;
}
}
JCMethodDecl equalsMethod = createEquals(typeNode, members, callSuper, fieldAccess,
needsCanEqual, source, onParam);
injectMethod(typeNode, equalsMethod);
if (needsCanEqual && canEqualExists == MemberExistsResult.NOT_EXISTS) {
JCMethodDecl canEqualMethod = createCanEqual(typeNode, source, copyAnnotations(onParam));
```

```
injectMethod(typeNode, canEqualMethod);
}
if (cacheHashCode){
if (fieldExists(HASH_CODE_CACHE_NAME, typeNode) != MemberExistsResult.NOT_EXISTS) {
String msg = String.format("Not caching the result of hashCode: A field named %s already exists.",
HASH_CODE_CACHE_NAME);
source.addWarning(msg);
cacheHashCode = false;
} else {
createHashCodeCacheField(typeNode, source);
}
}
JCMethodDecl hashCodeMethod = createHashCode(typeNode, members, callSuper, cacheHashCode,
fieldAccess, source);
injectMethod(typeNode, hashCodeMethod);
}
private void createHashCodeCacheField(JavacNode typeNode, JavacNode source) {
JavacTreeMaker maker = typeNode.getTreeMaker();
JCModifiers mods = maker.Modifiers(Flags.PRIVATE | Flags.TRANSIENT);
JCVariableDecl hashCodeCacheField = maker.VarDef(mods,
typeNode.toName(HASH_CODE_CACHE_NAME), maker.TypeIdent(CTC_INT), null);
injectFieldAndMarkGenerated(typeNode, hashCodeCacheField);
recursiveSetGeneratedBy(hashCodeCacheField, source);
}
public JCMethodDecl createHashCode(JavacNode typeNode, java.util.List<Included<JavacNode,
EqualsAndHashCode.Include>> members, boolean callSuper, boolean cacheHashCode, FieldAccess
fieldAccess, JavacNode source) {
JavacTreeMaker maker = typeNode.getTreeMaker();
JCAnnotation overrideAnnotation = maker.Annotation(genJavaLangTypeRef(typeNode, "Override"),
List.<JCExpression>nil());
List<JCAnnotation> annsOnMethod = List.of(overrideAnnotation);
CheckerFrameworkVersion checkerFramework = getCheckerFrameworkVersion(typeNode);
if (cacheHashCode && checkerFramework.generatePure()) {
annsOnMethod = annsOnMethod.prepend(maker.Annotation(genTypeRef(typeNode,
CheckerFrameworkVersion.NAME__PURE), List.<JCExpression>nil()));
} else if (checkerFramework.generateSideEffectFree()) {
annsOnMethod = annsOnMethod.prepend(maker.Annotation(genTypeRef(typeNode,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
}
JCModifiers mods = maker.Modifiers(Flags.PUBLIC, annsOnMethod);
JCExpression returnType = maker.TypeIdent(CTC_INT);
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
Name primeName = typeNode.toName(PRIME_NAME);
Name resultName = typeNode.toName(RESULT_NAME);
long finalFlag = JavacHandlerUtil.addFinalIfNeeded(0L, typeNode.getContext());
boolean isEmpty = members.isEmpty();
/* if (this.$hashCodeCache != 0) return this.$hashCodeCache; */ {
if (cacheHashCode) {
JCFieldAccess hashCodeCacheFieldAccess = createHashCodeCacheFieldAccess(typeNode, maker);
JCExpression cacheNotZero = maker.Binary(CTC_NOT_EQUAL, hashCodeCacheFieldAccess,
maker.Literal(CTC_INT, 0));
hashCodeCacheFieldAccess = createHashCodeCacheFieldAccess(typeNode, maker);
statements.append(maker.If(cacheNotZero, maker.Return(hashCodeCacheFieldAccess), null));
```

```
}
}
/* final int PRIME = X; */ {
if (!isEmpty) {
statements.append(maker.VarDef(maker.Modifiers(finalFlag), primeName,
maker.TypeIdent(CTC_INT), maker.Literal(HandlerUtil.primeForHashcode())));
}
}
/* int result = ... */ {
final JCExpression init;
if (callSuper) {
/* ... super.hashCode(); */
init = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(typeNode.toName("super")), typeNode.toName("hashCode")),
List.<JCExpression>nil());
} else {
/* ... 1; */
init = maker.Literal(1);
}
statements.append(maker.VarDef(maker.Modifiers(isEmpty && !cacheHashCode ? finalFlag : 0L),
resultName, maker.TypeIdent(CTC_INT), init));
}
for (Included<JavacNode, EqualsAndHashCode.Include> member : members) {
JavacNode memberNode = member.getNode();
JCExpression fType = removeTypeUseAnnotations(getFieldType(memberNode, fieldAccess));
boolean isMethod = memberNode.getKind() == Kind.METHOD;
JCExpression fieldAccessor = isMethod ? createMethodAccessor(maker, memberNode) :
createFieldAccessor(maker, memberNode, fieldAccess);
if (fType instanceof JCPrimitiveTypeTree) {
switch (((JCPrimitiveTypeTree) fType).getPrimitiveTypeKind()) {
case BOOLEAN:
/* this.fieldName ? X : Y */
statements.append(createResultCalculation(typeNode,
maker.Parens(maker.Conditional(fieldAccessor,
maker.Literal(HandlerUtil.primeForTrue()), maker.Literal(HandlerUtil.primeForFalse())))));
break;
case LONG: {
Name dollarFieldName = memberNode.toName((isMethod ? "$$" : "$") + memberNode.getName());
statements.append(maker.VarDef(maker.Modifiers(finalFlag), dollarFieldName,
maker.TypeIdent(CTC_LONG), fieldAccessor));
statements.append(createResultCalculation(typeNode, longToIntForHashCode(maker,
maker.Ident(dollarFieldName), maker.Ident(dollarFieldName))));
}
break;
case FLOAT:
/* Float.floatToIntBits(this.fieldName) */
statements.append(createResultCalculation(typeNode, maker.Apply(
List.<JCExpression>nil(),
genJavaLangTypeRef(typeNode, "Float", "floatToIntBits"),
List.of(fieldAccessor))));
break;
case DOUBLE: {
/* longToIntForHashCode(Double.doubleToLongBits(this.fieldName)) */
```

```
Name dollarFieldName = memberNode.toName((isMethod ? "$$" : "$") + memberNode.getName());
JCExpression init = maker.Apply(
List.<JCExpression>nil(),
genJavaLangTypeRef(typeNode, "Double", "doubleToLongBits"),
List.of(fieldAccessor));
statements.append(maker.VarDef(maker.Modifiers(finalFlag), dollarFieldName,
maker.TypeIdent(CTC_LONG), init));
statements.append(createResultCalculation(typeNode, longToIntForHashCode(maker,
maker.Ident(dollarFieldName), maker.Ident(dollarFieldName))));
}
break;
default:
case BYTE:
case SHORT:
case INT:
case CHAR:
/* just the field */
statements.append(createResultCalculation(typeNode, fieldAccessor));
break;
}
} else if (fType instanceof JCArrayTypeTree) {
JCArrayTypeTree array = (JCArrayTypeTree) fType;
/* java.util.Arrays.deepHashCode(this.fieldName) //use just hashCode() for primitive arrays. */
boolean multiDim = removeTypeUseAnnotations(array.elemtype) instanceof JCArrayTypeTree;
boolean primitiveArray = removeTypeUseAnnotations(array.elemtype) instanceof
JCPrimitiveTypeTree;
boolean useDeepHC = multiDim || !primitiveArray;
JCExpression hcMethod = chainDots(typeNode, "java", "util", "Arrays", useDeepHC ? "deepHashCode"
: "hashCode");
statements.append(createResultCalculation(typeNode, maker.Apply(List.<JCExpression>nil(),
hcMethod, List.of(fieldAccessor))));
} else /* objects */ {
/* final java.lang.Object $fieldName = this.fieldName; */
/* ($fieldName == null ? NULL_PRIME : $fieldName.hashCode()) */
Name dollarFieldName = memberNode.toName((isMethod ? "$$" : "$") + memberNode.getName());
statements.append(maker.VarDef(maker.Modifiers(finalFlag), dollarFieldName,
genJavaLangTypeRef(typeNode, "Object"), fieldAccessor));
JCExpression hcCall = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(dollarFieldName), typeNode.toName("hashCode")),
List.<JCExpression>nil());
JCExpression thisEqualsNull = maker.Binary(CTC_EQUAL, maker.Ident(dollarFieldName),
maker.Literal(CTC_BOT, null));
statements.append(createResultCalculation(typeNode,
maker.Parens(maker.Conditional(thisEqualsNull, maker.Literal(HandlerUtil.primeForNull()), hcCall))));
}
}
/*
* if (result == 0) result = Integer.MIN_VALUE;
* this.$hashCodeCache = result;
*
*/ {
if (cacheHashCode) {
```

```
statements.append(maker.If(maker.Binary(CTC_EQUAL, maker.Ident(resultName),
maker.Literal(CTC_INT, 0)),
maker.Exec(maker.Assign(maker.Ident(resultName), genJavaLangTypeRef(typeNode, "Integer",
"MIN_VALUE"))), null));
JCFieldAccess cacheHashCodeFieldAccess = createHashCodeCacheFieldAccess(typeNode, maker);
statements.append(maker.Exec(maker.Assign(cacheHashCodeFieldAccess,
maker.Ident(resultName))));
}
}
/* return result; */ {
statements.append(maker.Return(maker.Ident(resultName)));
}
JCBlock body = maker.Block(0, statements.toList());
return recursiveSetGeneratedBy(maker.MethodDef(mods, typeNode.toName("hashCode"), returnType,
List.<JCTypeParameter>nil(), List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null), source);
}
private JCFieldAccess createHashCodeCacheFieldAccess(JavacNode typeNode, JavacTreeMaker
maker) {
JCIdent receiver = maker.Ident(typeNode.toName("this"));
JCFieldAccess cacheHashCodeFieldAccess = maker.Select(receiver,
typeNode.toName(HASH_CODE_CACHE_NAME));
return cacheHashCodeFieldAccess;
}
public JCExpressionStatement createResultCalculation(JavacNode typeNode, JCExpression expr) {
/* result = result * PRIME + expr; */
JavacTreeMaker maker = typeNode.getTreeMaker();
Name resultName = typeNode.toName(RESULT_NAME);
JCExpression mult = maker.Binary(CTC_MUL, maker.Ident(resultName),
maker.Ident(typeNode.toName(PRIME_NAME)));
JCExpression add = maker.Binary(CTC_PLUS, mult, expr);
return maker.Exec(maker.Assign(maker.Ident(resultName), add));
}
/** The 2 references must be clones of each other. */
public JCExpression longToIntForHashCode(JavacTreeMaker maker, JCExpression ref1,
JCExpression ref2) {
/* (int) (ref >>> 32 ^ ref) */
JCExpression shift = maker.Binary(CTC_UNSIGNED_SHIFT_RIGHT, ref1, maker.Literal(32));
JCExpression xorBits = maker.Binary(CTC_BITXOR, shift, ref2);
return maker.TypeCast(maker.TypeIdent(CTC_INT), maker.Parens(xorBits));
}
public JCExpression createTypeReference(JavacNode type, boolean addWildcards) {
java.util.List<String> list = new ArrayList<String>();
java.util.List<Integer> genericsCount = addWildcards ? new ArrayList<Integer>() : null;
list.add(type.getName());
if (addWildcards) genericsCount.add(((JCClassDecl) type.get()).typarams.size());
boolean staticContext = (((JCClassDecl) type.get()).getModifiers().flags & Flags.STATIC) != 0;
JavacNode tNode = type.up();
while (tNode != null && tNode.getKind() == Kind.TYPE && !tNode.getName().isEmpty()) {
list.add(tNode.getName());
if (addWildcards) genericsCount.add(staticContext ? 0 : ((JCClassDecl) tNode.get()).typarams.size());
if (!staticContext) staticContext = (((JCClassDecl) tNode.get()).getModifiers().flags & Flags.STATIC) !=
0;
tNode = tNode.up();
```

```
}
Collections.reverse(list);
if (addWildcards) Collections.reverse(genericsCount);
JavacTreeMaker maker = type.getTreeMaker();
JCExpression chain = maker.Ident(type.toName(list.get(0)));
if (addWildcards) chain = wildcardify(maker, chain, genericsCount.get(0));
for (int i = 1; i < list.size(); i++) {
chain = maker.Select(chain, type.toName(list.get(i)));
if (addWildcards) chain = wildcardify(maker, chain, genericsCount.get(i));
}
return chain;
}
private JCExpression wildcardify(JavacTreeMaker maker, JCExpression expr, int count) {
if (count == 0) return expr;
ListBuffer<JCExpression> wildcards = new ListBuffer<JCExpression>();
for (int i = 0 ; i < count ; i++) {
wildcards.append(maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null));
}
return maker.TypeApply(expr, wildcards.toList());
}
public JCMethodDecl createEquals(JavacNode typeNode, java.util.List<Included<JavacNode,
EqualsAndHashCode.Include>> members, boolean callSuper, FieldAccess fieldAccess, boolean
needsCanEqual, JavacNode source, List<JCAnnotation> onParam) {
JavacTreeMaker maker = typeNode.getTreeMaker();
Name oName = typeNode.toName("o");
Name otherName = typeNode.toName("other");
Name thisName = typeNode.toName("this");
List<JCAnnotation> annsOnParamOnMethod = List.nil();
JCAnnotation overrideAnnotation = maker.Annotation(genJavaLangTypeRef(typeNode, "Override"),
List.<JCExpression>nil());
List<JCAnnotation> annsOnMethod = List.of(overrideAnnotation);
CheckerFrameworkVersion checkerFramework = getCheckerFrameworkVersion(typeNode);
if (checkerFramework.generateSideEffectFree()) {
annsOnMethod = annsOnMethod.prepend(maker.Annotation(genTypeRef(typeNode,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
}
JCModifiers mods = maker.Modifiers(Flags.PUBLIC, annsOnMethod);
JCExpression objectType;
if (annsOnParamOnMethod.isEmpty()) {
objectType = genJavaLangTypeRef(typeNode, "Object");
} else {
objectType = chainDots(typeNode, "java", "lang", "Object");
objectType = maker.AnnotatedType(annsOnParamOnMethod, objectType);
}
JCExpression returnType = maker.TypeIdent(CTC_BOOLEAN);
long finalFlag = JavacHandlerUtil.addFinalIfNeeded(0L, typeNode.getContext());
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
JCVariableDecl param = maker.VarDef(maker.Modifiers(finalFlag | Flags.PARAMETER, onParam),
oName, objectType, null);
JavacHandlerUtil.createRelevantNullableAnnotation(typeNode, param);
final List<JCVariableDecl> params = List.of(param);
/* if (o == this) return true; */ {
statements.append(maker.If(maker.Binary(CTC_EQUAL, maker.Ident(oName),
```

```
maker.Ident(thisName)), returnBool(maker, true), null));
}
/* if (!(o instanceof Outer.Inner.MyType)) return false; */ {
JCUnary notInstanceOf = maker.Unary(CTC_NOT,
maker.Parens(maker.TypeTest(maker.Ident(oName), createTypeReference(typeNode, false))));
statements.append(maker.If(notInstanceOf, returnBool(maker, false), null));
}
/* Outer.Inner.MyType<?> other = (Outer.Inner.MyType<?>) o; */ {
if (!members.isEmpty() || needsCanEqual) {
final JCExpression selfType1 = createTypeReference(typeNode, true), selfType2 =
createTypeReference(typeNode, true);
statements.append(
maker.VarDef(maker.Modifiers(finalFlag), otherName, selfType1, maker.TypeCast(selfType2,
maker.Ident(oName))));
}
}
/* if (!other.canEqual((java.lang.Object) this)) return false; */ {
if (needsCanEqual) {
List<JCExpression> exprNil = List.nil();
JCExpression thisRef = maker.Ident(thisName);
JCExpression castThisRef = maker.TypeCast(genJavaLangTypeRef(typeNode, "Object"), thisRef);
JCExpression equalityCheck = maker.Apply(exprNil,
maker.Select(maker.Ident(otherName), typeNode.toName("canEqual")),
List.of(castThisRef));
statements.append(maker.If(maker.Unary(CTC_NOT, equalityCheck), returnBool(maker, false), null));
}
}
/* if (!super.equals(o)) return false; */
if (callSuper) {
JCMethodInvocation callToSuper = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(typeNode.toName("super")), typeNode.toName("equals")),
List.<JCExpression>of(maker.Ident(oName)));
JCUnary superNotEqual = maker.Unary(CTC_NOT, callToSuper);
statements.append(maker.If(superNotEqual, returnBool(maker, false), null));
}
for (Included<JavacNode, EqualsAndHashCode.Include> member : members) {
JavacNode memberNode = member.getNode();
boolean isMethod = memberNode.getKind() == Kind.METHOD;
JCExpression fType = removeTypeUseAnnotations(getFieldType(memberNode, fieldAccess));
JCExpression thisFieldAccessor = isMethod ? createMethodAccessor(maker, memberNode) :
createFieldAccessor(maker, memberNode, fieldAccess);
JCExpression otherFieldAccessor = isMethod ? createMethodAccessor(maker, memberNode,
maker.Ident(otherName)) : createFieldAccessor(maker, memberNode, fieldAccess,
maker.Ident(otherName));
if (fType instanceof JCPrimitiveTypeTree) {
switch (((JCPrimitiveTypeTree)fType).getPrimitiveTypeKind()) {
case FLOAT:
/* if (Float.compare(this.fieldName, other.fieldName) != 0) return false; */
statements.append(generateCompareFloatOrDouble(thisFieldAccessor, otherFieldAccessor, maker,
typeNode, false));
break;
case DOUBLE:
/* if (Double.compare(this.fieldName, other.fieldName) != 0) return false; */
```

```
statements.append(generateCompareFloatOrDouble(thisFieldAccessor, otherFieldAccessor, maker,
typeNode, true));
break;
default:
/* if (this.fieldName != other.fieldName) return false; */
statements.append(
maker.If(maker.Binary(CTC_NOT_EQUAL, thisFieldAccessor, otherFieldAccessor), returnBool(maker,
false), null));
break;
}
} else if (fType instanceof JCArrayTypeTree) {
JCArrayTypeTree array = (JCArrayTypeTree) fType;
/* if (!java.util.Arrays.deepEquals(this.fieldName, other.fieldName)) return false; //use equals for
primitive arrays. */
boolean multiDim = removeTypeUseAnnotations(array.elemtype) instanceof JCArrayTypeTree;
boolean primitiveArray = removeTypeUseAnnotations(array.elemtype) instanceof
JCPrimitiveTypeTree;
boolean useDeepEquals = multiDim || !primitiveArray;
JCExpression eqMethod = chainDots(typeNode, "java", "util", "Arrays", useDeepEquals ? "deepEquals"
: "equals");
List<JCExpression> args = List.of(thisFieldAccessor, otherFieldAccessor);
statements.append(maker.If(maker.Unary(CTC_NOT,
maker.Apply(List.<JCExpression>nil(), eqMethod, args)), returnBool(maker, false), null));
} else /* objects */ {
/* final java.lang.Object this$fieldName = this.fieldName; */
/* final java.lang.Object other$fieldName = other.fieldName; */
/* if (this$fieldName == null ? other$fieldName != null : !this$fieldName.equals(other$fieldName)) return
false; */
Name thisDollarFieldName = memberNode.toName("this" + (isMethod ? "$$" : "$") +
memberNode.getName());
Name otherDollarFieldName = memberNode.toName("other" + (isMethod ? "$$" : "$") +
memberNode.getName());
statements.append(maker.VarDef(maker.Modifiers(finalFlag), thisDollarFieldName,
genJavaLangTypeRef(typeNode, "Object"), thisFieldAccessor));
statements.append(maker.VarDef(maker.Modifiers(finalFlag), otherDollarFieldName,
genJavaLangTypeRef(typeNode, "Object"), otherFieldAccessor));
JCExpression thisEqualsNull = maker.Binary(CTC_EQUAL, maker.Ident(thisDollarFieldName),
maker.Literal(CTC_BOT, null));
JCExpression otherNotEqualsNull = maker.Binary(CTC_NOT_EQUAL,
maker.Ident(otherDollarFieldName), maker.Literal(CTC_BOT, null));
JCExpression thisEqualsThat = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(thisDollarFieldName), typeNode.toName("equals")),
List.<JCExpression>of(maker.Ident(otherDollarFieldName)));
JCExpression fieldsAreNotEqual = maker.Conditional(thisEqualsNull, otherNotEqualsNull,
maker.Unary(CTC_NOT, thisEqualsThat));
statements.append(maker.If(fieldsAreNotEqual, returnBool(maker, false), null));
}
}
/* return true; */ {
statements.append(returnBool(maker, true));
}
JCBlock body = maker.Block(0, statements.toList());
```

```
return recursiveSetGeneratedBy(maker.MethodDef(mods, typeNode.toName("equals"), returnType,
List.<JCTypeParameter>nil(), params, List.<JCExpression>nil(), body, null), source);
}
public JCMethodDecl createCanEqual(JavacNode typeNode, JavacNode source, List<JCAnnotation>
onParam) {
/* protected boolean canEqual(final java.lang.Object other) {
* return other instanceof Outer.Inner.MyType;
* }
*/
JavacTreeMaker maker = typeNode.getTreeMaker();
List<JCAnnotation> annsOnMethod = List.nil();
CheckerFrameworkVersion checkerFramework = getCheckerFrameworkVersion(typeNode);
if (checkerFramework.generatePure()) {
annsOnMethod = annsOnMethod.prepend(maker.Annotation(genTypeRef(typeNode,
CheckerFrameworkVersion.NAME__PURE), List.<JCExpression>nil()));
}
JCModifiers mods = maker.Modifiers(Flags.PROTECTED, annsOnMethod);
JCExpression returnType = maker.TypeIdent(CTC_BOOLEAN);
Name canEqualName = typeNode.toName("canEqual");
JCExpression objectType = genJavaLangTypeRef(typeNode, "Object");
Name otherName = typeNode.toName("other");
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, typeNode.getContext());
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags, onParam), otherName, objectType,
null);
createRelevantNullableAnnotation(typeNode, param);
List<JCVariableDecl> params = List.of(param);
JCBlock body = maker.Block(0, List.<JCStatement>of(
maker.Return(maker.TypeTest(maker.Ident(otherName), createTypeReference(typeNode, false)))));
return recursiveSetGeneratedBy(maker.MethodDef(mods, canEqualName, returnType,
List.<JCTypeParameter>nil(), params, List.<JCExpression>nil(), body, null), source);
}
public JCStatement generateCompareFloatOrDouble(JCExpression thisDotField, JCExpression
otherDotField,
JavacTreeMaker maker, JavacNode node, boolean isDouble) {
/* if (Float.compare(fieldName, other.fieldName) != 0) return false; */
JCExpression clazz = genJavaLangTypeRef(node, isDouble ? "Double" : "Float");
List<JCExpression> args = List.of(thisDotField, otherDotField);
JCBinary compareCallEquals0 = maker.Binary(CTC_NOT_EQUAL, maker.Apply(
List.<JCExpression>nil(), maker.Select(clazz, node.toName("compare")), args), maker.Literal(0));
return maker.If(compareCallEquals0, returnBool(maker, false), null);
}
public JCStatement returnBool(JavacTreeMaker maker, boolean bool) {
return maker.Return(maker.Literal(CTC_BOOLEAN, bool ? 1 : 0));
}
}
```

lombok/javac/handlers/HandleExtensionMethod.java

```
/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
```

* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static com.sun.tools.javac.code.Flags.*;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import static lombok.javac.handlers.JavacResolver.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import javax.lang.model.element.ElementKind;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.experimental.ExtensionMethod;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacResolution;
import lombok.spi.Provides;
import com.sun.source.tree.MethodInvocationTree;
import com.sun.source.util.TreeScanner;
import com.sun.tools.javac.code.Symbol;
import com.sun.tools.javac.code.Symbol.ClassSymbol;
import com.sun.tools.javac.code.Symbol.MethodSymbol;
import com.sun.tools.javac.code.Symbol.TypeSymbol;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.code.Type.ClassType;
import com.sun.tools.javac.code.Type.ErrorType;
import com.sun.tools.javac.code.Type.ForAll;
import com.sun.tools.javac.code.Type.MethodType;
import com.sun.tools.javac.code.Types;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
```

```
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
/**
* Handles the {@link ExtensionMethod} annotation for javac.
*/
@Provides
@HandlerPriority(66560) // 2^16 + 2^10; we must run AFTER HandleVal which is at 2^16
public class HandleExtensionMethod extends JavacAnnotationHandler<ExtensionMethod> {
@Override
public void handle(final AnnotationValues<ExtensionMethod> annotation, final JCAnnotation source,
final JavacNode annotationNode) {
handleExperimentalFlagUsage(annotationNode,
ConfigurationKeys.EXTENSION_METHOD_FLAG_USAGE, "@ExtensionMethod");
deleteAnnotationIfNeccessary(annotationNode, ExtensionMethod.class);
JavacNode typeNode = annotationNode.up();
boolean isClassEnumInterfaceOrRecord = isClassEnumInterfaceOrRecord(typeNode);
if (!isClassEnumInterfaceOrRecord) {
annotationNode.addError("@ExtensionMethod can only be used on a class, an enum, an interface or a
record");
return;
}
boolean suppressBaseMethods = annotation.getInstance().suppressBaseMethods();
List<Object> extensionProviders = annotation.getActualExpressions("value");
if (extensionProviders.isEmpty()) {
annotationNode.addError(String.format("@%s has no effect since no extension types were specified.",
ExtensionMethod.class.getName()));
return;
}
final List<Extension> extensions = getExtensions(annotationNode, extensionProviders);
if (extensions.isEmpty()) return;
new ExtensionMethodReplaceVisitor(annotationNode, extensions, suppressBaseMethods).replace();
annotationNode.rebuild();
}
public List<Extension> getExtensions(final JavacNode typeNode, final List<Object>
extensionProviders) {
List<Extension> extensions = new ArrayList<Extension>();
for (Object extensionProvider : extensionProviders) {
if (!(extensionProvider instanceof JCFieldAccess)) continue;
JCFieldAccess provider = (JCFieldAccess) extensionProvider;
if (!("class".equals(provider.name.toString()))) continue;
Type providerType = CLASS.resolveMember(typeNode, provider.selected);
if (providerType == null) continue;
if ((providerType.tsym.flags() & (INTERFACE | ANNOTATION)) != 0) continue;
extensions.add(getExtension(typeNode, (ClassType) providerType));
}
return extensions;
}
public Extension getExtension(final JavacNode typeNode, final ClassType
extensionMethodProviderType) {
List<MethodSymbol> extensionMethods = new ArrayList<MethodSymbol>();
TypeSymbol tsym = extensionMethodProviderType.asElement();
if (tsym != null) for (Symbol member : tsym.getEnclosedElements()) {
if (member.getKind() != ElementKind.METHOD) continue;
MethodSymbol method = (MethodSymbol) member;
```

```
if ((method.flags() & STATIC) == 0) continue;
if ((method.flags() & PUBLIC) == 0) continue;
if (method.params().isEmpty()) continue;
extensionMethods.add(method);
}
return new Extension(extensionMethods, tsym);
}
private static class Extension {
final List<MethodSymbol> extensionMethods;
final TypeSymbol extensionProvider;
public Extension(List<MethodSymbol> extensionMethods, TypeSymbol extensionProvider) {
this.extensionMethods = extensionMethods;
this.extensionProvider = extensionProvider;
}
}
private static class ExtensionMethodReplaceVisitor extends TreeScanner<Void, Void> {
final JavacNode annotationNode;
final List<Extension> extensions;
final boolean suppressBaseMethods;
public ExtensionMethodReplaceVisitor(JavacNode annotationNode, List<Extension> extensions,
boolean suppressBaseMethods) {
this.annotationNode = annotationNode;
this.extensions = extensions;
this.suppressBaseMethods = suppressBaseMethods;
}
public void replace() {
annotationNode.up().get().accept(this, null);
}
@Override
public Void visitMethodInvocation(final MethodInvocationTree tree, final Void p) {
scan(tree.getTypeArguments(), p);
scan(tree.getMethodSelect(), p);
handleMethodCall((JCMethodInvocation) tree);
scan(tree.getArguments(), p);
return null;
}
private void handleMethodCall(final JCMethodInvocation methodCall) {
JavacNode methodCallNode = annotationNode.getAst().get(methodCall);
if (methodCallNode == null) {
// This should mean the node does not exist in the source at all. This is the case for generated nodes,
such as implicit super() calls.
return;
}
JavacNode surroundingType = upToTypeNode(methodCallNode);
TypeSymbol surroundingTypeSymbol = ((JCClassDecl)surroundingType.get()).sym;
JCExpression receiver = receiverOf(methodCall);
String methodName = methodNameOf(methodCall);
if ("this".equals(receiver.toString()) || "this".equals(methodName) || "super".equals(methodName))
return;
Map<JCTree, JCTree> resolution = new
JavacResolution(methodCallNode.getContext()).resolveMethodMember(methodCallNode);
JCTree resolvedMethodCall = resolution.get(methodCall);
if (resolvedMethodCall == null || resolvedMethodCall.type == null) return;
```

```
if (!suppressBaseMethods && !(resolvedMethodCall.type instanceof ErrorType)) return;
JCTree resolvedReceiver = resolution.get(receiver);
if (resolvedReceiver == null || resolvedReceiver.type == null) return;
Type receiverType = resolvedReceiver.type;
if (receiverType.isErroneous()) return;
// Skip static method access
Symbol sym = null;
if (resolvedReceiver instanceof JCIdent) {
sym = ((JCIdent) resolvedReceiver).sym;
} else if (resolvedReceiver instanceof JCFieldAccess) {
sym = ((JCFieldAccess) resolvedReceiver).sym;
}
if (sym instanceof ClassSymbol) return;
Types types = Types.instance(annotationNode.getContext());
for (Extension extension : extensions) {
TypeSymbol extensionProvider = extension.extensionProvider;
if (surroundingTypeSymbol == extensionProvider) continue;
for (MethodSymbol extensionMethod : extension.extensionMethods) {
if (!methodName.equals(extensionMethod.name.toString())) continue;
Type extensionMethodType = extensionMethod.type;
if (!MethodType.class.isInstance(extensionMethodType) &&
!ForAll.class.isInstance(extensionMethodType)) continue;
Type firstArgType = types.erasure(extensionMethodType.asMethodType().argtypes.get(0));
if (!types.isAssignable(receiverType, firstArgType)) continue;
methodCall.args = methodCall.args.prepend(receiver);
methodCall.meth = chainDotsString(annotationNode, extensionProvider.toString() + "." +
methodName);
recursiveSetGeneratedBy(methodCall.meth, methodCallNode);
return;
}
}
}
private String methodNameOf(final JCMethodInvocation methodCall) {
if (methodCall.meth instanceof JCIdent) {
return ((JCIdent) methodCall.meth).name.toString();
} else {
return ((JCFieldAccess) methodCall.meth).name.toString();
}
}
private JCExpression receiverOf(final JCMethodInvocation methodCall) {
if (methodCall.meth instanceof JCIdent) {
return annotationNode.getTreeMaker().Ident(annotationNode.toName("this"));
} else {
return ((JCFieldAccess) methodCall.meth).selected;
}
}
}
}
```

lombok/javac/handlers/HandleFieldDefaults.java

/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.experimental.FieldDefaults;
import lombok.experimental.NonFinal;
import lombok.experimental.PackagePrivate;
import lombok.javac.JavacASTAdapter;
import lombok.javac.JavacASTVisitor;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
/**
* Handles the {@code lombok.FieldDefaults} annotation for javac.
*/
@Provides(JavacASTVisitor.class)
@HandlerPriority(-2048) //-2^11; to ensure @Value picks up on messing with the fields' 'final' state, run earlier.
public class HandleFieldDefaults extends JavacASTAdapter {
public boolean generateFieldDefaultsForType(JavacNode typeNode, JavacNode errorNode, AccessLevel level, boolean makeFinal, boolean checkForTypeLevelFieldDefaults) {
if (checkForTypeLevelFieldDefaults) {
if (hasAnnotation(FieldDefaults.class, typeNode)) {
//The annotation will make it happen, so we can skip it.

```
return true;
}
}
if (!isClassOrEnum(typeNode)) {
errorNode.addError("@FieldDefaults is only supported on a class or an enum.");
return false;
}
for (JavacNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
//Skip fields that start with $
if (fieldDecl.name.toString().startsWith("$")) continue;
setFieldDefaultsForField(field, level, makeFinal);
}
return true;
}
public void setFieldDefaultsForField(JavacNode fieldNode, AccessLevel level, boolean makeFinal) {
JCVariableDecl field = (JCVariableDecl) fieldNode.get();
if (level != null && level != AccessLevel.NONE) {
if ((field.mods.flags & (Flags.PUBLIC | Flags.PRIVATE | Flags.PROTECTED)) == 0) {
if (!hasAnnotationAndDeleteIfNeccessary(PackagePrivate.class, fieldNode)) {
if ((field.mods.flags & Flags.STATIC) == 0) {
field.mods.flags |= toJavacModifier(level);
}
}
}
}
if (makeFinal && (field.mods.flags & Flags.FINAL) == 0) {
if (!hasAnnotationAndDeleteIfNeccessary(NonFinal.class, fieldNode)) {
if ((field.mods.flags & Flags.STATIC) == 0) {
field.mods.flags |= Flags.FINAL;
}
}
}
fieldNode.rebuild();
}
@Override public void visitType(JavacNode typeNode, JCClassDecl type) {
AnnotationValues<FieldDefaults> fieldDefaults = null;
JavacNode source = typeNode;
boolean levelIsExplicit = false;
boolean makeFinalIsExplicit = false;
FieldDefaults fd = null;
for (JavacNode jn : typeNode.down()) {
if (jn.getKind() != Kind.ANNOTATION) continue;
JCAnnotation ann = (JCAnnotation) jn.get();
JCTree typeTree = ann.annotationType;
if (typeTree == null) continue;
String typeTreeToString = typeTree.toString();
if (!typeTreeToString.equals("FieldDefaults") &&
!typeTreeToString.equals("lombok.experimental.FieldDefaults")) continue;
if (!typeMatches(FieldDefaults.class, jn, typeTree)) continue;
source = jn;
fieldDefaults = createAnnotation(FieldDefaults.class, jn);
```

```
levelIsExplicit = fieldDefaults.isExplicit("level");
makeFinalIsExplicit = fieldDefaults.isExplicit("makeFinal");
handleExperimentalFlagUsage(jn, ConfigurationKeys.FIELD_DEFAULTS_FLAG_USAGE,
"@FieldDefaults");
fd = fieldDefaults.getInstance();
if (!levelIsExplicit && !makeFinalIsExplicit) {
jn.addError("This does nothing; provide either level or makeFinal or both.");
}
if (levelIsExplicit && fd.level() == AccessLevel.NONE) {
jn.addError("AccessLevel.NONE doesn't mean anything here. Pick another value.");
levelIsExplicit = false;
}
deleteAnnotationIfNeccessary(jn, FieldDefaults.class);
deleteImportFromCompilationUnit(jn, "lombok.AccessLevel");
break;
}
if (fd == null && (type.mods.flags & (Flags.INTERFACE | Flags.ANNOTATION)) != 0) return;
boolean defaultToPrivate = levelIsExplicit ? false : Boolean.TRUE.equals(typeNode.getAst().readConfi
guration(ConfigurationKeys.FIELD_DEFAULTS_PRIVATE_EVERYWHERE));
boolean defaultToFinal = makeFinalIsExplicit ? false : Boolean.TRUE.equals(typeNode.getAst().readC
onfiguration(ConfigurationKeys.FIELD_DEFAULTS_FINAL_EVERYWHERE));
if (!defaultToPrivate && !defaultToFinal && fieldDefaults == null) return;
// Do not apply field defaults to records if set using the config system
if (fieldDefaults == null && !isClassOrEnum(typeNode)) return;
AccessLevel fdAccessLevel = (fieldDefaults != null && levelIsExplicit) ? fd.level() : defaultToPrivate ?
AccessLevel.PRIVATE : null;
boolean fdToFinal = (fieldDefaults != null && makeFinalIsExplicit) ? fd.makeFinal() : defaultToFinal;
generateFieldDefaultsForType(typeNode, source, fdAccessLevel, fdToFinal, false);
}
}
```

lombok/javac/handlers/HandleFieldNameConstants.java

```
/*
* Copyright (C) 2014-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.ArrayList;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.IdentifierName;
import lombok.core.handlers.HandlerUtil;
import lombok.experimental.FieldNameConstants;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.Name;
@Provides
public class HandleFieldNameConstants extends JavacAnnotationHandler<FieldNameConstants> {
private static final IdentifierName FIELDS = IdentifierName.valueOf("Fields");
public void generateFieldNameConstantsForType(JavacNode typeNode, JavacNode errorNode,
AccessLevel level, boolean asEnum, IdentifierName innerTypeName, boolean onlyExplicit, boolean
uppercase) {
if (!isClassEnumOrRecord(typeNode)) {
errorNode.addError("@FieldNameConstants is only supported on a class, an enum or a record.");
return;
}
if (!isStaticAllowed(typeNode)) {
errorNode.addError("@FieldNameConstants is not supported on non-static nested classes.");
return;
}
java.util.List<JavacNode> qualified = new ArrayList<JavacNode>();
for (JavacNode field : typeNode.down()) {
if (fieldQualifiesForFieldNameConstantsGeneration(field, onlyExplicit)) qualified.add(field);
}
if (qualified.isEmpty()) {
```

```
errorNode.addWarning("No fields qualify for @FieldNameConstants, therefore this annotation does
nothing");
} else {
createInnerTypeFieldNameConstants(typeNode, errorNode, level, qualified, asEnum, innerTypeName,
uppercase);
}
}
private boolean fieldQualifiesForFieldNameConstantsGeneration(JavacNode field, boolean
onlyExplicit) {
if (field.getKind() != Kind.FIELD) return false;
boolean exclAnn =
JavacHandlerUtil.hasAnnotationAndDeleteIfNeccessary(FieldNameConstants.Exclude.class, field);
boolean inclAnn =
JavacHandlerUtil.hasAnnotationAndDeleteIfNeccessary(FieldNameConstants.Include.class, field);
if (exclAnn) return false;
if (inclAnn) return true;
if (onlyExplicit) return false;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
if (fieldDecl.name.toString().startsWith("$")) return false;
if ((fieldDecl.mods.flags & Flags.STATIC) != 0) return false;
return true;
}
public void handle(AnnotationValues<FieldNameConstants> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleExperimentalFlagUsage(annotationNode,
ConfigurationKeys.FIELD_NAME_CONSTANTS_FLAG_USAGE, "@FieldNameConstants");
deleteAnnotationIfNeccessary(annotationNode, FieldNameConstants.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode node = annotationNode.up();
FieldNameConstants annotationInstance = annotation.getInstance();
AccessLevel level = annotationInstance.level();
boolean asEnum = annotationInstance.asEnum();
boolean usingLombokv1_18_2 = annotation.isExplicit("prefix") || annotation.isExplicit("suffix") ||
node.getKind() == Kind.FIELD;
if (usingLombokv1_18_2) {
annotationNode.addError("@FieldNameConstants has been redesigned in lombok v1.18.4; please
upgrade your project dependency on lombok. See
https://projectlombok.org/features/experimental/FieldNameConstants for more information.");
return;
}
if (level == AccessLevel.NONE) {
annotationNode.addWarning("AccessLevel.NONE is not compatible with @FieldNameConstants. If you
don't want the inner type, simply remove @FieldNameConstants.");
return;
}
IdentifierName innerTypeName;
try {
innerTypeName = IdentifierName.valueOf(annotationInstance.innerTypeName());
} catch(IllegalArgumentException e) {
annotationNode.addError("InnerTypeName " + annotationInstance.innerTypeName() + " is not a valid
Java identifier.");
return;
}
```

```
if (innerTypeName == null) innerTypeName = annotationNode.getAst().readConfiguration(Configuratio
nKeys.FIELD_NAME_CONSTANTS_INNER_TYPE_NAME);
if (innerTypeName == null) innerTypeName = FIELDS;
Boolean uppercase = annotationNode.getAst().readConfiguration(ConfigurationKeys.FIELD_NAME_C
ONSTANTS_UPPERCASE);
if (uppercase == null) uppercase = false;
generateFieldNameConstantsForType(node, annotationNode, level, asEnum, innerTypeName,
annotationInstance.onlyExplicitlyIncluded(), uppercase);
}
private void createInnerTypeFieldNameConstants(JavacNode typeNode, JavacNode errorNode,
AccessLevel level, java.util.List<JavacNode> fields, boolean asEnum, IdentifierName innerTypeName,
boolean uppercase) {
if (fields.isEmpty()) return;
JavacTreeMaker maker = typeNode.getTreeMaker();
JCModifiers mods = maker.Modifiers(toJavacModifier(level) | (asEnum ? Flags.ENUM : Flags.STATIC |
Flags.FINAL));
Name fieldsName = typeNode.toName(innerTypeName.getName());
JavacNode fieldsType = findInnerClass(typeNode, innerTypeName.getName());
boolean genConstr = false;
if (fieldsType == null) {
JCClassDecl innerType = maker.ClassDef(mods, fieldsName, List.<JCTypeParameter>nil(), null,
List.<JCExpression>nil(), List.<JCTree>nil());
fieldsType = injectType(typeNode, innerType);
recursiveSetGeneratedBy(innerType, errorNode);
genConstr = true;
} else {
JCClassDecl builderTypeDeclaration = (JCClassDecl) fieldsType.get();
long f = builderTypeDeclaration.getModifiers().flags;
if (asEnum && (f & Flags.ENUM) == 0) {
errorNode.addError("Existing " + innerTypeName + " must be declared as an 'enum'.");
return;
}
if (!asEnum && (f & Flags.STATIC) == 0) {
errorNode.addError("Existing " + innerTypeName + " must be declared as a 'static class'.");
return;
}
genConstr = constructorExists(fieldsType) == MemberExistsResult.NOT_EXISTS;
}
if (genConstr) {
JCModifiers genConstrMods = maker.Modifiers(Flags.GENERATEDCONSTR | (asEnum ? 0L :
Flags.PRIVATE));
JCBlock genConstrBody = maker.Block(0L,
List.<JCStatement>of(maker.Exec(maker.Apply(List.<JCExpression>nil(),
maker.Ident(typeNode.toName("super")), List.<JCExpression>nil()))));
JCMethodDecl c = maker.MethodDef(genConstrMods, typeNode.toName("<init>"), null,
List.<JCTypeParameter>nil(), List.<JCVariableDecl>nil(), List.<JCExpression>nil(), genConstrBody,
null);
recursiveSetGeneratedBy(c, errorNode);
injectMethod(fieldsType, c);
}
java.util.List<JCVariableDecl> generated = new ArrayList<JCVariableDecl>();
for (JavacNode field : fields) {
Name fName = ((JCVariableDecl) field.get()).name;
```

```
if (uppercase) fName = typeNode.toName(HandlerUtil.camelCaseToConstant(fName.toString()));
if (fieldExists(fName.toString(), fieldsType) != MemberExistsResult.NOT_EXISTS) continue;
JCModifiers constantValueMods = maker.Modifiers(Flags.PUBLIC | Flags.STATIC | Flags.FINAL |
(asEnum ? Flags.ENUM : 0L));
JCExpression returnType;
JCExpression init;
if (asEnum) {
returnType = maker.Ident(fieldsName);
init = maker.NewClass(null, List.<JCExpression>nil(), maker.Ident(fieldsName),
List.<JCExpression>nil(), null);
} else {
returnType = chainDots(field, "java", "lang", "String");
init = maker.Literal(field.getName());
}
JCVariableDecl constantField = maker.VarDef(constantValueMods, fName, returnType, init);
injectField(fieldsType, constantField, false, true);
setGeneratedBy(constantField, errorNode);
generated.add(constantField);
}
for (JCVariableDecl cf : generated) recursiveSetGeneratedBy(cf, errorNode);
}
}
```

lombok/javac/handlers/HandleGetter.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.*;
import static lombok.javac.JavacAugments.JCTree_keepPosition;
import static lombok.javac.JavacTreeMaker.TypeTag.*;
```

```
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.experimental.Accessors;
import lombok.experimental.Delegate;
import lombok.Getter;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.delombok.LombokOptionsFactory;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.JavacTreeMaker.TypeTag;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBinary;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCIf;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCSynchronized;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
/**
* Handles the {@code lombok.Getter} annotation for javac.
*/
@Provides
public class HandleGetter extends JavacAnnotationHandler<Getter> {
private static final String GETTER_NODE_NOT_SUPPORTED_ERR = "@Getter is only supported on
a class, an enum, or a field.";
public void generateGetterForType(JavacNode typeNode, JavacNode errorNode, AccessLevel level,
boolean checkForTypeLevelGetter, List<JCAnnotation> onMethod) {
if (checkForTypeLevelGetter) {
if (hasAnnotation(Getter.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
}
if (!isClassOrEnum(typeNode)) {
```

```
errorNode.addError(GETTER_NODE_NOT_SUPPORTED_ERR);
return;
}
for (JavacNode field : typeNode.down()) {
if (fieldQualifiesForGetterGeneration(field)) generateGetterForField(field, errorNode.get(), level, false,
onMethod);
}
}
public static boolean fieldQualifiesForGetterGeneration(JavacNode field) {
if (field.getKind() != Kind.FIELD) return false;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
//Skip fields that start with $
if (fieldDecl.name.toString().startsWith("$")) return false;
//Skip static fields.
if ((fieldDecl.mods.flags & Flags.STATIC) != 0) return false;
return true;
}
/**
* Generates a getter on the stated field.
*
* Used by {@link HandleData}.
*
* The difference between this call and the handle method is as follows:
*
* If there is a {@code lombok.Getter} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
* If not, the getter is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*
* @param fieldNode The node representing the field you want a getter for.
* @param pos The node responsible for generating the getter (the {@code @Data} or {@code
@Getter} annotation).
*/
public void generateGetterForField(JavacNode fieldNode, DiagnosticPosition pos, AccessLevel level,
boolean lazy, List<JCAnnotation> onMethod) {
if (hasAnnotation(Getter.class, fieldNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
createGetterForField(level, fieldNode, fieldNode, false, lazy, onMethod);
}
@Override public void handle(AnnotationValues<Getter> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.GETTER_FLAG_USAGE, "@Getter");
Collection<JavacNode> fields = annotationNode.upFromAnnotationToFields();
deleteAnnotationIfNeccessary(annotationNode, Getter.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode node = annotationNode.up();
Getter annotationInstance = annotation.getInstance();
AccessLevel level = annotationInstance.value();
boolean lazy = annotationInstance.lazy();
if (lazy) handleFlagUsage(annotationNode, ConfigurationKeys.GETTER_LAZY_FLAG_USAGE,
"@Getter(lazy=true)");
```

```
if (level == AccessLevel.NONE) {
if (lazy) annotationNode.addWarning("'lazy' does not work with AccessLevel.NONE.");
return;
}
if (node == null) return;
List<JCAnnotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@Getter(onMethod", annotationNode);
if (!onMethod.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@Getter(onMethod=...)");
}
switch (node.getKind()) {
case FIELD:
createGetterForFields(level, fields, annotationNode, true, lazy, onMethod);
break;
case TYPE:
if (lazy) annotationNode.addError("'lazy' is not supported for @Getter on a type.");
generateGetterForType(node, annotationNode, level, false, onMethod);
break;
}
}
public void createGetterForFields(AccessLevel level, Collection<JavacNode> fieldNodes, JavacNode
errorNode, boolean whineIfExists, boolean lazy, List<JCAnnotation> onMethod) {
for (JavacNode fieldNode : fieldNodes) {
createGetterForField(level, fieldNode, errorNode, whineIfExists, lazy, onMethod);
}
}
public void createGetterForField(AccessLevel level,
JavacNode fieldNode, JavacNode source, boolean whineIfExists, boolean lazy, List<JCAnnotation>
onMethod) {
if (fieldNode.getKind() != Kind.FIELD || fieldNode.isEnumMember()) {
source.addError(GETTER_NODE_NOT_SUPPORTED_ERR);
return;
}
JCVariableDecl fieldDecl = (JCVariableDecl) fieldNode.get();
if (lazy) {
if ((fieldDecl.mods.flags & Flags.PRIVATE) == 0 || (fieldDecl.mods.flags & Flags.FINAL) == 0) {
source.addError("'lazy' requires the field to be private and final.");
return;
}
if ((fieldDecl.mods.flags & Flags.TRANSIENT) != 0) {
source.addError("'lazy' is not supported on transient fields.");
return;
}
if (fieldDecl.init == null) {
source.addError("'lazy' requires field initialization.");
return;
}
}
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
String methodName = toGetterName(fieldNode, accessors);
if (methodName == null) {
source.addWarning("Not generating getter for this field: It does not fit your @Accessors prefix list.");
```

```
return;
}
for (String altName : toAllGetterNames(fieldNode, accessors)) {
switch (methodExists(altName, fieldNode, false, 0)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String altNameExpl = "";
if (!altName.equals(methodName)) altNameExpl = String.format(" (%s)", altName);
source.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", methodName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
//continue scanning the other alt names.
}
}
long access = toJavacModifier(level) | (fieldDecl.mods.flags & Flags.STATIC);
injectMethod(fieldNode.up(), source, createGetter(access, fieldNode, fieldNode.getTreeMaker(),
source, lazy, onMethod));
}
public JCMethodDecl createGetter(long access, JavacNode field, JavacTreeMaker treeMaker,
JavacNode source, boolean lazy, List<JCAnnotation> onMethod) {
JCVariableDecl fieldNode = (JCVariableDecl) field.get();
// Remember the type; lazy will change it
JCExpression methodType = copyType(treeMaker, fieldNode, source);
AnnotationValues<Accessors> accessors = JavacHandlerUtil.getAccessorsForField(field);
// Generate the methodName; lazy will change the field type
Name methodName = field.toName(toGetterName(field, accessors));
boolean makeFinal = shouldMakeFinal(field, accessors);
List<JCStatement> statements;
boolean addSuppressWarningsUnchecked = false;
if (lazy && !inNetbeansEditor(field)) {
JCTree_keepPosition.set(fieldNode.init, true);
statements = createLazyGetterBody(treeMaker, field, source);
addSuppressWarningsUnchecked = LombokOptionsFactory.getDelombokOptions(field.getContext()).g
etFormatPreferences().generateSuppressWarnings();
} else {
statements = createSimpleGetterBody(treeMaker, field);
}
JCBlock methodBody = treeMaker.Block(0, statements);
List<JCTypeParameter> methodGenericParams = List.nil();
List<JCVariableDecl> parameters = List.nil();
List<JCExpression> throwsClauses = List.nil();
JCExpression annotationMethodDefaultValue = null;
List<JCAnnotation> copyableAnnotations = findCopyableAnnotations(field);
List<JCAnnotation> delegates = findDelegatesAndRemoveFromField(field);
List<JCAnnotation> annsOnMethod = copyAnnotations(onMethod).appendList(copyableAnnotations);
if (field.isFinal()) {
```

```
if (getCheckerFrameworkVersion(field).generatePure()) annsOnMethod =
annsOnMethod.prepend(treeMaker.Annotation(genTypeRef(field,
CheckerFrameworkVersion.NAME__PURE), List.<JCExpression>nil()));
} else {
if (getCheckerFrameworkVersion(field).generateSideEffectFree()) annsOnMethod =
annsOnMethod.prepend(treeMaker.Annotation(genTypeRef(field,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
}
if (isFieldDeprecated(field)) annsOnMethod =
annsOnMethod.prepend(treeMaker.Annotation(genJavaLangTypeRef(field, "Deprecated"),
List.<JCExpression>nil()));
if (makeFinal) access |= Flags.FINAL;
JCMethodDecl decl = recursiveSetGeneratedBy(treeMaker.MethodDef(treeMaker.Modifiers(access,
annsOnMethod), methodName, methodType,
methodGenericParams, parameters, throwsClauses, methodBody, annotationMethodDefaultValue),
source);
decl.mods.annotations = decl.mods.annotations.appendList(delegates);
if (addSuppressWarningsUnchecked) {
ListBuffer<JCExpression> suppressions = new ListBuffer<JCExpression>();
if (!Boolean.FALSE.equals(field.getAst().readConfiguration(ConfigurationKeys.ADD_SUPPRESSWAR
NINGS_ANNOTATIONS))) {
suppressions.append(treeMaker.Literal("all"));
}
suppressions.append(treeMaker.Literal("unchecked"));
addAnnotation(decl.mods, field, source, "java.lang.SuppressWarnings", treeMaker.NewArray(null,
List.<JCExpression>nil(), suppressions.toList()));
}
copyJavadoc(field, decl, CopyJavadoc.GETTER);
return decl;
}
public static List<JCAnnotation> findDelegatesAndRemoveFromField(JavacNode field) {
JCVariableDecl fieldNode = (JCVariableDecl) field.get();
List<JCAnnotation> delegates = List.nil();
for (JCAnnotation annotation : fieldNode.mods.annotations) {
if (typeMatches(Delegate.class, field, annotation.annotationType)) {
delegates = delegates.append(annotation);
}
}
if (!delegates.isEmpty()) {
ListBuffer<JCAnnotation> withoutDelegates = new ListBuffer<JCAnnotation>();
for (JCAnnotation annotation : fieldNode.mods.annotations) {
if (!delegates.contains(annotation)) {
withoutDelegates.append(annotation);
}
}
fieldNode.mods.annotations = withoutDelegates.toList();
field.rebuild();
}
return delegates;
}
public List<JCStatement> createSimpleGetterBody(JavacTreeMaker treeMaker, JavacNode field) {
return List.<JCStatement>of(treeMaker.Return(createFieldAccessor(treeMaker, field,
FieldAccess.ALWAYS_FIELD)));
```

```
}
private static final String AR = "java.util.concurrent.atomic.AtomicReference";
private static final List<JCExpression> NIL_EXPRESSION = List.nil();
public static final java.util.Map<TypeTag, String> TYPE_MAP;
static {
Map<TypeTag, String> m = new HashMap<TypeTag, String>();
m.put(CTC_INT, "Integer");
m.put(CTC_DOUBLE, "Double");
m.put(CTC_FLOAT, "Float");
m.put(CTC_SHORT, "Short");
m.put(CTC_BYTE, "Byte");
m.put(CTC_LONG, "Long");
m.put(CTC_BOOLEAN, "Boolean");
m.put(CTC_CHAR, "Character");
TYPE_MAP = Collections.unmodifiableMap(m);
}
public List<JCStatement> createLazyGetterBody(JavacTreeMaker maker, JavacNode fieldNode,
JavacNode source) {
/*
java.lang.Object value = this.fieldName.get();
if (value == null) {
synchronized (this.fieldName) {
value = this.fieldName.get();
if (value == null) {
final RawValueType actualValue = INITIALIZER_EXPRESSION;
[IF PRIMITIVE]
value = actualValue;
[ELSE]
value = actualValue == null ? this.fieldName : actualValue;
[END IF]
this.fieldName.set(value);
}
}
}
[IF PRIMITIVE]
return (BoxedValueType) value;
[ELSE]
return (BoxedValueType) (value == this.fieldName ? null : value);
[END IF]
*/
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
JCVariableDecl field = (JCVariableDecl) fieldNode.get();
JCExpression copyOfRawFieldType = copyType(maker, field, source);
JCExpression copyOfBoxedFieldType = null;
field.type = null;
boolean isPrimitive = false;
if (field.vartype instanceof JCPrimitiveTypeTree) {
String boxed = TYPE_MAP.get(typeTag(field.vartype));
if (boxed != null) {
isPrimitive = true;
field.vartype = genJavaLangTypeRef(fieldNode, boxed);
copyOfBoxedFieldType = genJavaLangTypeRef(fieldNode, boxed);
}
```

```
}
if (copyOfBoxedFieldType == null) copyOfBoxedFieldType = copyType(maker, field, source);
Name valueName = fieldNode.toName("$value");
Name actualValueName = fieldNode.toName("actualValue");
/* java.lang.Object value = this.fieldName.get();*/ {
JCExpression valueVarType = genJavaLangTypeRef(fieldNode, "Object");
statements.append(maker.VarDef(maker.Modifiers(0L), valueName, valueVarType, callGet(fieldNode,
createFieldAccessor(maker, fieldNode, FieldAccess.ALWAYS_FIELD))));
}
/* if (value == null) { */ {
JCSynchronized synchronizedStatement;
/* synchronized (this.fieldName) { */ {
ListBuffer<JCStatement> synchronizedStatements = new ListBuffer<JCStatement>();
/* value = this.fieldName.get(); */ {
JCExpressionStatement newAssign = maker.Exec(maker.Assign(maker.Ident(valueName),
callGet(fieldNode, createFieldAccessor(maker, fieldNode, FieldAccess.ALWAYS_FIELD))));
synchronizedStatements.append(newAssign);
}
/* if (value == null) { */ {
ListBuffer<JCStatement> innerIfStatements = new ListBuffer<JCStatement>();
/* final RawValueType actualValue = INITIALIZER_EXPRESSION; */ {
innerIfStatements.append(maker.VarDef(maker.Modifiers(Flags.FINAL), actualValueName,
copyOfRawFieldType, field.init));
}
/* [IF primitive] value = actualValue; */ {
if (isPrimitive) {
JCStatement statement = maker.Exec(maker.Assign(maker.Ident(valueName),
maker.Ident(actualValueName)));
innerIfStatements.append(statement);
}
}
/* [ELSE] value = actualValue == null ? this.fieldName : actualValue; */ {
if (!isPrimitive) {
JCExpression actualValueIsNull = maker.Binary(CTC_EQUAL, maker.Ident(actualValueName),
maker.Literal(CTC_BOT, null));
JCExpression thisDotFieldName = createFieldAccessor(maker, fieldNode,
FieldAccess.ALWAYS_FIELD);
JCExpression ternary = maker.Conditional(actualValueIsNull, thisDotFieldName,
maker.Ident(actualValueName));
JCStatement statement = maker.Exec(maker.Assign(maker.Ident(valueName), ternary));
innerIfStatements.append(statement);
}
}
/* this.fieldName.set(value); */ {
JCStatement statement = callSet(fieldNode, createFieldAccessor(maker, fieldNode,
FieldAccess.ALWAYS_FIELD), maker.Ident(valueName));
innerIfStatements.append(statement);
}
JCBinary isNull = maker.Binary(CTC_EQUAL, maker.Ident(valueName), maker.Literal(CTC_BOT,
null));
JCIf ifStatement = maker.If(isNull, maker.Block(0, innerIfStatements.toList()), null);
synchronizedStatements.append(ifStatement);
}
```

```
synchronizedStatement = maker.Synchronized(createFieldAccessor(maker, fieldNode,
FieldAccess.ALWAYS_FIELD), maker.Block(0, synchronizedStatements.toList()));
}
JCBinary isNull = maker.Binary(CTC_EQUAL, maker.Ident(valueName), maker.Literal(CTC_BOT,
null));
JCIf ifStatement = maker.If(isNull, maker.Block(0, List.<JCStatement>of(synchronizedStatement)),
null);
statements.append(ifStatement);
}
/* [IF PRIMITIVE] return (BoxedValueType) value; */ {
if (isPrimitive) {
statements.append(maker.Return(maker.TypeCast(copyOfBoxedFieldType,
maker.Ident(valueName))));
}
}
/* [ELSE] return (BoxedValueType) (value == this.fieldName ? null : value); */ {
if (!isPrimitive) {
JCExpression valueEqualsSelf = maker.Binary(CTC_EQUAL, maker.Ident(valueName),
createFieldAccessor(maker, fieldNode, FieldAccess.ALWAYS_FIELD));
JCExpression ternary = maker.Conditional(valueEqualsSelf, maker.Literal(CTC_BOT, null),
maker.Ident(valueName));
JCExpression typeCast = maker.TypeCast(copyOfBoxedFieldType, maker.Parens(ternary));
statements.append(maker.Return(typeCast));
}
}
// update the field type and init last
/* private final java.util.concurrent.atomic.AtomicReference<Object> fieldName = new
java.util.concurrent.atomic.AtomicReference<Object>(); */ {
field.vartype = recursiveSetGeneratedBy(
maker.TypeApply(chainDotsString(fieldNode, AR),
List.<JCExpression>of(genJavaLangTypeRef(fieldNode, "Object"))), source);
field.init = recursiveSetGeneratedBy(maker.NewClass(null, NIL_EXPRESSION, copyType(maker, field,
source), NIL_EXPRESSION, null), source);
}
return statements.toList();
}
public JCMethodInvocation callGet(JavacNode source, JCExpression receiver) {
JavacTreeMaker maker = source.getTreeMaker();
return maker.Apply(NIL_EXPRESSION, maker.Select(receiver, source.toName("get")),
NIL_EXPRESSION);
}
public JCStatement callSet(JavacNode source, JCExpression receiver, JCExpression value) {
JavacTreeMaker maker = source.getTreeMaker();
return maker.Exec(maker.Apply(NIL_EXPRESSION, maker.Select(receiver, source.toName("set")),
List.<JCExpression>of(value)));
}
public JCExpression copyType(JavacTreeMaker treeMaker, JCVariableDecl fieldNode, JavacNode
source) {
return fieldNode.type != null ? treeMaker.Type(fieldNode.type) : cloneType(treeMaker,
fieldNode.vartype, source);
}
}
```

lombok/javac/handlers/HandleHelper.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Iterator;
import com.sun.source.tree.MethodInvocationTree;
import com.sun.source.tree.TreeVisitor;
import com.sun.source.util.TreeScanner;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCCase;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
```

```
import lombok.experimental.Helper;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
@Provides
public class HandleHelper extends JavacAnnotationHandler<Helper> {
private List<JCStatement> getStatementsFromJcNode(JCTree tree) {
if (tree instanceof JCBlock) return ((JCBlock) tree).stats;
if (tree instanceof JCCase) return ((JCCase) tree).stats;
if (tree instanceof JCMethodDecl) return ((JCMethodDecl) tree).body.stats;
return null;
}
private void setStatementsOfJcNode(JCTree tree, List<JCStatement> statements) {
if (tree instanceof JCBlock) ((JCBlock) tree).stats = statements;
else if (tree instanceof JCCase) ((JCCase) tree).stats = statements;
else if (tree instanceof JCMethodDecl) ((JCMethodDecl) tree).body.stats = statements;
else throw new IllegalArgumentException("Can't set statements on node type: " + tree.getClass());
}
@Override public void handle(AnnotationValues<Helper> annotation, JCAnnotation ast, final
JavacNode annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.HELPER_FLAG_USAGE,
"@Helper");
deleteAnnotationIfNeccessary(annotationNode, Helper.class);
JavacNode annotatedType = annotationNode.up();
JavacNode containingBlock = annotatedType == null ? null : annotatedType.directUp();
List<JCStatement> origStatements = getStatementsFromJcNode(containingBlock == null ? null :
containingBlock.get());
if (annotatedType == null || annotatedType.getKind() != Kind.TYPE || origStatements == null) {
annotationNode.addError("@Helper is legal only on method-local classes.");
return;
}
JCClassDecl annotatedType_ = (JCClassDecl) annotatedType.get();
Iterator<JCStatement> it = origStatements.iterator();
while (it.hasNext()) {
if (it.next() == annotatedType_) {
break;
}
}
java.util.List<String> knownMethodNames = new ArrayList<String>();
for (JavacNode ch : annotatedType.down()) {
if (ch.getKind() != Kind.METHOD) continue;
String n = ch.getName();
if (n == null || n.isEmpty() || n.charAt(0) == '<') continue;
knownMethodNames.add(n);
}
Collections.sort(knownMethodNames);
final String[] knownMethodNames_ = knownMethodNames.toArray(new String[0]);
final Name helperName = annotationNode.toName("$" + annotatedType_.name);
final boolean[] helperUsed = new boolean[1];
final JavacTreeMaker maker = annotationNode.getTreeMaker();
TreeVisitor<Void, Void> visitor = new TreeScanner<Void, Void>() {
@Override public Void visitMethodInvocation(MethodInvocationTree node, Void p) {
```

```
JCMethodInvocation jcmi = (JCMethodInvocation) node;
apply(jcmi);
return super.visitMethodInvocation(node, p);
}
private void apply(JCMethodInvocation jcmi) {
if (!(jcmi.meth instanceof JCIdent)) return;
JCIdent jci = (JCIdent) jcmi.meth;
if (Arrays.binarySearch(knownMethodNames_, jci.name.toString()) < 0) return;
jcmi.meth = maker.Select(maker.Ident(helperName), jci.name);
recursiveSetGeneratedBy(jcmi.meth, annotationNode);
helperUsed[0] = true;
}
};
while (it.hasNext()) {
JCStatement stat = it.next();
stat.accept(visitor, null);
}
if (!helperUsed[0]) {
annotationNode.addWarning("No methods of this helper class are ever used.");
return;
}
ListBuffer<JCStatement> newStatements = new ListBuffer<JCStatement>();
boolean mark = false;
for (JCStatement stat : origStatements) {
newStatements.append(stat);
if (mark || stat != annotatedType_) continue;
mark = true;
JCExpression init = maker.NewClass(null, List.<JCExpression>nil(),
maker.Ident(annotatedType_.name), List.<JCExpression>nil(), null);
JCExpression varType = maker.Ident(annotatedType_.name);
JCVariableDecl decl = maker.VarDef(maker.Modifiers(Flags.FINAL), helperName, varType, init);
recursiveSetGeneratedBy(decl, annotationNode);
newStatements.append(decl);
}
setStatementsOfJcNode(containingBlock.get(), newStatements.toList());
}
}
```

lombok/javac/handlers/HandleJacksonized.java

```
/*
* Copyright (C) 2020-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCTypeApply;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import lombok.Builder;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.handlers.HandlerUtil;
import lombok.experimental.SuperBuilder;
import lombok.extern.jackson.Jacksonized;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
/**
* This (javac) handler deals with {@code @Jacksonized} modifying the (already
* generated) {@code @Builder} or {@code @SuperBuilder} to conform to Jackson's
* needs for builders.
*/
@Provides
@HandlerPriority(-512) // Above Handle(Super)Builder's level (builders must be already generated).
public class HandleJacksonized extends JavacAnnotationHandler<Jacksonized> {
@Override public void handle(AnnotationValues<Jacksonized> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.JACKSONIZED_FLAG_USAGE,
"@Jacksonized");
JavacNode annotatedNode = annotationNode.up();
deleteAnnotationIfNeccessary(annotationNode, Jacksonized.class);
JavacNode tdNode;
if (annotatedNode.getKind() != Kind.TYPE)
tdNode = annotatedNode.up(); // @Jacksonized on a constructor or a static factory method.
```

```
else
tdNode = annotatedNode; // @Jacksonized on the class.
JCClassDecl td = (JCClassDecl) tdNode.get();
JavacNode builderAnnotationNode = findAnnotation(Builder.class, annotatedNode);
JavacNode superBuilderAnnotationNode = findAnnotation(SuperBuilder.class, annotatedNode);
if (builderAnnotationNode == null && superBuilderAnnotationNode == null) {
annotationNode.addWarning("@Jacksonized requires @Builder or @SuperBuilder for it to mean
anything.");
return;
}
if (builderAnnotationNode != null && superBuilderAnnotationNode != null) {
annotationNode.addError("@Jacksonized cannot process both @Builder and @SuperBuilder on the
same class.");
return;
}
boolean isAbstract = (td.mods.flags & Flags.ABSTRACT) != 0;
if (isAbstract) {
annotationNode.addError("Builders on abstract classes cannot be @Jacksonized (the builder would
never be used).");
return;
}
AnnotationValues<Builder> builderAnnotation = builderAnnotationNode != null ?
createAnnotation(Builder.class, builderAnnotationNode) :
null;
AnnotationValues<SuperBuilder> superBuilderAnnotation = superBuilderAnnotationNode != null ?
createAnnotation(SuperBuilder.class, superBuilderAnnotationNode) :
null;
String setPrefix = builderAnnotation != null ?
builderAnnotation.getInstance().setterPrefix() :
superBuilderAnnotation.getInstance().setterPrefix();
String buildMethodName = builderAnnotation != null ?
builderAnnotation.getInstance().buildMethodName() :
superBuilderAnnotation.getInstance().buildMethodName();
JavacTreeMaker maker = annotatedNode.getTreeMaker();
// Now lets find the generated builder class.
String builderClassName = getBuilderClassName(annotationNode, annotatedNode, td,
builderAnnotation, maker);
JCClassDecl builderClass = null;
for (JCTree member : td.getMembers()) {
if (member instanceof JCClassDecl && ((JCClassDecl)
member).getSimpleName().contentEquals(builderClassName)) {
builderClass = (JCClassDecl) member;
break;
}
}
if (builderClass == null) {
annotationNode.addError("Could not find @(Super)Builder's generated builder class for @Jacksonized
processing. If there are other compiler errors, fix them first.");
return;
}
// Insert @JsonDeserialize on annotated class.
if (hasAnnotation("com.fasterxml.jackson.databind.annotation.JsonDeserialize", tdNode)) {
```

```
annotationNode.addError("@JsonDeserialize already exists on class. Either delete @JsonDeserialize,
or remove @Jacksonized and manually configure Jackson.");
return;
}
JCExpression jsonDeserializeType = chainDots(annotatedNode, "com", "fasterxml", "jackson",
"databind", "annotation", "JsonDeserialize");
JCExpression builderClassExpression = namePlusTypeParamsToTypeReference(maker, tdNode,
annotationNode.toName(builderClassName), false, List.<JCTypeParameter>nil());
JCFieldAccess builderClassReference = maker.Select(builderClassExpression,
annotatedNode.toName("class"));
JCExpression assign = maker.Assign(maker.Ident(annotationNode.toName("builder")),
builderClassReference);
JCAnnotation annotationJsonDeserialize = maker.Annotation(jsonDeserializeType, List.of(assign));
recursiveSetGeneratedBy(annotationJsonDeserialize, annotationNode);
td.mods.annotations = td.mods.annotations.append(annotationJsonDeserialize);
// Copy annotations from the class to the builder class.
List<JCAnnotation> copyableAnnotations = findJacksonAnnotationsOnClass(tdNode);
List<JCAnnotation> copiedAnnotations = copyAnnotations(copyableAnnotations);
for (JCAnnotation anno : copiedAnnotations) {
recursiveSetGeneratedBy(anno, annotationNode);
}
builderClass.mods.annotations = builderClass.mods.annotations.appendList(copiedAnnotations);
// Insert @JsonPOJOBuilder on the builder class.
JCExpression jsonPOJOBuilderType = chainDots(annotatedNode, "com", "fasterxml", "jackson",
"databind", "annotation", "JsonPOJOBuilder");
JCExpression withPrefixExpr = maker.Assign(maker.Ident(annotationNode.toName("withPrefix")),
maker.Literal(setPrefix));
JCExpression buildMethodNameExpr =
maker.Assign(maker.Ident(annotationNode.toName("buildMethodName")),
maker.Literal(buildMethodName));
JCAnnotation annotationJsonPOJOBuilder = maker.Annotation(jsonPOJOBuilderType,
List.of(withPrefixExpr, buildMethodNameExpr));
recursiveSetGeneratedBy(annotationJsonPOJOBuilder, annotatedNode);
builderClass.mods.annotations =
builderClass.mods.annotations.append(annotationJsonPOJOBuilder);
// @SuperBuilder? Make it package-private!
if (superBuilderAnnotationNode != null)
builderClass.mods.flags = builderClass.mods.flags & ~Flags.PRIVATE;
}
private String getBuilderClassName(JavacNode annotationNode, JavacNode annotatedNode,
JCClassDecl td, AnnotationValues<Builder> builderAnnotation, JavacTreeMaker maker) {
String builderClassName = builderAnnotation != null ?
builderAnnotation.getInstance().builderClassName() : null;
if (builderClassName == null || builderClassName.isEmpty()) {
builderClassName =
annotationNode.getAst().readConfiguration(ConfigurationKeys.BUILDER_CLASS_NAME);
if (builderClassName == null || builderClassName.isEmpty())
builderClassName = "*Builder";
JCMethodDecl fillParametersFrom = annotatedNode.get() instanceof JCMethodDecl ?
(JCMethodDecl)annotatedNode.get() : null;
String replacement;
if (fillParametersFrom != null && !fillParametersFrom.getName().toString().equals("<init>")) {
// @Builder on a method: Use name of return type for builder class name.
```

```
JCExpression returnType = fillParametersFrom.restype;
List<JCTypeParameter> typeParams = fillParametersFrom.typarams;
if (returnType instanceof JCTypeApply) {
returnType = cloneType(maker, returnType, annotatedNode);
}
replacement = HandleBuilder.returnTypeToBuilderClassName(annotationNode, td, returnType,
typeParams);
} else {
// @Builder on class or constructor: Use the class name.
replacement = td.name.toString();
}
builderClassName = builderClassName.replace("*", replacement);
}
if (builderAnnotation == null)
builderClassName += "Impl"; // For @SuperBuilder, all Jackson annotations must be put on the
BuilderImpl class.
return builderClassName;
}
private static List<JCAnnotation> findJacksonAnnotationsOnClass(JavacNode node) {
ListBuffer<JCAnnotation> result = new ListBuffer<JCAnnotation>();
for (JavacNode child : node.down()) {
if (child.getKind() == Kind.ANNOTATION) {
JCAnnotation annotation = (JCAnnotation) child.get();
for (String bn : HandlerUtil.JACKSON_COPY_TO_BUILDER_ANNOTATIONS) {
if (typeMatches(bn, node, annotation.annotationType)) {
result.append(annotation);
break;
}
}
}
}
return result.toList();
}
}
```

lombok/javac/handlers/HandleLocked.java

```
/*
* Copyright (C) 2021-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.Locked;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
import static com.sun.tools.javac.tree.JCTree.*;
/**
* Handles the {@code lombok.Locked} annotation for javac.
*/
@Provides
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the statements generated by it.
public class HandleLocked extends JavacAnnotationHandler<Locked> {
private static final String ANNOTATION_NAME = "@Locked";
private static final String[] LOCK_TYPE_CLASS = new String[]{"java", "util", "concurrent", "locks", "Lock"};
private static final String[] LOCK_IMPL_CLASS = new String[]{"java", "util", "concurrent", "locks", "ReentrantLock"};
@Override public void handle(AnnotationValues<Locked> annotation, JCAnnotation ast, JavacNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.handle(annotationValue, ast, annotationNode, Locked.class, ANNOTATION_NAME, LOCK_TYPE_CLASS, LOCK_IMPL_CLASS);
}
}
```

lombok/javac/handlers/HandleLockedRead.java

```
/*
* Copyright (C) 2021-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.Locked;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
import static com.sun.tools.javac.tree.JCTree.*;
/**
* Handles the {@code lombok.Locked.Read} annotation for javac.
*/
@Provides
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the
statements generated by it.
public class HandleLockedRead extends JavacAnnotationHandler<Locked.Read> {
private static final String LOCK_METHOD = "readLock";
private static final String ANNOTATION_NAME = "@Locked.Read";
private static final String[] LOCK_TYPE_CLASS = new String[]{"java", "util", "concurrent", "locks",
"ReadWriteLock"};
private static final String[] LOCK_IMPL_CLASS = new String[]{"java", "util", "concurrent", "locks",
"ReentrantReadWriteLock"};
@Override public void handle(AnnotationValues<Locked.Read> annotation, JCAnnotation ast,
JavacNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.handle(annotationValue, ast, annotationNode, Locked.Read.class,
ANNOTATION_NAME, LOCK_TYPE_CLASS, LOCK_IMPL_CLASS, LOCK_METHOD);
}
}
```

lombok/javac/handlers/HandleLockedUtil.java

```
/*
* Copyright (C) 2021-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
```

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.util.List;
import lombok.core.AST;
import lombok.ConfigurationKeys;
import lombok.Locked;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import java.lang.annotation.Annotation;
import static com.sun.tools.javac.tree.JCTree.*;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
/**
* Container for static utility methods used by the Locked[.Read/Write] annotations.
*/
public final class HandleLockedUtil {
private static final String INSTANCE_LOCK_NAME = "$lock";
private static final String STATIC_LOCK_NAME = "$LOCK";
private static final List<JCExpression> NIL_EXPRESSION = List.nil();
private HandleLockedUtil() {
//Prevent instantiation
}
/**
* See {@link #handle(String, JCAnnotation, JavacNode, Class, String, String[], String[], String)} for
* {@code lockableMethodName = null}.
*/
public static <T extends Annotation> void handle(String annotationValue, JCTree.JCAnnotation ast,
JavacNode annotationNode, Class<T> annotationClass, String annotationName, String[]
lockTypeClass, String[] lockImplClass) {
handle(annotationValue, ast, annotationNode, annotationClass, annotationName, lockTypeClass,
lockImplClass, null);
}
/**
* Called when an annotation is found that is likely to match {@link Locked}, {@link Locked.Read}, or
* {@link Locked.Write}.
*
* Be aware that you'll be called for ANY annotation node in the source that looks like a match. There is,
* for example, no guarantee that the annotation node belongs to a method, even if you set your
* TargetType in the annotation to methods only.
*
* @param annotationValue The value of the annotation. This will be the name of the object used for
locking.
* @param ast The javac AST node representing the annotation.
```

```
* @param annotationNode The Lombok AST wrapper around the 'ast' parameter. You can use this object
* to travel back up the chain (something javac AST can't do) to the parent of the annotation, as well
* as access useful methods such as generating warnings or errors focused on the annotation.
* @param annotationClass The specific annotation class. This should be one of {@code Locked}, {@code Locked.Read},
* or {@code Locked.Write}.
* @param annotationName The name of the annotation to use when referencing it in errors.
* @param lockTypeClass The type of the variable when generating a lock to use.
* See {@link JavacHandlerUtil#chainDots(JavacNode, String, String, String...)}.
* @param lockImplClass Call the constructor of this class to generate a lock to use.
* See {@link JavacHandlerUtil#chainDots(JavacNode, String, String, String...)}.
* @param lockableMethodName The name of the method in the {@code lockClass} that returns a
* {@link java.util.concurrent.locks.Lock} object. When this is {@code null}, it is assumed that {@code lockClass}
* itself can be locked/unlocked.
* @param <T> The annotation type.
*/
public static <T extends Annotation> void handle(String annotationValue, JCTree.JCAnnotation ast, JavacNode annotationNode, Class<T> annotationClass, String annotationName, String[] lockTypeClass, String[] lockImplClass, String lockableMethodName) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOCKED_FLAG_USAGE, annotationName);
if (inNetbeansEditor(annotationNode)) return;
deleteAnnotationIfNeccessary(annotationNode, annotationClass);
JavacNode methodNode = annotationNode.up();
if (methodNode == null || methodNode.getKind() != AST.Kind.METHOD || !(methodNode.get() instanceof JCMethodDecl)) {
annotationNode.addError(annotationName + " is legal only on methods.");
return;
}
JCMethodDecl method = (JCMethodDecl) methodNode.get();
if ((method.mods.flags & Flags.ABSTRACT) != 0) {
annotationNode.addError(annotationName + " is legal only on concrete methods.");
return;
}
JavacNode typeNode = upToTypeNode(annotationNode);
if (!isClassOrEnum(typeNode)) {
annotationNode.addError(annotationName + " is legal only on methods in classes and enums.");
return;
}
boolean isStatic = (method.mods.flags & Flags.STATIC) != 0;
String lockName = annotationValue;
boolean autoMake = false;
if (lockName.length() == 0) {
autoMake = true;
lockName = isStatic ? STATIC_LOCK_NAME : INSTANCE_LOCK_NAME;
}
JavacTreeMaker maker = methodNode.getTreeMaker().at(ast.pos);
MemberExistsResult exists = MemberExistsResult.NOT_EXISTS;
JCExpression lockVarType = chainDots(methodNode, ast.pos, null, null, lockTypeClass);
if (typeNode != null && typeNode.get() instanceof JCClassDecl) {
for (JCTree def : ((JCClassDecl) typeNode.get()).defs) {
if (def instanceof JCVariableDecl) {
```

```
if (((JCVariableDecl) def).name.contentEquals(lockName)) {
JCVariableDecl varDeclDef = (JCVariableDecl) def;
exists = getGeneratedBy(varDeclDef) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
boolean st = ((varDeclDef.mods.flags) & Flags.STATIC) != 0;
if (isStatic != st && exists == MemberExistsResult.EXISTS_BY_LOMBOK) {
annotationNode.addError("The generated field " + lockName + " does not match the static status of this
method");
return;
}
isStatic = st;
if (exists == MemberExistsResult.EXISTS_BY_LOMBOK &&
!lockVarType.toString().equals(varDeclDef.vartype.toString())) {
annotationNode.addError("Expected field " + lockName + " to be of type " + lockVarType + " but got
type " + varDeclDef.vartype + ". Did you mix @Locked with @Locked.Read/Write on the same
generated field?");
return;
}
}
}
}
}
if (exists == MemberExistsResult.NOT_EXISTS) {
if (!autoMake) {
annotationNode.addError("The field " + lockName + " does not exist.");
return;
}
JCExpression lockImplType = chainDots(methodNode, ast.pos, null, null, lockImplClass);
JCNewClass lockInstance = maker.NewClass(null, NIL_EXPRESSION, lockImplType,
NIL_EXPRESSION, null);
JCVariableDecl newLockField = recursiveSetGeneratedBy(maker.VarDef(
maker.Modifiers(Flags.PRIVATE | Flags.FINAL | (isStatic ? Flags.STATIC : 0)),
methodNode.toName(lockName), lockVarType, lockInstance), annotationNode);
injectFieldAndMarkGenerated(methodNode.up(), newLockField);
}
if (method.body == null) return;
JCExpression lockNode;
if (isStatic) {
lockNode = namePlusTypeParamsToTypeReference(maker, typeNode,
methodNode.toName(lockName), false, List.<JCTypeParameter>nil());
} else {
lockNode = maker.Select(maker.Ident(methodNode.toName("this")), methodNode.toName(lockName));
}
JCExpressionStatement acquireLock = maker.Exec(maker.Apply(NIL_EXPRESSION,
maker.Select(getLockable(maker, typeNode, methodNode, lockableMethodName, lockNode),
annotationNode.toName("lock")), NIL_EXPRESSION));
JCExpressionStatement releaseLock = maker.Exec(maker.Apply(NIL_EXPRESSION,
maker.Select(getLockable(maker, typeNode, methodNode, lockableMethodName, lockNode),
annotationNode.toName("unlock")), NIL_EXPRESSION));
JCTry tryBlock = setGeneratedBy(maker.Try(method.body, List.<JCCatch>nil(),
recursiveSetGeneratedBy(maker.Block(0, List.<JCStatement>of(releaseLock)), annotationNode)),
annotationNode);
```

```
method.body = setGeneratedBy(maker.Block(0,
List.<JCStatement>of(recursiveSetGeneratedBy(acquireLock, annotationNode), tryBlock)),
annotationNode);
methodNode.rebuild();
}
private static JCExpression getLockable(JavacTreeMaker maker, JavacNode typeNode, JavacNode
methodNode, String lockableMethodName, JCExpression lockNode) {
if (lockableMethodName == null) {
return cloneType(maker, lockNode, typeNode);
} else {
return maker.Apply(NIL_EXPRESSION, maker.Select(cloneType(maker, lockNode, typeNode),
methodNode.toName(lockableMethodName)), NIL_EXPRESSION);
}
}
}
```

lombok/javac/handlers/HandleLockedWrite.java

```
/*
* Copyright (C) 2021-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.Locked;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
import static com.sun.tools.javac.tree.JCTree.*;
/**
* Handles the {@code lombok.Locked.Write} annotation for javac.
*/
@Provides
```

@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the statements generated by it.
public class HandleLockedWrite extends JavacAnnotationHandler<Locked.Write> {
private static final String LOCK_METHOD = "writeLock";
private static final String ANNOTATION_NAME = "@Locked.Write";
private static final String[] LOCK_TYPE_CLASS = new String[]{"java", "util", "concurrent", "locks", "ReadWriteLock"};
private static final String[] LOCK_IMPL_CLASS = new String[]{"java", "util", "concurrent", "locks", "ReentrantReadWriteLock"};
@Override public void handle(AnnotationValues<Locked.Write> annotation, JCAnnotation ast, JavacNode annotationNode) {
String annotationValue = annotation.getInstance().value();
HandleLockedUtil.handle(annotationValue, ast, annotationNode, Locked.Write.class, ANNOTATION_NAME, LOCK_TYPE_CLASS, LOCK_IMPL_CLASS, LOCK_METHOD);
}
}

lombok/javac/handlers/HandleLog.java

/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.CTC_BOT;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.configuration.IdentifierName;
import lombok.core.configuration.LogDeclaration;
import lombok.core.configuration.LogDeclaration.LogFactoryParameter;
import lombok.core.handlers.LoggingFramework;
import lombok.javac.Javac;

```
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.Name;
public class HandleLog {
private static final IdentifierName LOG = IdentifierName.valueOf("log");
private HandleLog() {
throw new UnsupportedOperationException();
}
public static void processAnnotation(LoggingFramework framework, AnnotationValues<?> annotation,
JavacNode annotationNode) {
deleteAnnotationIfNeccessary(annotationNode, framework.getAnnotationClass());
JavacNode typeNode = annotationNode.up();
switch (typeNode.getKind()) {
case TYPE:
IdentifierName logFieldName =
annotationNode.getAst().readConfiguration(ConfigurationKeys.LOG_ANY_FIELD_NAME);
if (logFieldName == null) logFieldName = LOG;
boolean useStatic = !Boolean.FALSE.equals(annotationNode.getAst().readConfiguration(Configuration
Keys.LOG_ANY_FIELD_IS_STATIC));
if ((((JCClassDecl) typeNode.get()).mods.flags & Flags.INTERFACE) != 0) {
annotationNode.addError(framework.getAnnotationAsString() + " is legal only on classes and enums.");
return;
}
if (fieldExists(logFieldName.getName(), typeNode) != MemberExistsResult.NOT_EXISTS) {
annotationNode.addWarning("Field '" + logFieldName + "' already exists.");
return;
}
if (isRecord(typeNode) && !useStatic) {
annotationNode.addError("Logger fields must be static in records.");
return;
}
if (useStatic && !isStaticAllowed(typeNode)) {
annotationNode.addError(framework.getAnnotationAsString() + " is not supported on non-static nested
classes.");
return;
}
Object valueGuess = annotation.getValueGuess("topic");
JCExpression loggerTopic = (JCExpression) annotation.getActualExpression("topic");
if (valueGuess instanceof String && ((String) valueGuess).trim().isEmpty()) loggerTopic = null;
if (framework.getDeclaration().getParametersWithTopic() == null && loggerTopic != null) {
annotationNode.addError(framework.getAnnotationAsString() + " does not allow a topic.");
```

```
loggerTopic = null;
}
if (framework.getDeclaration().getParametersWithoutTopic() == null && loggerTopic == null) {
annotationNode.addError(framework.getAnnotationAsString() + " requires a topic.");
loggerTopic = typeNode.getTreeMaker().Literal("");
}
JCFieldAccess loggingType = selfType(typeNode);
createField(framework, typeNode, loggingType, annotationNode, logFieldName.getName(), useStatic,
loggerTopic);
break;
default:
annotationNode.addError("@Log is legal only on types.");
break;
}
}
public static JCFieldAccess selfType(JavacNode typeNode) {
JavacTreeMaker maker = typeNode.getTreeMaker();
Name name = ((JCClassDecl) typeNode.get()).name;
return maker.Select(maker.Ident(name), typeNode.toName("class"));
}
private static boolean createField(LoggingFramework framework, JavacNode typeNode,
JCFieldAccess loggingType, JavacNode source, String logFieldName, boolean useStatic,
JCExpression loggerTopic) {
JavacTreeMaker maker = typeNode.getTreeMaker();
LogDeclaration logDeclaration = framework.getDeclaration();
// private static final <loggerType> log = <factoryMethod>(<parameter>);
JCExpression loggerType = chainDotsString(typeNode, logDeclaration.getLoggerType().getName());
JCExpression factoryMethod = chainDotsString(typeNode,
logDeclaration.getLoggerFactoryType().getName() + "." +
logDeclaration.getLoggerFactoryMethod().getName());
java.util.List<LogFactoryParameter> parameters = loggerTopic != null ?
logDeclaration.getParametersWithTopic() : logDeclaration.getParametersWithoutTopic();
JCExpression[] factoryParameters = createFactoryParameters(typeNode, loggingType, parameters,
loggerTopic);
JCMethodInvocation factoryMethodCall = maker.Apply(List.<JCExpression>nil(), factoryMethod,
List.<JCExpression>from(factoryParameters));
JCVariableDecl fieldDecl = recursiveSetGeneratedBy(maker.VarDef(
maker.Modifiers(Flags.PRIVATE | Flags.FINAL | (useStatic ? Flags.STATIC : 0)),
typeNode.toName(logFieldName), loggerType, factoryMethodCall), source);
if (isRecord(typeNode) && Javac.getJavaCompilerVersion() < 16) {
// This is a workaround for https://bugs.openjdk.java.net/browse/JDK-8243057 - note that the
workaround is only for static fields in records, but our infra _requires_ `static` for logs in records (think
about it).
injectField(typeNode, fieldDecl);
} else {
injectFieldAndMarkGenerated(typeNode, fieldDecl);
}
return true;
}
private static JCExpression[] createFactoryParameters(JavacNode typeNode, JCFieldAccess
loggingType, java.util.List<LogFactoryParameter> parameters, JCExpression loggerTopic) {
JCExpression[] expressions = new JCExpression[parameters.size()];
JavacTreeMaker maker = typeNode.getTreeMaker();
```

```
for (int i = 0; i < parameters.size(); i++) {
LogFactoryParameter parameter = parameters.get(i);
switch (parameter) {
case TYPE:
expressions[i] = cloneType(maker, loggingType, typeNode);
break;
case NAME:
JCExpression method = maker.Select(loggingType, typeNode.toName("getName"));
expressions[i] = maker.Apply(List.<JCExpression>nil(), method, List.<JCExpression>nil());
break;
case TOPIC:
if (loggerTopic instanceof JCLiteral) {
expressions[i] = maker.Literal(((JCLiteral) loggerTopic).value);
} else {
expressions[i] = cloneType(maker, loggerTopic, typeNode);
}
break;
case NULL:
expressions[i] = maker.Literal(CTC_BOT, null);
break;
default:
throw new IllegalStateException("Unknown logger factory parameter type: " + parameter);
}
}
return expressions;
}
/**
* Handles the {@link lombok.extern.apachecommons.CommonsLog} annotation for javac.
*/
@Provides
public static class HandleCommonsLog extends
JavacAnnotationHandler<lombok.extern.apachecommons.CommonsLog> {
@Override public void handle(AnnotationValues<lombok.extern.apachecommons.CommonsLog>
annotation, JCAnnotation ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_COMMONS_FLAG_USAGE,
"@apachecommons.CommonsLog", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.COMMONS, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.java.Log} annotation for javac.
*/
@Provides
public static class HandleJulLog extends JavacAnnotationHandler<lombok.extern.java.Log> {
@Override public void handle(AnnotationValues<lombok.extern.java.Log> annotation, JCAnnotation
ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_JUL_FLAG_USAGE, "@java.Log",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.JUL, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.log4j.Log4j} annotation for javac.
```

```
*/
@Provides
public static class HandleLog4jLog extends JavacAnnotationHandler<lombok.extern.log4j.Log4j> {
@Override public void handle(AnnotationValues<lombok.extern.log4j.Log4j> annotation, JCAnnotation
ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_LOG4J_FLAG_USAGE, "@Log4j",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.LOG4J, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.log4j.Log4j2} annotation for javac.
*/
@Provides
public static class HandleLog4j2Log extends JavacAnnotationHandler<lombok.extern.log4j.Log4j2> {
@Override public void handle(AnnotationValues<lombok.extern.log4j.Log4j2> annotation,
JCAnnotation ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_LOG4J2_FLAG_USAGE, "@Log4j2",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.LOG4J2, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.slf4j.Slf4j} annotation for javac.
*/
@Provides
public static class HandleSlf4jLog extends JavacAnnotationHandler<lombok.extern.slf4j.Slf4j> {
@Override public void handle(AnnotationValues<lombok.extern.slf4j.Slf4j> annotation, JCAnnotation
ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_SLF4J_FLAG_USAGE, "@Slf4j",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.SLF4J, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.slf4j.XSlf4j} annotation for javac.
*/
@Provides
public static class HandleXSlf4jLog extends JavacAnnotationHandler<lombok.extern.slf4j.XSlf4j> {
@Override public void handle(AnnotationValues<lombok.extern.slf4j.XSlf4j> annotation, JCAnnotation
ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_XSLF4J_FLAG_USAGE, "@XSlf4j",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.XSLF4J, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.jbosslog.JBossLog} annotation for javac.
*/
@Provides
public static class HandleJBossLog extends
JavacAnnotationHandler<lombok.extern.jbosslog.JBossLog> {
```

```
@Override public void handle(AnnotationValues<lombok.extern.jbosslog.JBossLog> annotation,
JCAnnotation ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_JBOSSLOG_FLAG_USAGE,
"@JBossLog", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.JBOSSLOG, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.extern.flogger.Flogger} annotation for javac.
*/
@Provides
public static class HandleFloggerLog extends
JavacAnnotationHandler<lombok.extern.flogger.Flogger> {
@Override public void handle(AnnotationValues<lombok.extern.flogger.Flogger> annotation,
JCAnnotation ast, JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_FLOGGER_FLAG_USAGE, "@Flogger",
ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
processAnnotation(LoggingFramework.FLOGGER, annotation, annotationNode);
}
}
/**
* Handles the {@link lombok.CustomLog} annotation for javac.
*/
@Provides
public static class HandleCustomLog extends JavacAnnotationHandler<lombok.CustomLog> {
@Override public void handle(AnnotationValues<lombok.CustomLog> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.LOG_CUSTOM_FLAG_USAGE,
"@CustomLog", ConfigurationKeys.LOG_ANY_FLAG_USAGE, "any @Log");
LogDeclaration logDeclaration =
annotationNode.getAst().readConfiguration(ConfigurationKeys.LOG_CUSTOM_DECLARATION);
if (logDeclaration == null) {
annotationNode.addError("The @CustomLog is not configured; please set
lombok.log.custom.declaration in lombok.config.");
return;
}
LoggingFramework framework = new LoggingFramework(lombok.CustomLog.class, logDeclaration);
processAnnotation(framework, annotation, annotationNode);
}
}
}
```

lombok/javac/handlers/HandleNonNull.java

```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
```

* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.javac.Javac.*;
import static lombok.javac.JavacTreeMaker.TreeTag.treeTag;
import static lombok.javac.JavacTreeMaker.TypeTag.typeTag;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.ArrayList;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCAssert;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCBinary;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCIf;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCParens;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCSynchronized;
import com.sun.tools.javac.tree.JCTree.JCThrow;
import com.sun.tools.javac.tree.JCTree.JCTry;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.NonNull;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
```

```
import lombok.core.HandlerPriority;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
@Provides
@HandlerPriority(value = 512) // 2^9; onParameter=@__(@NonNull) has to run first.
public class HandleNonNull extends JavacAnnotationHandler<NonNull> {
private JCMethodDecl createRecordArgslessConstructor(JavacNode typeNode, JavacNode source,
JCMethodDecl existingCtr) {
JavacTreeMaker maker = typeNode.getTreeMaker();
java.util.List<JCVariableDecl> fields = new ArrayList<JCVariableDecl>();
for (JavacNode child : typeNode.down()) {
if (child.getKind() == Kind.FIELD) {
JCVariableDecl v = (JCVariableDecl) child.get();
if ((v.mods.flags & RECORD) != 0) {
fields.add(v);
}
}
}
ListBuffer<JCVariableDecl> params = new ListBuffer<JCVariableDecl>();
for (int i = 0; i < fields.size(); i++) {
JCVariableDecl arg = fields.get(i);
JCModifiers mods = maker.Modifiers(GENERATED_MEMBER | Flags.PARAMETER,
arg.mods.annotations);
params.append(maker.VarDef(mods, arg.name, arg.vartype, null));
}
JCModifiers mods = maker.Modifiers(toJavacModifier(AccessLevel.PUBLIC) |
COMPACT_RECORD_CONSTRUCTOR, List.<JCAnnotation>nil());
JCBlock body = maker.Block(0L, List.<JCStatement>nil());
if (existingCtr == null) {
JCMethodDecl constr = maker.MethodDef(mods, typeNode.toName("<init>"), null,
List.<JCTypeParameter>nil(), params.toList(), List.<JCExpression>nil(), body, null);
return recursiveSetGeneratedBy(constr, source);
} else {
existingCtr.mods = mods;
existingCtr.body = body;
existingCtr = recursiveSetGeneratedBy(existingCtr, source);
addSuppressWarningsAll(existingCtr.mods, typeNode,
typeNode.getNodeFor(getGeneratedBy(existingCtr)), typeNode.getContext());
addGenerated(existingCtr.mods, typeNode, typeNode.getNodeFor(getGeneratedBy(existingCtr)),
typeNode.getContext());
return existingCtr;
}
}
/**
* If the provided typeNode is a record, returns the compact constructor (there should only be one, but if
the file is
* not semantically sound there might be more). If the only one in existence is the default auto-generated
one, it is removed,
* a new explicit one is created, and that one is returned in a list.
*
* Otherwise, an empty list is returned.
```

```
*/
private List<JCMethodDecl> addCompactConstructorIfNeeded(JavacNode typeNode, JavacNode
source) {
List<JCMethodDecl> answer = List.nil();
if (typeNode == null || !(typeNode.get() instanceof JCClassDecl)) return answer;
JCClassDecl cDecl = (JCClassDecl) typeNode.get();
if ((cDecl.mods.flags & RECORD) == 0) return answer;
boolean generateConstructor = false;
JCMethodDecl existingCtr = null;
for (JCTree def : cDecl.defs) {
if (def instanceof JCMethodDecl) {
JCMethodDecl md = (JCMethodDecl) def;
if (md.name.contentEquals("<init>")) {
if ((md.mods.flags & Flags.GENERATEDCONSTR) != 0) {
existingCtr = md;
existingCtr.mods.flags = existingCtr.mods.flags & ~Flags.GENERATEDCONSTR;
generateConstructor = true;
} else {
if (!isTolerate(typeNode, md)) {
if ((md.mods.flags & COMPACT_RECORD_CONSTRUCTOR) != 0) {
generateConstructor = false;
answer = answer.prepend(md);
}
}
}
}
}
}
if (generateConstructor) {
JCMethodDecl ctr;
if (existingCtr != null) {
ctr = createRecordArgslessConstructor(typeNode, source, existingCtr);
} else {
ctr = createRecordArgslessConstructor(typeNode, source, null);
injectMethod(typeNode, ctr);
}
answer = answer.prepend(ctr);
}
return answer;
}
private void addNullCheckIfNeeded(JCMethodDecl method, JavacNode paramNode, JavacNode
source) {
// Possibly, if 'declaration instanceof ConstructorDeclaration', fetch declaration.constructorCall, search it
for any references to our parameter,
// and if they exist, create a new method in the class: 'private static <T> T lombok$nullCheck(T expr,
String msg) {if (expr == null) throw NPE; return expr;}' and
// wrap all references to it in the super/this to a call to this method.
JCStatement nullCheck = recursiveSetGeneratedBy(generateNullCheck(source.getTreeMaker(),
paramNode, source), source);
if (nullCheck == null) {
// @NonNull applied to a primitive. Kinda pointless. Let's generate a warning.
source.addWarning("@NonNull is meaningless on a primitive.");
return;
```

```
}
List<JCStatement> statements = method.body.stats;
String expectedName = paramNode.getName();
/* Abort if the null check is already there, delving into try and synchronized statements */ {
List<JCStatement> stats = statements;
int idx = 0;
while (stats.size() > idx) {
JCStatement stat = stats.get(idx++);
if (JavacHandlerUtil.isConstructorCall(stat)) continue;
if (stat instanceof JCTry) {
stats = ((JCTry) stat).body.stats;
idx = 0;
continue;
}
if (stat instanceof JCSynchronized) {
stats = ((JCSynchronized) stat).body.stats;
idx = 0;
continue;
}
String varNameOfNullCheck = returnVarNameIfNullCheck(stat);
if (varNameOfNullCheck == null) break;
if (varNameOfNullCheck.equals(expectedName)) return;
}
}
List<JCStatement> tail = statements;
List<JCStatement> head = List.nil();
for (JCStatement stat : statements) {
if (JavacHandlerUtil.isConstructorCall(stat) || (JavacHandlerUtil.isGenerated(stat) &&
isNullCheck(stat))) {
tail = tail.tail;
head = head.prepend(stat);
continue;
}
break;
}
List<JCStatement> newList = tail.prepend(nullCheck);
for (JCStatement stat : head) newList = newList.prepend(stat);
method.body.stats = newList;
source.getAst().setChanged();
}
@Override public void handle(AnnotationValues<NonNull> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.NON_NULL_FLAG_USAGE, "@NonNull");
final JavacNode node;
if (annotationNode.up().getKind() == Kind.TYPE_USE) {
node = annotationNode.directUp().directUp();
} else {
node = annotationNode.up();
}
if (node.getKind() == Kind.FIELD) {
// This is meaningless unless the field is used to generate a method (@Setter,
@RequiredArgsConstructor, etc),
```

```
// but in that case those handlers will take care of it. However, we DO check if the annotation is applied
to
// a primitive, because those handlers trigger on any annotation named @NonNull and we only want the
warning
// behaviour on _OUR_ 'lombok.NonNull'.
try {
if (isPrimitive(((JCVariableDecl) node.get()).vartype)) {
annotationNode.addWarning("@NonNull is meaningless on a primitive.");
}
} catch (Exception ignore) {}
JCVariableDecl fDecl = (JCVariableDecl) node.get();
if ((fDecl.mods.flags & RECORD) != 0) {
// well, these kinda double as parameters (of the compact constructor), so we do some work here.
List<JCMethodDecl> compactConstructors = addCompactConstructorIfNeeded(node.up(),
annotationNode);
for (JCMethodDecl ctr : compactConstructors) {
addNullCheckIfNeeded(ctr, node, annotationNode);
}
}
return;
}
if (node.getKind() != Kind.ARGUMENT) return;
JCMethodDecl declaration;
try {
declaration = (JCMethodDecl) node.up().get();
} catch (Exception e) {
return;
}
if (declaration.body == null) {
// This used to be a warning, but as @NonNull also has a documentary purpose, better to not warn
about this. Since 1.16.7
return;
}
if ((declaration.mods.flags & (GENERATED_MEMBER | COMPACT_RECORD_CONSTRUCTOR)) !=
0) {
// The 'real' annotations are on the `record Foo(@NonNull Obj x)` part and we just see these
// syntax-sugared over. We deal with it on the field declaration variant, as those are always there,
// not dependent on whether you write out the compact constructor or not.
return;
}
addNullCheckIfNeeded(declaration, node, annotationNode);
}
public boolean isNullCheck(JCStatement stat) {
return returnVarNameIfNullCheck(stat) != null;
}
/**
* Checks if the statement is of the form 'if (x == null) {throw WHATEVER;}' or 'assert x != null;',
* where the block braces are optional. If it is of this form, returns "x".
* If it is not of this form, returns null.
*/
public String returnVarNameIfNullCheck(JCStatement stat) {
boolean isIf = stat instanceof JCIf;
boolean isExpression = stat instanceof JCExpressionStatement;
```

```
if (!isIf && !(stat instanceof JCAssert) && !isExpression) return null;
if (isExpression) {
/* Check if the statements contains a call to checkNotNull or requireNonNull */
JCExpression expression = ((JCExpressionStatement) stat).expr;
if (expression instanceof JCAssign) expression = ((JCAssign) expression).rhs;
if (!(expression instanceof JCMethodInvocation)) return null;
JCMethodInvocation invocation = (JCMethodInvocation) expression;
JCExpression method = invocation.meth;
Name name = null;
if (method instanceof JCFieldAccess) {
name = ((JCFieldAccess) method).name;
} else if (method instanceof JCIdent) {
name = ((JCIdent) method).name;
}
if (name == null || (!name.contentEquals("checkNotNull") && !name.contentEquals("requireNonNull")))
return null;
if (invocation.args.isEmpty()) return null;
JCExpression firstArgument = invocation.args.head;
if (!(firstArgument instanceof JCIdent)) return null;
return ((JCIdent) firstArgument).toString();
}
if (isIf) {
/* Check that the if's statement is a throw statement, possibly in a block. */
JCStatement then = ((JCIf) stat).thenpart;
if (then instanceof JCBlock) {
List<JCStatement> stats = ((JCBlock) then).stats;
if (stats.length() == 0) return null;
then = stats.get(0);
}
if (!(then instanceof JCThrow)) return null;
}
/* Check that the if's conditional is like 'x == null'. Return from this method (don't generate
a nullcheck) if 'x' is equal to our own variable's name: There's already a nullcheck here. */ {
JCExpression cond = isIf ? ((JCIf) stat).cond : ((JCAssert) stat).cond;
while (cond instanceof JCParens) cond = ((JCParens) cond).expr;
if (!(cond instanceof JCBinary)) return null;
JCBinary bin = (JCBinary) cond;
if (isIf) {
if (!CTC_EQUAL.equals(treeTag(bin))) return null;
} else {
if (!CTC_NOT_EQUAL.equals(treeTag(bin))) return null;
}
if (!(bin.lhs instanceof JCIdent)) return null;
if (!(bin.rhs instanceof JCLiteral)) return null;
if (!CTC_BOT.equals(typeTag(bin.rhs))) return null;
return ((JCIdent) bin.lhs).name.toString();
}
}
}
```

lombok/javac/handlers/HandlePrintAST.java

/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintStream;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import lombok.Lombok;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.core.PrintAST;
import lombok.javac.JavacASTVisitor;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
/**
* Handles the {@code lombok.core.PrintAST} annotation for javac.
*/
@Provides
@HandlerPriority(536870912) // 2^29; this handler is customarily run at the very end.
public class HandlePrintAST extends JavacAnnotationHandler<PrintAST> {
@Override public void handle(AnnotationValues<PrintAST> annotation, JCAnnotation ast, JavacNode
annotationNode) {
PrintStream stream = System.out;
String fileName = annotation.getInstance().outfile();
if (fileName.length() > 0) try {
stream = new PrintStream(new File(fileName));
} catch (FileNotFoundException e) {
throw Lombok.sneakyThrow(e);
}
try {
annotationNode.up().traverse(new JavacASTVisitor.Printer(annotation.getInstance().printContent(),
stream));

```
} finally {
if (stream != System.out) {
try {
stream.close();
} catch (Exception e) {
throw Lombok.sneakyThrow(e);
}
}
}
}
}
```

lombok/javac/handlers/HandleSetter.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.javac.Javac.*;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.Collection;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.Setter;
import lombok.core.AST.Kind;
import lombok.experimental.Accessors;
import lombok.core.AnnotationValues;
import lombok.javac.Javac;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
```

```
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
/**
* Handles the {@code lombok.Setter} annotation for javac.
*/
@Provides
public class HandleSetter extends JavacAnnotationHandler<Setter> {
private static final String SETTER_NODE_NOT_SUPPORTED_ERR = "@Setter is only supported on a
class or a field.";
public void generateSetterForType(JavacNode typeNode, JavacNode errorNode, AccessLevel level,
boolean checkForTypeLevelSetter, List<JCAnnotation> onMethod, List<JCAnnotation> onParam) {
if (checkForTypeLevelSetter) {
if (hasAnnotation(Setter.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
}
if (!isClass(typeNode)) {
errorNode.addError(SETTER_NODE_NOT_SUPPORTED_ERR);
return;
}
for (JavacNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
//Skip fields that start with $
if (fieldDecl.name.toString().startsWith("$")) continue;
//Skip static fields.
if ((fieldDecl.mods.flags & Flags.STATIC) != 0) continue;
//Skip final fields.
if ((fieldDecl.mods.flags & Flags.FINAL) != 0) continue;
generateSetterForField(field, errorNode, level, onMethod, onParam);
}
}
/**
* Generates a setter on the stated field.
*
* Used by {@link HandleData}.
*
* The difference between this call and the handle method is as follows:
*
* If there is a {@code lombok.Setter} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
```

```
* If not, the setter is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*
* @param fieldNode The node representing the field you want a setter for.
* @param pos The node responsible for generating the setter (the {@code @Data} or {@code
@Setter} annotation).
*/
public void generateSetterForField(JavacNode fieldNode, JavacNode sourceNode, AccessLevel level,
List<JCAnnotation> onMethod, List<JCAnnotation> onParam) {
if (hasAnnotation(Setter.class, fieldNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
createSetterForField(level, fieldNode, sourceNode, false, onMethod, onParam);
}
@Override public void handle(AnnotationValues<Setter> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.SETTER_FLAG_USAGE, "@Setter");
Collection<JavacNode> fields = annotationNode.upFromAnnotationToFields();
deleteAnnotationIfNeccessary(annotationNode, Setter.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode node = annotationNode.up();
AccessLevel level = annotation.getInstance().value();
if (level == AccessLevel.NONE || node == null) return;
List<JCAnnotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@Setter(onMethod", annotationNode);
if (!onMethod.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@Setter(onMethod=...)");
}
List<JCAnnotation> onParam = unboxAndRemoveAnnotationParameter(ast, "onParam",
"@Setter(onParam", annotationNode);
if (!onParam.isEmpty()) {
handleFlagUsage(annotationNode, ConfigurationKeys.ON_X_FLAG_USAGE,
"@Setter(onParam=...)");
}
switch (node.getKind()) {
case FIELD:
createSetterForFields(level, fields, annotationNode, true, onMethod, onParam);
break;
case TYPE:
generateSetterForType(node, annotationNode, level, false, onMethod, onParam);
break;
}
}
public void createSetterForFields(AccessLevel level, Collection<JavacNode> fieldNodes, JavacNode
errorNode, boolean whineIfExists, List<JCAnnotation> onMethod, List<JCAnnotation> onParam) {
for (JavacNode fieldNode : fieldNodes) {
createSetterForField(level, fieldNode, errorNode, whineIfExists, onMethod, onParam);
}
}
public void createSetterForField(AccessLevel level, JavacNode fieldNode, JavacNode sourceNode,
boolean whineIfExists, List<JCAnnotation> onMethod, List<JCAnnotation> onParam) {
```

```
if (fieldNode.getKind() != Kind.FIELD) {
fieldNode.addError(SETTER_NODE_NOT_SUPPORTED_ERR);
return;
}
AnnotationValues<Accessors> accessors = JavacHandlerUtil.getAccessorsForField(fieldNode);
JCVariableDecl fieldDecl = (JCVariableDecl) fieldNode.get();
String methodName = toSetterName(fieldNode, accessors);
if (methodName == null) {
fieldNode.addWarning("Not generating setter for this field: It does not fit your @Accessors prefix list.");
return;
}
if ((fieldDecl.mods.flags & Flags.FINAL) != 0) {
fieldNode.addWarning("Not generating setter for this field: Setters cannot be generated for final
fields.");
return;
}
for (String altName : toAllSetterNames(fieldNode, accessors)) {
switch (methodExists(altName, fieldNode, false, 1)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (whineIfExists) {
String altNameExpl = "";
if (!altName.equals(methodName)) altNameExpl = String.format(" (%s)", altName);
fieldNode.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", methodName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
//continue scanning the other alt names.
}
}
long access = toJavacModifier(level) | (fieldDecl.mods.flags & Flags.STATIC);
JCMethodDecl createdSetter = createSetter(access, fieldNode, fieldNode.getTreeMaker(),
sourceNode, onMethod, onParam);
injectMethod(fieldNode.up(), createdSetter);
}
public static JCMethodDecl createSetter(long access, JavacNode field, JavacTreeMaker treeMaker,
JavacNode source, List<JCAnnotation> onMethod, List<JCAnnotation> onParam) {
AnnotationValues<Accessors> accessors = JavacHandlerUtil.getAccessorsForField(field);
String setterName = toSetterName(field, accessors);
boolean returnThis = shouldReturnThis(field, accessors);
JCMethodDecl setter = createSetter(access, false, field, treeMaker, setterName, null, null, returnThis,
source, onMethod, onParam);
return setter;
}
public static JCMethodDecl createSetter(long access, boolean deprecate, JavacNode field,
JavacTreeMaker treeMaker, String setterName, Name paramName, Name booleanFieldToSet,
boolean shouldReturnThis, JavacNode source, List<JCAnnotation> onMethod, List<JCAnnotation>
onParam) {
JCExpression returnType = null;
```

```
JCReturn returnStatement = null;
if (shouldReturnThis) {
returnType = cloneSelfType(field);
returnType = addCheckerFrameworkReturnsReceiver(returnType, treeMaker, field,
getCheckerFrameworkVersion(source));
returnStatement = treeMaker.Return(treeMaker.Ident(field.toName("this")));
}
return createSetter(access, deprecate, field, treeMaker, setterName, paramName, booleanFieldToSet,
returnType, returnStatement, source, onMethod, onParam);
}
public static JCMethodDecl createSetterWithRecv(long access, boolean deprecate, JavacNode field,
JavacTreeMaker treeMaker, String setterName, Name paramName, Name booleanFieldToSet,
boolean shouldReturnThis, JavacNode source, List<JCAnnotation> onMethod, List<JCAnnotation>
onParam, JCVariableDecl recv) {
JCExpression returnType = null;
JCReturn returnStatement = null;
if (shouldReturnThis) {
returnType = cloneSelfType(field);
returnType = addCheckerFrameworkReturnsReceiver(returnType, treeMaker, field,
getCheckerFrameworkVersion(source));
returnStatement = treeMaker.Return(treeMaker.Ident(field.toName("this")));
}
JCMethodDecl d = createSetterWithRecv(access, deprecate, field, treeMaker, setterName,
paramName, booleanFieldToSet, returnType, returnStatement, source, onMethod, onParam, recv);
return d;
}
public static JCMethodDecl createSetter(long access, boolean deprecate, JavacNode field,
JavacTreeMaker treeMaker, String setterName, Name paramName, Name booleanFieldToSet,
JCExpression methodType, JCStatement returnStatement, JavacNode source, List<JCAnnotation>
onMethod, List<JCAnnotation> onParam) {
return createSetterWithRecv(access, deprecate, field, treeMaker, setterName, paramName,
booleanFieldToSet, methodType, returnStatement, source, onMethod, onParam, null);
}
public static JCMethodDecl createSetterWithRecv(long access, boolean deprecate, JavacNode field,
JavacTreeMaker treeMaker, String setterName, Name paramName, Name booleanFieldToSet,
JCExpression methodType, JCStatement returnStatement, JavacNode source, List<JCAnnotation>
onMethod, List<JCAnnotation> onParam, JCVariableDecl recv) {
if (setterName == null) return null;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
if (paramName == null) paramName = fieldDecl.name;
JCExpression fieldRef = createFieldAccessor(treeMaker, field, FieldAccess.ALWAYS_FIELD);
JCAssign assign = treeMaker.Assign(fieldRef, treeMaker.Ident(paramName));
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
List<JCAnnotation> copyableAnnotations = findCopyableAnnotations(field);
Name methodName = field.toName(setterName);
List<JCAnnotation> annsOnParam = copyAnnotations(onParam).appendList(copyableAnnotations);
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, field.getContext());
JCExpression pType = cloneType(treeMaker, fieldDecl.vartype, source);
JCVariableDecl param = treeMaker.VarDef(treeMaker.Modifiers(flags, annsOnParam), paramName,
pType, null);
if (!hasNonNullAnnotations(field) && !hasNonNullAnnotations(field, onParam)) {
statements.append(treeMaker.Exec(assign));
} else {
```

```
JCStatement nullCheck = generateNullCheck(treeMaker, fieldDecl.vartype, paramName, source, null);
if (nullCheck != null) statements.append(nullCheck);
statements.append(treeMaker.Exec(assign));
}
if (booleanFieldToSet != null) {
JCAssign setBool = treeMaker.Assign(treeMaker.Ident(booleanFieldToSet),
treeMaker.Literal(CTC_BOOLEAN, 1));
statements.append(treeMaker.Exec(setBool));
}
if (methodType == null) {
//WARNING: Do not use field.getSymbolTable().voidType - that field has gone through non-backwards
compatible API changes within javac1.6.
methodType = treeMaker.Type(Javac.createVoidType(field.getSymbolTable(), CTC_VOID));
returnStatement = null;
}
if (returnStatement != null) statements.append(returnStatement);
JCBlock methodBody = treeMaker.Block(0, statements.toList());
List<JCTypeParameter> methodGenericParams = List.nil();
List<JCVariableDecl> parameters = List.of(param);
List<JCExpression> throwsClauses = List.nil();
JCExpression annotationMethodDefaultValue = null;
List<JCAnnotation> annsOnMethod = mergeAnnotations(copyAnnotations(onMethod),
findCopyableToSetterAnnotations(field));
if (isFieldDeprecated(field) || deprecate) {
annsOnMethod = annsOnMethod.prepend(treeMaker.Annotation(genJavaLangTypeRef(field,
"Deprecated"), List.<JCExpression>nil()));
}
AnnotationValues<Accessors> accessors = JavacHandlerUtil.getAccessorsForField(field);
if (shouldMakeFinal(field, accessors)) access |= Flags.FINAL;
JCMethodDecl methodDef;
if (recv != null && treeMaker.hasMethodDefWithRecvParam()) {
methodDef = treeMaker.MethodDefWithRecvParam(treeMaker.Modifiers(access, annsOnMethod),
methodName, methodType,
methodGenericParams, recv, parameters, throwsClauses, methodBody,
annotationMethodDefaultValue);
} else {
methodDef = treeMaker.MethodDef(treeMaker.Modifiers(access, annsOnMethod), methodName,
methodType,
methodGenericParams, parameters, throwsClauses, methodBody, annotationMethodDefaultValue);
}
if (returnStatement != null) createRelevantNonNullAnnotation(source, methodDef);
JCMethodDecl decl = recursiveSetGeneratedBy(methodDef, source);
copyJavadoc(field, decl, CopyJavadoc.SETTER, returnStatement != null);
return decl;
}
}
```

lombok/javac/handlers/HandleSingularRemove.java

```
/*
* Copyright (C) 2023 The Project Lombok Authors.
```

*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import lombok.Singular;
import lombok.core.AlreadyHandledAnnotations;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
@Provides
@HandlerPriority(32768)
@AlreadyHandledAnnotations
public class HandleSingularRemove extends JavacAnnotationHandler<Singular> {
@Override public void handle(AnnotationValues<Singular> annotation, JCAnnotation ast, JavacNode
annotationNode) {
deleteAnnotationIfNeccessary(annotationNode, Singular.class);
deleteImportFromCompilationUnit(annotationNode, Singular.class.getName());
}
}
```

lombok/javac/handlers/HandleSneakyThrows.java

/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:

*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import lombok.ConfigurationKeys;
import lombok.SneakyThrows;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTry;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import lombok.javac.Javac;
/**
* Handles the {@code lombok.SneakyThrows} annotation for javac.
*/
@Provides
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the
statements generated by it.
public class HandleSneakyThrows extends JavacAnnotationHandler<SneakyThrows> {
@Override public void handle(AnnotationValues<SneakyThrows> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.SNEAKY_THROWS_FLAG_USAGE,
"@SneakyThrows");
deleteAnnotationIfNeccessary(annotationNode, SneakyThrows.class);
Collection<String> exceptionNames = annotation.getRawExpressions("value");
if (exceptionNames.isEmpty()) {
exceptionNames = Collections.singleton("java.lang.Throwable");
}
```

```
java.util.List<String> exceptions = new ArrayList<String>();
for (String exception : exceptionNames) {
if (exception.endsWith(".class")) exception = exception.substring(0, exception.length() - 6);
exceptions.add(exception);
}
JavacNode owner = annotationNode.up();
switch (owner.getKind()) {
case METHOD:
handleMethod(annotationNode, (JCMethodDecl)owner.get(), exceptions);
break;
default:
annotationNode.addError("@SneakyThrows is legal only on methods and constructors.");
break;
}
}
public void handleMethod(JavacNode annotation, JCMethodDecl method, Collection<String>
exceptions) {
JavacNode methodNode = annotation.up();
if ( (method.mods.flags & Flags.ABSTRACT) != 0) {
annotation.addError("@SneakyThrows can only be used on concrete methods.");
return;
}
if (method.body == null || method.body.stats.isEmpty()) {
generateEmptyBlockWarning(methodNode, annotation, false);
return;
}
final JCStatement constructorCall = method.body.stats.get(0);
final boolean isConstructorCall = isConstructorCall(constructorCall);
List<JCStatement> contents = isConstructorCall ? method.body.stats.tail : method.body.stats;
if (contents == null || contents.isEmpty()) {
generateEmptyBlockWarning(methodNode, annotation, true);
return;
}
for (String exception : exceptions) {
contents = List.of(buildTryCatchBlock(methodNode, contents, exception, annotation));
}
method.body.stats = isConstructorCall ? List.of(constructorCall).appendList(contents) : contents;
methodNode.rebuild();
}
public void generateEmptyBlockWarning(JavacNode methodNode, JavacNode annotation, boolean
hasConstructorCall) {
if (hasConstructorCall) {
annotation.addWarning("Calls to sibling / super constructors are always excluded from
@SneakyThrows; @SneakyThrows has been ignored because there is no other code in this
constructor.");
} else {
annotation.addWarning("This method or constructor is empty; @SneakyThrows has been ignored.");
}
}
public JCStatement buildTryCatchBlock(JavacNode node, List<JCStatement> contents, String
exception, JavacNode source) {
JavacTreeMaker maker = node.getTreeMaker();
JCBlock tryBlock = setGeneratedBy(maker.Block(0, contents), source);
```

```
JCExpression varType = chainDots(node, exception.split("\\."));
JCVariableDecl catchParam = maker.VarDef(maker.Modifiers(Flags.FINAL | Flags.PARAMETER),
node.toName("$ex"), varType, null);
JCExpression lombokLombokSneakyThrowNameRef = chainDots(node, "lombok", "Lombok",
"sneakyThrow");
JCBlock catchBody = maker.Block(0, List.<JCStatement>of(maker.Throw(maker.Apply(
List.<JCExpression>nil(), lombokLombokSneakyThrowNameRef,
List.<JCExpression>of(maker.Ident(node.toName("$ex")))))));
JCTry tryStatement = maker.Try(tryBlock, List.of(recursiveSetGeneratedBy(maker.Catch(catchParam,
catchBody), source)), null);
if (JavacHandlerUtil.inNetbeansEditor(node)) {
//set span (start and end position) of the try statement and the main block
//this allows NetBeans to dive into the statement correctly:
JCCompilationUnit top = (JCCompilationUnit) node.top().get();
int startPos = contents.head.pos;
int endPos = Javac.getEndPosition(contents.last().pos(), top);
tryBlock.pos = startPos;
tryStatement.pos = startPos;
Javac.storeEnd(tryBlock, endPos, top);
Javac.storeEnd(tryStatement, endPos, top);
}
return setGeneratedBy(tryStatement, source);
}
}
```

lombok/javac/handlers/HandleStandardException.java


```
/*
* Copyright (C) 2021-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.javac.handlers;
```

```
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.*;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.experimental.StandardException;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.delombok.LombokOptionsFactory;
import lombok.javac.Javac;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil.*;
import lombok.spi.Provides;
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
@Provides
public class HandleStandardException extends JavacAnnotationHandler<StandardException> {
@Override
public void handle(AnnotationValues<StandardException> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.STANDARD_EXCEPTION_FLAG_USAGE,
"@StandardException");
deleteAnnotationIfNeccessary(annotationNode, StandardException.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode typeNode = annotationNode.up();
if (!isClass(typeNode)) {
annotationNode.addError("@StandardException is only supported on a class");
return;
}
JCTree extending = Javac.getExtendsClause((JCClassDecl) typeNode.get());
if (extending == null) {
annotationNode.addError("@StandardException requires that you extend a Throwable type");
return;
}
AccessLevel access = annotation.getInstance().access();
if (access == null) access = AccessLevel.PUBLIC;
if (access == AccessLevel.NONE) {
annotationNode.addError("AccessLevel.NONE is not valid here");
access = AccessLevel.PUBLIC;
}
generateNoArgsConstructor(typeNode, access, annotationNode);
generateMsgOnlyConstructor(typeNode, access, annotationNode);
generateCauseOnlyConstructor(typeNode, access, annotationNode);
generateFullConstructor(typeNode, access, annotationNode);
}
private void generateNoArgsConstructor(JavacNode typeNode, AccessLevel level, JavacNode source)
{
```

```
if (hasConstructor(typeNode) != MemberExistsResult.NOT_EXISTS) return;
JavacTreeMaker maker = typeNode.getTreeMaker();
JCExpression stringArgument = maker.TypeCast(genJavaLangTypeRef(typeNode, "String"),
maker.Literal(CTC_BOT, null));
JCExpression throwableArgument = maker.TypeCast(genJavaLangTypeRef(typeNode, "Throwable"),
maker.Literal(CTC_BOT, null));
List<JCExpression> args = List.<JCExpression>of(stringArgument, throwableArgument);
JCStatement thisCall = maker.Exec(maker.Apply(List.<JCExpression>nil(),
maker.Ident(typeNode.toName("this")), args));
JCMethodDecl constr = createConstructor(level, typeNode, false, false, source, List.of(thisCall));
injectMethod(typeNode, constr);
}
private void generateMsgOnlyConstructor(JavacNode typeNode, AccessLevel level, JavacNode
source) {
if (hasConstructor(typeNode, String.class) != MemberExistsResult.NOT_EXISTS) return;
JavacTreeMaker maker = typeNode.getTreeMaker();
JCExpression stringArgument = maker.Ident(typeNode.toName("message"));
JCExpression throwableArgument = maker.TypeCast(genJavaLangTypeRef(typeNode, "Throwable"),
maker.Literal(CTC_BOT, null));
List<JCExpression> args = List.<JCExpression>of(stringArgument, throwableArgument);
JCStatement thisCall = maker.Exec(maker.Apply(List.<JCExpression>nil(),
maker.Ident(typeNode.toName("this")), args));
JCMethodDecl constr = createConstructor(level, typeNode, true, false, source, List.of(thisCall));
injectMethod(typeNode, constr);
}
private void generateCauseOnlyConstructor(JavacNode typeNode, AccessLevel level, JavacNode
source) {
if (hasConstructor(typeNode, Throwable.class) != MemberExistsResult.NOT_EXISTS) return;
JavacTreeMaker maker = typeNode.getTreeMaker();
Name causeName = typeNode.toName("cause");
JCExpression causeDotGetMessage = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(causeName), typeNode.toName("getMessage")), List.<JCExpression>nil());
JCExpression msgExpression = maker.Conditional(maker.Binary(CTC_NOT_EQUAL,
maker.Ident(causeName), maker.Literal(CTC_BOT, null)), causeDotGetMessage,
maker.Literal(CTC_BOT, null));
List<JCExpression> args = List.<JCExpression>of(msgExpression, maker.Ident(causeName));
JCStatement thisCall = maker.Exec(maker.Apply(List.<JCExpression>nil(),
maker.Ident(typeNode.toName("this")), args));
JCMethodDecl constr = createConstructor(level, typeNode, false, true, source, List.of(thisCall));
injectMethod(typeNode, constr);
}
private void generateFullConstructor(JavacNode typeNode, AccessLevel level, JavacNode source) {
if (hasConstructor(typeNode, String.class, Throwable.class) != MemberExistsResult.NOT_EXISTS)
return;
JavacTreeMaker maker = typeNode.getTreeMaker();
Name causeName = typeNode.toName("cause");
Name superName = typeNode.toName("super");
List<JCExpression> args = List.<JCExpression>of(maker.Ident(typeNode.toName("message")));
JCStatement superCall = maker.Exec(maker.Apply(List.<JCExpression>nil(),
maker.Ident(superName), args));
JCExpression causeNotNull = maker.Binary(CTC_NOT_EQUAL, maker.Ident(causeName),
maker.Literal(CTC_BOT, null));
```

```
JCStatement initCauseCall = maker.Exec(maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(superName), typeNode.toName("initCause")),
List.<JCExpression>of(maker.Ident(causeName))));
JCStatement initCause = maker.If(causeNotNull, initCauseCall, null);
JCMethodDecl constr = createConstructor(level, typeNode, true, true, source, List.of(superCall,
initCause));
injectMethod(typeNode, constr);
}
private static MemberExistsResult hasConstructor(JavacNode node, Class<?>... paramTypes) {
node = upToTypeNode(node);
if (node != null && node.get() instanceof JCClassDecl) {
for (JCTree def : ((JCClassDecl) node.get()).defs) {
if (def instanceof JCMethodDecl) {
JCMethodDecl md = (JCMethodDecl) def;
if (md.name.contentEquals("<init>") && (md.mods.flags & Flags.GENERATEDCONSTR) == 0) {
if (!paramsMatch(node, md.params, paramTypes)) continue;
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
}
}
return MemberExistsResult.NOT_EXISTS;
}
private static boolean paramsMatch(JavacNode node, List<JCVariableDecl> a, Class<?>[] b) {
if (a == null) return b == null || b.length == 0;
if (b == null) return a.size() == 0;
if (a.size() != b.length) return false;
for (int i = 0; i < a.size(); i++) {
JCVariableDecl param = a.get(i);
Class<?> c = b[i];
if (!typeMatches(c, node, param.vartype)) return false;
}
return true;
}
private static void addConstructorProperties(JCModifiers mods, JavacNode node, boolean msgParam,
boolean causeParam) {
if (!msgParam && !causeParam) return;
JavacTreeMaker maker = node.getTreeMaker();
JCExpression constructorPropertiesType = chainDots(node, "java", "beans", "ConstructorProperties");
ListBuffer<JCExpression> fieldNames = new ListBuffer<JCExpression>();
if (msgParam) fieldNames.append(maker.Literal("message"));
if (causeParam) fieldNames.append(maker.Literal("cause"));
JCExpression fieldNamesArray = maker.NewArray(null, List.<JCExpression>nil(), fieldNames.toList());
JCAnnotation annotation = maker.Annotation(constructorPropertiesType, List.of(fieldNamesArray));
mods.annotations = mods.annotations.append(annotation);
}
@SuppressWarnings("deprecation") private static JCMethodDecl createConstructor(AccessLevel level,
JavacNode typeNode, boolean msgParam, boolean causeParam, JavacNode source,
List<JCStatement> statements) {
JavacTreeMaker maker = typeNode.getTreeMaker();
boolean addConstructorProperties;
```

```
if ((!msgParam && !causeParam) || isLocalType(typeNode) || !LombokOptionsFactory.getDelombokOpt
ions(typeNode.getContext()).getFormatPreferences().generateConstructorProperties()) {
addConstructorProperties = false;
} else {
Boolean v = typeNode.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTOR_ADD_C
ONSTRUCTOR_PROPERTIES);
addConstructorProperties = v != null ? v.booleanValue() :
Boolean.FALSE.equals(typeNode.getAst().readConfiguration(ConfigurationKeys.ANY_CONSTRUCTO
R_SUPPRESS_CONSTRUCTOR_PROPERTIES));
}
ListBuffer<JCVariableDecl> params = new ListBuffer<JCVariableDecl>();
if (msgParam) {
Name fieldName = typeNode.toName("message");
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, typeNode.getContext());
JCExpression pType = genJavaLangTypeRef(typeNode, "String");
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags), fieldName, pType, null);
params.append(param);
}
if (causeParam) {
Name fieldName = typeNode.toName("cause");
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, typeNode.getContext());
JCExpression pType = genJavaLangTypeRef(typeNode, "Throwable");
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags), fieldName, pType, null);
params.append(param);
}
JCModifiers mods = maker.Modifiers(toJavacModifier(level), List.<JCAnnotation>nil());
if (addConstructorProperties) addConstructorProperties(mods, typeNode, msgParam, causeParam);
return recursiveSetGeneratedBy(maker.MethodDef(mods, typeNode.toName("<init>"),
null, List.<JCTypeParameter>nil(), params.toList(), List.<JCExpression>nil(),
maker.Block(0L, statements), null), source);
}
public static boolean isLocalType(JavacNode type) {
Kind kind = type.up().getKind();
if (kind == Kind.COMPILATION_UNIT) return false;
if (kind == Kind.TYPE) return isLocalType(type.up());
return true;
}
}
```

lombok/javac/handlers/HandleSuperBuilder.java


```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
```

* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static lombok.javac.handlers.HandleBuilder.*;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import javax.lang.model.element.Modifier;
import com.sun.tools.javac.code.BoundKind;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeApply;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.JCWildcard;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.AccessLevel;
import lombok.Builder;
import lombok.Builder.ObtainVia;
import lombok.ConfigurationKeys;
import lombok.Singular;
import lombok.ToString;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
```

```
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.experimental.NonFinal;
import lombok.experimental.SuperBuilder;
import lombok.javac.Javac;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.HandleBuilder.BuilderFieldData;
import lombok.javac.handlers.HandleBuilder.BuilderJob;
import lombok.javac.handlers.JavacHandlerUtil.CopyJavadoc;
import lombok.javac.handlers.JavacHandlerUtil.JCAnnotatedTypeReflect;
import lombok.javac.handlers.JavacHandlerUtil.MemberExistsResult;
import lombok.javac.handlers.JavacSingularsRecipes.ExpressionMaker;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
import lombok.javac.handlers.JavacSingularsRecipes.StatementMaker;
import lombok.spi.Provides;
@Provides
@HandlerPriority(-1024) //-2^10; to ensure we've picked up @FieldDefault's changes (-2048) but
@Value hasn't removed itself yet (-512), so that we can error on presence of it on the builder classes.
public class HandleSuperBuilder extends JavacAnnotationHandler<SuperBuilder> {
private static final String SELF_METHOD = "self";
private static final String FILL_VALUES_METHOD_NAME = "$fillValuesFrom";
private static final String STATIC_FILL_VALUES_METHOD_NAME =
"$fillValuesFromInstanceIntoBuilder";
private static final String INSTANCE_VARIABLE_NAME = "instance";
private static final String BUILDER_VARIABLE_NAME = "b";
class SuperBuilderJob extends BuilderJob {
JavacNode builderAbstractType;
String builderAbstractClassName;
JavacNode builderImplType;
String builderImplClassName;
List<JCTypeParameter> builderTypeParams_;
void init(AnnotationValues<SuperBuilder> annValues, SuperBuilder ann, JavacNode node) {
accessOuters = accessInners = AccessLevel.PUBLIC;
oldFluent = true;
oldChain = true;
builderMethodName = ann.builderMethodName();
buildMethodName = ann.buildMethodName();
toBuilder = ann.toBuilder();
if (builderMethodName == null) builderMethodName = "builder";
if (buildMethodName == null) buildMethodName = "build";
builderClassName = getBuilderClassNameTemplate(node, null);
}
void setBuilderToImpl() {
builderType = builderImplType;
builderClassName = builderImplClassName;
builderTypeParams = typeParams;
}
void setBuilderToAbstract() {
```

```
builderType = builderAbstractType;
builderClassName = builderAbstractClassName;
builderTypeParams = builderTypeParams_;
}
}
@Override
public void handle(AnnotationValues<SuperBuilder> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.SUPERBUILDER_FLAG_USAGE,
"@SuperBuilder");
SuperBuilderJob job = new SuperBuilderJob();
job.sourceNode = annotationNode;
job.checkerFramework = getCheckerFrameworkVersion(annotationNode);
job.isStatic = true;
SuperBuilder annInstance = annotation.getInstance();
job.init(annotation, annInstance, annotationNode);
boolean generateBuilderMethod;
if (job.builderMethodName.isEmpty()) {
generateBuilderMethod = false;
} else if (!checkName("builderMethodName", job.builderMethodName, annotationNode)) {
return;
} else {
generateBuilderMethod = true;
}
if (!checkName("buildMethodName", job.buildMethodName, annotationNode)) return;
// Do not delete the SuperBuilder annotation here, we need it for @Jacksonized.
JavacNode parent = annotationNode.up();
job.builderFields = new ArrayList<BuilderFieldData>();
job.typeParams = List.nil();
List<JCExpression> buildMethodThrownExceptions = List.nil();
List<JCExpression> superclassTypeParams = List.nil();
boolean addCleaning = false;
if (!isClass(parent)) {
annotationNode.addError("@SuperBuilder is only supported on classes.");
return;
}
if (!isStaticAllowed(parent)) {
annotationNode.addError("@SuperBuilder is not supported on non-static nested classes.");
return;
}
job.parentType = parent;
JCClassDecl td = (JCClassDecl) parent.get();
// Gather all fields of the class that should be set by the builder.
ArrayList<JavacNode> nonFinalNonDefaultedFields = null;
boolean valuePresent = (hasAnnotation(lombok.Value.class, parent) ||
hasAnnotation("lombok.experimental.Value", parent));
for (JavacNode fieldNode : HandleConstructor.findAllFields(parent, true)) {
JCVariableDecl fd = (JCVariableDecl) fieldNode.get();
JavacNode isDefault = findAnnotation(Builder.Default.class, fieldNode, false);
boolean isFinal = (fd.mods.flags & Flags.FINAL) != 0 || (valuePresent &&
!hasAnnotation(NonFinal.class, fieldNode));
BuilderFieldData bfd = new BuilderFieldData();
bfd.rawName = fd.name;
```

```
bfd.name = removePrefixFromField(fieldNode);
bfd.builderFieldName = bfd.name;
bfd.annotations = findCopyableAnnotations(fieldNode);
bfd.type = fd.vartype;
bfd.singularData = getSingularData(fieldNode, annInstance.setterPrefix());
bfd.originalFieldNode = fieldNode;
if (bfd.singularData != null && isDefault != null) {
isDefault.addError("@Builder.Default and @Singular cannot be mixed.");
findAnnotation(Builder.Default.class, fieldNode, true);
isDefault = null;
}
if (fd.init == null && isDefault != null) {
isDefault.addWarning("@Builder.Default requires an initializing expression (' = something;').");
findAnnotation(Builder.Default.class, fieldNode, true);
isDefault = null;
}
if (fd.init != null && isDefault == null) {
if (isFinal) continue;
if (nonFinalNonDefaultedFields == null) nonFinalNonDefaultedFields = new ArrayList<JavacNode>();
nonFinalNonDefaultedFields.add(fieldNode);
}
if (isDefault != null) {
bfd.nameOfDefaultProvider = parent.toName(DEFAULT_PREFIX + bfd.name);
bfd.nameOfSetFlag = parent.toName(bfd.name + SET_PREFIX);
bfd.builderFieldName = parent.toName(bfd.name + VALUE_PREFIX);
JCMethodDecl md = HandleBuilder.generateDefaultProvider(bfd.nameOfDefaultProvider, fieldNode,
td.typarams, job);
if (md != null) injectMethod(parent, md);
}
addObtainVia(bfd, fieldNode);
job.builderFields.add(bfd);
}
job.typeParams = job.builderTypeParams = td.typarams;
job.builderClassName = job.replaceBuilderClassName(td.name);
if (!checkName("builderClassName", job.builderClassName, annotationNode)) return;
// <C, B> are the generics for our builder.
String classGenericName = "C";
String builderGenericName = "B";
// We have to make sure that the generics' names do not collide with any generics on the annotated
class,
// the classname itself, or any member type name of the annotated class.
// For instance, if there are generics <B, B2, C> on the annotated class, use "C2" and "B3" for our
builder.
java.util.HashSet<String> usedNames = gatherUsedTypeNames(job.typeParams, td);
classGenericName = generateNonclashingNameFor(classGenericName, usedNames);
builderGenericName = generateNonclashingNameFor(builderGenericName, usedNames);
JavacTreeMaker maker = annotationNode.getTreeMaker();
{
JCExpression annotatedClass = namePlusTypeParamsToTypeReference(maker, parent,
job.typeParams);
JCTypeParameter c = maker.TypeParameter(parent.toName(classGenericName),
List.<JCExpression>of(annotatedClass));
```

```
ListBuffer<JCExpression> typeParamsForBuilder = getTypeParamExpressions(job.typeParams,
maker, job.sourceNode);
typeParamsForBuilder.append(maker.Ident(parent.toName(classGenericName)));
typeParamsForBuilder.append(maker.Ident(parent.toName(builderGenericName)));
JCTypeApply typeApply = maker.TypeApply(namePlusTypeParamsToTypeReference(maker, parent,
job.getBuilderClassName(), false, List.<JCTypeParameter>nil()), typeParamsForBuilder.toList());
JCTypeParameter d = maker.TypeParameter(parent.toName(builderGenericName),
List.<JCExpression>of(typeApply));
if (job.typeParams == null || job.typeParams.isEmpty()) {
job.builderTypeParams_ = List.of(c, d);
} else {
job.builderTypeParams_ = job.typeParams.append(c).append(d);
}
}
JCTree extendsClause = Javac.getExtendsClause(td);
JCExpression superclassBuilderClass = null;
if (extendsClause instanceof JCTypeApply) {
// Remember the type arguments, because we need them for the extends clause of our abstract builder
class.
superclassTypeParams = ((JCTypeApply) extendsClause).getTypeArguments();
// A class name with a generics type, e.g., "Superclass<A>".
extendsClause = ((JCTypeApply) extendsClause).getType();
}
if (extendsClause instanceof JCFieldAccess) {
Name superclassName = ((JCFieldAccess) extendsClause).getIdentifier();
String builderClassNameTemplate = BuilderJob.getBuilderClassNameTemplate(annotationNode, null);
String superclassBuilderClassName = job.replaceBuilderClassName(superclassName.toString(),
builderClassNameTemplate);
superclassBuilderClass = parent.getTreeMaker().Select(cloneType(maker, (JCFieldAccess)
extendsClause, annotationNode), parent.toName(superclassBuilderClassName));
} else if (extendsClause != null) {
String builderClassNameTemplate = BuilderJob.getBuilderClassNameTemplate(annotationNode, null);
String superclassBuilderClassName = job.replaceBuilderClassName(extendsClause.toString(),
builderClassNameTemplate);
superclassBuilderClass = chainDots(parent, extendsClause.toString(), superclassBuilderClassName);
}
// Check validity of @ObtainVia fields, and add check if adding cleaning for @Singular is necessary.
for (BuilderFieldData bfd : job.builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
if (bfd.singularData.getSingularizer().requiresCleaning()) {
addCleaning = true;
break;
}
}
if (bfd.obtainVia != null) {
if (bfd.obtainVia.field().isEmpty() == bfd.obtainVia.method().isEmpty()) {
bfd.obtainViaNode.addError("The syntax is either @ObtainVia(field = \"fieldName\") or
@ObtainVia(method = \"methodName\").");
return;
}
if (bfd.obtainVia.method().isEmpty() && bfd.obtainVia.isStatic()) {
bfd.obtainViaNode.addError("@ObtainVia(isStatic = true) is not valid unless 'method' has been set.");
return;
```

```
}
}
}
job.builderAbstractClassName = job.builderClassName = job.replaceBuilderClassName(td.name);
job.builderImplClassName = job.builderAbstractClassName + "Impl";
// Create the abstract builder class.
job.builderAbstractType = findInnerClass(parent, job.builderClassName);
if (job.builderAbstractType == null) {
job.builderAbstractType = generateBuilderAbstractClass(job, superclassBuilderClass,
superclassTypeParams, classGenericName, builderGenericName);
recursiveSetGeneratedBy(job.builderAbstractType.get(), annotationNode);
} else {
JCClassDecl builderTypeDeclaration = (JCClassDecl) job.builderAbstractType.get();
if (!builderTypeDeclaration.getModifiers().getFlags().contains(Modifier.STATIC)
|| !builderTypeDeclaration.getModifiers().getFlags().contains(Modifier.ABSTRACT)) {
annotationNode.addError("Existing Builder must be an abstract static inner class.");
return;
}
sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(job.builderAbstractType,
annotationNode);
// Generate errors for @Singular BFDs that have one already defined node.
for (BuilderFieldData bfd : job.builderFields) {
SingularData sd = bfd.singularData;
if (sd == null) continue;
JavacSingularizer singularizer = sd.getSingularizer();
if (singularizer == null) continue;
if (singularizer.checkForAlreadyExistingNodesAndGenerateError(job.builderAbstractType, sd)) {
bfd.singularData = null;
}
}
}
// Generate the fields in the abstract builder class that hold the values for the instance.
job.setBuilderToAbstract();
generateBuilderFields(job.builderType, job.builderFields, annotationNode);
if (addCleaning) {
JCVariableDecl uncleanField = maker.VarDef(maker.Modifiers(Flags.PRIVATE),
job.toName("$lombokUnclean"), maker.TypeIdent(CTC_BOOLEAN), null);
recursiveSetGeneratedBy(uncleanField, annotationNode);
injectFieldAndMarkGenerated(job.builderType, uncleanField);
}
if (job.toBuilder) {
// Generate $fillValuesFrom() method in the abstract builder.
JCMethodDecl fvm = generateFillValuesMethod(job, superclassBuilderClass != null,
builderGenericName, classGenericName);
recursiveSetGeneratedBy(fvm, annotationNode);
injectMethod(job.builderType, fvm);
// Generate $fillValuesFromInstanceIntoBuilder() method in the builder implementation class.
JCMethodDecl sfvm = generateStaticFillValuesMethod(job, annInstance.setterPrefix());
recursiveSetGeneratedBy(sfvm, annotationNode);
injectMethod(job.builderType, sfvm);
}
// Create the setter methods in the abstract builder.
for (BuilderFieldData bfd : job.builderFields) {
```

```
generateSetterMethodsForBuilder(job, bfd, builderGenericName, annInstance.setterPrefix());
}
// Generate abstract self() and build() methods in the abstract builder.
JCMethodDecl asm = generateAbstractSelfMethod(job, superclassBuilderClass != null,
builderGenericName);
recursiveSetGeneratedBy(asm, annotationNode);
injectMethod(job.builderType, asm);
JCMethodDecl abm = generateAbstractBuildMethod(job, superclassBuilderClass != null,
classGenericName);
recursiveSetGeneratedBy(abm, annotationNode);
injectMethod(job.builderType, abm);
// Create the toString() method for the abstract builder.
java.util.List<Included<JavacNode, ToString.Include>> fieldNodes = new
ArrayList<Included<JavacNode, ToString.Include>>();
for (BuilderFieldData bfd : job.builderFields) {
for (JavacNode f : bfd.createdFields) {
fieldNodes.add(new Included<JavacNode, ToString.Include>(f, null, true, false));
}
}
// Let toString() call super.toString() if there is a superclass, so that it also shows fields from the
superclass' builder.
JCMethodDecl toStringMethod = HandleToString.createToString(job.builderType, fieldNodes, true,
superclassBuilderClass != null, FieldAccess.ALWAYS_FIELD, annotationNode);
if (toStringMethod != null) injectMethod(job.builderType, toStringMethod);
// If clean methods are requested, add them now.
if (addCleaning) {
JCMethodDecl md = generateCleanMethod(job.builderFields, job.builderType, annotationNode);
recursiveSetGeneratedBy(md, annotationNode);
injectMethod(job.builderType, md);
}
boolean isAbstract = (td.mods.flags & Flags.ABSTRACT) != 0;
if (!isAbstract) {
// Only non-abstract classes get the Builder implementation.
// Create the builder implementation class.
job.builderImplType = findInnerClass(parent, job.builderImplClassName);
if (job.builderImplType == null) {
job.builderImplType = generateBuilderImplClass(job);
recursiveSetGeneratedBy(job.builderImplType.get(), annotationNode);
} else {
JCClassDecl builderImplTypeDeclaration = (JCClassDecl) job.builderImplType.get();
if (!builderImplTypeDeclaration.getModifiers().getFlags().contains(Modifier.STATIC)
|| builderImplTypeDeclaration.getModifiers().getFlags().contains(Modifier.ABSTRACT)) {
annotationNode.addError("Existing BuilderImpl must be a non-abstract static inner class.");
return;
}
sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(job.builderImplType, annotationNode);
}
// Create a simple constructor for the BuilderImpl class.
JCMethodDecl cd = HandleConstructor.createConstructor(AccessLevel.PRIVATE,
List.<JCAnnotation>nil(), job.builderImplType, List.<JavacNode>nil(), false, annotationNode);
if (cd != null) injectMethod(job.builderImplType, cd);
job.setBuilderToImpl();
// Create the self() and build() methods in the BuilderImpl.
```

```
JCMethodDecl selfMethod = generateSelfMethod(job);
recursiveSetGeneratedBy(selfMethod, annotationNode);
injectMethod(job.builderType, selfMethod);
if (methodExists(job.buildMethodName, job.builderType, -1) == MemberExistsResult.NOT_EXISTS) {
JCMethodDecl buildMethod = generateBuildMethod(job, buildMethodThrownExceptions);
recursiveSetGeneratedBy(buildMethod, annotationNode);
injectMethod(job.builderType, buildMethod);
}
}
// Generate a constructor in the annotated class that takes a builder as argument.
if (!constructorExists(job.parentType, job.builderAbstractClassName)) {
job.setBuilderToAbstract();
generateBuilderBasedConstructor(job, superclassBuilderClass != null);
}
if (!isAbstract) {
// Only non-abstract classes get the builder() and toBuilder() methods.
// Add the builder() method to the annotated class.
// Allow users to specify their own builder() methods, e.g., to provide default values.
if (generateBuilderMethod && methodExists(job.builderMethodName, job.parentType, -1) !=
MemberExistsResult.NOT_EXISTS) generateBuilderMethod = false;
if (generateBuilderMethod) {
JCMethodDecl builderMethod = generateBuilderMethod(job);
if (builderMethod != null) {
recursiveSetGeneratedBy(builderMethod, annotationNode);
injectMethod(job.parentType, builderMethod);
}
}
// Add the toBuilder() method to the annotated class.
if (job.toBuilder) {
switch (methodExists(TO_BUILDER_METHOD_NAME, job.parentType, 0)) {
case EXISTS_BY_USER:
break;
case NOT_EXISTS:
JCMethodDecl md = generateToBuilderMethod(job);
if (md != null) {
recursiveSetGeneratedBy(md, annotationNode);
injectMethod(job.parentType, md);
}
break;
default:
// Should not happen.
}
}
}
if (nonFinalNonDefaultedFields != null && generateBuilderMethod) {
for (JavacNode fieldNode : nonFinalNonDefaultedFields) {
fieldNode.addWarning("@SuperBuilder will ignore the initializing expression entirely. If you want the
initializing expression to serve as default, add @Builder.Default. If it is not supposed to be settable
during building, make the field final.");
}
}
}
/**
```

```
* Creates and returns the abstract builder class and injects it into the annotated class.
*/
private JavacNode generateBuilderAbstractClass(SuperBuilderJob job, JCExpression
superclassBuilderClass, List<JCExpression> superclassTypeParams, String classGenericName, String
builderGenericName) {
JavacTreeMaker maker = job.parentType.getTreeMaker();
JCModifiers mods = maker.Modifiers(Flags.STATIC | Flags.ABSTRACT | Flags.PUBLIC);
// Keep any type params of the annotated class.
ListBuffer<JCTypeParameter> allTypeParams = new ListBuffer<JCTypeParameter>();
allTypeParams.appendList(copyTypeParams(job.sourceNode, job.typeParams));
// Add builder-specific type params required for inheritable builders.
// 1. The return type for the build() method, named "C", which extends the annotated class.
JCExpression annotatedClass = namePlusTypeParamsToTypeReference(maker, job.parentType,
job.typeParams);
allTypeParams.append(maker.TypeParameter(job.toName(classGenericName),
List.<JCExpression>of(annotatedClass)));
// 2. The return type for all setter methods, named "B", which extends this builder class.
Name builderClassName = job.toName(job.builderClassName);
ListBuffer<JCExpression> typeParamsForBuilder = getTypeParamExpressions(job.typeParams,
maker, job.sourceNode);
typeParamsForBuilder.append(maker.Ident(job.toName(classGenericName)));
typeParamsForBuilder.append(maker.Ident(job.toName(builderGenericName)));
JCTypeApply typeApply = maker.TypeApply(namePlusTypeParamsToTypeReference(maker,
job.parentType, builderClassName, false, List.<JCTypeParameter>nil()),
typeParamsForBuilder.toList());
allTypeParams.append(maker.TypeParameter(job.toName(builderGenericName),
List.<JCExpression>of(typeApply)));
JCExpression extending = null;
if (superclassBuilderClass != null) {
// If the annotated class extends another class, we want this builder to extend the builder of the
superclass.
// 1. Add the type parameters of the superclass.
typeParamsForBuilder = getTypeParamExpressions(superclassTypeParams, maker, job.sourceNode);
// 2. Add the builder type params <C, B>.
typeParamsForBuilder.append(maker.Ident(job.toName(classGenericName)));
typeParamsForBuilder.append(maker.Ident(job.toName(builderGenericName)));
extending = maker.TypeApply(superclassBuilderClass, typeParamsForBuilder.toList());
}
JCClassDecl builder = maker.ClassDef(mods, builderClassName, allTypeParams.toList(), extending,
List.<JCExpression>nil(), List.<JCTree>nil());
recursiveSetGeneratedBy(builder, job.sourceNode);
return injectType(job.parentType, builder);
}
/**
* Creates and returns the concrete builder implementation class and injects it into the annotated class.
*/
private JavacNode generateBuilderImplClass(SuperBuilderJob job) {
JavacTreeMaker maker = job.getTreeMaker();
JCModifiers mods = maker.Modifiers(Flags.STATIC | Flags.PRIVATE | Flags.FINAL);
// Extend the abstract builder.
JCExpression extending = namePlusTypeParamsToTypeReference(maker, job.parentType,
job.toName(job.builderAbstractClassName), false, List.<JCTypeParameter>nil());
// Add builder-specific type params required for inheritable builders.
```

```
// 1. The return type for the build() method (named "C" in the abstract builder), which is the annotated class.
JCExpression annotatedClass = namePlusTypeParamsToTypeReference(maker, job.parentType, job.typeParams);
// 2. The return type for all setter methods (named "B" in the abstract builder), which is this builder class.
JCExpression builderImplClassExpression = namePlusTypeParamsToTypeReference(maker, job.parentType, job.toName(job.builderImplClassName), false, job.typeParams);
ListBuffer<JCExpression> typeParamsForBuilder = getTypeParamExpressions(job.typeParams, maker, job.sourceNode);
typeParamsForBuilder.append(annotatedClass);
typeParamsForBuilder.append(builderImplClassExpression);
extending = maker.TypeApply(extending, typeParamsForBuilder.toList());
JCClassDecl builder = maker.ClassDef(mods, job.toName(job.builderImplClassName), copyTypeParams(job.parentType, job.typeParams), extending, List.<JCExpression>nil(), List.<JCTree>nil());
recursiveSetGeneratedBy(builder, job.sourceNode);
return injectType(job.parentType, builder);
}
/**
* Generates a constructor that has a builder as the only parameter.
* The values from the builder are used to initialize the fields of new instances.
*
* @param callBuilderBasedSuperConstructor
* If {@code true}, the constructor will explicitly call a super
* constructor with the builder as argument. Requires
* {@code builderClassAsParameter != null}.
*/
private void generateBuilderBasedConstructor(SuperBuilderJob job, boolean callBuilderBasedSuperConstructor) {
JavacTreeMaker maker = job.getTreeMaker();
AccessLevel level = AccessLevel.PROTECTED;
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
Name builderVariableName = job.toName(BUILDER_VARIABLE_NAME);
for (BuilderFieldData bfd : job.builderFields) {
JCExpression rhs;
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.singularData.getSingularizer().appendBuildCode(bfd.singularData, bfd.originalFieldNode, job.sourceNode, statements, bfd.builderFieldName, "b");
rhs = maker.Ident(bfd.singularData.getPluralName());
} else {
rhs = maker.Select(maker.Ident(builderVariableName), bfd.builderFieldName);
}
JCFieldAccess fieldInThis = maker.Select(maker.Ident(job.toName("this")), bfd.rawName);
JCStatement assign = maker.Exec(maker.Assign(fieldInThis, rhs));
// In case of @Builder.Default, set the value to the default if it was not set in the builder.
if (bfd.nameOfSetFlag != null) {
JCFieldAccess setField = maker.Select(maker.Ident(builderVariableName), bfd.nameOfSetFlag);
fieldInThis = maker.Select(maker.Ident(job.toName("this")), bfd.rawName);
JCExpression parentTypeRef = namePlusTypeParamsToTypeReference(maker, job.parentType, List.<JCTypeParameter>nil());
JCAssign assignDefault = maker.Assign(fieldInThis, maker.Apply(typeParameterNames(maker, ((JCClassDecl) job.parentType.get()).typarams), maker.Select(parentTypeRef,
```

```
bfd.nameOfDefaultProvider), List.<JCExpression>nil()));
statements.append(maker.If(setField, assign, maker.Exec(assignDefault)));
} else {
statements.append(assign);
}
if (hasNonNullAnnotations(bfd.originalFieldNode)) {
JCStatement nullCheck = generateNullCheck(maker, bfd.originalFieldNode, job.sourceNode);
if (nullCheck != null) statements.append(nullCheck);
}
}
List<JCAnnotation> annsOnMethod = job.checkerFramework.generateSideEffectFree() ?
List.of(maker.Annotation(genTypeRef(job.parentType,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil())) :
List.<JCAnnotation>nil();
JCModifiers mods = maker.Modifiers(toJavacModifier(level), annsOnMethod);
// Create a constructor that has just the builder as parameter.
ListBuffer<JCVariableDecl> params = new ListBuffer<JCVariableDecl>();
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, job.getContext());
// First add all generics that are present on the parent type.
ListBuffer<JCExpression> typeParamsForBuilderParameter =
getTypeParamExpressions(job.typeParams, maker, job.sourceNode);
// Now add the <?, ?>.
JCWildcard wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParamsForBuilderParameter.append(wildcard);
wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParamsForBuilderParameter.append(wildcard);
JCTypeApply paramType = maker.TypeApply(namePlusTypeParamsToTypeReference(maker,
job.parentType, job.getBuilderClassName(), false, List.<JCTypeParameter>nil()),
typeParamsForBuilderParameter.toList());
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags), builderVariableName, paramType,
null);
params.append(param);
if (callBuilderBasedSuperConstructor) {
// The first statement must be the call to the super constructor.
JCMethodInvocation callToSuperConstructor = maker.Apply(List.<JCExpression>nil(),
maker.Ident(job.toName("super")),
List.<JCExpression>of(maker.Ident(builderVariableName)));
statements.prepend(maker.Exec(callToSuperConstructor));
}
JCMethodDecl constr = recursiveSetGeneratedBy(maker.MethodDef(mods, job.toName("<init>"),
null, List.<JCTypeParameter>nil(), params.toList(), List.<JCExpression>nil(),
maker.Block(0L, statements.toList()), null), job.sourceNode);
injectMethod(job.parentType, constr);
}
private JCMethodDecl generateBuilderMethod(SuperBuilderJob job) {
JavacTreeMaker maker = job.getTreeMaker();
JCExpression call = maker.NewClass(null, List.<JCExpression>nil(),
namePlusTypeParamsToTypeReference(maker, job.parentType,
job.toName(job.builderImplClassName), false, job.typeParams), List.<JCExpression>nil(), null);
JCStatement statement = maker.Return(call);
JCBlock body = maker.Block(0, List.<JCStatement>of(statement));
int modifiers = Flags.PUBLIC;
modifiers |= Flags.STATIC;
```

```
// Add any type params of the annotated class to the return type.
ListBuffer<JCExpression> typeParameterNames = new ListBuffer<JCExpression>();
typeParameterNames.appendList(typeParameterNames(maker, job.typeParams));
// Now add the <?, ?>.
JCWildcard wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParameterNames.append(wildcard);
wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParameterNames.append(wildcard);
// And return type annotations.
List<JCAnnotation> annsOnParamType = List.nil();
if (job.checkerFramework.generateUnique()) annsOnParamType =
List.of(maker.Annotation(genTypeRef(job.parentType, CheckerFrameworkVersion.NAME__UNIQUE),
List.<JCExpression>nil()));
JCTypeApply returnType = maker.TypeApply(namePlusTypeParamsToTypeReference(maker,
job.parentType, job.toName(job.builderAbstractClassName), false, List.<JCTypeParameter>nil(),
annsOnParamType), typeParameterNames.toList());
List<JCAnnotation> annsOnMethod = job.checkerFramework.generateSideEffectFree() ?
List.of(maker.Annotation(genTypeRef(job.parentType,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil())) :
List.<JCAnnotation>nil();
JCMethodDecl methodDef = maker.MethodDef(maker.Modifiers(modifiers, annsOnMethod),
job.toName(job.builderMethodName), returnType, copyTypeParams(job.sourceNode, job.typeParams),
List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
createRelevantNonNullAnnotation(job.parentType, methodDef);
return methodDef;
}
/**
* Generates a <code>toBuilder()</code> method in the annotated class that looks like this:
* <pre>
* public ParentBuilder&lt;?, ?&gt; toBuilder() {
* return new <i>Foobar</i>BuilderImpl().$fillValuesFrom(this);
* }
* </pre>
*/
private JCMethodDecl generateToBuilderMethod(SuperBuilderJob job) {
JavacTreeMaker maker = job.getTreeMaker();
JCExpression newClass = maker.NewClass(null, List.<JCExpression>nil(),
namePlusTypeParamsToTypeReference(maker, job.parentType,
job.toName(job.builderImplClassName), false, job.typeParams), List.<JCExpression>nil(), null);
List<JCExpression> methodArgs = List.<JCExpression>of(maker.Ident(job.toName("this")));
JCMethodInvocation invokeFillMethod = maker.Apply(List.<JCExpression>nil(),
maker.Select(newClass, job.toName(FILL_VALUES_METHOD_NAME)), methodArgs);
JCStatement statement = maker.Return(invokeFillMethod);
JCBlock body = maker.Block(0, List.<JCStatement>of(statement));
int modifiers = Flags.PUBLIC;
// Add any type params of the annotated class to the return type.
ListBuffer<JCExpression> typeParameterNames = new ListBuffer<JCExpression>();
typeParameterNames.appendList(typeParameterNames(maker, job.typeParams));
// Now add the <?, ?>.
JCWildcard wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParameterNames.append(wildcard);
wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParameterNames.append(wildcard);
```

```
JCTypeApply returnType = maker.TypeApply(namePlusTypeParamsToTypeReference(maker,
job.parentType, job.toName(job.builderAbstractClassName), false, List.<JCTypeParameter>nil()),
typeParameterNames.toList());
List<JCAnnotation> annsOnMethod = job.checkerFramework.generateSideEffectFree() ?
List.of(maker.Annotation(genTypeRef(job.parentType,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil())) :
List.<JCAnnotation>nil();
JCMethodDecl methodDef = maker.MethodDef(maker.Modifiers(modifiers, annsOnMethod),
job.toName(TO_BUILDER_METHOD_NAME), returnType, List.<JCTypeParameter>nil(),
List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
createRelevantNonNullAnnotation(job.parentType, methodDef);
return methodDef;
}
/**
* Generates a <code>$fillValuesFrom()</code> method in the abstract builder class that looks
* like this:
* <pre>
* protected B $fillValuesFrom(final C instance) {
* super.$fillValuesFrom(instance);
* FoobarBuilder.$fillValuesFromInstanceIntoBuilder(instance, this);
* return self();
* }
* </pre>
*/
private JCMethodDecl generateFillValuesMethod(SuperBuilderJob job, boolean inherited, String
builderGenericName, String classGenericName) {
JavacTreeMaker maker = job.getTreeMaker();
List<JCAnnotation> annotations = List.nil();
if (inherited) {
JCAnnotation overrideAnnotation = maker.Annotation(genJavaLangTypeRef(job.builderType,
"Override"), List.<JCExpression>nil());
annotations = List.of(overrideAnnotation);
}
JCModifiers modifiers = maker.Modifiers(Flags.PROTECTED, annotations);
Name name = job.toName(FILL_VALUES_METHOD_NAME);
JCExpression returnType = maker.Ident(job.toName(builderGenericName));
JCExpression classGenericNameExpr = maker.Ident(job.toName(classGenericName));
JCVariableDecl param = maker.VarDef(maker.Modifiers(Flags.PARAMETER | Flags.FINAL),
job.toName(INSTANCE_VARIABLE_NAME), classGenericNameExpr, null);
ListBuffer<JCStatement> body = new ListBuffer<JCStatement>();
if (inherited) {
// Call super.
JCMethodInvocation callToSuper = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(job.toName("super")), name),
List.<JCExpression>of(maker.Ident(job.toName(INSTANCE_VARIABLE_NAME))));
body.append(maker.Exec(callToSuper));
}
// Call the builder implemention's helper method that actually fills the values from the instance.
JCExpression ref = namePlusTypeParamsToTypeReference(maker, job.parentType,
job.getBuilderClassName(), false, List.<JCTypeParameter>nil());
JCMethodInvocation callStaticFillValuesMethod = maker.Apply(List.<JCExpression>nil(),
maker.Select(ref, job.toName(STATIC_FILL_VALUES_METHOD_NAME)),
```

```
List.<JCExpression>of(maker.Ident(job.toName(INSTANCE_VARIABLE_NAME)),
maker.Ident(job.toName("this"))));
body.append(maker.Exec(callStaticFillValuesMethod));
JCReturn returnStatement = maker.Return(maker.Apply(List.<JCExpression>nil(),
maker.Ident(job.toName(SELF_METHOD)), List.<JCExpression>nil()));
body.append(returnStatement);
JCBlock bodyBlock = maker.Block(0, body.toList());
return maker.MethodDef(modifiers, name, returnType, List.<JCTypeParameter>nil(), List.of(param),
List.<JCExpression>nil(), bodyBlock, null);
}
/**
* Generates a <code>$fillValuesFromInstanceIntoBuilder()</code> method in
* the builder implementation class that copies all fields from the instance
* to the builder. It looks like this:
*
* <pre>
* protected B $fillValuesFromInstanceIntoBuilder(Foobar instance, FoobarBuilder&lt;?, ?&gt; b) {
* b.field(instance.field);
* }
* </pre>
*/
private JCMethodDecl generateStaticFillValuesMethod(SuperBuilderJob job, String setterPrefix) {
JavacTreeMaker maker = job.getTreeMaker();
List<JCAnnotation> annotations = List.nil();
JCModifiers modifiers = maker.Modifiers(Flags.PRIVATE | Flags.STATIC, annotations);
Name name = job.toName(STATIC_FILL_VALUES_METHOD_NAME);
JCExpression returnType = maker.TypeIdent(CTC_VOID);
// 1st parameter: "Foobar instance"
JCVariableDecl paramInstance = maker.VarDef(maker.Modifiers(Flags.PARAMETER | Flags.FINAL),
job.toName(INSTANCE_VARIABLE_NAME), cloneSelfType(job.parentType), null);
// 2nd parameter: "FoobarBuilder<?, ?> b" (plus generics on the annotated type)
// First add all generics that are present on the parent type.
ListBuffer<JCExpression> typeParamsForBuilderParameter =
getTypeParamExpressions(job.typeParams, maker, job.sourceNode);
// Now add the <?, ?>.
JCWildcard wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParamsForBuilderParameter.append(wildcard);
wildcard = maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null);
typeParamsForBuilderParameter.append(wildcard);
JCTypeApply builderType = maker.TypeApply(namePlusTypeParamsToTypeReference(maker,
job.parentType, job.getBuilderClassName(), false, List.<JCTypeParameter>nil()),
typeParamsForBuilderParameter.toList());
JCVariableDecl paramBuilder = maker.VarDef(maker.Modifiers(Flags.PARAMETER | Flags.FINAL),
job.toName(BUILDER_VARIABLE_NAME), builderType, null);
ListBuffer<JCStatement> body = new ListBuffer<JCStatement>();
// Call the builder's setter methods to fill the values from the instance.
for (BuilderFieldData bfd : job.builderFields) {
JCExpressionStatement exec = createSetterCallWithInstanceValue(bfd, job, setterPrefix);
body.append(exec);
}
JCBlock bodyBlock = maker.Block(0, body.toList());
return maker.MethodDef(modifiers, name, returnType, copyTypeParams(job.builderType,
job.typeParams), List.of(paramInstance, paramBuilder), List.<JCExpression>nil(), bodyBlock, null);
```

```
}
private JCExpressionStatement createSetterCallWithInstanceValue(BuilderFieldData bfd,
SuperBuilderJob job, String setterPrefix) {
JavacTreeMaker maker = job.getTreeMaker();
JCExpression[] tgt = new JCExpression[bfd.singularData == null ? 1 : 2];
if (bfd.obtainVia == null || !bfd.obtainVia.field().isEmpty()) {
for (int i = 0; i < tgt.length; i++) {
tgt[i] = maker.Select(maker.Ident(job.toName(INSTANCE_VARIABLE_NAME)), bfd.obtainVia == null ?
bfd.rawName : job.toName(bfd.obtainVia.field()));
}
} else {
if (bfd.obtainVia.isStatic()) {
for (int i = 0; i < tgt.length; i++) {
JCExpression typeRef = namePlusTypeParamsToTypeReference(maker, job.parentType,
List.<JCTypeParameter>nil());
JCExpression c = maker.Select(typeRef, job.toName(bfd.obtainVia.method()));
tgt[i] = maker.Apply(List.<JCExpression>nil(), c,
List.<JCExpression>of(maker.Ident(job.toName(INSTANCE_VARIABLE_NAME))));
}
} else {
for (int i = 0; i < tgt.length; i++) {
JCExpression c = maker.Select(maker.Ident(job.toName(INSTANCE_VARIABLE_NAME)),
job.toName(bfd.obtainVia.method()));
tgt[i] = maker.Apply(List.<JCExpression>nil(), c, List.<JCExpression>nil());
}
}
}
JCExpression arg;
if (bfd.singularData == null) {
arg = tgt[0];
} else {
JCExpression eqNull = maker.Binary(CTC_EQUAL, tgt[0], maker.Literal(CTC_BOT, null));
JCExpression emptyCollection =
bfd.singularData.getSingularizer().getEmptyExpression(bfd.singularData.getTargetFqn(), maker,
bfd.singularData, job.parentType, job.sourceNode);
arg = maker.Conditional(eqNull, emptyCollection, tgt[1]);
}
String setterName = HandlerUtil.buildAccessorName(job.sourceNode, setterPrefix,
bfd.name.toString());
JCMethodInvocation apply = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(job.toName(BUILDER_VARIABLE_NAME)), job.toName(setterName)),
List.of(arg));
JCExpressionStatement exec = maker.Exec(apply);
return exec;
}
private JCMethodDecl generateAbstractSelfMethod(SuperBuilderJob job, boolean override, String
builderGenericName) {
JavacTreeMaker maker = job.getTreeMaker();
List<JCAnnotation> annotations = List.nil();
JCAnnotation overrideAnnotation = override ?
maker.Annotation(genJavaLangTypeRef(job.builderType, "Override"), List.<JCExpression>nil()) : null;
JCAnnotation sefAnnotation = job.checkerFramework.generatePure() ?
maker.Annotation(genTypeRef(job.builderType, CheckerFrameworkVersion.NAME__PURE),
```

```
List.<JCExpression>nil()) : null;
if (sefAnnotation != null) annotations = annotations.prepend(sefAnnotation);
if (overrideAnnotation != null) annotations = annotations.prepend(overrideAnnotation);
JCModifiers modifiers = maker.Modifiers(Flags.PROTECTED | Flags.ABSTRACT, annotations);
Name name = job.toName(SELF_METHOD);
JCExpression returnType = maker.Ident(job.toName(builderGenericName));
returnType = addCheckerFrameworkReturnsReceiver(returnType, maker, job.builderType,
job.checkerFramework);
return maker.MethodDef(modifiers, name, returnType, List.<JCTypeParameter>nil(),
List.<JCVariableDecl>nil(), List.<JCExpression>nil(), null, null);
}
private JCMethodDecl generateSelfMethod(SuperBuilderJob job) {
JavacTreeMaker maker = job.getTreeMaker();
JCAnnotation overrideAnnotation = maker.Annotation(genJavaLangTypeRef(job.builderType,
"Override"), List.<JCExpression>nil());
JCAnnotation sefAnnotation = job.checkerFramework.generatePure() ?
maker.Annotation(genTypeRef(job.builderType, CheckerFrameworkVersion.NAME__PURE),
List.<JCExpression>nil()) : null;
List<JCAnnotation> annsOnMethod = List.nil();
if (sefAnnotation != null) annsOnMethod = annsOnMethod.prepend(sefAnnotation);
annsOnMethod = annsOnMethod.prepend(overrideAnnotation);
JCModifiers modifiers = maker.Modifiers(Flags.PROTECTED, annsOnMethod);
Name name = job.toName(SELF_METHOD);
JCExpression returnType = namePlusTypeParamsToTypeReference(maker, job.builderType.up(),
job.getBuilderClassName(), false, job.typeParams);
returnType = addCheckerFrameworkReturnsReceiver(returnType, maker, job.builderType,
job.checkerFramework);
JCStatement statement = maker.Return(maker.Ident(job.toName("this")));
JCBlock body = maker.Block(0, List.<JCStatement>of(statement));
return maker.MethodDef(modifiers, name, returnType, List.<JCTypeParameter>nil(),
List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
}
private JCMethodDecl generateAbstractBuildMethod(SuperBuilderJob job, boolean override, String
classGenericName) {
JavacTreeMaker maker = job.getTreeMaker();
List<JCAnnotation> annotations = List.nil();
if (override) {
JCAnnotation overrideAnnotation = maker.Annotation(genJavaLangTypeRef(job.builderType,
"Override"), List.<JCExpression>nil());
annotations = List.of(overrideAnnotation);
}
if (job.checkerFramework.generateSideEffectFree()) annotations =
annotations.prepend(maker.Annotation(genTypeRef(job.builderType,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
JCModifiers modifiers = maker.Modifiers(Flags.PUBLIC | Flags.ABSTRACT, annotations);
Name name = job.toName(job.buildMethodName);
JCExpression returnType = maker.Ident(job.toName(classGenericName));
JCVariableDecl recv = HandleBuilder.generateReceiver(job);
JCMethodDecl methodDef;
if (recv != null && maker.hasMethodDefWithRecvParam()) {
methodDef = maker.MethodDefWithRecvParam(modifiers, name, returnType,
List.<JCTypeParameter>nil(), recv, List.<JCVariableDecl>nil(), List.<JCExpression>nil(), null, null);
} else {
```

```
methodDef = maker.MethodDef(modifiers, name, returnType, List.<JCTypeParameter>nil(),
List.<JCVariableDecl>nil(), List.<JCExpression>nil(), null, null);
}
return methodDef;
}
private JCMethodDecl generateBuildMethod(SuperBuilderJob job, List<JCExpression>
thrownExceptions) {
JavacTreeMaker maker = job.getTreeMaker();
JCExpression call;
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
// Use a constructor that only has this builder as parameter.
List<JCExpression> builderArg = List.<JCExpression>of(maker.Ident(job.toName("this")));
call = maker.NewClass(null, List.<JCExpression>nil(), cloneSelfType(job.parentType), builderArg, null);
statements.append(maker.Return(call));
JCBlock body = maker.Block(0, statements.toList());
JCAnnotation overrideAnnotation = maker.Annotation(genJavaLangTypeRef(job.builderType,
"Override"), List.<JCExpression>nil());
List<JCAnnotation> annsOnMethod = List.of(overrideAnnotation);
if (job.checkerFramework.generateSideEffectFree()) annsOnMethod =
annsOnMethod.prepend(maker.Annotation(genTypeRef(job.builderType,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
JCModifiers modifiers = maker.Modifiers(Flags.PUBLIC, annsOnMethod);
JCVariableDecl recv = HandleBuilder.generateReceiver(job);
JCMethodDecl methodDef;
if (recv != null && maker.hasMethodDefWithRecvParam()) {
methodDef = maker.MethodDefWithRecvParam(modifiers, job.toName(job.buildMethodName),
cloneSelfType(job.parentType), List.<JCTypeParameter>nil(), recv, List.<JCVariableDecl>nil(),
thrownExceptions, body, null);
} else {
methodDef = maker.MethodDef(modifiers, job.toName(job.buildMethodName),
cloneSelfType(job.parentType), List.<JCTypeParameter>nil(), List.<JCVariableDecl>nil(),
thrownExceptions, body, null);
}
createRelevantNonNullAnnotation(job.builderType, methodDef);
return methodDef;
}
private JCMethodDecl generateCleanMethod(java.util.List<BuilderFieldData> builderFields, JavacNode
type, JavacNode source) {
JavacTreeMaker maker = type.getTreeMaker();
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
for (BuilderFieldData bfd : builderFields) {
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
bfd.singularData.getSingularizer().appendCleaningCode(bfd.singularData, type, source, statements);
}
}
statements.append(maker.Exec(maker.Assign(maker.Select(maker.Ident(type.toName("this")),
type.toName("$lombokUnclean")), maker.Literal(CTC_BOOLEAN, 0))));
JCBlock body = maker.Block(0, statements.toList());
return maker.MethodDef(maker.Modifiers(Flags.PUBLIC), type.toName("$lombokClean"),
maker.Type(Javac.createVoidType(type.getSymbolTable(), CTC_VOID)),
List.<JCTypeParameter>nil(), List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
}
```

```
private void generateBuilderFields(JavacNode builderType, java.util.List<BuilderFieldData>
builderFields, JavacNode source) {
int len = builderFields.size();
java.util.List<JavacNode> existing = new ArrayList<JavacNode>();
for (JavacNode child : builderType.down()) {
if (child.getKind() == Kind.FIELD) existing.add(child);
}
java.util.List<JCVariableDecl> generated = new ArrayList<JCVariableDecl>();
for (int i = len - 1; i >= 0; i--) {
BuilderFieldData bfd = builderFields.get(i);
if (bfd.singularData != null && bfd.singularData.getSingularizer() != null) {
java.util.List<JavacNode> fields = bfd.singularData.getSingularizer().generateFields(bfd.singularData,
builderType, source);
for (JavacNode field : fields) {
generated.add((JCVariableDecl) field.get());
}
bfd.createdFields.addAll(fields);
} else {
JavacNode field = null, setFlag = null;
for (JavacNode exists : existing) {
Name n = ((JCVariableDecl) exists.get()).name;
if (n.equals(bfd.builderFieldName)) field = exists;
if (n.equals(bfd.nameOfSetFlag)) setFlag = exists;
}
JavacTreeMaker maker = builderType.getTreeMaker();
if (field == null) {
JCModifiers mods = maker.Modifiers(Flags.PRIVATE);
JCVariableDecl newField = maker.VarDef(mods, bfd.builderFieldName, cloneType(maker, bfd.type,
source), null);
field = injectFieldAndMarkGenerated(builderType, newField);
generated.add(newField);
}
if (setFlag == null && bfd.nameOfSetFlag != null) {
JCModifiers mods = maker.Modifiers(Flags.PRIVATE);
JCVariableDecl newField = maker.VarDef(mods, bfd.nameOfSetFlag,
maker.TypeIdent(CTC_BOOLEAN), null);
injectFieldAndMarkGenerated(builderType, newField);
generated.add(newField);
}
bfd.createdFields.add(field);
}
}
for (JCVariableDecl gen : generated) recursiveSetGeneratedBy(gen, source);
}
private void generateSetterMethodsForBuilder(final SuperBuilderJob job, BuilderFieldData fieldNode,
final String builderGenericName, String setterPrefix) {
boolean deprecate = isFieldDeprecated(fieldNode.originalFieldNode);
final JavacTreeMaker maker = job.getTreeMaker();
ExpressionMaker returnTypeMaker = new ExpressionMaker() { @Override public JCExpression make()
{
return maker.Ident(job.toName(builderGenericName));
}};
```

```
StatementMaker returnStatementMaker = new StatementMaker() { @Override public JCStatement
make() {
return maker.Return(maker.Apply(List.<JCExpression>nil(),
maker.Ident(job.toName(SELF_METHOD)), List.<JCExpression>nil()));
}};
if (fieldNode.singularData == null || fieldNode.singularData.getSingularizer() == null) {
generateSimpleSetterMethodForBuilder(job, deprecate, fieldNode.createdFields.get(0),
fieldNode.name, fieldNode.nameOfSetFlag, returnTypeMaker.make(), returnStatementMaker.make(),
fieldNode.annotations, fieldNode.originalFieldNode, setterPrefix);
} else {
fieldNode.singularData.getSingularizer().generateMethods(job.checkerFramework,
fieldNode.singularData, deprecate, job.builderType,
job.sourceNode, true, returnTypeMaker, returnStatementMaker, AccessLevel.PUBLIC);
}
}
private void generateSimpleSetterMethodForBuilder(SuperBuilderJob job, boolean deprecate,
JavacNode fieldNode, Name paramName, Name nameOfSetFlag, JCExpression returnType,
JCStatement returnStatement, List<JCAnnotation> annosOnParam, JavacNode originalFieldNode,
String setterPrefix) {
String setterName = HandlerUtil.buildAccessorName(job.sourceNode, setterPrefix,
paramName.toString());
Name setterName_ = job.builderType.toName(setterName);
for (JavacNode child : job.builderType.down()) {
if (child.getKind() != Kind.METHOD) continue;
JCMethodDecl methodDecl = (JCMethodDecl) child.get();
Name existingName = methodDecl.name;
if (existingName.equals(setterName_) && !isTolerate(fieldNode, methodDecl)) return;
}
JavacTreeMaker maker = fieldNode.getTreeMaker();
List<JCAnnotation> methodAnns =
JavacHandlerUtil.findCopyableToSetterAnnotations(originalFieldNode);
returnType = addCheckerFrameworkReturnsReceiver(returnType, maker, job.builderType,
job.checkerFramework);
JCMethodDecl newMethod = HandleSetter.createSetter(Flags.PUBLIC, deprecate, fieldNode, maker,
setterName, paramName, nameOfSetFlag, returnType, returnStatement, job.sourceNode,
methodAnns, annosOnParam);
if (job.sourceNode.up().getKind() == Kind.METHOD) {
copyJavadocFromParam(originalFieldNode.up(), newMethod, paramName.toString());
} else {
copyJavadoc(originalFieldNode, newMethod, CopyJavadoc.SETTER, true);
}
injectMethod(job.builderType, newMethod);
}
private void addObtainVia(BuilderFieldData bfd, JavacNode node) {
for (JavacNode child : node.down()) {
if (!annotationTypeMatches(ObtainVia.class, child)) continue;
AnnotationValues<ObtainVia> ann = createAnnotation(ObtainVia.class, child);
bfd.obtainVia = ann.getInstance();
bfd.obtainViaNode = child;
deleteAnnotationIfNeccessary(child, ObtainVia.class);
return;
}
}
```

```
/**
* Returns the explicitly requested singular annotation on this node (field
* or parameter), or null if there's no {@code @Singular} annotation on it.
*
* @param node The node (field or method param) to inspect for its name and potential {@code
@Singular} annotation.
* @param setterPrefix the prefix for setter methods
*/
private SingularData getSingularData(JavacNode node, String setterPrefix) {
for (JavacNode child : node.down()) {
if (!annotationTypeMatches(Singular.class, child)) continue;
Name pluralName = node.getKind() == Kind.FIELD ? removePrefixFromField(node) : ((JCVariableDecl)
node.get()).name;
AnnotationValues<Singular> ann = createAnnotation(Singular.class, child);
Singular singularInstance = ann.getInstance();
deleteAnnotationIfNeccessary(child, Singular.class);
String explicitSingular = singularInstance.value();
if (explicitSingular.isEmpty()) {
if (Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKeys.SINGULAR_AUTO))) {
node.addError("The singular must be specified explicitly (e.g. @Singular(\"task\")) because auto
singularization is disabled.");
explicitSingular = pluralName.toString();
} else {
explicitSingular = autoSingularize(pluralName.toString());
if (explicitSingular == null) {
node.addError("Can't singularize this name; please specify the singular explicitly (i.e.
@Singular(\"sheep\"))");
explicitSingular = pluralName.toString();
}
}
}
Name singularName = node.toName(explicitSingular);
JCExpression type = null;
if (node.get() instanceof JCVariableDecl) type = ((JCVariableDecl) node.get()).vartype;
String name = null;
List<JCExpression> typeArgs = List.nil();
if (type instanceof JCTypeApply) {
typeArgs = ((JCTypeApply) type).arguments;
type = ((JCTypeApply) type).clazz;
}
name = type.toString();
String targetFqn = JavacSingularsRecipes.get().toQualified(name);
JavacSingularizer singularizer = JavacSingularsRecipes.get().getSingularizer(targetFqn, node);
if (singularizer == null) {
node.addError("Lombok does not know how to create the singular-form builder methods for type '" +
name + "'; they won't be generated.");
return null;
}
return new SingularData(child, singularName, pluralName, typeArgs, targetFqn, singularizer,
singularInstance.ignoreNullCollections(), setterPrefix);
}
return null;
}
```

```
private java.util.HashSet<String> gatherUsedTypeNames(List<JCTypeParameter> typeParams,
JCClassDecl td) {
java.util.HashSet<String> usedNames = new HashSet<String>();
// 1. Add type parameter names.
for (JCTypeParameter typeParam : typeParams)
usedNames.add(typeParam.getName().toString());
// 2. Add class name.
usedNames.add(td.name.toString());
// 3. Add used type names.
for (JCTree member : td.getMembers()) {
if (member.getKind() == com.sun.source.tree.Tree.Kind.VARIABLE && member instanceof
JCVariableDecl) {
JCTree type = ((JCVariableDecl)member).getType();
if (type instanceof JCIdent)
usedNames.add(((JCIdent)type).getName().toString());
}
}
// 4. Add extends and implements clauses.
addFirstToken(usedNames, Javac.getExtendsClause(td));
for (JCExpression impl : td.getImplementsClause()) {
addFirstToken(usedNames, impl);
}
return usedNames;
}
private void addFirstToken(java.util.Set<String> usedNames, JCTree type) {
if (type == null)
return;
if (type instanceof JCTypeApply) {
type = ((JCTypeApply)type).clazz;
}
while (type instanceof JCFieldAccess && ((JCFieldAccess)type).selected != null) {
// Add the first token, because only that can collide.
type = ((JCFieldAccess)type).selected;
}
usedNames.add(type.toString());
}
private String generateNonclashingNameFor(String classGenericName, java.util.HashSet<String>
typeParamStrings) {
if (!typeParamStrings.contains(classGenericName)) return classGenericName;
int counter = 2;
while (typeParamStrings.contains(classGenericName + counter)) counter++;
return classGenericName + counter;
}
private JavacNode findInnerClass(JavacNode parent, String name) {
for (JavacNode child : parent.down()) {
if (child.getKind() != Kind.TYPE) continue;
JCClassDecl td = (JCClassDecl) child.get();
if (td.name.contentEquals(name)) return child;
}
return null;
}
private ListBuffer<JCExpression> getTypeParamExpressions(List<? extends JCTree> typeParams,
JavacTreeMaker maker, JavacNode source) {
```

```
ListBuffer<JCExpression> typeParamsForBuilderParameter = new ListBuffer<JCExpression>();
for (JCTree typeParam : typeParams) {
if (typeParam instanceof JCTypeParameter) {
typeParamsForBuilderParameter.append(maker.Ident(((JCTypeParameter) typeParam).getName()));
} else if (typeParam instanceof JCIdent) {
typeParamsForBuilderParameter.append(maker.Ident(((JCIdent) typeParam).getName()));
} else if (typeParam instanceof JCFieldAccess) {
typeParamsForBuilderParameter.append(copySelect(maker, (JCFieldAccess) typeParam));
} else if (typeParam instanceof JCTypeApply) {
typeParamsForBuilderParameter.append(cloneType(maker, (JCTypeApply) typeParam, source));
} else if (typeParam instanceof JCArrayTypeTree) {
typeParamsForBuilderParameter.append(cloneType(maker, (JCArrayTypeTree) typeParam, source));
} else if (JCAnnotatedTypeReflect.is(typeParam)) {
typeParamsForBuilderParameter.append(cloneType(maker, (JCExpression) typeParam, source));
}
}
return typeParamsForBuilderParameter;
}
private JCExpression copySelect(JavacTreeMaker maker, JCFieldAccess typeParam) {
java.util.List<Name> chainNames = new ArrayList<Name>();
JCExpression expression = typeParam;
while (expression != null) {
if (expression instanceof JCFieldAccess) {
chainNames.add(((JCFieldAccess) expression).getIdentifier());
expression = ((JCFieldAccess) expression).getExpression();
} else if (expression instanceof JCIdent) {
chainNames.add(((JCIdent) expression).getName());
expression = null;
}
}
Collections.reverse(chainNames);
JCExpression typeParameter = null;
for (Name name : chainNames) {
if (typeParameter == null) {
typeParameter = maker.Ident(name);
} else {
typeParameter = maker.Select(typeParameter, name);
}
}
return typeParameter;
}
/**
* Checks if there is a manual constructor in the given type with a single parameter (builder).
*/
private boolean constructorExists(JavacNode type, String builderClassName) {
if (type != null && type.get() instanceof JCClassDecl) {
for (JCTree def : ((JCClassDecl)type.get()).defs) {
if (def instanceof JCMethodDecl) {
JCMethodDecl md = (JCMethodDecl) def;
String name = md.name.toString();
boolean matches = name.equals("<init>");
if (isTolerate(type, md))
continue;
```

```
if (matches && md.params != null && md.params.length() == 1) {
// Cannot use typeMatches() here, because the parameter could be fully-qualified, partially-qualified, or not qualified.
// A string-compare of the last part should work. If it's a false-positive, users could still @Tolerate it.
String typeName = md.params.get(0).getType().toString();
int lastIndexOfDot = typeName.lastIndexOf('.');
if (lastIndexOfDot >= 0) {
typeName = typeName.substring(lastIndexOfDot+1);
}
if ((builderClassName+"<?, ?>").equals(typeName))
return true;
}
}
}
}
return false;
}
}
```

lombok/javac/handlers/HandleSuperBuilderRemove.java

```
/*
* Copyright (C) 2020-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import lombok.core.AlreadyHandledAnnotations;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.experimental.SuperBuilder;
```

```
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.spi.Provides;
@Provides
@HandlerPriority(32768)
@AlreadyHandledAnnotations
public class HandleSuperBuilderRemove extends JavacAnnotationHandler<SuperBuilder> {
@Override public void handle(AnnotationValues<SuperBuilder> annotation, JCAnnotation ast,
JavacNode annotationNode) {
deleteAnnotationIfNeccessary(annotationNode, SuperBuilder.class);
}
}
```

lombok/javac/handlers/HandleSynchronized.java

```
/*
* Copyright (C) 2009-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import lombok.ConfigurationKeys;
import lombok.Synchronized;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil.MemberExistsResult;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
```

```
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCNewArray;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
/**
* Handles the {@code lombok.Synchronized} annotation for javac.
*/
@Provides
@HandlerPriority(value = 1024) // 2^10; @NonNull must have run first, so that we wrap around the
statements generated by it.
public class HandleSynchronized extends JavacAnnotationHandler<Synchronized> {
private static final String INSTANCE_LOCK_NAME = "$lock";
private static final String STATIC_LOCK_NAME = "$LOCK";
@Override public void handle(AnnotationValues<Synchronized> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.SYNCHRONIZED_FLAG_USAGE,
"@Synchronized");
if (inNetbeansEditor(annotationNode)) return;
deleteAnnotationIfNeccessary(annotationNode, Synchronized.class);
JavacNode methodNode = annotationNode.up();
if (methodNode == null || methodNode.getKind() != Kind.METHOD || !(methodNode.get() instanceof
JCMethodDecl)) {
annotationNode.addError("@Synchronized is legal only on methods.");
return;
}
JCMethodDecl method = (JCMethodDecl)methodNode.get();
if ((method.mods.flags & Flags.ABSTRACT) != 0) {
annotationNode.addError("@Synchronized is legal only on concrete methods.");
return;
}
JavacNode typeNode = upToTypeNode(annotationNode);
if (!isClassOrEnum(typeNode)) {
annotationNode.addError("@Synchronized is legal only on methods in classes and enums.");
return;
}
boolean isStatic = (method.mods.flags & Flags.STATIC) != 0;
String lockName = annotation.getInstance().value();
boolean autoMake = false;
if (lockName.length() == 0) {
autoMake = true;
lockName = isStatic ? STATIC_LOCK_NAME : INSTANCE_LOCK_NAME;
}
JavacTreeMaker maker = methodNode.getTreeMaker().at(ast.pos);
MemberExistsResult exists = MemberExistsResult.NOT_EXISTS;
if (typeNode != null && typeNode.get() instanceof JCClassDecl) {
for (JCTree def : ((JCClassDecl) typeNode.get()).defs) {
if (def instanceof JCVariableDecl) {
```

```
if (((JCVariableDecl) def).name.contentEquals(lockName)) {
exists = getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
boolean st = ((((JCVariableDecl) def).mods.flags) & Flags.STATIC) != 0;
if (isStatic && !st) {
annotationNode.addError("The field " + lockName + " is non-static and this cannot be used on this static
method");
return;
}
isStatic = st;
}
}
}
}
if (exists == MemberExistsResult.NOT_EXISTS) {
if (!autoMake) {
annotationNode.addError("The field " + lockName + " does not exist.");
return;
}
JCExpression objectType = genJavaLangTypeRef(methodNode, ast.pos, "Object");
//We use 'new Object[0];' because unlike 'new Object();', empty arrays *ARE* serializable!
JCNewArray newObjectArray = maker.NewArray(genJavaLangTypeRef(methodNode, ast.pos,
"Object"),
List.<JCExpression>of(maker.Literal(CTC_INT, 0)), null);
JCVariableDecl fieldDecl = recursiveSetGeneratedBy(maker.VarDef(
maker.Modifiers(Flags.PRIVATE | Flags.FINAL | (isStatic ? Flags.STATIC : 0)),
methodNode.toName(lockName), objectType, newObjectArray), annotationNode);
injectFieldAndMarkGenerated(methodNode.up(), fieldDecl);
}
if (method.body == null) return;
JCExpression lockNode;
if (isStatic) {
lockNode = namePlusTypeParamsToTypeReference(maker, typeNode,
methodNode.toName(lockName), false, List.<JCTypeParameter>nil());
} else {
lockNode = maker.Select(maker.Ident(methodNode.toName("this")), methodNode.toName(lockName));
}
recursiveSetGeneratedBy(lockNode, annotationNode);
method.body = setGeneratedBy(maker.Block(0,
List.<JCStatement>of(setGeneratedBy(maker.Synchronized(lockNode, method.body),
annotationNode))), annotationNode);
methodNode.rebuild();
}
}
```

lombok/javac/handlers/HandleToString.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
```

* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import static lombok.javac.Javac.*;
import java.util.Collection;
import lombok.ConfigurationKeys;
import lombok.ToString;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CallSuperType;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.AST.Kind;
import lombok.core.handlers.InclusionExclusionUtils;
import lombok.core.handlers.InclusionExclusionUtils.Included;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
/**
* Handles the {@code ToString} annotation for javac.
*/
@Provides
public class HandleToString extends JavacAnnotationHandler<ToString> {
```

```
@Override public void handle(AnnotationValues<ToString> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.TO_STRING_FLAG_USAGE, "@ToString");
deleteAnnotationIfNeccessary(annotationNode, ToString.class);
ToString ann = annotation.getInstance();
boolean onlyExplicitlyIncluded = annotationNode.getAst().getBooleanAnnotationValue(annotation,
"onlyExplicitlyIncluded", ConfigurationKeys.TO_STRING_ONLY_EXPLICITLY_INCLUDED);
java.util.List<Included<JavacNode, ToString.Include>> members =
InclusionExclusionUtils.handleToStringMarking(annotationNode.up(), onlyExplicitlyIncluded,
annotation, annotationNode);
if (members == null) return;
Boolean callSuper = ann.callSuper();
if (!annotation.isExplicit("callSuper")) callSuper = null;
Boolean doNotUseGettersConfiguration = annotationNode.getAst().readConfiguration(ConfigurationKe
ys.TO_STRING_DO_NOT_USE_GETTERS);
boolean doNotUseGetters = annotation.isExplicit("doNotUseGetters") || doNotUseGettersConfiguration
== null ? ann.doNotUseGetters() : doNotUseGettersConfiguration;
FieldAccess fieldAccess = doNotUseGetters ? FieldAccess.PREFER_FIELD : FieldAccess.GETTER;
boolean includeFieldNames = annotationNode.getAst().getBooleanAnnotationValue(annotation,
"includeFieldNames", ConfigurationKeys.TO_STRING_INCLUDE_FIELD_NAMES);
generateToString(annotationNode.up(), annotationNode, members, includeFieldNames, callSuper,
true, fieldAccess);
}
public void generateToStringForType(JavacNode typeNode, JavacNode errorNode) {
if (hasAnnotation(ToString.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
AnnotationValues<ToString> anno = AnnotationValues.of(ToString.class);
boolean includeFieldNames = typeNode.getAst().getBooleanAnnotationValue(anno,
"includeFieldNames", ConfigurationKeys.TO_STRING_INCLUDE_FIELD_NAMES);
boolean onlyExplicitlyIncluded = typeNode.getAst().getBooleanAnnotationValue(anno,
"onlyExplicitlyIncluded", ConfigurationKeys.TO_STRING_ONLY_EXPLICITLY_INCLUDED);
Boolean doNotUseGettersConfiguration =
typeNode.getAst().readConfiguration(ConfigurationKeys.TO_STRING_DO_NOT_USE_GETTERS);
FieldAccess access = doNotUseGettersConfiguration == null || !doNotUseGettersConfiguration ?
FieldAccess.GETTER : FieldAccess.PREFER_FIELD;
java.util.List<Included<JavacNode, ToString.Include>> members =
InclusionExclusionUtils.handleToStringMarking(typeNode, onlyExplicitlyIncluded, null, null);
generateToString(typeNode, errorNode, members, includeFieldNames, null, false, access);
}
public void generateToString(JavacNode typeNode, JavacNode source,
java.util.List<Included<JavacNode, ToString.Include>> members,
boolean includeFieldNames, Boolean callSuper, boolean whineIfExists, FieldAccess fieldAccess) {
if (!isClassOrEnum(typeNode)) {
source.addError("@ToString is only supported on a class or enum.");
return;
}
switch (methodExists("toString", typeNode, 0)) {
case NOT_EXISTS:
if (callSuper == null) {
if (isDirectDescendantOfObject(typeNode)) {
callSuper = false;
```

```
} else {
CallSuperType cst =
typeNode.getAst().readConfiguration(ConfigurationKeys.TO_STRING_CALL_SUPER);
if (cst == null) cst = CallSuperType.SKIP;
switch (cst) {
default:
case SKIP:
callSuper = false;
break;
case WARN:
source.addWarning("Generating toString implementation but without a call to superclass, even though
this class does not extend java.lang.Object. If this is intentional, add '@ToString(callSuper=false)' to
your type.");
callSuper = false;
break;
case CALL:
callSuper = true;
break;
}
}
}
JCMethodDecl method = createToString(typeNode, members, includeFieldNames, callSuper,
fieldAccess, source);
injectMethod(typeNode, method);
break;
case EXISTS_BY_LOMBOK:
break;
default:
case EXISTS_BY_USER:
if (whineIfExists) {
source.addWarning("Not generating toString(): A method with that name already exists");
}
break;
}
}
static JCMethodDecl createToString(JavacNode typeNode, Collection<Included<JavacNode,
ToString.Include>> members,
boolean includeNames, boolean callSuper, FieldAccess fieldAccess, JavacNode source) {
JavacTreeMaker maker = typeNode.getTreeMaker();
JCAnnotation overrideAnnotation = maker.Annotation(genJavaLangTypeRef(typeNode, "Override"),
List.<JCExpression>nil());
List<JCAnnotation> annsOnMethod = List.of(overrideAnnotation);
if (getCheckerFrameworkVersion(typeNode).generateSideEffectFree()) annsOnMethod =
annsOnMethod.prepend(maker.Annotation(genTypeRef(typeNode,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
JCModifiers mods = maker.Modifiers(Flags.PUBLIC, annsOnMethod);
JCExpression returnType = genJavaLangTypeRef(typeNode, "String");
boolean first = true;
String typeName = getTypeName(typeNode);
boolean isEnum = typeNode.isEnumType();
String infix = ", ";
String suffix = ")";
String prefix;
```

```
if (callSuper) {
prefix = "(super=";
} else if (members.isEmpty()) {
prefix = isEnum ? "" : "()";
} else if (includeNames) {
Included<JavacNode, ToString.Include> firstMember = members.iterator().next();
String name = firstMember.getInc() == null ? "" : firstMember.getInc().name();
if (name.isEmpty()) name = firstMember.getNode().getName();
prefix = "(" + name + "=";
} else {
prefix = "(";
}
JCExpression current;
if (!isEnum) {
current = maker.Literal(typeName + prefix);
} else {
current = maker.Binary(CTC_PLUS, maker.Literal(typeName + "."),
maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(typeNode.toName("this")), typeNode.toName("name")),
List.<JCExpression>nil()));
if (!prefix.isEmpty()) current = maker.Binary(CTC_PLUS, current, maker.Literal(prefix));
}
if (callSuper) {
JCMethodInvocation callToSuper = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(typeNode.toName("super")), typeNode.toName("toString")),
List.<JCExpression>nil());
current = maker.Binary(CTC_PLUS, current, callToSuper);
first = false;
}
for (Included<JavacNode, ToString.Include> member : members) {
JCExpression expr;
JCExpression memberAccessor;
JavacNode memberNode = member.getNode();
if (memberNode.getKind() == Kind.METHOD) {
memberAccessor = createMethodAccessor(maker, memberNode);
} else {
memberAccessor = createFieldAccessor(maker, memberNode, fieldAccess);
}
JCExpression memberType = removeTypeUseAnnotations(getFieldType(memberNode, fieldAccess));
// The distinction between primitive and object will be useful if we ever add a 'hideNulls' option.
@SuppressWarnings("unused")
boolean fieldIsPrimitive = memberType instanceof JCPrimitiveTypeTree;
boolean fieldIsPrimitiveArray = memberType instanceof JCArrayTypeTree && ((JCArrayTypeTree)
memberType).elemtype instanceof JCPrimitiveTypeTree;
boolean fieldIsObjectArray = !fieldIsPrimitiveArray && memberType instanceof JCArrayTypeTree;
if (fieldIsPrimitiveArray || fieldIsObjectArray) {
JCExpression tsMethod = chainDots(typeNode, "java", "util", "Arrays", fieldIsObjectArray ?
"deepToString" : "toString");
expr = maker.Apply(List.<JCExpression>nil(), tsMethod, List.<JCExpression>of(memberAccessor));
} else expr = memberAccessor;
if (first) {
current = maker.Binary(CTC_PLUS, current, expr);
first = false;
```

```
continue;
}
if (includeNames) {
String n = member.getInc() == null ? "" : member.getInc().name();
if (n.isEmpty()) n = memberNode.getName();
current = maker.Binary(CTC_PLUS, current, maker.Literal(infix + n + "="));
} else {
current = maker.Binary(CTC_PLUS, current, maker.Literal(infix));
}
current = maker.Binary(CTC_PLUS, current, expr);
}
if (!first) current = maker.Binary(CTC_PLUS, current, maker.Literal(suffix));
JCStatement returnStatement = maker.Return(current);
JCBlock body = maker.Block(0, List.of(returnStatement));
JCMethodDecl methodDef = maker.MethodDef(mods, typeNode.toName("toString"), returnType,
List.<JCTypeParameter>nil(), List.<JCVariableDecl>nil(), List.<JCExpression>nil(), body, null);
createRelevantNonNullAnnotation(typeNode, methodDef);
return recursiveSetGeneratedBy(methodDef, source);
}
public static String getTypeName(JavacNode typeNode) {
String typeName = typeNode.getName();
JavacNode upType = typeNode.up();
while (upType.getKind() == Kind.TYPE && !upType.getName().isEmpty()) {
typeName = upType.getName() + "." + typeName;
upType = upType.up();
}
return typeName;
}
}
```

lombok/javac/handlers/HandleUtilityClass.java


```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
```

```
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.handleExperimentalFlagUsage;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.code.Symbol.ClassSymbol;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.code.Type.ClassType;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.Name;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.experimental.UtilityClass;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.spi.Provides;
/**
* Handles the {@code @UtilityClass} annotation for javac.
*/
@HandlerPriority(-4096) //-2^12; to ensure @FieldDefaults picks up on the 'static' we set here.
@Provides
public class HandleUtilityClass extends JavacAnnotationHandler<UtilityClass> {
@Override public void handle(AnnotationValues<UtilityClass> annotation, JCAnnotation ast,
JavacNode annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.UTILITY_CLASS_FLAG_USAGE,
"@UtilityClass");
deleteAnnotationIfNeccessary(annotationNode, UtilityClass.class);
JavacNode typeNode = annotationNode.up();
if (!checkLegality(typeNode, annotationNode)) return;
changeModifiersAndGenerateConstructor(annotationNode.up(), annotationNode);
}
private static boolean checkLegality(JavacNode typeNode, JavacNode errorNode) {
if (!isClass(typeNode)) {
errorNode.addError("@UtilityClass is only supported on a class.");
return false;
}
// It might be an inner class. This is okay, but only if it is / can be a static inner class. Thus, all of its
parents have to be static inner classes until the top-level.
JavacNode typeWalk = typeNode;
while (true) {
```

```
typeWalk = typeWalk.up();
switch (typeWalk.getKind()) {
case TYPE:
JCClassDecl typeDef = (JCClassDecl) typeWalk.get();
if ((typeDef.mods.flags & (Flags.STATIC | Flags.ANNOTATION | Flags.ENUM | Flags.INTERFACE)) !=
0) continue;
if (typeWalk.up().getKind() == Kind.COMPILATION_UNIT) return true;
errorNode.addError("@UtilityClass automatically makes the class static, however, this class cannot be
made static.");
return false;
case COMPILATION_UNIT:
return true;
default:
errorNode.addError("@UtilityClass cannot be placed on a method local or anonymous inner class, or
any class nested in such a class.");
return false;
}
}
}
private void changeModifiersAndGenerateConstructor(JavacNode typeNode, JavacNode errorNode) {
JCClassDecl classDecl = (JCClassDecl) typeNode.get();
boolean makeConstructor = true;
classDecl.mods.flags |= Flags.FINAL;
boolean markStatic = true;
if (typeNode.up().getKind() == Kind.COMPILATION_UNIT) markStatic = false;
if (markStatic && typeNode.up().getKind() == Kind.TYPE) {
JCClassDecl typeDecl = (JCClassDecl) typeNode.up().get();
if ((typeDecl.mods.flags & (Flags.INTERFACE | Flags.ANNOTATION)) != 0) markStatic = false;
}
if (markStatic) classDecl.mods.flags |= Flags.STATIC;
for (JavacNode element : typeNode.down()) {
if (element.getKind() == Kind.FIELD) {
JCVariableDecl fieldDecl = (JCVariableDecl) element.get();
fieldDecl.mods.flags |= Flags.STATIC;
} else if (element.getKind() == Kind.METHOD) {
JCMethodDecl methodDecl = (JCMethodDecl) element.get();
if (methodDecl.name.contentEquals("<init>")) {
if (getGeneratedBy(methodDecl) == null && (methodDecl.mods.flags & Flags.GENERATEDCONSTR)
== 0) {
element.addError("@UtilityClasses cannot have declared constructors.");
makeConstructor = false;
continue;
}
}
methodDecl.mods.flags |= Flags.STATIC;
} else if (element.getKind() == Kind.TYPE) {
JCClassDecl innerClassDecl = (JCClassDecl) element.get();
innerClassDecl.mods.flags |= Flags.STATIC;
ClassSymbol innerClassSymbol = innerClassDecl.sym;
if (innerClassSymbol != null) {
if (innerClassSymbol.type instanceof ClassType) {
((ClassType) innerClassSymbol.type).setEnclosingType(Type.noType);
}
```

```
innerClassSymbol.erasure_field = null;
}
}
}
if (makeConstructor) createPrivateDefaultConstructor(typeNode);
}
private void createPrivateDefaultConstructor(JavacNode typeNode) {
JavacTreeMaker maker = typeNode.getTreeMaker();
JCModifiers mods = maker.Modifiers(Flags.PRIVATE, List.<JCAnnotation>nil());
Name name = typeNode.toName("<init>");
JCBlock block = maker.Block(0L, createThrowStatement(typeNode, maker));
JCMethodDecl methodDef = maker.MethodDef(mods, name, null, List.<JCTypeParameter>nil(),
List.<JCVariableDecl>nil(), List.<JCExpression>nil(), block, null);
JCMethodDecl constructor = recursiveSetGeneratedBy(methodDef, typeNode);
JavacHandlerUtil.injectMethod(typeNode, constructor);
}
private List<JCStatement> createThrowStatement(JavacNode typeNode, JavacTreeMaker maker) {
JCExpression exceptionType = genJavaLangTypeRef(typeNode, "UnsupportedOperationException");
List<JCExpression> jceBlank = List.nil();
JCExpression message = maker.Literal("This is a utility class and cannot be instantiated");
JCExpression exceptionInstance = maker.NewClass(null, jceBlank, exceptionType, List.of(message),
null);
JCStatement throwStatement = maker.Throw(exceptionInstance);
return List.of(throwStatement);
}
}
```

lombok/javac/handlers/HandleVal.java


```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.javac.handlers;
```

```
import static lombok.core.handlers.HandlerUtil.handleFlagUsage;
import static lombok.javac.handlers.HandleDelegate.HANDLE_DELEGATE_PRIORITY;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.lang.reflect.Field;
import lombok.ConfigurationKeys;
import lombok.val;
import lombok.var;
import lombok.core.HandlerPriority;
import lombok.javac.JavacASTAdapter;
import lombok.javac.JavacASTVisitor;
import lombok.javac.JavacNode;
import lombok.javac.JavacResolution;
import lombok.javac.ResolutionResetNeeded;
import lombok.permit.Permit;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.code.Symtab;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCEnhancedForLoop;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCForLoop;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCNewArray;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
@Provides(JavacASTVisitor.class)
@HandlerPriority(HANDLE_DELEGATE_PRIORITY + 100) // run slightly after HandleDelegate;
resolution needs to work, so if the RHS expression is i.e. a call to a generated getter, we have to run
after that getter has been generated.
@ResolutionResetNeeded
public class HandleVal extends JavacASTAdapter {
private static boolean eq(String typeTreeToString, String key) {
return typeTreeToString.equals(key) || typeTreeToString.equals("lombok." + key) ||
typeTreeToString.equals("lombok.experimental." + key);
}
@SuppressWarnings("deprecation") @Override
public void endVisitLocal(JavacNode localNode, JCVariableDecl local) {
JCTree typeTree = local.vartype;
if (typeTree == null) return;
String typeTreeToString = typeTree.toString();
JavacNode typeNode = localNode.getNodeFor(typeTree);
if (!(eq(typeTreeToString, "val") || eq(typeTreeToString, "var"))) return;
boolean isVal = typeMatches(val.class, localNode, typeTree);
boolean isVar = typeMatches(var.class, localNode, typeTree);
if (!(isVal || isVar)) return;
if (isVal) handleFlagUsage(localNode, ConfigurationKeys.VAL_FLAG_USAGE, "val");
if (isVar) handleFlagUsage(localNode, ConfigurationKeys.VAR_FLAG_USAGE, "var");
JCTree parentRaw = localNode.directUp().get();
if (isVal && parentRaw instanceof JCForLoop) {
localNode.addError("'val' is not allowed in old-style for loops");
return;
```

```
}
if (parentRaw instanceof JCForLoop && ((JCForLoop) parentRaw).getInitializer().size() > 1) {
localNode.addError("'var' is not allowed in old-style for loops if there is more than 1 initializer");
return;
}
JCExpression rhsOfEnhancedForLoop = null;
if (local.init == null) {
if (parentRaw instanceof JCEnhancedForLoop) {
JCEnhancedForLoop efl = (JCEnhancedForLoop) parentRaw;
JCTree var = EnhancedForLoopReflect.getVarOrRecordPattern(efl);
if (var == local) rhsOfEnhancedForLoop = efl.expr;
}
}
final String annotation = typeTreeToString;
if (rhsOfEnhancedForLoop == null && local.init == null) {
localNode.addError("'" + annotation + "' on a local variable requires an initializer expression");
return;
}
if (local.init instanceof JCNewArray && ((JCNewArray)local.init).elemtype == null) {
localNode.addError("'" + annotation + "' is not compatible with array initializer expressions. Use the full
form (new int[] { ... } instead of just { ... })");
return;
}
if (localNode.shouldDeleteLombokAnnotations()) {
JavacHandlerUtil.deleteImportFromCompilationUnit(localNode, val.class.getName());
JavacHandlerUtil.deleteImportFromCompilationUnit(localNode,
lombok.experimental.var.class.getName());
JavacHandlerUtil.deleteImportFromCompilationUnit(localNode, var.class.getName());
}
if (isVal) local.mods.flags |= Flags.FINAL;
if (!localNode.shouldDeleteLombokAnnotations()) {
JCAnnotation valAnnotation =
recursiveSetGeneratedBy(localNode.getTreeMaker().Annotation(local.vartype,
List.<JCExpression>nil()), typeNode);
local.mods.annotations = local.mods.annotations == null ? List.of(valAnnotation) :
local.mods.annotations.append(valAnnotation);
}
if (localNode.getSourceVersion() >= 10) {
local.vartype = null;
localNode.getAst().setChanged();
return;
}
if (JavacResolution.platformHasTargetTyping()) {
local.vartype =
localNode.getAst().getTreeMaker().Ident(localNode.getAst().toName("___Lombok_VAL_Attrib__"));
} else {
local.vartype = JavacResolution.createJavaLangObject(localNode.getAst());
}
Type type;
try {
if (rhsOfEnhancedForLoop == null) {
if (local.init.type == null) {
if (isVar && local.init instanceof JCLiteral && ((JCLiteral) local.init).value == null) {
```

```
localNode.addError("variable initializer is 'null'");
}
JavacResolution resolver = new JavacResolution(localNode.getContext());
try {
type = ((JCExpression) resolver.resolveMethodMember(localNode).get(local.init)).type;
} catch (RuntimeException e) {
System.err.println("Exception while resolving: " + localNode + "(" + localNode.getFileName() + ")");
throw e;
}
} else {
type = local.init.type;
if (type.isErroneous()) {
try {
JavacResolution resolver = new JavacResolution(localNode.getContext());
local.type = Symtab.instance(localNode.getContext()).unknownType;
type = ((JCExpression) resolver.resolveMethodMember(localNode).get(local.init)).type;
} catch (RuntimeException e) {
System.err.println("Exception while resolving: " + localNode + "(" + localNode.getFileName() + ")");
throw e;
}
}
}
} else {
if (rhsOfEnhancedForLoop.type == null) {
JavacResolution resolver = new JavacResolution(localNode.getContext());
type = ((JCExpression)
resolver.resolveMethodMember(localNode.directUp()).get(rhsOfEnhancedForLoop)).type;
} else {
type = rhsOfEnhancedForLoop.type;
}
}
try {
JCExpression replacement;
if (rhsOfEnhancedForLoop != null) {
Type componentType = JavacResolution.ifTypeIsIterableToComponent(type, localNode.getAst());
if (componentType == null) replacement = JavacResolution.createJavaLangObject(localNode.getAst());
else replacement = JavacResolution.typeToJCTree(componentType, localNode.getAst(), false);
} else {
replacement = JavacResolution.typeToJCTree(type, localNode.getAst(), false);
}
if (replacement != null) {
local.vartype = replacement;
} else {
local.vartype = JavacResolution.createJavaLangObject(localNode.getAst());
}
localNode.getAst().setChanged();
} catch (JavacResolution.TypeNotConvertibleException e) {
localNode.addError("Cannot use '" + annotation + "' here because initializer expression does not have a
representable type: " + e.getMessage());
local.vartype = JavacResolution.createJavaLangObject(localNode.getAst());
}
} catch (RuntimeException e) {
local.vartype = JavacResolution.createJavaLangObject(localNode.getAst());
```

```
throw e;
} finally {
recursiveSetGeneratedBy(local.vartype, typeNode);
}
}
private static class EnhancedForLoopReflect {
private static final Field varOrRecordPattern = Permit.permissiveGetField(JCEnhancedForLoop.class,
"varOrRecordPattern");
private static JCTree getVarOrRecordPattern(JCEnhancedForLoop loop) {
if (varOrRecordPattern == null) {
return loop.var;
}
try {
return (JCTree) varOrRecordPattern.get(loop);
} catch (Exception ignore) {}
return null;
}
}
}
```

lombok/javac/handlers/HandleValue.java

```
/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AnnotationValues;
import lombok.core.HandlerPriority;
import lombok.experimental.NonFinal;
```

```
import lombok.Value;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.handlers.HandleConstructor.SkipIfConstructorExists;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.util.List;
/**
* Handles the {@code lombok.Value} annotation for javac.
*/
@Provides
@HandlerPriority(-512) //-2^9; to ensure @EqualsAndHashCode and such pick up on this handler
making the class final and messing with the fields' access levels, run earlier.
public class HandleValue extends JavacAnnotationHandler<Value> {
private HandleFieldDefaults handleFieldDefaults = new HandleFieldDefaults();
private HandleConstructor handleConstructor = new HandleConstructor();
private HandleGetter handleGetter = new HandleGetter();
private HandleEqualsAndHashCode handleEqualsAndHashCode = new
HandleEqualsAndHashCode();
private HandleToString handleToString = new HandleToString();
@Override public void handle(AnnotationValues<Value> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.VALUE_FLAG_USAGE, "@Value");
deleteAnnotationIfNeccessary(annotationNode, Value.class, "lombok.experimental.Value");
JavacNode typeNode = annotationNode.up();
boolean notAClass = !isClass(typeNode);
if (notAClass) {
annotationNode.addError("@Value is only supported on a class.");
return;
}
String staticConstructorName = annotation.getInstance().staticConstructor();
if (!hasAnnotationAndDeleteIfNeccessary(NonFinal.class, typeNode)) {
JCModifiers jcm = ((JCClassDecl) typeNode.get()).mods;
if ((jcm.flags & Flags.FINAL) == 0) {
jcm.flags |= Flags.FINAL;
}
}
handleFieldDefaults.generateFieldDefaultsForType(typeNode, annotationNode,
AccessLevel.PRIVATE, true, true);
handleConstructor.generateAllArgsConstructor(typeNode, AccessLevel.PUBLIC,
staticConstructorName, SkipIfConstructorExists.YES, annotationNode);
handleConstructor.generateExtraNoArgsConstructor(typeNode, annotationNode);
handleGetter.generateGetterForType(typeNode, annotationNode, AccessLevel.PUBLIC, true,
List.<JCAnnotation>nil());
handleEqualsAndHashCode.generateEqualsAndHashCodeForType(typeNode, annotationNode);
handleToString.generateToStringForType(typeNode, annotationNode);
}
}
```

lombok/javac/handlers/HandleWith.java

```
/*
* Copyright (C) 2012-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.Collection;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.With;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.experimental.Accessors;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil.CopyJavadoc;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCConditional;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCNewClass;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
```

```
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
/**
* Handles the {@code lombok.With} annotation for javac.
*/
@Provides
public class HandleWith extends JavacAnnotationHandler<With> {
public void generateWithForType(JavacNode typeNode, JavacNode errorNode, AccessLevel level,
boolean checkForTypeLevelWith) {
if (checkForTypeLevelWith) {
if (hasAnnotation(With.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
}
JCClassDecl typeDecl = null;
if (typeNode.get() instanceof JCClassDecl) typeDecl = (JCClassDecl) typeNode.get();
long modifiers = typeDecl == null ? 0 : typeDecl.mods.flags;
boolean notAClass = (modifiers & (Flags.INTERFACE | Flags.ANNOTATION | Flags.ENUM)) != 0;
if (typeDecl == null || notAClass) {
errorNode.addError("@With is only supported on a class or a field.");
return;
}
for (JavacNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
//Skip fields that start with $
if (fieldDecl.name.toString().startsWith("$")) continue;
//Skip static fields.
if ((fieldDecl.mods.flags & Flags.STATIC) != 0) continue;
//Skip final initialized fields.
if ((fieldDecl.mods.flags & Flags.FINAL) != 0 && fieldDecl.init != null) continue;
generateWithForField(field, errorNode.get(), level);
}
}
/**
* Generates a with on the stated field.
*
* Used by {@link HandleValue}.
*
* The difference between this call and the handle method is as follows:
*
* If there is a {@code lombok.With} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
* If not, the with is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*
* @param fieldNode The node representing the field you want a with for.
* @param pos The node responsible for generating the with (the {@code @Value} or {@code @With}
annotation).
```

```
*/
public void generateWithForField(JavacNode fieldNode, DiagnosticPosition pos, AccessLevel level) {
if (hasAnnotation(With.class, fieldNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
createWithForField(level, fieldNode, fieldNode, false, List.<JCAnnotation>nil(),
List.<JCAnnotation>nil());
}
@Override public void handle(AnnotationValues<With> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleFlagUsage(annotationNode, ConfigurationKeys.WITH_FLAG_USAGE, "@With");
Collection<JavacNode> fields = annotationNode.upFromAnnotationToFields();
deleteAnnotationIfNeccessary(annotationNode, With.class, "lombok.experimental.Wither");
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode node = annotationNode.up();
AccessLevel level = annotation.getInstance().value();
if (level == AccessLevel.NONE || node == null) return;
List<JCAnnotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@With(onMethod", annotationNode);
List<JCAnnotation> onParam = unboxAndRemoveAnnotationParameter(ast, "onParam",
"@With(onParam", annotationNode);
switch (node.getKind()) {
case FIELD:
createWithForFields(level, fields, annotationNode, true, onMethod, onParam);
break;
case TYPE:
if (!onMethod.isEmpty()) annotationNode.addError("'onMethod' is not supported for @With on a type.");
if (!onParam.isEmpty()) annotationNode.addError("'onParam' is not supported for @With on a type.");
generateWithForType(node, annotationNode, level, false);
break;
}
}
public void createWithForFields(AccessLevel level, Collection<JavacNode> fieldNodes, JavacNode
errorNode, boolean whineIfExists, List<JCAnnotation> onMethod, List<JCAnnotation> onParam) {
for (JavacNode fieldNode : fieldNodes) {
createWithForField(level, fieldNode, errorNode, whineIfExists, onMethod, onParam);
}
}
public void createWithForField(AccessLevel level, JavacNode fieldNode, JavacNode source, boolean
strictMode, List<JCAnnotation> onMethod, List<JCAnnotation> onParam) {
JavacNode typeNode = fieldNode.up();
boolean makeAbstract = typeNode != null && typeNode.getKind() == Kind.TYPE && (((JCClassDecl)
typeNode.get()).mods.flags & Flags.ABSTRACT) != 0;
if (fieldNode.getKind() != Kind.FIELD) {
fieldNode.addError("@With is only supported on a class or a field.");
return;
}
AnnotationValues<Accessors> accessors = getAccessorsForField(fieldNode);
JCVariableDecl fieldDecl = (JCVariableDecl) fieldNode.get();
String methodName = toWithName(fieldNode, accessors);
if (methodName == null) {
```

```
fieldNode.addWarning("Not generating a withX method for this field: It does not fit your @Accessors
prefix list.");
return;
}
if ((fieldDecl.mods.flags & Flags.STATIC) != 0) {
if (strictMode) {
fieldNode.addWarning("Not generating " + methodName + " for this field: With methods cannot be
generated for static fields.");
}
return;
}
if ((fieldDecl.mods.flags & Flags.FINAL) != 0 && fieldDecl.init != null) {
if (strictMode) {
fieldNode.addWarning("Not generating " + methodName + " for this field: With methods cannot be
generated for final, initialized fields.");
}
return;
}
if (fieldDecl.name.toString().startsWith("$")) {
if (strictMode) {
fieldNode.addWarning("Not generating " + methodName + " for this field: With methods cannot be
generated for fields starting with $.");
}
return;
}
for (String altName : toAllWithNames(fieldNode, accessors)) {
switch (methodExists(altName, fieldNode, false, 1)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (strictMode) {
String altNameExpl = "";
if (!altName.equals(methodName)) altNameExpl = String.format(" (%s)", altName);
fieldNode.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", methodName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
//continue scanning the other alt names.
}
}
long access = toJavacModifier(level);
JCMethodDecl createdWith = createWith(access, fieldNode, fieldNode.getTreeMaker(), source,
onMethod, onParam, makeAbstract);
createRelevantNonNullAnnotation(fieldNode, createdWith);
recursiveSetGeneratedBy(createdWith, source);
injectMethod(typeNode, createdWith);
}
public JCMethodDecl createWith(long access, JavacNode field, JavacTreeMaker maker, JavacNode
source, List<JCAnnotation> onMethod, List<JCAnnotation> onParam, boolean makeAbstract) {
String withName = toWithName(field);
```

```
if (withName == null) return null;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
List<JCAnnotation> copyableAnnotations = findCopyableAnnotations(field);
Name methodName = field.toName(withName);
JCExpression returnType = cloneSelfType(field);
JCBlock methodBody = null;
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, field.getContext());
List<JCAnnotation> annsOnParam = copyAnnotations(onParam).appendList(copyableAnnotations);
JCExpression pType = cloneType(maker, fieldDecl.vartype, source);
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags, annsOnParam), fieldDecl.name, pType,
null);
if (!makeAbstract) {
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
JCExpression selfType = cloneSelfType(field);
if (selfType == null) return null;
ListBuffer<JCExpression> args = new ListBuffer<JCExpression>();
for (JavacNode child : field.up().down()) {
if (child.getKind() != Kind.FIELD) continue;
JCVariableDecl childDecl = (JCVariableDecl) child.get();
// Skip fields that start with $
if (childDecl.name.toString().startsWith("$")) continue;
long fieldFlags = childDecl.mods.flags;
// Skip static fields.
if ((fieldFlags & Flags.STATIC) != 0) continue;
// Skip initialized final fields.
if (((fieldFlags & Flags.FINAL) != 0) && childDecl.init != null) continue;
if (child.get() == field.get()) {
args.append(maker.Ident(fieldDecl.name));
} else {
args.append(createFieldAccessor(maker, child, FieldAccess.ALWAYS_FIELD));
}
}
JCNewClass newClass = maker.NewClass(null, List.<JCExpression>nil(), selfType, args.toList(), null);
JCExpression identityCheck = maker.Binary(CTC_EQUAL, createFieldAccessor(maker, field,
FieldAccess.ALWAYS_FIELD), maker.Ident(fieldDecl.name));
JCConditional conditional = maker.Conditional(identityCheck, maker.Ident(field.toName("this")),
newClass);
JCReturn returnStatement = maker.Return(conditional);
if (!hasNonNullAnnotations(field)) {
statements.append(returnStatement);
} else {
JCStatement nullCheck = generateNullCheck(maker, field, source);
if (nullCheck != null) statements.append(nullCheck);
statements.append(returnStatement);
}
methodBody = maker.Block(0, statements.toList());
}
List<JCTypeParameter> methodGenericParams = List.nil();
List<JCVariableDecl> parameters = List.of(param);
List<JCExpression> throwsClauses = List.nil();
JCExpression annotationMethodDefaultValue = null;
List<JCAnnotation> annsOnMethod = copyAnnotations(onMethod);
CheckerFrameworkVersion checkerFramework = getCheckerFrameworkVersion(source);
```

```
if (checkerFramework.generateSideEffectFree()) annsOnMethod =
annsOnMethod.prepend(maker.Annotation(genTypeRef(source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
if (isFieldDeprecated(field)) annsOnMethod =
annsOnMethod.prepend(maker.Annotation(genJavaLangTypeRef(field, "Deprecated"),
List.<JCExpression>nil()));
if (makeAbstract) access |= Flags.ABSTRACT;
AnnotationValues<Accessors> accessors = JavacHandlerUtil.getAccessorsForField(field);
boolean makeFinal = shouldMakeFinal(field, accessors);
if (makeFinal) access |= Flags.FINAL;
JCMethodDecl decl = recursiveSetGeneratedBy(maker.MethodDef(maker.Modifiers(access,
annsOnMethod), methodName, returnType,
methodGenericParams, parameters, throwsClauses, methodBody, annotationMethodDefaultValue),
source);
copyJavadoc(field, decl, CopyJavadoc.WITH);
return decl;
}
}
```

lombok/javac/handlers/HandleWithBy.java


```
/*
* Copyright (C) 2020-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.javac.handlers;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.Collection;
import javax.lang.model.type.TypeKind;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
```

```
import lombok.core.LombokImmutableList;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.experimental.Accessors;
import lombok.experimental.WithBy;
import lombok.javac.Javac;
import lombok.javac.JavacAnnotationHandler;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.JavacTreeMaker.TypeTag;
import lombok.javac.handlers.JavacHandlerUtil.CopyJavadoc;
import lombok.spi.Provides;
import com.sun.tools.javac.code.BoundKind;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCNewClass;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
/**
* Handles the {@code lombok.With} annotation for javac.
*/
@Provides
public class HandleWithBy extends JavacAnnotationHandler<WithBy> {
public void generateWithByForType(JavacNode typeNode, JavacNode errorNode, AccessLevel level,
boolean checkForTypeLevelWithBy) {
if (checkForTypeLevelWithBy) {
if (hasAnnotation(WithBy.class, typeNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
}
JCClassDecl typeDecl = null;
if (typeNode.get() instanceof JCClassDecl) typeDecl = (JCClassDecl) typeNode.get();
long modifiers = typeDecl == null ? 0 : typeDecl.mods.flags;
boolean notAClass = (modifiers & (Flags.INTERFACE | Flags.ANNOTATION | Flags.ENUM)) != 0;
if (typeDecl == null || notAClass) {
errorNode.addError("@WithBy is only supported on a class or a field.");
return;
}
for (JavacNode field : typeNode.down()) {
if (field.getKind() != Kind.FIELD) continue;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
//Skip fields that start with $
```

```
if (fieldDecl.name.toString().startsWith("$")) continue;
//Skip static fields.
if ((fieldDecl.mods.flags & Flags.STATIC) != 0) continue;
//Skip final initialized fields.
if ((fieldDecl.mods.flags & Flags.FINAL) != 0 && fieldDecl.init != null) continue;
generateWithByForField(field, errorNode.get(), level);
}
}
/**
* Generates a withBy on the stated field.
*
* The difference between this call and the handle method is as follows:
*
* If there is a {@code lombok.experimental.WithBy} annotation on the field, it is used and the
* same rules apply (e.g. warning if the method already exists, stated access level applies).
* If not, the with is still generated if it isn't already there, though there will not
* be a warning if its already there. The default access level is used.
*
* @param fieldNode The node representing the field you want a with for.
* @param pos The node responsible for generating the {@code @WithBy} annotation.
*/
public void generateWithByForField(JavacNode fieldNode, DiagnosticPosition pos, AccessLevel level)
{
if (hasAnnotation(WithBy.class, fieldNode)) {
//The annotation will make it happen, so we can skip it.
return;
}
createWithByForField(level, fieldNode, fieldNode, false, List.<JCAnnotation>nil());
}
@Override public void handle(AnnotationValues<WithBy> annotation, JCAnnotation ast, JavacNode
annotationNode) {
handleExperimentalFlagUsage(annotationNode, ConfigurationKeys.WITHBY_FLAG_USAGE,
"@WithBy");
deleteAnnotationIfNeccessary(annotationNode, WithBy.class);
deleteImportFromCompilationUnit(annotationNode, "lombok.AccessLevel");
JavacNode node = annotationNode.up();
AccessLevel level = annotation.getInstance().value();
if (level == AccessLevel.NONE || node == null) return;
List<JCAnnotation> onMethod = unboxAndRemoveAnnotationParameter(ast, "onMethod",
"@WithBy(onMethod", annotationNode);
switch (node.getKind()) {
case FIELD:
createWithByForFields(level, annotationNode.upFromAnnotationToFields(), annotationNode, true,
onMethod);
break;
case TYPE:
if (!onMethod.isEmpty()) annotationNode.addError("'onMethod' is not supported for @WithBy on a
type.");
generateWithByForType(node, annotationNode, level, false);
break;
}
}
```

```
public void createWithByForFields(AccessLevel level, Collection<JavacNode> fieldNodes, JavacNode
errorNode, boolean whineIfExists, List<JCAnnotation> onMethod) {
for (JavacNode fieldNode : fieldNodes) {
createWithByForField(level, fieldNode, errorNode, whineIfExists, onMethod);
}
}
public void createWithByForField(AccessLevel level, JavacNode fieldNode, JavacNode source,
boolean strictMode, List<JCAnnotation> onMethod) {
JavacNode typeNode = fieldNode.up();
boolean makeAbstract = typeNode != null && typeNode.getKind() == Kind.TYPE && (((JCClassDecl)
typeNode.get()).mods.flags & Flags.ABSTRACT) != 0;
if (fieldNode.getKind() != Kind.FIELD) {
fieldNode.addError("@WithBy is only supported on a class or a field.");
return;
}
AnnotationValues<Accessors> accessors = JavacHandlerUtil.getAccessorsForField(fieldNode);
JCVariableDecl fieldDecl = (JCVariableDecl) fieldNode.get();
String methodName = toWithByName(fieldNode, accessors);
if (methodName == null) {
fieldNode.addWarning("Not generating a withXBy method for this field: It does not fit your @Accessors
prefix list.");
return;
}
if ((fieldDecl.mods.flags & Flags.STATIC) != 0) {
if (strictMode) fieldNode.addWarning("Not generating " + methodName + " for this field: WithBy
methods cannot be generated for static fields.");
return;
}
if ((fieldDecl.mods.flags & Flags.FINAL) != 0 && fieldDecl.init != null) {
if (strictMode) fieldNode.addWarning("Not generating " + methodName + " for this field: WithBy
methods cannot be generated for final, initialized fields.");
return;
}
if (fieldDecl.name.toString().startsWith("$")) {
if (strictMode) fieldNode.addWarning("Not generating " + methodName + " for this field: WithBy
methods cannot be generated for fields starting with $.");
return;
}
for (String altName : toAllWithByNames(fieldNode, accessors)) {
switch (methodExists(altName, fieldNode, false, 1)) {
case EXISTS_BY_LOMBOK:
return;
case EXISTS_BY_USER:
if (strictMode) {
String altNameExpl = "";
if (!altName.equals(methodName)) altNameExpl = String.format(" (%s)", altName);
fieldNode.addWarning(
String.format("Not generating %s(): A method with that name already exists%s", methodName,
altNameExpl));
}
return;
default:
case NOT_EXISTS:
```

```
//continue scanning the other alt names.
}
}
long access = toJavacModifier(level);
JCMethodDecl createdWithBy = createWithBy(access, fieldNode, fieldNode.getTreeMaker(), source,
onMethod, makeAbstract);
recursiveSetGeneratedBy(createdWithBy, source);
injectMethod(typeNode, createdWithBy);
}
private static final LombokImmutableList<String> NAME_JUF_FUNCTION =
LombokImmutableList.of("java", "util", "function", "Function");
private static final LombokImmutableList<String> NAME_JUF_OP = LombokImmutableList.of("java",
"util", "function", "UnaryOperator");
private static final LombokImmutableList<String> NAME_JUF_DOUBLEOP =
LombokImmutableList.of("java", "util", "function", "DoubleUnaryOperator");
private static final LombokImmutableList<String> NAME_JUF_INTOP =
LombokImmutableList.of("java", "util", "function", "IntUnaryOperator");
private static final LombokImmutableList<String> NAME_JUF_LONGOP =
LombokImmutableList.of("java", "util", "function", "LongUnaryOperator");
public JCMethodDecl createWithBy(long access, JavacNode field, JavacTreeMaker maker, JavacNode
source, List<JCAnnotation> onMethod, boolean makeAbstract) {
String withByName = toWithByName(field);
if (withByName == null) return null;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
Name methodName = field.toName(withByName);
JCExpression returnType = cloneSelfType(field);
JCBlock methodBody = null;
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, field.getContext());
LombokImmutableList<String> functionalInterfaceName = null;
TypeTag requiredCast = null;
JCExpression parameterizer = null;
boolean superExtendsStyle = true;
String applyMethodName = "apply";
if (fieldDecl.vartype instanceof JCPrimitiveTypeTree) {
TypeKind kind = ((JCPrimitiveTypeTree) fieldDecl.vartype).getPrimitiveTypeKind();
if (kind == TypeKind.CHAR) {
requiredCast = Javac.CTC_CHAR;
functionalInterfaceName = NAME_JUF_INTOP;
} else if (kind == TypeKind.SHORT) {
requiredCast = Javac.CTC_SHORT;
functionalInterfaceName = NAME_JUF_INTOP;
} else if (kind == TypeKind.BYTE) {
requiredCast = Javac.CTC_BYTE;
functionalInterfaceName = NAME_JUF_INTOP;
} else if (kind == TypeKind.INT) {
functionalInterfaceName = NAME_JUF_INTOP;
} else if (kind == TypeKind.LONG) {
functionalInterfaceName = NAME_JUF_LONGOP;
} else if (kind == TypeKind.FLOAT) {
functionalInterfaceName = NAME_JUF_DOUBLEOP;
requiredCast = Javac.CTC_FLOAT;
} else if (kind == TypeKind.DOUBLE) {
functionalInterfaceName = NAME_JUF_DOUBLEOP;
```

```
} else if (kind == TypeKind.BOOLEAN) {
functionalInterfaceName = NAME_JUF_OP;
parameterizer = JavacHandlerUtil.genJavaLangTypeRef(field, "Boolean");
superExtendsStyle = false;
}
}
if (functionalInterfaceName == null) {
functionalInterfaceName = NAME_JUF_FUNCTION;
parameterizer = cloneType(maker, fieldDecl.vartype, source);
}
if (functionalInterfaceName == NAME_JUF_INTOP) applyMethodName = "applyAsInt";
if (functionalInterfaceName == NAME_JUF_LONGOP) applyMethodName = "applyAsLong";
if (functionalInterfaceName == NAME_JUF_DOUBLEOP) applyMethodName = "applyAsDouble";
JCExpression varType = chainDots(field, functionalInterfaceName);
if (parameterizer != null && superExtendsStyle) {
JCExpression parameterizer1 = parameterizer;
JCExpression parameterizer2 = cloneType(maker, parameterizer, source);
// TODO: Apply copyable annotations to 'parameterizer' and 'parameterizer2'.
JCExpression arg1 = maker.Wildcard(maker.TypeBoundKind(BoundKind.SUPER), parameterizer1);
JCExpression arg2 = maker.Wildcard(maker.TypeBoundKind(BoundKind.EXTENDS), parameterizer2);
varType = maker.TypeApply(varType, List.of(arg1, arg2));
}
if (parameterizer != null && !superExtendsStyle) {
varType = maker.TypeApply(varType, List.of(parameterizer));
}
Name paramName = field.toName("transformer");
JCVariableDecl param = maker.VarDef(maker.Modifiers(flags), paramName, varType, null);
if (!makeAbstract) {
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
JCExpression selfType = cloneSelfType(field);
if (selfType == null) return null;
ListBuffer<JCExpression> args = new ListBuffer<JCExpression>();
for (JavacNode child : field.up().down()) {
if (child.getKind() != Kind.FIELD) continue;
JCVariableDecl childDecl = (JCVariableDecl) child.get();
// Skip fields that start with $
if (childDecl.name.toString().startsWith("$")) continue;
long fieldFlags = childDecl.mods.flags;
// Skip static fields.
if ((fieldFlags & Flags.STATIC) != 0) continue;
// Skip initialized final fields.
if (((fieldFlags & Flags.FINAL) != 0) && childDecl.init != null) continue;
if (child.get() == field.get()) {
JCExpression invoke = maker.Apply(List.<JCExpression>nil(), maker.Select(maker.Ident(paramName),
field.toName(applyMethodName)), List.<JCExpression>of(createFieldAccessor(maker, child,
FieldAccess.ALWAYS_FIELD)));
if (requiredCast != null) invoke = maker.TypeCast(maker.TypeIdent(requiredCast), invoke);
args.append(invoke);
} else {
args.append(createFieldAccessor(maker, child, FieldAccess.ALWAYS_FIELD));
}
}
JCNewClass newClass = maker.NewClass(null, List.<JCExpression>nil(), selfType, args.toList(), null);
```

```
JCReturn returnStatement = maker.Return(newClass);
statements.append(returnStatement);
methodBody = maker.Block(0, statements.toList());
}
List<JCTypeParameter> methodGenericParams = List.nil();
List<JCVariableDecl> parameters = List.of(param);
List<JCExpression> throwsClauses = List.nil();
JCExpression annotationMethodDefaultValue = null;
List<JCAnnotation> annsOnMethod = copyAnnotations(onMethod);
CheckerFrameworkVersion checkerFramework = getCheckerFrameworkVersion(source);
if (checkerFramework.generateSideEffectFree()) annsOnMethod =
annsOnMethod.prepend(maker.Annotation(genTypeRef(source,
CheckerFrameworkVersion.NAME__SIDE_EFFECT_FREE), List.<JCExpression>nil()));
if (isFieldDeprecated(field)) annsOnMethod =
annsOnMethod.prepend(maker.Annotation(genJavaLangTypeRef(field, "Deprecated"),
List.<JCExpression>nil()));
if (makeAbstract) access = access | Flags.ABSTRACT;
AnnotationValues<Accessors> accessors = JavacHandlerUtil.getAccessorsForField(field);
boolean makeFinal = shouldMakeFinal(field, accessors);
if (makeFinal) access |= Flags.FINAL;
createRelevantNonNullAnnotation(source, param);
JCMethodDecl decl = recursiveSetGeneratedBy(maker.MethodDef(maker.Modifiers(access,
annsOnMethod), methodName, returnType,
methodGenericParams, parameters, throwsClauses, methodBody, annotationMethodDefaultValue),
source);
copyJavadoc(field, decl, CopyJavadoc.WITH_BY);
createRelevantNonNullAnnotation(source, decl);
return decl;
}
}
```

lombok/javac/handlers/JavacHandlerUtil.java


```
/*
* Copyright (C) 2009-2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
```

* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers;
import static com.sun.tools.javac.code.Flags.GENERATEDCONSTR;
import static lombok.core.handlers.HandlerUtil.*;
import static lombok.javac.Javac.*;
import static lombok.javac.JavacAugments.JCTree_generatedNode;
import static lombok.javac.JavacAugments.JCTree_keepPosition;
import java.lang.annotation.Annotation;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.regex.Pattern;
import com.sun.tools.javac.code.Attribute;
import com.sun.tools.javac.code.BoundKind;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.code.Scope;
import com.sun.tools.javac.code.Symbol;
import com.sun.tools.javac.code.Symbol.ClassSymbol;
import com.sun.tools.javac.code.Symbol.TypeSymbol;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.comp.Annotate;
import com.sun.tools.javac.comp.AttrContext;
import com.sun.tools.javac.comp.Enter;
import com.sun.tools.javac.comp.Env;
import com.sun.tools.javac.comp.MemberEnter;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCImport;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCNewArray;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeApply;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
```

```
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.JCWildcard;
import com.sun.tools.javac.tree.JCTree.TypeBoundKind;
import com.sun.tools.javac.tree.TreeMaker;
import com.sun.tools.javac.tree.TreeScanner;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import com.sun.tools.javac.util.Options;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.Data;
import lombok.Getter;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.AnnotationValues.AnnotationValue;
import lombok.core.CleanupTask;
import lombok.core.LombokImmutableList;
import lombok.core.TypeResolver;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.configuration.NullAnnotationLibrary;
import lombok.core.configuration.NullCheckExceptionType;
import lombok.core.configuration.TypeName;
import lombok.core.handlers.HandlerUtil;
import lombok.core.handlers.HandlerUtil.FieldAccess;
import lombok.delombok.LombokOptionsFactory;
import lombok.experimental.Accessors;
import lombok.experimental.Tolerate;
import lombok.javac.Javac;
import lombok.javac.JavacAugments;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.JavacTreeMaker.TypeTag;
import lombok.permit.Permit;
/**
* Container for static utility methods useful to handlers written for javac.
*/
public class JavacHandlerUtil {
private JavacHandlerUtil() {
//Prevent instantiation
}
private static class MarkingScanner extends TreeScanner {
private final JavacNode source;
MarkingScanner(JavacNode source) {
this.source = source;
}
@Override public void scan(JCTree tree) {
if (tree == null) return;
if (JCTree_keepPosition.get(tree)) return;
setGeneratedBy(tree, source);
super.scan(tree);
```

```
}
}
/**
* Contributed by Jan Lahoda; many lombok transformations should not be run (or a lite version should be run) when the netbeans editor
* is running javac on the open source file to find inline errors and such. As class files are compiled separately this does not affect
* actual runtime behaviour or file output of the netbeans IDE.
*/
public static boolean inNetbeansEditor(JavacNode node) {
return inNetbeansEditor(node.getContext());
}
public static boolean inNetbeansEditor(Context context) {
Options options = Options.instance(context);
return (options.keySet().contains("ide") && !options.keySet().contains("backgroundCompilation"));
}
public static boolean inNetbeansCompileOnSave(Context context) {
Options options = Options.instance(context);
return (options.keySet().contains("ide") && options.keySet().contains("backgroundCompilation"));
}
public static JCTree getGeneratedBy(JCTree node) {
return JCTree_generatedNode.get(node);
}
public static boolean isGenerated(JCTree node) {
return getGeneratedBy(node) != null;
}
public static <T extends JCTree> T recursiveSetGeneratedBy(T node, JavacNode source) {
if (node == null) return null;
setGeneratedBy(node, source);
node.accept(new MarkingScanner(source));
return node;
}
public static <T extends JCTree> T setGeneratedBy(T node, JavacNode sourceNode) {
if (node == null) return null;
if (sourceNode == null) {
JCTree_generatedNode.clear(node);
return node;
}
JCTree_generatedNode.set(node, sourceNode.get());
if (JCTree_keepPosition.get(node)) return node;
if (inNetbeansEditor(sourceNode.getContext()) && !isParameter(node)) return node;
node.pos = sourceNode.getStartPos();
storeEnd(node, sourceNode.getEndPosition(), (JCCompilationUnit) sourceNode.top().get());
return node;
}
public static boolean isParameter(JCTree node) {
return node instanceof JCVariableDecl && (((JCVariableDecl) node).mods.flags & Flags.PARAMETER) != 0;
}
public static boolean hasAnnotation(String type, JavacNode node) {
return hasAnnotation(type, node, false);
}
public static boolean hasAnnotation(Class<? extends Annotation> type, JavacNode node) {
```

```
return hasAnnotation(type, node, false);
}
public static boolean hasAnnotationAndDeleteIfNeccessary(Class<? extends Annotation> type,
JavacNode node) {
return hasAnnotation(type, node, true);
}
private static boolean hasAnnotation(Class<? extends Annotation> type, JavacNode node, boolean
delete) {
if (node == null) return false;
if (type == null) return false;
switch (node.getKind()) {
case ARGUMENT:
case FIELD:
case LOCAL:
case TYPE:
case METHOD:
for (JavacNode child : node.down()) {
if (annotationTypeMatches(type, child)) {
if (delete) deleteAnnotationIfNeccessary(child, type);
return true;
}
}
// intentional fallthrough
default:
return false;
}
}
private static boolean hasAnnotation(String type, JavacNode node, boolean delete) {
if (node == null) return false;
if (type == null) return false;
switch (node.getKind()) {
case ARGUMENT:
case FIELD:
case LOCAL:
case TYPE:
case METHOD:
for (JavacNode child : node.down()) {
if (annotationTypeMatches(type, child)) {
if (delete) deleteAnnotationIfNeccessary(child, type);
return true;
}
}
// intentional fallthrough
default:
return false;
}
}
public static JavacNode findInnerClass(JavacNode parent, String name) {
for (JavacNode child : parent.down()) {
if (child.getKind() != Kind.TYPE) continue;
JCClassDecl td = (JCClassDecl) child.get();
if (td.name.contentEquals(name)) return child;
}
```

```
return null;
}
public static JavacNode findAnnotation(Class<? extends Annotation> type, JavacNode node) {
return findAnnotation(type, node, false);
}
public static JavacNode findAnnotation(Class<? extends Annotation> type, JavacNode node, boolean
delete) {
if (node == null) return null;
if (type == null) return null;
switch (node.getKind()) {
case ARGUMENT:
case FIELD:
case LOCAL:
case TYPE:
case METHOD:
for (JavacNode child : node.down()) {
if (annotationTypeMatches(type, child)) {
if (delete) deleteAnnotationIfNeccessary(child, type);
return child;
}
}
// intentional fallthrough
default:
return null;
}
}
/**
* Checks if the Annotation AST Node provided is likely to be an instance of the provided annotation
type.
*
* @param type An actual annotation type, such as {@code lombok.Getter.class}.
* @param node A Lombok AST node representing an annotation in source code.
*/
public static boolean annotationTypeMatches(Class<? extends Annotation> type, JavacNode node) {
if (node.getKind() != Kind.ANNOTATION) return false;
return typeMatches(type, node, ((JCAnnotation) node.get()).annotationType);
}
/**
* Checks if the Annotation AST Node provided is likely to be an instance of the provided annotation
type.
*
* @param type An actual annotation type, such as {@code lombok.Getter.class}.
* @param node A Lombok AST node representing an annotation in source code.
*/
public static boolean annotationTypeMatches(String type, JavacNode node) {
if (node.getKind() != Kind.ANNOTATION) return false;
return typeMatches(type, node, ((JCAnnotation) node.get()).annotationType);
}
/**
* Checks if the given TypeReference node is likely to be a reference to the provided class.
*
* @param type An actual type. This method checks if {@code typeNode} is likely to be a reference to
this type.
```

```
* @param node A Lombok AST node. Any node in the appropriate compilation unit will do (used to get access to import statements).
* @param typeNode A type reference to check.
*/
public static boolean typeMatches(Class<?> type, JavacNode node, JCTree typeNode) {
return typeMatches(type.getName(), node, typeNode);
}
/**
* Checks if the given TypeReference node is likely to be a reference to the provided class.
*
* @param type An actual type. This method checks if {@code typeNode} is likely to be a reference to this type.
* @param node A Lombok AST node. Any node in the appropriate compilation unit will do (used to get access to import statements).
* @param typeNode A type reference to check.
*/
public static boolean typeMatches(String type, JavacNode node, JCTree typeNode) {
String typeName = getTypeName(typeNode);
return typeMatches(type, node, typeName);
}
private static boolean typeMatches(String type, JavacNode node, String typeName) {
if (typeName == null || typeName.length() == 0) return false;
int lastIndexA = typeName.lastIndexOf('.') + 1;
int lastIndexB = Math.max(type.lastIndexOf('.'), type.lastIndexOf('$')) + 1;
int len = typeName.length() - lastIndexA;
if (len != type.length() - lastIndexB) return false;
for (int i = 0; i < len; i++) if (typeName.charAt(i + lastIndexA) != type.charAt(i + lastIndexB)) return false;
TypeResolver resolver = node.getImportListAsTypeResolver();
return resolver.typeMatches(node, type, typeName);
}
private static String getTypeName(JCTree typeNode) {
return typeNode == null ? null : typeNode.toString();
}
/**
* Returns if a field is marked deprecated, either by {@code @Deprecated} or in javadoc
* @param field the field to check
* @return {@code true} if a field is marked deprecated, either by {@code @Deprecated} or in javadoc, otherwise {@code false}
*/
public static boolean isFieldDeprecated(JavacNode field) {
if (!(field.get() instanceof JCVariableDecl)) return false;
JCVariableDecl fieldNode = (JCVariableDecl) field.get();
if ((fieldNode.mods.flags & Flags.DEPRECATED) != 0) {
return true;
}
for (JavacNode child : field.down()) {
if (annotationTypeMatches(Deprecated.class, child)) {
return true;
}
}
return false;
}
public static CheckerFrameworkVersion getCheckerFrameworkVersion(JavacNode node) {
```

```
CheckerFrameworkVersion cfv =
node.getAst().readConfiguration(ConfigurationKeys.CHECKER_FRAMEWORK);
return cfv == null ? CheckerFrameworkVersion.NONE : cfv;
}
/**
* Returns if a node is marked deprecated (as picked up on by the parser).
* @param node the node to check (type, method, or field decl).
*/
public static boolean nodeHasDeprecatedFlag(JCTree node) {
if (node instanceof JCVariableDecl) return (((JCVariableDecl) node).mods.flags &
Flags.DEPRECATED) != 0;
if (node instanceof JCMethodDecl) return (((JCMethodDecl) node).mods.flags & Flags.DEPRECATED)
!= 0;
if (node instanceof JCClassDecl) return (((JCClassDecl) node).mods.flags & Flags.DEPRECATED) !=
0;
return false;
}
/**
* Creates an instance of {@code AnnotationValues} for the provided AST Node.
*
* @param type An annotation class type, such as {@code lombok.Getter.class}.
* @param node A Lombok AST node representing an annotation in source code.
*/
public static <A extends Annotation> AnnotationValues<A> createAnnotation(Class<A> type, final
JavacNode node) {
return createAnnotation(type, (JCAnnotation) node.get(), node);
}
/**
* Creates an instance of {@code AnnotationValues} for the provided AST Node
* and Annotation expression.
*
* @param type An annotation class type, such as {@code lombok.Getter.class}.
* @param anno the annotation expression
* @param node A Lombok AST node representing an annotation in source code.
*/
public static <A extends Annotation> AnnotationValues<A> createAnnotation(Class<A> type,
JCAnnotation anno, final JavacNode node) {
Map<String, AnnotationValue> values = new HashMap<String, AnnotationValue>();
List<JCExpression> arguments = anno.getArguments();
for (JCExpression arg : arguments) {
String mName;
JCExpression rhs;
java.util.List<String> raws = new ArrayList<String>();
java.util.List<Object> guesses = new ArrayList<Object>();
java.util.List<Object> expressions = new ArrayList<Object>();
final java.util.List<DiagnosticPosition> positions = new ArrayList<DiagnosticPosition>();
if (arg instanceof JCAssign) {
JCAssign assign = (JCAssign) arg;
mName = assign.lhs.toString();
rhs = assign.rhs;
} else {
rhs = arg;
mName = "value";
```

```
}
if (rhs instanceof JCNewArray) {
List<JCExpression> elems = ((JCNewArray) rhs).elems;
for (JCExpression inner : elems) {
raws.add(inner.toString());
expressions.add(inner);
if (inner instanceof JCAnnotation) {
try {
@SuppressWarnings("unchecked")
Class<A> innerClass = (Class<A>) Class.forName(inner.type.toString());
guesses.add(createAnnotation(innerClass, (JCAnnotation) inner, node));
} catch (ClassNotFoundException ex) {
guesses.add(calculateGuess(inner));
}
} else {
guesses.add(calculateGuess(inner));
}
positions.add(inner.pos());
}
} else {
raws.add(rhs.toString());
expressions.add(rhs);
if (rhs instanceof JCAnnotation) {
try {
@SuppressWarnings("unchecked")
Class<A> innerClass = (Class<A>) Class.forName(rhs.type.toString());
guesses.add(createAnnotation(innerClass, (JCAnnotation) rhs, node));
} catch (ClassNotFoundException ex) {
guesses.add(calculateGuess(rhs));
}
} else {
guesses.add(calculateGuess(rhs));
}
positions.add(rhs.pos());
}
values.put(mName, new AnnotationValue(node, raws, expressions, guesses, true) {
@Override public void setError(String message, int valueIdx) {
if (valueIdx < 0) node.addError(message);
else node.addError(message, positions.get(valueIdx));
}
@Override public void setWarning(String message, int valueIdx) {
if (valueIdx < 0) node.addWarning(message);
else node.addWarning(message, positions.get(valueIdx));
}
});
}
for (Method m : type.getDeclaredMethods()) {
if (!Modifier.isPublic(m.getModifiers())) continue;
String name = m.getName();
if (!values.containsKey(name)) {
values.put(name, new AnnotationValue(node, new ArrayList<String>(), new ArrayList<Object>(), new
ArrayList<Object>(), false) {
@Override public void setError(String message, int valueIdx) {
```

```
node.addError(message);
}
@Override public void setWarning(String message, int valueIdx) {
node.addWarning(message);
}
});
}
}
return new AnnotationValues<A>(type, values, node);
}
/**
* Removes the annotation from javac's AST (it remains in lombok's AST),
* then removes any import statement that imports this exact annotation (not star imports).
* Only does this if the DeleteLombokAnnotations class is in the context.
*/
public static void deleteAnnotationIfNeccessary(JavacNode annotation, String annotationType) {
deleteAnnotationIfNeccessary0(annotation, annotationType);
}
/**
* Removes the annotation from javac's AST (it remains in lombok's AST),
* then removes any import statement that imports this exact annotation (not star imports).
* Only does this if the DeleteLombokAnnotations class is in the context.
*/
public static void deleteAnnotationIfNeccessary(JavacNode annotation, Class<? extends Annotation>
annotationType) {
deleteAnnotationIfNeccessary0(annotation, annotationType.getName());
}
/**
* Removes the annotation from javac's AST (it remains in lombok's AST),
* then removes any import statement that imports this exact annotation (not star imports).
* Only does this if the DeleteLombokAnnotations class is in the context.
*/
public static void deleteAnnotationIfNeccessary(JavacNode annotation, Class<? extends Annotation>
annotationType1, Class<? extends Annotation> annotationType2) {
deleteAnnotationIfNeccessary0(annotation, annotationType1.getName(), annotationType2.getName());
}
/**
* Removes the annotation from javac's AST (it remains in lombok's AST),
* then removes any import statement that imports this exact annotation (not star imports).
* Only does this if the DeleteLombokAnnotations class is in the context.
*/
public static void deleteAnnotationIfNeccessary(JavacNode annotation, Class<? extends Annotation>
annotationType1, String annotationType2) {
deleteAnnotationIfNeccessary0(annotation, annotationType1.getName(), annotationType2);
}
private static void deleteAnnotationIfNeccessary0(JavacNode annotation, String... annotationTypes) {
if (inNetbeansEditor(annotation)) return;
if (!annotation.shouldDeleteLombokAnnotations()) return;
JavacNode parentNode = annotation.directUp();
switch (parentNode.getKind()) {
case FIELD:
case ARGUMENT:
case LOCAL:
```

```
JCVariableDecl variable = (JCVariableDecl) parentNode.get();
variable.mods.annotations = filterList(variable.mods.annotations, annotation.get());
if ((variable.mods.flags & GENERATED_MEMBER) != 0) {
JavacNode typeNode = upToTypeNode(annotation);
if (isRecord(typeNode)) {
RecordComponentReflect.deleteAnnotation((JCClassDecl) typeNode.get(), variable, annotation.get());
}
}
break;
case METHOD:
JCMethodDecl method = (JCMethodDecl) parentNode.get();
method.mods.annotations = filterList(method.mods.annotations, annotation.get());
break;
case TYPE:
try {
JCClassDecl type = (JCClassDecl) parentNode.get();
type.mods.annotations = filterList(type.mods.annotations, annotation.get());
} catch (ClassCastException e) {
//something rather odd has been annotated. Better to just break only delombok instead of everything.
}
break;
default:
//This really shouldn't happen, but if it does, better just break delombok instead of breaking everything.
return;
}
parentNode.getAst().setChanged();
for (String annotationType : annotationTypes) {
deleteImportFromCompilationUnit(annotation, annotationType);
}
}
public static void deleteImportFromCompilationUnit(JavacNode node, String name) {
if (inNetbeansEditor(node)) return;
if (!node.shouldDeleteLombokAnnotations()) return;
JCCompilationUnit unit = (JCCompilationUnit) node.top().get();
for (JCTree def : unit.defs) {
if (!(def instanceof JCImport)) continue;
JCImport imp0rt = (JCImport) def;
if (imp0rt.staticImport) continue;
if (!Javac.getQualid(imp0rt).toString().equals(name)) continue;
JavacAugments.JCImport_deletable.set(imp0rt, true);
}
}
private static List<JCAnnotation> filterList(List<JCAnnotation> annotations, JCTree jcTree) {
ListBuffer<JCAnnotation> newAnnotations = new ListBuffer<JCAnnotation>();
for (JCAnnotation ann : annotations) {
if (jcTree != ann) newAnnotations.append(ann);
}
return newAnnotations.toList();
}
private static List<JCAnnotation> filterListByPos(List<JCAnnotation> annotations, JCTree jcTree) {
ListBuffer<JCAnnotation> newAnnotations = new ListBuffer<JCAnnotation>();
for (JCAnnotation ann : annotations) {
if (jcTree.pos != ann.pos) newAnnotations.append(ann);
```

```
}
return newAnnotations.toList();
}
static class RecordComponentReflect {
private static final Field astField;
private static final Field originalAnnos;
private static final Method findRecordComponentToRemove;
static {
Field a = null;
Field o = null;
Method m = null;
try {
Class<?> forName = Class.forName("com.sun.tools.javac.code.Symbol$RecordComponent");
a = Permit.permissiveGetField(forName, "ast");
o = Permit.permissiveGetField(forName, "originalAnnos");
m = Permit.permissiveGetMethod(ClassSymbol.class, "findRecordComponentToRemove",
JCVariableDecl.class);
} catch (Throwable e) {
// Ignore
}
astField = a;
originalAnnos = o;
findRecordComponentToRemove = m;
}
static void deleteAnnotation(JCClassDecl record, JCVariableDecl component, JCTree annotation) {
if ((astField == null && originalAnnos == null) || findRecordComponentToRemove == null) return;
try {
Object toRemove = Permit.invokeSneaky(findRecordComponentToRemove, record.sym, component);
if (astField != null) {
// OpenJDK
JCVariableDecl variable = Permit.get(astField, toRemove);
variable.mods.annotations = filterListByPos(variable.mods.annotations, annotation);
} else {
// Zulu JDK 17
List<JCAnnotation> annotations = Permit.get(originalAnnos, toRemove);
Permit.set(originalAnnos, toRemove, filterListByPos(annotations, annotation));
}
} catch (Throwable e) {
// Ignore
}
}
}
/** Serves as return value for the methods that check for the existence of fields and methods. */
public enum MemberExistsResult {
NOT_EXISTS, EXISTS_BY_LOMBOK, EXISTS_BY_USER;
}
/**
* Translates the given field into all possible getter names.
* Convenient wrapper around {@link HandlerUtil#toAllGetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllGetterNames(JavacNode field) {
```

```
return HandlerUtil.toAllGetterNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* Translates the given field into all possible getter names.
* Convenient wrapper around {@link HandlerUtil#toAllGetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllGetterNames(JavacNode field, AnnotationValues<Accessors>
accessors) {
return HandlerUtil.toAllGetterNames(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
* @return the likely getter name for the stated field. (e.g. private boolean foo; to isFoo).
*
* Convenient wrapper around {@link HandlerUtil#toGetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toGetterName(JavacNode field) {
return HandlerUtil.toGetterName(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* @return the likely getter name for the stated field. (e.g. private boolean foo; to isFoo).
*
* Convenient wrapper around {@link HandlerUtil#toGetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toGetterName(JavacNode field, AnnotationValues<Accessors> accessors) {
return HandlerUtil.toGetterName(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
* Translates the given field into all possible setter names.
* Convenient wrapper around {@link HandlerUtil#toAllSetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllSetterNames(JavacNode field) {
return HandlerUtil.toAllSetterNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* Translates the given field into all possible setter names.
* Convenient wrapper around {@link HandlerUtil#toAllSetterNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllSetterNames(JavacNode field, AnnotationValues<Accessors>
accessors) {
return HandlerUtil.toAllSetterNames(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
* @return the likely setter name for the stated field. (e.g. private boolean foo; to setFoo).
*
```

```
* Convenient wrapper around {@link HandlerUtil#toSetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toSetterName(JavacNode field) {
return HandlerUtil.toSetterName(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* @return the likely setter name for the stated field. (e.g. private boolean foo; to setFoo).
*
* Convenient wrapper around {@link HandlerUtil#toSetterName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toSetterName(JavacNode field, AnnotationValues<Accessors> accessors) {
return HandlerUtil.toSetterName(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
* Translates the given field into all possible with names.
* Convenient wrapper around {@link HandlerUtil#toAllWithNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithNames(JavacNode field) {
return HandlerUtil.toAllWithNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* Translates the given field into all possible with names.
* Convenient wrapper around {@link HandlerUtil#toAllWithNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithNames(JavacNode field, AnnotationValues<Accessors>
accessors) {
return HandlerUtil.toAllWithNames(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
* Translates the given field into all possible withBy names.
* Convenient wrapper around {@link HandlerUtil#toAllWithByNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithByNames(JavacNode field) {
return HandlerUtil.toAllWithByNames(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* Translates the given field into all possible withBy names.
* Convenient wrapper around {@link HandlerUtil#toAllWithByNames(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static java.util.List<String> toAllWithByNames(JavacNode field, AnnotationValues<Accessors>
accessors) {
return HandlerUtil.toAllWithByNames(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
```

```
* @return the likely with name for the stated field. (e.g. private boolean foo; to withFoo).
*
* Convenient wrapper around {@link HandlerUtil#toWithName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithName(JavacNode field) {
return HandlerUtil.toWithName(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* @return the likely with name for the stated field. (e.g. private boolean foo; to withFoo).
*
* Convenient wrapper around {@link HandlerUtil#toWithName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithName(JavacNode field, AnnotationValues<Accessors> accessors) {
return HandlerUtil.toWithName(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
* @return the likely withBy name for the stated field. (e.g. private boolean foo; to withFooBy).
*
* Convenient wrapper around {@link HandlerUtil#toWithByName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithByName(JavacNode field) {
return HandlerUtil.toWithByName(field.getAst(), getAccessorsForField(field), field.getName(),
isBoolean(field));
}
/**
* @return the likely withBy name for the stated field. (e.g. private boolean foo; to withFooBy).
*
* Convenient wrapper around {@link HandlerUtil#toWithByName(lombok.core.AnnotationValues,
CharSequence, boolean)}.
*/
public static String toWithByName(JavacNode field, AnnotationValues<Accessors> accessors) {
return HandlerUtil.toWithByName(field.getAst(), accessors, field.getName(), isBoolean(field));
}
/**
* When generating a setter, the setter either returns void (beanspec) or Self (fluent).
* This method scans for the {@code Accessors} annotation to figure that out.
*/
public static boolean shouldReturnThis(JavacNode field, AnnotationValues<Accessors> accessors) {
if ((((JCVariableDecl) field.get()).mods.flags & Flags.STATIC) != 0) return false;
return HandlerUtil.shouldReturnThis0(accessors, field.getAst());
}
/**
* When generating a setter/getter/wither, should it be made final?
*/
public static boolean shouldMakeFinal(JavacNode field, AnnotationValues<Accessors> accessors) {
if ((((JCVariableDecl) field.get()).mods.flags & Flags.STATIC) != 0) return false;
return HandlerUtil.shouldMakeFinal0(accessors, field.getAst());
}
public static JCExpression cloneSelfType(JavacNode childOfType) {
```

```
JavacNode typeNode = childOfType;
JavacTreeMaker maker = childOfType.getTreeMaker();
while (typeNode != null && typeNode.getKind() != Kind.TYPE) typeNode = typeNode.up();
return JavacHandlerUtil.namePlusTypeParamsToTypeReference(maker, typeNode, ((JCClassDecl)
typeNode.get()).typarams);
}
public static boolean isBoolean(JavacNode field) {
JCExpression varType = ((JCVariableDecl) field.get()).vartype;
return isBoolean(varType);
}
public static boolean isBoolean(JCExpression varType) {
return varType != null && varType.toString().equals("boolean");
}
public static Name removePrefixFromField(JavacNode field) {
java.util.List<String> prefixes = null;
for (JavacNode node : field.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
AnnotationValues<Accessors> ann = createAnnotation(Accessors.class, node);
if (ann.isExplicit("prefix")) prefixes = Arrays.asList(ann.getInstance().prefix());
break;
}
}
if (prefixes == null) {
JavacNode current = field.up();
outer:
while (current != null) {
for (JavacNode node : current.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
AnnotationValues<Accessors> ann = createAnnotation(Accessors.class, node);
if (ann.isExplicit("prefix")) prefixes = Arrays.asList(ann.getInstance().prefix());
break outer;
}
}
current = current.up();
}
}
if (prefixes == null) prefixes =
field.getAst().readConfiguration(ConfigurationKeys.ACCESSORS_PREFIX);
if (!prefixes.isEmpty()) {
CharSequence newName = removePrefix(field.getName(), prefixes);
if (newName != null) return field.toName(newName.toString());
}
return ((JCVariableDecl) field.get()).name;
}
public static AnnotationValues<Accessors> getAccessorsForField(JavacNode field) {
AnnotationValues<Accessors> values = null;
for (JavacNode node : field.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
values = createAnnotation(Accessors.class, node);
break;
}
}
JavacNode current = field.up();
```

```
while (current != null) {
for (JavacNode node : current.down()) {
if (annotationTypeMatches(Accessors.class, node)) {
AnnotationValues<Accessors> onType = createAnnotation(Accessors.class, node);
values = values == null ? onType : values.integrate(onType);
break;
}
}
current = current.up();
}
return values == null ? AnnotationValues.of(Accessors.class, field) : values;
}
/**
* Checks if there is a field with the provided name.
*
* @param fieldName the field name to check for.
* @param node Any node that represents the Type (JCClassDecl) to look in, or any child node thereof.
*/
public static MemberExistsResult fieldExists(String fieldName, JavacNode node) {
node = upToTypeNode(node);
if (node != null && node.get() instanceof JCClassDecl) {
for (JCTree def : ((JCClassDecl)node.get()).defs) {
if (def instanceof JCVariableDecl) {
if (((JCVariableDecl)def).name.contentEquals(fieldName)) {
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
}
}
return MemberExistsResult.NOT_EXISTS;
}
public static MemberExistsResult methodExists(String methodName, JavacNode node, int params) {
return methodExists(methodName, node, true, params);
}
/**
* Checks if there is a method with the provided name. In case of multiple methods (overloading), only
* the first method decides if EXISTS_BY_USER or EXISTS_BY_LOMBOK is returned.
*
* @param methodName the method name to check for.
* @param node Any node that represents the Type (JCClassDecl) to look in, or any child node thereof.
* @param caseSensitive If the search should be case sensitive.
* @param params The number of parameters the method should have; varargs count as 0-*. Set to -1
to find any method with the appropriate name regardless of parameter count.
*/
public static MemberExistsResult methodExists(String methodName, JavacNode node, boolean
caseSensitive, int params) {
node = upToTypeNode(node);
if (node != null && node.get() instanceof JCClassDecl) {
top: for (JCTree def : ((JCClassDecl)node.get()).defs) {
if (def instanceof JCMethodDecl) {
JCMethodDecl md = (JCMethodDecl) def;
String name = md.name.toString();
```

```
boolean matches = caseSensitive ? name.equals(methodName) :
name.equalsIgnoreCase(methodName);
if (matches) {
if (params > -1) {
List<JCVariableDecl> ps = md.params;
int minArgs = 0;
int maxArgs = 0;
if (ps != null && ps.length() > 0) {
minArgs = ps.length();
if ((ps.last().mods.flags & Flags.VARARGS) != 0) {
maxArgs = Integer.MAX_VALUE;
minArgs--;
} else {
maxArgs = minArgs;
}
}
if (params < minArgs || params > maxArgs) continue;
}
if (isTolerate(node, md)) continue top;
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
}
}
return MemberExistsResult.NOT_EXISTS;
}
public static boolean isTolerate(JavacNode node, JCTree.JCMethodDecl md) {
List<JCAnnotation> annotations = md.getModifiers().getAnnotations();
if (annotations != null) for (JCTree.JCAnnotation anno : annotations) {
if (typeMatches(Tolerate.class, node, anno.getAnnotationType())) return true;
}
return false;
}
/**
* Checks if there is a (non-default) constructor. In case of multiple constructors (overloading), only
* the first constructor decides if EXISTS_BY_USER or EXISTS_BY_LOMBOK is returned.
*
* @param node Any node that represents the Type (JCClassDecl) to look in, or any child node thereof.
*/
public static MemberExistsResult constructorExists(JavacNode node) {
node = upToTypeNode(node);
if (node != null && node.get() instanceof JCClassDecl) {
for (JCTree def : ((JCClassDecl) node.get()).defs) {
if (def instanceof JCMethodDecl) {
JCMethodDecl md = (JCMethodDecl) def;
if (md.name.contentEquals("<init>")) {
if ((md.mods.flags & Flags.GENERATEDCONSTR) != 0) continue;
if (isTolerate(node, md)) continue;
return getGeneratedBy(def) == null ? MemberExistsResult.EXISTS_BY_USER :
MemberExistsResult.EXISTS_BY_LOMBOK;
}
}
```

```
}
}
return MemberExistsResult.NOT_EXISTS;
}
public static boolean isConstructorCall(final JCStatement statement) {
if (!(statement instanceof JCExpressionStatement)) return false;
JCExpression expr = ((JCExpressionStatement) statement).expr;
if (!(expr instanceof JCMethodInvocation)) return false;
JCExpression invocation = ((JCMethodInvocation) expr).meth;
String name;
if (invocation instanceof JCFieldAccess) {
name = ((JCFieldAccess) invocation).name.toString();
} else if (invocation instanceof JCIdent) {
name = ((JCIdent) invocation).name.toString();
} else {
name = "";
}
return "super".equals(name) || "this".equals(name);
}
/**
* Turns an {@code AccessLevel} instance into the flag bit used by javac.
*/
public static int toJavacModifier(AccessLevel accessLevel) {
switch (accessLevel) {
case MODULE:
case PACKAGE:
return 0;
default:
case PUBLIC:
return Flags.PUBLIC;
case NONE:
case PRIVATE:
return Flags.PRIVATE;
case PROTECTED:
return Flags.PROTECTED;
}
}
private static class GetterMethod {
private final Name name;
private final JCExpression type;
GetterMethod(Name name, JCExpression type) {
this.name = name;
this.type = type;
}
}
private static GetterMethod findGetter(JavacNode field) {
JCVariableDecl decl = (JCVariableDecl)field.get();
JavacNode typeNode = field.up();
for (String potentialGetterName : toAllGetterNames(field)) {
for (JavacNode potentialGetter : typeNode.down()) {
if (potentialGetter.getKind() != Kind.METHOD) continue;
JCMethodDecl method = (JCMethodDecl) potentialGetter.get();
if (!method.name.toString().equalsIgnoreCase(potentialGetterName)) continue;
```

```
/** static getX() methods don't count. */
if ((method.mods.flags & Flags.STATIC) != 0) continue;
/** Nor do getters with a non-empty parameter list. */
if (method.params != null && method.params.size() > 0) continue;
return new GetterMethod(method.name, method.restype);
}
}
// Check if the field has a @Getter annotation.
boolean hasGetterAnnotation = false;
for (JavacNode child : field.down()) {
if (child.getKind() == Kind.ANNOTATION && annotationTypeMatches(Getter.class, child)) {
AnnotationValues<Getter> ann = createAnnotation(Getter.class, child);
if (ann.getInstance().value() == AccessLevel.NONE) return null; //Definitely WONT have a getter.
hasGetterAnnotation = true;
}
}
// Check if the class has a @Getter annotation.
if (!hasGetterAnnotation && HandleGetter.fieldQualifiesForGetterGeneration(field)) {
//Check if the class has @Getter or @Data annotation.
JavacNode containingType = field.up();
if (containingType != null) for (JavacNode child : containingType.down()) {
if (child.getKind() == Kind.ANNOTATION && annotationTypeMatches(Data.class, child))
hasGetterAnnotation = true;
if (child.getKind() == Kind.ANNOTATION && annotationTypeMatches(Getter.class, child)) {
AnnotationValues<Getter> ann = createAnnotation(Getter.class, child);
if (ann.getInstance().value() == AccessLevel.NONE) return null; //Definitely WONT have a getter.
hasGetterAnnotation = true;
}
}
}
if (hasGetterAnnotation) {
String getterName = toGetterName(field);
if (getterName == null) return null;
return new GetterMethod(field.toName(getterName), decl.vartype);
}
return null;
}
static boolean lookForGetter(JavacNode field, FieldAccess fieldAccess) {
if (fieldAccess == FieldAccess.GETTER) return true;
if (fieldAccess == FieldAccess.ALWAYS_FIELD) return false;
// If @Getter(lazy = true) is used, then using it is mandatory.
for (JavacNode child : field.down()) {
if (child.getKind() != Kind.ANNOTATION) continue;
if (annotationTypeMatches(Getter.class, child)) {
AnnotationValues<Getter> ann = createAnnotation(Getter.class, child);
if (ann.getInstance().lazy()) return true;
}
}
return false;
}
/**
* Returns the type of the field, unless a getter exists for this field, in which case the return type of the
getter is returned.
```

```
*
* @see #createFieldAccessor(TreeMaker, JavacNode, FieldAccess)
*/
static JCExpression getFieldType(JavacNode field, FieldAccess fieldAccess) {
if (field.getKind() == Kind.METHOD) return ((JCMethodDecl) field.get()).restype;
boolean lookForGetter = lookForGetter(field, fieldAccess);
GetterMethod getter = lookForGetter ? findGetter(field) : null;
if (getter == null) {
return ((JCVariableDecl) field.get()).vartype;
}
return getter.type;
}
/**
* Creates an expression that reads the field. Will either be {@code this.field} or {@code this.getField()}
depending on whether or not there's a getter.
*/
static JCExpression createFieldAccessor(JavacTreeMaker maker, JavacNode field, FieldAccess
fieldAccess) {
return createFieldAccessor(maker, field, fieldAccess, null);
}
static JCExpression createFieldAccessor(JavacTreeMaker maker, JavacNode field, FieldAccess
fieldAccess, JCExpression receiver) {
boolean lookForGetter = lookForGetter(field, fieldAccess);
GetterMethod getter = lookForGetter ? findGetter(field) : null;
JCVariableDecl fieldDecl = (JCVariableDecl) field.get();
if (getter == null) {
if (receiver == null) {
if ((fieldDecl.mods.flags & Flags.STATIC) == 0) {
receiver = maker.Ident(field.toName("this"));
} else {
JavacNode containerNode = field.up();
if (containerNode != null && containerNode.get() instanceof JCClassDecl) {
JCClassDecl container = (JCClassDecl) field.up().get();
receiver = maker.Ident(container.name);
}
}
}
return receiver == null ? maker.Ident(fieldDecl.name) : maker.Select(receiver, fieldDecl.name);
}
if (receiver == null) receiver = maker.Ident(field.toName("this"));
JCMethodInvocation call = maker.Apply(List.<JCExpression>nil(),
maker.Select(receiver, getter.name), List.<JCExpression>nil());
return call;
}
static JCExpression createMethodAccessor(JavacTreeMaker maker, JavacNode method) {
return createMethodAccessor(maker, method, null);
}
static JCExpression createMethodAccessor(JavacTreeMaker maker, JavacNode method,
JCExpression receiver) {
JCMethodDecl methodDecl = (JCMethodDecl) method.get();
if (receiver == null && (methodDecl.mods.flags & Flags.STATIC) == 0) {
receiver = maker.Ident(method.toName("this"));
} else if (receiver == null) {
```

```
JavacNode containerNode = method.up();
if (containerNode != null && containerNode.get() instanceof JCClassDecl) {
JCClassDecl container = (JCClassDecl) method.up().get();
receiver = maker.Ident(container.name);
}
}
JCMethodInvocation call = maker.Apply(List.<JCExpression>nil(),
receiver == null ? maker.Ident(methodDecl.name) : maker.Select(receiver, methodDecl.name),
List.<JCExpression>nil());
return call;
}
/**
* Adds the given new field declaration to the provided type AST Node.
* The field carries the &#64;{@link SuppressWarnings}("all") annotation.
* Also takes care of updating the JavacAST.
*/
public static JavacNode injectFieldAndMarkGenerated(JavacNode typeNode, JCVariableDecl field) {
return injectField(typeNode, field, true);
}
/**
* Adds the given new field declaration to the provided type AST Node.
*
* Also takes care of updating the JavacAST.
*/
public static JavacNode injectField(JavacNode typeNode, JCVariableDecl field) {
return injectField(typeNode, field, false);
}
public static JavacNode injectField(JavacNode typeNode, JCVariableDecl field, boolean
addGenerated) {
return injectField(typeNode, field, addGenerated, false);
}
public static JavacNode injectField(JavacNode typeNode, JCVariableDecl field, boolean
addGenerated, boolean specialEnumHandling) {
JCClassDecl type = (JCClassDecl) typeNode.get();
if (addGenerated) {
addSuppressWarningsAll(field.mods, typeNode, typeNode.getNodeFor(getGeneratedBy(field)),
typeNode.getContext());
addGenerated(field.mods, typeNode, typeNode.getNodeFor(getGeneratedBy(field)),
typeNode.getContext());
}
List<JCTree> insertAfter = null;
List<JCTree> insertBefore = type.defs;
while (true) {
boolean skip = false;
if (insertBefore.head instanceof JCVariableDecl) {
JCVariableDecl f = (JCVariableDecl) insertBefore.head;
if ((!specialEnumHandling && isEnumConstant(f)) || isGenerated(f)) skip = true;
} else if (insertBefore.head instanceof JCMethodDecl) {
if ((((JCMethodDecl) insertBefore.head).mods.flags & GENERATEDCONSTR) != 0) skip = true;
}
if (skip) {
insertAfter = insertBefore;
insertBefore = insertBefore.tail;
```

```
continue;
}
break;
}
List<JCTree> fieldEntry = List.<JCTree>of(field);
fieldEntry.tail = insertBefore;
if (insertAfter == null) {
type.defs = fieldEntry;
} else {
insertAfter.tail = fieldEntry;
}
EnterReflect.memberEnter(field, typeNode);
return typeNode.add(field, Kind.FIELD);
}
public static boolean isEnumConstant(final JCVariableDecl field) {
return (field.mods.flags & Flags.ENUM) != 0;
}
static class JCAnnotatedTypeReflect {
private static final Class<?> TYPE;
private static final Constructor<?> CONSTRUCTOR;
private static final Field ANNOTATIONS, UNDERLYING_TYPE;
static {
Class<?> t = null;
Constructor<?> c = null;
Field a = null;
Field u = null;
try {
t = Class.forName("com.sun.tools.javac.tree.JCTree$JCAnnotatedType");
c = Permit.getConstructor(t, List.class, JCExpression.class);
a = Permit.permissiveGetField(t, "annotations");
u = Permit.permissiveGetField(t, "underlyingType");
} catch (Exception e) {
// Ignore
}
TYPE = t;
CONSTRUCTOR = c;
ANNOTATIONS = a;
UNDERLYING_TYPE = u;
}
static boolean is(JCTree obj) {
if (obj == null) return false;
return obj.getClass() == TYPE;
}
@SuppressWarnings("unchecked")
static List<JCAnnotation> getAnnotations(JCTree obj) {
if (ANNOTATIONS == null) return List.nil();
try {
return (List<JCAnnotation>) ANNOTATIONS.get(obj);
} catch (Exception e) {
return List.nil();
}
}
static void setAnnotations(JCTree obj, List<JCAnnotation> anns) {
```

```
if (ANNOTATIONS == null) return;
try {
ANNOTATIONS.set(obj, anns);
} catch (Exception e) {
// Ignore
}
}
static JCExpression getUnderlyingType(JCTree obj) {
if (UNDERLYING_TYPE == null) return null;
try {
return (JCExpression) UNDERLYING_TYPE.get(obj);
} catch (Exception e) {
return null;
}
}
static JCExpression create(List<JCAnnotation> annotations, JCExpression underlyingType) {
if (CONSTRUCTOR == null) return underlyingType;
try {
return (JCExpression) CONSTRUCTOR.newInstance(annotations, underlyingType);
} catch (Exception e) {
return underlyingType;
}
}
}
static class JCAnnotationReflect {
private static final Field ATTRIBUTE;
static {
ATTRIBUTE = Permit.permissiveGetField(JCAnnotation.class, "attribute");
}
static Attribute.Compound getAttribute(JCAnnotation jcAnnotation) {
if (ATTRIBUTE != null) {
try {
return (Attribute.Compound) ATTRIBUTE.get(jcAnnotation);
} catch (Exception e) {
// Ignore
}
}
return null;
}
static void setAttribute(JCAnnotation jcAnnotation, Attribute.Compound attribute) {
if (ATTRIBUTE != null) {
try {
Permit.set(ATTRIBUTE, jcAnnotation, attribute);
} catch (Exception e) {
// Ignore
}
}
}
}
// jdk9 support, types have changed, names stay the same
static class ClassSymbolMembersField {
private static final Field membersField;
private static final Method removeMethod;
```

```
private static final Method enterMethod;
static {
Field f = null;
Method r = null;
Method e = null;
try {
f = Permit.getField(ClassSymbol.class, "members_field");
r = Permit.getMethod(f.getType(), "remove", Symbol.class);
e = Permit.getMethod(f.getType(), "enter", Symbol.class);
} catch (Exception ex) {}
membersField = f;
removeMethod = r;
enterMethod = e;
}
static void remove(ClassSymbol from, Symbol toRemove) {
if (from == null) return;
try {
Scope scope = (Scope) membersField.get(from);
if (scope == null) return;
Permit.invoke(removeMethod, scope, toRemove);
} catch (Exception e) {}
}
static void enter(ClassSymbol from, Symbol toEnter) {
if (from == null) return;
try {
Scope scope = (Scope) membersField.get(from);
if (scope == null) return;
Permit.invoke(enterMethod, scope, toEnter);
} catch (Exception e) {}
}
}
/**
* Adds the given new method declaration to the provided type AST Node.
* Can also inject constructors.
*
* Also takes care of updating the JavacAST.
*/
public static void injectMethod(JavacNode typeNode, JCMethodDecl method) {
JavacNode source = typeNode.getNodeFor(getGeneratedBy(method));
injectMethod(typeNode, source, method);
}
/**
* Adds the given new method declaration to the provided type AST Node.
* Can also inject constructors.
*
* Also takes care of updating the JavacAST.
*
* Ordinarily the 'source' is determined automatically. In rare cases generally involving combinations between
* annotations, some of which require re-attribution such as {@code @Delegate}, that doesn't work. This method
* exists if you explicitly have the source.
*/
```

```
public static void injectMethod(JavacNode typeNode, JavacNode source, JCMethodDecl method) {
JCClassDecl type = (JCClassDecl) typeNode.get();
if (method.getName().contentEquals("<init>")) {
//Scan for default constructor, and remove it.
int idx = 0;
for (JCTree def : type.defs) {
if (def instanceof JCMethodDecl) {
if ((((JCMethodDecl) def).mods.flags & Flags.GENERATEDCONSTR) != 0) {
JavacNode tossMe = typeNode.getNodeFor(def);
if (tossMe != null) tossMe.up().removeChild(tossMe);
type.defs = addAllButOne(type.defs, idx);
ClassSymbolMembersField.remove(type.sym, ((JCMethodDecl) def).sym);
break;
}
}
idx++;
}
}
addSuppressWarningsAll(method.mods, typeNode, typeNode.getNodeFor(getGeneratedBy(method)),
typeNode.getContext());
addGenerated(method.mods, typeNode, source, typeNode.getContext());
type.defs = type.defs.append(method);
EnterReflect.memberEnter(method, typeNode);
typeNode.add(method, Kind.METHOD);
}
/**
* Adds an inner type (class, interface, enum) to the given type. Cannot inject top-level types.
*
* @param typeNode parent type to inject new type into
* @param type New type (class, interface, etc) to inject.
* @return
*/
public static JavacNode injectType(JavacNode typeNode, final JCClassDecl type) {
JCClassDecl typeDecl = (JCClassDecl) typeNode.get();
addSuppressWarningsAll(type.mods, typeNode, typeNode.getNodeFor(getGeneratedBy(type)),
typeNode.getContext());
addGenerated(type.mods, typeNode, typeNode.getNodeFor(getGeneratedBy(type)),
typeNode.getContext());
typeDecl.defs = typeDecl.defs.append(type);
EnterReflect.classEnter(type, typeNode);
return typeNode.add(type, Kind.TYPE);
}
static class EnterReflect {
private static final Method classEnter;
private static final Method memberEnter;
private static final Method blockAnnotations;
private static final Method unblockAnnotations;
static {
classEnter = Permit.permissiveGetMethod(Enter.class, "classEnter", JCTree.class, Env.class);
memberEnter = Permit.permissiveGetMethod(MemberEnter.class, "memberEnter", JCTree.class,
Env.class);
Method block = Permit.permissiveGetMethod(Annotate.class, "blockAnnotations");
if (block == null) block = Permit.permissiveGetMethod(Annotate.class, "enterStart");
```

```
blockAnnotations = block;
Method unblock = Permit.permissiveGetMethod(Annotate.class, "unblockAnnotations");
if (unblock == null) unblock = Permit.permissiveGetMethod(Annotate.class, "enterDone");
unblockAnnotations = unblock;
}
static Type classEnter(JCTree tree, JavacNode parent) {
Enter enter = Enter.instance(parent.getContext());
Env<AttrContext> classEnv = enter.getEnv((TypeSymbol) parent.getElement());
if (classEnv == null) return null;
Type type = (Type) Permit.invokeSneaky(classEnter, enter, tree, classEnv);
if (type == null) return null;
type.complete();
return type;
}
static void memberEnter(JCTree tree, JavacNode parent) {
Context context = parent.getContext();
MemberEnter me = MemberEnter.instance(context);
Annotate annotate = Annotate.instance(context);
Enter enter = Enter.instance(context);
Env<AttrContext> classEnv = enter.getEnv((TypeSymbol) parent.getElement());
if (classEnv == null) return;
Permit.invokeSneaky(blockAnnotations, annotate);
Permit.invokeSneaky(memberEnter, me, tree, classEnv);
Permit.invokeSneaky(unblockAnnotations, annotate);
}
}
public static long addFinalIfNeeded(long flags, Context context) {
boolean addFinal =
LombokOptionsFactory.getDelombokOptions(context).getFormatPreferences().generateFinalParams();
if (addFinal) flags |= Flags.FINAL;
return flags;
}
public static JCExpression genTypeRef(JavacNode node, String complexName) {
String[] parts = complexName.split("\\.");
if (parts.length > 2 && parts[0].equals("java") && parts[1].equals("lang")) {
String[] subParts = new String[parts.length - 2];
System.arraycopy(parts, 2, subParts, 0, subParts.length);
return genJavaLangTypeRef(node, subParts);
}
return chainDots(node, parts);
}
public static JCExpression genJavaLangTypeRef(JavacNode node, String... simpleNames) {
if (LombokOptionsFactory.getDelombokOptions(node.getContext()).getFormatPreferences().javaLang
AsFqn()) {
return chainDots(node, "java", "lang", simpleNames);
} else {
return chainDots(node, null, null, simpleNames);
}
}
public static JCExpression genJavaLangTypeRef(JavacNode node, int pos, String... simpleNames) {
if (LombokOptionsFactory.getDelombokOptions(node.getContext()).getFormatPreferences().javaLang
AsFqn()) {
return chainDots(node, pos, "java", "lang", simpleNames);
```

```
} else {
return chainDots(node, pos, null, null, simpleNames);
}
}
public static void addSuppressWarningsAll(JCModifiers mods, JavacNode node, JavacNode source,
Context context) {
if (!LombokOptionsFactory.getDelombokOptions(context).getFormatPreferences().generateSuppressW
arnings()) return;
boolean addJLSuppress = !Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKey
s.ADD_SUPPRESSWARNINGS_ANNOTATIONS));
if (addJLSuppress) {
for (JCAnnotation ann : mods.annotations) {
JCTree type = ann.getAnnotationType();
Name n = null;
if (type instanceof JCIdent) n = ((JCIdent) type).name;
else if (type instanceof JCFieldAccess) n = ((JCFieldAccess) type).name;
if (n != null && n.contentEquals("SuppressWarnings")) {
addJLSuppress = false;
}
}
}
if (addJLSuppress) addAnnotation(mods, node, source, "java.lang.SuppressWarnings",
node.getTreeMaker().Literal("all"));
if (Boolean.TRUE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_FINDBUGS_SUPP
RESSWARNINGS_ANNOTATIONS))) {
JavacTreeMaker maker = node.getTreeMaker();
JCExpression arg = maker.Assign(maker.Ident(node.toName("justification")), maker.Literal("generated
code"));
addAnnotation(mods, node, source, "edu.umd.cs.findbugs.annotations.SuppressFBWarnings", arg);
}
}
public static void addGenerated(JCModifiers mods, JavacNode node, JavacNode source, Context
context) {
if
(!LombokOptionsFactory.getDelombokOptions(context).getFormatPreferences().generateGenerated())
return;
if (HandlerUtil.shouldAddGenerated(node)) {
addAnnotation(mods, node, source, "javax.annotation.Generated",
node.getTreeMaker().Literal("lombok"));
}
if (Boolean.TRUE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_JAKARTA_GENE
RATED_ANNOTATIONS))) {
addAnnotation(mods, node, source, "jakarta.annotation.Generated",
node.getTreeMaker().Literal("lombok"));
}
if (!Boolean.FALSE.equals(node.getAst().readConfiguration(ConfigurationKeys.ADD_LOMBOK_GENE
RATED_ANNOTATIONS))) {
addAnnotation(mods, node, source, "lombok.Generated", null);
}
}
public static void addAnnotation(JCModifiers mods, JavacNode node, JavacNode source, String
annotationTypeFqn, JCExpression arg) {
boolean isJavaLangBased;
```

```
String simpleName; {
int idx = annotationTypeFqn.lastIndexOf('.');
simpleName = idx == -1 ? annotationTypeFqn : annotationTypeFqn.substring(idx + 1);
isJavaLangBased = idx == 9 && annotationTypeFqn.regionMatches(0, "java.lang.", 0, 10);
}
for (JCAnnotation ann : mods.annotations) {
JCTree annType = ann.getAnnotationType();
if (annType instanceof JCIdent) {
Name lastPart = ((JCIdent) annType).name;
if (lastPart.contentEquals(simpleName)) return;
}
if (annType instanceof JCFieldAccess) {
if (annType.toString().equals(annotationTypeFqn)) return;
}
}
JavacTreeMaker maker = node.getTreeMaker();
JCExpression annType = isJavaLangBased ? genJavaLangTypeRef(node, simpleName) :
chainDotsString(node, annotationTypeFqn);
List<JCExpression> argList = arg != null ? List.of(arg) : List.<JCExpression>nil();
JCAnnotation annotation = recursiveSetGeneratedBy(maker.Annotation(annType, argList), source);
mods.annotations = mods.annotations.append(annotation);
}
static JCExpression addCheckerFrameworkReturnsReceiver(JCExpression returnType,
JavacTreeMaker maker, JavacNode typeNode, CheckerFrameworkVersion cfv) {
if (cfv.generateReturnsReceiver()) {
JCAnnotation rrAnnotation = maker.Annotation(genTypeRef(typeNode,
CheckerFrameworkVersion.NAME__RETURNS_RECEIVER), List.<JCExpression>nil());
returnType = maker.AnnotatedType(List.of(rrAnnotation), returnType);
}
return returnType;
}
private static List<JCTree> addAllButOne(List<JCTree> defs, int idx) {
ListBuffer<JCTree> out = new ListBuffer<JCTree>();
int i = 0;
for (JCTree def : defs) {
if (i++ != idx) out.append(def);
}
return out.toList();
}
/**
* In javac, dotted access of any kind, from {@code java.lang.String} to {@code var.methodName}
* is represented by a fold-left of {@code Select} nodes with the leftmost string represented by
* a {@code Ident} node. This method generates such an expression.
* <p>
* The position of the generated node(s) will be unpositioned (-1).
*
* For example, maker.Select(maker.Select(maker.Ident(NAME[java]), NAME[lang]), NAME[String]).
*
* @see com.sun.tools.javac.tree.JCTree.JCIdent
* @see com.sun.tools.javac.tree.JCTree.JCFieldAccess
*/
public static JCExpression chainDots(JavacNode node, String elem1, String elem2, String... elems) {
return chainDots(node, -1, elem1, elem2, elems);
```

```
}
public static JCExpression chainDots(JavacNode node, String[] elems) {
return chainDots(node, -1, null, null, elems);
}
public static JCExpression chainDots(JavacNode node, LombokImmutableList<String> elems) {
assert elems != null;
JavacTreeMaker maker = node.getTreeMaker();
JCExpression e = null;
for (String elem : elems) {
if (e == null) e = maker.Ident(node.toName(elem));
else e = maker.Select(e, node.toName(elem));
}
return e;
}
/**
* In javac, dotted access of any kind, from {@code java.lang.String} to {@code var.methodName}
* is represented by a fold-left of {@code Select} nodes with the leftmost string represented by
* a {@code Ident} node. This method generates such an expression.
* <p>
* The position of the generated node(s) will be equal to the {@code pos} parameter.
*
* For example, maker.Select(maker.Select(maker.Ident(NAME[java]), NAME[lang]), NAME[String]).
*
* @see com.sun.tools.javac.tree.JCTree.JCIdent
* @see com.sun.tools.javac.tree.JCTree.JCFieldAccess
*/
public static JCExpression chainDots(JavacNode node, int pos, String elem1, String elem2, String...
elems) {
assert elems != null;
JavacTreeMaker maker = node.getTreeMaker();
if (pos != -1) maker = maker.at(pos);
JCExpression e = null;
if (elem1 != null) e = maker.Ident(node.toName(elem1));
if (elem2 != null) e = e == null ? maker.Ident(node.toName(elem2)) : maker.Select(e,
node.toName(elem2));
for (int i = 0 ; i < elems.length ; i++) {
e = e == null ? maker.Ident(node.toName(elems[i])) : maker.Select(e, node.toName(elems[i]));
}
assert e != null;
return e;
}
/**
* In javac, dotted access of any kind, from {@code java.lang.String} to {@code var.methodName}
* is represented by a fold-left of {@code Select} nodes with the leftmost string represented by
* a {@code Ident} node. This method generates such an expression.
*
* For example, maker.Select(maker.Select(maker.Ident(NAME[java]), NAME[lang]), NAME[String]).
*
* @see com.sun.tools.javac.tree.JCTree.JCIdent
* @see com.sun.tools.javac.tree.JCTree.JCFieldAccess
*/
public static JCExpression chainDotsString(JavacNode node, String elems) {
return chainDots(node, null, null, elems.split("\\."));
```

```
}
/**
* Searches the given field node for annotations and returns each one that matches the provided regular
expression pattern.
*
* Only the simple name is checked - the package and any containing class are ignored.
*/
public static List<JCAnnotation> findAnnotations(JavacNode fieldNode, Pattern namePattern) {
ListBuffer<JCAnnotation> result = new ListBuffer<JCAnnotation>();
for (JavacNode child : fieldNode.down()) {
if (child.getKind() == Kind.ANNOTATION) {
JCAnnotation annotation = (JCAnnotation) child.get();
String name = annotation.annotationType.toString();
int idx = name.lastIndexOf(".");
String suspect = idx == -1 ? name : name.substring(idx + 1);
if (namePattern.matcher(suspect).matches()) {
result.append(annotation);
}
}
}
return result.toList();
}
public static String scanForNearestAnnotation(JavacNode node, String... anns) {
while (node != null) {
for (JavacNode ann : node.down()) {
if (ann.getKind() != Kind.ANNOTATION) continue;
JCAnnotation a = (JCAnnotation) ann.get();
for (String annToFind : anns) if (typeMatches(annToFind, node, a.annotationType)) return annToFind;
}
node = node.up();
}
return null;
}
public static boolean hasNonNullAnnotations(JavacNode node) {
for (JavacNode child : node.down()) {
if (child.getKind() == Kind.ANNOTATION) {
JCAnnotation annotation = (JCAnnotation) child.get();
String annotationTypeName = getTypeName(annotation.annotationType);
for (String nn : NONNULL_ANNOTATIONS) if (typeMatches(nn, node, annotationTypeName)) return
true;
}
}
return false;
}
public static boolean hasNonNullAnnotations(JavacNode node, List<JCAnnotation> anns) {
if (anns == null) return false;
for (JCAnnotation ann : anns) {
String annotationTypeName = getTypeName(ann.annotationType);
for (String nn : NONNULL_ANNOTATIONS) if (typeMatches(nn, node, annotationTypeName)) return
true;
}
return false;
}
```

```
/**
* Searches the given field node for annotations and returns each one that is 'copyable' (either via
configuration or from the base list).
*/
public static List<JCAnnotation> findCopyableAnnotations(JavacNode node) {
java.util.List<TypeName> configuredCopyable =
node.getAst().readConfiguration(ConfigurationKeys.COPYABLE_ANNOTATIONS);
ListBuffer<JCAnnotation> result = new ListBuffer<JCAnnotation>();
for (JavacNode child : node.down()) {
if (child.getKind() == Kind.ANNOTATION) {
JCAnnotation annotation = (JCAnnotation) child.get();
String annotationTypeName = getTypeName(annotation.annotationType);
boolean match = false;
for (TypeName cn : configuredCopyable) if (cn != null && typeMatches(cn.toString(), node,
annotationTypeName)) {
result.append(annotation);
match = true;
break;
}
if (!match) for (String bn : BASE_COPYABLE_ANNOTATIONS) if (typeMatches(bn, node,
annotationTypeName)) {
result.append(annotation);
break;
}
}
}
return copyAnnotations(result.toList());
}
/**
* Searches the given field node for annotations that are specifically intentioned to be copied to the
setter.
*/
public static List<JCAnnotation> findCopyableToSetterAnnotations(JavacNode node) {
return findAnnotationsInList(node, COPY_TO_SETTER_ANNOTATIONS);
}
/**
* Searches the given field node for annotations that are specifically intentioned to be copied to the
builder's singular method.
*/
public static List<JCAnnotation> findCopyableToBuilderSingularSetterAnnotations(JavacNode node) {
return findAnnotationsInList(node, COPY_TO_BUILDER_SINGULAR_SETTER_ANNOTATIONS);
}
/**
* Searches the given field node for annotations that are in the given list, and returns those.
*/
private static List<JCAnnotation> findAnnotationsInList(JavacNode node, java.util.List<String>
annotationsToFind) {
JCAnnotation anno = null;
String annoName = null;
for (JavacNode child : node.down()) {
if (child.getKind() == Kind.ANNOTATION) {
if (anno != null) {
annoName = "";
```

```
break;
}
JCAnnotation annotation = (JCAnnotation) child.get();
annoName = annotation.annotationType.toString();
anno = annotation;
}
}
if (annoName == null) return List.nil();
if (!annoName.isEmpty()) {
for (String bn : annotationsToFind) if (typeMatches(bn, node, annoName)) return List.of(anno);
}
ListBuffer<JCAnnotation> result = new ListBuffer<JCAnnotation>();
for (JavacNode child : node.down()) {
if (child.getKind() == Kind.ANNOTATION) {
JCAnnotation annotation = (JCAnnotation) child.get();
String annotationTypeName = getTypeName(annotation.annotationType);
boolean match = false;
if (!match) for (String bn : annotationsToFind) if (typeMatches(bn, node, annotationTypeName)) {
result.append(annotation);
break;
}
}
}
return copyAnnotations(result.toList());
}
/**
* Generates a new statement that checks if the given variable is null, and if so, throws a configured
exception with the
* variable name as message.
*/
public static JCStatement generateNullCheck(JavacTreeMaker maker, JavacNode variable,
JavacNode source) {
return generateNullCheck(maker, (JCVariableDecl) variable.get(), source);
}
/**
* Generates a new statement that checks if the given local is null, and if so, throws a configured
exception with the
* local variable name as message.
*/
public static JCStatement generateNullCheck(JavacTreeMaker maker, JCExpression typeNode, Name
varName, JavacNode source, String customMessage) {
NullCheckExceptionType exceptionType =
source.getAst().readConfiguration(ConfigurationKeys.NON_NULL_EXCEPTION_TYPE);
if (exceptionType == null) exceptionType = NullCheckExceptionType.NULL_POINTER_EXCEPTION;
if (typeNode != null && isPrimitive(typeNode)) return null;
JCLiteral message = maker.Literal(exceptionType.toExceptionMessage(varName.toString(),
customMessage));
LombokImmutableList<String> method = exceptionType.getMethod();
if (method != null) {
return maker.Exec(maker.Apply(List.<JCExpression>nil(), chainDots(source, method),
List.of(maker.Ident(varName), message)));
}
if (exceptionType == NullCheckExceptionType.ASSERTION) {
```

```
return maker.Assert(maker.Binary(CTC_NOT_EQUAL, maker.Ident(varName),
maker.Literal(CTC_BOT, null)), message);
}
JCExpression exType = genTypeRef(source, exceptionType.getExceptionType());
JCExpression exception = maker.NewClass(null, List.<JCExpression>nil(), exType,
List.<JCExpression>of(message), null);
JCStatement throwStatement = maker.Throw(exception);
JCBlock throwBlock = maker.Block(0, List.of(throwStatement));
return maker.If(maker.Binary(CTC_EQUAL, maker.Ident(varName), maker.Literal(CTC_BOT, null)),
throwBlock, null);
}
/**
* Generates a new statement that checks if the given variable is null, and if so, throws a configured
exception with the
* variable name as message.
*
* This is a special case method reserved for use when the provided declaration differs from the
* variable's declaration, i.e. in a constructor or setter where the local parameter is named the same but
with the prefix
* stripped as a result of @Accessors.prefix.
*/
public static JCStatement generateNullCheck(JavacTreeMaker maker, JCVariableDecl varDecl,
JavacNode source) {
return generateNullCheck(maker, varDecl.vartype, varDecl.name, source, null);
}
/**
* Given a list of field names and a node referring to a type, finds each name in the list that does not
match a field within the type.
*/
public static List<Integer> createListOfNonExistentFields(List<String> list, JavacNode type, boolean
excludeStandard, boolean excludeTransient) {
boolean[] matched = new boolean[list.size()];
for (JavacNode child : type.down()) {
if (list.isEmpty()) break;
if (child.getKind() != Kind.FIELD) continue;
JCVariableDecl field = (JCVariableDecl)child.get();
if (excludeStandard) {
if ((field.mods.flags & Flags.STATIC) != 0) continue;
if (field.name.toString().startsWith("$")) continue;
}
if (excludeTransient && (field.mods.flags & Flags.TRANSIENT) != 0) continue;
int idx = list.indexOf(child.getName());
if (idx > -1) matched[idx] = true;
}
ListBuffer<Integer> problematic = new ListBuffer<Integer>();
for (int i = 0 ; i < list.size() ; i++) {
if (!matched[i]) problematic.append(i);
}
return problematic.toList();
}
static List<JCAnnotation> unboxAndRemoveAnnotationParameter(JCAnnotation ast, String
parameterName, String errorName, JavacNode annotationNode) {
ListBuffer<JCExpression> params = new ListBuffer<JCExpression>();
```

```
ListBuffer<JCAnnotation> result = new ListBuffer<JCAnnotation>();
outer:
for (JCExpression param : ast.args) {
boolean allowRaw;
String nameOfParam = "value";
JCExpression valueOfParam = null;
if (param instanceof JCAssign) {
JCAssign assign = (JCAssign) param;
if (assign.lhs instanceof JCIdent) {
JCIdent ident = (JCIdent) assign.lhs;
nameOfParam = ident.name.toString();
}
valueOfParam = assign.rhs;
}
/* strip trailing underscores */ {
int lastIdx;
for (lastIdx = nameOfParam.length() ; lastIdx > 0; lastIdx--) {
if (nameOfParam.charAt(lastIdx - 1) != '_') break;
}
allowRaw = lastIdx < nameOfParam.length();
nameOfParam = nameOfParam.substring(0, lastIdx);
}
if (!parameterName.equals(nameOfParam)) {
params.append(param);
continue outer;
}
int endPos = Javac.getEndPosition(param.pos(), (JCCompilationUnit) annotationNode.top().get());
annotationNode.getAst().removeFromDeferredDiagnostics(param.pos, endPos);
if (valueOfParam instanceof JCAnnotation) {
String dummyAnnotationName = ((JCAnnotation) valueOfParam).annotationType.toString();
dummyAnnotationName = dummyAnnotationName.replace("_", "").replace("$", "").replace("x",
"").replace("X", "");
if (dummyAnnotationName.length() > 0) {
if (allowRaw) {
result.append((JCAnnotation) valueOfParam);
} else {
addError(errorName, annotationNode);
continue outer;
}
} else {
for (JCExpression expr : ((JCAnnotation) valueOfParam).args) {
if (expr instanceof JCAssign && ((JCAssign) expr).lhs instanceof JCIdent) {
JCIdent id = (JCIdent) ((JCAssign) expr).lhs;
if ("value".equals(id.name.toString())) {
expr = ((JCAssign) expr).rhs;
} else {
addError(errorName, annotationNode);
}
}
if (expr instanceof JCAnnotation) {
result.append((JCAnnotation) expr);
} else if (expr instanceof JCNewArray) {
for (JCExpression expr2 : ((JCNewArray) expr).elems) {
```

```
if (expr2 instanceof JCAnnotation) {
result.append((JCAnnotation) expr2);
} else {
addError(errorName, annotationNode);
continue outer;
}
}
} else {
addError(errorName, annotationNode);
continue outer;
}
}
}
} else if (valueOfParam instanceof JCNewArray) {
JCNewArray arr = (JCNewArray) valueOfParam;
if (arr.elems.isEmpty()) {
// Just remove it, this is always fine.
} else if (allowRaw) {
for (JCExpression jce : arr.elems) {
if (jce instanceof JCAnnotation) result.append((JCAnnotation) jce);
else addError(errorName, annotationNode);
}
} else {
addError(errorName, annotationNode);
}
} else {
addError(errorName, annotationNode);
}
}
for (JCAnnotation annotation : result) {
clearTypes(annotation);
}
ast.args = params.toList();
return result.toList();
}
/**
* Removes all type information from the provided tree.
*/
private static void clearTypes(JCTree tree) {
tree.accept(new TreeScanner() {
@Override public void scan(JCTree tree) {
if (tree == null) return;
tree.type = null;
super.scan(tree);
}
@Override public void visitClassDef(JCClassDecl tree) {
tree.sym = null;
super.visitClassDef(tree);
}
@Override public void visitMethodDef(JCMethodDecl tree) {
tree.sym = null;
super.visitMethodDef(tree);
}
```

```
@Override public void visitVarDef(JCVariableDecl tree) {
tree.sym = null;
super.visitVarDef(tree);
}
@Override public void visitSelect(JCFieldAccess tree) {
tree.sym = null;
super.visitSelect(tree);
}
@Override public void visitIdent(JCIdent tree) {
tree.sym = null;
super.visitIdent(tree);
}
@Override public void visitAnnotation(JCAnnotation tree) {
JCAnnotationReflect.setAttribute(tree, null);
super.visitAnnotation(tree);
}
});
}
private static void addError(String errorName, JavacNode node) {
if (node.getLatestJavaSpecSupported() < 8) {
node.addError("The correct format up to JDK7 is " + errorName + "=@__({@SomeAnnotation,
@SomeOtherAnnotation}))");
} else {
node.addError("The correct format for JDK8+ is " + errorName + "_={@SomeAnnotation,
@SomeOtherAnnotation})");
}
}
public static List<JCTypeParameter> copyTypeParams(JavacNode source, List<JCTypeParameter>
params) {
if (params == null || params.isEmpty()) return params;
ListBuffer<JCTypeParameter> out = new ListBuffer<JCTypeParameter>();
JavacTreeMaker maker = source.getTreeMaker();
for (JCTypeParameter tp : params) {
List<JCExpression> bounds = tp.bounds;
if (bounds != null && !bounds.isEmpty()) {
ListBuffer<JCExpression> boundsCopy = new ListBuffer<JCExpression>();
for (JCExpression expr : tp.bounds) {
boundsCopy.append(cloneType(maker, expr, source));
}
bounds = boundsCopy.toList();
}
out.append(maker.TypeParameter(tp.name, bounds));
}
return out.toList();
}
public static List<JCAnnotation> getTypeUseAnnotations(JCExpression from) {
if (!JCAnnotatedTypeReflect.is(from)) return List.nil();
return JCAnnotatedTypeReflect.getAnnotations(from);
}
public static JCExpression removeTypeUseAnnotations(JCExpression from) {
if (!JCAnnotatedTypeReflect.is(from)) return from;
return JCAnnotatedTypeReflect.getUnderlyingType(from);
}
```

```
public static JCExpression namePlusTypeParamsToTypeReference(JavacTreeMaker maker,
JavacNode type, List<JCTypeParameter> params) {
JCClassDecl td = (JCClassDecl) type.get();
boolean instance = !type.isStatic();
return namePlusTypeParamsToTypeReference(maker, type.up(), td.name, instance, params,
List.<JCAnnotation>nil());
}
public static JCExpression namePlusTypeParamsToTypeReference(JavacTreeMaker maker,
JavacNode type, List<JCTypeParameter> params, List<JCAnnotation> annotations) {
JCClassDecl td = (JCClassDecl) type.get();
boolean instance = !type.isStatic();
return namePlusTypeParamsToTypeReference(maker, type.up(), td.name, instance, params,
annotations);
}
public static JCExpression namePlusTypeParamsToTypeReference(JavacTreeMaker maker,
JavacNode parentType, Name typeName, boolean instance, List<JCTypeParameter> params) {
return namePlusTypeParamsToTypeReference(maker, parentType, typeName, instance, params,
List.<JCAnnotation>nil());
}
public static JCExpression namePlusTypeParamsToTypeReference(JavacTreeMaker maker,
JavacNode parentType, Name typeName, boolean instance, List<JCTypeParameter> params,
List<JCAnnotation> annotations) {
JCExpression r = null;
if (parentType != null && parentType.getKind() == Kind.TYPE && !parentType.getName().isEmpty()) {
JCClassDecl td = (JCClassDecl) parentType.get();
boolean outerInstance = instance && !parentType.isStatic();
List<JCTypeParameter> outerParams = instance ? td.typarams : List.<JCTypeParameter>nil();
r = namePlusTypeParamsToTypeReference(maker, parentType.up(), td.name, outerInstance,
outerParams, List.<JCAnnotation>nil());
}
r = r == null ? maker.Ident(typeName) : maker.Select(r, typeName);
if (!annotations.isEmpty()) r = JCAnnotatedTypeReflect.create(annotations, r);
if (!params.isEmpty()) r = maker.TypeApply(r, typeParameterNames(maker, params));
return r;
}
public static List<JCExpression> typeParameterNames(JavacTreeMaker maker,
List<JCTypeParameter> params) {
ListBuffer<JCExpression> typeArgs = new ListBuffer<JCExpression>();
for (JCTypeParameter param : params) {
typeArgs.append(maker.Ident(param.name));
}
return typeArgs.toList();
}
public static void sanityCheckForMethodGeneratingAnnotationsOnBuilderClass(JavacNode typeNode,
JavacNode errorNode) {
List<String> disallowed = List.nil();
for (JavacNode child : typeNode.down()) {
for (String annType : INVALID_ON_BUILDERS) {
if (annotationTypeMatches(annType, child)) {
int lastIndex = annType.lastIndexOf('.');
disallowed = disallowed.append(lastIndex == -1 ? annType : annType.substring(lastIndex + 1));
}
}
```

```
}
int size = disallowed.size();
if (size == 0) return;
if (size == 1) {
errorNode.addError("@" + disallowed.head + " is not allowed on builder classes.");
return;
}
StringBuilder out = new StringBuilder();
for (String a : disallowed) out.append("@").append(a).append(", ");
out.setLength(out.length() - 2);
errorNode.addError(out.append(" are not allowed on builder classes.").toString());
}
static List<JCAnnotation> copyAnnotations(List<? extends JCExpression> in) {
ListBuffer<JCAnnotation> out = new ListBuffer<JCAnnotation>();
for (JCExpression expr : in) {
if (!(expr instanceof JCAnnotation)) continue;
out.append((JCAnnotation) expr.clone());
}
return out.toList();
}
static List<JCAnnotation> mergeAnnotations(List<JCAnnotation> a, List<JCAnnotation> b) {
if (a == null || a.isEmpty()) return b;
if (b == null || b.isEmpty()) return a;
ListBuffer<JCAnnotation> out = new ListBuffer<JCAnnotation>();
for (JCAnnotation ann : a) out.append(ann);
for (JCAnnotation ann : b) out.append(ann);
return out.toList();
}
/**
* Returns {@code true} if the provided node is an actual class and not some other type declaration (so,
not an annotation definition, interface, enum, or record).
*/
public static boolean isClass(JavacNode typeNode) {
return isClassAndDoesNotHaveFlags(typeNode, Flags.INTERFACE | Flags.ENUM |
Flags.ANNOTATION | RECORD);
}
/**
* Returns {@code true} if the provided node is an actual class or enum and not some other type
declaration (so, not an annotation definition, interface, or record).
*/
public static boolean isClassOrEnum(JavacNode typeNode) {
return isClassAndDoesNotHaveFlags(typeNode, Flags.INTERFACE | Flags.ANNOTATION |
RECORD);
}
/**
* Returns {@code true} if the provided node is an actual class an enum or an interface and not some
other type declaration (so, not an annotation definition, or record).
*/
public static boolean isClassEnumOrInterface(JavacNode typeNode) {
return isClassAndDoesNotHaveFlags(typeNode, Flags.ANNOTATION | RECORD);
}
/**
```

```
* Returns {@code true} if the provided node is an actual class, an enum, an interface or a record and
not some other type declaration (so, not an annotation definition).
*/
public static boolean isClassEnumInterfaceOrRecord(JavacNode typeNode) {
return isClassAndDoesNotHaveFlags(typeNode, Flags.ANNOTATION);
}
/**
* Returns {@code true} if the provided node is an actual class, an enum or a record and not some other
type declaration (so, not an annotation definition or interface).
*/
public static boolean isClassEnumOrRecord(JavacNode typeNode) {
return isClassAndDoesNotHaveFlags(typeNode, Flags.INTERFACE | Flags.ANNOTATION);
}
/**
* Returns {@code true} if the provided node is a record declaration (so, not an annotation definition,
interface, enum, or plain class).
*/
public static boolean isRecord(JavacNode typeNode) {
return typeNode.getKind() == Kind.TYPE && (((JCClassDecl) typeNode.get()).mods.flags & RECORD)
!= 0;
}
public static boolean isClassAndDoesNotHaveFlags(JavacNode typeNode, long flags) {
JCClassDecl typeDecl = null;
if (typeNode.get() instanceof JCClassDecl) typeDecl = (JCClassDecl) typeNode.get();
else return false;
long typeDeclflags = typeDecl == null ? 0 : typeDecl.mods.flags;
return (typeDeclflags & flags) == 0;
}
/**
* Returns {@code true} if the provided node supports static methods and types (top level or static class)
*/
public static boolean isStaticAllowed(JavacNode typeNode) {
return typeNode.isStatic() || typeNode.up() == null || typeNode.up().getKind() ==
Kind.COMPILATION_UNIT || isRecord(typeNode);
}
public static JavacNode upToTypeNode(JavacNode node) {
if (node == null) throw new NullPointerException("node");
while ((node != null) && !(node.get() instanceof JCClassDecl)) node = node.up();
return node;
}
public static List<JCExpression> cloneTypes(JavacTreeMaker maker, List<JCExpression> in,
JavacNode source) {
if (in.isEmpty()) return List.nil();
if (in.size() == 1) return List.of(cloneType(maker, in.get(0), source));
ListBuffer<JCExpression> lb = new ListBuffer<JCExpression>();
for (JCExpression expr : in) lb.append(cloneType(maker, expr, source));
return lb.toList();
}
/**
* Creates a full clone of a given javac AST type node. Every part is cloned (every identifier, every
select, every wildcard, every type apply, every type_use annotation).
*
```

```
* If there's any node in the tree that we don't know how to clone, that part isn't cloned. However, we wouldn't know what could possibly show up that we
* can't currently clone; that's just a safeguard.
*
* This should be used if the type looks the same in the code, but resolves differently. For example, a static method that has some generics in it named after
* the class's own parameter, but as its a static method, the static method's notion of {@code T} is different from the class notion of {@code T}. If you're duplicating
* a type used in the class context, you need to use this method.
*/
public static JCExpression cloneType(JavacTreeMaker maker, JCExpression in, JavacNode source) {
JCExpression out = cloneType0(maker, in);
if (out != null) recursiveSetGeneratedBy(out, source);
return out;
}
private static JCExpression cloneType0(JavacTreeMaker maker, JCTree in) {
if (in == null) return null;
if (in instanceof JCPrimitiveTypeTree) {
return maker.TypeIdent(TypeTag.typeTag(in));
}
if (in instanceof JCIdent) {
return maker.Ident(((JCIdent) in).name);
}
if (in instanceof JCFieldAccess) {
JCFieldAccess fa = (JCFieldAccess) in;
return maker.Select(cloneType0(maker, fa.selected), fa.name);
}
if (in instanceof JCArrayTypeTree) {
JCArrayTypeTree att = (JCArrayTypeTree) in;
return maker.TypeArray(cloneType0(maker, att.elemtype));
}
if (in instanceof JCTypeApply) {
JCTypeApply ta = (JCTypeApply) in;
ListBuffer<JCExpression> lb = new ListBuffer<JCExpression>();
for (JCExpression typeArg : ta.arguments) {
lb.append(cloneType0(maker, typeArg));
}
return maker.TypeApply(cloneType0(maker, ta.clazz), lb.toList());
}
if (in instanceof JCWildcard) {
JCWildcard w = (JCWildcard) in;
JCExpression newInner = cloneType0(maker, w.inner);
TypeBoundKind newKind;
switch (w.getKind()) {
case SUPER_WILDCARD:
newKind = maker.TypeBoundKind(BoundKind.SUPER);
break;
case EXTENDS_WILDCARD:
newKind = maker.TypeBoundKind(BoundKind.EXTENDS);
break;
default:
case UNBOUNDED_WILDCARD:
newKind = maker.TypeBoundKind(BoundKind.UNBOUND);
```

```
break;
}
return maker.Wildcard(newKind, newInner);
}
if (JCAnnotatedTypeReflect.is(in)) {
JCExpression underlyingType = cloneType0(maker, JCAnnotatedTypeReflect.getUnderlyingType(in));
List<JCAnnotation> anns = copyAnnotations(JCAnnotatedTypeReflect.getAnnotations(in));
return JCAnnotatedTypeReflect.create(anns, underlyingType);
}
// This is somewhat unsafe, but it's better than outright throwing an exception here. Returning null will
just cause an exception down the pipeline.
return (JCExpression) in;
}
public static enum CopyJavadoc {
VERBATIM {
@Override public String apply(final JCCompilationUnit cu, final JavacNode node) {
return Javac.getDocComment(cu, node.get());
}
},
GETTER {
@Override public String apply(final JCCompilationUnit cu, final JavacNode node) {
final JCTree n = node.get();
String javadoc = Javac.getDocComment(cu, n);
// step 1: Check if there is a 'GETTER' section. If yes, that becomes the new method's javadoc.
String out = getJavadocSection(javadoc, "GETTER");
final boolean sectionBased = out != null;
if (!sectionBased) {
out = stripLinesWithTagFromJavadoc(stripSectionsFromJavadoc(javadoc), JavadocTag.PARAM);
}
node.getAst().cleanupTask("javadocfilter-getter", n, new CleanupTask() {
@Override public void cleanup() {
String javadoc = Javac.getDocComment(cu, n);
if (javadoc == null || javadoc.isEmpty()) return;
javadoc = stripSectionsFromJavadoc(javadoc);
if (!sectionBased) {
javadoc = stripLinesWithTagFromJavadoc(stripSectionsFromJavadoc(javadoc),
JavadocTag.RETURN);
}
Javac.setDocComment(cu, n, javadoc);
}
});
return out;
}
},
SETTER {
@Override public String apply(final JCCompilationUnit cu, final JavacNode node) {
return applySetter(cu, node, "SETTER");
}
},
WITH {
@Override public String apply(final JCCompilationUnit cu, final JavacNode node) {
return addReturnsUpdatedSelfIfNeeded(applySetter(cu, node, "WITH|WITHER"));
}
```

```
},
WITH_BY {
@Override public String apply(final JCCompilationUnit cu, final JavacNode node) {
return applySetter(cu, node, "WITHBY|WITH_BY");
}
};
public abstract String apply(final JCCompilationUnit cu, final JavacNode node);
private static String applySetter(final JCCompilationUnit cu, JavacNode node, String sectionName) {
final JCTree n = node.get();
String javadoc = Javac.getDocComment(cu, n);
// step 1: Check if there is a 'SETTER' section. If yes, that becomes the new method's javadoc.
String out = getJavadocSection(javadoc, sectionName);
final boolean sectionBased = out != null;
if (!sectionBased) {
out = stripLinesWithTagFromJavadoc(stripSectionsFromJavadoc(javadoc), JavadocTag.RETURN);
}
node.getAst().cleanupTask("javadocfilter-setter", n, new CleanupTask() {
@Override public void cleanup() {
String javadoc = Javac.getDocComment(cu, n);
if (javadoc == null || javadoc.isEmpty()) return;
javadoc = stripSectionsFromJavadoc(javadoc);
if (!sectionBased) {
javadoc = stripLinesWithTagFromJavadoc(stripSectionsFromJavadoc(javadoc), JavadocTag.PARAM);
}
Javac.setDocComment(cu, n, javadoc);
}
});
return shouldReturnThis(node, JavacHandlerUtil.getAccessorsForField(node)) ?
addReturnsThisIfNeeded(out) : out;
}
}
public static void copyJavadoc(JavacNode from, JCTree to, CopyJavadoc copyMode) {
copyJavadoc(from, to, copyMode, false);
}
/**
* Copies javadoc on one node to the other.
*
* in 'GETTER' copyMode, first a 'GETTER' segment is searched for. If it exists, that will become the
javadoc for the 'to' node, and this section is
* stripped out of the 'from' node. If no 'GETTER' segment is found, then the entire javadoc is taken
minus any {@code @param} lines and other sections.
* any {@code @return} lines are stripped from 'from'.
*
* in 'SETTER' mode, stripping works similarly to 'GETTER' mode, except {@code param} are copied
and stripped from the original and {@code @return} are skipped.
*/
public static void copyJavadoc(JavacNode from, JCTree to, CopyJavadoc copyMode, boolean
forceAddReturn) {
if (copyMode == null) copyMode = CopyJavadoc.VERBATIM;
try {
JCCompilationUnit cu = ((JCCompilationUnit) from.top().get());
String newJavadoc = copyMode.apply(cu, from);
if (forceAddReturn) {
```

```
newJavadoc = addReturnsThisIfNeeded(newJavadoc);
}
Javac.setDocComment(cu, to, newJavadoc);
} catch (Exception ignore) {}
}
public static void copyJavadocFromParam(JavacNode from, JCMethodDecl to, String paramName) {
try {
JCCompilationUnit cu = ((JCCompilationUnit) from.top().get());
String methodComment = Javac.getDocComment(cu, from.get());
String newJavadoc = addReturnsThisIfNeeded(getParamJavadoc(methodComment, paramName));
Javac.setDocComment(cu, to, newJavadoc);
} catch (Exception ignore) {}
}
public static boolean isDirectDescendantOfObject(JavacNode typeNode) {
if (!(typeNode.get() instanceof JCClassDecl)) throw new IllegalArgumentException("not a type node");
JCTree extending = Javac.getExtendsClause((JCClassDecl) typeNode.get());
if (extending == null) return true;
String p = extending.toString();
return p.equals("Object") || p.equals("java.lang.Object");
}
public static void createRelevantNullableAnnotation(JavacNode typeNode, JCMethodDecl mth) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToMethodDecl(typeNode, mth, lib.getNullableAnnotation(), lib.isTypeUse());
}
public static void createRelevantNonNullAnnotation(JavacNode typeNode, JCMethodDecl mth) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToMethodDecl(typeNode, mth, lib.getNonNullAnnotation(), lib.isTypeUse());
}
public static void createRelevantNonNullAnnotation(JavacNode typeNode, JCVariableDecl arg) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToVarDecl(typeNode, arg, lib.getNonNullAnnotation(), lib.isTypeUse());
}
public static void createRelevantNullableAnnotation(JavacNode typeNode, JCVariableDecl arg) {
NullAnnotationLibrary lib =
typeNode.getAst().readConfiguration(ConfigurationKeys.ADD_NULL_ANNOTATIONS);
if (lib == null) return;
applyAnnotationToVarDecl(typeNode, arg, lib.getNullableAnnotation(), lib.isTypeUse());
}
private static void applyAnnotationToMethodDecl(JavacNode typeNode, JCMethodDecl mth, String
annType, boolean typeUse) {
if (annType == null) return;
JavacTreeMaker maker = typeNode.getTreeMaker();
JCAnnotation m = maker.Annotation(genTypeRef(typeNode, annType), List.<JCExpression>nil());
if (typeUse) {
JCExpression resType = mth.restype;
if (resType instanceof JCTypeApply) {
JCTypeApply ta = (JCTypeApply) resType;
```

```
if (ta.clazz instanceof JCFieldAccess) {
mth.restype = maker.TypeApply(maker.AnnotatedType(List.of(m), ta.clazz), ta.arguments);
return;
}
resType = ta.clazz;
}
if (resType instanceof JCFieldAccess || resType instanceof JCArrayTypeTree) {
mth.restype = maker.AnnotatedType(List.of(m), resType);
return;
}
if (JCAnnotatedTypeReflect.is(resType)) {
List<JCAnnotation> annotations = JCAnnotatedTypeReflect.getAnnotations(resType);
JCAnnotatedTypeReflect.setAnnotations(resType, annotations.prepend(m));
return;
}
if (resType instanceof JCPrimitiveTypeTree || resType instanceof JCIdent) {
mth.mods.annotations = mth.mods.annotations == null ? List.of(m) :
mth.mods.annotations.prepend(m);
}
} else {
mth.mods.annotations = mth.mods.annotations == null ? List.of(m) :
mth.mods.annotations.prepend(m);
}
}
private static void applyAnnotationToVarDecl(JavacNode typeNode, JCVariableDecl arg, String
annType, boolean typeUse) {
if (annType == null) return;
JavacTreeMaker maker = typeNode.getTreeMaker();
JCAnnotation m = maker.Annotation(genTypeRef(typeNode, annType), List.<JCExpression>nil());
if (typeUse) {
JCExpression varType = arg.vartype;
JCTypeApply ta = null;
if (varType instanceof JCTypeApply) {
ta = (JCTypeApply) varType;
varType = ta.clazz;
}
if (varType instanceof JCFieldAccess || varType instanceof JCArrayTypeTree) {
varType = maker.AnnotatedType(List.of(m), varType);
if (ta != null) ta.clazz = varType;
else arg.vartype = varType;
return;
}
if (JCAnnotatedTypeReflect.is(varType)) {
List<JCAnnotation> annotations = JCAnnotatedTypeReflect.getAnnotations(varType);
JCAnnotatedTypeReflect.setAnnotations(varType, annotations.prepend(m));
return;
}
if (varType instanceof JCPrimitiveTypeTree || varType instanceof JCIdent) {
arg.mods.annotations = arg.mods.annotations == null ? List.of(m) : arg.mods.annotations.prepend(m);
}
} else {
arg.mods.annotations = arg.mods.annotations == null ? List.of(m) : arg.mods.annotations.prepend(m);
}
```

}
}

lombok/javac/handlers/JavacResolver.java

```
/*
* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import lombok.javac.JavacNode;
import lombok.javac.JavacResolution;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
public enum JavacResolver {
CLASS {
@Override
public Type resolveMember(JavacNode node, JCExpression expr) {
Type type = expr.type;
if (type == null) {
try {
new JavacResolution(node.getContext()).resolveClassMember(node);
type = expr.type;
} catch (Exception ignore) {
}
}
return type;
}
},
METHOD {
public Type resolveMember(JavacNode node, JCExpression expr) {
```

```
Type type = expr.type;
if (type == null) {
try {
JCExpression resolvedExpression = ((JCExpression) new
JavacResolution(node.getContext()).resolveMethodMember(node).get(expr));
if (resolvedExpression != null) type = resolvedExpression.type;
} catch (Exception ignore) {
}
}
return type;
}
},
CLASS_AND_METHOD {
@Override
public Type resolveMember(JavacNode node, JCExpression expr) {
Type type = METHOD.resolveMember(node, expr);
if (type == null) {
JavacNode classNode = node;
while (classNode != null && noneOf(classNode.get(), JCBlock.class, JCMethodDecl.class,
JCVariableDecl.class)) {
classNode = classNode.up();
}
if (classNode != null) {
type = CLASS.resolveMember(classNode, expr);
}
}
return type;
}
private boolean noneOf(Object o, Class<?>... clazzes) {
for (Class<?> clazz : clazzes) {
if (clazz.isInstance(o)) return false;
}
return true;
}
};
public abstract Type resolveMember(final JavacNode node, final JCExpression expr);
}
```

lombok/javac/handlers/JavacSingularsRecipes.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.io.IOException;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import com.sun.source.tree.Tree.Kind;
import com.sun.tools.javac.code.BoundKind;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.JCWildcard;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.AccessLevel;
import lombok.ConfigurationKeys;
import lombok.core.LombokImmutableList;
import lombok.core.SpiLoadUtil;
import lombok.core.TypeLibrary;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.core.handlers.HandlerUtil;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.HandleBuilder.BuilderJob;
public class JavacSingularsRecipes {
public interface ExpressionMaker {
JCExpression make();
}
public interface StatementMaker {
JCStatement make();
}
private static final JavacSingularsRecipes INSTANCE = new JavacSingularsRecipes();
```

```
private final Map<String, JavacSingularizer> singularizers = new HashMap<String,
JavacSingularizer>();
private final TypeLibrary singularizableTypes = new TypeLibrary();
private JavacSingularsRecipes() {
try {
loadAll(singularizableTypes, singularizers);
singularizableTypes.lock();
} catch (IOException e) {
System.err.println("Lombok's @Singularizable feature is broken due to misconfigured SPI files: " + e);
}
}
private static void loadAll(TypeLibrary library, Map<String, JavacSingularizer> map) throws
IOException {
for (JavacSingularizer handler : SpiLoadUtil.findServices(JavacSingularizer.class,
JavacSingularizer.class.getClassLoader())) {
for (String type : handler.getSupportedTypes()) {
JavacSingularizer existingSingularizer = map.get(type);
if (existingSingularizer != null) {
JavacSingularizer toKeep =
existingSingularizer.getClass().getName().compareTo(handler.getClass().getName()) > 0 ? handler :
existingSingularizer;
System.err.println("Multiple singularizers found for type " + type + "; the alphabetically first class is
used: " + toKeep.getClass().getName());
map.put(type, toKeep);
} else {
map.put(type, handler);
library.addType(type);
}
}
}
}
public static JavacSingularsRecipes get() {
return INSTANCE;
}
public String toQualified(String typeReference) {
java.util.List<String> q = singularizableTypes.toQualifieds(typeReference);
if (q.isEmpty()) return null;
return q.get(0);
}
public JavacSingularizer getSingularizer(String fqn, JavacNode node) {
final JavacSingularizer singularizer = singularizers.get(fqn);
final boolean useGuavaInstead = Boolean.TRUE.equals(node.getAst().readConfiguration(Configuratio
nKeys.SINGULAR_USE_GUAVA));
return useGuavaInstead ? singularizer.getGuavaInstead(node) : singularizer;
}
public static final class SingularData {
private final JavacNode annotation;
private final Name singularName;
private final Name pluralName;
private final List<JCExpression> typeArgs;
private final String targetFqn;
private final JavacSingularizer singularizer;
private final String setterPrefix;
```

```
private final boolean ignoreNullCollections;
public SingularData(JavacNode annotation, Name singularName, Name pluralName,
List<JCExpression> typeArgs, String targetFqn, JavacSingularizer singularizer, boolean
ignoreNullCollections) {
this(annotation, singularName, pluralName, typeArgs, targetFqn, singularizer, ignoreNullCollections,
"");
}
public SingularData(JavacNode annotation, Name singularName, Name pluralName,
List<JCExpression> typeArgs, String targetFqn, JavacSingularizer singularizer, boolean
ignoreNullCollections, String setterPrefix) {
this.annotation = annotation;
this.singularName = singularName;
this.pluralName = pluralName;
this.typeArgs = typeArgs;
this.targetFqn = targetFqn;
this.singularizer = singularizer;
this.setterPrefix = setterPrefix;
this.ignoreNullCollections = ignoreNullCollections;
}
public JavacNode getAnnotation() {
return annotation;
}
public Name getSingularName() {
return singularName;
}
public Name getPluralName() {
return pluralName;
}
public String getSetterPrefix() {
return setterPrefix;
}
public List<JCExpression> getTypeArgs() {
return typeArgs;
}
public String getTargetFqn() {
return targetFqn;
}
public JavacSingularizer getSingularizer() {
return singularizer;
}
public boolean isIgnoreNullCollections() {
return ignoreNullCollections;
}
public String getTargetSimpleType() {
int idx = targetFqn.lastIndexOf(".");
return idx == -1 ? targetFqn : targetFqn.substring(idx + 1);
}
}
public static abstract class JavacSingularizer {
public abstract LombokImmutableList<String> getSupportedTypes();
protected JavacSingularizer getGuavaInstead(JavacNode node) {
return this;
}
```

```
protected JCModifiers makeMods(JavacTreeMaker maker, JavacNode node, boolean deprecate,
AccessLevel access, List<JCAnnotation> methodAnnotations) {
JCAnnotation deprecateAnn = deprecate ? maker.Annotation(genJavaLangTypeRef(node,
"Deprecated"), List.<JCExpression>nil()) : null;
List<JCAnnotation> annsOnMethod = (deprecateAnn != null) ? List.of(deprecateAnn) :
List.<JCAnnotation>nil();
annsOnMethod = mergeAnnotations(annsOnMethod,methodAnnotations);
return maker.Modifiers(toJavacModifier(access), annsOnMethod);
}
/** Checks if any of the to-be-generated nodes (fields, methods) already exist. If so, errors on these
(singulars don't support manually writing some of it, and returns true). */
public boolean checkForAlreadyExistingNodesAndGenerateError(JavacNode builderType,
SingularData data) {
for (JavacNode child : builderType.down()) {
switch (child.getKind()) {
case FIELD: {
JCVariableDecl field = (JCVariableDecl) child.get();
Name name = field.name;
if (name == null) break;
if (getGeneratedBy(field) != null) continue;
for (Name fieldToBeGenerated : listFieldsToBeGenerated(data, builderType)) {
if (!fieldToBeGenerated.equals(name)) continue;
child.addError("Manually adding a field that @Singular @Builder would generate is not supported. If
you want to manually manage the builder aspect for this field/parameter, don't use @Singular.");
return true;
}
break;
}
case METHOD: {
JCMethodDecl method = (JCMethodDecl) child.get();
Name name = method.name;
if (name == null) break;
if (getGeneratedBy(method) != null) continue;
for (Name methodToBeGenerated : listMethodsToBeGenerated(data, builderType)) {
if (!methodToBeGenerated.equals(name)) continue;
child.addError("Manually adding a method that @Singular @Builder would generate is not supported. If
you want to manually manage the builder aspect for this field/parameter, don't use @Singular.");
return true;
}
break;
}}
}
return false;
}
public java.util.List<Name> listFieldsToBeGenerated(SingularData data, JavacNode builderType) {
return Collections.singletonList(data.pluralName);
}
public java.util.List<Name> listMethodsToBeGenerated(SingularData data, JavacNode builderType) {
Name p = data.pluralName;
Name s = data.singularName;
if (p.equals(s)) return Collections.singletonList(p);
return Arrays.asList(p, s);
}
```

```
public abstract java.util.List<JavacNode> generateFields(SingularData data, JavacNode builderType,
JavacNode source);
/**
* Generates the singular, plural, and clear methods for the given {@link SingularData}.
* Uses the given {@code builderType} as return type if {@code chain == true}, {@code void} otherwise.
* If you need more control over the return type and value, use
* {@link #generateMethods(SingularData, boolean, JavacNode, JCTree, boolean, ExpressionMaker,
StatementMaker)}.
*/
public void generateMethods(final BuilderJob job, SingularData data, boolean deprecate) {
//job.checkerFramework, job.builderType, job.source, job.oldFluent, job.oldChain, job.accessInners
//CheckerFrameworkVersion cfv, final JavacNode builderType, JavacNode source, boolean fluent, final
boolean chain, AccessLevel access) {
final JavacTreeMaker maker = job.builderType.getTreeMaker();
ExpressionMaker returnTypeMaker = new ExpressionMaker() { @Override public JCExpression make()
{
return job.oldChain ?
cloneSelfType(job.builderType) :
maker.Type(createVoidType(job.builderType.getSymbolTable(), CTC_VOID));
}};
StatementMaker returnStatementMaker = new StatementMaker() { @Override public JCStatement
make() {
return job.oldChain ? maker.Return(maker.Ident(job.builderType.toName("this"))) : null;
}};
generateMethods(job.checkerFramework, data, deprecate, job.builderType, job.sourceNode,
job.oldFluent, returnTypeMaker, returnStatementMaker, job.accessInners);
}
/**
* Generates the singular, plural, and clear methods for the given {@link SingularData}.
* Uses the given {@code returnTypeMaker} and {@code returnStatementMaker} for the generated
methods.
*/
public abstract void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, JavacNode builderType, JavacNode source, boolean fluent, ExpressionMaker
returnTypeMaker, StatementMaker returnStatementMaker, AccessLevel access);
protected void doGenerateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, JavacNode builderType, JavacNode source, boolean fluent, ExpressionMaker
returnTypeMaker, StatementMaker returnStatementMaker, AccessLevel access) {
JavacTreeMaker maker = builderType.getTreeMaker();
generateSingularMethod(cfv, deprecate, maker, returnTypeMaker.make(),
returnStatementMaker.make(), data, builderType, source, fluent, access);
generatePluralMethod(cfv, deprecate, maker, returnTypeMaker.make(), returnStatementMaker.make(),
data, builderType, source, fluent, access);
generateClearMethod(cfv, deprecate, maker, returnTypeMaker.make(), returnStatementMaker.make(),
data, builderType, source, access);
}
private void finishAndInjectMethod(CheckerFrameworkVersion cfv, JavacTreeMaker maker,
JCExpression returnType, JCStatement returnStatement, SingularData data, JavacNode builderType,
JavacNode source, boolean deprecate, ListBuffer<JCStatement> statements, Name methodName,
List<JCVariableDecl> jcVariableDecls, List<JCAnnotation> methodAnnotations, AccessLevel access,
Boolean ignoreNullCollections) {
if (returnStatement != null) statements.append(returnStatement);
JCBlock body = maker.Block(0, statements.toList());
```

```
JCModifiers mods = makeMods(maker, builderType, deprecate, access, methodAnnotations);
List<JCTypeParameter> typeParams = List.nil();
List<JCExpression> thrown = List.nil();
if (ignoreNullCollections != null) {
if (ignoreNullCollections.booleanValue()) {
for (JCVariableDecl d : jcVariableDecls) createRelevantNullableAnnotation(builderType, d);
} else {
for (JCVariableDecl d : jcVariableDecls) createRelevantNonNullAnnotation(builderType, d);
}
}
returnType = addCheckerFrameworkReturnsReceiver(returnType, maker, builderType, cfv);
JCMethodDecl method = maker.MethodDef(mods, methodName, returnType, typeParams,
jcVariableDecls, thrown, body, null);
if (returnStatement != null) createRelevantNonNullAnnotation(builderType, method);
recursiveSetGeneratedBy(method, source);
injectMethod(builderType, method);
}
private void generateClearMethod(CheckerFrameworkVersion cfv, boolean deprecate,
JavacTreeMaker maker, JCExpression returnType, JCStatement returnStatement, SingularData data,
JavacNode builderType, JavacNode source, AccessLevel access) {
JCStatement clearStatement = generateClearStatements(maker, data, builderType);
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
statements.append(clearStatement);
Name methodName = builderType.toName(HandlerUtil.buildAccessorName(source, "clear",
data.getPluralName().toString()));
finishAndInjectMethod(cfv, maker, returnType, returnStatement, data, builderType, source, deprecate,
statements, methodName, List.<JCVariableDecl>nil(), List.<JCAnnotation>nil(), access, null);
}
protected abstract JCStatement generateClearStatements(JavacTreeMaker maker, SingularData data,
JavacNode builderType);
private void generateSingularMethod(CheckerFrameworkVersion cfv, boolean deprecate,
JavacTreeMaker maker, JCExpression returnType, JCStatement returnStatement, SingularData data,
JavacNode builderType, JavacNode source, boolean fluent, AccessLevel access) {
ListBuffer<JCStatement> statements = generateSingularMethodStatements(maker, data, builderType,
source);
List<JCVariableDecl> params = generateSingularMethodParameters(maker, data, builderType,
source);
Name name = data.getSingularName();
String setterPrefix = data.getSetterPrefix();
if (setterPrefix.isEmpty() && !fluent) setterPrefix = getAddMethodName();
if (!setterPrefix.isEmpty()) name = builderType.toName(HandlerUtil.buildAccessorName(source,
setterPrefix, name.toString()));
statements.prepend(createConstructBuilderVarIfNeeded(maker, data, builderType, source));
List<JCAnnotation> methodAnnotations =
copyAnnotations(findCopyableToBuilderSingularSetterAnnotations(data.annotation.up()));
finishAndInjectMethod(cfv, maker, returnType, returnStatement, data, builderType, source, deprecate,
statements, name, params, methodAnnotations, access, null);
}
protected JCVariableDecl generateSingularMethodParameter(int typeIndex, JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source, Name name) {
long flags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER, builderType.getContext());
JCExpression type = cloneParamType(typeIndex, maker, data.getTypeArgs(), builderType, source);
List<JCAnnotation> typeUseAnns = getTypeUseAnnotations(type);
```

```
type = removeTypeUseAnnotations(type);
JCModifiers mods = typeUseAnns.isEmpty() ? maker.Modifiers(flags) : maker.Modifiers(flags,
typeUseAnns);
return maker.VarDef(mods, name, type, null);
}
protected JCStatement generateSingularMethodAddStatement(JavacTreeMaker maker, JavacNode
builderType, Name argumentName, String builderFieldName) {
JCExpression thisDotFieldDotAdd = chainDots(builderType, "this", builderFieldName, "add");
JCExpression invokeAdd = maker.Apply(List.<JCExpression>nil(), thisDotFieldDotAdd,
List.<JCExpression>of(maker.Ident(argumentName)));
return maker.Exec(invokeAdd);
}
protected abstract ListBuffer<JCStatement> generateSingularMethodStatements(JavacTreeMaker
maker, SingularData data, JavacNode builderType, JavacNode source);
protected abstract List<JCVariableDecl> generateSingularMethodParameters(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source);
private void generatePluralMethod(CheckerFrameworkVersion cfv, boolean deprecate,
JavacTreeMaker maker, JCExpression returnType, JCStatement returnStatement, SingularData data,
JavacNode builderType, JavacNode source, boolean fluent, AccessLevel access) {
ListBuffer<JCStatement> statements = generatePluralMethodStatements(maker, data, builderType,
source);
Name name = data.getPluralName();
String setterPrefix = data.getSetterPrefix();
if (setterPrefix.isEmpty() && !fluent) setterPrefix = getAddMethodName() + "All";
if (!setterPrefix.isEmpty()) name = builderType.toName(HandlerUtil.buildAccessorName(source,
setterPrefix, name.toString()));
JCExpression paramType = getPluralMethodParamType(builderType);
paramType = addTypeArgs(getTypeArgumentsCount(), true, builderType, paramType,
data.getTypeArgs(), source);
long paramFlags = JavacHandlerUtil.addFinalIfNeeded(Flags.PARAMETER,
builderType.getContext());
boolean ignoreNullCollections = data.isIgnoreNullCollections();
JCModifiers paramMods = maker.Modifiers(paramFlags);
JCVariableDecl param = maker.VarDef(paramMods, data.getPluralName(), paramType, null);
statements.prepend(createConstructBuilderVarIfNeeded(maker, data, builderType, source));
if (ignoreNullCollections) {
JCExpression incomingIsNotNull = maker.Binary(CTC_NOT_EQUAL,
maker.Ident(data.getPluralName()), maker.Literal(CTC_BOT, null));
JCStatement onNotNull = maker.Block(0, statements.toList());
statements = new ListBuffer<JCStatement>();
statements.append(maker.If(incomingIsNotNull, onNotNull, null));
} else {
statements.prepend(JavacHandlerUtil.generateNullCheck(maker, null, data.getPluralName(),
builderType, "%s cannot be null"));
}
List<JCAnnotation> methodAnnotations =
copyAnnotations(findCopyableToSetterAnnotations(data.annotation.up()));
finishAndInjectMethod(cfv, maker, returnType, returnStatement, data, builderType, source, deprecate,
statements, name, List.of(param), methodAnnotations, access, ignoreNullCollections);
}
protected ListBuffer<JCStatement> generatePluralMethodStatements(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
```

```
JCExpression thisDotFieldDotAdd = chainDots(builderType, "this", data.getPluralName().toString(),
getAddMethodName() + "All");
JCExpression invokeAdd = maker.Apply(List.<JCExpression>nil(), thisDotFieldDotAdd,
List.<JCExpression>of(maker.Ident(data.getPluralName())));
statements.append(maker.Exec(invokeAdd));
return statements;
}
protected abstract JCExpression getPluralMethodParamType(JavacNode builderType);
protected abstract JCStatement createConstructBuilderVarIfNeeded(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source);
public abstract void appendBuildCode(SingularData data, JavacNode builderType, JavacNode source,
ListBuffer<JCStatement> statements, Name targetVariableName, String builderVariable);
public boolean shadowedDuringBuild() {
return true;
}
public boolean requiresCleaning() {
try {
return !getClass().getMethod("appendCleaningCode", SingularData.class, JavacNode.class,
JCTree.class, ListBuffer.class).getDeclaringClass().equals(JavacSingularizer.class);
} catch (NoSuchMethodException e) {
return false;
}
}
public void appendCleaningCode(SingularData data, JavacNode builderType, JavacNode source,
ListBuffer<JCStatement> statements) {
}
// -- Utility methods --
/**
* Adds the requested number of type arguments to the provided type, copying each argument in
{@code typeArgs}. If typeArgs is too long, the extra elements are ignored.
* If {@code typeArgs} is null or too short, {@code java.lang.Object} will be substituted for each missing
type argument.
*
* @param count The number of type arguments requested.
* @param addExtends If {@code true}, all bounds are either '? extends X' or just '?'. If false, the
reverse is applied, and '? extends Foo' is converted to Foo, '?' to Object, etc.
* @param node Some node in the same AST. Just used to obtain makers and contexts and such.
* @param type The type to add generics to.
* @param typeArgs the list of type args to clone.
* @param source The source annotation that is the root cause of this code generation.
*/
protected JCExpression addTypeArgs(int count, boolean addExtends, JavacNode node, JCExpression
type, List<JCExpression> typeArgs, JavacNode source) {
JavacTreeMaker maker = node.getTreeMaker();
List<JCExpression> clonedAndFixedTypeArgs = createTypeArgs(count, addExtends, node, typeArgs,
source);
return maker.TypeApply(type, clonedAndFixedTypeArgs);
}
protected List<JCExpression> createTypeArgs(int count, boolean addExtends, JavacNode node,
List<JCExpression> typeArgs, JavacNode source) {
JavacTreeMaker maker = node.getTreeMaker();
if (count < 0) throw new IllegalArgumentException("count is negative");
if (count == 0) return List.nil();
```

```
ListBuffer<JCExpression> arguments = new ListBuffer<JCExpression>();
if (typeArgs != null) for (JCExpression orig : typeArgs) {
if (!addExtends) {
if (orig.getKind() == Kind.UNBOUNDED_WILDCARD || orig.getKind() == Kind.SUPER_WILDCARD) {
arguments.append(genJavaLangTypeRef(node, "Object"));
} else if (orig.getKind() == Kind.EXTENDS_WILDCARD) {
JCExpression inner;
try {
inner = (JCExpression) ((JCWildcard) orig).inner;
} catch (Exception e) {
inner = genJavaLangTypeRef(node, "Object");
}
arguments.append(cloneType(maker, inner, source));
} else {
arguments.append(cloneType(maker, orig, source));
}
} else {
if (orig.getKind() == Kind.UNBOUNDED_WILDCARD || orig.getKind() == Kind.SUPER_WILDCARD) {
arguments.append(maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null));
} else if (orig.getKind() == Kind.EXTENDS_WILDCARD) {
arguments.append(cloneType(maker, orig, source));
} else {
arguments.append(maker.Wildcard(maker.TypeBoundKind(BoundKind.EXTENDS), cloneType(maker,
orig, source)));
}
}
if (--count == 0) break;
}
while (count-- > 0) {
if (addExtends) {
arguments.append(maker.Wildcard(maker.TypeBoundKind(BoundKind.UNBOUND), null));
} else {
arguments.append(genJavaLangTypeRef(node, "Object"));
}
}
return arguments.toList();
}
/** Generates '<em>builderVariable</em>.<em>name</em>.size()' as an expression; if nullGuard is
true, it's this.name == null ? 0 : this.name.size(). */
protected JCExpression getSize(JavacTreeMaker maker, JavacNode builderType, Name name,
boolean nullGuard, boolean parens, String builderVariable) {
Name thisName = builderType.toName(builderVariable);
JCExpression fn = maker.Select(maker.Select(maker.Ident(thisName), name),
builderType.toName("size"));
JCExpression sizeInvoke = maker.Apply(List.<JCExpression>nil(), fn, List.<JCExpression>nil());
if (nullGuard) {
JCExpression isNull = maker.Binary(CTC_EQUAL, maker.Select(maker.Ident(thisName), name),
maker.Literal(CTC_BOT, null));
JCExpression out = maker.Conditional(isNull, maker.Literal(CTC_INT, 0), sizeInvoke);
if (parens) return maker.Parens(out);
return out;
}
return sizeInvoke;
```

```
}
protected JCExpression cloneParamType(int index, JavacTreeMaker maker, List<JCExpression>
typeArgs, JavacNode builderType, JavacNode source) {
if (typeArgs == null || typeArgs.size() <= index) {
return genJavaLangTypeRef(builderType, "Object");
} else {
JCExpression originalType = typeArgs.get(index);
if (originalType.getKind() == Kind.UNBOUNDED_WILDCARD || originalType.getKind() ==
Kind.SUPER_WILDCARD) {
return genJavaLangTypeRef(builderType, "Object");
} else if (originalType.getKind() == Kind.EXTENDS_WILDCARD) {
try {
return cloneType(maker, (JCExpression) ((JCWildcard) originalType).inner, source);
} catch (Exception e) {
return genJavaLangTypeRef(builderType, "Object");
}
} else {
return cloneType(maker, originalType, source);
}
}
}
protected abstract String getAddMethodName();
protected abstract int getTypeArgumentsCount();
protected abstract String getEmptyMaker(String target);
public JCExpression getEmptyExpression(String target, JavacTreeMaker maker, SingularData data,
JavacNode builderType, JavacNode source) {
String emptyMaker = getEmptyMaker(target);
List<JCExpression> typeArgs = createTypeArgs(getTypeArgumentsCount(), false, builderType,
data.getTypeArgs(), source);
return maker.Apply(typeArgs, chainDots(builderType, emptyMaker.split("\\.")),
List.<JCExpression>nil());
}
}
}
```

lombok/javac/handlers/package-info.java

```
/*
* Copyright (C) 2009-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* Contains the classes that implement the transformations for all of lombok's various features on the
javac v1.6 platform.
*
* NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.
*/
package lombok.javac.handlers;

lombok/javac/handlers/singulars/JavacGuavaMapSingularizer.java

/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import lombok.core.LombokImmutableList;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.spi.Provides;
@Provides(JavacSingularizer.class)
public class JavacGuavaMapSingularizer extends JavacGuavaSingularizer {
// TODO cgcc.ImmutableMultimap, cgcc.ImmutableListMultimap, cgcc.ImmutableSetMultimap
// TODO cgcc.ImmutableClassToInstanceMap
// TODO cgcc.ImmutableRangeMap
private static final LombokImmutableList<String> SUFFIXES =

```
LombokImmutableList.of("key", "value");
private static final LombokImmutableList<String> SUPPORTED_TYPES = LombokImmutableList.of(
"com.google.common.collect.ImmutableMap",
"com.google.common.collect.ImmutableBiMap",
"com.google.common.collect.ImmutableSortedMap"
);
@Override public LombokImmutableList<String> getSupportedTypes() {
return SUPPORTED_TYPES;
}
@Override protected LombokImmutableList<String> getArgumentSuffixes() {
return SUFFIXES;
}
@Override protected String getAddMethodName() {
return "put";
}
@Override protected String getAddAllTypeName() {
return "java.util.Map";
}
}
```

lombok/javac/handlers/singulars/JavacGuavaSetListSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import lombok.core.LombokImmutableList;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.spi.Provides;
@Provides(JavacSingularizer.class)
public class JavacGuavaSetListSingularizer extends JavacGuavaSingularizer {
```

```
// TODO com.google.common.collect.ImmutableRangeSet
// TODO com.google.common.collect.ImmutableMultiset and
com.google.common.collect.ImmutableSortedMultiset
private static final LombokImmutableList<String> SUFFIXES = LombokImmutableList.of("");
private static final LombokImmutableList<String> SUPPORTED_TYPES = LombokImmutableList.of(
"com.google.common.collect.ImmutableCollection",
"com.google.common.collect.ImmutableList",
"com.google.common.collect.ImmutableSet",
"com.google.common.collect.ImmutableSortedSet"
);
@Override public LombokImmutableList<String> getSupportedTypes() {
return SUPPORTED_TYPES;
}
@Override protected LombokImmutableList<String> getArgumentSuffixes() {
return SUFFIXES;
}
@Override protected String getAddMethodName() {
return "add";
}
@Override protected String getAddAllTypeName() {
return "java.lang.Iterable";
}
}
```

lombok/javac/handlers/singulars/JavacGuavaSingularizer.java

```
/*
* Copyright (C) 2015-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import static lombok.javac.Javac.*;
```

```
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.Collections;
import lombok.AccessLevel;
import lombok.core.GuavaTypeMap;
import lombok.core.LombokImmutableList;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil;
import lombok.javac.handlers.JavacSingularsRecipes.ExpressionMaker;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
import lombok.javac.handlers.JavacSingularsRecipes.StatementMaker;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
abstract class JavacGuavaSingularizer extends JavacSingularizer {
protected String getSimpleTargetTypeName(SingularData data) {
return GuavaTypeMap.getGuavaTypeName(data.getTargetFqn());
}
@Override protected String getEmptyMaker(String target) {
return "com.google.common.collect." + GuavaTypeMap.getGuavaTypeName(target) + ".of";
}
protected String getBuilderMethodName(SingularData data) {
String simpleTypeName = getSimpleTargetTypeName(data);
if ("ImmutableSortedSet".equals(simpleTypeName) ||
"ImmutableSortedMap".equals(simpleTypeName)) return "naturalOrder";
return "builder";
}
@Override public java.util.List<JavacNode> generateFields(SingularData data, JavacNode
builderType, JavacNode source) {
JavacTreeMaker maker = builderType.getTreeMaker();
String simpleTypeName = getSimpleTargetTypeName(data);
JCExpression type = JavacHandlerUtil.chainDots(builderType, "com", "google", "common", "collect",
simpleTypeName, "Builder");
type = addTypeArgs(getTypeArgumentsCount(), false, builderType, type, data.getTypeArgs(), source);
JCVariableDecl buildField = maker.VarDef(maker.Modifiers(Flags.PRIVATE), data.getPluralName(),
type, null);
return Collections.singletonList(injectFieldAndMarkGenerated(builderType, buildField));
}
@Override public void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, JavacNode builderType, JavacNode source, boolean fluent, ExpressionMaker
returnTypeMaker, StatementMaker returnStatementMaker, AccessLevel access) {
doGenerateMethods(cfv, data, deprecate, builderType, source, fluent, returnTypeMaker,
returnStatementMaker, access);
}
@Override
protected JCStatement generateClearStatements(JavacTreeMaker maker, SingularData data,
JavacNode builderType) {
```

```
JCExpression thisDotField = maker.Select(maker.Ident(builderType.toName("this")),
data.getPluralName());
return maker.Exec(maker.Assign(thisDotField, maker.Literal(CTC_BOT, null)));
}
@Override
protected List<JCVariableDecl> generateSingularMethodParameters(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
Name[] names = generateSingularMethodParameterNames(data, builderType);
ListBuffer<JCVariableDecl> params = new ListBuffer<JCVariableDecl>();
for (int i = 0; i < names.length; i++) {
params.append(generateSingularMethodParameter(i, maker, data, builderType, source, names[i]));
}
return params.toList();
}
@Override
protected ListBuffer<JCStatement> generateSingularMethodStatements(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
Name[] names = generateSingularMethodParameterNames(data, builderType);
JCExpression thisDotFieldDotAdd = chainDots(builderType, "this", data.getPluralName().toString(),
getAddMethodName());
ListBuffer<JCExpression> invokeAddExprBuilder = new ListBuffer<JCExpression>();
for (Name name : names) {
invokeAddExprBuilder.append(maker.Ident(name));
}
List<JCExpression> invokeAddExpr = invokeAddExprBuilder.toList();
JCExpression invokeAdd = maker.Apply(List.<JCExpression>nil(), thisDotFieldDotAdd,
invokeAddExpr);
JCStatement st = maker.Exec(invokeAdd);
return new ListBuffer<JCStatement>().append(st);
}
private Name[] generateSingularMethodParameterNames(SingularData data, JavacNode builderType)
{
LombokImmutableList<String> suffixes = getArgumentSuffixes();
Name[] names = new Name[suffixes.size()];
for (int i = 0; i < names.length; i++) {
String s = suffixes.get(i);
Name n = data.getSingularName();
names[i] = s.isEmpty() ? n : builderType.toName(s);
}
return names;
}
@Override
protected JCExpression getPluralMethodParamType(JavacNode builderType) {
return genTypeRef(builderType, getAddAllTypeName());
}
@Override public void appendBuildCode(SingularData data, JavacNode builderType, JavacNode
source, ListBuffer<JCStatement> statements, Name targetVariableName, String builderVariable) {
JavacTreeMaker maker = builderType.getTreeMaker();
List<JCExpression> jceBlank = List.nil();
JCExpression varType = chainDotsString(builderType, data.getTargetFqn());
int argumentsCount = getTypeArgumentsCount();
varType = addTypeArgs(argumentsCount, false, builderType, varType, data.getTypeArgs(), source);
JCExpression empty; {
```

```
//ImmutableX.of()
JCExpression emptyMethod = chainDots(builderType, "com", "google", "common", "collect",
getSimpleTargetTypeName(data), "of");
List<JCExpression> invokeTypeArgs = createTypeArgs(argumentsCount, false, builderType,
data.getTypeArgs(), source);
empty = maker.Apply(invokeTypeArgs, emptyMethod, jceBlank);
}
JCExpression invokeBuild; {
//this.pluralName.build();
invokeBuild = maker.Apply(jceBlank, chainDots(builderType, builderVariable,
data.getPluralName().toString(), "build"), jceBlank);
}
JCExpression isNull; {
//this.pluralName == null
isNull = maker.Binary(CTC_EQUAL, maker.Select(maker.Ident(builderType.toName(builderVariable)),
data.getPluralName()), maker.Literal(CTC_BOT, null));
}
JCExpression init = maker.Conditional(isNull, empty, invokeBuild); // this.pluralName == null ?
ImmutableX.of() : this.pluralName.build()
JCStatement jcs = maker.VarDef(maker.Modifiers(0L), data.getPluralName(), varType, init);
statements.append(jcs);
}
@Override
protected JCStatement createConstructBuilderVarIfNeeded(JavacTreeMaker maker, SingularData
data, JavacNode builderType, JavacNode source) {
List<JCExpression> jceBlank = List.nil();
JCExpression thisDotField = maker.Select(maker.Ident(builderType.toName("this")),
data.getPluralName());
JCExpression thisDotField2 = maker.Select(maker.Ident(builderType.toName("this")),
data.getPluralName());
JCExpression cond = maker.Binary(CTC_EQUAL, thisDotField, maker.Literal(CTC_BOT, null));
JCExpression create = maker.Apply(jceBlank, chainDots(builderType, "com", "google", "common",
"collect", getSimpleTargetTypeName(data), getBuilderMethodName(data)), jceBlank);
JCStatement thenPart = maker.Exec(maker.Assign(thisDotField2, create));
return maker.If(cond, thenPart, null);
}
protected abstract LombokImmutableList<String> getArgumentSuffixes();
protected abstract String getAddAllTypeName();
@Override
protected int getTypeArgumentsCount() {
return getArgumentSuffixes().size();
}
}
```

lombok/javac/handlers/singulars/JavacGuavaTableSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
```

```
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import lombok.core.LombokImmutableList;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.spi.Provides;
@Provides(JavacSingularizer.class)
public class JavacGuavaTableSingularizer extends JavacGuavaSingularizer {
private static final LombokImmutableList<String> SUFFIXES =
LombokImmutableList.of("rowKey", "columnKey", "value");
private static final LombokImmutableList<String> SUPPORTED_TYPES =
LombokImmutableList.of("com.google.common.collect.ImmutableTable");
@Override public LombokImmutableList<String> getSupportedTypes() {
return SUPPORTED_TYPES;
}
@Override protected LombokImmutableList<String> getArgumentSuffixes() {
return SUFFIXES;
}
@Override protected String getAddMethodName() {
return "put";
}
@Override protected String getAddAllTypeName() {
return "com.google.common.collect.Table";
}
}
```

lombok/javac/handlers/singulars/JavacJavaUtilListSetSingularizer.java

```
/*
* Copyright (C) 2015-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
```

* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.handlers.singulars;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.Collections;
import lombok.AccessLevel;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil;
import lombok.javac.handlers.JavacSingularsRecipes.ExpressionMaker;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
import lombok.javac.handlers.JavacSingularsRecipes.StatementMaker;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
abstract class JavacJavaUtilListSetSingularizer extends JavacJavaUtilSingularizer {
@Override protected JavacSingularizer getGuavaInstead(JavacNode node) {
return new JavacGuavaSetListSingularizer();
}
@Override public java.util.List<Name> listFieldsToBeGenerated(SingularData data, JavacNode
builderType) {
return super.listFieldsToBeGenerated(data, builderType);
}
@Override public java.util.List<Name> listMethodsToBeGenerated(SingularData data, JavacNode
builderType) {
return super.listMethodsToBeGenerated(data, builderType);
}
@Override public java.util.List<JavacNode> generateFields(SingularData data, JavacNode
builderType, JavacNode source) {
JavacTreeMaker maker = builderType.getTreeMaker();
JCExpression type = JavacHandlerUtil.chainDots(builderType, "java", "util", "ArrayList");
type = addTypeArgs(1, false, builderType, type, data.getTypeArgs(), source);
```

```
JCVariableDecl buildField = maker.VarDef(maker.Modifiers(Flags.PRIVATE), data.getPluralName(),
type, null);
return Collections.singletonList(injectFieldAndMarkGenerated(builderType, buildField));
}
@Override public void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, JavacNode builderType, JavacNode source, boolean fluent, ExpressionMaker
returnTypeMaker, StatementMaker returnStatementMaker, AccessLevel access) {
doGenerateMethods(cfv, data, deprecate, builderType, source, fluent, returnTypeMaker,
returnStatementMaker, access);
}
@Override
protected JCStatement generateClearStatements(JavacTreeMaker maker, SingularData data,
JavacNode builderType) {
List<JCExpression> jceBlank = List.nil();
JCExpression thisDotField = maker.Select(maker.Ident(builderType.toName("this")),
data.getPluralName());
JCExpression thisDotFieldDotClear =
maker.Select(maker.Select(maker.Ident(builderType.toName("this")), data.getPluralName()),
builderType.toName("clear"));
JCStatement clearCall = maker.Exec(maker.Apply(jceBlank, thisDotFieldDotClear, jceBlank));
JCExpression cond = maker.Binary(CTC_NOT_EQUAL, thisDotField, maker.Literal(CTC_BOT, null));
return maker.If(cond, clearCall, null);
}
@Override
protected ListBuffer<JCStatement> generateSingularMethodStatements(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
return new ListBuffer<JCStatement>()
.append(generateSingularMethodAddStatement(maker, builderType, data.getSingularName(),
data.getPluralName().toString()));
}
@Override
protected List<JCVariableDecl> generateSingularMethodParameters(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
JCVariableDecl param = generateSingularMethodParameter(0, maker, data, builderType, source,
data.getSingularName());
return List.of(param);
}
@Override
protected JCExpression getPluralMethodParamType(JavacNode builderType) {
return chainDots(builderType, "java", "util", "Collection");
}
@Override
protected JCStatement createConstructBuilderVarIfNeeded(JavacTreeMaker maker, SingularData
data, JavacNode builderType, JavacNode source) {
return createConstructBuilderVarIfNeeded(maker, data, builderType, false, source);
}
@Override
protected String getAddMethodName() {
return "add";
}
@Override
protected int getTypeArgumentsCount() {
return 1;
```

}
}

lombok/javac/handlers/singulars/JavacJavaUtilListSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import lombok.core.LombokImmutableList;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
import lombok.spi.Provides;
import com.sun.tools.javac.tree.JCTree.JCCase;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
@Provides(JavacSingularizer.class)
public class JavacJavaUtilListSingularizer extends JavacJavaUtilListSetSingularizer {
@Override public LombokImmutableList<String> getSupportedTypes() {
return LombokImmutableList.of("java.util.List", "java.util.Collection", "java.lang.Iterable");
}
@Override protected String getEmptyMaker(String target) {
return "java.util.Collections.emptyList";
}
```

```
@Override public void appendBuildCode(SingularData data, JavacNode builderType, JavacNode
source, ListBuffer<JCStatement> statements, Name targetVariableName, String builderVariable) {
JavacTreeMaker maker = builderType.getTreeMaker();
List<JCExpression> jceBlank = List.nil();
ListBuffer<JCCase> cases = new ListBuffer<JCCase>();
/* case 0: (empty); break; */ {
JCStatement assignStat; {
// pluralName = java.util.Collections.emptyList();
JCExpression invoke = maker.Apply(jceBlank, chainDots(builderType, "java", "util", "Collections",
"emptyList"), jceBlank);
assignStat = maker.Exec(maker.Assign(maker.Ident(data.getPluralName()), invoke));
}
JCStatement breakStat = maker.Break(null);
JCCase emptyCase = maker.Case(maker.Literal(CTC_INT, 0), List.of(assignStat, breakStat));
cases.append(emptyCase);
}
/* case 1: (singletonList); break; */ {
JCStatement assignStat; {
// pluralName = java.util.Collections.singletonList(this.pluralName.get(0));
JCExpression zeroLiteral = maker.Literal(CTC_INT, 0);
JCExpression arg = maker.Apply(jceBlank, chainDots(builderType, builderVariable,
data.getPluralName().toString(), "get"), List.of(zeroLiteral));
List<JCExpression> args = List.of(arg);
JCExpression invoke = maker.Apply(jceBlank, chainDots(builderType, "java", "util", "Collections",
"singletonList"), args);
assignStat = maker.Exec(maker.Assign(maker.Ident(data.getPluralName()), invoke));
}
JCStatement breakStat = maker.Break(null);
JCCase singletonCase = maker.Case(maker.Literal(CTC_INT, 1), List.of(assignStat, breakStat));
cases.append(singletonCase);
}
/* default: Create with right size, then addAll */ {
List<JCStatement> defStats = createListCopy(maker, data, builderType, source, builderVariable);
JCCase defaultCase = maker.Case(null, defStats);
cases.append(defaultCase);
}
JCStatement switchStat = maker.Switch(getSize(maker, builderType, data.getPluralName(), true, false,
builderVariable), cases.toList());
JCExpression localShadowerType = chainDotsString(builderType, data.getTargetFqn());
localShadowerType = addTypeArgs(1, false, builderType, localShadowerType, data.getTypeArgs(),
source);
JCStatement varDefStat = maker.VarDef(maker.Modifiers(0L), data.getPluralName(),
localShadowerType, null);
statements.append(varDefStat);
statements.append(switchStat);
}
private List<JCStatement> createListCopy(JavacTreeMaker maker, SingularData data, JavacNode
builderType, JavacNode source, String builderVariable) {
List<JCExpression> jceBlank = List.nil();
Name thisName = builderType.toName(builderVariable);
JCExpression argToUnmodifiable; {
// new java.util.ArrayList<Generics>(this.pluralName);
List<JCExpression> constructorArgs = List.nil();
```

```
JCExpression thisDotPluralName = maker.Select(maker.Ident(thisName), data.getPluralName());
constructorArgs = List.<JCExpression>of(thisDotPluralName);
JCExpression targetTypeExpr = chainDots(builderType, "java", "util", "ArrayList");
targetTypeExpr = addTypeArgs(1, false, builderType, targetTypeExpr, data.getTypeArgs(), source);
argToUnmodifiable = maker.NewClass(null, jceBlank, targetTypeExpr, constructorArgs, null);
}
JCStatement unmodifiableStat; {
// pluralname = Collections.unmodifiableInterfaceType(-newlist-);
JCExpression invoke = maker.Apply(jceBlank, chainDots(builderType, "java", "util", "Collections",
"unmodifiableList"), List.of(argToUnmodifiable));
unmodifiableStat = maker.Exec(maker.Assign(maker.Ident(data.getPluralName()), invoke));
}
return List.of(unmodifiableStat);
}
}
```

lombok/javac/handlers/singulars/JavacJavaUtilMapSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import java.util.Arrays;
import lombok.AccessLevel;
import lombok.core.LombokImmutableList;
import lombok.core.configuration.CheckerFrameworkVersion;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacHandlerUtil;
```

```
import lombok.javac.handlers.JavacSingularsRecipes.ExpressionMaker;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
import lombok.javac.handlers.JavacSingularsRecipes.StatementMaker;
import lombok.spi.Provides;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
@Provides(JavacSingularizer.class)
public class JavacJavaUtilMapSingularizer extends JavacJavaUtilSingularizer {
@Override public LombokImmutableList<String> getSupportedTypes() {
return LombokImmutableList.of("java.util.Map", "java.util.SortedMap", "java.util.NavigableMap");
}
@Override protected String getEmptyMaker(String target) {
if (target.endsWith("NavigableMap")) return "java.util.Collections.emptyNavigableMap";
if (target.endsWith("SortedMap")) return "java.util.Collections.emptySortedMap";
return "java.util.Collections.emptyMap";
}
@Override protected JavacSingularizer getGuavaInstead(JavacNode node) {
return new JavacGuavaMapSingularizer();
}
@Override public java.util.List<Name> listFieldsToBeGenerated(SingularData data, JavacNode
builderType) {
String p = data.getPluralName().toString();
return Arrays.asList(builderType.toName(p + "$key"), builderType.toName(p + "$value"));
}
@Override public java.util.List<Name> listMethodsToBeGenerated(SingularData data, JavacNode
builderType) {
return super.listMethodsToBeGenerated(data, builderType);
}
@Override public java.util.List<JavacNode> generateFields(SingularData data, JavacNode
builderType, JavacNode source) {
JavacTreeMaker maker = builderType.getTreeMaker();
JCVariableDecl buildKeyField; {
JCExpression type = JavacHandlerUtil.chainDots(builderType, "java", "util", "ArrayList");
type = addTypeArgs(1, false, builderType, type, data.getTypeArgs(), source);
buildKeyField = maker.VarDef(maker.Modifiers(Flags.PRIVATE),
builderType.toName(data.getPluralName() + "$key"), type, null);
}
JCVariableDecl buildValueField; {
JCExpression type = JavacHandlerUtil.chainDots(builderType, "java", "util", "ArrayList");
List<JCExpression> tArgs = data.getTypeArgs();
if (tArgs != null && tArgs.size() > 1) tArgs = tArgs.tail;
else tArgs = List.nil();
type = addTypeArgs(1, false, builderType, type, tArgs, source);
buildValueField = maker.VarDef(maker.Modifiers(Flags.PRIVATE),
builderType.toName(data.getPluralName() + "$value"), type, null);
}
```

```
JavacNode valueFieldNode = injectFieldAndMarkGenerated(builderType, buildValueField);
JavacNode keyFieldNode = injectFieldAndMarkGenerated(builderType, buildKeyField);
return Arrays.asList(keyFieldNode, valueFieldNode);
}
@Override public void generateMethods(CheckerFrameworkVersion cfv, SingularData data, boolean
deprecate, JavacNode builderType, JavacNode source, boolean fluent, ExpressionMaker
returnTypeMaker, StatementMaker returnStatementMaker, AccessLevel access) {
doGenerateMethods(cfv, data, deprecate, builderType, source, fluent, returnTypeMaker,
returnStatementMaker, access);
}
@Override
protected JCStatement generateClearStatements(JavacTreeMaker maker, SingularData data,
JavacNode builderType) {
List<JCExpression> jceBlank = List.nil();
JCExpression thisDotKeyField = chainDots(builderType, "this", data.getPluralName() + "$key");
JCExpression thisDotKeyFieldDotClear = chainDots(builderType, "this", data.getPluralName() + "$key",
"clear");
JCExpression thisDotValueFieldDotClear = chainDots(builderType, "this", data.getPluralName() +
"$value", "clear");
JCStatement clearKeyCall = maker.Exec(maker.Apply(jceBlank, thisDotKeyFieldDotClear, jceBlank));
JCStatement clearValueCall = maker.Exec(maker.Apply(jceBlank, thisDotValueFieldDotClear,
jceBlank));
JCExpression cond = maker.Binary(CTC_NOT_EQUAL, thisDotKeyField, maker.Literal(CTC_BOT,
null));
JCBlock clearCalls = maker.Block(0, List.of(clearKeyCall, clearValueCall));
return maker.If(cond, clearCalls, null);
}
@Override
protected ListBuffer<JCStatement> generateSingularMethodStatements(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
Name keyName = builderType.toName(data.getSingularName().toString() + "Key");
Name valueName = builderType.toName(data.getSingularName().toString() + "Value");
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
/* Generates: this.pluralname$key.add(singularnameKey); */
statements.append(generateSingularMethodAddStatement(maker, builderType, keyName,
data.getPluralName() + "$key"));
/* Generates: this.pluralname$value.add(singularnameValue); */
statements.append(generateSingularMethodAddStatement(maker, builderType, valueName,
data.getPluralName() + "$value"));
return statements;
}
@Override
protected List<JCVariableDecl> generateSingularMethodParameters(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
Name keyName = builderType.toName(data.getSingularName().toString() + "Key");
Name valueName = builderType.toName(data.getSingularName().toString() + "Value");
JCVariableDecl paramKey = generateSingularMethodParameter(0, maker, data, builderType, source,
keyName);
JCVariableDecl paramValue = generateSingularMethodParameter(1, maker, data, builderType, source,
valueName);
return List.of(paramKey, paramValue);
}
@Override
```

```
protected ListBuffer<JCStatement> generatePluralMethodStatements(JavacTreeMaker maker,
SingularData data, JavacNode builderType, JavacNode source) {
List<JCExpression> jceBlank = List.nil();
ListBuffer<JCStatement> statements = new ListBuffer<JCStatement>();
long baseFlags = JavacHandlerUtil.addFinalIfNeeded(0, builderType.getContext());
Name entryName = builderType.toName("$lombokEntry");
JCExpression forEachType = chainDots(builderType, "java", "util", "Map", "Entry");
forEachType = addTypeArgs(2, true, builderType, forEachType, data.getTypeArgs(), source);
JCExpression keyArg = maker.Apply(List.<JCExpression>nil(), maker.Select(maker.Ident(entryName),
builderType.toName("getKey")), List.<JCExpression>nil());
JCExpression valueArg = maker.Apply(List.<JCExpression>nil(),
maker.Select(maker.Ident(entryName), builderType.toName("getValue")), List.<JCExpression>nil());
JCExpression addKey = maker.Apply(List.<JCExpression>nil(), chainDots(builderType, "this",
data.getPluralName() + "$key", "add"), List.of(keyArg));
JCExpression addValue = maker.Apply(List.<JCExpression>nil(), chainDots(builderType, "this",
data.getPluralName() + "$value", "add"), List.of(valueArg));
JCBlock forEachBody = maker.Block(0, List.<JCStatement>of(maker.Exec(addKey),
maker.Exec(addValue)));
JCExpression entrySetInvocation = maker.Apply(jceBlank,
maker.Select(maker.Ident(data.getPluralName()), builderType.toName("entrySet")), jceBlank);
JCStatement forEach = maker.ForeachLoop(maker.VarDef(maker.Modifiers(baseFlags), entryName,
forEachType, null), entrySetInvocation, forEachBody);
statements.append(forEach);
return statements;
}
@Override
protected JCExpression getPluralMethodParamType(JavacNode builderType) {
return chainDots(builderType, "java", "util", "Map");
}
@Override
protected JCStatement createConstructBuilderVarIfNeeded(JavacTreeMaker maker, SingularData
data, JavacNode builderType, JavacNode source) {
return createConstructBuilderVarIfNeeded(maker, data, builderType, true, source);
}
@Override public void appendBuildCode(SingularData data, JavacNode builderType, JavacNode
source, ListBuffer<JCStatement> statements, Name targetVariableName, String builderVariable) {
JavacTreeMaker maker = builderType.getTreeMaker();
if (data.getTargetFqn().equals("java.util.Map")) {
statements.appendList(createJavaUtilSetMapInitialCapacitySwitchStatements(maker, data,
builderType, true, "emptyMap", "singletonMap", "LinkedHashMap", source, builderVariable));
} else {
statements.appendList(createJavaUtilSimpleCreationAndFillStatements(maker, data, builderType, true,
true, false, true, "TreeMap", source, builderVariable));
}
}
@Override
protected String getAddMethodName() {
return "put";
}
@Override
protected int getTypeArgumentsCount() {
return 2;
}
```

}

lombok/javac/handlers/singulars/JavacJavaUtilSetSingularizer.java

```
/*
* Copyright (C) 2015-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import lombok.core.LombokImmutableList;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
import lombok.spi.Provides;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
@Provides(JavacSingularizer.class)
public class JavacJavaUtilSetSingularizer extends JavacJavaUtilListSetSingularizer {
@Override public LombokImmutableList<String> getSupportedTypes() {
return LombokImmutableList.of("java.util.Set", "java.util.SortedSet", "java.util.NavigableSet");
}
@Override protected String getEmptyMaker(String target) {
if (target.endsWith("SortedSet")) return "java.util.Collections.emptySortedSet";
if (target.endsWith("NavigableSet")) return "java.util.Collections.emptyNavigableSet";
return "java.util.Collections.emptySet";
}
@Override public void appendBuildCode(SingularData data, JavacNode builderType, JavacNode
source, ListBuffer<JCStatement> statements, Name targetVariableName, String builderVariable) {
JavacTreeMaker maker = builderType.getTreeMaker();
if (data.getTargetFqn().equals("java.util.Set")) {
```

```
statements.appendList(createJavaUtilSetMapInitialCapacitySwitchStatements(maker, data,
builderType, false, "emptySet", "singleton", "LinkedHashSet", source, builderVariable));
} else {
statements.appendList(createJavaUtilSimpleCreationAndFillStatements(maker, data, builderType,
false, true, false, true, "TreeSet", source, builderVariable));
}
}
}
```

lombok/javac/handlers/singulars/JavacJavaUtilSingularizer.java

```
/*
* Copyright (C) 2015-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.handlers.singulars;
import static lombok.javac.Javac.*;
import static lombok.javac.handlers.JavacHandlerUtil.*;
import com.sun.tools.javac.tree.JCTree.JCCase;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import com.sun.tools.javac.util.Name;
import lombok.javac.JavacNode;
import lombok.javac.JavacTreeMaker;
import lombok.javac.handlers.JavacSingularsRecipes.JavacSingularizer;
import lombok.javac.handlers.JavacSingularsRecipes.SingularData;
abstract class JavacJavaUtilSingularizer extends JavacSingularizer {
protected List<JCStatement> createJavaUtilSetMapInitialCapacitySwitchStatements(JavacTreeMaker
maker, SingularData data, JavacNode builderType, boolean mapMode, String emptyCollectionMethod,
String singletonCollectionMethod, String targetType, JavacNode source, String builderVariable) {
```

```
List<JCExpression> jceBlank = List.nil();
ListBuffer<JCCase> cases = new ListBuffer<JCCase>();
if (emptyCollectionMethod != null) { // case 0: (empty); break;
JCStatement assignStat; {
// pluralName = java.util.Collections.emptyCollectionMethod();
JCExpression invoke = maker.Apply(jceBlank, chainDots(builderType, "java", "util", "Collections",
emptyCollectionMethod), jceBlank);
assignStat = maker.Exec(maker.Assign(maker.Ident(data.getPluralName()), invoke));
}
JCStatement breakStat = maker.Break(null);
JCCase emptyCase = maker.Case(maker.Literal(CTC_INT, 0), List.of(assignStat, breakStat));
cases.append(emptyCase);
}
if (singletonCollectionMethod != null) { // case 1: (singleton); break;
JCStatement assignStat; {
// !mapMode: pluralName = java.util.Collections.singletonCollectionMethod(this.pluralName.get(0));
// mapMode: pluralName = java.util.Collections.singletonCollectionMethod(this.pluralName$key.get(0),
this.pluralName$value.get(0));
JCExpression zeroLiteral = maker.Literal(CTC_INT, 0);
JCExpression arg = maker.Apply(jceBlank, chainDots(builderType, builderVariable,
data.getPluralName() + (mapMode ? "$key" : ""), "get"), List.of(zeroLiteral));
List<JCExpression> args;
if (mapMode) {
JCExpression zeroLiteralClone = maker.Literal(CTC_INT, 0);
JCExpression arg2 = maker.Apply(jceBlank, chainDots(builderType, builderVariable,
data.getPluralName() + (mapMode ? "$value" : ""), "get"), List.of(zeroLiteralClone));
args = List.of(arg, arg2);
} else {
args = List.of(arg);
}
JCExpression invoke = maker.Apply(jceBlank, chainDots(builderType, "java", "util", "Collections",
singletonCollectionMethod), args);
assignStat = maker.Exec(maker.Assign(maker.Ident(data.getPluralName()), invoke));
}
JCStatement breakStat = maker.Break(null);
JCCase singletonCase = maker.Case(maker.Literal(CTC_INT, 1), List.of(assignStat, breakStat));
cases.append(singletonCase);
}
{ // default:
List<JCStatement> statements = createJavaUtilSimpleCreationAndFillStatements(maker, data,
builderType, mapMode, false, true, emptyCollectionMethod == null, targetType, source,
builderVariable);
JCCase defaultCase = maker.Case(null, statements);
cases.append(defaultCase);
}
JCStatement switchStat = maker.Switch(getSize(maker, builderType, mapMode ?
builderType.toName(data.getPluralName() + "$key") : data.getPluralName(), true, false,
builderVariable), cases.toList());
JCExpression localShadowerType = chainDotsString(builderType, data.getTargetFqn());
localShadowerType = addTypeArgs(mapMode ? 2 : 1, false, builderType, localShadowerType,
data.getTypeArgs(), source);
JCStatement varDefStat = maker.VarDef(maker.Modifiers(0L), data.getPluralName(),
localShadowerType, null);
```

```
return List.of(varDefStat, switchStat);
}
protected JCStatement createConstructBuilderVarIfNeeded(JavacTreeMaker maker, SingularData
data, JavacNode builderType, boolean mapMode, JavacNode source) {
List<JCExpression> jceBlank = List.nil();
Name v1Name = mapMode ? builderType.toName(data.getPluralName() + "$key") :
data.getPluralName();
Name v2Name = mapMode ? builderType.toName(data.getPluralName() + "$value") : null;
JCExpression thisDotField = maker.Select(maker.Ident(builderType.toName("this")), v1Name);
JCExpression cond = maker.Binary(CTC_EQUAL, thisDotField, maker.Literal(CTC_BOT, null));
thisDotField = maker.Select(maker.Ident(builderType.toName("this")), v1Name);
JCExpression v1Type = chainDots(builderType, "java", "util", "ArrayList");
v1Type = addTypeArgs(1, false, builderType, v1Type, data.getTypeArgs(), source);
JCExpression constructArrayList = maker.NewClass(null, jceBlank, v1Type, jceBlank, null);
JCStatement initV1 = maker.Exec(maker.Assign(thisDotField, constructArrayList));
JCStatement thenPart;
if (mapMode) {
thisDotField = maker.Select(maker.Ident(builderType.toName("this")), v2Name);
JCExpression v2Type = chainDots(builderType, "java", "util", "ArrayList");
List<JCExpression> tArgs = data.getTypeArgs();
if (tArgs != null && tArgs.tail != null) tArgs = tArgs.tail;
else tArgs = List.nil();
v2Type = addTypeArgs(1, false, builderType, v2Type, tArgs, source);
constructArrayList = maker.NewClass(null, jceBlank, v2Type, jceBlank, null);
JCStatement initV2 = maker.Exec(maker.Assign(thisDotField, constructArrayList));
thenPart = maker.Block(0, List.of(initV1, initV2));
} else {
thenPart = initV1;
}
return maker.If(cond, thenPart, null);
}
protected List<JCStatement> createJavaUtilSimpleCreationAndFillStatements(JavacTreeMaker maker,
SingularData data, JavacNode builderType, boolean mapMode, boolean defineVar, boolean
addInitialCapacityArg, boolean nullGuard, String targetType, JavacNode source, String builderVariable)
{
List<JCExpression> jceBlank = List.nil();
Name thisName = builderType.toName(builderVariable);
JCStatement createStat; {
// pluralName = new java.util.TargetType(initialCap);
List<JCExpression> constructorArgs = List.nil();
if (addInitialCapacityArg) {
Name varName = mapMode ? builderType.toName(data.getPluralName() + "$key") :
data.getPluralName();
// this.varName.size() < MAX_POWER_OF_2 ? 1 + this.varName.size() + (this.varName.size() - 3) / 3 :
Integer.MAX_VALUE;
// lessThanCutOff = this.varName.size() < MAX_POWER_OF_2
JCExpression lessThanCutoff = maker.Binary(CTC_LESS_THAN, getSize(maker, builderType,
varName, nullGuard, true, builderVariable), maker.Literal(CTC_INT, 0x40000000));
JCExpression integerMaxValue = genJavaLangTypeRef(builderType, "Integer", "MAX_VALUE");
JCExpression sizeFormulaLeft = maker.Binary(CTC_PLUS, maker.Literal(CTC_INT, 1),
getSize(maker, builderType, varName, nullGuard, true, builderVariable));
JCExpression sizeFormulaRightLeft = maker.Parens(maker.Binary(CTC_MINUS, getSize(maker,
builderType, varName, nullGuard, true, builderVariable), maker.Literal(CTC_INT, 3)));
```

```
JCExpression sizeFormulaRight = maker.Binary(CTC_DIV, sizeFormulaRightLeft,
maker.Literal(CTC_INT, 3));
JCExpression sizeFormula = maker.Binary(CTC_PLUS, sizeFormulaLeft, sizeFormulaRight);
constructorArgs = List.<JCExpression>of(maker.Conditional(lessThanCutoff, sizeFormula,
integerMaxValue));
}
JCExpression targetTypeExpr = chainDots(builderType, "java", "util", targetType);
targetTypeExpr = addTypeArgs(mapMode ? 2 : 1, false, builderType, targetTypeExpr,
data.getTypeArgs(), source);
JCExpression constructorCall = maker.NewClass(null, jceBlank, targetTypeExpr, constructorArgs,
null);
if (defineVar) {
JCExpression localShadowerType = chainDotsString(builderType, data.getTargetFqn());
localShadowerType = addTypeArgs(mapMode ? 2 : 1, false, builderType, localShadowerType,
data.getTypeArgs(), source);
createStat = maker.VarDef(maker.Modifiers(0L), data.getPluralName(), localShadowerType,
constructorCall);
} else {
createStat = maker.Exec(maker.Assign(maker.Ident(data.getPluralName()), constructorCall));
}
}
JCStatement fillStat; {
if (mapMode) {
// for (int $i = 0; $i < this.pluralname$key.size(); i++) pluralname.put(this.pluralname$key.get($i),
this.pluralname$value.get($i));
Name ivar = builderType.toName("$i");
Name keyVarName = builderType.toName(data.getPluralName() + "$key");
JCExpression pluralnameDotPut = maker.Select(maker.Ident(data.getPluralName()),
builderType.toName("put"));
JCExpression arg1 = maker.Apply(jceBlank, chainDots(builderType, builderVariable,
data.getPluralName() + "$key", "get"), List.<JCExpression>of(maker.Ident(ivar)));
JCExpression arg2 = maker.Apply(jceBlank, chainDots(builderType, builderVariable,
data.getPluralName() + "$value", "get"), List.<JCExpression>of(maker.Ident(ivar)));
// [jdk9] We add an unneccessary (V) cast here. Not doing so gives an error in javac (build
9-ea+156-jigsaw-nightly-h6072-20170212):
// error: method put in interface Map<K#2,V#2> cannot be applied to given types;
arg2 = maker.TypeCast(createTypeArgs(2, false, builderType, data.getTypeArgs(), source).get(1),
arg2);
JCStatement putStatement = maker.Exec(maker.Apply(jceBlank, pluralnameDotPut, List.of(arg1,
arg2)));
JCStatement forInit = maker.VarDef(maker.Modifiers(0L), ivar, maker.TypeIdent(CTC_INT),
maker.Literal(CTC_INT, 0));
JCExpression checkExpr = maker.Binary(CTC_LESS_THAN, maker.Ident(ivar), getSize(maker,
builderType, keyVarName, nullGuard, true, builderVariable));
JCExpression incrementExpr = maker.Unary(CTC_POSTINC, maker.Ident(ivar));
fillStat = maker.ForLoop(List.of(forInit), checkExpr, List.of(maker.Exec(incrementExpr)), putStatement);
} else {
// pluralname.addAll(this.pluralname);
JCExpression thisDotPluralName = maker.Select(maker.Ident(thisName), data.getPluralName());
fillStat = maker.Exec(maker.Apply(jceBlank, maker.Select(maker.Ident(data.getPluralName()),
builderType.toName("addAll")), List.of(thisDotPluralName)));
}
if (nullGuard) {
```

```
JCExpression thisDotField = maker.Select(maker.Ident(thisName), mapMode ?
builderType.toName(data.getPluralName() + "$key") : data.getPluralName());
JCExpression nullCheck = maker.Binary(CTC_NOT_EQUAL, thisDotField, maker.Literal(CTC_BOT,
null));
fillStat = maker.If(nullCheck, fillStat, null);
}
}
JCStatement unmodifiableStat; {
// pluralname = Collections.unmodifiableInterfaceType(pluralname);
JCExpression arg = maker.Ident(data.getPluralName());
JCExpression invoke = maker.Apply(jceBlank, chainDots(builderType, "java", "util", "Collections",
"unmodifiable" + data.getTargetSimpleType()), List.of(arg));
unmodifiableStat = maker.Exec(maker.Assign(maker.Ident(data.getPluralName()), invoke));
}
return List.of(createStat, fillStat, unmodifiableStat);
}
}
```

lombok/javac/package-info.java

```
/*
* Copyright (C) 2009-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* Includes the javac specific implementations of the lombok AST and annotation introspection support.
*
* <strong>NB: This package is not public API in the sense that contents of this package,
* even public classes / methods / etc, may change in point releases.</strong>
*/
package lombok.javac;
```

lombok/package-info.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* This package contains all the annotations and support classes you need as a user of lombok.
* All other packages are only relevant to those who are extending lombok for their own uses, except:
*
* <ul>
* <li>{@code lombok.extern.*} – These packages contains lombok annotations that solve boilerplate issues for libraries not part of the JRE itself.
* <li>{@code lombok.experimental} – This package contains lombok features that are new or likely to change before committing to long-term support.
* </ul>
*
* @see <a href="https://projectlombok.org/features/all">Lombok features</a>
*/
package lombok;
```

lombok/val.java

```
/*
* Copyright (C) 2010-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
```

```
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
/**
* Use {@code val} as the type of any local variable declaration (even in a for-each statement), and the type will be inferred from the initializing expression.
* For example: {@code val x = 10.0;} will infer {@code double}, and {@code val y = new ArrayList<String>();} will infer {@code ArrayList<String>}. The local variable
* will also be made final.
* <p>
* Note that this is an annotation type because {@code val x = 10;} will be desugared to {@code @val final int x = 10;}
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/val">the project lombok features page for &#64;val</a>.
*/
public @interface val {
}
```

lombok/var.java

```
/*
* Copyright (C) 2010-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
/**
```

```
* Use {@code var} as the type of any local variable declaration (even in a {@code for} statement), and the type will be inferred from the initializing expression
* (any further assignments to the variable are not involved in this type inference).
* <p>
* For example: {@code var x = 10.0;} will infer {@code double}, and {@code var y = new ArrayList<String>();} will infer {@code ArrayList<String>}.
* <p>
* Note that this is an annotation type because {@code var x = 10;} will be desugared to {@code @var int x = 10;}
* <p>
* Complete documentation is found at <a href="https://projectlombok.org/features/var">the project lombok features page for &#64;var</a>.
*/
public @interface var {
}
```

lombok/launch/Agent.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.launch;
import java.lang.instrument.Instrumentation;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
final class Agent {
public static void agentmain(String agentArgs, Instrumentation instrumentation) throws Throwable {
runLauncher(agentArgs, instrumentation, true);
}
public static void premain(String agentArgs, Instrumentation instrumentation) throws Throwable {
runLauncher(agentArgs, instrumentation, false);
}
```

```
private static void runLauncher(String agentArgs, Instrumentation instrumentation, boolean injected)
throws Throwable {
ClassLoader cl = Main.getShadowClassLoader();
try {
Class<?> c = cl.loadClass("lombok.core.AgentLauncher");
Method m = c.getDeclaredMethod("runAgents", String.class, Instrumentation.class, boolean.class,
Class.class);
m.invoke(null, agentArgs, instrumentation, injected, Agent.class);
} catch (InvocationTargetException e) {
throw e.getCause();
}
}
}
```

lombok/launch/AnnotationProcessor.java

```
/*
* Copyright (C) 2014-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.launch;
import java.lang.reflect.Field;
import java.util.Set;
import javax.annotation.processing.AbstractProcessor;
import javax.annotation.processing.Completion;
import javax.annotation.processing.ProcessingEnvironment;
import javax.annotation.processing.RoundEnvironment;
import javax.annotation.processing.SupportedAnnotationTypes;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.AnnotationMirror;
import javax.lang.model.element.Element;
import javax.lang.model.element.ExecutableElement;
import javax.lang.model.element.TypeElement;
import sun.misc.Unsafe;
```

```
class AnnotationProcessorHider {
public static class AstModificationNotifierData {
public volatile static boolean lombokInvoked = false;
}
public static class AnnotationProcessor extends AbstractProcessor {
private final AbstractProcessor instance = createWrappedInstance();
@Override public Set<String> getSupportedOptions() {
return instance.getSupportedOptions();
}
@Override public Set<String> getSupportedAnnotationTypes() {
return instance.getSupportedAnnotationTypes();
}
@Override public SourceVersion getSupportedSourceVersion() {
return instance.getSupportedSourceVersion();
}
@Override public void init(ProcessingEnvironment processingEnv) {
disableJava9SillyWarning();
AstModificationNotifierData.lombokInvoked = true;
instance.init(processingEnv);
super.init(processingEnv);
}
// sunapi suppresses javac's warning about using Unsafe; 'all' suppresses eclipse's warning about the
unspecified 'sunapi' key. Leave them both.
// Yes, javac's definition of the word 'all' is quite contrary to what the dictionary says it means. 'all' does
NOT include 'sunapi' according to javac.
@SuppressWarnings({"sunapi", "all"})
private void disableJava9SillyWarning() {
// JVM9 complains about using reflection to access packages from a module that aren't exported. This
makes no sense; the whole point of reflection
// is to get past such issues. The only comment from the jigsaw team lead on this was some unspecified
mumbling about security which makes no sense,
// as the SecurityManager is invoked to check such things. Therefore this warning is a bug, so we shall
patch java to fix it.
try {
Field theUnsafe = Unsafe.class.getDeclaredField("theUnsafe");
theUnsafe.setAccessible(true);
Unsafe u = (Unsafe) theUnsafe.get(null);
Class<?> cls = Class.forName("jdk.internal.module.IllegalAccessLogger");
Field logger = cls.getDeclaredField("logger");
u.putObjectVolatile(cls, u.staticFieldOffset(logger), null);
} catch (Throwable t) {
// We shall ignore it; the effect of this code failing is that the user gets to see a warning they remove
with various --add-opens magic.
}
}
@Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment
roundEnv) {
return instance.process(annotations, roundEnv);
}
@Override public Iterable<? extends Completion> getCompletions(Element element, AnnotationMirror
annotation, ExecutableElement member, String userText) {
return instance.getCompletions(element, annotation, member, userText);
}
```

```
private static AbstractProcessor createWrappedInstance() {
ClassLoader cl = Main.getShadowClassLoader();
try {
Class<?> mc = cl.loadClass("lombok.core.AnnotationProcessor");
return (AbstractProcessor) mc.getDeclaredConstructor().newInstance();
} catch (Throwable t) {
if (t instanceof Error) throw (Error) t;
if (t instanceof RuntimeException) throw (RuntimeException) t;
throw new RuntimeException(t);
}
}
}
@SupportedAnnotationTypes("lombok.*")
public static class ClaimingProcessor extends AbstractProcessor {
@Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment
roundEnv) {
return true;
}
@Override public SourceVersion getSupportedSourceVersion() {
return SourceVersion.latest();
}
}
}
```

lombok/launch/ClassFileMetaData.java

```
/*
* Copyright (C) 2010-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.launch;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
```

```
import java.util.List;
/**
* Utility to read the constant pool, header, and inheritance information of any class file.
*
* THIS IS A COPY OF lombok.bytecode.ClassFileMetaData - it is used by the shadowclassloader which can't rely on anything from outside the lombok.launch package, but everything in the launch package has to be package private.
* If there is a bug or an update is needed here - make sure to also update it in the lombok.bytecode package.
*/
class ClassFileMetaData {
private static final byte UTF8 = 1;
private static final byte INTEGER = 3;
private static final byte FLOAT = 4;
private static final byte LONG = 5;
private static final byte DOUBLE = 6;
private static final byte CLASS = 7;
private static final byte STRING = 8;
private static final byte FIELD = 9;
private static final byte METHOD = 10;
private static final byte INTERFACE_METHOD = 11;
private static final byte NAME_TYPE = 12;
// New in java7: support for methodhandles and invokedynamic
private static final byte METHOD_HANDLE = 15;
private static final byte METHOD_TYPE = 16;
private static final byte DYNAMIC = 17;
private static final byte INVOKE_DYNAMIC = 18;
// New in java9: support for modules
private static final byte MODULE = 19;
private static final byte PACKAGE = 20;
private static final int NOT_FOUND = -1;
private static final int START_OF_CONSTANT_POOL = 8;
private final byte[] byteCode;
private final int maxPoolSize;
private final int[] offsets;
private final byte[] types;
private final String[] utf8s;
private final int endOfPool;
public ClassFileMetaData(byte[] byteCode) {
this.byteCode = byteCode;
maxPoolSize = readValue(START_OF_CONSTANT_POOL);
offsets = new int[maxPoolSize];
types = new byte[maxPoolSize];
utf8s = new String[maxPoolSize];
int position = 10;
for (int i = 1; i < maxPoolSize; i++) {
byte type = byteCode[position];
types[i] = type;
position++;
offsets[i] = position;
switch (type) {
case UTF8:
int length = readValue(position);
```

```
position += 2;
utf8s[i] = decodeString(position, length);
position += length;
break;
case CLASS:
case STRING:
case METHOD_TYPE:
case MODULE:
case PACKAGE:
position += 2;
break;
case METHOD_HANDLE:
position += 3;
break;
case INTEGER:
case FLOAT:
case FIELD:
case METHOD:
case INTERFACE_METHOD:
case NAME_TYPE:
case INVOKE_DYNAMIC:
case DYNAMIC:
position += 4;
break;
case LONG:
case DOUBLE:
position += 8;
i++;
break;
case 0:
break;
default:
throw new AssertionError("Unknown constant pool type " + type);
}
}
endOfPool = position;
}
private String decodeString(int pos, int size) {
int end = pos + size;
// the resulting string might be smaller
char[] result = new char[size];
int length = 0;
while (pos < end) {
int first = (byteCode[pos++] & 0xFF);
if (first < 0x80) {
result[length++] = (char)first;
} else if ((first & 0xE0) == 0xC0) {
int x = (first & 0x1F) << 6;
int y = (byteCode[pos++] & 0x3F);
result[length++] = (char)(x | y);
} else {
int x = (first & 0x0F) << 12;
int y = (byteCode[pos++] & 0x3F) << 6;
```

```
int z = (byteCode[pos++] & 0x3F);
result[length++] = (char)(x | y | z);
}
}
return new String(result, 0, length);
}
public int[] getOffsets(byte type) {
int[] out = new int[types.length];
int ptr = 0;
for (int i = 0; i < types.length; i++) {
if (types[i] != type) continue;
out[ptr++] = offsets[i];
}
return Arrays.copyOf(out, ptr);
}
/**
* Checks if the constant pool contains the provided 'raw' string. These are used as source material for
further JVM types, such as string constants, type references, etcetera.
*/
public boolean containsUtf8(String value) {
return findUtf8(value) != NOT_FOUND;
}
/**
* Checks if the constant pool contains a reference to the provided class.
*
* NB: Most uses of a type do <em>NOT</em> show up as a class in the constant pool.
* For example, the parameter types and return type of any method you invoke or declare, are stored as
signatures and not as type references,
* but the type to which any method you invoke belongs, is. Read the JVM Specification for more
information.
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
*/
public boolean usesClass(String className) {
return findClass(className) != NOT_FOUND;
}
/**
* Checks if the constant pool contains a reference to a given field, either for writing or reading.
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
*/
public boolean usesField(String className, String fieldName) {
int classIndex = findClass(className);
if (classIndex == NOT_FOUND) return false;
int fieldNameIndex = findUtf8(fieldName);
if (fieldNameIndex == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == FIELD && readValue(offsets[i]) == classIndex) {
int nameAndTypeIndex = readValue(offsets[i] + 2);
if (readValue(offsets[nameAndTypeIndex]) == fieldNameIndex) return true;
}
}
return false;
```

```
}
/**
* Checks if the constant pool contains a reference to a given method, with any signature (return type
and parameter types).
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
*/
public boolean usesMethod(String className, String methodName) {
int classIndex = findClass(className);
if (classIndex == NOT_FOUND) return false;
int methodNameIndex = findUtf8(methodName);
if (methodNameIndex == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (isMethod(i) && readValue(offsets[i]) == classIndex) {
int nameAndTypeIndex = readValue(offsets[i] + 2);
if (readValue(offsets[nameAndTypeIndex]) == methodNameIndex) return true;
}
}
return false;
}
/**
* Checks if the constant pool contains a reference to a given method.
*
* @param className must be provided JVM-style, such as {@code java/lang/String}
* @param descriptor must be provided JVM-style, such as {@code (IZ)Ljava/lang/String;}
*/
public boolean usesMethod(String className, String methodName, String descriptor) {
int classIndex = findClass(className);
if (classIndex == NOT_FOUND) return false;
int nameAndTypeIndex = findNameAndType(methodName, descriptor);
if (nameAndTypeIndex == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (isMethod(i) &&
readValue(offsets[i]) == classIndex &&
readValue(offsets[i] + 2) == nameAndTypeIndex) return true;
}
return false;
}
/**
* Checks if the constant pool contains the provided string constant, which implies the constant is used
somewhere in the code.
*
* NB: String literals get concatenated by the compiler.
* NB2: This method does NOT do any kind of normalization.
*/
public boolean containsStringConstant(String value) {
int index = findUtf8(value);
if (index == NOT_FOUND) return false;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == STRING && readValue(offsets[i]) == index) return true;
}
return false;
}
```

```
/**
* Checks if the constant pool contains the provided long constant, which implies the constant is used somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsLong(long value) {
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == LONG && readLong(i) == value) return true;
}
return false;
}
/**
* Checks if the constant pool contains the provided double constant, which implies the constant is used somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsDouble(double value) {
boolean isNan = Double.isNaN(value);
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == DOUBLE) {
double d = readDouble(i);
if (d == value || (isNan && Double.isNaN(d))) return true;
}
}
return false;
}
/**
* Checks if the constant pool contains the provided int constant, which implies the constant is used somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsInteger(int value) {
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == INTEGER && readInteger(i) == value) return true;
}
return false;
}
/**
* Checks if the constant pool contains the provided float constant, which implies the constant is used somewhere in the code.
*
* NB: compile-time constant expressions are evaluated at compile time.
*/
public boolean containsFloat(float value) {
boolean isNan = Float.isNaN(value);
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == FLOAT) {
float f = readFloat(i);
if (f == value || (isNan && Float.isNaN(f))) return true;
}
```

```
}
return false;
}
private long readLong(int index) {
int pos = offsets[index];
return ((long)read32(pos)) << 32 | (read32(pos + 4) & 0x00000000FFFFFFFFL);
}
private double readDouble(int index) {
return Double.longBitsToDouble(readLong(index));
}
private int readInteger(int index) {
return read32(offsets[index]);
}
private float readFloat(int index) {
return Float.intBitsToFloat(readInteger(index));
}
private int read32(int pos) {
return (byteCode[pos] & 0xFF) << 24 | (byteCode[pos + 1] & 0xFF) << 16 | (byteCode[pos + 2] & 0xFF)
<< 8 | (byteCode[pos + 3] & 0xFF);
}
/**
* Returns the name of the class in JVM format, such as {@code java/lang/String}
*/
public String getClassName() {
return getClassName(readValue(endOfPool + 2));
}
/**
* Returns the name of the superclass in JVM format, such as {@code java/lang/Object}
*
* NB: If you try this on Object itself, you'll get {@code null}.<br />
* NB2: For interfaces and annotation interfaces, you'll always get {@code java/lang/Object}
*/
public String getSuperClassName() {
return getClassName(readValue(endOfPool + 4));
}
/**
* Returns the name of all implemented interfaces.
*/
public List<String> getInterfaces() {
int size = readValue(endOfPool + 6);
if (size == 0) return Collections.emptyList();
List<String> result = new ArrayList<String>();
for (int i = 0; i < size; i++) {
result.add(getClassName(readValue(endOfPool + 8 + (i * 2))));
}
return result;
}
/**
* A {@code toString()} like utility to dump all contents of the constant pool into a string.
*
* NB: No guarantees are made about the exact layout of this string. It is for informational purposes only,
don't try to parse it.<br />
```

```
* NB2: After a double or long, there's a JVM spec-mandated gap, which is listed as {@code (cont.)} in
the returned string.
*/
public String poolContent() {
StringBuilder result = new StringBuilder();
for (int i = 1; i < maxPoolSize; i++) {
result.append(String.format("#%02x: ", i));
int pos = offsets[i];
switch(types[i]) {
case UTF8:
result.append("Utf8 ").append(utf8s[i]);
break;
case CLASS:
result.append("Class ").append(getClassName(i));
break;
case STRING:
result.append("String \"").append(utf8s[readValue(pos)]).append("\"");
break;
case INTEGER:
result.append("int ").append(readInteger(i));
break;
case FLOAT:
result.append("float ").append(readFloat(i));
break;
case FIELD:
appendAccess(result.append("Field "), i);
break;
case METHOD:
case INTERFACE_METHOD:
appendAccess(result.append("Method "), i);
break;
case NAME_TYPE:
appendNameAndType(result.append("Name&Type "), i);
break;
case LONG:
result.append("long ").append(readLong(i));
break;
case DOUBLE:
result.append("double ").append(readDouble(i));
break;
case METHOD_HANDLE:
result.append("MethodHandle...");
break;
case METHOD_TYPE:
result.append("MethodType...");
break;
case DYNAMIC:
result.append("Dynamic...");
break;
case INVOKE_DYNAMIC:
result.append("InvokeDynamic...");
break;
case 0:
```

```
result.append("(cont.)");
break;
}
result.append("\n");
}
return result.toString();
}
private void appendAccess(StringBuilder result, int index) {
int pos = offsets[index];
result.append(getClassName(readValue(pos))).append(".");
appendNameAndType(result, readValue(pos + 2));
}
private void appendNameAndType(StringBuilder result, int index) {
int pos = offsets[index];
result.append(utf8s[readValue(pos)]).append(":").append(utf8s[readValue(pos + 2)]);
}
private String getClassName(int classIndex) {
if (classIndex < 1) return null;
return utf8s[readValue(offsets[classIndex])];
}
private boolean isMethod(int i) {
byte type = types[i];
return type == METHOD || type == INTERFACE_METHOD;
}
private int findNameAndType(String name, String descriptor) {
int nameIndex = findUtf8(name);
if (nameIndex == NOT_FOUND) return NOT_FOUND;
int descriptorIndex = findUtf8(descriptor);
if (descriptorIndex == NOT_FOUND) return NOT_FOUND;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == NAME_TYPE &&
readValue(offsets[i]) == nameIndex &&
readValue(offsets[i] + 2) == descriptorIndex) return i;
}
return NOT_FOUND;
}
private int findUtf8(String value) {
for (int i = 1; i < maxPoolSize; i++) {
if (value.equals(utf8s[i])) {
return i;
}
}
return NOT_FOUND;
}
private int findClass(String className) {
int index = findUtf8(className);
if (index == -1) return NOT_FOUND;
for (int i = 1; i < maxPoolSize; i++) {
if (types[i] == CLASS && readValue(offsets[i]) == index) return i;
}
return NOT_FOUND;
}
/**
```

```
* Reads a 16-bit value at {@code position}
*/
private int readValue(int position) {
return ((byteCode[position] & 0xFF) << 8) | (byteCode[position + 1] & 0xFF);
}
}
```

lombok/launch/Main.java

```
/*
* Copyright (C) 2014-2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.launch;
import java.lang.reflect.InvocationTargetException;
import java.util.Arrays;
class Main {
private static ShadowClassLoader classLoader;
static synchronized ClassLoader getShadowClassLoader() {
if (classLoader == null) {
classLoader = new ShadowClassLoader(Main.class.getClassLoader(), "lombok", null,
Arrays.<String>asList(), Arrays.asList("lombok.patcher.Symbols"));
}
return classLoader;
}
static synchronized void prependClassLoader(ClassLoader loader) {
getShadowClassLoader();
classLoader.prependParent(loader);
}
public static void main(String[] args) throws Throwable {
ClassLoader cl = getShadowClassLoader();
Class<?> mc = cl.loadClass("lombok.core.Main");
try {
mc.getMethod("main", String[].class).invoke(null, new Object[] {args});
```

```
} catch (InvocationTargetException e) {
throw e.getCause();
}
}
}
```

lombok/launch/PackageShader.java

```
/*
* Copyright (C) 2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.launch;
import java.io.UnsupportedEncodingException;
import java.nio.charset.Charset;
/**
* Low-level shader tool, that inline 'rewrites' bytecode so that a given package name acts like it has a
different name.
*
* This is useful particularly in light of the java module system which disallows 'split packages' - different
loader systems / class file sources
* that both have the same package. Generally you'd take one of your dependency libraries (such as
lombok's dependency on ObjectWeb's ASM bytecode parser) and
* rename that dependency to avoid conflicts with other code that also wants to use ASM.
*
* This shader doesn't use ASM or any other bytecode manip library; it just scans constant pools.
*
* It's limited in its capabilities: A shading op (rename 'this' package to 'that') requires same-length
strings so that the replacement can be done without moving things around.
*/
class PackageShader {
private final byte[][] froms; // contains package prefixes that need shading.
private final byte[][] tos; // contains the post-shading package prefixes. froms.length == tos.length.
/**
```

```
* @param shadeOps provide pairs of strings; each pair is a from,to tuple listing binary package prefixes, e.g. "Lorg/objectweb/asm", "Lorg/lombokweb/asm".
*/
public PackageShader(String... shadeOps) {
if (shadeOps.length % 2 != 0) throw new IllegalArgumentException("Provide pairs: real package name to shaded package name (you provided an odd number of strings; even number required)");
Charset ascii = Charset.forName("US-ASCII");
int len = shadeOps.length / 2;
froms = new byte[len][];
tos = new byte[len][];
for (int i = 0; i < len; i ++) {
String in = shadeOps[i << 1];
String out = shadeOps[(i << 1) | 1];
if (in.contains(".") || out.contains(".")) throw new IllegalArgumentException("Binary name prefixes are required (use slashes and dollars instead of dots to separate type name elements); they look like e.g. 'java/util/'. Violating entry: " + in + " -> " + out);
froms[i] = in.getBytes(ascii);
tos[i] = out.getBytes(ascii);
if (froms[i].length != tos[i].length) throw new IllegalArgumentException("Pair [" + in + " -> " + out + "] is invalid: Both strings must be the same length");
}
}
private static final byte CONSTANTPOOLTYPE_UTF8 = 1;
/**
* Fuxxes the constant pool of the provided bytecode, rewriting any constant pool entries that represent types that start with any of the registered package shade requests to the shaded variant.
*
* The byte array is modified in-line.
*
* @param b Bytecode
* @return {@code true} if at least one shading modification has been applied.
*/
public boolean apply(byte[] b) {
ClassFileMetaData md = new ClassFileMetaData(b);
boolean changes = false;
// We want to replace the package name but it can occur in the middle of a 'signature' string, e.g. ([I[Ljava/util/List;)Ljava/util/List; - so replace from start and _after_ every capital-L.
// 'startPoints' tracks every location inside the constant pool UTF8 value that _could_ contain a fully qualified type name: Position 0 plus after _every_ capital-L in the UTF-8.
int[] startPoints = new int[260];
int maxStartPoints;
for (int offset : md.getOffsets(CONSTANTPOOLTYPE_UTF8)) {
int len = readValue(b, offset);
offset += 2;
startPoints[0] = offset;
maxStartPoints = 1;
for (int i = offset, max = offset + len; i < max; i++) {
if (b[i] == 'L') startPoints[maxStartPoints++] = i + 1;
}
for (int i = 0; i < froms.length; i++) {
outer: for (int startPointIdx = 0; startPointIdx < maxStartPoints; startPointIdx++) {
int indexIntoSignature = startPoints[startPointIdx];
if (len - (indexIntoSignature - offset) < froms[i].length) continue;
```

```
for (int p = 0; p < froms[i].length; p++) {
if (b[indexIntoSignature + p] != froms[i][p]) continue outer;
}
System.arraycopy(tos[i], 0, b, indexIntoSignature, tos[i].length);
changes = true;
}
}
}
return changes;
}
/**
* Reads a 16-bit value at {@code position}
*/
private static int readValue(byte[] b, int position) {
return ((b[position] & 0xFF) << 8) | (b[position + 1] & 0xFF);
}
public String reverseResourceName(String name) {
outer: for (int i = 0; i < tos.length; i++) {
int len = tos[i].length;
if (name.length() < len) continue;
for (int p = 0; p < len; p++) {
if (name.charAt(p) != tos[i][p]) continue outer;
}
try {
String out = new String(froms[i], 0, froms[i].length, "US-ASCII") + name.substring(len);
return out;
} catch (UnsupportedEncodingException e) {
// Can't happen - US-ASCII is guaranteed by the JVM spec
return name;
}
}
return name;
}
}
```

lombok/launch/ShadowClassLoader.java

```
/*
* Copyright (C) 2014-2023 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.launch;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.UnsupportedEncodingException;
import java.net.MalformedURLException;
import java.net.URI;
import java.net.URL;
import java.net.URLDecoder;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Enumeration;
import java.util.HashMap;
import java.util.HashSet;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Set;
import java.util.Vector;
import java.util.WeakHashMap;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import java.util.jar.JarEntry;
import java.util.jar.JarFile;
import java.util.zip.ZipEntry;
import java.util.zip.ZipInputStream;
/**
* The shadow classloader serves to completely hide almost all classes in a given jar file by using a different file ending.
*
* The shadow classloader also serves to link in a project as it is being developed (a 'bin' dir from an IDE for example).
* <p>
* Classes loaded by the shadowloader use ".SCL.sclSuffix" in addition to ".class". In other words, most of the class files in a given jar end in this suffix, which
* serves to hide them from any tool that isn't aware of the suffix (such as IDEs generating auto-complete dialogs, and javac's classpath in general). Only shadowloader can actually
* load these classes.
* <p>
* The shadowloader will pick up an alternate (priority) classpath, using normal class files, from the system property "<code>shadow.override.sclSuffix</code>".

```
* This shadow classpath looks just like a normal java classpath; the path separator is applied (semi-colon on windows, colon elsewhere), and entries can consist of directories,
* jar files, or directories ending in "/*" to pick up all jars inside it.
* <p>
* Load order is as follows if at least one override is present:
* <li>First, if the resource is found in one of the paths stated in the shadow classpath, find that.
* <li>Next, ask the <code>parent</code> loader, which is passed during construction of the ShadowClassLoader.
* <li>Notably, this jar's contents are always skipped! (The idea of the shadow classpath is that this jar only functions as a launcher, not as a source of your actual application).
* </ul>
*
* If no overrides are present, the load order is as follows:
* <li>First, if the resource is found in our own jar (trying ".SCL.<em>sclSuffix</em>" first for any resource request ending in ".class"), return that.
* <li>Next, check any jar files other than our own, loading them via this classloader, if they have a file <code>META-INF/ShadowClassLoader</code> that contains a line of text with <em>sclSuffix</em>.
* <li>Next, ask the <code>parent</code> loader.
* </ul>
*
* Use ShadowClassLoader to accomplish the following things:<ul>
* <li>Avoid contaminating the namespace of any project using an SCL-based jar. Autocompleters in IDEs will NOT suggest anything other than actual public API.
* <li>Like jarjar, allows folding in dependencies such as ASM without foisting these dependencies on projects that use this jar. shadowloader obviates the need for jarjar.
* <li>Allows an agent (which MUST be in jar form) to still load everything except this loader infrastructure from class files generated by the IDE, which should
* considerably help debugging, as you can now rely on the IDE's built-in auto-recompile features instead of having to run a full build everytime, and it should help
* with hot code replace and the like (this is what the {@code shadow.override} feature is for).
* </ul>
*
* Implementation note: {@code lombok.eclipse.agent.EclipseLoaderPatcher} <em>relies</em> on this class having no dependencies on any other class except the JVM boot class and
* package private classes inside this package <strong>explicitly</strong> listed in EclipseLoaderPatcherTransplants's {@code overrideLoadResult} (near the top). This notably
* including any other classes in this package, <strong>including</strong> inner classes. So, don't write closures, anonymous inner class literals,
* enums, or anything else that could cause the compilation of this file to produce more than 1 class file. In general, actually passing load control to this loader is a bit tricky
* so ensure that this class has zero dependencies on anything except java core classes.
*/
class ShadowClassLoader extends ClassLoader {
private static final String SELF_NAME = "lombok/launch/ShadowClassLoader.class";
private static final ConcurrentMap<String, Class<?>> highlanderMap = new ConcurrentHashMap<String, Class<?>>();
private final String SELF_BASE;
private final File SELF_BASE_FILE;
private final int SELF_BASE_LENGTH;
private final List<File> override = new ArrayList<File>();
private final String sclSuffix;
private final List<String> parentExclusion = new ArrayList<String>();
private final List<String> highlanders = new ArrayList<String>();
```

```
private final Set<ClassLoader> prependedParentLoaders = Collections.newSetFromMap(new
IdentityHashMap<ClassLoader, Boolean>());
private final PackageShader shader = new PackageShader(
"org/objectweb/asm/", "org/lombokweb/asm/");
public void prependParent(ClassLoader loader) {
if (loader == null) return;
if (loader == getParent()) return;
if (loader == this) return;
prependedParentLoaders.add(loader);
}
/**
* @param source The 'parent' classloader.
* @param sclSuffix The suffix of the shadowed class files in our own jar. For example, if this is {@code
lombok}, then the class files in your jar should be {@code foo/Bar.SCL.lombok} and not {@code
foo/Bar.class}.
* @param selfBase The (preferably absolute) path to our own jar. This jar will be searched for
class/SCL.sclSuffix files.
* @param parentExclusion For example {@code "lombok."}; upon invocation of loadClass of this
loader, the parent loader ({@code source}) will NOT be invoked if the class to be loaded begins with
anything in the parent exclusion list. No exclusion is applied for getResource(s).
* @param highlanders SCL will put in extra effort to ensure that these classes (in simple class spec, so
{@code foo.bar.baz.ClassName}) are only loaded once as a class, even if many different classloaders
try to load classes, such as equinox/OSGi.
*/
ShadowClassLoader(ClassLoader source, String sclSuffix, String selfBase, List<String>
parentExclusion, List<String> highlanders) {
super(source);
this.sclSuffix = sclSuffix;
if (parentExclusion != null) for (String pe : parentExclusion) {
pe = pe.replace(".", "/");
if (!pe.endsWith("/")) pe = pe + "/";
this.parentExclusion.add(pe);
}
if (highlanders != null) for (String hl : highlanders) this.highlanders.add(hl);
if (selfBase != null) {
SELF_BASE = selfBase;
SELF_BASE_LENGTH = selfBase.length();
} else {
URL sclClassUrl = ShadowClassLoader.class.getResource("ShadowClassLoader.class");
String sclClassStr = sclClassUrl == null ? null : sclClassUrl.toString();
if (sclClassStr == null || !sclClassStr.endsWith(SELF_NAME)) {
ClassLoader cl = ShadowClassLoader.class.getClassLoader();
throw new RuntimeException("ShadowLoader can't find itself. SCL loader type: " + (cl == null ?
"*NULL*" : cl.getClass().toString()));
}
SELF_BASE_LENGTH = sclClassStr.length() - SELF_NAME.length();
String decoded = urlDecode(sclClassStr.substring(0, SELF_BASE_LENGTH));
SELF_BASE = decoded;
}
if (SELF_BASE.startsWith("jar:file:") && SELF_BASE.endsWith("!/")) SELF_BASE_FILE = new
File(SELF_BASE.substring(9, SELF_BASE.length() - 2));
else if (SELF_BASE.startsWith("file:")) SELF_BASE_FILE = new File(SELF_BASE.substring(5));
else SELF_BASE_FILE = new File(SELF_BASE);
```

```
String scl = System.getProperty("shadow.override." + sclSuffix);
if (scl != null && !scl.isEmpty()) {
for (String part : scl.split("\\s*" + (File.pathSeparatorChar == ';' ? ";" : ":") + "\\s*")) {
if (part.endsWith("/*") || part.endsWith(File.separator + "*")) {
addOverrideJarDir(part.substring(0, part.length() - 2));
} else {
addOverrideClasspathEntry(part);
}
}
}
}
private final Map<String, Object> mapJarPathToTracker = new HashMap<String, Object>();
private static final Map<Object, String> mapTrackerToJarPath = new WeakHashMap<Object, String>();
private static final Map<Object, Set<String>> mapTrackerToJarContents = new
WeakHashMap<Object, Set<String>>();
/**
* This cache ensures that any given jar file is only opened once in order to determine the full contents of
it.
* We use 'trackers' to make sure that the bulk of the memory taken up by this cache (the list of strings
representing the content of a jar file)
* gets garbage collected if all ShadowClassLoaders that ever tried to request a listing of this jar file, are
garbage collected.
*/
private Set<String> getOrMakeJarListing(final String absolutePathToJar) {
synchronized (mapTrackerToJarPath) {
/*
* 1) Check our private instance JarPath-to-Tracker Mappings:
*/
Object ourTracker = mapJarPathToTracker.get(absolutePathToJar);
if (ourTracker != null) {
/*
* Yes, we are already tracking this Jar. Just return its contents...
*/
return mapTrackerToJarContents.get(ourTracker);
}
/*
* 2) Not tracked by us as yet. Check statically whether others have tracked this JarPath:
*/
for (Entry<Object, String> entry : mapTrackerToJarPath.entrySet()) {
if (entry.getValue().equals(absolutePathToJar)) {
/*
* Yes, 3rd party is tracking this jar. We must track too, then return its contents.
*/
Object otherTracker = entry.getKey();
mapJarPathToTracker.put(absolutePathToJar, otherTracker);
return mapTrackerToJarContents.get(otherTracker);
}
}
/*
* 3) Not tracked by anyone so far. Build, publish, track & return Jar contents...
*/
Object newTracker = new Object();
Set<String> jarMembers = getJarMemberSet(absolutePathToJar);
```

```
mapTrackerToJarContents.put(newTracker, jarMembers);
mapTrackerToJarPath.put(newTracker, absolutePathToJar);
mapJarPathToTracker.put(absolutePathToJar, newTracker);
return jarMembers;
}
}
/**
* Return a {@link Set} of members in the Jar identified by {@code absolutePathToJar}.
*
* @param absolutePathToJar Cache key
* @return a Set with the Jar member-names
*/
private Set<String> getJarMemberSet(String absolutePathToJar) {
/*
* Note:
* Our implementation returns a HashSet. initialCapacity and loadFactor are carefully tweaked for speed
and RAM optimization purposes.
*
* Benchmark:
* The HashSet implementation is about 10% slower to build (only happens once) than the ArrayList.
* The HashSet with shiftBits = 1 was about 33 times(!) faster than the ArrayList for retrievals.
*/
try {
int shiftBits = 1; // (fast, but big) 0 <= shiftBits <= 5, say (slower & compact)
JarFile jar = new JarFile(absolutePathToJar);
/*
* Find the first power of 2 >= JarSize (as calculated in HashSet constructor)
*/
int jarSizePower2 = Integer.highestOneBit(jar.size());
if (jarSizePower2 != jar.size()) jarSizePower2 <<= 1;
if (jarSizePower2 == 0) jarSizePower2 = 1;
Set<String> jarMembers = new HashSet<String>(jarSizePower2 >> shiftBits, 1 << shiftBits);
try {
Enumeration<JarEntry> entries = jar.entries();
while (entries.hasMoreElements()) {
JarEntry jarEntry = entries.nextElement();
if (jarEntry.isDirectory()) continue;
jarMembers.add(jarEntry.getName());
}
} catch (Exception ignore) {
// ignored; if the jar can't be read, treating it as if the jar contains no classes is just what we want.
} finally {
jar.close();
}
return jarMembers;
}
catch (Exception newJarFileException) {
return Collections.emptySet();
}
}
/**
* Looks up {@code altName} in {@code location}, and if that isn't found, looks up {@code name};
{@code altName} can be null in which case it is skipped.
```

```
*/
private URL getResourceFromLocation(String name, String altName, File location) {
if (location.isDirectory()) {
try {
if (altName != null) {
File f = new File(location, altName);
if (f.isFile() && f.canRead()) return f.toURI().toURL();
}
File f = new File(location, name);
if (f.isFile() && f.canRead()) return f.toURI().toURL();
return null;
} catch (MalformedURLException e) {
return null;
}
}
if (!location.isFile() || !location.canRead()) return null;
File absoluteFile; {
try {
absoluteFile = location.getCanonicalFile();
} catch (Exception e) {
absoluteFile = location.getAbsoluteFile();
}
}
Set<String> jarContents = getOrMakeJarListing(absoluteFile.getAbsolutePath());
String absoluteUri = absoluteFile.toURI().toString();
try {
if (jarContents.contains(altName)) {
return new URI("jar:" + absoluteUri + "!/" + altName).toURL();
}
} catch (Exception ignore) {
// intentional fallthrough
}
try {
if (jarContents.contains(name)) {
return new URI("jar:" + absoluteUri + "!/" + name).toURL();
}
} catch(Exception ignore) {
// intentional fallthrough
}
return null;
}
private boolean partOfShadow(String item, String name) {
return !name.startsWith("java/")
&& !name.startsWith("sun/")
&& (inOwnBase(item, name) || isPartOfShadowSuffix(item, name, sclSuffix));
}
/**
* Checks if the stated item is located inside the same classpath root as the jar that hosts
ShadowClassLoader.class. {@code item} and {@code name} refer to the same thing.
*/
private boolean inOwnBase(String item, String name) {
if (item == null) return false;
```

```
return (item.length() == SELF_BASE_LENGTH + name.length()) && SELF_BASE.regionMatches(0,
item, 0, SELF_BASE_LENGTH);
}
private static boolean sclFileContainsSuffix(InputStream in, String suffix) throws IOException {
BufferedReader br = new BufferedReader(new InputStreamReader(in, "UTF-8"));
for (String line = br.readLine(); line != null; line = br.readLine()) {
line = line.trim();
if (line.isEmpty() || line.charAt(0) == '#') continue;
if (line.equals(suffix)) return true;
}
return false;
}
private static String urlDecode(String in) {
final String plusFixed = in.replaceAll("\\+", "%2B");
try {
return URLDecoder.decode(plusFixed, "UTF-8");
} catch (UnsupportedEncodingException e) {
throw new InternalError("UTF-8 not supported");
}
}
private Map<String, Boolean> fileRootCache = new HashMap<String, Boolean>();
private boolean isPartOfShadowSuffixFileBased(String fileRoot, String suffix) {
String key = fileRoot + "::" + suffix;
Boolean existing = fileRootCache.get(key);
if (existing != null) return existing.booleanValue();
File f = new File(fileRoot + "/META-INF/ShadowClassLoader");
try {
FileInputStream fis = new FileInputStream(f);
try {
boolean v = sclFileContainsSuffix(fis, suffix);
fileRootCache.put(key, v);
return v;
} finally {
fis.close();
}
} catch (FileNotFoundException fnfEx) {
fileRootCache.put(key, false);
return false;
} catch (IOException e) {
fileRootCache.put(key, false);
return false; // *unexpected*
}
}
private Map<String, Boolean> jarLocCache = new HashMap<String, Boolean>();
private boolean isPartOfShadowSuffixJarBased(String jarLoc, String suffix) {
String key = jarLoc + "::" + suffix;
Boolean existing = jarLocCache.get(key);
if (existing != null) return existing.booleanValue();
if (jarLoc.startsWith("file:/")) jarLoc = urlDecode(jarLoc.substring(5));
try {
FileInputStream jar = new FileInputStream(jarLoc);
try {
ZipInputStream zip = new ZipInputStream(jar);
```

```
try {
while (true) {
ZipEntry entry = zip.getNextEntry();
if (entry == null) {
jarLocCache.put(key, false);
return false;
}
if (!"META-INF/ShadowClassLoader".equals(entry.getName())) continue;
boolean v = sclFileContainsSuffix(zip, suffix);
jarLocCache.put(key, v);
return v;
}
} finally {
zip.close();
}
} finally {
jar.close();
}
} catch (FileNotFoundException fnfEx) {
jarLocCache.put(key, false);
return false;
} catch (IOException ex) {
jarLocCache.put(key, false);
return false; // *unexpected*
}
}
private boolean isPartOfShadowSuffix(String url, String name, String suffix) {
// Instead of throwing an exception or logging, weird, unexpected cases just return false.
// This is better than throwing an exception, because exceptions would make your build tools unusable.
// Such cases are marked with the comment: // *unexpected*
if (url == null) return false;
if (url.startsWith("file:/")) {
url = urlDecode(url.substring(5));
if (url.length() <= name.length() || !url.endsWith(name) || url.charAt(url.length() - name.length() - 1) != '/')
{
return false; // *unexpected*
}
String fileRoot = url.substring(0, url.length() - name.length() - 1);
return isPartOfShadowSuffixFileBased(fileRoot, suffix);
} else if (url.startsWith("jar:")) {
int sep = url.indexOf('!');
if (sep == -1) {
return false; // *unexpected*
}
String jarLoc = url.substring(4, sep);
return isPartOfShadowSuffixJarBased(jarLoc, suffix);
}
return false;
}
private String toSclResourceName(String name) {
return "SCL." + sclSuffix + "/" + name.substring(0, name.length() - 6) + ".SCL." + sclSuffix;
}
@Override public Enumeration<URL> getResources(String name) throws IOException {
```

```
String altName = null;
if (name.endsWith(".class")) altName = toSclResourceName(name);
// Vector? Yes, we need one:
// * We can NOT make inner classes here (this class is loaded with special voodoo magic in eclipse, as a one off, it's not a full loader.
// * We need to return an enumeration.
// * We can't make one on the fly.
// * ArrayList can't make these.
Vector<URL> vector = new Vector<URL>();
for (File ce : override) {
URL url = getResourceFromLocation(name, altName, ce);
if (url != null) vector.add(url);
}
if (override.isEmpty()) {
URL fromSelf = getResourceFromLocation(name, altName, SELF_BASE_FILE);
if (fromSelf != null) vector.add(fromSelf);
}
Enumeration<URL> sec = super.getResources(name);
while (sec.hasMoreElements()) {
URL item = sec.nextElement();
if (isPartOfShadowSuffix(item.toString(), name, sclSuffix)) vector.add(item);
}
if (altName != null) {
Enumeration<URL> tern = super.getResources(altName);
while (tern.hasMoreElements()) {
URL item = tern.nextElement();
if (isPartOfShadowSuffix(item.toString(), altName, sclSuffix)) vector.add(item);
}
}
return vector.elements();
}
@Override public URL getResource(String name) {
return getResource_(name, false);
}
private URL getResource_(String name, boolean noSuper) {
String altName = null;
name = shader == null ? name : shader.reverseResourceName(name);
if (name.endsWith(".class")) altName = toSclResourceName(name);
for (File ce : override) {
URL url = getResourceFromLocation(name, altName, ce);
if (url != null) return url;
}
if (!override.isEmpty()) {
if (noSuper) return null;
if (altName != null) {
try {
URL res = getResourceSkippingSelf(altName);
if (res != null) return res;
} catch (IOException ignore) {}
}
try {
return getResourceSkippingSelf(name);
} catch (IOException e) {
```

```
return null;
}
}
URL url = getResourceFromLocation(name, altName, SELF_BASE_FILE);
if (url != null) return url;
if (altName != null) {
URL res = super.getResource(altName);
if (res != null && (!noSuper || partOfShadow(res.toString(), altName))) return res;
}
URL res = super.getResource(name);
if (res != null && (!noSuper || partOfShadow(res.toString(), name))) return res;
return null;
}
private boolean exclusionListMatch(String name) {
for (String pe : parentExclusion) {
if (name.startsWith(pe)) return true;
}
return false;
}
private URL getResourceSkippingSelf(String name) throws IOException {
URL candidate = super.getResource(name);
if (candidate == null) return null;
if (!partOfShadow(candidate.toString(), name)) return candidate;
Enumeration<URL> en = super.getResources(name);
while (en.hasMoreElements()) {
candidate = en.nextElement();
if (!partOfShadow(candidate.toString(), name)) return candidate;
}
return null;
}
@Override public Class<?> loadClass(String name, boolean resolve) throws ClassNotFoundException
{
{
Class<?> alreadyLoaded = findLoadedClass(name);
if (alreadyLoaded != null) return alreadyLoaded;
}
if (highlanders.contains(name)) {
Class<?> c = highlanderMap.get(name);
if (c != null) return c;
}
String fileNameOfClass = name.replace(".", "/") + ".class";
URL res = getResource_(fileNameOfClass, true);
if (res == null) {
if (!exclusionListMatch(fileNameOfClass)) {
try {
// First search in the prepended classloaders, the class might be there already
for (ClassLoader pre : prependedParentLoaders) {
try {
Class<?> loadClass = pre.loadClass(name);
if (loadClass != null) return loadClass;
} catch (Throwable e) {
continue;
}
```

```
}
return super.loadClass(name, resolve);
} catch (ClassNotFoundException cnfe) {
res = getResource_("secondaryLoading.SCL." + sclSuffix + "/" + name.replace(".", "/") + ".SCL." +
sclSuffix, true);
if (res == null) throw cnfe;
}
}
}
if (res == null) throw new ClassNotFoundException(name);
return urlToDefineClass(name, res, resolve);
}
private Class<?> urlToDefineClass(String name, URL res, boolean resolve) throws
ClassNotFoundException {
byte[] b;
int p = 0;
try {
InputStream in = res.openStream();
try {
b = new byte[65536];
while (true) {
int r = in.read(b, p, b.length - p);
if (r == -1) break;
p += r;
if (p == b.length) {
byte[] nb = new byte[b.length * 2];
System.arraycopy(b, 0, nb, 0, p);
b = nb;
}
}
} finally {
in.close();
}
} catch (IOException e) {
throw new ClassNotFoundException("I/O exception reading class " + name, e);
}
if (shader != null) shader.apply(b);
Class<?> c;
try {
c = defineClass(name, b, 0, p);
} catch (LinkageError e) {
if (highlanders.contains(name)) {
Class<?> alreadyDefined = highlanderMap.get(name);
if (alreadyDefined != null) return alreadyDefined;
}
try {
c = this.findLoadedClass(name);
} catch (LinkageError e2) {
throw e;
}
if (c == null) throw e;
}
if (highlanders.contains(name)) {
```

```
Class<?> alreadyDefined = highlanderMap.putIfAbsent(name, c);
if (alreadyDefined != null) c = alreadyDefined;
}
if (resolve) resolveClass(c);
return c;
}
public void addOverrideJarDir(String dir) {
File f = new File(dir);
for (File j : f.listFiles()) {
if (j.getName().toLowerCase().endsWith(".jar") && j.canRead() && j.isFile()) override.add(j);
}
}
public void addOverrideClasspathEntry(String entry) {
override.add(new File(entry));
}
}
```

lombok/core/ClassLiteral.java

```
/*
* Copyright (C) 2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
public class ClassLiteral {
private final String className;
public ClassLiteral(String className) {
this.className = className;
}
public String getClassName() {
return className;
}
}
```

lombok/core/FieldAugment.java

```
/*
* Copyright (C) 2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.lang.ref.WeakReference;
import java.lang.reflect.Field;
import java.lang.reflect.Modifier;
import java.util.HashMap;
import java.util.Map;
import java.util.WeakHashMap;
import lombok.permit.Permit;
public abstract class FieldAugment<T, F> {
private static Object getDefaultValue(Class<?> type) {
if (type == boolean.class) return false;
if (type == int.class) return 0;
if (!type.isPrimitive()) return null;
if (type == long.class) return 0L;
if (type == short.class) return (short) 0;
if (type == byte.class) return (byte) 0;
if (type == char.class) return '\0';
if (type == float.class) return 0.0F;
if (type == double.class) return 0.0D;
// We can't get here unless java added some primitive types, but, hey.
return null;
}
/**
* (Virtually) adds a field to an existing type and returns an object that can be used to read and write this field.
* <p>
* If the type already declares a non-final instance field with the given name and a compatible field type, that field will be used.
```

```
* Otherwise the field will be provided virtually.
* <p>
* <em>WARNING</em>: The values put into the augment should NOT reference in any way the object you've added the augment to, or memory leaks may occur.
* If you do need to add such references, use {@link #circularSafeAugment(Class, Class, String, Object)} instead.
* <p>
* This code assumes that for any combination of {@code type} and {@code name} this method is only called once.
* Otherwise, whether state is shared is undefined.
*
* @param type to augment
* @param fieldType type of the field
* @param name of the field
* @param defaultValue the value of the augment if it hasn't been set yet.
* @throws NullPointerException if {@code type}, {@code fieldType} or {@code name} is {@code null}
*/
public static <T, F> FieldAugment<T, F> augment(Class<T> type, Class<? super F> fieldType, String name) {
checkNotNull(type, "type");
checkNotNull(fieldType, "fieldType");
checkNotNull(name, "name");
if (type.isInterface()) {
return new InterfaceFieldAugment<T, F>(name, fieldType);
}
@SuppressWarnings("unchecked")
F defaultValue = (F) getDefaultValue(fieldType);
FieldAugment<T, F> ret = tryCreateReflectionAugment(type, fieldType, name, defaultValue);
return ret != null ? ret : new MapFieldAugment<T, F>(defaultValue);
}
/**
* (Virtually) adds a field to an existing type and returns an object that can be used to read and write this field.
* <p>
* This method does the same as {@link #augment(Class, Class, String, Object)}, except it is safe to set values that reference back to their containing object.
*/
public static <T, F> FieldAugment<T, F> circularSafeAugment(Class<T> type, Class<? super F> fieldType, String name) {
checkNotNull(type, "type");
checkNotNull(fieldType, "fieldType");
checkNotNull(name, "name");
@SuppressWarnings("unchecked")
F defaultValue = (F) getDefaultValue(fieldType);
FieldAugment<T, F> ret = tryCreateReflectionAugment(type, fieldType, name, defaultValue);
return ret != null ? ret : new MapWeakFieldAugment<T, F>(defaultValue);
}
/**
* Creates a reflection-based augment which will directly access the listed field name. If this field does not exist or the field
* is not capable of storing the requested type, {@code null} is returned instead.
*/
```

```
private static <T, F> FieldAugment<T, F> tryCreateReflectionAugment(Class<T> type, Class<? super
F> fieldType, String name, F defaultValue) {
Field f = findField(type, fieldType, name);
if (f != null && typeIsAssignmentCompatible(f.getType(), fieldType)) return new
ReflectionFieldAugment<T, F>(f, fieldType, defaultValue);
return null;
}
private static Field findField(Class<?> type, Class<?> wantedType, String name) {
try {
Field f = Permit.getField(type, name);
if (Modifier.isStatic(f.getModifiers()) || Modifier.isFinal(f.getModifiers())) return null;
if (!typeIsAssignmentCompatible(f.getType(), wantedType)) return null;
return f;
} catch (Exception e) {
return null;
}
}
private static boolean typeIsAssignmentCompatible(Class<?> fieldType, Class<?> wantedType) {
if (fieldType == java.lang.Object.class) return true;
if (fieldType == wantedType) return true;
if (fieldType.isPrimitive()) return fieldType == wantedType;
if (wantedType == int.class && (fieldType == Number.class || fieldType == Integer.class)) return true;
if (wantedType == long.class && (fieldType == Number.class || fieldType == Long.class)) return true;
if (wantedType == short.class && (fieldType == Number.class || fieldType == Short.class)) return true;
if (wantedType == byte.class && (fieldType == Number.class || fieldType == Byte.class)) return true;
if (wantedType == char.class && (fieldType == Number.class || fieldType == Character.class)) return
true;
if (wantedType == float.class && (fieldType == Number.class || fieldType == Float.class)) return true;
if (wantedType == double.class && (fieldType == Number.class || fieldType == Double.class)) return
true;
if (wantedType == boolean.class && fieldType == Boolean.class) return true;
return fieldType.isAssignableFrom(wantedType);
}
private FieldAugment() {
// prevent external instantiation
}
/**
* @throws NullPointerException if {@code object} is {@code null}
*/
public abstract F get(T object);
/**
* @throws NullPointerException if {@code object} or {@code value} is {@code null}
*/
public final void set(T object, F value) {
getAndSet(object, value);
}
/**
* @return the value of the field <strong>before</strong> the operation.
* @throws NullPointerException if {@code object} or {@code value} is {@code null}.
*/
public abstract F getAndSet(T object, F value);
/**
* @return the value of the field <strong>before</strong> the operation.
```

```
* @throws NullPointerException if {@code object} is {@code null}
*/
public abstract F clear(T object);
/**
* @return the value of the field <strong>after</strong> the operation. If the value was equal to {@code
expected} or already cleared {@code null}, otherwise the current value.
* @throws NullPointerException if {@code object} or {@code expected} is {@code null}
*/
public abstract F compareAndClear(T object, F expected);
/**
* @return the value of the field <strong>after</strong> the operation.
* @throws NullPointerException if {@code object} or {@code value} is {@code null}
*/
public abstract F setIfAbsent(T object, F value);
/**
* @return the value of the field <strong>after</strong> the operation.
* @throws NullPointerException if {@code object}, {@code expected} or {@code value} is {@code null}
*/
public abstract F compareAndSet(T object, F expected, F value);
private static final class InterfaceFieldAugment<T, F> extends FieldAugment<T, F> {
private final String name;
private final Class<? super F> fieldType;
private Map<Class<T>, FieldAugment<T, F>> map = new HashMap<Class<T>,
FieldAugment<T,F>>();
private InterfaceFieldAugment(String name, Class<? super F> fieldType) {
this.name = name;
this.fieldType = fieldType;
}
private synchronized FieldAugment<T, F> getDelegate(T object) {
@SuppressWarnings("unchecked")
Class<T> c = (Class<T>) object.getClass();
FieldAugment<T,F> fieldAugment = map.get(c);
if (fieldAugment == null) {
fieldAugment = augment(c, fieldType, name);
map.put(c, fieldAugment);
}
return fieldAugment;
}
@Override public F get(T object) {
return getDelegate(object).get(object);
}
@Override public F getAndSet(T object, F value) {
return getDelegate(object).getAndSet(object, value);
}
@Override public F clear(T object) {
return getDelegate(object).clear(object);
}
@Override public F compareAndClear(T object, F expected) {
return getDelegate(object).compareAndClear(object, expected);
}
@Override public F setIfAbsent(T object, F value) {
return getDelegate(object).setIfAbsent(object, value);
}
```

```
@Override public F compareAndSet(T object, F expected, F value) {
return getDelegate(object).compareAndSet(object, expected, value);
}
}
private static class ReflectionFieldAugment<T, F> extends FieldAugment<T, F> {
private final Object lock = new Object();
private final Field field;
private final Class<F> targetType;
private final F defaultValue;
@SuppressWarnings("unchecked")
ReflectionFieldAugment(Field field, Class<? super F> targetType, F defaultValue) {
this.field = field;
this.targetType = (Class<F>) targetType;
this.defaultValue = defaultValue;
}
@Override public F get(T object) {
checkNotNull(object, "object");
try {
F value;
synchronized (lock) {
value = targetType.cast(field.get(object));
}
return value == null ? defaultValue : value;
} catch (IllegalAccessException e) {
throw new IllegalStateException(e);
}
}
@Override public F getAndSet(T object, F value) {
checkNotNull(object, "object");
checkNotNull(value, "value");
try {
F oldValue;
synchronized (lock) {
oldValue = targetType.cast(field.get(object));
field.set(object, value);
}
return oldValue == null ? defaultValue : oldValue;
} catch (IllegalAccessException e) {
throw new IllegalStateException(e);
}
}
@Override public F clear(T object) {
checkNotNull(object, "object");
try {
F oldValue;
synchronized (lock) {
oldValue = targetType.cast(field.get(object));
field.set(object, defaultValue);
}
return oldValue == null ? defaultValue : oldValue;
} catch (IllegalAccessException e) {
throw new IllegalStateException(e);
}
```

```
}
@Override public F compareAndClear(T object, F expected) {
checkNotNull(object, "object");
checkNotNull(expected, "expected");
try {
F oldValue;
synchronized (lock) {
oldValue = targetType.cast(field.get(object));
if (expected.equals(oldValue)) {
field.set(object, defaultValue);
return defaultValue;
}
}
return oldValue;
} catch (IllegalAccessException e) {
throw new IllegalStateException(e);
}
}
@Override public F setIfAbsent(T object, F value) {
checkNotNull(object, "object");
checkNotNull(value, "value");
try {
synchronized (lock) {
F oldValue = targetType.cast(field.get(object));
if (oldValue != null && !oldValue.equals(defaultValue)) return oldValue;
field.set(object, value);
return value;
}
} catch (IllegalAccessException e) {
throw new IllegalStateException(e);
}
}
@Override public F compareAndSet(T object, F expected, F value) {
checkNotNull(object, "object");
checkNotNull(expected, "expected");
checkNotNull(value, "value");
try {
synchronized (lock) {
F oldValue = targetType.cast(field.get(object));
if (!expected.equals(oldValue)) return oldValue == null ? defaultValue : oldValue;
field.set(object, value);
return value;
}
} catch (IllegalAccessException e) {
throw new IllegalStateException(e);
}
}
}
private static class MapFieldAugment<T, F> extends FieldAugment<T, F> {
final Map<T, Object> values = new WeakHashMap<T, Object>();
final F defaultValue;
MapFieldAugment(F defaultValue) {
this.defaultValue = defaultValue;
```

```
}
@Override
public F get(T object) {
checkNotNull(object, "object");
synchronized (values) {
return read(object);
}
}
@Override
public F getAndSet(T object, F value) {
checkNotNull(object, "object");
checkNotNull(value, "value");
synchronized (values) {
F result = read(object);
write(object, value);
return result;
}
}
@Override
public F clear(T object) {
checkNotNull(object, "object");
synchronized (values) {
F result = read(object);
values.remove(object);
return result;
}
}
@Override
public F compareAndClear(T object, F expected) {
checkNotNull(object, "object");
checkNotNull(expected, "expected");
synchronized (values) {
F result = read(object);
if (result == null) {
return null;
}
if (!expected.equals(result)) {
return result;
}
values.remove(object);
return null;
}
}
@Override
public F setIfAbsent(T object, F value) {
checkNotNull(object, "object");
checkNotNull(value, "value");
synchronized (values) {
F result = read(object);
if (result != null) {
return result;
}
write(object, value);
```

```
return value;
}
}
@Override
public F compareAndSet(T object, F expected, F value) {
checkNotNull(object, "object");
checkNotNull(expected, "expected");
checkNotNull(value, "value");
synchronized (values) {
F result = read(object);
if (!expected.equals(result)) {
return result;
}
write(object, value);
return value;
}
}
@SuppressWarnings("unchecked")
F read(T object) {
F value = (F) values.get(object);
return value == null ? defaultValue : value;
}
void write(T object, F value) {
values.put(object, value);
}
}
static class MapWeakFieldAugment<T, F> extends MapFieldAugment<T, F> {
MapWeakFieldAugment(F defaultValue) {
super(defaultValue);
}
@SuppressWarnings("unchecked")
F read(T object) {
WeakReference<F> read = (WeakReference<F>)values.get(object);
if (read == null) return defaultValue;
F result = read.get();
if (result == null) values.remove(object);
return result == null ? defaultValue : result;
}
void write(T object, F value) {
values.put(object, new WeakReference<F>(value));
}
}
private static <T> T checkNotNull(T object, String name) {
if (object == null) throw new NullPointerException(name);
return object;
}
}
```

lombok/core/FieldSelect.java

```
/*
```

* Copyright (C) 2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.core;
public class FieldSelect {
private final String finalPart;
public FieldSelect(String finalPart) {
this.finalPart = finalPart;
}
public String getFinalPart() {
return finalPart;
}
}
```

lombok/core/JavaIdentifiers.java

/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,

```
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.regex.Pattern;
/**
* Utility functions for validating potential java verifiers.
*/
public class JavaIdentifiers {
private JavaIdentifiers() {}
private static final LombokImmutableList<String> KEYWORDS = LombokImmutableList.of(
"public", "private", "protected",
"default", "switch", "case",
"for", "do", "goto", "const", "strictfp", "while", "if", "else",
"byte", "short", "int", "long", "float", "double", "void", "boolean", "char",
"null", "false", "true",
"continue", "break", "return", "instanceof",
"synchronized", "volatile", "transient", "final", "static",
"interface", "class", "extends", "implements", "throws",
"throw", "catch", "try", "finally", "abstract", "assert",
"enum", "import", "package", "native", "new", "super", "this");
public static boolean isValidJavaIdentifier(String identifier) {
if (identifier == null) return false;
if (identifier.isEmpty()) return false;
if (!Character.isJavaIdentifierStart(identifier.charAt(0))) return false;
for (int i = 1; i < identifier.length(); i++) {
if (!Character.isJavaIdentifierPart(identifier.charAt(i))) return false;
}
return !isKeyword(identifier);
}
public static boolean isKeyword(String keyword) {
return KEYWORDS.contains(keyword);
}
/** Matches any of the 8 primitive names, such as {@code boolean}. */
private static final Pattern PRIMITIVE_TYPE_NAME_PATTERN =
Pattern.compile("^(?:boolean|byte|short|int|long|float|double|char)$");
public static boolean isPrimitive(String typeName) {
return PRIMITIVE_TYPE_NAME_PATTERN.matcher(typeName).matches();
}
}
```

lombok/core/LombokImmutableList.java

```
/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
```

```
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;
public final class LombokImmutableList<T> implements Iterable<T> {
private Object[] content;
private static final LombokImmutableList<?> EMPTY = new LombokImmutableList<Object>(new
Object[0]);
@SuppressWarnings("unchecked")
public static <T> LombokImmutableList<T> of() {
return (LombokImmutableList<T>) EMPTY;
}
public static <T> LombokImmutableList<T> of(T a) {
return new LombokImmutableList<T>(new Object[] {a});
}
public static <T> LombokImmutableList<T> of(T a, T b) {
return new LombokImmutableList<T>(new Object[] {a, b});
}
public static <T> LombokImmutableList<T> of(T a, T b, T c) {
return new LombokImmutableList<T>(new Object[] {a, b, c});
}
public static <T> LombokImmutableList<T> of(T a, T b, T c, T d) {
return new LombokImmutableList<T>(new Object[] {a, b, c, d});
}
public static <T> LombokImmutableList<T> of(T a, T b, T c, T d, T e) {
return new LombokImmutableList<T>(new Object[] {a, b, c, d, e});
}
@SuppressWarnings({"all", "unchecked"})
public static <T> LombokImmutableList<T> of(T a, T b, T c, T d, T e, T f, T... g) {
Object[] rest = g == null ? new Object[] {null} : g;
Object[] val = new Object[rest.length + 6];
System.arraycopy(rest, 0, val, 6, rest.length);
val[0] = a;
val[1] = b;
val[2] = c;
val[3] = d;
val[4] = e;
```

```
val[5] = f;
return new LombokImmutableList<T>(val);
}
public static <T> LombokImmutableList<T> copyOf(Collection<? extends T> list) {
return new LombokImmutableList<T>(list.toArray());
}
public static <T> LombokImmutableList<T> copyOf(Iterable<? extends T> iterable) {
List<T> list = new ArrayList<T>();
for (T o : iterable) list.add(o);
return copyOf(list);
}
public static <T> LombokImmutableList<T> copyOf(T[] array) {
Object[] content = new Object[array.length];
System.arraycopy(array, 0, content, 0, array.length);
return new LombokImmutableList<T>(content);
}
private LombokImmutableList(Object[] content) {
this.content = content;
}
public LombokImmutableList<T> replaceElementAt(int idx, T newValue) {
Object[] newContent = content.clone();
newContent[idx] = newValue;
return new LombokImmutableList<T>(newContent);
}
public LombokImmutableList<T> append(T newValue) {
int len = content.length;
Object[] newContent = new Object[len + 1];
System.arraycopy(content, 0, newContent, 0, len);
newContent[len] = newValue;
return new LombokImmutableList<T>(newContent);
}
public LombokImmutableList<T> prepend(T newValue) {
int len = content.length;
Object[] newContent = new Object[len + 1];
System.arraycopy(content, 0, newContent, 1, len);
newContent[0] = newValue;
return new LombokImmutableList<T>(newContent);
}
public int indexOf(T val) {
int len = content.length;
if (val == null) {
for (int i = 0; i < len; i++) if (content[i] == null) return i;
return -1;
}
for (int i = 0; i < len; i++) if (val.equals(content[i])) return i;
return -1;
}
public LombokImmutableList<T> removeElement(T val) {
int idx = indexOf(val);
return idx == -1 ? this : removeElementAt(idx);
}
public LombokImmutableList<T> removeElementAt(int idx) {
int len = content.length;
```

```
Object[] newContent = new Object[len - 1];
if (idx > 0) System.arraycopy(content, 0, newContent, 0, idx);
if (idx < len - 1) System.arraycopy(content, idx + 1, newContent, idx, len - idx - 1);
return new LombokImmutableList<T>(newContent);
}
public boolean isEmpty() {
return content.length == 0;
}
public int size() {
return content.length;
}
@SuppressWarnings("unchecked")
public T get(int idx) {
return (T) content[idx];
}
public boolean contains(T in) {
if (in == null) {
for (Object e : content) if (e == null) return true;
return false;
}
for (Object e : content) if (in.equals(e)) return true;
return false;
}
public Iterator<T> iterator() {
return new Iterator<T>() {
private int idx = 0;
@Override public boolean hasNext() {
return idx < content.length;
}
@SuppressWarnings("unchecked")
@Override public T next() {
if (idx < content.length) return (T) content[idx++];
throw new NoSuchElementException();
}
@Override public void remove() {
throw new UnsupportedOperationException("List is immutable");
}
};
}
@Override public String toString() {
return Arrays.toString(content);
}
@Override public boolean equals(Object obj) {
if (!(obj instanceof LombokImmutableList)) return false;
if (obj == this) return true;
return Arrays.equals(content, ((LombokImmutableList<?>) obj).content);
}
@Override public int hashCode() {
return Arrays.hashCode(content);
}
}
```

lombok/core/SpiLoadUtil.java

```
/*
* Copyright (C) 2009-2011 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.lang.annotation.Annotation;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.ParameterizedType;
import java.lang.reflect.Type;
import java.net.URL;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Enumeration;
import java.util.Iterator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
/**
* The java core libraries have a SPI discovery system, but it works only in Java 1.6 and up. For at least
Eclipse,
* lombok actually works in java 1.5, so we've rolled our own SPI discovery system.
*
* It is not API compatible with {@code ServiceLoader}.
*
* @see java.util.ServiceLoader
*/
public class SpiLoadUtil {
private SpiLoadUtil() {
```

```
//Prevent instantiation
}
/**
* Method that conveniently turn the {@code Iterable}s returned by the other methods in this class to a
* {@code List}.
*
* @see #findServices(Class)
* @see #findServices(Class, ClassLoader)
*/
public static <T> List<T> readAllFromIterator(Iterable<T> findServices) {
List<T> list = new ArrayList<T>();
for (T t : findServices) list.add(t);
return list;
}
/**
* Returns an iterator of instances that, at least according to the spi discovery file, are implementations
* of the stated class.
*
* Like ServiceLoader, each listed class is turned into an instance by calling the public no-args
constructor.
*
* Convenience method that calls the more elaborate {@link #findServices(Class, ClassLoader)} method
with
* this {@link java.lang.Thread}'s context class loader as {@code ClassLoader}.
*
* @param target class to find implementations for.
*/
public static <C> Iterable<C> findServices(Class<C> target) throws IOException {
return findServices(target, Thread.currentThread().getContextClassLoader());
}
/**
* Returns an iterator of class objects that, at least according to the spi discovery file, are
implementations
* of the stated class.
*
* Like ServiceLoader, each listed class is turned into an instance by calling the public no-args
constructor.
*
* @param target class to find implementations for.
* @param loader The classloader object to use to both the spi discovery files, as well as the loader to
use
* to make the returned instances.
*/
public static <C> Iterable<C> findServices(final Class<C> target, ClassLoader loader) throws
IOException {
if (loader == null) loader = ClassLoader.getSystemClassLoader();
Enumeration<URL> resources = loader.getResources("META-INF/services/" + target.getName());
final Set<String> entries = new LinkedHashSet<String>();
while (resources.hasMoreElements()) {
URL url = resources.nextElement();
readServicesFromUrl(entries, url);
}
final Iterator<String> names = entries.iterator();
```

```
final ClassLoader fLoader = loader;
return new Iterable<C> () {
@Override public Iterator<C> iterator() {
return new Iterator<C>() {
@Override public boolean hasNext() {
return names.hasNext();
}
@Override public C next() {
try {
return target.cast(Class.forName(names.next(), true, fLoader).getConstructor().newInstance());
} catch (Exception e) {
Throwable t = e;
if (t instanceof InvocationTargetException) t = t.getCause();
if (t instanceof RuntimeException) throw (RuntimeException) t;
if (t instanceof Error) throw (Error) t;
throw new RuntimeException(t);
}
}
@Override public void remove() {
throw new UnsupportedOperationException();
}
};
}
};
}
private static void readServicesFromUrl(Collection<String> list, URL url) throws IOException {
InputStream in = url.openStream();
BufferedReader r = null;
try {
if (in == null) return;
r = new BufferedReader(new InputStreamReader(in, "UTF-8"));
while (true) {
String line = r.readLine();
if (line == null) break;
int idx = line.indexOf('#');
if (idx != -1) line = line.substring(0, idx);
line = line.trim();
if (line.length() == 0) continue;
list.add(line);
}
} finally {
try {
if (r != null) r.close();
if (in != null) in.close();
} catch (Throwable ignore) {}
}
}
/**
* This method will find the @{code T} in {@code public class Foo extends BaseType<T>}.
*
* It returns an annotation type because it is used exclusively to figure out which annotations are
* being handled by {@link lombok.eclipse.EclipseAnnotationHandler} and {@link
lombok.javac.JavacAnnotationHandler}.
```

```
*/
public static Class<? extends Annotation> findAnnotationClass(Class<?> c, Class<?> base) {
if (c == Object.class || c == null) return null;
Class<? extends Annotation> answer = null;
answer = findAnnotationHelper(base, c.getGenericSuperclass());
if (answer != null) return answer;
for (Type iface : c.getGenericInterfaces()) {
answer = findAnnotationHelper(base, iface);
if (answer != null) return answer;
}
Class<? extends Annotation> potential = findAnnotationClass(c.getSuperclass(), base);
if (potential != null) return potential;
for (Class<?> iface : c.getInterfaces()) {
potential = findAnnotationClass(iface, base);
if (potential != null) return potential;
}
return null;
}
@SuppressWarnings("unchecked")
private static Class<? extends Annotation> findAnnotationHelper(Class<?> base, Type iface) {
if (iface instanceof ParameterizedType) {
ParameterizedType p = (ParameterizedType)iface;
if (!base.equals(p.getRawType())) return null;
Type target = p.getActualTypeArguments()[0];
if (target instanceof Class<?>) {
if (Annotation.class.isAssignableFrom((Class<?>) target)) {
return (Class<? extends Annotation>) target;
}
}
throw new ClassCastException("Not an annotation type: " + target);
}
return null;
}
}
```

lombok/core/debug/FileLog.java

```
/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core.debug;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.PrintWriter;
import java.io.StringWriter;
public class FileLog {
private static FileOutputStream fos;
public static void log(String message) {
log(message, null);
}
public synchronized static void log(String message, Throwable t) {
try {
if (fos == null) {
fos = new FileOutputStream(new File(System.getProperty("user.home"),
"LOMBOK-DEBUG-OUT.txt"));
Runtime.getRuntime().addShutdownHook(new Thread() {
public void run() {
try {
fos.close();
} catch (Throwable ignore) {}
}
});
}
fos.write(message.getBytes("UTF-8"));
fos.write('\n');
if (t != null) {
StringWriter sw = new StringWriter();
t.printStackTrace(new PrintWriter(sw));
fos.write(sw.toString().getBytes("UTF-8"));
fos.write('\n');
}
fos.flush();
} catch (IOException e) {
throw new IllegalStateException("Internal lombok file-based debugging not possible", e);
}
}
}
```

lombok/eclipse/Eclipse.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
```

* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.regex.Pattern;
import lombok.core.ClassLiteral;
import lombok.core.FieldSelect;
import lombok.core.JavaIdentifiers;
import lombok.permit.Permit;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.CaseStatement;
import org.eclipse.jdt.internal.compiler.ast.ClassLiteralAccess;
import org.eclipse.jdt.internal.compiler.ast.Clinit;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.Literal;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.TryStatement;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.ast.UnaryExpression;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.impl.CompilerOptions;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
public class Eclipse {
private static final Annotation[] EMPTY_ANNOTATIONS_ARRAY = new Annotation[0];
/**
* Eclipse's Parser class is instrumented to not attempt to fill in the body of any method or initializer
* or field initialization if this flag is set. Set it on the flag field of
* any method, field, or initializer you create!

```
*/
public static final int ECLIPSE_DO_NOT_TOUCH_FLAG = ASTNode.Bit24;
/* This section includes flags that are in ecj files too new vs. the deps we compile against.
* Specifically: org.eclipse.jdt.internal.compiler.ast.ASTNode and
org.eclipse.jdt.internal.compiler.lookup.ExtraCompilerModifiers
*/
public static final int AccRecord = ASTNode.Bit25; // ECM.AccRecord
public static final int IsCanonicalConstructor = ASTNode.Bit10; // ASTNode.IsCanonicalConstructor
public static final int IsImplicit = ASTNode.Bit11; // ASTNode.IsImplicit
public static final int HasTypeAnnotations = ASTNode.Bit21; // ASTNode.HasTypeAnnotations
private static final Pattern SPLIT_AT_DOT = Pattern.compile("\\.");
private Eclipse() {
//Prevent instantiation
}
/**
* For 'speed' reasons, Eclipse works a lot with char arrays. I have my doubts this was a fruitful exercise,
* but we need to deal with it. This turns [[java][lang][String]] into "java.lang.String".
*/
public static String toQualifiedName(char[][] typeName) {
int len = typeName.length - 1; // number of dots
if (len == 0) return new String(typeName[0]);
for (char[] c : typeName) len += c.length;
char[] ret = new char[len];
char[] part = typeName[0];
System.arraycopy(part, 0, ret, 0, part.length);
int pos = part.length;
for (int i = 1; i < typeName.length; i++) {
ret[pos++] = '.';
part = typeName[i];
System.arraycopy(part, 0, ret, pos, part.length);
pos += part.length;
}
return new String(ret);
}
public static char[][] fromQualifiedName(String typeName) {
String[] split = SPLIT_AT_DOT.split(typeName);
char[][] result = new char[split.length][];
for (int i = 0; i < split.length; i++) {
result[i] = split[i].toCharArray();
}
return result;
}
public static long pos(ASTNode node) {
return ((long) node.sourceStart << 32) | (node.sourceEnd & 0xFFFFFFFFL);
}
public static long[] poss(ASTNode node, int repeat) {
long p = ((long) node.sourceStart << 32) | (node.sourceEnd & 0xFFFFFFFFL);
long[] out = new long[repeat];
Arrays.fill(out, p);
return out;
}
/**
```

```
* Checks if an eclipse-style array-of-array-of-characters to represent a fully qualified name
('foo.bar.baz'), matches a plain
* string containing the same fully qualified name with dots in the string.
*/
public static boolean nameEquals(char[][] typeName, String string) {
int pos = 0, len = string.length();
for (int i = 0; i < typeName.length; i++) {
char[] t = typeName[i];
if (i > 0) {
if (pos == len) return false;
if (string.charAt(pos++) != '.') return false;
}
for (int j = 0; j < t.length; j++) {
if (pos == len) return false;
if (string.charAt(pos++) != t[j]) return false;
}
}
return true;
}
public static boolean hasClinit(TypeDeclaration parent) {
if (parent.methods == null) return false;
for (AbstractMethodDeclaration method : parent.methods) {
if (method instanceof Clinit) return true;
}
return false;
}
/**
* Searches the given field node for annotations and returns each one that matches the provided regular
expression pattern.
*
* Only the simple name is checked - the package and any containing class are ignored.
*/
public static Annotation[] findAnnotations(AbstractVariableDeclaration field, Pattern namePattern) {
List<Annotation> result = new ArrayList<Annotation>();
if (field.annotations == null) return EMPTY_ANNOTATIONS_ARRAY;
for (Annotation annotation : field.annotations) {
TypeReference typeRef = annotation.type;
if (typeRef != null && typeRef.getTypeName() != null) {
char[][] typeName = typeRef.getTypeName();
String suspect = new String(typeName[typeName.length - 1]);
if (namePattern.matcher(suspect).matches()) {
result.add(annotation);
}
}
}
return result.toArray(EMPTY_ANNOTATIONS_ARRAY);
}
/**
* Checks if the given type reference represents a primitive type.
*/
public static boolean isPrimitive(TypeReference ref) {
if (ref.dimensions() > 0) return false;
return JavaIdentifiers.isPrimitive(toQualifiedName(ref.getTypeName()));
```

```
}
/**
* Returns the actual value of the given Literal or Literal-like node.
*/
public static Object calculateValue(Expression e) {
if (e instanceof Literal) {
((Literal) e).computeConstant();
switch (e.constant.typeID()) {
case TypeIds.T_int: return e.constant.intValue();
case TypeIds.T_byte: return e.constant.byteValue();
case TypeIds.T_short: return e.constant.shortValue();
case TypeIds.T_char: return e.constant.charValue();
case TypeIds.T_float: return e.constant.floatValue();
case TypeIds.T_double: return e.constant.doubleValue();
case TypeIds.T_boolean: return e.constant.booleanValue();
case TypeIds.T_long: return e.constant.longValue();
case TypeIds.T_JavaLangString: return e.constant.stringValue();
default: return null;
}
} else if (e instanceof ClassLiteralAccess) {
return new ClassLiteral(Eclipse.toQualifiedName(((ClassLiteralAccess) e).type.getTypeName()));
} else if (e instanceof SingleNameReference) {
return new FieldSelect(new String(((SingleNameReference)e).token));
} else if (e instanceof QualifiedNameReference) {
String qName = Eclipse.toQualifiedName(((QualifiedNameReference) e).tokens);
int idx = qName.lastIndexOf('.');
return new FieldSelect(idx == -1 ? qName : qName.substring(idx+1));
} else if (e instanceof UnaryExpression) {
if ("-".equals(((UnaryExpression) e).operatorToString())) {
Object inner = calculateValue(((UnaryExpression) e).expression);
if (inner instanceof Integer) return - ((Integer) inner).intValue();
if (inner instanceof Byte) return - ((Byte) inner).byteValue();
if (inner instanceof Short) return - ((Short) inner).shortValue();
if (inner instanceof Long) return - ((Long) inner).longValue();
if (inner instanceof Float) return - ((Float) inner).floatValue();
if (inner instanceof Double) return - ((Double) inner).doubleValue();
return null;
}
}
return null;
}
private static long latestEcjCompilerVersionConstantCached = 0;
public static long getLatestEcjCompilerVersionConstant() {
if (latestEcjCompilerVersionConstantCached != 0) return latestEcjCompilerVersionConstantCached;
int highestVersionSoFar = 0;
for (Field f : ClassFileConstants.class.getDeclaredFields()) {
try {
if (f.getName().startsWith("JDK")) {
String versionString = f.getName().substring("JDK".length());
if (versionString.startsWith("1_")) versionString = versionString.substring("1_".length());
int thisVersion = Integer.parseInt(versionString);
if (thisVersion > highestVersionSoFar) {
highestVersionSoFar = thisVersion;
```

```
latestEcjCompilerVersionConstantCached = (Long) f.get(null);
}
}
} catch (Exception ignore) {}
}
if (highestVersionSoFar > 6 && !ecjSupportsJava7Features()) {
latestEcjCompilerVersionConstantCached = ClassFileConstants.JDK1_6;
}
return latestEcjCompilerVersionConstantCached;
}
private static int ecjCompilerVersionCached = -1;
public static int getEcjCompilerVersion() {
if (ecjCompilerVersionCached >= 0) return ecjCompilerVersionCached;
for (Field f : CompilerOptions.class.getDeclaredFields()) {
try {
String fName = f.getName();
String versionNumber = null;
if (fName.startsWith("VERSION_1_")) {
versionNumber = fName.substring("VERSION_1_".length());
} else if (fName.startsWith("VERSION_")) {
versionNumber = fName.substring("VERSION_".length());
} else continue;
ecjCompilerVersionCached = Math.max(ecjCompilerVersionCached,
Integer.parseInt(versionNumber));
} catch (Exception ignore) {}
}
if (ecjCompilerVersionCached < 5) ecjCompilerVersionCached = 5;
if (!ecjSupportsJava7Features()) ecjCompilerVersionCached = Math.min(6,
ecjCompilerVersionCached);
return ecjCompilerVersionCached;
}
/**
* Certain ECJ versions that only go up to -source 6 report that they support -source 7 and even fail to
error when -source 7 is applied.
* We detect this and correctly say that no more than -source 6 is supported. (when this is the case, this
method returns false).
*/
private static boolean ecjSupportsJava7Features() {
try {
TryStatement.class.getDeclaredField("resources");
return true;
} catch (NoSuchFieldException e) {
return false;
}
}
private static final Field CASE_STATEMENT_CONSTANT_EXPRESSIONS =
Permit.permissiveGetField(CaseStatement.class, "constantExpressions");
private static final Constructor<CaseStatement> CASE_STATEMENT_CONSTRUCTOR_SINGLE;
private static final Constructor<CaseStatement> CASE_STATEMENT_CONSTRUCTOR_ARRAY;
private static final Expression[] EMPTY_EXPRESSIONS;
static {
Constructor<CaseStatement> constructorSingle = null, constructorArray = null;
Expression[] emptyExpressions = new Expression[0];
```

```
try {
constructorSingle = Permit.getConstructor(CaseStatement.class, Expression.class, int.class, int.class);
} catch (NoSuchMethodException ignore) {
}
try {
constructorArray = Permit.getConstructor(CaseStatement.class, Expression[].class, int.class, int.class);
} catch (NoSuchMethodException ignore) {
}
try {
emptyExpressions = Permit.get(Permit.permissiveGetField(Expression.class, "NO_EXPRESSIONS"),
null);
} catch (Throwable ignore) {
}
CASE_STATEMENT_CONSTRUCTOR_SINGLE = constructorSingle;
CASE_STATEMENT_CONSTRUCTOR_ARRAY = constructorArray;
EMPTY_EXPRESSIONS = emptyExpressions;
}
public static CaseStatement createCaseStatement(Expression expr) {
final CaseStatement stat;
if (CASE_STATEMENT_CONSTRUCTOR_SINGLE != null) {
stat = Permit.newInstanceSneaky(CASE_STATEMENT_CONSTRUCTOR_SINGLE, expr, 0, 0);
if (stat != null && expr != null && CASE_STATEMENT_CONSTANT_EXPRESSIONS != null) {
try {
Permit.set(CASE_STATEMENT_CONSTANT_EXPRESSIONS, stat, new Expression[] {expr});
} catch (IllegalAccessException ignore) {
}
}
} else {
Expression[] expressions = EMPTY_EXPRESSIONS;
if (expr != null) {
expressions = new Expression[] {expr};
}
stat = Permit.newInstanceSneaky(CASE_STATEMENT_CONSTRUCTOR_ARRAY, expressions, 0, 0);
}
return stat;
}
}
```

lombok/eclipse/Java14Bits.java

```
package lombok.eclipse;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
public class Java14Bits {
private Java14Bits() { }
public static final int AccRecord = ASTNode.Bit25;
public static final int IsCanonicalConstructor = ASTNode.Bit10; // record declaration
public static final int IsImplicit = ASTNode.Bit11; // record declaration / generated statements in
compact constructor
}
```

lombok/javac/CommentCatcher.java

```
/*
* Copyright (C) 2011-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.lang.reflect.InvocationTargetException;
import java.util.Collections;
import java.util.List;
import lombok.core.FieldAugment;
import lombok.permit.Permit;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.util.Context;
public class CommentCatcher {
private final JavaCompiler compiler;
public static final FieldAugment<JCCompilationUnit, List<CommentInfo>>
JCCompilationUnit_comments = FieldAugment.augment(JCCompilationUnit.class, List.class,
"lombok$comments");
public static final FieldAugment<JCCompilationUnit, List<Integer>> JCCompilationUnit_textBlockStarts
= FieldAugment.augment(JCCompilationUnit.class, List.class, "lombok$textBlockStarts");
public static CommentCatcher create(Context context, boolean findTextBlocks) {
registerCommentsCollectingScannerFactory(context, findTextBlocks);
JavaCompiler compiler = new JavaCompiler(context);
setInCompiler(compiler, context);
compiler.keepComments = true;
compiler.genEndPos = true;
return new CommentCatcher(compiler);
}
private CommentCatcher(JavaCompiler compiler) {
this.compiler = compiler;
}
public JavaCompiler getCompiler() {
```

```
return compiler;
}
public void setComments(JCCompilationUnit ast, List<CommentInfo> comments) {
if (comments != null) {
JCCompilationUnit_comments.set(ast, comments);
} else {
JCCompilationUnit_comments.clear(ast);
}
}
public List<CommentInfo> getComments(JCCompilationUnit ast) {
List<CommentInfo> list = JCCompilationUnit_comments.get(ast);
return list == null ? Collections.<CommentInfo>emptyList() : list;
}
public List<Integer> getTextBlockStarts(JCCompilationUnit ast) {
List<Integer> list = JCCompilationUnit_textBlockStarts.get(ast);
return list == null ? Collections.<Integer>emptyList() : list;
}
private static void registerCommentsCollectingScannerFactory(Context context, boolean
findTextBlocks) {
try {
Class<?> scannerFactory;
int javaCompilerVersion = Javac.getJavaCompilerVersion();
if (javaCompilerVersion <= 6) {
scannerFactory = Class.forName("lombok.javac.java6.CommentCollectingScannerFactory");
} else if (javaCompilerVersion == 7) {
scannerFactory = Class.forName("lombok.javac.java7.CommentCollectingScannerFactory");
} else {
scannerFactory = Class.forName("lombok.javac.java8.CommentCollectingScannerFactory");
if (findTextBlocks) Permit.getField(scannerFactory, "findTextBlocks").set(null, true);
}
Permit.getMethod(scannerFactory, "preRegister", Context.class).invoke(null, context);
} catch (InvocationTargetException e) {
throw Javac.sneakyThrow(e.getCause());
} catch (Exception e) {
throw Javac.sneakyThrow(e);
}
}
private static void setInCompiler(JavaCompiler compiler, Context context) {
try {
Class<?> parserFactory;
int javaCompilerVersion = Javac.getJavaCompilerVersion();
if (javaCompilerVersion <= 6) {
parserFactory = Class.forName("lombok.javac.java6.CommentCollectingParserFactory");
} else if (javaCompilerVersion == 7) {
parserFactory = Class.forName("lombok.javac.java7.CommentCollectingParserFactory");
} else if (javaCompilerVersion == 8) {
parserFactory = Class.forName("lombok.javac.java8.CommentCollectingParserFactory");
} else {
parserFactory = Class.forName("lombok.javac.java9.CommentCollectingParserFactory");
}
Permit.getMethod(parserFactory, "setInCompiler", JavaCompiler.class, Context.class).invoke(null,
compiler, context);
} catch (InvocationTargetException e) {
```

```
throw Javac.sneakyThrow(e.getCause());
} catch (Exception e) {
throw Javac.sneakyThrow(e);
}
}
}
```

lombok/javac/CommentInfo.java

```
/*
* Copyright (C) 2009-2011 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
public final class CommentInfo {
public enum StartConnection {
/* Comment's start immediately follows a newline. */
START_OF_LINE,
/* Comment's start does not immediately follow a newline, but there is a newline between this
* and the previous comment (or pos 0 if first comment). */
ON_NEXT_LINE,
/* Comment's start immediately follows previous comment's end (or pos 0 if first comment). */
DIRECT_AFTER_PREVIOUS,
/* Comment's start does not immediately follow previous comment's end, but there is no newline in
* between this and previous comment (or pos 0 if first comment). */
AFTER_PREVIOUS
}
public enum EndConnection {
/* Comment is followed immediately by another node (not whitespace, not newline). */
DIRECT_AFTER_COMMENT,
/* Comment is followed by some non-newline whitespace then by another node. */
AFTER_COMMENT,
/* Comment is followed by optionally some whitespace then a newline. */
ON_NEXT_LINE
```

```
}
public final int pos;
public final int prevEndPos;
public final String content;
public final int endPos;
public final StartConnection start;
public final EndConnection end;
public CommentInfo(int prevEndPos, int pos, int endPos, String content, StartConnection start,
EndConnection end) {
this.pos = pos;
this.prevEndPos = prevEndPos;
this.endPos = endPos;
this.content = content;
this.start = start;
this.end = end;
}
public boolean isJavadoc() {
return content.startsWith("/**") && content.length() > 4;
}
@Override
public String toString() {
return String.format("%d: %s (%s,%s)", pos, content, start, end);
}
}
```

lombok/javac/Javac.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import static lombok.javac.JavacTreeMaker.TreeTag.treeTag;
import static lombok.javac.JavacTreeMaker.TypeTag.typeTag;
```

```
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import javax.lang.model.type.NoType;
import javax.lang.model.type.TypeKind;
import javax.lang.model.type.TypeVisitor;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.code.Source;
import com.sun.tools.javac.code.Symtab;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.parser.Tokens.Comment;
import com.sun.tools.javac.tree.DocCommentTable;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCImport;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import lombok.core.ClassLiteral;
import lombok.core.FieldSelect;
import lombok.core.JavaIdentifiers;
import lombok.javac.JavacTreeMaker.TreeTag;
import lombok.javac.JavacTreeMaker.TypeTag;
import lombok.permit.Permit;
/**
* Container for static utility methods relevant to lombok's operation on javac.
*/
public class Javac {
private Javac() {
// prevent instantiation
}
private static final Pattern VERSION_PARSER = Pattern.compile("^(\\d{1,6})\\.?(\\d{1,6})?.*$");
private static final Pattern SOURCE_PARSER = Pattern.compile("^JDK(\\d{1,6})_?(\\d{1,6})?.*$");
private static final AtomicInteger compilerVersion = new AtomicInteger(-1);
/* This section includes flags that would ordinarily be in Flags, but which are 'too new' (we don't compile
against older versions of javac for compatibility). */
public static final long RECORD = 1L << 61; // ClassSymbols, MethodSymbols, VarSymbols (Marks
types as being records, as well as the 'fields' in the compact declaration, and the canonical constructor)
public static final long COMPACT_RECORD_CONSTRUCTOR = 1L << 51; // MethodSymbols (the
'implicit' many-args constructor that records have)
public static final long UNINITIALIZED_FIELD = 1L << 51; // VarSymbols (To identify fields that the
compact record constructor won't initialize)
```

```
public static final long GENERATED_MEMBER = 1L << 24; // MethodSymbols, VarSymbols (marks methods and the constructor generated in records)
public static final long SEALED = 1L << 62; // ClassSymbols (Flag to indicate sealed class/interface declaration)
public static final long NON_SEALED = 1L << 63; // ClassSymbols (Flag to indicate that the class/interface was declared with the non-sealed modifier)
/**
* Returns the version of this java compiler, i.e. the JDK that it shipped in. For example, for javac v1.7, this returns {@code 7}.
*/
public static int getJavaCompilerVersion() {
int cv = compilerVersion.get();
if (cv != -1) return cv;
/* Main algorithm: Use JavaCompiler's intended method to do this */ {
Matcher m = VERSION_PARSER.matcher(JavaCompiler.version());
if (m.matches()) {
int major = Integer.parseInt(m.group(1));
if (major == 1) {
int minor = Integer.parseInt(m.group(2));
return setVersion(minor);
}
if (major >= 9) return setVersion(major);
}
}
/* Fallback algorithm one: Check Source's values. Lets hope oracle never releases a javac that recognizes future versions for -source */ {
String name = Source.values()[Source.values().length - 1].name();
Matcher m = SOURCE_PARSER.matcher(name);
if (m.matches()) {
int major = Integer.parseInt(m.group(1));
if (major == 1) {
int minor = Integer.parseInt(m.group(2));
return setVersion(minor);
}
if (major >= 9) return setVersion(major);
}
}
return setVersion(6);
}
private static int setVersion(int version) {
compilerVersion.set(version);
return version;
}
private static final Class<?> DOCCOMMENTTABLE_CLASS;
static {
Class<?> c = null;
try {
c = Class.forName("com.sun.tools.javac.tree.DocCommentTable");
} catch (Throwable ignore) {}
DOCCOMMENTTABLE_CLASS = c;
}
public static boolean instanceOfDocCommentTable(Object o) {
return DOCCOMMENTTABLE_CLASS != null && DOCCOMMENTTABLE_CLASS.isInstance(o);
```

```
}
/**
* Checks if the given expression (that really ought to refer to a type
* expression) represents a primitive type.
*/
public static boolean isPrimitive(JCExpression ref) {
return JavaIdentifiers.isPrimitive(ref.toString());
}
/**
* Turns an expression into a guessed intended literal. Only works for
* literals, as you can imagine.
*
* Will for example turn a TrueLiteral into 'Boolean.valueOf(true)'.
*/
public static Object calculateGuess(JCExpression expr) {
if (expr instanceof JCLiteral) {
JCLiteral lit = (JCLiteral) expr;
if (lit.getKind() == com.sun.source.tree.Tree.Kind.BOOLEAN_LITERAL) {
return ((Number) lit.value).intValue() == 0 ? false : true;
}
return lit.value;
}
if (expr instanceof JCIdent || expr instanceof JCFieldAccess) {
String x = expr.toString();
if (x.endsWith(".class")) return new ClassLiteral(x.substring(0, x.length() - 6));
int idx = x.lastIndexOf('.');
if (idx > -1) x = x.substring(idx + 1);
return new FieldSelect(x);
}
return null;
}
public static final TypeTag CTC_BOOLEAN = typeTag("BOOLEAN");
public static final TypeTag CTC_INT = typeTag("INT");
public static final TypeTag CTC_DOUBLE = typeTag("DOUBLE");
public static final TypeTag CTC_FLOAT = typeTag("FLOAT");
public static final TypeTag CTC_SHORT = typeTag("SHORT");
public static final TypeTag CTC_BYTE = typeTag("BYTE");
public static final TypeTag CTC_LONG = typeTag("LONG");
public static final TypeTag CTC_CHAR = typeTag("CHAR");
public static final TypeTag CTC_VOID = typeTag("VOID");
public static final TypeTag CTC_NONE = typeTag("NONE");
public static final TypeTag CTC_BOT = typeTag("BOT");
public static final TypeTag CTC_ERROR = typeTag("ERROR");
public static final TypeTag CTC_UNKNOWN = typeTag("UNKNOWN");
public static final TypeTag CTC_UNDETVAR = typeTag("UNDETVAR");
public static final TypeTag CTC_CLASS = typeTag("CLASS");
public static final TreeTag CTC_NOT_EQUAL = treeTag("NE");
public static final TreeTag CTC_LESS_THAN = treeTag("LT");
public static final TreeTag CTC_GREATER_THAN = treeTag("GT");
public static final TreeTag CTC_LESS_OR_EQUAL= treeTag("LE");
public static final TreeTag CTC_GREATER_OR_EQUAL = treeTag("GE");
public static final TreeTag CTC_POS = treeTag("POS");
public static final TreeTag CTC_NEG = treeTag("NEG");
```

```
public static final TreeTag CTC_NOT = treeTag("NOT");
public static final TreeTag CTC_COMPL = treeTag("COMPL");
public static final TreeTag CTC_BITXOR = treeTag("BITXOR");
public static final TreeTag CTC_UNSIGNED_SHIFT_RIGHT = treeTag("USR");
public static final TreeTag CTC_MUL = treeTag("MUL");
public static final TreeTag CTC_DIV = treeTag("DIV");
public static final TreeTag CTC_PLUS = treeTag("PLUS");
public static final TreeTag CTC_MINUS = treeTag("MINUS");
public static final TreeTag CTC_EQUAL = treeTag("EQ");
public static final TreeTag CTC_PREINC = treeTag("PREINC");
public static final TreeTag CTC_PREDEC = treeTag("PREDEC");
public static final TreeTag CTC_POSTINC = treeTag("POSTINC");
public static final TreeTag CTC_POSTDEC = treeTag("POSTDEC");
private static final Method getExtendsClause, getEndPosition, storeEnd;
static {
getExtendsClause = getMethod(JCClassDecl.class, "getExtendsClause", new Class<?>[0]);
if (getJavaCompilerVersion() < 8) {
getEndPosition = getMethod(DiagnosticPosition.class, "getEndPosition", java.util.Map.class);
storeEnd = getMethod(java.util.Map.class, "put", Object.class, Object.class);
} else {
getEndPosition = getMethod(DiagnosticPosition.class, "getEndPosition",
"com.sun.tools.javac.tree.EndPosTable");
Method storeEndMethodTemp;
Class<?> endPosTable;
try {
endPosTable = Class.forName("com.sun.tools.javac.tree.EndPosTable");
} catch (ClassNotFoundException ex) {
throw sneakyThrow(ex);
}
try {
storeEndMethodTemp = Permit.getMethod(endPosTable, "storeEnd", JCTree.class, int.class);
} catch (NoSuchMethodException e) {
try {
endPosTable = Class.forName("com.sun.tools.javac.parser.JavacParser$AbstractEndPosTable");
storeEndMethodTemp = Permit.getMethod(endPosTable, "storeEnd", JCTree.class, int.class);
} catch (NoSuchMethodException ex) {
throw sneakyThrow(ex);
} catch (ClassNotFoundException ex) {
throw sneakyThrow(ex);
}
}
storeEnd = storeEndMethodTemp;
}
Permit.setAccessible(getEndPosition);
Permit.setAccessible(storeEnd);
}
private static Method getMethod(Class<?> clazz, String name, Class<?>... paramTypes) {
try {
return Permit.getMethod(clazz, name, paramTypes);
} catch (NoSuchMethodException e) {
throw sneakyThrow(e);
}
}
```

```
private static Method getMethod(Class<?> clazz, String name, String... paramTypes) {
try {
Class<?>[] c = new Class[paramTypes.length];
for (int i = 0; i < paramTypes.length; i++) c[i] = Class.forName(paramTypes[i]);
return Permit.getMethod(clazz, name, c);
} catch (NoSuchMethodException e) {
throw sneakyThrow(e);
} catch (ClassNotFoundException e) {
throw sneakyThrow(e);
}
}
/**
* In some versions, the field's type is {@code JCTree}, in others it is {@code JCExpression}, which at the JVM level are not the same.
*/
public static JCTree getExtendsClause(JCClassDecl decl) {
try {
return (JCTree) getExtendsClause.invoke(decl);
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
} catch (InvocationTargetException e) {
throw sneakyThrow(e.getCause());
}
}
/**
* In some versions, the field's type is {@code JCTree}, in others it is {@code JCFieldAccess}, which at the JVM level are not the same.
*/
private static final Field JCIMPORT_QUALID = Permit.permissiveGetField(JCImport.class, "qualid");
public static JCTree getQualid(JCImport tree) {
try {
return (JCTree) JCIMPORT_QUALID.get(tree);
} catch (IllegalAccessException e) {
throw sneakyThrow(e.getCause());
}
}
public static Object getDocComments(JCCompilationUnit cu) {
try {
return JCCOMPILATIONUNIT_DOCCOMMENTS.get(cu);
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
}
}
public static String getDocComment(JCCompilationUnit cu, JCTree node) {
Object dc = getDocComments(cu);
if (dc instanceof Map) return (String) ((Map<?, ?>) dc).get(node);
if (instanceOfDocCommentTable(dc)) return JavadocOps_8.getJavadoc(dc, node);
return null;
}
/**
* Checks if the javadoc comment associated with {@code tree} has a position set.
*
* Returns true if there is no javadoc comment on the node, or it has position (position isn't -1).
```

```
*/
public static boolean validateDocComment(JCCompilationUnit cu, JCTree tree) {
Object dc = getDocComments(cu);
if (!instanceOfDocCommentTable(dc)) return true;
return JavadocOps_8.validateJavadoc(dc, tree);
}
@SuppressWarnings("unchecked")
public static void setDocComment(JCCompilationUnit cu, JCTree node, String javadoc) {
if (javadoc == null) return;
Object dc = getDocComments(cu);
if (dc instanceof Map) {
((Map<JCTree, String>) dc).put(node, javadoc);
return;
}
if (instanceOfDocCommentTable(dc)) {
JavadocOps_8.setJavadoc(dc, node, javadoc);
return;
}
}
private static class JavadocOps_8 {
static String getJavadoc(Object dc, JCTree node) {
DocCommentTable dct = (DocCommentTable) dc;
Comment javadoc = dct.getComment(node);
if (javadoc == null) return null;
return javadoc.getText();
}
public static boolean validateJavadoc(Object dc, JCTree node) {
DocCommentTable dct = (DocCommentTable) dc;
Comment javadoc = dct.getComment(node);
return javadoc == null || javadoc.getText() == null || javadoc.getSourcePos(0) >= 0;
}
static void setJavadoc(Object dc, JCTree node, String javadoc) {
DocCommentTable dct = (DocCommentTable) dc;
Comment newCmt = createJavadocComment(javadoc, node);
dct.putComment(node, newCmt);
}
private static Comment createJavadocComment(final String text, final JCTree field) {
return new Comment() {
@Override public String getText() {
return text;
}
@Override public int getSourcePos(int index) {
return field == null ? -1 : field.getStartPosition();
}
@Override public CommentStyle getStyle() {
return CommentStyle.JAVADOC;
}
@Override public boolean isDeprecated() {
return text.contains("@deprecated") && field instanceof JCVariableDecl && isFieldDeprecated(field);
}
};
}
}
```

```
public static boolean isFieldDeprecated(JCTree field) {
if (!(field instanceof JCVariableDecl)) return false;
JCVariableDecl fieldNode = (JCVariableDecl) field;
if ((fieldNode.mods.flags & Flags.DEPRECATED) != 0) return true;
if (fieldNode.mods.annotations != null) for (JCAnnotation ann : fieldNode.mods.annotations) {
String at = ann.getAnnotationType().toString();
return at.equals("Deprecated") || at.endsWith(".Deprecated");
}
return false;
}
public static void initDocComments(JCCompilationUnit cu) {
try {
JCCOMPILATIONUNIT_DOCCOMMENTS.set(cu, new HashMap<Object, String>());
} catch (IllegalArgumentException e) {
// That's fine - we're on JDK8, we'll fix that later.
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
}
}
public static int getEndPosition(DiagnosticPosition pos, JCCompilationUnit top) {
try {
Object endPositions = JCCOMPILATIONUNIT_ENDPOSITIONS.get(top);
return (Integer) getEndPosition.invoke(pos, endPositions);
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
} catch (InvocationTargetException e) {
throw sneakyThrow(e.getCause());
}
}
public static void storeEnd(JCTree tree, int pos, JCCompilationUnit top) {
try {
Object endPositions = JCCOMPILATIONUNIT_ENDPOSITIONS.get(top);
if (endPositions == null) return;
storeEnd.invoke(endPositions, tree, pos);
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
} catch (InvocationTargetException e) {
throw sneakyThrow(e.getCause());
}
}
private static final Class<?> JC_VOID_TYPE, JC_NO_TYPE;
static {
Class<?> c = null;
try {
c = Class.forName("com.sun.tools.javac.code.Type$JCVoidType");
} catch (Throwable ignore) {}
JC_VOID_TYPE = c;
c = null;
try {
c = Class.forName("com.sun.tools.javac.code.Type$JCNoType");
} catch (Throwable ignore) {}
JC_NO_TYPE = c;
}
```

```
private static final Field symtabVoidType = getFieldIfExists(Symtab.class, "voidType");
private static Field getFieldIfExists(Class<?> c, String fieldName) {
try {
return Permit.getField(c, "voidType");
} catch (Exception e) {
return null;
}
}
public static Type createVoidType(Symtab symbolTable, TypeTag tag) {
if (symtabVoidType != null) try {
return (Type) symtabVoidType.get(symbolTable);
} catch (IllegalAccessException ignore) {}
if (Javac.getJavaCompilerVersion() < 8) {
return new JCNoType(((Integer) tag.value).intValue());
} else {
try {
if (CTC_VOID.equals(tag)) {
return (Type) JC_VOID_TYPE.getConstructor().newInstance();
} else {
return (Type) JC_NO_TYPE.getConstructor().newInstance();
}
} catch (InvocationTargetException e) {
throw sneakyThrow(e.getCause());
} catch (NoSuchMethodException e) {
throw sneakyThrow(e);
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
} catch (InstantiationException e) {
throw sneakyThrow(e);
}
}
}
private static class JCNoType extends Type implements NoType {
public JCNoType(int tag) {
super(tag, null);
}
@Override
public TypeKind getKind() {
if (tag == ((Integer) CTC_VOID.value).intValue()) return TypeKind.VOID;
if (tag == ((Integer) CTC_NONE.value).intValue()) return TypeKind.NONE;
throw new AssertionError("Unexpected tag: " + tag);
}
@Override
public <R, P> R accept(TypeVisitor<R, P> v, P p) {
return v.visitNoType(this, p);
}
}
private static final Field JCCOMPILATIONUNIT_ENDPOSITIONS,
JCCOMPILATIONUNIT_DOCCOMMENTS;
static {
Field f = null;
try {
f = Permit.getField(JCCompilationUnit.class, "endPositions");
```

```
} catch (NoSuchFieldException e) {}
JCCOMPILATIONUNIT_ENDPOSITIONS = f;
f = null;
try {
f = Permit.getField(JCCompilationUnit.class, "docComments");
} catch (NoSuchFieldException e) {}
JCCOMPILATIONUNIT_DOCCOMMENTS = f;
}
static RuntimeException sneakyThrow(Throwable t) {
if (t == null) throw new NullPointerException("t");
Javac.<RuntimeException>sneakyThrow0(t);
return null;
}
@SuppressWarnings("unchecked")
private static <T extends Throwable> void sneakyThrow0(Throwable t) throws T {
throw (T)t;
}
}
```

lombok/javac/JavacTreeMaker.java

```
/*
* Copyright (C) 2013-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import com.sun.tools.javac.code.Attribute;
import com.sun.tools.javac.code.BoundKind;
```

```
import com.sun.tools.javac.code.Symbol;
import com.sun.tools.javac.code.Type;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCArrayAccess;
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCAssert;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCAssignOp;
import com.sun.tools.javac.tree.JCTree.JCBinary;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCBreak;
import com.sun.tools.javac.tree.JCTree.JCCase;
import com.sun.tools.javac.tree.JCTree.JCCatch;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCConditional;
import com.sun.tools.javac.tree.JCTree.JCContinue;
import com.sun.tools.javac.tree.JCTree.JCDoWhileLoop;
import com.sun.tools.javac.tree.JCTree.JCEnhancedForLoop;
import com.sun.tools.javac.tree.JCTree.JCErroneous;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCForLoop;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCIf;
import com.sun.tools.javac.tree.JCTree.JCImport;
import com.sun.tools.javac.tree.JCTree.JCInstanceOf;
import com.sun.tools.javac.tree.JCTree.JCLabeledStatement;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCNewArray;
import com.sun.tools.javac.tree.JCTree.JCNewClass;
import com.sun.tools.javac.tree.JCTree.JCParens;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCSkip;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCSwitch;
import com.sun.tools.javac.tree.JCTree.JCSynchronized;
import com.sun.tools.javac.tree.JCTree.JCThrow;
import com.sun.tools.javac.tree.JCTree.JCTry;
import com.sun.tools.javac.tree.JCTree.JCTypeApply;
import com.sun.tools.javac.tree.JCTree.JCTypeCast;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCUnary;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.JCWhileLoop;
import com.sun.tools.javac.tree.JCTree.JCWildcard;
import com.sun.tools.javac.tree.JCTree.LetExpr;
```

```
import com.sun.tools.javac.tree.JCTree.TypeBoundKind;
import com.sun.tools.javac.tree.TreeInfo;
import com.sun.tools.javac.tree.TreeMaker;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.Name;
import lombok.permit.Permit;
public class JavacTreeMaker {
private final TreeMaker tm;
public JavacTreeMaker(TreeMaker tm) {
this.tm = tm;
}
public TreeMaker getUnderlyingTreeMaker() {
return tm;
}
public JavacTreeMaker at(int pos) {
tm.at(pos);
return this;
}
private static final class FieldId<J> {
private final Class<?> owner;
private final String name;
private final Class<J> fieldType;
FieldId(Class<?> owner, String name, Class<J> fieldType) {
this.owner = owner;
this.name = name;
this.fieldType = fieldType;
}
}
private static final class MethodId<J> {
private final Class<?> owner;
private final String name;
private final Class<J> returnType;
private final Class<?>[] paramTypes;
MethodId(Class<?> owner, String name, Class<J> returnType, Class<?>... types) {
this.owner = owner;
this.name = name;
this.paramTypes = types;
this.returnType = returnType;
}
@Override public String toString() {
StringBuilder out = new StringBuilder();
out.append(returnType.getName()).append("
").append(owner.getName()).append(".").append(name).append("(");
boolean f = true;
for (Class<?> p : paramTypes) {
if (f) f = false;
else out.append(", ");
out.append(p.getName());
}
return out.append(")").toString();
}
}
private static class SchroedingerType {
```

```
final Object value;
private SchroedingerType(Object value) {
this.value = value;
}
@Override public int hashCode() {
return value == null ? -1 : value.hashCode();
}
@Override public boolean equals(Object obj) {
if (obj instanceof SchroedingerType) {
Object other = ((SchroedingerType) obj).value;
return value == null ? other == null : value.equals(other);
}
return false;
}
static Object getFieldCached(ConcurrentMap<String, Object> cache, String className, String
fieldName) {
Object value = cache.get(fieldName);
if (value != null) return value;
try {
value = Permit.getField(Class.forName(className), fieldName).get(null);
} catch (NoSuchFieldException e) {
throw Javac.sneakyThrow(e);
} catch (IllegalAccessException e) {
throw Javac.sneakyThrow(e);
} catch (ClassNotFoundException e) {
throw Javac.sneakyThrow(e);
}
cache.putIfAbsent(fieldName, value);
return value;
}
private static Field NOSUCHFIELDEX_MARKER;
static {
try {
NOSUCHFIELDEX_MARKER = Permit.getField(SchroedingerType.class,
"NOSUCHFIELDEX_MARKER");
} catch (NoSuchFieldException e) {
throw Javac.sneakyThrow(e);
}
}
static Object getFieldCached(ConcurrentMap<Class<?>, Field> cache, Object ref, String fieldName)
throws NoSuchFieldException {
Class<?> c = ref.getClass();
Field field = cache.get(c);
if (field == null) {
try {
field = Permit.getField(c, fieldName);
} catch (NoSuchFieldException e) {
cache.putIfAbsent(c, NOSUCHFIELDEX_MARKER);
throw Javac.sneakyThrow(e);
}
Permit.setAccessible(field);
Field old = cache.putIfAbsent(c, field);
if (old != null) field = old;
```

```
}
if (field == NOSUCHFIELDEX_MARKER) throw new NoSuchFieldException(fieldName);
try {
return field.get(ref);
} catch (IllegalAccessException e) {
throw Javac.sneakyThrow(e);
}
}
}
public static class TypeTag extends SchroedingerType {
private static final ConcurrentMap<String, Object> TYPE_TAG_CACHE = new
ConcurrentHashMap<String, Object>();
private static final ConcurrentMap<Class<?>, Field> FIELD_CACHE = new
ConcurrentHashMap<Class<?>, Field>();
private static final Method TYPE_TYPETAG_METHOD;
static {
Method m = null;
try {
m = Permit.getMethod(Type.class, "getTag");
} catch (NoSuchMethodException e) {}
TYPE_TYPETAG_METHOD = m;
}
private TypeTag(Object value) {
super(value);
}
public static TypeTag typeTag(JCTree o) {
try {
return new TypeTag(getFieldCached(FIELD_CACHE, o, "typetag"));
} catch (NoSuchFieldException e) {
throw Javac.sneakyThrow(e);
}
}
public static TypeTag typeTag(Type t) {
if (t == null) return Javac.CTC_VOID;
try {
return new TypeTag(getFieldCached(FIELD_CACHE, t, "tag"));
} catch (NoSuchFieldException e) {
if (TYPE_TYPETAG_METHOD == null) throw new IllegalStateException("Type " + t.getClass() + " has
neither 'tag' nor getTag()");
try {
return new TypeTag(TYPE_TYPETAG_METHOD.invoke(t));
} catch (IllegalAccessException ex) {
throw Javac.sneakyThrow(ex);
} catch (InvocationTargetException ex) {
throw Javac.sneakyThrow(ex.getCause());
}
}
}
public static TypeTag typeTag(String identifier) {
return new TypeTag(getFieldCached(TYPE_TAG_CACHE, Javac.getJavaCompilerVersion() < 8 ?
"com.sun.tools.javac.code.TypeTags" : "com.sun.tools.javac.code.TypeTag", identifier));
}
}
```

```
public static class TreeTag extends SchroedingerType {
private static final ConcurrentMap<String, Object> TREE_TAG_CACHE = new
ConcurrentHashMap<String, Object>();
private static final Field TAG_FIELD;
private static final Method TAG_METHOD;
private static final MethodId<Integer> OP_PREC = MethodId(TreeInfo.class, "opPrec", int.class,
TreeTag.class);
static {
Method m = null;
try {
m = Permit.getMethod(JCTree.class, "getTag");
} catch (NoSuchMethodException e) {}
if (m != null) {
TAG_FIELD = null;
TAG_METHOD = m;
} else {
Field f = null;
try {
f = Permit.getField(JCTree.class, "tag");
} catch (NoSuchFieldException e) {}
TAG_FIELD = f;
TAG_METHOD = null;
}
}
private TreeTag(Object value) {
super(value);
}
public static TreeTag treeTag(JCTree o) {
try {
if (TAG_METHOD != null) return new TreeTag(TAG_METHOD.invoke(o));
else return new TreeTag(TAG_FIELD.get(o));
} catch (InvocationTargetException e) {
throw Javac.sneakyThrow(e.getCause());
} catch (IllegalAccessException e) {
throw Javac.sneakyThrow(e);
}
}
public static TreeTag treeTag(String identifier) {
return new TreeTag(getFieldCached(TREE_TAG_CACHE, Javac.getJavaCompilerVersion() < 8 ?
"com.sun.tools.javac.tree.JCTree" : "com.sun.tools.javac.tree.JCTree$Tag", identifier));
}
public int getOperatorPrecedenceLevel() {
return invokeAny(null, OP_PREC, value);
}
public boolean isPrefixUnaryOp() {
return Javac.CTC_NEG.equals(this) || Javac.CTC_POS.equals(this) || Javac.CTC_NOT.equals(this) ||
Javac.CTC_COMPL.equals(this) || Javac.CTC_PREDEC.equals(this) ||
Javac.CTC_PREINC.equals(this);
}
}
static <J> MethodId<J> MethodId(Class<?> owner, String name, Class<J> returnType, Class<?>...
types) {
return new MethodId<J>(owner, name, returnType, types);
```

```
}
/**
* Creates a new method ID based on the name of the method to invoke, the return type of that method,
and the types of the parameters.
*
* A method matches if the return type matches, and for each parameter the following holds:
*
* Either (A) the type listed here is the same as, or a subtype of, the type of the method in javac's
TreeMaker, or
* (B) the type listed here is a subtype of SchroedingerType.
*/
static <J> MethodId<J> MethodId(String name, Class<J> returnType, Class<?>... types) {
return new MethodId<J>(TreeMaker.class, name, returnType, types);
}
/**
* Creates a new method ID based on the name of a method in this class, assuming the name of the
method to invoke in TreeMaker has the same name,
* the same return type, and the same parameters (under the same rules as the other MethodId
method).
*/
static <J> MethodId<J> MethodId(String name) {
for (Method m : JavacTreeMaker.class.getDeclaredMethods()) {
if (m.getName().equals(name)) {
@SuppressWarnings("unchecked") Class<J> r = (Class<J>) m.getReturnType();
Class<?>[] p = m.getParameterTypes();
return new MethodId<J>(TreeMaker.class, name, r, p);
}
}
throw new InternalError("Not found: " + name);
}
static <J> FieldId<J> FieldId(Class<?> owner, String name, Class<J> fieldType) {
return new FieldId<J>(owner, name, fieldType);
}
private static final ConcurrentHashMap<FieldId<?>, Object> FIELD_CACHE = new
ConcurrentHashMap<FieldId<?>, Object>();
private static boolean has(FieldId<?> f) {
Object field = FIELD_CACHE.get(f);
if (field == REFLECTIVE_ITEM_NOT_FOUND) return false;
if (field instanceof Field) return true;
try {
return getFromCache(f) != REFLECTIVE_ITEM_NOT_FOUND;
} catch (IllegalStateException e) {
return false;
}
}
private static <J> J get(Object owner, FieldId<J> f) {
Field field = getFromCache(f);
try {
return f.fieldType.cast(field.get(owner));
} catch (IllegalAccessException e) {
throw Javac.sneakyThrow(e);
}
}
```

```
private static <J> void set(Object owner, FieldId<J> f, J val) {
Field field = getFromCache(f);
try {
field.set(owner, val);
} catch (IllegalAccessException e) {
throw Javac.sneakyThrow(e);
} catch (IllegalArgumentException e) {
System.err.println("Type mismatch for: " + field);
throw e;
}
}
private static Field getFromCache(FieldId<?> f) {
Object s = FIELD_CACHE.get(f);
if (s == null) s = addToCache(f);
if (s == REFLECTIVE_ITEM_NOT_FOUND) throw new IllegalStateException("Lombok TreeMaker
frontend issue: no match when looking for field: " + f);
return (Field) s;
}
private static Object addToCache(FieldId<?> f) {
for (Field field : f.owner.getDeclaredFields()) {
if (f.name.equals(field.getName())) {
if (!Modifier.isPublic(field.getModifiers())) field.setAccessible(true);
return FIELD_CACHE.putIfAbsent(f, field);
}
}
return FIELD_CACHE.putIfAbsent(f, REFLECTIVE_ITEM_NOT_FOUND);
}
private static final Object REFLECTIVE_ITEM_NOT_FOUND = new Object[0];
private static final Object REFLECTIVE_ITEM_MULTIPLE_FOUND = new Object[0];
private static final ConcurrentHashMap<MethodId<?>, Object> METHOD_CACHE = new
ConcurrentHashMap<MethodId<?>, Object>();
private boolean has(MethodId<?> m) {
Object method = METHOD_CACHE.get(m);
if (method == REFLECTIVE_ITEM_NOT_FOUND) return false;
if (method instanceof Method) return true;
try {
return getFromCache(m) != REFLECTIVE_ITEM_NOT_FOUND;
} catch (IllegalStateException e) {
return false;
}
}
private <J> J invoke(MethodId<J> m, Object... args) {
return invokeAny(tm, m, args);
}
@SuppressWarnings("unchecked") private static <J> J invokeAny(Object owner, MethodId<J> m,
Object... args) {
Method method = getFromCache(m);
try {
if (m.returnType.isPrimitive()) {
return (J) method.invoke(owner, args);
}
return m.returnType.cast(method.invoke(owner, args));
} catch (InvocationTargetException e) {
```

```
throw Javac.sneakyThrow(e.getCause());
} catch (IllegalAccessException e) {
throw Javac.sneakyThrow(e);
} catch (IllegalArgumentException e) {
System.err.println("Type mismatch for: " + method);
throw e;
}
}
private static boolean tryResolve(MethodId<?> m) {
Object s = METHOD_CACHE.get(m);
if (s == null) s = addToCache(m);
if (s instanceof Method) return true;
return false;
}
private static Method getFromCache(MethodId<?> m) {
Object s = METHOD_CACHE.get(m);
if (s == null) s = addToCache(m);
if (s == REFLECTIVE_ITEM_MULTIPLE_FOUND) throw new IllegalStateException("Lombok
TreeMaker frontend issue: multiple matches when looking for method: " + m);
if (s == REFLECTIVE_ITEM_NOT_FOUND) throw new IllegalStateException("Lombok TreeMaker
frontend issue: no match when looking for method: " + m);
return (Method) s;
}
private static Object addToCache(MethodId<?> m) {
Method found = null;
outer:
for (Method method : m.owner.getDeclaredMethods()) {
if (!m.name.equals(method.getName())) continue;
Class<?>[] t = method.getParameterTypes();
if (t.length != m.paramTypes.length) continue;
for (int i = 0; i < t.length; i++) {
if (Symbol.class.isAssignableFrom(t[i])) continue outer;
if (!SchroedingerType.class.isAssignableFrom(m.paramTypes[i])) {
if (t[i].isPrimitive()) {
if (t[i] != m.paramTypes[i]) continue outer;
} else {
if (!t[i].isAssignableFrom(m.paramTypes[i])) continue outer;
}
}
}
if (found == null) found = method;
else {
METHOD_CACHE.putIfAbsent(m, REFLECTIVE_ITEM_MULTIPLE_FOUND);
return REFLECTIVE_ITEM_MULTIPLE_FOUND;
}
}
if (found == null) {
METHOD_CACHE.putIfAbsent(m, REFLECTIVE_ITEM_NOT_FOUND);
return REFLECTIVE_ITEM_NOT_FOUND;
}
Permit.setAccessible(found);
Object marker = METHOD_CACHE.putIfAbsent(m, found);
if (marker == null) return found;
```

```
return marker;
}
//javac versions: 6-8
private static final MethodId<JCCompilationUnit> TopLevel = MethodId("TopLevel");
public JCCompilationUnit TopLevel(List<JCAnnotation> packageAnnotations, JCExpression pid,
List<JCTree> defs) {
return invoke(TopLevel, packageAnnotations, pid, defs);
}
//javac versions: 6-8
private static final MethodId<JCImport> Import = MethodId("Import");
public JCImport Import(JCTree qualid, boolean staticImport) {
return invoke(Import, qualid, staticImport);
}
//javac versions: 6-8
private static final MethodId<JCClassDecl> ClassDef = MethodId("ClassDef");
public JCClassDecl ClassDef(JCModifiers mods, Name name, List<JCTypeParameter> typarams,
JCExpression extending, List<JCExpression> implementing, List<JCTree> defs) {
return invoke(ClassDef, mods, name, typarams, extending, implementing, defs);
}
//javac versions: 6-8
private static final MethodId<JCMethodDecl> MethodDef = MethodId("MethodDef",
JCMethodDecl.class, JCModifiers.class, Name.class, JCExpression.class, List.class, List.class,
List.class, JCBlock.class, JCExpression.class);
public JCMethodDecl MethodDef(JCModifiers mods, Name name, JCExpression resType,
List<JCTypeParameter> typarams, List<JCVariableDecl> params, List<JCExpression> thrown,
JCBlock body, JCExpression defaultValue) {
return invoke(MethodDef, mods, name, resType, typarams, params, thrown, body, defaultValue);
}
//javac versions: 8
private static final MethodId<JCMethodDecl> MethodDefWithRecvParam = MethodId("MethodDef",
JCMethodDecl.class, JCModifiers.class, Name.class, JCExpression.class, List.class,
JCVariableDecl.class, List.class, List.class, JCBlock.class, JCExpression.class);
public boolean hasMethodDefWithRecvParam() {
return has(MethodDefWithRecvParam);
}
public JCMethodDecl MethodDefWithRecvParam(JCModifiers mods, Name name, JCExpression
resType, List<JCTypeParameter> typarams, JCVariableDecl recvparam, List<JCVariableDecl>
params, List<JCExpression> thrown, JCBlock body, JCExpression defaultValue) {
return invoke(MethodDefWithRecvParam, mods, name, resType, typarams, recvparam, params,
thrown, body, defaultValue);
}
//javac versions: 6-8
private static final MethodId<JCVariableDecl> VarDef = MethodId("VarDef");
public JCVariableDecl VarDef(JCModifiers mods, Name name, JCExpression vartype, JCExpression
init) {
JCVariableDecl varDef = invoke(VarDef, mods, name, vartype, init);
// We use 'position of the type is -1' as indicator in delombok that the original node was written using
JDK10's 'var' feature, because javac desugars 'var' to the real type and doesn't leave any markers
other than the
// node position to indicate that it did so. Unfortunately, that means vardecls we generate look like 'var'
to delombok. Adjust the position to avoid this.
if (varDef.vartype != null && varDef.vartype.pos == -1) varDef.vartype.pos = 0;
return varDef;
```

```
}
//javac versions: 8
private static final MethodId<JCVariableDecl> ReceiverVarDef = MethodId("ReceiverVarDef");
public JCVariableDecl ReceiverVarDef(JCModifiers mods, JCExpression name, JCExpression vartype)
{
return invoke(ReceiverVarDef, mods, name, vartype);
}
//javac versions: 6-8
private static final MethodId<JCSkip> Skip = MethodId("Skip");
public JCSkip Skip() {
return invoke(Skip);
}
//javac versions: 6-8
private static final MethodId<JCBlock> Block = MethodId("Block");
public JCBlock Block(long flags, List<JCStatement> stats) {
return invoke(Block, flags, stats);
}
//javac versions: 6-8
private static final MethodId<JCDoWhileLoop> DoLoop = MethodId("DoLoop");
public JCDoWhileLoop DoLoop(JCStatement body, JCExpression cond) {
return invoke(DoLoop, body, cond);
}
//javac versions: 6-8
private static final MethodId<JCWhileLoop> WhileLoop = MethodId("WhileLoop");
public JCWhileLoop WhileLoop(JCExpression cond, JCStatement body) {
return invoke(WhileLoop, cond, body);
}
//javac versions: 6-8
private static final MethodId<JCForLoop> ForLoop = MethodId("ForLoop");
public JCForLoop ForLoop(List<JCStatement> init, JCExpression cond, List<JCExpressionStatement>
step, JCStatement body) {
return invoke(ForLoop, init, cond, step, body);
}
//javac versions: 6-8
private static final MethodId<JCEnhancedForLoop> ForeachLoop = MethodId("ForeachLoop");
public JCEnhancedForLoop ForeachLoop(JCVariableDecl var, JCExpression expr, JCStatement body)
{
return invoke(ForeachLoop, var, expr, body);
}
//javac versions: 6-8
private static final MethodId<JCLabeledStatement> Labelled = MethodId("Labelled");
public JCLabeledStatement Labelled(Name label, JCStatement body) {
return invoke(Labelled, label, body);
}
//javac versions: 6-8
private static final MethodId<JCSwitch> Switch = MethodId("Switch");
public JCSwitch Switch(JCExpression selector, List<JCCase> cases) {
return invoke(Switch, selector, cases);
}
//javac versions: 6-11
private static final MethodId<JCCase> Case11 = MethodId("Case", JCCase.class, JCExpression.class,
com.sun.tools.javac.util.List.class);
//javac version: 12+
```

```
public static class Case {
private static final Class<?> CASE_KIND_CLASS = classForName(TreeMaker.class,
"com.sun.source.tree.CaseTree$CaseKind");
static final MethodId<JCCase> Case12 = MethodId("Case", JCCase.class, CASE_KIND_CLASS,
com.sun.tools.javac.util.List.class, com.sun.tools.javac.util.List.class, JCTree.class);
static final MethodId<JCCase> Case21 = MethodId("Case", JCCase.class, CASE_KIND_CLASS,
com.sun.tools.javac.util.List.class, JCExpression.class, com.sun.tools.javac.util.List.class,
JCTree.class);
static final Object CASE_KIND_STATEMENT = CASE_KIND_CLASS.getEnumConstants()[0];
}
static Class<?> classForName(Class<?> context, String name) {
try {
return context.getClassLoader().loadClass(name);
} catch (ClassNotFoundException e) {
Error x = new NoClassDefFoundError(e.getMessage());
x.setStackTrace(e.getStackTrace());
throw x;
}
}
public JCCase Case(JCExpression pat, List<JCStatement> stats) {
if (tryResolve(Case11)) return invoke(Case11, pat, stats);
List<JCTree> labels;
if (pat == null) {
labels = tryResolve(DefaultCaseLabel) ? List.of(DefaultCaseLabel()) : List.<JCTree>nil();
} else if (tryResolve(ConstantCaseLabel)) {
labels = List.<JCTree>of(ConstantCaseLabel(pat));
} else {
labels = List.<JCTree>of(pat);
}
if (tryResolve(Case.Case12)) {
return invoke(Case.Case12, Case.CASE_KIND_STATEMENT, labels, stats, null);
}
return invoke(Case.Case21, Case.CASE_KIND_STATEMENT, labels, null, stats, null);
}
//javac versions: 17
private static final MethodId<JCTree> DefaultCaseLabel = MethodId("DefaultCaseLabel",
JCTree.class);
public JCTree DefaultCaseLabel() {
return invoke(DefaultCaseLabel);
}
//javac versions: 19
private static final MethodId<JCTree> ConstantCaseLabel = MethodId("ConstantCaseLabel",
JCTree.class, JCExpression.class);
public JCTree ConstantCaseLabel(JCExpression expr) {
return invoke(ConstantCaseLabel, expr);
}
//javac versions: 6-8
private static final MethodId<JCSynchronized> Synchronized = MethodId("Synchronized");
public JCSynchronized Synchronized(JCExpression lock, JCBlock body) {
return invoke(Synchronized, lock, body);
}
//javac versions: 6-8
```

```
private static final MethodId<JCTry> Try = MethodId("Try", JCTry.class, JCBlock.class, List.class,
JCBlock.class);
public JCTry Try(JCBlock body, List<JCCatch> catchers, JCBlock finalizer) {
return invoke(Try, body, catchers, finalizer);
}
//javac versions: 7-8
private static final MethodId<JCTry> TryWithResources = MethodId("Try", JCTry.class, List.class,
JCBlock.class, List.class, JCBlock.class);
public JCTry Try(List<JCTree> resources, JCBlock body, List<JCCatch> catchers, JCBlock finalizer) {
return invoke(TryWithResources, resources, body, catchers, finalizer);
}
//javac versions: 6-8
private static final MethodId<JCCatch> Catch = MethodId("Catch");
public JCCatch Catch(JCVariableDecl param, JCBlock body) {
return invoke(Catch, param, body);
}
//javac versions: 6-8
private static final MethodId<JCConditional> Conditional = MethodId("Conditional");
public JCConditional Conditional(JCExpression cond, JCExpression thenpart, JCExpression elsepart) {
return invoke(Conditional, cond, thenpart, elsepart);
}
//javac versions: 6-8
private static final MethodId<JCIf> If = MethodId("If");
public JCIf If(JCExpression cond, JCStatement thenpart, JCStatement elsepart) {
return invoke(If, cond, thenpart, elsepart);
}
//javac versions: 6-8
private static final MethodId<JCExpressionStatement> Exec = MethodId("Exec");
public JCExpressionStatement Exec(JCExpression expr) {
return invoke(Exec, expr);
}
//javac version: 6-11
private static final MethodId<JCBreak> Break11 = MethodId("Break", JCBreak.class, Name.class);
//javac version: 12+
private static final MethodId<JCBreak> Break12 = MethodId("Break", JCBreak.class,
JCExpression.class);
public JCBreak Break(Name label) {
if (tryResolve(Break11)) return invoke(Break11, label);
return invoke(Break12, label != null ? Ident(label) : null);
}
//javac versions: 6-8
private static final MethodId<JCContinue> Continue = MethodId("Continue");
public JCContinue Continue(Name label) {
return invoke(Continue, label);
}
//javac versions: 6-8
private static final MethodId<JCReturn> Return = MethodId("Return");
public JCReturn Return(JCExpression expr) {
return invoke(Return, expr);
}
//javac versions: 6-8
private static final MethodId<JCThrow> Throw = MethodId("Throw");
public JCThrow Throw(JCExpression expr) {
```

```
return invoke(Throw, expr);
}
//javac versions: 6-8
private static final MethodId<JCAssert> Assert = MethodId("Assert");
public JCAssert Assert(JCExpression cond, JCExpression detail) {
return invoke(Assert, cond, detail);
}
//javac versions: 6-8
private static final MethodId<JCMethodInvocation> Apply = MethodId("Apply");
public JCMethodInvocation Apply(List<JCExpression> typeargs, JCExpression fn, List<JCExpression>
args) {
return invoke(Apply, typeargs, fn, args);
}
//javac versions: 6-8
private static final MethodId<JCNewClass> NewClass = MethodId("NewClass");
public JCNewClass NewClass(JCExpression encl, List<JCExpression> typeargs, JCExpression clazz,
List<JCExpression> args, JCClassDecl def) {
return invoke(NewClass, encl, typeargs, clazz, args, def);
}
//javac versions: 6-8
private static final MethodId<JCNewArray> NewArray = MethodId("NewArray");
public JCNewArray NewArray(JCExpression elemtype, List<JCExpression> dims, List<JCExpression>
elems) {
return invoke(NewArray, elemtype, dims, elems);
}
//javac versions: 8
// private static final MethodId<JCLambda> Lambda = MethodId("Lambda");
// public JCLambda Lambda(List<JCVariableDecl> params, JCTree body) {
// return invoke(Lambda, params, body);
// }
//javac versions: 6-8
private static final MethodId<JCParens> Parens = MethodId("Parens");
public JCParens Parens(JCExpression expr) {
return invoke(Parens, expr);
}
//javac versions: 6-8
private static final MethodId<JCAssign> Assign = MethodId("Assign");
public JCAssign Assign(JCExpression lhs, JCExpression rhs) {
return invoke(Assign, lhs, rhs);
}
//javac versions: 6-8
//opcode = [6-7] int [8] JCTree.Tag
private static final MethodId<JCAssignOp> Assignop = MethodId("Assignop");
public JCAssignOp Assignop(TreeTag opcode, JCTree lhs, JCTree rhs) {
return invoke(Assignop, opcode.value, lhs, rhs);
}
//javac versions: 6-8
//opcode = [6-7] int [8] JCTree.Tag
private static final MethodId<JCUnary> Unary = MethodId("Unary");
public JCUnary Unary(TreeTag opcode, JCExpression arg) {
return invoke(Unary, opcode.value, arg);
}
//javac versions: 6-8
```

```
//opcode = [6-7] int [8] JCTree.Tag
private static final MethodId<JCBinary> Binary = MethodId("Binary");
public JCBinary Binary(TreeTag opcode, JCExpression lhs, JCExpression rhs) {
return invoke(Binary, opcode.value, lhs, rhs);
}
//javac versions: 6-8
private static final MethodId<JCTypeCast> TypeCast = MethodId("TypeCast");
public JCTypeCast TypeCast(JCTree expr, JCExpression type) {
return invoke(TypeCast, expr, type);
}
//javac versions: 6-8
private static final MethodId<JCInstanceOf> TypeTest = MethodId("TypeTest");
public JCInstanceOf TypeTest(JCExpression expr, JCTree clazz) {
return invoke(TypeTest, expr, clazz);
}
//javac versions: 6-8
private static final MethodId<JCArrayAccess> Indexed = MethodId("Indexed");
public JCArrayAccess Indexed(JCExpression indexed, JCExpression index) {
return invoke(Indexed, indexed, index);
}
//javac versions: 6-8
private static final MethodId<JCFieldAccess> Select = MethodId("Select");
public JCFieldAccess Select(JCExpression selected, Name selector) {
return invoke(Select, selected, selector);
}
//javac versions: 8
// private static final MethodId<JCMemberReference> Reference = MethodId("Reference");
// public JCMemberReference Reference(JCMemberReference.ReferenceMode mode, Name name,
JCExpression expr, List<JCExpression> typeargs) {
// return invoke(Reference, mode, name, expr, typeargs);
// }
//javac versions: 6-8
private static final MethodId<JCIdent> Ident = MethodId("Ident", JCIdent.class, Name.class);
public JCIdent Ident(Name idname) {
return invoke(Ident, idname);
}
//javac versions: 6-8
//tag = [6-7] int [8] TypeTag
private static final MethodId<JCLiteral> Literal = MethodId("Literal", JCLiteral.class, TypeTag.class,
Object.class);
public JCLiteral Literal(TypeTag tag, Object value) {
return invoke(Literal, tag.value, value);
}
//javac versions: 6-8
//typetag = [6-7] int [8] TypeTag
private static final MethodId<JCPrimitiveTypeTree> TypeIdent = MethodId("TypeIdent");
public JCPrimitiveTypeTree TypeIdent(TypeTag typetag) {
return invoke(TypeIdent, typetag.value);
}
//javac versions: 6-8
private static final MethodId<JCArrayTypeTree> TypeArray = MethodId("TypeArray");
public JCArrayTypeTree TypeArray(JCExpression elemtype) {
return invoke(TypeArray, elemtype);
```

```
}
//javac versions: 6-8
private static final MethodId<JCTypeApply> TypeApply = MethodId("TypeApply");
public JCTypeApply TypeApply(JCExpression clazz, List<JCExpression> arguments) {
return invoke(TypeApply, clazz, arguments);
}
//javac versions: 7-8
// private static final MethodId<JCTypeUnion> TypeUnion = MethodId("TypeUnion");
// public JCTypeUnion TypeUnion(List<JCExpression> components) {
// return invoke(TypeUnion, compoonents);
// }
//javac versions: 8
// private static final MethodId<JCTypeIntersection> TypeIntersection = MethodId("TypeIntersection");
// public JCTypeIntersection TypeIntersection(List<JCExpression> components) {
// return invoke(TypeIntersection, components);
// }
//javac versions: 6-8
private static final MethodId<JCTypeParameter> TypeParameter = MethodId("TypeParameter",
JCTypeParameter.class, Name.class, List.class);
public JCTypeParameter TypeParameter(Name name, List<JCExpression> bounds) {
return invoke(TypeParameter, name, bounds);
}
//javac versions: 8
private static final MethodId<JCTypeParameter> TypeParameterWithAnnos =
MethodId("TypeParameter", JCTypeParameter.class, Name.class, List.class, List.class);
public JCTypeParameter TypeParameter(Name name, List<JCExpression> bounds,
List<JCAnnotation> annos) {
return invoke(TypeParameterWithAnnos, name, bounds, annos);
}
//javac versions: 6-8
private static final MethodId<JCWildcard> Wildcard = MethodId("Wildcard");
public JCWildcard Wildcard(TypeBoundKind kind, JCTree type) {
return invoke(Wildcard, kind, type);
}
//javac versions: 6-8
private static final MethodId<TypeBoundKind> TypeBoundKind = MethodId("TypeBoundKind");
public TypeBoundKind TypeBoundKind(BoundKind kind) {
return invoke(TypeBoundKind, kind);
}
//javac versions: 6-8
private static final MethodId<JCAnnotation> Annotation = MethodId("Annotation", JCAnnotation.class,
JCTree.class, List.class);
public JCAnnotation Annotation(JCTree annotationType, List<JCExpression> args) {
return invoke(Annotation, annotationType, args);
}
//javac versions: 8
private static final MethodId<JCAnnotation> TypeAnnotation = MethodId("TypeAnnotation",
JCAnnotation.class, JCTree.class, List.class);
public JCAnnotation TypeAnnotation(JCTree annotationType, List<JCExpression> args) {
return invoke(TypeAnnotation, annotationType, args);
}
//javac versions: 6-8
```

```
private static final MethodId<JCModifiers> ModifiersWithAnnotations = MethodId("Modifiers",
JCModifiers.class, long.class, List.class);
public JCModifiers Modifiers(long flags, List<JCAnnotation> annotations) {
return invoke(ModifiersWithAnnotations, flags, annotations);
}
//javac versions: 6-8
private static final MethodId<JCModifiers> Modifiers = MethodId("Modifiers", JCModifiers.class,
long.class);
public JCModifiers Modifiers(long flags) {
return invoke(Modifiers, flags);
}
//javac versions: 8
// private static final MethodId<JCAnnotatedType> AnnotatedType = MethodId("AnnotatedType");
// public JCAnnotatedType AnnotatedType(List<JCAnnotation> annotations, JCExpression
underlyingType) {
// return invoke(AnnotatedType, annotations, underlyingType);
// }
//javac versions: 6-8
private static final MethodId<JCErroneous> Erroneous = MethodId("Erroneous", JCErroneous.class);
public JCErroneous Erroneous() {
return invoke(Erroneous);
}
//javac versions: 6-8
private static final MethodId<JCErroneous> ErroneousWithErrs = MethodId("Erroneous",
JCErroneous.class, List.class);
public JCErroneous Erroneous(List<? extends JCTree> errs) {
return invoke(ErroneousWithErrs, errs);
}
//javac versions: 6-8
private static final MethodId<LetExpr> LetExpr = MethodId("LetExpr", LetExpr.class, List.class,
JCTree.class);
public LetExpr LetExpr(List<JCVariableDecl> defs, JCTree expr) {
return invoke(LetExpr, defs, expr);
}
//javac versions: 6-8
private static final MethodId<JCClassDecl> AnonymousClassDef = MethodId("AnonymousClassDef");
public JCClassDecl AnonymousClassDef(JCModifiers mods, List<JCTree> defs) {
return invoke(AnonymousClassDef, mods, defs);
}
//javac versions: 6-8
private static final MethodId<LetExpr> LetExprSingle = MethodId("LetExpr", LetExpr.class,
JCVariableDecl.class, JCTree.class);
public LetExpr LetExpr(JCVariableDecl def, JCTree expr) {
return invoke(LetExprSingle, def, expr);
}
//javac versions: 6-8
private static final MethodId<JCIdent> IdentVarDecl = MethodId("Ident", JCIdent.class,
JCVariableDecl.class);
public JCExpression Ident(JCVariableDecl param) {
return invoke(IdentVarDecl, param);
}
//javac versions: 6-8
private static final MethodId<List<JCExpression>> Idents = MethodId("Idents");
```

```
public List<JCExpression> Idents(List<JCVariableDecl> params) {
return invoke(Idents, params);
}
//javac versions: 6-8
private static final MethodId<JCMethodInvocation> App2 = MethodId("App",
JCMethodInvocation.class, JCExpression.class, List.class);
public JCMethodInvocation App(JCExpression meth, List<JCExpression> args) {
return invoke(App2, meth, args);
}
//javac versions: 6-8
private static final MethodId<JCMethodInvocation> App1 = MethodId("App",
JCMethodInvocation.class, JCExpression.class);
public JCMethodInvocation App(JCExpression meth) {
return invoke(App1, meth);
}
//javac versions: 6-8
private static final MethodId<List<JCAnnotation>> Annotations = MethodId("Annotations");
public List<JCAnnotation> Annotations(List<Attribute.Compound> attributes) {
return invoke(Annotations, attributes);
}
//javac versions: 6-8
private static final MethodId<JCLiteral> LiteralWithValue = MethodId("Literal", JCLiteral.class,
Object.class);
public JCLiteral Literal(Object value) {
return invoke(LiteralWithValue, value);
}
//javac versions: 6-8
private static final MethodId<JCAnnotation> AnnotationWithAttributeOnly = MethodId("Annotation",
JCAnnotation.class, Attribute.class);
public JCAnnotation Annotation(Attribute a) {
return invoke(AnnotationWithAttributeOnly, a);
}
//javac versions: 8
private static final MethodId<JCAnnotation> TypeAnnotationWithAttributeOnly =
MethodId("TypeAnnotation", JCAnnotation.class, Attribute.class);
public JCAnnotation TypeAnnotation(Attribute a) {
return invoke(TypeAnnotationWithAttributeOnly, a);
}
//javac versions: 8
private static final MethodId<JCExpression> AnnotatedType = MethodId("AnnotatedType",
JCExpression.class, List.class, JCExpression.class);
public JCExpression AnnotatedType(List<JCAnnotation> annotations, JCExpression underlyingType) {
return invoke(AnnotatedType, annotations, underlyingType);
}
//javac versions: 6-8
private static final MethodId<JCStatement> Call = MethodId("Call");
public JCStatement Call(JCExpression apply) {
return invoke(Call, apply);
}
//javac versions: 6-8
private static final MethodId<JCExpression> Type = MethodId("Type");
public JCExpression Type(Type type) {
return invoke(Type, type);
```

```
}
private static final FieldId<JCVariableDecl> MethodDecl_recvParam = FieldId(JCMethodDecl.class,
"recvparam", JCVariableDecl.class);
//javac versions: 8+
public boolean hasReceiverParameter() {
return has(MethodDecl_recvParam);
}
public JCVariableDecl getReceiverParameter(JCMethodDecl method) {
return get(method, MethodDecl_recvParam);
}
public void setReceiverParameter(JCMethodDecl method, JCVariableDecl param) {
set(method, MethodDecl_recvParam, param);
}
}
```

lombok/javac/PackageName.java

```
/*
* Copyright (C) 2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac;
import java.lang.reflect.Method;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import lombok.permit.Permit;
// Supports JDK6-9
public class PackageName {
private static final Method packageNameMethod = getPackageNameMethod();
private static Method getPackageNameMethod() {
try {
return Permit.getMethod(JCCompilationUnit.class, "getPackageName");
```

```
} catch (Exception e) {
return null;
}
}
public static String getPackageName(JCCompilationUnit cu) {
JCTree t = getPackageNode(cu);
return t != null ? t.toString() : null;
}
public static JCTree getPackageNode(JCCompilationUnit cu) {
if (packageNameMethod != null) try {
Object pkg = packageNameMethod.invoke(cu);
return (pkg instanceof JCFieldAccess || pkg instanceof JCIdent) ? (JCTree) pkg : null;
} catch (Exception e) {}
return cu.pid instanceof JCFieldAccess || cu.pid instanceof JCIdent ? cu.pid : null;
}
}
```

lombok/javac/TreeMirrorMaker.java


```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.javac;
import java.util.Collections;
import java.util.IdentityHashMap;
import java.util.Iterator;
import java.util.Map;
import static lombok.javac.Javac.*;
import lombok.javac.JavacTreeMaker.TypeTag;
import com.sun.source.tree.LabeledStatementTree;
import com.sun.source.tree.VariableTree;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
```

```
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.TreeCopier;
import com.sun.tools.javac.tree.TreeScanner;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.List;
/**
* Makes a copy of any AST node, with some exceptions.
* Exceptions:<ul>
* <li>The symbol ('sym') of a copied variable isn't copied.
* <li>all labels are removed.
* </ul>
*
* The purpose of this class is to make a copy, and then the copy is attributed (resolution info is added).
These exceptions
* are to work around apparent bugs (or at least inconsistencies) in javac sources.
*/
public class TreeMirrorMaker extends TreeCopier<Void> {
private final IdentityHashMap<JCTree, JCTree> originalToCopy = new IdentityHashMap<JCTree,
JCTree>();
public TreeMirrorMaker(JavacTreeMaker maker, Context context) {
super(maker.getUnderlyingTreeMaker());
}
@Override public <T extends JCTree> T copy(T original) {
T copy = super.copy(original);
putIfAbsent(originalToCopy, original, copy);
return copy;
}
@Override public <T extends JCTree> T copy(T original, Void p) {
T copy = super.copy(original, p);
putIfAbsent(originalToCopy, original, copy);
return copy;
}
@Override public <T extends JCTree> List<T> copy(List<T> originals) {
List<T> copies = super.copy(originals);
if (originals != null) {
Iterator<T> it1 = originals.iterator();
Iterator<T> it2 = copies.iterator();
while (it1.hasNext()) putIfAbsent(originalToCopy, it1.next(), it2.next());
}
return copies;
}
@Override public <T extends JCTree> List<T> copy(List<T> originals, Void p) {
List<T> copies = super.copy(originals, p);
if (originals != null) {
Iterator<T> it1 = originals.iterator();
Iterator<T> it2 = copies.iterator();
while (it1.hasNext()) putIfAbsent(originalToCopy, it1.next(), it2.next());
}
return copies;
}
public Map<JCTree, JCTree> getOriginalToCopyMap() {
return Collections.unmodifiableMap(originalToCopy);
}
```

```
// Monitor the following issues when making changes here.
// - https://github.com/projectlombok/lombok/issues/278
// - https://github.com/projectlombok/lombok/issues/729
@Override public JCTree visitVariable(VariableTree node, Void p) {
JCVariableDecl original = node instanceof JCVariableDecl ? (JCVariableDecl) node : null;
JCVariableDecl copy = (JCVariableDecl) super.visitVariable(node, p);
if (original == null) return copy;
copy.sym = original.sym;
if (copy.sym != null) copy.type = original.type;
if (copy.type != null) {
boolean wipeSymAndType = copy.type.isErroneous();
if (!wipeSymAndType) {
TypeTag typeTag = TypeTag.typeTag(copy.type);
wipeSymAndType = (CTC_NONE.equals(typeTag) || CTC_ERROR.equals(typeTag) ||
CTC_UNKNOWN.equals(typeTag) || CTC_UNDETVAR.equals(typeTag));
}
if (wipeSymAndType) {
copy.sym = null;
copy.type = null;
} else {
if (original.vartype != null) {
copy.vartype.type = original.vartype.type;
original.vartype.accept(new TreeScanner() {
@Override public void scan(JCTree tree) {
super.scan(tree);
originalToCopy.get(tree).type = tree.type;
}
@Override public void visitSelect(JCFieldAccess tree) {
super.visitSelect(tree);
((JCFieldAccess) originalToCopy.get(tree)).sym = tree.sym;
}
});
}
}
}
return copy;
}
// Fix for NPE in HandleVal. See https://github.com/projectlombok/lombok/issues/372
// This and visitVariable is rather hacky but we're working around evident bugs or at least
inconsistencies in javac.
@Override public JCTree visitLabeledStatement(LabeledStatementTree node, Void p) {
return node.getStatement().accept(this, p);
}
private <K, V> void putIfAbsent(Map<K, V> map, K key, V value) {
if (!map.containsKey(key)) {
map.put(key, value);
}
}
}
```

lombok/javac/java6/CommentCollectingParser.java

```
/*
* Copyright (C) 2013-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java6;
import static lombok.javac.CommentCatcher.JCCompilationUnit_comments;
import lombok.javac.CommentInfo;
import com.sun.tools.javac.parser.EndPosParser;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.Parser;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.util.List;
class CommentCollectingParser extends EndPosParser {
private final Lexer lexer;
protected CommentCollectingParser(Parser.Factory fac, Lexer S, boolean keepDocComments) {
super(fac, S, keepDocComments);
lexer = S;
}
@Override public JCCompilationUnit compilationUnit() {
JCCompilationUnit result = super.compilationUnit();
if (lexer instanceof CommentCollectingScanner) {
List<CommentInfo> comments = ((CommentCollectingScanner)lexer).getComments();
JCCompilationUnit_comments.set(result, comments);
}
return result;
}
}
```

lombok/javac/java6/CommentCollectingParserFactory.java

```
/*
```

* Copyright (C) 2013-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.java6;
import java.lang.reflect.Field;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.Parser;
import com.sun.tools.javac.util.Context;
import lombok.permit.Permit;
public class CommentCollectingParserFactory extends Parser.Factory {
static Context.Key<Parser.Factory> key() {
return parserFactoryKey;
}
protected CommentCollectingParserFactory(Context context) {
super(context);
}
@Override public Parser newParser(Lexer S, boolean keepDocComments, boolean genEndPos) {
Object x = new CommentCollectingParser(this, S, true);
return (Parser) x;
// CCP is based on a stub which extends nothing, but at runtime the stub is replaced with either
//javac6's EndPosParser which extends Parser, or javac7's EndPosParser which implements Parser.
//Either way this will work out.
}
public static void setInCompiler(JavaCompiler compiler, Context context) {
context.put(CommentCollectingParserFactory.key(), (Parser.Factory)null);
Field field;
try {
field = Permit.getField(JavaCompiler.class, "parserFactory");
field.set(compiler, new CommentCollectingParserFactory(context));
} catch (Exception e) {
throw new IllegalStateException("Could not set comment sensitive parser in the compiler", e);
}
}
}
```

lombok/javac/java6/CommentCollectingScanner.java

```
/*
* Copyright (C) 2011-2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java6;
import java.nio.CharBuffer;
import lombok.javac.CommentInfo;
import lombok.javac.CommentInfo.EndConnection;
import lombok.javac.CommentInfo.StartConnection;
import com.sun.tools.javac.parser.Scanner;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
public class CommentCollectingScanner extends Scanner {
private final ListBuffer<CommentInfo> comments = new ListBuffer<CommentInfo>();
private int endComment = 0;
public CommentCollectingScanner(CommentCollectingScannerFactory factory, CharBuffer charBuffer)
{
super(factory, charBuffer);
}
public CommentCollectingScanner(CommentCollectingScannerFactory factory, char[] input, int
inputLength) {
super(factory, input, inputLength);
}
@Override
protected void processComment(CommentStyle style) {
int prevEndPos = Math.max(prevEndPos(), endComment);
int pos = pos();
int endPos = endPos();
endComment = endPos;
String content = new String(getRawCharacters(pos, endPos));
StartConnection start = determineStartConnection(prevEndPos, pos);
EndConnection end = determineEndConnection(endPos);
```

```
CommentInfo comment = new CommentInfo(prevEndPos, pos, endPos, content, start, end);
comments.append(comment);
super.processComment(style);
}
private EndConnection determineEndConnection(int pos) {
boolean first = true;
for (int i = pos;; i++) {
char c;
try {
c = getRawCharacters(i, i + 1)[0];
} catch (IndexOutOfBoundsException e) {
c = '\n';
}
if (isNewLine(c)) {
return EndConnection.ON_NEXT_LINE;
}
if (Character.isWhitespace(c)) {
first = false;
continue;
}
return first ? EndConnection.DIRECT_AFTER_COMMENT : EndConnection.AFTER_COMMENT;
}
}
private StartConnection determineStartConnection(int from, int to) {
if (from == to) {
return StartConnection.DIRECT_AFTER_PREVIOUS;
}
char[] between = getRawCharacters(from, to);
if (isNewLine(between[between.length - 1])) {
return StartConnection.START_OF_LINE;
}
for (char c : between) {
if (isNewLine(c)) {
return StartConnection.ON_NEXT_LINE;
}
}
return StartConnection.AFTER_PREVIOUS;
}
private boolean isNewLine(char c) {
return c == '\n' || c == '\r';
}
public List<CommentInfo> getComments() {
return comments.toList();
}
}
```

lombok/javac/java6/CommentCollectingScannerFactory.java

```
/*
```

* Copyright (C) 2011-2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.java6;
import java.nio.CharBuffer;
import com.sun.tools.javac.parser.Scanner;
import com.sun.tools.javac.util.Context;
public class CommentCollectingScannerFactory extends Scanner.Factory {
@SuppressWarnings("all")
public static void preRegister(final Context context) {
if (context.get(scannerFactoryKey) == null) {
// Careful! There is voodoo magic here!
//
// Context.Factory is parameterized. make() is for javac6 and below; make(Context) is for javac7 and up.
// this anonymous inner class definition is intentionally 'raw' - the return type of both 'make' methods is 'T',
// which means the compiler will only generate the correct "real" override method (with returntype Object, which is
// the lower bound for T, as a synthetic accessor for the make with returntype ScannerFactory) for that make method which
// is actually on the classpath (either make() for javac6-, or make(Context) for javac7+).
//
// We normally solve this issue via src/stubs, with BOTH make methods listed, but for some reason the presence of a stubbed out
// Context (or even a complete copy, it doesn't matter) results in a really strange eclipse bug, where any mention of any kind
// of com.sun.tools.javac.tree.TreeMaker in a source file disables ALL usage of 'go to declaration' and auto-complete in the entire
// source file.
//
// Thus, in short:
// * Do NOT parameterize the anonymous inner class literal.
// * Leave the return types as 'j.l.Object'.
// * Leave both make methods intact; deleting one has no effect on javac6- / javac7+, but breaks the other. Hard to test for.
```

```
// * Do not stub com.sun.tools.javac.util.Context or any of its inner types, like Factory.
@SuppressWarnings("all")
class MyFactory implements Context.Factory {
// This overrides the javac6- version of make.
public Object make() {
return new CommentCollectingScannerFactory(context);
}
// This overrides the javac7+ version of make.
public Object make(Context c) {
return new CommentCollectingScannerFactory(c);
}
}
@SuppressWarnings("unchecked") Context.Factory<Scanner.Factory> factory = new MyFactory();
context.put(scannerFactoryKey, factory);
}
}
/** Create a new scanner factory. */
protected CommentCollectingScannerFactory(Context context) {
super(context);
}
@Override
public Scanner newScanner(CharSequence input) {
if (input instanceof CharBuffer) {
return new CommentCollectingScanner(this, (CharBuffer)input);
}
char[] array = input.toString().toCharArray();
return newScanner(array, array.length);
}
@Override
public Scanner newScanner(char[] input, int inputLength) {
return new CommentCollectingScanner(this, input, inputLength);
}
}
```

lombok/javac/java7/CommentCollectingParser.java


```
/*
* Copyright (C) 2013-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java7;
import static lombok.javac.CommentCatcher.JCCompilationUnit_comments;
import java.util.List;
import lombok.javac.CommentInfo;
import com.sun.tools.javac.parser.EndPosParser;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.ParserFactory;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
class CommentCollectingParser extends EndPosParser {
private final Lexer lexer;
protected CommentCollectingParser(ParserFactory fac, Lexer S,
boolean keepDocComments, boolean keepLineMap) {
super(fac, S, keepDocComments, keepLineMap);
lexer = S;
}
public JCCompilationUnit parseCompilationUnit() {
JCCompilationUnit result = super.parseCompilationUnit();
if (lexer instanceof CommentCollectingScanner) {
List<CommentInfo> comments = ((CommentCollectingScanner)lexer).getComments();
JCCompilationUnit_comments.set(result, comments);
}
return result;
}
}
```

lombok/javac/java7/CommentCollectingParserFactory.java

```
/*
* Copyright (C) 2013-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java7;
import java.lang.reflect.Field;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.Parser;
import com.sun.tools.javac.parser.ParserFactory;
import com.sun.tools.javac.parser.ScannerFactory;
import com.sun.tools.javac.util.Context;
import lombok.permit.Permit;
public class CommentCollectingParserFactory extends ParserFactory {
private final Context context;
static Context.Key<ParserFactory> key() {
return parserFactoryKey;
}
protected CommentCollectingParserFactory(Context context) {
super(context);
this.context = context;
}
public Parser newParser(CharSequence input, boolean keepDocComments, boolean keepEndPos,
boolean keepLineMap) {
ScannerFactory scannerFactory = ScannerFactory.instance(context);
Lexer lexer = scannerFactory.newScanner(input, true);
Object x = new CommentCollectingParser(this, lexer, true, keepLineMap);
return (Parser) x;
// CCP is based on a stub which extends nothing, but at runtime the stub is replaced with either
//javac6's EndPosParser which extends Parser, or javac7's EndPosParser which implements Parser.
//Either way this will work out.
}
public static void setInCompiler(JavaCompiler compiler, Context context) {
context.put(CommentCollectingParserFactory.key(), (ParserFactory)null);
Field field;
try {
field = Permit.getField(JavaCompiler.class, "parserFactory");
field.set(compiler, new CommentCollectingParserFactory(context));
} catch (Exception e) {
throw new IllegalStateException("Could not set comment sensitive parser in the compiler", e);
}
}
}
```

lombok/javac/java7/CommentCollectingScanner.java

```
/*
* Copyright (C) 2011-2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
```

```
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java7;
import java.nio.CharBuffer;
import lombok.javac.CommentInfo;
import lombok.javac.CommentInfo.EndConnection;
import lombok.javac.CommentInfo.StartConnection;
import com.sun.tools.javac.parser.Scanner;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
public class CommentCollectingScanner extends Scanner {
private final ListBuffer<CommentInfo> comments = new ListBuffer<CommentInfo>();
private int endComment = 0;
public CommentCollectingScanner(CommentCollectingScannerFactory factory, CharBuffer charBuffer)
{
super(factory, charBuffer);
}
public CommentCollectingScanner(CommentCollectingScannerFactory factory, char[] input, int
inputLength) {
super(factory, input, inputLength);
}
@Override
protected void processComment(CommentStyle style) {
int prevEndPos = Math.max(prevEndPos(), endComment);
int pos = pos();
int endPos = endPos();
endComment = endPos;
String content = new String(getRawCharacters(pos, endPos));
StartConnection start = determineStartConnection(prevEndPos, pos);
EndConnection end = determineEndConnection(endPos);
CommentInfo comment = new CommentInfo(prevEndPos, pos, endPos, content, start, end);
comments.append(comment);
super.processComment(style);
}
private EndConnection determineEndConnection(int pos) {
boolean first = true;
for (int i = pos;; i++) {
char c;
```

```
try {
c = getRawCharacters(i, i + 1)[0];
} catch (IndexOutOfBoundsException e) {
c = '\n';
}
if (isNewLine(c)) {
return EndConnection.ON_NEXT_LINE;
}
if (Character.isWhitespace(c)) {
first = false;
continue;
}
return first ? EndConnection.DIRECT_AFTER_COMMENT : EndConnection.AFTER_COMMENT;
}
}
private StartConnection determineStartConnection(int from, int to) {
if (from == to) {
return StartConnection.DIRECT_AFTER_PREVIOUS;
}
char[] between = getRawCharacters(from, to);
if (isNewLine(between[between.length - 1])) {
return StartConnection.START_OF_LINE;
}
for (char c : between) {
if (isNewLine(c)) {
return StartConnection.ON_NEXT_LINE;
}
}
return StartConnection.AFTER_PREVIOUS;
}
private boolean isNewLine(char c) {
return c == '\n' || c == '\r';
}
public List<CommentInfo> getComments() {
return comments.toList();
}
}
```

lombok/javac/java7/CommentCollectingScannerFactory.java

```
/*
* Copyright (C) 2011-2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
```

```
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java7;
import java.nio.CharBuffer;
import com.sun.tools.javac.parser.Scanner;
import com.sun.tools.javac.parser.ScannerFactory;
import com.sun.tools.javac.util.Context;
public class CommentCollectingScannerFactory extends ScannerFactory {
@SuppressWarnings("all")
public static void preRegister(final Context context) {
if (context.get(scannerFactoryKey) == null) {
// Careful! There is voodoo magic here!
//
// Context.Factory is parameterized. make() is for javac6 and below; make(Context) is for javac7 and
up.
// this anonymous inner class definition is intentionally 'raw' - the return type of both 'make' methods is
'T',
// which means the compiler will only generate the correct "real" override method (with returntype
Object, which is
// the lower bound for T, as a synthetic accessor for the make with returntype ScannerFactory) for that
make method which
// is actually on the classpath (either make() for javac6-, or make(Context) for javac7+).
//
// We normally solve this issue via src/stubs, with BOTH make methods listed, but for some reason the
presence of a stubbed out
// Context (or even a complete copy, it doesn't matter) results in a really strange eclipse bug, where any
mention of any kind
// of com.sun.tools.javac.tree.TreeMaker in a source file disables ALL usage of 'go to declaration' and
auto-complete in the entire
// source file.
//
// Thus, in short:
// * Do NOT parameterize the anonymous inner class literal.
// * Leave the return types as 'j.l.Object'.
// * Leave both make methods intact; deleting one has no effect on javac6- / javac7+, but breaks the
other. Hard to test for.
// * Do not stub com.sun.tools.javac.util.Context or any of its inner types, like Factory.
@SuppressWarnings("all")
class MyFactory implements Context.Factory {
// This overrides the javac6- version of make.
public Object make() {
return new CommentCollectingScannerFactory(context);
}
```

```
// This overrides the javac7+ version.
public Object make(Context c) {
return new CommentCollectingScannerFactory(c);
}
}
@SuppressWarnings("unchecked") Context.Factory<ScannerFactory> factory = new MyFactory();
context.put(scannerFactoryKey, factory);
}
}
/** Create a new scanner factory. */
protected CommentCollectingScannerFactory(Context context) {
super(context);
}
@Override
public Scanner newScanner(CharSequence input, boolean keepDocComments) {
if (input instanceof CharBuffer) {
return new CommentCollectingScanner(this, (CharBuffer)input);
}
char[] array = input.toString().toCharArray();
return newScanner(array, array.length, keepDocComments);
}
@Override
public Scanner newScanner(char[] input, int inputLength, boolean keepDocComments) {
return new CommentCollectingScanner(this, input, inputLength);
}
}
```

lombok/javac/java8/CommentCollectingParser.java


```
/*
* Copyright (C) 2013-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.javac.java8;
```

```
import static lombok.javac.CommentCatcher.JCCompilationUnit_comments;
import static lombok.javac.CommentCatcher.JCCompilationUnit_textBlockStarts;
import com.sun.tools.javac.parser.JavacParser;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.ParserFactory;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
class CommentCollectingParser extends JavacParser {
private final Lexer lexer;
protected CommentCollectingParser(ParserFactory fac, Lexer S,
boolean keepDocComments, boolean keepLineMap, boolean keepEndPositions) {
super(fac, S, keepDocComments, keepLineMap, keepEndPositions);
lexer = S;
}
public JCCompilationUnit parseCompilationUnit() {
JCCompilationUnit result = super.parseCompilationUnit();
if (lexer instanceof CommentCollectingScanner) {
JCCompilationUnit_comments.set(result, ((CommentCollectingScanner) lexer).getComments());
JCCompilationUnit_textBlockStarts.set(result, ((CommentCollectingScanner)
lexer).getTextBlockStarts());
}
return result;
}
}
```

lombok/javac/java8/CommentCollectingParserFactory.java

```
/*
* Copyright (C) 2013-2014 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java8;
import java.lang.reflect.Field;
```

```
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.parser.JavacParser;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.ParserFactory;
import com.sun.tools.javac.parser.ScannerFactory;
import com.sun.tools.javac.util.Context;
import lombok.permit.Permit;
public class CommentCollectingParserFactory extends ParserFactory {
private final Context context;
static Context.Key<ParserFactory> key() {
return parserFactoryKey;
}
protected CommentCollectingParserFactory(Context context) {
super(context);
this.context = context;
}
public JavacParser newParser(CharSequence input, boolean keepDocComments, boolean
keepEndPos, boolean keepLineMap) {
ScannerFactory scannerFactory = ScannerFactory.instance(context);
Lexer lexer = scannerFactory.newScanner(input, true);
Object x = new CommentCollectingParser(this, lexer, true, keepLineMap, keepEndPos);
return (JavacParser) x;
// CCP is based on a stub which extends nothing, but at runtime the stub is replaced with either
//javac6's EndPosParser which extends Parser, or javac8's JavacParser which implements Parser.
//Either way this will work out.
}
public JavacParser newParser(CharSequence input, boolean keepDocComments, boolean
keepEndPos, boolean keepLineMap, boolean parseModuleInfo) {
ScannerFactory scannerFactory = ScannerFactory.instance(context);
Lexer lexer = scannerFactory.newScanner(input, true);
Object x = new CommentCollectingParser(this, lexer, true, keepLineMap, keepEndPos);
return (JavacParser) x;
// CCP is based on a stub which extends nothing, but at runtime the stub is replaced with either
//javac6's EndPosParser which extends Parser, or javac8's JavacParser which implements Parser.
//Either way this will work out.
}
public static void setInCompiler(JavaCompiler compiler, Context context) {
context.put(CommentCollectingParserFactory.key(), (ParserFactory) null);
Field field;
try {
field = Permit.getField(JavaCompiler.class, "parserFactory");
field.set(compiler, new CommentCollectingParserFactory(context));
} catch (Exception e) {
throw new IllegalStateException("Could not set comment sensitive parser in the compiler", e);
}
}
}
```

lombok/javac/java8/CommentCollectingScanner.java

```
/*
```

* Copyright (C) 2013-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.javac.java8;
import lombok.javac.CommentInfo;
import com.sun.tools.javac.parser.Scanner;
import com.sun.tools.javac.parser.ScannerFactory;
import com.sun.tools.javac.util.List;
public class CommentCollectingScanner extends Scanner {
private CommentCollectingTokenizer tokenizer;
public CommentCollectingScanner(ScannerFactory fac, CommentCollectingTokenizer tokenizer) {
super(fac, tokenizer);
this.tokenizer = tokenizer;
}
public List<CommentInfo> getComments() {
return tokenizer.getComments();
}
public List<Integer> getTextBlockStarts() {
return tokenizer.getTextBlockStarts();
}
}
```

lombok/javac/java8/CommentCollectingScannerFactory.java

/*
* Copyright (C) 2011-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is

```
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java8;
import java.nio.Buffer;
import java.nio.CharBuffer;
import com.sun.tools.javac.parser.Scanner;
import com.sun.tools.javac.parser.ScannerFactory;
import com.sun.tools.javac.util.Context;
public class CommentCollectingScannerFactory extends ScannerFactory {
public static boolean findTextBlocks;
@SuppressWarnings("all")
public static void preRegister(final Context context) {
if (context.get(scannerFactoryKey) == null) {
// Careful! There is voodoo magic here!
//
// Context.Factory is parameterized. make() is for javac6 and below; make(Context) is for javac7 and
up.
// this anonymous inner class definition is intentionally 'raw' - the return type of both 'make' methods is
'T',
// which means the compiler will only generate the correct "real" override method (with returntype
Object, which is
// the lower bound for T, as a synthetic accessor for the make with returntype ScannerFactory) for that
make method which
// is actually on the classpath (either make() for javac6-, or make(Context) for javac7+).
//
// We normally solve this issue via src/stubs, with BOTH make methods listed, but for some reason the
presence of a stubbed out
// Context (or even a complete copy, it doesn't matter) results in a really strange eclipse bug, where any
mention of any kind
// of com.sun.tools.javac.tree.TreeMaker in a source file disables ALL usage of 'go to declaration' and
auto-complete in the entire
// source file.
//
// Thus, in short:
// * Do NOT parameterize the anonymous inner class literal.
// * Leave the return types as 'j.l.Object'.
// * Leave both make methods intact; deleting one has no effect on javac6- / javac7+, but breaks the
other. Hard to test for.
// * Do not stub com.sun.tools.javac.util.Context or any of its inner types, like Factory.
@SuppressWarnings("all")
class MyFactory implements Context.Factory {
// This overrides the javac6- version of make.
```

```
public Object make() {
return new CommentCollectingScannerFactory(context);
}
// This overrides the javac7+ version.
public Object make(Context c) {
return new CommentCollectingScannerFactory(c);
}
}
@SuppressWarnings("unchecked") Context.Factory<ScannerFactory> factory = new MyFactory();
context.put(scannerFactoryKey, factory);
}
}
/** Create a new scanner factory. */
protected CommentCollectingScannerFactory(Context context) {
super(context);
}
@SuppressWarnings("all")
@Override
public Scanner newScanner(CharSequence input, boolean keepDocComments) {
char[] array;
int limit;
if (input instanceof CharBuffer && ((CharBuffer) input).hasArray()) {
CharBuffer cb = (CharBuffer) input;
((Buffer)cb.compact()).flip();
array = cb.array();
limit = cb.limit();
} else {
array = input.toString().toCharArray();
limit = array.length;
}
if (array.length == limit) {
// work around a bug where the last comment in a file falls away in this case.
char[] d = new char[limit + 1];
System.arraycopy(array, 0, d, 0, limit);
array = d;
}
return newScanner(array, limit, keepDocComments);
}
@Override
public Scanner newScanner(char[] input, int inputLength, boolean keepDocComments) {
return new CommentCollectingScanner(this, CommentCollectingTokenizer.create(this, input,
inputLength, findTextBlocks));
}
}
```

lombok/javac/java8/CommentCollectingTokenizer.java

```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
```

```
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java8;
import java.nio.CharBuffer;
import com.sun.tools.javac.parser.JavaTokenizer;
import com.sun.tools.javac.parser.ScannerFactory;
import com.sun.tools.javac.parser.Tokens.Comment;
import com.sun.tools.javac.parser.Tokens.Comment.CommentStyle;
import com.sun.tools.javac.parser.Tokens.Token;
import com.sun.tools.javac.parser.UnicodeReader;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.ListBuffer;
import lombok.javac.CommentInfo;
import lombok.javac.CommentInfo.EndConnection;
import lombok.javac.CommentInfo.StartConnection;
class CommentCollectingTokenizer extends JavaTokenizer {
private static final boolean tokenizerIsUnicodeReader =
JavaTokenizer.class.getSuperclass().getSimpleName().equals("UnicodeReader");
private int prevEndPosition = 0;
private final ListBuffer<CommentInfo> comments = new ListBuffer<CommentInfo>();
private final ListBuffer<Integer> textBlockStarts;
private int endComment = 0;
static CommentCollectingTokenizer create(ScannerFactory fac, char[] buf, int inputLength, boolean
findTextBlocks) {
if (tokenizerIsUnicodeReader) {
return new CommentCollectingTokenizer(fac, buf, inputLength, findTextBlocks, true);
}
return new CommentCollectingTokenizer(fac, buf, inputLength, findTextBlocks);
}
// pre java 16
private CommentCollectingTokenizer(ScannerFactory fac, char[] buf, int inputLength, boolean
findTextBlocks) {
super(fac, new PositionUnicodeReader(fac, buf, inputLength));
textBlockStarts = findTextBlocks ? new ListBuffer<Integer>() : null;
}
// from java 16
```

```
private CommentCollectingTokenizer(ScannerFactory fac, char[] buf, int inputLength, boolean
findTextBlocks, boolean java16Signature) {
super(fac, buf, inputLength);
textBlockStarts = findTextBlocks ? new ListBuffer<Integer>() : null;
}
int pos() {
if (tokenizerIsUnicodeReader) {
return position();
}
return ((PositionUnicodeReader) reader).pos();
}
@Override public Token readToken() {
Token token = super.readToken();
prevEndPosition = pos();
if (textBlockStarts != null && (prevEndPosition - token.pos > 5) &&
token.getClass().getName().endsWith("$StringToken")) {
char[] start = reader().getRawCharacters(token.pos, token.pos + 3);
if (start[0] == "" && start[1] == "" && start[2] == "") textBlockStarts.append(token.pos);
}
return token;
}
@Override
protected Comment processComment(int pos, int endPos, CommentStyle style) {
int prevEndPos = Math.max(prevEndPosition, endComment);
endComment = endPos;
String content = new String(reader().getRawCharacters(pos, endPos));
StartConnection start = determineStartConnection(prevEndPos, pos);
EndConnection end = determineEndConnection(endPos);
CommentInfo comment = new CommentInfo(prevEndPos, pos, endPos, content, start, end);
comments.append(comment);
return super.processComment(pos, endPos, style);
}
private EndConnection determineEndConnection(int pos) {
boolean first = true;
for (int i = pos;; i++) {
char c;
try {
c = reader().getRawCharacters(i, i + 1)[0];
} catch (IndexOutOfBoundsException e) {
c = '\n';
}
if (isNewLine(c)) {
return EndConnection.ON_NEXT_LINE;
}
if (Character.isWhitespace(c)) {
first = false;
continue;
}
return first ? EndConnection.DIRECT_AFTER_COMMENT : EndConnection.AFTER_COMMENT;
}
}
private StartConnection determineStartConnection(int from, int to) {
if (from == to) {
```

```
return StartConnection.DIRECT_AFTER_PREVIOUS;
}
char[] between = reader().getRawCharacters(from, to);
if (isNewLine(between[between.length - 1])) {
return StartConnection.START_OF_LINE;
}
for (char c : between) {
if (isNewLine(c)) {
return StartConnection.ON_NEXT_LINE;
}
}
return StartConnection.AFTER_PREVIOUS;
}
private boolean isNewLine(char c) {
return c == '\n' || c == '\r';
}
public List<CommentInfo> getComments() {
return comments.toList();
}
public List<Integer> getTextBlockStarts() {
return textBlockStarts == null ? List.<Integer>nil() : textBlockStarts.toList();
}
private UnicodeReader reader() {
if (tokenizerIsUnicodeReader) {
return (UnicodeReader) (Object) this;
}
return reader;
}
static class PositionUnicodeReader extends UnicodeReader {
protected PositionUnicodeReader(ScannerFactory sf, char[] input, int inputLength) {
super(sf, input, inputLength);
}
public PositionUnicodeReader(ScannerFactory sf, CharBuffer buffer) {
super(sf, buffer);
}
int pos() {
return bp;
}
}
}
```

lombok/javac/java9/CommentCollectingParser.java


```
/*
* Copyright (C) 2013-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
```

```
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java9;
import static lombok.javac.CommentCatcher.JCCompilationUnit_comments;
import static lombok.javac.CommentCatcher.JCCompilationUnit_textBlockStarts;
import lombok.javac.java8.CommentCollectingScanner;
import com.sun.tools.javac.parser.JavacParser;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.ParserFactory;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
class CommentCollectingParser extends JavacParser {
private final Lexer lexer;
protected CommentCollectingParser(ParserFactory fac, Lexer S,
boolean keepDocComments, boolean keepLineMap, boolean keepEndPositions, boolean
parseModuleInfo) {
super(fac, S, keepDocComments, keepLineMap, keepEndPositions, parseModuleInfo);
lexer = S;
}
public JCCompilationUnit parseCompilationUnit() {
JCCompilationUnit result = super.parseCompilationUnit();
if (lexer instanceof CommentCollectingScanner) {
JCCompilationUnit_comments.set(result, ((CommentCollectingScanner) lexer).getComments());
JCCompilationUnit_textBlockStarts.set(result, ((CommentCollectingScanner)
lexer).getTextBlockStarts());
}
return result;
}
}
```

lombok/javac/java9/CommentCollectingParserFactory.java

```
/*
* Copyright (C) 2013-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
```

```
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.javac.java9;
import java.lang.reflect.Field;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.parser.JavacParser;
import com.sun.tools.javac.parser.Lexer;
import com.sun.tools.javac.parser.ParserFactory;
import com.sun.tools.javac.parser.ScannerFactory;
import com.sun.tools.javac.util.Context;
import lombok.permit.Permit;
public class CommentCollectingParserFactory extends ParserFactory {
private final Context context;
static Context.Key<ParserFactory> key() {
return parserFactoryKey;
}
protected CommentCollectingParserFactory(Context context) {
super(context);
this.context = context;
}
public JavacParser newParser(CharSequence input, boolean keepDocComments, boolean
keepEndPos, boolean keepLineMap) {
return newParser(input, keepDocComments, keepEndPos, keepLineMap, false);
}
public JavacParser newParser(CharSequence input, boolean keepDocComments, boolean
keepEndPos, boolean keepLineMap, boolean parseModuleInfo) {
ScannerFactory scannerFactory = ScannerFactory.instance(context);
Lexer lexer = scannerFactory.newScanner(input, true);
Object x = new CommentCollectingParser(this, lexer, true, keepLineMap, keepEndPos,
parseModuleInfo);
return (JavacParser) x;
// CCP is based on a stub which extends nothing, but at runtime the stub is replaced with either
//javac6's EndPosParser which extends Parser, or javac-9's JavacParser which implements Parser.
//Either way this will work out.
}
public static void setInCompiler(JavaCompiler compiler, Context context) {
context.put(CommentCollectingParserFactory.key(), (ParserFactory) null);
Field field;
try {
field = Permit.getField(JavaCompiler.class, "parserFactory");
field.set(compiler, new CommentCollectingParserFactory(context));
```

```
} catch (Exception e) {
throw new IllegalStateException("Could not set comment sensitive parser in the compiler", e);
}
}
}
```

lombok/permit/Permit.java

```
/*
* Copyright (C) 2018-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.permit;
import java.lang.reflect.AccessibleObject;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.net.URL;
import javax.tools.JavaFileManager;
import org.eclipse.jdt.internal.compiler.CompilationResult;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.util.List;
// sunapi suppresses javac's warning about using Unsafe; 'all' suppresses eclipse's warning about the unspecified 'sunapi' key. Leave them both.
// Yes, javac's definition of the word 'all' is quite contrary to what the dictionary says it means. 'all' does NOT include 'sunapi' according to javac.
@SuppressWarnings({"sunapi", "all"})
public class Permit {
private Permit() {}
private static final long ACCESSIBLE_OVERRIDE_FIELD_OFFSET;
```

```
private static final IllegalAccessException INIT_ERROR;
private static final sun.misc.Unsafe UNSAFE = (sun.misc.Unsafe)
reflectiveStaticFieldAccess(sun.misc.Unsafe.class, "theUnsafe");
static {
Field f;
long g;
Throwable ex;
try {
g = getOverrideFieldOffset();
ex = null;
} catch (Throwable t) {
f = null;
g = -1L;
ex = t;
}
ACCESSIBLE_OVERRIDE_FIELD_OFFSET = g;
if (ex == null) INIT_ERROR = null;
else if (ex instanceof IllegalAccessException) INIT_ERROR = (IllegalAccessException) ex;
else {
INIT_ERROR = new IllegalAccessException("Cannot initialize Unsafe-based permit");
INIT_ERROR.initCause(ex);
}
}
public static <T extends AccessibleObject> T setAccessible(T accessor) {
if (INIT_ERROR == null) {
UNSAFE.putBoolean(accessor, ACCESSIBLE_OVERRIDE_FIELD_OFFSET, true);
} else {
accessor.setAccessible(true);
}
return accessor;
}
private static long getOverrideFieldOffset() throws Throwable {
Field f = null;
Throwable saved = null;
try {
f = AccessibleObject.class.getDeclaredField("override");
} catch (Throwable t) {
saved = t;
}
if (f != null) {
return UNSAFE.objectFieldOffset(f);
}
// The below seems very risky, but for all AccessibleObjects in java today it does work, and starting with
JDK12, making the field accessible is no longer possible.
try {
return UNSAFE.objectFieldOffset(Fake.class.getDeclaredField("override"));
} catch (Throwable t) {
throw saved;
}
}
static class Fake {
boolean override;
Object accessCheckCache;
```

```
}
public static Method getMethod(Class<?> c, String mName, Class<?>... parameterTypes) throws
NoSuchMethodException {
Method m = null;
Class<?> oc = c;
while (c != null) {
try {
m = c.getDeclaredMethod(mName, parameterTypes);
break;
} catch (NoSuchMethodException e) {}
c = c.getSuperclass();
}
if (m == null) throw new NoSuchMethodException(oc.getName() + " :: " + mName + "(args)");
return setAccessible(m);
}
public static Method permissiveGetMethod(Class<?> c, String mName, Class<?>... parameterTypes) {
try {
return getMethod(c, mName, parameterTypes);
} catch (Exception ignore) {
return null;
}
}
public static Field getField(Class<?> c, String fName) throws NoSuchFieldException {
Field f = null;
Class<?> oc = c;
while (c != null) {
try {
f = c.getDeclaredField(fName);
break;
} catch (NoSuchFieldException e) {}
c = c.getSuperclass();
}
if (f == null) throw new NoSuchFieldException(oc.getName() + " :: " + fName);
return setAccessible(f);
}
public static Field permissiveGetField(Class<?> c, String fName) {
try {
return getField(c, fName);
} catch (Exception ignore) {
return null;
}
}
public static <T> T permissiveReadField(Class<T> type, Field f, Object instance) {
try {
return type.cast(f.get(instance));
} catch (Exception ignore) {
return null;
}
}
public static <T> Constructor<T> getConstructor(Class<T> c, Class<?>... parameterTypes) throws
NoSuchMethodException {
return setAccessible(c.getDeclaredConstructor(parameterTypes));
}
```

```
private static Object reflectiveStaticFieldAccess(Class<?> c, String fName) {
try {
Field f = c.getDeclaredField(fName);
f.setAccessible(true);
return f.get(null);
} catch (Exception e) {
return null;
}
}
public static boolean isDebugReflection() {
return !"false".equals(System.getProperty("lombok.debug.reflection", "false"));
}
public static void handleReflectionDebug(Throwable t, Throwable initError) {
if (!isDebugReflection()) return;
System.err.println("** LOMBOK REFLECTION exception: " + t.getClass() + ": " + (t.getMessage() ==
null ? "(no message)" : t.getMessage()));
t.printStackTrace(System.err);
if (initError != null) {
System.err.println("*** ADDITIONALLY, exception occurred setting up reflection: ");
initError.printStackTrace(System.err);
}
}
public static Object invoke(Method m, Object receiver, Object... args) throws IllegalAccessException,
InvocationTargetException {
return invoke(null, m, receiver, args);
}
public static Object invoke(Throwable initError, Method m, Object receiver, Object... args) throws
IllegalAccessException, InvocationTargetException {
try {
return m.invoke(receiver, args);
} catch (IllegalAccessException e) {
handleReflectionDebug(e, initError);
throw e;
} catch (RuntimeException e) {
handleReflectionDebug(e, initError);
throw e;
} catch (Error e) {
handleReflectionDebug(e, initError);
throw e;
}
}
public static Object invokeSneaky(Method m, Object receiver, Object... args) {
return invokeSneaky(null, m, receiver, args);
}
public static Object invokeSneaky(Throwable initError, Method m, Object receiver, Object... args) {
try {
return m.invoke(receiver, args);
} catch (NoClassDefFoundError e) {
handleReflectionDebug(e, initError);
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
return null;
} catch (NullPointerException e) {
```

```
handleReflectionDebug(e, initError);
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
return null;
} catch (IllegalAccessException e) {
handleReflectionDebug(e, initError);
throw sneakyThrow(e);
} catch (InvocationTargetException e) {
throw sneakyThrow(e.getCause());
} catch (RuntimeException e) {
handleReflectionDebug(e, initError);
throw e;
} catch (Error e) {
handleReflectionDebug(e, initError);
throw e;
}
}
public static <T> T newInstance(Constructor<T> c, Object... args) throws IllegalAccessException,
InvocationTargetException, InstantiationException {
return newInstance(null, c, args);
}
public static <T> T newInstance(Throwable initError, Constructor<T> c, Object... args) throws
IllegalAccessException, InvocationTargetException, InstantiationException {
try {
return c.newInstance(args);
} catch (IllegalAccessException e) {
handleReflectionDebug(e, initError);
throw e;
} catch (InstantiationException e) {
handleReflectionDebug(e, initError);
throw e;
} catch (RuntimeException e) {
handleReflectionDebug(e, initError);
throw e;
} catch (Error e) {
handleReflectionDebug(e, initError);
throw e;
}
}
public static <T> T newInstanceSneaky(Constructor<T> c, Object... args) {
return newInstanceSneaky(null, c, args);
}
public static <T> T newInstanceSneaky(Throwable initError, Constructor<T> c, Object... args) {
try {
return c.newInstance(args);
} catch (NoClassDefFoundError e) {
handleReflectionDebug(e, initError);
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
return null;
} catch (NullPointerException e) {
handleReflectionDebug(e, initError);
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
```

```
//do anything useful here.
return null;
} catch (IllegalAccessException e) {
handleReflectionDebug(e, initError);
throw sneakyThrow(e);
} catch (InstantiationException e) {
handleReflectionDebug(e, initError);
throw sneakyThrow(e);
} catch (InvocationTargetException e) {
throw sneakyThrow(e.getCause());
} catch (RuntimeException e) {
handleReflectionDebug(e, initError);
throw e;
} catch (Error e) {
handleReflectionDebug(e, initError);
throw e;
}
}
public static <T> T get(Field f, Object receiver) throws IllegalAccessException {
try {
return (T) f.get(receiver);
} catch (IllegalAccessException e) {
handleReflectionDebug(e, null);
throw e;
} catch (RuntimeException e) {
handleReflectionDebug(e, null);
throw e;
} catch (Error e) {
handleReflectionDebug(e, null);
throw e;
}
}
public static void set(Field f, Object receiver, Object newValue) throws IllegalAccessException {
try {
f.set(receiver, newValue);
} catch (IllegalAccessException e) {
handleReflectionDebug(e, null);
throw e;
} catch (RuntimeException e) {
handleReflectionDebug(e, null);
throw e;
} catch (Error e) {
handleReflectionDebug(e, null);
throw e;
}
}
public static void reportReflectionProblem(Throwable initError, String msg) {
if (!isDebugReflection()) return;
System.err.println("** LOMBOK REFLECTION issue: " + msg);
if (initError != null) {
System.err.println("*** ADDITIONALLY, exception occurred setting up reflection: ");
initError.printStackTrace(System.err);
}
```

```
}
public static RuntimeException sneakyThrow(Throwable t) {
if (t == null) throw new NullPointerException("t");
return Permit.<RuntimeException>sneakyThrow0(t);
}
@SuppressWarnings("unchecked")
private static <T extends Throwable> T sneakyThrow0(Throwable t) throws T {
throw (T)t;
}
}
```

lombok/permit/dummy/Child.java

```
package lombok.permit.dummy;
@SuppressWarnings("all")
public abstract class Child extends Parent {
private transient volatile boolean foo;
private transient volatile Object[] bar;
private transient volatile Object baz;
}
```

lombok/permit/dummy/GrandChild.java

```
package lombok.permit.dummy;
@SuppressWarnings("all")
public final class GrandChild extends Child {
private Class<?> a;
private int b;
private String c;
private Class<?> d;
private Class<?>[] e;
private Class<?>[] f;
private int g;
private transient String h;
private transient Object i;
private byte[] j;
private byte[] k;
private byte[] l;
private volatile Object m;
private Object n;
}
```

lombok/permit/dummy/Parent.java

```
package lombok.permit.dummy;
import java.io.OutputStream;
@SuppressWarnings("all")
public class Parent {
boolean first;
```

```
static final Object staticObj = OutputStream.class;
volatile Object second;
private static volatile boolean staticSecond;
private static volatile boolean staticThird;
}
```

lombok/permit/dummy/package-info.java

```
/**
* This package recreates the type hierarchy of {@code java.lang.reflect.AccessibleObject} and friends (such as {@code java.lang.reflect.Method});
* its purpose is to allow us to ask {@code sun.misc.internal.Unsafe} about the exact offset of the {@code override} field of {@code AccessibleObject};
* asking about that field directly doesn't work after jdk14, presumably because the fields of AO are expressly hidden somehow.
*
* NB: It's usually 12, on the vast majority of OS, VM, and architecture combos.
*/
package lombok.permit.dummy;
```

lombok/permit/package-info.java

```
/**
* This is a reduced copy of the nqzero Permit-reflect project.
* https://github.com/nqzero/permit-reflect
*
* Many thanks to nqzero. The permit-reflect project is, like lombok itself, licensed under the MIT license.
* See https://github.com/nqzero/permit-reflect/blob/master/License for license info.
*/
package lombok.permit;
```

lombok/eclipse/agent/EclipseLoaderPatcher.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import lombok.patcher.ClassRootFinder;
import lombok.patcher.Hook;
import lombok.patcher.MethodTarget;
import lombok.patcher.ScriptManager;
import lombok.patcher.StackRequest;
import lombok.patcher.scripts.ScriptBuilder;
public class EclipseLoaderPatcher {
private static final String TRANSPLANTS_CLASS_NAME =
"lombok.eclipse.agent.EclipseLoaderPatcherTransplants";
static final String[] OSGI_TYPES = {
"org/eclipse/osgi/internal/baseadaptor/DefaultClassLoader",
"org/eclipse/osgi/framework/adapter/core/AbstractClassLoader",
"org/eclipse/osgi/internal/loader/ModuleClassLoader"
};
public static void patchEquinoxLoaders(ScriptManager sm, Class<?> launchingContext) {
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.osgi.internal.baseadaptor.DefaultClassLoader", "loadClass",
"java.lang.Class", "java.lang.String", "boolean"))
.target(new MethodTarget("org.eclipse.osgi.framework.adapter.core.AbstractClassLoader",
"loadClass",
"java.lang.Class", "java.lang.String", "boolean"))
.target(new MethodTarget("org.eclipse.osgi.internal.loader.ModuleClassLoader", "loadClass",
"java.lang.Class", "java.lang.String", "boolean"))
.decisionMethod(new Hook(TRANSPLANTS_CLASS_NAME, "overrideLoadDecide", "boolean",
"java.lang.ClassLoader", "java.lang.String", "boolean"))
.valueMethod(new Hook(TRANSPLANTS_CLASS_NAME, "overrideLoadResult", "java.lang.Class",
"java.lang.ClassLoader", "java.lang.String", "boolean"))
.transplant()
.request(StackRequest.THIS, StackRequest.PARAM1, StackRequest.PARAM2).build());
sm.addScript(ScriptBuilder.addField().setPublic().setVolatile()
.fieldType("Ljava/lang/ClassLoader;")
.fieldName("lombok$shadowLoader")
.targetClass("org.eclipse.osgi.internal.baseadaptor.DefaultClassLoader")
.targetClass("org.eclipse.osgi.framework.adapter.core.AbstractClassLoader")
.targetClass("org.eclipse.osgi.internal.loader.ModuleClassLoader")
.build());
sm.addScript(ScriptBuilder.addField().setPublic().setVolatile().setStatic()
.fieldType("Ljava/lang/Class;")
.fieldName("lombok$shadowLoaderClass")
.targetClass("org.eclipse.osgi.internal.baseadaptor.DefaultClassLoader")
.targetClass("org.eclipse.osgi.framework.adapter.core.AbstractClassLoader")
.targetClass("org.eclipse.osgi.internal.loader.ModuleClassLoader")
.build());
sm.addScript(ScriptBuilder.addField().setPublic().setStatic().setFinal()
.fieldType("Ljava/lang/String;")
.fieldName("lombok$location")
.targetClass("org.eclipse.osgi.internal.baseadaptor.DefaultClassLoader")
```

```
.targetClass("org.eclipse.osgi.framework.adapter.core.AbstractClassLoader")
.targetClass("org.eclipse.osgi.internal.loader.ModuleClassLoader")
.value(ClassRootFinder.findClassRootOfClass(launchingContext))
.build());
}
}
```

lombok/eclipse/agent/EclipseLoaderPatcherTransplants.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import java.io.InputStream;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.jar.JarFile;
import java.util.zip.ZipEntry;
/**
* Contains all the code to be transplanted into eclipse.
*
* Do not use:
*
* * Annotations
* * Generics
* * Varargs
* * Auto (un)boxing
* * class literals
```

```
*
* The above because this code is compiled with -source 1.4, and is transplanted.
*
* NB: The suppress warnings will be stripped out before compilation.
*/
@SuppressWarnings("all")
public class EclipseLoaderPatcherTransplants {
public static boolean overrideLoadDecide(ClassLoader original, String name, boolean resolve) {
return name.startsWith("lombok.");
}
public static Class overrideLoadResult(ClassLoader original, String name, boolean resolve) throws
ClassNotFoundException {
boolean bootstrap = name.equals("lombok.launch.PackageShader") ||
name.equals("lombok.launch.ClassFileMetaData");
try {
Field shadowLoaderField = original.getClass().getField("lombok$shadowLoader");
ClassLoader shadowLoader = (ClassLoader) shadowLoaderField.get(original);
if (bootstrap || shadowLoader == null) {
synchronized ("lombok$shadowLoader$globalLock".intern()) {
shadowLoader = (ClassLoader) shadowLoaderField.get(original);
if (bootstrap || shadowLoader == null) {
Class shadowClassLoaderClass = (Class)
original.getClass().getField("lombok$shadowLoaderClass").get(null);
Class classLoaderClass = Class.forName("java.lang.ClassLoader");
String jarLoc = (String) original.getClass().getField("lombok$location").get(null);
if (bootstrap || shadowClassLoaderClass == null) {
JarFile jf = new JarFile(jarLoc);
InputStream in = null;
try {
ZipEntry entry = jf.getEntry(bootstrap ? (name.replace(".", "/") + ".class") :
"lombok/launch/ShadowClassLoader.class");
if (entry == null) {
bootstrap = false;
entry = jf.getEntry("lombok/launch/ShadowClassLoader.class");
}
in = jf.getInputStream(entry);
byte[] bytes = new byte[65536];
int len = 0;
while (true) {
int r = in.read(bytes, len, bytes.length - len);
if (r == -1) break;
len += r;
if (len == bytes.length) throw new IllegalStateException((bootstrap ? name :
"lombok.launch.ShadowClassLoader") + " too large.");
}
in.close();
try {
/* Since Java 16 reflective access to ClassLoader.defineClass is no longer permitted. The
recommended solution
* is to use MethodHandles.lookup().defineClass which is useless here because it is limited to classes in
the
* same package. Fortunately this code gets transplanted into a ClassLoader and we can call the parent
method
```

```
* using a MethodHandle. To support old Java versions we use a reflective version of the code snippet below.
*
* Lookup lookup = MethodHandles.lookup();
* MethodType type = MethodType.methodType(Class.class, new Class[] {String.class, byte[].class, int.class, int.class});
* MethodHandle method = lookup.findVirtual(original.getClass(), "defineClass", type);
* shadowClassLoaderClass = (Class) method.invokeWithArguments(original, "lombok.launch.ShadowClassLoader", bytes, new Integer(0), new Integer(len)})
*/
Class methodHandles = Class.forName("java.lang.invoke.MethodHandles");
Class methodHandle = Class.forName("java.lang.invoke.MethodHandle");
Class methodType = Class.forName("java.lang.invoke.MethodType");
Class methodHandlesLookup = Class.forName("java.lang.invoke.MethodHandles$Lookup");
Method lookupMethod = methodHandles.getDeclaredMethod("lookup", null);
Method methodTypeMethod = methodType.getDeclaredMethod("methodType", new Class[] {Class.class, Class[].class});
Method findVirtualMethod = methodHandlesLookup.getDeclaredMethod("findVirtual", new Class[] {Class.class, String.class, methodType});
Method invokeMethod = methodHandle.getDeclaredMethod("invokeWithArguments", new Class[] {Object[].class});
Object lookup = lookupMethod.invoke(null, null);
Object type = methodTypeMethod.invoke(null, new Object[] {Class.class, new Class[] {String.class, byte[].class, int.class, int.class}});
Object method = findVirtualMethod.invoke(lookup, new Object[] {original.getClass(), "defineClass", type});
Class c = (Class) invokeMethod.invoke(method, new Object[] {new Object[] {original, bootstrap ? name : "lombok.launch.ShadowClassLoader", bytes, new Integer(0), new Integer(len)}});
if (bootstrap) return c;
shadowClassLoaderClass = c;
} catch (ClassNotFoundException e) {
// Ignore, old Java
}
if (shadowClassLoaderClass == null) {
Class[] paramTypes = new Class[4];
paramTypes[0] = "".getClass();
paramTypes[1] = new byte[0].getClass();
paramTypes[2] = Integer.TYPE;
paramTypes[3] = paramTypes[2];
Method defineClassMethod = classLoaderClass.getDeclaredMethod("defineClass", paramTypes);
defineClassMethod.setAccessible(true);
Class c = (Class) defineClassMethod.invoke(original, new Object[] {bootstrap ? name : "lombok.launch.ShadowClassLoader", bytes, new Integer(0), new Integer(len)});
if (bootstrap) return c;
shadowClassLoaderClass = c;
}
original.getClass().getField("lombok$shadowLoaderClass").set(null, shadowClassLoaderClass);
} finally {
if (in != null) in.close();
jf.close();
}
}
Class[] paramTypes = new Class[5];
```

```
paramTypes[0] = classLoaderClass;
paramTypes[1] = "".getClass();
paramTypes[2] = paramTypes[1];
paramTypes[3] = Class.forName("java.util.List");
paramTypes[4] = paramTypes[3];
Constructor constructor = shadowClassLoaderClass.getDeclaredConstructor(paramTypes);
constructor.setAccessible(true);
shadowLoader = (ClassLoader) constructor.newInstance(new Object[] {original, "lombok", jarLoc,
Arrays.asList(new Object[] {"lombok."}), Arrays.asList(new Object[] {"lombok.patcher.Symbols"})});
shadowLoaderField.set(original, shadowLoader);
}
}
}
if (resolve) {
Class[] paramTypes = new Class[2];
paramTypes[0] = "".getClass();
paramTypes[1] = Boolean.TYPE;
Method m = shadowLoader.getClass().getDeclaredMethod("loadClass", new Class[] {String.class,
boolean.class});
m.setAccessible(true);
return (Class) m.invoke(shadowLoader, new Object[] {name, Boolean.TRUE});
} else {
return shadowLoader.loadClass(name);
}
} catch (Exception ex) {
Throwable t = ex;
if (t instanceof InvocationTargetException) t = t.getCause();
if (t instanceof RuntimeException) throw (RuntimeException) t;
if (t instanceof Error) throw (Error) t;
throw new RuntimeException(t);
}
}
}
```

lombok/eclipse/agent/EclipsePatcher.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

```
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import static lombok.patcher.scripts.ScriptBuilder.*;
import static lombok.eclipse.agent.EclipseLoaderPatcher.OSGI_TYPES;
import java.io.File;
import java.lang.instrument.Instrumentation;
import java.net.URLClassLoader;
import java.security.ProtectionDomain;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import lombok.core.AgentLauncher;
import lombok.patcher.Filter;
import lombok.patcher.Hook;
import lombok.patcher.MethodTarget;
import lombok.patcher.ScriptManager;
import lombok.patcher.StackRequest;
import lombok.patcher.TargetMatcher;
import lombok.patcher.TransplantMapper;
import lombok.patcher.scripts.ScriptBuilder;
/**
* This is a java-agent that patches some of eclipse's classes so AST Nodes are handed off to Lombok
* for modification before Eclipse actually uses them to compile, render errors, show code outlines,
* create auto-completion dialogs, and anything else eclipse does with java code. See the *Transformer
* classes in this package for more information about which classes are transformed and how they are
* transformed.
*/
public class EclipsePatcher implements AgentLauncher.AgentLaunchable {
@Override public void runAgent(String agentArgs, Instrumentation instrumentation, boolean injected,
Class<?> launchingContext) throws Exception {
registerPatchScripts(instrumentation, injected, launchingContext);
}
private static void registerPatchScripts(Instrumentation instrumentation, boolean
reloadExistingClasses, Class<?> launchingContext) {
ScriptManager sm = new ScriptManager();
sm.registerTransformer(instrumentation);
sm.setFilter(new Filter() {
@Override public boolean shouldTransform(ClassLoader loader, String className, Class<?>
classBeingDefined, ProtectionDomain protectionDomain, byte[] classfileBuffer) {
if (loader != null) {
if (loader.getClass().getName().startsWith("org.sonar.classloader.")) return false; // Relevant to bug
#2351
if (loader.toString().contains("com.alexnederlof:jasperreports-plugin")) return false; //Relevant to bug
#1036
if (loader.toString().contains("com.pro-crafting.tools:jasperreports-plugin")) return false; //Relevant to
bug #1036
}
if (!(loader instanceof URLClassLoader)) return true;
```

```
ClassLoader parent = loader.getParent();
if (parent == null) return true;
return !parent.getClass().getName().startsWith("org.eclipse.jdt.apt.core.internal.AnnotationProcessorFa
ctoryLoader");
}
});
final boolean forceBaseResourceNames = shouldForceBaseResourceNames();
sm.setTransplantMapper(new TransplantMapper() {
public String mapResourceName(int classFileFormatVersion, String resourceName) {
if (classFileFormatVersion < 50 || forceBaseResourceNames) return resourceName;
return "Class50/" + resourceName;
}
});
EclipseLoaderPatcher.patchEquinoxLoaders(sm, launchingContext);
patchCatchReparse(sm);
patchIdentifierEndReparse(sm);
patchRetrieveEllipsisStartPosition(sm);
patchRetrieveRightBraceOrSemiColonPosition(sm);
patchRetrieveProperRightBracketPosition(sm);
patchRetrieveStartBlockPosition(sm);
patchSetGeneratedFlag(sm);
patchDomAstReparseIssues(sm);
patchHideGeneratedNodes(sm);
patchPostCompileHookEclipse(sm);
patchFixSourceTypeConverter(sm);
patchListRewriteHandleGeneratedMethods(sm);
patchSyntaxAndOccurrencesHighlighting(sm);
patchSortMembersOperation(sm);
patchExtractInterfaceAndPullUp(sm);
patchAboutDialog(sm);
patchEclipseDebugPatches(sm);
patchJavadoc(sm);
patchASTConverterLiterals(sm);
patchASTNodeSearchUtil(sm);
patchFieldInitializer(sm);
patchPostCompileHookEcj(sm);
patchAvoidReparsingGeneratedCode(sm);
patchLombokizeAST(sm);
patchEcjTransformers(sm);
patchExtensionMethod(sm);
patchRenameField(sm);
patchInline(sm);
patchNullCheck(sm);
patchCrossModuleClassLoading(sm);
patchForTests(sm);
if (reloadExistingClasses) sm.reloadClasses(instrumentation);
}
private static boolean shouldForceBaseResourceNames() {
String shadowOverride = System.getProperty("shadow.override.lombok", "");
if (shadowOverride == null || shadowOverride.length() == 0) return false;
for (String part : shadowOverride.split("\\s*" + (File.pathSeparatorChar == ';' ? ";" : ":") + "\\s*")) {
if (part.equalsIgnoreCase("lombok.jar")) return false;
}
```

```
return true;
}
private static void patchRenameField(ScriptManager sm) {
/* RefactoringSearchEngine.search will not return results when renaming field and Data Annotation is
present. Return a fake Element to make checks pass */
sm.addScript(ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.rename.RenameFieldProcessor",
"checkAccessorDeclarations", "org.eclipse.ltk.core.refactoring.RefactoringStatus",
"org.eclipse.core.runtime.IProgressMonitor", "org.eclipse.jdt.core.IMethod"))
.methodToWrap(new Hook("org.eclipse.jdt.internal.corext.refactoring.RefactoringSearchEngine",
"search", "org.eclipse.jdt.internal.corext.refactoring.SearchResultGroup[]", "org.eclipse.jdt.core.search.
SearchPattern","org.eclipse.jdt.core.search.IJavaSearchScope","org.eclipse.core.runtime.IProgressMo
nitor","org.eclipse.ltk.core.refactoring.RefactoringStatus"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "createFakeSearchResult",
"org.eclipse.jdt.internal.corext.refactoring.SearchResultGroup[]",
"org.eclipse.jdt.internal.corext.refactoring.SearchResultGroup[]", "java.lang.Object"))
.requestExtra(StackRequest.THIS)
.transplant()
.build());
/* Prevent adding generated nodes as possible search matches */
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.core.search.matching.MatchingNodeSet",
"addPossibleMatch", "void", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "isGenerated", "boolean",
"org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.request(StackRequest.PARAM1)
.transplant()
.build());
}
private static void patchExtractInterfaceAndPullUp(ScriptManager sm) {
/* Fix sourceEnding for generated nodes to avoid null pointer */
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.SourceElementNotifier",
"notifySourceElementRequestor", "void",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration",
"org.eclipse.jdt.internal.compiler.ast.TypeDeclaration",
"org.eclipse.jdt.internal.compiler.ast.ImportReference"))
.methodToWrap(new Hook("org.eclipse.jdt.internal.compiler.util.HashtableOfObjectToInt", "get", "int",
"java.lang.Object"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "getSourceEndFixed", "int",
"int", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.requestExtra(StackRequest.PARAM1)
.transplant().build());
/* Make sure the generated source element is found instead of the annotation */
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new
MethodTarget("org.eclipse.jdt.internal.corext.refactoring.structure.ExtractInterfaceProcessor",
"createMethodDeclaration", "void",
"org.eclipse.jdt.internal.corext.refactoring.structure.CompilationUnitRewrite",
"org.eclipse.jdt.core.dom.rewrite.ASTRewrite",
"org.eclipse.jdt.core.dom.AbstractTypeDeclaration",
"org.eclipse.jdt.core.dom.MethodDeclaration"
))
```

```
.methodToWrap(new Hook("org.eclipse.jface.text.IDocument", "get", "java.lang.String", "int", "int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"getRealMethodDeclarationSource", "java.lang.String", "java.lang.String", "java.lang.Object",
"org.eclipse.jdt.core.dom.MethodDeclaration"))
.requestExtra(StackRequest.THIS, StackRequest.PARAM4)
.transplant().build());
/* Get real node source instead of the annotation */
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.structure.HierarchyProcessor",
"createPlaceholderForSingleVariableDeclaration",
"org.eclipse.jdt.core.dom.SingleVariableDeclaration",
"org.eclipse.jdt.core.dom.SingleVariableDeclaration",
"org.eclipse.jdt.core.ICompilationUnit",
"org.eclipse.jdt.core.dom.rewrite.ASTRewrite"
))
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.structure.HierarchyProcessor",
"createPlaceholderForType", "org.eclipse.jdt.core.dom.Type",
"org.eclipse.jdt.core.dom.Type",
"org.eclipse.jdt.core.ICompilationUnit",
"org.eclipse.jdt.core.dom.rewrite.ASTRewrite"
))
.methodToWrap(new Hook("org.eclipse.jdt.core.IBuffer", "getText", "java.lang.String", "int", "int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "getRealNodeSource",
"java.lang.String", "java.lang.String", "org.eclipse.jdt.core.dom.ASTNode"))
.requestExtra(StackRequest.PARAM1)
.transplant()
.build());
/* Do not add @Override's for generated methods */
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.core.dom.rewrite.ListRewrite", "insertFirst"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"isListRewriteOnGeneratedNode", "boolean", "org.eclipse.jdt.core.dom.rewrite.ListRewrite"))
.request(StackRequest.THIS)
.transplant().build());
/* Do not add comments for generated methods */
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new
MethodTarget("org.eclipse.jdt.internal.corext.refactoring.structure.ExtractInterfaceProcessor",
"createMethodComment"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "isGenerated", "boolean",
"org.eclipse.jdt.core.dom.ASTNode"))
.request(StackRequest.PARAM2)
.transplant().build());
/* Do not add a modifier to the generating annotation during pull up
*
* Example: Pull up a protected method (canEqual()/@EqualsAndHashCode)
*/
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.structure.MemberVisibilityAdjustor
$IncomingMemberVisibilityAdjustment", "rewriteVisibility"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "skipRewriteVisibility",
"boolean", "org.eclipse.jdt.internal.corext.refactoring.structure.MemberVisibilityAdjustor$IncomingMem
berVisibilityAdjustment"))
```

```
.request(StackRequest.THIS)
.transplant()
.build());
/*
* ImportRemover sometimes removes lombok imports if a generated method/type gets changed.
Skipping all generated nodes fixes this behavior.
*
* Example: Create a class (Use.java) that uses a generated method (Test t; t.toString();) and pull up
this generated method.
*/
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.structure.ImportRemover",
"registerRemovedNode", "void", "org.eclipse.jdt.core.dom.ASTNode"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "isGenerated", "boolean",
"org.eclipse.jdt.core.dom.ASTNode"))
.request(StackRequest.PARAM1)
.transplant()
.build());
}
private static void patchInline(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.code.SourceProvider",
"getCodeBlocks", "java.lang.String[]", "org.eclipse.jdt.internal.corext.refactoring.code.CallContext",
"org.eclipse.jdt.core.dom.rewrite.ImportRewrite"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "getRealCodeBlocks",
"java.lang.String[]", "java.lang.String[]",
"org.eclipse.jdt.internal.corext.refactoring.code.SourceProvider",
"org.eclipse.jdt.internal.corext.refactoring.code.CallContext"))
.request(StackRequest.RETURN_VALUE, StackRequest.THIS, StackRequest.PARAM1)
.transplant()
.build());
}
private static void patchAboutDialog(ScriptManager sm) {
/*
* Add a line about lombok (+ version info) to eclipse's about dialog.
* This is doable without patching, but we intentionally patch it so that presence of the lombok info
* in the about dialog can be used to ascertain that patching in general is doing something.
*/
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.core.internal.runtime.Product", "getProperty", "java.lang.String",
"java.lang.String"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$LombokDeps",
"addLombokNotesToEclipseAboutDialog", "java.lang.String", "java.lang.String", "java.lang.String"))
.request(StackRequest.RETURN_VALUE, StackRequest.PARAM1)
.transplant().build());
}
private static void patchSyntaxAndOccurrencesHighlighting(ScriptManager sm) {
/*
* Skip generated nodes for "visual effects" (syntax highlighting && highlight occurrences)
*/
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.ui.search.OccurrencesFinder", "addUsage"))
.target(new MethodTarget("org.eclipse.jdt.internal.ui.search.OccurrencesFinder", "addWrite"))
```

```
.target(new
MethodTarget("org.eclipse.jdt.internal.ui.javaeditor.SemanticHighlightingReconciler$PositionCollector",
"visit", "boolean", "org.eclipse.jdt.core.dom.SimpleName"))
.target(new
MethodTarget("org.eclipse.jdt.internal.ui.javaeditor.SemanticHighlightingReconciler$PositionCollector",
"visitLiteral", "boolean", "org.eclipse.jdt.core.dom.Expression"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "isGenerated", "boolean",
"org.eclipse.jdt.core.dom.ASTNode"))
.valueMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "returnFalse", "boolean",
"java.lang.Object"))
.request(StackRequest.PARAM1)
.build());
}
private static void patchListRewriteHandleGeneratedMethods(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.replaceMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.dom.rewrite.ASTRewriteAnalyzer$ListRewriter",
"rewriteList"))
.methodToReplace(new Hook("org.eclipse.jdt.internal.core.dom.rewrite.RewriteEvent", "getChildren",
"org.eclipse.jdt.internal.core.dom.rewrite.RewriteEvent[]"))
.replacementMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"listRewriteHandleGeneratedMethods", "org.eclipse.jdt.internal.core.dom.rewrite.RewriteEvent[]",
"org.eclipse.jdt.internal.core.dom.rewrite.RewriteEvent"))
.build());
}
private static void patchSortMembersOperation(ScriptManager sm) {
/* Fixes "sort members" action with @Data @Log
* I would have liked to patch sortMembers, but kept getting a VerifyError: Illegal type in constant pool
* So now I just patch all calling methods
*/
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.SortElementsOperation$2", "visit", "boolean",
"org.eclipse.jdt.core.dom.CompilationUnit"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.CompilationUnit", "types", "java.util.List"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedNodes",
"java.util.List", "java.util.List"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.SortElementsOperation$2", "visit", "boolean",
"org.eclipse.jdt.core.dom.AnnotationTypeDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.AnnotationTypeDeclaration", "bodyDeclarations",
"java.util.List"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedNodes",
"java.util.List", "java.util.List"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.SortElementsOperation$2", "visit", "boolean",
"org.eclipse.jdt.core.dom.AnonymousClassDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.AnonymousClassDeclaration",
"bodyDeclarations", "java.util.List"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedNodes",
"java.util.List", "java.util.List"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
```

```
.target(new MethodTarget("org.eclipse.jdt.internal.core.SortElementsOperation$2", "visit", "boolean",
"org.eclipse.jdt.core.dom.TypeDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.TypeDeclaration", "bodyDeclarations",
"java.util.List"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedNodes",
"java.util.List", "java.util.List"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.SortElementsOperation$2", "visit", "boolean",
"org.eclipse.jdt.core.dom.EnumDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.EnumDeclaration", "bodyDeclarations",
"java.util.List"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedNodes",
"java.util.List", "java.util.List"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.SortElementsOperation$2", "visit", "boolean",
"org.eclipse.jdt.core.dom.EnumDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.EnumDeclaration", "enumConstants",
"java.util.List"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedNodes",
"java.util.List", "java.util.List"))
.transplant().build());
}
private static void patchDomAstReparseIssues(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.replaceMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.dom.rewrite.ASTRewriteAnalyzer", "visit"))
.methodToReplace(new Hook("org.eclipse.jdt.internal.core.dom.rewrite.TokenScanner",
"getTokenEndOffset", "int", "int", "int"))
.replacementMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"getTokenEndOffsetFixed", "int", "org.eclipse.jdt.internal.core.dom.rewrite.TokenScanner", "int", "int",
"java.lang.Object"))
.requestExtra(StackRequest.PARAM1)
.transplant()
.build());
}
private static void patchPostCompileHookEclipse(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.builder.IncrementalImageBuilder",
"writeClassFileContents"))
.target(new MethodTarget("org.eclipse.jdt.internal.core.builder.AbstractImageBuilder",
"writeClassFileContents"))
.methodToWrap(new Hook("org.eclipse.jdt.internal.compiler.ClassFile", "getBytes", "byte[]"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$LombokDeps", "runPostCompiler", "byte[]",
"byte[]", "java.lang.String"))
.requestExtra(StackRequest.PARAM3)
.build());
}
private static void patchPostCompileHookEcj(ScriptManager sm) {
sm.addScriptIfNotWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.tool.EclipseCompilerImpl",
"outputClassFiles"))
```

```
.methodToWrap(new Hook("javax.tools.JavaFileObject", "openOutputStream",
"java.io.OutputStream"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$LombokDeps", "runPostCompiler",
"java.io.OutputStream", "java.io.OutputStream"))
.transplant().build());
sm.addScriptIfNotWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.util.Util", "writeToDisk"))
.methodToWrap(new Hook("java.io.BufferedOutputStream", "<init>", "void", "java.io.OutputStream",
"int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$LombokDeps", "runPostCompiler",
"java.io.BufferedOutputStream", "java.io.BufferedOutputStream", "java.lang.String", "java.lang.String"))
.requestExtra(StackRequest.PARAM2, StackRequest.PARAM3)
.transplant().build());
}
private static void patchHideGeneratedNodes(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.dom.LinkedNodeFinder", "findByNode",
"org.eclipse.jdt.core.dom.SimpleName[]", "org.eclipse.jdt.core.dom.ASTNode",
"org.eclipse.jdt.core.dom.SimpleName"))
.target(new MethodTarget("org.eclipse.jdt.internal.corext.dom.LinkedNodeFinder", "findByBinding"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"removeGeneratedSimpleNames", "org.eclipse.jdt.core.dom.SimpleName[]",
"org.eclipse.jdt.core.dom.SimpleName[]"))
.request(StackRequest.RETURN_VALUE).build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.dom.LinkedNodeFinder", "findByNode",
"org.eclipse.jdt.core.dom.Name[]", "org.eclipse.jdt.core.dom.ASTNode",
"org.eclipse.jdt.core.dom.Name"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedNames",
"org.eclipse.jdt.core.dom.Name[]",
"org.eclipse.jdt.core.dom.Name[]"))
.request(StackRequest.RETURN_VALUE).build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTNode", "accept", "void",
"org.eclipse.jdt.core.dom.ASTVisitor"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"isBlockedVisitorAndGenerated", "boolean", "org.eclipse.jdt.core.dom.ASTNode",
"org.eclipse.jdt.core.dom.ASTVisitor"))
.request(StackRequest.THIS, StackRequest.PARAM1)
.build());
patchRefactorScripts(sm);
patchFormatters(sm);
}
private static void patchFormatters(ScriptManager sm) {
// before Eclipse Mars
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.setSymbolDuringMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.formatter.DefaultCodeFormatter",
"formatCompilationUnit"))
.callToWrap(new Hook("org.eclipse.jdt.internal.core.util.CodeSnippetParsingUtil",
"parseCompilationUnit", "org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration", "char[]",
"java.util.Map", "boolean"))
.symbol("lombok.disable")
.build());
```

```
// Eclipse Mars and beyond
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.setSymbolDuringMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.formatter.DefaultCodeFormatter",
"parseSourceCode"))
.callToWrap(new Hook("org.eclipse.jdt.core.dom.ASTParser", "createAST",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.core.runtime.IProgressMonitor"))
.symbol("lombok.disable")
.build());
}
private static void patchRefactorScripts(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.core.dom.rewrite.ASTRewrite", "replace"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.rewrite.ASTRewrite", "remove"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"skipRewritingGeneratedNodes", "boolean",
"org.eclipse.jdt.core.dom.ASTNode"))
.transplant().request(StackRequest.PARAM1).build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.rename.RenameTypeProcessor",
"addConstructorRenames"))
.methodToWrap(new Hook("org.eclipse.jdt.core.IType", "getMethods", "org.eclipse.jdt.core.IMethod[]"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "removeGeneratedMethods",
"org.eclipse.jdt.core.IMethod[]",
"org.eclipse.jdt.core.IMethod[]"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new
MethodTarget("org.eclipse.jdt.internal.corext.refactoring.rename.TempOccurrenceAnalyzer", "visit",
"boolean", "org.eclipse.jdt.core.dom.SimpleName"))
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.rename.RenameAnalyzeUtil$Probl
emNodeFinder$NameNodeVisitor", "visit", "boolean", "org.eclipse.jdt.core.dom.SimpleName"))
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.rename.RenameTypeParameterPr
ocessor$RenameTypeParameterVisitor", "visit", "boolean", "org.eclipse.jdt.core.dom.SimpleName"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "isGenerated", "boolean",
"org.eclipse.jdt.core.dom.ASTNode"))
.valueMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "returnTrue", "boolean",
"java.lang.Object"))
.request(StackRequest.PARAM1)
.transplant().build());
}
private static void patchCatchReparse(ScriptManager sm) {
sm.addScript(ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "retrieveStartingCatchPosition"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveStartingCatchPosition", "int", "int", "int"))
.transplant().request(StackRequest.RETURN_VALUE, StackRequest.PARAM1).build());
}
private static void patchIdentifierEndReparse(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "retrieveIdentifierEndPosition"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveIdentifierEndPosition", "int", "int", "int", "int"))
```

```
.transplant().request(StackRequest.RETURN_VALUE, StackRequest.PARAM1,
StackRequest.PARAM2).build());
}
private static void patchRetrieveEllipsisStartPosition(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "retrieveEllipsisStartPosition"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveEllipsisStartPosition", "int", "int", "int"))
.transplant().request(StackRequest.RETURN_VALUE, StackRequest.PARAM2).build());
}
private static void patchRetrieveStartBlockPosition(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "retrieveStartBlockPosition"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "fixRetrieveStartBlockPosition",
"int", "int", "int"))
.transplant().request(StackRequest.RETURN_VALUE, StackRequest.PARAM2).build());
}
private static void patchRetrieveRightBraceOrSemiColonPosition(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.ASTNode", "boolean",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.ASTConverter",
"retrieveRightBraceOrSemiColonPosition", "int", "int", "int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveRightBraceOrSemiColonPosition", "int", "int",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration"))
.requestExtra(StackRequest.PARAM2)
.transplant()
.build());
sm.addScript(ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.ASTNode", "boolean",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.ASTConverter", "retrieveRightBrace", "int", "int",
"int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveRightBraceOrSemiColonPosition", "int", "int",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration"))
.requestExtra(StackRequest.PARAM2)
.transplant()
.build());
sm.addScript(ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.FieldDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.ASTConverter", "retrieveRightBrace", "int", "int",
"int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveRightBraceOrSemiColonPosition", "int", "int",
"org.eclipse.jdt.internal.compiler.ast.FieldDeclaration"))
.requestExtra(StackRequest.PARAM1)
.transplant()
.build());
```

```
// sm.addScript(ScriptBuilder.wrapReturnValue()
// .target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"retrieveRightBraceOrSemiColonPosition"))
// .target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "retrieveRightBrace"))
// .wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveRightBraceOrSemiColonPosition", "int", "int", "int"))
// .transplant().request(StackRequest.RETURN_VALUE, StackRequest.PARAM2).build());
}
private static void patchRetrieveProperRightBracketPosition(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "extractSubArrayType",
"org.eclipse.jdt.core.dom.ArrayType", "org.eclipse.jdt.core.dom.ArrayType", "int", "int"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.ASTConverter",
"retrieveProperRightBracketPosition", "int", "int", "int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveProperRightBracketPosition", "int", "int", "org.eclipse.jdt.core.dom.Type"))
.requestExtra(StackRequest.PARAM1)
.transplant()
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertToArray",
"org.eclipse.jdt.core.dom.ArrayType", "org.eclipse.jdt.core.dom.Type", "int", "int", "int",
"org.eclipse.jdt.internal.compiler.ast.Annotation[][]"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.ASTConverter",
"retrieveProperRightBracketPosition", "int", "int", "int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"fixRetrieveProperRightBracketPosition", "int", "int", "org.eclipse.jdt.core.dom.Type"))
.requestExtra(StackRequest.PARAM1)
.transplant()
.build());
}
private static void patchSetGeneratedFlag(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
.targetClass("org.eclipse.jdt.internal.compiler.ast.ASTNode")
.fieldName("$generatedBy")
.fieldType("Lorg/eclipse/jdt/internal/compiler/ast/ASTNode;")
.setPublic().setTransient().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
.targetClass("org.eclipse.jdt.core.dom.ASTNode")
.fieldName("$isGenerated").fieldType("Z")
.setPublic().setTransient().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new TargetMatcher() {
@Override public boolean matches(String classSpec, String methodName, String descriptor) {
if (!"convert".equals(methodName)) return false;
List<String> fullDesc = MethodTarget.decomposeFullDesc(descriptor);
if ("V".equals(fullDesc.get(0))) return false;
if (fullDesc.size() < 2) return false;
if (!fullDesc.get(1).startsWith("Lorg/eclipse/jdt/internal/compiler/ast/")) return false;
return true;
}
@Override public String describe() {
return "ASTConverter:[all relevant]";
```

```
}
@Override public Collection<String> getAffectedClasses() {
return Collections.singleton("org.eclipse.jdt.core.dom.ASTConverter");
}
}).request(StackRequest.PARAM1, StackRequest.RETURN_VALUE)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.ASTNode", "boolean",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration"))
.request(StackRequest.PARAM2, StackRequest.RETURN_VALUE)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertToFieldDeclaration",
"org.eclipse.jdt.core.dom.FieldDeclaration", "org.eclipse.jdt.internal.compiler.ast.FieldDeclaration"))
/* Targets beneath are only patched because the resulting dom nodes should be marked if generated.
* However I couldn't find a usecase where these were actually used
*/
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertToType",
"org.eclipse.jdt.core.dom.Type", "org.eclipse.jdt.internal.compiler.ast.NameReference"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertType",
"org.eclipse.jdt.core.dom.Type", "org.eclipse.jdt.internal.compiler.ast.TypeReference"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"convertToVariableDeclarationExpression", "org.eclipse.jdt.core.dom.VariableDeclarationExpression",
"org.eclipse.jdt.internal.compiler.ast.LocalDeclaration"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"convertToSingleVariableDeclaration", "org.eclipse.jdt.core.dom.SingleVariableDeclaration",
"org.eclipse.jdt.internal.compiler.ast.LocalDeclaration"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"convertToVariableDeclarationFragment", "org.eclipse.jdt.core.dom.VariableDeclarationFragment",
"org.eclipse.jdt.internal.compiler.ast.FieldDeclaration"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"convertToVariableDeclarationFragment", "org.eclipse.jdt.core.dom.VariableDeclarationFragment",
"org.eclipse.jdt.internal.compiler.ast.LocalDeclaration"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"convertToVariableDeclarationStatement", "org.eclipse.jdt.core.dom.VariableDeclarationStatement",
"org.eclipse.jdt.internal.compiler.ast.LocalDeclaration"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "createBaseType",
"org.eclipse.jdt.core.dom.Type", "org.eclipse.jdt.internal.compiler.ast.TypeReference", "long[]",
"org.eclipse.jdt.internal.compiler.ast.Annotation[][]", "char[][]", "int", "int", "boolean"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "createQualifiedType",
"org.eclipse.jdt.core.dom.QualifiedType", "org.eclipse.jdt.internal.compiler.ast.TypeReference", "long[]",
"org.eclipse.jdt.internal.compiler.ast.Annotation[][]", "char[][]", "int", "org.eclipse.jdt.core.dom.Type"))
/* Targets above are only patched because the resulting dom nodes should be marked if generated. */
.request(StackRequest.PARAM1, StackRequest.RETURN_VALUE)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant().build());
/* Set generated flag for SimpleNames */
```

```
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new TargetMatcher() {
@Override public boolean matches(String classSpec, String methodName, String descriptor) {
if (!methodName.startsWith("convert")) return false;
List<String> fullDesc = MethodTarget.decomposeFullDesc(descriptor);
if (fullDesc.size() < 2) return false;
if (!fullDesc.get(1).startsWith("Lorg/eclipse/jdt/internal/compiler/ast/")) return false;
return true;
}
@Override public String describe() {
return "ASTConverter::(all relevant)";
}
@Override public Collection<String> getAffectedClasses() {
return Collections.singleton("org.eclipse.jdt.core.dom.ASTConverter");
}
}).methodToWrap(new Hook("org.eclipse.jdt.core.dom.SimpleName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.ASTNode", "boolean",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.SimpleName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM2)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.ASTNode", "boolean",
"org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.Block", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM2)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertType",
"org.eclipse.jdt.core.dom.Type", "org.eclipse.jdt.internal.compiler.ast.TypeReference"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.PrimitiveType", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant()
.build());
```

```
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertType",
"org.eclipse.jdt.core.dom.Type", "org.eclipse.jdt.internal.compiler.ast.TypeReference"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.SimpleType", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant()
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertType",
"org.eclipse.jdt.core.dom.Type", "org.eclipse.jdt.internal.compiler.ast.TypeReference"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.ParameterizedType", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant()
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convertType",
"org.eclipse.jdt.core.dom.Type", "org.eclipse.jdt.internal.compiler.ast.TypeReference"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.QualifiedType", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "setIsGeneratedFlag", "void",
"org.eclipse.jdt.core.dom.ASTNode", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant()
.build());
/* Set generated flag for QualifiedNames */
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"setQualifiedNameNameAndSourceRanges", "org.eclipse.jdt.core.dom.QualifiedName", "char[][]",
"long[]", "int", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"setQualifiedNameNameAndSourceRanges", "org.eclipse.jdt.core.dom.QualifiedName", "char[][]",
"long[]", "int", "org.eclipse.jdt.internal.compiler.ast.TypeReference"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.SimpleName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM4)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"setQualifiedNameNameAndSourceRanges", "org.eclipse.jdt.core.dom.QualifiedName", "char[][]",
"long[]", "int", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"setQualifiedNameNameAndSourceRanges", "org.eclipse.jdt.core.dom.QualifiedName", "char[][]",
"long[]", "int", "org.eclipse.jdt.internal.compiler.ast.TypeReference"))
```

```
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.QualifiedName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM4)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"setQualifiedNameNameAndSourceRanges", "org.eclipse.jdt.core.dom.QualifiedName", "char[][]",
"long[]", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.SimpleName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM3)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter",
"setQualifiedNameNameAndSourceRanges", "org.eclipse.jdt.core.dom.QualifiedName", "char[][]",
"long[]", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.QualifiedName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM3)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "setTypeNameForAnnotation",
"void", "org.eclipse.jdt.internal.compiler.ast.Annotation", "org.eclipse.jdt.core.dom.Annotation"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.SimpleName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "setTypeNameForAnnotation",
"void", "org.eclipse.jdt.internal.compiler.ast.Annotation", "org.eclipse.jdt.core.dom.Annotation"))
.methodToWrap(new Hook("org.eclipse.jdt.core.dom.QualifiedName", "<init>", "void",
"org.eclipse.jdt.core.dom.AST"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"setIsGeneratedFlagForName", "void",
"org.eclipse.jdt.core.dom.Name", "java.lang.Object"))
.transplant().build());
}
private static void patchAvoidReparsingGeneratedCode(ScriptManager sm) {
final String PARSER_SIG = "org.eclipse.jdt.internal.compiler.parser.Parser";
sm.addScript(ScriptBuilder.exitEarly()
```

```
.target(new MethodTarget(PARSER_SIG, "parse", "void",
"org.eclipse.jdt.internal.compiler.ast.MethodDeclaration",
"org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "checkBit24", "boolean",
"java.lang.Object"))
.transplant()
.request(StackRequest.PARAM1).build());
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget(PARSER_SIG, "parse", "void",
"org.eclipse.jdt.internal.compiler.ast.ConstructorDeclaration",
"org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration", "boolean"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "checkBit24", "boolean",
"java.lang.Object"))
.transplant()
.request(StackRequest.PARAM1).build());
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget(PARSER_SIG, "parse", "void",
"org.eclipse.jdt.internal.compiler.ast.Initializer",
"org.eclipse.jdt.internal.compiler.ast.TypeDeclaration",
"org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "checkBit24", "boolean",
"java.lang.Object"))
.transplant()
.request(StackRequest.PARAM1).build());
}
private static void patchLombokizeAST(ScriptManager sm) {
sm.addScript(ScriptBuilder.addField()
.targetClass("org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration")
.fieldName("$lombokAST").fieldType("Ljava/lang/Object;")
.setPublic().setTransient().build());
sm.addScript(ScriptBuilder.addField()
.targetClass("org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration")
.fieldName("$transformationState").fieldType("Ljava/lang/Object;")
.setPublic().setTransient().build());
final String PARSER_SIG = "org.eclipse.jdt.internal.compiler.parser.Parser";
final String CUD_SIG = "org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration";
final String OBJECT_SIG = "java.lang.Object";
sm.addScript(ScriptBuilder.wrapReturnValue()
.target(new MethodTarget(PARSER_SIG, "getMethodBodies", "void", CUD_SIG))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Transform", "transform", "void",
OBJECT_SIG, OBJECT_SIG))
.request(StackRequest.THIS, StackRequest.PARAM1).build());
sm.addScript(ScriptBuilder.wrapReturnValue()
.target(new MethodTarget(PARSER_SIG, "endParse", CUD_SIG, "int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Transform", "transform_swapped", "void",
OBJECT_SIG, OBJECT_SIG))
.request(StackRequest.THIS, StackRequest.RETURN_VALUE).build());
}
private static void patchEcjTransformers(ScriptManager sm) {
addPatchesForDelegate(sm);
addPatchesForVal(sm);
addPatchesForValEclipse(sm);
}
```

```
private static void addPatchesForDelegate(ScriptManager sm) {
final String CLASSSCOPE_SIG = "org.eclipse.jdt.internal.compiler.lookup.ClassScope";
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget(CLASSSCOPE_SIG, "buildFieldsAndMethods", "void"))
.request(StackRequest.THIS)
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$Delegate", "handleDelegateForType",
"boolean", "java.lang.Object"))
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.setSymbolDuringMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.SelectionRequestor", "acceptSourceMethod"))
.callToWrap(new Hook("org.eclipse.jdt.core.IType", "getMethods", "org.eclipse.jdt.core.IMethod[]"))
.symbol("lombok.skipdelegates")
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
.fieldName("$delegateMethods")
.fieldType("Ljava/util/Map;")
.setPublic()
.setTransient()
.targetClass("org.eclipse.jdt.internal.core.CompilationUnit")
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.internal.core.SourceTypeElementInfo", "getChildren",
"org.eclipse.jdt.core.IJavaElement[]"))
.request(StackRequest.RETURN_VALUE, StackRequest.THIS)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Delegate",
"addGeneratedDelegateMethods", "java.lang.Object[]", "java.lang.Object", "java.lang.Object"))
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.core.JavaElement", "getElementInfo",
"org.eclipse.jdt.internal.compiler.env.IElementInfo"))
.request(StackRequest.THIS)
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$Delegate", "isDelegateSourceMethod",
"boolean", "java.lang.Object"))
.valueMethod(new Hook("lombok.launch.PatchFixesHider$Delegate", "returnElementInfo",
"java.lang.Object", "java.lang.Object"))
.build());
}
private static void addPatchesForValEclipse(ScriptManager sm) {
final String LOCALDECLARATION_SIG = "org.eclipse.jdt.internal.compiler.ast.LocalDeclaration";
final String PARSER_SIG = "org.eclipse.jdt.internal.compiler.parser.Parser";
final String VARIABLEDECLARATIONSTATEMENT_SIG =
"org.eclipse.jdt.core.dom.VariableDeclarationStatement";
final String SINGLEVARIABLEDECLARATION_SIG =
"org.eclipse.jdt.core.dom.SingleVariableDeclaration";
final String ASTCONVERTER_SIG = "org.eclipse.jdt.core.dom.ASTConverter";
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
.fieldName("$initCopy")
.fieldType("Lorg/eclipse/jdt/internal/compiler/ast/ASTNode;")
.setPublic()
.setTransient()
.targetClass("org.eclipse.jdt.internal.compiler.ast.LocalDeclaration")
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
```

```
.fieldName("$iterableCopy")
.fieldType("Lorg/eclipse/jdt/internal/compiler/ast/ASTNode;")
.setPublic()
.setTransient()
.targetClass("org.eclipse.jdt.internal.compiler.ast.LocalDeclaration")
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget(PARSER_SIG, "consumeExitVariableWithInitialization", "void"))
.request(StackRequest.THIS)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$ValPortal",
"copyInitializationOfLocalDeclaration", "void", "java.lang.Object"))
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget(PARSER_SIG, "consumeEnhancedForStatementHeader", "void"))
.request(StackRequest.THIS)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$ValPortal",
"copyInitializationOfForEachIterable", "void", "java.lang.Object"))
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget(ASTCONVERTER_SIG, "setModifiers", "void",
VARIABLEDECLARATIONSTATEMENT_SIG, LOCALDECLARATION_SIG))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$ValPortal",
"addFinalAndValAnnotationToVariableDeclarationStatement",
"void", "java.lang.Object", "java.lang.Object", "java.lang.Object"))
.request(StackRequest.THIS, StackRequest.PARAM1, StackRequest.PARAM2).build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget(ASTCONVERTER_SIG, "setModifiers", "void",
SINGLEVARIABLEDECLARATION_SIG, LOCALDECLARATION_SIG))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$ValPortal",
"addFinalAndValAnnotationToSingleVariableDeclaration",
"void", "java.lang.Object", "java.lang.Object", "java.lang.Object"))
.request(StackRequest.THIS, StackRequest.PARAM1, StackRequest.PARAM2).build());
}
private static void addPatchesForVal(ScriptManager sm) {
final String LOCALDECLARATION_SIG = "org.eclipse.jdt.internal.compiler.ast.LocalDeclaration";
final String FOREACHSTATEMENT_SIG = "org.eclipse.jdt.internal.compiler.ast.ForeachStatement";
final String EXPRESSION_SIG = "org.eclipse.jdt.internal.compiler.ast.Expression";
final String BLOCKSCOPE_SIG = "org.eclipse.jdt.internal.compiler.lookup.BlockScope";
final String TYPEBINDING_SIG = "org.eclipse.jdt.internal.compiler.lookup.TypeBinding";
final String OBJECT_SIG = "java.lang.Object";
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget(LOCALDECLARATION_SIG, "resolve", "void", BLOCKSCOPE_SIG))
.request(StackRequest.THIS, StackRequest.PARAM1)
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$Val", "handleValForLocalDeclaration",
"boolean", OBJECT_SIG, OBJECT_SIG))
.build());
sm.addScript(ScriptBuilder.replaceMethodCall()
.target(new MethodTarget(LOCALDECLARATION_SIG, "resolve", "void", BLOCKSCOPE_SIG))
.target(new MethodTarget(LOCALDECLARATION_SIG, "resolve", "void", BLOCKSCOPE_SIG,
"boolean"))
.methodToReplace(new Hook(EXPRESSION_SIG, "resolveType", TYPEBINDING_SIG,
BLOCKSCOPE_SIG))
.requestExtra(StackRequest.THIS)
```

```
.replacementMethod(new Hook("lombok.launch.PatchFixesHider$Val",
"skipResolveInitializerIfAlreadyCalled2", TYPEBINDING_SIG, EXPRESSION_SIG,
BLOCKSCOPE_SIG, LOCALDECLARATION_SIG))
.transplant()
.build());
sm.addScript(ScriptBuilder.replaceMethodCall()
.target(new MethodTarget(FOREACHSTATEMENT_SIG, "resolve", "void", BLOCKSCOPE_SIG))
.methodToReplace(new Hook(EXPRESSION_SIG, "resolveType", TYPEBINDING_SIG,
BLOCKSCOPE_SIG))
.replacementMethod(new Hook("lombok.launch.PatchFixesHider$Val",
"skipResolveInitializerIfAlreadyCalled", TYPEBINDING_SIG, EXPRESSION_SIG,
BLOCKSCOPE_SIG))
.transplant()
.build());
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget(FOREACHSTATEMENT_SIG, "resolve", "void", BLOCKSCOPE_SIG))
.request(StackRequest.THIS, StackRequest.PARAM1)
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$Val", "handleValForForEach", "boolean",
OBJECT_SIG, OBJECT_SIG))
.build());
}
private static void patchFixSourceTypeConverter(ScriptManager sm) {
final String SOURCE_TYPE_CONVERTER_SIG =
"org.eclipse.jdt.internal.compiler.parser.SourceTypeConverter";
final String I_ANNOTATABLE_SIG = "org.eclipse.jdt.core.IAnnotatable";
final String ANNOTATION_SIG = "org.eclipse.jdt.internal.compiler.ast.Annotation";
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget(SOURCE_TYPE_CONVERTER_SIG, "convertAnnotations",
ANNOTATION_SIG + "[]", I_ANNOTATABLE_SIG))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "convertAnnotations",
ANNOTATION_SIG + "[]", ANNOTATION_SIG + "[]", I_ANNOTATABLE_SIG))
.request(StackRequest.PARAM1, StackRequest.RETURN_VALUE).build());
}
private static void patchEclipseDebugPatches(ScriptManager sm) {
final String ASTNODE_SIG = "org.eclipse.jdt.core.dom.ASTNode";
final String PATCH_DEBUG = "lombok.eclipse.agent.PatchDiagnostics";
sm.addScriptIfWitness(OSGI_TYPES, exitEarly()
.target(new MethodTarget(ASTNODE_SIG, "setSourceRange", "void", "int", "int"))
.request(StackRequest.THIS)
.request(StackRequest.PARAM1)
.request(StackRequest.PARAM2)
.decisionMethod(new Hook(PATCH_DEBUG, "setSourceRangeCheck", "boolean", "java.lang.Object",
"int", "int"))
.build());
}
private static void patchExtensionMethod(ScriptManager sm) {
final String PATCH_EXTENSIONMETHOD = "lombok.launch.PatchFixesHider$ExtensionMethod";
final String PATCH_EXTENSIONMETHOD_COMPLETIONPROPOSAL_PORTAL =
"lombok.eclipse.agent.PatchExtensionMethodCompletionProposalPortal";
final String MESSAGE_SEND_SIG = "org.eclipse.jdt.internal.compiler.ast.MessageSend";
final String TYPE_BINDING_SIG = "org.eclipse.jdt.internal.compiler.lookup.TypeBinding";
final String SCOPE_SIG = "org.eclipse.jdt.internal.compiler.lookup.Scope";
final String BLOCK_SCOPE_SIG = "org.eclipse.jdt.internal.compiler.lookup.BlockScope";
```

```
final String TYPE_BINDINGS_SIG = "org.eclipse.jdt.internal.compiler.lookup.TypeBinding[]";
final String PROBLEM_REPORTER_SIG =
"org.eclipse.jdt.internal.compiler.problem.ProblemReporter";
final String METHOD_BINDING_SIG = "org.eclipse.jdt.internal.compiler.lookup.MethodBinding";
final String COMPLETION_PROPOSAL_COLLECTOR_SIG =
"org.eclipse.jdt.ui.text.java.CompletionProposalCollector";
final String I_JAVA_COMPLETION_PROPOSAL_SIG =
"org.eclipse.jdt.ui.text.java.IJavaCompletionProposal[]";
final String AST_NODE = "org.eclipse.jdt.internal.compiler.ast.ASTNode";
final String OBJECT_SIG = "java.lang.Object";
sm.addScript(wrapReturnValue()
.target(new MethodTarget(MESSAGE_SEND_SIG, "resolveType", TYPE_BINDING_SIG,
BLOCK_SCOPE_SIG))
.request(StackRequest.RETURN_VALUE)
.request(StackRequest.THIS)
.request(StackRequest.PARAM1)
.wrapMethod(new Hook(PATCH_EXTENSIONMETHOD, "resolveType", OBJECT_SIG, OBJECT_SIG,
OBJECT_SIG, OBJECT_SIG))
.cast()
.build());
sm.addScript(replaceMethodCall()
.target(new MethodTarget(MESSAGE_SEND_SIG, "resolveType", TYPE_BINDING_SIG,
BLOCK_SCOPE_SIG))
.methodToReplace(new Hook(PROBLEM_REPORTER_SIG, "errorNoMethodFor", "void",
MESSAGE_SEND_SIG, TYPE_BINDING_SIG, TYPE_BINDINGS_SIG))
.replacementMethod(new Hook(PATCH_EXTENSIONMETHOD, "errorNoMethodFor", "void",
OBJECT_SIG, OBJECT_SIG, OBJECT_SIG, OBJECT_SIG))
.build());
sm.addScript(replaceMethodCall()
.target(new MethodTarget(MESSAGE_SEND_SIG, "resolveType", TYPE_BINDING_SIG,
BLOCK_SCOPE_SIG))
.methodToReplace(new Hook(PROBLEM_REPORTER_SIG, "invalidMethod", "void",
MESSAGE_SEND_SIG, METHOD_BINDING_SIG))
.replacementMethod(new Hook(PATCH_EXTENSIONMETHOD, "invalidMethod", "void",
OBJECT_SIG, OBJECT_SIG, OBJECT_SIG))
.build());
// Since eclipse mars; they added a param.
sm.addScript(replaceMethodCall()
.target(new MethodTarget(MESSAGE_SEND_SIG, "resolveType", TYPE_BINDING_SIG,
BLOCK_SCOPE_SIG))
.methodToReplace(new Hook(PROBLEM_REPORTER_SIG, "invalidMethod", "void",
MESSAGE_SEND_SIG, METHOD_BINDING_SIG, SCOPE_SIG))
.replacementMethod(new Hook(PATCH_EXTENSIONMETHOD, "invalidMethod", "void",
OBJECT_SIG, OBJECT_SIG, OBJECT_SIG, OBJECT_SIG))
.build());
sm.addScript(replaceMethodCall()
.target(new MethodTarget(MESSAGE_SEND_SIG, "resolveType", TYPE_BINDING_SIG,
BLOCK_SCOPE_SIG))
.methodToReplace(new Hook(PROBLEM_REPORTER_SIG, "nonStaticAccessToStaticMethod",
"void", AST_NODE, METHOD_BINDING_SIG))
.replacementMethod(new Hook(PATCH_EXTENSIONMETHOD, "nonStaticAccessToStaticMethod",
"void", OBJECT_SIG, OBJECT_SIG, OBJECT_SIG, OBJECT_SIG))
.requestExtra(StackRequest.THIS)
```

```
.build());
sm.addScriptIfWitness(OSGI_TYPES, wrapReturnValue()
.target(new MethodTarget(COMPLETION_PROPOSAL_COLLECTOR_SIG,
"getJavaCompletionProposals", I_JAVA_COMPLETION_PROPOSAL_SIG))
.request(StackRequest.RETURN_VALUE)
.request(StackRequest.THIS)
.wrapMethod(new Hook(PATCH_EXTENSIONMETHOD_COMPLETIONPROPOSAL_PORTAL,
"getJavaCompletionProposals", I_JAVA_COMPLETION_PROPOSAL_SIG, "java.lang.Object[]",
"java.lang.Object"))
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.core.search.SearchPattern", "createPattern",
"org.eclipse.jdt.core.search.SearchPattern", "org.eclipse.jdt.core.IJavaElement", "int", "int"))
.wrapMethod(new Hook(PATCH_EXTENSIONMETHOD, "modifyMethodPattern", "java.lang.Object",
"java.lang.Object"))
.cast()
.request(StackRequest.RETURN_VALUE)
.build());
}
private static void patchNullCheck(ScriptManager sm) {
/* Avoid warnings caused by the null check generated for lombok.NonNull if NonNullByDefault is used.
*/
/* Avoid "Redundant null check: comparing '@NonNull String' against null" */
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.problem.ProblemReporter",
"expressionNonNullComparison", "boolean", "org.eclipse.jdt.internal.compiler.ast.Expression",
"boolean"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "isGenerated", "boolean",
"org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.valueMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "returnTrue", "boolean",
"java.lang.Object"))
.request(StackRequest.PARAM1)
.transplant().build());
/* Avoid "Dead code" */
sm.addScript(ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.problem.ProblemReporter",
"fakeReachable", "void", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "isGenerated", "boolean",
"org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.request(StackRequest.PARAM1)
.transplant().build());
}
private static void patchJavadoc(ScriptManager sm) {
// Moved to new package and changed to instance method in 2024-03
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.manipulation.internal.javadoc.CoreJavadocAccess",
"getHTMLContent", "java.lang.String", "org.eclipse.jdt.core.IJavaElement", "boolean"))
.methodToWrap(new Hook("org.eclipse.jdt.core.manipulation.internal.javadoc.CoreJavadocAccess",
"getHTMLContentFromSource", "java.lang.String", "org.eclipse.jdt.core.IJavaElement"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Javadoc", "getHTMLContentFromSource",
"java.lang.String", "java.lang.String", "java.lang.Object", "org.eclipse.jdt.core.IJavaElement"))
.requestExtra(StackRequest.THIS, StackRequest.PARAM1)
.build());
```

```
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.ui.text.javadoc.JavadocContentAccess2",
"getHTMLContent", "java.lang.String", "org.eclipse.jdt.core.IJavaElement", "boolean"))
.methodToWrap(new Hook("org.eclipse.jdt.internal.ui.text.javadoc.JavadocContentAccess2",
"getHTMLContentFromSource", "java.lang.String", "org.eclipse.jdt.core.IJavaElement"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Javadoc", "getHTMLContentFromSource",
"java.lang.String", "java.lang.String", "org.eclipse.jdt.core.IJavaElement"))
.requestExtra(StackRequest.PARAM1)
.build());
/* This is a copy for the language server implementation that also supports markdown */
sm.addScript(ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.ls.core.internal.javadoc.JavadocContentAccess2",
"getHTMLContent", "java.lang.String", "org.eclipse.jdt.core.IJavaElement", "boolean"))
.methodToWrap(new Hook("org.eclipse.jdt.ls.core.internal.javadoc.JavadocContentAccess2",
"getHTMLContentFromSource", "java.lang.String", "org.eclipse.jdt.core.IJavaElement"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Javadoc", "getHTMLContentFromSource",
"java.lang.String", "java.lang.String", "org.eclipse.jdt.core.IJavaElement"))
.requestExtra(StackRequest.PARAM1)
.build());
/* This is an older version that uses IMember instead of IJavaElement */
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.ui.text.javadoc.JavadocContentAccess2",
"getHTMLContent", "java.lang.String", "org.eclipse.jdt.core.IMember", "boolean"))
.methodToWrap(new Hook("org.eclipse.jdt.internal.ui.text.javadoc.JavadocContentAccess2",
"getHTMLContentFromSource", "java.lang.String", "org.eclipse.jdt.core.IMember"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Javadoc", "getHTMLContentFromSource",
"java.lang.String", "java.lang.String", "org.eclipse.jdt.core.IJavaElement"))
.requestExtra(StackRequest.PARAM1)
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
.fieldName("$javadoc")
.fieldType("Ljava/util/Map;")
.setPublic()
.setTransient()
.targetClass("org.eclipse.jdt.internal.core.CompilationUnit")
.build());
}
private static void patchASTConverterLiterals(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.Expression", "org.eclipse.jdt.internal.compiler.ast.StringLiteral"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.Expression", "org.eclipse.jdt.internal.compiler.ast.TextBlock"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.CharacterLiteral", "org.eclipse.jdt.internal.compiler.ast.CharLiteral"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.NumberLiteral", "org.eclipse.jdt.internal.compiler.ast.DoubleLiteral"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.NumberLiteral", "org.eclipse.jdt.internal.compiler.ast.FloatLiteral"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.NumberLiteral", "org.eclipse.jdt.internal.compiler.ast.LongLiteral"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.NumberLiteral", "org.eclipse.jdt.internal.compiler.ast.LongLiteralMinValue"))
```

```
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.NumberLiteral", "org.eclipse.jdt.internal.compiler.ast.IntLiteral"))
.target(new MethodTarget("org.eclipse.jdt.core.dom.ASTConverter", "convert",
"org.eclipse.jdt.core.dom.NumberLiteral", "org.eclipse.jdt.internal.compiler.ast.IntLiteralMinValue"))
.methodToWrap(new Hook("java.lang.String", "<init>", "void", "char[]", "int", "int"))
.requestExtra(StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes", "getRealNodeSource",
"java.lang.String", "java.lang.String", "org.eclipse.jdt.internal.compiler.ast.ASTNode"))
.transplant()
.build());
}
private static void patchASTNodeSearchUtil(ScriptManager sm) {
/*
* If an annotation generates more than one method the normal node search returns one of them
instead of the right one.
* This patch method also compares method name and method parameter types to identify the right
method.
*/
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.internal.corext.refactoring.structure.ASTNodeSearchUtil",
"getMethodDeclarationNode", "org.eclipse.jdt.core.dom.MethodDeclaration",
"org.eclipse.jdt.core.IMethod", "org.eclipse.jdt.core.dom.CompilationUnit"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$PatchFixes",
"getRealMethodDeclarationNode", "org.eclipse.jdt.core.dom.MethodDeclaration",
"org.eclipse.jdt.core.dom.MethodDeclaration", "org.eclipse.jdt.core.IMethod",
"org.eclipse.jdt.core.dom.CompilationUnit"))
.request(StackRequest.RETURN_VALUE, StackRequest.PARAM1, StackRequest.PARAM2)
.transplant()
.build());
}
private static void patchFieldInitializer(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
.targetClass("org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor")
.fieldName("$fieldInfo")
.fieldType("Ljava/lang/Object;")
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.addField()
.targetClass("org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor")
.fieldName("$sourceFieldElementInfo")
.fieldType("Lorg/eclipse/jdt/internal/core/SourceFieldElementInfo;")
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor",
"exitField", "void", "int", "int", "int"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$FieldInitializer", "storeFieldInfo",
"boolean", "org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor"))
.request(StackRequest.THIS)
.transplant()
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor",
"exitField", "void", "int", "int", "int"))
.methodToWrap(new Hook("org.eclipse.jdt.internal.core.SourceFieldElementInfo", "<init>", "void"))
```

```
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$FieldInitializer",
"storeSourceFieldElementInfo", "void", "org.eclipse.jdt.internal.core.SourceFieldElementInfo",
"org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor"))
.requestExtra(StackRequest.THIS)
.transplant()
.build());
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor",
"exitField", "void", "int", "int", "int"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$FieldInitializer", "overwriteInitializer", "void",
"org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor"))
.request(StackRequest.THIS)
.transplant()
.build());
}
private static void patchCrossModuleClassLoading(ScriptManager sm) {
sm.addScriptIfWitness(OSGI_TYPES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.parser.Parser", "<clinit>"))
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$ModuleClassLoading", "parserClinit",
"void"))
.build());
}
private static void patchForTests(ScriptManager sm) {
String[] ECLIPSE_TEST_CLASSES = new String[] {"lombok/transform/TestWithEcj",
"lombok/eclipse/EclipseRunner"};
// Add support for javadoc in tests
sm.addScriptIfWitness(ECLIPSE_TEST_CLASSES, ScriptBuilder.replaceMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.ast.TypeDeclaration", "printBody",
"java.lang.StringBuilder", "int", "java.lang.StringBuilder"))
.methodToReplace(new Hook("org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "print",
"java.lang.StringBuilder", "int", "java.lang.StringBuilder"))
.replacementMethod(new Hook("lombok.launch.PatchFixesHider$Tests", "printMethod",
"java.lang.StringBuilder", "org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "int",
"java.lang.StringBuilder", "org.eclipse.jdt.internal.compiler.ast.TypeDeclaration"))
.requestExtra(StackRequest.THIS)
.build());
sm.addScriptIfWitness(ECLIPSE_TEST_CLASSES, ScriptBuilder.replaceMethodCall()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.ast.TypeDeclaration", "printBody",
"java.lang.StringBuffer", "int", "java.lang.StringBuffer"))
.methodToReplace(new Hook("org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "print",
"java.lang.StringBuffer", "int", "java.lang.StringBuffer"))
.replacementMethod(new Hook("lombok.launch.PatchFixesHider$Tests", "printMethod",
"java.lang.StringBuffer", "org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "int",
"java.lang.StringBuffer", "org.eclipse.jdt.internal.compiler.ast.TypeDeclaration"))
.requestExtra(StackRequest.THIS)
.build());
sm.addScriptIfWitness(ECLIPSE_TEST_CLASSES, ScriptBuilder.wrapReturnValue()
.target(new MethodTarget("org.osgi.framework.FrameworkUtil", "getBundle",
"org.osgi.framework.Bundle", "java.lang.Class"))
.request(StackRequest.RETURN_VALUE, StackRequest.PARAM1)
.wrapMethod(new Hook("lombok.launch.PatchFixesHider$Tests", "getBundle", "java.lang.Object",
"java.lang.Object", "java.lang.Class"))
.build());
```

```
// Remove implicit canonical constructors in tests
sm.addScriptIfWitness(ECLIPSE_TEST_CLASSES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "print",
"java.lang.StringBuilder", "int", "java.lang.StringBuilder"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$Tests", "isImplicitCanonicalConstructor",
"boolean", "org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "java.lang.Object"))
.valueMethod(new Hook("lombok.launch.PatchFixesHider$Tests", "returnStringBuilder",
"java.lang.StringBuilder", "java.lang.Object", "java.lang.StringBuilder"))
.request(StackRequest.THIS, StackRequest.PARAM2)
.transplant()
.build());
sm.addScriptIfWitness(ECLIPSE_TEST_CLASSES, ScriptBuilder.exitEarly()
.target(new MethodTarget("org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "print",
"java.lang.StringBuffer", "int", "java.lang.StringBuffer"))
.decisionMethod(new Hook("lombok.launch.PatchFixesHider$Tests", "isImplicitCanonicalConstructor",
"boolean", "org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration", "java.lang.Object"))
.valueMethod(new Hook("lombok.launch.PatchFixesHider$Tests", "returnStringBuffer",
"java.lang.StringBuffer", "java.lang.Object", "java.lang.StringBuffer"))
.request(StackRequest.THIS, StackRequest.PARAM2)
.transplant()
.build());
}
}
```

lombok/eclipse/agent/ExtensionMethodCompletionProposal.java

```
/*
* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import static org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants.AccStatic;
```

```
import static org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants.AccVarargs;
import java.util.Arrays;
import org.eclipse.core.runtime.IProgressMonitor;
import org.eclipse.jdt.core.CompletionProposal;
import org.eclipse.jdt.core.IMethod;
import org.eclipse.jdt.core.JavaModelException;
import org.eclipse.jdt.core.Signature;
import org.eclipse.jdt.core.compiler.CharOperation;
import org.eclipse.jdt.internal.codeassist.CompletionEngine;
import org.eclipse.jdt.internal.codeassist.InternalCompletionProposal;
import org.eclipse.jdt.internal.codeassist.complete.CompletionOnMemberAccess;
import org.eclipse.jdt.internal.codeassist.complete.CompletionOnQualifiedNameReference;
import org.eclipse.jdt.internal.codeassist.complete.CompletionOnSingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.lookup.MethodBinding;
import org.eclipse.jdt.internal.compiler.lookup.TypeBinding;
import org.eclipse.jdt.internal.core.NameLookup;
public class ExtensionMethodCompletionProposal extends InternalCompletionProposal {
private char[] fullSignature;
private char[][] parameterNames;
public ExtensionMethodCompletionProposal(final int replacementOffset) {
super(CompletionProposal.METHOD_REF, replacementOffset - 1);
}
public void setMethodBinding(final MethodBinding method, final ASTNode node) {
// Add proposal parameter names, sometimes its empty...
if (method.parameterNames != null && method.parameterNames.length > 0) {
setParameterNames(Arrays.copyOfRange(method.parameterNames, 1,
method.parameterNames.length));
} else {
// Copy signature for parameter name resolution, this is more reliable but slower
fullSignature = CompletionEngine.getSignature(method);
}
MethodBinding original = method.original();
TypeBinding[] parameters = Arrays.copyOf(method.parameters, method.parameters.length);
method.parameters = Arrays.copyOfRange(method.parameters, 1, method.parameters.length);
TypeBinding[] originalParameters = null;
if (original != method) {
originalParameters = Arrays.copyOf(method.original().parameters,
method.original().parameters.length);
method.original().parameters = Arrays.copyOfRange(method.original().parameters, 1,
method.original().parameters.length);
}
int length = method.parameters == null ? 0 : method.parameters.length;
char[][] parameterPackageNames = new char[length][];
char[][] parameterTypeNames = new char[length][];
for (int i = 0; i < length; i++) {
TypeBinding type = method.original().parameters[i];
parameterPackageNames[i] = type.qualifiedPackageName();
parameterTypeNames[i] = type.qualifiedSourceName();
}
char[] completion = CharOperation.concat(method.selector, new char[] { '(', ')' });
setDeclarationSignature(CompletionEngine.getSignature(method.declaringClass));
setSignature(CompletionEngine.getSignature(method));
```

```
if (original != method) {
setOriginalSignature(CompletionEngine.getSignature(original));
}
setDeclarationPackageName(method.declaringClass.qualifiedPackageName());
setDeclarationTypeName(method.declaringClass.qualifiedSourceName());
setParameterPackageNames(parameterPackageNames);
setParameterTypeNames(parameterTypeNames);
setPackageName(method.returnType.qualifiedPackageName());
setTypeName(method.returnType.qualifiedSourceName());
setName(method.selector);
setCompletion(completion);
setFlags(method.modifiers & (~AccStatic));
// Remove varargs flag if it is the only parameter
if (method.isVarargs() && length == 0) {
setFlags(getFlags() & (~AccVarargs));
}
int index = node.sourceEnd + 1;
if (node instanceof CompletionOnQualifiedNameReference) {
index -= ((CompletionOnQualifiedNameReference) node).completionIdentifier.length;
}
if (node instanceof CompletionOnMemberAccess) {
index -= ((CompletionOnMemberAccess) node).token.length;
}
if (node instanceof CompletionOnSingleNameReference) {
index -= ((CompletionOnSingleNameReference) node).token.length;
}
setReplaceRange(index, index);
setTokenRange(index, index);
setRelevance(100);
method.parameters = parameters;
if (original != method) {
method.original().parameters = originalParameters;
}
}
@Override
public char[][] findParameterNames(IProgressMonitor monitor) {
if (parameterNames != null) {
return parameterNames;
}
NameLookup.Answer answer = this.nameLookup.findType(
new String(this.declarationTypeName),
new String(this.declarationPackageName),
false,
NameLookup.ACCEPT_CLASSES & NameLookup.ACCEPT_INTERFACES,
true/* consider secondary types */,
false/* do NOT wait for indexes */,
false/*don't check restrictions*/,
null);
if (answer != null && answer.type != null) {
char[][] parameterTypes = Signature.getParameterTypes(fullSignature);
String[] args = new String[parameterTypes.length];
for (int i = 0; i < parameterTypes.length; i++) {
args[i] = new String(parameterTypes[i]);
```

```
}
IMethod method = answer.type.getMethod(new String(this.getName()), args);
IMethod[] methods = answer.type.findMethods(method);
if (methods != null && methods.length > 0) {
method = methods[0];
}
if (method != null) {
try {
String[] parameterNames = method.getParameterNames();
char[][] parameterNamesAsChar = new char[parameterNames.length - 1][];
for (int i = 0; i < parameterNamesAsChar.length; i++) {
parameterNamesAsChar[i] = parameterNames[i + 1].toCharArray();
}
setParameterNames(parameterNamesAsChar);
} catch (JavaModelException e) {
// Nope
}
}
}
// Seems like we failed, fallback
if (parameterNames == null) {
parameterNames = super.findParameterNames(monitor);
}
return parameterNames;
}
@Override
public void setParameterNames(char[][] parameterNames) {
this.parameterNames = parameterNames;
super.setParameterNames(parameterNames);
}
}
```

lombok/eclipse/agent/MavenEcjBootstrapApp.java


```
/*
* Copyright (C) 2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import java.io.File;
import java.io.FileOutputStream;
import java.io.FileWriter;
import java.io.InputStream;
import java.io.PrintStream;
import java.util.List;
import com.zwitserloot.cmdreader.CmdReader;
import com.zwitserloot.cmdreader.Description;
import com.zwitserloot.cmdreader.InvalidCommandLineException;
import com.zwitserloot.cmdreader.Shorthand;
import lombok.core.LombokApp;
import lombok.spi.Provides;
@Provides
public class MavenEcjBootstrapApp extends LombokApp {
@Override public String getAppName() {
return "createMavenECJBootstrap";
}
@Override public String getAppDescription() {
return "Creates .mvn/jvm.config and .mvn/lombok-bootstrap.jar for\n" +
"use with the ECJ compiler.";
}
private static class CmdArgs {
@Shorthand("w")
@Description("Overwrite existing files. Defaults to false.")
boolean overwrite = false;
@Shorthand("o")
@Description("The root of a Maven project. Defaults to the current working directory.")
String output;
@Shorthand({"h", "?"})
@Description("Shows this help text")
boolean help;
}
@Override public int runApp(List<String> rawArgs) throws Exception {
CmdReader<CmdArgs> reader = CmdReader.of(CmdArgs.class);
CmdArgs args;
try {
args = reader.make(rawArgs.toArray(new String[0]));
} catch (InvalidCommandLineException e) {
printHelp(reader, e.getMessage(), System.err);
return 1;
}
if (args.help) {
printHelp(reader, null, System.out);
return 0;
}
return createBootstrap(args.output, args.overwrite);
}
private int createBootstrap(String root, boolean overwrite) {
```

```
File mvn = new File(root, ".mvn");
int result = 0;
if (result == 0) result = makeMvn(mvn);
if (result == 0) result = makeJvmConfig(mvn, overwrite);
if (result == 0) result = makeJar(mvn, overwrite);
return result;
}
private int makeMvn(File mvn) {
int result = 0;
Exception err = null;
try {
if (!mvn.exists() && !mvn.mkdirs()) result = 1;
} catch (Exception e) {
result = 1;
err = e;
}
if (result != 0) {
System.err.println("Could not create " + mvn.getPath());
if (err != null) err.printStackTrace(System.err);
}
return result;
}
private int makeJvmConfig(File mvn, boolean overwrite) {
File jvmConfig = new File(mvn, "jvm.config");
if (jvmConfig.exists() && !overwrite) {
System.err.println(canonical(jvmConfig) + " exists but '-w' not specified.");
return 1;
}
try {
FileWriter writer = new FileWriter(jvmConfig);
writer.write("-javaagent:.mvn/lombok-bootstrap.jar");
writer.flush();
writer.close();
System.out.println("Successfully created: " + canonical(jvmConfig));
return 0;
} catch (Exception e) {
System.err.println("Could not create: " + canonical(jvmConfig));
e.printStackTrace(System.err);
return 1;
}
}
private int makeJar(File mvn, boolean overwrite) {
File jar = new File(mvn, "lombok-bootstrap.jar");
if (jar.exists() && !overwrite) {
System.err.println(canonical(jar) + " but '-w' not specified.");
return 1;
}
try {
InputStream input =
MavenEcjBootstrapApp.class.getResourceAsStream("/lombok/launch/mavenEcjBootstrapAgent.jar");
FileOutputStream output = new FileOutputStream(jar);
try {
byte[] buffer = new byte[4096];
```

```
int length;
while ((length = input.read(buffer)) > 0) output.write(buffer, 0, length);
output.flush();
output.close();
System.out.println("Successfully created: " + canonical(jar));
return 0;
} finally {
try {
output.close();
} catch (Exception ignore) {}
}
} catch (Exception e) {
System.err.println("Could not create: " + canonical(jar));
e.printStackTrace(System.err);
return 1;
}
}
private static String canonical(File out) {
try {
return out.getCanonicalPath();
} catch (Exception e) {
return out.getAbsolutePath();
}
}
private void printHelp(CmdReader<CmdArgs> reader, String message, PrintStream out) {
if (message != null) {
out.println(message);
out.println("-----------------------------");
}
out.println(reader.generateCommandLineHelp("java -jar lombok.jar createMavenECJBootstrap"));
}
}
```

lombok/eclipse/agent/PatchDelegate.java


```
/*
* Copyright (C) 2010-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import static lombok.eclipse.EcjAugments.*;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import org.eclipse.jdt.core.ElementChangedEvent;
import org.eclipse.jdt.core.IJavaElement;
import org.eclipse.jdt.core.ILocalVariable;
import org.eclipse.jdt.core.IType;
import org.eclipse.jdt.core.JavaModelException;
import org.eclipse.jdt.core.Signature;
import org.eclipse.jdt.core.compiler.CharOperation;
import org.eclipse.jdt.internal.compiler.CompilationResult;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Argument;
import org.eclipse.jdt.internal.compiler.ast.ArrayInitializer;
import org.eclipse.jdt.internal.compiler.ast.ClassLiteralAccess;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.MemberValuePair;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.MethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ReturnStatement;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.Statement;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeParameter;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.lookup.AnnotationBinding;
import org.eclipse.jdt.internal.compiler.lookup.ArrayBinding;

```
import org.eclipse.jdt.internal.compiler.lookup.BaseTypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.Binding;
import org.eclipse.jdt.internal.compiler.lookup.ClassScope;
import org.eclipse.jdt.internal.compiler.lookup.ExtraCompilerModifiers;
import org.eclipse.jdt.internal.compiler.lookup.FieldBinding;
import org.eclipse.jdt.internal.compiler.lookup.MethodBinding;
import org.eclipse.jdt.internal.compiler.lookup.ParameterizedTypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.ReferenceBinding;
import org.eclipse.jdt.internal.compiler.lookup.SourceTypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.TypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeVariableBinding;
import org.eclipse.jdt.internal.compiler.lookup.UnresolvedReferenceBinding;
import org.eclipse.jdt.internal.compiler.lookup.WildcardBinding;
import org.eclipse.jdt.internal.core.CompilationUnit;
import org.eclipse.jdt.internal.core.DeltaProcessor;
import org.eclipse.jdt.internal.core.JavaElement;
import org.eclipse.jdt.internal.core.JavaElementDelta;
import org.eclipse.jdt.internal.core.JavaModelManager;
import org.eclipse.jdt.internal.core.LocalVariable;
import org.eclipse.jdt.internal.core.SourceMethod;
import org.eclipse.jdt.internal.core.SourceMethodInfo;
import org.eclipse.jdt.internal.core.SourceType;
import org.eclipse.jdt.internal.core.SourceTypeElementInfo;
import lombok.core.AST.Kind;
import lombok.eclipse.Eclipse;
import lombok.eclipse.EclipseAST;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.TransformEclipseAST;
import lombok.eclipse.handlers.SetGeneratedByVisitor;
import lombok.patcher.Symbols;
import lombok.permit.Permit;
public class PatchDelegate {
private static class ClassScopeEntry {
ClassScopeEntry(ClassScope scope) {
this.scope = scope;
}
final ClassScope scope;
String corruptedPath;
}
private static ThreadLocal<List<ClassScopeEntry>> visited = new
ThreadLocal<List<ClassScopeEntry>>() {
protected List<ClassScopeEntry> initialValue() {
return new ArrayList<ClassScopeEntry>();
}
};
private static String nameOfScope(ClassScope scope) {
TypeDeclaration decl = scope.referenceContext;
if (decl == null) return "(unknown)";
if (decl.name == null || decl.name.length == 0) return "(unknown)";
return new String(decl.name);
}
private static boolean hasDelegateMarkedFieldsOrMethods(TypeDeclaration decl) {
```

```
if (decl.fields != null) for (FieldDeclaration field : decl.fields) {
if (field.annotations == null) continue;
for (Annotation ann : field.annotations) {
if (isDelegate(ann, decl)) return true;
}
}
if (decl.methods != null) for (AbstractMethodDeclaration method : decl.methods) {
if (method.annotations == null) continue;
for (Annotation ann : method.annotations) {
if (isDelegate(ann, decl)) return true;
}
}
return false;
}
public static boolean handleDelegateForType(ClassScope scope) {
if (TransformEclipseAST.disableLombok) return false;
CompilationUnitDeclaration cud = scope.compilationUnitScope().referenceContext;
if (scope == scope.compilationUnitScope().topLevelTypes[0].scope) {
if (eclipseAvailable) {
EclipseOnlyMethods.cleanupDelegateMethods(cud);
}
}
if (!hasDelegateMarkedFieldsOrMethods(scope.referenceContext)) return false;
List<ClassScopeEntry> stack = visited.get();
StringBuilder corrupted = null;
for (ClassScopeEntry entry : stack) {
if (corrupted != null) {
corrupted.append(" -> ").append(nameOfScope(entry.scope));
} else if (entry.scope == scope) {
corrupted = new StringBuilder().append(nameOfScope(scope));
}
}
if (corrupted != null) {
boolean found = false;
String path = corrupted.toString();
for (ClassScopeEntry entry : stack) {
if (!found && entry.scope == scope) found = true;
if (found) entry.corruptedPath = path;
}
} else {
ClassScopeEntry entry = new ClassScopeEntry(scope);
stack.add(entry);
try {
TypeDeclaration decl = scope.referenceContext;
if (decl != null) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
List<BindingTuple> methodsToDelegate = new ArrayList<BindingTuple>();
fillMethodBindingsForFields(cud, scope, methodsToDelegate);
if (entry.corruptedPath != null) {
eclipseAst.get(scope.referenceContext).addError("No @Delegate methods created because there's a
loop: " + entry.corruptedPath);
} else {
generateDelegateMethods(eclipseAst.get(decl), methodsToDelegate, DelegateReceiver.FIELD);
```

```
}
methodsToDelegate.clear();
fillMethodBindingsForMethods(cud, scope, methodsToDelegate);
if (entry.corruptedPath != null) {
eclipseAst.get(scope.referenceContext).addError("No @Delegate methods created because there's a
loop: " + entry.corruptedPath);
} else {
generateDelegateMethods(eclipseAst.get(decl), methodsToDelegate, DelegateReceiver.METHOD);
}
}
} finally {
stack.remove(stack.size() - 1);
if (stack.isEmpty() && eclipseAvailable) {
EclipseOnlyMethods.notifyDelegateMethodsAdded(cud);
}
}
}
return false;
}
/**
* Returns a string containing the signature of a method that appears (erased) at least twice in the list.
* If no duplicates are present, {@code null} is returned.
*/
private static String containsDuplicates(List<BindingTuple> tuples) {
Set<String> sigs = new HashSet<String>();
for (BindingTuple tuple : tuples) {
if (!sigs.add(printSig(tuple.parameterized))) return printSig(tuple.parameterized);
}
return null;
}
public static void markHandled(Annotation annotation) {
Annotation_applied.set(annotation, true);
}
private static void fillMethodBindingsForFields(CompilationUnitDeclaration cud, ClassScope scope,
List<BindingTuple> methodsToDelegate) {
TypeDeclaration decl = scope.referenceContext;
if (decl == null) return;
if (decl.fields != null) for (FieldDeclaration field : decl.fields) {
if (field.annotations == null) continue;
for (Annotation ann : field.annotations) {
if (!isDelegate(ann, decl)) continue;
if (Annotation_applied.getAndSet(ann, true)) continue;
if ((field.modifiers & ClassFileConstants.AccStatic) != 0) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
eclipseAst.get(ann).addError(LEGALITY_OF_DELEGATE);
break;
}
List<ClassLiteralAccess> rawTypes = rawTypes(ann, "types");
List<ClassLiteralAccess> excludedRawTypes = rawTypes(ann, "excludes");
List<BindingTuple> methodsToExclude = new ArrayList<BindingTuple>();
List<BindingTuple> methodsToDelegateForThisAnn = new ArrayList<BindingTuple>();
try {
for (ClassLiteralAccess cla : excludedRawTypes) {
```

```
addAllMethodBindings(methodsToExclude, cla.type.resolveType(decl.initializerScope), new
HashSet<String>(), field.name, ann);
}
Set<String> banList = findAlreadyImplementedMethods(decl);
for (BindingTuple excluded : methodsToExclude) banList.add(printSig(excluded.parameterized));
if (rawTypes.isEmpty()) {
addAllMethodBindings(methodsToDelegateForThisAnn, field.type.resolveType(decl.initializerScope),
banList, field.name, ann);
} else {
for (ClassLiteralAccess cla : rawTypes) {
addAllMethodBindings(methodsToDelegateForThisAnn, cla.type.resolveType(decl.initializerScope),
banList, field.name, ann);
}
}
} catch (DelegateRecursion e) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
eclipseAst.get(ann).addError(String.format(RECURSION_NOT_ALLOWED, new String(e.member),
new String(e.type)));
break;
}
// Not doing this right now because of problems - see commented-out-method for info.
// removeExistingMethods(methodsToDelegate, decl, scope);
String dupe = containsDuplicates(methodsToDelegateForThisAnn);
if (dupe != null) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
eclipseAst.get(ann).addError("The method '" + dupe + "' is being delegated by more than one specified
type.");
} else {
methodsToDelegate.addAll(methodsToDelegateForThisAnn);
}
}
}
}
private static final String LEGALITY_OF_DELEGATE = "@Delegate is legal only on instance fields or
no-argument instance methods.";
private static final String RECURSION_NOT_ALLOWED = "@Delegate does not support recursion
(delegating to a type that itself has @Delegate members). Member \"%s\" is @Delegate in type \"%s\"";
private static void fillMethodBindingsForMethods(CompilationUnitDeclaration cud, ClassScope scope,
List<BindingTuple> methodsToDelegate) {
TypeDeclaration decl = scope.referenceContext;
if (decl == null) return;
if (decl.methods != null) for (AbstractMethodDeclaration methodDecl : decl.methods) {
if (methodDecl.annotations == null) continue;
for (Annotation ann : methodDecl.annotations) {
if (!isDelegate(ann, decl)) continue;
if (Annotation_applied.getAndSet(ann, true)) continue;
if (!(methodDecl instanceof MethodDeclaration)) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
eclipseAst.get(ann).addError(LEGALITY_OF_DELEGATE);
break;
}
if (methodDecl.arguments != null) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
```

```
eclipseAst.get(ann).addError(LEGALITY_OF_DELEGATE);
break;
}
if ((methodDecl.modifiers & ClassFileConstants.AccStatic) != 0) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
eclipseAst.get(ann).addError(LEGALITY_OF_DELEGATE);
break;
}
MethodDeclaration method = (MethodDeclaration) methodDecl;
List<ClassLiteralAccess> rawTypes = rawTypes(ann, "types");
List<ClassLiteralAccess> excludedRawTypes = rawTypes(ann, "excludes");
List<BindingTuple> methodsToExclude = new ArrayList<BindingTuple>();
List<BindingTuple> methodsToDelegateForThisAnn = new ArrayList<BindingTuple>();
try {
for (ClassLiteralAccess cla : excludedRawTypes) {
addAllMethodBindings(methodsToExclude, cla.type.resolveType(decl.initializerScope), new
HashSet<String>(), method.selector, ann);
}
Set<String> banList = findAlreadyImplementedMethods(decl);
for (BindingTuple excluded : methodsToExclude) banList.add(printSig(excluded.parameterized));
if (rawTypes.isEmpty()) {
if (method.returnType == null) continue;
addAllMethodBindings(methodsToDelegateForThisAnn,
method.returnType.resolveType(decl.initializerScope), banList, method.selector, ann);
} else {
for (ClassLiteralAccess cla : rawTypes) {
addAllMethodBindings(methodsToDelegateForThisAnn, cla.type.resolveType(decl.initializerScope),
banList, method.selector, ann);
}
}
} catch (DelegateRecursion e) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
eclipseAst.get(ann).addError(String.format(RECURSION_NOT_ALLOWED, new String(e.member),
new String(e.type)));
break;
}
// Not doing this right now because of problems - see commented-out-method for info.
// removeExistingMethods(methodsToDelegate, decl, scope);
String dupe = containsDuplicates(methodsToDelegateForThisAnn);
if (dupe != null) {
EclipseAST eclipseAst = TransformEclipseAST.getAST(cud, true);
eclipseAst.get(ann).addError("The method '" + dupe + "' is being delegated by more than one specified
type.");
} else {
methodsToDelegate.addAll(methodsToDelegateForThisAnn);
}
}
}
}
private static boolean isDelegate(Annotation ann, TypeDeclaration decl) {
if (ann.type == null) return false;
if (!charArrayEquals("Delegate", ann.type.getLastToken())) return false;
TypeBinding tb = ann.type.resolveType(decl.initializerScope);
```

```
if (tb == null) return false;
if (!charArrayEquals("lombok", tb.qualifiedPackageName()) &&
!charArrayEquals("lombok.experimental", tb.qualifiedPackageName())) return false;
if (!charArrayEquals("Delegate", tb.qualifiedSourceName())) return false;
return true;
}
private static List<ClassLiteralAccess> rawTypes(Annotation ann, String name) {
List<ClassLiteralAccess> rawTypes = new ArrayList<ClassLiteralAccess>();
for (MemberValuePair pair : ann.memberValuePairs()) {
if (charArrayEquals(name, pair.name)) {
if (pair.value instanceof ArrayInitializer) {
for (Expression expr : ((ArrayInitializer)pair.value).expressions) {
if (expr instanceof ClassLiteralAccess) rawTypes.add((ClassLiteralAccess) expr);
}
}
if (pair.value instanceof ClassLiteralAccess) {
rawTypes.add((ClassLiteralAccess) pair.value);
}
}
}
return rawTypes;
}
/*
* We may someday finish this method. Steps to be completed:
*
* (A) Turn any Parameterized anythings into non-parameterized versions. Resolving parameterized
stuff will definitely not work safely.
* (B) scope.problemReporter() will need to return a noop reporter as various errors are marked off.
* (C) Find a way to do _something_ for references to typevars (i.e. 'T') which are declared on the
method itself.
* (D) getTypeBinding isn't public, so call it via reflection.
*/
// private static TypeBinding safeResolveAndErase(TypeReference ref, Scope scope) {
// if (ref.resolvedType != null) {
// return ref.resolvedType.isValidBinding() ? ref.resolvedType : null;
// }
//
// try {
// TypeBinding bind = ref.getTypeBinding(scope);
// if (!bind.isValidBinding()) return null;
// } catch (AbortCompilation e) {
// return null;
// }
// return bind.erasure();
// }
/*
* Not using this because calling clone.resolveType() taints a bunch of caches and reports erroneous
errors.
*/
// private static void removeExistingMethods(List<BindingTuple> list, TypeDeclaration decl,
ClassScope scope) {
// for (AbstractMethodDeclaration methodDecl : decl.methods) {
// if (!(methodDecl instanceof MethodDeclaration)) continue;
```

```
// MethodDeclaration md = (MethodDeclaration) methodDecl;
// char[] name = md.selector;
// TypeBinding[] args = md.arguments == null ? new TypeBinding[0] : new
TypeBinding[md.arguments.length];
// for (int i = 0; i < args.length; i++) {
// TypeReference clone = Eclipse.copyType(md.arguments[i].type, md.arguments[i]);
// args[i] = clone.resolveType(scope).erasure(); // This is the problematic line
// }
// Iterator<BindingTuple> it = list.iterator();
// methods:
// while (it.hasNext()) {
// MethodBinding mb = it.next().parameterized;
// if (!Arrays.equals(mb.selector, name)) continue;
// int paramLen = mb.parameters == null ? 0 : mb.parameters.length;
// if (paramLen != args.length) continue;
// if (md.typeParameters == null || md.typeParameters.length == 0) {
// for (int i = 0; i < paramLen; i++) {
// if (!mb.parameters[i].erasure().isEquivalentTo(args[i])) continue methods;
// }
// } else {
// for (int i = 0; i < paramLen; i++) {
// if (!mb.parameters[i].erasure().isEquivalentTo(args[i])) ;
// }
// //BUG #???: We erase the method's parameter types using the class scope, but we should be using
the method scope.
// // In practice this is no problem UNLESS the method has type parameters, such as <T> T[]
toArray(T[] in).
// // In this case the class scope cannot resolve the T[] parameter and erase it to Object[], which is a big
problem because
// // it would mean manually writing <X> X[] toArray(X[] in) does NOT stop lombok from ALSO trying to
make the delegated toArray method,
// // thus causing an error (2 methods with post-erasure duplicate signatures). Our 'fix' for this is to treat
any method with type parameters
// // as if each parameter's type matches anything else; so, if the name matches and the parameter
count, we DONT generate it, even if its just
// // an overloaded method.
// //
// // The reason we do this now is because making that MethodScope properly is effectively impossible
at this step, so we need to do the resolving
// // ourselves, which involves chasing down array bindings (T[]), following the path down type variables,
i.e. <X extends Y, Y extends T>, and then
// // resolving the final result of this exercise against the class scope.
//
// // When this crappy incomplete workaround of ours occurs, we end up in this else block, which does
nothing and thus we fall through and remove
// // the method.
// }
// it.remove(); // Method already exists in this class - don't create a delegating implementation.
// }
// }
// }
private static void generateDelegateMethods(EclipseNode typeNode, List<BindingTuple> methods,
DelegateReceiver delegateReceiver) {
```

```
CompilationUnitDeclaration top = (CompilationUnitDeclaration) typeNode.top().get();
List<MethodDeclaration> addedMethods = new ArrayList<MethodDeclaration>();
for (BindingTuple pair : methods) {
EclipseNode annNode = typeNode.getAst().get(pair.responsible);
MethodDeclaration method = createDelegateMethod(pair.fieldName, typeNode, pair,
top.compilationResult, annNode, delegateReceiver);
if (method != null) {
SetGeneratedByVisitor visitor = new SetGeneratedByVisitor(annNode.get());
method.traverse(visitor, ((TypeDeclaration)typeNode.get()).scope);
injectMethod(typeNode, method);
addedMethods.add(method);
}
}
if (eclipseAvailable) {
EclipseOnlyMethods.collectGeneratedDelegateMethods(top, typeNode, addedMethods);
}
}
public static void checkConflictOfTypeVarNames(BindingTuple binding, EclipseNode typeNode) throws
CantMakeDelegates {
TypeVariableBinding[] typeVars = binding.parameterized.typeVariables();
if (typeVars == null || typeVars.length == 0) return;
Set<String> usedInOurType = new HashSet<String>();
EclipseNode enclosingType = typeNode;
while (enclosingType != null) {
if (enclosingType.getKind() == Kind.TYPE) {
TypeParameter[] typeParameters = ((TypeDeclaration)enclosingType.get()).typeParameters;
if (typeParameters != null) {
for (TypeParameter param : typeParameters) {
if (param.name != null) usedInOurType.add(new String(param.name));
}
}
}
enclosingType = enclosingType.up();
}
Set<String> usedInMethodSig = new HashSet<String>();
for (TypeVariableBinding var : typeVars) {
char[] sourceName = var.sourceName();
if (sourceName != null) usedInMethodSig.add(new String(sourceName));
}
usedInMethodSig.retainAll(usedInOurType);
if (usedInMethodSig.isEmpty()) return;
// We might be delegating a List<T>, and we are making method <T> toArray(). A conflict is possible.
// But only if the toArray method also uses type vars from its class, otherwise we're only shadowing,
// which is okay as we'll add a @SuppressWarnings.
TypeVarFinder finder = new TypeVarFinder();
finder.visitRaw(binding.base);
Set<String> names = new HashSet<String>(finder.getTypeVariables());
names.removeAll(usedInMethodSig);
if (!names.isEmpty()) {
// We have a confirmed conflict. We could dig deeper as this may still be a false alarm, but its already
an exceedingly rare case.
CantMakeDelegates cmd = new CantMakeDelegates();
cmd.conflicted = usedInMethodSig;
```

```
throw cmd;
}
}
public static class CantMakeDelegates extends Exception {
public Set<String> conflicted;
}
public static class TypeVarFinder extends EclipseTypeBindingScanner {
private Set<String> typeVars = new HashSet<String>();
public Set<String> getTypeVariables() {
return typeVars;
}
@Override public void visitTypeVariable(TypeVariableBinding binding) {
if (binding.sourceName != null) typeVars.add(new String(binding.sourceName));
super.visitTypeVariable(binding);
}
}
public abstract static class EclipseTypeBindingScanner {
public void visitRaw(Binding binding) {
if (binding == null) return;
if (binding instanceof MethodBinding) visitMethod((MethodBinding) binding);
if (binding instanceof BaseTypeBinding) visitBase((BaseTypeBinding) binding);
if (binding instanceof ArrayBinding) visitArray((ArrayBinding) binding);
if (binding instanceof UnresolvedReferenceBinding) visitUnresolved((UnresolvedReferenceBinding)
binding);
if (binding instanceof WildcardBinding) visitWildcard((WildcardBinding) binding);
if (binding instanceof TypeVariableBinding) visitTypeVariable((TypeVariableBinding) binding);
if (binding instanceof ParameterizedTypeBinding) visitParameterized((ParameterizedTypeBinding)
binding);
if (binding instanceof ReferenceBinding) visitReference((ReferenceBinding) binding);
}
public void visitReference(ReferenceBinding binding) {
}
public void visitParameterized(ParameterizedTypeBinding binding) {
visitRaw(binding.genericType());
TypeVariableBinding[] typeVars = binding.typeVariables();
if (typeVars != null) for (TypeVariableBinding child : typeVars) {
visitRaw(child);
}
}
public void visitTypeVariable(TypeVariableBinding binding) {
visitRaw(binding.superclass);
ReferenceBinding[] supers = binding.superInterfaces();
if (supers != null) for (ReferenceBinding child : supers) {
visitRaw(child);
}
}
public void visitWildcard(WildcardBinding binding) {
visitRaw(binding.bound);
}
public void visitUnresolved(UnresolvedReferenceBinding binding) {
}
public void visitArray(ArrayBinding binding) {
visitRaw(binding.leafComponentType());
```

```
}
public void visitBase(BaseTypeBinding binding) {
}
public void visitMethod(MethodBinding binding) {
if (binding.parameters != null) for (TypeBinding child : binding.parameters) {
visitRaw(child);
}
visitRaw(binding.returnType);
if (binding.thrownExceptions != null) for (TypeBinding child : binding.thrownExceptions) {
visitRaw(child);
}
TypeVariableBinding[] typeVars = binding.typeVariables();
if (typeVars != null) for (TypeVariableBinding child : typeVars) {
visitRaw(child.superclass);
ReferenceBinding[] supers = child.superInterfaces();
if (supers != null) for (ReferenceBinding child2 : supers) {
visitRaw(child2);
}
}
}
}
private static MethodDeclaration createDelegateMethod(char[] name, EclipseNode typeNode,
BindingTuple pair, CompilationResult compilationResult, EclipseNode annNode, DelegateReceiver
delegateReceiver) {
/* public <T, U, ...> ReturnType methodName(ParamType1 name1, ParamType2 name2, ...) throws T1,
T2, ... {
* (return) delegate.<T, U>methodName(name1, name2);
* }
*/
boolean isVarargs = (pair.base.modifiers & ClassFileConstants.AccVarargs) != 0;
try {
checkConflictOfTypeVarNames(pair, typeNode);
} catch (CantMakeDelegates e) {
annNode.addError("There's a conflict in the names of type parameters. Fix it by renaming the following
type parameters of your class: " + e.conflicted);
return null;
}
ASTNode source = annNode.get();
int pS = source.sourceStart, pE = source.sourceEnd;
MethodBinding binding = pair.parameterized;
MethodDeclaration method = new MethodDeclaration(compilationResult);
setGeneratedBy(method, source);
method.sourceStart = pS; method.sourceEnd = pE;
method.modifiers = ClassFileConstants.AccPublic;
method.returnType = makeType(binding.returnType, source, false);
boolean isDeprecated = binding.isDeprecated();
method.selector = binding.selector;
if (binding.thrownExceptions != null && binding.thrownExceptions.length > 0) {
method.thrownExceptions = new TypeReference[binding.thrownExceptions.length];
for (int i = 0; i < method.thrownExceptions.length; i++) {
method.thrownExceptions[i] = makeType(binding.thrownExceptions[i], source, false);
}
}
```

```
MessageSend call = new MessageSend();
call.sourceStart = pS; call.sourceEnd = pE;
call.nameSourcePosition = pos(source);
setGeneratedBy(call, source);
call.receiver = delegateReceiver.get(source, name);
call.selector = binding.selector;
if (binding.typeVariables != null && binding.typeVariables.length > 0) {
method.typeParameters = new TypeParameter[binding.typeVariables.length];
call.typeArguments = new TypeReference[binding.typeVariables.length];
for (int i = 0; i < method.typeParameters.length; i++) {
method.typeParameters[i] = new TypeParameter();
method.typeParameters[i].sourceStart = pS; method.typeParameters[i].sourceEnd = pE;
setGeneratedBy(method.typeParameters[i], source);
method.typeParameters[i].name = binding.typeVariables[i].sourceName;
call.typeArguments[i] = new SingleTypeReference(binding.typeVariables[i].sourceName, pos(source));
setGeneratedBy(call.typeArguments[i], source);
ReferenceBinding super1 = binding.typeVariables[i].superclass;
ReferenceBinding[] super2 = binding.typeVariables[i].superInterfaces;
if (super2 == null) super2 = new ReferenceBinding[0];
if (super1 != null || super2.length > 0) {
int offset = super1 == null ? 0 : 1;
method.typeParameters[i].bounds = new TypeReference[super2.length + offset - 1];
if (super1 != null) method.typeParameters[i].type = makeType(super1, source, false);
else method.typeParameters[i].type = makeType(super2[0], source, false);
int ctr = 0;
for (int j = (super1 == null) ? 1 : 0; j < super2.length; j++) {
method.typeParameters[i].bounds[ctr] = makeType(super2[j], source, false);
method.typeParameters[i].bounds[ctr++].bits |= ASTNode.IsSuperType;
}
}
}
}
if (isDeprecated) {
method.annotations = new Annotation[] { generateDeprecatedAnnotation(source) };
}
method.bits |= ECLIPSE_DO_NOT_TOUCH_FLAG;
if (binding.parameters != null && binding.parameters.length > 0) {
method.arguments = new Argument[binding.parameters.length];
call.arguments = new Expression[method.arguments.length];
for (int i = 0; i < method.arguments.length; i++) {
AbstractMethodDeclaration sourceElem;
try {
sourceElem = pair.base.sourceMethod();
} catch (Exception e) {
sourceElem = null;
}
char[] argName;
if (sourceElem == null) argName = ("arg" + i).toCharArray();
else {
argName = sourceElem.arguments[i].name;
}
method.arguments[i] = new Argument(
argName, pos(source),
```

```
makeType(binding.parameters[i], source, false),
ClassFileConstants.AccFinal);
setGeneratedBy(method.arguments[i], source);
call.arguments[i] = new SingleNameReference(argName, pos(source));
setGeneratedBy(call.arguments[i], source);
}
if (isVarargs) {
method.arguments[method.arguments.length - 1].type.bits |= ASTNode.IsVarArgs;
}
}
Statement body;
if (method.returnType instanceof SingleTypeReference &&
((SingleTypeReference)method.returnType).token == TypeConstants.VOID) {
body = call;
} else {
body = new ReturnStatement(call, source.sourceStart, source.sourceEnd);
setGeneratedBy(body, source);
}
method.statements = new Statement[] {body};
return method;
}
private static boolean eclipseAvailable = true;
static {
try {
CompilationUnit.class.getName();
} catch (Throwable t) {
eclipseAvailable = false;
}
}
public static Object[] addGeneratedDelegateMethods(Object[] returnValue, Object javaElement) {
if (Symbols.hasSymbol("lombok.skipdelegates")) return returnValue;
if (!eclipseAvailable) return returnValue;
return EclipseOnlyMethods.addGeneratedDelegateMethodsToChildren(returnValue, javaElement);
}
public static Object returnElementInfo(Object delegateSourceMethod) {
Field field = Permit.permissiveGetField(delegateSourceMethod.getClass(), "sourceMethodInfo");
return Permit.permissiveReadField(Object.class, field, delegateSourceMethod);
}
public static boolean isDelegateSourceMethod(Object sourceMethod) {
return sourceMethod.getClass().getName().equals("lombok.eclipse.agent.PatchDelegate$EclipseOnly
Methods$DelegateSourceMethod");
}
public static class EclipseOnlyMethods {
private static void cleanupDelegateMethods(CompilationUnitDeclaration cud) {
CompilationUnit compilationUnit = getCompilationUnit(cud);
if (compilationUnit != null) {
EclipseAugments.CompilationUnit_delegateMethods.clear(compilationUnit);
}
}
public static void collectGeneratedDelegateMethods(CompilationUnitDeclaration top, EclipseNode
typeNode, List<MethodDeclaration> addedMethods) {
String qualifiedName = new String(CharOperation.concatWith(getQualifiedInnerName(typeNode.up(),
typeNode.getName().toCharArray()), '$'));
```

```
SourceType sourceType = getSourceType(top, qualifiedName);
List<SourceMethod> generatedMethods = getGeneratedMethods(sourceType);
if (generatedMethods == null) return;
for (MethodDeclaration md : addedMethods) {
generatedMethods.add(DelegateSourceMethod.forMethodDeclaration(sourceType, md));
}
}
public static Object[] addGeneratedDelegateMethodsToChildren(Object[] returnValue, Object
javaElement) {
List<SourceMethod> delegateMethods = getGeneratedMethods((SourceType)
((SourceTypeElementInfo) javaElement).getHandle());
if (delegateMethods != null) {
return concat((IJavaElement[]) returnValue, delegateMethods.toArray(new IJavaElement[0]),
IJavaElement.class);
}
return returnValue;
}
private static void notifyDelegateMethodsAdded(CompilationUnitDeclaration cud) {
CompilationUnit compilationUnit = getCompilationUnit(cud);
if (compilationUnit != null) {
DeltaProcessor deltaProcessor = JavaModelManager.getJavaModelManager().getDeltaProcessor();
deltaProcessor.fire(new JavaElementDelta(compilationUnit),
ElementChangedEvent.POST_CHANGE);
}
}
private static CompilationUnit getCompilationUnit(Object iCompilationUnit) {
if (iCompilationUnit instanceof CompilationUnit) {
CompilationUnit compilationUnit = (CompilationUnit) iCompilationUnit;
return compilationUnit.originalFromClone();
}
return null;
}
private static CompilationUnit getCompilationUnit(CompilationUnitDeclaration cud) {
return getCompilationUnit(cud.compilationResult.compilationUnit);
}
private static final class DelegateSourceMethod extends SourceMethod {
private DelegateSourceMethodInfo sourceMethodInfo;
private static DelegateSourceMethod forMethodDeclaration(JavaElement parent, MethodDeclaration
method) {
Argument[] arguments = method.arguments != null ? method.arguments : new Argument[0];
String[] parameterTypes = new String[arguments.length];
for (int i = 0; i < arguments.length; i++) {
parameterTypes[i] = Signature.createTypeSignature(CharOperation.concatWith(arguments[i].type.getP
arameterizedTypeName(), '.'), false);
}
return new DelegateSourceMethod(parent, new String(method.selector), parameterTypes, method);
}
private DelegateSourceMethod(JavaElement parent, String name, String[] parameterTypes,
MethodDeclaration md) {
super(parent, name, parameterTypes);
sourceMethodInfo = new DelegateSourceMethodInfo(this, md);
}
@Override public Object getElementInfo() throws JavaModelException {
```

```
return sourceMethodInfo;
}
/**
* Disable refactoring for delegate methods
*/
@Override public boolean isReadOnly() {
return true;
}
/**
* This is required to prevent duplicate entries in the outline
*/
@Override public boolean equals(Object o) {
return this == o;
}
public static final class DelegateSourceMethodInfo extends SourceMethodInfo {
DelegateSourceMethodInfo(DelegateSourceMethod delegateSourceMethod, MethodDeclaration md) {
int pS = md.sourceStart;
int pE = md.sourceEnd;
Argument[] methodArguments = md.arguments != null ? md.arguments : new Argument[0];
char[][] argumentNames = new char[methodArguments.length][];
arguments = new ILocalVariable[methodArguments.length];
for (int i = 0; i < methodArguments.length; i++) {
Argument argument = methodArguments[i];
argumentNames[i] = argument.name;
arguments[i] = new LocalVariable(delegateSourceMethod, new String(argument.name), pS, pE, pS,
pS, delegateSourceMethod.getParameterTypes()[i], argument.annotations, argument.modifiers, true);
}
setArgumentNames(argumentNames);
setSourceRangeStart(pS);
setSourceRangeEnd(pE);
setNameSourceStart(pS);
setNameSourceEnd(pE);
setExceptionTypeNames(CharOperation.NO_CHAR_CHAR);
setReturnType(md.returnType == null ? new char[]{'v', 'o','i', 'd'} :
CharOperation.concatWith(md.returnType.getParameterizedTypeName(), '.'));
setFlags(md.modifiers);
}
}
}
private static List<SourceMethod> getGeneratedMethods(SourceType sourceType) {
if (sourceType != null) {
CompilationUnit compilationUnit = getCompilationUnit(sourceType.getCompilationUnit());
if (compilationUnit != null) {
ConcurrentMap<String, List<SourceMethod>> map =
EclipseAugments.CompilationUnit_delegateMethods.setIfAbsent(compilationUnit, new
ConcurrentHashMap<String, List<SourceMethod>>());
List<SourceMethod> newList = new ArrayList<SourceMethod>();
List<SourceMethod> oldList = map.putIfAbsent(sourceType.getTypeQualifiedName(), newList);
return oldList != null ? oldList : newList;
}
}
return null;
}
```

```
private static SourceType getSourceType(CompilationUnitDeclaration cud, String typeName) {
CompilationUnit compilationUnit = getCompilationUnit(cud);
if (compilationUnit != null) {
try {
for (IType type : compilationUnit.getAllTypes()) {
if (type instanceof SourceType && type.getTypeQualifiedName().equals(typeName)) {
return (SourceType) type;
}
}
} catch (JavaModelException e) {
// Ignore
}
}
return null;
}
}
private static final class Reflection {
public static final Method classScopeBuildFieldsAndMethodsMethod;
public static final Throwable initCause;
static {
Method m = null;
Throwable c = null;
try {
m = Permit.getMethod(ClassScope.class, "buildFieldsAndMethods");
} catch (Throwable t) {
c = t;
// That's problematic, but as long as no local classes are used we don't actually need it.
// Better fail on local classes than crash altogether.
}
classScopeBuildFieldsAndMethodsMethod = m;
initCause = c;
}
}
private static void addAllMethodBindings(List<BindingTuple> list, TypeBinding binding, Set<String>
banList, char[] fieldName, ASTNode responsible) throws DelegateRecursion {
banList.addAll(METHODS_IN_OBJECT);
addAllMethodBindings0(list, binding, banList, fieldName, responsible);
}
private static class DelegateRecursion extends Throwable {
final char[] type, member;
public DelegateRecursion(char[] type, char[] member) {
this.type = type;
this.member = member;
}
}
private static void addAllMethodBindings0(List<BindingTuple> list, TypeBinding binding, Set<String>
banList, char[] fieldName, ASTNode responsible) throws DelegateRecursion {
if (binding instanceof SourceTypeBinding) {
ClassScope scope = ((SourceTypeBinding) binding).scope;
if (scope != null) scope.environment().globalOptions.storeAnnotations = true;
}
if (binding == null) return;
TypeBinding inner;
```

```
if (binding instanceof ParameterizedTypeBinding) {
inner = ((ParameterizedTypeBinding) binding).genericType();
} else {
inner = binding;
}
if (inner instanceof SourceTypeBinding) {
ClassScope cs = ((SourceTypeBinding)inner).scope;
if (cs != null) {
try {
Permit.invoke(Reflection.initCause, Reflection.classScopeBuildFieldsAndMethodsMethod, cs);
} catch (Exception e) {
// See 'Reflection' class for why we ignore this exception.
}
}
}
if (!(binding instanceof ReferenceBinding)) return;
ReferenceBinding rb = (ReferenceBinding) binding;
MethodBinding[] availableMethods = rb.availableMethods();
FieldBinding[] availableFields = rb.availableFields();
failIfContainsAnnotation(binding, availableMethods);
failIfContainsAnnotation(binding, availableFields);
MethodBinding[] parameterizedSigs = availableMethods;
MethodBinding[] baseSigs = parameterizedSigs;
if (binding instanceof ParameterizedTypeBinding) {
baseSigs = ((ParameterizedTypeBinding)binding).genericType().availableMethods();
if (baseSigs.length != parameterizedSigs.length) {
// The last known state of eclipse source says this can't happen, so we rely on it,
// but if this invariant is broken, better to go with 'arg0' naming instead of crashing.
baseSigs = parameterizedSigs;
}
}
for (int i = 0; i < parameterizedSigs.length; i++) {
MethodBinding mb = parameterizedSigs[i];
String sig = printSig(mb);
if (mb.isStatic()) continue;
if (mb.isBridge()) continue;
if (mb.isConstructor()) continue;
if (mb.isDefaultAbstract()) continue;
if (!mb.isPublic()) continue;
if (mb.isSynthetic()) continue;
if (!banList.add(sig)) continue; // If add returns false, it was already in there.
BindingTuple pair = new BindingTuple(mb, baseSigs[i], fieldName, responsible);
list.add(pair);
}
addAllMethodBindings0(list, rb.superclass(), banList, fieldName, responsible);
ReferenceBinding[] interfaces = rb.superInterfaces();
if (interfaces != null) {
for (ReferenceBinding iface : interfaces) addAllMethodBindings0(list, iface, banList, fieldName,
responsible);
}
}
private static Set<String> findAlreadyImplementedMethods(TypeDeclaration decl) {
Set<String> sigs = new HashSet<String>();
```

```
for (AbstractMethodDeclaration md : decl.methods) {
if (md.isStatic()) continue;
if ((md.modifiers & ClassFileConstants.AccBridge) != 0) continue;
if (md.isConstructor()) continue;
if ((md.modifiers & ExtraCompilerModifiers.AccDefaultAbstract) != 0) continue;
if ((md.modifiers & ClassFileConstants.AccPublic) == 0) continue;
if ((md.modifiers & ClassFileConstants.AccSynthetic) != 0) continue;
sigs.add(printSig(md, decl.scope));
}
return sigs;
}
private static final char[] STRING_LOMBOK = new char[] {'l', 'o', 'm', 'b', 'o', 'k'};
private static final char[] STRING_EXPERIMENTAL = new char[] {'e', 'x', 'p', 'e', 'r', 'i', 'm', 'e', 'n', 't', 'a',
'l'};
private static final char[] STRING_DELEGATE = new char[] {'D', 'e', 'l', 'e', 'g', 'a', 't', 'e'};
private static void failIfContainsAnnotation(TypeBinding parent, Binding[] bindings) throws
DelegateRecursion {
if (bindings == null) return;
for (Binding b : bindings) {
AnnotationBinding[] anns = null;
if (b instanceof MethodBinding) anns = ((MethodBinding) b).getAnnotations();
if (b instanceof FieldBinding) anns = ((FieldBinding) b).getAnnotations();
// anns = b.getAnnotations() would make a heck of a lot more sense, but that is a late addition to ecj, so
would cause NoSuchMethodErrors! Don't use that!
if (anns == null) continue;
for (AnnotationBinding ann : anns) {
char[][] name = null;
try {
name = ann.getAnnotationType().compoundName;
} catch (Exception ignore) {}
if (name == null || name.length < 2 || name.length > 3) continue;
if (!Arrays.equals(STRING_LOMBOK, name[0])) continue;
if (!Arrays.equals(STRING_DELEGATE, name[name.length - 1])) continue;
if (name.length == 3 && !Arrays.equals(STRING_EXPERIMENTAL, name[1])) continue;
throw new DelegateRecursion(parent.readableName(), b.readableName());
}
}
}
private static final class BindingTuple {
BindingTuple(MethodBinding parameterized, MethodBinding base, char[] fieldName, ASTNode
responsible) {
this.parameterized = parameterized;
this.base = base;
this.fieldName = fieldName;
this.responsible = responsible;
}
final MethodBinding parameterized, base;
final char[] fieldName;
final ASTNode responsible;
@Override public String toString() {
return String.format("{param: %s, base: %s, fieldName: %s}", parameterized == null ? "(null)" :
printSig(parameterized), base == null ? "(null)" : printSig(base), new String(fieldName));
}
```

```
}
private static final List<String> METHODS_IN_OBJECT = Collections.unmodifiableList(Arrays.asList(
"hashCode()",
"canEqual(java.lang.Object)", //Not in j.l.Object, but it goes with hashCode and equals so if we ignore
those two, we should ignore this one.
"equals(java.lang.Object)",
"wait()",
"wait(long)",
"wait(long, int)",
"notify()",
"notifyAll()",
"toString()",
"getClass()",
"clone()",
"finalize()"));
private static String printSig(MethodBinding binding) {
StringBuilder signature = new StringBuilder();
signature.append(binding.selector);
signature.append("(");
boolean first = true;
if (binding.parameters != null) for (TypeBinding param : binding.parameters) {
if (!first) signature.append(", ");
first = false;
signature.append(typeBindingToSignature(param));
}
signature.append(")");
return signature.toString();
}
private static String printSig(AbstractMethodDeclaration md, ClassScope scope) {
StringBuilder signature = new StringBuilder();
signature.append(md.selector);
signature.append("(");
boolean first = true;
if (md.arguments != null) {
TypeParameter[] typeParameters = md.typeParameters();
Map<String, TypeParameter> typeParametersMap = new HashMap<String, TypeParameter>();
if (typeParameters != null) {
for (TypeParameter typeParameter : typeParameters) {
typeParametersMap.put(new String(typeParameter.name), typeParameter);
}
}
for (Argument argument : md.arguments) {
TypeBinding typeBinding = makeTypeBinding(argument.type, typeParametersMap, scope);
if (!first) signature.append(", ");
first = false;
signature.append(typeBindingToSignature(typeBinding));
}
}
signature.append(")");
return signature.toString();
}
private static TypeBinding makeTypeBinding(TypeReference typeReference, Map<String,
TypeParameter> typeParametersMap, ClassScope scope) {
```

```
char[][] typeName = typeReference.getTypeName();
String typeNameString = Eclipse.toQualifiedName(typeName);
TypeParameter typeParameter = typeParametersMap.get(typeNameString);
if (typeParameter != null) {
if (typeParameter.type != null) {
typeName = typeParameter.type.getTypeName();
} else {
typeName = TypeConstants.JAVA_LANG_OBJECT;
}
}
TypeBinding typeBinding = scope.getType(typeName, typeName.length);
if (typeReference.dimensions() > 0) {
typeBinding = scope.createArrayType(typeBinding, typeReference.dimensions());
}
return typeBinding;
}
private static String typeBindingToSignature(TypeBinding binding) {
binding = binding.erasure();
if (binding != null && binding.isBaseType()) {
return new String (binding.sourceName());
} else if (binding instanceof ReferenceBinding) {
String pkg = binding.qualifiedPackageName() == null ? "" : new
String(binding.qualifiedPackageName());
String qsn = binding.qualifiedSourceName() == null ? "" : new String(binding.qualifiedSourceName());
return pkg.isEmpty() ? qsn : (pkg + "." + qsn);
} else if (binding instanceof ArrayBinding) {
StringBuilder out = new StringBuilder();
out.append(typeBindingToSignature(binding.leafComponentType()));
for (int i = 0; i < binding.dimensions(); i++) out.append("[]");
return out.toString();
}
return "";
}
private static boolean charArrayEquals(String s, char[] c) {
if (s == null) return c == null;
if (c == null) return false;
if (s.length() != c.length) return false;
for (int i = 0; i < s.length(); i++) if (s.charAt(i) != c[i]) return false;
return true;
}
private enum DelegateReceiver {
METHOD {
public Expression get(final ASTNode source, char[] name) {
MessageSend call = new MessageSend();
call.sourceStart = source.sourceStart; call.sourceEnd = source.sourceEnd;
call.nameSourcePosition = pos(source);
setGeneratedBy(call, source);
call.selector = name;
call.receiver = new ThisReference(source.sourceStart, source.sourceEnd);
setGeneratedBy(call.receiver, source);
return call;
}
},
```

```
FIELD {
public Expression get(final ASTNode source, char[] name) {
FieldReference fieldRef = new FieldReference(name, pos(source));
setGeneratedBy(fieldRef, source);
fieldRef.receiver = new ThisReference(source.sourceStart, source.sourceEnd);
setGeneratedBy(fieldRef.receiver, source);
return fieldRef;
}
};
public abstract Expression get(final ASTNode source, char[] name);
}
}
```

lombok/eclipse/agent/PatchDelegatePortal.java

```
/*
* Copyright (C) 2012-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import java.lang.reflect.Method;
import lombok.permit.Permit;
public class PatchDelegatePortal {
static final String CLASS_SCOPE = "org.eclipse.jdt.internal.compiler.lookup.ClassScope";
static final String I_JAVA_ELEMENT_ARRAY = "[Lorg.eclipse.jdt.core.IJavaElement;";
static final String SOURCE_TYPE_ELEMENT_INFO =
"org.eclipse.jdt.internal.core.SourceTypeElementInfo";
public static boolean handleDelegateForType(Object classScope) {
Boolean v = (Boolean) Permit.invokeSneaky(Reflection.problemHandleDelegate,
Reflection.handleDelegateForType, null, classScope);
if (v == null) return false;
return v.booleanValue();
}
public static Object[] addGeneratedDelegateMethods(Object returnValue, Object javaElement) {
```

```
return (Object[]) Permit.invokeSneaky(Reflection.problemAddGeneratedDelegateMethods,
Reflection.addGeneratedDelegateMethods, null, returnValue, javaElement);
}
private static final class Reflection {
public static final Method handleDelegateForType;
public static final Method addGeneratedDelegateMethods;
public static final Throwable problemHandleDelegate;
public static final Throwable problemAddGeneratedDelegateMethods;
static {
Method m = null, n = null;
Throwable problemHandleDelegate_ = null;
Throwable problemAddGeneratedDelegateMethods_ = null;
try {
m = Permit.getMethod(PatchDelegate.class, "handleDelegateForType",
Class.forName(CLASS_SCOPE));
} catch (Throwable t) {
// That's problematic, but as long as no local classes are used we don't actually need it.
// Better fail on local classes than crash altogether.
problemHandleDelegate_ = t;
}
handleDelegateForType = m;
problemHandleDelegate = problemHandleDelegate_;
try {
n = Permit.getMethod(PatchDelegate.class, "addGeneratedDelegateMethods", Object[].class,
Object.class);
} catch (Throwable t) {
// That's problematic, but as long as no local classes are used we don't actually need it.
// Better fail on local classes than crash altogether.
problemAddGeneratedDelegateMethods_ = t;
}
addGeneratedDelegateMethods = n;
problemAddGeneratedDelegateMethods = problemAddGeneratedDelegateMethods_;
}
}
}
```

lombok/eclipse/agent/PatchDiagnostics.java

```
/*
* Copyright (C) 2012-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
```

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
public class PatchDiagnostics {
/**
*
* org.eclipse.jdt.core.dom.ASTNode#setSourceRange starts off doing some sanity checks on the input arguments, and, if these fail the sanity check,
* a {@code IllegalArgumentException} is thrown. However, these IAEs do not have any message and are thus pretty much useless. We do the exact same
* checks, and throw the exact same exception (thus, effectively, we don't change how eclipse operates), but, we <em>do</em> provide a useful message.
*/
public static boolean setSourceRangeCheck(Object astNode, int startPosition, int length) {
String nodeTxt;
if (startPosition >= 0 && length < 0) {
if (astNode == null) nodeTxt = "(NULL NODE)";
else nodeTxt = astNode.getClass() + ": " + astNode.toString();
throw new IllegalArgumentException("startPos = " + startPosition + " and length is " + length + ".\n" +
"This breaks the rule that lengths are not allowed to be negative. Affected Node:\n" + nodeTxt);
}
if (startPosition < 0 && length != 0) {
if (astNode == null) nodeTxt = "(NULL NODE)";
else nodeTxt = astNode.getClass() + ": " + astNode.toString();
throw new IllegalArgumentException("startPos = " + startPosition + " and length is " + length + ".\n" +
"This breaks the rule that length must be 0 if startPosition is negative. Affected Node:\n" + nodeTxt);
}
return false;
}
}
```

lombok/eclipse/agent/PatchExtensionMethod.java

```
/*
* Copyright (C) 2012-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
```

*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.eclipse.agent;
import static lombok.eclipse.handlers.EclipseHandlerUtil.createAnnotation;
import java.lang.ref.WeakReference;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import lombok.core.AST.Kind;
import lombok.core.AnnotationValues;
import lombok.core.AnnotationValues.AnnotationValueDecodeFail;
import lombok.core.FieldAugment;
import lombok.eclipse.EclipseAST;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.TransformEclipseAST;
import lombok.eclipse.handlers.EclipseHandlerUtil;
import lombok.experimental.ExtensionMethod;
import lombok.permit.Permit;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.ClassLiteralAccess;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.MessageSend;
import org.eclipse.jdt.internal.compiler.ast.NameReference;
import org.eclipse.jdt.internal.compiler.ast.QualifiedNameReference;
import org.eclipse.jdt.internal.compiler.ast.SingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.SuperReference;
import org.eclipse.jdt.internal.compiler.ast.ThisReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.impl.ReferenceContext;
import org.eclipse.jdt.internal.compiler.lookup.Binding;
import org.eclipse.jdt.internal.compiler.lookup.BlockScope;
import org.eclipse.jdt.internal.compiler.lookup.MethodBinding;
import org.eclipse.jdt.internal.compiler.lookup.ProblemMethodBinding;
import org.eclipse.jdt.internal.compiler.lookup.ReferenceBinding;
import org.eclipse.jdt.internal.compiler.lookup.Scope;
import org.eclipse.jdt.internal.compiler.lookup.TypeBinding;
import org.eclipse.jdt.internal.compiler.problem.ProblemReporter;
import org.eclipse.jdt.internal.core.search.matching.MethodPattern;
public class PatchExtensionMethod {
```

```
static class Extension {
List<MethodBinding> extensionMethods;
boolean suppressBaseMethods;
}
private static class PostponedNoMethodError implements PostponedError {
private final ProblemReporter problemReporter;
private final WeakReference<MessageSend> messageSendRef;
private final TypeBinding recType;
private final TypeBinding[] params;
PostponedNoMethodError(ProblemReporter problemReporter, MessageSend messageSend,
TypeBinding recType, TypeBinding[] params) {
this.problemReporter = problemReporter;
this.messageSendRef = new WeakReference<MessageSend>(messageSend);
this.recType = recType;
this.params = params;
}
public void fire() {
MessageSend messageSend = messageSendRef.get();
if (messageSend != null) problemReporter.errorNoMethodFor(messageSend, recType, params);
}
}
private static class PostponedInvalidMethodError implements PostponedError {
private final ProblemReporter problemReporter;
private final WeakReference<MessageSend> messageSendRef;
private final MethodBinding method;
private final Scope scope;
private static final Method shortMethod = getMethod("invalidMethod", MessageSend.class,
MethodBinding.class);
private static final Method longMethod = getMethod("invalidMethod", MessageSend.class,
MethodBinding.class, Scope.class);
private static Throwable initProblem;
private static Method getMethod(String name, Class<?>... types) {
try {
return Permit.getMethod(ProblemReporter.class, name, types);
} catch (Exception e) {
initProblem = e;
return null;
}
}
PostponedInvalidMethodError(ProblemReporter problemReporter, MessageSend messageSend,
MethodBinding method, Scope scope) {
this.problemReporter = problemReporter;
this.messageSendRef = new WeakReference<MessageSend>(messageSend);
this.method = method;
this.scope = scope;
}
static void invoke(ProblemReporter problemReporter, MessageSend messageSend, MethodBinding
method, Scope scope) {
if (messageSend != null) {
try {
if (shortMethod != null) Permit.invoke(initProblem, shortMethod, problemReporter, messageSend,
method);
```

```
else if (longMethod != null) Permit.invoke(initProblem, longMethod, problemReporter, messageSend,
method, scope);
else Permit.reportReflectionProblem(initProblem, "method named 'invalidMethod' not found in
ProblemReporter.class");
} catch (IllegalAccessException e) {
throw new RuntimeException(e);
} catch (InvocationTargetException e) {
Throwable t = e.getCause();
if (t instanceof Error) throw (Error) t;
if (t instanceof RuntimeException) throw (RuntimeException) t;
throw new RuntimeException(t);
}
}
}
public void fire() {
MessageSend messageSend = messageSendRef.get();
invoke(problemReporter, messageSend, method, scope);
}
}
private static class PostponedNonStaticAccessToStaticMethodError implements PostponedError {
private final ProblemReporter problemReporter;
private ASTNode location;
private MethodBinding method;
private ReferenceContext referenceContext;
PostponedNonStaticAccessToStaticMethodError(ProblemReporter problemReporter, ASTNode
location, MethodBinding method) {
this.problemReporter = problemReporter;
this.location = location;
this.method = method;
this.referenceContext = problemReporter.referenceContext;
}
public void fire() {
problemReporter.referenceContext = this.referenceContext;
problemReporter.nonStaticAccessToStaticMethod(location, method);
}
}
private static interface PostponedError {
public void fire();
}
public static EclipseNode getTypeNode(TypeDeclaration decl) {
CompilationUnitDeclaration cud = decl.scope.compilationUnitScope().referenceContext;
EclipseAST astNode = TransformEclipseAST.getAST(cud, false);
EclipseNode node = astNode.get(decl);
if (node == null) {
astNode = TransformEclipseAST.getAST(cud, true);
node = astNode.get(decl);
}
return node;
}
public static Annotation getAnnotation(Class<? extends java.lang.annotation.Annotation>
expectedType, EclipseNode node) {
TypeDeclaration decl = (TypeDeclaration) node.get();
if (decl.annotations != null) for (Annotation ann : decl.annotations) {
```

```
if (EclipseHandlerUtil.typeMatches(expectedType, node, ann.type)) return ann;
}
return null;
}
static EclipseNode upToType(EclipseNode typeNode) {
EclipseNode node = typeNode;
do {
node = node.up();
} while ((node != null) && (node.getKind() != Kind.TYPE));
return node;
}
static List<Extension> getApplicableExtensionMethods(EclipseNode typeNode, Annotation ann,
TypeBinding receiverType) {
List<Extension> extensions = new ArrayList<Extension>();
if ((typeNode != null) && (ann != null) && (receiverType != null)) {
BlockScope blockScope = ((TypeDeclaration) typeNode.get()).initializerScope;
EclipseNode annotationNode = typeNode.getNodeFor(ann);
AnnotationValues<ExtensionMethod> annotation = createAnnotation(ExtensionMethod.class,
annotationNode);
boolean suppressBaseMethods = false;
try {
suppressBaseMethods = annotation.getInstance().suppressBaseMethods();
} catch (AnnotationValueDecodeFail fail) {
fail.owner.setError(fail.getMessage(), fail.idx);
}
for (Object extensionMethodProvider : annotation.getActualExpressions("value")) {
if (extensionMethodProvider instanceof ClassLiteralAccess) {
TypeBinding binding = ((ClassLiteralAccess) extensionMethodProvider).type.resolveType(blockScope);
if (binding == null) continue;
if (!binding.isClass() && !binding.isEnum()) continue;
Extension e = new Extension();
e.extensionMethods = getApplicableExtensionMethodsDefinedInProvider(typeNode,
(ReferenceBinding) binding, receiverType);
e.suppressBaseMethods = suppressBaseMethods;
extensions.add(e);
}
}
}
return extensions;
}
private static List<MethodBinding> getApplicableExtensionMethodsDefinedInProvider(EclipseNode
typeNode, ReferenceBinding extensionMethodProviderBinding,
TypeBinding receiverType) {
List<MethodBinding> extensionMethods = new ArrayList<MethodBinding>();
for (MethodBinding method : extensionMethodProviderBinding.methods()) {
if (!method.isStatic()) continue;
if (!method.isPublic()) continue;
if (method.parameters == null || method.parameters.length == 0) continue;
TypeBinding firstArgType = method.parameters[0];
if (receiverType.isProvablyDistinct(firstArgType) &&
!receiverType.isCompatibleWith(firstArgType.erasure())) continue;
extensionMethods.add(method);
}
```

```
return extensionMethods;
}
private static final FieldAugment<MessageSend, PostponedError> MessageSend_postponedErrors =
FieldAugment.augment(MessageSend.class, PostponedError.class, "lombok$postponedErrors");
public static void errorNoMethodFor(ProblemReporter problemReporter, MessageSend messageSend,
TypeBinding recType, TypeBinding[] params) {
MessageSend_postponedErrors.set(messageSend, new PostponedNoMethodError(problemReporter,
messageSend, recType, params));
}
public static void invalidMethod(ProblemReporter problemReporter, MessageSend messageSend,
MethodBinding method) {
MessageSend_postponedErrors.set(messageSend, new
PostponedInvalidMethodError(problemReporter, messageSend, method, null));
}
public static void invalidMethod(ProblemReporter problemReporter, MessageSend messageSend,
MethodBinding method, Scope scope) {
MessageSend_postponedErrors.set(messageSend, new
PostponedInvalidMethodError(problemReporter, messageSend, method, scope));
}
public static void nonStaticAccessToStaticMethod(ProblemReporter problemReporter, ASTNode
location, MethodBinding method, MessageSend messageSend) {
MessageSend_postponedErrors.set(messageSend, new
PostponedNonStaticAccessToStaticMethodError(problemReporter, location, method));
}
public static TypeBinding resolveType(TypeBinding resolvedType, MessageSend methodCall,
BlockScope scope) {
List<Extension> extensions = new ArrayList<Extension>();
TypeDeclaration decl = scope.classScope().referenceContext;
EclipseNode owningType = null;
for (EclipseNode typeNode = getTypeNode(decl); typeNode != null; typeNode = upToType(typeNode))
{
Annotation ann = getAnnotation(ExtensionMethod.class, typeNode);
if (ann != null) {
extensions.addAll(0, getApplicableExtensionMethods(typeNode, ann,
methodCall.receiver.resolvedType));
if (owningType == null) owningType = typeNode;
}
}
boolean skip = false;
if (methodCall.receiver instanceof ThisReference && (((ThisReference)methodCall.receiver).bits &
ASTNode.IsImplicitThis) != 0) skip = true;
if (methodCall.receiver instanceof SuperReference) skip = true;
if (methodCall.receiver instanceof NameReference) {
Binding binding = ((NameReference)methodCall.receiver).binding;
if (binding instanceof TypeBinding) skip = true;
}
// It's impossible to resolve the right method without types
if (Reflection.argumentsHaveErrors != null) {
try {
if ((Boolean) Reflection.argumentsHaveErrors.get(methodCall)) skip = true;
} catch (IllegalAccessException ignore) {
// ignore
}
```

```
}
if (!skip) for (Extension extension : extensions) {
if (!extension.suppressBaseMethods && !(methodCall.binding instanceof ProblemMethodBinding))
continue;
for (MethodBinding extensionMethod : extension.extensionMethods) {
if (!Arrays.equals(methodCall.selector, extensionMethod.selector)) continue;
MessageSend_postponedErrors.clear(methodCall);
if (methodCall.receiver instanceof ThisReference) {
methodCall.receiver.bits &= ~ASTNode.IsImplicitThis;
}
List<Expression> arguments = new ArrayList<Expression>();
arguments.add(methodCall.receiver);
if (methodCall.arguments != null) arguments.addAll(Arrays.asList(methodCall.arguments));
Expression[] originalArgs = methodCall.arguments;
methodCall.arguments = arguments.toArray(new Expression[0]);
List<TypeBinding> argumentTypes = new ArrayList<TypeBinding>();
for (Expression argument : arguments) {
TypeBinding argumentType = argument.resolvedType;
if (argumentType == null && requiresPolyBinding(argument)) {
argumentType = Reflection.getPolyTypeBinding(argument);
}
if (argumentType == null) {
argumentType = TypeBinding.NULL;
}
argumentTypes.add(argumentType);
}
if (methodCall.receiver instanceof MessageSend) {
if (Reflection.inferenceContexts != null) {
try {
Permit.set(Reflection.inferenceContexts, methodCall.receiver, null);
} catch (IllegalAccessException ignore) {
// ignore
}
}
}
MethodBinding fixedBinding = scope.getMethod(extensionMethod.declaringClass,
methodCall.selector, argumentTypes.toArray(new TypeBinding[0]), methodCall);
if (fixedBinding instanceof ProblemMethodBinding) {
methodCall.arguments = originalArgs;
// Sometimes the declaring class is null, in that case we have to create a new ProblemMethodBinding
using the extension method's declaring class
if (fixedBinding.declaringClass == null) {
fixedBinding = new ProblemMethodBinding(fixedBinding.selector, fixedBinding.parameters,
extensionMethod.declaringClass, fixedBinding.problemId());
}
PostponedInvalidMethodError.invoke(scope.problemReporter(), methodCall, fixedBinding, scope);
} else {
// If the extension method uses varargs, the last fixed binding parameter is an array but
// the method arguments are not. Even thought we already know that the method is fine we still
// have to compare each parameter with the type of the array to support autoboxing/unboxing.
boolean isVarargs = fixedBinding.isVarargs();
for (int i = 0, iend = arguments.size(); i < iend; i++) {
Expression arg = arguments.get(i);
```

```
TypeBinding[] parameters = fixedBinding.parameters;
TypeBinding param;
if (isVarargs && i >= parameters.length - 1) {
// Extract the array element type for all vararg arguments
param = parameters[parameters.length - 1].leafComponentType();
} else {
param = parameters[i];
}
// Resolve types for polys
if (requiresPolyBinding(arg)) {
arg.setExpectedType(param);
arg.resolveType(scope);
}
if (arg.resolvedType != null) {
arg.computeConversion(scope, param, arg.resolvedType);
}
}
methodCall.receiver = createNameRef(extensionMethod.declaringClass, methodCall);
methodCall.actualReceiverType = extensionMethod.declaringClass;
methodCall.binding = fixedBinding;
methodCall.resolvedType = methodCall.binding.returnType;
methodCall.statementEnd = methodCall.sourceEnd;
if (Reflection.argumentTypes != null) {
try {
Reflection.argumentTypes.set(methodCall, argumentTypes.toArray(new TypeBinding[0]));
} catch (IllegalAccessException ignore) {
// ignore
}
}
}
return methodCall.resolvedType;
}
}
PostponedError error = MessageSend_postponedErrors.get(methodCall);
if (error != null) error.fire();
MessageSend_postponedErrors.clear(methodCall);
return resolvedType;
}
public static Object modifyMethodPattern(Object original) {
if (original != null && original instanceof MethodPattern) {
MethodPattern methodPattern = (MethodPattern) original;
if (methodPattern.parameterCount > 0) {
methodPattern.varargs = true;
}
}
return original;
}
private static boolean requiresPolyBinding(Expression argument) {
return Reflection.isFunctionalExpression(argument) || argument instanceof ConditionalExpression &&
Reflection.isPolyExpression(argument);
}
private static NameReference createNameRef(TypeBinding typeBinding, ASTNode source) {
long p = ((long) source.sourceStart << 32) | source.sourceEnd;
```

```
char[] pkg = typeBinding.qualifiedPackageName();
char[] basename = typeBinding.qualifiedSourceName();
StringBuilder sb = new StringBuilder();
if (pkg != null) sb.append(pkg);
if (sb.length() > 0) sb.append(".");
sb.append(basename);
String tName = sb.toString();
if (tName.indexOf('.') == -1) {
return new SingleNameReference(basename, p);
} else {
char[][] sources;
String[] in = tName.split("\\.");
sources = new char[in.length][];
for (int i = 0; i < in.length; i++) sources[i] = in[i].toCharArray();
long[] poss = new long[in.length];
Arrays.fill(poss, p);
return new QualifiedNameReference(sources, poss, source.sourceStart, source.sourceEnd);
}
}
private static final class Reflection {
public static final Field argumentTypes = Permit.permissiveGetField(MessageSend.class,
"argumentTypes");
public static final Field argumentsHaveErrors = Permit.permissiveGetField(MessageSend.class,
"argumentsHaveErrors");
public static final Field inferenceContexts = Permit.permissiveGetField(MessageSend.class,
"inferenceContexts");
private static final Method isPolyExpression = Permit.permissiveGetMethod(Expression.class,
"isPolyExpression");
private static final Class<?> functionalExpression;
private static final Constructor<?> polyTypeBindingConstructor;
static {
Class<?> a = null;
Constructor<?> b = null;
try {
a = Class.forName("org.eclipse.jdt.internal.compiler.ast.FunctionalExpression");
} catch (Exception e) {
// Ignore
}
try {
b = Permit.getConstructor(Class.forName("org.eclipse.jdt.internal.compiler.lookup.PolyTypeBinding"),
Expression.class);
} catch (Exception e) {
// Ignore
}
functionalExpression = a;
polyTypeBindingConstructor = b;
}
public static boolean isFunctionalExpression(Expression expression) {
if (functionalExpression == null) return false;
return functionalExpression.isInstance(expression);
}
public static boolean isPolyExpression(Expression expression) {
if (isPolyExpression == null) return false;
```

```
try {
return (Boolean) isPolyExpression.invoke(expression);
} catch (Exception e) {
// Ignore
}
return false;
}
public static TypeBinding getPolyTypeBinding(Expression expression) {
if (polyTypeBindingConstructor == null) return null;
try {
return (TypeBinding) polyTypeBindingConstructor.newInstance(expression);
} catch (Exception e) {
// Ignore
}
return null;
}
}
}
```

lombok/eclipse/agent/PatchExtensionMethodCompletionProposal.java

```
/*
* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import static lombok.eclipse.agent.PatchExtensionMethod.*;
import java.lang.reflect.AccessibleObject;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Arrays;
```

```
import java.util.List;
import lombok.eclipse.EclipseNode;
import lombok.eclipse.agent.PatchExtensionMethod.Extension;
import lombok.experimental.ExtensionMethod;
import lombok.permit.Permit;
import org.eclipse.jdt.core.CompletionProposal;
import org.eclipse.jdt.internal.codeassist.InternalCompletionContext;
import org.eclipse.jdt.internal.codeassist.InternalCompletionProposal;
import org.eclipse.jdt.internal.codeassist.InternalExtendedCompletionContext;
import org.eclipse.jdt.internal.codeassist.complete.CompletionOnMemberAccess;
import org.eclipse.jdt.internal.codeassist.complete.CompletionOnQualifiedNameReference;
import org.eclipse.jdt.internal.codeassist.complete.CompletionOnSingleNameReference;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.FieldReference;
import org.eclipse.jdt.internal.compiler.ast.NameReference;
import org.eclipse.jdt.internal.compiler.ast.SuperReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.lookup.Binding;
import org.eclipse.jdt.internal.compiler.lookup.ClassScope;
import org.eclipse.jdt.internal.compiler.lookup.LookupEnvironment;
import org.eclipse.jdt.internal.compiler.lookup.MethodBinding;
import org.eclipse.jdt.internal.compiler.lookup.Scope;
import org.eclipse.jdt.internal.compiler.lookup.TypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.VariableBinding;
import org.eclipse.jdt.internal.core.SearchableEnvironment;
import org.eclipse.jdt.internal.ui.text.java.AbstractJavaCompletionProposal;
import org.eclipse.jdt.ui.text.java.CompletionProposalCollector;
import org.eclipse.jdt.ui.text.java.IJavaCompletionProposal;
public class PatchExtensionMethodCompletionProposal {
public static IJavaCompletionProposal[] getJavaCompletionProposals(IJavaCompletionProposal[]
javaCompletionProposals,
CompletionProposalCollector completionProposalCollector) {
List<IJavaCompletionProposal> proposals = new
ArrayList<IJavaCompletionProposal>(Arrays.asList(javaCompletionProposals));
if (canExtendCodeAssist()) {
for (Extension extension : getExtensionMethods(completionProposalCollector)) {
for (MethodBinding method : extension.extensionMethods) {
if (!isMatchingProposal(method, completionProposalCollector)) {
continue;
}
ExtensionMethodCompletionProposal newProposal = new ExtensionMethodCompletionProposal(0);
copyNameLookupAndCompletionEngine(completionProposalCollector, newProposal);
ASTNode node = getAssistNode(completionProposalCollector);
newProposal.setMethodBinding(method, node);
createAndAddJavaCompletionProposal(completionProposalCollector, newProposal, proposals);
}
}
}
return proposals.toArray(new IJavaCompletionProposal[0]);
}
private static List<Extension> getExtensionMethods(CompletionProposalCollector
completionProposalCollector) {
```

```
List<Extension> extensions = new ArrayList<Extension>();
ClassScope classScope = getClassScope(completionProposalCollector);
if (classScope != null) {
TypeDeclaration decl = classScope.referenceContext;
TypeBinding firstParameterType = getFirstParameterType(decl, completionProposalCollector);
for (EclipseNode typeNode = getTypeNode(decl); typeNode != null; typeNode = upToType(typeNode))
{
Annotation ann = getAnnotation(ExtensionMethod.class, typeNode);
extensions.addAll(0, getApplicableExtensionMethods(typeNode, ann, firstParameterType));
}
}
return extensions;
}
private static boolean isMatchingProposal(MethodBinding method, CompletionProposalCollector
completionProposalCollector) {
try {
InternalCompletionContext context = (InternalCompletionContext)
Reflection.contextField.get(completionProposalCollector);
String searchToken = new String(context.getToken());
String extensionMethodName = new String(method.selector);
return extensionMethodName.contains(searchToken);
} catch (IllegalAccessException e) {
return true;
}
}
static TypeBinding getFirstParameterType(TypeDeclaration decl, CompletionProposalCollector
completionProposalCollector) {
TypeBinding firstParameterType = null;
ASTNode node = getAssistNode(completionProposalCollector);
if (node == null) return null;
if (!(node instanceof CompletionOnQualifiedNameReference) && !(node instanceof
CompletionOnSingleNameReference) && !(node instanceof CompletionOnMemberAccess)) return null;
// Never offer on 'super.<autocomplete>'.
if (node instanceof FieldReference && ((FieldReference)node).receiver instanceof SuperReference)
return null;
if (node instanceof NameReference) {
Binding binding = ((NameReference) node).binding;
// Unremark next block to allow a 'blank' autocomplete to list any extensions that apply to the current
scope, but make sure we're not in a static context first, which this doesn't do.
// Lacking good use cases, and having this particular concept be a little tricky on javac, means for now
we don't support extension methods like this. this.X() will be fine, though.
/* if ((node instanceof SingleNameReference) && (((SingleNameReference) node).token.length == 0)) {
firstParameterType = decl.binding;
} else */if (binding instanceof VariableBinding) {
firstParameterType = ((VariableBinding) binding).type;
}
} else if (node instanceof FieldReference) {
firstParameterType = ((FieldReference) node).actualReceiverType;
}
return firstParameterType;
}
private static ASTNode getAssistNode(CompletionProposalCollector completionProposalCollector) {
try {
```

```
InternalCompletionContext context = (InternalCompletionContext)
Reflection.contextField.get(completionProposalCollector);
InternalExtendedCompletionContext extendedContext = (InternalExtendedCompletionContext)
Reflection.extendedContextField.get(context);
if (extendedContext == null) return null;
return (ASTNode) Reflection.assistNodeField.get(extendedContext);
} catch (Exception ignore) {
return null;
}
}
private static ClassScope getClassScope(CompletionProposalCollector completionProposalCollector) {
ClassScope scope = null;
try {
InternalCompletionContext context = (InternalCompletionContext)
Reflection.contextField.get(completionProposalCollector);
InternalExtendedCompletionContext extendedContext = (InternalExtendedCompletionContext)
Reflection.extendedContextField.get(context);
if (extendedContext != null) {
Scope assistScope = ((Scope) Reflection.assistScopeField.get(extendedContext));
if (assistScope != null) {
scope = assistScope.classScope();
}
}
} catch (IllegalAccessException ignore) {
// ignore
}
return scope;
}
private static void copyNameLookupAndCompletionEngine(CompletionProposalCollector
completionProposalCollector, InternalCompletionProposal newProposal) {
try {
InternalCompletionContext context = (InternalCompletionContext)
Reflection.contextField.get(completionProposalCollector);
InternalExtendedCompletionContext extendedContext = (InternalExtendedCompletionContext)
Reflection.extendedContextField.get(context);
LookupEnvironment lookupEnvironment = (LookupEnvironment)
Reflection.lookupEnvironmentField.get(extendedContext);
Reflection.nameLookupField.set(newProposal, ((SearchableEnvironment)
lookupEnvironment.nameEnvironment).nameLookup);
Reflection.completionEngineField.set(newProposal, lookupEnvironment.typeRequestor);
} catch (IllegalAccessException ignore) {
// ignore
}
}
private static void createAndAddJavaCompletionProposal(CompletionProposalCollector
completionProposalCollector, CompletionProposal newProposal,
List<IJavaCompletionProposal> proposals) {
try {
proposals.add((IJavaCompletionProposal)
Reflection.createJavaCompletionProposalMethod.invoke(completionProposalCollector, newProposal));
} catch (Exception ignore) {
// ignore
}
```

```
}
private static boolean canExtendCodeAssist() {
return Reflection.isComplete();
}
static class Reflection {
public static final Field replacementOffsetField;
public static final Field contextField;
public static final Field extendedContextField;
public static final Field assistNodeField;
public static final Field assistScopeField;
public static final Field lookupEnvironmentField;
public static final Field completionEngineField;
public static final Field nameLookupField;
public static final Method createJavaCompletionProposalMethod;
static {
replacementOffsetField = accessField(AbstractJavaCompletionProposal.class, "fReplacementOffset");
contextField = accessField(CompletionProposalCollector.class, "fContext");
extendedContextField = accessField(InternalCompletionContext.class, "extendedContext");
assistNodeField = accessField(InternalExtendedCompletionContext.class, "assistNode");
assistScopeField = accessField(InternalExtendedCompletionContext.class, "assistScope");
lookupEnvironmentField = accessField(InternalExtendedCompletionContext.class,
"lookupEnvironment");
completionEngineField = accessField(InternalCompletionProposal.class, "completionEngine");
nameLookupField = accessField(InternalCompletionProposal.class, "nameLookup");
createJavaCompletionProposalMethod = accessMethod(CompletionProposalCollector.class,
"createJavaCompletionProposal", CompletionProposal.class);
}
static boolean isComplete() {
Object[] requiredFieldsAndMethods = { replacementOffsetField, contextField, extendedContextField,
assistNodeField, assistScopeField, lookupEnvironmentField, completionEngineField,
nameLookupField, createJavaCompletionProposalMethod };
for (Object o : requiredFieldsAndMethods) if (o == null) return false;
return true;
}
private static Field accessField(Class<?> clazz, String fieldName) {
try {
return makeAccessible(clazz.getDeclaredField(fieldName));
} catch (Exception e) {
return null;
}
}
private static Method accessMethod(Class<?> clazz, String methodName, Class<?> parameter) {
try {
return makeAccessible(clazz.getDeclaredMethod(methodName, parameter));
} catch (Exception e) {
return null;
}
}
private static <T extends AccessibleObject> T makeAccessible(T object) {
return Permit.setAccessible(object);
}
}
}
```

lombok/eclipse/agent/PatchExtensionMethodCompletionProposalPortal.java

```
/*
* Copyright (C) 2012 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import lombok.Lombok;
import org.eclipse.jdt.ui.text.java.IJavaCompletionProposal;
public class PatchExtensionMethodCompletionProposalPortal {
private static final String COMPLETION_PROPOSAL_COLLECTOR =
"org.eclipse.jdt.ui.text.java.CompletionProposalCollector";
private static final String I_JAVA_COMPLETION_PROPOSAL_ARRAY =
"[Lorg.eclipse.jdt.ui.text.java.IJavaCompletionProposal;";
public static IJavaCompletionProposal[] getJavaCompletionProposals(Object[]
javaCompletionProposals, Object completionProposalCollector) {
try {
return (IJavaCompletionProposal[]) ReflectionForUi.getJavaCompletionProposals.invoke(null,
javaCompletionProposals, completionProposalCollector);
} catch (NoClassDefFoundError e) {
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
return (IJavaCompletionProposal[]) javaCompletionProposals;
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
} catch (NullPointerException e) {
if (!"false".equals(System.getProperty("lombok.debug.reflection", "false"))) {
```

```
e.initCause(ReflectionForUi.problem);
throw e;
}
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
return (IJavaCompletionProposal[]) javaCompletionProposals;
}
}
private static final class ReflectionForUi {
public static final Method getJavaCompletionProposals;
public static final Throwable problem;
static {
Method p = null;
Throwable problem_ = null;
try {
p = PatchExtensionMethodCompletionProposal.class.getMethod("getJavaCompletionProposals",
Class.forName(I_JAVA_COMPLETION_PROPOSAL_ARRAY),
Class.forName(COMPLETION_PROPOSAL_COLLECTOR));
} catch (Throwable t) {
// That's problematic, but as long as no local classes are used we don't actually need it.
// Better fail on local classes than crash altogether.
problem_ = t;
}
getJavaCompletionProposals = p;
problem = problem_;
}
}
}
```

lombok/eclipse/agent/PatchFixesShadowLoaded.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
```

```
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import java.io.BufferedOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import lombok.core.DiagnosticsReceiver;
import lombok.core.PostCompiler;
import lombok.core.Version;
public class PatchFixesShadowLoaded {
public static String addLombokNotesToEclipseAboutDialog(String origReturnValue, String key) {
if ("aboutText".equals(key)) {
if (origReturnValue.contains(" is installed. https://projectlombok.org")) return origReturnValue;
return origReturnValue + "\n\nLombok " + Version.getFullVersion() + " is installed.
https://projectlombok.org/";
}
return origReturnValue;
}
public static byte[] runPostCompiler(byte[] bytes, String fileName) {
byte[] transformed = PostCompiler.applyTransformations(bytes, fileName,
DiagnosticsReceiver.CONSOLE);
return transformed == null ? bytes : transformed;
}
public static OutputStream runPostCompiler(OutputStream out) throws IOException {
return PostCompiler.wrapOutputStream(out, "TEST", DiagnosticsReceiver.CONSOLE);
}
public static BufferedOutputStream runPostCompiler(BufferedOutputStream out, String path, String
name) throws IOException {
String fileName = path + "/" + name;
return new BufferedOutputStream(PostCompiler.wrapOutputStream(out, fileName,
DiagnosticsReceiver.CONSOLE));
}
}
```

lombok/eclipse/agent/PatchJavadoc.java

```
/*
* Copyright (C) 2020-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import static lombok.eclipse.EcjAugments.CompilationUnit_javadoc;
import java.lang.reflect.Method;
import java.util.Map;
import org.eclipse.jdt.core.ICompilationUnit;
import org.eclipse.jdt.core.IJavaElement;
import org.eclipse.jdt.core.IMember;
import org.eclipse.jdt.internal.core.CompilationUnit;
import org.eclipse.jdt.internal.core.SourceMethod;
import org.eclipse.jdt.internal.ui.text.javadoc.JavadocContentAccess2;
import lombok.permit.Permit;
public class PatchJavadoc {
public static String getHTMLContentFromSource(Object instance, String original, Object member) {
if (original != null) {
return original;
}
if (member instanceof SourceMethod) {
SourceMethod sourceMethod = (SourceMethod) member;
ICompilationUnit iCompilationUnit = sourceMethod.getCompilationUnit();
if (iCompilationUnit instanceof CompilationUnit) {
CompilationUnit compilationUnit = (CompilationUnit) iCompilationUnit;
Map<String, String> docs = CompilationUnit_javadoc.get(compilationUnit);
if (docs == null) return null;
String signature = Signature.getSignature(sourceMethod);
String rawJavadoc = docs.get(signature);
if (rawJavadoc == null) return null;
return Reflection.javadoc2HTML(instance, (IMember) member, (IJavaElement) member, rawJavadoc);
}
}
return null;
}
private static class Signature {
static final String getSignature(SourceMethod sourceMethod) {
StringBuilder sb = new StringBuilder();
sb.append(sourceMethod.getParent().getElementName());
sb.append(".");
sb.append(sourceMethod.getElementName());
sb.append("(");
for (String type : sourceMethod.getParameterTypes()) {
sb.append(org.eclipse.jdt.core.Signature.toString(type));
}
sb.append(")");
return sb.toString();
}
}
/**
```

```
* The method <code>javadoc2HTML</code> changed 2014-12 to accept an
* additional IJavaElement parameter. To support older versions, try to
* find that one too.
*/
private static class Reflection {
private static final Method javadoc2HTML;
private static final Method oldJavadoc2HTML;
private static final Method reallyOldJavadoc2HTML;
private static final Method lsJavadoc2HTML;
static {
Method a = null, b = null, c = null, d = null;
try {
a = Permit.getMethod(JavadocContentAccess2.class, "javadoc2HTML", IMember.class,
IJavaElement.class, String.class);
} catch (Throwable t) {}
try {
b = Permit.getMethod(JavadocContentAccess2.class, "javadoc2HTML", IMember.class, String.class);
} catch (Throwable t) {}
try {
c =
Permit.getMethod(Class.forName("org.eclipse.jdt.ls.core.internal.javadoc.JavadocContentAccess2"),
"javadoc2HTML", IMember.class, IJavaElement.class, String.class);
} catch (Throwable t) {}
try {
d = Permit.getMethod(Class.forName("org.eclipse.jdt.core.manipulation.internal.javadoc.CoreJavadoc
Access"), "javadoc2HTML", IMember.class, IJavaElement.class, String.class);
} catch (Throwable t) {}
oldJavadoc2HTML = a;
reallyOldJavadoc2HTML = b;
lsJavadoc2HTML = c;
javadoc2HTML = d;
}
private static String javadoc2HTML(Object instance, IMember member, IJavaElement element, String
rawJavadoc) {
if (javadoc2HTML != null) {
try {
return (String) javadoc2HTML.invoke(instance, member, element, rawJavadoc);
} catch (Throwable t) {
return null;
}
}
if (oldJavadoc2HTML != null) {
try {
return (String) oldJavadoc2HTML.invoke(instance, member, element, rawJavadoc);
} catch (Throwable t) {
return null;
}
}
if (lsJavadoc2HTML != null) {
try {
return (String) lsJavadoc2HTML.invoke(instance, member, element, rawJavadoc);
} catch (Throwable t) {
return null;
```

```
}
}
if (reallyOldJavadoc2HTML != null) {
try {
return (String) reallyOldJavadoc2HTML.invoke(instance, member, rawJavadoc);
} catch (Throwable t) {
return null;
}
}
return null;
}
}
}
```

lombok/eclipse/agent/PatchVal.java

```
/*
* Copyright (C) 2010-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import lombok.permit.Permit;
import org.eclipse.jdt.core.compiler.CategorizedProblem;
import org.eclipse.jdt.internal.compiler.CompilationResult;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.ast.ConditionalExpression;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.ForeachStatement;
import org.eclipse.jdt.internal.compiler.ast.FunctionalExpression;
import org.eclipse.jdt.internal.compiler.ast.ImportReference;
import org.eclipse.jdt.internal.compiler.ast.LambdaExpression;
```

```
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.classfmt.ClassFileConstants;
import org.eclipse.jdt.internal.compiler.impl.ReferenceContext;
import org.eclipse.jdt.internal.compiler.lookup.ArrayBinding;
import org.eclipse.jdt.internal.compiler.lookup.Binding;
import org.eclipse.jdt.internal.compiler.lookup.BlockScope;
import org.eclipse.jdt.internal.compiler.lookup.CompilationUnitScope;
import org.eclipse.jdt.internal.compiler.lookup.ImportBinding;
import org.eclipse.jdt.internal.compiler.lookup.ParameterizedTypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.ReferenceBinding;
import org.eclipse.jdt.internal.compiler.lookup.Scope;
import org.eclipse.jdt.internal.compiler.lookup.TypeBinding;
import org.eclipse.jdt.internal.compiler.lookup.TypeConstants;
import org.eclipse.jdt.internal.compiler.lookup.TypeIds;
import org.eclipse.jdt.internal.compiler.problem.AbortCompilation;
import java.lang.reflect.Field;
import static lombok.Lombok.sneakyThrow;
import static lombok.eclipse.Eclipse.*;
import static lombok.eclipse.handlers.EclipseHandlerUtil.*;
import static org.eclipse.jdt.core.compiler.CategorizedProblem.CAT_TYPE;
public class PatchVal {
// This is half of the work for 'val' support - the other half is in PatchValEclipse. This half is enough for
ecj.
// Creates a copy of the 'initialization' field on a LocalDeclaration if the type of the LocalDeclaration is
'val', because the completion parser will null this out,
// which in turn stops us from inferring the intended type for 'val x = 5;'. We look at the copy.
// Also patches .resolve() on LocalDeclaration itself to just-in-time replace the 'val' vartype with the right
one.
public static boolean matches(String key, char[] array) {
if (array == null || key.length() != array.length) return false;
for (int i = 0; i < array.length; i++) {
if (key.charAt(i) != array[i]) return false;
}
return true;
}
public static boolean couldBe(ImportBinding[] imports, String key, TypeReference ref) {
String[] keyParts = key.split("\\.");
if (ref instanceof SingleTypeReference) {
char[] token = ((SingleTypeReference)ref).token;
if (!matches(keyParts[keyParts.length - 1], token)) return false;
if (imports == null) return true;
top:
for (ImportBinding ib : imports) {
ImportReference ir = ib.reference;
if (ir == null) continue;
if (ir.isStatic()) continue;
boolean star = ((ir.bits & ASTNode.OnDemand) != 0);
int len = keyParts.length - (star ? 1 : 0);
char[][] t = ir.tokens;
```

```
if (len != t.length) continue;
for (int i = 0; i < len; i++) {
if (keyParts[i].length() != t[i].length) continue top;
for (int j = 0; j < t[i].length; j++) if (keyParts[i].charAt(j) != t[i][j]) continue top;
}
return true;
}
return false;
}
if (ref instanceof QualifiedTypeReference) {
char[][] tokens = ((QualifiedTypeReference)ref).tokens;
if (keyParts.length != tokens.length) return false;
for(int i = 0; i < tokens.length; ++i) {
String part = keyParts[i];
char[] token = tokens[i];
if (!matches(part, token)) return false;
}
return true;
}
return false;
}
public static boolean couldBe(ImportReference[] imports, String key, TypeReference ref) {
String[] keyParts = key.split("\\.");
if (ref instanceof SingleTypeReference) {
char[] token = ((SingleTypeReference) ref).token;
if (!matches(keyParts[keyParts.length - 1], token)) return false;
if (imports == null) return true;
top:
for (ImportReference ir : imports) {
if (ir.isStatic()) continue;
boolean star = ((ir.bits & ASTNode.OnDemand) != 0);
int len = keyParts.length - (star ? 1 : 0);
char[][] t = ir.tokens;
if (len != t.length) continue;
for (int i = 0; i < len; i++) {
if (keyParts[i].length() != t[i].length) continue top;
for (int j = 0; j < t[i].length; j++) if (keyParts[i].charAt(j) != t[i][j]) continue top;
}
return true;
}
return false;
}
if (ref instanceof QualifiedTypeReference) {
char[][] tokens = ((QualifiedTypeReference) ref).tokens;
if (keyParts.length != tokens.length) return false;
for(int i = 0; i < tokens.length; ++i) {
String part = keyParts[i];
char[] token = tokens[i];
if (!matches(part, token)) return false;
}
return true;
}
return false;
```

```
}
private static boolean is(TypeReference ref, BlockScope scope, String key) {
Scope s = scope.parent;
while (s != null && !(s instanceof CompilationUnitScope)) {
Scope ns = s.parent;
s = ns == s ? null : ns;
}
ImportBinding[] imports = null;
if (s instanceof CompilationUnitScope) imports = ((CompilationUnitScope) s).imports;
if (!couldBe(imports, key, ref)) return false;
TypeBinding resolvedType = ref.resolvedType;
if (resolvedType == null) resolvedType = ref.resolveType(scope, false);
if (resolvedType == null) return false;
char[] pkg = resolvedType.qualifiedPackageName();
char[] nm = resolvedType.qualifiedSourceName();
int pkgFullLength = pkg.length > 0 ? pkg.length + 1: 0;
char[] fullName = new char[pkgFullLength + nm.length];
if(pkg.length > 0) {
System.arraycopy(pkg, 0, fullName, 0, pkg.length);
fullName[pkg.length] = '.';
}
System.arraycopy(nm, 0, fullName, pkgFullLength, nm.length);
return matches(key, fullName);
}
public static final class Reflection {
private static final Field initCopyField, iterableCopyField;
static {
Field a = null, b = null;
try {
a = Permit.getField(LocalDeclaration.class, "$initCopy");
b = Permit.getField(LocalDeclaration.class, "$iterableCopy");
} catch (Throwable t) {
//ignore - no $initCopy exists when running in ecj.
}
initCopyField = a;
iterableCopyField = b;
}
}
public static boolean handleValForLocalDeclaration(LocalDeclaration local, BlockScope scope) {
if (local == null || !LocalDeclaration.class.equals(local.getClass())) return false;
boolean decomponent = false;
boolean val = isVal(local, scope);
boolean var = isVar(local, scope);
if (!(val || var)) return false;
if (val) {
StackTraceElement[] st = new Throwable().getStackTrace();
for (int i = 0; i < st.length - 2 && i < 10; i++) {
if (st[i].getClassName().equals("lombok.launch.PatchFixesHider$Val")) {
boolean valInForStatement =
st[i + 1].getClassName().equals("org.eclipse.jdt.internal.compiler.ast.LocalDeclaration") &&
st[i + 2].getClassName().equals("org.eclipse.jdt.internal.compiler.ast.ForStatement");
if (valInForStatement) return false;
break;
```

```
}
}
}
Expression init = local.initialization;
if (init == null && Reflection.initCopyField != null) {
try {
init = (Expression) Reflection.initCopyField.get(local);
} catch (Exception e) {
// init remains null.
}
}
if (init == null && Reflection.iterableCopyField != null) {
try {
init = (Expression) Reflection.iterableCopyField.get(local);
decomponent = true;
} catch (Exception e) {
// init remains null.
}
}
TypeReference replacement = null;
// Java 10+: Lombok uses the native 'var' support and transforms 'val' to 'final var'.
if (hasNativeVarSupport(scope) && val) {
replacement = new SingleTypeReference("var".toCharArray(), pos(local.type));
local.initialization = init;
init = null;
}
if (init != null) {
if (init.getClass().getName().equals("org.eclipse.jdt.internal.compiler.ast.LambdaExpression")) {
return false;
}
TypeBinding resolved = null;
try {
resolved = decomponent ? getForEachComponentType(init, scope) : resolveForExpression(init,
scope);
} catch (NullPointerException e) {
// This definitely occurs if as part of resolving the initializer expression, a
// lambda expression in it must also be resolved (such as when lambdas are part of
// a ternary expression). This can't result in a viable 'val' matching, so, we
// just go with 'Object' and let the IDE print the appropriate errors.
resolved = null;
}
if (resolved == null) {
if (init instanceof ConditionalExpression) {
ConditionalExpression cexp = (ConditionalExpression) init;
Expression ifTrue = cexp.valueIfTrue;
Expression ifFalse = cexp.valueIfFalse;
TypeBinding ifTrueResolvedType = ifTrue.resolvedType;
CompilationResult compilationResult = scope.referenceCompilationUnit().compilationResult;
CategorizedProblem[] problems = compilationResult.problems;
CategorizedProblem lastProblem = problems[compilationResult.problemCount - 1];
if (ifTrueResolvedType != null && ifFalse.resolvedType == null && lastProblem.getCategoryID() ==
CAT_TYPE) {
int problemCount = compilationResult.problemCount;
```

```
for (int i = 0; i < problemCount; ++i) {
if (problems[i] == lastProblem) {
problems[i] = null;
if (i + 1 < problemCount) {
System.arraycopy(problems, i + 1, problems, i, problemCount - i + 1);
}
break;
}
}
compilationResult.removeProblem(lastProblem);
if (!compilationResult.hasErrors()) {
clearIgnoreFurtherInvestigationField(scope.referenceContext());
setValue(getField(CompilationResult.class, "hasMandatoryErrors"), compilationResult, false);
}
if (ifFalse instanceof FunctionalExpression) {
FunctionalExpression functionalExpression = (FunctionalExpression) ifFalse;
functionalExpression.setExpectedType(ifTrueResolvedType);
}
if (ifFalse.resolvedType == null) {
resolveForExpression(ifFalse, scope);
}
resolved = ifTrueResolvedType;
}
}
}
if (resolved != null) {
try {
replacement = makeType(resolved, local.type, false);
if (!decomponent) init.resolvedType = replacement.resolveType(scope);
} catch (Exception e) {
// Some type thing failed.
}
}
}
if (val) local.modifiers |= ClassFileConstants.AccFinal;
local.annotations = addValAnnotation(local.annotations, local.type, scope);
local.type = replacement != null ? replacement : new
QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT, poss(local.type, 3));
return false;
}
private static boolean isVar(LocalDeclaration local, BlockScope scope) {
return is(local.type, scope, "lombok.experimental.var") || is(local.type, scope, "lombok.var");
}
private static boolean isVal(LocalDeclaration local, BlockScope scope) {
return is(local.type, scope, "lombok.val");
}
private static boolean hasNativeVarSupport(Scope scope) {
long sl = scope.problemReporter().options.sourceLevel >> 16;
long cl = scope.problemReporter().options.complianceLevel >> 16;
if (sl == 0) sl = cl;
if (cl == 0) cl = sl;
return Math.min((int)(sl - 44), (int)(cl - 44)) >= 10;
}
```

```
public static boolean handleValForForEach(ForeachStatement forEach, BlockScope scope) {
if (forEach.elementVariable == null) return false;
boolean val = isVal(forEach.elementVariable, scope);
boolean var = isVar(forEach.elementVariable, scope);
if (!(val || var)) return false;
if (hasNativeVarSupport(scope)) return false;
TypeBinding component = getForEachComponentType(forEach.collection, scope);
if (component == null) return false;
TypeReference replacement = makeType(component, forEach.elementVariable.type, false);
if (val) forEach.elementVariable.modifiers |= ClassFileConstants.AccFinal;
forEach.elementVariable.annotations = addValAnnotation(forEach.elementVariable.annotations,
forEach.elementVariable.type, scope);
forEach.elementVariable.type = replacement != null ? replacement :
new QualifiedTypeReference(TypeConstants.JAVA_LANG_OBJECT,
poss(forEach.elementVariable.type, 3));
return false;
}
private static Annotation[] addValAnnotation(Annotation[] originals, TypeReference originalRef,
BlockScope scope) {
Annotation[] newAnn;
if (originals != null) {
newAnn = new Annotation[1 + originals.length];
System.arraycopy(originals, 0, newAnn, 0, originals.length);
} else {
newAnn = new Annotation[1];
}
TypeReference qualifiedTypeRef = generateQualifiedTypeRef(originalRef,
originalRef.getTypeName());
newAnn[newAnn.length - 1] = new
org.eclipse.jdt.internal.compiler.ast.MarkerAnnotation(qualifiedTypeRef, qualifiedTypeRef.sourceStart);
return newAnn;
}
private static TypeBinding getForEachComponentType(Expression collection, BlockScope scope) {
if (collection != null) {
TypeBinding resolved = collection.resolvedType;
if (resolved == null) resolved = resolveForExpression(collection, scope);
if (resolved == null) return null;
if (resolved.isArrayType()) {
resolved = ((ArrayBinding) resolved).elementsType();
return resolved;
} else if (resolved instanceof ReferenceBinding) {
ReferenceBinding iterableType = ((ReferenceBinding)
resolved).findSuperTypeOriginatingFrom(TypeIds.T_JavaLangIterable, false);
TypeBinding[] arguments = null;
if (iterableType != null) switch (iterableType.kind()) {
case Binding.GENERIC_TYPE : // for (T t : Iterable<T>) - in case used inside Iterable itself
arguments = iterableType.typeVariables();
break;
case Binding.PARAMETERIZED_TYPE : // for(E e : Iterable<E>)
arguments = ((ParameterizedTypeBinding)iterableType).arguments;
break;
case Binding.RAW_TYPE : // for(Object e : Iterable)
return null;
```

```
}
if (arguments != null && arguments.length == 1) {
return arguments[0];
}
}
}
return null;
}
private static TypeBinding resolveForExpression(Expression collection, BlockScope scope) {
try {
return collection.resolveType(scope);
} catch (ArrayIndexOutOfBoundsException e) {
// Known cause of issues; for example: val e = mth("X"), where mth takes 2 arguments.
return null;
} catch (AbortCompilation e) {
return null;
}
}
private static void clearIgnoreFurtherInvestigationField(ReferenceContext currentContext) {
if (currentContext instanceof AbstractMethodDeclaration) {
AbstractMethodDeclaration methodDeclaration = (AbstractMethodDeclaration) currentContext;
methodDeclaration.ignoreFurtherInvestigation = false;
} else if (currentContext instanceof LambdaExpression) {
LambdaExpression lambdaExpression = (LambdaExpression) currentContext;
setValue(getField(LambdaExpression.class, "ignoreFurtherInvestigation"), lambdaExpression, false);
Scope parent = lambdaExpression.enclosingScope.parent;
while (parent != null) {
switch(parent.kind) {
case Scope.CLASS_SCOPE:
case Scope.METHOD_SCOPE:
ReferenceContext parentAST = parent.referenceContext();
if (parentAST != lambdaExpression) {
clearIgnoreFurtherInvestigationField(parentAST);
return;
}
default:
parent = parent.parent;
break;
}
}
} else if (currentContext instanceof TypeDeclaration) {
TypeDeclaration typeDeclaration = (TypeDeclaration) currentContext;
typeDeclaration.ignoreFurtherInvestigation = false;
} else if (currentContext instanceof CompilationUnitDeclaration) {
CompilationUnitDeclaration typeDeclaration = (CompilationUnitDeclaration) currentContext;
typeDeclaration.ignoreFurtherInvestigation = false;
} else {
throw new UnsupportedOperationException("clearIgnoreFurtherInvestigationField for " +
currentContext.getClass());
}
}
private static void setValue(Field field, Object object, Object value) {
try {
```

```
field.set(object, value);
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
}
}
private static Field getField(Class<?> clazz, String name) {
try {
return Permit.getField(clazz, name);
} catch (NoSuchFieldException e) {
throw sneakyThrow(e);
}
}
}
```

lombok/eclipse/agent/PatchValEclipse.java

```
/*
* Copyright (C) 2010-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*
* Thanks to Stephen Haberman for a patch to solve some NPEs in Eclipse.
*/
package lombok.eclipse.agent;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.List;
import lombok.Lombok;
import lombok.permit.Permit;
import org.eclipse.jdt.core.dom.AST;
import org.eclipse.jdt.core.dom.IExtendedModifier;
import org.eclipse.jdt.core.dom.MarkerAnnotation;
import org.eclipse.jdt.core.dom.Modifier;
```

```
import org.eclipse.jdt.core.dom.Modifier.ModifierKeyword;
import org.eclipse.jdt.core.dom.Name;
import org.eclipse.jdt.core.dom.QualifiedName;
import org.eclipse.jdt.core.dom.SimpleName;
import org.eclipse.jdt.core.dom.SingleVariableDeclaration;
import org.eclipse.jdt.core.dom.VariableDeclarationStatement;
import org.eclipse.jdt.internal.compiler.ast.ASTNode;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.ForeachStatement;
import org.eclipse.jdt.internal.compiler.ast.ImportReference;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.QualifiedTypeReference;
import org.eclipse.jdt.internal.compiler.ast.SingleTypeReference;
import org.eclipse.jdt.internal.compiler.ast.TypeReference;
import org.eclipse.jdt.internal.compiler.parser.Parser;
public class PatchValEclipse {
public static void copyInitializationOfForEachIterable(Parser parser) {
ASTNode[] astStack;
int astPtr;
try {
astStack = (ASTNode[]) Reflection.astStackField.get(parser);
astPtr = (Integer) Reflection.astPtrField.get(parser);
} catch (Exception e) {
// Most likely we're in ecj or some other plugin usage of the eclipse compiler. No need for this.
return;
}
ForeachStatement foreachDecl = (ForeachStatement) astStack[astPtr];
ASTNode init = foreachDecl.collection;
if (init == null) return;
boolean val = couldBeVal(parser == null ? null : parser.compilationUnit == null ? null :
parser.compilationUnit.imports, foreachDecl.elementVariable.type);
boolean var = couldBeVar(parser == null ? null : parser.compilationUnit == null ? null :
parser.compilationUnit.imports, foreachDecl.elementVariable.type);
if (foreachDecl.elementVariable == null || !(val || var)) return;
try {
if (Reflection.iterableCopyField != null) Reflection.iterableCopyField.set(foreachDecl.elementVariable,
init);
} catch (Exception e) {
// In ecj mode this field isn't there and we don't need the copy anyway, so, we ignore the exception.
}
}
public static void copyInitializationOfLocalDeclaration(Parser parser) {
ASTNode[] astStack;
int astPtr;
try {
astStack = (ASTNode[]) Reflection.astStackField.get(parser);
astPtr = (Integer)Reflection.astPtrField.get(parser);
} catch (Exception e) {
// Most likely we're in ecj or some other plugin usage of the eclipse compiler. No need for this.
return;
}
AbstractVariableDeclaration variableDecl = (AbstractVariableDeclaration) astStack[astPtr];
```

```
if (!(variableDecl instanceof LocalDeclaration)) return;
ASTNode init = variableDecl.initialization;
if (init == null) return;
boolean val = couldBeVal(parser == null ? null : parser.compilationUnit == null ? null :
parser.compilationUnit.imports, variableDecl.type);
boolean var = couldBeVar(parser == null ? null : parser.compilationUnit == null ? null :
parser.compilationUnit.imports, variableDecl.type);
if (!(val || var)) return;
try {
if (Reflection.initCopyField != null) Reflection.initCopyField.set(variableDecl, init);
} catch (Exception e) {
// In ecj mode this field isn't there and we don't need the copy anyway, so, we ignore the exception.
}
}
private static boolean couldBeVal(ImportReference[] imports, TypeReference type) {
return PatchVal.couldBe(imports, "lombok.val", type);
}
private static boolean couldBeVar(ImportReference[] imports, TypeReference type) {
return PatchVal.couldBe(imports, "lombok.experimental.var", type) || PatchVal.couldBe(imports,
"lombok.var", type);
}
public static void addFinalAndValAnnotationToSingleVariableDeclaration(Object converter,
SingleVariableDeclaration out, LocalDeclaration in) {
@SuppressWarnings("unchecked") List<IExtendedModifier> modifiers = out.modifiers();
addFinalAndValAnnotationToModifierList(converter, modifiers, out.getAST(), in);
}
public static void addFinalAndValAnnotationToVariableDeclarationStatement(Object converter,
VariableDeclarationStatement out, LocalDeclaration in) {
@SuppressWarnings("unchecked") List<IExtendedModifier> modifiers = out.modifiers();
addFinalAndValAnnotationToModifierList(converter, modifiers, out.getAST(), in);
}
public static void addFinalAndValAnnotationToModifierList(Object converter, List<IExtendedModifier>
modifiers, AST ast, LocalDeclaration in) {
// First check that 'in' has the final flag on, and a @val / @lombok.val / @var / @lombok.var annotation.
if (in.annotations == null) return;
boolean found = false;
Annotation valAnnotation = null, varAnnotation = null;
for (Annotation ann : in.annotations) {
if (couldBeVal(null, ann.type)) {
found = true;
valAnnotation = ann;
}
if (couldBeVar(null, ann.type)) {
found = true;
varAnnotation = ann;
}
}
if (!found) return;
// Now check that 'out' is missing either of these.
if (modifiers == null) return; // This is null only if the project is 1.4 or less. Lombok doesn't work in that.
boolean finalIsPresent = false;
boolean valIsPresent = false;
boolean varIsPresent = false;
```

```
for (Object present : modifiers) {
if (present instanceof Modifier) {
ModifierKeyword keyword = ((Modifier) present).getKeyword();
if (keyword == null) continue;
if (keyword.toFlagValue() == Modifier.FINAL) finalIsPresent = true;
}
if (present instanceof org.eclipse.jdt.core.dom.Annotation) {
Name typeName = ((org.eclipse.jdt.core.dom.Annotation) present).getTypeName();
if (typeName != null) {
String fullyQualifiedName = typeName.getFullyQualifiedName();
if ("val".equals(fullyQualifiedName) || "lombok.val".equals(fullyQualifiedName)) valIsPresent = true;
if ("var".equals(fullyQualifiedName) || "lombok.var".equals(fullyQualifiedName) ||
"lombok.experimental.var".equals(fullyQualifiedName)) varIsPresent = true;
}
}
}
if (!finalIsPresent && valAnnotation != null) {
modifiers.add(createModifier(ast, ModifierKeyword.FINAL_KEYWORD, valAnnotation.sourceStart,
valAnnotation.sourceEnd));
}
if (!valIsPresent && valAnnotation != null) {
MarkerAnnotation newAnnotation = createValVarAnnotation(ast, valAnnotation,
valAnnotation.sourceStart, valAnnotation.sourceEnd);
try {
Reflection.astConverterRecordNodes.invoke(converter, newAnnotation, valAnnotation);
Reflection.astConverterRecordNodes.invoke(converter, newAnnotation.getTypeName(),
valAnnotation.type);
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
}
modifiers.add(newAnnotation);
}
if (!varIsPresent && varAnnotation != null) {
MarkerAnnotation newAnnotation = createValVarAnnotation(ast, varAnnotation,
varAnnotation.sourceStart, varAnnotation.sourceEnd);
try {
Reflection.astConverterRecordNodes.invoke(converter, newAnnotation, varAnnotation);
Reflection.astConverterRecordNodes.invoke(converter, newAnnotation.getTypeName(),
varAnnotation.type);
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
}
modifiers.add(newAnnotation);
}
}
public static Modifier createModifier(AST ast, ModifierKeyword keyword, int start, int end) {
Modifier modifier = null;
try {
modifier = Reflection.modifierConstructor.newInstance(ast);
```

```
} catch (InstantiationException e) {
throw Lombok.sneakyThrow(e);
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e);
}
if (modifier != null) {
modifier.setKeyword(keyword);
modifier.setSourceRange(start, end - start + 1);
}
return modifier;
}
public static MarkerAnnotation createValVarAnnotation(AST ast, Annotation original, int start, int end) {
MarkerAnnotation out = null;
try {
out = Reflection.markerAnnotationConstructor.newInstance(ast);
} catch (InstantiationException e) {
throw Lombok.sneakyThrow(e);
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e);
}
char[][] tokens;
if (original.type instanceof SingleTypeReference) {
tokens = new char[1][];
tokens[0] = ((SingleTypeReference) original.type).token;
} else if (original.type instanceof QualifiedTypeReference) {
tokens = ((QualifiedTypeReference) original.type).tokens;
} else {
return null;
}
if (out != null) {
SimpleName valName = ast.newSimpleName(new String(tokens[tokens.length - 1]));
valName.setSourceRange(start, end - start + 1);
if (tokens.length == 1) {
out.setTypeName(valName);
setIndex(valName, 1);
} else if (tokens.length == 2) {
SimpleName lombokName = ast.newSimpleName("lombok");
lombokName.setSourceRange(start, end - start + 1);
setIndex(lombokName, 1);
setIndex(valName, 2);
QualifiedName fullName = ast.newQualifiedName(lombokName, valName);
setIndex(fullName, 1);
fullName.setSourceRange(start, end - start + 1);
out.setTypeName(fullName);
} else {
SimpleName lombokName = ast.newSimpleName("lombok");
lombokName.setSourceRange(start, end - start + 1);
SimpleName experimentalName = ast.newSimpleName("experimental");
lombokName.setSourceRange(start, end - start + 1);
```

```
setIndex(lombokName, 1);
setIndex(experimentalName, 2);
setIndex(valName, 3);
QualifiedName lombokExperimentalName = ast.newQualifiedName(lombokName, experimentalName);
lombokExperimentalName.setSourceRange(start, end - start + 1);
setIndex(lombokExperimentalName, 1);
QualifiedName fullName = ast.newQualifiedName(lombokExperimentalName, valName);
setIndex(fullName, 1);
fullName.setSourceRange(start, end - start + 1);
out.setTypeName(fullName);
}
out.setSourceRange(start, end - start + 1);
}
return out;
}
private static final Field FIELD_NAME_INDEX;
static {
Field f = null;
try {
f = Permit.getField(Name.class, "index");
} catch (Throwable t) {
// Leave it null, in which case we don't set index. That'll result in error log messages but its better than
crashing here.
}
FIELD_NAME_INDEX = f;
}
private static void setIndex(Name name, int index) {
try {
if (FIELD_NAME_INDEX != null) FIELD_NAME_INDEX.set(name, index);
} catch (Exception e) {
// Don't do anything - safest fallback behaviour.
}
}
public static final class Reflection {
private static final Field initCopyField, iterableCopyField;
private static final Field astStackField, astPtrField;
private static final Constructor<Modifier> modifierConstructor;
private static final Constructor<MarkerAnnotation> markerAnnotationConstructor;
private static final Method astConverterRecordNodes;
static {
Field a = null, b = null, c = null, d = null;
Constructor<Modifier> f = null;
Constructor<MarkerAnnotation> g = null;
Method h = null;
try {
a = Permit.getField(LocalDeclaration.class, "$initCopy");
b = Permit.getField(LocalDeclaration.class, "$iterableCopy");
} catch (Throwable t) {
//ignore - no $initCopy exists when running in ecj.
}
try {
c = Permit.getField(Parser.class, "astStack");
d = Permit.getField(Parser.class, "astPtr");
```

```
f = Permit.getConstructor(Modifier.class, AST.class);
g = Permit.getConstructor(MarkerAnnotation.class, AST.class);
Class<?> z = Class.forName("org.eclipse.jdt.core.dom.ASTConverter");
h = Permit.getMethod(z, "recordNodes", org.eclipse.jdt.core.dom.ASTNode.class,
org.eclipse.jdt.internal.compiler.ast.ASTNode.class);
} catch (Throwable t) {
// Most likely we're in ecj or some other plugin usage of the eclipse compiler. No need for this.
}
initCopyField = a;
iterableCopyField = b;
astStackField = c;
astPtrField = d;
modifierConstructor = f;
markerAnnotationConstructor = g;
astConverterRecordNodes = h;
}
}
}
```

lombok/eclipse/agent/PatchValEclipsePortal.java

```
/*
* Copyright (C) 2011 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.eclipse.agent;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import lombok.Lombok;
public class PatchValEclipsePortal {
static final String LOCALDECLARATION_SIG = "org.eclipse.jdt.internal.compiler.ast.LocalDeclaration";
static final String PARSER_SIG = "org.eclipse.jdt.internal.compiler.parser.Parser";
static final String VARIABLEDECLARATIONSTATEMENT_SIG =
"org.eclipse.jdt.core.dom.VariableDeclarationStatement";
```

```
static final String SINGLEVARIABLEDECLARATION_SIG =
"org.eclipse.jdt.core.dom.SingleVariableDeclaration";
static final String ASTCONVERTER_SIG = "org.eclipse.jdt.core.dom.ASTConverter";
public static void copyInitializationOfForEachIterable(Object parser) {
try {
Reflection.copyInitializationOfForEachIterable.invoke(null, parser);
} catch (NoClassDefFoundError e) {
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
} catch (NullPointerException e) {
if (!"false".equals(System.getProperty("lombok.debug.reflection", "false"))) {
e.initCause(Reflection.problem);
throw e;
}
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
}
}
public static void copyInitializationOfLocalDeclaration(Object parser) {
try {
Reflection.copyInitializationOfLocalDeclaration.invoke(null, parser);
} catch (NoClassDefFoundError e) {
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
} catch (NullPointerException e) {
if (!"false".equals(System.getProperty("lombok.debug.reflection", "false"))) {
e.initCause(Reflection.problem);
throw e;
}
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
}
}
public static void addFinalAndValAnnotationToVariableDeclarationStatement(Object converter, Object
out, Object in) {
try {
Reflection.addFinalAndValAnnotationToVariableDeclarationStatement.invoke(null, converter, out, in);
} catch (NoClassDefFoundError e) {
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
} catch (NullPointerException e) {
```

```
if (!"false".equals(System.getProperty("lombok.debug.reflection", "false"))) {
e.initCause(Reflection.problem);
throw e;
}
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
}
}
public static void addFinalAndValAnnotationToSingleVariableDeclaration(Object converter, Object out,
Object in) {
try {
Reflection.addFinalAndValAnnotationToSingleVariableDeclaration.invoke(null, converter, out, in);
} catch (NoClassDefFoundError e) {
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
} catch (IllegalAccessException e) {
throw Lombok.sneakyThrow(e);
} catch (InvocationTargetException e) {
throw Lombok.sneakyThrow(e.getCause());
} catch (NullPointerException e) {
if (!"false".equals(System.getProperty("lombok.debug.reflection", "false"))) {
e.initCause(Reflection.problem);
throw e;
}
//ignore, we don't have access to the correct ECJ classes, so lombok can't possibly
//do anything useful here.
}
}
private static final class Reflection {
public static final Method copyInitializationOfForEachIterable;
public static final Method copyInitializationOfLocalDeclaration;
public static final Method addFinalAndValAnnotationToVariableDeclarationStatement;
public static final Method addFinalAndValAnnotationToSingleVariableDeclaration;
public static final Throwable problem;
static {
Method m = null, n = null, o = null, p = null;
Throwable problem_ = null;
try {
m = PatchValEclipse.class.getMethod("copyInitializationOfForEachIterable",
Class.forName(PARSER_SIG));
n = PatchValEclipse.class.getMethod("copyInitializationOfLocalDeclaration",
Class.forName(PARSER_SIG));
o = PatchValEclipse.class.getMethod("addFinalAndValAnnotationToVariableDeclarationStatement",
Object.class,
Class.forName(VARIABLEDECLARATIONSTATEMENT_SIG),
Class.forName(LOCALDECLARATION_SIG));
p = PatchValEclipse.class.getMethod("addFinalAndValAnnotationToSingleVariableDeclaration",
Object.class,
Class.forName(SINGLEVARIABLEDECLARATION_SIG),
Class.forName(LOCALDECLARATION_SIG));
} catch (Throwable t) {
// That's problematic, but as long as no local classes are used we don't actually need it.
// Better fail on local classes than crash altogether.
```

```
problem_ = t;
}
copyInitializationOfForEachIterable = m;
copyInitializationOfLocalDeclaration = n;
addFinalAndValAnnotationToVariableDeclarationStatement = o;
addFinalAndValAnnotationToSingleVariableDeclaration = p;
problem = problem_;
}
}
}
```

lombok/eclipse/agent/package-info.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* Contains the mechanism that instruments eclipse by being loaded as a javaagent.
*/
package lombok.eclipse.agent;
```

lombok/launch/PatchFixesHider.java

```
/*
* Copyright (C) 2010-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
```

*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.launch;
import static lombok.eclipse.EcjAugments.*;
import static lombok.eclipse.Eclipse.*;
import java.io.BufferedOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.security.CodeSource;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Stack;
import org.eclipse.core.runtime.CoreException;
import org.eclipse.jdt.core.IAnnotatable;
import org.eclipse.jdt.core.IAnnotation;
import org.eclipse.jdt.core.IJavaElement;
import org.eclipse.jdt.core.IMethod;
import org.eclipse.jdt.core.IType;
import org.eclipse.jdt.core.JavaModelException;
import org.eclipse.jdt.core.dom.AST;
import org.eclipse.jdt.core.dom.ASTNode;
import org.eclipse.jdt.core.dom.MethodDeclaration;
import org.eclipse.jdt.core.dom.Name;
import org.eclipse.jdt.core.dom.ReturnStatement;
import org.eclipse.jdt.core.dom.SimpleName;
import org.eclipse.jdt.core.dom.SingleVariableDeclaration;
import org.eclipse.jdt.core.dom.Type;
import org.eclipse.jdt.core.search.SearchMatch;
import org.eclipse.jdt.internal.compiler.ISourceElementRequestor.FieldInfo;
import org.eclipse.jdt.internal.compiler.ast.AbstractMethodDeclaration;
import org.eclipse.jdt.internal.compiler.ast.AbstractVariableDeclaration;
import org.eclipse.jdt.internal.compiler.ast.Annotation;
import org.eclipse.jdt.internal.compiler.ast.Expression;
import org.eclipse.jdt.internal.compiler.ast.FieldDeclaration;
import org.eclipse.jdt.internal.compiler.ast.LocalDeclaration;
import org.eclipse.jdt.internal.compiler.ast.TypeDeclaration;
import org.eclipse.jdt.internal.compiler.lookup.BlockScope;
import org.eclipse.jdt.internal.compiler.lookup.TypeBinding;
import org.eclipse.jdt.internal.compiler.parser.Parser;
```

```
import org.eclipse.jdt.internal.core.CompilationUnitStructureRequestor;
import org.eclipse.jdt.internal.core.SourceField;
import org.eclipse.jdt.internal.core.SourceFieldElementInfo;
import org.eclipse.jdt.internal.core.dom.rewrite.NodeRewriteEvent;
import org.eclipse.jdt.internal.core.dom.rewrite.RewriteEvent;
import org.eclipse.jdt.internal.core.dom.rewrite.TokenScanner;
import org.eclipse.jdt.internal.corext.refactoring.SearchResultGroup;
import org.eclipse.jdt.internal.corext.refactoring.code.CallContext;
import org.eclipse.jdt.internal.corext.refactoring.code.SourceProvider;
import org.eclipse.jdt.internal.corext.refactoring.structure.MemberVisibilityAdjustor.IncomingMemberVis
ibilityAdjustment;
import lombok.eclipse.handlers.EclipseHandlerUtil;
import lombok.permit.Permit;
/** These contain a mix of the following:
* <ul>
* <li> 'dependency free' method wrappers that cross the shadowloader barrier.
* <li> methods that directly patch, <em>but</em>, these should ALWAYS be transplanted.
* </ul>
*
* <strong>This class lives on the outside of the shadowloader barrier, and as a consequence, cannot
access any other lombok code except other
* code in the {@code lombok.launch} package!</strong>.
* <p>
* This class is package private with lots of public inner static classes. This hides all of them from IDE
autocomplete dialogs and such but at the JVM
* level the inner static class are just plain public, which is important, because calls to the contents of
these inner static classes are injected into
* various eclipse classes verbatim, and if they weren't public, the verifier wouldn't accept it.
*/
final class PatchFixesHider {
/** These utility methods are only used 'internally', but because of transplant methods, the class (and its
methods) still have to be public! */
public static final class Util {
private static ClassLoader shadowLoader;
public static ClassLoader getShadowLoader() {
if (shadowLoader == null) {
try {
Class.forName("lombok.core.LombokNode");
// If we get here, then lombok is already available.
shadowLoader = Util.class.getClassLoader();
} catch (ClassNotFoundException e) {
// If we get here, it isn't, and we should use the shadowloader.
shadowLoader = Main.getShadowClassLoader();
}
}
return shadowLoader;
}
public static Class<?> shadowLoadClass(String name) {
try {
return Class.forName(name, true, getShadowLoader());
} catch (ClassNotFoundException e) {
throw sneakyThrow(e);
}
```

```
}
public static Method findMethod(Class<?> type, String name, Class<?>... parameterTypes) {
try {
return type.getDeclaredMethod(name, parameterTypes);
} catch (NoSuchMethodException e) {
throw sneakyThrow(e);
}
}
public static Method findMethod(Class<?> type, String name, String... parameterTypes) {
for (Method m : type.getDeclaredMethods()) {
if (name.equals(m.getName()) && sameTypes(m.getParameterTypes(), parameterTypes)) {
return m;
}
}
throw sneakyThrow(new NoSuchMethodException(type.getName() + "::" + name));
}
public static Method findMethodAnyArgs(Class<?> type, String name) {
for (Method m : type.getDeclaredMethods()) if (name.equals(m.getName())) return m;
throw sneakyThrow(new NoSuchMethodException(type.getName() + "::" + name));
}
public static Object invokeMethod(Method method, Object... args) {
try {
return method.invoke(null, args);
} catch (IllegalAccessException e) {
throw sneakyThrow(e);
} catch (InvocationTargetException e) {
throw sneakyThrow(e.getCause());
}
}
private static RuntimeException sneakyThrow(Throwable t) {
if (t == null) throw new NullPointerException("t");
Util.<RuntimeException>sneakyThrow0(t);
return null;
}
@SuppressWarnings("unchecked")
private static <T extends Throwable> void sneakyThrow0(Throwable t) throws T {
throw (T)t;
}
private static boolean sameTypes(Class<?>[] types, String[] typeNames) {
if (types.length != typeNames.length) return false;
for (int i = 0; i < types.length; i++) {
if (!types[i].getName().equals(typeNames[i])) return false;
}
return true;
}
private static void prependToClassLoader(ClassLoader currentClassLoader, ClassLoader prepend) {
try {
Method prependParentMethod = Permit.getMethod(currentClassLoader.getClass(), "prependParent",
ClassLoader.class);
Permit.invoke(prependParentMethod, currentClassLoader, prepend);
} catch (Throwable t) {
// Ignore
}
```

```
}
private static ClassLoader findJdtCoreClassLoader(ClassLoader classLoader) {
try {
Method getBundleMethod = Permit.getMethod(classLoader.getClass(), "getBundle");
Object bundle = Permit.invoke(getBundleMethod, classLoader);
Method getBundleContextMethod = Permit.getMethod(bundle.getClass(), "getBundleContext");
Object bundleContext = Permit.invoke(getBundleContextMethod, bundle);
Method getBundlesMethod = Permit.getMethod(bundleContext.getClass(), "getBundles");
Object[] bundles = (Object[]) Permit.invoke(getBundlesMethod, bundleContext);
for (Object searchBundle : bundles) {
if (searchBundle.toString().startsWith("org.eclipse.jdt.core_")) {
Method getModuleClassLoaderMethod = Permit.getMethod(searchBundle.getClass(),
"getModuleClassLoader", boolean.class);
return (ClassLoader) Permit.invoke(getModuleClassLoaderMethod, searchBundle, false);
}
}
} catch (Throwable t) {
// Ignore
}
return null;
}
}
/** Contains patch fixes that are dependent on lombok internals. */
public static final class LombokDeps {
public static final Method ADD_LOMBOK_NOTES;
public static final Method POST_COMPILER_BYTES_STRING;
public static final Method POST_COMPILER_OUTPUTSTREAM;
public static final Method POST_COMPILER_BUFFEREDOUTPUTSTREAM_STRING_STRING;
static {
Class<?> shadowed = Util.shadowLoadClass("lombok.eclipse.agent.PatchFixesShadowLoaded");
ADD_LOMBOK_NOTES = Util.findMethod(shadowed, "addLombokNotesToEclipseAboutDialog",
String.class, String.class);
POST_COMPILER_BYTES_STRING = Util.findMethod(shadowed, "runPostCompiler", byte[].class,
String.class);
POST_COMPILER_OUTPUTSTREAM = Util.findMethod(shadowed, "runPostCompiler",
OutputStream.class);
POST_COMPILER_BUFFEREDOUTPUTSTREAM_STRING_STRING = Util.findMethod(shadowed,
"runPostCompiler", BufferedOutputStream.class, String.class, String.class);
}
public static String addLombokNotesToEclipseAboutDialog(String origReturnValue, String key) {
try {
return (String) Util.invokeMethod(LombokDeps.ADD_LOMBOK_NOTES, origReturnValue, key);
} catch (Throwable t) {
return origReturnValue;
}
}
public static byte[] runPostCompiler(byte[] bytes, String fileName) {
return (byte[]) Util.invokeMethod(LombokDeps.POST_COMPILER_BYTES_STRING, bytes, fileName);
}
public static OutputStream runPostCompiler(OutputStream out) throws IOException {
return (OutputStream) Util.invokeMethod(LombokDeps.POST_COMPILER_OUTPUTSTREAM, out);
}
```

```
public static BufferedOutputStream runPostCompiler(BufferedOutputStream out, String path, String
name) throws IOException {
return (BufferedOutputStream) Util.invokeMethod(LombokDeps.POST_COMPILER_BUFFEREDOUTP
UTSTREAM_STRING_STRING, out, path, name);
}
}
public static final class ModuleClassLoading {
public static void parserClinit() {
ClassLoader jdtCoreClassLoader = Util.findJdtCoreClassLoader(Parser.class.getClassLoader());
ClassLoader currentClassLoader = ModuleClassLoading.class.getClassLoader();
Util.prependToClassLoader(currentClassLoader, jdtCoreClassLoader);
}
}
public static final class Transform {
private static Method TRANSFORM;
private static Method TRANSFORM_SWAPPED;
private static synchronized void init(ClassLoader prepend) {
if (TRANSFORM != null) return;
Main.prependClassLoader(prepend);
ClassLoader currentClassLoader = Transform.class.getClassLoader();
Util.prependToClassLoader(currentClassLoader, prepend);
Class<?> shadowed = Util.shadowLoadClass("lombok.eclipse.TransformEclipseAST");
TRANSFORM = Util.findMethodAnyArgs(shadowed, "transform");
TRANSFORM_SWAPPED = Util.findMethodAnyArgs(shadowed, "transform_swapped");
}
public static void transform(Object parser, Object ast) throws IOException {
init(parser.getClass().getClassLoader());
Util.invokeMethod(TRANSFORM, parser, ast);
}
public static void transform_swapped(Object ast, Object parser) throws IOException {
init(parser.getClass().getClassLoader());
Util.invokeMethod(TRANSFORM_SWAPPED, ast, parser);
}
}
/** Contains patch code to support {@code @Delegate} */
public static final class Delegate {
private static final Method HANDLE_DELEGATE_FOR_TYPE;
private static final Method ADD_GENERATED_DELEGATE_METHODS;
public static final Method IS_DELEGATE_SOURCE_METHOD;
public static final Method RETURN_ELEMENT_INFO;
static {
Class<?> shadowedPortal = Util.shadowLoadClass("lombok.eclipse.agent.PatchDelegatePortal");
HANDLE_DELEGATE_FOR_TYPE = Util.findMethod(shadowedPortal, "handleDelegateForType",
Object.class);
ADD_GENERATED_DELEGATE_METHODS = Util.findMethod(shadowedPortal,
"addGeneratedDelegateMethods", Object.class, Object.class);
Class<?> shadowed = Util.shadowLoadClass("lombok.eclipse.agent.PatchDelegate");
IS_DELEGATE_SOURCE_METHOD = Util.findMethod(shadowed, "isDelegateSourceMethod",
Object.class);
RETURN_ELEMENT_INFO = Util.findMethod(shadowed, "returnElementInfo", Object.class);
}
public static boolean handleDelegateForType(Object classScope) {
return (Boolean) Util.invokeMethod(HANDLE_DELEGATE_FOR_TYPE, classScope);
```

```
}
public static Object[] addGeneratedDelegateMethods(Object returnValue, Object javaElement) {
return (Object[]) Util.invokeMethod(ADD_GENERATED_DELEGATE_METHODS, returnValue,
javaElement);
}
public static boolean isDelegateSourceMethod(Object sourceMethod) {
return (Boolean) Util.invokeMethod(IS_DELEGATE_SOURCE_METHOD, sourceMethod);
}
public static Object returnElementInfo(Object delegateSourceMethod) {
return Util.invokeMethod(RETURN_ELEMENT_INFO, delegateSourceMethod);
}
}
/** Contains patch code to support {@code val} (eclipse specific) */
public static final class ValPortal {
private static final Method COPY_INITIALIZATION_OF_FOR_EACH_ITERABLE;
private static final Method COPY_INITIALIZATION_OF_LOCAL_DECLARATION;
private static final Method
ADD_FINAL_AND_VAL_ANNOTATION_TO_VARIABLE_DECLARATION_STATEMENT;
private static final Method
ADD_FINAL_AND_VAL_ANNOTATION_TO_SINGLE_VARIABLE_DECLARATION;
static {
Class<?> shadowed = Util.shadowLoadClass("lombok.eclipse.agent.PatchValEclipsePortal");
COPY_INITIALIZATION_OF_FOR_EACH_ITERABLE = Util.findMethod(shadowed,
"copyInitializationOfForEachIterable", Object.class);
COPY_INITIALIZATION_OF_LOCAL_DECLARATION = Util.findMethod(shadowed,
"copyInitializationOfLocalDeclaration", Object.class);
ADD_FINAL_AND_VAL_ANNOTATION_TO_VARIABLE_DECLARATION_STATEMENT =
Util.findMethod(shadowed, "addFinalAndValAnnotationToVariableDeclarationStatement", Object.class,
Object.class, Object.class);
ADD_FINAL_AND_VAL_ANNOTATION_TO_SINGLE_VARIABLE_DECLARATION =
Util.findMethod(shadowed, "addFinalAndValAnnotationToSingleVariableDeclaration", Object.class,
Object.class, Object.class);
}
public static void copyInitializationOfForEachIterable(Object parser) {
Util.invokeMethod(COPY_INITIALIZATION_OF_FOR_EACH_ITERABLE, parser);
}
public static void copyInitializationOfLocalDeclaration(Object parser) {
Util.invokeMethod(COPY_INITIALIZATION_OF_LOCAL_DECLARATION, parser);
}
public static void addFinalAndValAnnotationToVariableDeclarationStatement(Object converter, Object
out, Object in) {
Util.invokeMethod(ADD_FINAL_AND_VAL_ANNOTATION_TO_VARIABLE_DECLARATION_STATE
MENT, converter, out, in);
}
public static void addFinalAndValAnnotationToSingleVariableDeclaration(Object converter, Object out,
Object in) {
Util.invokeMethod(ADD_FINAL_AND_VAL_ANNOTATION_TO_SINGLE_VARIABLE_DECLARATION,
converter, out, in);
}
}
/** Contains patch code to support {@code val} (eclipse and ecj) */
public static final class Val {
private static final String BLOCK_SCOPE_SIG = "org.eclipse.jdt.internal.compiler.lookup.BlockScope";
```

```
private static final String LOCAL_DECLARATION_SIG =
"org.eclipse.jdt.internal.compiler.ast.LocalDeclaration";
private static final String FOREACH_STATEMENT_SIG =
"org.eclipse.jdt.internal.compiler.ast.ForeachStatement";
private static final Method HANDLE_VAL_FOR_LOCAL_DECLARATION;
private static final Method HANDLE_VAL_FOR_FOR_EACH;
static {
Class<?> shadowed = Util.shadowLoadClass("lombok.eclipse.agent.PatchVal");
HANDLE_VAL_FOR_LOCAL_DECLARATION = Util.findMethod(shadowed,
"handleValForLocalDeclaration", LOCAL_DECLARATION_SIG, BLOCK_SCOPE_SIG);
HANDLE_VAL_FOR_FOR_EACH = Util.findMethod(shadowed, "handleValForForEach",
FOREACH_STATEMENT_SIG, BLOCK_SCOPE_SIG);
}
public static boolean handleValForLocalDeclaration(Object local, Object scope) {
return (Boolean) Util.invokeMethod(HANDLE_VAL_FOR_LOCAL_DECLARATION, local, scope);
}
public static boolean handleValForForEach(Object forEach, Object scope) {
return (Boolean) Util.invokeMethod(HANDLE_VAL_FOR_FOR_EACH, forEach, scope);
}
/**
* Patches local declaration to not call .resolveType() on the initializer expression if we've already done
so (calling it twice causes weird errors)
* This and the next method must be transplanted so that the return type is loaded in the correct class
loader
*/
public static TypeBinding skipResolveInitializerIfAlreadyCalled(Expression expr, BlockScope scope) {
if (expr.resolvedType != null) return expr.resolvedType;
try {
return expr.resolveType(scope);
} catch (NullPointerException e) {
return null;
} catch (ArrayIndexOutOfBoundsException e) {
// This will occur internally due to for example 'val x = mth("X");', where mth takes 2 arguments.
return null;
}
}
public static TypeBinding skipResolveInitializerIfAlreadyCalled2(Expression expr, BlockScope scope,
LocalDeclaration decl) {
if (decl != null && LocalDeclaration.class.equals(decl.getClass()) && expr.resolvedType != null) return
expr.resolvedType;
try {
return expr.resolveType(scope);
} catch (NullPointerException e) {
return null;
} catch (ArrayIndexOutOfBoundsException e) {
// This will occur internally due to for example 'val x = mth("X");', where mth takes 2 arguments.
return null;
}
}
}
/** Contains patch code to support {@code @ExtensionMethod} */
public static final class ExtensionMethod {
private static final String MESSAGE_SEND_SIG = "org.eclipse.jdt.internal.compiler.ast.MessageSend";
```

```
private static final String TYPE_BINDING_SIG = "org.eclipse.jdt.internal.compiler.lookup.TypeBinding";
private static final String SCOPE_SIG = "org.eclipse.jdt.internal.compiler.lookup.Scope";
private static final String BLOCK_SCOPE_SIG = "org.eclipse.jdt.internal.compiler.lookup.BlockScope";
private static final String TYPE_BINDINGS_SIG =
"[Lorg.eclipse.jdt.internal.compiler.lookup.TypeBinding;";
private static final String PROBLEM_REPORTER_SIG =
"org.eclipse.jdt.internal.compiler.problem.ProblemReporter";
private static final String METHOD_BINDING_SIG =
"org.eclipse.jdt.internal.compiler.lookup.MethodBinding";
private static final String AST_NODE_SIG = "org.eclipse.jdt.internal.compiler.ast.ASTNode";
private static final Method RESOLVE_TYPE;
private static final Method ERROR_NO_METHOD_FOR;
private static final Method INVALID_METHOD, INVALID_METHOD2;
private static final Method NON_STATIC_ACCESS_TO_STATIC_METHOD;
private static final Method MODIFY_METHOD_PATTERN;
static {
Class<?> shadowed = Util.shadowLoadClass("lombok.eclipse.agent.PatchExtensionMethod");
RESOLVE_TYPE = Util.findMethod(shadowed, "resolveType", TYPE_BINDING_SIG,
MESSAGE_SEND_SIG, BLOCK_SCOPE_SIG);
ERROR_NO_METHOD_FOR = Util.findMethod(shadowed, "errorNoMethodFor",
PROBLEM_REPORTER_SIG, MESSAGE_SEND_SIG, TYPE_BINDING_SIG,
TYPE_BINDINGS_SIG);
INVALID_METHOD = Util.findMethod(shadowed, "invalidMethod", PROBLEM_REPORTER_SIG,
MESSAGE_SEND_SIG, METHOD_BINDING_SIG);
INVALID_METHOD2 = Util.findMethod(shadowed, "invalidMethod", PROBLEM_REPORTER_SIG,
MESSAGE_SEND_SIG, METHOD_BINDING_SIG, SCOPE_SIG);
NON_STATIC_ACCESS_TO_STATIC_METHOD = Util.findMethod(shadowed,
"nonStaticAccessToStaticMethod", PROBLEM_REPORTER_SIG, AST_NODE_SIG,
METHOD_BINDING_SIG, MESSAGE_SEND_SIG);
MODIFY_METHOD_PATTERN = Util.findMethod(shadowed, "modifyMethodPattern", Object.class);
}
public static Object resolveType(Object resolvedType, Object methodCall, Object scope) {
return Util.invokeMethod(RESOLVE_TYPE, resolvedType, methodCall, scope);
}
public static void errorNoMethodFor(Object problemReporter, Object messageSend, Object recType,
Object params) {
Util.invokeMethod(ERROR_NO_METHOD_FOR, problemReporter, messageSend, recType, params);
}
public static void invalidMethod(Object problemReporter, Object messageSend, Object method) {
Util.invokeMethod(INVALID_METHOD, problemReporter, messageSend, method);
}
public static void invalidMethod(Object problemReporter, Object messageSend, Object method, Object
scope) {
Util.invokeMethod(INVALID_METHOD2, problemReporter, messageSend, method, scope);
}
public static void nonStaticAccessToStaticMethod(Object problemReporter, Object location, Object
method, Object messageSend) {
Util.invokeMethod(NON_STATIC_ACCESS_TO_STATIC_METHOD, problemReporter, location,
method, messageSend);
}
public static Object modifyMethodPattern(Object original) {
return Util.invokeMethod(MODIFY_METHOD_PATTERN, original);
}
```

```
}
/** Contains patch code to support Javadoc for generated methods */
public static final class Javadoc {
private static final Method GET_HTML;
static {
Class<?> shadowed = Util.shadowLoadClass("lombok.eclipse.agent.PatchJavadoc");
GET_HTML = Util.findMethod(shadowed, "getHTMLContentFromSource", Object.class, String.class,
Object.class);
}
public static String getHTMLContentFromSource(String original, IJavaElement member) {
return (String) Util.invokeMethod(GET_HTML, null, original, member);
}
public static String getHTMLContentFromSource(String original, Object instance, IJavaElement
member) {
return (String) Util.invokeMethod(GET_HTML, instance, original, member);
}
}
/**
* Contains a mix of methods: ecj only, ecj+eclipse, and eclipse only. As a consequence, _EVERY_
method from here used for ecj MUST be
* transplanted, as ecj itself cannot load this class (signatures refer to things that don't exist in ecj-only
mode).
* <p>
* Because of usage of transplant(), a bunch of these contain direct code and don't try to cross the
shadowloader barrier.
*/
public static final class PatchFixes {
public static boolean isGenerated(org.eclipse.jdt.core.dom.ASTNode node) {
boolean result = false;
try {
result = ((Boolean)node.getClass().getField("$isGenerated").get(node)).booleanValue();
if (!result && node.getParent() != null && node.getParent() instanceof
org.eclipse.jdt.core.dom.QualifiedName) {
result = isGenerated(node.getParent());
}
} catch (Exception e) {
// better to assume it isn't generated
}
return result;
}
public static boolean isGenerated(org.eclipse.jdt.internal.compiler.ast.ASTNode node) {
boolean result = false;
try {
result = node.getClass().getField("$generatedBy").get(node) != null;
} catch (Exception e) {
// better to assume it isn't generated
}
return result;
}
public static boolean isGenerated(org.eclipse.jdt.core.IMember member) {
boolean result = false;
try {
```

```
result = member.getNameRange().getLength() <= 0 ||
member.getNameRange().equals(member.getSourceRange());
} catch (JavaModelException e) {
// better to assume it isn't generated
}
return result;
}
public static boolean isBlockedVisitorAndGenerated(org.eclipse.jdt.core.dom.ASTNode node,
org.eclipse.jdt.core.dom.ASTVisitor visitor) {
if (visitor == null) return false;
String className = visitor.getClass().getName();
if (!(className.startsWith("org.eclipse.jdt.internal.corext.fix") ||
className.startsWith("org.eclipse.jdt.internal.ui.fix") ||
className.startsWith("org.eclipse.jdt.ls.core.internal.semantictokens.SemanticTokensVisitor"))) return
false;
if (className.equals("org.eclipse.jdt.internal.corext.fix.VariableDeclarationFixCore$WrittenNamesFind
er")) return false;
return isGenerated(node);
}
public static boolean isListRewriteOnGeneratedNode(org.eclipse.jdt.core.dom.rewrite.ListRewrite
rewrite) {
return isGenerated(rewrite.getParent());
}
public static boolean returnFalse(java.lang.Object object) {
return false;
}
public static boolean returnTrue(java.lang.Object object) {
return true;
}
@java.lang.SuppressWarnings({"unchecked", "rawtypes"}) public static java.util.List
removeGeneratedNodes(java.util.List list) {
try {
java.util.List realNodes = new java.util.ArrayList(list.size());
for (java.lang.Object node : list) {
if(!isGenerated(((org.eclipse.jdt.core.dom.ASTNode)node))) {
realNodes.add(node);
}
}
return realNodes;
} catch (Exception e) {
}
return list;
}
public static java.lang.String getRealMethodDeclarationSource(java.lang.String original, Object
processor, org.eclipse.jdt.core.dom.MethodDeclaration declaration) throws Exception {
if (!isGenerated(declaration)) return original;
List<org.eclipse.jdt.core.dom.Annotation> annotations = new
ArrayList<org.eclipse.jdt.core.dom.Annotation>();
for (Object modifier : declaration.modifiers()) {
if (modifier instanceof org.eclipse.jdt.core.dom.Annotation) {
org.eclipse.jdt.core.dom.Annotation annotation = (org.eclipse.jdt.core.dom.Annotation)modifier;
String qualifiedAnnotationName = annotation.resolveTypeBinding().getQualifiedName();
```

```
if (!"java.lang.Override".equals(qualifiedAnnotationName) &&
!"java.lang.SuppressWarnings".equals(qualifiedAnnotationName)) annotations.add(annotation);
}
}
StringBuilder signature = new StringBuilder();
addAnnotations(annotations, signature);
try {
if ((Boolean)processor.getClass().getDeclaredField("fPublic").get(processor)) signature.append("public
");
if ((Boolean)processor.getClass().getDeclaredField("fAbstract").get(processor))
signature.append("abstract ");
} catch (Throwable t) { }
signature
.append(declaration.getReturnType2().toString())
.append(" ").append(declaration.getName().getFullyQualifiedName())
.append("(");
boolean first = true;
for (Object parameter : declaration.parameters()) {
if (!first) signature.append(", ");
first = false;
// We should also add the annotations of the parameters
signature.append(parameter);
}
signature.append(");");
return signature.toString();
}
// part of getRealMethodDeclarationSource(...)
public static void addAnnotations(List<org.eclipse.jdt.core.dom.Annotation> annotations, StringBuilder
signature) {
/*
* We SHOULD be able to handle the following cases:
* @Override
* @Override()
* @SuppressWarnings("all")
* @SuppressWarnings({"all", "unused"})
* @SuppressWarnings(value = "all")
* @SuppressWarnings(value = {"all", "unused"})
* @EqualsAndHashCode(callSuper=true, of="id")
*
* Currently, we only seem to correctly support:
* @Override
* @Override() N.B. We lose the parentheses here, since there are no values. No big deal.
* @SuppressWarnings("all")
*/
for (org.eclipse.jdt.core.dom.Annotation annotation : annotations) {
List<String> values = new ArrayList<String>();
if (annotation.isSingleMemberAnnotation()) {
org.eclipse.jdt.core.dom.SingleMemberAnnotation smAnn =
(org.eclipse.jdt.core.dom.SingleMemberAnnotation) annotation;
values.add(smAnn.getValue().toString());
} else if (annotation.isNormalAnnotation()) {
org.eclipse.jdt.core.dom.NormalAnnotation normalAnn = (org.eclipse.jdt.core.dom.NormalAnnotation)
annotation;
```

```
for (Object value : normalAnn.values()) values.add(value.toString());
}
signature.append("@").append(annotation.getTypeName().getFullyQualifiedName());
if (!values.isEmpty()) {
signature.append("(");
boolean first = true;
for (String string : values) {
if (!first) signature.append(", ");
first = false;
signature.append("").append(string).append("");
}
signature.append(")");
}
signature.append(" ");
}
}
public static org.eclipse.jdt.core.dom.MethodDeclaration
getRealMethodDeclarationNode(org.eclipse.jdt.core.dom.MethodDeclaration original,
org.eclipse.jdt.core.IMethod sourceMethod, org.eclipse.jdt.core.dom.CompilationUnit cuUnit) throws
JavaModelException {
if (!isGenerated(original)) return original;
IType declaringType = sourceMethod.getDeclaringType();
Stack<IType> typeStack = new Stack<IType>();
while (declaringType != null) {
typeStack.push(declaringType);
declaringType = declaringType.getDeclaringType();
}
IType rootType = typeStack.pop();
org.eclipse.jdt.core.dom.AbstractTypeDeclaration typeDeclaration = findTypeDeclaration(rootType,
cuUnit.types());
while (!typeStack.isEmpty() && typeDeclaration != null) {
typeDeclaration = findTypeDeclaration(typeStack.pop(), typeDeclaration.bodyDeclarations());
}
String targetMethodName = sourceMethod.getElementName();
List<String> targetMethodParameterTypes = new ArrayList<String>();
for (String parameterType : sourceMethod.getParameterTypes()) {
targetMethodParameterTypes.add(org.eclipse.jdt.core.Signature.toString(parameterType));
}
if (typeStack.isEmpty() && typeDeclaration != null) {
for (Object declaration : typeDeclaration.bodyDeclarations()) {
if (declaration instanceof org.eclipse.jdt.core.dom.MethodDeclaration) {
org.eclipse.jdt.core.dom.MethodDeclaration methodDeclaration =
(org.eclipse.jdt.core.dom.MethodDeclaration) declaration;
if (!methodDeclaration.getName().toString().equals(targetMethodName)) continue;
if (methodDeclaration.parameters().size() != targetMethodParameterTypes.size()) continue;
if (!isGenerated(methodDeclaration)) continue;
boolean parameterTypesEquals = true;
for (int i = 0; i < methodDeclaration.parameters().size(); i++) {
SingleVariableDeclaration variableDeclaration = (SingleVariableDeclaration)
methodDeclaration.parameters().get(i);
if (!variableDeclaration.getType().toString().equals(targetMethodParameterTypes.get(i))) {
parameterTypesEquals = false;
break;
```

```
}
}
if (parameterTypesEquals) {
return methodDeclaration;
}
}
}
}
return original;
}
// part of getRealMethodDeclarationNode
public static org.eclipse.jdt.core.dom.AbstractTypeDeclaration findTypeDeclaration(IType searchType,
List<?> nodes) {
for (Object object : nodes) {
if (object instanceof org.eclipse.jdt.core.dom.AbstractTypeDeclaration) {
org.eclipse.jdt.core.dom.AbstractTypeDeclaration typeDeclaration =
(org.eclipse.jdt.core.dom.AbstractTypeDeclaration) object;
if (typeDeclaration.getName().toString().equals(searchType.getElementName()))
return typeDeclaration;
}
}
return null;
}
public static int getSourceEndFixed(int sourceEnd, org.eclipse.jdt.internal.compiler.ast.ASTNode node)
throws Exception {
if (sourceEnd == -1) {
org.eclipse.jdt.internal.compiler.ast.ASTNode object =
(org.eclipse.jdt.internal.compiler.ast.ASTNode)node.getClass().getField("$generatedBy").get(node);
if (object != null) {
return object.sourceEnd;
}
}
return sourceEnd;
}
public static int fixRetrieveStartingCatchPosition(int original, int start) {
return original == -1 ? start : original;
}
public static int fixRetrieveIdentifierEndPosition(int original, int start, int end) {
if (original == -1) return end;
if (original < start) return end;
return original;
}
public static int fixRetrieveEllipsisStartPosition(int original, int end) {
return original == -1 ? end : original;
}
public static int fixRetrieveStartBlockPosition(int original, int start) {
return original == -1 ? start : original;
}
public static int fixRetrieveRightBraceOrSemiColonPosition(int original, int end) {
// if (original == -1) {
// Thread.dumpStack();
// }
return original == -1 ? end : original;
```

```
}
public static int fixRetrieveRightBraceOrSemiColonPosition(int retVal, AbstractMethodDeclaration amd)
{
if (retVal != -1 || amd == null) return retVal;
boolean isGenerated = ASTNode_generatedBy.get(amd) != null;
if (isGenerated) return amd.declarationSourceEnd;
return -1;
}
public static int fixRetrieveRightBraceOrSemiColonPosition(int retVal, FieldDeclaration fd) {
if (retVal != -1 || fd == null) return retVal;
boolean isGenerated = ASTNode_generatedBy.get(fd) != null;
if (isGenerated) return fd.declarationSourceEnd;
return -1;
}
public static int fixRetrieveProperRightBracketPosition(int retVal, Type type) {
if (retVal != -1 || type == null) return retVal;
if (isGenerated(type)) return type.getStartPosition() + type.getLength() - 1;
return -1;
}
public static final int ALREADY_PROCESSED_FLAG = 0x800000; //Bit 24
public static boolean checkBit24(Object node) throws Exception {
int bits = (Integer)(node.getClass().getField("bits").get(node));
return (bits & ALREADY_PROCESSED_FLAG) != 0;
}
public static boolean skipRewritingGeneratedNodes(org.eclipse.jdt.core.dom.ASTNode node) throws
Exception {
return ((Boolean) node.getClass().getField("$isGenerated").get(node)).booleanValue();
}
public static void setIsGeneratedFlag(org.eclipse.jdt.core.dom.ASTNode domNode,
org.eclipse.jdt.internal.compiler.ast.ASTNode internalNode) throws Exception {
if (internalNode == null || domNode == null) return;
boolean isGenerated = ASTNode_generatedBy.get(internalNode) != null;
if (isGenerated) domNode.getClass().getField("$isGenerated").set(domNode, true);
}
public static void setIsGeneratedFlagForName(org.eclipse.jdt.core.dom.Name name, Object
internalNode) throws Exception {
if (internalNode instanceof org.eclipse.jdt.internal.compiler.ast.ASTNode) {
boolean isGenerated = ASTNode_generatedBy.get((org.eclipse.jdt.internal.compiler.ast.ASTNode)
internalNode) != null;
if (isGenerated) name.getClass().getField("$isGenerated").set(name, true);
}
}
public static RewriteEvent[] listRewriteHandleGeneratedMethods(RewriteEvent parent) {
RewriteEvent[] children = parent.getChildren();
List<RewriteEvent> newChildren = new ArrayList<RewriteEvent>();
List<RewriteEvent> modifiedChildren = new ArrayList<RewriteEvent>();
for (int i = 0; i < children.length; i++) {
RewriteEvent child = children[i];
boolean isGenerated = isGenerated((org.eclipse.jdt.core.dom.ASTNode) child.getOriginalValue());
if (isGenerated) {
boolean isReplacedOrRemoved = child.getChangeKind() == RewriteEvent.REPLACED ||
child.getChangeKind() == RewriteEvent.REMOVED;
```

```
boolean convertingFromMethod = child.getOriginalValue() instanceof
org.eclipse.jdt.core.dom.MethodDeclaration;
if (isReplacedOrRemoved && convertingFromMethod && child.getNewValue() != null) {
modifiedChildren.add(new NodeRewriteEvent(null, child.getNewValue()));
}
} else {
newChildren.add(child);
}
}
// Since Eclipse doesn't honor the "insert at specified location" for already existing members,
// we'll just add them last
newChildren.addAll(modifiedChildren);
return newChildren.toArray(new RewriteEvent[0]);
}
public static int getTokenEndOffsetFixed(TokenScanner scanner, int token, int startOffset, Object
domNode) throws CoreException {
boolean isGenerated = false;
try {
isGenerated = (Boolean) domNode.getClass().getField("$isGenerated").get(domNode);
} catch (Exception e) {
// If this fails, better to break some refactor scripts than to crash eclipse.
}
if (isGenerated) return -1;
return scanner.getTokenEndOffset(token, startOffset);
}
public static IMethod[] removeGeneratedMethods(IMethod[] methods) throws Exception {
List<IMethod> result = new ArrayList<IMethod>();
for (IMethod m : methods) {
if (!isGenerated(m)) result.add(m);
}
return result.size() == methods.length ? methods : result.toArray(new IMethod[0]);
}
public static SearchResultGroup[] createFakeSearchResult(SearchResultGroup[] returnValue,
Object/*
* org.eclipse.jdt.internal.corext.refactoring.rename.
* RenameFieldProcessor
*/ processor) throws Exception {
if (returnValue == null || returnValue.length == 0) {
// if no matches were found, check if Data annotation is present on the class
Field declaredField = processor.getClass().getDeclaredField("fField");
if (declaredField != null) {
declaredField.setAccessible(true);
SourceField fField = (SourceField) declaredField.get(processor);
IAnnotation dataAnnotation = fField.getDeclaringType().getAnnotation("Data");
if (dataAnnotation != null) {
// add fake item, to make refactoring checks pass
return new SearchResultGroup[] {new SearchResultGroup(null, new SearchMatch[1])};
}
}
}
return returnValue;
}
public static SimpleName[] removeGeneratedSimpleNames(SimpleName[] in) throws Exception {
```

```
Field f = SimpleName.class.getField("$isGenerated");
int count = 0;
for (int i = 0; i < in.length; i++) {
if (in[i] == null || !((Boolean)f.get(in[i])).booleanValue()) count++;
}
if (count == in.length) return in;
SimpleName[] newSimpleNames = new SimpleName[count];
count = 0;
for (int i = 0; i < in.length; i++) {
if (in[i] == null || !((Boolean)f.get(in[i])).booleanValue()) newSimpleNames[count++] = in[i];
}
return newSimpleNames;
}
public static Name[] removeGeneratedNames(Name[] in) throws Exception {
Field f = Name.class.getField("$isGenerated");
int count = 0;
for (int i = 0; i < in.length; i++) {
if (in[i] == null || !((Boolean)f.get(in[i])).booleanValue()) count++;
}
if (count == in.length) return in;
Name[] newNames = new Name[count];
count = 0;
for (int i = 0; i < in.length; i++) {
if (in[i] == null || !((Boolean)f.get(in[i])).booleanValue()) newNames[count++] = in[i];
}
return newNames;
}
public static Annotation[] convertAnnotations(Annotation[] out, IAnnotatable annotatable) {
IAnnotation[] in;
try {
in = annotatable.getAnnotations();
} catch (Exception e) {
return out;
}
if (out == null) return null;
int toWrite = 0;
for (int idx = 0; idx < out.length; idx++) {
String oName = new String(out[idx].type.getLastToken());
boolean found = false;
for (IAnnotation i : in) {
String name = i.getElementName();
int li = name.lastIndexOf('.');
if (li > -1) name = name.substring(li + 1);
if (name.equals(oName)) {
found = true;
break;
}
}
if (!found) out[idx] = null;
else toWrite++;
}
Annotation[] replace = out;
if (toWrite < out.length) {
```

```
replace = new Annotation[toWrite];
int idx = 0;
for (int i = 0; i < out.length; i++) {
if (out[i] == null) continue;
replace[idx++] = out[i];
}
}
return replace;
}
public static String getRealNodeSource(String original, org.eclipse.jdt.internal.compiler.ast.ASTNode
node) {
if (!isGenerated(node)) return original;
return node.toString();
}
public static java.lang.String getRealNodeSource(java.lang.String original,
org.eclipse.jdt.core.dom.ASTNode node) throws Exception {
if (!isGenerated(node)) return original;
return node.toString();
}
public static boolean skipRewriteVisibility(IncomingMemberVisibilityAdjustment adjustment) {
return isGenerated(adjustment.getMember());
}
public static String[] getRealCodeBlocks(String[] blocks, SourceProvider sourceProvider, CallContext
callContext) {
MethodDeclaration methodDeclaration = sourceProvider.getDeclaration();
if (!isGenerated(methodDeclaration)) {
return blocks;
}
try {
// Replace parameter references with actual argument
AST ast = methodDeclaration.getAST();
List<?> parameters = methodDeclaration.parameters();
for (int i = 0; i < parameters.size(); i++) {
SingleVariableDeclaration param = (SingleVariableDeclaration) parameters.get(i);
Object data = param.getProperty("org.eclipse.jdt.internal.corext.refactoring.code.ParameterData");
List<SimpleName> names = Permit.get(Permit.permissiveGetField(data.getClass(), "fReferences"),
data);
for (SimpleName simpleName : names) {
ASTNode copy = ASTNode.copySubtree(ast, callContext.arguments[i]);
simpleName.getParent().setStructuralProperty(simpleName.getLocationInParent(), copy);
}
}
// Convert AST to source
StringBuilder sb = new StringBuilder();
for (Object statement : methodDeclaration.getBody().statements()) {
if (callContext.callMode != ASTNode.RETURN_STATEMENT && statement instanceof
ReturnStatement) {
ReturnStatement returnStatement = (ReturnStatement) statement;
sb.append(returnStatement.getExpression());
} else {
sb.append(statement);
}
}
```

```
return new String[] {sb.toString().trim()};
} catch (Throwable e) {
return blocks;
}
}
}
public static class FieldInitializer {
public static final Field INFO_STACK;
public static final Field FIELD_INFO;
public static final Field SOURCE_FIELD_ELEMENT_INFO;
public static final Field INITIALIZATION_SOURCE;
public static final Field NODE;
public static final boolean INITIALIZED;
static {
INFO_STACK = Permit.permissiveGetField(CompilationUnitStructureRequestor.class, "infoStack");
FIELD_INFO = Permit.permissiveGetField(CompilationUnitStructureRequestor.class, "$fieldInfo");
SOURCE_FIELD_ELEMENT_INFO =
Permit.permissiveGetField(CompilationUnitStructureRequestor.class, "$sourceFieldElementInfo");
INITIALIZATION_SOURCE = Permit.permissiveGetField(SourceFieldElementInfo.class,
"initializationSource");
NODE = Permit.permissiveGetField(FieldInfo.class, "node");
INITIALIZED = INFO_STACK != null && FIELD_INFO != null && SOURCE_FIELD_ELEMENT_INFO !=
null && INITIALIZATION_SOURCE != null && NODE != null;
}
public static boolean storeFieldInfo(CompilationUnitStructureRequestor
compilationUnitStructureRequestor) {
try {
if (INITIALIZED) {
Stack<?> infoStack = Permit.get(INFO_STACK, compilationUnitStructureRequestor);
Object fieldInfo = infoStack.peek();
Permit.set(FIELD_INFO, compilationUnitStructureRequestor, fieldInfo);
}
} catch (Exception e) {
// do not break eclipse
}
return false;
}
public static void storeSourceFieldElementInfo(SourceFieldElementInfo fieldInfo,
CompilationUnitStructureRequestor compilationUnitStructureRequestor) {
try {
if (INITIALIZED) {
Permit.set(SOURCE_FIELD_ELEMENT_INFO, compilationUnitStructureRequestor, fieldInfo);
}
} catch (Exception e) {
// do not break eclipse
}
}
public static void overwriteInitializer(CompilationUnitStructureRequestor
compilationUnitStructureRequestor) {
try {
if (INITIALIZED) {
FieldInfo fieldInfo = Permit.get(FIELD_INFO, compilationUnitStructureRequestor);
Permit.set(FIELD_INFO, compilationUnitStructureRequestor, null);
```

```
SourceFieldElementInfo sourceFieldElementInfo =
Permit.get(SOURCE_FIELD_ELEMENT_INFO,compilationUnitStructureRequestor);
Permit.set(SOURCE_FIELD_ELEMENT_INFO, compilationUnitStructureRequestor, null);
if (sourceFieldElementInfo.getInitializationSource() != null) {
AbstractVariableDeclaration node = Permit.get(NODE, fieldInfo);
if (PatchFixes.isGenerated(node)) {
Permit.set(INITIALIZATION_SOURCE, sourceFieldElementInfo,
node.initialization.toString().toCharArray());
}
}
}
} catch (Exception e) {
// do not break eclipse
}
}
}
public static class Tests {
public static StringBuffer printMethod(AbstractMethodDeclaration methodDeclaration, int tab,
StringBuffer output, TypeDeclaration type) {
return (StringBuffer) printMethod(methodDeclaration, tab, (Object) output, type);
}
public static StringBuilder printMethod(AbstractMethodDeclaration methodDeclaration, int tab,
StringBuilder output, TypeDeclaration type) {
return (StringBuilder) printMethod(methodDeclaration, tab, (Object) output, type);
}
public static Object printMethod(AbstractMethodDeclaration methodDeclaration, int tab, Object output,
TypeDeclaration type) {
Map<String, String> docs =
CompilationUnit_javadoc.get(methodDeclaration.compilationResult.compilationUnit);
Method printIndent = Permit.permissiveGetMethod(org.eclipse.jdt.internal.compiler.ast.ASTNode.class,
"printIndent", int.class, output.getClass());
if (docs != null) {
String signature = EclipseHandlerUtil.getSignature(type, methodDeclaration);
String rawJavadoc = docs.get(signature);
if (rawJavadoc != null) {
for (String line : rawJavadoc.split("\r?\n")) {
try {
Appendable sb = (Appendable) Permit.invoke(printIndent, null, tab, output);
sb.append(line).append("\n");
} catch (Throwable e) {
// Ignore
}
}
}
}
Method printMethodDeclaration = Permit.permissiveGetMethod(AbstractMethodDeclaration.class,
"print", int.class, output.getClass());
Permit.invokeSneaky(printMethodDeclaration, methodDeclaration, tab, output);
return output;
}
public static Object getBundle(Object original, Class<?> c) {
if (original != null) {
return original;
```

```
}
CodeSource codeSource = c.getProtectionDomain().getCodeSource();
if (codeSource == null) {
return null;
}
String jar = codeSource.getLocation().getFile();
String bundleName = jar.substring(jar.lastIndexOf("/") + 1, jar.indexOf("_"));
org.osgi.framework.Bundle[] bundles =
org.eclipse.core.runtime.adaptor.EclipseStarter.getSystemBundleContext().getBundles();
for (org.osgi.framework.Bundle bundle : bundles) {
if (bundleName.equals(bundle.getSymbolicName())) {
return bundle;
}
}
return null;
}
public static boolean isImplicitCanonicalConstructor(AbstractMethodDeclaration method, Object
parameter) {
return (method.bits & IsCanonicalConstructor) != 0 && (method.bits & IsImplicit) != 0;
}
public static StringBuffer returnStringBuffer(Object p1, StringBuffer buffer) {
return buffer;
}
public static StringBuilder returnStringBuilder(Object p1, StringBuilder buffer) {
return buffer;
}
}
}
```

lombok/launch/MavenEcjBootstrapAgent.java


```
/*
* Copyright (C) 2022 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
```

```
*/
package lombok.launch;
import java.lang.instrument.ClassFileTransformer;
import java.lang.instrument.IllegalClassFormatException;
import java.lang.instrument.Instrumentation;
import java.net.URL;
import java.net.URLClassLoader;
import java.security.ProtectionDomain;
import java.util.jar.JarFile;
/**
* This Java agent does not transform bytecode, but acts as a watcher that can
* figure out when it is appropriate to load Lombok itself within a Maven
* execution.
*
* It relies on several facts:
* <ul>
* <li>maven-compiler-plugin contains an {@code AbstractCompilerMojo} class that
* compiler instances extend.
* <li>Maven loaders are {@code ClassRealms}, which extend {@code URLClassLoader}.
* <li>Each plugin dependency in the <em>pom.xml </em>is represented as a file URL on the
* ClassRealm that points to the artifact.
* <li>URLs to Maven artifacts contain the group and artifact ids
* ({@code [...]/groupid/artifactid/ver/artifactid-ver.jar}).
* <li>The Lombok Java agent class is {@code lombok.launch.Agent}.
* </ul>
* Given all of the above, the transformer simply waits for {@code AbstractCompilerMojo}
* to be loaded, then uses the loader to find the path to the Lombok jar file,
* and finally loads the Lombok agent using reflection.
*/
public final class MavenEcjBootstrapAgent {
private static final String MAVEN_COMPILER_TRIGGER_CLASS =
"org/apache/maven/plugin/compiler/AbstractCompilerMojo";
private static final String LOMBOK_URL_IDENTIFIER = "/org/projectlombok/lombok/";
private static final String LOMBOK_AGENT_CLASS = "lombok.launch.Agent";
private static final byte[] NOT_TRANSFORMED = null;
private MavenEcjBootstrapAgent() {}
public static void premain(final String agentArgs, final Instrumentation instrumentation) {
instrumentation.addTransformer(new ClassFileTransformer() {
@Override public byte[] transform(final ClassLoader loader, final String className, final Class<?> cbr,
final ProtectionDomain pd, final byte[] cfb) throws IllegalClassFormatException {
if (MAVEN_COMPILER_TRIGGER_CLASS.equals(className)) {
for (final URL url : ((URLClassLoader) loader).getURLs()) {
if (url.getPath().contains(LOMBOK_URL_IDENTIFIER)) {
try {
instrumentation.appendToSystemClassLoaderSearch(new JarFile(url.getPath()));
MavenEcjBootstrapAgent.class.getClassLoader().loadClass(LOMBOK_AGENT_CLASS).getDeclared
Method("premain", String.class, Instrumentation.class).invoke(null, agentArgs, instrumentation);
instrumentation.removeTransformer(this);
break;
} catch (Exception e) {
// There are no appropriate loggers available at
// this point in time.
e.printStackTrace(System.err);
```

```
}
}
}
}
return NOT_TRANSFORMED;
}
});
}
}
```

lombok/installer/AppleNativeLook.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import java.awt.Image;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
/**
* Mac OS X specific code to gussy up the GUI a little bit, mostly with a nice dock icon. Well, nicer than
* the standard icon, at any rate.
*/
class AppleNativeLook {
public static void go() throws Exception {
Class<?> appClass = Class.forName("com.apple.eawt.Application");
Object app = appClass.getMethod("getApplication").invoke(null);
appClass.getMethod("removeAboutMenuItem").invoke(app);
appClass.getMethod("removePreferencesMenuItem").invoke(app);
BufferedImage image = ImageIO.read(AppleNativeLook.class.getResource("lombokIcon.png"));
appClass.getMethod("setDockIconImage", Image.class).invoke(app, image);
}
}
```

lombok/installer/CorruptedIdeLocationException.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
/**
* Represents an installation location problem.
* You should throw it upon creation of a {@code IdeLocation} class
* if the provided location looks like your kind of IDE but there's something wrong with it.
*/
public class CorruptedIdeLocationException extends Exception {
private final String ideType;
public CorruptedIdeLocationException(String message, String ideType, Throwable cause) {
super(message, cause);
this.ideType = ideType;
}
public String getIdeType() {
return ideType;
}
void showDialog(JFrame appWindow) {
JOptionPane.showMessageDialog(appWindow, getMessage(), "Cannot configure " + ideType + "
installation", JOptionPane.WARNING_MESSAGE);
}
}
```

lombok/installer/IdeLocation.java

/*
* Copyright (C) 2009-2016 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import java.io.File;
import java.io.IOException;
import java.net.URL;
import lombok.patcher.ClassRootFinder;
/**
* Represents a location that contains an IDE.
*/
public abstract class IdeLocation {
/** Toggling the 'selected' checkbox in the GUI is tracked via this boolean */
boolean selected = true;
public abstract String install() throws InstallException;
public abstract void uninstall() throws UninstallException;
public abstract String getName();
public abstract boolean hasLombok();
public abstract URL getIdeIcon();
/**
* Returns a File object pointing to our own jar file. Will obviously fail if the installer was started via
* a jar that wasn't accessed via the file-system, or if its started via e.g. unpacking the jar.
*/
public static File findOurJar() {
return new File(ClassRootFinder.findClassRootOfClass(OsUtils.class));
}
@Override public String toString() {
return getName();
}
/**
* Returns a full path to the provided file.
* Returns the canonical path, unless that is not available, in which case it returns the absolute path.
*/
public static String canonical(File p) {

```
try {
return p.getCanonicalPath();
} catch (IOException e) {
String x = p.getAbsolutePath();
return x == null ? p.getPath() : x;
}
}
}
```

lombok/installer/IdeLocationProvider.java

```
/*
* Copyright (C) 2009-2016 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import java.util.List;
import java.util.regex.Pattern;
public interface IdeLocationProvider {
/**
* @throws CorruptedIdeLocationException
* Only throw this exception if the location seems like a proper installation except there's something
wrong with it.
* Do not throw it (just return {@code null}) if there's nothing there or it looks absolutely nothing like your
IDE.
*/
IdeLocation create(String path) throws CorruptedIdeLocationException;
/**
* Return the usual name of the IDE executable or other obvious marker of an IDE installation on the
current platform.
*/
Pattern getLocationSelectors();
/**
* Look for installations of your IDE in the usual places.
```

*
* @param locations Add to this list any valid locations that you found.
* @param problems
* Add to this list any locations that look like installations,
* but have problems that prevent you from installing/uninstalling from them. DONT add to this list
* any common locations that have no installation at all - only add near misses.
*/
void findIdes(List<IdeLocation> locations, List<CorruptedIdeLocationException> problems);
}

lombok/installer/InstallException.java

/*
* Copyright (C) 2009-2010 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
/**
* Thrown when installation of lombok into an IDE fails.
*/
public class InstallException extends Exception {
private boolean warning;
public InstallException(String message, Throwable cause) {
super(message, cause);
}
public InstallException(boolean warning, String message, Throwable cause) {
super(message, cause);
this.warning = warning;
}
public boolean isWarning() {
return warning;
}
}

lombok/installer/Installer.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import java.awt.HeadlessException;
import java.io.File;
import java.io.IOException;
import java.net.URI;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.regex.Pattern;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import lombok.Lombok;
import lombok.core.LombokApp;
import lombok.core.SpiLoadUtil;
import lombok.core.Version;
import lombok.installer.OsUtils.OS;
import lombok.patcher.ClassRootFinder;
import lombok.spi.Provides;
import com.zwitserloot.cmdreader.CmdReader;
import com.zwitserloot.cmdreader.Description;
import com.zwitserloot.cmdreader.InvalidCommandLineException;
import com.zwitserloot.cmdreader.Sequential;
import com.zwitserloot.cmdreader.Shorthand;
/**
* The lombok installer proper.
* Uses swing to show a simple GUI that can add and remove the java agent to Eclipse installations.
* Also offers info on what this installer does in case people want to instrument their Eclipse manually,
```

```
* and looks in some common places on Mac OS X, Linux and Windows.
*/
public class Installer {
static final URI ABOUT_LOMBOK_URL = URI.create("https://projectlombok.org");
static final List<IdeLocationProvider> locationProviders;
static {
List<IdeLocationProvider> list = new ArrayList<IdeLocationProvider>();
try {
for (IdeLocationProvider provider : SpiLoadUtil.findServices(IdeLocationProvider.class)) {
list.add(provider);
}
} catch (IOException e) {
throw Lombok.sneakyThrow(e);
}
locationProviders = Collections.unmodifiableList(list);
}
static List<Pattern> getIdeExecutableNames() {
List<Pattern> list = new ArrayList<Pattern>();
for (IdeLocationProvider provider : locationProviders) {
Pattern p = provider.getLocationSelectors();
if (p != null) list.add(p);
}
return list;
}
static IdeLocation tryAllProviders(String location) throws CorruptedIdeLocationException {
for (IdeLocationProvider provider : locationProviders) {
IdeLocation loc = provider.create(location);
if (loc != null) return loc;
}
return null;
}
static void autoDiscover(List<IdeLocation> locations, List<CorruptedIdeLocationException> problems) {
for (IdeLocationProvider provider : locationProviders) {
provider.findIdes(locations, problems);
}
}
public static boolean isSelf(String jar) {
String self = ClassRootFinder.findClassRootOfClass(Installer.class);
if (self == null) return false;
File a = new File(jar).getAbsoluteFile();
File b = new File(self).getAbsoluteFile();
try { a = a.getCanonicalFile(); } catch (IOException ignore) {}
try { b = b.getCanonicalFile(); } catch (IOException ignore) {}
return a.equals(b);
}
@Provides
public static class GraphicalInstallerApp extends LombokApp {
@Override public String getAppName() {
return "installer";
}
@Override public String getAppDescription() {
return "Runs the graphical installer tool (default).";
}
```

```
@Override public List<String> getAppAliases() {
return Arrays.asList("");
}
@Override public int runApp(List<String> args) throws Exception {
return guiInstaller();
}
}
@Provides
public static class CommandLineInstallerApp extends LombokApp {
@Override public String getAppName() {
return "install";
}
@Override public String getAppDescription() {
return "Runs the 'handsfree' command line scriptable installer.";
}
@Override public int runApp(List<String> args) throws Exception {
return cliInstaller(false, args);
}
}
@Provides
public static class CommandLineUninstallerApp extends LombokApp {
@Override public String getAppName() {
return "uninstall";
}
@Override public String getAppDescription() {
return "Runs the 'handsfree' command line scriptable uninstaller.";
}
@Override public int runApp(List<String> args) throws Exception {
return cliInstaller(true, args);
}
}
private static int guiInstaller() {
if (OsUtils.getOS() == OS.MAC_OS_X) {
System.setProperty("com.apple.mrj.application.apple.menu.about.name", "Lombok Installer");
System.setProperty("com.apple.macos.use-file-dialog-packages", "true");
}
try {
SwingUtilities.invokeLater(new Runnable() {
public void run() {
try {
try {
UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
} catch (Exception ignore) {}
new InstallerGUI().show();
} catch (HeadlessException e) {
printHeadlessInfo();
}
}
});
synchronized (InstallerGUI.exitMarker) {
while (!Thread.interrupted() && InstallerGUI.exitMarker.get() == null) {
try {
InstallerGUI.exitMarker.wait();
```

```
} catch (InterruptedException e) {
return 1;
}
}
Integer errCode = InstallerGUI.exitMarker.get();
return errCode == null ? 1 : errCode;
}
} catch (HeadlessException e) {
printHeadlessInfo();
return 1;
}
}
private static class CmdArgs {
@Description("Specify paths to a location to install/uninstall. Use 'auto' to apply to all automatically
discoverable installations.")
@Sequential
List<String> path = new ArrayList<String>();
@Shorthand({"h", "?"})
@Description("Shows this help text")
boolean help;
}
public static int cliInstaller(boolean uninstall, List<String> rawArgs) {
CmdReader<CmdArgs> reader = CmdReader.of(CmdArgs.class);
CmdArgs args;
try {
args = reader.make(rawArgs.toArray(new String[0]));
} catch (InvalidCommandLineException e) {
System.err.println(e.getMessage());
System.err.println("--------------------------");
System.err.println(generateCliHelp(uninstall, reader));
return 1;
}
if (args.help) {
System.out.println(generateCliHelp(uninstall, reader));
return 0;
}
if (args.path.isEmpty()) {
System.err.println("ERROR: Nothing to do!");
System.err.println("--------------------------");
System.err.println(generateCliHelp(uninstall, reader));
return 1;
}
final List<IdeLocation> locations = new ArrayList<IdeLocation>();
final List<CorruptedIdeLocationException> problems = new
ArrayList<CorruptedIdeLocationException>();
if (args.path.contains("auto")) autoDiscover(locations, problems);
for (String rawPath : args.path) {
if (!rawPath.equals("auto")) {
try {
IdeLocation loc = tryAllProviders(rawPath);
if (loc != null) locations.add(loc);
else problems.add(new CorruptedIdeLocationException("Can't find any IDE at: " + rawPath, null, null));
} catch (CorruptedIdeLocationException e) {
```

```
problems.add(e);
}
}
}
int validLocations = locations.size();
for (IdeLocation loc : locations) {
try {
if (uninstall) {
loc.uninstall();
} else {
loc.install();
}
System.out.printf("Lombok %s %s: %s\n", uninstall ? "uninstalled" : "installed", uninstall ? "from" : "to",
loc.getName());
} catch (InstallException e) {
if (e.isWarning()) {
System.err.printf("Warning while installing at %s:\n", loc.getName());
} else {
System.err.printf("Installation at %s failed:\n", loc.getName());
validLocations--;
}
System.err.println(e.getMessage());
} catch (UninstallException e) {
if (e.isWarning()) {
System.err.printf("Warning while uninstalling at %s:\n", loc.getName());
} else {
System.err.printf("Uninstall at %s failed:\n", loc.getName());
validLocations--;
}
System.err.println(e.getMessage());
}
}
for (CorruptedIdeLocationException problem : problems) {
System.err.println("WARNING: " + problem.getMessage());
}
if (validLocations == 0) {
System.err.println("WARNING: Zero valid locations found; so nothing was done!");
}
return 0;
}
private static String generateCliHelp(boolean uninstall, CmdReader<CmdArgs> reader) {
return reader.generateCommandLineHelp("java -jar lombok.jar " + (uninstall ? "uninstall" : "install"));
}
/**
* If run in headless mode, the installer can't show its fancy GUI. There's little point in running
* the installer without a GUI environment, as Eclipse doesn't run in headless mode either, so
* we'll make do with showing some basic info on Lombok as well as instructions for using lombok with
javac.
*/
private static void printHeadlessInfo() {
System.out.printf("About lombok v%s\n" +
"Lombok makes java better by providing very spicy additions to the Java programming language," +
"such as using @Getter to automatically generate a getter method for any field.\n\n" +
```

```
"Browse to %s for more information. To install lombok on Eclipse, re-run this jar file on a " +
"graphical computer system - this message is being shown because your terminal is not graphics capable.\n" +
"Alternatively, use the command line installer (java -jar lombok.jar install --help).\n" +
"If you are just using 'javac' or a tool that calls on javac, no installation is neccessary; just " +
"make sure lombok.jar is in the classpath when you compile. Example:\n\n" +
" java -cp lombok.jar MyCode.java\n",
Version.getVersion(), ABOUT_LOMBOK_URL);
}
}
```

lombok/installer/InstallerGUI.java

```
/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import java.awt.Color;
import java.awt.Component;
import java.awt.Container;
import java.awt.Cursor;
import java.awt.Dimension;
import java.awt.FileDialog;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.Rectangle;
import java.awt.Toolkit;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.font.TextAttribute;
```

```
import java.io.File;
import java.io.FilenameFilter;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.net.URI;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;
import java.util.regex.Pattern;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.ImageIcon;
import javax.swing.JButton;
import javax.swing.JCheckBox;
import javax.swing.JComponent;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextPane;
import javax.swing.Scrollable;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import javax.swing.filechooser.FileFilter;
import javax.swing.text.html.HTMLDocument;
import lombok.core.Version;
import lombok.installer.OsUtils.OS;
/**
* The lombok GUI installer.
* Uses swing to show a simple GUI that can add and remove the java agent to IDE installations.
* Also offers info on what this installer does in case people want to instrument their IDE manually.
*/
public class InstallerGUI {
private static final int INSTALLER_WINDOW_WIDTH = 662;
static final AtomicReference<Integer> exitMarker = new AtomicReference<Integer>();
private JFrame appWindow;
private JComponent loadingExpl;
private Component javacArea;
private Component ideArea;
private Component uninstallArea;
private Component howIWorkArea;
private Component successArea;
private Box uninstallBox;
private JHyperLink uninstallButton;
private JLabel uninstallPlaceholder;
private JButton installButton;
```

```
private List<IdeLocation> toUninstall;
private final Set<String> installSpecificMessages = new LinkedHashSet<String>();
/**
* Creates a new installer that starts out invisible.
* Call the {@link #show()} method on a freshly created installer to render it.
*/
public InstallerGUI() {
appWindow = new JFrame(String.format("Project Lombok v%s - Installer", Version.getVersion()));
appWindow.setLocationByPlatform(true);
appWindow.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
appWindow.setResizable(false);
appWindow.setIconImage(Toolkit.getDefaultToolkit().getImage(Installer.class.getResource("lombokIco
n.png")));
try {
javacArea = buildJavacArea();
ideArea = buildIdeArea();
uninstallArea = buildUninstallArea();
uninstallArea.setVisible(false);
howIWorkArea = buildHowIWorkArea();
howIWorkArea.setVisible(false);
successArea = buildSuccessArea();
successArea.setVisible(false);
buildChrome(appWindow.getContentPane());
appWindow.pack();
} catch (Throwable t) {
handleException(t);
}
}
private void handleException(final Throwable t) {
SwingUtilities.invokeLater(new Runnable() {
@Override public void run() {
JOptionPane.showMessageDialog(appWindow, "There was a problem during the installation
process:\n" + t, "Uh Oh!", JOptionPane.ERROR_MESSAGE);
t.printStackTrace();
System.exit(1);
}
});
}
private Component buildHowIWorkArea() {
JPanel container = new JPanel();
container.setLayout(new GridBagLayout());
GridBagConstraints constraints = new GridBagConstraints();
constraints.anchor = GridBagConstraints.WEST;
container.add(new JLabel(HOW_I_WORK_TITLE), constraints);
constraints.gridy = 1;
constraints.insets = new Insets(8, 0, 0, 16);
container.add(new JLabel(String.format(HOW_I_WORK_EXPLANATION, File.separator)),
constraints);
Box buttonBar = Box.createHorizontalBox();
JButton backButton = new JButton("Okay - Good to know!");
buttonBar.add(Box.createHorizontalGlue());
buttonBar.add(backButton);
backButton.addActionListener(new ActionListener() {
```

```
@Override public void actionPerformed(ActionEvent e) {
howIWorkArea.setVisible(false);
javacArea.setVisible(true);
ideArea.setVisible(true);
successArea.setVisible(false);
appWindow.pack();
}
});
constraints.gridy = 2;
container.add(buttonBar, constraints);
container.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH, 415));
container.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH, 415));
return container;
}
private void showSuccess(String installSpecific) {
successExplanation.setText(SUCCESS_EXPLANATION.replace("%%%", installSpecific));
howIWorkArea.setVisible(false);
javacArea.setVisible(false);
ideArea.setVisible(false);
successArea.setVisible(true);
appWindow.pack();
}
private JLabel successExplanation;
private Component buildSuccessArea() {
JPanel container = new JPanel();
container.setLayout(new GridBagLayout());
GridBagConstraints constraints = new GridBagConstraints();
constraints.anchor = GridBagConstraints.WEST;
JLabel title;
container.add(title = new JLabel(SUCCESS_TITLE), constraints);
title.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH - 82, 20));
title.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH - 82, 20));
constraints.gridy = 1;
constraints.insets = new Insets(8, 0, 0, 16);
container.add(successExplanation = new JLabel(SUCCESS_EXPLANATION), constraints);
successExplanation.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH - 82, 175));
successExplanation.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH - 82, 175));
constraints.gridy++;
constraints.fill = GridBagConstraints.BOTH;
JTextPane notes = new JTextPane();
notes.setContentType("text/html");
notes.setText(readChangeLog());
notes.setEditable(false);
notes.setOpaque(false);
notes.setBorder(null);
notes.setSelectionStart(0);
notes.setSelectionEnd(0);
Font font = UIManager.getFont("Label.font");
String bodyRule = "body { font-family: " + font.getFamily() + "; font-size: " + font.getSize() + "pt; }";
((HTMLDocument) notes.getDocument()).getStyleSheet().addRule(bodyRule);
JScrollPane scroller = new JScrollPane(notes);
container.add(scroller, constraints);
scroller.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH - 82, 200));
```

```
scroller.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH - 82, 200));
container.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH, 415));
container.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH, 415));
return container;
}
private String readChangeLog() {
InputStream in = Installer.class.getResourceAsStream("/latestchanges.html");
try {
char[] buff = new char[8192];
StringBuilder contents = new StringBuilder();
InputStreamReader reader = new InputStreamReader(in, "UTF-8");
while (true) {
int read = reader.read(buff);
if (read == -1) break;
contents.append(buff, 0, read);
}
return "<html>" + contents + "</html>";
} catch (Exception e) {
return "No Changelog available";
}
finally {
try {
in.close();
} catch (Exception ignore){ /**/}
}
}
private Component buildUninstallArea() {
JPanel container = new JPanel();
container.setLayout(new GridBagLayout());
GridBagConstraints constraints = new GridBagConstraints();
constraints.anchor = GridBagConstraints.WEST;
container.add(new JLabel(UNINSTALL_TITLE), constraints);
constraints.gridy = 1;
constraints.insets = new Insets(8, 0, 0, 16);
container.add(new JLabel(UNINSTALL_EXPLANATION), constraints);
uninstallBox = Box.createVerticalBox();
constraints.gridy = 2;
constraints.fill = GridBagConstraints.HORIZONTAL;
container.add(uninstallBox, constraints);
constraints.fill = GridBagConstraints.HORIZONTAL;
constraints.gridy = 3;
container.add(new JLabel("Are you sure?"), constraints);
Box buttonBar = Box.createHorizontalBox();
JButton noButton = new JButton("No - Don't uninstall");
buttonBar.add(noButton);
buttonBar.add(Box.createHorizontalGlue());
JButton yesButton = new JButton("Yes - uninstall Lombok");
buttonBar.add(yesButton);
noButton.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent e) {
uninstallArea.setVisible(false);
javacArea.setVisible(true);
ideArea.setVisible(true);
```

```
appWindow.pack();
}
});
yesButton.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent e) {
doUninstall();
}
});
constraints.gridy = 4;
container.add(buttonBar, constraints);
container.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH, 415));
container.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH, 415));
return container;
}
private Component buildJavacArea() {
JPanel container = new JPanel();
container.setLayout(new GridBagLayout());
GridBagConstraints constraints = new GridBagConstraints();
constraints.anchor = GridBagConstraints.WEST;
constraints.insets = new Insets(8, 0, 0, 16);
container.add(new JLabel(JAVAC_TITLE), constraints);
constraints.gridy = 1;
constraints.weightx = 1.0;
constraints.fill = GridBagConstraints.HORIZONTAL;
container.add(new JLabel(JAVAC_EXPLANATION), constraints);
JLabel example = new JLabel(JAVAC_EXAMPLE);
constraints.gridy = 2;
container.add(example, constraints);
container.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH, 105));
container.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH, 105));
return container;
}
private Component buildIdeArea() {
JPanel container = new JPanel();
container.setLayout(new GridBagLayout());
GridBagConstraints constraints = new GridBagConstraints();
constraints.anchor = GridBagConstraints.WEST;
constraints.insets = new Insets(8, 0, 0, 16);
container.add(new JLabel(IDE_TITLE), constraints);
constraints.gridy = 1;
container.add(new JLabel(IDE_EXPLANATION), constraints);
constraints.gridy = 2;
loadingExpl = Box.createHorizontalBox();
loadingExpl.add(new JLabel(new ImageIcon(Installer.class.getResource("loading.gif"))));
loadingExpl.add(new JLabel(IDE_LOADING_EXPLANATION));
container.add(loadingExpl, constraints);
constraints.weightx = 1.0;
constraints.gridy = 3;
constraints.fill = GridBagConstraints.HORIZONTAL;
idesList = new IdesList();
JScrollPane idesListScroll = new JScrollPane(idesList);
idesListScroll.setBackground(Color.WHITE);
idesListScroll.getViewport().setBackground(Color.WHITE);
```

```
container.add(idesListScroll, constraints);
Thread findIdesThread = new Thread() {
@Override public void run() {
try {
final List<IdeLocation> locations = new ArrayList<IdeLocation>();
final List<CorruptedIdeLocationException> problems = new
ArrayList<CorruptedIdeLocationException>();
Installer.autoDiscover(locations, problems);
SwingUtilities.invokeLater(new Runnable() {
@Override public void run() {
for (IdeLocation location : locations) {
try {
idesList.addLocation(location);
} catch (Throwable t) {
handleException(t);
}
}
for (CorruptedIdeLocationException problem : problems) {
problem.showDialog(appWindow);
}
loadingExpl.setVisible(false);
if (locations.size() + problems.size() == 0) {
JOptionPane.showMessageDialog(appWindow,
"I can't find any IDEs on your computer.\n" +
"If you have IDEs installed on this computer, please use the " +
"'Specify Location...' button to manually point out the \n" +
"location of your IDE installation to me. Thanks!",
"Can't find IDE", JOptionPane.INFORMATION_MESSAGE);
}
}
});
} catch (Throwable t) {
handleException(t);
}
}
};
findIdesThread.start();
Box buttonBar = Box.createHorizontalBox();
JButton specifyIdeLocationButton = new JButton("Specify location...");
buttonBar.add(specifyIdeLocationButton);
specifyIdeLocationButton.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent event) {
final List<Pattern> exeNames = Installer.getIdeExecutableNames();
String file = null;
if (OsUtils.getOS() == OS.MAC_OS_X) {
FileDialog chooser = new FileDialog(appWindow);
chooser.setMode(FileDialog.LOAD);
chooser.setFilenameFilter(new FilenameFilter() {
@Override public boolean accept(File dir, String fileName) {
for (Pattern exeName : exeNames) if (exeName.matcher(fileName).matches()) return true;
return false;
}
});
```

```
chooser.setVisible(true);
if (chooser.getDirectory() != null && chooser.getFile() != null) {
file = new File(chooser.getDirectory(), chooser.getFile()).getAbsolutePath();
}
} else {
JFileChooser chooser = new JFileChooser();
chooser.setAcceptAllFileFilterUsed(false);
chooser.setFileSelectionMode(JFileChooser.FILES_AND_DIRECTORIES);
chooser.setFileFilter(new FileFilter() {
@Override public boolean accept(File f) {
if (f.isDirectory()) return true;
for (Pattern exeName : exeNames) if (exeName.matcher(f.getName()).matches()) return true;
return false;
}
@Override public String getDescription() {
return "IDE Installation";
}
});
switch (chooser.showDialog(appWindow, "Select")) {
case JFileChooser.APPROVE_OPTION:
file = chooser.getSelectedFile().getAbsolutePath();
}
}
if (file != null) {
try {
IdeLocation loc = Installer.tryAllProviders(file);
if (loc != null) idesList.addLocation(loc);
else JOptionPane.showMessageDialog(appWindow, "I can't find any IDE that lombok supports at
location: " + file, "No IDE found", JOptionPane.WARNING_MESSAGE);
} catch (CorruptedIdeLocationException e) {
e.showDialog(appWindow);
} catch (Throwable t) {
handleException(t);
}
}
}
});
buttonBar.add(Box.createHorizontalGlue());
installButton = new JButton("Install / Update");
buttonBar.add(installButton);
installButton.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent e) {
List<IdeLocation> locationsToInstall = new ArrayList<IdeLocation>(idesList.getSelectedIdes());
if (locationsToInstall.isEmpty()) {
JOptionPane.showMessageDialog(appWindow, "You haven't selected any IDE installations!.", "No
Selection", JOptionPane.WARNING_MESSAGE);
return;
}
install(locationsToInstall);
}
});
constraints.gridy = 4;
constraints.weightx = 0;
```

```
container.add(buttonBar, constraints);
constraints.gridy = 5;
constraints.fill = GridBagConstraints.NONE;
JHyperLink showMe = new JHyperLink("Show me what this installer will do to my IDE installation.");
container.add(showMe, constraints);
showMe.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent e) {
showWhatIDo();
}
});
constraints.gridy = 6;
uninstallButton = new JHyperLink("Uninstall lombok from selected IDE installations.");
uninstallPlaceholder = new JLabel("<html> </html>");
uninstallButton.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent e) {
List<IdeLocation> locationsToUninstall = new ArrayList<IdeLocation>();
for (IdeLocation location : idesList.getSelectedIdes()) {
if (location.hasLombok()) locationsToUninstall.add(location);
}
if (locationsToUninstall.isEmpty()) {
JOptionPane.showMessageDialog(appWindow, "You haven't selected any IDE installations that have
been lombok-enabled.", "No Selection", JOptionPane.WARNING_MESSAGE);
return;
}
uninstall(locationsToUninstall);
}
});
container.add(uninstallButton, constraints);
uninstallPlaceholder.setVisible(false);
container.add(uninstallPlaceholder, constraints);
container.setPreferredSize(new Dimension(INSTALLER_WINDOW_WIDTH, 296));
container.setMinimumSize(new Dimension(INSTALLER_WINDOW_WIDTH, 296));
return container;
}
private void showWhatIDo() {
javacArea.setVisible(false);
ideArea.setVisible(false);
howIWorkArea.setVisible(true);
successArea.setVisible(false);
appWindow.pack();
}
private void uninstall(List<IdeLocation> locations) {
javacArea.setVisible(false);
ideArea.setVisible(false);
uninstallBox.removeAll();
uninstallBox.add(Box.createRigidArea(new Dimension(1, 16)));
for (IdeLocation location : locations) {
JLabel label = new JLabel(location.getName());
label.setFont(label.getFont().deriveFont(Font.BOLD));
uninstallBox.add(label);
}
uninstallBox.add(Box.createRigidArea(new Dimension(1, 16)));
toUninstall = locations;
```

```
uninstallArea.setVisible(true);
appWindow.pack();
}
private void install(final List<IdeLocation> toInstall) {
JPanel spinner = new JPanel();
spinner.setOpaque(true);
spinner.setLayout(new FlowLayout());
spinner.add(new JLabel(new ImageIcon(Installer.class.getResource("loading.gif"))));
final Container appWindowContent = appWindow.getContentPane();
appWindow.setContentPane(spinner);
final AtomicInteger successes = new AtomicInteger();
final AtomicBoolean failure = new AtomicBoolean();
new Thread() {
@Override public void run() {
for (IdeLocation loc : toInstall) {
try {
installSpecificMessages.add(loc.install());
successes.incrementAndGet();
} catch (final InstallException e) {
if (e.isWarning()) {
try {
SwingUtilities.invokeAndWait(new Runnable() {
@Override public void run() {
JOptionPane.showMessageDialog(appWindow,
e.getMessage(), "Install Problem", JOptionPane.WARNING_MESSAGE);
}
});
} catch (Exception e2) {
e2.printStackTrace();
//Shouldn't happen.
throw new RuntimeException(e2);
}
} else {
failure.set(true);
try {
SwingUtilities.invokeAndWait(new Runnable() {
@Override public void run() {
JOptionPane.showMessageDialog(appWindow,
e.getMessage(), "Install Problem", JOptionPane.ERROR_MESSAGE);
}
});
} catch (Exception e2) {
e2.printStackTrace();
//Shouldn't happen.
throw new RuntimeException(e2);
}
}
}
}
if (successes.get() > 0) {
try {
SwingUtilities.invokeAndWait(new Runnable() {
@Override public void run() {
```

```
appWindow.setContentPane(appWindowContent);
appWindow.pack();
StringBuilder installSpecific = new StringBuilder();
for (String installSpecificMessage : installSpecificMessages) {
installSpecific.append("<br>").append(installSpecificMessage);
}
showSuccess(installSpecific.toString());
}
});
} catch (Exception e) {
// Shouldn't happen.
throw new RuntimeException(e);
}
} else {
SwingUtilities.invokeLater(new Runnable() {
@Override public void run() {
synchronized (exitMarker) {
exitMarker.set(failure.get() ? 1 : 0);
exitMarker.notifyAll();
}
}
});
}
}
}.start();
}
private void doUninstall() {
JPanel spinner = new JPanel();
spinner.setOpaque(true);
spinner.setLayout(new FlowLayout());
spinner.add(new JLabel(new ImageIcon(Installer.class.getResource("/lombok/installer/loading.gif"))));
final Container originalContentPane = appWindow.getContentPane();
appWindow.setContentPane(spinner);
final AtomicInteger successes = new AtomicInteger();
new Thread(new Runnable() {
@Override public void run() {
for (IdeLocation loc : toUninstall) {
try {
loc.uninstall();
successes.incrementAndGet();
} catch (final UninstallException e) {
if (e.isWarning()) {
try {
SwingUtilities.invokeAndWait(new Runnable() {
@Override public void run() {
JOptionPane.showMessageDialog(appWindow,
e.getMessage(), "Uninstall Problem", JOptionPane.WARNING_MESSAGE);
}
});
} catch (Exception e2) {
e2.printStackTrace();
//Shouldn't happen.
throw new RuntimeException(e2);
```

```
}
} else {
try {
SwingUtilities.invokeAndWait(new Runnable() {
@Override public void run() {
JOptionPane.showMessageDialog(appWindow,
e.getMessage(), "Uninstall Problem", JOptionPane.ERROR_MESSAGE);
}
});
} catch (Exception e2) {
e2.printStackTrace();
//Shouldn't happen.
throw new RuntimeException(e2);
}
}
}
}
SwingUtilities.invokeLater(new Runnable() {
@Override public void run() {
if (successes.get() > 0) {
JOptionPane.showMessageDialog(appWindow, "Lombok has been removed from the selected IDE
installations.", "Uninstall successful", JOptionPane.INFORMATION_MESSAGE);
appWindow.setVisible(false);
System.exit(0);
return;
}
appWindow.setContentPane(originalContentPane);
}
});
}
}).start();
}
private IdesList idesList = new IdesList();
void selectedLomboksChanged(List<IdeLocation> selectedIdes) {
boolean uninstallAvailable = false;
boolean installAvailable = false;
for (IdeLocation loc : selectedIdes) {
if (loc.hasLombok()) uninstallAvailable = true;
installAvailable = true;
}
uninstallButton.setVisible(uninstallAvailable);
uninstallPlaceholder.setVisible(!uninstallAvailable);
installButton.setEnabled(installAvailable);
}
private class IdesList extends JPanel implements Scrollable {
private static final long serialVersionUID = 1L;
List<IdeLocation> locations = new ArrayList<IdeLocation>();
IdesList() {
setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
setBackground(Color.WHITE);
}
List<IdeLocation> getSelectedIdes() {
List<IdeLocation> list = new ArrayList<IdeLocation>();
```

```
for (IdeLocation loc : locations) if (loc.selected) list.add(loc);
return list;
}
void fireSelectionChange() {
selectedLomboksChanged(getSelectedIdes());
}
void addLocation(final IdeLocation location) {
if (locations.contains(location)) return;
Box box = Box.createHorizontalBox();
box.setBackground(Color.WHITE);
final JCheckBox checkbox = new JCheckBox(location.getName());
checkbox.setBackground(Color.WHITE);
box.add(new JLabel(new ImageIcon(location.getIdeIcon())));
box.add(checkbox);
checkbox.setSelected(true);
checkbox.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent e) {
location.selected = checkbox.isSelected();
fireSelectionChange();
}
});
if (location.hasLombok()) {
box.add(new JLabel(new ImageIcon(Installer.class.getResource("lombokIcon.png"))));
}
box.add(Box.createHorizontalGlue());
locations.add(location);
add(box);
getParent().doLayout();
fireSelectionChange();
}
@Override public Dimension getPreferredScrollableViewportSize() {
return new Dimension(1, 100);
}
@Override public int getScrollableBlockIncrement(Rectangle visibleRect, int orientation, int direction) {
return 12;
}
@Override public boolean getScrollableTracksViewportHeight() {
return false;
}
@Override public boolean getScrollableTracksViewportWidth() {
return true;
}
@Override public int getScrollableUnitIncrement(Rectangle visibleRect, int orientation, int direction) {
return 1;
}
}
private void buildChrome(Container appWindowContainer) {
JLabel leftGraphic = new JLabel(new ImageIcon(Installer.class.getResource("lombok.png")));
GridBagConstraints constraints = new GridBagConstraints();
appWindowContainer.setLayout(new GridBagLayout());
constraints.gridheight = 3;
constraints.gridwidth = 1;
constraints.gridx = 0;
```

```
constraints.gridy = 0;
constraints.insets = new Insets(8, 8, 8, 8);
appWindowContainer.add(leftGraphic, constraints);
constraints.insets = new Insets(0, 0, 0, 0);
constraints.gridx++;
constraints.gridy++;
constraints.gridheight = 1;
constraints.fill = GridBagConstraints.HORIZONTAL;
constraints.ipadx = 16;
constraints.ipady = 14;
appWindowContainer.add(javacArea, constraints);
constraints.gridy++;
appWindowContainer.add(ideArea, constraints);
appWindowContainer.add(uninstallArea, constraints);
appWindowContainer.add(howIWorkArea, constraints);
appWindowContainer.add(successArea, constraints);
constraints.gridy++;
constraints.gridwidth = 2;
constraints.gridx = 0;
constraints.weightx = 0;
constraints.weighty = 0;
constraints.ipadx = 0;
constraints.ipady = 0;
constraints.fill = GridBagConstraints.HORIZONTAL;
constraints.anchor = GridBagConstraints.SOUTHWEST;
constraints.insets = new Insets(0, 16, 8, 8);
appWindow.add(buildButtonBar(), constraints);
}
private Box buildButtonBar() {
Box buttonBar = Box.createHorizontalBox();
JHyperLink aboutLink = new JHyperLink(Installer.ABOUT_LOMBOK_URL.toString());
aboutLink.addActionListener(openBrowser(aboutLink, Installer.ABOUT_LOMBOK_URL));
buttonBar.add(aboutLink);
buttonBar.add(Box.createRigidArea(new Dimension(16, 1)));
JLabel versionLabel = new JLabel();
versionLabel.setText("v" + Version.getVersion());
buttonBar.add(versionLabel);
buttonBar.add(Box.createRigidArea(new Dimension(16, 1)));
JHyperLink changelogLink = new JHyperLink("View full changelog");
changelogLink.addActionListener(openBrowser(changelogLink,
Installer.ABOUT_LOMBOK_URL.resolve("/changelog.html")));
buttonBar.add(changelogLink);
buttonBar.add(Box.createHorizontalGlue());
JButton quitButton = new JButton("Quit Installer");
quitButton.addActionListener(new ActionListener() {
@Override public void actionPerformed(ActionEvent e) {
appWindow.setVisible(false);
System.exit(0);
}
});
buttonBar.add(quitButton);
return buttonBar;
}
```

```
private ActionListener openBrowser(final JHyperLink hyperlink, final URI location) {
return new ActionListener() {
@Override public void actionPerformed(ActionEvent event) {
hyperlink.setForeground(new Color(85, 145, 90));
try {
//java.awt.Desktop doesn't exist in 1.5.
Object desktop = Class.forName("java.awt.Desktop").getMethod("getDesktop").invoke(null);
Class.forName("java.awt.Desktop").getMethod("browse", URI.class).invoke(desktop, location);
} catch (Exception e) {
Runtime rt = Runtime.getRuntime();
try {
switch (OsUtils.getOS()) {
case WINDOWS:
String[] cmd = new String[4];
cmd[0] = "cmd.exe";
cmd[1] = "/C";
cmd[2] = "start";
cmd[3] = location.toString();
rt.exec(cmd);
break;
case MAC_OS_X:
rt.exec("open " + location.toString());
break;
default:
case UNIX:
rt.exec("firefox " + location.toString());
break;
}
} catch (Exception e2) {
JOptionPane.showMessageDialog(appWindow,
"Well, this is embarrassing. I don't know how to open a webbrowser.\n" +
"I guess you'll have to open it. Browse to:\n" + location +
" for more information about Lombok.",
"I'm embarrassed", JOptionPane.INFORMATION_MESSAGE);
}
}
}
};
}
/**
* Makes the installer window visible.
*/
public void show() {
appWindow.setVisible(true);
if (OsUtils.getOS() == OS.MAC_OS_X) {
try {
AppleNativeLook.go();
} catch (Throwable ignore) {
//We're just prettying up the app. If it fails, meh.
}
}
}
private static final String IDE_TITLE =
```

```
"<html><font size=\"+1\"><b><i>IDEs </i></b></font></html>";
private static final String IDE_EXPLANATION =
"<html>Lombok can update your Eclipse or eclipse-based IDE to fully support all Lombok
features.<br>" +
"Select IDE installations below and hit 'Install/Update'.</html>";
private static final String IDE_LOADING_EXPLANATION =
"Scanning your drives for IDE installations...";
private static final String JAVAC_TITLE =
"<html><font size=\"+1\"><b><i>Javac </i></b></font>       (and tools
that invoke javac such as <i>ant</i> and <i>maven</i>)</html>";
private static final String JAVAC_EXPLANATION =
"<html>Lombok works 'out of the box' with javac.<br>Just make sure the lombok.jar is in your classpath
when you compile.";
private static final String JAVAC_EXAMPLE =
"<html>Example: <code>javac -cp lombok.jar MyCode.java</code></html>";
private static final String UNINSTALL_TITLE =
"<html><font size=\"+1\"><b><i>Uninstall </i></b></font></html>";
private static final String UNINSTALL_EXPLANATION =
"<html>Uninstall Lombok from the following IDE Installations?</html>";
private static final String HOW_I_WORK_TITLE =
"<html><font size=\"+1\"><b><i>What this installer does </i></b></font></html>";
private static final String HOW_I_WORK_EXPLANATION =
"<html><h2>Eclipse</h2><ol>" +
"<li>First, I copy myself (lombok.jar) to your Eclipse install directory.</li>" +
"<li>Then, I edit the <i>eclipse.ini</i> file to add the following entry:<br>" +
"<pre>-javaagent:lombok.jar</pre></li></ol>" +
"On Mac OS X, eclipse.ini is hidden in<br>" +
"<code>Eclipse.app/Contents/MacOS</code> so that's where I place the jar files.</html>";
private static final String SUCCESS_TITLE = "<html><font size=\"+1\"><b><i>Install successful
</i></b></font></html>";
private static final String SUCCESS_EXPLANATION = "<html>Lombok has been installed on the
selected IDE installations.<br>" +
"Don't forget to:<ul><li> add <code>lombok.jar</code> to your projects,<li><b>exit and start</b> your
IDE,<li><b>rebuild</b> all projects!</ul>%%%</html>";
private static class JHyperLink extends JButton {
private static final long serialVersionUID = 1L;
public JHyperLink(String text) {
super();
setFont(getFont().deriveFont(Collections.singletonMap(TextAttribute.UNDERLINE, 1)));
setText(text);
setBorder(null);
setContentAreaFilled(false);
setForeground(Color.BLUE);
setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
setMargin(new Insets(0, 0, 0, 0));
}
}
}
```

lombok/installer/OsUtils.java

```
/*
* Copyright (C) 2009-2016 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicBoolean;
import lombok.core.Version;
/**
* Implement and provide this class to add auto-finding a certain brand of IDEs to the lombok installer.
*/
public final class OsUtils {
private static final AtomicBoolean windowsDriveInfoLibLoaded = new AtomicBoolean(false);
private OsUtils() {
// Prevent instantiation
}
private static void loadWindowsDriveInfoLib() throws IOException {
if (!windowsDriveInfoLibLoaded.compareAndSet(false, true)) return;
final String prefix = "lombok-" + Version.getVersion() + "-";
File temp = File.createTempFile("lombok", ".mark");
File dll1 = new File(temp.getParentFile(), prefix + "WindowsDriveInfo-i386.dll");
File dll2 = new File(temp.getParentFile(), prefix + "WindowsDriveInfo-x86_64.dll");
temp.delete();
dll1.deleteOnExit();
dll2.deleteOnExit();
try {
if (unpackDLL("WindowsDriveInfo-i386.binary", dll1)) {
System.load(dll1.getAbsolutePath());
return;
}
} catch (Throwable ignore) {}
```

```
try {
if (unpackDLL("WindowsDriveInfo-x86_64.binary", dll2)) {
System.load(dll2.getAbsolutePath());
}
} catch (Throwable ignore) {}
}
private static boolean unpackDLL(String dllName, File target) throws IOException {
InputStream in = OsUtils.class.getResourceAsStream(dllName);
try {
try {
FileOutputStream out = new FileOutputStream(target);
try {
byte[] b = new byte[32000];
while (true) {
int r = in.read(b);
if (r == -1) break;
out.write(b, 0, r);
}
} finally {
out.close();
}
} catch (IOException e) {
//Fall through - if there is a file named lombok-WindowsDriveInfo-arch.dll, we'll try it.
return target.exists() && target.canRead();
}
} finally {
in.close();
}
return true;
}
/**
* Returns all drive letters on windows that represent fixed disks.
*
* Floppy drives, optical drives, USB sticks, and network drives should all be excluded.
*
* @return A List of drive letters, such as ["C", "D", "X"].
*/
public static List<String> getDrivesOnWindows() throws Throwable {
loadWindowsDriveInfoLib();
List<String> drives = new ArrayList<String>();
WindowsDriveInfo info = new WindowsDriveInfo();
for (String drive : info.getLogicalDrives()) {
if (info.isFixedDisk(drive)) drives.add(drive);
}
return drives;
}
public static enum OS {
MAC_OS_X("\n"), WINDOWS("\r\n"), UNIX("\n");
private final String lineEnding;
OS(String lineEnding) {
this.lineEnding = lineEnding;
}
public String getLineEnding() {
```

```
return lineEnding;
}
}
public static OS getOS() {
String prop = System.getProperty("os.name", "").toLowerCase();
if (prop.matches("^.*\\bmac\\b.*$")) return OS.MAC_OS_X;
if (prop.matches("^.*\\bdarwin\\b.*$")) return OS.MAC_OS_X;
if (prop.matches("^.*\\bwin(dows|32|64)?\\b.*$")) return OS.WINDOWS;
return OS.UNIX;
}
}
```

lombok/installer/UninstallException.java

```
/*
* Copyright (C) 2009-2010 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
/**
* Thrown when uninstallation of lombok into an IDE fails.
*/
public class UninstallException extends Exception {
private boolean warning;
public UninstallException(String message, Throwable cause) {
super(message, cause);
}
public UninstallException(boolean warning, String message, Throwable cause) {
super(message, cause);
this.warning = warning;
}
public boolean isWarning() {
return warning;
}
```

}

lombok/installer/WindowsDriveInfo.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer;
import java.util.ArrayList;
import java.util.List;
/**
* This class uses native calls on windows to figure out all drives,
* and, for each drive, if its a harddisk or something else.
*
* The output is essentially equivalent to running windows executable:
* <pre>fsutil fsinfo drives</pre>
* and
* <pre>fsutil fsinfo drivetype C:</pre>
*
* except that (A) fsutil requires privileges, (B) someone might have moved
* it out of the path or some such, and (C) its output is internationalized,
* so unless you want to include a table of how to say "Fixed Disk" in 300
* languages, this really is a superior solution.
* <p>
* To compile it, you'll need windows, as well as MinGW:
* http://sourceforge.net/projects/mingw/files/
* <p>
* Fetch gcc 4.0.4+, you don't need anything extra. Toss /c/mingw/bin in
* your git bash prompt's path (/etc/profile) and then run:
*
* $ gcc -c \
-I "/c/Program Files/Java/jdk1.6.0_14/include" \
-I "/c/Program Files/Java/jdk1.6.0_14/include/win32" \
```

```
-D__int64="long long" lombok_installer_WindowsDriveInfo.c
*
* $ dllwrap.exe --add-stdcall-alias \
-o WindowsDriveInfo-i386.dll \
lombok_installer_WindowsDriveInfo.o
*
* You may get a warning along the lines of "Creating an export definition".
* This is expected behaviour.
*
* <p>
* Now download MinGW-w64 to build the 64-bit version of the dll (you thought you were done, weren't
you?)
* from: http://sourceforge.net/projects/mingw-w64/files/
* (under toolchains targetting Win64 / Release for GCC 4.4.0 (or later) / the version for your OS.)
*
* Then, do this all over again, but this time with the x86_64-w64-mingw32-gcc and
* x86_64-w64-mingw32-dllwrap versions that are part of the MinGW-w64 distribution.
* Name the dll 'WindowsDriveInfo-x86_64.dll'.
*
* Both the 32-bit and 64-bit DLLs that this produces have been checked into the git repository
* under src/lombok/installer so you won't need to build them again unless you make some changes to
* the code in the winsrc directory.
*/
public class WindowsDriveInfo {
/**
* Return a list of all available drive letters, such as ["A", "C", "D"].
*/
public List<String> getLogicalDrives() {
int flags = getLogicalDrives0();
List<String> letters = new ArrayList<String>();
for (int i = 0; i < 26; i++) {
if ((flags & (1 << i)) != 0) letters.add(Character.toString((char)('A' + i)));
}
return letters;
}
/**
* Calls kernel32's GetLogicalDrives, which returns an int containing
* flags; bit 0 corresponds to drive A, bit 25 to drive Z. on = disk exists.
*/
private native int getLogicalDrives0();
/**
* Feed it a drive letter (such as 'A') to see if it is a fixed disk.
*/
public boolean isFixedDisk(String letter) {
if (letter.length() != 1) throw new IllegalArgumentException("Supply 1 letter, not: " + letter);
char drive = Character.toUpperCase(letter.charAt(0));
if (drive < 'A' || drive > 'Z') throw new IllegalArgumentException(
"A drive is indicated by a letter, so A-Z inclusive. Not " + drive);
return getDriveType(drive + ":\\") == 3L;
}
/**
* Mirror of kernel32's GetDriveTypeA. You must pass in 'A:\\' -
* so including both a colon and a backslash!
```

```
*
* 0 = error
* 1 = doesn't exist
* 2 = removable drive
* 3 = fixed disk
* 4 = remote (network) disk
* 5 = cd-rom
* 6 = ram disk
*/
private native int getDriveType(String name);
public static void main(String[] args) {
System.loadLibrary("WindowsDriveInfo");
WindowsDriveInfo info = new WindowsDriveInfo();
for (String letter : info.getLogicalDrives()) {
System.out.printf("Drive %s: - %s\n", letter,
info.isFixedDisk(letter) ? "Fixed Disk" : "Not Fixed Disk");
}
}
}
```

lombok/installer/eclipse/AngularIDELocationProvider.java

```
/*
* Copyright (C) 2018-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class AngularIDELocationProvider extends EclipseProductLocationProvider {
```

```
private static final EclipseProductDescriptor ANGULAR = new StandardProductDescriptor(
"Angular IDE",
"angularide",
"angular",
AngularIDELocationProvider.class.getResource("angular.png"),
Collections.<String>emptySet()
);
public AngularIDELocationProvider() {
super(ANGULAR);
}
}
```

lombok/installer/eclipse/EclipseLocationProvider.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class EclipseLocationProvider extends EclipseProductLocationProvider {
private static final EclipseProductDescriptor ECLIPSE = new StandardProductDescriptor(
"Eclipse",
"eclipse",
"eclipse",
EclipseLocationProvider.class.getResource("eclipse.png"),
Collections.<String>emptySet()
);
public EclipseLocationProvider() {
```

```
super(ECLIPSE);
}
}
```

lombok/installer/eclipse/EclipseProductDescriptor.java

```
/*
* Copyright (C) 2016 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.net.URL;
import java.util.List;
import java.util.regex.Pattern;
public interface EclipseProductDescriptor {
String getProductName();
String getWindowsExecutableName();
String getUnixAppName();
String getMacAppName();
String getDirectoryName();
List<String> getExecutableNames();
List<String> getSourceDirsOnWindows();
List<String> getSourceDirsOnMac();
List<String> getSourceDirsOnUnix();
String getIniFileName();
Pattern getLocationSelectors();
URL getIdeIcon();
}
```

lombok/installer/eclipse/EclipseProductLocation.java

/*
* Copyright (C) 2009-2017 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.net.URL;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import lombok.installer.CorruptedIdeLocationException;
import lombok.installer.IdeLocation;
import lombok.installer.InstallException;
import lombok.installer.Installer;
import lombok.installer.OsUtils;
import lombok.installer.UninstallException;
/**
* Represents an Eclipse installation.
* An instance can figure out if an Eclipse installation has been lombok-ified, and can
* install and uninstall lombok from the Eclipse installation.
*/
public final class EclipseProductLocation extends IdeLocation {
private static final String OS_NEWLINE = OsUtils.getOS().getLineEnding();
private final EclipseProductDescriptor descriptor;
private final String[] name;
private final File[] eclipseIniPath;

```
private final String[] pathToLombokJarPrefix;
private final boolean[] hasLombok;
EclipseProductLocation(EclipseProductDescriptor descriptor, String[] nameOfLocation, File[]
pathToEclipseIni) throws CorruptedIdeLocationException {
this.descriptor = descriptor;
this.name = nameOfLocation;
this.eclipseIniPath = pathToEclipseIni;
this.pathToLombokJarPrefix = new String[pathToEclipseIni.length];
this.hasLombok = new boolean[pathToEclipseIni.length];
for (int i = 0; i < pathToEclipseIni.length; i++) {
File p1 = pathToEclipseIni[i].getParentFile();
File p2 = p1 == null ? null : p1.getParentFile();
File p3 = p2 == null ? null : p2.getParentFile();
if (p1 != null && p1.getName().equals("Eclipse") && p2 != null && p2.getName().equals("Contents") &&
p3 != null && p3.getName().endsWith(".app")) {
this.pathToLombokJarPrefix[i] = "../Eclipse/";
} else {
this.pathToLombokJarPrefix[i] = "";
}
try {
this.hasLombok[i] = checkForLombok(eclipseIniPath[i]);
} catch (IOException e) {
throw new CorruptedIdeLocationException(
"I can't read the configuration file of the " + descriptor.getProductName() + " installed at " + name + "\n"
+
"You may need to run this installer with root privileges if you want to modify that " +
descriptor.getProductName() + ".", descriptor.getProductName(), e);
}
}
}
@Override public int hashCode() {
return Arrays.hashCode(eclipseIniPath);
}
@Override public boolean equals(Object o) {
if (!(o instanceof EclipseProductLocation)) return false;
return Arrays.deepEquals(((EclipseProductLocation)o).eclipseIniPath, eclipseIniPath);
}
/**
* Returns the name of this location; generally the path to the eclipse executable.
*/
@Override
public String getName() {
return name[0];
}
/**
* @return true if the Eclipse installation has been instrumented with lombok.
*/
@Override
public boolean hasLombok() {
for (int i = 0; i < hasLombok.length; i++) {
if (!hasLombok[i]) return false;
}
return true;
```

```
}
private static final Pattern JAVA_AGENT_LINE_MATCHER = Pattern.compile(
"^\\-javaagent\\:.*lombok.*\\.jar$", Pattern.CASE_INSENSITIVE);
private static final Pattern BOOTCLASSPATH_LINE_MATCHER = Pattern.compile(
"^\\-Xbootclasspath\\/a\\:(.*lombok.*\\.jar.*)$", Pattern.CASE_INSENSITIVE);
private static boolean checkForLombok(File iniFile) throws IOException {
if (!iniFile.exists()) return false;
FileInputStream fis = new FileInputStream(iniFile);
try {
BufferedReader br = new BufferedReader(new InputStreamReader(fis));
String line;
while ((line = br.readLine()) != null) {
if (JAVA_AGENT_LINE_MATCHER.matcher(line.trim()).matches()) {
br.close();
return true;
}
}
br.close();
return false;
} finally {
fis.close();
}
}
/** Returns directories that may contain lombok.jar files that need to be deleted. */
private List<File> getUninstallDirs() {
Set<String> result = new HashSet<String>();
for (int i = 0; i < name.length; i++) {
File x = new File(name[i]);
if (!x.isDirectory()) x = x.getParentFile();
if (x.isDirectory()) result.add(x.getAbsolutePath());
result.add(eclipseIniPath[i].getAbsolutePath());
}
List<File> out = new ArrayList<File>();
for (String r : result) out.add(new File(r));
return out;
}
/**
* Uninstalls lombok from this location.
* It's a no-op if lombok wasn't there in the first place,
* and it will remove a half-succeeded lombok installation as well.
*
* @throws UninstallException
* If there's an obvious I/O problem that is preventing
* installation. bugs in the uninstall code will probably throw
* other exceptions; this is intentional.
*/
@Override
public void uninstall() throws UninstallException {
final List<File> lombokJarsForWhichCantDeleteSelf = new ArrayList<File>();
StringBuilder newContents = new StringBuilder();
for (int i = 0; i < eclipseIniPath.length; i++) {
if (eclipseIniPath[i].exists()) {
try {
```

```
FileInputStream fis = new FileInputStream(eclipseIniPath[i]);
try {
BufferedReader br = new BufferedReader(new InputStreamReader(fis));
String line;
while ((line = br.readLine()) != null) {
if (JAVA_AGENT_LINE_MATCHER.matcher(line).matches()) continue;
Matcher m = BOOTCLASSPATH_LINE_MATCHER.matcher(line);
if (m.matches()) {
StringBuilder elemBuilder = new StringBuilder();
elemBuilder.append("-Xbootclasspath/a:");
boolean first = true;
for (String elem : m.group(1).split(Pattern.quote(File.pathSeparator))) {
if (elem.toLowerCase().endsWith("lombok.jar")) continue;
/* legacy code -see previous comment that starts with 'legacy' */ {
if (elem.toLowerCase().endsWith("lombok.eclipse.agent.jar")) continue;
}
if (first) first = false;
else elemBuilder.append(File.pathSeparator);
elemBuilder.append(elem);
}
if (!first) newContents.append(elemBuilder.toString()).append(OS_NEWLINE);
continue;
}
newContents.append(line).append(OS_NEWLINE);
}
br.close();
} finally {
fis.close();
}
FileOutputStream fos = new FileOutputStream(eclipseIniPath[i]);
try {
fos.write(newContents.toString().getBytes());
} finally {
fos.close();
}
} catch (IOException e) {
throw new UninstallException("Cannot uninstall lombok from " + name + generateWriteErrorMessage(),
e);
}
}
for (File dir : getUninstallDirs()) {
File lombokJar = new File(dir, "lombok.jar");
if (lombokJar.exists()) {
if (!lombokJar.delete()) {
if (OsUtils.getOS() == OsUtils.OS.WINDOWS && Installer.isSelf(lombokJar.getAbsolutePath())) {
lombokJarsForWhichCantDeleteSelf.add(lombokJar);
} else {
throw new UninstallException(
"Can't delete " + lombokJar.getAbsolutePath() + generateWriteErrorMessage(), null);
}
}
}
/* legacy code - lombok at one point used to have a separate jar for the eclipse agent.
```

```
* Leave this code in to delete it for those upgrading from an old version. */ {
File agentJar = new File(dir, "lombok.eclipse.agent.jar");
if (agentJar.exists()) {
agentJar.delete();
}
}
}
if (!lombokJarsForWhichCantDeleteSelf.isEmpty()) {
throw new UninstallException(true, String.format(
"lombok.jar cannot delete itself on windows.\nHowever, lombok has been uncoupled from your %s.\n" +
"You can safely delete this jar file. You can find it at:\n%s",
descriptor.getProductName(), lombokJarsForWhichCantDeleteSelf.get(0).getAbsolutePath()), null);
}
}
}
private static String generateWriteErrorMessage() {
String osSpecificError;
switch (OsUtils.getOS()) {
default:
case MAC_OS_X:
case UNIX:
osSpecificError = ":\nStart terminal, go to the directory with lombok.jar, and run: sudo java -jar
lombok.jar";
break;
case WINDOWS:
osSpecificError = ":\nStart a new cmd (dos box) with admin privileges, go to the directory with
lombok.jar, and run: java -jar lombok.jar";
break;
}
return ", probably because this installer does not have the access rights.\n" +
"Try re-running the installer with administrative privileges" + osSpecificError;
}
/**
* Install lombok into the Eclipse at this location.
* If lombok is already there, it is overwritten neatly (upgrade mode).
*
* @throws InstallException
* If there's an obvious I/O problem that is preventing
* installation. bugs in the install code will probably throw
* other exceptions; this is intentional.
*/
@Override
public String install() throws InstallException {
// On Linux, for whatever reason, relative paths in your eclipse.ini file don't work, but only for -javaagent.
// On Windows, since the Oomph, the generated shortcut starts in the wrong directory.
// So the default is to use absolute paths, breaking lombok when you move the eclipse directory.
// Or not break when you copy your directory, but break later when you remove the original one.
boolean fullPathRequired = !"false".equals(System.getProperty("lombok.installer.fullpath", "true"));
boolean installSucceeded = false;
StringBuilder newContents = new StringBuilder();
for (int i = 0; i < eclipseIniPath.length; i++) {
installSucceeded = false;
File lombokJar = new File(eclipseIniPath[i].getParentFile(), "lombok.jar");
```

```
/* No need to copy lombok.jar to itself, obviously. On windows this would generate an error so we check for this. */
if (!Installer.isSelf(lombokJar.getAbsolutePath())) {
File ourJar = findOurJar();
byte[] b = new byte[524288];
boolean readSucceeded = true;
try {
FileOutputStream out = new FileOutputStream(lombokJar);
try {
readSucceeded = false;
InputStream in = new FileInputStream(ourJar);
try {
while (true) {
int r = in.read(b);
if (r == -1) break;
if (r > 0) readSucceeded = true;
out.write(b, 0, r);
}
} finally {
in.close();
}
} finally {
out.close();
}
} catch (IOException e) {
try {
lombokJar.delete();
} catch (Throwable ignore) { /* Nothing we can do about that. */ }
if (!readSucceeded) {
throw new InstallException(
"I can't read my own jar file (trying: " + ourJar.toString() + "). I think you've found a bug in this installer!\nI suggest you restart it " +
"and use the 'what do I do' link, to manually install lombok. Also, tell us about this at:\n" +
"http://groups.google.com/group/project-lombok - Thanks!\n\n[DEBUG INFO] " + e.getClass() + ": " + e.getMessage() + "\nBase: " + OsUtils.class.getResource("OsUtils.class"), e);
}
throw new InstallException("I can't write to your " + descriptor.getProductName() + " directory at " + name + generateWriteErrorMessage(), e);
}
}
/* legacy - delete lombok.eclipse.agent.jar if its there, which lombok no longer uses. */ {
new File(lombokJar.getParentFile(), "lombok.eclipse.agent.jar").delete();
}
try {
FileInputStream fis = new FileInputStream(eclipseIniPath[i]);
try {
BufferedReader br = new BufferedReader(new InputStreamReader(fis));
String line;
while ((line = br.readLine()) != null) {
if (JAVA_AGENT_LINE_MATCHER.matcher(line).matches()) continue;
Matcher m = BOOTCLASSPATH_LINE_MATCHER.matcher(line);
if (m.matches()) {
StringBuilder elemBuilder = new StringBuilder();
```

```
elemBuilder.append("-Xbootclasspath/a:");
boolean first = true;
for (String elem : m.group(1).split(Pattern.quote(File.pathSeparator))) {
if (elem.toLowerCase().endsWith("lombok.jar")) continue;
/* legacy code -see previous comment that starts with 'legacy' */ {
if (elem.toLowerCase().endsWith("lombok.eclipse.agent.jar")) continue;
}
if (first) first = false;
else elemBuilder.append(File.pathSeparator);
elemBuilder.append(elem);
}
if (!first) newContents.append(elemBuilder.toString()).append(OS_NEWLINE);
continue;
}
newContents.append(line).append(OS_NEWLINE);
}
br.close();
} finally {
fis.close();
}
String pathPrefix;
if (fullPathRequired) {
pathPrefix = lombokJar.getParentFile().getCanonicalPath() + File.separator;
} else {
pathPrefix = pathToLombokJarPrefix[i];
}
// NB: You may be tempted to escape this, but don't; there is no possibility to escape this, but
// eclipse/java reads the string following the colon in 'raw' fashion. Spaces, colons - all works fine.
newContents.append(String.format(
"-javaagent:%s", pathPrefix + "lombok.jar")).append(OS_NEWLINE);
FileOutputStream fos = new FileOutputStream(eclipseIniPath[i]);
try {
fos.write(newContents.toString().getBytes());
} finally {
fos.close();
}
installSucceeded = true;
} catch (IOException e) {
throw new InstallException("Cannot install lombok at " + name + generateWriteErrorMessage(), e);
} finally {
if (!installSucceeded) try {
lombokJar.delete();
} catch (Throwable ignore) {}
}
if (!installSucceeded) {
throw new InstallException("I can't find the " + descriptor.getIniFileName() + " file. Is this a real " +
descriptor.getProductName() + " installation?", null);
}
}
return "If you start " + descriptor.getProductName() + " with a custom -vm parameter, you'll need to
add:<br>" +
"<code>-vmargs -javaagent:lombok.jar</code><br>as parameter as well.";
}
```

```
@Override public URL getIdeIcon() {
return descriptor.getIdeIcon();
}
}
```

lombok/installer/eclipse/EclipseProductLocationProvider.java

```
/*
* Copyright (C) 2009-2016 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import static java.util.Arrays.asList;
import static lombok.installer.IdeLocation.canonical;
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.regex.Pattern;
import lombok.installer.CorruptedIdeLocationException;
import lombok.installer.OsUtils;
import lombok.installer.IdeLocation;
import lombok.installer.IdeLocationProvider;
public class EclipseProductLocationProvider implements IdeLocationProvider {
private final EclipseProductDescriptor descriptor;
EclipseProductLocationProvider(EclipseProductDescriptor descriptor) {
this.descriptor = descriptor;
}
/**
* Create a new EclipseLocation by pointing at either the directory contains the Eclipse executable, or
the executable itself,
```

```
* or an eclipse.ini file.
*
* @throws CorruptedIdeLocationException
* If this isn't an Eclipse executable or a directory with an
* Eclipse executable.
* @throws NullPointerException if {@code path} is {@code null}.
*/
@Override public final IdeLocation create(String path) throws CorruptedIdeLocationException {
return create0(path);
}
private IdeLocation create0(String path) throws CorruptedIdeLocationException {
if (path == null) throw new NullPointerException("path");
String iniName = descriptor.getIniFileName();
File p = new File(path);
if (!p.exists()) return null;
if (p.isDirectory()) {
for (String possibleExeName : descriptor.getExecutableNames()) {
File f = new File(p, possibleExeName);
if (f.exists()) return findEclipseIniFromExe(f, 0);
}
File f = new File(p, iniName);
if (f.exists()) return makeLocation(new String[] {canonical(p)}, new File[] {f});
}
if (p.isFile()) {
if (p.getName().equalsIgnoreCase(iniName)) {
return makeLocation(new String[] {canonical(p.getParentFile())}, new File[] {p});
}
}
if (descriptor.getExecutableNames().contains(p.getName().toLowerCase())) {
return findEclipseIniFromExe(p, 0);
}
return null;
}
private IdeLocation findEclipseIniFromExe(File exePath, int loopCounter) throws
CorruptedIdeLocationException {
String iniName = descriptor.getIniFileName();
List<Object> foundResults = new ArrayList<Object>(); // even indices are the canonical path (Strings),
the immediately following odd index is the file. (java.io.File).
/* Try looking for eclipse.ini as sibling to the executable */ {
File ini = new File(exePath.getParentFile(), iniName);
if (ini.isFile()) {
foundResults.add(canonical(exePath));
foundResults.add(ini);
}
}
String macAppName = descriptor.getMacAppName();
/* Starting with Eclipse Mars (with the oomph installer), the structure has changed, and it's now at
Eclipse.app/Contents/Eclipse/eclipse.ini*/ {
File ini = new File(exePath.getParentFile(), macAppName + "/Contents/Eclipse/" + iniName);
if (ini.isFile()) {
foundResults.add(canonical(exePath));
foundResults.add(ini);
}
```

```
}
/* Try looking for Eclipse.app/Contents/MacOS/eclipse.ini as sibling to executable; this works on Mac
OS X. */ {
File ini = new File(exePath.getParentFile(), macAppName + "/Contents/MacOS/" + iniName);
if (ini.isFile()) {
foundResults.add(canonical(exePath));
foundResults.add(ini);
}
}
if (foundResults.size() > 0) {
String[] paths = new String[foundResults.size() / 2];
File[] files = new File[paths.length];
for (int i = 0; i < paths.length; i++) {
paths[i] = (String) foundResults.get(i * 2);
files[i] = (File) foundResults.get((i * 2) + 1);
}
return makeLocation(paths, files);
}
/* If executable is a soft link, follow it and retry. */ {
if (loopCounter < 50) {
try {
String oPath = exePath.getAbsolutePath();
String nPath = exePath.getCanonicalPath();
if (!oPath.equals(nPath)) try {
IdeLocation loc = findEclipseIniFromExe(new File(nPath), loopCounter + 1);
if (loc != null) return loc;
} catch (CorruptedIdeLocationException ignore) {
// Unlinking didn't help find an eclipse, so continue.
}
} catch (IOException ignore) { /* okay, that didn't work, assume it isn't a soft link then. */ }
}
}
/* If executable is a linux LSB-style path, then look in the usual places that package managers like
apt-get use.*/ {
String path = exePath.getAbsolutePath();
try {
path = exePath.getCanonicalPath();
} catch (IOException ignore) { /* We'll stick with getAbsolutePath()'s result then. */ }
String unixAppName = descriptor.getUnixAppName();
if (path.equals("/usr/bin/" + unixAppName) || path.equals("/bin/" + unixAppName) ||
path.equals("/usr/local/bin/" + unixAppName)) {
File ini = new File("/usr/lib/" + unixAppName + "/" + iniName);
if (ini.isFile()) return makeLocation(new String[] {path}, new File[] {ini});
ini = new File("/usr/local/lib/" + unixAppName + "/" + iniName);
if (ini.isFile()) return makeLocation(new String[] {path}, new File[] {ini});
ini = new File("/usr/local/etc/" + unixAppName + "/" + iniName);
if (ini.isFile()) return makeLocation(new String[] {path}, new File[] {ini});
ini = new File("/etc/" + iniName);
if (ini.isFile()) return makeLocation(new String[] {path}, new File[] {ini});
}
}
/* If we get this far, we lose. */
return null;
```

```
}
private IdeLocation makeLocation(String[] name, File[] ini) throws CorruptedIdeLocationException {
return new EclipseProductLocation(descriptor, name, ini);
}
@Override public Pattern getLocationSelectors() {
return descriptor.getLocationSelectors();
}
/**
* Calls the OS-dependent 'find Eclipse' routine. If the local OS doesn't have a routine written for it,
* null is returned.
*
* @param locations
* List of valid eclipse locations - provide an empty list; this
* method will fill it.
* @param problems
* List of eclipse locations that seem to contain half-baked
* eclipses that can't be installed. Provide an empty list; this
* method will fill it.
*/
@Override
public void findIdes(List<IdeLocation> locations, List<CorruptedIdeLocationException> problems) {
switch (OsUtils.getOS()) {
case WINDOWS:
new WindowsFinder().findEclipse(locations, problems);
break;
case MAC_OS_X:
new MacFinder().findEclipse(locations, problems);
break;
default:
case UNIX:
new UnixFinder().findEclipse(locations, problems);
break;
}
}
private List<File> transformToFiles(List<String> fileNames) {
List<File> files = new ArrayList<File>();
for (String fileName : fileNames) {
files.add(new File(fileName));
}
return files;
}
private List<File> getFlatSourceLocationsOnUnix() {
List<File> dirs = new ArrayList<File>();
dirs.add(new File("/usr/bin/"));
dirs.add(new File("/usr/local/bin/"));
dirs.add(new File(System.getProperty("user.home", "."), "bin/"));
return dirs;
}
private List<File> getNestedSourceLocationOnUnix() {
List<File> dirs = new ArrayList<File>();
dirs.add(new File("/usr/local/share"));
dirs.add(new File("/usr/local"));
dirs.add(new File("/usr/share"));
```

```
return dirs;
}
private class UnixFinder extends DirectoryFinder {
UnixFinder() {
super(getNestedSourceLocationOnUnix(), getFlatSourceLocationsOnUnix());
}
@Override protected String findEclipseOnPlatform(File dir) {
File possible = new File(dir, descriptor.getUnixAppName());
return (possible.exists()) ? possible.getAbsolutePath() : null;
}
}
/**
* Returns a list of paths of Eclipse installations.
*
* The search process works by scanning for each 'source dir' for either an eclipse installation or a folder containing the text returned
* by getDirName(). If such a folder is found, this process is applied recursively. On windows, this process is run on each drive letter
* which represents a physical hard disk. If the native windows API call to determine these drive letters fails, only 'C:' is checked.
*/
private List<String> getSourceDirsOnWindowsWithDriveLetters() {
List<String> driveLetters = asList("C");
try {
driveLetters = OsUtils.getDrivesOnWindows();
} catch (Throwable ignore) {
ignore.printStackTrace();
}
List<String> sourceDirs = new ArrayList<String>();
for (String letter : driveLetters) {
for (String possibleSource : descriptor.getSourceDirsOnWindows()) {
if (!isDriveSpecificOnWindows(possibleSource)) {
sourceDirs.add(letter + ":" + possibleSource);
}
}
}
for (String possibleSource : descriptor.getSourceDirsOnWindows()) {
if (isDriveSpecificOnWindows(possibleSource)) sourceDirs.add(possibleSource);
}
return sourceDirs;
}
private boolean isDriveSpecificOnWindows(String path) {
return path.length() > 1 && path.charAt(1) == ':';
}
private class WindowsFinder extends DirectoryFinder {
WindowsFinder() {
super(transformToFiles(getSourceDirsOnWindowsWithDriveLetters()), Collections.<File>emptyList());
}
/** Checks if the provided directory contains 'eclipse.exe', and if so, returns the directory, otherwise null.
*/
@Override
protected String findEclipseOnPlatform(File dir) {
File possible = new File(dir, descriptor.getWindowsExecutableName());
```

```
return (possible.isFile()) ? dir.getAbsolutePath() : null;
}
}
private class MacFinder extends DirectoryFinder {
MacFinder() {
super(transformToFiles(descriptor.getSourceDirsOnMac()), Collections.<File>emptyList());
}
protected String findEclipseOnPlatform(File dir) {
if (dir.getName().toLowerCase().equals(descriptor.getMacAppName().toLowerCase())) return
dir.getParent();
if (dir.getName().toLowerCase().contains(descriptor.getDirectoryName())) {
if (new File(dir, descriptor.getMacAppName()).exists()) return dir.toString();
}
return null;
}
}
private abstract class DirectoryFinder {
private final List<File> flatSourceDirs;
private final List<File> nestedSourceDirs;
DirectoryFinder(List<File> nestedSourceDirs, List<File> flatSourceDirs) {
this.nestedSourceDirs = nestedSourceDirs;
this.flatSourceDirs = flatSourceDirs;
}
void findEclipse(List<IdeLocation> locations, List<CorruptedIdeLocationException> problems) {
for (File dir : nestedSourceDirs) recurseDirectory(locations, problems, dir);
for (File dir : flatSourceDirs) findEclipse(locations, problems, dir);
}
abstract String findEclipseOnPlatform(File dir);
void recurseDirectory(List<IdeLocation> locations, List<CorruptedIdeLocationException> problems,
File dir) {
recurseDirectory0(locations, problems, dir, 0, false);
}
private void recurseDirectory0(List<IdeLocation> locations, List<CorruptedIdeLocationException>
problems, File f, int loopCounter, boolean nameFound) {
//Various try/catch/ignore statements are in this for loop. Weird conditions on the disk can cause
exceptions,
//such as an unformatted drive causing a NullPointerException on listFiles. Best action is almost
invariably to just
//continue onwards.
File[] listFiles = f.listFiles();
if (listFiles == null) return;
for (File dir : listFiles) {
if (!dir.isDirectory()) continue;
try {
if (nameFound || dir.getName().toLowerCase().contains(descriptor.getDirectoryName())) {
findEclipse(locations, problems, dir);
if (loopCounter < 50) recurseDirectory0(locations, problems, dir, loopCounter + 1, true);
}
} catch (Exception ignore) {}
}
}
private void findEclipse(List<IdeLocation> locations, List<CorruptedIdeLocationException> problems,
File dir) {
```

```
String eclipseLocation = findEclipseOnPlatform(dir);
if (eclipseLocation != null) {
try {
IdeLocation newLocation = create(eclipseLocation);
if (newLocation != null) locations.add(newLocation);
} catch (CorruptedIdeLocationException e) {
problems.add(e);
}
}
}
}
}
```

lombok/installer/eclipse/JbdsLocationProvider.java

```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class JbdsLocationProvider extends EclipseProductLocationProvider {
private static final EclipseProductDescriptor JBDS = new StandardProductDescriptor(
"JBoss Developer Studio",
"jbdevstudio",
"studio",
JbdsLocationProvider.class.getResource("jbds.png"),
Collections.<String>emptySet()
);
public JbdsLocationProvider() {
super(JBDS);
```

```
}
}
```

lombok/installer/eclipse/MyEclipseLocationProvider.java

```
/*
* Copyright (C) 2016-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class MyEclipseLocationProvider extends EclipseProductLocationProvider {
private static final EclipseProductDescriptor MY_ECLIPSE = new StandardProductDescriptor(
"MyEclipse",
"myeclipse",
"myeclipse",
MyEclipseLocationProvider.class.getResource("myeclipse.png"),
Collections.<String>emptySet()
);
public MyEclipseLocationProvider() {
super(MY_ECLIPSE);
}
}
```

lombok/installer/eclipse/RhcrLocationProvider.java

```
/*
* Copyright (C) 2020-2021 The Project Lombok Authors.
```

```
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class RhcrLocationProvider extends EclipseProductLocationProvider {
private static final EclipseProductDescriptor RHCR = new StandardProductDescriptor(
"Red Hat CodeReady Studio",
"codereadystudio",
"studio",
RhcrLocationProvider.class.getResource("rhds.png"),
Collections.<String>emptySet()
);
public RhcrLocationProvider() {
super(RHCR);
}
}
```

lombok/installer/eclipse/RhdsLocationProvider.java

```
/*
* Copyright (C) 2013-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
```

```
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class RhdsLocationProvider extends EclipseProductLocationProvider {
private static final EclipseProductDescriptor RHDS = new StandardProductDescriptor(
"Red Hat JBoss Developer Studio",
"devstudio",
"studio",
RhdsLocationProvider.class.getResource("rhds.png"),
Collections.<String>emptySet()
);
public RhdsLocationProvider() {
super(RHDS);
}
}
```

lombok/installer/eclipse/STS4LocationProvider.java

```
/*
* Copyright (C) 2018-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
```

```
import java.util.Arrays;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class STS4LocationProvider extends EclipseProductLocationProvider {
private static final EclipseProductDescriptor STS4 = new StandardProductDescriptor("Spring Tools
Suite 4",
"SpringToolSuite4",
"sts",
STS4LocationProvider.class.getResource("STS.png"),
Collections.unmodifiableList(Arrays.asList("springsource", "spring-tool-suite"))
);
public STS4LocationProvider() {
super(STS4);
}
}
```

lombok/installer/eclipse/STSLocationProvider.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.util.Collections;
import lombok.installer.IdeLocationProvider;
import lombok.spi.Provides;
@Provides(IdeLocationProvider.class)
public class STSLocationProvider extends EclipseProductLocationProvider {
private static final EclipseProductDescriptor STS = new StandardProductDescriptor("STS",
"STS",
"sts",
STSLocationProvider.class.getResource("STS.png"),
```

```
Collections.singleton("springsource")
);
public STSLocationProvider() {
super(STS);
}
}
```

lombok/installer/eclipse/StandardProductDescriptor.java

```
/*
* Copyright (C) 2016 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.installer.eclipse;
import java.io.File;
import java.net.URL;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import java.util.regex.Pattern;
import lombok.installer.OsUtils;
public class StandardProductDescriptor implements EclipseProductDescriptor {
private static final String USER_HOME = System.getProperty("user.home", ".");
private static final String[] BASE_WINDOWS_ROOTS = {"\\", "\\Program Files", "\\Program Files (x86)",
"\\ProgramData\\Chocolatey\\lib"};
private static final String[] WINDOWS_ROOTS = windowsRoots();
private static final String[] MAC_ROOTS = {"/Applications", USER_HOME};
private static final String[] UNIX_ROOTS = {USER_HOME};
private final String productName;
private final String windowsName;
```

```
private final String unixName;
private final String macAppName;
private final List<String> executableNames;
private final List<String> sourceDirsOnWindows;
private final List<String> sourceDirsOnMac;
private final List<String> sourceDirsOnUnix;
private final String iniFileName;
private final Pattern locationSelectors;
private final String directoryName;
private final URL ideIcon;
public StandardProductDescriptor(String productName, String baseName, String directoryName, URL
ideIcon, Collection<String> alternativeDirectoryNames) {
this.productName = productName;
this.windowsName = baseName + ".exe";
this.unixName = baseName;
this.macAppName = baseName + ".app";
this.executableNames = executableNames(baseName);
this.sourceDirsOnWindows = generateAlternatives(WINDOWS_ROOTS, "\\",
alternativeDirectoryNames);
this.sourceDirsOnMac = generateAlternatives(MAC_ROOTS, "/", alternativeDirectoryNames);
this.sourceDirsOnUnix = generateAlternatives(UNIX_ROOTS, "/", alternativeDirectoryNames);
this.iniFileName = baseName + ".ini";
this.locationSelectors = getLocationSelectors(baseName);
this.directoryName = directoryName.toLowerCase();
this.ideIcon = ideIcon;
}
@Override public String getProductName() {
return productName;
}
@Override public String getWindowsExecutableName() {
return windowsName;
}
@Override public String getUnixAppName() {
return unixName;
}
@Override public String getMacAppName() {
return macAppName;
}
@Override public String getDirectoryName() {
return directoryName;
}
@Override public List<String> getExecutableNames() {
return executableNames;
}
@Override public List<String> getSourceDirsOnWindows() {
return sourceDirsOnWindows;
}
@Override public List<String> getSourceDirsOnMac() {
return sourceDirsOnMac;
}
@Override public List<String> getSourceDirsOnUnix() {
return sourceDirsOnUnix;
}
```

```
@Override public String getIniFileName() {
return iniFileName;
}
@Override public Pattern getLocationSelectors() {
return locationSelectors;
}
@Override public URL getIdeIcon() {
return ideIcon;
}
private static Pattern getLocationSelectors(String baseName) {
return Pattern.compile(String.format(platformPattern(), baseName.toLowerCase()),
Pattern.CASE_INSENSITIVE);
}
private static String platformPattern() {
switch (OsUtils.getOS()) {
case MAC_OS_X:
return "^(%s|%<s\\.ini|%<s\\.app)$";
case WINDOWS:
return "^(%sc?\\.exe|%<s\\.ini)$";
default:
case UNIX:
return "^(%s|%<s\\.ini)$";
}
}
private static List<String> executableNames(String baseName) {
String base = baseName.toLowerCase();
return Collections.unmodifiableList(Arrays.asList(base, base + ".app", base + ".exe", base + "c.exe"));
}
private static List<String> generateAlternatives(String[] roots, String pathSeparator, Collection<String>
alternatives) {
List<String> result = new ArrayList<String>();
for (String root : roots) {
result.add(concat(root, pathSeparator, ""));
for (String alternative : alternatives) {
result.add(concat(root, pathSeparator, alternative));
}
}
return Collections.unmodifiableList(result);
}
private static String concat(String base, String pathSeparator, String alternative) {
if (alternative.isEmpty()) {
return base;
}
if (base.endsWith(pathSeparator)) {
return base + alternative.replaceAll("[\\/]", "\\" + pathSeparator);
}
return base + pathSeparator + alternative.replaceAll("[\\/]", "\\" + pathSeparator);
}
private static String[] windowsRoots() {
String localAppData = windowsLocalAppData();
String[] out = new String[BASE_WINDOWS_ROOTS.length + (localAppData == null ? 1 : 2)];
System.arraycopy(BASE_WINDOWS_ROOTS, 0, out, 0, BASE_WINDOWS_ROOTS.length);
out[BASE_WINDOWS_ROOTS.length] = USER_HOME;
```

```
if (localAppData != null) out[BASE_WINDOWS_ROOTS.length + 1] = localAppData;
return out;
}
private static String windowsLocalAppData() {
String localAppData = System.getenv("LOCALAPPDATA");
File file = localAppData == null ? null : new File(localAppData);
return file != null && file.exists() && file.canRead() && file.isDirectory() ? localAppData : null;
}
}
```

lombok/installer/package-info.java

```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
/**
* This package contains the lombok installer. It explains to any user that double-clicks the lombok.jar
what
* lombok is about, and has the ability to instrument (or remove existing Lombok instrumentation) from
any
* Eclipse installation. This package also contains the graphics uses in the installer in SVG format.
*/
package lombok.installer;
```

lombok/delombok/Delombok.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
```

* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.io.OutputStreamWriter;
import java.io.PrintStream;
import java.io.UnsupportedEncodingException;
import java.io.Writer;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.net.URI;
import java.net.URLDecoder;
import java.nio.charset.Charset;
import java.nio.charset.UnsupportedCharsetException;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.IdentityHashMap;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.ListIterator;
import java.util.Map;
import java.util.Set;
import java.util.regex.Pattern;
import javax.annotation.processing.AbstractProcessor;
import javax.tools.DiagnosticListener;
import javax.tools.JavaFileManager;
import javax.tools.JavaFileObject;
import lombok.Lombok;
import lombok.javac.CommentCatcher;
import lombok.javac.Javac;

```
import lombok.javac.JavacAugments;
import lombok.javac.LombokOptions;
import lombok.javac.apt.LombokProcessor;
import lombok.permit.Permit;
import com.sun.tools.javac.code.Symtab;
import com.sun.tools.javac.comp.Todo;
import com.sun.tools.javac.file.BaseFileManager;
import com.sun.tools.javac.main.Arguments;
import com.sun.tools.javac.main.JavaCompiler;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCImport;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.ListBuffer;
import com.zwitserloot.cmdreader.CmdReader;
import com.zwitserloot.cmdreader.Description;
import com.zwitserloot.cmdreader.Excludes;
import com.zwitserloot.cmdreader.FullName;
import com.zwitserloot.cmdreader.InvalidCommandLineException;
import com.zwitserloot.cmdreader.Mandatory;
import com.zwitserloot.cmdreader.Sequential;
import com.zwitserloot.cmdreader.Shorthand;
public class Delombok {
private Charset charset = Charset.defaultCharset();
private Context context = new Context();
private Writer presetWriter;
public void setWriter(Writer writer) {
this.presetWriter = writer;
}
private PrintStream feedback = System.err;
private boolean verbose;
private boolean noCopy;
private boolean onlyChanged;
private boolean force = false;
private boolean disablePreview;
private String classpath, sourcepath, bootclasspath, modulepath;
private LinkedHashMap<File, File> fileToBase = new LinkedHashMap<File, File>();
private List<File> filesToParse = new ArrayList<File>();
private Map<String, String> formatPrefs = new HashMap<String, String>();
private List<AbstractProcessor> additionalAnnotationProcessors = new
ArrayList<AbstractProcessor>();
/** If null, output to standard out. */
private File output = null;
private static class CmdArgs {
@Shorthand("v")
@Description("Print the name of each file as it is being delombok-ed.")
@Excludes("quiet")
private boolean verbose;
@Shorthand("f")
@Description("Sets formatting rules. Use --format-help to list all available rules. Unset format rules are
inferred by scanning the source for usages.")
private List<String> format = new ArrayList<String>();
@FullName("format-help")
```

```
private boolean formatHelp;
@Shorthand("q")
@Description("No warnings or errors will be emitted to standard error")
@Excludes("verbose")
private boolean quiet;
@Shorthand("e")
@Description("Sets the encoding of your source files. Defaults to the system default charset. Example:
\"UTF-8\"")
private String encoding;
@Shorthand("p")
@Description("Print delombok-ed code to standard output instead of saving it in target directory")
private boolean print;
@Shorthand("d")
@Description("Directory to save delomboked files to")
@Mandatory(onlyIfNot={"print", "help", "format-help"})
private String target;
@Shorthand("c")
@Description("Classpath (analogous to javac -cp option)")
private String classpath;
@Shorthand("s")
@Description("Sourcepath (analogous to javac -sourcepath option)")
private String sourcepath;
@Description("override Bootclasspath (analogous to javac -bootclasspath option)")
private String bootclasspath;
@Description("Module path (analogous to javac --module-path option)")
@FullName("module-path")
private String modulepath;
@Description("Files to delombok. Provide either a file, or a directory. If you use a directory, all files in it
(recursive) are delombok-ed")
@Sequential
private List<String> input = new ArrayList<String>();
@Description("Lombok will only delombok source files. Without this option, non-java, non-class files
are copied to the target directory.")
@Shorthand("n")
private boolean nocopy;
@Description("Output only changed files (implies -n)")
private boolean onlyChanged;
@Description("By default lombok enables preview features if available (introduced in JDK 12). With this
option, lombok won't do that.")
@FullName("disable-preview")
private boolean disablePreview;
private boolean help;
}
static {
LombokProcessor.addOpensForLombok();
}
private static String indentAndWordbreak(String in, int indent, int maxLen) {
StringBuilder out = new StringBuilder();
StringBuilder line = new StringBuilder();
StringBuilder word = new StringBuilder();
int len = in.length();
for (int i = 0; i < (len + 1); i++) {
char c = i == len ? ' ' : in.charAt(i);
```

```
if (c == ' ') {
if (line.length() + word.length() < maxLen) {
line.append(word);
} else {
if (out.length() > 0) out.append("\n");
for (int j = 0; j < indent; j++) out.append(" ");
out.append(line);
line.setLength(0);
line.append(word.toString().trim());
}
word.setLength(0);
}
if (i < len) word.append(c);
}
if (line.length() > 0) {
if (out.length() > 0) out.append("\n");
for (int j = 0; j < indent; j++) out.append(" ");
out.append(line);
}
return out.toString();
}
static String getPathOfSelf() {
String url = Delombok.class.getResource("Delombok.class").toString();
if (url.endsWith("lombok/delombok/Delombok.class")) {
url = urlDecode(url.substring(0, url.length() - "lombok/delombok/Delombok.class".length()));
} else if (url.endsWith("lombok/delombok/Delombok.SCL.lombok")) {
url = urlDecode(url.substring(0, url.length() - "lombok/delombok/Delombok.SCL.lombok".length()));
} else {
return null;
}
if (url.startsWith("jar:file:") && url.endsWith("!/")) return url.substring(9, url.length() - 2);
if (url.startsWith("file:")) return url.substring(5);
return null;
}
private static String urlDecode(String in) {
try {
return URLDecoder.decode(in, "UTF-8");
} catch (UnsupportedEncodingException e) {
throw new InternalError("UTF-8 not supported");
}
}
public static void main(String[] rawArgs) {
try {
rawArgs = fileExpand(rawArgs);
} catch (IOException e) {
System.out.println(e.getMessage());
System.exit(1);
}
CmdReader<CmdArgs> reader = CmdReader.of(CmdArgs.class);
CmdArgs args;
try {
args = reader.make(rawArgs);
} catch (InvalidCommandLineException e) {
```

```
System.err.println("ERROR: " + e.getMessage());
System.err.println(cmdHelp(reader));
System.exit(1);
return;
}
if (args.help || (args.input.isEmpty() && !args.formatHelp)) {
if (!args.help) System.err.println("ERROR: no files or directories to delombok specified.");
System.err.println(cmdHelp(reader));
System.exit(args.help ? 0 : 1);
return;
}
Delombok delombok = new Delombok();
if (args.quiet) delombok.setFeedback(new PrintStream(new OutputStream() {
@Override public void write(int b) throws IOException {
//dummy - do nothing.
}
}));
if (args.formatHelp) {
System.out.println("Available format keys (to use, -f key:value -f key2:value2 -f ... ):");
for (Map.Entry<String, String> e : FormatPreferences.getKeysAndDescriptions().entrySet()) {
System.out.print(" ");
System.out.print(e.getKey());
System.out.println(":");
System.out.println(indentAndWordbreak(e.getValue(), 4, 70));
}
System.out.println("Example: -f indent:4 -f emptyLines:indent");
System.out.println("The '-f pretty' option is shorthand for '-f suppressWarnings:skip -f generated:skip -f
danceAroundIdeChecks:skip -f generateDelombokComment:skip -f javaLangAsFQN:skip'");
System.exit(0);
return;
}
try {
delombok.setFormatPreferences(formatOptionsToMap(args.format));
} catch (InvalidFormatOptionException e) {
System.out.println(e.getMessage() + " Try --format-help.");
System.exit(1);
return;
}
if (args.encoding != null) {
try {
delombok.setCharset(args.encoding);
} catch (UnsupportedCharsetException e) {
System.err.println("ERROR: Not a known charset: " + args.encoding);
System.exit(1);
return;
}
}
if (args.verbose) delombok.setVerbose(true);
if (args.nocopy || args.onlyChanged) delombok.setNoCopy(true);
if (args.disablePreview) delombok.setDisablePreview(true);
if (args.onlyChanged) delombok.setOnlyChanged(true);
if (args.print) {
delombok.setOutputToStandardOut();
```

```
} else {
delombok.setOutput(new File(args.target));
}
if (args.classpath != null) delombok.setClasspath(args.classpath);
if (args.sourcepath != null) delombok.setSourcepath(args.sourcepath);
if (args.bootclasspath != null) delombok.setBootclasspath(args.bootclasspath);
if (args.modulepath != null) delombok.setModulepath(args.modulepath);
try {
for (String in : args.input) {
File f = new File(in).getAbsoluteFile();
if (f.isFile()) {
delombok.addFile(f.getParentFile(), f.getName());
} else if (f.isDirectory()) {
delombok.addDirectory(f);
} else if (!f.exists()) {
if (!args.quiet) System.err.println("WARNING: does not exist - skipping: " + f);
} else {
if (!args.quiet) System.err.println("WARNING: not a standard file or directory - skipping: " + f);
}
}
delombok.delombok();
} catch (Exception e) {
if (!args.quiet) {
String msg = e.getMessage();
if (msg != null && msg.startsWith("DELOMBOK: ")) System.err.println(msg.substring("DELOMBOK:
".length()));
else e.printStackTrace();
System.exit(1);
return;
}
}
}
private static String cmdHelp(CmdReader<CmdArgs> reader) {
String x = reader.generateCommandLineHelp("delombok");
int idx = x.indexOf('\n');
return x.substring(0, idx) + "\n You can use @filename.args to read arguments from the file
'filename.args'.\n" + x.substring(idx);
}
private static String[] fileExpand(String[] rawArgs) throws IOException {
String[] out = rawArgs;
int offset = 0;
for (int i = 0; i < rawArgs.length; i++) {
if (rawArgs[i].length() > 0 && rawArgs[i].charAt(0) == '@') {
String[] parts = readArgsFromFile(rawArgs[i].substring(1));
String[] newOut = new String[out.length + parts.length - 1];
System.arraycopy(out, 0, newOut, 0, i + offset);
System.arraycopy(parts, 0, newOut, i + offset, parts.length);
System.arraycopy(out, i + offset + 1, newOut, i + offset + parts.length, out.length - (i + offset + 1));
offset += parts.length - 1;
out = newOut;
}
}
return out;
```

```
}
private static String[] readArgsFromFile(String file) throws IOException {
InputStream in = new FileInputStream(file);
StringBuilder s = new StringBuilder();
try {
InputStreamReader isr = new InputStreamReader(in, "UTF-8");
try {
char[] c = new char[4096];
while (true) {
int r = isr.read(c);
if (r == -1) break;
s.append(c, 0, r);
}
} finally {
isr.close();
}
} finally {
in.close();
}
List<String> x = new ArrayList<String>();
StringBuilder a = new StringBuilder();
int state = 1;
for (int i = 0; i < s.length(); i++) {
char c = s.charAt(i);
if (state < 0) {
state = -state;
if (c != '\n') a.append(c);
continue;
}
if (state == 1) {
if (c == '\\') {
state = -1;
continue;
}
if (c == '"') {
state = 2;
continue;
}
if (c == '\'') {
state = 3;
continue;
}
if (Character.isWhitespace(c)) {
String aa = a.toString();
if (!aa.isEmpty()) x.add(aa);
a.setLength(0);
continue;
}
a.append(c);
continue;
}
if (state == 2) {
if (c == '\\') {
```

```
state = -2;
continue;
}
if (c == '"') {
state = 1;
x.add(a.toString());
a.setLength(0);
continue;
}
a.append(c);
continue;
}
if (state == 3) {
if (c == '\'') {
state = 1;
x.add(a.toString());
a.setLength(0);
continue;
}
a.append(c);
continue;
}
}
if (state == 1) {
String aa = a.toString();
if (!aa.isEmpty()) x.add(aa);
} else if (state < 0) {
throw new IOException("Unclosed backslash escape in @ file");
} else if (state == 2) {
throw new IOException("Unclosed \" in @ file");
} else if (state == 3) {
throw new IOException("Unclosed ' in @ file");
}
return x.toArray(new String[0]);
}
public static class InvalidFormatOptionException extends Exception {
public InvalidFormatOptionException(String msg) {
super(msg);
}
}
public static Map<String, String> formatOptionsToMap(List<String> formatOptions) throws
InvalidFormatOptionException {
boolean prettyEnabled = false;
Map<String, String> formatPrefs = new HashMap<String, String>();
for (String format : formatOptions) {
int idx = format.indexOf(':');
if (idx == -1) {
if (format.equalsIgnoreCase("pretty")) {
prettyEnabled = true;
continue;
} else {
throw new InvalidFormatOptionException("Format keys need to be 2 values separated with a colon.");
}
```

```
}
String key = format.substring(0, idx);
String value = format.substring(idx + 1);
boolean valid = false;
for (String k : FormatPreferences.getKeysAndDescriptions().keySet()) {
if (k.equalsIgnoreCase(key)) {
valid = true;
break;
}
}
if (!valid) throw new InvalidFormatOptionException("Unknown format key: '" + key + "'.");
formatPrefs.put(key.toLowerCase(), value);
}
if (prettyEnabled) {
if (!formatPrefs.containsKey("suppresswarnings")) formatPrefs.put("suppresswarnings", "skip");
if (!formatPrefs.containsKey("generated")) formatPrefs.put("generated", "skip");
if (!formatPrefs.containsKey("dancearoundidechecks")) formatPrefs.put("dancearoundidechecks",
"skip");
if (!formatPrefs.containsKey("generatedelombokcomment"))
formatPrefs.put("generatedelombokcomment", "skip");
if (!formatPrefs.containsKey("javalangasfqn")) formatPrefs.put("javalangasfqn", "skip");
}
return formatPrefs;
}
public void setFormatPreferences(Map<String, String> prefs) {
this.formatPrefs = prefs;
}
public void setCharset(String charsetName) throws UnsupportedCharsetException {
if (charsetName == null) {
charset = Charset.defaultCharset();
return;
}
charset = Charset.forName(charsetName);
}
public void setDiagnosticsListener(DiagnosticListener<JavaFileObject> diagnostics) {
if (diagnostics != null) context.put(DiagnosticListener.class, diagnostics);
}
public void setForceProcess(boolean force) {
this.force = force;
}
public void setFeedback(PrintStream feedback) {
this.feedback = feedback;
}
public void setClasspath(String classpath) {
this.classpath = classpath;
}
public void setSourcepath(String sourcepath) {
this.sourcepath = sourcepath;
}
public void setBootclasspath(String bootclasspath) {
this.bootclasspath = bootclasspath;
}
public void setVerbose(boolean verbose) {
```

```
this.verbose = verbose;
}
public void setNoCopy(boolean noCopy) {
this.noCopy = noCopy;
}
public void setDisablePreview(boolean disablePreview) {
this.disablePreview = disablePreview;
}
public void setOnlyChanged(boolean onlyChanged) {
this.onlyChanged = onlyChanged;
}
public void setOutput(File dir) {
if (dir.isFile() || (!dir.isDirectory() && dir.getName().endsWith(".java"))) throw new
IllegalArgumentException(
"DELOMBOK: delombok will only write to a directory. " +
"If you want to delombok a single file, use -p to output to standard output, then redirect this to a file:\n" +
"delombok MyJavaFile.java -p >MyJavaFileDelombok.java");
output = dir;
}
public void setOutputToStandardOut() {
this.output = null;
}
public void setModulepath(String modulepath) {
this.modulepath = modulepath;
}
public void addDirectory(File base) throws IOException {
addDirectory0(false, base, "", 0);
}
public void addDirectory1(boolean copy, File base, String name) throws IOException {
File f = new File(base, name);
if (f.isFile()) {
String extension = getExtension(f);
if (extension.equals("java")) addFile(base, name);
else if (extension.equals("class")) skipClass(name);
else copy(copy, base, name);
} else if (!f.exists()) {
feedback.printf("Skipping %s because it does not exist.\n", canonical(f));
} else if (!f.isDirectory()) {
feedback.printf("Skipping %s because it is a special file type.\n", canonical(f));
}
}
private void addDirectory0(boolean inHiddenDir, File base, String suffix, int loop) throws IOException {
File dir = suffix.isEmpty() ? base : new File(base, suffix);
if (dir.isDirectory()) {
boolean thisDirIsHidden = !inHiddenDir && new File(canonical(dir)).getName().startsWith(".");
if (loop >= 100) {
feedback.printf("Over 100 subdirectories? I'm guessing there's a loop in your directory structure.
Skipping: %s\n", suffix);
} else {
File[] list = dir.listFiles();
if (list.length > 0) {
if (thisDirIsHidden && !noCopy && output != null) {
```

```
feedback.printf("Only processing java files (not copying non-java files) in %s because it's a hidden
directory.\n", canonical(dir));
}
for (File f : list) {
addDirectory0(inHiddenDir || thisDirIsHidden, base, suffix + (suffix.isEmpty() ? "" : File.separator) +
f.getName(), loop + 1);
}
} else {
if (!thisDirIsHidden && !noCopy && !inHiddenDir && output != null && !suffix.isEmpty()) {
File emptyDir = new File(output, suffix);
emptyDir.mkdirs();
if (verbose) feedback.printf("Creating empty directory: %s\n", canonical(emptyDir));
}
}
}
} else {
addDirectory1(!inHiddenDir && !noCopy, base, suffix);
}
}
private void skipClass(String fileName) {
if (verbose) feedback.printf("Skipping class file: %s\n", fileName);
}
private void copy(boolean copy, File base, String fileName) throws IOException {
if (output == null) {
feedback.printf("Skipping resource file: %s\n", fileName);
return;
}
if (!copy) {
if (verbose) feedback.printf("Skipping resource file: %s\n", fileName);
return;
}
if (verbose) feedback.printf("Copying resource file: %s\n", fileName);
byte[] b = new byte[65536];
File inFile = new File(base, fileName);
FileInputStream in = new FileInputStream(inFile);
try {
File outFile = new File(output, fileName);
outFile.getParentFile().mkdirs();
FileOutputStream out = new FileOutputStream(outFile);
try {
while (true) {
int r = in.read(b);
if (r == -1) break;
out.write(b, 0, r);
}
} finally {
out.close();
}
} finally {
in.close();
}
}
public void addFile(File base, String fileName) throws IOException {
```

```
if (output != null && canonical(base).equals(canonical(output))) throw new IOException(
"DELOMBOK: Output file and input file refer to the same filesystem location. Specify a separate path
for output.");
File f = new File(base, fileName);
filesToParse.add(f);
fileToBase.put(f, base);
}
public void addAdditionalAnnotationProcessor(AbstractProcessor processor) {
additionalAnnotationProcessors.add(processor);
}
private static <T> com.sun.tools.javac.util.List<T> toJavacList(List<T> list) {
com.sun.tools.javac.util.List<T> out = com.sun.tools.javac.util.List.nil();
ListIterator<T> li = list.listIterator(list.size());
while (li.hasPrevious()) out = out.prepend(li.previous());
return out;
}
private static final Field MODULE_FIELD = getModuleField();
private static Field getModuleField() {
try {
return Permit.getField(JCCompilationUnit.class, "modle");
} catch (NoSuchFieldException e) {
return null;
} catch (SecurityException e) {
return null;
}
}
public boolean delombok() throws IOException {
LombokOptions options = LombokOptionsFactory.getDelombokOptions(context);
options.deleteLombokAnnotations();
options.putJavacOption("ENCODING", charset.name());
if (classpath != null) options.putJavacOption("CLASSPATH", unpackClasspath(classpath));
if (sourcepath != null) options.putJavacOption("SOURCEPATH", sourcepath);
if (bootclasspath != null) options.putJavacOption("BOOTCLASSPATH",
unpackClasspath(bootclasspath));
options.setFormatPreferences(new FormatPreferences(formatPrefs));
options.put("compilePolicy", "check");
if (Javac.getJavaCompilerVersion() >= 9) {
Arguments args = Arguments.instance(context);
List<String> argsList = new ArrayList<String>();
if (classpath != null) {
argsList.add("--class-path");
argsList.add(options.get("--class-path"));
}
if (sourcepath != null) {
argsList.add("--source-path");
argsList.add(options.get("--source-path"));
}
if (bootclasspath != null) {
argsList.add("--boot-class-path");
argsList.add(options.get("--boot-class-path"));
}
if (charset != null) {
argsList.add("-encoding");
```

```
argsList.add(charset.name());
}
String pathToSelfJar = getPathOfSelf();
if (pathToSelfJar != null) {
argsList.add("--module-path");
argsList.add((modulepath == null || modulepath.isEmpty()) ? pathToSelfJar : (pathToSelfJar +
File.pathSeparator + modulepath));
} else if (modulepath != null && !modulepath.isEmpty()) {
argsList.add("--module-path");
argsList.add(modulepath);
}
if (!disablePreview && Javac.getJavaCompilerVersion() >= 11) argsList.add("--enable-preview");
if (Javac.getJavaCompilerVersion() >= 21) argsList.add("-proc:full");
if (Javac.getJavaCompilerVersion() < 15) {
String[] argv = argsList.toArray(new String[0]);
args.init("javac", argv);
} else {
args.init("javac", argsList);
}
options.put("diags.legacy", "TRUE");
options.put("allowStringFolding", "FALSE");
} else {
if (modulepath != null && !modulepath.isEmpty()) throw new IllegalStateException("DELOMBOK:
Option --module-path requires usage of JDK9 or higher.");
}
CommentCatcher catcher = CommentCatcher.create(context, Javac.getJavaCompilerVersion() >= 13);
JavaCompiler compiler = catcher.getCompiler();
List<JCCompilationUnit> roots = new ArrayList<JCCompilationUnit>();
Map<JCCompilationUnit, File> baseMap = new IdentityHashMap<JCCompilationUnit, File>();
Set<AbstractProcessor> processors = new LinkedHashSet<AbstractProcessor>();
processors.add(new lombok.javac.apt.LombokProcessor());
processors.addAll(additionalAnnotationProcessors);
if (Javac.getJavaCompilerVersion() >= 9) {
JavaFileManager jfm_ = context.get(JavaFileManager.class);
if (jfm_ instanceof BaseFileManager) {
Arguments args = Arguments.instance(context);
((BaseFileManager) jfm_).setContext(context); // reinit with options
((BaseFileManager) jfm_).handleOptions(args.getDeferredFileManagerOptions());
}
}
if (Javac.getJavaCompilerVersion() < 9) {
compiler.initProcessAnnotations(processors);
} else {
compiler.initProcessAnnotations(processors, Collections.<JavaFileObject>emptySet(),
Collections.<String>emptySet());
}
Object unnamedModule = null;
if (Javac.getJavaCompilerVersion() >= 9) unnamedModule =
Symtab.instance(context).unnamedModule;
for (File fileToParse : filesToParse) {
JCCompilationUnit unit = compiler.parse(fileToParse.getAbsolutePath());
if (Javac.getJavaCompilerVersion() >= 9) try {
MODULE_FIELD.set(unit, unnamedModule);
```

```
} catch (IllegalAccessException e) {
throw new RuntimeException(e);
}
baseMap.put(unit, fileToBase.get(fileToParse));
roots.add(unit);
}
if (compiler.errorCount() > 0) {
// At least one parse error. No point continuing (a real javac run doesn't either).
return false;
}
for (JCCompilationUnit unit : roots) {
catcher.setComments(unit, new DocCommentIntegrator().integrate(catcher.getComments(unit), unit));
}
if (Javac.getJavaCompilerVersion() >= 9) {
compiler.initModules(com.sun.tools.javac.util.List.from(roots.toArray(new JCCompilationUnit[0])));
}
com.sun.tools.javac.util.List<JCCompilationUnit> trees = compiler.enterTrees(toJavacList(roots));
JavaCompiler delegate;
if (Javac.getJavaCompilerVersion() < 9) {
delegate = compiler.processAnnotations(trees, com.sun.tools.javac.util.List.<String>nil());
} else {
delegate = compiler;
Collection<String> c = com.sun.tools.javac.util.List.nil();
compiler.processAnnotations(trees, c);
}
Object care = callAttributeMethodOnJavaCompiler(delegate, delegate.todo);
callFlowMethodOnJavaCompiler(delegate, care);
FormatPreferences fps = new FormatPreferences(formatPrefs);
for (JCCompilationUnit unit : roots) {
DelombokResult result = new DelombokResult(catcher.getComments(unit),
catcher.getTextBlockStarts(unit), unit, force || options.isChanged(unit), fps);
if (onlyChanged && !result.isChanged() && !options.isChanged(unit)) {
if (verbose) feedback.printf("File: %s [%s]\n", unit.sourcefile.getName(), "unchanged (skipped)");
continue;
}
ListBuffer<JCTree> newDefs = new ListBuffer<JCTree>();
for (JCTree def : unit.defs) {
if (def instanceof JCImport) {
Boolean b = JavacAugments.JCImport_deletable.get((JCImport) def);
if (b == null || !b.booleanValue()) newDefs.append(def);
} else {
newDefs.append(def);
}
}
unit.defs = newDefs.toList();
if (verbose) feedback.printf("File: %s [%s%s]\n", unit.sourcefile.getName(), result.isChanged() ?
"delomboked" : "unchanged", force && !options.isChanged(unit) ? " (forced)" : "");
Writer rawWriter;
if (presetWriter != null) rawWriter = createUnicodeEscapeWriter(presetWriter);
else if (output == null) rawWriter = createStandardOutWriter();
else rawWriter = createFileWriter(output, baseMap.get(unit), unit.sourcefile.toUri());
BufferedWriter writer = new BufferedWriter(rawWriter);
try {
```

```
result.print(writer);
} finally {
if (output != null) {
writer.close();
} else {
writer.flush();
}
}
}
delegate.close();
return true;
}
private String unpackClasspath(String cp) {
String[] parts = cp.split(Pattern.quote(File.pathSeparator));
StringBuilder out = new StringBuilder();
for (String p : parts) {
if (!p.endsWith("*")) {
if (out.length() > 0) out.append(File.pathSeparator);
out.append(p);
continue;
}
File f = new File(p.substring(0, p.length() - 2));
File[] files = f.listFiles();
if (files == null) continue;
for (File file : files) {
if (file.isFile()) {
if (out.length() > 0) out.append(File.pathSeparator);
out.append(p, 0, p.length() - 1);
out.append(file.getName());
}
}
}
return out.toString();
}
private static Method attributeMethod;
/** Method is needed because the call signature has changed between javac6 and javac7; no matter
what we compile against, using delombok in the other means VerifyErrors. */
private static Object callAttributeMethodOnJavaCompiler(JavaCompiler compiler, Todo arg) {
if (attributeMethod == null) {
try {
attributeMethod = Permit.getMethod(JavaCompiler.class, "attribute", java.util.Queue.class);
} catch (NoSuchMethodException e) {
try {
attributeMethod = Permit.getMethod(JavaCompiler.class, "attribute",
com.sun.tools.javac.util.ListBuffer.class);
} catch (NoSuchMethodException e2) {
throw Lombok.sneakyThrow(e2);
}
}
}
return Permit.invokeSneaky(attributeMethod, compiler, arg);
}
private static Method flowMethod;
```

```
/** Method is needed because the call signature has changed between javac6 and javac7; no matter what we compile against, using delombok in the other means VerifyErrors. */
private static void callFlowMethodOnJavaCompiler(JavaCompiler compiler, Object arg) {
if (flowMethod == null) {
try {
flowMethod = Permit.getMethod(JavaCompiler.class, "flow", java.util.Queue.class);
} catch (NoSuchMethodException e) {
try {
flowMethod = Permit.getMethod(JavaCompiler.class, "flow", com.sun.tools.javac.util.List.class);
} catch (NoSuchMethodException e2) {
throw Lombok.sneakyThrow(e2);
}
}
}
Permit.invokeSneaky(flowMethod, compiler, arg);
}
private static String canonical(File dir) {
try {
return dir.getCanonicalPath();
} catch (Exception e) {
return dir.getAbsolutePath();
}
}
private static String getExtension(File dir) {
String name = dir.getName();
int idx = name.lastIndexOf('.');
return idx == -1 ? "" : name.substring(idx+1);
}
private Writer createFileWriter(File outBase, File inBase, URI file) throws IOException {
URI base = inBase.toURI();
URI relative = base.relativize(base.resolve(file));
File outFile;
if (relative.isAbsolute()) {
outFile = new File(outBase, new File(relative).getName());
} else {
outFile = new File(outBase, relative.getPath());
}
outFile.getParentFile().mkdirs();
FileOutputStream out = new FileOutputStream(outFile);
return createUnicodeEscapeWriter(out);
}
private Writer createStandardOutWriter() {
return createUnicodeEscapeWriter(System.out);
}
private Writer createUnicodeEscapeWriter(Writer writer) {
return new UnicodeEscapeWriter(writer, charset);
}
private Writer createUnicodeEscapeWriter(OutputStream out) {
return new UnicodeEscapeWriter(new OutputStreamWriter(out, charset), charset);
}
}
```

lombok/delombok/DelombokApp.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.lang.reflect.InvocationTargetException;
import java.net.MalformedURLException;
import java.net.URL;
import java.util.Arrays;
import java.util.Enumeration;
import java.util.List;
import java.util.Map;
import java.util.jar.JarEntry;
import java.util.jar.JarFile;
import lombok.core.LombokApp;
import lombok.permit.Permit;
import lombok.spi.Provides;
@Provides
public class DelombokApp extends LombokApp {
@Override public int runApp(List<String> args) throws Exception {
try {
Class.forName("com.sun.tools.javac.main.JavaCompiler");
runDirectly(args);
return 0;
} catch (ClassNotFoundException e) {
Class<?> delombokClass = loadDelombok(args);
if (delombokClass == null) {
return 1;
```

```
}
try {
Permit.invoke(Permit.getMethod(loadDelombok(args), "main", String[].class), null, new Object[]
{args.toArray(new String[0])});
} catch (InvocationTargetException e1) {
Throwable t = e1.getCause();
if (t instanceof Error) throw (Error) t;
if (t instanceof Exception) throw (Exception) t;
throw e1;
}
return 0;
}
}
public static Class<?> loadDelombok(List<String> args) throws Exception {
//tools.jar is probably not on the classpath. We're going to try and find it, and then load the rest via a
ClassLoader that includes tools.jar.
final File toolsJar = findToolsJar();
if (toolsJar == null) {
String examplePath = "/path/to/tools.jar";
if (File.separator.equals("\\")) examplePath = "C:\\path\\to\\tools.jar";
StringBuilder sb = new StringBuilder();
for (String arg : args) {
if (sb.length() > 0) sb.append(' ');
if (arg.contains(" ")) {
sb.append("").append(arg).append("");
} else {
sb.append(arg);
}
}
System.err.printf("Can't find tools.jar. Rerun delombok as: java -cp lombok.jar%1$s%2$s
lombok.launch.Main delombok %3$s\n",
File.pathSeparator, examplePath, sb.toString());
return null;
}
// The jar file is used for the lifetime of the classLoader, therefore the lifetime of delombok.
// Since we only read from it, not closing it should not be a problem.
@SuppressWarnings({"resource", "all"}) final JarFile toolsJarFile = new JarFile(toolsJar);
ClassLoader loader = new ClassLoader(DelombokApp.class.getClassLoader()) {
private Class<?> loadStreamAsClass(String name, boolean resolve, InputStream in) throws
ClassNotFoundException {
try {
try {
byte[] b = new byte[65536];
ByteArrayOutputStream out = new ByteArrayOutputStream();
while (true) {
int r = in.read(b);
if (r == -1) break;
out.write(b, 0, r);
}
in.close();
byte[] data = out.toByteArray();
Class<?> c = defineClass(name, data, 0, data.length);
if (resolve) resolveClass(c);
```

```
return c;
} finally {
in.close();
}
} catch (Exception e2) {
throw new ClassNotFoundException(name, e2);
}
}
@Override protected synchronized Class<?> loadClass(String name, boolean resolve) throws
ClassNotFoundException {
String rawName, altName; {
String binName = name.replace(".", "/");
rawName = binName + ".class";
altName = binName + ".SCL.lombok";
}
JarEntry entry = toolsJarFile.getJarEntry(rawName);
if (entry == null) {
if (name.startsWith("lombok.")) {
InputStream res = getParent().getResourceAsStream(rawName);
if (res == null) res = getParent().getResourceAsStream(altName);
return loadStreamAsClass(name, resolve, res);
}
return super.loadClass(name, resolve);
}
try {
return loadStreamAsClass(name, resolve, toolsJarFile.getInputStream(entry));
} catch (IOException e2) {
throw new ClassNotFoundException(name, e2);
}
}
@Override public URL getResource(String name) {
JarEntry entry = toolsJarFile.getJarEntry(name);
if (entry == null) return super.getResource(name);
try {
return new URL("jar:file:" + toolsJar.getAbsolutePath() + "!" + name);
} catch (MalformedURLException ignore) {
return null;
}
}
@Override public Enumeration<URL> getResources(final String name) throws IOException {
JarEntry entry = toolsJarFile.getJarEntry(name);
final Enumeration<URL> parent = super.getResources(name);
if (entry == null) return super.getResources(name);
return new Enumeration<URL>() {
private boolean first = false;
@Override public boolean hasMoreElements() {
return !first || parent.hasMoreElements();
}
@Override public URL nextElement() {
if (!first) {
first = true;
try {
return new URL("jar:file:" + toolsJar.getAbsolutePath() + "!" + name);
```

```
} catch (MalformedURLException ignore) {
return parent.nextElement();
}
}
return parent.nextElement();
}
};
}
};
return loader.loadClass("lombok.delombok.Delombok");
}
private void runDirectly(List<String> args) {
Delombok.main(args.toArray(new String[0]));
}
private static File findToolsJar() {
try {
File toolsJar = findToolsJarViaRT();
if (toolsJar != null) return toolsJar;
} catch (Throwable ignore) {
//fallthrough
}
try {
File toolsJar = findToolsJarViaProperties();
if (toolsJar != null) return toolsJar;
} catch (Throwable ignore) {
//fallthrough
}
try {
File toolsJar = findToolsJarViaEnvironment();
return toolsJar;
} catch (Throwable ignore) {
//fallthrough
}
return null;
}
private static File findToolsJarViaEnvironment() {
for (Map.Entry<String, String> s : System.getenv().entrySet()) {
if ("JAVA_HOME".equalsIgnoreCase(s.getKey())) {
return extensiveCheckToolsJar(new File(s.getValue()));
}
}
return null;
}
private static File findToolsJarViaProperties() {
File home = new File(System.getProperty("java.home", "."));
return extensiveCheckToolsJar(home);
}
private static File extensiveCheckToolsJar(File base) {
File toolsJar = checkToolsJar(base);
if (toolsJar != null) return toolsJar;
toolsJar = checkToolsJar(new File(base, "lib"));
if (toolsJar != null) return toolsJar;
toolsJar = checkToolsJar(new File(base.getParentFile(), "lib"));
```

```
if (toolsJar != null) return toolsJar;
toolsJar = checkToolsJar(new File(new File(base, "jdk"), "lib"));
if (toolsJar != null) return toolsJar;
return null;
}
private static File findToolsJarViaRT() {
String url = ClassLoader.getSystemClassLoader().getResource("java/lang/String.class").toString();
if (!url.startsWith("jar:file:")) return null;
url = url.substring("jar:file:".length());
int idx = url.indexOf('!');
if (idx == -1) return null;
url = url.substring(0, idx);
File toolsJar = checkToolsJar(new File(url).getParentFile());
if (toolsJar != null) return toolsJar;
toolsJar = checkToolsJar(new File(new File(url).getParentFile().getParentFile().getParentFile(), "lib"));
if (toolsJar != null) return toolsJar;
return null;
}
private static File checkToolsJar(File d) {
if (d.getName().equals("tools.jar") && d.isFile() && d.canRead()) return d;
d = new File(d, "tools.jar");
if (d.getName().equals("tools.jar") && d.isFile() && d.canRead()) return d;
return null;
}
@Override public String getAppName() {
return "delombok";
}
@Override public List<String> getAppAliases() {
return Arrays.asList("unlombok");
}
@Override public String getAppDescription() {
return "Applies lombok transformations without compiling your\njava code (so, 'unpacks' lombok
annotations and such).";
}
}
```

lombok/delombok/DelombokResult.java

```
/*
* Copyright (C) 2009-2019 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
```

```
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import java.io.IOException;
import java.io.Writer;
import java.util.Date;
import java.util.List;
import javax.tools.JavaFileObject;
import lombok.javac.CommentInfo;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
public class DelombokResult {
private final List<CommentInfo> comments;
private final List<Integer> textBlockStarts;
private final JCCompilationUnit compilationUnit;
private final boolean changed;
private final FormatPreferences formatPreferences;
public DelombokResult(List<CommentInfo> comments, List<Integer> textBlockStarts,
JCCompilationUnit compilationUnit, boolean changed, FormatPreferences formatPreferences) {
this.comments = comments;
this.textBlockStarts = textBlockStarts;
this.compilationUnit = compilationUnit;
this.changed = changed;
this.formatPreferences = formatPreferences;
}
public void print(Writer out) throws IOException {
if (!changed) {
CharSequence content = getContent();
if (content != null) {
out.write(content.toString());
return;
}
}
if (formatPreferences.generateDelombokComment()) {
out.write("// Generated by delombok at ");
out.write(String.valueOf(new Date()));
out.write(System.getProperty("line.separator"));
}
com.sun.tools.javac.util.List<CommentInfo> comments_;
int[] textBlockStarts_;
if (comments instanceof com.sun.tools.javac.util.List) comments_ =
(com.sun.tools.javac.util.List<CommentInfo>) comments;
else comments_ = com.sun.tools.javac.util.List.from(comments.toArray(new CommentInfo[0]));
textBlockStarts_ = new int[textBlockStarts.size()];
int idx = 0;
for (int tbs : textBlockStarts) textBlockStarts_[idx++] = tbs;
FormatPreferences preferences = new FormatPreferenceScanner().scan(formatPreferences,
getContent());
```

```
//compilationUnit.accept(new PrettyCommentsPrinter(out, compilationUnit, comments_, preferences));
compilationUnit.accept(new PrettyPrinter(out, compilationUnit, comments_, textBlockStarts_,
preferences));
}
private CharSequence getContent() throws IOException {
JavaFileObject sourceFile = compilationUnit.getSourceFile();
if (sourceFile == null) return null;
return sourceFile.getCharContent(true);
}
public boolean isChanged() {
return changed;
}
}
```

lombok/delombok/DocCommentIntegrator.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.NavigableMap;
import java.util.TreeMap;
import java.util.regex.Pattern;
import com.sun.tools.javac.parser.Tokens.Comment;
import com.sun.tools.javac.tree.DocCommentTable;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
```

```
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.TreeScanner;
import lombok.javac.CommentInfo;
import lombok.javac.Javac;
import lombok.javac.handlers.JavacHandlerUtil;
public class DocCommentIntegrator {
/**
* Returns the same comment list as when this integrator was created, minus all doc comments that
have been successfully integrated into the compilation unit.
*/
public List<CommentInfo> integrate(List<CommentInfo> comments, JCCompilationUnit unit) {
List<CommentInfo> out = new ArrayList<CommentInfo>();
CommentInfo lastExcisedComment = null;
JCTree lastNode = null;
NavigableMap<Integer, JCTree> positionMap = buildNodePositionMap(unit);
for (CommentInfo cmt : comments) {
if (!cmt.isJavadoc()) {
out.add(cmt);
continue;
}
Entry<Integer, JCTree> entry = positionMap.ceilingEntry(cmt.endPos);
if (entry == null) {
out.add(cmt);
continue;
}
JCTree node = entry.getValue();
if (node == lastNode) {
out.add(lastExcisedComment);
}
if (!attach(unit, node, cmt)) {
out.add(cmt);
} else {
lastNode = node;
lastExcisedComment = cmt;
}
}
return out;
}
private NavigableMap<Integer, JCTree> buildNodePositionMap(JCCompilationUnit unit) {
final NavigableMap<Integer, JCTree> positionMap = new TreeMap<Integer, JCTree>();
unit.accept(new TreeScanner() {
@Override
public void visitClassDef(JCClassDecl tree) {
positionMap.put(tree.pos, tree);
super.visitClassDef(tree);
}
@Override
public void visitMethodDef(JCMethodDecl tree) {
positionMap.put(tree.pos, tree);
super.visitMethodDef(tree);
}
@Override
public void visitVarDef(JCVariableDecl tree) {
```

```
positionMap.put(tree.pos, tree);
super.visitVarDef(tree);
}
});
return positionMap;
}
private static final Pattern CONTENT_STRIPPER = Pattern.compile("^(?:\\s*\\*)?(.*?)$",
Pattern.MULTILINE);
@SuppressWarnings("unchecked") private boolean attach(JCCompilationUnit top, final JCTree node,
CommentInfo cmt) {
String docCommentContent = cmt.content;
if (docCommentContent.startsWith("/**")) docCommentContent = docCommentContent.substring(3);
if (docCommentContent.endsWith("*/")) docCommentContent = docCommentContent.substring(0,
docCommentContent.length() - 2);
docCommentContent = CONTENT_STRIPPER.matcher(docCommentContent).replaceAll("$1");
docCommentContent = docCommentContent.trim();
if (Javac.getDocComments(top) == null) Javac.initDocComments(top);
Object map_ = Javac.getDocComments(top);
if (map_ instanceof Map) {
((Map<JCTree, String>) map_).put(node, docCommentContent);
return true;
} else if (Javac.instanceOfDocCommentTable(map_)) {
CommentAttacher_8.attach(node, docCommentContent, cmt.pos, map_);
return true;
}
return false;
}
/* Container for code which will cause class loader exceptions on javac below 8. By being in a separate
class, we avoid the problem. */
private static class CommentAttacher_8 {
static void attach(final JCTree node, String docCommentContent, final int pos, Object map_) {
final String docCommentContent_ = docCommentContent;
((DocCommentTable) map_).putComment(node, new Comment() {
@Override public String getText() {
return docCommentContent_;
}
@Override public int getSourcePos(int index) {
return pos + index;
}
@Override public CommentStyle getStyle() {
return CommentStyle.JAVADOC;
}
@Override public boolean isDeprecated() {
return JavacHandlerUtil.nodeHasDeprecatedFlag(node);
}
});
}
}
}
```

lombok/delombok/FormatPreferenceScanner.java

```
/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import java.io.CharArrayReader;
import java.io.IOException;
import java.io.Reader;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
/**
* Scans a java source file to figure out certain format preferences.
* Currently:<ul>
* <li>Indent style (tab or 2-8 spaces are supported). Default: tab</li>
* <li>Empty lines still carry the appropriate indent, vs. empty lines are empty. Default: empty</li>
*/
public class FormatPreferenceScanner {
/** Checks validity of preferences, and returns with a non-null value if ALL format keys are available,
thus negating the need for a scan. */
private FormatPreferences tryEasy(FormatPreferences preferences, boolean force) {
int count = 0;
for (Map.Entry<String, String> e : preferences.rawMap.entrySet()) {
if (!"scan".equalsIgnoreCase(e.getValue())) count++;
}
if (force || count >= FormatPreferences.KEYS.size()) return preferences;
return null;
}
public FormatPreferences scan(FormatPreferences preferences, final CharSequence source) {
FormatPreferences fps = tryEasy(preferences, source == null);
if (fps != null) return fps;
try {
return scan_(preferences, new Reader() {
int pos = 0;
```

```
int max = source.length();
@Override public void close() throws IOException {
}
@Override public int read(char[] b, int p, int len) throws IOException {
int read = 0;
if (pos >= max) return -1;
for (int i = p; i < p + len; i++) {
b[i] = source.charAt(pos++);
read++;
if (pos == max) return read;
}
return len;
}
});
} catch (IOException e) {
throw new RuntimeException(e); //Can't happen; CAR doesn't throw these.
}
}
public FormatPreferences scan(FormatPreferences preferences, char[] source) {
FormatPreferences fps = tryEasy(preferences, source == null);
if (fps != null) return fps;
try {
return scan_(preferences, new CharArrayReader(source));
} catch (IOException e) {
throw new RuntimeException(e); //Can't happen; CAR doesn't throw these.
}
}
public FormatPreferences scan(FormatPreferences preferences, Reader in) throws IOException {
FormatPreferences fps = tryEasy(preferences, in == null);
if (fps != null) return fps;
return scan_(preferences, in);
}
private static FormatPreferences scan_(FormatPreferences preferences, Reader in) throws
IOException {
int filledEmpties = 0;
List<String> indents = new ArrayList<String>();
char[] buffer = new char[32700];
int pos = 1;
int end = 0;
StringBuilder indentSoFar = new StringBuilder();
boolean inIndent = true;
boolean inComment = false;
char lastChar = ' ';
while (true) {
if (pos >= end) {
int r = in.read(buffer);
if (r == -1) break;
pos = 0;
end = r;
continue;
}
char c = buffer[pos++];
if (inComment) {
```

```
if (lastChar == '*' && c == '/') inComment = false;
lastChar = c;
continue;
}
if (lastChar == '/' && c == '*') {
inComment = true;
lastChar = ' ';
indentSoFar.setLength(0);
inIndent = false;
continue;
}
if (inIndent) {
boolean w = Character.isWhitespace(c);
if (c == '\n') {
if (indentSoFar.length() > 0 && indentSoFar.charAt(indentSoFar.length() -1) == '\r') {
indentSoFar.setLength(indentSoFar.length() - 1);
}
if (indentSoFar.length() > 0) {
filledEmpties++;
} else {
}
indents.add(indentSoFar.toString());
indentSoFar.setLength(0);
lastChar = c;
continue;
}
if (w) {
indentSoFar.append(c);
lastChar = c;
continue;
}
if (indentSoFar.length() > 0) {
indents.add(indentSoFar.toString());
indentSoFar.setLength(0);
}
lastChar = c;
inIndent = false;
continue;
}
lastChar = c;
if (c == '\n') {
inIndent = true;
indentSoFar.setLength(0);
}
}
String indent = null;
int lowestSpaceCount = Integer.MAX_VALUE;
for (String ind : indents) {
if (ind.indexOf('\t') > -1) {
indent = "\t";
break;
}
if (ind.length() < 2 || ind.length() > 8) continue;
```

```
if (ind.length() < lowestSpaceCount) lowestSpaceCount = ind.length();
}
if (lowestSpaceCount == Integer.MAX_VALUE) indent = "\t";
if (indent == null) {
char[] id = new char[lowestSpaceCount];
Arrays.fill(id, ' ');
indent = new String(id);
}
return new FormatPreferences(preferences.rawMap, indent, filledEmpties > 0);
}
}
```

lombok/delombok/FormatPreferences.java

```
/*
* Copyright (C) 2013-2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import java.util.Arrays;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;
public final class FormatPreferences {
private final String indent;
private final Boolean filledEmpties;
private final boolean generateFinalParams;
private final boolean generateConstructorProperties;
private final boolean generateSuppressWarnings, danceAroundIdeChecks,
generateDelombokComment, javaLangAsFqn;
final Map<String, String> rawMap;
private final boolean generateGenerated;
static final Map<String, String> KEYS;
static {
```

```
Map<String, String> keys = new LinkedHashMap<String, String>();
keys.put("indent", "The indent to use. 'tab' can be used to represent 1 tab. A number means that many spaces. Default: 'tab'");
keys.put("emptyLines", "Either 'indent' or 'blank'. indent means: Indent an empty line to the right level. Default: 'blank'");
keys.put("finalParams", "Either 'generate' or 'skip'. generate means: All lombok-generated methods set all parameters to final. Default: 'generate'");
keys.put("constructorProperties", "Either 'generate' or 'skip'. generate means: All lombok-generated constructors with 1 or more arguments get an @ConstructorProperties annotation. Default: 'generate'");
keys.put("suppressWarnings", "Either 'generate' or 'skip'. generate means: All lombok-generated methods, types, and fields get a @SuppressWarnings annotation. Default: 'generate'");
keys.put("generated", "Either 'generate' or 'skip'. generate means: All lombok-generated methods, types, and fields get a @javax.annotation.Generated(\"lombok\") annotation. Default: 'generate'");
keys.put("danceAroundIdeChecks", "Either 'generate' or 'skip'. generate means: Lombok will intentionally obfuscate some generated code to avoid IDE warnings. Default: 'generate'");
keys.put("generateDelombokComment", "Either 'generate' or 'skip'. generate means: Any file modified by delombok will have a comment stating this at the top. Default: 'generate'");
keys.put("javaLangAsFQN", "Either 'generate' or 'skip'. generate means: Any generated reference to java.lang classes are prefixed with `java.lang.`. Default: 'generate'");
KEYS = Collections.unmodifiableMap(keys);
}
public FormatPreferences(Map<String, String> preferences) {
this(preferences, null, null);
}
public FormatPreferences(Map<String, String> preferences, String indent, Boolean filledEmpties) {
this.rawMap = preferences;
if (preferences == null) preferences = Collections.emptyMap();
String indent_ = preferences.get("indent");
if (indent_ != null && !"scan".equalsIgnoreCase(indent_)) {
try {
int id = Integer.parseInt(indent_);
if (id > 0 && id < 32) {
char[] c = new char[id];
Arrays.fill(c, ' ');
indent_ = new String(c);
}
} catch (NumberFormatException ignore) {}
indent = indent_.replace("\\t", "\t").replace("tab", "\t");
}
String empties_ = preferences.get("emptyLines".toLowerCase());
if ("indent".equalsIgnoreCase(empties_)) filledEmpties = true;
else if ("blank".equalsIgnoreCase(empties_)) filledEmpties = false;
else if (empties_ != null && !"scan".equalsIgnoreCase(empties_)) {
throw new IllegalArgumentException("Legal values for 'emptyLines' are 'scan', 'indent', or 'blank'.");
}
this.indent = indent;
this.filledEmpties = filledEmpties;
this.generateFinalParams = unrollBoolean(preferences, "finalParams", "generate", "skip", true);
this.generateConstructorProperties = unrollBoolean(preferences, "constructorProperties", "generate", "skip", true);
this.generateSuppressWarnings = unrollBoolean(preferences, "suppressWarnings", "generate", "skip", true);
this.generateGenerated = unrollBoolean(preferences, "generated", "generate", "skip", true);
```

```
this.danceAroundIdeChecks = unrollBoolean(preferences, "danceAroundIdeChecks", "generate",
"skip", true);
this.generateDelombokComment = unrollBoolean(preferences, "generateDelombokComment",
"generate", "skip", true);
this.javaLangAsFqn = unrollBoolean(preferences, "javaLangAsFQN", "generate", "skip", true);
}
private static boolean unrollBoolean(Map<String, String> preferences, String name, String trueStr,
String falseStr, boolean defaultVal) {
String v_ = preferences.get(name.toLowerCase());
if (v_ == null) return defaultVal;
if (trueStr.equalsIgnoreCase(v_)) return true;
if (falseStr.equalsIgnoreCase(v_)) return false;
throw new IllegalArgumentException("Legal values for '" + name + "' are '" + trueStr + "', or '" + falseStr
+ "'.");
}
public static Map<String, String> getKeysAndDescriptions() {
return KEYS;
}
/** If true, empty lines should still be appropriately indented. If false, empty lines should be completely
blank. */
public boolean fillEmpties() {
return filledEmpties == null ? false : filledEmpties;
}
public String indent() {
return indent == null ? "\t" : indent;
}
public boolean generateSuppressWarnings() {
return generateSuppressWarnings;
}
public boolean generateGenerated() {
return generateGenerated;
}
public boolean generateFinalParams() {
return generateFinalParams;
}
public boolean danceAroundIdeChecks() {
return danceAroundIdeChecks;
}
public boolean generateDelombokComment() {
return generateDelombokComment;
}
public boolean javaLangAsFqn() {
return javaLangAsFqn;
}
public boolean generateConstructorProperties() {
return generateConstructorProperties;
}
@Override public String toString() {
return rawMap.toString();
}
}
```

lombok/delombok/LombokOptionsFactory.java

```
/*
* Copyright (C) 2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import lombok.javac.Javac;
import lombok.javac.Javac6BasedLombokOptions;
import lombok.javac.Javac8BasedLombokOptions;
import lombok.javac.Javac9BasedLombokOptions;
import lombok.javac.LombokOptions;
import com.sun.tools.javac.util.Context;
import com.sun.tools.javac.util.Options;
public class LombokOptionsFactory {
enum LombokOptionCompilerVersion {
JDK7_AND_LOWER {
@Override LombokOptions createAndRegisterOptions(Context context) {
return Javac6BasedLombokOptions.replaceWithDelombokOptions(context);
}
},
JDK8 {
@Override LombokOptions createAndRegisterOptions(Context context) {
return Javac8BasedLombokOptions.replaceWithDelombokOptions(context);
}
},
JDK9 {
@Override LombokOptions createAndRegisterOptions(Context context) {
return Javac9BasedLombokOptions.replaceWithDelombokOptions(context);
}
};
abstract LombokOptions createAndRegisterOptions(Context context);
}
public static LombokOptions getDelombokOptions(Context context) {
```

```
Options rawOptions = Options.instance(context);
if (rawOptions instanceof LombokOptions) return (LombokOptions) rawOptions;
LombokOptions options;
if (Javac.getJavaCompilerVersion() < 8) {
options = LombokOptionCompilerVersion.JDK7_AND_LOWER.createAndRegisterOptions(context);
} else if (Javac.getJavaCompilerVersion() == 8) {
options = LombokOptionCompilerVersion.JDK8.createAndRegisterOptions(context);
} else {
options = LombokOptionCompilerVersion.JDK9.createAndRegisterOptions(context);
}
return options;
}
}
```

lombok/delombok/PrettyPrinter.java

```
/*
* Copyright (C) 2016-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok;
import static com.sun.tools.javac.code.Flags.*;
import static lombok.javac.Javac.*;
import static lombok.javac.JavacTreeMaker.TreeTag.treeTag;
import static lombok.javac.JavacTreeMaker.TypeTag.typeTag;
import java.io.IOException;
import java.io.Writer;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import com.sun.tools.javac.tree.DocCommentTable;
import com.sun.tools.javac.tree.JCTree;
```

```
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCArrayAccess;
import com.sun.tools.javac.tree.JCTree.JCArrayTypeTree;
import com.sun.tools.javac.tree.JCTree.JCAssert;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCAssignOp;
import com.sun.tools.javac.tree.JCTree.JCBinary;
import com.sun.tools.javac.tree.JCTree.JCBlock;
import com.sun.tools.javac.tree.JCTree.JCBreak;
import com.sun.tools.javac.tree.JCTree.JCCase;
import com.sun.tools.javac.tree.JCTree.JCCatch;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCConditional;
import com.sun.tools.javac.tree.JCTree.JCContinue;
import com.sun.tools.javac.tree.JCTree.JCDoWhileLoop;
import com.sun.tools.javac.tree.JCTree.JCEnhancedForLoop;
import com.sun.tools.javac.tree.JCTree.JCErroneous;
import com.sun.tools.javac.tree.JCTree.JCExpression;
import com.sun.tools.javac.tree.JCTree.JCExpressionStatement;
import com.sun.tools.javac.tree.JCTree.JCFieldAccess;
import com.sun.tools.javac.tree.JCTree.JCForLoop;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCIf;
import com.sun.tools.javac.tree.JCTree.JCImport;
import com.sun.tools.javac.tree.JCTree.JCInstanceOf;
import com.sun.tools.javac.tree.JCTree.JCLabeledStatement;
import com.sun.tools.javac.tree.JCTree.JCLiteral;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCMethodInvocation;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCNewArray;
import com.sun.tools.javac.tree.JCTree.JCNewClass;
import com.sun.tools.javac.tree.JCTree.JCParens;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCReturn;
import com.sun.tools.javac.tree.JCTree.JCSkip;
import com.sun.tools.javac.tree.JCTree.JCStatement;
import com.sun.tools.javac.tree.JCTree.JCSwitch;
import com.sun.tools.javac.tree.JCTree.JCSynchronized;
import com.sun.tools.javac.tree.JCTree.JCThrow;
import com.sun.tools.javac.tree.JCTree.JCTry;
import com.sun.tools.javac.tree.JCTree.JCTypeApply;
import com.sun.tools.javac.tree.JCTree.JCTypeCast;
import com.sun.tools.javac.tree.JCTree.JCTypeParameter;
import com.sun.tools.javac.tree.JCTree.JCUnary;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.JCWhileLoop;
import com.sun.tools.javac.tree.JCTree.JCWildcard;
import com.sun.tools.javac.tree.JCTree.TypeBoundKind;
import com.sun.tools.javac.util.JCDiagnostic.DiagnosticPosition;
import com.sun.tools.javac.util.List;
import com.sun.tools.javac.util.Name;
```

```
import com.sun.tools.javac.util.Position;
import lombok.javac.CommentInfo;
import lombok.javac.Javac;
import lombok.javac.PackageName;
import lombok.permit.Permit;
import lombok.javac.CommentInfo.EndConnection;
import lombok.javac.CommentInfo.StartConnection;
import lombok.javac.JavacTreeMaker.TreeTag;
import lombok.javac.JavacTreeMaker.TypeTag;
public class PrettyPrinter extends JCTree.Visitor {
private static final String LINE_SEP = System.getProperty("line.separator");
private static final Map<TreeTag, String> OPERATORS;
static {
Map<TreeTag, String> map = new HashMap<TreeTag, String>();
map.put(treeTag("POS"), "+");
map.put(treeTag("NEG"), "-");
map.put(treeTag("NOT"), "!");
map.put(treeTag("COMPL"), "~");
map.put(treeTag("PREINC"), "++");
map.put(treeTag("PREDEC"), "--");
map.put(treeTag("POSTINC"), "++");
map.put(treeTag("POSTDEC"), "--");
map.put(treeTag("NULLCHK"), "<*nullchk*>");
map.put(treeTag("OR"), "||");
map.put(treeTag("AND"), "&&");
map.put(treeTag("EQ"), "==");
map.put(treeTag("NE"), "!=");
map.put(treeTag("LT"), "<");
map.put(treeTag("GT"), ">");
map.put(treeTag("LE"), "<=");
map.put(treeTag("GE"), ">=");
map.put(treeTag("BITOR"), "|");
map.put(treeTag("BITXOR"), "^");
map.put(treeTag("BITAND"), "&");
map.put(treeTag("SL"), "<<");
map.put(treeTag("SR"), ">>");
map.put(treeTag("USR"), ">>>");
map.put(treeTag("PLUS"), "+");
map.put(treeTag("MINUS"), "-");
map.put(treeTag("MUL"), "*");
map.put(treeTag("DIV"), "/");
map.put(treeTag("MOD"), "%");
map.put(treeTag("BITOR_ASG"), "|=");
map.put(treeTag("BITXOR_ASG"), "^=");
map.put(treeTag("BITAND_ASG"), "&=");
map.put(treeTag("SL_ASG"), "<<=");
map.put(treeTag("SR_ASG"), ">>=");
map.put(treeTag("USR_ASG"), ">>>=");
map.put(treeTag("PLUS_ASG"), "+=");
map.put(treeTag("MINUS_ASG"), "-=");
map.put(treeTag("MUL_ASG"), "*=");
map.put(treeTag("DIV_ASG"), "/=");
map.put(treeTag("MOD_ASG"), "%=");
```

```
OPERATORS = map;
}
private final Writer out;
private final JCCompilationUnit compilationUnit;
private List<CommentInfo> comments;
private final int[] textBlockStarts;
private final FormatPreferences formatPreferences;
private final Map<JCTree, String> docComments;
private final DocCommentTable docTable;
private int indent = 0;
@SuppressWarnings({"unchecked", "rawtypes"})
public PrettyPrinter(Writer out, JCCompilationUnit cu, List<CommentInfo> comments, int[]
textBlockStarts, FormatPreferences preferences) {
this.out = out;
this.comments = comments;
this.textBlockStarts = textBlockStarts;
this.compilationUnit = cu;
this.formatPreferences = preferences;
/* load doc comments */ {
Object dc = getDocComments(compilationUnit);
if (dc instanceof Map<?, ?>) {
this.docComments = (Map) dc;
this.docTable = null;
} else if (dc instanceof DocCommentTable) {
this.docComments = null;
this.docTable = (DocCommentTable) dc;
} else {
this.docComments = null;
this.docTable = null;
}
}
}
private int endPos(JCTree tree) {
return getEndPosition(tree, compilationUnit);
}
private boolean needsAlign, needsNewLine, onNewLine = true, needsSpace, aligned;
public static final class UncheckedIOException extends RuntimeException {
UncheckedIOException(IOException source) {
super(toMsg(source));
setStackTrace(source.getStackTrace());
}
private static String toMsg(Throwable t) {
String msg = t.getMessage();
String n = t.getClass().getSimpleName();
if (msg == null || msg.isEmpty()) return n;
return n + ": " + msg;
}
}
private void align() {
if (!onNewLine) return;
try {
for (int i = 0; i < indent; i++) out.write(formatPreferences.indent());
} catch (IOException e) {
```

```
throw new UncheckedIOException(e);
}
onNewLine = false;
aligned = true;
needsAlign = false;
}
private void print(JCTree tree) {
if (tree == null) {
print("/*missing*/");
return;
}
consumeComments(tree);
tree.accept(this);
consumeTrailingComments(endPos(tree));
}
private void print(List<? extends JCTree> trees, String infix) {
boolean first = true;
JCTree prev = null;
for (JCTree tree : trees) {
if (suppress(tree)) continue;
if (!first && infix != null && !infix.isEmpty()) {
if ("\n".equals(infix)) println(prev);
else print(infix);
}
first = false;
print(tree);
prev = tree;
}
}
private boolean suppress(JCTree tree) {
if (tree instanceof JCBlock) {
JCBlock block = (JCBlock) tree;
return (Position.NOPOS == block.pos) && block.stats.isEmpty();
}
if (tree instanceof JCExpressionStatement) {
JCExpression expr = ((JCExpressionStatement)tree).expr;
if (expr instanceof JCMethodInvocation) {
JCMethodInvocation inv = (JCMethodInvocation) expr;
if (!inv.typeargs.isEmpty() || !inv.args.isEmpty()) return false;
if (!(inv.meth instanceof JCIdent)) return false;
if (tree.pos != expr.pos) return false; // Explicit super call
return ((JCIdent) inv.meth).name.toString().equals("super");
}
}
return false;
}
private void print(Name name) {
if (name.isEmpty()) {
print("_");
} else {
print(name.toString());
}
}
```

```
private void print(CharSequence s) {
boolean align = needsAlign;
if (needsNewLine && !onNewLine) println();
if (align && !aligned) align();
try {
if (needsSpace && !onNewLine && !aligned) out.write(' ');
out.write(s.toString());
} catch (IOException e) {
throw new UncheckedIOException(e);
}
needsSpace = false;
onNewLine = false;
aligned = false;
}
private void println() {
try {
out.write(LINE_SEP);
} catch (IOException e) {
throw new UncheckedIOException(e);
}
onNewLine = true;
aligned = false;
needsNewLine = false;
}
private void println(JCTree completed) {
if (completed != null) {
int endPos = endPos(completed);
consumeTrailingComments(endPos);
}
try {
out.write(LINE_SEP);
} catch (IOException e) {
throw new UncheckedIOException(e);
}
onNewLine = true;
aligned = false;
needsNewLine = false;
}
private void println(CharSequence s) {
print(s);
println();
}
private void println(CharSequence s, JCTree completed) {
print(s);
println(completed);
}
private void aPrint(CharSequence s) {
align();
print(s);
}
private void aPrintln(CharSequence s) {
align();
print(s);
```

```
println();
}
private void aPrintln(CharSequence s, JCTree completed) {
align();
print(s);
println(completed);
}
private void consumeComments(int until) {
CommentInfo head = comments.head;
while (comments.nonEmpty() && head.pos < until) {
printComment(head);
comments = comments.tail;
head = comments.head;
}
}
private void consumeComments(JCTree tree) {
consumeComments(tree.pos);
}
private void consumeTrailingComments(int from) {
boolean prevNewLine = onNewLine;
CommentInfo head = comments.head;
boolean stop = false;
while (comments.nonEmpty() && head.prevEndPos == from && !stop && !(head.start ==
StartConnection.ON_NEXT_LINE || head.start == StartConnection.START_OF_LINE)) {
from = head.endPos;
printComment(head);
stop = (head.end == EndConnection.ON_NEXT_LINE);
comments = comments.tail;
head = comments.head;
}
if (!onNewLine && prevNewLine) {
println();
}
}
private String getJavadocFor(JCTree node) {
if (docComments != null) return docComments.get(node);
if (docTable != null) return docTable.getCommentText(node);
return null;
}
private int dims(JCExpression vartype) {
if (vartype instanceof JCArrayTypeTree) {
return 1 + dims(((JCArrayTypeTree) vartype).elemtype);
} else if (isJcAnnotatedType(vartype)) {
JCTree underlyingType = readObject(vartype, "underlyingType", (JCTree) null);
if (underlyingType instanceof JCArrayTypeTree) return 1 + dims (((JCArrayTypeTree)
underlyingType).elemtype);
}
return 0;
}
private void printComment(CommentInfo comment) {
switch (comment.start) {
case DIRECT_AFTER_PREVIOUS:
needsSpace = false;
```

```
break;
case AFTER_PREVIOUS:
needsSpace = true;
break;
case START_OF_LINE:
needsNewLine = true;
needsAlign = false;
break;
case ON_NEXT_LINE:
if (!onNewLine) {
needsNewLine = true;
needsAlign = true;
} else if (!aligned) {
needsAlign = true;
}
break;
}
if (onNewLine && !aligned && comment.start != StartConnection.START_OF_LINE) needsAlign = true;
print(comment.content);
switch (comment.end) {
case ON_NEXT_LINE:
if (!aligned) {
needsNewLine = true;
needsAlign = true;
}
break;
case AFTER_COMMENT:
needsSpace = true;
break;
case DIRECT_AFTER_COMMENT:
// do nothing
break;
}
}
private void printDocComment(JCTree tree) {
String dc = getJavadocFor(tree);
if (dc == null) return;
aPrintln("/**");
boolean atStart = true;
for (String line : dc.split("\\r?\\n")) {
if (atStart && line.trim().isEmpty()) {
atStart = false;
continue;
}
atStart = false;
aPrint(" *");
if (!line.isEmpty() && !Character.isWhitespace(line.charAt(0))) {
print(" ");
}
println(line);
}
aPrintln(" */");
}
```

```
private Name __INIT__, __VALUE__;
private Name name_init(Name someName) {
if (__INIT__ == null) __INIT__ = someName.table.fromChars("<init>".toCharArray(), 0, 6);
return __INIT__;
}
private Name name_value(Name someName) {
if (__VALUE__ == null) __VALUE__ = someName.table.fromChars("value".toCharArray(), 0, 5);
return __VALUE__;
}
@Override public void visitTopLevel(JCCompilationUnit tree) {
printDocComment(tree);
JCTree n = PackageName.getPackageNode(tree);
if (n != null) {
consumeComments(tree);
aPrint("package ");
print(n);
println(";", n);
}
boolean first = true;
for (JCTree child : tree.defs) {
if (!(child instanceof JCImport)) continue;
if (first) println();
first = false;
print(child);
}
for (JCTree child : tree.defs) {
if (child instanceof JCImport) continue;
print(child);
}
consumeComments(Integer.MAX_VALUE);
}
@Override public void visitImport(JCImport tree) {
JCTree qualid = Javac.getQualid(tree);
if (qualid instanceof JCFieldAccess) {
JCFieldAccess fa = ((JCFieldAccess) qualid);
if (fa.name.length() == 1 && fa.name.contentEquals("*")) {
if (fa.selected instanceof JCFieldAccess) {
JCFieldAccess lombokExperimental = (JCFieldAccess) fa.selected;
if (lombokExperimental.name.contentEquals("experimental") && lombokExperimental.selected
instanceof JCIdent && ((JCIdent) lombokExperimental.selected).name.contentEquals("lombok")) {
// do not ever print lombok.experimental.*.
return;
}
}
}
}
aPrint("import ");
if (tree.staticImport) print("static ");
print(qualid);
println(";", tree);
}
private Name currentTypeName;
@Override public void visitClassDef(JCClassDecl tree) {
```

```
println();
printDocComment(tree);
align();
print(tree.mods);
boolean isInterface = (tree.mods.flags & INTERFACE) != 0;
boolean isAnnotationInterface = isInterface && (tree.mods.flags & ANNOTATION) != 0;
boolean isEnum = (tree.mods.flags & ENUM) != 0;
boolean isRecord = (tree.mods.flags & RECORD) != 0;
if (isAnnotationInterface) print("@interface ");
else if (isInterface) print("interface ");
else if (isEnum) print("enum ");
else if (isRecord) print("record ");
else print("class ");
print(tree.name);
Name prevTypeName = currentTypeName;
currentTypeName = tree.name;
if (tree.typarams.nonEmpty()) {
print("<");
print(tree.typarams, ", ");
print(">");
}
if (isRecord) printRecordConstructor(tree.defs);
JCTree extendsClause = getExtendsClause(tree);
if (extendsClause != null) {
print(" extends ");
print(extendsClause);
}
if (tree.implementing.nonEmpty()) {
print(isInterface ? " extends " : " implements ");
print(tree.implementing, ", ");
}
List<JCExpression> permitting = readObject(tree, "permitting", List.<JCExpression>nil());
if (permitting.nonEmpty()) {
print(" permits ");
print(permitting, ", ");
}
println(" {");
indent++;
printClassMembers(tree.defs, isEnum, isInterface);
consumeComments(endPos(tree));
indent--;
aPrintln("}", tree);
currentTypeName = prevTypeName;
}
private void printRecordConstructor(List<JCTree> members) {
boolean first = true;
print("(");
for (JCTree member : members) {
if (member instanceof JCVariableDecl) {
JCVariableDecl variableDecl = (JCVariableDecl) member;
if ((variableDecl.mods.flags & GENERATED_MEMBER) != 0) {
if (!first) print(", ");
first = false;
```

```
printAnnotations(variableDecl.mods.annotations, false);
printVarDef0(variableDecl);
}
}
}
print(")");
}
private void printClassMembers(List<JCTree> members, boolean isEnum, boolean isInterface) {
Class<?> prefType = null;
int typeOfPrevEnumMember = isEnum ? 3 : 0; // 1 = normal, 2 = with body, 3 = no enum field yet.
boolean prevWasEnumMember = isEnum;
for (JCTree member : members) {
if (typeOfPrevEnumMember == 3 && member instanceof JCMethodDecl && (((JCMethodDecl)
member).mods.flags & GENERATEDCONSTR) != 0) continue;
boolean isEnumVar = isEnum && member instanceof JCVariableDecl && (((JCVariableDecl)
member).mods.flags & ENUM) != 0;
if (!isEnumVar && prevWasEnumMember) {
prevWasEnumMember = false;
if (typeOfPrevEnumMember == 3) align();
println(";");
}
if (isEnumVar) {
if (prefType != null && prefType != JCVariableDecl.class) println();
switch (typeOfPrevEnumMember) {
case 1:
print(", ");
break;
case 2:
println(",");
align();
break;
}
print(member);
JCTree init = ((JCVariableDecl) member).init;
typeOfPrevEnumMember = init instanceof JCNewClass && ((JCNewClass) init).def != null ? 2 : 1;
} else if (member instanceof JCVariableDecl) {
if ((((JCVariableDecl) member).mods.flags & GENERATED_MEMBER) != 0) continue;
if (prefType != null && prefType != JCVariableDecl.class) println();
if (isInterface) flagMod = -1L & ~(PUBLIC | STATIC | FINAL);
print(member);
} else if (member instanceof JCMethodDecl) {
if ((((JCMethodDecl) member).mods.flags & GENERATEDCONSTR) != 0) continue;
if (prefType != null) println();
if (isInterface) flagMod = -1L & ~(PUBLIC | ABSTRACT);
print(member);
} else if (member instanceof JCClassDecl) {
if (prefType != null) println();
if (isInterface) flagMod = -1L & ~(PUBLIC | STATIC);
print(member);
} else {
if (prefType != null) println();
print(member);
}
```

```
prefType = member.getClass();
}
if (prevWasEnumMember) {
prevWasEnumMember = false;
if (typeOfPrevEnumMember == 3) align();
println(";");
}
}
@Override public void visitTypeParameter(JCTypeParameter tree) {
List<JCExpression> annotations = readObject(tree, "annotations", List.<JCExpression>nil());
if (!annotations.isEmpty()) {
print(annotations, " ");
print(" ");
}
print(tree.name);
if (tree.bounds.nonEmpty()) {
print(" extends ");
print(tree.bounds, " & ");
}
consumeComments(tree);
}
@Override public void visitVarDef(JCVariableDecl tree) {
printDocComment(tree);
align();
if ((tree.mods.flags & ENUM) != 0) {
printEnumMember(tree);
return;
}
printAnnotations(tree.mods.annotations, true);
printModifierKeywords(tree.mods);
printVarDef0(tree);
println(";", tree);
}
private void printVarDefInline(JCVariableDecl tree) {
printAnnotations(tree.mods.annotations, false);
printModifierKeywords(tree.mods);
printVarDef0(tree);
}
private boolean innermostArrayBracketsAreVarargs = false;
private void printVarDef0(JCVariableDecl tree) {
boolean varargs = (tree.mods.flags & VARARGS) != 0;
/* story time!
in 'new int[5][6];', the 5 is the outermost and the 6 is the innermost: That means: 5 int arrays, each
capable of containing 6 elements.
But that's actually a crazy way to read it; you'd think that in FOO[], you should interpret that as 'an array
of FOO', but that's not correct;
if FOO is for example 'int[]', it's: "Modify the component type of FOO to be an array of whatever it was
before.. unless FOO isn't an array, in which case,
this is an array of FOO". Which is weird.
This is particularly poignant with vargs. In: "int[]... x", the ... are actually the _INNER_ type even though
varargs by definition is a modification of
how to interpret the outer. The JLS just sort of lets that be: To indicate varargs, replace the lexically last
[] with dots even though that's the wrong
```

```
[] to modify!
This becomes an utter shambles when annotations-on-arrays become involved. The annotation on the
INNER most type is to be placed right before the ...;
and because of that, we have to do crazy stuff with this innermostArrayBracketsAreVarargs flag.
*/
try {
innermostArrayBracketsAreVarargs = varargs;
if (tree.vartype == null || tree.vartype.pos == -1) {
print("var");
} else {
print(tree.vartype);
}
} finally {
innermostArrayBracketsAreVarargs = false;
}
print(" ");
print(tree.name);
if (tree.init != null) {
print(" = ");
print(tree.init);
}
}
private void printEnumMember(JCVariableDecl tree) {
printAnnotations(tree.mods.annotations, true);
print(tree.name);
if (tree.init instanceof JCNewClass) {
JCNewClass constructor = (JCNewClass) tree.init;
if (constructor.args != null && constructor.args.nonEmpty()) {
print("(");
print(constructor.args, ", ");
print(")");
}
if (constructor.def != null && constructor.def.defs != null) {
println(" {");
indent++;
printClassMembers(constructor.def.defs, false, false);
consumeComments(endPos(tree));
indent--;
aPrint("}");
}
}
}
// TODO: Test postfix syntax for methods (?), for decls. Multiline vardefs, possibly with comments.
enums with bodies. constructor-local generics, method-local generics, also do/while, finally,
try-with-resources, lambdas, annotations in java8 places...
// TODO: Whatever is JCAnnotatedType? We handle it in the 7+ bucket in the old one...
@Override public void visitTypeApply(JCTypeApply tree) {
print(tree.clazz);
print("<");
boolean temp = innermostArrayBracketsAreVarargs;
innermostArrayBracketsAreVarargs = false;
print(tree.arguments, ", ");
innermostArrayBracketsAreVarargs = temp;
```

```
print(">");
}
@Override public void visitWildcard(JCWildcard tree) {
switch (tree.getKind()) {
default:
case UNBOUNDED_WILDCARD:
print("?");
return;
case EXTENDS_WILDCARD:
print("? extends ");
print(tree.inner);
return;
case SUPER_WILDCARD:
print("? super ");
print(tree.inner);
return;
}
}
@Override public void visitLiteral(JCLiteral tree) {
TypeTag typeTag = typeTag(tree);
if (CTC_INT.equals(typeTag)) print("" + tree.value);
else if (CTC_LONG.equals(typeTag)) print(tree.value + "L");
else if (CTC_FLOAT.equals(typeTag)) print(tree.value + "F");
else if (CTC_DOUBLE.equals(typeTag)) print("" + tree.value);
else if (CTC_CHAR.equals(typeTag)) {
print("\'" + quoteChar((char)((Number)tree.value).intValue()) + "\'");
}
else if (CTC_BOOLEAN.equals(typeTag)) print(((Number)tree.value).intValue() == 1 ? "true" : "false");
else if (CTC_BOT.equals(typeTag)) print("null");
else {
if (Arrays.binarySearch(textBlockStarts, tree.pos) < 0) {
print("\"" + quoteChars(tree.value.toString()) + "\"");
} else {
printTextBlock(tree.value.toString());
}
}
}
private void printTextBlock(String s) {
println("\"\"\"");
needsAlign = true;
indent++;
StringBuilder sb = new StringBuilder();
boolean lineStart = true;
for (int i = 0; i < s.length(); i++) {
char c = s.charAt(i);
if (c != ' ' && c != '\t') lineStart = false;
if (c == '\n') {
println(sb);
sb.setLength(0);
needsAlign = true;
lineStart = true;
continue;
}
```

```
if (c == '\t' && lineStart) sb.append("\t");
else sb.append(quoteChar(s.charAt(i)));
}
print(sb);
print("\"\"\"");
indent--;
}
@Override public void visitMethodDef(JCMethodDecl tree) {
boolean isConstructor = tree.name == name_init(tree.name);
if (isConstructor && (tree.mods.flags & GENERATEDCONSTR) != 0) return;
printDocComment(tree);
align();
print(tree.mods);
if (tree.typarams != null && tree.typarams.nonEmpty()) {
print("<");
print(tree.typarams, ", ");
print("> ");
}
if (isConstructor) {
print(currentTypeName == null ? "<init>" : currentTypeName);
} else {
print(tree.restype);
print(" ");
print(tree.name);
}
boolean argsLessConstructor = false;
if (isConstructor && (tree.mods.flags & COMPACT_RECORD_CONSTRUCTOR) != 0) {
argsLessConstructor = true;
}
boolean first = true;
if (!argsLessConstructor) {
print("(");
JCVariableDecl recvparam = readObject(tree, "recvparam", null);
if (recvparam != null) {
printVarDefInline(recvparam);
first = false;
}
for (JCVariableDecl param : tree.params) {
if (!first) print(", ");
first = false;
printVarDefInline(param);
}
print(")");
}
if (tree.thrown.nonEmpty()) {
print(" throws ");
print(tree.thrown, ", ");
}
if (tree.defaultValue != null) {
print(" default ");
print(tree.defaultValue);
}
if (tree.body != null) {
```

```
print(" ");
print(tree.body);
} else println(";", tree);
}
@Override public void visitSkip(JCSkip that) {
if (onNewLine && !aligned) {
align();
}
println(";");
}
@Override public void visitAnnotation(JCAnnotation tree) {
print("@");
print(tree.annotationType);
if (tree.args.isEmpty()) return;
print("(");
boolean done = false;
if (tree.args.length() == 1 && tree.args.get(0) instanceof JCAssign) {
JCAssign arg1 = (JCAssign) tree.args.get(0);
JCIdent arg1Name = arg1.lhs instanceof JCIdent ? ((JCIdent) arg1.lhs) : null;
if (arg1Name != null && arg1Name.name == name_value(arg1Name.name)) {
print(arg1.rhs);
done = true;
}
}
if (!done) print(tree.args, ", ");
print(")");
}
@Override public void visitTypeArray(JCArrayTypeTree tree) {
printTypeArray0(tree);
}
@Override public void visitNewArray(JCNewArray tree) {
JCTree elem = tree.elemtype;
int dims = 0;
if (elem != null) {
print("new ");
while (elem instanceof JCArrayTypeTree) {
dims++;
elem = ((JCArrayTypeTree) elem).elemtype;
}
print(elem);
for (JCExpression expr : tree.dims) {
print("[");
print(expr);
print("]");
}
}
for (int i = 0; i < dims; i++) print("[]");
if (tree.elems != null) {
if (elem != null) print("[] ");
print("{");
print(tree.elems, ", ");
print("}");
}
```

```
}
@Override public void visitNewClass(JCNewClass tree) {
if (tree.encl != null) {
print(tree.encl);
print(".");
}
boolean moveFirstParameter = tree.args.nonEmpty() && tree.args.head instanceof JCUnary &&
tree.args.head.toString().startsWith("<*nullchk*>");
if (moveFirstParameter) {
print(((JCUnary) tree.args.head).arg);
print(".");
}
print("new ");
if (!tree.typeargs.isEmpty()) {
print("<");
print(tree.typeargs, ", ");
print(">");
}
print(tree.clazz);
print("(");
if (moveFirstParameter) {
print(tree.args.tail, ", ");
} else {
print(tree.args, ", ");
}
print(")");
if (tree.def != null) {
Name previousTypeName = currentTypeName;
currentTypeName = null;
println(" {");
indent++;
print(tree.def.defs, "");
indent--;
aPrint("}");
currentTypeName = previousTypeName;
}
}
@Override public void visitIndexed(JCArrayAccess tree) {
print(tree.indexed);
print("[");
print(tree.index);
print("]");
}
@Override public void visitTypeIdent(JCPrimitiveTypeTree tree) {
TypeTag typeTag = typeTag(tree);
if (CTC_BYTE.equals(typeTag)) print("byte");
else if (CTC_CHAR.equals(typeTag)) print("char");
else if (CTC_SHORT.equals(typeTag)) print("short");
else if (CTC_INT.equals(typeTag)) print("int");
else if (CTC_LONG.equals(typeTag)) print("long");
else if (CTC_FLOAT.equals(typeTag)) print("float");
else if (CTC_DOUBLE.equals(typeTag)) print("double");
else if (CTC_BOOLEAN.equals(typeTag)) print("boolean");
```

```
else if (CTC_VOID.equals(typeTag)) print("void");
else print("error");
}
@Override public void visitLabelled(JCLabeledStatement tree) {
aPrint(tree.label);
print(":");
if (tree.body instanceof JCSkip || suppress(tree)) {
println(" ;", tree);
} else if (tree.body instanceof JCBlock) {
print(" ");
print(tree.body);
} else {
println(tree);
print(tree.body);
}
}
private long flagMod = -1L;
private static final long DEFAULT = 1L<<43;
@Override public void visitModifiers(JCModifiers tree) {
printAnnotations(tree.annotations, true);
printModifierKeywords(tree);
}
private void printAnnotations(List<JCAnnotation> annotations, boolean newlines) {
for (JCAnnotation ann : annotations) {
print(ann);
if (newlines) {
println();
align();
} else print(" ");
}
}
private void printModifierKeywords(JCModifiers tree) {
long v = flagMod & tree.flags;
flagMod = -1L;
if ((v & SYNTHETIC) != 0) print("/* synthetic */ ");
if ((v & PUBLIC) != 0) print("public ");
if ((v & PRIVATE) != 0) print("private ");
if ((v & PROTECTED) != 0) print("protected ");
if ((v & STATIC) != 0) print("static ");
if ((v & FINAL) != 0) print("final ");
if ((v & SYNCHRONIZED) != 0) print("synchronized ");
if ((v & VOLATILE) != 0) print("volatile ");
if ((v & TRANSIENT) != 0) print("transient ");
if ((v & NATIVE) != 0) print("native ");
if ((v & ABSTRACT) != 0) print("abstract ");
if ((v & SEALED) != 0) print("sealed ");
if ((v & NON_SEALED) != 0) print("non-sealed ");
if ((v & STRICTFP) != 0) print("strictfp ");
if ((v & DEFAULT) != 0 && (v & INTERFACE) == 0) print("default ");
}
@Override public void visitSelect(JCFieldAccess tree) {
print(tree.selected);
print(".");
```

```
print(tree.name);
}
@Override public void visitIdent(JCIdent tree) {
print(tree.name);
}
@Override public void visitApply(JCMethodInvocation tree) {
if (tree.typeargs.nonEmpty()) {
if (tree.meth instanceof JCFieldAccess) {
JCFieldAccess fa = (JCFieldAccess) tree.meth;
print(fa.selected);
print(".<");
print(tree.typeargs, ", ");
print(">");
print(fa.name);
} else {
print("<");
print(tree.typeargs, ", ");
print(">");
print(tree.meth);
}
} else {
print(tree.meth);
}
print("(");
print(tree.args, ", ");
print(")");
}
@Override public void visitAssert(JCAssert tree) {
aPrint("assert ");
print(tree.cond);
if (tree.detail != null) {
print(" : ");
print(tree.detail);
}
println(";", tree);
}
@Override public void visitAssign(JCAssign tree) {
print(tree.lhs);
print(" = ");
print(tree.rhs);
}
@Override public void visitAssignop(JCAssignOp tree) {
print(tree.lhs);
String opname = operator(treeTag(tree));
print(" " + opname + " ");
print(tree.rhs);
}
private static final int PREFIX = 14;
@Override public void visitUnary(JCUnary tree) {
String op = operator(treeTag(tree));
if (treeTag(tree).getOperatorPrecedenceLevel() == PREFIX) {
print(op);
print(tree.arg);
```

```
} else {
print(tree.arg);
print(op);
}
}
@Override public void visitBinary(JCBinary tree) {
String op = operator(treeTag(tree));
print(tree.lhs);
print(" ");
print(op);
print(" ");
print(tree.rhs);
}
@Override public void visitTypeTest(JCInstanceOf tree) {
print(tree.expr);
print(" instanceof ");
/** With java14, instead of a type (field 'clazz'), there's now a pattern (field 'pattern', of type JCTree,
which is either a JCPattern for new-style instanceof, or if not it's the same as what 'clazz' held) */
JCTree c = readObject(tree, "clazz", null); // JDK-13
if (c == null) c = readObject(tree, "pattern", null); // JDK14+
print(c);
}
@Override public void visitTypeCast(JCTypeCast tree) {
print("(");
print(tree.clazz);
print(") ");
print(tree.expr);
}
@Override public void visitBlock(JCBlock tree) {
if (tree.pos == Position.NOPOS && tree.stats.isEmpty()) return;
if (onNewLine) align();
if ((tree.flags & STATIC) != 0) print("static ");
println("{");
indent++;
print(tree.stats, "");
consumeComments(endPos(tree));
indent--;
aPrintln("}", tree);
}
@Override public void visitBreak(JCBreak tree) {
aPrint("break");
JCExpression value = readObject(tree, "value", null); // JDK 12+
if (value != null) {
print(" ");
print(value);
} else {
Name label = readObject(tree, "label", null);
if (label != null) {
print(" ");
print(label);
}
}
println(";", tree);
```

```
}
@Override public void visitContinue(JCContinue tree) {
aPrint("continue");
if (tree.label != null) {
print(" ");
print(tree.label);
}
println(";", tree);
}
@Override public void visitConditional(JCConditional tree) {
print(tree.cond);
print(" ? ");
print(tree.truepart);
print(" : ");
print(tree.falsepart);
}
@Override public void visitParens(JCParens tree) {
print("(");
print(tree.expr);
print(")");
}
@Override public void visitReturn(JCReturn tree) {
aPrint("return");
if (tree.expr != null) {
print(" ");
print(tree.expr);
}
println(";", tree);
}
@Override public void visitThrow(JCThrow tree) {
aPrint("throw ");
print(tree.expr);
println(";", tree);
}
@Override public void visitWhileLoop(JCWhileLoop tree) {
aPrint("while ");
if (tree.cond instanceof JCParens) {
print(tree.cond);
} else {
print("(");
print(tree.cond);
print(")");
}
print(" ");
print(tree.body);
// make sure to test while (true) ; and while(true){} and while(true) x = 5;
}
@Override public void visitForLoop(JCForLoop tree) {
aPrint("for (");
if (tree.init.nonEmpty()) {
// ForInit is either a StatementExpressionList or a LocalVariableDeclaration
if (tree.init.head instanceof JCVariableDecl) {
boolean first = true;
```

```
int dims = 0;
for (JCStatement i : tree.init) {
JCVariableDecl vd = (JCVariableDecl) i;
if (first) {
printVarDefInline(vd);
dims = dims(vd.vartype);
} else {
print(", ");
print(vd.name);
int dimDiff = dims(vd.vartype) - dims;
for (int j = 0; j < dimDiff; j++) print("[]");
if (vd.init != null) {
print(" = ");
print(vd.init);
}
}
first = false;
}
} else {
boolean first = true;
for (JCStatement exprStatement : tree.init) {
if (!first) print(", ");
first = false;
print(((JCExpressionStatement) exprStatement).expr);
}
}
}
print("; ");
if (tree.cond != null) print(tree.cond);
print("; ");
boolean first = true;
for (JCExpressionStatement exprStatement : tree.step) {
if (!first) print(", ");
first = false;
print(exprStatement.expr);
}
print(") ");
print(tree.body);
}
@Override public void visitForeachLoop(JCEnhancedForLoop tree) {
aPrint("for (");
JCTree varOrRecordPattern = readObject(tree, "varOrRecordPattern", null);
if (varOrRecordPattern instanceof JCVariableDecl) {
printVarDefInline((JCVariableDecl) varOrRecordPattern);
} else if (varOrRecordPattern != null) {
print(varOrRecordPattern);
} else {
printVarDefInline(tree.var);
}
print(" : ");
print(tree.expr);
print(") ");
print(tree.body);
```

```
}
@Override public void visitIf(JCIf tree) {
aPrint("if ");
if (tree.cond instanceof JCParens) {
print(tree.cond);
} else {
print("(");
print(tree.cond);
print(")");
}
print(" ");
if (tree.thenpart instanceof JCBlock) {
println("{");
indent++;
print(((JCBlock) tree.thenpart).stats, "");
indent--;
if (tree.elsepart == null) {
aPrintln("}", tree);
} else {
aPrint("}");
}
} else {
print(tree.thenpart);
}
if (tree.elsepart != null) {
aPrint(" else ");
print(tree.elsepart);
}
}
@Override public void visitExec(JCExpressionStatement tree) {
align();
print(tree.expr);
println(";", tree);
}
@Override public void visitDoLoop(JCDoWhileLoop tree) {
aPrint("do ");
if (tree.body instanceof JCBlock) {
println("{");
indent++;
print(((JCBlock) tree.body).stats, "");
indent--;
aPrint("}");
} else print(tree.body);
print(" while ");
if (tree.cond instanceof JCParens) {
print(tree.cond);
} else {
print("(");
print(tree.cond);
print(")");
}
println(";", tree);
}
```

```
@Override public void visitSynchronized(JCSynchronized tree) {
aPrint("synchronized ");
if (tree.lock instanceof JCParens) {
print(tree.lock);
} else {
print("(");
print(tree.lock);
print(")");
}
print(" ");
print(tree.body);
}
@Override public void visitCase(JCCase tree) {
// Starting with JDK12, switches allow multiple expressions per case, and can take the form of an
expression (preview feature).
List<JCTree> pats = readObject(tree, "labels", null); // JDK 17+
if (pats == null) {
pats = readObject(tree, "pats", null); // JDK 12-17
}
if (pats == null) {
JCTree pat = readObject(tree, "pat", null); // JDK -11
pats = pat == null ? List.<JCTree>nil() : List.of(pat);
}
if (pats.isEmpty() || pats.size() == 1 &&
pats.head.getClass().getName().endsWith("$JCDefaultCaseLabel")) {
aPrint("default");
} else {
aPrint("case ");
print(pats, ", ");
JCExpression guard = readObject(tree, "guard", null); // JDK 21+
if (guard != null) {
print(" when ");
print(guard);
}
}
Enum<?> caseKind = readObject(tree, "caseKind", null); // JDK 12+
if (caseKind != null && caseKind.name().equalsIgnoreCase("RULE")) {
print(" -> ");
if (tree.stats.head instanceof JCBreak) {
JCBreak b = (JCBreak) tree.stats.head;
print((JCExpression) readObject(b, "value", null));
print(";");
needsNewLine = true;
needsAlign = true;
} else if (tree.stats.head.getClass().getName().endsWith("$JCYield")) {
print((JCExpression) readObject(tree.stats.head, "value", null));
print(";");
needsNewLine = true;
needsAlign = true;
} else {
print(tree.stats.head);
if (tree.stats.head instanceof JCBlock) needsNewLine = false;
}
```

```
} else {
println(": ");
indent++;
print(tree.stats, "");
indent--;
}
}
@Override public void visitCatch(JCCatch tree) {
print(" catch (");
print(tree.param);
print(") ");
print(tree.body);
}
@Override public void visitSwitch(JCSwitch tree) {
aPrint("switch ");
if (tree.selector instanceof JCParens) {
print(tree.selector);
} else {
print("(");
print(tree.selector);
print(")");
}
println(" {");
boolean ruleStyle = isCaseRuleStyle(tree.cases.head);
if (ruleStyle) indent++;
print(tree.cases, "");
if (ruleStyle) indent--;
aPrintln("}", tree);
}
void printSwitchExpression(JCTree tree) {
aPrint("switch ");
JCExpression selector = readObject(tree, "selector", null);
if (selector instanceof JCParens) {
print(selector);
} else {
print("(");
print(selector);
print(")");
}
println(" {");
List<JCCase> cases = readObject(tree, "cases", null);
boolean ruleStyle = isCaseRuleStyle(cases.head);
if (ruleStyle) indent++;
print(cases, "");
if (ruleStyle) indent--;
aPrint("}");
}
void printYieldExpression(JCTree tree) {
aPrint("yield ");
JCExpression value = readObject(tree, "value", null);
print(value);
println(";", tree);
}
```

```
void printBindingPattern(JCTree tree) {
JCTree var = readObject(tree, "var", tree);
if (var instanceof JCVariableDecl) {
printVarDef0((JCVariableDecl) var);
} else {
print((JCExpression) readObject(var, "vartype", null));
print(" ");
print((Name) readObject(var, "name", null));
}
}
void printDefaultCase(JCTree tree) {
print("default");
}
void printGuardPattern(JCTree tree) {
print((JCTree) readObject(tree, "patt", null));
print(" && ");
print((JCExpression) readObject(tree, "expr", null));
}
void printParenthesizedPattern(JCTree tree) {
print("(");
print((JCTree) readObject(tree, "pattern", null));
print(")");
}
void printConstantCaseLabel(JCTree tree) {
print((JCTree) readObject(tree, "expr", null));
}
void printPatternCaseLabel(JCTree tree) {
print((JCTree) readObject(tree, "pat", null));
JCTree guard = readObject(tree, "guard", null);
if (guard != null) {
print(" when ");
print(guard);
}
}
void printRecordPattern(JCTree tree) {
print((JCTree) readObject(tree, "deconstructor", null));
print("(");
print(readObject(tree, "nested", List.<JCTree>nil()), ", ");
print(")");
JCVariableDecl var = readObject(tree, "var", null);
if (var != null) {
print(" ");
print(var.name);
}
}
@Override public void visitTry(JCTry tree) {
aPrint("try ");
List<?> resources = readObject(tree, "resources", List.nil());
int len = resources.length();
switch (len) {
case 0:
break;
case 1:
```

```
print("(");
JCTree resource = (JCTree) resources.get(0);
if (resource instanceof JCVariableDecl) {
JCVariableDecl decl = (JCVariableDecl) resource;
flagMod = -1L & ~FINAL;
printVarDefInline(decl);
} else {
print(resource);
}
print(") ");
break;
default:
println("(");
indent++;
int c = 0;
for (Object i : resources) {
align();
if (i instanceof JCVariableDecl) {
flagMod = -1L & ~FINAL;
printVarDefInline((JCVariableDecl) i);
} else {
print((JCTree) i);
}
if (++c == len) {
print(") ");
} else {
println(";", (JCTree) i);
}
}
indent--;
}
println("{");
indent++;
for (JCStatement stat : tree.body.stats) print(stat);
indent--;
aPrint("}");
for (JCCatch catchBlock : tree.catchers) {
printCatch(catchBlock);
}
if (tree.finalizer != null) {
println(" finally {");
indent++;
for (JCStatement stat : tree.finalizer.stats) print(stat);
indent--;
aPrint("}");
}
println(tree);
}
private void printCatch(JCCatch catchBlock) {
print(" catch (");
printVarDefInline(catchBlock.param); // ExprType1 | ExprType2 handled via JCTypeUnion.
println(") {");
indent++;
```

```
for (JCStatement stat : catchBlock.body.stats) print(stat);
indent--;
aPrint("}");
}
public void visitErroneous(JCErroneous tree) {
print("(ERROR)");
}
private static String operator(TreeTag tag) {
String op = OPERATORS.get(tag);
if (op == null) return "(?op?)";
return op;
}
private static String quoteChars(String s) {
StringBuilder sb = new StringBuilder();
for (int i = 0; i < s.length(); i++) sb.append(quoteChar(s.charAt(i)));
return sb.toString();
}
private static String quoteChar(char ch) {
switch (ch) {
case '\b': return "\\b";
case '\f': return "\\f";
case '\n': return "\\n";
case '\r': return "\\r";
case '\t': return "\\t";
case '\'': return "\\'";
case '\"': return "\\\"";
case '\\': return "\\\\";
default:
if (ch < 32) return String.format("\\%03o", (int) ch);
return String.valueOf(ch);
}
}
private static final Method getExtendsClause, getEndPosition, storeEnd;
static {
getExtendsClause = getMethod(JCClassDecl.class, "getExtendsClause", new Class<?>[0]);
if (getJavaCompilerVersion() < 8) {
getEndPosition = getMethod(DiagnosticPosition.class, "getEndPosition", java.util.Map.class);
storeEnd = getMethod(java.util.Map.class, "put", Object.class, Object.class);
} else {
getEndPosition = getMethod(DiagnosticPosition.class, "getEndPosition",
"com.sun.tools.javac.tree.EndPosTable");
Method storeEndMethodTemp;
Class<?> endPosTable;
try {
endPosTable = Class.forName("com.sun.tools.javac.tree.EndPosTable");
} catch (ClassNotFoundException ex) {
throw sneakyThrow(ex);
}
try {
storeEndMethodTemp = Permit.getMethod(endPosTable, "storeEnd", JCTree.class, int.class);
} catch (NoSuchMethodException e) {
try {
endPosTable = Class.forName("com.sun.tools.javac.parser.JavacParser$AbstractEndPosTable");
```

```
storeEndMethodTemp = Permit.getMethod(endPosTable, "storeEnd", JCTree.class, int.class);
} catch (NoSuchMethodException ex) {
throw sneakyThrow(ex);
} catch (ClassNotFoundException ex) {
throw sneakyThrow(ex);
}
}
storeEnd = storeEndMethodTemp;
}
Permit.setAccessible(getEndPosition);
Permit.setAccessible(storeEnd);
}
private static Method getMethod(Class<?> clazz, String name, Class<?>... paramTypes) {
try {
return Permit.getMethod(clazz, name, paramTypes);
} catch (NoSuchMethodException e) {
throw sneakyThrow(e);
}
}
private static Method getMethod(Class<?> clazz, String name, String... paramTypes) {
try {
Class<?>[] c = new Class[paramTypes.length];
for (int i = 0; i < paramTypes.length; i++) c[i] = Class.forName(paramTypes[i]);
return Permit.getMethod(clazz, name, c);
} catch (NoSuchMethodException e) {
throw sneakyThrow(e);
} catch (ClassNotFoundException e) {
throw sneakyThrow(e);
}
}
public static JCTree getExtendsClause(JCClassDecl decl) {
return (JCTree) Permit.invokeSneaky(getExtendsClause, decl);
}
static RuntimeException sneakyThrow(Throwable t) {
if (t == null) throw new NullPointerException("t");
PrettyPrinter.<RuntimeException>sneakyThrow0(t);
return null;
}
@SuppressWarnings("unchecked")
private static <T extends Throwable> void sneakyThrow0(Throwable t) throws T {
throw (T)t;
}
private static final Map<Class<?>, Map<String, Field>> reflectionCache = new HashMap<Class<?>,
Map<String, Field>>();
@SuppressWarnings("unchecked")
private <T> T readObject(JCTree tree, String fieldName, T defaultValue) {
Class<?> tClass = tree.getClass();
Map<String, Field> c = reflectionCache.get(tClass);
if (c == null) reflectionCache.put(tClass, c = new HashMap<String, Field>());
Field f = c.get(fieldName);
if (f == null) {
try {
f = Permit.getField(tClass, fieldName);
```

```
} catch (Exception e) {
return defaultValue;
}
c.put(fieldName, f);
}
try {
return (T) f.get(tree);
} catch (Exception e) {
return defaultValue;
}
}
public void visitTypeBoundKind(TypeBoundKind tree) {
print(String.valueOf(tree.kind));
}
@Override public void visitTree(JCTree tree) {
String className = tree.getClass().getName();
if (className.endsWith("$JCTypeUnion")) {
List<JCExpression> types = readObject(tree, "alternatives", List.<JCExpression>nil());
print(types, " | ");
} else if (className.endsWith("$JCTypeIntersection")) {
print(readObject(tree, "bounds", List.<JCExpression>nil()), " & ");
} else if (className.endsWith("$JCMemberReference")) {
printMemberReference0(tree);
} else if (className.endsWith("$JCLambda")) {
printLambda0(tree);
} else if (className.endsWith("$JCAnnotatedType")) {
printAnnotatedType0(tree);
} else if (className.endsWith("$JCPackageDecl")) {
// Starting with JDK9, this is inside the import list, but we've already printed it. Just ignore it.
} else if (className.endsWith("$JCSwitchExpression")) { // Introduced as preview feature in JDK12
printSwitchExpression(tree);
} else if (className.endsWith("$JCYield")) { // Introduced as preview feature in JDK13, part of switch
expressions.
printYieldExpression(tree);
} else if (className.endsWith("$JCBindingPattern")) { // Introduced as preview in JDK14
printBindingPattern(tree);
} else if (className.endsWith("$JCDefaultCaseLabel")) { // Introduced in JDK17
printDefaultCase(tree);
} else if (className.endsWith("$JCGuardPattern")) { // Introduced in JDK17
printGuardPattern(tree);
} else if (className.endsWith("$JCParenthesizedPattern")) { // Introduced in JDK17
printParenthesizedPattern(tree);
} else if (className.endsWith("$JCConstantCaseLabel")) { // Introduced in JDK19
printConstantCaseLabel(tree);
} else if (className.endsWith("$JCPatternCaseLabel")) { // Introduced in JDK19
printPatternCaseLabel(tree);
} else if (className.endsWith("$JCRecordPattern")) { // Introduced in JDK19
printRecordPattern(tree);
} else if (className.endsWith("$JCAnyPattern")) { // Introduced in JDK22
print("_");
} else {
throw new AssertionError("Unhandled tree type: " + tree.getClass() + ": " + tree);
}
```

```
}
private boolean isCaseRuleStyle(JCCase tree) {
if (tree == null) return false;
Enum<?> caseKind = readObject(tree, "caseKind", null); // JDK 12+
return caseKind != null && caseKind.name().equalsIgnoreCase("RULE");
}
private boolean jcAnnotatedTypeInit = false;
private Class<?> jcAnnotatedTypeClass = null;
private boolean isJcAnnotatedType(Object o) {
if (o == null) return false;
if (jcAnnotatedTypeInit) return jcAnnotatedTypeClass == o.getClass();
Class<?> c = o.getClass();
if (c.getName().endsWith("$JCAnnotatedType")) {
jcAnnotatedTypeClass = c;
jcAnnotatedTypeInit = true;
return true;
}
return false;
}
private void printMemberReference0(JCTree tree) {
print(readObject(tree, "expr", (JCExpression) null));
print("::");
List<JCExpression> typeArgs = readObject(tree, "typeargs", List.<JCExpression>nil());
if (typeArgs != null && !typeArgs.isEmpty()) {
print("<");
print(typeArgs, ", ");
print(">");
}
print(readObject(tree, "mode", new Object()).toString().equals("INVOKE") ? readObject(tree, "name",
(Name) null) : "new");
}
private void printLambda0(JCTree tree) {
List<JCVariableDecl> params = readObject(tree, "params", List.<JCVariableDecl>nil());
boolean explicit = true;
int paramLength = params.size();
try {
explicit = readObject(tree, "paramKind", new Object()).toString().equals("EXPLICIT");
} catch (Exception e) {}
boolean useParens = paramLength != 1 || explicit;
if (useParens) print("(");
if (explicit) {
boolean first = true;
for (JCVariableDecl vd : params) {
if (!first) print(", ");
first = false;
printVarDefInline(vd);
}
} else {
String sep = "";
for (JCVariableDecl param : params) {
print(sep);
print(param.name);
sep = ", ";
```

```
}
}
if (useParens) print(")");
print(" -> ");
JCTree body = readObject(tree, "body", (JCTree) null);
if (body instanceof JCBlock) {
println("{");
indent++;
print(((JCBlock) body).stats, "");
indent--;
aPrint("}");
} else {
print(body);
}
}
private void printAnnotatedType0(JCTree tree) {
JCTree underlyingType = readObject(tree, "underlyingType", (JCTree) null);
if (underlyingType instanceof JCFieldAccess) {
print(((JCFieldAccess) underlyingType).selected);
print(".");
print(readObject(tree, "annotations", List.<JCExpression>nil()), " ");
print(" ");
print(((JCFieldAccess) underlyingType).name);
} else if (underlyingType instanceof JCArrayTypeTree) {
printTypeArray0(tree);
} else {
print(readObject(tree, "annotations", List.<JCExpression>nil()), " ");
print(" ");
print(underlyingType);
}
}
private void printTypeArray0(JCTree tree) {
JCTree inner = tree;
int dimCount = 0;
while (true) {
if (inner instanceof JCArrayTypeTree) {
inner = ((JCArrayTypeTree) inner).elemtype;
dimCount++;
continue;
} else if (isJcAnnotatedType(inner)) {
JCTree underlyingType = readObject(inner, "underlyingType", (JCTree) null);
if (underlyingType instanceof JCArrayTypeTree) {
inner = ((JCArrayTypeTree) underlyingType).elemtype;
dimCount++;
continue;
}
}
break;
}
print(inner);
inner = tree;
while (true) {
if (inner instanceof JCArrayTypeTree) {
```

```
dimCount--;
print((dimCount == 0 && innermostArrayBracketsAreVarargs) ? "..." : "[]");
inner = ((JCArrayTypeTree) inner).elemtype;
continue;
} else if (isJcAnnotatedType(inner)) {
JCTree underlyingType = readObject(inner, "underlyingType", (JCTree) null);
if (underlyingType instanceof JCArrayTypeTree) {
dimCount--;
print(" ");
print(readObject(inner, "annotations", List.<JCExpression>nil()), " ");
print(" ");
print((dimCount == 0 && innermostArrayBracketsAreVarargs) ? "..." : "[]");
inner = ((JCArrayTypeTree) underlyingType).elemtype;
continue;
}
}
break;
}
}
}
```

lombok/delombok/UnicodeEscapeWriter.java


```
/*
* Copyright (C) 2009 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.delombok;
import java.io.IOException;
import java.io.Writer;
import java.nio.charset.Charset;
import java.nio.charset.CharsetEncoder;
public class UnicodeEscapeWriter extends Writer {
private final Writer writer;
```

```
private CharsetEncoder encoder;
public UnicodeEscapeWriter(Writer writer, Charset charset) {
this.writer = writer;
encoder = charset.newEncoder();
}
@Override
public void close() throws IOException {
writer.close();
}
@Override
public void flush() throws IOException {
writer.flush();
}
@Override
public final void write(char[] cbuf, int off, int len) throws IOException {
int start = off;
int index = start;
int end = off + len;
while (index < end) {
if (!encoder.canEncode(cbuf[index])) {
writer.write(cbuf, start, index - start);
writeUnicodeEscape(cbuf[index]);
start = index + 1;
}
index++;
}
if (start < end) {
writer.write(cbuf, start, end - start);
}
}
protected void writeUnicodeEscape(char c) throws IOException {
writer.write(String.format("\\u%04x", (int) c));
}
}
```

lombok/delombok/ant/DelombokTask.java

```
/*
* Copyright (C) 2009-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.delombok.ant;
import java.io.File;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.List;
import org.apache.tools.ant.BuildException;
import org.apache.tools.ant.Location;
import org.apache.tools.ant.Task;
import org.apache.tools.ant.types.FileSet;
import org.apache.tools.ant.types.Path;
import org.apache.tools.ant.types.Reference;
@SuppressWarnings("unused") // we use reflection to transfer fields.
class Tasks {
public static class Format {
private String value;
@Override public int hashCode() {
final int prime = 31;
int result = 1;
result = prime * result + ((value == null) ? 0 : value.hashCode());
return result;
}
@Override public boolean equals(Object obj) {
if (this == obj) return true;
if (obj == null) return false;
if (getClass() != obj.getClass()) return false;
Format other = (Format) obj;
if (value == null) {
if (other.value != null) return false;
} else if (!value.equals(other.value)) return false;
return true;
}
@Override public String toString() {
return "FormatOption [value=" + value + "]";
}
public String getValue() {
return value;
}
public void setValue(String value) {
this.value = value;
}
}
public static class Delombok extends Task {
private File fromDir, toDir;
```

```
private Path classpath;
private Path sourcepath;
private Path modulepath;
private boolean verbose;
private String encoding;
private Path path;
private List<Format> formatOptions = new ArrayList<Format>();
public void setClasspath(Path classpath) {
if (this.classpath == null) {
this.classpath = classpath;
} else {
this.classpath.append(classpath);
}
}
public Path createClasspath() {
if (classpath == null) classpath = new Path(getProject());
return classpath.createPath();
}
public void setClasspathRef(Reference r) {
createClasspath().setRefid(r);
}
public void setSourcepath(Path sourcepath) {
if (this.sourcepath == null) {
this.sourcepath = sourcepath;
} else {
this.sourcepath.append(sourcepath);
}
}
public Path createSourcepath() {
if (sourcepath == null) sourcepath = new Path(getProject());
return sourcepath.createPath();
}
public void setSourcepathRef(Reference r) {
createSourcepath().setRefid(r);
}
public void setModulepath(Path modulepath) {
if (this.modulepath == null) {
this.modulepath = modulepath;
} else {
this.modulepath.append(modulepath);
}
}
public Path createModulepath() {
if (modulepath == null) modulepath = new Path(getProject());
return modulepath.createPath();
}
public void setModulepathRef(Reference r) {
createModulepath().setRefid(r);
}
public void setFrom(File dir) {
this.fromDir = dir;
}
public void setTo(File dir) {
```

```
this.toDir = dir;
}
public void setVerbose(boolean verbose) {
this.verbose = verbose;
}
public void setEncoding(String encoding) {
this.encoding = encoding;
}
public void addFileset(FileSet set) {
if (path == null) path = new Path(getProject());
path.add(set);
}
public void addFormat(Format format) {
formatOptions.add(format);
}
private static ClassLoader shadowLoader;
public static Class<?> shadowLoadClass(String name) {
try {
if (shadowLoader == null) {
try {
Class.forName("lombok.core.LombokNode");
// If we get here, then lombok is already available.
shadowLoader = Delombok.class.getClassLoader();
} catch (ClassNotFoundException e) {
// If we get here, it isn't, and we should use the shadowloader.
Class<?> launcherMain = Class.forName("lombok.launch.Main");
Method m = launcherMain.getDeclaredMethod("getShadowClassLoader");
m.setAccessible(true);
shadowLoader = (ClassLoader) m.invoke(null);
}
}
return Class.forName(name, true, shadowLoader);
} catch (Throwable t) {
if (t instanceof InvocationTargetException) t = t.getCause();
if (t instanceof RuntimeException) throw (RuntimeException) t;
if (t instanceof Error) throw (Error) t;
throw new RuntimeException(t);
}
}
@Override
public void execute() throws BuildException {
Location loc = getLocation();
try {
Object instance =
shadowLoadClass("lombok.delombok.ant.DelombokTaskImpl").getConstructor().newInstance();
for (Field selfField : getClass().getDeclaredFields()) {
if (selfField.isSynthetic() || Modifier.isStatic(selfField.getModifiers())) continue;
Field otherField = instance.getClass().getDeclaredField(selfField.getName());
otherField.setAccessible(true);
if (selfField.getName().equals("formatOptions")) {
List<String> rep = new ArrayList<String>();
for (Format f : formatOptions) {
if (f.getValue() == null) throw new BuildException("'value' property required for <format>");
```

```
rep.add(f.getValue());
}
otherField.set(instance, rep);
} else {
otherField.set(instance, selfField.get(this));
}
}
Method m = instance.getClass().getMethod("execute", Location.class);
m.invoke(instance, loc);
} catch (Throwable t) {
if (t instanceof InvocationTargetException) t = t.getCause();
if (t instanceof RuntimeException) throw (RuntimeException) t;
if (t instanceof Error) throw (Error) t;
throw new RuntimeException(t);
}
}
}
}
```

lombok/delombok/ant/DelombokTaskImpl.java


```
/*
* Copyright (C) 2009-2018 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok.delombok.ant;
import java.io.File;
import java.io.IOException;
import java.nio.charset.UnsupportedCharsetException;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import lombok.delombok.Delombok;
import lombok.delombok.Delombok.InvalidFormatOptionException;
```

```
import org.apache.tools.ant.BuildException;
import org.apache.tools.ant.Location;
import org.apache.tools.ant.types.Path;
import org.apache.tools.ant.types.resources.FileResource;
public class DelombokTaskImpl {
private File fromDir, toDir;
private Path classpath;
private Path sourcepath;
private Path modulepath;
private boolean verbose;
private String encoding;
private Path path;
private List<String> formatOptions = new ArrayList<String>();
public void execute(Location location) throws BuildException {
if (fromDir == null && path == null) throw new BuildException("Either 'from' attribute, or nested <fileset>
tags are required.");
if (fromDir != null && path != null) throw new BuildException("You can't specify both 'from' attribute and
nested filesets. You need one or the other.");
if (toDir == null) throw new BuildException("The to attribute is required.");
Delombok delombok = new Delombok();
if (verbose) delombok.setVerbose(true);
try {
if (encoding != null) delombok.setCharset(encoding);
} catch (UnsupportedCharsetException e) {
throw new BuildException("Unknown charset: " + encoding, location);
}
if (classpath != null) delombok.setClasspath(classpath.toString());
if (sourcepath != null) delombok.setSourcepath(sourcepath.toString());
if (modulepath != null) delombok.setModulepath(modulepath.toString());
try {
delombok.setFormatPreferences(Delombok.formatOptionsToMap(formatOptions));
} catch (InvalidFormatOptionException e) {
throw new BuildException(e.getMessage() + " Run java -jar lombok.jar --format-help for detailed format
help.");
}
delombok.setOutput(toDir);
try {
if (fromDir != null) delombok.addDirectory(fromDir);
else {
Iterator<?> it = path.iterator();
while (it.hasNext()) {
FileResource fileResource = (FileResource) it.next();
File baseDir = fileResource.getBaseDir();
if (baseDir == null) {
File file = fileResource.getFile();
delombok.addFile(file.getParentFile(), file.getName());
} else {
delombok.addFile(baseDir, fileResource.getName());
}
}
}
delombok.delombok();
} catch (IOException e) {
```

```
throw new BuildException("I/O problem during delombok", e, location);
}
}
}
```

lombok/transform/TestLombokFilesIdempotent.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.transform;
import java.io.File;
import lombok.DirectoryRunner;
import org.junit.runner.RunWith;
@RunWith(DirectoryRunner.class)
public class TestLombokFilesIdempotent extends DirectoryRunner.TestParams {
@Override
public File getBeforeDirectory() {
return getAfterDirectory();
}
@Override
public File getAfterDirectory() {
return new File("test/transform/resource/after-delombok");
}
@Override
public File getMessagesDirectory() {
return new File("test/transform/resource/messages-idempotent");
}
@Override
public DirectoryRunner.Compiler getCompiler() {
return DirectoryRunner.Compiler.DELOMBOK;
}
@Override
```

```
public boolean printErrors() {
return true;
}
@Override
public boolean expectChanges() {
return false;
}
@Override public String testNamePrefix() {
return "idempotent-";
}
}
```

lombok/transform/TestSourceFiles.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.transform;
import java.io.File;
import lombok.DirectoryRunner;
import org.junit.runner.RunWith;
@RunWith(DirectoryRunner.class)
public class TestSourceFiles extends DirectoryRunner.TestParams {
@Override
public DirectoryRunner.Compiler getCompiler() {
return DirectoryRunner.Compiler.DELOMBOK;
}
@Override
public boolean printErrors() {
return true;
}
@Override
public File getBeforeDirectory() {
```

```
return new File("test/pretty/resource/before");
}
@Override
public File getAfterDirectory() {
return new File("test/pretty/resource/after");
}
@Override
public File getMessagesDirectory() {
return new File("test/pretty/resource/messages");
}
@Override
public boolean expectChanges() {
return false;
}
@Override public String testNamePrefix() {
return "prettyprint-";
}
}
```

lombok/transform/TestWithDelombok.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.transform;
import java.io.File;
import lombok.DirectoryRunner;
import org.junit.runner.RunWith;
@RunWith(DirectoryRunner.class)
public class TestWithDelombok extends DirectoryRunner.TestParams {
@Override
public DirectoryRunner.Compiler getCompiler() {
return DirectoryRunner.Compiler.DELOMBOK;
```

```
}
@Override
public boolean printErrors() {
return true;
}
@Override
public File getBeforeDirectory() {
return new File("test/transform/resource/before");
}
@Override
public File getAfterDirectory() {
return new File("test/transform/resource/after-delombok");
}
@Override
public File getMessagesDirectory() {
return new File("test/transform/resource/messages-delombok");
}
@Override
public boolean expectChanges() {
return true;
}
@Override public String testNamePrefix() {
return "javac-";
}
}
```

lombok/transform/TestWithEcj.java

```
/*
* Copyright (C) 2009-2021 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.transform;
import java.io.File;
```

```
import lombok.DirectoryRunner;
import org.junit.runner.RunWith;
// You need to add the following vm-parameter to run the test:
// -javaagent:${project_loc:lombok}/dist/lombok.jar
// When running inside eclipse's junit tester, you don't actually need to run 'ant dist' after updating code, though.
@RunWith(DirectoryRunner.class)
public class TestWithEcj extends DirectoryRunner.TestParams {
@Override
public DirectoryRunner.Compiler getCompiler() {
return DirectoryRunner.Compiler.ECJ;
}
@Override
public boolean printErrors() {
return true;
}
@Override
public File getBeforeDirectory() {
return new File("test/transform/resource/before");
}
@Override
public File getAfterDirectory() {
return new File("test/transform/resource/after-ecj");
}
@Override
public File getMessagesDirectory() {
return new File("test/transform/resource/messages-ecj");
}
@Override
public boolean expectChanges() {
return true;
}
@Override public String testNamePrefix() {
return "ecj-";
}
}
```

lombok/AbstractRunTests.java


```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
```

```
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import static org.junit.Assert.*;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.URI;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Iterator;
import java.util.LinkedHashSet;
import java.util.List;
import org.junit.Assert;
import lombok.DirectoryRunner.FileTester;
import lombok.core.LombokConfiguration;
import lombok.core.LombokImmutableList;
import lombok.core.configuration.ConfigurationKeysLoader;
import lombok.core.configuration.ConfigurationResolver;
import lombok.core.configuration.ConfigurationResolverFactory;
import lombok.javac.CapturingDiagnosticListener.CompilerMessage;
import lombok.transform.TestLombokFilesIdempotent;
import lombok.transform.TestSourceFiles;
public abstract class AbstractRunTests {
public final FileTester createTester(final DirectoryRunner.TestParams params, final File file, String
platform, int version, final boolean copyToSource) throws IOException {
ConfigurationKeysLoader.LoaderLoader.loadAllConfigurationKeys();
AssertionError directiveFailure = null;
LombokTestSource sourceDirectives = null;
try {
sourceDirectives = LombokTestSource.readDirectives(file);
if (sourceDirectives.isIgnore()) return null;
if (!sourceDirectives.versionWithinLimit(version)) return null;
if (!sourceDirectives.runOnPlatform(platform)) return null;
} catch (AssertionError ae) {
directiveFailure = ae;
}
String fileName = file.getName();
final LombokTestSource expected = LombokTestSource.read(params.getAfterDirectory(),
params.getMessagesDirectory(), fileName);
if (expected.isIgnore()) return null;
```

```
if (!expected.versionWithinLimit(params.getVersion())) return null;
if (!expected.versionWithinLimit(version)) return null;
if (expected.isSkipIdempotent() && params instanceof TestLombokFilesIdempotent) return null;
final LombokTestSource sourceDirectives_ = sourceDirectives;
final AssertionError directiveFailure_ = directiveFailure;
return new FileTester() {
@Override public void runTest() throws Throwable {
if (directiveFailure_ != null) throw directiveFailure_;
LombokConfiguration.overrideConfigurationResolverFactory(new ConfigurationResolverFactory() {
@Override public ConfigurationResolver createResolver(URI sourceLocation) {
return sourceDirectives_.getConfiguration();
}
});
TestParameters testParameters = new TestParameters();
testParameters.setEncoding(sourceDirectives_.getSpecifiedEncoding());
testParameters.setFormatPreferences(sourceDirectives_.getFormatPreferences());
testParameters.setMinVersion(sourceDirectives_.minVersion());
testParameters.setVerifyDiet(sourceDirectives_.isVerifyDiet());
boolean checkPositions = !(params instanceof TestLombokFilesIdempotent || params instanceof
TestSourceFiles) && !sourceDirectives_.isSkipCompareContent();
testParameters.setCheckPositions(checkPositions);
String javaVersionString = System.getProperty("compiler.compliance.level");
if (javaVersionString != null) {
long version = Long.parseLong(javaVersionString);
testParameters.setSourceVersion(version);
testParameters.setTargetVersion(version);
}
TransformationResult result = transformCode(file, testParameters);
boolean changed = result.isChanged();
boolean forceUnchanged = sourceDirectives_.forceUnchanged() ||
sourceDirectives_.isSkipCompareContent();
if (params.expectChanges() && !forceUnchanged && !changed) result.addMessage(new
CompilerMessage(-1, -1, true, "not flagged modified"));
if (!params.expectChanges() && changed) result.addMessage(new CompilerMessage(-1, -1, true,
"unexpected modification"));
compare(file.getName(), expected, result, params.printErrors(),
sourceDirectives_.isSkipCompareContent() || expected.isSkipCompareContent(), copyToSource);
}
};
}
protected abstract TransformationResult transformCode(File file, TestParameters parameters) throws
Throwable;
protected String readFile(File file) throws IOException {
BufferedReader reader;
try {
reader = new BufferedReader(new FileReader(file));
} catch (FileNotFoundException e) {
return null;
}
StringBuilder result = new StringBuilder();
String line;
while ((line = reader.readLine()) != null) {
result.append(line);
```

```
result.append("\n");
}
reader.close();
return result.toString();
}
private static void dumpToFile(File file, String content) throws IOException {
List<String> directives = new ArrayList<String>();
if (file.exists()) {
BufferedReader br = new BufferedReader(new InputStreamReader(new FileInputStream(file),
"UTF-8"));
try {
while (true) {
String line = br.readLine();
if (line == null) break;
if (line.startsWith("//")) directives.add(line);
else if (!line.isEmpty()) break;
}
} finally {
br.close();
}
}
FileOutputStream fos = new FileOutputStream(file);
try {
for (String directive : directives) fos.write((directive + "\n").getBytes("UTF-8"));
boolean atStart = true;
for (String line : content.split("(\\r?\\n)")) {
if (line.isEmpty()) continue;
if (atStart && line.startsWith("//")) continue;
atStart = false;
fos.write((line + "\n").getBytes("UTF-8"));
}
} finally {
fos.close();
}
}
private static void dumpToFile(File file, Collection<CompilerMessage> content) throws IOException {
FileOutputStream fos = new FileOutputStream(file);
System.err.println(">>> DUMPING ERR MSG: " + file.getAbsolutePath());
try {
for (CompilerMessage message : content) {
fos.write(CompilerMessageMatcher.asCompilerMessageMatcher(message).toString().getBytes("UTF-8
"));
fos.write('\n');
}
} finally {
fos.close();
}
}
private void compare(String name, LombokTestSource expected, TransformationResult result, boolean
printErrors, boolean skipCompareContent, boolean copyToSource) throws Throwable {
String actualFile = result.getOutput();
LinkedHashSet<CompilerMessage> actualMessages = result.getMessages();
if (!skipCompareContent) try {
```

```
compareContent(name, expected.getContent(), actualFile);
} catch (Throwable e) {
if (copyToSource) {
dumpToFile(expected.getSourceFile(), actualFile);
System.out.println("UPDATED: " + expected.getSourceFile());
} else {
if (printErrors) {
System.out.println("***** " + name + " *****");
System.out.println(e.getMessage());
System.out.println("**** Expected ******");
System.out.println(expected.getContent());
System.out.println("**** Actual ******");
System.out.println(actualFile);
if (actualMessages != null && !actualMessages.isEmpty()) {
System.out.println("**** Actual Errors *****");
for (CompilerMessage actualMessage : actualMessages) {
System.out.println(actualMessage);
}
}
System.out.println("********************");
}
throw e;
}
}
try {
compareMessages(name, expected.getMessages(), actualMessages);
} catch (Throwable e) {
if (copyToSource) {
if (actualMessages.isEmpty()) {
if (expected.getMessagesFile().exists()) {
expected.getMessagesFile().delete();
System.out.println("DELETED: " + expected.getMessagesFile());
}
} else {
dumpToFile(expected.getMessagesFile(), actualMessages);
System.out.println("UPDATED: " + expected.getMessagesFile());
}
} else {
if (printErrors) {
System.out.println("***** " + name + " *****");
System.out.println(e.getMessage());
System.out.println("**** Expected ******");
for (CompilerMessageMatcher expectedMessage : expected.getMessages()) {
System.out.println(expectedMessage);
}
System.out.println("**** Actual ******");
for (CompilerMessage actualMessage : actualMessages) {
System.out.println(actualMessage);
}
System.out.println("**** Actual File ******");
System.out.println(lineNumber(actualFile));
System.out.println("********************");
}
```

```
throw e;
}
}
}
private CharSequence lineNumber(String content) {
StringBuilder out = new StringBuilder();
int pos = 0;
int ln = 1;
while (true) {
out.append(String.format("%4d ", ln));
int idx = content.indexOf('\n', pos);
if (idx == -1) {
return out.append(content.substring(pos));
}
out.append(content.substring(pos, idx + 1));
ln++;
pos = idx + 1;
}
}
@SuppressWarnings("null") /* eclipse bug workaround; it falsely thinks stuffAc will always be null. */
private static void compareMessages(String name, LombokImmutableList<CompilerMessageMatcher>
expected, LinkedHashSet<CompilerMessage> actual) {
Iterator<CompilerMessageMatcher> expectedIterator = expected.iterator();
Iterator<CompilerMessage> actualIterator = actual.iterator();
CompilerMessage stuffAc = null;
while (true) {
boolean exHasNext = expectedIterator.hasNext();
boolean acHasNext = stuffAc != null || actualIterator.hasNext();
if (!exHasNext && !acHasNext) break;
if (exHasNext && acHasNext) {
CompilerMessageMatcher cmm = expectedIterator.next();
CompilerMessage cm = stuffAc == null ? actualIterator.next() : stuffAc;
if (cmm.matches(cm)) continue;
if (cmm.isOptional()) stuffAc = cm;
fail(String.format("[%s] Expected message '%s' but got message '%s'", name, cmm, cm));
throw new AssertionError("fail should have aborted already.");
}
while (expectedIterator.hasNext()) {
CompilerMessageMatcher next = expectedIterator.next();
if (next.isOptional()) continue;
fail(String.format("[%s] Expected message '%s' but ran out of actual messages", name, next));
}
if (acHasNext) fail(String.format("[%s] Unexpected message: %s", name, actualIterator.next()));
break;
}
}
private static void compareContent(String name, String expectedFile, String actualFile) {
String[] expectedLines = expectedFile.split("(\\r?\\n)");
String[] actualLines = actualFile.split("(\\r?\\n)");
for (int i = 0; i < expectedLines.length; i++) {
if (expectedLines[i].isEmpty() || expectedLines[i].startsWith("//")) expectedLines[i] = "";
else break;
}
```

```
for (int i = 0; i < actualLines.length; i++) {
if (actualLines[i].isEmpty() || actualLines[i].startsWith("//")) actualLines[i] = "";
else break;
}
expectedLines = removeBlanks(expectedLines);
actualLines = removeBlanks(actualLines);
int size = Math.min(expectedLines.length, actualLines.length);
if (size == 0 && expectedLines.length + actualLines.length > 0) {
Assert.fail("Missing / empty expected file: " + name);
}
for (int i = 0; i < size; i++) {
String expected = trimRight(expectedLines[i]);
String actual = trimRight(actualLines[i]);
assertEquals(String.format("Difference in %s on line %d", name, i + 1), expected, actual);
}
if (expectedLines.length > actualLines.length) {
fail(String.format("Missing line %d in generated %s: %s", size + 1, name, expectedLines[size]));
}
if (expectedLines.length < actualLines.length) {
fail(String.format("Extra line %d in generated %s: %s", size + 1, name, actualLines[size]));
}
}
private static String trimRight(String in) {
int endIdx = in.length() - 1;
while (endIdx > -1 && Character.isWhitespace(in.charAt(endIdx))) {
endIdx--;
}
return in.substring(0, endIdx + 1);
}
private static String[] removeBlanks(String[] in) {
List<String> out = new ArrayList<String>();
for (String s : in) {
if (!s.trim().isEmpty()) out.add(s);
}
return out.toArray(new String[0]);
}
}
```

lombok/CompilerMessageMatcher.java

```
/*
* Copyright (C) 2012-2013 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import lombok.javac.CapturingDiagnosticListener.CompilerMessage;
public class CompilerMessageMatcher {
/** Line Number (starting at 1) */
private final List<Integer> lineNumbers = new ArrayList<Integer>();
private final List<List<String>> messages = new ArrayList<List<String>>();
private boolean optional;
private CompilerMessageMatcher() {}
public boolean isOptional() {
return optional;
}
public static CompilerMessageMatcher asCompilerMessageMatcher(CompilerMessage message) {
CompilerMessageMatcher cmm = new CompilerMessageMatcher();
cmm.lineNumbers.add((int) message.getLine());
cmm.messages.add(Arrays.asList(message.getMessage().split("\\s+")));
return cmm;
}
@Override public String toString() {
StringBuilder out = new StringBuilder();
for (int i = 0; i < lineNumbers.size(); i++) {
out.append(lineNumbers.get(i)).append(" ");
for (String part : messages.get(i)) out.append(part).append(" ");
if (out.length() > 0) out.setLength(out.length() - 1);
out.append(" |||| ");
}
if (out.length() > 0) out.setLength(out.length() - 6);
return out.toString();
}
public boolean matches(CompilerMessage message) {
outer:
for (int i = 0; i < lineNumbers.size(); i++) {
//Allow an off-by-1 in line numbers; when running tests that sometimes happens for as yet unknown
reasons.
if (message.getLine() != lineNumbers.get(i) && message.getLine() -1 != lineNumbers.get(i)) continue;
```

```
for (String token : messages.get(i)) {
if (!message.getMessage().contains(token)) continue outer;
}
return true;
}
return false;
}
public static List<CompilerMessageMatcher> readAll(InputStream rawIn) throws IOException {
BufferedReader in = new BufferedReader(new InputStreamReader(rawIn, "UTF-8"));
List<CompilerMessageMatcher> out = new ArrayList<CompilerMessageMatcher>();
for (String line = in.readLine(); line != null; line = in.readLine()) {
CompilerMessageMatcher cmm = read(line);
if (cmm != null) out.add(cmm);
}
return out;
}
private static final Pattern PATTERN = Pattern.compile("^(-?\\d+) (.*)$");
private static CompilerMessageMatcher read(String line) {
line = line.trim();
if (line.isEmpty()) return null;
boolean optional = false;
if (line.startsWith("OPTIONAL ")) {
line = line.substring(9);
optional = true;
}
String[] parts = line.split("\\s*\\|\\|\\|\\|\\|\\s*");
CompilerMessageMatcher cmm = new CompilerMessageMatcher();
cmm.optional = optional;
for (String part : parts) {
Matcher m = PATTERN.matcher(part);
if (!m.matches()) throw new IllegalArgumentException("Typo in test file: " + line);
cmm.lineNumbers.add(Integer.parseInt(m.group(1)));
cmm.messages.add(Arrays.asList(m.group(2).split("\\s+")));
}
return cmm;
}
}
```

lombok/DirectoryRunner.java

```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
```

```
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.io.File;
import java.io.FileFilter;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import org.junit.runner.Description;
import org.junit.runner.Runner;
import org.junit.runner.notification.Failure;
import org.junit.runner.notification.RunNotifier;
import lombok.eclipse.Eclipse;
import lombok.javac.Javac;
public class DirectoryRunner extends Runner {
/** Add 1 or more file names to reduce the testset to just the named file(s). No files = test it all. */
private static final List<String> DEBUG_FOCUS_ON_FILE = Arrays.asList(
);
/*
* If this is set to true, you are decreeing that all tests pass, and that any test that dares to say it does not, means the test is wrong and not lombok: The framework will <strong>update</strong> the 'target' so that it now passes.
* This is, naturally, an exceedingly dangerous tool. The general way to use it is: First ensure all tests pass, then, do some minor refactor that massively impacts the tests, then, enable this, run the tests, then turn this off again,
* then, thoroughly review your commit diff.
*/
private static final boolean ITS_ALL_GOOD = false;
public enum Compiler {
DELOMBOK {
@Override public int getVersion() {
return Javac.getJavaCompilerVersion();
}
},
JAVAC {
@Override public int getVersion() {
return DELOMBOK.getVersion();
}
},
ECJ {
@Override public int getVersion() {
String javaVersionString = System.getProperty("compiler.compliance.level");
```

```
return javaVersionString != null ? Integer.parseInt(javaVersionString) :
Eclipse.getEcjCompilerVersion();
}
};
public abstract int getVersion();
}
public static abstract class TestParams {
public abstract Compiler getCompiler();
public abstract boolean printErrors();
public abstract File getBeforeDirectory();
public abstract File getAfterDirectory();
public abstract File getMessagesDirectory();
/** Version of the JDK dialect that the compiler can understand; for example, if you return '7', you
should know what try-with-resources is. */
public int getVersion() {
return getCompiler().getVersion();
}
public boolean accept(File file) {
return true;
}
public abstract boolean expectChanges();
public String testNamePrefix() {
return "";
}
}
private static final FileFilter JAVA_FILE_FILTER = new FileFilter() {
@Override public boolean accept(File file) {
if (!file.isFile() || !file.getName().endsWith(".java")) return false;
boolean positiveFilter = false;
for (String dfof : DEBUG_FOCUS_ON_FILE) {
if (dfof.isEmpty()) continue;
if (!dfof.endsWith(".java")) dfof = dfof + ".java";
boolean invert = dfof.startsWith("!");
if (invert) dfof = dfof.substring(1);
positiveFilter = positiveFilter || !invert;
int starIdx = dfof.indexOf('*');
if (starIdx == -1) {
if (file.getName().equals(dfof)) return !invert;
} else {
if (file.getName().startsWith(dfof.substring(0, starIdx)) &&
file.getName().endsWith(dfof.substring(starIdx + 1))) return !invert;
}
}
return !positiveFilter;
}
};
private final Description description;
private final Map<String, Description> tests = new TreeMap<String, Description>();
private final Throwable failure;
private final TestParams params;
public DirectoryRunner(Class<?> testClass) throws Exception {
description = Description.createSuiteDescription(testClass);
this.params = (TestParams) testClass.getConstructor().newInstance();
```

```
Throwable error = null;
try {
addTests(testClass);
} catch (Throwable t) {
error = t;
}
this.failure = error;
}
private void addTests(Class<?> testClass) throws Exception {
for (File file : params.getBeforeDirectory().listFiles(JAVA_FILE_FILTER)) {
if (!params.accept(file)) continue;
Description testDescription = Description.createTestDescription(testClass,
this.params.testNamePrefix() + file.getName());
description.addChild(testDescription);
tests.put(file.getName(), testDescription);
}
}
@Override
public Description getDescription() {
return description;
}
@Override
public void run(RunNotifier notifier) {
if (failure != null) {
reportInitializationFailure(notifier, description, failure);
return;
}
for (Map.Entry<String, Description> entry : tests.entrySet()) {
Description testDescription = entry.getValue();
FileTester tester;
try {
tester = createTester(entry.getKey());
} catch (IOException e) {
reportInitializationFailure(notifier, testDescription, e);
continue;
}
if (tester == null) {
notifier.fireTestIgnored(testDescription);
continue;
}
notifier.fireTestStarted(testDescription);
try {
tester.runTest();
} catch (Throwable t) {
notifier.fireTestFailure(new Failure(testDescription, t));
}
notifier.fireTestFinished(testDescription);
}
}
private void reportInitializationFailure(RunNotifier notifier, Description description, Throwable
throwable) {
notifier.fireTestStarted(description);
notifier.fireTestFailure(new Failure(description, throwable));
```

```
notifier.fireTestFinished(description);
}
private FileTester createTester(String fileName) throws IOException {
File file = new File(params.getBeforeDirectory(), fileName);
switch (params.getCompiler()) {
case DELOMBOK:
return new RunTestsViaDelombok().createTester(params, file, "javac", params.getVersion(),
ITS_ALL_GOOD);
case ECJ:
String platform = RunTestsViaEcj.eclipseAvailable() ? "eclipse" : "ecj";
return new RunTestsViaEcj().createTester(params, file, platform, params.getVersion(),
ITS_ALL_GOOD);
default:
case JAVAC:
throw new UnsupportedOperationException();
}
}
public interface FileTester {
void runTest() throws Throwable;
}
}
```

lombok/LombokTestSource.java

```
/*
* Copyright (C) 2014-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
```

```
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import org.junit.Assert;
import lombok.core.LombokImmutableList;
import lombok.core.configuration.BubblingConfigurationResolver;
import lombok.core.configuration.ConfigurationFile;
import lombok.core.configuration.ConfigurationFileToSource;
import lombok.core.configuration.ConfigurationParser;
import lombok.core.configuration.ConfigurationProblemReporter;
import lombok.core.configuration.ConfigurationResolver;
import lombok.core.configuration.ConfigurationSource;
import lombok.core.configuration.SingleConfigurationSource;
public class LombokTestSource {
private final File sourceFile;
private final File messagesFile;
private final String content;
private final LombokImmutableList<CompilerMessageMatcher> messages;
private final Map<String, String> formatPreferences;
private final boolean ignore;
private final boolean skipCompareContent;
private final boolean skipIdempotent;
private final boolean unchanged;
private final boolean verifyDiet;
private final int versionLowerLimit, versionUpperLimit;
private final ConfigurationResolver configuration;
private final String specifiedEncoding;
private final List<String> platforms;
public boolean runOnPlatform(String platform) {
if (platforms == null || platforms.isEmpty()) return true;
int inclusiveCount = 0;
for (String pl : platforms) {
if (pl.startsWith("!")) continue;
inclusiveCount++;
if (pl.equalsIgnoreCase(platform)) return true;
}
if (inclusiveCount == platforms.size()) {
return false;
}
for (String pl : platforms) {
if (!pl.startsWith("!")) continue;
if (pl.regionMatches(true, 1, platform, 0, platform.length())) return false;
}
return true;
}
public boolean versionWithinLimit(int version) {
```

```
return version >= versionLowerLimit && version <= versionUpperLimit;
}
public File getSourceFile() {
return sourceFile;
}
public File getMessagesFile() {
return messagesFile;
}
public String getContent() {
return content;
}
public LombokImmutableList<CompilerMessageMatcher> getMessages() {
return messages;
}
public boolean isIgnore() {
return ignore;
}
public boolean forceUnchanged() {
return unchanged;
}
public boolean isSkipCompareContent() {
return skipCompareContent;
}
public boolean isSkipIdempotent() {
return skipIdempotent;
}
public boolean isVerifyDiet() {
return verifyDiet;
}
public String getSpecifiedEncoding() {
return specifiedEncoding;
}
public ConfigurationResolver getConfiguration() {
return configuration;
}
public Map<String, String> getFormatPreferences() {
return formatPreferences;
}
private static final Pattern VERSION_STYLE_1 = Pattern.compile("^(\\d+)(?:\\s+.*)?$");
private static final Pattern VERSION_STYLE_2 = Pattern.compile("^\\:(\\d+)(?:\\s+.*)?$");
private static final Pattern VERSION_STYLE_3 = Pattern.compile("^(\\d+):(?:\\s+.*)?$");
private static final Pattern VERSION_STYLE_4 = Pattern.compile("^(\\d+):(\\d+)(?:\\s+.*)?$");
private int[] parseVersionLimit(String spec) {
/* Single version: '5' */ {
Matcher m = VERSION_STYLE_1.matcher(spec);
if (m.matches()) {
int v = Integer.parseInt(m.group(1));
return new int[] {v, v};
}
}
/* Upper bound: ':5' (inclusive) */ {
Matcher m = VERSION_STYLE_2.matcher(spec);
if (m.matches()) return new int[] {0, Integer.parseInt(m.group(1))};
```

```
}
/* Lower bound '5:' (inclusive) */ {
Matcher m = VERSION_STYLE_3.matcher(spec);
if (m.matches()) return new int[] {Integer.parseInt(m.group(1)), Integer.MAX_VALUE};
}
/* Range '7:8' (inclusive) */ {
Matcher m = VERSION_STYLE_4.matcher(spec);
if (m.matches()) return new int[] {Integer.parseInt(m.group(1)), Integer.parseInt(m.group(2))};
}
return null;
}
private static final Pattern IGNORE_PATTERN = Pattern.compile("^\\s*ignore\\s*(?:[-:].*)?$",
Pattern.CASE_INSENSITIVE);
private static final Pattern UNCHANGED_PATTERN = Pattern.compile("^\\s*unchanged\\s*(?:[-:].*)?$",
Pattern.CASE_INSENSITIVE);
private static final Pattern SKIP_COMPARE_CONTENT_PATTERN = Pattern.compile("^\\s*skip[-
]?compare[- ]?contents?\\s*(?:[-:].*)?$", Pattern.CASE_INSENSITIVE);
private static final Pattern SKIP_IDEMPOTENT_PATTERN = Pattern.compile("^\\s*skip[-
]?idempotent\\s*(?:[-:].*)?$", Pattern.CASE_INSENSITIVE);
private static final Pattern ISSUE_REF_PATTERN = Pattern.compile("^\\s*issue #?\\d+(:?\\s+.*)?$",
Pattern.CASE_INSENSITIVE);
private static final Pattern VERIFY_DIET_PATTERN = Pattern.compile("^\\s*eclipse:\\s*verify[-
]?diet\\s*(?:[-:].*)?$", Pattern.CASE_INSENSITIVE);
private LombokTestSource(File sourceFile, File messagesFile, String content,
List<CompilerMessageMatcher> messages, List<String> directives) {
this.sourceFile = sourceFile;
this.messagesFile = messagesFile;
this.content = content;
this.messages = messages == null ? LombokImmutableList.<CompilerMessageMatcher>of() :
LombokImmutableList.copyOf(messages);
StringBuilder conf = new StringBuilder();
int versionLower = 0;
int versionUpper = Integer.MAX_VALUE;
boolean ignore = false;
boolean skipCompareContent = false;
boolean skipIdempotent = false;
boolean unchanged = false;
boolean verifyDiet = false;
String encoding = null;
Map<String, String> formats = new HashMap<String, String>();
String[] platformLimit = null;
for (String directive : directives) {
directive = directive.trim();
String lc = directive.toLowerCase();
if (IGNORE_PATTERN.matcher(directive).matches()) {
ignore = true;
continue;
}
if (UNCHANGED_PATTERN.matcher(directive).matches()) {
unchanged = true;
continue;
}
if (SKIP_COMPARE_CONTENT_PATTERN.matcher(directive).matches()) {
```

```
skipCompareContent = true;
continue;
}
if (SKIP_IDEMPOTENT_PATTERN.matcher(directive).matches()) {
skipIdempotent = true;
continue;
}
if (ISSUE_REF_PATTERN.matcher(directive).matches()) {
continue;
}
if (VERIFY_DIET_PATTERN.matcher(directive).matches()) {
verifyDiet = true;
continue;
}
if (lc.startsWith("platform ")) {
String platformDesc = lc.substring("platform ".length());
int idx = platformDesc.indexOf(':');
if (idx != -1) platformDesc = platformDesc.substring(0, idx).trim();
platformLimit = platformDesc.split("\\s*,\\s*");
continue;
}
if (lc.startsWith("version ")) {
int[] limits = parseVersionLimit(lc.substring(7).trim());
if (limits == null) {
Assert.fail("Directive line \"" + directive + "\" in "' + sourceFile.getAbsolutePath() + "' invalid: version
must be followed by a single integer.");
throw new RuntimeException();
}
versionLower = limits[0];
versionUpper = limits[1];
continue;
}
if (lc.startsWith("conf:")) {
String confLine = directive.substring(5).trim();
conf.append(confLine).append("\n");
continue;
}
if (lc.startsWith("encoding:")) {
encoding = directive.substring(9).trim();
continue;
}
if (lc.startsWith("format:")) {
String formatLine = directive.substring(7).trim();
int idx = formatLine.indexOf('=');
if (idx == -1) throw new IllegalArgumentException("To add a format directive, use: \"//FORMAT:
javaLangAsFQN = skip\"");
String key = formatLine.substring(0, idx).trim();
String value = formatLine.substring(idx + 1).trim();
formats.put(key.toLowerCase(), value);
continue;
}
if (lc.startsWith("issue ")) continue;
```

```
Assert.fail("Directive line \"" + directive + "\" in '" + sourceFile.getAbsolutePath() + "' invalid:
unrecognized directive.");
throw new RuntimeException();
}
this.specifiedEncoding = encoding;
this.versionLowerLimit = versionLower;
this.versionUpperLimit = versionUpper;
this.ignore = ignore;
this.skipCompareContent = skipCompareContent;
this.skipIdempotent = skipIdempotent;
this.unchanged = unchanged;
this.verifyDiet = verifyDiet;
this.platforms = platformLimit == null ? null : Arrays.asList(platformLimit);
ConfigurationProblemReporter reporter = new ConfigurationProblemReporter() {
@Override public void report(String sourceDescription, String problem, int lineNumber, CharSequence
line) {
Assert.fail("Problem on directive line: " + problem + " at conf line #" + lineNumber + " (" + line + ")");
}
};
final ConfigurationFile configurationFile =
ConfigurationFile.fromCharSequence(sourceFile.getAbsoluteFile().getPath(), conf,
ConfigurationFile.getLastModifiedOrMissing(sourceFile));
final ConfigurationSource source = SingleConfigurationSource.parse(configurationFile, new
ConfigurationParser(reporter));
ConfigurationFileToSource sourceFinder = new ConfigurationFileToSource() {
@Override public ConfigurationSource parsed(ConfigurationFile fileLocation) {
return fileLocation.equals(configurationFile) ? source : null;
}
};
this.configuration = new BubblingConfigurationResolver(configurationFile, sourceFinder);
this.formatPreferences = Collections.unmodifiableMap(formats);
}
public static LombokTestSource readDirectives(File file) throws IOException {
List<String> directives = new ArrayList<String>();
{
InputStream rawIn = new FileInputStream(file);
try {
BufferedReader in = new BufferedReader(new InputStreamReader(rawIn, "UTF-8"));
try {
for (String i = in.readLine(); i != null; i = in.readLine()) {
if (i.isEmpty()) continue;
if (i.startsWith("//")) {
directives.add(i.substring(2));
} else {
break;
}
}
}
finally {
in.close();
}
}
finally {
```

```
rawIn.close();
}
}
return new LombokTestSource(file, null, "", null, directives);
}
public static LombokTestSource read(File sourceFolder, File messagesFolder, String fileName) throws
IOException {
return read0(sourceFolder, messagesFolder, fileName, "UTF-8");
}
private static LombokTestSource read0(File sourceFolder, File messagesFolder, String fileName,
String encoding) throws IOException {
StringBuilder content = null;
List<String> directives = new ArrayList<String>();
File sourceFile = new File(sourceFolder, fileName);
if (sourceFile.exists()) {
InputStream rawIn = new FileInputStream(sourceFile);
try {
BufferedReader in = new BufferedReader(new InputStreamReader(rawIn, encoding));
try {
for (String i = in.readLine(); i != null; i = in.readLine()) {
if (content != null) {
content.append(i).append("\n");
continue;
}
if (i.isEmpty()) continue;
if (i.startsWith("//")) {
directives.add(i.substring(2));
} else {
content = new StringBuilder();
content.append(i).append("\n");
}
}
}
finally {
in.close();
}
}
finally {
rawIn.close();
}
}
if (content == null) content = new StringBuilder();
List<CompilerMessageMatcher> messages = null;
File messagesFile = null;
if (messagesFolder != null) {
messagesFile = new File(messagesFolder, fileName + ".messages");
try {
InputStream rawIn = new FileInputStream(messagesFile);
try {
messages = CompilerMessageMatcher.readAll(rawIn);
}
finally {
rawIn.close();
```

```
}
} catch (FileNotFoundException e) {
messages = null;
messagesFile = null;
}
}
LombokTestSource source = new LombokTestSource(sourceFile, messagesFile, content.toString(),
messages, directives);
String specifiedEncoding = source.getSpecifiedEncoding();
// The source file has an 'encoding' header to test encoding issues. Of course, reading the encoding
header
// requires knowing the encoding of the file first. In practice we get away with it, because UTF-8 and
US-ASCII are compatible enough.
// The fix is therefore to read in as UTF-8 initially, and if the file requests that it should be read as
another encoding, toss it all
// and reread that way.
if (specifiedEncoding == null || specifiedEncoding.equalsIgnoreCase(encoding)) return source;
return read0(sourceFolder, messagesFolder, fileName, specifiedEncoding);
}
public int minVersion() {
return Math.max(6, versionLowerLimit);
}
public int maxVersion() {
return versionUpperLimit;
}
}
```

lombok/RunTestsViaDelombok.java


```
/*
* Copyright (C) 2009-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```


```
package lombok;
```

```
import static org.junit.Assert.fail;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.PrintStream;
import java.io.StringWriter;
import java.io.UnsupportedEncodingException;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Deque;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.Stack;
import javax.annotation.processing.AbstractProcessor;
import javax.annotation.processing.ProcessingEnvironment;
import javax.annotation.processing.RoundEnvironment;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.Element;
import javax.lang.model.element.TypeElement;
import com.sun.source.util.TreePath;
import com.sun.source.util.Trees;
import com.sun.tools.javac.code.Flags;
import com.sun.tools.javac.code.Symbol;
import com.sun.tools.javac.tree.JCTree;
import com.sun.tools.javac.tree.JCTree.JCAnnotation;
import com.sun.tools.javac.tree.JCTree.JCAssign;
import com.sun.tools.javac.tree.JCTree.JCClassDecl;
import com.sun.tools.javac.tree.JCTree.JCCompilationUnit;
import com.sun.tools.javac.tree.JCTree.JCIdent;
import com.sun.tools.javac.tree.JCTree.JCMethodDecl;
import com.sun.tools.javac.tree.JCTree.JCModifiers;
import com.sun.tools.javac.tree.JCTree.JCPrimitiveTypeTree;
import com.sun.tools.javac.tree.JCTree.JCVariableDecl;
import com.sun.tools.javac.tree.JCTree.TypeBoundKind;
import com.sun.tools.javac.tree.TreeScanner;
import lombok.delombok.Delombok;
import lombok.javac.CapturingDiagnosticListener;
import lombok.javac.Javac;
public class RunTestsViaDelombok extends AbstractRunTests {
private Delombok delombok = new Delombok();
@Override
public TransformationResult transformCode(final File file, TestParameters parameters) throws
Throwable {
TransformationResult result = new TransformationResult();
delombok.setVerbose(true);
ChangedChecker cc = new ChangedChecker();
delombok.setFeedback(cc.feedback);
delombok.setForceProcess(true);
delombok.setCharset(parameters.getEncoding() == null ? "UTF-8" : parameters.getEncoding());
```

```
delombok.setFormatPreferences(parameters.getFormatPreferences());
delombok.setDiagnosticsListener(new CapturingDiagnosticListener(file, result.getMessages()));
if (parameters.isCheckPositions()) delombok.addAdditionalAnnotationProcessor(new
ValidatePositionProcessor(parameters.getMinVersion()));
delombok.addAdditionalAnnotationProcessor(new ValidateTypesProcessor());
delombok.addAdditionalAnnotationProcessor(new ValidateNoDuplicateTreeNodeProcessor());
delombok.addFile(file.getAbsoluteFile().getParentFile(), file.getName());
delombok.setSourcepath(file.getAbsoluteFile().getParent());
String bcp = System.getProperty("delombok.bootclasspath");
if (bcp != null) delombok.setBootclasspath(bcp);
StringWriter sw = new StringWriter();
delombok.setWriter(sw);
Locale originalLocale = Locale.getDefault();
try {
Locale.setDefault(Locale.ENGLISH);
delombok.delombok();
result.setOutput(sw.toString());
result.setChanged(cc.isChanged());
return result;
} finally {
Locale.setDefault(originalLocale);
}
}
public static class ValidatePositionProcessor extends TreeProcessor {
private final int version;
public ValidatePositionProcessor(int version) {
this.version = version;
}
private String craftFailMsg(String problematicNode, Deque<JCTree> astContext) {
StringBuilder msg = new StringBuilder(problematicNode).append(" position of node not set: ");
for (JCTree t : astContext) {
msg.append("\n ").append(t.getClass().getSimpleName());
String asStr = t.toString();
if (asStr.length() < 80) msg.append(": ").append(asStr);
else if (t instanceof JCClassDecl) msg.append(": ").append(((JCClassDecl) t).name);
else if (t instanceof JCMethodDecl) msg.append(": ").append(((JCMethodDecl) t).name);
else if (t instanceof JCVariableDecl) msg.append(": ").append(((JCVariableDecl) t).name);
}
return msg.append("\n-------").toString();
}
@Override void processCompilationUnit(final JCCompilationUnit unit) {
final Deque<JCTree> astContext = new ArrayDeque<JCTree>();
unit.accept(new TreeScanner() {
@Override public void scan(JCTree tree) {
if (tree == null) return;
if (tree instanceof JCMethodDecl && (((JCMethodDecl) tree).mods.flags &
Flags.GENERATEDCONSTR) != 0) return;
astContext.push(tree);
try {
if (tree instanceof JCModifiers) return;
if (!Javac.validateDocComment(unit, tree)) {
fail("Start position of doc comment (" + Javac.getDocComment(unit, tree) + ") of " + tree + " not set");
}
```

```
boolean check = true;
if (version < 8 && tree instanceof TypeBoundKind) {
// TypeBoundKind works differently in java6, and as a consequence,
// the position is not set properly.
// Given status of j6/j7, not worth properly testing.
check = false;
}
if (version < 8 && tree instanceof JCIdent) {
// explicit `super()` invocations do not appear to have end pos in j6/7.
if ("super".equals("" + ((JCIdent) tree).name)) check = false;
}
if (tree instanceof JCVariableDecl && (((JCVariableDecl) tree).mods.flags &
Javac.GENERATED_MEMBER) != 0) return;
if (check && tree.pos == -1) fail(craftFailMsg("Start", astContext));
if (check && Javac.getEndPosition(tree, unit) == -1) {
fail(craftFailMsg("End", astContext));
}
} finally {
try {
super.scan(tree);
} finally {
astContext.pop();
}
}
}
@Override public void visitMethodDef(JCMethodDecl tree) {
super.visitMethodDef(tree);
}
@Override public void visitVarDef(JCVariableDecl tree) {
if ((tree.mods.flags & Flags.ENUM) != 0) return;
super.visitVarDef(tree);
}
@Override public void visitAnnotation(JCAnnotation tree) {
scan(tree.annotationType);
// Javac parser maps @Annotation("val") to @Annotation(value = "val") but does not add an end
position for the new JCIdent...
if (tree.args.length() == 1 && tree.args.head instanceof JCAssign && ((JCIdent)((JCAssign)
tree.args.head).lhs).name.toString().equals("value")) {
scan(((JCAssign) tree.args.head).rhs);
} else {
scan(tree.args);
}
}
});
}
}
public static class ValidateTypesProcessor extends TreeProcessor {
@Override void processCompilationUnit(final JCCompilationUnit unit) {
final Stack<JCTree> parents = new Stack<JCTree>();
parents.add(unit);
unit.accept(new TreeScanner() {
private JCTree parent;
@Override public void scan(JCTree tree) {
```

```
parent = parents.peek();
parents.push(tree);
super.scan(tree);
parents.pop();
}
@Override public void visitClassDef(JCClassDecl tree) {
// Skip anonymous or local classes, they have no symbol
if (!(parent instanceof JCClassDecl || parent instanceof JCCompilationUnit)) return;
validateSymbol(tree, tree.sym);
super.visitClassDef(tree);
};
@Override public void visitMethodDef(JCMethodDecl tree) {
validateSymbol(tree, tree.sym);
super.visitMethodDef(tree);
}
@Override public void visitVarDef(JCVariableDecl tree) {
// Skip local variables
if (!(parent instanceof JCClassDecl || parent instanceof JCMethodDecl)) return;
validateSymbol(tree, tree.sym);
super.visitVarDef(tree);
}
private void validateSymbol(JCTree tree, Symbol sym) {
if (sym == null) {
fail("Missing symbol for " + tree);
}
// Only classes have enclosed elements, skip everything else
if (!sym.owner.getKind().isClass()) return;
if (!sym.owner.getEnclosedElements().contains(sym)) {
fail(tree + " not added to parent");
}
}
});
}
}
public static class ValidateNoDuplicateTreeNodeProcessor extends TreeProcessor {
private String craftFailMsg(Collection<JCTree> astContext) {
StringBuilder msg = new StringBuilder();
for (JCTree t : astContext) {
msg.append("\n ").append(t.getClass().getSimpleName());
String asStr = t.toString();
if (asStr.length() < 80) msg.append(": ").append(asStr);
else if (t instanceof JCClassDecl) msg.append(": ").append(((JCClassDecl) t).name);
else if (t instanceof JCMethodDecl) msg.append(": ").append(((JCMethodDecl) t).name);
else if (t instanceof JCVariableDecl) msg.append(": ").append(((JCVariableDecl) t).name);
}
return msg.append("\n-------").toString();
}
@Override
void processCompilationUnit(JCCompilationUnit unit) {
final Deque<JCTree> parents = new ArrayDeque<JCTree>();
parents.add(unit);
final Map<JCTree, List<JCTree>> knownTreeNode = new IdentityHashMap<JCTree, List<JCTree>>();
unit.accept(new TreeScanner() {
```

```
private JCTree parent;
@Override
public void scan(JCTree tree) {
parent = parents.peek();
if (tree == null) return;
if (tree instanceof JCPrimitiveTypeTree) return;
// javac generates duplicates for record members
if (tree instanceof JCVariableDecl && (((JCVariableDecl) tree).mods.flags &
Javac.GENERATED_MEMBER) != 0) return;
List<JCTree> knownNodeContext = knownTreeNode.put(tree, new ArrayList<JCTree>(parents));
if (knownNodeContext != null) {
// javac generates two JCVariableDecl elements for 'int a, b;'
if (parent instanceof JCVariableDecl) {
if (tree instanceof JCModifiers) return;
if (tree instanceof JCIdent) return;
}
fail("Node " + tree + " found twice:" + craftFailMsg(knownNodeContext) + craftFailMsg(parents));
}
parents.push(tree);
super.scan(tree);
parents.pop();
}
/**
* We always generate shallow copies for annotations
*/
@Override
public void visitAnnotation(JCAnnotation tree) {
return;
}
});
}
}
public static abstract class TreeProcessor extends AbstractProcessor {
private Trees trees;
@Override public synchronized void init(ProcessingEnvironment processingEnv) {
super.init(processingEnv);
trees = Trees.instance(processingEnv);
}
@Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment
roundEnv) {
for (Element element : roundEnv.getRootElements()) {
JCCompilationUnit unit = toUnit(element);
if (unit != null) {
processCompilationUnit(unit);
}
}
return false;
}
abstract void processCompilationUnit(JCCompilationUnit unit);
@Override public Set<String> getSupportedAnnotationTypes() {
return Collections.singleton("*");
}
@Override public SourceVersion getSupportedSourceVersion() {
```

```
return SourceVersion.latest();
}
private JCCompilationUnit toUnit(Element element) {
TreePath path = null;
if (trees != null) {
try {
path = trees.getPath(element);
} catch (NullPointerException ignore) {
// Happens if a package-info.java doesn't contain a package declaration.
// https://github.com/projectlombok/lombok/issues/2184
// We can safely ignore those, since they do not need any processing
}
}
if (path == null) return null;
return (JCCompilationUnit) path.getCompilationUnit();
}
}
static class ChangedChecker {
private final ByteArrayOutputStream bytes = new ByteArrayOutputStream();
private final PrintStream feedback;
ChangedChecker() throws UnsupportedEncodingException {
feedback = new PrintStream(bytes, true, "UTF-8");
}
boolean isChanged() throws UnsupportedEncodingException {
feedback.flush();
return bytes.toString("UTF-8").endsWith("[delomboked]\n");
}
}
}
```

lombok/RunTestsViaEcj.java


```
/*
* Copyright (C) 2010-2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
```

```
* THE SOFTWARE.
*/
package lombok;
import java.io.File;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.atomic.AtomicReference;
import org.eclipse.core.internal.registry.ExtensionRegistry;
import org.eclipse.core.internal.runtime.Activator;
import org.eclipse.core.internal.runtime.PlatformActivator;
import org.eclipse.core.resources.ResourcesPlugin;
import org.eclipse.core.runtime.IExtensionRegistry;
import org.eclipse.core.runtime.RegistryFactory;
import org.eclipse.core.runtime.adaptor.EclipseStarter;
import org.eclipse.core.runtime.spi.IRegistryProvider;
import org.eclipse.jdt.core.JavaCore;
import org.eclipse.jdt.core.compiler.CategorizedProblem;
import org.eclipse.jdt.core.compiler.CharOperation;
import org.eclipse.jdt.core.dom.AST;
import org.eclipse.jdt.core.dom.CompilationUnit;
import org.eclipse.jdt.internal.compiler.CompilationResult;
import org.eclipse.jdt.internal.compiler.Compiler;
import org.eclipse.jdt.internal.compiler.ICompilerRequestor;
import org.eclipse.jdt.internal.compiler.IErrorHandlingPolicy;
import org.eclipse.jdt.internal.compiler.ast.CompilationUnitDeclaration;
import org.eclipse.jdt.internal.compiler.batch.FileSystem;
import org.eclipse.jdt.internal.compiler.env.ICompilationUnit;
import org.eclipse.jdt.internal.compiler.impl.CompilerOptions;
import org.eclipse.jdt.internal.compiler.problem.DefaultProblemFactory;
import org.eclipse.jdt.internal.core.DefaultWorkingCopyOwner;
import org.eclipse.jdt.internal.core.JavaModelManager;
import org.osgi.framework.Bundle;
import org.osgi.framework.BundleContext;
import lombok.eclipse.Eclipse;
import lombok.javac.CapturingDiagnosticListener.CompilerMessage;
public class RunTestsViaEcj extends AbstractRunTests {
protected CompilerOptions ecjCompilerOptions(TestParameters parameters) {
CompilerOptions options = new CompilerOptions();
Map<String, String> warnings = new HashMap<String, String>();
long ecjCompilerVersionConstant = Eclipse.getLatestEcjCompilerVersionConstant();
long ecjCompilerVersion = Eclipse.getEcjCompilerVersion();
if (parameters.getSourceVersion() != null) {
ecjCompilerVersionConstant = (parameters.getSourceVersion() + 44) << 16;
ecjCompilerVersion = parameters.getSourceVersion();
} else {
// Preview features are only allowed if the maximum compiler version is equal to the source version
warnings.put("org.eclipse.jdt.core.compiler.problem.enablePreviewFeatures", "enabled");
}
options.complianceLevel = ecjCompilerVersionConstant;
```

```
options.sourceLevel = ecjCompilerVersionConstant;
options.targetJDK = ecjCompilerVersionConstant;
options.docCommentSupport = false;
options.parseLiteralExpressionsAsConstants = true;
options.inlineJsrBytecode = true;
options.reportUnusedDeclaredThrownExceptionExemptExceptionAndThrowable = false;
options.reportUnusedDeclaredThrownExceptionIncludeDocCommentReference = false;
options.reportUnusedDeclaredThrownExceptionWhenOverriding = false;
options.reportUnusedParameterIncludeDocCommentReference = false;
options.reportUnusedParameterWhenImplementingAbstract = false;
options.reportUnusedParameterWhenOverridingConcrete = false;
options.reportDeadCodeInTrivialIfStatement = false;
options.generateClassFiles = false;
warnings.put(CompilerOptions.OPTION_ReportUnusedLocal, "ignore");
warnings.put(CompilerOptions.OPTION_ReportUnusedLabel, "ignore");
warnings.put(CompilerOptions.OPTION_ReportUnusedImport, "ignore");
warnings.put(CompilerOptions.OPTION_ReportUnusedPrivateMember, "ignore");
warnings.put(CompilerOptions.OPTION_ReportIndirectStaticAccess, "warning");
warnings.put(CompilerOptions.OPTION_ReportNonStaticAccessToStatic, "warning");
warnings.put("org.eclipse.jdt.core.compiler.problem.reportPreviewFeatures", "ignore");
warnings.put(CompilerOptions.OPTION_Source, (ecjCompilerVersion < 9 ? "1." : "") +
ecjCompilerVersion);
warnings.put("org.eclipse.jdt.core.compiler.codegen.useStringConcatFactory", "disabled");
options.set(warnings);
return options;
}
protected IErrorHandlingPolicy ecjErrorHandlingPolicy() {
return new IErrorHandlingPolicy() {
public boolean stopOnFirstError() {
return false;
}
public boolean proceedOnErrors() {
return false;
}
@SuppressWarnings("all") // Added to the interface in later ecj version.
public boolean ignoreAllErrors() {
return false;
}
};
}
private ICompilationUnit getSourceUnit(File file, String source) {
if (eclipseAvailable()) return new TestCompilationUnitEclipse(file.getName(), source);
return new TestCompilationUnitEcj(file.getName(), source);
}
@Override
public TransformationResult transformCode(File file, final TestParameters parameters) throws
Throwable {
final TransformationResult result = new TransformationResult();
final AtomicReference<CompilationResult> compilationResult_ = new
AtomicReference<CompilationResult>();
final AtomicReference<CompilationUnitDeclaration> compilationUnit_ = new
AtomicReference<CompilationUnitDeclaration>();
ICompilerRequestor bitbucketRequestor = new ICompilerRequestor() {
```

```
@Override public void acceptResult(CompilationResult result) {
compilationResult_.set(result);
}
};
String source = readFile(file);
char[] sourceArray = source.toCharArray();
final ICompilationUnit sourceUnit;
try {
sourceUnit = getSourceUnit(file, source);
} catch (Throwable t) {
t.printStackTrace();
result.setChanged(false);
return result;
}
Compiler ecjCompiler = new Compiler(createFileSystem(file), ecjErrorHandlingPolicy(),
ecjCompilerOptions(parameters), bitbucketRequestor, new DefaultProblemFactory(Locale.ENGLISH))
{
@Override protected synchronized void addCompilationUnit(ICompilationUnit inUnit,
CompilationUnitDeclaration parsedUnit) {
if (inUnit == sourceUnit) {
compilationUnit_.set(parsedUnit);
if (parameters.isVerifyDiet()) {
result.setOutput(parsedUnit.toString());
}
}
super.addCompilationUnit(inUnit, parsedUnit);
}
};
// initializeEclipseBundles();
ecjCompiler.compile(new ICompilationUnit[] {sourceUnit});
CompilationResult compilationResult = compilationResult_.get();
CategorizedProblem[] problems = compilationResult.getAllProblems();
if (problems != null) for (CategorizedProblem p : problems) {
result.addMessage(new CompilerMessage(p.getSourceLineNumber(), p.getSourceStart(), p.isError(),
p.getMessage()));
}
CompilationUnitDeclaration cud = compilationUnit_.get();
if (cud == null) {
result.setOutput("---- No CompilationUnit provided by ecj ----");
} else {
if (!parameters.isVerifyDiet()) {
String output = cud.toString();
// starting somewhere around ecj16, the print code is a bit too cavalier with printing modifiers.
output = output.replace("non-sealed @val", "@val");
result.setOutput(output);
}
if (eclipseAvailable()) {
EclipseDomConversion.toDomAst(cud, sourceArray);
}
}
result.setChanged(true);
return result;
}
```

```
@SuppressWarnings("unused")
private static class EclipseInitializer {
static void initializeEclipseBundles() throws Exception {
// This code does not work yet, it's research-in-progress.
// The problem is that parts of the eclipse handler (in `PatchValEclipse` and friends) do not work unless
// an actual eclipse exists; PatchVal causes code to run that will end up running
`ResourcesPlugin.getWorkspace()`, which
// goes down a rabbit hole of pinging off of various static fields (or `getX()` calls which return static
fields), all
// of which are `null` until the plugin they belong to is properly initialized.
// This code is work in progress to 'hack' the initialization of each plugin one-by-one, but I doubt this is
the right
// way to do it, as I bet it's fragile (will break when eclipse updates rather easily), and who knows how
many fields
// and things need to be initialized.
// A better plan would be to start an actual, real eclipse, by telling `EclipseStarter.startup` to launch
some sort of
// application (or at least a bunch of bundles/products/apps, including the JDT). This will then take long
enough that
// it'll need to be cached and re-used for each test or the Eclipse test run would take far too long.
BundleContext context = EclipseStarter.startup(new String[0], null);
RegistryFactory.setDefaultRegistryProvider(new IRegistryProvider() {
private final ExtensionRegistry REG = new ExtensionRegistry(null, null, null);
@Override public IExtensionRegistry getRegistry() {
return REG;
}
});
new Activator().start(context);
new PlatformActivator().start(context);
for (Bundle b : context.getBundles()) System.out.println("BUNDLE: " + b.getSymbolicName());
new ResourcesPlugin().start(context);
JavaModelManager.getJavaModelManager().startup();
}
}
static boolean eclipseAvailable() {
try {
Class.forName("org.eclipse.jdt.core.dom.CompilationUnit");
} catch (Throwable t) {
return false;
}
return true;
}
private static final String bootRuntimePath = System.getProperty("delombok.bootclasspath");
private static class EclipseDomConversion {
static CompilationUnit toDomAst(CompilationUnitDeclaration cud, final char[] source) {
Map<String, String> options = new HashMap<String, String>();
options.put(JavaCore.COMPILER_SOURCE, "11");
options.put("org.eclipse.jdt.core.compiler.problem.enablePreviewFeatures", "enabled");
org.eclipse.jdt.internal.core.CompilationUnit ccu = new
org.eclipse.jdt.internal.core.CompilationUnit(null, null, null) {
@Override public char[] getContents() {
return source;
}
```

```
};
return AST.convertCompilationUnit(4, cud, options, false, ccu, 0, null);
}
}
private FileSystem createFileSystem(File file) {
List<String> classpath = new ArrayList<String>();
if (new File("bin/main").exists()) classpath.add("bin/main");
classpath.add("dist/lombok.jar");
classpath.add("build/teststubs");
if (bootRuntimePath == null || bootRuntimePath.isEmpty()) throw new IllegalStateException("System
property delombok.bootclasspath is not set; set it to the rt of java6 or java8");
classpath.add(bootRuntimePath);
for (File f : new File("lib/test").listFiles()) {
String fn = f.getName();
if (fn.length() < 4) continue;
if (!fn.substring(fn.length() - 4).toLowerCase().equals(".jar")) continue;
classpath.add("lib/test/" + fn);
}
for (Iterator<String> i = classpath.iterator(); i.hasNext();) {
if (FileSystem.getClasspath(i.next(), "UTF-8", null) == null) {
i.remove();
}
}
return new FileSystem(classpath.toArray(new String[0]), new String[] {file.getAbsolutePath()}, "UTF-8");
}
private static final class TestCompilationUnitEcj implements ICompilationUnit {
private final char[] name, source, mainTypeName;
TestCompilationUnitEcj(String name, String source) {
this.source = source.toCharArray();
this.name = name.toCharArray();
char[] fileNameCharArray = getFileName();
int start = CharOperation.lastIndexOf(File.separatorChar, fileNameCharArray) + 1;
int end = CharOperation.lastIndexOf('.', fileNameCharArray);
if (end == -1) {
end = fileNameCharArray.length;
}
mainTypeName = CharOperation.subarray(fileNameCharArray, start, end);
}
@Override public char[] getFileName() {
return name;
}
@Override public char[] getContents() {
return source;
}
@Override public char[] getMainTypeName() {
return mainTypeName;
}
@Override public char[][] getPackageName() {
return null;
}
@Override public boolean ignoreOptionalProblems() {
return false;
}
```

```
}
private static final class TestCompilationUnitEclipse extends
org.eclipse.jdt.internal.core.CompilationUnit {
private final char[] source;
private final char[] mainTypeName;
private TestCompilationUnitEclipse(String name, String source) {
super(null, name, DefaultWorkingCopyOwner.PRIMARY);
this.source = source.toCharArray();
char[] fileNameCharArray = getFileName();
int start = CharOperation.lastIndexOf(File.separatorChar, fileNameCharArray) + 1;
int end = CharOperation.lastIndexOf('.', fileNameCharArray);
if (end == -1) {
end = fileNameCharArray.length;
}
mainTypeName = CharOperation.subarray(fileNameCharArray, start, end);
}
@Override public char[] getContents() {
return source;
}
@Override public char[] getMainTypeName() {
return mainTypeName;
}
@Override public boolean ignoreOptionalProblems() {
return false;
}
@Override public char[][] getPackageName() {
return null;
}
public char[] getModuleName() {
return null;
}
}
}
```

lombok/TestBase.java

```
/*
* Copyright (C) 2011-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
```

* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import org.junit.runners.Suite.SuiteClasses;
@RunWith(Suite.class)
@SuiteClasses({lombok.core.configuration.RunConfigurationTests.class,
lombok.core.RunCoreTests.class})
public class TestBase {
}

lombok/TestEclipse.java

/*
* Copyright (C) 2011-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import org.junit.runners.Suite.SuiteClasses;
@RunWith(Suite.class)
@SuiteClasses({lombok.transform.TestWithEcj.class})
public class TestEclipse {
}

lombok/TestJavac.java

```
/*
* Copyright (C) 2011-2020 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import org.junit.runners.Suite.SuiteClasses;
@RunWith(Suite.class)
@SuiteClasses({lombok.bytecode.RunBytecodeTests.class,
lombok.transform.TestLombokFilesIdempotent.class, lombok.transform.TestSourceFiles.class,
lombok.transform.TestWithDelombok.class})
public class TestJavac {
}
```

lombok/TestParameters.java

```
/*
* Copyright (C) 2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

```
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.util.Map;
public class TestParameters {
private Long sourceVersion;
private Long targetVersion;
// Javac only
private Integer minVersion;
private String encoding;
private Map<String, String> formatPreferences;
private boolean checkPositions;
// Eclipse/Ecj only
private boolean verifyDiet;
public Long getSourceVersion() {
return sourceVersion;
}
public void setSourceVersion(Long sourceVersion) {
this.sourceVersion = sourceVersion;
}
public Long getTargetVersion() {
return targetVersion;
}
public void setTargetVersion(Long targetVersion) {
this.targetVersion = targetVersion;
}
public Integer getMinVersion() {
return minVersion;
}
public void setMinVersion(Integer minVersion) {
this.minVersion = minVersion;
}
public String getEncoding() {
return encoding;
}
public void setEncoding(String encoding) {
this.encoding = encoding;
}
public Map<String, String> getFormatPreferences() {
return formatPreferences;
}
public void setFormatPreferences(Map<String, String> formatPreferences) {
this.formatPreferences = formatPreferences;
}
public boolean isCheckPositions() {
return checkPositions;
}
public void setCheckPositions(boolean checkPositions) {
this.checkPositions = checkPositions;
}
public boolean isVerifyDiet() {
```

```
return verifyDiet;
}
public void setVerifyDiet(boolean verifyDiet) {
this.verifyDiet = verifyDiet;
}
}
```

lombok/TransformationResult.java

```
/*
* Copyright (C) 2024 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok;
import java.util.LinkedHashSet;
import lombok.javac.CapturingDiagnosticListener.CompilerMessage;
public class TransformationResult {
private boolean changed;
private boolean success;
private LinkedHashSet<CompilerMessage> messages = new LinkedHashSet<CompilerMessage>();
private String output;
public void addMessage(CompilerMessage message) {
getMessages().add(message);
}
public void setOutput(String output) {
this.output = output;
}
public String getOutput() {
return output;
}
public boolean isChanged() {
return changed;
}
public void setChanged(boolean changed) {
```

```
this.changed = changed;
}
public boolean isSuccess() {
return success;
}
public void setSuccess(boolean success) {
this.success = success;
}
public LinkedHashSet<CompilerMessage> getMessages() {
return messages;
}
}
```

lombok/core/RunCoreTests.java

```
/*
* Copyright (C) 2015 The Project Lombok Authors.
*
* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
package lombok.core;
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import org.junit.runners.Suite.SuiteClasses;
@RunWith(Suite.class)
@SuiteClasses({TestSingulars.class})
public class RunCoreTests {
}
```

lombok/core/TestSingulars.java

```
/*
* Copyright (C) 2015-2017 The Project Lombok Authors.
*
```

* Permission is hereby granted, free of charge, to any person obtaining a copy
* of this software and associated documentation files (the "Software"), to deal
* in the Software without restriction, including without limitation the rights
* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
* copies of the Software, and to permit persons to whom the Software is
* furnished to do so, subject to the following conditions:
*
* The above copyright notice and this permission notice shall be included in
* all copies or substantial portions of the Software.
*
* THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
* IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
* FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
* AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
* LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
* OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
* THE SOFTWARE.
*/
```
package lombok.core;
import static lombok.core.handlers.Singulars.autoSingularize;
import static org.junit.Assert.*;
import org.junit.Test;
public class TestSingulars {
@Test
public void testSingulars() {
assertNull("axes", autoSingularize("axes"));
assertEquals("adjective", autoSingularize("adjectives"));
assertEquals("bus", autoSingularize("buses"));
assertEquals("octopus", autoSingularize("octopodes"));
assertNull("octopi", autoSingularize("octopi"));
assertEquals("elf", autoSingularize("elves"));
assertEquals("jack", autoSingularize("jacks"));
assertEquals("colloquy", autoSingularize("colloquies"));
assertNull("series", autoSingularize("series"));
assertEquals("man", autoSingularize("men"));
assertNull("highwaymen", autoSingularize("highwaymen"));
assertEquals("caveMan", autoSingularize("caveMen"));
assertNull("jackss", autoSingularize("jackss"));
assertNull("virus", autoSingularize("virus"));
assertEquals("quiz", autoSingularize("quizzes"));
assertEquals("database", autoSingularize("databases"));
assertEquals("dataBase", autoSingularize("dataBases"));
assertEquals("Query", autoSingularize("Queries"));
assertEquals("Movie", autoSingularize("Movies"));
assertEquals("cafe", autoSingularize("cafes"));
assertNull("caves", autoSingularize("caves"));
assertEquals("leaf", autoSingularize("leaves"));
assertEquals("autosave", autoSingularize("autosaves"));
}
}
```